# Wayward House Mathematics

**A series for people who want to understand, not just pass**

Paul Hobson

2026-04-04

# Table of contents


**1. Mathematics** **1**
1.1. Start here . . . 1
1.2. The volumes . . . 1
1.3. How to use this series . . . 2

**2. How to Use This Series** **3**
2.1. Where to start . . . 3
2.2. How chapters are structured . . . 3
2.3. The exercises . . . 4
2.4. What the notation is doing . . . 4
2.5. The destination fields . . . 4
2.6. The prerequisites . . . 4

**I. Volume 1 — Numbers and operations** **5**

**3. Volume 1 — Numbers and operations** **7**
3.1. Chapters . . . 7

**4. Numbers and the number line** **9**
4.1. What the notation is saying . . . 9
4.2. The method . . . 14
4.3. Worked examples . . . 15
4.4. Where this goes . . . 21
4.5. Exercises . . . 21

**5. Ratio and proportion** **23**
5.1. What the notation is saying . . . 23
5.2. The method . . . 27
5.3. Worked examples . . . 31
5.4. Where this goes . . . 32
5.5. Exercises . . . 33

**6. Percent and rates** **35**
6.1. What the notation is saying . . . 35
6.2. The method . . . 36
6.3. Worked examples . . . 39
6.4. Where this goes . . . 45
6.5. Exercises . . . 45

**7. Integers and signed arithmetic** **47**
7.1. What the notation is saying . . . 47
7.2. The method . . . 51
7.3. Worked examples . . . 55
7.4. Where this goes . . . 57
7.5. Exercises . . . 57

**8. Fractions and decimals** **59**
8.1. What the notation is saying . . . 59
8.2. The method . . . 63
8.3. Worked examples . . . 68
8.4. Where this goes . . . 69
8.5. Exercises . . . 70

**9. Statistics and probability** **71**
9.1. What the notation is saying . . . 71
9.2. The method . . . 72
9.3. Worked examples . . . 77
9.4. Where this goes . . . 83
9.5. Exercises . . . 84

**II. Volume 2 — Patterns and algebra** **85**

**10. Volume 2 — Patterns and algebra** **87**
10.1. Chapters . . . 87

**11. Linear equations** **89**
11.1. What the notation is saying . . . 89
11.2. The method . . . 93
11.3. Worked examples . . . 97
11.4. Where this goes . . . 99
11.5. Exercises . . . 99

**12. Inequalities** **103**
12.1. What the notation is saying . . . 103
12.2. The method . . . 104
12.2.1. The sign-flip rule . . . 104
12.2.2. Solving a linear inequality, step by step . . . 105
12.2.3. Reading the solution on a number line . . . 106
12.3. Worked examples . . . 109
12.4. Where this goes . . . 113
12.5. Exercises . . . 114

**13. Patterns and sequences** **119**
13.1. What makes a sequence a sequence . . . 119
13.2. What the notation is saying . . . 120
13.3. Arithmetic sequences . . . 120
13.3.1. Recognising one . . . 120

13.3.2. The nth term formula . . . 120
13.4. Worked examples — arithmetic . . . 123
13.5. Geometric sequences . . . 125
13.5.1. Recognising one . . . 125
13.5.2. The nth term formula . . . 125
13.6. Worked examples — geometric . . . 128
13.7. Telling them apart . . . 129
13.8. Sequences as functions . . . 129
13.9. Where this goes . . . 130
13.10. Exercises . . . 130

**14. Graphing relations** **135**
14.1. What the notation is saying . . . 141
14.1.1. The coordinate plane . . . 141
14.1.2. A relation as a table of pairs . . . 145
14.2. The method . . . 145
14.2.1. Reading slope: rise over run . . . 145
14.2.2. Method 1: Graphing from a table of values . . . 146
14.2.3. Method 2: Graphing from slope-intercept form . . . 147
14.2.4. Method 3: Finding the equation from a point and a slope . . . 147
14.3. Why this works . . . 150
14.4. Worked examples . . . 151
14.5. A note on curves . . . 153
14.6. Where this goes . . . 153
14.7. Exercises . . . 154

**III. Volume 3 — Shape and measure** **159**

**15. Volume 3 — Shape and measure** **161**
15.1. Chapters . . . 161

**16. Geometry and measurement** **163**
16.1. What the notation is saying . . . 163
16.2. The method . . . 164
16.2.1. Rectangles . . . 164
16.2.2. Triangles . . . 164
16.2.3. Circles . . . 165
16.2.4. The Pythagorean theorem . . . 167
16.2.5. Volume of a rectangular prism . . . 170
16.2.6. Volume of a cylinder . . . 170
16.2.7. Units and unit conversion . . . 171
16.3. Worked examples . . . 172
16.4. Where this goes . . . 173
16.5. Exercises . . . 174

**17. Trigonometry** **179**
17.1. What the notation is saying . . . 179

17.2. The method . . . 183
17.2.1. Finding an unknown side . . . 183
17.2.2. Finding an unknown angle . . . 183
17.3. Worked examples . . . 184
17.4. Where this goes . . . 189
17.5. Exercises . . . 189

**18. Polynomials and factoring** **195**
18.1. What the notation is saying . . . 195
18.2. The method . . . 196
18.2.1. Part a: Expanding . . . 196
18.2.2. Part b: Factoring . . . 197
18.2.3. Part c: Solving quadratic equations . . . 198
18.2.4. Part d: Reading the graph . . . 199
18.3. Why this works . . . 202
18.4. Worked examples . . . 203
18.5. Where this goes . . . 207
18.6. Exercises . . . 208

**19. Functions and relations** **213**
19.1. What the notation is saying . . . 213
19.2. The method . . . 218
19.2.1. Part 1: Evaluating a function . . . 218
19.2.2. Part 2: Domain and range . . . 219
19.2.3. Part 3: Composition . . . 219
19.2.4. Part 4: Inverses . . . 223
19.3. Worked examples . . . 228
19.4. Where this goes . . . 231
19.5. Exercises . . . 231

**IV. Volume 4 — Functions and change** **237**

**20. Volume 4 — Functions and change** **239**
20.1. Chapters . . . 239

**21. Exponents and logarithms** **241**
21.1. What the notation is saying . . . 241
21.1.1. The exponent laws . . . 241
21.1.2. The exponential function . . . 242
21.1.3. The natural base $e$ . . . 242
21.2. The method . . . 245
21.2.1. Part 1: Working with exponent laws . . . 245
21.2.2. Part 2: Solving exponential equations by inspection . . . 245
21.2.3. Part 3: Logarithms . . . 246
21.2.4. Part 4: Log laws and solving logarithmic equations . . . 248
21.3. Worked examples . . . 250
21.4. Where this goes . . . 252
21.5. Exercises . . . 252

**22. Complex numbers** **257**
22.1. What the notation is saying . . . 257
22.1.1. The imaginary unit . . . 257
22.1.2. Complex numbers . . . 258
22.1.3. The complex plane . . . 258
22.2. The method . . . 261
22.2.1. Part 1: Arithmetic . . . 261
22.2.2. Part 2: Modulus and argument . . . 262
22.2.3. Part 3: Polar form and Euler's formula . . . 262
22.2.4. Part 4: De Moivre's theorem . . . 265
22.3. Worked examples . . . 266
22.3.1. Example 1 — Engineering: AC circuit impedance . . . 266
22.3.2. Example 2 — Physics: Quantum wavefunction amplitude . . . 267
22.3.3. Example 3 — Computing: FFT butterfly . . . 267
22.4. Where this goes . . . 268
22.5. Exercises . . . 268

**23. Systems of equations** **273**
23.1. What the notation is saying . . . 273
23.1.1. The general form . . . 274
23.1.2. Compact matrix notation . . . 274
23.2. The method . . . 274
23.2.1. Method 1 — Graphical . . . 275
23.2.2. Method 2 — Substitution . . . 278
23.2.3. Method 3 — Elimination . . . 278
23.3. Why this works . . . 279
23.4. Worked examples . . . 279
23.5. Where this goes . . . 284
23.6. Exercises . . . 284

**24. Sequences and series** **289**
24.1. What the notation is saying . . . 289
24.2. The method . . . 292
24.2.1. Arithmetic series . . . 292
24.2.2. Geometric series . . . 293
24.2.3. Infinite geometric series . . . 294
24.2.4. Sigma notation: working with sums . . . 297
24.3. Worked examples . . . 298
24.4. Where this goes . . . 299
24.5. Exercises . . . 300

**V. Volume 5 — Analysis and proof** **305**

**25. Volume 5 — Analysis and proof** **307**
25.1. Chapters . . . 307

**26. Limits and continuity** **309**
26.1. The idea before the notation . . . 309

26.2. What the notation is saying . . . 310
26.2.1. One-sided limits . . . 310
26.2.2. The limit versus the value — three cases . . . 311
26.3. The method — evaluating limits . . . 314
26.3.1. Technique 1: Direct substitution . . . 314
26.3.2. Technique 2: Factor and cancel . . . 315
26.3.3. Technique 3: Rationalise . . . 316
26.3.4. Technique 4: One-sided limits from graphs . . . 317
26.4. Continuity . . . 317
26.4.1. The three types of discontinuity . . . 318
26.5. The epsilon-delta definition . . . 321
26.6. Worked examples . . . 323
26.6.1. Example 1 — Direct substitution (polynomial) . . . 323
26.6.2. Example 2 — Factoring (removable discontinuity) . . . 323
26.6.3. Example 3 — Rationalisation (resolving an indeterminate form) . . . 324
26.6.4. Example 4 — One-sided limits (regime change) . . . 324
26.7. Where this goes . . . 325
26.8. Exercises . . . 326

**27. Differential calculus** **331**
27.1. The limit definition of the derivative . . . 331
27.2. What the notation is saying . . . 332
27.3. The method — differentiation rules . . . 335
27.3.1. Power rule . . . 336
27.3.2. Sum/difference and constant multiple rules . . . 337
27.3.3. Product rule . . . 337
27.3.4. Chain rule . . . 337
27.3.5. Derivatives of key functions . . . 340
27.4. Higher derivatives . . . 341
27.5. Applications: optimisation . . . 341
27.6. Worked examples . . . 342
27.7. Where this goes . . . 344
27.8. Exercises . . . 344

**28. Integral calculus** **349**
28.1. What the notation is saying . . . 349
28.1.1. Riemann sums . . . 349
28.2. The Fundamental Theorem of Calculus . . . 353
28.2.1. Part 1 — Differentiation undoes integration . . . 353
28.2.2. Part 2 — Evaluating definite integrals . . . 354
28.3. Integration rules . . . 355
28.4. Integration by substitution . . . 356
28.5. Worked examples . . . 360
28.5.1. Example 1 — Power rule and the FTC (computing / probability) . . . 360
28.5.2. Example 2 — FTC Part 2 (science / measurement) . . . 361
28.5.3. Example 3 — Substitution (engineering / signals) . . . 362
28.5.4. Example 3b — Area between two curves . . . 362
28.5.5. Example 4 — Area between two curves (business / economic surplus) . . . 363

28.6. Where this goes . . . 363
28.7. Applications . . . 364
28.8. Exercises . . . 364

**29. Proof and mathematical reasoning** **369**
29.1. A pattern that lies to you . . . 369
29.2. What a proof is . . . 372
29.2.1. The notation of logical structure . . . 372
29.3. Four proof methods . . . 373
29.3.1. 1. Direct proof . . . 373
29.3.2. 2. Proof by contrapositive . . . 373
29.3.3. 3. Proof by contradiction . . . 374
29.3.4. 4. Mathematical induction . . . 374
29.4. What makes a proof fail . . . 377
29.4.1. 1. Assuming the conclusion . . . 377
29.4.2. 2. Missing a case . . . 378
29.4.3. 3. An invalid step . . . 378
29.5. Worked examples . . . 379
29.5.1. Example 1: Direct proof — the product of two odd numbers . . . 379
29.5.2. Example 2: Proof by contradiction — infinitely many primes . . . 379
29.5.3. Example 3: Proof by induction — $2^n > n$ for all $n \geq 1$ . . . 380
29.6. Where this goes . . . 380
29.7. Exercises . . . 381

**VI. Volume 6 — Probability and data** **389**

**30. Volume 6 — Probability and data** **391**
30.1. Chapters . . . 391

**31. Probability theory** **393**
31.1. What the notation is saying . . . 393
31.1.1. Sample spaces and events . . . 393
31.2. The rules . . . 397
31.2.1. Rule 1: Complement . . . 397
31.2.2. Rule 2: Addition rule . . . 397
31.2.3. Rule 3: Conditional probability . . . 398
31.2.4. Rule 4: Multiplication rule . . . 405
31.2.5. Rule 5: Independence . . . 405
31.2.6. Bayes' theorem . . . 408
31.3. Worked examples . . . 409
31.3.1. Example 1 — Medical testing (sciences) . . . 409
31.3.2. Example 2 — Cards without replacement (finance) . . . 410
31.3.3. Example 3 — Spam filter (computing) . . . 410
31.3.4. Example 4 — Weather (geography and environment) . . . 411
31.4. Where this goes . . . 412
31.4.1. Probability distributions . . . 412
31.4.2. Bayesian reasoning in computing and machine learning . . . 412

31.5. Applications . . . 413
31.6. Exercises . . . 413
31.6.1. Exercise 1 . . . 414
31.6.2. Exercise 2 . . . 415
31.6.3. Exercise 3 . . . 415
31.6.4. Exercise 4 . . . 416
31.6.5. Exercise 5 . . . 416
31.6.6. Exercise 6 — Multi-step problem . . . 416

**32. Probability distributions** **419**
32.1. What the notation is saying . . . 419
32.1.1. Random variables . . . 419
32.1.2. Mean, variance, and standard deviation . . . 420
32.2. The binomial distribution . . . 420
32.2.1. Deriving the formula . . . 421
32.2.2. Mean and variance of the binomial . . . 422
32.3. The normal distribution . . . 425
32.3.1. Why the bell curve appears everywhere . . . 426
32.3.2. The 68-95-99.7 rule . . . 426
32.3.3. Standardising: the z-score . . . 426
32.4. Worked examples . . . 431
32.4.1. Example 1 — Binomial exact (quality control) . . . 431
32.4.2. Example 2 — Binomial complement (healthcare) . . . 433
32.4.3. Example 3 — Normal z-score (education) . . . 434
32.4.4. Example 4 — Normal reverse lookup (pass with distinction) . . . 435
32.5. Where this goes . . . 436
32.6. Applications . . . 436
32.7. Exercises . . . 436

**33. Statistical Inference** **441**
33.1. Populations and samples . . . 441
33.1.1. What makes a sample useful . . . 442
33.2. The sampling distribution . . . 442
33.3. The Central Limit Theorem . . . 446
33.3.1. Why this is surprising . . . 447
33.3.2. Working with the standard error — a numerical example . . . 447
33.4. Confidence intervals . . . 448
33.4.1. Building a 95% confidence interval . . . 448
33.4.2. Worked example — confidence interval . . . 449
33.4.3. What “95% confident” actually means . . . 449
33.5. Hypothesis testing . . . 450
33.5.1. The logic of the test . . . 450
33.5.2. Setting up the hypotheses . . . 450
33.5.3. The test statistic . . . 451
33.5.4. The p-value . . . 451
33.5.5. The decision rule . . . 451
33.6. Worked example — full hypothesis test . . . 456

33.7. Type I and Type II errors . . . 458
33.7.1. Why this tradeoff matters . . . 458
33.8. Connecting to the bigger picture . . . 458
33.9. Where this goes . . . 459
33.10. Exercises . . . 459

**34. Data Analysis** **465**
34.1. 1 · Types of data . . . 465
34.2. 2 · Describing one variable . . . 466
34.2.1. Centre . . . 466
34.2.2. Spread . . . 467
34.2.3. Shape: the five-number summary and box plot . . . 468
34.3. 3 · Describing relationships . . . 468
34.4. 4 · Pearson correlation . . . 469
34.5. 5 · Correlation does not imply causation . . . 473
34.6. 6 · Linear regression . . . 475
34.7. 7 · $R^2$ and residual plots . . . 478
34.7.1. The coefficient of determination . . . 478
34.7.2. Residual plots . . . 479
34.8. 8 · End-to-end worked example . . . 480
34.9. Exercises . . . 482
34.10. Where this chapter leaves you . . . 487

**VII. Volume 7 — Engineering mathematics: ODEs** **489**

**35. Volume 7 — Engineering mathematics** **491**
35.1. Sections . . . 491

**36. First-order ODEs** **493**
36.1. The ODE problem . . . 493
36.1.1. Order and linearity . . . 494
36.1.2. General and particular solutions . . . 494
36.1.3. Why existence and uniqueness matters . . . 494
36.2. Separable ODEs . . . 495
36.2.1. Worked example 1: exponential growth and decay . . . 495
36.2.2. Worked example 2: Newton's law of cooling . . . 496
36.3. Linear first-order ODEs . . . 499
36.3.1. The integrating factor . . . 499
36.3.2. Worked example: RC circuit . . . 500
36.4. Applications . . . 504
36.4.1. Mixing problem . . . 504
36.4.2. Drug pharmacokinetics: one-compartment model . . . 505
36.5. Existence and uniqueness . . . 506
36.6. Exercises . . . 506
36.7. Where this goes . . . 513

**37. Second-order ODEs** **515**
37.1. Structure of the general equation . . . 515

37.2. Homogeneous equations: the characteristic equation . . . 516
37.2.1. Case 1: Two distinct real roots ($\Delta > 0$) . . . 517
37.2.2. Case 2: Repeated real root ($\Delta = 0$) . . . 517
37.2.3. Case 3: Complex conjugate roots ($\Delta < 0$) . . . 518
37.3. The spring-mass-damper system . . . 521
37.3.1. Worked numerical example . . . 522
37.4. Method of undetermined coefficients . . . 523
37.4.1. Standard guess table . . . 524
37.4.2. Worked example 1: exponential forcing . . . 524
37.4.3. Worked example 2: sinusoidal forcing . . . 525
37.4.4. Modification rule in action . . . 525
37.5. Forced oscillation and resonance . . . 526
37.6. Reduction of order . . . 530
37.7. Where this leads . . . 531
37.8. Exercises . . . 531

**38. Systems and phase plane** **537**
38.1. Systems of first-order ODEs . . . 537
38.1.1. From a single ODE to a system . . . 537
38.1.2. Matrix form . . . 538
38.1.3. More general systems . . . 538
38.2. The phase plane . . . 539
38.2.1. Trajectories as curves . . . 539
38.2.2. Direction fields . . . 539
38.2.3. Equilibrium points . . . 539
38.3. Linear systems and the eigenvalue method . . . 540
38.3.1. The trial solution . . . 540
38.3.2. General solution . . . 540
38.3.3. Classification of equilibria . . . 540
38.4. Stability . . . 547
38.4.1. Lyapunov's definition . . . 547
38.4.2. Engineering implications . . . 547
38.5. Nonlinear systems: linearisation . . . 547
38.5.1. Why linearise? . . . 547
38.5.2. The Jacobian . . . 548
38.5.3. The pendulum . . . 548
38.6. Lotka-Volterra predator-prey . . . 549
38.6.1. The equations . . . 549
38.6.2. Equilibria . . . 549
38.6.3. Classifying via the Jacobian . . . 549
38.6.4. Physical interpretation . . . 550
38.7. Application: Coupled spring system . . . 555
38.8. Exercises . . . 555
38.9. Where this goes . . . 559

**39. Laplace transforms** **561**
39.1. Definition and motivation . . . 561
39.2. Table of transforms . . . 562

39.3. The transform of derivatives . . . 567
39.4. Solving initial value problems . . . 568
39.4.1. First-order example . . . 568
39.4.2. Second-order example . . . 569
39.5. Partial fractions . . . 570
39.5.1. Distinct real poles . . . 570
39.5.2. Complex conjugate poles . . . 570
39.6. The Heaviside step function and piecewise forcing . . . 571
39.6.1. Worked example: RC circuit with a step input . . . 571
39.7. The Dirac delta function . . . 575
39.7.1. Impulse response of a second-order system . . . 575
39.8. Convolution theorem . . . 576
39.9. Where this goes . . . 577
39.10. Exercises . . . 577

**VIII. Volume 7 — Linear algebra and vector calculus 583**

**40. Matrices and linear systems 585**
40.1. Matrices and operations . . . 585
40.1.1. Special types . . . 586
40.1.2. Matrix addition and scalar multiplication . . . 586
40.1.3. Matrix multiplication . . . 586
40.2. Linear systems: Ax = b . . . 592
40.2.1. The augmented matrix . . . 593
40.2.2. What a solution means geometrically . . . 593
40.3. Gaussian elimination . . . 594
40.3.1. Worked example: 3×3 system . . . 594
40.3.2. The pivot . . . 595
40.3.3. Gauss-Jordan elimination and RREF . . . 597
40.4. Solution types and rank . . . 598
40.4.1. The three cases . . . 598
40.4.2. Rank . . . 601
40.4.3. Rank-nullity theorem . . . 602
40.4.4. What rank tells you before you compute . . . 602
40.5. LU factorisation . . . 604
40.5.1. Solving with LU . . . 604
40.5.2. Worked example: 3×3 LU factorisation . . . 605
40.5.3. When LU fails . . . 605
40.5.4. Numerical perspective . . . 606
40.6. Applications . . . 606
40.6.1. Circuit mesh analysis: Kirchhoff's voltage law . . . 606
40.6.2. Structural force distribution . . . 607
40.7. Exercises . . . 607
40.7.1. Exercise 1: Node admittance matrix multiplication . . . 609
40.7.2. Exercise 2: Mesh current analysis by Gaussian elimination . . . 610
40.7.3. Exercise 3: RREF and solution type (underdetermined structural system) . . . 611
40.7.4. Exercise 4: Rank of a sensor measurement matrix . . . 612

40.7.5. Exercise 5: LU factorisation for repeated solves . . . 612
40.7.6. Exercise 6: Write and solve the linear system for a 3-mesh circuit . . . 614

**41. Eigenvalues and eigenvectors** **617**
41.1. The eigenvalue problem . . . 617
41.2. Characteristic equation . . . 620
41.3. Finding eigenvalues and eigenvectors — worked examples . . . 621
41.3.1. Example 1: A 2×2 electrical network matrix . . . 621
41.3.2. Example 2: A 3×3 two-storey building stiffness matrix . . . 623
41.4. Repeated eigenvalues . . . 624
41.5. Diagonalisation . . . 625
41.6. Symmetric matrices and the spectral theorem . . . 626
41.7. Applications . . . 626
41.7.1. Vibrational modes of a two-mass spring system . . . 626
41.7.2. Power iteration and PageRank . . . 627
41.7.3. Principal component analysis (PCA) — a preview . . . 631
41.8. Exercises . . . 631
41.9. Where this goes . . . 638

**42. Vector differential calculus** **639**
42.1. Vectors: dot and cross products . . . 639
42.1.1. The dot product . . . 639
42.1.2. The cross product . . . 640
42.2. Scalar and vector fields . . . 640
42.3. The gradient . . . 640
42.3.1. Partial derivatives . . . 640
42.3.2. Definition . . . 641
42.3.3. Geometric meaning . . . 641
42.3.4. Directional derivative . . . 644
42.3.5. Total differential . . . 645
42.4. Divergence . . . 645
42.4.1. Definition . . . 645
42.4.2. Physical meaning . . . 645
42.5. Curl . . . 646
42.5.1. Definition . . . 646
42.5.2. Physical meaning . . . 647
42.6. The Laplacian . . . 649
42.6.1. Definition . . . 649
42.6.2. Physical meaning . . . 650
42.7. Applications . . . 650
42.7.1. Gradient descent and optimisation . . . 650
42.7.2. Electric field from voltage potential . . . 651
42.7.3. Continuity equation in fluid mechanics . . . 651
42.8. Exercises . . . 652

**43. Vector integral calculus** **657**
43.1. Line integrals . . . 657
43.1.1. Scalar line integrals . . . 657

43.1.2. Vector line integrals (work) . . . 658
43.2. Path independence and conservative fields . . . 662
43.2.1. Fundamental Theorem for line integrals . . . 662
43.3. Green's theorem . . . 663
43.3.1. Where it comes from: cellular decomposition . . . 663
43.4. Surface integrals . . . 666
43.4.1. Parametric surfaces . . . 667
43.4.2. Flux integrals . . . 667
43.5. The Divergence theorem . . . 668
43.5.1. Physical interpretation . . . 668
43.5.2. Cellular decomposition in 3D . . . 668
43.6. Stokes' theorem . . . 671
43.6.1. Connection to Green's theorem . . . 672
43.6.2. The cellular argument for Stokes . . . 672
43.7. The unified picture . . . 673
43.8. Applications . . . 674
43.8.1. Electric flux and Gauss's law . . . 674
43.8.2. Mass flow through a surface (fluid mechanics) . . . 674
43.8.3. Work done against gravity (conservative field) . . . 674
43.8.4. Stokes' theorem and Faraday's law . . . 675
43.9. Exercises . . . 675
43.10. Where this goes . . . 682

**IX. Volume 7 — Fourier analysis and PDEs** **683**

**44. Fourier series** **685**
44.1. Periodic functions . . . 685
44.2. The Fourier series . . . 686
44.2.1. Why these formulas: orthogonality . . . 686
44.3. Worked example: the square wave . . . 687
44.4. Convergence and the Gibbs phenomenon . . . 695
44.5. Even and odd functions . . . 695
44.6. Half-range series . . . 696
44.7. Complex Fourier series . . . 696
44.8. Parseval's theorem . . . 701
44.9. Applications . . . 702
44.9.1. Signal analysis and filtering . . . 702
44.9.2. Heat equation initial conditions . . . 702
44.10. Exercises . . . 702
44.10.1. Exercise 1: Fourier coefficients of the square wave . . . 704
44.10.2. Exercise 2: Sawtooth wave Fourier coefficients . . . 705
44.10.3. Exercise 3: Half-range cosine series for $f(x) = x$ . . . 706
44.10.4. Exercise 4: Half-range sine series for $f(x) = 1$ . . . 706
44.10.5. Exercise 5: Parseval's theorem and a series sum . . . 707
44.10.6. Exercise 6: Heat equation — reading off the solution from a Fourier series . . . 708

**45. Fourier transforms** **711**
45.1. From Fourier series to the Fourier integral . . . 711
45.2. The Fourier transform . . . 715
45.3. Key transform pairs . . . 719
45.3.1. Rectangular pulse . . . 719
45.3.2. Gaussian . . . 720
45.3.3. Dirac delta . . . 720
45.4. Properties of the Fourier transform . . . 721
45.5. The convolution theorem in engineering . . . 721
45.6. Solving differential equations with Fourier transforms . . . 727
45.7. Connection to the Laplace transform . . . 728
45.8. Exercises . . . 728
45.8.1. Exercise 1: Transform of a rectangular pulse . . . 730
45.8.2. Exercise 2: Time-shifting theorem . . . 730
45.8.3. Exercise 3: Differentiation property — solving an ODE . . . 731
45.8.4. Exercise 4: Convolution theorem — LTI system response . . . 732
45.8.5. Exercise 5: Parseval's theorem — energy in the frequency domain . . . 733

**46. Partial differential equations** **735**
46.1. Classification of second-order PDEs . . . 735
46.2. Separation of variables . . . 740
46.3. The heat equation . . . 741
46.3.1. Derivation . . . 741
46.3.2. Boundary and initial conditions . . . 741
46.3.3. Solution by separation of variables . . . 741
46.4. The wave equation . . . 744
46.4.1. Setup . . . 744
46.4.2. Solution by separation of variables . . . 744
46.4.3. Standing waves and natural frequencies . . . 745
46.4.4. D'Alembert's solution . . . 746
46.5. The Laplace equation . . . 747
46.5.1. Setup . . . 747
46.5.2. Dirichlet problem in a rectangle . . . 747
46.6. Summary: the method in four steps . . . 748
46.7. Exercises . . . 748
46.7.1. Exercise 1: Heat equation — single-mode initial condition . . . 750
46.7.2. Exercise 2: Heat equation — arbitrary initial condition . . . 751
46.7.3. Exercise 3: Wave equation — plucked string . . . 751
46.7.4. Exercise 4: Laplace equation — steady-state temperature . . . 752
46.7.5. Exercise 5: Classify these PDEs . . . 753
46.7.6. Exercise 6: Non-homogeneous boundary conditions . . . 754

**X. Volume 7 — Complex analysis** **755**

**47. Complex functions** **757**
47.1. Complex numbers: brief review . . . 761
47.2. Complex functions . . . 765

47.3. Complex differentiability . . . 765
47.3.1. Deriving the Cauchy-Riemann equations . . . 766
47.4. Analytic functions . . . 773
47.4.1. Harmonic functions . . . 774
47.5. Elementary analytic functions . . . 780
47.5.1. Exponential . . . 780
47.5.2. Trigonometric . . . 784
47.5.3. Logarithm . . . 784
47.5.4. Complex powers . . . 790
47.6. Conformal mappings: preview . . . 790
47.7. Exercises . . . 795
47.7.1. Exercise 1: Cauchy-Riemann equations for $f(z) = z^2$ . . . 796
47.7.2. Exercise 2: Finding a harmonic conjugate . . . 797
47.7.3. Exercise 3: $e^z$ is entire . . . 798
47.7.4. Exercise 4: Where is $f(z) = |z|^2$ differentiable? . . . 798
47.7.5. Exercise 5: Powers via De Moivre . . . 799

**48. Complex integration** **803**
48.1. Contours and contour integrals . . . 803
48.1.1. The ML inequality . . . 810
48.1.2. Integral of $1/z^{n+1}$ around a circle . . . 815
48.2. Cauchy's theorem . . . 820
48.3. Cauchy's integral formula . . . 825
48.3.1. Derivatives of all orders . . . 826
48.4. Major consequences . . . 826
48.4.1. Liouville's theorem . . . 826
48.4.2. Fundamental theorem of algebra . . . 829
48.4.3. Morera's theorem . . . 830
48.5. Application: real integrals via contour integration . . . 830
48.6. Exercises . . . 842
48.6.1. Exercise 1: Direct computation of a contour integral . . . 844
48.6.2. Exercise 2: Cauchy's theorem — zero integral . . . 844
48.6.3. Exercise 3: Cauchy's integral formula . . . 845
48.6.4. Exercise 4: Derivative formula . . . 845
48.6.5. Exercise 5: ML inequality estimate . . . 846

**49. Series and residues** **847**
49.1. Taylor series for analytic functions . . . 847
49.2. Laurent series . . . 854
49.3. Classification of isolated singularities . . . 865
49.4. Residues . . . 869
49.4.1. Computing residues . . . 875
49.5. The residue theorem . . . 876
49.6. Evaluating real integrals . . . 876
49.6.1. Type 1: trigonometric integrals $\int_0^{2\pi} R(\cos\theta, \sin\theta)\, d\theta$ . . . 876
49.6.2. Type 2: improper integrals $\int_{-\infty}^{\infty} R(x)\, dx$ . . . 876
49.7. Exercises . . . 886
49.7.1. Exercise 1: Laurent series around a pole . . . 887

49.7.2. Exercise 2: Classifying $\sin z/z^2$ at $z = 0$ . . . 888
49.7.3. Exercise 3: Residue at a simple pole . . . 888
49.7.4. Exercise 4: Residue at a double pole . . . 889
49.7.5. Exercise 5: Trigonometric integral . . . 889
49.7.6. Exercise 6: Improper real integral via residues . . . 890

**50. Conformal mapping** **891**
50.1. Conformal mappings . . . 891
50.2. Möbius transformations . . . 902
50.2.1. Fixed points . . . 902
50.3. Standard conformal maps . . . 913
50.3.1. $w = z^2$ . . . 913
50.3.2. $w = e^z$ . . . 914
50.3.3. $w = \log z$ . . . 914
50.4. The Joukowski transformation . . . 918
50.5. Application to potential theory . . . 925
50.5.1. Complex potential . . . 926
50.5.2. Transforming the domain . . . 926
50.6. Exercises . . . 931
50.6.1. Exercise 1: Image of a circle under $w = z^2$ . . . 932
50.6.2. Exercise 2: Image of a strip under $w = e^z$ . . . 933
50.6.3. Exercise 3: Möbius transformation from three point pairs . . . 934
50.6.4. Exercise 4: Joukowski map — circle to ellipse . . . 934
50.6.5. Exercise 5: Potential theory — flow in a corner . . . 935

**XI. Volume 7 — Numerical methods** **937**

**51. Numerical methods** **939**
51.1. Root-finding . . . 939
51.1.1. The problem . . . 939
51.1.2. Bisection method . . . 939
51.1.3. Newton-Raphson method . . . 943
51.1.4. Secant method . . . 947
51.2. Interpolation . . . 947
51.2.1. The problem . . . 947
51.2.2. Lagrange interpolation . . . 947
51.2.3. Newton's divided-difference form . . . 949
51.2.4. Runge's phenomenon and why splines exist . . . 949
51.3. Numerical integration . . . 949
51.3.1. The problem . . . 949
51.3.2. Trapezoid rule . . . 950
51.3.3. Simpson's 1/3 rule . . . 951
51.3.4. Simpson's 3/8 rule . . . 951
51.3.5. Gaussian quadrature . . . 954
51.4. Where this goes . . . 955
51.5. Exercises . . . 956

**52. Numerical linear algebra** **957**
52.1. What the notation is saying . . . 957
52.2. Gaussian elimination with partial pivoting . . . 958
52.2.1. Why naive elimination fails . . . 958
52.2.2. Partial pivoting: the fix . . . 959
52.2.3. Worked example: partial pivoting on a 3×3 system . . . 959
52.3. LU decomposition . . . 962
52.3.1. Doolittle form . . . 963
52.3.2. Why LU earns its keep . . . 963
52.3.3. Worked example: Doolittle LU on a 3×3 system . . . 963
52.4. Condition number . . . 965
52.4.1. Definition . . . 965
52.4.2. Ill-conditioning in practice: the 3×3 Hilbert matrix . . . 965
52.5. Iterative methods . . . 968
52.5.1. Jacobi iteration . . . 968
52.5.2. Gauss-Seidel iteration . . . 969
52.5.3. Convergence criterion: diagonal dominance . . . 969
52.5.4. Worked example: Jacobi and Gauss-Seidel on a 3×3 system . . . 969
52.6. Where this goes . . . 972
52.7. Exercises . . . 972

**53. Numerical ODEs and PDEs** **975**
53.1. The setup: replacing dy/dx with Δy/Δx . . . 975
53.2. Euler's method . . . 975
53.2.1. Forward Euler . . . 975
53.2.2. Backward (implicit) Euler . . . 976
53.2.3. Worked example: forward Euler on $y' = -2y$ . . . 976
53.3. Runge-Kutta methods . . . 979
53.3.1. RK2: two-slope methods . . . 979
53.3.2. RK4: the standard workhorse . . . 979
53.3.3. Worked example: RK4 on $y' = y$, one step . . . 980
53.4. Systems of ODEs . . . 980
53.4.1. Worked example: damped oscillator . . . 981
53.5. Finite differences for PDEs . . . 981
53.5.1. The 1D heat equation . . . 982
53.5.2. Crank-Nicolson: unconditional stability . . . 985
53.5.3. Boundary-value problems: tridiagonal systems from central differences . . . 985
53.6. Worked example: FTCS one step . . . 986
53.7. Where this goes . . . 986
53.8. Applications . . . 987
53.9. Exercises . . . 987

**XII. Volume 7 — Optimisation and probability** **989**

**54. Linear programming** **991**
54.1. LP formulation . . . 991
54.2. Graphical method . . . 993

54.3. Simplex method . . . 998
54.4. Duality . . . 1002
54.5. Sensitivity analysis . . . 1004
54.6. Where this goes . . . 1004
54.7. Exercises . . . 1004

**55. Graphs and networks** **1007**
55.1. Graph fundamentals . . . 1007
55.1.1. Adjacency matrix . . . 1007
55.1.2. Degree, paths, and connectivity . . . 1008
55.2. Shortest path: Dijkstra's algorithm . . . 1008
55.2.1. Worked trace — 6-node graph . . . 1008
55.2.2. Correctness sketch . . . 1011
55.3. Minimum spanning tree . . . 1012
55.3.1. Cut property . . . 1012
55.3.2. Kruskal's algorithm . . . 1012
55.3.3. Prim's algorithm . . . 1013
55.4. Network flows . . . 1013
55.4.1. Ford-Fulkerson method . . . 1014
55.4.2. Max-flow min-cut theorem . . . 1014
55.4.3. Worked example — 5-node network . . . 1014
55.5. Where this goes . . . 1019
55.6. Exercises . . . 1019

**56. Probability theory** **1023**
56.1. Probability axioms . . . 1023
56.1.1. Sample space and events . . . 1023
56.1.2. Kolmogorov axioms . . . 1023
56.1.3. Conditional probability . . . 1024
56.1.4. Bayes' theorem . . . 1028
56.2. Random variables . . . 1030
56.2.1. Expectation . . . 1030
56.2.2. Variance . . . 1031
56.3. Key discrete distributions . . . 1031
56.3.1. Bernoulli . . . 1031
56.3.2. Binomial . . . 1032
56.3.3. Poisson . . . 1032
56.4. Key continuous distributions . . . 1040
56.4.1. Uniform . . . 1040
56.4.2. Exponential . . . 1040
56.4.3. Normal . . . 1041
56.5. Joint distributions . . . 1047
56.6. Central Limit Theorem . . . 1051
56.7. Where this goes . . . 1057
56.8. Exercises . . . 1057

**57. Mathematical statistics** **1059**
57.1. Point estimation . . . 1059
57.1.1. Setting up the likelihood . . . 1059
57.1.2. MLE for the Bernoulli distribution . . . 1060
57.1.3. MLE for the Exponential distribution . . . 1060
57.1.4. Unbiasedness and consistency . . . 1061
57.2. Confidence intervals . . . 1061
57.2.1. The problem with small samples . . . 1061
57.2.2. Constructing a 95% CI for $\mu$ . . . 1062
57.2.3. What a confidence interval means — and what it does not . . . 1062
57.3. Hypothesis testing . . . 1066
57.3.1. The framework . . . 1067
57.3.2. One-sample $t$-test . . . 1067
57.3.3. Two-sample $t$-test . . . 1068
57.3.4. $\chi^2$ goodness-of-fit . . . 1068
57.4. Linear regression . . . 1068
57.4.1. The model . . . 1068
57.4.2. OLS estimators . . . 1069
57.4.3. $R^2$ — coefficient of determination . . . 1070
57.4.4. $t$-test for $\beta_1$ . . . 1070
57.4.5. Residual analysis . . . 1070
57.5. One-way ANOVA . . . 1074
57.5.1. The F-test . . . 1074
57.6. Where this goes . . . 1075
57.7. Exercises . . . 1076

**XIII.Volume 8 — Upper-year engineering mathematics** **1079**

**58. Volume 8 — Upper-year engineering mathematics** **1081**
58.1. Chapters . . . 1081
58.2. U of A anchors . . . 1082
58.3. Pipeline notes . . . 1082
58.4. Optional viewpoints . . . 1082

**59. Control, feedback, and stability** **1083**
59.1. The control problem . . . 1083
59.2. What stability means . . . 1090
59.3. The core method . . . 1091
59.4. Worked example 1: cruise control with proportional feedback . . . 1091
59.5. Worked example 2: state feedback for a two-state system . . . 1095
59.6. Worked example 3: stabilising a scientific instrument . . . 1099
59.7. Where this goes . . . 1100
59.8. Exercises . . . 1100
59.8.1. Exercise 1 . . . 1102
59.8.2. Exercise 2 . . . 1103
59.8.3. Exercise 3 . . . 1103
59.8.4. Exercise 4 . . . 1104

**60. Discrete-time systems and signal processing** **1109**
60.1. The discrete-time view . . . 1109
60.2. Why sampling can lie to you . . . 1118
60.3. Difference equations and recurrence . . . 1119
60.4. The core method . . . 1121
60.5. Worked example 1: a moving-average sensor filter . . . 1121
60.6. Worked example 2: discrete-time stability of a sampled update . . . 1122
60.7. Worked example 3: aliasing in a seismic record . . . 1122
60.8. Where this goes . . . 1123
60.9. Exercises . . . 1123
60.9.1. Exercise 1 . . . 1123
60.9.2. Exercise 2 . . . 1123
60.9.3. Exercise 3 . . . 1124
60.9.4. Exercise 4 . . . 1124

**61. Continuum mechanics and transport** **1129**
61.1. The local balance law . . . 1129
61.2. Constitutive laws . . . 1135
61.3. Why signs and directions matter . . . 1136
61.4. The core method . . . 1137
61.5. Worked example 1: heat conduction in one dimension . . . 1138
61.6. Worked example 2: contaminant diffusion in groundwater . . . 1139
61.7. Worked example 3: linear elasticity in one dimension . . . 1140
61.8. Where this goes . . . 1141
61.9. Exercises . . . 1141
61.9.1. Exercise 1 . . . 1141
61.9.2. Exercise 2 . . . 1141
61.9.3. Exercise 3 . . . 1142
61.9.4. Exercise 4 . . . 1142

**62. Computational methods for engineering models** **1145**
62.1. From equation to algebraic system . . . 1145
62.2. Three dominant viewpoints . . . 1150
62.3. Stability, consistency, and convergence . . . 1150
62.4. The core method . . . 1153
62.5. Worked example 1: explicit diffusion update . . . 1153
62.6. Worked example 2: a finite-volume mass balance . . . 1154
62.7. Worked example 3: why finite elements dominate structural analysis . . . 1154
62.8. Where this goes . . . 1155
62.9. Exercises . . . 1156
62.9.1. Exercise 1 . . . 1156
62.9.2. Exercise 2 . . . 1156
62.9.3. Exercise 3 . . . 1156
62.9.4. Exercise 4 . . . 1156

**63. Estimation, inverse problems, and filtering** **1159**
63.1. Forward and inverse viewpoints . . . 1159
63.2. The simplest update logic . . . 1166

63.3. Parameter estimation and state estimation . . . 1166
63.4. The core method . . . 1167
63.5. Worked example 1: one-step scalar filter . . . 1167
63.6. Worked example 2: linear least-squares parameter fit . . . 1169
63.7. Worked example 3: remote sensing as an inverse problem . . . 1169
63.8. The Kalman filter as the canonical example . . . 1170
63.9. Where this goes . . . 1170
63.10. Exercises . . . 1171
63.10.1. Exercise 1 . . . 1171
63.10.2. Exercise 2 . . . 1171
63.10.3. Exercise 3 . . . 1171
63.10.4. Exercise 4 . . . 1172

**64. Reliability, stochastic systems, and quality 1175**
64.1. Reliability as survival over time . . . 1175
64.1.1. Weibull model for non-constant hazard . . . 1176
64.2. Queueing as accumulation under uncertainty . . . 1180
64.3. Quality as probability of staying in tolerance . . . 1182
64.4. The core method . . . 1184
64.5. Worked example 1: exponential reliability . . . 1184
64.6. Worked example 2: a queue near saturation . . . 1185
64.7. Worked example 3: defect probability from a normal model . . . 1186
64.8. Where this goes . . . 1186
64.9. Exercises . . . 1186
64.9.1. Exercise 1 . . . 1187
64.9.2. Exercise 2 . . . 1187
64.9.3. Exercise 3 . . . 1187
64.9.4. Exercise 4 . . . 1187

**65. Nonlinear optimisation for design and operations 1189**
65.1. From linear to nonlinear thinking . . . 1189
65.2. Constrained optimisation and the Lagrangian idea . . . 1195
65.3. Tradeoffs are the real subject . . . 1196
65.4. The core method . . . 1196
65.5. Worked example 1: unconstrained quadratic design objective . . . 1197
65.6. Worked example 2: equality-constrained optimisation . . . 1197
65.7. Worked example 3: design under a penalty tradeoff . . . 1198
65.8. Where this goes . . . 1199
65.9. Exercises . . . 1199
65.9.1. Exercise 1 . . . 1199
65.9.2. Exercise 2 . . . 1199
65.9.3. Exercise 3 . . . 1200
65.9.4. Exercise 4 . . . 1200

# 1. Mathematics

A series for people who want to understand, not just pass

Most mathematics education answers the question *how*. This series tries to answer *what*, *why*, and *where this goes*.

It starts where the difficulty usually starts — not at the beginning of mathematics, but at the point where the notation begins to feel like it belongs to someone else. Each volume takes a set of tools, shows what they actually are, and points toward where they're used in the world.

The series runs from the foundations of arithmetic through to upper-year engineering mathematics. You don't have to start at Volume 1. Start where you lost the thread.

## 1.1. Start here

If you want orientation before choosing a chapter, begin with How to Use This Series. It explains where to start, how chapters are structured, what the notation is doing, and how the volumes connect to different fields.

## 1.2. The volumes

| Volume | Content | Roughly equivalent to |
|---|---|---|
| 1 | Numbers and operations | Alberta Gr 7–8 |
| 2 | Patterns and algebra | Alberta Gr 8–9 |
| 3 | Shape and measure | Alberta Gr 9–10 |
| 4 | Functions and change | Alberta Gr 10–11 |
| 5 | Analysis and proof | Alberta Gr 11–12 |
| 6 | Probability and data | Alberta Gr 11–12 |
| 7 | Engineering mathematics | U of A first and second year engineering maths |
| 8 | Upper-year engineering mathematics | U of A years 3 and 4 mathematical methods |

The grade equivalents are approximate and are there for orientation only. They are not labels.

## 1.3. How to use this series

Each chapter opens with what you need coming in, and closes with what it opens up. The exercises are puzzles, not drills — they're designed to be interesting rather than repetitive.

If a piece of notation stops you, look for the plain-language reading immediately beside it. Notation is a compression tool, not a gatekeeping device. This series treats it as such.

# 2. How to Use This Series

Where to start, how to navigate, what the notation means

## 2.1. Where to start

You do not have to start at Volume 1.

The series runs from arithmetic to engineering mathematics, but it is not a ladder. Find the place where you lost the thread and start there, then move outward from that point.

If you are not sure where that is, a rough guide:

| Where you are | Start here |
|---|---|
| Struggled with high school maths | Volume 1 or 2 |
| Passed high school, never felt sure why | Volume 3 or 4 |
| About to start university maths | Volume 5 |
| In first- or second-year engineering, finding the maths hard | Volume 7 |
| In third- or fourth-year engineering, the maths has become embedded in discipline courses | Volume 8 |

## 2.2. How chapters are structured

Every chapter opens with two or three concrete situations that share the same underlying mathematical structure. The mathematics comes from the situation, not the other way round.

After the opening:

1. The notation is introduced and read in plain English beside it
2. The method is shown, stepped through, with a verification
3. A brief explanation of why the method works
4. Two to four worked examples from different fields
5. Where the chapter leads — the specific topics it opens up
6. Exercises

## 2.3. The exercises

Exercises come in four types. The type is stated at the start of each exercise set.

**Puzzle** — convergent, satisfying, one clean answer. The interesting work is setting up the equation, not solving it.

**Drill** — repetitive by design. Some things require fluency, and fluency requires repetition.

**Investigation** — open-ended, more than one valid answer. These ask you to explore.

**Project** — extended, usually involving a real dataset or context.

## 2.4. What the notation is doing

Notation is a compression tool. It lets you write in three symbols what would otherwise take three sentences. This series introduces each symbol when it is first needed, reads it in plain English immediately beside it, and notes how heavy a lift it is — some symbols need one sentence, some need a whole section.

If a piece of notation stops you, look for the plain-language reading in the same paragraph. It will be there.

## 2.5. The destination fields

Every chapter is tagged with the fields it serves:

- **Engineering** — structural, electrical, mechanical, chemical, software
- **Hard sciences** — physics, chemistry, biology, earth sciences
- **Finance and quant** — financial modelling, actuarial science, econometrics
- **Computing and data** — CS, machine learning, data science, cryptography
- **Geography and environment** — GIS, climate science, environmental modelling

The tags are there to answer the question "why am I learning this". Each chapter's worked examples cover at least three of its tagged fields.

## 2.6. The prerequisites

Each chapter states what you need coming in. If a prerequisite chapter is incomplete in your knowledge, step back to the most recent chapter that still feels unstable and rebuild from there.

The prerequisites are genuine — not padding. If a chapter says it requires limits, it means you will need limits to follow the argument.

# Part I.

# Volume 1 — Numbers and operations

# 3. Volume 1 — Numbers and operations

The central move of this volume is placement.

Every number has a location: on a line, relative to zero, relative to other numbers. This volume is about understanding that structure — not as abstract symbol manipulation, but as a description of real things. Debts and credits. Divided portions. Rates of change. Samples from populations.

You already know that numbers exist. What this volume builds is fluency with how they relate — how to scale them (ratio), express them as parts of wholes (fractions and decimals), talk about change (percents and rates), move in both directions from zero (integers), and reason about collections of them (statistics and probability).

None of this requires anything beyond counting. But it requires counting carefully, in the right direction, with the right units. That discipline is what the rest of the series stands on.

## 3.1. Chapters

| Chapter | Title | Core move |
|---|---|---|
| 1 | Numbers and the number line | Placing any number exactly |
| 2 | Ratio and proportion | Comparing two quantities of the same kind |
| 3 | Percent and rates | Expressing change as a fraction of something |
| 4 | Integers and signed arithmetic | Extending the number line in both directions |
| 5 | Fractions and decimals | Arithmetic with parts of wholes |
| 6 | Statistics and probability | Reasoning honestly from limited data |

# 4. Numbers and the number line

Placing any number exactly

A bank balance of −$40 is not the same as having no money. It tells you that you are 40 dollars below zero. An altitude of −85 metres means 85 metres below sea level. A temperature of −18°C means 18 degrees below zero.

In each case, the number does two jobs at once. It tells you which side of zero you are on, and it tells you how far from zero you are. The number line gives us a clear picture of both ideas. In this chapter, we use that picture to place numbers, compare them, and understand what their symbols mean.

## 4.1. What the notation is saying

Start with something simple. Imagine a straight road with a marker at the centre. Label that marker **0**. Now think of every number as a place on that road.

Numbers greater than zero go to the right. Numbers less than zero go to the left. A number's position tells you its value.

$$\cdots \quad -4 \quad -3 \quad -2 \quad -1 \quad 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad \cdots$$

This picture is called the **number line**. It helps us compare numbers by location. A number further left is smaller. A number further right is larger.

For example, $-5 < -1$ because $-5$ is further left on the line. This means that even though 5 is a greater distance from zero than 1, $-5$ is still the smaller number.

Fractions and decimals belong on the same line. For example, $\frac{1}{2}$ sits halfway between 0 and 1, and $-0.7$ sits between $-1$ and 0. This means the number line is not just for whole numbers. It has a place for every number we will use.

Now we can introduce one more idea: distance from zero. Sometimes we care about where a number is. Sometimes we care only about how far it is from zero.

The symbol $|x|$ means the **absolute value** of $x$. It tells us the distance from zero and ignores direction.

For example, $|-3| = 3$ and $|3| = 3$.

This means that both $-3$ and 3 are the same distance from zero, even though they are on opposite sides.

The symbols $<$ and $>$ are called **inequality symbols**. They compare positions on the line. The symbol $<$ means "is less than," and the symbol $>$ means "is greater than."

So when we write $-5 < -1$, we are saying that $-5$ is to the left of $-1$. This means $-5$ is the smaller number.

---

*Use the slider below to place any number on the line. The arrow shows its distance from zero — that distance is its absolute value.*

```
//| echo: false
nlExplorer = {
  const d3 = await require("d3@7");

  const W = 560, H = 160;
  const ML = 40, MR = 40, MT = 50, MB = 40;
  const innerW = W - ML - MR;

  const xMin = -10, xMax = 10;
  const xScale = d3.scaleLinear([xMin, xMax], [0, innerW]);

  const container = d3.create("div")
    .style("font-family", "sans-serif")
    .style("max-width", W + "px");

  // --- slider ---
  const sliderLabel = container.append("label")
    .style("display", "block")
    .style("margin-bottom", "6px")
    .style("font-size", "0.9rem");
  sliderLabel.append("span").text("Position: ");
  const valueSpan = sliderLabel.append("span")
    .style("font-weight", "bold");

  const slider = container.append("input")
    .attr("type", "range")
    .attr("min", xMin)
    .attr("max", xMax)
    .attr("step", 1)
    .attr("value", -3)
    .style("width", "100%")
    .style("margin-bottom", "6px");

  // --- SVG ---
  const svg = container.append("svg")
    .attr("width", W)
    .attr("height", H)
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("aria-label", "Interactive number line");
```

```
const g = svg.append("g")
  .attr("transform", `translate(${ML},${MT})`);

// axis line
g.append("line")
  .attr("x1", 0).attr("y1", 0)
  .attr("x2", innerW).attr("y2", 0)
  .attr("stroke", "#333").attr("stroke-width", 2);

// arrowheads on axis
g.append("polygon")
  .attr("points", `${innerW},0 ${innerW-8},-4 ${innerW-8},4`)
  .attr("fill", "#333");
g.append("polygon")
  .attr("points", `0,0 8,-4 8,4`)
  .attr("fill", "#333");

// tick marks and labels
for (let v = xMin; v <= xMax; v++) {
  const x = xScale(v);
  g.append("line")
    .attr("x1", x).attr("y1", -6)
    .attr("x2", x).attr("y2", 6)
    .attr("stroke", v === 0 ? "#222" : "#777")
    .attr("stroke-width", v === 0 ? 2 : 1);
  g.append("text")
    .attr("x", x)
    .attr("y", 22)
    .attr("text-anchor", "middle")
    .attr("font-size", v === 0 ? "13px" : "11px")
    .attr("font-weight", v === 0 ? "bold" : "normal")
    .attr("fill", "#333")
    .text(v);
}

// zero label
g.append("text")
  .attr("x", xScale(0))
  .attr("y", -14)
  .attr("text-anchor", "middle")
  .attr("font-size", "11px")
  .attr("fill", "#555")
  .text("zero");

// distance arrow (from zero to point) — drawn under the axis
const arrowGroup = g.append("g").attr("transform", "translate(0,0)");
```

```
const arrowLine = arrowGroup.append("line")
  .attr("y1", 36).attr("y2", 36)
  .attr("stroke", "#d97706")
  .attr("stroke-width", 2.5)
  .attr("stroke-dasharray", "5,3");

const arrowHead = arrowGroup.append("polygon")
  .attr("fill", "#d97706");

const distLabel = arrowGroup.append("text")
  .attr("y", 56)
  .attr("text-anchor", "middle")
  .attr("font-size", "12px")
  .attr("fill", "#b45309")
  .attr("font-weight", "bold");

// point marker
const pointGroup = g.append("g");
const pointCircle = pointGroup.append("circle")
  .attr("r", 9)
  .attr("fill", "#2563eb")
  .attr("stroke", "#1e3a8a")
  .attr("stroke-width", 2);

const pointLabel = pointGroup.append("text")
  .attr("text-anchor", "middle")
  .attr("y", -16)
  .attr("font-size", "13px")
  .attr("font-weight", "bold")
  .attr("fill", "#1e3a8a");

const dirLabel = pointGroup.append("text")
  .attr("text-anchor", "middle")
  .attr("y", -30)
  .attr("font-size", "11px")
  .attr("fill", "#475569");

// absolute value text below SVG
const absDiv = container.append("div")
  .style("font-size", "1rem")
  .style("text-align", "center")
  .style("margin-top", "4px")
  .style("color", "#1e293b");

function update(v) {
  const x = xScale(v);
  const x0 = xScale(0);
```

```
    const absV = Math.abs(v);

    valueSpan.text(v);

    // point
    pointGroup.attr("transform", `translate(${x},0)`);
    pointLabel.text(v);
    dirLabel.text(v > 0 ? "positive \u2192" : v < 0 ? "\u2190 negative" : "zero");

    // distance arrow
    if (v !== 0) {
      const left = Math.min(x, x0);
      const right = Math.max(x, x0);
      arrowLine
        .attr("x1", left + 4)
        .attr("x2", right - 4)
        .attr("visibility", "visible");
      if (v > 0) {
        arrowHead
          .attr("points", `${right},36 ${right-8},32 ${right-8},40`)
          .attr("visibility", "visible");
      } else {
        arrowHead
          .attr("points", `${left},36 ${left+8},32 ${left+8},40`)
          .attr("visibility", "visible");
      }
      distLabel
        .attr("x", (left + right) / 2)
        .attr("visibility", "visible")
        .text(`distance from zero = ${absV}`);
    } else {
      arrowLine.attr("visibility", "hidden");
      arrowHead.attr("visibility", "hidden");
      distLabel.attr("visibility", "hidden");
    }

    absDiv.text(`|${v}| = ${absV}`);
  }

  update(+slider.property("value"));

  slider.on("input", function() {
    update(+this.value);
  });

  return container.node();
}
```

## 4.2. The method

### Placing a number on the line

Suppose you want to place a number exactly. There is a simple method.

1. Determine the sign. If the number is positive, move right from zero. If the number is negative, move left from zero.
2. Determine the magnitude. The magnitude is the size of the number without its direction, so it tells you how far to move.
3. Mark the position. Each number has one exact place on the line.

This means that sign gives direction, and magnitude gives distance.

### Ordering numbers

To order numbers, place them on the line and read from left to right. Left means smaller. Right means larger.

For example:

$$-10 < -2 < 0 < 3 < 7$$

This means $-10$ is the smallest number in the list, and 7 is the largest.

### Absolute value

We have already described absolute value as distance from zero. The formula below says the same thing in a precise way.

$|x| = x$ if $x \geq 0$; $|x| = -x$ if $x < 0$.

Here, $x$ means any number. The symbol $\geq 0$ means "greater than or equal to zero." So if $x$ is already zero or positive, its absolute value is just $x$. If $x$ is negative, we use $-x$ to turn it into the matching positive distance.

For example,

$$|-7| = -(-7) = 7$$

This means that the number $-7$ is 7 units from zero.

We will justify ideas like $-(-7) = 7$ more fully in Chapter 4, when we study signed arithmetic in detail.

> **Why this works**
> "Negative" means "opposite direction from positive" — not "less real" or "doesn't exist." Absolute value removes direction because sometimes you only care about size (distance, error) not which way. When you multiply two negatives to get a positive, it is because reversing direction twice returns you to where you started.

## 4.3. Worked examples

**Example 1 — Sport.** A football team's goal difference across four matches: won by 3, lost by 2, won by 1, lost by 4. Express these as signed numbers and order them from worst to best.

First translate the results into signed numbers:

+3, −2, +1, −4

Place each on the number line and read left to right:

$$-4 < -2 < 1 < 3$$

So the order from worst to best is:

lost by 4, lost by 2, won by 1, won by 3.

This means more negative results are worse, and more positive results are better.

---

**Example 2 — Geography.** A group records these elevations relative to sea level: mountain peak +2,450 m; valley floor −60 m; coastal town +8 m; lake bed −340 m. What is the vertical distance from the lake bed to the peak?

We want the distance between two positions on the number line: 2450 and −340.

Set up the calculation:

$$|2450 - (-340)|$$

Now simplify step by step. Subtracting a negative is the same as adding the matching positive:

$$|2450 - (-340)| = |2450 + 340| = 2790$$

So the vertical distance is 2790 m.

The absolute value bars show that we want a distance, which must be positive.

---

**Example 3 — Weather.** Five cities report overnight temperatures: −12°C, +3°C, −7°C, −19°C, +1°C. Which temperature is furthest from zero? Which is closest?

Find the absolute value of each temperature:

$$|-12| = 12, \quad |3| = 3, \quad |-7| = 7, \quad |-19| = 19, \quad |1| = 1$$

Furthest from zero: −19°C (absolute value 19). Closest to zero: +1°C (absolute value 1).

This means absolute value helps us compare size without worrying about direction.

---

**Example 4 — Money.** Your weekly pocket money is $15. You owe a friend $8 from last week. You spend $4 on a snack. Express your running total as signed numbers from the start of the week.

Start at 0.

Receive $15: move to +15.

Repay the $8 debt: move 8 units left to +7.

Spend $4: move 4 more units left to +3.

Each transaction is a movement along the number line. You end up at +3, meaning $3 remaining.

This means gains move you right and costs move you left.

---

*The interactive below lets you explore addition and subtraction as movement along the line. Set a starting value A, choose an operation, and adjust B to see where you land.*

```
//| echo: false
nlArithmetic = {
  const d3 = await require("d3@7");

  const W = 560, H = 170;
  const ML = 40, MR = 40, MT = 60, MB = 30;
  const innerW = W - ML - MR;
  const xMin = -10, xMax = 10;
  const xScale = d3.scaleLinear([xMin, xMax], [0, innerW]);

  const container = d3.create("div")
    .style("font-family", "sans-serif")
    .style("max-width", W + "px");

  // --- controls row ---
  const controls = container.append("div")
    .style("display", "flex")
    .style("gap", "16px")
    .style("align-items", "flex-end")
    .style("flex-wrap", "wrap")
    .style("margin-bottom", "8px");

  function makeNumberInput(parent, labelText, defaultVal) {
    const wrap = parent.append("label")
      .style("display", "flex")
      .style("flex-direction", "column")
      .style("font-size", "0.85rem")
      .style("gap", "3px");
```

```
  wrap.append("span").text(labelText);
  const inp = wrap.append("input")
    .attr("type", "range")
    .attr("min", -10)
    .attr("max", 10)
    .attr("step", 1)
    .attr("value", defaultVal)
    .style("width", "120px");
  const val = wrap.append("span")
    .style("text-align", "center")
    .style("font-weight", "bold");
  return { inp, val };
}

const { inp: inpA, val: valA } = makeNumberInput(controls, "A (start)", 3);
const opWrap = controls.append("label")
  .style("display", "flex")
  .style("flex-direction", "column")
  .style("font-size", "0.85rem")
  .style("gap", "3px");
opWrap.append("span").text("Operation");
const opSel = opWrap.append("select")
  .style("font-size", "1rem")
  .style("padding", "2px 6px");
opSel.append("option").attr("value", "add").text("+ (add)");
opSel.append("option").attr("value", "sub").text("− (subtract)");

const { inp: inpB, val: valB } = makeNumberInput(controls, "B (amount)", -5);

// --- result line ---
const resultDiv = container.append("div")
  .style("font-size", "1rem")
  .style("text-align", "center")
  .style("margin-bottom", "6px")
  .style("font-weight", "bold")
  .style("color", "#1e293b");

// --- SVG ---
const svg = container.append("svg")
  .attr("width", W)
  .attr("height", H)
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("aria-label", "Number line showing arithmetic as movement");

const g = svg.append("g")
  .attr("transform", `translate(${ML},${MT})`);
```

```
// axis
g.append("line")
  .attr("x1", 0).attr("y1", 0)
  .attr("x2", innerW).attr("y2", 0)
  .attr("stroke", "#333").attr("stroke-width", 2);
g.append("polygon")
  .attr("points", `${innerW},0 ${innerW-8},-4 ${innerW-8},4`)
  .attr("fill", "#333");
g.append("polygon")
  .attr("points", `0,0 8,-4 8,4`)
  .attr("fill", "#333");

// ticks
for (let v = xMin; v <= xMax; v++) {
  const x = xScale(v);
  g.append("line")
    .attr("x1", x).attr("y1", -5).attr("x2", x).attr("y2", 5)
    .attr("stroke", v === 0 ? "#222" : "#888")
    .attr("stroke-width", v === 0 ? 2 : 1);
  g.append("text")
    .attr("x", x).attr("y", 20)
    .attr("text-anchor", "middle")
    .attr("font-size", "11px")
    .attr("fill", "#444")
    .text(v);
}

// movement arrow (drawn below axis)
const moveArrow = g.append("g");
const moveLine = moveArrow.append("line")
  .attr("y1", 34).attr("y2", 34)
  .attr("stroke", "#7c3aed").attr("stroke-width", 3);
const moveHead = moveArrow.append("polygon")
  .attr("fill", "#7c3aed");
const moveLabel = moveArrow.append("text")
  .attr("y", 55)
  .attr("text-anchor", "middle")
  .attr("font-size", "11px")
  .attr("fill", "#5b21b6");

// point A
const ptA = g.append("g");
ptA.append("circle").attr("r", 8)
  .attr("fill", "#0369a1").attr("stroke", "#075985").attr("stroke-width", 2);
ptA.append("text")
  .attr("y", -14).attr("text-anchor", "middle")
  .attr("font-size", "12px").attr("fill", "#075985").attr("font-weight", "bold")
```

```
  .attr("class", "ptA-label");
ptA.append("text")
  .attr("y", -26).attr("text-anchor", "middle")
  .attr("font-size", "10px").attr("fill", "#0369a1")
  .text("A");

// point result
const ptR = g.append("g");
ptR.append("circle").attr("r", 8)
  .attr("fill", "#16a34a").attr("stroke", "#15803d").attr("stroke-width", 2);
ptR.append("text")
  .attr("y", -14).attr("text-anchor", "middle")
  .attr("font-size", "12px").attr("fill", "#15803d").attr("font-weight", "bold")
  .attr("class", "ptR-label");
ptR.append("text")
  .attr("y", -26).attr("text-anchor", "middle")
  .attr("font-size", "10px").attr("fill", "#16a34a")
  .text("Result");

function clamp(v) {
  return Math.max(xMin, Math.min(xMax, v));
}

function update() {
  const a = +inpA.property("value");
  const b = +inpB.property("value");
  const op = opSel.property("value");
  const result = clamp(op === "add" ? a + b : a - b);
  const opChar = op === "add" ? "+" : "−";
  const bDisplay = op === "sub" ? b : b;

  valA.text(a);
  valB.text(b);

  const xA = xScale(a);
  const xR = xScale(result);

  ptA.attr("transform", `translate(${xA},0)`);
  ptA.select(".ptA-label").text(a);

  ptR.attr("transform", `translate(${xR},0)`);
  ptR.select(".ptR-label").text(result);

  // movement arrow
  if (a !== result) {
    const leftX = Math.min(xA, xR);
    const rightX = Math.max(xA, xR);
```

```
      moveLine.attr("x1", leftX + 4).attr("x2", rightX - 4).attr("visibility", "visible");
      if (result > a) {
        moveHead.attr("points", `${rightX},34 ${rightX-8},30 ${rightX-8},38`).attr("visibility", "vi
      } else {
        moveHead.attr("points", `${leftX},34 ${leftX+8},30 ${leftX+8},38`).attr("visibility", "visib
      }
      moveLabel
        .attr("x", (leftX + rightX) / 2)
        .attr("visibility", "visible")
        .text(`move ${Math.abs(result - a)} ${result > a ? "right" : "left"}`);
    } else {
      moveLine.attr("visibility", "hidden");
      moveHead.attr("visibility", "hidden");
      moveLabel.attr("visibility", "hidden");
    }

    resultDiv.text(`${a} ${opChar} ${b} = ${result}${Math.abs(a + (op === "add" ? b : -b)) > 10 ? "
  }

  update();
  inpA.on("input", update);
  inpB.on("input", update);
  opSel.on("change", update);

  // legend
  const legend = container.append("div")
    .style("display", "flex").style("gap", "16px").style("font-size", "0.8rem")
    .style("margin-top", "4px").style("flex-wrap", "wrap");
  const items = [
    { color: "#0369a1", label: "A — starting value" },
    { color: "#16a34a", label: "Result" },
    { color: "#7c3aed", label: "Movement (the operation)" }
  ];
  items.forEach(({ color, label }) => {
    const item = legend.append("div").style("display", "flex").style("align-items", "center").style(
    item.append("div")
      .style("width", "14px").style("height", "14px")
      .style("border-radius", "50%").style("background", color).style("flex-shrink", "0");
    item.append("span").text(label);
  });

  return container.node();
}
```

## 4.4. Where this goes

This chapter gives us the picture that the rest of the volume uses. We began by placing numbers relative to zero. Next, we will use the same idea in more situations.

In **Integers and signed arithmetic** (Chapter 4), addition and subtraction become movements left and right on this line. In **Fractions and decimals** (Chapter 5), we fill in the spaces between whole numbers more carefully.

Later in the series, the number line becomes an axis in a graph. So this is not a one-time picture. It is a model we will keep using and extending.

**Where this shows up**

- A weather forecaster reading −18°C is placing a temperature on the number line with zero at freezing.
- A diver tracking depth uses a signed number with zero at the surface.
- A games scoreboard showing −5 (penalty points) is using the same idea.
- An app showing your bank balance in red is showing you a negative number.

Same structure. Different labels.

## 4.5. Exercises

Each problem has a clean answer. The interesting part is setting up which numbers go where before you compute. Sketching a quick number line will help.

1. A diver descends from the surface to −28 m, then swims to a coral shelf at −15 m, then ascends to −5 m before surfacing. List these depths in order from deepest to shallowest.
2. Four cities report overnight temperatures: Edmonton −23°C, Vancouver +4°C, Winnipeg −31°C, Toronto −8°C. Order from coldest to warmest. What is the difference in temperature between the warmest and coldest cities?
3. A football team's weekly goal differences are: Mon −1, Tue +2, Wed −3, Thu +4, Fri −1. On which day was the result furthest from zero? *(Use absolute values to compare.)*
4. Five friends pool money for a trip. Their balances in the group fund are: +$12, −$5, +$3, −$18, +$8. Order from least to most. Who owes the most?
5. Place these numbers on a number line sketch and order them from least to greatest: −1.5, 2, 0, −3, 0.7, −0.5.
6. A submarine is at −180 m. A fish swims at −45 m. A buoy floats at +2 m. What is the distance between the submarine and the buoy? Between the fish and the submarine?

# 5. Ratio and proportion

Comparing two quantities of the same kind

A recipe for 4 people needs 300 g of flour. You're cooking for 10. How much flour do you need?

A map is drawn at 1:25,000. A road measures 8 cm on the map. How long is it on the ground?

You and two friends split a streaming subscription. The monthly cost is $14.99 for three people. One friend joins. What does each person pay now?

These three questions share the same calculation underneath.

## 5.1. What the notation is saying

Start with a simple picture. If a drink is mixed in the ratio 2:3, that means 2 equal parts of one ingredient for every 3 equal parts of the other. Ratio compares quantities by how many equal parts they have.

The ratio $a : b$ means "for every $a$ of the first thing, there are $b$ of the second." It is a comparison by division. We can also write that comparison as the fraction $\frac{a}{b}$.

This means a ratio tells us how one quantity matches another, not just their total.

Now think about scaling. If 4 people need 300 g of flour, then 8 people need twice as much. If 2 people need 300 mL of paint, then 6 people need three times as much. In both cases, the quantities change together.

When two quantities always change by the same scale factor, they are called **proportional**.

$$y = kx$$

Here, $x$ and $y$ are the two quantities, and $k$ is a fixed number. That fixed number tells us how much of $y$ goes with 1 unit of $x$.

This means $k$ is the constant multiplier linking the two quantities.

We also write this as $y \propto x$, read as "$y$ is proportional to $x$."

A proportion equation sets two ratios equal:

$$\frac{a}{b} = \frac{c}{d}$$

*5. Ratio and proportion*

Read: "*a* is to *b* as *c* is to *d*." If you know three of the four values, you can find the fourth.

This means two comparisons are describing the same relationship.

---

*Adjust the two parts of a ratio below. The bars show how A and B relate to each other and to the total.*

```
//| echo: false
ratioVis = {
  const d3 = await require("d3@7");

  const W = 560, H = 200;
  const ML = 20, MR = 20, MT = 20, MB = 20;
  const innerW = W - ML - MR;

  const container = d3.create("div")
    .style("font-family", "sans-serif")
    .style("max-width", W + "px");

  // controls
  const controls = container.append("div")
    .style("display", "flex")
    .style("gap", "24px")
    .style("align-items", "flex-end")
    .style("flex-wrap", "wrap")
    .style("margin-bottom", "10px");

  function makeSlider(parent, labelText, defaultVal) {
    const wrap = parent.append("label")
      .style("display", "flex")
      .style("flex-direction", "column")
      .style("font-size", "0.85rem")
      .style("gap", "3px");
    const header = wrap.append("div")
      .style("display", "flex").style("justify-content", "space-between");
    header.append("span").text(labelText);
    const valSpan = header.append("span").style("font-weight", "bold");
    const inp = wrap.append("input")
      .attr("type", "range")
      .attr("min", 1).attr("max", 20).attr("step", 1)
      .attr("value", defaultVal)
      .style("width", "160px");
    return { inp, valSpan };
  }

  const { inp: inpA, valSpan: vsA } = makeSlider(controls, "Part A", 3);
  const { inp: inpB, valSpan: vsB } = makeSlider(controls, "Part B", 5);
```

```
// info line
const infoDiv = container.append("div")
  .style("font-size", "0.95rem")
  .style("text-align", "center")
  .style("margin-bottom", "10px")
  .style("color", "#1e293b");

// SVG
const svg = container.append("svg")
  .attr("width", W)
  .attr("height", H)
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("aria-label", "Ratio bar visualisation");

const g = svg.append("g").attr("transform", `translate(${ML},${MT})`);

const BAR_H = 36;
const GAP = 16;
const rows = [
  { label: "A", y: 0, fill: "#2563eb", stroke: "#1e3a8a" },
  { label: "B", y: BAR_H + GAP, fill: "#d97706", stroke: "#92400e" },
  { label: "Total (A+B)", y: (BAR_H + GAP) * 2, fill: "#64748b", stroke: "#334155" }
];

// background tracks
rows.forEach(row => {
  g.append("rect")
    .attr("x", 0).attr("y", row.y)
    .attr("width", innerW).attr("height", BAR_H)
    .attr("fill", "#f1f5f9").attr("rx", 4);
});

// bar rects
const barRects = rows.map(row => {
  return g.append("rect")
    .attr("x", 0).attr("y", row.y)
    .attr("height", BAR_H)
    .attr("fill", row.fill).attr("rx", 4);
});

// row labels (left side, outside)
rows.forEach(row => {
  g.append("text")
    .attr("x", -8)
    .attr("y", row.y + BAR_H / 2 + 4)
    .attr("text-anchor", "end")
    .attr("font-size", "12px")
```

```
    .attr("fill", "#334155")
    .attr("font-weight", "bold")
    .text(row.label);
});

// value labels inside bars
const barLabels = rows.map(row => {
  return g.append("text")
    .attr("y", row.y + BAR_H / 2 + 5)
    .attr("font-size", "13px")
    .attr("font-weight", "bold")
    .attr("fill", "#fff");
});

function update() {
  const a = +inpA.property("value");
  const b = +inpB.property("value");
  const total = a + b;
  const maxVal = 20; // slider max for A and B; total can reach 40

  vsA.text(a);
  vsB.text(b);

  const values = [a, b, total];
  const maxDisplay = 40; // total bar always represents full width when a=b=20

  values.forEach((v, i) => {
    const w = (v / maxDisplay) * innerW;
    barRects[i].attr("width", Math.max(w, 4));
    barLabels[i]
      .attr("x", Math.max(w / 2, 20))
      .attr("text-anchor", "middle")
      .text(v);
  });

  const decimal = (a / b).toFixed(3);
  infoDiv.html(
    `A : B = <strong>${a} : ${b}</strong>  |  ` +
    `A / B = <strong>${decimal}</strong>  |  ` +
    `Total parts = <strong>${total}</strong>`
  );
}

update();
inpA.on("input", update);
inpB.on("input", update);
```

```
  // legend
  const legend = container.append("div")
    .style("display", "flex").style("gap", "16px").style("font-size", "0.8rem")
    .style("margin-top", "6px").style("flex-wrap", "wrap");
  [
    { color: "#2563eb", label: "Part A" },
    { color: "#d97706", label: "Part B" },
    { color: "#64748b", label: "Total (A + B)" }
  ].forEach(({ color, label }) => {
    const item = legend.append("div").style("display", "flex").style("align-items", "center").sty
    item.append("div")
      .style("width", "14px").style("height", "14px").style("border-radius", "3px")
      .style("background", color).style("flex-shrink", "0");
    item.append("span").text(label);
  });

  return container.node();
}
```

---

## 5.2. The method

### Finding a missing value in a proportion

Suppose you know

$$\frac{a}{b} = \frac{c}{d}$$

and you want to find $d$.

Multiply both sides by $bd$. This clears the denominators and gives

$$ad = bc$$

Now divide by $a$:

$$d = \frac{bc}{a}$$

Here, $a$, $b$, and $c$ are known values, and $d$ is the missing one. This means the missing value is chosen so the two ratios stay equal.

### Dividing a quantity in a given ratio

To divide total $T$ in the ratio $a : b$:

1. Count the total number of parts: $a + b$.
2. Find one part: $T \div (a + b)$.
3. Multiply by each share: first amount = $a\times$ one part, second amount = $b\times$ one part.

Here, $T$ is the total being shared. This means we first split the whole into equal pieces, then count how many pieces each side gets.

**Cascaded ratios**

Two ratios applied in series multiply. A 3:1 ratio followed by a 4:1 ratio gives a combined ratio of $3 \times 4 = 12 : 1$.

This means one scale change happens after another.

**Gear ratio from tooth counts**

For meshing gears, the ratio equals the number of teeth on the driven gear divided by the teeth on the driving gear.

This means a larger driven gear turns more slowly, because each turn of the small gear moves only part of the larger one.

**Finding a constant of proportionality**

Given one known pair $(x_0, y_0)$ where $y \propto x$:

$$k = \frac{y_0}{x_0}$$

Then for any other $x$: $y = kx$.

Here, $(x_0, y_0)$ is one matching pair of values. This means once you know one pair, you can build the whole proportional relationship.

> **Why this works**
> A ratio is a relative relationship — it does not depend on the size of either quantity, only on their comparison. When two quantities are proportional, their ratio $k$ is constant by definition. Cross-multiplication works because multiplying both sides of $\frac{a}{b} = \frac{c}{d}$ by $bd$ gives $ad = bc$ — a balanced equation.

---

*Enter three known values in a proportion. The fourth is calculated for you. Use this to check your worked-example answers.*

```
//| echo: false
proportionSolver = {
  const d3 = await require("d3@7");

  const container = d3.create("div")
    .style("font-family", "sans-serif")
```

```
  .style("max-width", "560px");

container.append("p")
  .style("font-size", "0.85rem")
  .style("margin-bottom", "10px")
  .style("color", "#475569")
  .text("Given a/b = c/d, enter a, b, and c — d is calculated. Enter any three values to find t

// controls grid
const grid = container.append("div")
  .style("display", "grid")
  .style("grid-template-columns", "repeat(4, 1fr)")
  .style("gap", "12px")
  .style("align-items", "end")
  .style("margin-bottom", "14px");

function makeField(parent, labelText, defaultVal, isResult) {
  const wrap = parent.append("label")
    .style("display", "flex")
    .style("flex-direction", "column")
    .style("font-size", "0.85rem")
    .style("gap", "4px");
  wrap.append("span")
    .style("font-weight", "bold")
    .style("color", isResult ? "#15803d" : "#1e293b")
    .text(labelText);
  const inp = wrap.append("input")
    .attr("type", "number")
    .attr("step", "any")
    .attr("value", defaultVal)
    .style("width", "100%")
    .style("font-size", "1rem")
    .style("padding", "4px 6px")
    .style("border", isResult ? "2px solid #16a34a" : "1px solid #cbd5e1")
    .style("border-radius", "4px")
    .style("background", isResult ? "#f0fdf4" : "#fff")
    .property("readOnly", isResult);
  return inp;
}

const inpA2 = makeField(grid, "a", 3, false);
const inpB2 = makeField(grid, "b", 4, false);
const inpC2 = makeField(grid, "c", 6, false);
const inpD2 = makeField(grid, "d = b· c ÷ a", "?", true);

// equation display
const eqDiv = container.append("div")
```

```
    .style("font-size", "1.1rem")
    .style("text-align", "center")
    .style("margin-bottom", "10px")
    .style("padding", "10px")
    .style("background", "#f8fafc")
    .style("border-radius", "6px")
    .style("border", "1px solid #e2e8f0");

  // explanation line
  const explDiv = container.append("div")
    .style("font-size", "0.85rem")
    .style("color", "#475569")
    .style("text-align", "center");

  function update() {
    const a = parseFloat(inpA2.property("value"));
    const b = parseFloat(inpB2.property("value"));
    const c = parseFloat(inpC2.property("value"));

    if (isNaN(a) || isNaN(b) || isNaN(c) || a === 0) {
      inpD2.property("value", "—");
      eqDiv.text("Enter values for a, b, and c above (a must not be zero).");
      explDiv.text("");
      return;
    }

    const d = (b * c) / a;
    const dDisplay = Number.isInteger(d) ? d : d.toFixed(4);
    inpD2.property("value", dDisplay);

    eqDiv.html(
      `${a} / ${b} = ${c} / <strong style="color:#15803d">${dDisplay}</strong>`
    );
    explDiv.text(`d = b × c ÷ a = ${b} × ${c} ÷ ${a} = ${dDisplay}`);
  }

  update();
  inpA2.on("input", update);
  inpB2.on("input", update);
  inpC2.on("input", update);

  return container.node();
}
```

## 5.3. Worked examples

**Example 1 — Recipe scaling.** A recipe for 4 people needs 300 g flour, 150 g butter, and 80 g sugar. How much of each do you need for 10 people?

The number of people changes from 4 to 10, so we find the scale factor:

$$\frac{10}{4} = 2.5$$

Now multiply each ingredient by 2.5:

$$\text{flour: } 300 \times 2.5 = 750 \text{ g}$$
$$\text{butter: } 150 \times 2.5 = 375 \text{ g}$$
$$\text{sugar: } 80 \times 2.5 = 200 \text{ g}$$

This means every ingredient grows by the same factor, so the recipe keeps the same taste.

---

**Example 2 — Geography: map scale.** A road appears as 7.4 cm on a 1:50,000 map. What is the actual length of the road?

Map scale $1 : 50,000$ means 1 unit on the map = 50,000 units on the ground.

So 7.4 cm on the map means:

$$\text{actual length} = 7.4 \text{ cm} \times 50,000 = 370,000 \text{ cm} = 3.7 \text{ km}$$

This means every map centimetre stands for a much larger real distance.

---

**Example 3 — Splitting costs.** Three friends split a monthly phone plan costing $36 in the ratio 2:3:1 (based on how much data each uses). How much does each pay?

First count the total parts:

$$2 + 3 + 1 = 6$$

Now find one part:

$$\$36 \div 6 = \$6$$

Friend A (2 parts): $2 \times \$6 = \$12$ Friend B (3 parts): $3 \times \$6 = \$18$ Friend C (1 part): $1 \times \$6 = \$6$

Check: $$12 + $$18 + $$6 = $36. Yes.

This means the cost is shared by use, not equally.

---

**Example 4 — Finance: dividing profit.** Two partners invest in a business in the ratio 3:5. The year's profit is $24,000. How much does each receive?

First count the total parts:

$$3 + 5 = 8$$

Then find one part:

$$\$24,000 \div 8 = \$3,000$$

Partner A (3 parts): $3 ×$$3,000 = $9,000 Partner B (5 parts): $5 \times \$3,000 = \$15,000$

Check: $$9,000 + $$15,000 = $24,000. Yes.

This means the profit is divided in the same ratio as the investment.

## 5.4. Where this goes

This chapter gives us a way to compare quantities by scale instead of by difference. That idea comes back immediately in the next chapter.

**Percent and rates** (Chapter 3) is a special case of proportion, where the comparison is made out of 100.

Later, proportion reappears inside trigonometry (the sine ratio is a ratio of lengths) and in logarithms (where proportional growth becomes additive). In engineering mathematics, dimensional analysis — checking that equations are consistent in their units — is a systematic application of ratio.

The notation $y \propto x$ is also the simplest case of a functional relationship. Volume 3's treatment of functions generalises this to any rule that maps inputs to outputs.

**Where this shows up**

- Scaling a recipe up or down is proportion — every ingredient changes by the same factor.
- Reading a map involves ratio: every measurement on paper is a proportional reduction of the real distance.
- Splitting a bill by how much each person ordered is dividing a quantity in a ratio.
- A gear ratio on a bike tells you how many times the back wheel turns for each turn of the pedals.

The arithmetic is identical in every case.

## 5.5. Exercises

1. A recipe for 4 people needs 300 g flour, 150 g butter, and 80 g sugar. Scale the recipe to serve 7 people. *(Exact grams to one decimal place.)*
2. A map has scale 1:25,000. Two towns are 14.6 cm apart on the map. What is the actual distance in kilometres?
3. A car engine produces 250 Nm of torque. It drives through a gearbox with a 3.8:1 ratio, then a differential with a 3.2:1 ratio. What is the torque at the rear wheels?
4. Silver and copper are mixed in the ratio 7:3 by mass to make an alloy. You need 2.5 kg of alloy. How many grams of each metal do you need?
5. A 500 mL bottle of concentrated cleaning fluid requires diluting 1:15 before use. How many litres of diluted product can you make from one bottle?
6. Three investors share profits in the ratio 2:3:7. Total profit this year is $48,000. How much does each investor receive?
7. A gear system has a driver gear with 24 teeth and a driven gear with 60 teeth. What is the gear ratio? If the driver rotates at 900 rpm, what is the speed of the driven gear?

# 6. Percent and rates

Expressing change as a fraction of something

Your phone battery drains 3% per hour on standby. A savings account pays 4.5% annual interest. A streaming service puts its prices up by 8%. Your favourite team's win rate this season is 60%.

All four are the same operation: a fraction expressed out of a hundred, applied to a quantity. The difference is what the quantity is, and whether the percent is applied once or repeatedly.

## 6.1. What the notation is saying

Start with a familiar example. If 25 students out of 100 passed a test, we say 25%. That means 25 out of every 100.

**Percent** means per hundred. $p\%$ is the same as $\frac{p}{100}$.

This means percent is just a special kind of fraction.

To find $p\%$ of a quantity $Q$: multiply $Q$ by $\frac{p}{100}$.

$$\text{amount} = Q \times \frac{p}{100}$$

Here, $Q$ is the starting quantity and $p$ is the percent. This means the formula finds the part of the whole named by the percent.

**Percent change** compares a change to the original:

$$\text{percent change} = \frac{\text{new} - \text{original}}{\text{original}} \times 100$$

This means we compare the change to where we started, not just the size of the change by itself.

**A rate** is a ratio with different units: speed is km/h, data use is MB/day, interest is \$/year. The structure is the same as percent — one quantity compared to another.

This means a rate tells us "how much for each unit of time, distance, mass, or something else."

**Compound growth** applies a rate repeatedly. After $n$ periods at rate $r$ (expressed as a decimal), a quantity $Q_0$ becomes:

$$Q_n = Q_0 \times (1 + r)^n$$

Here, $Q_0$ is the starting amount, $r$ is the rate written as a decimal, and $n$ is the number of periods.

This means we multiply by the same growth factor again and again.

**Compound decay** uses $1 - r$ instead:

$$Q_n = Q_0 \times (1 - r)^n$$

The exponent $n$ is why compound growth and decay are not linear — they accelerate in a way that simple percent change does not.

This means the amount changes on top of the new amount each time, not just the original amount.

## 6.2. The method

**Simple percent of a quantity**

$p\%$ of $Q = Q \times \frac{p}{100}$

Convert percent to decimal first: $7\% = 0.07$; $120\% = 1.20$.

This means "of" in percent problems usually tells you to multiply.

**Percent increase / decrease**

To increase $Q$ by $p\%$: multiply by $(1 + \frac{p}{100})$ — one step does the same job as finding the percent and then adding it.

To decrease $Q$ by $p\%$: multiply by $(1 - \frac{p}{100})$.

Here, $(1 + \frac{p}{100})$ and $(1 - \frac{p}{100})$ are called multipliers. This means one multiplier can do the whole change in a single step.

**Finding the original after a percent change**

If after a $p\%$ increase the value is $V$, work backwards by dividing by the multiplier rather than multiplying:

$$Q_0 = \frac{V}{1 + \frac{p}{100}}$$

This means reversing a percent increase uses division, not subtraction.

**Compound growth**

$Q_n = Q_0 \times (1 + r)^n$ where $r = \frac{p}{100}$.

This means we are applying the same percent again and again over time.

> **Why this works**
> Each compounding period, the new starting point is the previous result. Multiplying by $(1 + r)$ once applies the rate for one period. Applying it again gives $(1 + r)^2$ — the rate is now applied to the already-grown quantity. The exponent is just repeated multiplication. This is why compound growth eventually overtakes any flat trend: the base grows, so the next period's growth is larger than the last.

---

**Interactive: Percent change visualiser.** Adjust the original value and the percent change. The bar shows the result and the single-step multiplier used.

```
//| echo: false
d3 = require("d3@7")
```

```
//| echo: false
viewof origVal = Inputs.number(
  { label: "Original value", min: 1, max: 10000, step: 1, value: 100 }
)
```

```
//| echo: false
viewof pctChange = Inputs.range(
  [-50, 100], { label: "Percent change (%)", step: 1, value: 20 }
)
```

```
//| echo: false
{
  const W = 560;
  const H = 160;
  const BAR_Y = 60;
  const BAR_H = 36;
  const LABEL_Y = BAR_Y + BAR_H + 20;
  const MULT_Y = LABEL_Y + 22;
  const PAD_L = 12;
  const PAD_R = 12;

  // Derived values — guard against NaN/zero
  const orig = Number.isFinite(origVal) && origVal > 0 ? origVal : 100;
  const pct  = Number.isFinite(pctChange) ? pctChange : 0;
  const multiplier = 1 + pct / 100;
  const newVal = orig * multiplier;

  // Scale: always show at least max(orig, newVal); min width 1
  const maxVal = Math.max(orig, newVal, 1);
  const usableW = W - PAD_L - PAD_R;
```

```
const scale = usableW / maxVal;

const origW  = Math.max(orig  * scale, 1);
const newW   = Math.max(newVal * scale, 1);

// Colours: increase = teal, decrease = amber, neutral = slate
const isIncrease = pct > 0;
const isDecrease = pct < 0;
const origColour = "#4b5563";  // slate-600
const newColour  = isIncrease ? "#0d9488"  // teal-600
                 : isDecrease ? "#d97706"  // amber-600
                 : "#4b5563";

const sign = pct >= 0 ? "+" : "";
const multSign = multiplier >= 1 ? "×" : "×";

const multStr = multiplier.toFixed(2);
const newValStr = Number.isInteger(newVal)
  ? newVal.toString()
  : newVal.toFixed(2);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", "max-width:" + W + "px; font-family: inherit;");

// Original bar
svg.append("rect")
  .attr("x", PAD_L)
  .attr("y", BAR_Y)
  .attr("width", origW)
  .attr("height", BAR_H)
  .attr("fill", origColour)
  .attr("rx", 3);

// Original label (above bar, left-aligned)
svg.append("text")
  .attr("x", PAD_L)
  .attr("y", BAR_Y - 6)
  .attr("fill", origColour)
  .attr("font-size", "12px")
  .attr("font-weight", "600")
  .text(`Original: ${orig}`);

// New value bar (drawn below original as a second row)
const BAR2_Y = BAR_Y + BAR_H + 10;
```

```
  svg.append("rect")
    .attr("x", PAD_L)
    .attr("y", BAR2_Y)
    .attr("width", newW)
    .attr("height", BAR_H)
    .attr("fill", newColour)
    .attr("rx", 3);

  // New value label (above second bar, right of bar if space allows)
  const newLabelX = PAD_L + newW + 6;
  const newLabelAnchor = newLabelX + 120 < W ? "start" : "end";
  const newLabelXAdj   = newLabelAnchor === "end" ? PAD_L + newW - 4 : newLabelX;

  svg.append("text")
    .attr("x", newLabelXAdj)
    .attr("y", BAR2_Y + BAR_H / 2 + 5)
    .attr("fill", newColour)
    .attr("font-size", "13px")
    .attr("font-weight", "700")
    .attr("text-anchor", newLabelAnchor)
    .text(`New value: ${newValStr}`);

  // Multiplier label at bottom
  const changeLabel = isIncrease
    ? `${sign}${pct}% means × ${multStr}  (multiply by ${multStr} in one step)`
    : isDecrease
    ? `${sign}${pct}% means × ${multStr}  (multiply by ${multStr} in one step)`
    : `0% means × 1.00  (no change)`;

  svg.append("text")
    .attr("x", PAD_L)
    .attr("y", BAR2_Y + BAR_H + 20)
    .attr("fill", "#374151")
    .attr("font-size", "12px")
    .text(changeLabel);

  return svg.node();
}
```

---

## 6.3. Worked examples

**Example 1 — Savings: compound interest.** You put $2,000 into a savings account at 3.5% annual interest, compounded every year. What is the balance after 5 years?

Set up the compound formula with $Q_0 = 2000$, $r = 0.035$, $n = 5$:

$$Q_5 = 2000 \times (1.035)^5$$

Work out $(1.035)^5$ — that is 1.035 multiplied by itself five times. Using a calculator: $(1.035)^5 = 1.1877$ (to 4 d.p.).

$$Q_5 = 2000 \times 1.1877 = 2375.40$$

So after 5 years, the value is **$2,375.40**.

Compare to simple interest: $5 \times 3.5\% \times 2000 = 350$, giving **$2,350**. The compounded version is **$25.40** more — the gap grows larger the longer you leave it.

This means compound interest pays interest on earlier interest as well.

---

**Example 2 — Phone plan.** A mobile plan costs $25 per month. The provider raises prices by 12%. What is the new monthly cost? How much extra do you pay over a year?

Use the increase multiplier:

$$25 \times (1 + 0.12) = 25 \times 1.12 = \$28$$

Extra per month:

$$28 - 25 = \$3$$

Extra over a year:

$$3 \times 12 = \$36$$

This means a small monthly increase can become a bigger yearly cost.

---

**Example 3 — Geography: population.** A city has a population of 850,000. The annual growth rate is 1.4%. What is the projected population in 10 years?

$$P_{10} = 850{,}000 \times (1.014)^{10}$$

$(1.014)^{10} = 1.1495$ (to 4 d.p.)

$$P_{10} = 850{,}000 \times 1.1495 \approx 977{,}000$$

Here, $P_{10}$ means the population after 10 years. This means the city grows by the same percentage each year, but the number added each year gets larger.

---

**Example 4 — Science: radioactive decay.** A sample contains 5,000 atoms of a radioactive isotope that decays at 12% per hour. How many atoms remain after 4 hours?

This is compound decay — each hour, 12% is lost, so 88% remains.

$$N_4 = 5000 \times (1 - 0.12)^4 = 5000 \times (0.88)^4$$

$(0.88)^4 = 0.5997$

$$N_4 = 5000 \times 0.5997 \approx 2,999 \text{ atoms}$$

In 4 hours, roughly 40% of the atoms have decayed.

This means decay is the same structure as growth, but the multiplier is less than 1.

---

**Interactive: Compound growth explorer.** Adjust the annual rate and number of years. The bars show the compounded value year by year. The dashed line shows what simple interest would give over the same period.

```
//| echo: false
viewof annualRate = Inputs.range(
  [0, 20], { label: "Annual rate (%)", step: 0.5, value: 5 }
)
```

```
//| echo: false
viewof numYears = Inputs.range(
  [1, 20], { label: "Number of years", step: 1, value: 10 }
)
```

```
//| echo: false
{
  const W = 560;
  const H = 300;
  const MARGIN = { top: 30, right: 20, bottom: 50, left: 62 };
  const innerW = W - MARGIN.left - MARGIN.right;
  const innerH = H - MARGIN.top - MARGIN.bottom;

  const Q0 = 1000;  // base value — shown as index, labelled clearly
  const r  = (Number.isFinite(annualRate) ? annualRate : 5) / 100;
```

```
const N  = Number.isFinite(numYears) && numYears >= 1 ? Math.round(numYears) : 10;

// Build data arrays (year 0 to N)
const years = d3.range(0, N + 1);
const compound = years.map(n => Q0 * Math.pow(1 + r, n));
const simple   = years.map(n => Q0 * (1 + r * n));

const finalCompound = compound[N];
const finalSimple   = simple[N];
const extraStr = (finalCompound - finalSimple).toFixed(2);

const yMax = Math.max(...compound, ...simple) * 1.08;

const xScale = d3.scaleBand()
  .domain(years)
  .range([0, innerW])
  .padding(0.18);

const yScale = d3.scaleLinear()
  .domain([0, yMax])
  .range([innerH, 0]);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", `max-width:${W}px; font-family: inherit;`);

const g = svg.append("g")
  .attr("transform", `translate(${MARGIN.left},${MARGIN.top})`);

// Gridlines
g.append("g")
  .attr("class", "grid")
  .call(
    d3.axisLeft(yScale)
      .ticks(5)
      .tickSize(-innerW)
      .tickFormat("")
  )
  .call(gg => gg.select(".domain").remove())
  .call(gg => gg.selectAll("line").attr("stroke", "#e5e7eb").attr("stroke-dasharray", "3,3"));

// Compound bars
g.selectAll(".bar-compound")
  .data(years)
  .enter()
  .append("rect")
```

```
    .attr("class", "bar-compound")
    .attr("x", d => xScale(d))
    .attr("y", d => yScale(compound[d]))
    .attr("width", xScale.bandwidth())
    .attr("height", d => innerH - yScale(compound[d]))
    .attr("fill", "#0d9488")
    .attr("rx", 2);

// Simple interest line
const lineGen = d3.line()
  .x(d => xScale(d) + xScale.bandwidth() / 2)
  .y(d => yScale(simple[d]));

g.append("path")
  .datum(years)
  .attr("fill", "none")
  .attr("stroke", "#d97706")
  .attr("stroke-width", 2)
  .attr("stroke-dasharray", "5,4")
  .attr("d", lineGen);

// Simple interest dots
g.selectAll(".dot-simple")
  .data(years)
  .enter()
  .append("circle")
    .attr("cx", d => xScale(d) + xScale.bandwidth() / 2)
    .attr("cy", d => yScale(simple[d]))
    .attr("r", 3)
    .attr("fill", "#d97706");

// X axis
g.append("g")
  .attr("transform", `translate(0,${innerH})`)
  .call(
    d3.axisBottom(xScale)
      .tickValues(years.filter((_, i) => N <= 10 || i % 2 === 0))
      .tickFormat(d => `Y${d}`)
  )
  .call(gg => gg.select(".domain").attr("stroke", "#9ca3af"))
  .call(gg => gg.selectAll("text").attr("font-size", "11px"));

// X axis label
g.append("text")
  .attr("x", innerW / 2)
  .attr("y", innerH + 40)
  .attr("text-anchor", "middle")
```

```
    .attr("fill", "#6b7280")
    .attr("font-size", "12px")
    .text("Year");

  // Y axis
  g.append("g")
    .call(d3.axisLeft(yScale).ticks(5).tickFormat(d => `$${d}`))
    .call(gg => gg.select(".domain").attr("stroke", "#9ca3af"))
    .call(gg => gg.selectAll("text").attr("font-size", "11px"));

  // Final value label on last bar
  g.append("text")
    .attr("x", xScale(N) + xScale.bandwidth() / 2)
    .attr("y", yScale(finalCompound) - 6)
    .attr("text-anchor", "middle")
    .attr("fill", "#065f46")
    .attr("font-size", "11px")
    .attr("font-weight", "700")
    .text(`$${finalCompound.toFixed(0)}`);

  // Legend
  const legendY = -18;
  // Compound bar swatch
  g.append("rect")
    .attr("x", 0).attr("y", legendY - 9).attr("width", 12).attr("height", 12)
    .attr("fill", "#0d9488").attr("rx", 2);
  g.append("text")
    .attr("x", 16).attr("y", legendY)
    .attr("fill", "#374151").attr("font-size", "11px")
    .text("Compound growth");

  // Simple line swatch
  g.append("line")
    .attr("x1", 140).attr("y1", legendY - 3)
    .attr("x2", 152).attr("y2", legendY - 3)
    .attr("stroke", "#d97706").attr("stroke-width", 2).attr("stroke-dasharray", "4,3");
  g.append("text")
    .attr("x", 156).attr("y", legendY)
    .attr("fill", "#374151").attr("font-size", "11px")
    .text("Simple interest");

  // Extra label
  g.append("text")
    .attr("x", innerW)
    .attr("y", innerH + 40)
    .attr("text-anchor", "end")
    .attr("fill", "#0d9488")
```

```
    .attr("font-size", "12px")
    .attr("font-weight", "600")
    .text(`Compounding adds $${extraStr} extra over ${N} year${N === 1 ? "" : "s"}`);

  return svg.node();
}
```

---

## 6.4. Where this goes

This chapter builds directly on ratio and proportion. First we used comparisons. Now we use a comparison out of 100.

When percent is applied once, it is still a proportion problem. When it is applied again and again, it becomes exponential growth or decay. **Exponents and logarithms** (Volume 4, Chapter 1) studies that repeated growth more carefully.

**Statistics and probability** (Chapter 6) uses rates constantly: probability is a rate between 0 and 1, and relative frequency is a percent.

In finance, virtually every calculation — present value, future value, yield, annuity — is a variant of the compound formula. In engineering, efficiency, signal attenuation, and structural safety factors are all percent-based ratios. The formula here is simple. The consequences are not.

**Where this shows up**

- A savings account paying 4% a year is using the compound formula — the interest each year is calculated on the new, larger balance.
- A streaming service raising prices by 8% is applying a percent increase to a base price.
- A sports commentator quoting a 65% win rate is expressing a fraction as a percent.
- A phone showing "battery 23%" is telling you what fraction of capacity remains.

The arithmetic is identical. The stakes vary.

## 6.5. Exercises

1. A laptop costs $1,149. It is on sale at 15% off. What is the sale price?
2. A city had 320,000 residents in 2015 and 387,000 in 2025. What was the total percent change over that decade?
3. An investment grows from $4,500 to $5,200 over 3 years. What was the approximate annual growth rate? *(Estimate; you do not yet need logarithms — try 3–4 percent and refine.)*
4. A water tank contains 12,000 litres. It leaks at a rate of 5% of its current volume per day. How much water remains after 1 week?

5. A car's drivetrain transfers engine power to the wheels with 87% efficiency. If the engine produces 110 kW, what power reaches the wheels? What percentage is lost?
6. A country's annual inflation rate is 2.8%. A loaf of bread costs $1.40 today. What will it cost in 8 years at this rate?
7. A business owner offers a 10% early-payment discount. A customer receives an invoice for $3,400. They pay early. How much do they pay? What percent of the original price was the discount in pounds?

# 7. Integers and signed arithmetic

Moving in both directions

Your bank balance is \$40 and you spend \$60. You're now at −\$20.

It's −3°C at night. By morning it's dropped another 8 degrees. Where are you on the thermometer?

You're on the ground floor of a car park. The exit is two floors up; the lower deck is three floors down. Which direction is floor −3?

All three questions use the same operation: addition on the integer number line. Negative numbers aren't strange — they're just the left side of zero.

## 7.1. What the notation is saying

Start with the number line from Chapter 1. Positive numbers point one way from zero, and negative numbers point the other way.

An **integer** is any whole number: ..., −3, −2, −1, 0, 1, 2, 3, ...

This means integers are whole-number positions on the number line.

The negative sign in front of a number means "opposite direction": −5 is 5 units to the left of zero.

This means the sign tells you direction, while the number tells you size.

Adding a negative moves you left on the number line:

$$7 + (-3) = 7 - 3 = 4$$

This means "plus a negative" has the same effect as subtraction.

Subtracting a negative moves you right — you reverse direction twice:

$$5 - (-2) = 5 + 2 = 7$$

That second one trips people up. Think of it this way: if someone cancels a debt you owe, you end up better off. Taking away a negative is the same as adding a positive.

**The double negative rule:** $-(-a) = a$. Two negatives cancel out and you get the original positive back.

Here, $a$ means any number. This means the opposite of an opposite brings you back to the starting direction.

**Sign rules for multiplication and division:**

| Operation | Sign result |
|---|---|
| $(+) \times (+)$ | $+$ |
| $(-) \times (-)$ | $+$ |
| $(+) \times (-)$ | $-$ |
| $(-) \times (+)$ | $-$ |

Same signs give positive. Different signs give negative. The same rules apply to division.

This means multiplication and division care about whether the directions match or oppose each other.

---

**Interactive: Signed number line.** Set a starting value and a change. The arrow shows the movement; the equation updates live.

```
//| echo: false
d3 = require("d3@7")
```

```
//| echo: false
viewof startVal = Inputs.range(
  [-10, 10], { label: "Starting value", step: 1, value: 3 }
)
```

```
//| echo: false
viewof changeVal = Inputs.range(
  [-10, 10], { label: "Change (add this)", step: 1, value: -5 }
)
```

```
//| echo: false
{
  const W = 560;
  const H = 130;
  const LINE_Y = 58;
  const TICK_H = 10;
  const PAD_L = 28;
  const PAD_R = 28;
  const MIN_VAL = -15;
  const MAX_VAL = 15;

  const start  = Number.isFinite(startVal)  ? Math.round(startVal)  : 3;
  const change = Number.isFinite(changeVal) ? Math.round(changeVal) : -5;
```

```
const result = start + change;

// Clamp result display — result can go outside visible range; still show equation
const visibleResult = Math.max(MIN_VAL, Math.min(MAX_VAL, result));

const usableW = W - PAD_L - PAD_R;
const xScale = d3.scaleLinear()
  .domain([MIN_VAL, MAX_VAL])
  .range([PAD_L, PAD_L + usableW]);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", `max-width:${W}px; font-family: inherit;`);

// Number line
svg.append("line")
  .attr("x1", PAD_L - 8).attr("y1", LINE_Y)
  .attr("x2", PAD_L + usableW + 8).attr("y2", LINE_Y)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);

// Arrowheads on line ends
const arrowPath = (x, y, dir) =>
  `M${x},${y} L${x + dir * 7},${y - 5} L${x + dir * 7},${y + 5} Z`;
svg.append("path").attr("d", arrowPath(PAD_L - 8, LINE_Y, -1)).attr("fill", "#9ca3af");
svg.append("path").attr("d", arrowPath(PAD_L + usableW + 8, LINE_Y, 1)).attr("fill", "#9ca3af")

// Tick marks and labels
for (let v = MIN_VAL; v <= MAX_VAL; v++) {
  const x = xScale(v);
  const isMajor = v % 5 === 0;
  svg.append("line")
    .attr("x1", x).attr("y1", LINE_Y - (isMajor ? TICK_H : TICK_H * 0.6))
    .attr("x2", x).attr("y2", LINE_Y + (isMajor ? TICK_H : TICK_H * 0.6))
    .attr("stroke", isMajor ? "#6b7280" : "#d1d5db")
    .attr("stroke-width", isMajor ? 1.5 : 1);
  if (isMajor) {
    svg.append("text")
      .attr("x", x).attr("y", LINE_Y + TICK_H + 13)
      .attr("text-anchor", "middle")
      .attr("fill", "#6b7280").attr("font-size", "11px")
      .text(v);
  }
}

// Arrow showing movement (from start to result, clamped to visible range)
const arrowStartX  = xScale(Math.max(MIN_VAL, Math.min(MAX_VAL, start)));
```

```
const arrowEndX    = xScale(visibleResult);
const ARROW_Y      = LINE_Y - 22;
const isOutOfRange = result < MIN_VAL || result > MAX_VAL;

if (change !== 0) {
  const colour = change > 0 ? "#0d9488" : "#d97706";
  svg.append("line")
    .attr("x1", arrowStartX).attr("y1", ARROW_Y)
    .attr("x2", arrowEndX).attr("y2", ARROW_Y)
    .attr("stroke", colour).attr("stroke-width", 2.5);

  // Arrowhead at end
  const dir = change > 0 ? 1 : -1;
  svg.append("path")
    .attr("d", `M${arrowEndX},${ARROW_Y} L${arrowEndX - dir * 8},${ARROW_Y - 5} L${arrowEndX - di
    .attr("fill", colour);
}

// Start position dot
svg.append("circle")
  .attr("cx", arrowStartX).attr("cy", LINE_Y)
  .attr("r", 7)
  .attr("fill", "#4b5563")
  .attr("stroke", "#fff").attr("stroke-width", 1.5);

svg.append("text")
  .attr("x", arrowStartX).attr("cy", LINE_Y)
  .attr("y", LINE_Y - 12)
  .attr("text-anchor", "middle")
  .attr("fill", "#374151").attr("font-size", "11px").attr("font-weight", "600")
  .text(start);

// Result position dot
const resultColour = result > 0 ? "#0d9488" : result < 0 ? "#d97706" : "#4b5563";
svg.append("circle")
  .attr("cx", xScale(visibleResult)).attr("cy", LINE_Y)
  .attr("r", 7)
  .attr("fill", resultColour)
  .attr("stroke", "#fff").attr("stroke-width", 1.5);

svg.append("text")
  .attr("x", xScale(visibleResult))
  .attr("y", LINE_Y + TICK_H + 28)
  .attr("text-anchor", "middle")
  .attr("fill", resultColour).attr("font-size", "11px").attr("font-weight", "700")
  .text(isOutOfRange ? `${result} (off scale)` : result);
```

```
  // Equation label
  const changeSign = change >= 0 ? `+ ${change}` : `+ (${change})`;
  const eqLabel    = `${start}  ${changeSign}  =  ${result}`;
  svg.append("text")
    .attr("x", W / 2).attr("y", H - 6)
    .attr("text-anchor", "middle")
    .attr("fill", "#1f2937").attr("font-size", "13px").attr("font-weight", "700")
    .text(eqLabel);

  return svg.node();
}
```

---

## 7.2. The method

### Addition and subtraction

Rewrite every subtraction as addition of the opposite: $a - b = a + (-b)$.

Then use the number line: positive means move right, negative means move left. Your result is where you land.

Here, $a$ and $b$ are any numbers. This means one idea can handle both addition and subtraction.

### Multiplication

1. Multiply the absolute values (ignore signs for now).
2. Count the number of negative factors.
    - Even number of negatives $\longrightarrow$ positive result.
    - Odd number of negatives $\longrightarrow$ negative result.

This means the size and the sign can be handled separately.

### Division

Same rule: divide the absolute values, then apply the sign rule.

This means division follows the same sign pattern as multiplication.

### Order of operations with signed numbers

Brackets first. Then signs attached to numbers. Then multiply/divide left to right. Then add/subtract left to right.

This means the sign belongs to the number it is attached to.

> **Why this works**
> Think of multiplication as repeated addition. $3 \times (-4)$ means "add $-4$ three times": $(-4) + (-4) + (-4) = -12$. That gives $(+) \times (-) = (-)$.
> For $(-)\times(-)$: $-3\times(-4)$ means "add $-4$ negative three times." Adding a negative thing a negative number of times is removing it — which is positive. The algebraic proof uses the distributive law: $0 = (-3)(4 + (-4)) = (-3)(4) + (-3)(-4) = -12 + (-3)(-4)$, so $(-3)(-4) = 12$.

---

**Interactive: Sign rules for multiplication.** Click any cell in the table to see the calculation and the rule in action. All four combinations are visible at once.

```
//| echo: false
viewof selectedCell = {
  // Default to top-left cell on load
  const init = { a: 1, b: 1 };
  return Inputs.input(init);
}
```

```
//| echo: false
{
  // Fixed example magnitudes for each sign combination
  // Rows: sign of first number (+/-)
  // Cols: sign of second number (+/-)
  const COMBOS = [
    { aSign: +1, bSign: +1, aAbs: 3, bAbs: 4, row: 0, col: 0 },
    { aSign: +1, bSign: -1, aAbs: 3, bAbs: 4, row: 0, col: 1 },
    { aSign: -1, bSign: +1, aAbs: 3, bAbs: 4, row: 1, col: 0 },
    { aSign: -1, bSign: -1, aAbs: 3, bAbs: 4, row: 1, col: 1 },
  ];

  const W = 560;
  const H = 280;
  const CELL_W = 200;
  const CELL_H = 88;
  const GRID_X = 100;
  const GRID_Y = 50;
  const COL_HEADER_Y = 28;
  const ROW_HEADER_X = 52;

  const posColour  = "#0d9488";  // teal-600 — positive result
  const negColour  = "#d97706";  // amber-600 — negative result
  const selStroke  = "#1f2937";
  const dimFill    = "#f9fafb";

  const svg = d3.create("svg")
```

```
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", `max-width:${W}px; font-family: inherit; cursor: pointer;`);

// Column headers: sign of second number
const colLabels = ["Second number: +", "Second number: −"];
colLabels.forEach((label, ci) => {
  svg.append("text")
    .attr("x", GRID_X + ci * CELL_W + CELL_W / 2)
    .attr("y", COL_HEADER_Y)
    .attr("text-anchor", "middle")
    .attr("fill", "#374151")
    .attr("font-size", "12px")
    .attr("font-weight", "600")
    .text(label);
});

// Row headers: sign of first number
const rowLabels = ["First: +", "First: −"];
rowLabels.forEach((label, ri) => {
  svg.append("text")
    .attr("x", ROW_HEADER_X)
    .attr("y", GRID_Y + ri * CELL_H + CELL_H / 2 + 4)
    .attr("text-anchor", "middle")
    .attr("fill", "#374151")
    .attr("font-size", "12px")
    .attr("font-weight", "600")
    .text(label);
});

COMBOS.forEach(c => {
  const aVal   = c.aSign * c.aAbs;
  const bVal   = c.bSign * c.bAbs;
  const result = aVal * bVal;
  const isPos  = result > 0;
  const colour = isPos ? posColour : negColour;

  const sel = selectedCell;
  const isSelected = sel && sel.a === c.aSign && sel.b === c.bSign;

  const cellX = GRID_X + c.col * CELL_W;
  const cellY = GRID_Y + c.row * CELL_H;

  // Cell background
  const g = svg.append("g")
    .attr("transform", `translate(${cellX},${cellY})`)
    .style("cursor", "pointer")
```

```
      .on("click", () => {
        selectedCell.value = { a: c.aSign, b: c.bSign };
        selectedCell.dispatchEvent(new Event("input"));
      });

    g.append("rect")
      .attr("width", CELL_W - 4)
      .attr("height", CELL_H - 4)
      .attr("x", 2).attr("y", 2)
      .attr("rx", 6)
      .attr("fill", isSelected ? colour : dimFill)
      .attr("stroke", isSelected ? selStroke : "#d1d5db")
      .attr("stroke-width", isSelected ? 2 : 1);

    // Calculation line: e.g. "+3 × −4 = −12"
    const aStr = c.aSign > 0 ? `+${c.aAbs}` : `−${c.aAbs}`;
    const bStr = c.bSign > 0 ? `+${c.bAbs}` : `−${c.bAbs}`;
    const rStr = result > 0 ? `+${Math.abs(result)}` : `−${Math.abs(result)}`;

    g.append("text")
      .attr("x", (CELL_W - 4) / 2)
      .attr("y", 28)
      .attr("text-anchor", "middle")
      .attr("fill", isSelected ? "#fff" : "#1f2937")
      .attr("font-size", "16px")
      .attr("font-weight", "700")
      .text(`${aStr} × ${bStr} = ${rStr}`);

    // Sign rule label
    const aSignWord = c.aSign > 0 ? "positive" : "negative";
    const bSignWord = c.bSign > 0 ? "positive" : "negative";
    const rSignWord = isPos ? "positive" : "negative";
    const ruleLine1 = `${aSignWord} × ${bSignWord}`;
    const ruleLine2 = `= ${rSignWord}`;

    g.append("text")
      .attr("x", (CELL_W - 4) / 2)
      .attr("y", 50)
      .attr("text-anchor", "middle")
      .attr("fill", isSelected ? "#fff" : "#6b7280")
      .attr("font-size", "11px")
      .text(ruleLine1);

    g.append("text")
      .attr("x", (CELL_W - 4) / 2)
      .attr("y", 64)
      .attr("text-anchor", "middle")
```

```
      .attr("fill", isSelected ? "#fff" : colour)
      .attr("font-size", "11px")
      .attr("font-weight", "600")
      .text(ruleLine2);

    // Shape badge: circle = positive, diamond = negative (redundant encoding)
    const BADGE_X = (CELL_W - 4) / 2;
    const BADGE_Y = 78;
    if (isPos) {
      g.append("circle")
        .attr("cx", BADGE_X).attr("cy", BADGE_Y - 4)
        .attr("r", 5)
        .attr("fill", "none")
        .attr("stroke", isSelected ? "#fff" : colour)
        .attr("stroke-width", 1.5);
    } else {
      g.append("polygon")
        .attr("points", `${BADGE_X},${BADGE_Y - 9} ${BADGE_X + 6},${BADGE_Y - 4} ${BADGE_X},${BAD
        .attr("fill", "none")
        .attr("stroke", isSelected ? "#fff" : colour)
        .attr("stroke-width", 1.5);
    }
  });

  // Summary rule line at bottom
  svg.append("text")
    .attr("x", W / 2)
    .attr("y", H - 10)
    .attr("text-anchor", "middle")
    .attr("fill", "#374151")
    .attr("font-size", "12px")
    .text("Same signs → positive result   |   Different signs → negative result   |   Click any c

  return svg.node();
}
```

---

## 7.3. Worked examples

**Example 1 — Bank balance.** You start the week with $50 in your account. You spend $30 on food, $25 on a game, and then get paid $60 from a part-time shift. What's your balance at the end of the week?

Add up all the changes — spends are negative, income is positive:

$$50 + (-30) + (-25) + 60 = 50 - 30 - 25 + 60 = 55$$

End balance: $55.

This means the final balance is the starting amount plus every signed change.

---

**Example 2 — Temperature change.** The temperature at 6 am is −5°C. By midday it has risen 14°C. By midnight it has fallen 18°C from the midday high. What is the midnight temperature?

Work through the changes in order:

$$-5 + 14 = 9^\circ\text{C} \quad \text{(midday)}$$

$$9 - 18 = -9^\circ\text{C} \quad \text{(midnight)}$$

The midnight temperature is −9°C.

This means a rise is positive movement and a fall is negative movement.

---

**Example 3 — Score in a game.** A quiz game gives +5 for a correct answer and −3 for a wrong one. A player gets 4 right and 6 wrong. What is their score?

Correct answers: $4 \times 5 = 20$. Wrong answers: $6 \times (-3) = -18$ — six negatives, so the result is negative.

Total score: $20 + (-18) = 2$.

This means a player's total is the sum of positive and negative points.

---

**Example 4 — Running total with negatives.** Evaluate $(-4) \times (-3) + 2 \times (-5)$ without a calculator.

Handle the multiplications first — left to right:

$(-4) \times (-3) = 12$ (same signs $\longrightarrow$ positive), giving 12.

$2 \times (-5) = -10$ (different signs $\longrightarrow$ negative), giving $-10$.

Now add: $12 + (-10) = 2$.

This means we apply sign rules first in multiplication, then combine the results by addition.

## 7.4. Where this goes

This chapter takes the number line idea and turns it into a full set of rules for calculation.

Signed arithmetic is the prerequisite for **linear equations** (Volume 2, Chapter 1). When you solve $3x - 7 = -19$, you need to add 7 to both sides and handle the resulting $-12$ correctly.

In physics, signed quantities appear everywhere: displacement, velocity, acceleration, force, work, charge, and potential are all signed. The sign carries physical meaning — it encodes direction or polarity. Getting the sign wrong is not just a numerical error; it is a physical error.

**Where this shows up**

- A bank account or budget uses signed arithmetic every time money moves in or out — the balance is an integer sum.
- A weather forecast compares signed temperatures to report how much warmer or colder one day was than another.
- A physicist writing Newton's second law includes sign: $F = ma$, where $F < 0$ means the force acts in the negative direction.
- A programmer writing low-level code must track whether arithmetic will overflow a signed integer's range.

The rules are short. The mistakes, when you forget them, are expensive.

## 7.5. Exercises

1. A lift starts at floor 0 (ground). It goes down 3 floors, then up 7 floors, then down 2 floors. What floor is it on?
2. A player's scores across five rounds of a quiz game: −4, +7, −2, +9, −3. What is their total score?
3. Evaluate without a calculator:
   a) $(-8) \times (-6)$
   b) $(-15) \div 3$
   c) $4 \times (-9) \times (-2)$
   d) $(-3)^3$
4. The temperature at midnight is −6°C. It rises by 11°C by noon, then drops 15°C by the following midnight. What is the temperature at the end?
5. The lowest temperature ever recorded in Canada is −63°C (Snag, Yukon). The highest is +45°C (Lytton, BC). What is the range — the difference between highest and lowest?
6. A student owes three friends \$8 each. They earn \$40 doing odd jobs. After paying back all three debts, how much money do they have?
7. A share price opens at \$14.50. Changes over five days: −\$0.80, +\$1.20, −\$2.40, +\$0.35, −\$0.90. What is the closing price on day 5?

# 8. Fractions and decimals

Arithmetic with parts of wholes

A recipe needs $\frac{3}{4}$ of a cup of flour and you want to make half the recipe. How much flour do you use?

Four friends split a restaurant bill of \$37.60 equally. What does each person pay?

You run 2.5 km on Monday and 1.75 km on Wednesday. How far have you run in total?

These are all fraction and decimal problems. The numbers sit between the integers — and the arithmetic for handling them is a small set of rules you can learn once and use everywhere.

## 8.1. What the notation is saying

Start with a whole divided into equal pieces. If a pizza is cut into 4 equal slices and you take 3 of them, you have $\frac{3}{4}$ of the pizza.

The fraction $\frac{a}{b}$ means "$a$ divided by $b$." The top number is the **numerator** (how many parts you have); the bottom number is the **denominator** (how many equal parts make the whole).

This means a fraction names both the number of equal parts and the size of each part.

Decimals are fractions whose denominators are powers of ten:

$$0.3 = \frac{3}{10}, \quad 0.47 = \frac{47}{100}, \quad 0.006 = \frac{6}{1000}$$

They are interchangeable. $\frac{3}{4} = 0.75$. $\frac{1}{3} = 0.\overline{3}$ (the bar means the digit repeats forever).

This means fractions and decimals are two ways to write the same numbers.

**Equivalent fractions:** $\frac{2}{4} = \frac{1}{2}$ because you can divide numerator and denominator by 2. A fraction is in **lowest terms** when numerator and denominator share no common factor other than 1.

This means the writing can change while the value stays the same.

---

*Use the controls below to explore what a fraction looks like — as a divided rectangle and as a position on the number line.*

```
//| echo: false
viewof frac1_num = Inputs.range([1, 12], {
  value: 3,
  step: 1,
  label: "Numerator (top)"
})
```

```
//| echo: false
viewof frac1_den_raw = Inputs.range([1, 12], {
  value: 4,
  step: 1,
  label: "Denominator (bottom)"
})
```

```
//| echo: false
{
  // Enforce denominator >= numerator by silently clamping
  const num = frac1_num;
  const den = Math.max(frac1_den_raw, num);

  // --- GCD helper for display in lowest terms ---
  function gcd(a, b) { return b === 0 ? a : gcd(b, a % b); }
  const g = gcd(num, den);
  const rn = num / g;
  const rd = den / g;
  const decimal = num / den;
  const lowestLabel = rn === rd
    ? `${rn} (= 1)`
    : rd === 1
      ? `${rn}`
      : `${rn}/${rd}`;

  const W = 560, H_rect = 80, H_line = 70;
  const PAD = 40;

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H_rect + H_line + 20}`)
    .attr("width", "100%")
    .attr("style", "max-width:560px; display:block; margin:0 auto; font-family:sans-serif;");

  // ── Area model ──────────────────────────────────────────
  const rectW = W - 2 * PAD;
  const partW = rectW / den;

  const rectG = svg.append("g").attr("transform", `translate(${PAD}, 10)`);

  for (let i = 0; i < den; i++) {
```

```
  rectG.append("rect")
    .attr("x", i * partW)
    .attr("y", 0)
    .attr("width", partW - 1)
    .attr("height", H_rect)
    .attr("fill", i < num ? "#2563eb" : "#e5e7eb")
    .attr("stroke", "#94a3b8")
    .attr("stroke-width", 0.5)
    .attr("rx", 2);
}

// Fraction label centred above the shaded region
rectG.append("text")
  .attr("x", (num * partW) / 2)
  .attr("y", -4)
  .attr("text-anchor", "middle")
  .attr("font-size", 13)
  .attr("fill", "#1e3a5f")
  .text(`${num}/${den}${g > 1 ? ` = ${lowestLabel}` : ""} = ${decimal.toFixed(6).replace(/\.?0+

// Shaded count label inside rectangle
if (num > 0) {
  rectG.append("text")
    .attr("x", (num * partW) / 2)
    .attr("y", H_rect / 2 + 5)
    .attr("text-anchor", "middle")
    .attr("font-size", Math.min(14, partW * 0.7))
    .attr("fill", "#ffffff")
    .attr("font-weight", "bold")
    .text(num === den ? "whole" : `${num} part${num > 1 ? "s" : ""}`);
}

// Unshaded count label
if (num < den) {
  rectG.append("text")
    .attr("x", num * partW + ((den - num) * partW) / 2)
    .attr("y", H_rect / 2 + 5)
    .attr("text-anchor", "middle")
    .attr("font-size", Math.min(13, partW * 0.65))
    .attr("fill", "#64748b")
    .text(`${den - num} part${den - num > 1 ? "s" : ""}`);
}

// Area model axis label
rectG.append("text")
  .attr("x", rectW / 2)
  .attr("y", H_rect + 14)
```

```
  .attr("text-anchor", "middle")
  .attr("font-size", 11)
  .attr("fill", "#64748b")
  .text(`Rectangle divided into ${den} equal part${den > 1 ? "s" : ""} — ${num} shaded`);

// ── Number line ──────────────────────────────────────────
const lineY = H_rect + 20;
const lineG = svg.append("g").attr("transform", `translate(${PAD}, ${lineY + 10})`);

// Track line
lineG.append("line")
  .attr("x1", 0).attr("y1", 20)
  .attr("x2", rectW).attr("y2", 20)
  .attr("stroke", "#94a3b8").attr("stroke-width", 2);

// Tick at 0
lineG.append("line")
  .attr("x1", 0).attr("y1", 13).attr("x2", 0).attr("y2", 27)
  .attr("stroke", "#374151").attr("stroke-width", 2);
lineG.append("text")
  .attr("x", 0).attr("y", 40)
  .attr("text-anchor", "middle").attr("font-size", 12).attr("fill", "#374151")
  .text("0");

// Tick at 1
lineG.append("line")
  .attr("x1", rectW).attr("y1", 13).attr("x2", rectW).attr("y2", 27)
  .attr("stroke", "#374151").attr("stroke-width", 2);
lineG.append("text")
  .attr("x", rectW).attr("y", 40)
  .attr("text-anchor", "middle").attr("font-size", 12).attr("fill", "#374151")
  .text("1");

// Fraction position marker
const markerX = decimal * rectW;
lineG.append("circle")
  .attr("cx", markerX).attr("cy", 20)
  .attr("r", 7)
  .attr("fill", "#2563eb")
  .attr("stroke", "#1e3a5f")
  .attr("stroke-width", 1.5);

// Value label above marker
lineG.append("text")
  .attr("x", markerX)
  .attr("y", 10)
  .attr("text-anchor", "middle")
```

```
    .attr("font-size", 12)
    .attr("fill", "#1e3a5f")
    .attr("font-weight", "bold")
    .text(decimal.toFixed(4).replace(/\.?0+$/, "") || "0");

  // Axis label
  lineG.append("text")
    .attr("x", rectW / 2).attr("y", 55)
    .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#64748b")
    .text("Number line from 0 to 1 — dot shows the same value");

  return svg.node();
}
```

> ℹ If the denominator input goes below the numerator, the interactive clamps it to keep the fraction valid (≤ 1). Try setting numerator and denominator to the same value to see what "1 whole" looks like in both views.

---

## 8.2. The method

### Multiplying fractions

Multiply numerators together; multiply denominators together — that's it:

$$\frac{a}{b} \times \frac{c}{d} = \frac{ac}{bd}$$

Simplify before or after. Before is usually easier.

Here, $a$ and $c$ are the numerators, and $b$ and $d$ are the denominators. This means multiplication works directly across the top and across the bottom.

### Dividing fractions

Flip the second fraction and multiply — this is called multiplying by the reciprocal:

$$\frac{a}{b} \div \frac{c}{d} = \frac{a}{b} \times \frac{d}{c} = \frac{ad}{bc}$$

"Flip and multiply" works because $\frac{c}{d} \times \frac{d}{c} = 1$: the two fractions cancel each other out, leaving you with a clean multiplication.

This means division by a fraction asks how many of that fraction fit into the first number.

### Adding and subtracting fractions

Mixed numbers: convert first. Multiply the whole number by the denominator and add the numerator: $1\frac{1}{2} = \frac{(1\times2)+1}{2} = \frac{3}{2}$.

You need a common denominator before you can add. The simplest route: multiply both denominators to get one that works for both.

$$\frac{a}{b} + \frac{c}{d} = \frac{ad}{bd} + \frac{bc}{bd} = \frac{ad+bc}{bd}$$

Simplify the result.

This means we must rename the fractions so they count the same size of part before we combine them.

**Converting between fractions and decimals**

Fraction ⟶ decimal: divide numerator by denominator. Decimal ⟶ fraction: write the decimal over its place-value denominator, then simplify.

This means you can move between the two forms whenever one is easier to use.

> **Why this works**
> A fraction $\frac{a}{b}$ is a number. The rules for arithmetic with fractions follow from the same rules that govern whole numbers — distribution, commutativity, and the fact that multiplying by 1 changes nothing. To add $\frac{a}{b} + \frac{c}{d}$, rewrite the first fraction as $\frac{a}{b} \times \frac{d}{d} = \frac{ad}{bd}$ and the second as $\frac{c}{d} \times \frac{b}{b} = \frac{bc}{bd}$. Both multiplications are by 1, so neither fraction's value changes — only its representation. The result is $\frac{ad+bc}{bd}$. Division by a fraction is multiplication by its reciprocal because $\frac{a}{b} \times \frac{b}{a} = 1$ — they are inverses of each other.

---

*Use the calculator below to check any fraction arithmetic. Choose two fractions and an operation; the working is shown step by step.*

```
//| echo: false
viewof op_a_num = Inputs.range([1, 12], { value: 3, step: 1, label: "Fraction A — numerator" })
```

```
//| echo: false
viewof op_a_den = Inputs.range([1, 12], { value: 4, step: 1, label: "Fraction A — denominator" })
```

```
//| echo: false
viewof op_b_num = Inputs.range([1, 12], { value: 1, step: 1, label: "Fraction B — numerator" })
```

```
//| echo: false
viewof op_b_den = Inputs.range([1, 12], { value: 2, step: 1, label: "Fraction B — denominator" })
```

```
//| echo: false
viewof operation = Inputs.select(["×", "÷", "+", "−"], { label: "Operation", value: "×" })
```

```
//| echo: false
{
  function gcd(a, b) { return b === 0 ? a : gcd(b, a % b); }
  function simplify(n, d) {
    if (d === 0) return [n, d];
    const g = gcd(Math.abs(n), Math.abs(d));
    return [n / g, d / g];
  }
  function fracStr(n, d) {
    if (d === 1) return `${n}`;
    return `${n}/${d}`;
  }

  const an = op_a_num, ad = op_a_den;
  const bn = op_b_num, bd = op_b_den;

  let steps = [];
  let rn, rd;

  if (operation === "×") {
    rn = an * bn;
    rd = ad * bd;
    const [sn, sd] = simplify(rn, rd);
    steps = [
      `Multiply numerators: ${an} × ${bn} = ${rn}`,
      `Multiply denominators: ${ad} × ${bd} = ${rd}`,
      `Result: ${rn}/${rd}`,
      (sn !== rn || sd !== rd) ? `Simplified: ${fracStr(sn, sd)}` : `Already in lowest terms`,
      `As a decimal: ${(sn / sd).toFixed(6).replace(/\.?0+$/, "") || "0"}`
    ];
    [rn, rd] = [sn, sd];

  } else if (operation === "÷") {
    // Multiply by reciprocal of B
    const flipN = bd, flipD = bn;
    if (bn === 0) {
      steps = ["Cannot divide by zero — set Fraction B numerator to at least 1."];
      rn = NaN; rd = 1;
    } else {
      const rawN = an * flipN;
      const rawD = ad * flipD;
      const [sn, sd] = simplify(rawN, rawD);
      steps = [
        `Flip Fraction B: ${bn}/${bd} becomes ${flipN}/${flipD}`,
        `Multiply: ${an}/${ad} × ${flipN}/${flipD}`,
        `Numerators: ${an} × ${flipN} = ${rawN}`,
        `Denominators: ${ad} × ${flipD} = ${rawD}`,
```

```
        `Result: ${rawN}/${rawD}`,
        (sn !== rawN || sd !== rawD) ? `Simplified: ${fracStr(sn, sd)}` : `Already in lowest terms`,
        `As a decimal: ${(sn / sd).toFixed(6).replace(/\.?0+$/, "") || "0"}`
      ];
      [rn, rd] = [sn, sd];
    }

  } else {
    // Addition or subtraction — find common denominator
    const cd = ad * bd;  // common denominator (cross-multiply)
    const aN = an * bd;
    const bN = bn * ad;
    const rawN = operation === "+" ? aN + bN : aN - bN;
    const [sn, sd] = simplify(rawN, cd);
    steps = [
      `Common denominator: ${ad} × ${bd} = ${cd}`,
      `Rewrite A: ${an}/${ad} = ${aN}/${cd}`,
      `Rewrite B: ${bn}/${bd} = ${bN}/${cd}`,
      operation === "+"
        ? `Add numerators: ${aN} + ${bN} = ${rawN}`
        : `Subtract numerators: ${aN} − ${bN} = ${rawN}`,
      `Result: ${rawN}/${cd}`,
      (sn !== rawN || sd !== cd) ? `Simplified: ${fracStr(sn, sd)}` : `Already in lowest terms`,
      `As a decimal: ${isFinite(sn / sd) ? (sn / sd).toFixed(6).replace(/\.?0+$/, "") || "0" : "unde
    ];
    [rn, rd] = [sn, sd];
  }

  // ── Build SVG output ──────────────────────────────────────────
  const W = 560, lineH = 24, topPad = 14, sidePad = 20;
  const H = topPad + steps.length * lineH + 60 + 20;

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width", "100%")
    .attr("style", "max-width:560px; display:block; margin:0 auto; font-family:sans-serif; backgrou

  // Header: A op B =
  const headerText = `${an}/${ad}  ${operation}  ${bn}/${bd}`;
  svg.append("text")
    .attr("x", W / 2).attr("y", topPad + 16)
    .attr("text-anchor", "middle")
    .attr("font-size", 16).attr("font-weight", "bold").attr("fill", "#1e3a5f")
    .text(headerText);

  // Step-by-step lines
  steps.filter(Boolean).forEach((s, i) => {
```

```
  const isLast = i === steps.filter(Boolean).length - 1;
  svg.append("text")
    .attr("x", sidePad).attr("y", topPad + 40 + i * lineH)
    .attr("font-size", 13)
    .attr("fill", isLast ? "#166534" : "#374151")
    .attr("font-weight", isLast ? "bold" : "normal")
    .text(`${i + 1}. ${s}`);
});

// Result box (only when result is valid)
if (isFinite(rn / rd)) {
  const boxY = topPad + 40 + steps.filter(Boolean).length * lineH + 4;
  svg.append("rect")
    .attr("x", sidePad - 6).attr("y", boxY - 18)
    .attr("width", W - 2 * sidePad + 12).attr("height", 36)
    .attr("fill", "#dbeafe").attr("rx", 4).attr("stroke", "#93c5fd");

  // Area model for + and −: two coloured bars side by side in proportion
  if (operation === "+" || operation === "−") {
    // Show A and B as proportional bars, stacked / subtracted
    const barW = W - 2 * sidePad - 150;
    const barH = 18;
    const barX = sidePad + 145;
    const barY = boxY - 16;
    // A bar
    const aFrac = Math.min(an / ad, 1);
    const bFrac = Math.min(bn / bd, 1);
    svg.append("rect")
      .attr("x", barX).attr("y", barY)
      .attr("width", aFrac * barW).attr("height", barH)
      .attr("fill", "#2563eb").attr("opacity", 0.8).attr("rx", 2);
    if (operation === "+") {
      svg.append("rect")
        .attr("x", barX + aFrac * barW).attr("y", barY)
        .attr("width", bFrac * barW).attr("height", barH)
        .attr("fill", "#16a34a").attr("opacity", 0.8).attr("rx", 2);
    } else {
      // Subtraction: show B removed from A
      const removeW = Math.min(bFrac, aFrac) * barW;
      svg.append("rect")
        .attr("x", barX + aFrac * barW - removeW).attr("y", barY)
        .attr("width", removeW).attr("height", barH)
        .attr("fill", "#dc2626").attr("opacity", 0.5).attr("rx", 2)
        .attr("stroke", "#dc2626").attr("stroke-dasharray", "3,2");
    }
    svg.append("rect")
      .attr("x", barX).attr("y", barY)
```

```
        .attr("width", barW).attr("height", barH)
        .attr("fill", "none").attr("stroke", "#94a3b8").attr("rx", 2);
      svg.append("text")
        .attr("x", barX - 4).attr("y", barY + 13)
        .attr("text-anchor", "end").attr("font-size", 10).attr("fill", "#64748b")
        .text("0");
      svg.append("text")
        .attr("x", barX + barW + 4).attr("y", barY + 13)
        .attr("text-anchor", "start").attr("font-size", 10).attr("fill", "#64748b")
        .text("1");
    }
  }

  return svg.node();
}
```

---

## 8.3. Worked examples

**Example 1 — Halving a recipe.** A recipe needs $\frac{3}{4}$ cup of flour, $1\frac{1}{2}$ cups of oats, and $\frac{5}{8}$ cup of sugar. You want to make half the recipe. What quantities do you need?

Multiply each ingredient by $\frac{1}{2}$.

Convert the mixed number first: $1\frac{1}{2} = \frac{3}{2}$.

$$\frac{3}{4} \times \frac{1}{2} = \frac{3}{8} \text{ cup flour}$$

$$\frac{3}{2} \times \frac{1}{2} = \frac{3}{4} \text{ cup oats}$$

$$\frac{5}{8} \times \frac{1}{2} = \frac{5}{16} \text{ cup sugar}$$

This means making half a recipe multiplies every ingredient by $\frac{1}{2}$.

---

**Example 2 — Splitting costs.** Three friends split the cost of a camping trip. They agree to share costs in proportion to how many nights each stays: one friend stays $\frac{1}{3}$ of the nights, another $\frac{5}{12}$, and the third $\frac{1}{4}$. The total cost is \$120. What does each pay?

Check the fractions sum to 1 first — to make sure everyone's share is accounted for:

$$\frac{4}{12} + \frac{5}{12} + \frac{3}{12} = \frac{12}{12} = 1$$

Friend A: $\frac{1}{3} \times 120 = 40$

Friend B: $\frac{5}{12} \times 120 = 50$

Friend C: $\frac{1}{4} \times 120 = 30$

Total: \$120. Check.

This means each person pays the same fraction of the cost as their fraction of the stay.

---

**Example 3 — Running distances.** You run 2.5 km on Monday, 1.75 km on Wednesday, and $\frac{7}{4}$ km on Friday. What is your total distance for the week?

Convert $\frac{7}{4}$ to a decimal so all three are in the same form: $\frac{7}{4} = 1.75$ km.

$$2.5 + 1.75 + 1.75 = 6.0 \text{ km}$$

This means it often helps to rewrite all numbers in the same form before adding.

---

**Example 4 — What fraction is left?** A bag of rice is 0.38 used up. What fraction remains? Express it as a decimal and as a fraction in lowest terms.

Remaining: $1 - 0.38 = 0.62$.

As a fraction: $0.62 = \frac{62}{100} = \frac{31}{50}$.

$\frac{31}{50}$ of the bag is left.

This means "remaining" often starts with subtracting from 1 whole.

## 8.4. Where this goes

This chapter fills in the spaces between whole numbers and gives us rules for working with them.

Fraction arithmetic is the foundation of **algebra**. When you solve $\frac{2x}{3} = 8$, you multiply both sides by $\frac{3}{2}$ — a fraction operation.

In calculus, the derivative is defined as a limit of a fraction: $\frac{\Delta y}{\Delta x}$ as $\Delta x \to 0$. That fraction needs to be understood, not just computed. The conceptual step from "fraction" to "rate of change" is not large; it requires only that you see the notation clearly.

**Where this shows up**

- Any time you split something equally — a bill, a recipe, a distance — you're doing fraction arithmetic.
- A structural engineer applies a safety factor of $\frac{3}{2}$ (or 1.5) to every load — multiplying by a fraction.
- A financial model apportions costs, equity, and interest using decimal fractions at every step.
- A GIS analyst computing land-cover proportions is adding fractions that must sum to 1.

The arithmetic is identical. The precision matters.

## 8.5. Exercises

1. A metal bar is $4\frac{3}{8}$ m long. A $1\frac{5}{8}$ m section is cut off. How much remains?
2. A recipe calls for $\frac{2}{3}$ cup of milk. You want to make $2\frac{1}{2}$ times the recipe. How much milk do you need?
3. A sports pitch is 0.75 km long and 0.48 km wide. What is its area in km²? Express the answer as a decimal and as a fraction in lowest terms.
4. Three taps fill a tank. Each supplies $\frac{3}{8}$ of the tank's capacity per hour. A drain removes $\frac{5}{8}$ of the tank's capacity per hour. Is the tank filling or emptying? At what net fraction per hour?
5. A bag of trail mix is analysed: 0.44 nuts, 0.29 dried fruit, 0.18 chocolate chips, and the rest is seeds. What fraction is seeds? Express as a decimal and a simplified fraction.
6. You have a 500 mL bottle of juice concentrate. The instructions say to mix it at $\frac{1}{4}$ strength. What fraction of the final drink will be concentrate? How many mL of concentrate goes into a 250 mL glass?
7. Convert each to a decimal, then order from least to greatest: $\frac{7}{12}$, $\frac{3}{5}$, 0.58, $\frac{11}{20}$, $0.\overline{6}$.

# 9. Statistics and probability

Reasoning honestly from limited data

A football team scores 1, 3, 0, 2, 1, 4, 1 goals across seven matches. What is their typical score?

A weather forecast says 70% chance of rain. What does that actually mean?

Two students both average 65% on their tests, but one scores between 60–70% every time and the other swings from 20% to 100%. Are they the same?

These questions all need the same toolkit: averages, spread, and probability. The numbers tell you something — but only if you know which number to look at.

## 9.1. What the notation is saying

Start with a simple list of scores: 4, 6, 7, 7, 9. We could ask two different questions. What is a typical score? And how spread out are the scores?

**The mean** $\bar{x}$ is the sum of all values divided by the number of values. Given data $x_1, x_2, \ldots, x_n$:

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_n}{n} = \frac{\sum x_i}{n}$$

The $\Sigma$ (capital sigma) means "sum all of the following." It is notation for a loop: add every term $x_i$ for $i = 1$ to $n$.

Here, $x_1, x_2, \ldots, x_n$ are the data values, and $n$ is how many values there are. This means the mean is the total shared equally.

**The median** is the middle value when the data are ordered. If there are two middle values, take their mean. The median is less sensitive to extreme values than the mean.

This means the median tells us what sits in the middle, not what would happen if we shared everything equally.

**The range** is maximum minus minimum. It measures total spread but is heavily influenced by outliers.

This means range uses only the two end values.

**Probability** of event $A$ is:

$$P(A) = \frac{\text{number of favourable outcomes}}{\text{total number of equally likely outcomes}}$$

$P(A) = 0$: impossible. $P(A) = 1$: certain. All probabilities for a complete set of outcomes sum to 1.

Here, $A$ is the event we care about. This means probability is a fraction of all possible outcomes.

**Relative frequency**: when you can't count outcomes theoretically, estimate probability from data:

$$P(A) \approx \frac{\text{number of times A occurred}}{\text{total trials}}$$

This means data can give us an estimate of probability, even when we cannot work it out exactly in advance.

## 9.2. The method

### Computing the mean

1. Add all values — to get the total.
2. Divide by the count — to find the fair share per value.

This means the mean is the equal-share value.

### Computing the median

1. Order the values from least to greatest.
2. If $n$ is odd, the median is the value at position $\frac{n+1}{2}$.
3. If $n$ is even, average the values at positions $\frac{n}{2}$ and $\frac{n}{2} + 1$.

This means the median depends on order, so you must sort the data first.

### Computing the range and IQR

Range: max − min.

IQR (interquartile range): split the ordered data in half. Lower quartile Q1 = median of the lower half. Upper quartile Q3 = median of the upper half. IQR = $Q3 - Q1$.

The IQR describes the spread of the middle 50% of the data — it ignores the extremes at both ends.

This means IQR is often more stable than range when there are outliers.

### Computing probability from equally likely outcomes

1. Count total possible outcomes.
2. Count outcomes that match event $A$.
3. Divide.

This means probability compares successful outcomes to all possible ones.

> **Why this works**
> The mean is the "balance point" of a data set — if you placed equal weights at each value on a number line, the mean is where the line would balance. The median is the "middle" — it splits the data 50/50. They give the same answer for symmetric data; they diverge when the data is skewed. The median is more robust: one extreme outlier shifts the mean but not the median. Probability is defined between 0 and 1 because it is a fraction of a complete set of outcomes. When probabilities sum to more than 1, you have double-counted. When they sum to less than 1, you have missed some outcomes.

---

*Edit any of the seven scores below. Watch the mean, median, range, and IQR update instantly — and see the difference between mean and median when you push one value to an extreme.*

```
//| echo: false
viewof v1 = Inputs.number({ label: "Score 1", value: 62, min: 0, max: 100, step: 1 })
```

```
//| echo: false
viewof v2 = Inputs.number({ label: "Score 2", value: 71, min: 0, max: 100, step: 1 })
```

```
//| echo: false
viewof v3 = Inputs.number({ label: "Score 3", value: 58, min: 0, max: 100, step: 1 })
```

```
//| echo: false
viewof v4 = Inputs.number({ label: "Score 4", value: 74, min: 0, max: 100, step: 1 })
```

```
//| echo: false
viewof v5 = Inputs.number({ label: "Score 5", value: 66, min: 0, max: 100, step: 1 })
```

```
//| echo: false
viewof v6 = Inputs.number({ label: "Score 6", value: 70, min: 0, max: 100, step: 1 })
```

```
//| echo: false
viewof v7 = Inputs.number({ label: "Score 7", value: 63, min: 0, max: 100, step: 1 })
```

```
//| echo: false
{
  // ── Helpers ──────────────────────────────────────────────
  function medianOf(arr) {
    const s = [...arr].sort((a, b) => a - b);
    const mid = Math.floor(s.length / 2);
    return s.length % 2 === 1 ? s[mid] : (s[mid - 1] + s[mid]) / 2;
  }
  function quartiles(arr) {
```

```
  const s = [...arr].sort((a, b) => a - b);
  const n = s.length;
  const lower = s.slice(0, Math.floor(n / 2));
  const upper = n % 2 === 0 ? s.slice(n / 2) : s.slice(Math.ceil(n / 2));
  return { q1: medianOf(lower), q3: medianOf(upper) };
}

// ── Data ──────────────────────────────────────────────────
const raw = [v1, v2, v3, v4, v5, v6, v7].map(x => isFinite(x) ? x : 0);
const sorted = [...raw].sort((a, b) => a - b);
const n = raw.length;
const sum = raw.reduce((a, b) => a + b, 0);
const mean = sum / n;
const med = medianOf(raw);
const range_ = sorted[n - 1] - sorted[0];
const { q1, q3 } = quartiles(raw);
const iqr = q3 - q1;
const medIdx = Math.floor(n / 2); // index in sorted array (0-based)

// ── Layout ────────────────────────────────────────────────
const W = 560, PAD = 40, DOT_AREA_H = 120, STATS_H = 110;
const H = DOT_AREA_H + STATS_H + 20;

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", "max-width:560px; display:block; margin:0 auto; font-family:sans-serif;");

// ── Dot plot ──────────────────────────────────────────────
const plotW = W - 2 * PAD;
const xMin = 0, xMax = 100;
const xScale = v => PAD + (v - xMin) / (xMax - xMin) * plotW;
const axisY = 70;

// Axis line
svg.append("line")
  .attr("x1", PAD).attr("y1", axisY)
  .attr("x2", W - PAD).attr("y2", axisY)
  .attr("stroke", "#94a3b8").attr("stroke-width", 1.5);

// Tick marks every 10
for (let t = 0; t <= 100; t += 10) {
  const x = xScale(t);
  svg.append("line")
    .attr("x1", x).attr("y1", axisY - 4)
    .attr("x2", x).attr("y2", axisY + 4)
    .attr("stroke", "#94a3b8").attr("stroke-width", 1);
```

```
  svg.append("text")
    .attr("x", x).attr("y", axisY + 15)
    .attr("text-anchor", "middle").attr("font-size", 10).attr("fill", "#64748b")
    .text(t);
}

// Axis label
svg.append("text")
  .attr("x", W / 2).attr("y", axisY + 28)
  .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#64748b")
  .text("Test score (0–100)");

// Dots — stack duplicates
const stackCount = {};
raw.forEach(v => { stackCount[v] = (stackCount[v] || 0) + 1; });
const placed = {};
raw.forEach(v => {
  placed[v] = (placed[v] || 0);
  const cx = xScale(v);
  const cy = axisY - 12 - placed[v] * 16;
  svg.append("circle")
    .attr("cx", cx).attr("cy", cy).attr("r", 7)
    .attr("fill", "#475569").attr("stroke", "#1e293b").attr("stroke-width", 1);
  placed[v]++;
});

// Mean vertical line
svg.append("line")
  .attr("x1", xScale(mean)).attr("y1", 8)
  .attr("x2", xScale(mean)).attr("y2", axisY)
  .attr("stroke", "#2563eb").attr("stroke-width", 2)
  .attr("stroke-dasharray", "4,2");
svg.append("text")
  .attr("x", xScale(mean)).attr("y", 6)
  .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#2563eb")
  .attr("font-weight", "bold")
  .text(`Mean ${mean.toFixed(1)}`);

// Median vertical line
const medOffset = Math.abs(xScale(med) - xScale(mean)) < 20 ? 12 : 0;
svg.append("line")
  .attr("x1", xScale(med)).attr("y1", 8)
  .attr("x2", xScale(med)).attr("y2", axisY)
  .attr("stroke", "#dc2626").attr("stroke-width", 2)
  .attr("stroke-dasharray", "6,2");
svg.append("text")
  .attr("x", xScale(med)).attr("y", 6 + medOffset)
```

```
    .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#dc2626")
    .attr("font-weight", "bold")
    .text(`Median ${med}`);

  // ── Stats table ──────────────────────────────────────────────
  const tY = DOT_AREA_H + 4;

  // Background panel
  svg.append("rect")
    .attr("x", PAD - 10).attr("y", tY)
    .attr("width", W - 2 * PAD + 20).attr("height", STATS_H)
    .attr("fill", "#f1f5f9").attr("rx", 4);

  const col1 = PAD, col2 = PAD + 180, col3 = PAD + 360;
  const rowH = 22;

  const stats = [
    [`Mean  x̄  =  (${raw.join(" + ")}) ÷ ${n}`, `= ${mean.toFixed(2)}`],
    [`Median (middle of sorted list)`, `= ${med}`],
    [`Range  =  ${sorted[n-1]} − ${sorted[0]}`, `= ${range_}`],
    [`Q1 = ${q1}   Q3 = ${q3}   IQR = Q3 − Q1`, `= ${iqr}`],
  ];

  stats.forEach([label, val], i) => {
    svg.append("text")
      .attr("x", col1).attr("y", tY + 20 + i * rowH)
      .attr("font-size", 12).attr("fill", "#1e293b")
      .text(label);
    svg.append("text")
      .attr("x", W - PAD).attr("y", tY + 20 + i * rowH)
      .attr("text-anchor", "end").attr("font-size", 12)
      .attr("fill", i === 0 ? "#2563eb" : i === 1 ? "#dc2626" : "#374151")
      .attr("font-weight", "bold")
      .text(val);
  });

  // Sorted list display
  svg.append("text")
    .attr("x", PAD).attr("y", tY + STATS_H - 8)
    .attr("font-size", 11).attr("fill", "#64748b")
    .text(`Sorted: ${sorted.join(", ")}`);

  return svg.node();
}
```

**Try this:** change Score 1 from 62 to 2. Watch the mean drop noticeably while the median barely shifts. That is the difference between the two measures — and why it matters which one you use.

---

## 9.3. Worked examples

**Example 1 — Test scores.** A student scores the following in seven tests: 62, 71, 58, 74, 66, 70, 63. Find the mean, median, and range.

Ordered: 58, 62, 63, 66, 70, 71, 74.

Mean — add all values, then divide by 7:

$$\bar{x} = \frac{62 + 71 + 58 + 74 + 66 + 70 + 63}{7} = \frac{464}{7} = 66.3$$

Range — highest minus lowest:

$$74 - 58 = 16$$

Median — the 4th value in the ordered list:

$$66$$

The mean and median are close, which tells you the scores are fairly evenly spread — no extreme result is pulling the average up or down.

This means either measure gives a similar picture for this set of data.

---

**Example 1b — IQR for the same dataset.** Ordered data (7 values): 58, 62, 63, **66**, 70, 71, 74.

The median is the 4th value: 66. For an odd-count dataset, exclude the median when splitting into halves.

Lower half (values 1–3): 58, 62, 63. Q1 = median = 62.

Upper half (values 5–7): 70, 71, 74. Q3 = median = 71.

$$\text{IQR} = Q3 - Q1 = 71 - 62 = 9$$

The middle 50% of scores sit within a 9-point range.

This means the centre of the data is less spread out than the full range suggests.

---

**Example 2 — Weather probability.** Over the past 60 days, it has rained on 15 of them. Estimate the probability of rain on any given day.

$$P(\text{rain}) \approx \frac{15}{60} = \frac{1}{4} = 0.25 = 25\%$$

This is a relative frequency estimate — it uses past data rather than counting equally likely outcomes. The more data you have, the more reliable the estimate.

This means 25% is an estimate from evidence, not a guarantee.

---

**Example 3 — Picking from a group.** A class has 12 students who play sport: 5 play football, 4 play basketball, and 3 play tennis. One student is picked at random to represent the class. What is the probability they play football? What is the probability they do not play basketball?

$$P(\text{football}) = \frac{5}{12}$$

$$P(\text{not basketball}) = \frac{12-4}{12} = \frac{8}{12} = \frac{2}{3}$$

This means "not basketball" includes every outcome except basketball.

---

**Example 4 — Expected score.** A game show gives two options: take \$300 for certain, or spin a wheel that pays \$800 with probability 0.5 and \$0 with probability 0.5. What is the expected value of spinning the wheel?

$$E = (0.5 \times 800) + (0.5 \times 0) = 400 + 0 = 400$$

The expected value of spinning is 400 — higher than the certain \$300. But "expected value" means the average across many spins, not a guarantee for this one spin. You could walk away with nothing.

This means the best long-run average choice is not always the safest single choice.

---

*The simulator below shows the difference between theoretical probability and what actually happens in a finite number of draws. Results vary each time you click — that is the point.*

```
//| echo: false
viewof red_count = Inputs.range([1, 10], {
  value: 3,
  step: 1,
  label: "Red counters in bag"
})
```

```
//| echo: false
viewof blue_count = Inputs.range([1, 10], {
  value: 5,
  step: 1,
  label: "Blue counters in bag"
})
```

```
//| echo: false
viewof sim_button = Inputs.button("Simulate 100 draws")
```

```
//| echo: false
{
  // Trigger on button press (sim_button changes each click)
  sim_button;

  const total = red_count + blue_count;
  const pRed = red_count / total;
  const pBlue = blue_count / total;

  // Run 100 draws using Math.random()
  // Note: results vary on each click — this is intentional and illustrates
  // that probability is a long-run prediction, not a guarantee for any
  // finite sample.
  const N = 100;
  let simRed = 0, simBlue = 0;
  for (let i = 0; i < N; i++) {
    if (Math.random() < pRed) simRed++; else simBlue++;
  }
  const simPRed = simRed / N;
  const simPBlue = simBlue / N;

  // ── Layout ──────────────────────────────────────────────────
  const W = 560, H = 300, PAD_L = 120, PAD_R = 30, PAD_T = 30, PAD_B = 60;
  const plotW = W - PAD_L - PAD_R;
  const plotH = H - PAD_T - PAD_B;

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width", "100%")
    .attr("style", "max-width:560px; display:block; margin:0 auto; font-family:sans-serif;");
```

```
// —— Bag visualisation (left panel) ————————————————————————
// Show counters as small circles in a 4-column grid, coloured and labelled
const allCounters = [
  ...Array(red_count).fill("red"),
  ...Array(blue_count).fill("blue")
];
const COLS = 4;
const CR = 10, CG = 5; // circle radius, gap
const gridX0 = 6, gridY0 = PAD_T + 10;
allCounters.forEach((colour, i) => {
  const col = i % COLS;
  const row = Math.floor(i / COLS);
  const cx = gridX0 + col * (CR * 2 + CG) + CR;
  const cy = gridY0 + row * (CR * 2 + CG) + CR;
  svg.append("circle")
    .attr("cx", cx).attr("cy", cy).attr("r", CR)
    .attr("fill", colour === "red" ? "#dc2626" : "#2563eb")
    .attr("stroke", colour === "red" ? "#7f1d1d" : "#1e3a5f")
    .attr("stroke-width", 1.5);
});

// Bag label
svg.append("text")
  .attr("x", gridX0 + (COLS * (CR * 2 + CG)) / 2)
  .attr("y", H - PAD_B + 15)
  .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#374151")
  .text(`Bag: ${red_count} red, ${blue_count} blue`);

svg.append("text")
  .attr("x", gridX0 + (COLS * (CR * 2 + CG)) / 2)
  .attr("y", H - PAD_B + 27)
  .attr("text-anchor", "middle").attr("font-size", 10).attr("fill", "#64748b")
  .text(`Total: ${total}`);

// —— Bar chart ————————————————————————————————————————————
const barData = [
  { label: "Red",  theory: pRed,  sim: simPRed,  colour: "#dc2626", stripe: "#7f1d1d" },
  { label: "Blue", theory: pBlue, sim: simPBlue, colour: "#2563eb", stripe: "#1e3a5f" }
];

const groupW = plotW / barData.length;
const barW = groupW * 0.35;
const yScale = v => PAD_T + plotH * (1 - v);

// Y axis
svg.append("line")
  .attr("x1", PAD_L).attr("y1", PAD_T)
```

```
  .attr("x2", PAD_L).attr("y2", PAD_T + plotH)
  .attr("stroke", "#94a3b8").attr("stroke-width", 1.5);

// Y ticks
[0, 0.25, 0.5, 0.75, 1.0].forEach(v => {
  const y = yScale(v);
  svg.append("line")
    .attr("x1", PAD_L - 5).attr("y1", y)
    .attr("x2", PAD_L).attr("y2", y)
    .attr("stroke", "#94a3b8").attr("stroke-width", 1);
  svg.append("text")
    .attr("x", PAD_L - 8).attr("y", y + 4)
    .attr("text-anchor", "end").attr("font-size", 10).attr("fill", "#64748b")
    .text(v.toFixed(2));
  // Gridline
  svg.append("line")
    .attr("x1", PAD_L).attr("y1", y)
    .attr("x2", PAD_L + plotW).attr("y2", y)
    .attr("stroke", "#e2e8f0").attr("stroke-width", 1);
});

// Y axis label
svg.append("text")
  .attr("transform", `translate(${PAD_L - 40}, ${PAD_T + plotH / 2}) rotate(-90)`)
  .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#374151")
  .text("Probability / relative frequency");

// X axis
svg.append("line")
  .attr("x1", PAD_L).attr("y1", PAD_T + plotH)
  .attr("x2", PAD_L + plotW).attr("y2", PAD_T + plotH)
  .attr("stroke", "#94a3b8").attr("stroke-width", 1.5);

barData.forEach((d, gi) => {
  const gx = PAD_L + gi * groupW + groupW / 2;

  // Theory bar (solid fill)
  const th = yScale(d.theory);
  const thH = (PAD_T + plotH) - th;
  svg.append("rect")
    .attr("x", gx - barW - 2).attr("y", th)
    .attr("width", barW).attr("height", thH)
    .attr("fill", d.colour).attr("opacity", 0.9)
    .attr("rx", 2);

  // Sim bar (hatched: lighter fill + dashed outline)
  const sh = yScale(d.sim);
```

```
    const shH = (PAD_T + plotH) - sh;
    svg.append("rect")
      .attr("x", gx + 2).attr("y", sh)
      .attr("width", barW).attr("height", shH)
      .attr("fill", d.colour).attr("opacity", 0.35)
      .attr("stroke", d.colour).attr("stroke-width", 1.5)
      .attr("stroke-dasharray", "4,2")
      .attr("rx", 2);

    // Value labels on bars
    svg.append("text")
      .attr("x", gx - barW / 2 - 2).attr("y", th - 3)
      .attr("text-anchor", "middle").attr("font-size", 10)
      .attr("fill", d.stripe).attr("font-weight", "bold")
      .text(`${(d.theory * 100).toFixed(0)}%`);

    svg.append("text")
      .attr("x", gx + barW / 2 + 2).attr("y", sh - 3)
      .attr("text-anchor", "middle").attr("font-size", 10)
      .attr("fill", d.stripe).attr("font-weight", "bold")
      .text(`${(d.sim * 100).toFixed(0)}%`);

    // Group label (colour name)
    svg.append("text")
      .attr("x", gx).attr("y", PAD_T + plotH + 16)
      .attr("text-anchor", "middle").attr("font-size", 12).attr("fill", "#374151")
      .text(d.label);
  });

  // ── Legend ──────────────────────────────────────────────
  const legX = PAD_L + 10, legY = PAD_T + 6;
  // Solid = theory
  svg.append("rect")
    .attr("x", legX).attr("y", legY).attr("width", 14).attr("height", 10)
    .attr("fill", "#374151").attr("opacity", 0.8).attr("rx", 1);
  svg.append("text")
    .attr("x", legX + 18).attr("y", legY + 9)
    .attr("font-size", 10).attr("fill", "#374151")
    .text("Theoretical probability");

  svg.append("rect")
    .attr("x", legX + 150).attr("y", legY).attr("width", 14).attr("height", 10)
    .attr("fill", "#374151").attr("opacity", 0.35)
    .attr("stroke", "#374151").attr("stroke-width", 1.5)
    .attr("stroke-dasharray", "3,2").attr("rx", 1);
  svg.append("text")
    .attr("x", legX + 168).attr("y", legY + 9)
```

```
    .attr("font-size", 10).attr("fill", "#374151")
    .text(`Simulated (${N} draws)`);

  // — Summary line ————————————————————————————————————
  svg.append("text")
    .attr("x", W / 2).attr("y", H - 10)
    .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#374151")
    .text(`Red: theory ${(pRed*100).toFixed(1)}%  vs  simulated ${(simPRed*100).toFixed(0)}%  |

  return svg.node();
}
```

**Simulation note:** each click on "Simulate 100 draws" generates a fresh set of random draws. The simulated bars will be close to the theoretical bars but rarely identical — that gap between prediction and observation is real and expected. With 1 000 draws the bars would be closer; with 10 draws they might be very far apart. More data means a more reliable estimate.

## 9.4. Where this goes

This chapter gives us the first tools for reading data honestly. We are no longer asking only "what is the answer?" We are also asking "how typical is it?" and "how sure are we?"

**Volume 6** develops this much further: probability distributions, hypothesis testing, and regression.

The expected value calculation in Example 4 is also the foundation of decision theory, financial option pricing (Black-Scholes is built on expected values under a probability distribution), and machine learning loss functions. The concept is old. The applications are new.

**Where this shows up**

- Sports commentators and coaches use averages and ranges to compare players' performance over a season.
- A weather forecast's percentage chance of rain is a probability estimated from thousands of past days with similar conditions.
- A lab scientist reports every measurement with ± uncertainty — that is a statement about spread.
- A data scientist evaluates a classifier by its error rate — a probability computed from test data.

Statistics is not a course you take. It is a way of reading claims about the world.

## 9.5. Exercises

1. The daily high temperatures for one week (°C): −4, −7, −2, 1, 3, −1, −5. Find the mean, median, and range.
2. A student scores 62, 71, 85, 68, 90, 74, and 55 on seven tests. Find the mean and median. A new student joins the group with a score of 4. Recalculate the mean. Does the median change? What does this show about which measure is more stable under an extreme value?
3. A standard die has faces 1–6. What is the probability of rolling a number greater than 4? Of rolling an even number? Of rolling a 7?
4. A bag contains 4 red, 6 blue, and 2 green marbles. A marble is drawn at random.

   a) What is P(red)?
   b) What is P(not blue)?
   c) What is P(red or green)?

5. A school canteen tracks how many students choose each lunch option over 200 days: hot meal 110 times, sandwich 60 times, salad 30 times. Estimate the probability a randomly chosen day has more than half the students choosing a hot meal. Out of the next 50 days, how many would you expect?
6. A quiz game has three options:

   - Option A: 80% chance of +$500, 20% chance of −$200
   - Option B: 50% chance of +$900, 50% chance of −$100
   - Option C: certain return of +$280

   Compute the expected value of each option. Which is highest? Which would you choose, and why might expected value alone not be the only consideration?
7. A set of 9 test scores has a median of 14.2 and a mean of 15.1. One outlier is identified and removed, leaving 8 values with a mean of 14.6. What was the outlier value? *(Hint: total of 9 values = mean × 9. Total of remaining 8 = new mean × 8. Outlier = first total − second total.)*

# Part II.

# Volume 2 — Patterns and algebra

# 10. Volume 2 — Patterns and algebra

The central move of this volume is naming the gap.

You already know how numbers behave. This volume is about what happens when one of the numbers is missing — and about the notation that lets you work with that absence as if it were a thing you could hold.

By the end of this volume you will be able to write a relationship down symbolically, solve for unknowns, recognise when a relationship is linear versus something else, and read a graph as a statement about how two quantities relate.

These are not abstract skills. They are the toolkit that every quantitative field — engineering, science, finance, computing, geography — uses before it reaches for anything more sophisticated.

## 10.1. Chapters

| Chapter | Title | Core move |
|---|---|---|
| 1 | Linear equations | Isolating the unknown |
| 2 | Inequalities | When the answer is a range |
| 3 | Patterns and sequences | When the relationship repeats |
| 4 | Graphing relations | When the relationship has a shape |

# 11. Linear equations

Finding the unknown

You and three friends want to split a bill evenly, but one person already paid for something earlier. How much does each person owe now?

You earn $8.50 an hour at a weekend job. You need $120 for a gig ticket. How many shifts do you need?

You're buying snacks for a group. Each bag costs the same. You spend $14.40 in total. How much was each bag?

These are all the same question underneath: you know the relationship, you're missing one of the values. A linear equation is just that relationship written down, with a placeholder for the missing part.

## 11.1. What the notation is saying

Start with a sentence instead of symbols: "three bags and 7 dollars cost 22 dollars." An equation is that sentence written in a compact way.

$$3x + 7 = 22$$

Read it as: three of something, plus seven, gives twenty-two. The $x$ is the placeholder — a name for the gap in your knowledge. The equals sign means both sides describe the same amount.

This means an equation is a balance between two expressions.

Peel away what's around the unknown until it stands alone. Three of something plus seven is twenty-two, so three of something is fifteen, so the something is five. Check: $3 \times 5 + 7 = 22$. Yes.

That move — isolating the unknown — is what this chapter is about.

This means solving is the process of getting the unknown by itself.

The concept stays this simple. What gets harder is the notation accumulating, and equations appearing inside larger problems where it isn't obvious that's what they are.

---

**Interactive: Equation balance.** The scale below shows both sides of an equation. Choose an operation and a value. Apply it to both sides and watch the equation update — the balance holds because you're changing both sides equally.

```
//| echo: false
d3 = require("d3@7")
```

```
//| echo: false
viewof balanceOp = Inputs.select(["+", "−", "×", "÷"], {
  label: "Operation",
  value: "+"
})
```

```
//| echo: false
viewof balanceVal = Inputs.range([-10, 10], {
  label: "Value to apply to both sides",
  step: 1,
  value: 3
})
```

```
//| echo: false
{
  // Starting equation: 3x + 7 = 22
  // We track the numeric sides as they stand after one operation.
  // Left side: 3x + 7  →  represented symbolically
  // Right side: 22
  // For display we show the symbolic equation text and what each side becomes.

  const op  = balanceOp;
  const val = Number.isFinite(balanceVal) ? Math.round(balanceVal) : 3;

  // Guard against division by zero
  const divByZero = op === "÷" && val === 0;

  // Represent the original equation symbolically
  const origLeft  = "3x + 7";
  const origRight = "22";

  // Compute what happens to the right-hand side (a pure number)
  let newRight;
  if (op === "+")       newRight = 22 + val;
  else if (op === "−")  newRight = 22 - val;
  else if (op === "×")  newRight = 22 * val;
  else if (op === "÷")  newRight = divByZero ? null : 22 / val;

  // Build a symbolic description of the new left side
  let newLeftDesc;
```

```
const valStr = val < 0 ? `(${val})` : `${val}`;
if (op === "+")       newLeftDesc = `3x + 7 + ${valStr}`;
else if (op === "−")  newLeftDesc = `3x + 7 − ${valStr}`;
else if (op === "×")  newLeftDesc = `${valStr}(3x + 7)`;
else if (op === "÷")  newLeftDesc = divByZero ? "3x + 7 ÷ 0" : `(3x + 7) ÷ ${valStr}`;

// Display
const W = 560;
const H = 260;
const MID_X = W / 2;
const BEAM_Y = 120;
const PIVOT_Y = 80;
const PAN_Y   = 170;
const PAN_W   = 150;
const PAN_H   = 44;
const ROPE_H  = 50;

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", `max-width:${W}px; font-family: inherit;`);

// ── Pivot post ──
svg.append("line")
  .attr("x1", MID_X).attr("y1", PIVOT_Y - 10)
  .attr("x2", MID_X).attr("y2", BEAM_Y)
  .attr("stroke", "#9ca3af").attr("stroke-width", 3);

svg.append("circle")
  .attr("cx", MID_X).attr("cy", PIVOT_Y - 10)
  .attr("r", 6)
  .attr("fill", "#6b7280");

// ── Beam ──
svg.append("line")
  .attr("x1", 80).attr("y1", BEAM_Y)
  .attr("x2", W - 80).attr("y2", BEAM_Y)
  .attr("stroke", "#374151").attr("stroke-width", 4)
  .attr("stroke-linecap", "round");

// ── Ropes ──
const leftPanCX  = 130;
const rightPanCX = W - 130;

svg.append("line")
  .attr("x1", leftPanCX).attr("y1", BEAM_Y)
  .attr("x2", leftPanCX).attr("y2", PAN_Y - PAN_H / 2)
```

```
  .attr("stroke", "#9ca3af").attr("stroke-width", 2);

svg.append("line")
  .attr("x1", rightPanCX).attr("y1", BEAM_Y)
  .attr("x2", rightPanCX).attr("y2", PAN_Y - PAN_H / 2)
  .attr("stroke", "#9ca3af").attr("stroke-width", 2);

// — Pans —
const panFill = "#f3f4f6";
const panStroke = "#d1d5db";

svg.append("rect")
  .attr("x", leftPanCX - PAN_W / 2).attr("y", PAN_Y - PAN_H / 2)
  .attr("width", PAN_W).attr("height", PAN_H)
  .attr("rx", 6)
  .attr("fill", panFill).attr("stroke", panStroke).attr("stroke-width", 1.5);

svg.append("rect")
  .attr("x", rightPanCX - PAN_W / 2).attr("y", PAN_Y - PAN_H / 2)
  .attr("width", PAN_W).attr("height", PAN_H)
  .attr("rx", 6)
  .attr("fill", panFill).attr("stroke", panStroke).attr("stroke-width", 1.5);

// — Pan labels: new expressions —
const leftLabel  = divByZero ? "÷ 0 not defined" : newLeftDesc;
const rightLabel = divByZero ? "÷ 0 not defined" :
  (Number.isInteger(newRight) ? `${newRight}` : newRight.toFixed(2));

svg.append("text")
  .attr("x", leftPanCX).attr("y", PAN_Y + 5)
  .attr("text-anchor", "middle")
  .attr("fill", divByZero ? "#dc2626" : "#1f2937")
  .attr("font-size", divByZero ? "10px" : "12px")
  .attr("font-weight", "700")
  .text(leftLabel);

svg.append("text")
  .attr("x", rightPanCX).attr("y", PAN_Y + 5)
  .attr("text-anchor", "middle")
  .attr("fill", divByZero ? "#dc2626" : "#1f2937")
  .attr("font-size", "14px")
  .attr("font-weight", "700")
  .text(rightLabel);

// — Equation line above balance —
const origEq = `Original: ${origLeft} = ${origRight}`;
svg.append("text")
```

```
    .attr("x", MID_X).attr("y", 32)
    .attr("text-anchor", "middle")
    .attr("fill", "#374151").attr("font-size", "13px").attr("font-weight", "600")
    .text(origEq);

  // — Operation applied —
  const opSymbol = op === "÷" ? "÷" : op === "×" ? "×" : op;
  const opDesc = divByZero
    ? "Cannot divide by zero — the balance breaks"
    : `Apply  ${opSymbol} ${val}  to both sides`;

  svg.append("text")
    .attr("x", MID_X).attr("y", 52)
    .attr("text-anchor", "middle")
    .attr("fill", divByZero ? "#dc2626" : "#0d9488")
    .attr("font-size", "12px")
    .text(opDesc);

  // — "Both sides equal" label —
  if (!divByZero) {
    svg.append("text")
      .attr("x", MID_X).attr("y", H - 12)
      .attr("text-anchor", "middle")
      .attr("fill", "#6b7280").attr("font-size", "11px")
      .text("Both sides still describe the same amount — the balance holds.");
  }

  // — "=" centred between pans —
  svg.append("text")
    .attr("x", MID_X).attr("y", PAN_Y + 6)
    .attr("text-anchor", "middle")
    .attr("fill", "#6b7280").attr("font-size", "18px").attr("font-weight", "700")
    .text("=");

  return svg.node();
}
```

---

## 11.2. The method

The rule is: whatever you do to one side of the equals sign, do to the other. The equation is a balance. You can add weight to both sides, remove weight from both sides, or multiply both sides by the same amount — as long as you do the same thing to both, the balance holds.

This means we are not guessing the answer. We are changing the equation without breaking it.

$$3x + 7 = 22$$

Subtract 7 from both sides — this removes the 7 from the left, leaving just the term with $x$:

$$3x = 15$$

Divide both sides by 3 — this peels away the coefficient, leaving $x$ alone:

$$x = 5$$

That's it. Two operations, one unknown, done.

Here, 3 is the coefficient of $x$. This means it is the number multiplying the unknown.

> **Why this works**
> The equals sign is a statement that two things are the same amount. If you change one side without changing the other, they're no longer the same amount — you've broken the equation. Doing the same thing to both sides preserves the equality. That's the whole method.

---

**Interactive: Equation solver.** Set the coefficients $a$, $b$, and $c$ in the equation $ax + b = c$. The solution steps update live with your numbers.

```
//| echo: false
viewof solveA = Inputs.range([-10, 10], {
  label: "a  (coefficient of x)",
  step: 1,
  value: 3
})
```

```
//| echo: false
viewof solveB = Inputs.range([-20, 20], {
  label: "b  (constant on left)",
  step: 1,
  value: 7
})
```

```
//| echo: false
viewof solveC = Inputs.range([-50, 50], {
  label: "c  (right-hand side)",
  step: 1,
  value: 22
})
```

```
//| echo: false
{
  const a = Number.isFinite(solveA) ? Math.round(solveA) : 3;
  const b = Number.isFinite(solveB) ? Math.round(solveB) : 7;
  const c = Number.isFinite(solveC) ? Math.round(solveC) : 22;

  const noSolution = a === 0;
  const cMinusB    = c - b;
  const x          = noSolution ? null : cMinusB / a;

  // Format a coefficient: omit "1x" → "x", handle negatives neatly
  const coeffStr = (n) => {
    if (n === 1)  return "x";
    if (n === -1) return "−x";
    if (n < 0)    return `${n}x`;
    return `${n}x`;
  };

  // Format the left side "ax + b" nicely
  const leftSide = () => {
    const aPart = coeffStr(a);
    if (b === 0) return `${aPart}`;
    if (b > 0)   return `${aPart} + ${b}`;
    return `${aPart} − ${Math.abs(b)}`;
  };

  const W = 560;
  const H = noSolution ? 130 : 200;
  const ROW_H = 38;
  const LABEL_X = 20;
  const EQ_X = 180;

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width", "100%")
    .attr("style", `max-width:${W}px; font-family: inherit;`);

  // Header: equation as given
  svg.append("text")
    .attr("x", LABEL_X).attr("y", 28)
    .attr("fill", "#1f2937").attr("font-size", "15px").attr("font-weight", "700")
    .text(`Solve:  ${leftSide()} = ${c}`);

  if (noSolution) {
    svg.append("text")
      .attr("x", LABEL_X).attr("y", 70)
      .attr("fill", "#dc2626").attr("font-size", "13px").attr("font-weight", "600")
```

```
      .text("a = 0: no unique solution (x drops out of the equation).");

    svg.append("text")
      .attr("x", LABEL_X).attr("y", 96)
      .attr("fill", "#6b7280").attr("font-size", "12px")
      .text(b === c
        ? `With a = 0 the equation reads ${b} = ${c}, which is always true — any x works.`
        : `With a = 0 the equation reads ${b} = ${c}, which is never true — no x works.`
      );
  } else {
    // Step 1
    const step1Y = 64;
    svg.append("text")
      .attr("x", LABEL_X).attr("y", step1Y)
      .attr("fill", "#6b7280").attr("font-size", "12px")
      .text(`Step 1 — subtract ${b < 0 ? `(${b})` : b} from both sides`
            + (b < 0 ? "  [adding a negative]" : "")
            + ":");

    const bStr = b < 0 ? `+ (${b})` : `− ${b}`;
    svg.append("text")
      .attr("x", EQ_X).attr("y", step1Y + 20)
      .attr("fill", "#1f2937").attr("font-size", "14px").attr("font-weight", "600")
      .text(`${coeffStr(a)} = ${c} ${bStr}  =  ${cMinusB}`);

    // Step 2
    const step2Y = step1Y + 56;
    svg.append("text")
      .attr("x", LABEL_X).attr("y", step2Y)
      .attr("fill", "#6b7280").attr("font-size", "12px")
      .text(`Step 2 — divide both sides by ${a < 0 ? `(${a})` : a}:`);

    const xDisplay = Number.isInteger(x)
      ? `${x}`
      : x.toFixed(4).replace(/\.?0+$/, "");

    svg.append("text")
      .attr("x", EQ_X).attr("y", step2Y + 20)
      .attr("fill", "#1f2937").attr("font-size", "14px").attr("font-weight", "600")
      .text(`x = ${cMinusB} ÷ ${a}  =  ${xDisplay}`);

    // Answer box
    const ansY = step2Y + 46;
    svg.append("rect")
      .attr("x", EQ_X - 8).attr("y", ansY - 20)
      .attr("width", 220).attr("height", 32)
      .attr("rx", 6)
```

```
      .attr("fill", "#0d9488").attr("opacity", 0.12)
      .attr("stroke", "#0d9488").attr("stroke-width", 1.5);

    svg.append("text")
      .attr("x", EQ_X + 2).attr("y", ansY)
      .attr("fill", "#0d9488").attr("font-size", "15px").attr("font-weight", "700")
      .text(`Answer:  x = ${xDisplay}`);

    // Check
    const checkY = ansY + 28;
    const checkLeft  = a * (Number.isInteger(x) ? x : parseFloat(xDisplay)) + b;
    const checkRight = c;
    const checkOk    = Math.abs(checkLeft - checkRight) < 1e-9;
    svg.append("text")
      .attr("x", LABEL_X).attr("y", checkY)
      .attr("fill", checkOk ? "#6b7280" : "#dc2626")
      .attr("font-size", "11px")
      .text(checkOk
        ? `Check: ${a}×${xDisplay} + ${b} = ${checkLeft} ✓`
        : `Check: ${a}×${xDisplay} + ${b} = ${checkLeft}  (expected ${c}) — rounding`
      );
  }

  return svg.node();
}
```

---

## 11.3. Worked examples

**Example 1.** You're saving up for a festival wristband. The wristband costs $85. You already have $20 saved and earn $13 per hour doing odd jobs. How many hours do you need to work?

Let $h$ = hours worked.

$$20 + 13h = 85$$

Subtract 20 from both sides — to clear the fixed amount already saved:

$$13h = 65$$

Divide both sides by 13 — to get from "13 of something" to "one of something":

$$h = 5$$

Five hours. Check: $20 + 13 \times 5 = 20 + 65 = 85$. Yes.

This means the fixed amount you already have becomes the constant term, and the hourly pay becomes the coefficient of the unknown.

---

**Example 2.** Two streaming services. Service A costs \$8 per month plus \$2 per film you rent. Service B costs \$18 per month with unlimited films. How many film rentals per month makes the two services equal in cost?

Let $f$ = films rented per month.

Service A costs: $8 + 2f$

Service B costs: 18

They're equal when:

$$8 + 2f = 18$$

$$2f = 10$$

$$f = 5$$

At 5 films per month they cost the same. Above 5 films, Service B is cheaper.

This means the solution tells us the break-even point.

---

**Example 3.** You're making a fruit drink by mixing juice concentrate with water. The concentrate is 60% real juice. You want the final drink to be 20% real juice. You start with 1 litre of concentrate. How much water do you add?

The amount of real juice stays fixed at $0.60 \times 1 = 0.6$ litres. After adding $w$ litres of water, the total volume is $1 + w$ litres, and you want the juice fraction to be 20%:

$$\frac{0.6}{1 + w} = 0.2$$

Multiply both sides by $(1 + w)$ — to clear the fraction:

$$0.6 = 0.2(1 + w)$$

Expand the right side:

$$0.6 = 0.2 + 0.2w$$

Subtract 0.2 from both sides:

$$0.4 = 0.2w$$

Divide both sides by 0.2:

$$w = 2$$

Add 2 litres of water.

This means the amount of juice does not change, only the total volume does.

## 11.4. Where this goes

This chapter turns a verbal relationship into algebra and solves for the missing value. The next chapters keep the same idea but widen it.

Inequalities ask which values *satisfy* a relationship rather than pinning down one exact answer, and graphing shows what happens when you let the unknown vary continuously.

**Where this shows up**

- An electrical engineer writing Kirchhoff's voltage law is solving a linear equation for current or voltage.
- A nutritionist calculating how much of an ingredient hits a target percentage is solving a linear equation.
- A financial analyst finding a break-even point is solving a linear equation.
- A data scientist fitting a regression line is solving a system of linear equations.

The tool is the same. The domain changes.

## 11.5. Exercises

These are puzzles, not drills. Each one has a clean answer, but the interesting part is setting up the equation — translating the situation into the notation before you solve it.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
```

```
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

1. A bus pass costs $1.50 to buy, then $0.90 per journey. You spend $9.60 in total. How many journeys did you take?

```
//| echo: false
makeStepperHTML(1, [
  { op: "Write as an equation",           eq: "1.50 + 0.90j = 9.60" },
  { op: "Subtract 1.50 from both sides",  eq: "0.90j = 8.10" },
  { op: "Divide both sides by 0.90",      eq: "j = 9" },
  { op: "Check",                           eq: "1.50 + 0.90 \\times 9 = 1.50 + 8.10 = 9.60 \\chec
])
```

2. A recipe makes 12 cookies using 180g of flour. You have 480g of flour. How many cookies can you make? *(Set up an equation first. Don't just divide.)*

```
//| echo: false
makeStepperHTML(2, [
  { op: "Write as an equation",          eq: "\\frac{180}{12} = \\frac{480}{c}", note: "Flour per
  { op: "Simplify left side",             eq: "15 = \\frac{480}{c}" },
  { op: "Multiply both sides by c",       eq: "15c = 480" },
  { op: "Divide both sides by 15",        eq: "c = 32" },
  { op: "Check",                           eq: "\\frac{180}{12} = 15 \\text{ g/cookie},\\quad 15 \
])
```

3. Two cyclists start 120km apart and ride toward each other. One rides at 20km/h, the other at 25km/h. After how many hours do they meet?

```
//| echo: false
makeStepperHTML(3, [
  { op: "Write as an equation",           eq: "20t + 25t = 120", note: "Together they close 45 km ea
  { op: "Collect like terms",             eq: "45t = 120" },
  { op: "Divide both sides by 45",        eq: "t = \\frac{120}{45} = \\frac{8}{3} \\approx 2.67 \\te
  { op: "Check",                          eq: "20 \\times \\tfrac{8}{3} + 25 \\times \\tfrac{8}{3} =
])
```

When two quantities are changing in opposite directions and you want to know when they're equal, the unknown ends up on both sides of the equation. The fix is simple: collect all the terms with the unknown on one side, and all the plain numbers on the other.

For example, if one thing is shrinking and another is growing, you might get something like $500 - 35t = 200 + 15t$. Subtract $15t$ from both sides to collect the variable terms: $500 - 50t = 200$. Then subtract 200: $300 = 50t$. Divide by 50: $t = 6$. The same balance rule as before — it just takes one extra step to gather the unknowns first.

4. A water tank holds 800 litres and drains at 35 litres per minute. A second tank holds 200 litres and fills at 15 litres per minute. After how many minutes do they contain the same amount?

```
//| echo: false
makeStepperHTML(4, [
  { op: "Write as an equation",           eq: "800 - 35t = 200 + 15t", note: "Tank 1 decreases; tank
  { op: "Subtract 15t from both sides",   eq: "800 - 50t = 200" },
  { op: "Subtract 800 from both sides",   eq: "-50t = -600" },
  { op: "Divide both sides by −50",       eq: "t = 12" },
  { op: "Check",                         eq: "800 - 35 \\times 12 = 800 - 420 = 380;\\quad 200 + 15
])
```

5. You and two friends are splitting the cost of a group gift. The total cost is $96, but one friend already put in $18 as a deposit. How much does each of the three of you pay now?

```
//| echo: false
makeStepperHTML(5, [
  { op: "Write as an equation",           eq: "3p + 18 = 96", note: "Three people each pay p, plus t
  { op: "Subtract 18 from both sides",    eq: "3p = 78" },
  { op: "Divide both sides by 3",         eq: "p = 26" },
  { op: "Check",                         eq: "3 \\times 26 + 18 = 78 + 18 = 96 \\checkmark" }
])
```

# 12. Inequalities

The range of answers that still work

Your monthly budget for food is $120. There are dozens of ways to spend it — some that work, many that don't. The question isn't "what exactly do I spend?" It's "what combinations are even possible?"

A shelf is rated to hold 50 kg. The boxes you're stacking weigh 8 kg each. You don't need the exact weight — you need to know how many boxes you can put on before it becomes unsafe.

A password field requires between 8 and 20 characters. No single length is the answer. Any length in that range works.

These are all inequalities. An equation gives you one answer. An inequality gives you a whole region of answers — and the boundary that separates "works" from "doesn't work."

## 12.1. What the notation is saying

Start with an everyday rule: "your spending must stay below $120." That rule does not ask for one exact value. It allows many values and blocks others.

An equation uses = to say two things are exactly equal. An inequality uses a different symbol to say one thing is *bigger* or *smaller* than another.

This means an inequality describes a limit or range.

The four symbols:

| Symbol | Reads as | Example |
|---|---|---|
| $<$ | is less than | $3 < 7$ |
| $>$ | is greater than | $12 > 5$ |
| $\leq$ | is less than or equal to | $x \leq 10$ |
| $\geq$ | is greater than or equal to | $n \geq 4$ |

The pointy end of the symbol always faces the smaller value. So $3 < 7$ reads "3 is less than 7" — the small end points at 3, the open end opens toward 7. If you forget which way round it goes, just picture the symbol as an arrow squishing the smaller number.

The difference between $<$ and $\leq$ matters. If a shelf holds *at most* 50 kg, exactly 50 kg is fine — so you'd write $w \leq 50$. If something must be *strictly less than* 50, exactly 50 is not fine — so you'd write $w < 50$.

This means the boundary may or may not be included.

A **compound inequality** combines two inequalities into one statement. $8 \leq n \leq 20$ reads "n is at least 8 and at most 20" — both conditions must hold at once. This is the password length rule: any whole number from 8 to 20 satisfies it.

This means one value must satisfy both sides at the same time.

## 12.2. The method

Solving an inequality is almost identical to solving an equation. You do the same things: add, subtract, multiply, divide — keeping both sides balanced until the unknown is isolated.

This means most of the method is familiar from the last chapter.

**There is one exception. It's the only genuinely surprising thing in this chapter. It deserves its own section.**

### 12.2.1. The sign-flip rule

When you multiply or divide both sides of an inequality by a **negative number**, the direction of the inequality reverses.

Here's why, shown directly. Start with something you know is true:

$$3 < 5$$

Three is less than five. Yes. Now multiply both sides by $-1$:

$$-3 \quad \text{and} \quad -5$$

Which of these is smaller? $-5$ is. It's further left on the number line. So the relationship has flipped:

$$-3 > -5$$

The original said "3 is less than 5." After multiplying by $-1$, the true statement is "$-3$ is greater than $-5$."

This isn't a rule someone invented. It's what actually happens on the number line when you negate both sides. Multiplying by $-1$ reflects every number across zero — what was to the left of zero ends up to the right, and the ordering reverses.

**The sign-flip rule**
Multiplying or dividing both sides of an inequality by a *negative* number reverses the direction of the inequality sign.

$$3 < 5 \implies -3 > -5$$

If you forget to flip, your answer will point the wrong way — and every value in your "solution" will actually be wrong.

### 12.2.2. Solving a linear inequality, step by step

$$2x - 3 < 7$$

Add 3 to both sides (adding is safe — no sign flip):

$$2x < 10$$

Divide both sides by 2 (dividing by a *positive* number — no flip needed):

$$x < 5$$

The solution is every number less than 5. Not just one number: all of them.

This means the answer to an inequality is usually a set of numbers, not one number.

Now compare with one that triggers the flip:

$$-3x + 1 \geq 10$$

Subtract 1 from both sides:

$$-3x \geq 9$$

Divide both sides by $-3$. This is negative — flip the sign:

$$x \leq -3$$

The solution is every number less than or equal to $-3$. Check: try $x = -4$. Then $-3(-4)+1 = 12+1 = 13$, and $13 \geq 10$. Yes. Try $x = 0$. Then $-3(0) + 1 = 1$, and $1 \geq 10$. No — and $x = 0$ is not in our solution set, so that's correct.

This means checking a value inside and outside the solution helps you see whether the answer makes sense.

> **Why solving inequalities works**
> An inequality is a statement about the relationship between two quantities. Doing the same operation to both sides preserves that relationship — as long as the operation doesn't reverse the ordering. Addition and subtraction never reverse ordering. Multiplication and division by a *positive* number preserve ordering. Multiplication or division by a *negative* number reverses it. That's the whole story.

### 12.2.3. Reading the solution on a number line

A number line makes the solution visible. You mark the boundary value and shade the region that satisfies the inequality.

**Open circle** (∘) at the boundary: the boundary value is *not* included. Use this for strict inequalities ($<$ and $>$).

**Closed circle** (•) at the boundary: the boundary value *is* included. Use this for $\leq$ and $\geq$.

For $x < 5$: draw an open circle at 5, shade to the left.

For $x \leq 5$: draw a closed circle at 5, shade to the left.

For $x > -3$: draw an open circle at $-3$, shade to the right.

For a compound inequality like $-2 < x \leq 4$: open circle at $-2$, closed circle at 4, shade the segment between them.

**Interactive: Number line inequality explorer.** Use the slider to set the boundary value. Choose the inequality direction and whether the boundary is included. The shaded region and circle type update live.

```
//| echo: false
viewof nlBoundary = Inputs.range([-5, 20], {
  label: "Boundary value",
  step: 0.5,
  value: 5
})
```

```
//| echo: false
viewof nlSymbol = Inputs.select(["<", "⩽", ">", "⩾"], {
  label: "Inequality symbol  (x  __ boundary)",
  value: "<"
})
```

```
//| echo: false
{
  const boundary = nlBoundary;
  const sym = nlSymbol;
  const inclusive = sym === "⩽" || sym === "⩾";
  const shadedLeft = sym === "<" || sym === "⩽";
```

```
const W = 560, H = 110;
const PAD = 40;
const AXIS_Y = 60;
const TICK_H = 8;
const domainMin = -5, domainMax = 20;
const scale = (v) => PAD + (v - domainMin) / (domainMax - domainMin) * (W - 2 * PAD);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", `max-width:${W}px; font-family:inherit;`);

// Axis line
svg.append("line")
  .attr("x1", PAD).attr("y1", AXIS_Y)
  .attr("x2", W - PAD).attr("y2", AXIS_Y)
  .attr("stroke", "#9ca3af").attr("stroke-width", 2);

// Arrow heads
svg.append("polygon")
  .attr("points", `${W - PAD},${AXIS_Y} ${W - PAD - 8},${AXIS_Y - 4} ${W - PAD - 8},${AXIS_Y +
  .attr("fill", "#9ca3af");
svg.append("polygon")
  .attr("points", `${PAD},${AXIS_Y} ${PAD + 8},${AXIS_Y - 4} ${PAD + 8},${AXIS_Y + 4}`)
  .attr("fill", "#9ca3af");

// Tick marks and labels for integers in domain
for (let v = domainMin; v <= domainMax; v += 5) {
  const x = scale(v);
  svg.append("line")
    .attr("x1", x).attr("y1", AXIS_Y - TICK_H / 2)
    .attr("x2", x).attr("y2", AXIS_Y + TICK_H / 2)
    .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
  svg.append("text")
    .attr("x", x).attr("y", AXIS_Y + 22)
    .attr("text-anchor", "middle")
    .attr("font-size", "11px").attr("fill", "#6b7280")
    .text(v);
}

// Shaded region
const bx = scale(boundary);
const shadeColor = "#0d9488";
if (shadedLeft) {
  svg.append("rect")
    .attr("x", PAD).attr("y", AXIS_Y - 6)
    .attr("width", Math.max(0, bx - PAD)).attr("height", 12)
```

```
      .attr("fill", shadeColor).attr("opacity", 0.25);
    // Arrow showing infinite extent to the left
    svg.append("polygon")
      .attr("points", `${PAD},${AXIS_Y} ${PAD + 8},${AXIS_Y - 4} ${PAD + 8},${AXIS_Y + 4}`)
      .attr("fill", shadeColor).attr("opacity", 0.6);
  } else {
    svg.append("rect")
      .attr("x", bx).attr("y", AXIS_Y - 6)
      .attr("width", Math.max(0, W - PAD - bx)).attr("height", 12)
      .attr("fill", shadeColor).attr("opacity", 0.25);
    // Arrow showing infinite extent to the right
    svg.append("polygon")
      .attr("points", `${W - PAD},${AXIS_Y} ${W - PAD - 8},${AXIS_Y - 4} ${W - PAD - 8},${AXIS_Y + 4
      .attr("fill", shadeColor).attr("opacity", 0.6);
  }

  // Boundary circle
  if (inclusive) {
    svg.append("circle")
      .attr("cx", bx).attr("cy", AXIS_Y)
      .attr("r", 7).attr("fill", shadeColor).attr("stroke", shadeColor).attr("stroke-width", 2);
  } else {
    svg.append("circle")
      .attr("cx", bx).attr("cy", AXIS_Y)
      .attr("r", 7).attr("fill", "#fff").attr("stroke", shadeColor).attr("stroke-width", 2.5);
  }

  // Boundary label
  svg.append("text")
    .attr("x", bx).attr("y", AXIS_Y - 18)
    .attr("text-anchor", "middle")
    .attr("font-size", "12px").attr("fill", shadeColor).attr("font-weight", "600")
    .text(boundary % 1 === 0 ? boundary : boundary.toFixed(1));

  // Inequality statement
  const circleDesc = inclusive ? "closed circle (boundary included)" : "open circle (boundary exclu
  svg.append("text")
    .attr("x", W / 2).attr("y", H - 6)
    .attr("text-anchor", "middle")
    .attr("font-size", "11px").attr("fill", "#6b7280")
    .text(`x ${sym} ${boundary % 1 === 0 ? boundary : boundary.toFixed(1)}   —   ${circleDesc}`);

  return svg.node();
}
```

## 12.3. Worked examples

**Example 1 — Budgeting.** You have $120 to spend on food this month. You've already spent $47. You want to buy some items that cost $8 each. How many can you buy without going over budget?

Let $n$ = number of items.

$$47 + 8n \leq 120$$

Subtract 47 from both sides:

$$8n \leq 73$$

Divide both sides by 8 (positive — no flip):

$$n \leq 9.125$$

Since $n$ must be a whole number, the answer is $n \leq 9$. You can buy at most 9 items.

Check: $47 + 8 \times 9 = 47 + 72 = 119$. Within budget. And $47 + 8 \times 10 = 127$. Over. Correct.

This means real-world inequalities often need one more step: interpreting the algebra in whole numbers.

---

**Example 2 — Shelf load.** A wall-mounted shelf is rated to hold at most 50 kg. Each box you place on it weighs 8 kg.

How many boxes can the shelf safely hold?

The shelf load rating tells you how much weight the shelf can carry on top of it. The brackets that mount the shelf to the wall bear their own weight separately — they don't count against the shelf's carrying capacity.

Let $b$ = number of boxes.

$$8b \leq 50$$

$$b \leq 6.25$$

Since $b$ must be a whole number: at most 6 boxes.

Check: $8 \times 6 = 48 \leq 50$. ☒ Seven boxes: $8 \times 7 = 56 > 50$. ☒

This means safety limits are naturally written as inequalities.

---

**Example 3 — Password validation.** A website requires passwords to be at least 8 characters long but no longer than 20 characters. A user types a password. The code checks whether its length $n$ is acceptable.

The valid range is:

$$8 \leq n \leq 20$$

Any value of $n$ satisfying both conditions is acceptable. $n = 7$? Too short. $n = 8$? Fine — the lower boundary is included. $n = 20$? Fine. $n = 21$? Too long.

This is a compound inequality. In code, you'd check both conditions at once: `8 <= n && n <= 20` (or the equivalent in whatever language you're using). The mathematics and the code are the same structure.

This means inequalities are a direct way to describe allowed ranges.

---

**Example 4 — An inequality with a negative coefficient.** The temperature in a storage room must be kept strictly below 5°C. The thermostat setting $t$ (in °C) satisfies:

$$-2t + 14 > 4$$

Subtract 14 from both sides:

$$-2t > -10$$

Divide by $-2$ — negative, so flip:

$$t < 5$$

The thermostat must be set below 5°C. The sign flip was necessary here: if you forgot it and wrote $t > 5$, you'd be saying temperatures *above* 5°C are acceptable, which is exactly wrong.

This means the sign-flip rule is not a small detail. It changes the meaning of the answer completely.

**Interactive: Inequality explorer.** Set the coefficients $a$ and $b$, choose the constant $c$ and the inequality direction. The solution is shown algebraically and on a number line. The sign-flip warning lights up when $a$ is negative.

```
//| echo: false
viewof ieA = Inputs.range([-5, 5], { label: "a  (coefficient of x)", step: 1, value: 2 })
```

```
//| echo: false
viewof ieB = Inputs.range([-10, 10], { label: "b  (constant, left side)", step: 1, value: -3 })
```

```
//| echo: false
viewof ieC = Inputs.range([-20, 20], { label: "c  (right-hand side)", step: 1, value: 7 })
```

```
//| echo: false
viewof ieSym = Inputs.select(["<", "≤", ">", "≥"], { label: "Inequality symbol", value: "<" })
```

```
//| echo: false
{
  const a = Math.round(ieA), b = Math.round(ieB), c = Math.round(ieC), sym = ieSym;

  // Solve:  ax + b  sym  c   →   ax  sym  c - b   →   x  sym'  (c-b)/a
  // If a < 0, sym flips.
  const rhs = c - b;
  const needsFlip = a < 0;
  const flipped = { "<": ">", "≤": "≥", ">": "<", "≥": "≤" };
  const solvedSym = needsFlip ? flipped[sym] : sym;

  const W = 560, H = 200;
  const PAD = 40;
  const AXIS_Y = 150;
  const TICK_H = 8;

  const leftStr = a === 1 ? `x + ${b}` : a === -1 ? `-x + ${b}` : `${a}x + ${b}`;
  const lhsClean = b >= 0
    ? (a === 1 ? `x + ${b}` : a === -1 ? `-x + ${b}` : `${a}x + ${b}`)
    : (a === 1 ? `x - ${Math.abs(b)}` : a === -1 ? `-x - ${Math.abs(b)}` : `${a}x - ${Math.abs(b)

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width", "100%")
    .attr("style", `max-width:${W}px; font-family:inherit;`);

  // Step labels
  const steps = [
    `Original:  ${lhsClean}  ${sym}  ${c}`,
    `Subtract ${b} from both sides:  ${a === 1 ? "x" : a === -1 ? "-x" : `${a}x`}  ${sym}  ${rhs}
    a === 0
      ? (rhs > 0 && (sym === "<" || sym === "≤") ? "No solution (0x = 0, always)" : "All real num
      : needsFlip
        ? `Divide by ${a}  (negative — flip sign):  x  ${solvedSym}  ${rhs / a}`
        : `Divide by ${a}:  x  ${solvedSym}  ${rhs / a}`
  ];

  steps.forEach((txt, i) => {
    const isFlipStep = i === 2 && needsFlip;
    svg.append("text")
      .attr("x", 16).attr("y", 24 + i * 28)
```

```
      .attr("font-size", "12px")
      .attr("fill", isFlipStep ? "#b45309" : i === steps.length - 1 ? "#0d9488" : "#374151")
      .attr("font-weight", isFlipStep || i === steps.length - 1 ? "600" : "400")
      .text(txt);
  });

  if (needsFlip) {
    svg.append("text")
      .attr("x", 16).attr("y", 24 + steps.length * 28)
      .attr("font-size", "11px").attr("fill", "#b45309")
      .text("⚠ Sign flip applied — dividing by a negative number reverses the inequality.");
  }

  if (a !== 0) {
    const solution = rhs / a;
    const domainMin = Math.floor(solution) - 8;
    const domainMax = Math.ceil(solution) + 8;
    const scale = (v) => PAD + (v - domainMin) / (domainMax - domainMin) * (W - 2 * PAD);
    const bx = scale(solution);
    const shadedLeft = solvedSym === "<" || solvedSym === "≤";
    const inclusive = solvedSym === "≤" || solvedSym === "≥";
    const shadeColor = "#0d9488";

    // Axis
    svg.append("line")
      .attr("x1", PAD).attr("y1", AXIS_Y)
      .attr("x2", W - PAD).attr("y2", AXIS_Y)
      .attr("stroke", "#9ca3af").attr("stroke-width", 2);

    // Arrows
    svg.append("polygon")
      .attr("points", `${W - PAD},${AXIS_Y} ${W - PAD - 8},${AXIS_Y - 4} ${W - PAD - 8},${AXIS_Y + 4
      .attr("fill", "#9ca3af");
    svg.append("polygon")
      .attr("points", `${PAD},${AXIS_Y} ${PAD + 8},${AXIS_Y - 4} ${PAD + 8},${AXIS_Y + 4}`)
      .attr("fill", "#9ca3af");

    // Ticks
    for (let v = Math.ceil(domainMin); v <= Math.floor(domainMax); v += Math.ceil((domainMax - domai
      const x = scale(v);
      svg.append("line")
        .attr("x1", x).attr("y1", AXIS_Y - TICK_H / 2)
        .attr("x2", x).attr("y2", AXIS_Y + TICK_H / 2)
        .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
      svg.append("text")
        .attr("x", x).attr("y", AXIS_Y + 20)
        .attr("text-anchor", "middle")
```

```
        .attr("font-size", "10px").attr("fill", "#6b7280")
        .text(v);
    }

    // Shade
    if (shadedLeft) {
      svg.append("rect")
        .attr("x", PAD).attr("y", AXIS_Y - 5)
        .attr("width", Math.max(0, bx - PAD)).attr("height", 10)
        .attr("fill", shadeColor).attr("opacity", 0.25);
    } else {
      svg.append("rect")
        .attr("x", bx).attr("y", AXIS_Y - 5)
        .attr("width", Math.max(0, W - PAD - bx)).attr("height", 10)
        .attr("fill", shadeColor).attr("opacity", 0.25);
    }

    // Boundary circle
    if (inclusive) {
      svg.append("circle")
        .attr("cx", bx).attr("cy", AXIS_Y)
        .attr("r", 6).attr("fill", shadeColor).attr("stroke", shadeColor).attr("stroke-width", 2)
    } else {
      svg.append("circle")
        .attr("cx", bx).attr("cy", AXIS_Y)
        .attr("r", 6).attr("fill", "#fff").attr("stroke", shadeColor).attr("stroke-width", 2.5);
    }

    svg.append("text")
      .attr("x", bx).attr("y", AXIS_Y - 14)
      .attr("text-anchor", "middle")
      .attr("font-size", "11px").attr("fill", shadeColor).attr("font-weight", "600")
      .text(Number.isInteger(solution) ? solution : solution.toFixed(2));
  }

  return svg.node();
}
```

## 12.4. Where this goes

This chapter takes the solving method from equations and turns it into a method for handling limits and allowed ranges.

An equation pins down one value. An inequality describes a boundary — everything on one side is feasible, everything on the other isn't.

The most direct continuation is **linear programming** — how to find the best point within a region defined by multiple inequalities at once. That's in the Optimisation chapter in Volume 7. A manufacturer choosing how much of two products to make, subject to labour and material constraints, is solving a linear programme. The feasible region is defined entirely by inequalities like the ones in this chapter.

The second direction is **systems of inequalities** and their role in applied linear algebra. When a machine learning model classifies data, it's often drawing a boundary in a high-dimensional space — one side satisfies the classification condition, the other doesn't. That boundary is expressed as an inequality. The linear equations you learned in the previous chapter define the boundary; the inequality says which side is which. That thread runs from here through linear algebra (Volume 4) all the way to optimisation and machine learning.

**Where this shows up**

- A financial analyst building a budget model sets upper and lower bounds on each line of spending — those bounds are inequalities.
- A structural engineer checks that the load on every beam stays within its rated capacity — an inequality checked against the design at every point.
- A software developer writing input validation checks that every user-entered value lies in an acceptable range — a compound inequality executed in code.
- A supply chain planner ensures stock levels stay between a minimum (to avoid running out) and a maximum (to avoid overflow) — two inequalities, applied simultaneously.

## 12.5. Exercises

Each problem has a clean answer. The work is in translating the situation into the notation before you solve it.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
```

```
    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;
    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    row.appendChild(opCell);
    row.appendChild(eqCell);
    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }
    stepsDiv.appendChild(row);
  }
  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "□ Solution complete";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

function updateButtons() {
  nextBtn.disabled = current === totalSteps;
  nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
  counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
}

nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
controls.appendChild(nextBtn);
controls.appendChild(resetBtn);
controls.appendChild(counter);
container.appendChild(stepsDiv);
container.appendChild(controls);
```

```
  renderSteps();
  updateButtons();
  return container;
}
```

1. You can spend at most $50 per month on streaming services. You already pay $14.97 for two services. How many additional services at $9.99 per month can you add and still stay within your budget?

```
//| echo: false
makeStepperHTML(1, [
  { op: "Write the inequality",            eq: "14.97 + 9.99s \\leq 50" },
  { op: "Subtract 14.97 from both sides",  eq: "9.99s \\leq 35.03" },
  { op: "Divide both sides by 9.99",       eq: "s \\leq 3.506\\ldots" },
  { op: "Round down (whole services only)", eq: "s \\leq 3" },
  { op: "Check",                           eq: "14.97 + 9.99 \\times 3 = 14.97 + 29.97 = 44.94 \\le
])
```

2. A lift (elevator) has a maximum capacity of 630 kg. A group of adults with an average weight of 78 kg wants to use it. What is the largest number of people who can ride together safely?

```
//| echo: false
makeStepperHTML(2, [
  { op: "Write the inequality",          eq: "78p \\leq 630" },
  { op: "Divide both sides by 78",       eq: "p \\leq 8.076\\ldots" },
  { op: "Round down (whole people only)", eq: "p \\leq 8" },
  { op: "Check",                          eq: "78 \\times 8 = 624 \\leq 630 \\checkmark;\\quad 78 \\
])
```

3. Solve for $x$ and mark the solution on a number line:

$$-4x + 5 > 17$$

Explain in one sentence what happens to the inequality sign and why.

```
//| echo: false
makeStepperHTML(3, [
  { op: "Write the inequality",           eq: "-4x + 5 > 17" },
  { op: "Subtract 5 from both sides",     eq: "-4x > 12" },
  { op: "Divide by −4  (flip sign)",      eq: "x < -3", note: "Dividing by a negative number reverse
  { op: "Check: try x = −4",             eq: "-4(-4) + 5 = 16 + 5 = 21 > 17 \\checkmark" },
  { op: "Check: try x = −2 (not in solution)", eq: "-4(-2) + 5 = 8 + 5 = 13 \\not> 17 \\checkmark" }
])
```

4. A password must be at least 8 characters. It must also be fewer than 3 times the minimum length. Write this as a compound inequality for the password length $n$, then state the range of valid lengths.

```
//| echo: false
makeStepperHTML(4, [
  { op: "Write the lower bound",          eq: "n \\geq 8" },
  { op: "Write the upper bound",          eq: "n < 3 \\times 8 = 24" },
  { op: "Combine as compound inequality", eq: "8 \\leq n < 24" },
  { op: "State the valid lengths",        eq: "n \\in \\{8, 9, 10, \\ldots, 23\\}", note: "23 is
  { op: "Check boundaries",              eq: "n=8: \\geq 8 \\checkmark,\\; n=23: < 24 \\checkmark
])
```

5. Solve the compound inequality and state which whole numbers satisfy it:

$$1 < 3x - 2 \leq 13$$

```
//| echo: false
makeStepperHTML(5, [
  { op: "Write the compound inequality",  eq: "1 < 3x - 2 \\leq 13" },
  { op: "Add 2 to all three parts",       eq: "3 < 3x \\leq 15", note: "Adding the same amount to
  { op: "Divide all three parts by 3",    eq: "1 < x \\leq 5" },
  { op: "State whole-number solutions",   eq: "x \\in \\{2, 3, 4, 5\\}", note: "x must be strictl
  { op: "Check endpoints",               eq: "x=2:\\; 3(2)-2=4,\\; 1 < 4 \\leq 13 \\checkmark;\\q
])
```

6. A phone plan charges $0.08 per minute after a 200-minute monthly allowance. You want your total bill to be under $25. The base plan costs $12. How many additional minutes can you use?

```
//| echo: false
makeStepperHTML(6, [
  { op: "Write the inequality",           eq: "12 + 0.08m < 25" },
  { op: "Subtract 12 from both sides",    eq: "0.08m < 13" },
  { op: "Divide both sides by 0.08",      eq: "m < 162.5" },
  { op: "State the answer",               eq: "m \\leq 162 \\text{ additional minutes}", note: "S
  { op: "Check",                          eq: "12 + 0.08 \\times 162 = 12 + 12.96 = 24.96 < 25 \\
])
```

7. A company's safety guideline says a storage shelf should never hold more than 75% of its rated maximum. The shelf is rated to 80 kg. Boxes weigh 6 kg each. What is the maximum number of boxes you can store on this shelf within the guideline?

```
//| echo: false
makeStepperHTML(7, [
  { op: "Find the safe weight limit",     eq: "0.75 \\times 80 = 60 \\text{ kg}" },
  { op: "Write the inequality",           eq: "6b \\leq 60" },
  { op: "Divide both sides by 6",         eq: "b \\leq 10" },
  { op: "Check",                          eq: "6 \\times 10 = 60 \\leq 60 \\checkmark;\\quad 6 \\
])
```

# 13. Patterns and sequences

When the next step follows a rule

You decide to save $20 a week. After one week: $20. After two weeks: $40. After ten weeks: $200. You didn't need a formula for that — you just kept adding twenty.

A chain message says: send this to three friends. Each of those three sends it to three more. After one round: 3 people. After two rounds: 9 people. After three rounds: 27 people. That one grows differently. Faster. Worryingly faster.

Both of those are sequences. The first adds the same amount each time. The second multiplies by the same amount each time. Knowing which kind you're looking at — and being able to jump straight to any term without listing all the ones before it — is what this chapter is about.

## 13.1. What makes a sequence a sequence

A sequence is a list of numbers in a definite order, where each position has exactly one value. Position 1 has a value. Position 2 has a value. Position $n$ has a value. The order matters — you can't shuffle the terms and call it the same sequence.

This means a sequence is not just a pile of numbers. The position is part of the meaning.

What makes a sequence *useful* is when there's a pattern: a rule that connects each term to the one before it, or better yet, a rule that connects any term directly to its position number.

That second kind — where you can calculate any term from its position alone — is a function. You put a position in, you get a value out. That idea is one of the most important in all of mathematics, and sequences are where it shows up first in a concrete, tangible way.

This means sequences are our first clear example of a rule that maps inputs to outputs.

For now: spot the pattern, name the rule, find any term.

## 13.2. What the notation is saying

We write the terms of a sequence as:

$$a_1, a_2, a_3, \dots, a_n$$

Read that as: "a-one, a-two, a-three, up to a-n." The letter $a$ names the sequence; the subscript number is the position. So $a_1$ — read "a-one" — is the first term, and $a_n$ — read "a-n" — is the term at position $n$, wherever that happens to be.

This means the subscript tells you where the term sits in the list.

Two sequences, two different rules:

- **Arithmetic sequences** add the same amount each step. That fixed amount is called the **common difference**, written $d$.
- **Geometric sequences** multiply by the same amount each step. That fixed multiplier is called the **common ratio**, written $r$.

This means the quickest way to classify a sequence is to ask: "am I adding the same amount, or multiplying by the same amount?"

## 13.3. Arithmetic sequences

### 13.3.1. Recognising one

Look at the gaps between consecutive terms. If the gap is always the same, it's arithmetic.

$$3,\ 7,\ 11,\ 15,\ 19, \dots$$

The gap is 4 every time. So $d = 4$ and this is arithmetic.

$$100,\ 94,\ 88,\ 82, \dots$$

The gap is $-6$ every time. So $d = -6$. A decreasing sequence still counts — the difference is just negative.

### 13.3.2. The nth term formula

Listing terms one by one works fine if you only want the first few. But what if you want the 50th term? You don't want to list 50 items.

Notice what's happening with each term:

| Position | Term | Working |
|---|---|---|
| 1 | $a_1$ | just the starting value |
| 2 | $a_1 + d$ | added the difference once |
| 3 | $a_1 + 2d$ | added the difference twice |
| 4 | $a_1 + 3d$ | added the difference three times |
| $n$ | $a_1 + (n-1)d$ | added the difference $n-1$ times |

The pattern: to get to position $n$ from position 1, you add $d$ exactly $(n-1)$ times. So:

$$a_n = a_1 + (n-1)d$$

Read that aloud: "the nth term equals the first term, plus n minus one lots of the common difference."

The $(n-1)$ looks odd at first. Here's why it isn't: at position 1 you add the difference *zero* times, at position 2 you add it *once*, at position $n$ you add it $n-1$ times. The count of additions is always one less than the position.

Here, $a_1$ is the first term, $d$ is the common difference, and $n$ is the position number. This means the formula jumps straight to any term without listing the earlier ones.

**Interactive: Arithmetic sequence builder.** Set the first term and common difference. The dot plot shows the first 10 terms — notice the points fall on a straight line.

```
//| echo: false
viewof arithA1 = Inputs.range([1, 20], { label: "First term a₁", step: 1, value: 3 })
```

```
//| echo: false
viewof arithD = Inputs.range([-5, 10], { label: "Common difference d", step: 1, value: 4 })
```

```
//| echo: false
{
  const a1 = Math.round(arithA1);
  const d  = Math.round(arithD);
  const N  = 10;
  const terms = Array.from({ length: N }, (_, i) => ({ n: i + 1, v: a1 + i * d }));

  const W = 560, H = 280;
  const PAD_L = 60, PAD_R = 30, PAD_T = 40, PAD_B = 50;
  const plotW = W - PAD_L - PAD_R;
  const plotH = H - PAD_T - PAD_B;

  const vals = terms.map(t => t.v);
  const vMin = Math.min(...vals);
  const vMax = Math.max(...vals);
  const vRange = vMax === vMin ? 1 : vMax - vMin;
  const vLo = vMin - vRange * 0.15;
  const vHi = vMax + vRange * 0.15;
```

```
const xScale = (n) => PAD_L + (n - 1) / (N - 1) * plotW;
const yScale = (v) => PAD_T + plotH - (v - vLo) / (vHi - vLo) * plotH;

const d3local = await require("d3@7");
const svg = d3local.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", `max-width:${W}px; font-family:inherit;`);

// Gridlines
for (let i = 1; i <= N; i++) {
  svg.append("line")
    .attr("x1", xScale(i)).attr("y1", PAD_T)
    .attr("x2", xScale(i)).attr("y2", PAD_T + plotH)
    .attr("stroke", "#f3f4f6").attr("stroke-width", 1);
}

// Axes
svg.append("line")
  .attr("x1", PAD_L).attr("y1", PAD_T + plotH)
  .attr("x2", PAD_L + plotW).attr("y2", PAD_T + plotH)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
svg.append("line")
  .attr("x1", PAD_L).attr("y1", PAD_T)
  .attr("x2", PAD_L).attr("y2", PAD_T + plotH)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);

// x-axis labels
terms.forEach(t => {
  svg.append("text")
    .attr("x", xScale(t.n)).attr("y", PAD_T + plotH + 18)
    .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#6b7280")
    .text(t.n);
});
svg.append("text")
  .attr("x", PAD_L + plotW / 2).attr("y", H - 6)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#6b7280")
  .text("Position (n)");

// y-axis label
svg.append("text")
  .attr("transform", `translate(14, ${PAD_T + plotH / 2}) rotate(-90)`)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#6b7280")
  .text("Value (a⃗)");

// Line connecting dots
const lineGen = d3local.line().x(t => xScale(t.n)).y(t => yScale(t.v));
```

```
  svg.append("path")
    .datum(terms)
    .attr("d", lineGen)
    .attr("fill", "none").attr("stroke", "#0d9488").attr("stroke-width", 1.5)
    .attr("stroke-dasharray", "4 3").attr("opacity", 0.5);

  // Dots + value labels
  terms.forEach(t => {
    svg.append("circle")
      .attr("cx", xScale(t.n)).attr("cy", yScale(t.v))
      .attr("r", 5).attr("fill", "#0d9488").attr("stroke", "#fff").attr("stroke-width", 1.5);
    svg.append("text")
      .attr("x", xScale(t.n)).attr("y", yScale(t.v) - 10)
      .attr("text-anchor", "middle").attr("font-size", "10px").attr("fill", "#374151")
      .text(t.v);
  });

  // Formula label
  svg.append("text")
    .attr("x", PAD_L + plotW - 4).attr("y", PAD_T + 14)
    .attr("text-anchor", "end").attr("font-size", "12px").attr("fill", "#0d9488").attr("font-weig
    .text(`a₁ = ${a1},  d = ${d}`);

  return svg.node();
}
```

## 13.4. Worked examples — arithmetic

**Example 1 (sci).** Bacterial measurements are taken every 4 hours starting at hour 0. The recorded counts (in thousands) are: 12, 15, 18, 21, ...

The question is: what is the count at hour 36?

First, work out which measurement number that is. Hour 0 is measurement 1, hour 4 is measurement 2, and so on — each measurement is 4 hours after the previous one. So hour 36 is measurement number $\frac{36}{4} + 1 = 10$.

Now check whether the sequence is arithmetic: $15 - 12 = 3$, $18 - 15 = 3$, $21 - 18 = 3$. Yes, $d = 3$.

$$a_{10} = 12 + (10 - 1) \times 3 = 12 + 27 = 39$$

The count at hour 36 is 39 thousand. (This is arithmetic growth — constant addition. Real bacterial growth is geometric, but the measuring interval can create arithmetic-looking data over short spans.)

This means the first step in many sequence problems is converting the real-world situation into a position number.

---

**Example 2 (biz).** A gamer earns 150 points for completing level 1. Each subsequent level awards 25 more points than the previous one. How many points does level 18 award?

Here $a_1 = 150$ and $d = 25$.

$$a_{18} = 150 + (18 - 1) \times 25 = 150 + 425 = 575$$

Level 18 awards 575 points.

This means the common difference tells us how much each new level adds.

---

**Example 3 (comp).** A programmer is adding rows to a triangle pattern: row 1 has 1 tile, row 2 has 3 tiles, row 3 has 5 tiles.

The sequence is 1, 3, 5, 7, ... with $a_1 = 1$ and $d = 2$.

How many tiles does row 20 need?

$$a_{20} = 1 + (20 - 1) \times 2 = 1 + 38 = 39$$

Row 20 needs 39 tiles. A programmer building this pattern in a loop would use exactly this calculation to allocate memory or set loop bounds.

This means arithmetic sequences often appear when a pattern grows by equal steps.

---

**Example 4 (eng).** A steel cable loses 3mm of useful length each time it's re-terminated. It starts at 500mm. After how many re-terminations is it shorter than 450mm?

The sequence is $500, 497, 494, \ldots$ with $a_1 = 500$ and $d = -3$.

We want $a_n < 450$:

$$\begin{gathered} 500 + (n - 1)(-3) < 450 \\ -(n - 1) \times 3 < -50 \\ (n - 1) \times 3 > 50 \\ n - 1 > 16.67 \\ n > 17.67 \end{gathered}$$

So after the 18th re-termination the cable is below 450mm. Check: $a_{18} = 500 + 17 \times (-3) = 500 - 51 = 449$. Yes.

This means a sequence formula can also be used inside an inequality when we want to know when a threshold is crossed.

## 13.5. Geometric sequences

### 13.5.1. Recognising one

Look at the *ratios* between consecutive terms instead of the gaps. If the ratio is always the same, it's geometric.

$$2,\ 6,\ 18,\ 54, \ldots$$

Divide each term by the previous one: $6 \div 2 = 3$, $18 \div 6 = 3$, $54 \div 18 = 3$. The ratio is always 3, so $r = 3$.

$$80,\ 40,\ 20,\ 10, \ldots$$

Each term is half the previous one: $r = 0.5$. A shrinking sequence — the ratio just has to be positive and consistent.

### 13.5.2. The nth term formula

The same logic as arithmetic, but with multiplication:

| Position | Term | Working |
|---|---|---|
| 1 | $a_1$ | starting value |
| 2 | $a_1 \times r$ | multiplied once |
| 3 | $a_1 \times r^2$ | multiplied twice |
| 4 | $a_1 \times r^3$ | multiplied three times |
| $n$ | $a_1 \times r^{n-1}$ | multiplied $n - 1$ times |

So:

$$a_n = a_1 \times r^{n-1}$$

The $(n - 1)$ is there for the same reason as in arithmetic: at position 1 you multiply by $r$ zero times. Any number raised to the power zero equals 1 — so $r^0 = 1$, and the first term is just $a_1 \times 1 = a_1$. Each step up adds one more multiplication.

Here, $r$ is the common ratio. This means geometric sequences grow or shrink by repeated scaling.

**Connection to Vol 1 Ch 3:** if this looks like the compound growth formula, that's because it is. Compound interest is a geometric sequence. Each year's balance is the first year's amount multiplied by the growth factor (1 + rate), raised to the number of years. The structure is identical.

**Interactive: Sequence explorer.** Toggle between arithmetic and geometric. Adjust the first term and the common difference or ratio. The dot plot makes the shape of growth immediately visible — linear versus curved.

```
//| echo: false
viewof seqType = Inputs.toggle({ label: "Show geometric sequence (off = arithmetic)", value: false
```

```
//| echo: false
viewof seqA1 = Inputs.range([1, 100], { label: "First term a₁", step: 1, value: 5 })
```

```
//| echo: false
viewof seqParam = Inputs.range(seqType ? [0.5, 3.0] : [1, 10], {
  label: seqType ? "Common ratio r" : "Common difference d",
  step: seqType ? 0.1 : 1,
  value: seqType ? 1.8 : 4
})
```

```
//| echo: false
{
  const isGeo = seqType;
  const a1 = Math.round(seqA1);
  const param = seqParam;
  const N = 10;

  const terms = Array.from({ length: N }, (_, i) => ({
    n: i + 1,
    v: isGeo ? a1 * Math.pow(param, i) : a1 + i * param
  }));

  const W = 560, H = 280;
  const PAD_L = 65, PAD_R = 30, PAD_T = 40, PAD_B = 50;
  const plotW = W - PAD_L - PAD_R;
  const plotH = H - PAD_T - PAD_B;

  const vals = terms.map(t => t.v);
  const vMin = Math.min(...vals);
  const vMax = Math.max(...vals);
  const vRange = vMax === vMin ? 1 : vMax - vMin;
  const vLo = Math.min(0, vMin - vRange * 0.1);
  const vHi = vMax + vRange * 0.15;

  const xScale = (n) => PAD_L + (n - 1) / (N - 1) * plotW;
  const yScale = (v) => PAD_T + plotH - (v - vLo) / (vHi - vLo) * plotH;

  const d3local = await require("d3@7");
  const svg = d3local.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width", "100%")
    .attr("style", `max-width:${W}px; font-family:inherit;`);

  const colour = isGeo ? "#7c3aed" : "#0d9488";
```

```
const typeLabel = isGeo
  ? `Geometric: a₁ = ${a1}, r = ${param.toFixed(1)}`
  : `Arithmetic: a₁ = ${a1}, d = ${param}`;

// Gridlines
for (let i = 1; i <= N; i++) {
  svg.append("line")
    .attr("x1", xScale(i)).attr("y1", PAD_T)
    .attr("x2", xScale(i)).attr("y2", PAD_T + plotH)
    .attr("stroke", "#f3f4f6").attr("stroke-width", 1);
}

// Axes
svg.append("line")
  .attr("x1", PAD_L).attr("y1", PAD_T + plotH)
  .attr("x2", PAD_L + plotW).attr("y2", PAD_T + plotH)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
svg.append("line")
  .attr("x1", PAD_L).attr("y1", PAD_T)
  .attr("x2", PAD_L).attr("y2", PAD_T + plotH)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);

// x-axis labels
terms.forEach(t => {
  svg.append("text")
    .attr("x", xScale(t.n)).attr("y", PAD_T + plotH + 18)
    .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#6b7280")
    .text(t.n);
});
svg.append("text")
  .attr("x", PAD_L + plotW / 2).attr("y", H - 6)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#6b7280")
  .text("Position (n)");

// y-axis label
svg.append("text")
  .attr("transform", `translate(14, ${PAD_T + plotH / 2}) rotate(-90)`)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#6b7280")
  .text("Value (a□)");

// Line
const lineGen = d3local.line().x(t => xScale(t.n)).y(t => yScale(t.v));
svg.append("path")
  .datum(terms)
  .attr("d", lineGen)
  .attr("fill", "none").attr("stroke", colour).attr("stroke-width", 1.5)
  .attr("stroke-dasharray", isGeo ? "none" : "4 3").attr("opacity", 0.5);
```

```
  // Dots
  terms.forEach(t => {
    const cy = yScale(t.v);
    svg.append("circle")
      .attr("cx", xScale(t.n)).attr("cy", cy)
      .attr("r", 5).attr("fill", colour).attr("stroke", "#fff").attr("stroke-width", 1.5);
    const labelVal = t.v >= 1000 ? `${(t.v / 1000).toFixed(1)}k` :
                     t.v >= 10 ? Math.round(t.v) : t.v.toFixed(1);
    svg.append("text")
      .attr("x", xScale(t.n)).attr("y", cy - 10)
      .attr("text-anchor", "middle").attr("font-size", "9px").attr("fill", "#374151")
      .text(labelVal);
  });

  // Type label
  svg.append("text")
    .attr("x", PAD_L + 4).attr("y", PAD_T + 14)
    .attr("font-size", "12px").attr("fill", colour).attr("font-weight", "600")
    .text(typeLabel);

  return svg.node();
}
```

## 13.6. Worked examples — geometric

**Example 5 (biz).** A video gets 200 views on day 1. Each day it gets three times as many views as the day before. How many views on day 6?

Here $a_1 = 200$ and $r = 3$.

$$a_6 = 200 \times 3^{6-1} = 200 \times 3^5 = 200 \times 243 = 48{,}600$$

48,600 views on day 6. That's the shape of viral growth — it looks manageable for the first few steps, then becomes enormous.

This means repeated multiplication grows much faster than repeated addition.

---

**Example 6 (sci).** A radioactive sample has 640 micrograms at the start. Each hour, it retains half its mass. How much remains after 5 hours?

Here $a_1 = 640$ and $r = 0.5$. Term 1 is at $t = 0$ (the start), so after 5 hours we're looking at term 6.

$$a_6 = 640 \times 0.5^{6-1} = 640 \times 0.5^5 = 640 \times \frac{1}{32} = 20$$

20 micrograms remain after 5 hours.

This means geometric sequences can describe decay as well as growth.

## 13.7. Telling them apart

The question to ask: does the sequence *add* a fixed amount each step, or *multiply* by a fixed amount?

| Check | Arithmetic | Geometric |
|---|---|---|
| What's constant? | the difference between terms | the ratio between terms |
| Each term is… | previous term $+d$ | previous term $\times r$ |
| On a graph it looks like… | a straight line | a curve |
| Formula | $a_n = a_1 + (n-1)d$ | $a_n = a_1 \times r^{n-1}$ |

If neither test passes — the differences aren't constant and neither are the ratios — the sequence isn't arithmetic or geometric. There are other kinds, but these two cover most of what you'll see at this level.

## 13.8. Sequences as functions

Here's the thing to take away from this chapter.

A sequence is a function. You feed in a position number — 1, 2, 3, …, $n$ — and the formula gives you back a value. One input, one output, a definite rule. That's the definition of a function.

This means Chapter 4 will build on an idea you already have.

For an arithmetic sequence with $a_1 = 5$ and $d = 3$:

$$a_n = 5 + (n-1) \times 3 = 5 + 3n - 3 = 3n + 2$$

That last form — $3n + 2$ — should look familiar. It's a linear expression. The position $n$ plays the same role as $x$ in a linear equation. If you plotted the terms on a graph with position on the horizontal axis and value on the vertical, you'd get points lying on a straight line.

Geometric sequences, plotted the same way, give points lying on a curve — the same kind of exponential curve you saw in compound growth.

The bigger idea — that a rule can take any input and return a predictable output — is what Chapter 4 is about. Sequences are the clearest introduction to it because the inputs are always whole numbers, so you can list them and see the pattern directly. When Chapter 4 generalises to real-number inputs, you'll already have the idea.

## 13.9. Where this goes

This chapter connects patterns to formulas. The next chapter connects those formulas to graphs.

**Chapter 4 — Graphing relations** takes the connection from this chapter seriously. If $a_n = 3n + 2$ is a function of $n$, then it has a graph.

**Volume 4 — Sequences and series** returns to sequences at a higher level. A *series* is what you get when you add up the terms of a sequence: how much have you saved total after $n$ weeks? How many people has the chain message reached in total? The formulas become more involved, but the foundation — arithmetic or geometric, position determines value — is exactly what this chapter builds.

**Where this shows up**

- A financial analyst projecting investment returns is working with a geometric sequence — each period's value is the previous one scaled by a growth factor.
- A software engineer estimating how long an algorithm takes as the input grows is often asking whether the cost is arithmetic (grows linearly) or geometric (grows exponentially).
- A biologist tracking bacterial colony doublings is reading a geometric sequence, even if the lab notebook just shows raw counts.
- A structural engineer numbering equally-spaced sensor stations along a bridge is building an arithmetic sequence into the measurement system.

The difference between "adds the same amount" and "multiplies by the same amount" is the difference between manageable and overwhelming. Recognising which one you're dealing with is a genuinely useful skill.

## 13.10. Exercises

These are puzzles. Each has a clean answer, but the interesting part is identifying the type of sequence and setting up the formula before you calculate. Don't just list terms.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
```

```
  for (let i = 0; i < current; i++) {
    const s = steps[i];
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align
    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;
    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    row.appendChild(opCell);
    row.appendChild(eqCell);
    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }
    stepsDiv.appendChild(row);
  }
  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "□ Solution complete";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

function updateButtons() {
  nextBtn.disabled = current === totalSteps;
  nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
  counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
}

nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
controls.appendChild(nextBtn);
```

```
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

1. A social media account gains 40 followers per day. It starts with 120 followers. How many followers does it have after 3 weeks (21 days)? *(Identify: arithmetic or geometric? Write the formula, then calculate.)*

```
//| echo: false
makeStepperHTML(1, [
  { op: "Identify sequence type",        eq: "\\text{Arithmetic — gains a fixed 40 each day}", note
  { op: "Write the formula",             eq: "a_n = 120 + (n-1) \\times 40" },
  { op: "Find term at day 21 (n = 21)",  eq: "a_{21} = 120 + 20 \\times 40 = 120 + 800 = 920" },
  { op: "Check",                        eq: "a_1 = 120 \\checkmark,\\quad a_2 = 160 \\checkmark,\\
])
```

2. The sequence 5, 9, 13, 17, … — what is the 25th term?

```
//| echo: false
makeStepperHTML(2, [
  { op: "Find the common difference",    eq: "d = 9 - 5 = 4" },
  { op: "Write the formula",             eq: "a_n = 5 + (n-1) \\times 4" },
  { op: "Substitute n = 25",             eq: "a_{25} = 5 + 24 \\times 4 = 5 + 96 = 101" },
  { op: "Check",                        eq: "a_1 = 5 \\checkmark,\\quad a_2 = 9 \\checkmark,\\quad
])
```

3. A ball is dropped from 160cm. Each bounce reaches exactly 75% of the previous height. How high is the 5th bounce? Round to the nearest centimetre.

```
//| echo: false
makeStepperHTML(3, [
  { op: "Identify sequence type",        eq: "\\text{Geometric — multiplied by 0.75 each time}" },
  { op: "Write the formula",             eq: "a_n = 160 \\times 0.75^{n-1}", note: "The first bounce
  { op: "Substitute n = 5",              eq: "a_5 = 160 \\times 0.75^4 = 160 \\times 0.3164\\ldots
  { op: "Round to nearest cm",           eq: "a_5 \\approx 51 \\text{ cm}" },
  { op: "Check",                        eq: "160, 120, 90, 67.5, 50.6 \\ldots \\checkmark" }
])
```

4. A sequence begins 3, 6, 12, 24, …

   a. Write the formula for the nth term.
   b. Which term first exceeds 3,000?

```
//| echo: false
makeStepperHTML(4, [
  { op: "Find the common ratio",         eq: "r = 6 \\div 3 = 2" },
  { op: "Write the formula (part a)",    eq: "a_n = 3 \\times 2^{n-1}" },
  { op: "Try n = 10",                    eq: "a_{10} = 3 \\times 2^9 = 3 \\times 512 = 1536 < 300
  { op: "Try n = 11",                    eq: "a_{11} = 3 \\times 2^{10} = 3 \\times 1024 = 3072 >
  { op: "Answer (part b)",               eq: "\\text{Term 11 is the first to exceed 3,000}" }
])
```

5. Exam scores in a practice series improve by 4 marks each attempt: 46, 50, 54, 58, … After how many attempts does a student first reach or exceed 80 marks?

```
//| echo: false
makeStepperHTML(5, [
  { op: "Write the formula",             eq: "a_n = 46 + (n-1) \\times 4" },
  { op: "Set up inequality",             eq: "46 + (n-1) \\times 4 \\geq 80" },
  { op: "Subtract 46",                   eq: "(n-1) \\times 4 \\geq 34" },
  { op: "Divide by 4",                   eq: "n - 1 \\geq 8.5" },
  { op: "Add 1, round up",               eq: "n \\geq 9.5 \\Rightarrow n = 10" },
  { op: "Check",                         eq: "a_{10} = 46 + 9 \\times 4 = 46 + 36 = 82 \\geq 80 \
])
```

6. A fundraiser's social posts are shared in a pattern. On day 1 it reaches 5 people. Each day it reaches 4 times as many people as the day before. On which day does it first reach more than 50,000 people?

```
//| echo: false
makeStepperHTML(6, [
  { op: "Write the formula",             eq: "a_n = 5 \\times 4^{n-1}" },
  { op: "Try n = 7",                     eq: "a_7 = 5 \\times 4^6 = 5 \\times 4096 = 20{,}480 < 5
  { op: "Try n = 8",                     eq: "a_8 = 5 \\times 4^7 = 5 \\times 16{,}384 = 81{,}920
  { op: "Answer",                        eq: "\\text{Day 8 is the first day it exceeds 50,000 peo
])
```

7. Two sequences both start at 10.

   - Sequence A: add 15 each term.
   - Sequence B: multiply by 1.5 each term.

   By the 10th term, which is larger? By how much?

```
//| echo: false
makeStepperHTML(7, [
  { op: "Sequence A — formula",          eq: "a_n = 10 + (n-1) \\times 15" },
  { op: "Sequence A — 10th term",        eq: "a_{10} = 10 + 9 \\times 15 = 10 + 135 = 145" },
  { op: "Sequence B — formula",          eq: "b_n = 10 \\times 1.5^{n-1}" },
  { op: "Sequence B — 10th term",        eq: "b_{10} = 10 \\times 1.5^9 = 10 \\times 38.44\\ldots
  { op: "Compare",                       eq: "384.4 - 145 \\approx 239.4", note: "Geometric growt
])
```

# 14. Graphing relations

Seeing equations as pictures

You've checked a mobile data plan: $10 a month fixed, then $2 per extra GB. You want to know, at a glance, what you'd pay for 0, 1, 3, 5 gigabytes. You could solve an equation for each value — or you could draw a line and read off every answer at once.

That line is the graph of the equation. It doesn't replace the algebra. It shows you the whole relationship in one picture.

**Opening visual: the number line becomes the coordinate plane.** Press play and watch the one-dimensional address from earlier chapters turn into a point with two coordinates.

```
//| echo: false
{
  const W = 560, H = 360;
  const margin = { left: 48, right: 28, top: 28, bottom: 56 };
  const x0 = margin.left;
  const x1 = W - margin.right;
  const yMid = H * 0.58;
  const step = (x1 - x0) / 10;
  const originX = x0 + 5 * step;
  const originY = yMid;
  const teal = "#0d9488";
  const purple = "#7c3aed";
  const amber = "#f59e0b";
  const axisGrey = "#9ca3af";
  const gridGrey = "#f3f4f6";
  const textGrey = "#374151";

  let timer = null;
  let t = 0;

  const clamp = (v, a, b) => Math.max(a, Math.min(b, v));
  const lerp = (a, b, u) => a + (b - a) * u;
  const ease = u => 1 - Math.pow(1 - clamp(u, 0, 1), 3);

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }
```

```
const container = document.createElement("div");
container.style.cssText = "max-width: 620px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.55rem;";

const playBtn = document.createElement("button");
playBtn.textContent = "Begin";
playBtn.style.cssText = "padding:0.45rem 0.85rem; border:1px solid #d1d5db; border-radius:8px; bac

const replayBtn = document.createElement("button");
replayBtn.textContent = "Replay";
replayBtn.style.cssText = "padding:0.45rem 0.85rem; border:1px solid #d1d5db; border-radius:8px; b

const status = document.createElement("div");
status.style.cssText = "font-size:0.92rem; color:#4b5563;";
controls.append(playBtn, replayBtn, status);

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:white; border:1px solid #e5e7eb; border-radius:10px

const bg = svgEl("rect");
bg.setAttribute("x", "0");
bg.setAttribute("y", "0");
bg.setAttribute("width", W);
bg.setAttribute("height", H);
bg.setAttribute("fill", "white");
svg.appendChild(bg);

const gridGroup = svgEl("g");
const axisGroup = svgEl("g");
const labelGroup = svgEl("g");
const guideGroup = svgEl("g");
const pointGroup = svgEl("g");
svg.append(gridGroup, axisGroup, guideGroup, pointGroup, labelGroup);

function addText(x, y, text, opts = {}) {
  const node = svgEl("text");
  node.setAttribute("x", x);
  node.setAttribute("y", y);
  node.setAttribute("fill", opts.fill || textGrey);
  node.setAttribute("font-size", opts.size || 14);
  node.setAttribute("font-weight", opts.weight || "500");
  node.setAttribute("text-anchor", opts.anchor || "start");
  node.textContent = text;
```

```
  if (opts.opacity != null) node.setAttribute("opacity", opts.opacity);
  if (opts.rotate) node.setAttribute("transform", `rotate(${opts.rotate} ${x} ${y})`);
  labelGroup.appendChild(node);
  return node;
}

const axisCaption = addText(W / 2, 26, "From one dimension to two dimensions", { anchor: "middl
const phaseText = addText(W / 2, H - 18, "", { anchor: "middle", size: 13, fill: "#6b7280" });
const xLineLabel = addText(originX + 3 * step, originY + 36, "x = 3", { anchor: "middle", size:
const planeLabel = addText(originX + 3 * step + 8, originY - 4 * step - 10, "(3, 4)", { anchor:
const oldAddress = addText(originX + 3 * step, originY + 54, "x = 3 was its address on one line
const newAddress = addText(originX + 3 * step + 8, originY - 4 * step - 28, "(3, 4) is its addr
const fromVol1 = addText(originX, originY + 72, "The number line you know from Vol 1.", { ancho

const hAxis = svgEl("line");
const vAxis = svgEl("line");
const xArrow = svgEl("line");
const yArrow = svgEl("line");
[hAxis, vAxis, xArrow, yArrow].forEach(n => axisGroup.appendChild(n));

const xTag = addText(x1 + 4, originY - 8, "x", { fill: teal, weight: "700", opacity: 0 });
const yTag = addText(originX + 8, margin.top - 4, "y", { fill: purple, weight: "700", opacity:

const point = svgEl("circle");
point.setAttribute("r", "7");
point.setAttribute("fill", amber);
point.setAttribute("stroke", "#b45309");
point.setAttribute("stroke-width", "1.2");
pointGroup.appendChild(point);

const vGuide = svgEl("line");
const hGuide = svgEl("line");
[vGuide, hGuide].forEach(g => {
  g.setAttribute("stroke-width", "2");
  g.setAttribute("stroke-dasharray", "6 5");
  g.setAttribute("opacity", "0");
  guideGroup.appendChild(g);
});
vGuide.setAttribute("stroke", teal);
hGuide.setAttribute("stroke", purple);

function drawTicks(lineY) {
  const group = svgEl("g");
  for (let i = -5; i <= 5; i += 1) {
    const x = originX + i * step;
    const tick = svgEl("line");
    tick.setAttribute("x1", x);
```

```
      tick.setAttribute("x2", x);
      tick.setAttribute("y1", lineY - 6);
      tick.setAttribute("y2", lineY + 6);
      tick.setAttribute("stroke", axisGrey);
      tick.setAttribute("stroke-width", "1.5");
      group.appendChild(tick);

      if (i !== 0) {
        const label = svgEl("text");
        label.setAttribute("x", x);
        label.setAttribute("y", lineY + 20);
        label.setAttribute("text-anchor", "middle");
        label.setAttribute("font-size", "11");
        label.setAttribute("fill", axisGrey);
        label.textContent = i;
        group.appendChild(label);
      }
    }
    return group;
  }

  const hTicks = drawTicks(originY);
  axisGroup.appendChild(hTicks);

  function redraw() {
    const p1 = ease(t / 1.0);
    const p2 = ease((t - 1.0) / 1.5);
    const p3 = ease((t - 2.5) / 1.5);

    gridGroup.replaceChildren();
    if (p2 > 0) {
      for (let i = -5; i <= 5; i += 1) {
        const gx = originX + i * step;
        const gy = originY - i * step;

        const v = svgEl("line");
        v.setAttribute("x1", gx);
        v.setAttribute("x2", gx);
        v.setAttribute("y1", margin.top);
        v.setAttribute("y2", H - margin.bottom);
        v.setAttribute("stroke", gridGrey);
        v.setAttribute("stroke-width", "1");
        v.setAttribute("opacity", (0.9 * p2).toFixed(2));
        gridGroup.appendChild(v);

        const h = svgEl("line");
        h.setAttribute("x1", x0);
```

```
      h.setAttribute("x2", x1);
      h.setAttribute("y1", originY - i * step);
      h.setAttribute("y2", originY - i * step);
      h.setAttribute("stroke", gridGrey);
      h.setAttribute("stroke-width", "1");
      h.setAttribute("opacity", (0.9 * p2).toFixed(2));
      gridGroup.appendChild(h);
    }
  }

  hAxis.setAttribute("x1", x0);
  hAxis.setAttribute("y1", originY);
  hAxis.setAttribute("x2", x1);
  hAxis.setAttribute("y2", originY);
  hAxis.setAttribute("stroke", axisGrey);
  hAxis.setAttribute("stroke-width", "2");

  xArrow.setAttribute("x1", x1 - 12);
  xArrow.setAttribute("y1", originY - 6);
  xArrow.setAttribute("x2", x1);
  xArrow.setAttribute("y2", originY);
  xArrow.setAttribute("stroke", teal);
  xArrow.setAttribute("stroke-width", "2");
  xArrow.setAttribute("opacity", p2.toFixed(2));

  const angle = 90 * p2;
  const length = (H - margin.bottom) - margin.top;
  vAxis.setAttribute("x1", originX);
  vAxis.setAttribute("y1", originY);
  vAxis.setAttribute("x2", originX);
  vAxis.setAttribute("y2", originY - length);
  vAxis.setAttribute("stroke", axisGrey);
  vAxis.setAttribute("stroke-width", "2");
  vAxis.setAttribute("transform", `rotate(${-angle} ${originX} ${originY})`);
  vAxis.setAttribute("opacity", p2.toFixed(2));

  yArrow.setAttribute("x1", originX - 6);
  yArrow.setAttribute("y1", margin.top + 12);
  yArrow.setAttribute("x2", originX);
  yArrow.setAttribute("y2", margin.top);
  yArrow.setAttribute("stroke", purple);
  yArrow.setAttribute("stroke-width", "2");
  yArrow.setAttribute("opacity", p2.toFixed(2));

  xTag.setAttribute("opacity", p2.toFixed(2));
  yTag.setAttribute("opacity", p2.toFixed(2));
  fromVol1.setAttribute("opacity", (1 - 0.6 * p2).toFixed(2));
```

```
  const startX = originX + 3 * step;
  const startY = originY;
  const endX = startX;
  const endY = originY - 4 * step;
  point.setAttribute("cx", startX);
  point.setAttribute("cy", lerp(startY, endY, p3));

  xLineLabel.setAttribute("opacity", (1 - 0.5 * p3).toFixed(2));
  planeLabel.setAttribute("opacity", p3.toFixed(2));
  oldAddress.setAttribute("opacity", p3.toFixed(2));
  newAddress.setAttribute("opacity", p3.toFixed(2));

  vGuide.setAttribute("x1", startX);
  vGuide.setAttribute("x2", startX);
  vGuide.setAttribute("y1", originY);
  vGuide.setAttribute("y2", point.getAttribute("cy"));
  vGuide.setAttribute("opacity", p3.toFixed(2));

  hGuide.setAttribute("x1", originX);
  hGuide.setAttribute("x2", startX);
  hGuide.setAttribute("y1", originY);
  hGuide.setAttribute("y2", originY);
  hGuide.setAttribute("opacity", p3.toFixed(2));

  if (t < 1) {
    phaseText.textContent = "Phase 1: one number line, one address.";
    status.textContent = "A familiar number line from earlier chapters.";
  } else if (t < 2.5) {
    phaseText.textContent = "Phase 2: add a vertical axis and a grid.";
    status.textContent = "One dimension becomes two dimensions.";
  } else {
    phaseText.textContent = "Phase 3: lift the point into the plane.";
    status.textContent = "x = 3 becomes the point (3, 4).";
  }

  replayBtn.style.display = t >= 4 ? "inline-block" : "none";
}

function stopAnim() {
  if (timer) {
    cancelAnimationFrame(timer);
    timer = null;
  }
}

function startAnim() {
  stopAnim();
```

```
    t = 0;
    playBtn.textContent = "Playing...";
    replayBtn.style.display = "none";
    let start = null;
    function frame(ts) {
      if (start == null) start = ts;
      t = Math.min(4.1, (ts - start) / 1000);
      redraw();
      if (t < 4.1) {
        timer = requestAnimationFrame(frame);
      } else {
        playBtn.textContent = "Begin";
        timer = null;
      }
    }
    timer = requestAnimationFrame(frame);
  }

  playBtn.addEventListener("click", startAnim);
  replayBtn.addEventListener("click", startAnim);

  container.append(controls, svg);
  redraw();
  return container;
}
```

## 14.1. What the notation is saying

### 14.1.1. The coordinate plane

Start with a map idea. A graph gives each point an address, so we can locate it exactly.

Draw two number lines crossing at right angles. The horizontal one is the **x-axis**. The vertical one is the **y-axis**. The point where they cross is the **origin** — the zero of both axes.

Any point in the plane has an address: $(x, y)$, read "x across, y up". The $x$ tells you how far to move left or right from the origin. The $y$ tells you how far to move up or down.

So $(3, 2)$ means: go 3 to the right, then 2 up. And $(-1, 4)$ means: go 1 to the left, then 4 up. And $(0, -3)$ means: stay on the vertical axis, go 3 down.

This means the first number always controls the horizontal move, and the second controls the vertical move.

The two axes divide the plane into four regions, called **quadrants**. Top-right is Quadrant I (both $x$ and $y$ positive). Top-left is Quadrant II ($x$ negative, $y$ positive). Bottom-left is Quadrant III (both negative). Bottom-right is Quadrant IV ($x$ positive, $y$ negative). Most of the graphs in this chapter live in Quadrant

I, where both values are positive — because most real-world quantities like cost, distance, and time don't go negative.

This means the full coordinate plane is general, even when a problem uses only part of it.

**Interactive: Coordinate plane explorer.** Click anywhere on the grid to place a point. Watch the $(x, y)$ coordinates update and animated arrows trace the "go right, then up" path from the origin.

```
//| echo: false
{
  const W = 500, H = 500;
  const ORIGIN_X = W / 2, ORIGIN_Y = H / 2;
  const GRID_STEP = 40; // pixels per unit
  const UNITS = 5;      // axis runs from -5 to +5 in each direction (but we'll clip at W/H)

  let pointX = null, pointY = null;

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width", "100%")
    .attr("style", `max-width:${W}px; font-family:inherit; cursor:crosshair;`);

  const toSvg = (mathX, mathY) => ({
    sx: ORIGIN_X + mathX * GRID_STEP,
    sy: ORIGIN_Y - mathY * GRID_STEP
  });
  const toMath = (sx, sy) => ({
    mx: (sx - ORIGIN_X) / GRID_STEP,
    my: (ORIGIN_Y - sy) / GRID_STEP
  });

  // Quadrant labels
  const quadrants = [
    { label: "I", mx: 3, my: 3 },
    { label: "II", mx: -3, my: 3 },
    { label: "III", mx: -3, my: -3 },
    { label: "IV", mx: 3, my: -3 }
  ];
  quadrants.forEach(q => {
    const { sx, sy } = toSvg(q.mx, q.my);
    svg.append("text")
      .attr("x", sx).attr("y", sy)
      .attr("text-anchor", "middle").attr("dominant-baseline", "middle")
      .attr("font-size", "13px").attr("fill", "#e5e7eb").attr("font-weight", "600")
      .text(`Q${q.label}`);
  });

  // Grid lines
  for (let i = -UNITS; i <= UNITS; i++) {
```

```
  const { sx } = toSvg(i, 0);
  const { sy } = toSvg(0, i);
  svg.append("line")
    .attr("x1", sx).attr("y1", 0).attr("x2", sx).attr("y2", H)
    .attr("stroke", i === 0 ? "#6b7280" : "#f3f4f6").attr("stroke-width", i === 0 ? 2 : 1);
  svg.append("line")
    .attr("x1", 0).attr("y1", sy).attr("x2", W).attr("y2", sy)
    .attr("stroke", i === 0 ? "#6b7280" : "#f3f4f6").attr("stroke-width", i === 0 ? 2 : 1);
}

// Axis tick labels
for (let i = -UNITS; i <= UNITS; i++) {
  if (i === 0) continue;
  const { sx } = toSvg(i, 0);
  const { sy } = toSvg(0, i);
  svg.append("text")
    .attr("x", sx).attr("y", ORIGIN_Y + 16)
    .attr("text-anchor", "middle").attr("font-size", "10px").attr("fill", "#9ca3af")
    .text(i);
  svg.append("text")
    .attr("x", ORIGIN_X - 14).attr("y", sy + 4)
    .attr("text-anchor", "end").attr("font-size", "10px").attr("fill", "#9ca3af")
    .text(i);
}

// Axis labels
svg.append("text").attr("x", W - 8).attr("y", ORIGIN_Y - 8)
  .attr("font-size", "13px").attr("fill", "#6b7280").attr("font-weight", "600").text("x");
svg.append("text").attr("x", ORIGIN_X + 8).attr("y", 14)
  .attr("font-size", "13px").attr("fill", "#6b7280").attr("font-weight", "600").text("y");

// Arrow group (updated on click)
const arrowGroup = svg.append("g").attr("class", "arrows");
const pointGroup = svg.append("g").attr("class", "point");
const labelGroup = svg.append("g").attr("class", "label");

// Instruction text
const instrText = svg.append("text")
  .attr("x", W / 2).attr("y", H - 10)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#9ca3af")
  .text("Click anywhere on the grid to place a point");

svg.on("click", function(event) {
  const [sx, sy] = d3.pointer(event, svg.node());
  const { mx, my } = toMath(sx, sy);
  const rx = Math.round(mx * 2) / 2;
  const ry = Math.round(my * 2) / 2;
```

```
const { sx: px, sy: py } = toSvg(rx, ry);
const { sy: axisY } = toSvg(0, 0);
const { sx: axisX } = toSvg(0, 0);

arrowGroup.selectAll("*").remove();
pointGroup.selectAll("*").remove();
labelGroup.selectAll("*").remove();
instrText.text("");

// Horizontal arrow from origin to (rx, axisY)
if (Math.abs(rx) > 0.1) {
  arrowGroup.append("line")
    .attr("x1", ORIGIN_X).attr("y1", ORIGIN_Y)
    .attr("x2", px).attr("y2", ORIGIN_Y)
    .attr("stroke", "#0d9488").attr("stroke-width", 2).attr("stroke-dasharray", "5 3");
  arrowGroup.append("text")
    .attr("x", (ORIGIN_X + px) / 2).attr("y", ORIGIN_Y - 8)
    .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#0d9488")
    .text(rx > 0 ? `→ ${rx}` : `← ${Math.abs(rx)}`);
}

// Vertical arrow from (rx, 0) to (rx, ry)
if (Math.abs(ry) > 0.1) {
  arrowGroup.append("line")
    .attr("x1", px).attr("y1", ORIGIN_Y)
    .attr("x2", px).attr("y2", py)
    .attr("stroke", "#7c3aed").attr("stroke-width", 2).attr("stroke-dasharray", "5 3");
  arrowGroup.append("text")
    .attr("x", px + 10).attr("y", (ORIGIN_Y + py) / 2)
    .attr("font-size", "11px").attr("fill", "#7c3aed")
    .text(ry > 0 ? `↑ ${ry}` : `↓ ${Math.abs(ry)}`);
}

// Point
pointGroup.append("circle")
  .attr("cx", px).attr("cy", py).attr("r", 7)
  .attr("fill", "#f59e0b").attr("stroke", "#fff").attr("stroke-width", 2);

// Coordinate label
const labelX = px + (rx > 3 ? -60 : 12);
const labelY = py + (ry < -3 ? 20 : -12);
labelGroup.append("rect")
  .attr("x", labelX - 4).attr("y", labelY - 14)
  .attr("width", 68).attr("height", 20).attr("rx", 4)
  .attr("fill", "#fff").attr("stroke", "#e5e7eb").attr("stroke-width", 1);
labelGroup.append("text")
  .attr("x", labelX).attr("y", labelY)
```

```
      .attr("font-size", "12px").attr("fill", "#1f2937").attr("font-weight", "600")
      .text(`(${rx}, ${ry})`);
  });

  return svg.node();
}
```

### 14.1.2. A relation as a table of pairs

A **relation** is any set of input-output pairs. You give it an input value, it gives back an output value.

This means a relation connects one quantity to another.

The phone plan from the opening is a relation. Call the number of extra GB the input $x$, and the monthly cost in pounds the output $y$. The rule is:

$$y = 2x + 10$$

Read it as: "y equals 2 times x, plus 10." Plug in $x = 0$: you get $y = 10$. Plug in $x = 3$: you get $y = 16$. That pair of values is the point $(3, 16)$.

Here, $x$ is the input and $y$ is the output. This means every input produces a point on the graph.

Here's the table for a few values:

| $x$ (extra GB) | $y$ (cost, \$) | Point |
|---|---|---|
| 0 | 10 | $(0, 10)$ |
| 1 | 12 | $(1, 12)$ |
| 2 | 14 | $(2, 14)$ |
| 3 | 16 | $(3, 16)$ |
| 5 | 20 | $(5, 20)$ |

Plot those five points on the coordinate plane and they fall in a straight line. Every point on that line satisfies the equation. Every point off it doesn't.

That's the key idea: the graph and the equation are the same relationship in two different languages.

This means the graph should agree with the algebra at every point.

## 14.2. The method

### 14.2.1. Reading slope: rise over run

The equation $y = 2x + 10$ has the form $y = mx + b$.

The letter $m$ is the **slope** — how steeply the line rises or falls. Read it as: for every 1 step to the right, the line goes $m$ steps up (or down, if $m$ is negative).

This means slope is a rate of change.

To measure slope from any two points on the line, count:

$$m = \frac{\text{rise}}{\text{run}} = \frac{\text{change in } y}{\text{change in } x}$$

For the phone plan: pick $(1, 12)$ and $(3, 16)$. Rise = $16 - 12 = 4$. Run = $3 - 1 = 2$. Slope = $4 \div 2 = 2$. That matches the 2 in front of $x$.

This means the coefficient of $x$ tells us how quickly the output changes.

**Positive slope** means the line goes up left-to-right — the quantity grows as $x$ increases.

**Negative slope** means the line goes down left-to-right — the quantity shrinks as $x$ increases. For a negative slope like $-3$, the number of units down for each unit across is 3 — the size of the slope ignoring the sign. The sign tells you the direction; the number tells you how steep.

**Zero slope** means a flat horizontal line — the output doesn't change no matter what the input is.

The letter $b$ is the **y-intercept** — where the line crosses the y-axis. Substitute $x = 0$ into any linear equation and you get $y = b$. For the phone plan, $b = 10$: the line crosses the y-axis at $(0, 10)$, meaning you pay \$10 even with zero extra data. That's the standing charge.

This means the y-intercept is the starting value.

> **Why this works**
> Every point on a line satisfies its equation — that's the definition of the line. So if you want to draw the line, you only need two points: the equation holds at both, and a straight line through two points is determined completely. The y-intercept $(0, b)$ is almost always the easiest first point. From there, the slope tells you how to find the next one.

### 14.2.2. Method 1: Graphing from a table of values

1. Choose three or four values of $x$ — spread them out.
2. Substitute each into the equation to get the matching $y$.
3. Write down the $(x, y)$ pairs.
4. Plot the points on the coordinate plane.
5. Draw a straight line through them.

Use at least three points. If the third one doesn't fall on the line through the first two, you've made an arithmetic error somewhere.

This means a table is often the safest way to start.

### 14.2.3. Method 2: Graphing from slope-intercept form

When the equation is written as $y = mx + b$, you can graph it without a table:

1. Plot the y-intercept $(0, b)$ — that's your starting point.
2. From that point, apply the slope: go 1 unit right, go $m$ units up (or, if $m$ is negative, go that many units down).
3. Mark that second point.
4. Draw the line through both points.
5. Extend the line in both directions.

This is faster than a table once you're comfortable with it.

This means slope-intercept form shows two key features of the line at once.

### 14.2.4. Method 3: Finding the equation from a point and a slope

Sometimes you know the slope and a point on the line, but not the y-intercept. To find $b$, substitute the known point into $y = mx + b$ and solve.

For example: slope $m = 2$, and the line passes through $(3, 11)$. Substitute into $y = mx + b$:

$$11 = 2(3) + b$$

$$11 = 6 + b$$

$$b = 5$$

So the equation is $y = 2x + 5$. Check: does $(3, 11)$ satisfy it? $2(3) + 5 = 11$. Yes.

This means a known point can help us work backward to the missing intercept.

The same method works when you're given two points instead of a point and a slope — first calculate the slope from the two points using rise over run, then substitute one of the points to find $b$.

**Interactive: Linear graph builder.** Drag the sliders to change slope $m$ and y-intercept $b$. The line $y = mx + b$ redraws live. The slope triangle shows the rise and run. The equation updates above the graph.

```
//| echo: false
viewof graphM = Inputs.range([-3, 3], { label: "Slope m", step: 0.5, value: 1 })
```

```
//| echo: false
viewof graphB = Inputs.range([-5, 5], { label: "y-intercept b", step: 0.5, value: 2 })
```

```
//| echo: false
{
  const m = graphM;
  const b = graphB;

  const W = 500, H = 500;
  const X_MIN = -5, X_MAX = 10, Y_MIN = -5, Y_MAX = 10;
  const PAD_L = 48, PAD_R = 20, PAD_T = 48, PAD_B = 40;
  const plotW = W - PAD_L - PAD_R;
  const plotH = H - PAD_T - PAD_B;

  const xScale = (v) => PAD_L + (v - X_MIN) / (X_MAX - X_MIN) * plotW;
  const yScale = (v) => PAD_T + plotH - (v - Y_MIN) / (Y_MAX - Y_MIN) * plotH;

  const d3local = await require("d3@7");
  const svg = d3local.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width", "100%")
    .attr("style", `max-width:${W}px; font-family:inherit;`);

  // Background
  svg.append("rect")
    .attr("x", PAD_L).attr("y", PAD_T)
    .attr("width", plotW).attr("height", plotH)
    .attr("fill", "#f9fafb");

  // Grid
  for (let x = X_MIN; x <= X_MAX; x++) {
    svg.append("line")
      .attr("x1", xScale(x)).attr("y1", PAD_T)
      .attr("x2", xScale(x)).attr("y2", PAD_T + plotH)
      .attr("stroke", x === 0 ? "#9ca3af" : "#e5e7eb")
      .attr("stroke-width", x === 0 ? 2 : 1);
  }
  for (let y = Y_MIN; y <= Y_MAX; y++) {
    svg.append("line")
      .attr("x1", PAD_L).attr("y1", yScale(y))
      .attr("x2", PAD_L + plotW).attr("y2", yScale(y))
      .attr("stroke", y === 0 ? "#9ca3af" : "#e5e7eb")
      .attr("stroke-width", y === 0 ? 2 : 1);
  }

  // Axis tick labels
  for (let x = X_MIN; x <= X_MAX; x += 2) {
    if (x === 0) continue;
    svg.append("text")
      .attr("x", xScale(x)).attr("y", yScale(0) + 16)
```

```
    .attr("text-anchor", "middle").attr("font-size", "10px").attr("fill", "#9ca3af")
    .text(x);
}
for (let y = Y_MIN; y <= Y_MAX; y += 2) {
  if (y === 0) continue;
  svg.append("text")
    .attr("x", xScale(0) - 8).attr("y", yScale(y) + 4)
    .attr("text-anchor", "end").attr("font-size", "10px").attr("fill", "#9ca3af")
    .text(y);
}
// Axis labels
svg.append("text").attr("x", PAD_L + plotW - 4).attr("y", yScale(0) - 6)
  .attr("font-size", "12px").attr("fill", "#6b7280").attr("font-weight", "600").text("x");
svg.append("text").attr("x", xScale(0) + 6).attr("y", PAD_T + 10)
  .attr("font-size", "12px").attr("fill", "#6b7280").attr("font-weight", "600").text("y");

// Clip the line to the plot area
const lineY = (x) => m * x + b;
// Find x values where line enters/exits the plot area
const candidates = [
  { x: X_MIN, y: lineY(X_MIN) },
  { x: X_MAX, y: lineY(X_MAX) },
  m !== 0 ? { x: (Y_MIN - b) / m, y: Y_MIN } : null,
  m !== 0 ? { x: (Y_MAX - b) / m, y: Y_MAX } : null
].filter(p => p && p.x >= X_MIN - 0.01 && p.x <= X_MAX + 0.01 && p.y >= Y_MIN - 0.01 && p.y <=
candidates.sort((a, c) => a.x - c.x);

if (candidates.length >= 2) {
  const p1 = candidates[0], p2 = candidates[candidates.length - 1];
  svg.append("line")
    .attr("x1", xScale(p1.x)).attr("y1", yScale(p1.y))
    .attr("x2", xScale(p2.x)).attr("y2", yScale(p2.y))
    .attr("stroke", "#0d9488").attr("stroke-width", 2.5);
}

// y-intercept point
const bInView = b >= Y_MIN && b <= Y_MAX;
if (bInView) {
  svg.append("circle")
    .attr("cx", xScale(0)).attr("cy", yScale(b))
    .attr("r", 6).attr("fill", "#0d9488").attr("stroke", "#fff").attr("stroke-width", 2);
  svg.append("text")
    .attr("x", xScale(0) + 10).attr("y", yScale(b) - 8)
    .attr("font-size", "11px").attr("fill", "#0d9488")
    .text(`(0, ${b})`);
}
```

```
  // Slope triangle — drawn from (1, b+m) showing rise and run
  const triX0 = 1, triY0 = m * triX0 + b, triX1 = triX0 + 1, triY1 = m * triX1 + b;
  const triVisible = triX0 >= X_MIN && triX1 <= X_MAX &&
    Math.min(triY0, triY1) >= Y_MIN && Math.max(triY0, triY1) <= Y_MAX;

  if (triVisible && m !== 0) {
    // Horizontal leg (run = 1)
    svg.append("line")
      .attr("x1", xScale(triX0)).attr("y1", yScale(triY0))
      .attr("x2", xScale(triX1)).attr("y2", yScale(triY0))
      .attr("stroke", "#f59e0b").attr("stroke-width", 2).attr("stroke-dasharray", "3 2");
    svg.append("text")
      .attr("x", xScale(triX0 + 0.5)).attr("y", yScale(triY0) + 14)
      .attr("text-anchor", "middle").attr("font-size", "10px").attr("fill", "#f59e0b")
      .text("run = 1");
    // Vertical leg (rise = m)
    svg.append("line")
      .attr("x1", xScale(triX1)).attr("y1", yScale(triY0))
      .attr("x2", xScale(triX1)).attr("y2", yScale(triY1))
      .attr("stroke", "#7c3aed").attr("stroke-width", 2).attr("stroke-dasharray", "3 2");
    svg.append("text")
      .attr("x", xScale(triX1) + (m < 0 ? -48 : 10)).attr("y", yScale((triY0 + triY1) / 2) + 4)
      .attr("font-size", "10px").attr("fill", "#7c3aed")
      .text(`rise = ${m}`);
  }

  // Equation label
  const mStr = m === 1 ? "" : m === -1 ? "-" : `${m}`;
  const bStr = b === 0 ? "" : b > 0 ? ` + ${b}` : ` − ${Math.abs(b)}`;
  const eqStr = `y = ${mStr}x${bStr}`;
  svg.append("text")
    .attr("x", W / 2).attr("y", 28)
    .attr("text-anchor", "middle")
    .attr("font-size", "16px").attr("fill", "#0d9488").attr("font-weight", "700")
    .text(eqStr);

  return svg.node();
}
```

## 14.3. Why this works

**The graph is the equation**
Every pair $(x, y)$ that satisfies $y = mx + b$ lies on the line. Every pair that doesn't satisfy it lies off the line. The line isn't a picture of the equation — it *is* the equation, drawn in space. This means you can read information off the graph the same way you'd get it from the algebra: find an $x$ on the horizontal axis, trace up to the line, and read the matching $y$ on the vertical axis. Or go the other way: find a $y$, trace across to the line, read the $x$.
This two-way reading becomes powerful when you have two lines on the same graph. Where they cross, both equations are satisfied at once — that intersection point is the solution to a system of two equations.

## 14.4. Worked examples

**Example 1 — Distance and time (Sciences).**

You cycle to school at a steady 12 km/h. Your school is 6 km away. Write an equation for your distance from school, and graph it.

Let $x$ = time in hours since you left home. Let $y$ = distance from home, in kilometres.

At a steady speed:

$$y = 12x$$

This is $y = mx + b$ with $m = 12$ and $b = 0$.

| $x$ (hours) | $y$ (km from home) |
|---|---|
| 0 | 0 |
| 0.25 | 3 |
| 0.5 | 6 |

The slope is 12 — for every hour, you cover 12 km. The y-intercept is 0 — at time zero you're still at home. The graph is a line from the origin, rising steeply. Reading from the graph: after 0.25 hours (15 minutes) you're 3 km along.

You arrive at school when $y = 6$. Solve: $12x = 6$, so $x = 0.5$ hours. That's 30 minutes. Confirmed by the table.

This means a graph can answer many questions once the relationship is drawn.

**Slope = speed** in a distance-time graph. This is not a coincidence — it's exactly what slope means physically.

**Example 2 — Phone plan (Finance).**

Your data plan costs \$8 per month fixed, plus \$3 for each extra GB you use. Write and graph the monthly cost equation.

$$y = 3x + 8$$

where $x$ = extra GB used, $y$ = total cost in \$.

Method 2 (slope-intercept): Plot $(0, 8)$ — that's the standing charge. Slope is 3, so from $(0, 8)$ go 1 right and 3 up to $(1, 11)$. Draw the line.

| $x$ (extra GB) | $y$ (cost, \$) |
|---|---|
| 0 | 8 |
| 1 | 11 |
| 2 | 14 |
| 4 | 20 |

Reading from the graph: 3 extra GB costs \$17. When does the bill hit \$26? Set $y = 26$: $3x + 8 = 26$, so $3x = 18$, $x = 6$ GB. The algebra and the graph tell the same story.

This means the graph shows the whole pricing pattern, not just one case.

---

**Example 3 — Points scored (Sport).**

A basketball player scores an average of 18 points per game. She has already played 4 games. Write a formula for her total points, and find when she'll have 144 total.

$$y = 18x + 72$$

Wait — where does the 72 come from? She already has $18 \times 4 = 72$ points from those 4 games. So $b = 72$ (her running total at the start of tracking), and $m = 18$ (she adds 18 each game).

Plot: start at $(0, 72)$ — that's game zero from now, 72 points already. Each game adds 18. From the graph, you can see she hits 144 points after 4 more games.

Check: $18x + 72 = 144$, so $18x = 72$, $x = 4$. Yes.

This means slope can describe a running total changing over time.

**Slope = rate** — in any graph where the y-axis tracks a total and the x-axis tracks time or count, slope is the rate of change. How fast the total grows.

---

**Example 4 — Computing: screen coordinates.**

One thing worth knowing before this example: in maths, $y$ increases upward. On a screen — in game engines, browser graphics, and most image software — $y$ increases *downward*, so $(0, 0)$ is the top-left corner. This example uses mathematical coordinates. If you're writing actual game code, a character "moving up" would have a *decreasing* $y$ value in the engine's system.

A character starts at position $(x, y) = (50, 80)$ in mathematical coordinates and moves right at 5 pixels per frame and down at 3 pixels per frame.

After $t$ frames:

$$x = 50 + 5t \qquad y = 80 + 3t$$

These are two separate linear equations, one for each axis. After 10 frames: $x = 100$, $y = 110$. After 20 frames: $x = 150$, $y = 140$. The path is a straight line, and its slope (in these coordinates) is $\frac{3}{5}$ — 3 units down for every 5 across.

This means motion in two directions can be described by two linked linear rules.

## 14.5. A note on curves

All the examples above produce straight lines. But not all relationships are linear.

If you track your savings when you're earning interest on interest, the graph curves upward — it gets steeper as time goes on. If you track how a hot drink cools, it curves downward and flattens out. A ball thrown in the air traces a curve that dips back down.

A curve on a graph means the rate of change isn't constant. The slope isn't fixed — it depends on where you are. That's a very different situation from a straight line, where the slope is the same everywhere.

You'll meet curved graphs properly when we reach polynomials and functions in Volume 3. For now, notice the difference: a straight line means a constant rate. A curve means the rate is changing.

## 14.6. Where this goes

This chapter takes equations from earlier chapters and makes them visible. Instead of solving for one value, we can now see the whole relationship at once.

The slope of a line is a constant — pick any two points and you get the same number. In Volume 3, **Functions and relations** makes this more precise: you'll define a function formally, think about domain and range, and meet graphs that aren't lines. From there, in Volume 5, **Differential calculus** asks a harder question: if the slope of a curve isn't constant, what is the slope *at a single point*? The answer — the derivative — is built on exactly the rise-over-run idea from this chapter. You're already holding the seed of calculus in the phrase "change in $y$ over change in $x$."

The second direction is **systems of equations**. You've already seen that a linear equation graphs as a line. Two linear equations graph as two lines. If they cross, the crossing point solves both equations at once — you can read the answer off the graph, or find it algebraically. That's the foundation of linear algebra, which appears in Volume 4 and underpins nearly everything in engineering and data science.

**Where this shows up**

- A civil engineer plots a road's vertical profile as a distance-height graph. Slope is gradient, measured in percent. Regulations specify maximum gradients for different road types.
- A physicist reads the slope of a velocity-time graph to find acceleration. The area under the graph gives distance — a preview of integration.
- A business analyst draws cost and revenue lines on the same graph. The crossing point is break-even: the number of units where cost equals revenue.
- A game developer uses $(x, y)$ coordinates to position every object on screen. Movement is addition; rotation involves trig — which also starts with the coordinate plane.

## 14.7. Exercises

These are puzzles. Each one has a clean answer. The work is reading the situation carefully, building the equation, and deciding whether the graph or the algebra gives you what you need faster.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
```

```
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

1. A streaming service costs $6 per month. A rival service costs $2 per month plus $1 per film watched. Write equations for both costs and find how many films per month makes them equal. Which service is cheaper if you watch 5 films a month?

```
//| echo: false
makeStepperHTML(1, [
```

```
  { op: "Write as an equation: Service A",  eq: "y = 6" },
  { op: "Write as an equation: Service B",  eq: "y = x + 2" },
  { op: "Set them equal",                   eq: "6 = x + 2" },
  { op: "Solve for x",                      eq: "x = 4 \\text{ films}" },
  { op: "Compare at 5 films",               eq: "A: 6,\\quad B: 5 + 2 = 7", note: "Service A is chea
  { op: "Check",                            eq: "x=4:\\; 4+2=6 \\checkmark" }
])
```

2. A delivery driver starts a shift with a full tank holding 60 litres. She uses fuel at 0.08 litres per kilometre. Write an equation for fuel remaining after $x$ km. What is the slope, and what does it mean in context? How far can she drive before the tank is empty?

```
//| echo: false
makeStepperHTML(2, [
  { op: "Write as an equation",           eq: "y = 60 - 0.08x" },
  { op: "Identify the slope",             eq: "m = -0.08 \\text{ litres per km}", note: "Negative sl
  { op: "Set y = 0 (tank empty)",         eq: "0 = 60 - 0.08x" },
  { op: "Solve for x",                    eq: "0.08x = 60 \\Rightarrow x = 750 \\text{ km}" },
  { op: "Check",                          eq: "60 - 0.08 \\times 750 = 60 - 60 = 0 \\checkmark" }
])
```

3. Two friends start at the same place and walk in opposite directions. Priya walks at 5 km/h; Jordan walks at 4 km/h. Write an equation for the distance between them after $t$ hours. What is the slope? After how long are they 13.5 km apart?

```
//| echo: false
makeStepperHTML(3, [
  { op: "Write as an equation",           eq: "d = 5t + 4t = 9t", note: "They move away from each ot
  { op: "Identify the slope",             eq: "m = 9 \\text{ km/h (the rate at which they separate)
  { op: "Set d = 13.5",                   eq: "9t = 13.5" },
  { op: "Solve for t",                    eq: "t = 1.5 \\text{ hours}" },
  { op: "Check",                          eq: "9 \\times 1.5 = 13.5 \\checkmark" }
])
```

4. A runner's training log shows she ran 15 km in week 1 and increases her distance by 2 km each week. Write an equation for total distance run after $w$ weeks of training (not counting week 1 as week 0 — let week 1 be $w = 1$). Plot the first six weeks. In which week does she first exceed 100 km total?

```
//| echo: false
makeStepperHTML(4, [
  { op: "Write the weekly distance",       eq: "d_w = 15 + (w-1) \\times 2 = 2w + 13" },
  { op: "Sum the first w weeks",           eq: "T_w = \\sum_{k=1}^{w}(2k+13) = w^2 + 14w", note: "Su
  { op: "Set T_w > 100",                   eq: "w^2 + 14w > 100" },
  { op: "Try w = 6",                       eq: "36 + 84 = 120 > 100 \\checkmark" },
  { op: "Try w = 5",                       eq: "25 + 70 = 95 < 100" },
  { op: "Answer",                          eq: "\\text{She first exceeds 100 km total in week 6}" }
])
```

5. A line passes through the points $(2, 7)$ and $(5, 13)$. Find the slope. Then substitute one of the points into $y = mx + b$ to solve for the y-intercept $b$, and write the full equation. Check your equation works for both original points.

```
//| echo: false
makeStepperHTML(5, [
  { op: "Calculate slope",                eq: "m = \\frac{13-7}{5-2} = \\frac{6}{3} = 2" },
  { op: "Substitute (2, 7) into y=mx+b", eq: "7 = 2(2) + b \\Rightarrow 7 = 4 + b \\Rightarrow b
  { op: "Write the equation",             eq: "y = 2x + 3" },
  { op: "Check point (2, 7)",             eq: "2(2)+3 = 7 \\checkmark" },
  { op: "Check point (5, 13)",            eq: "2(5)+3 = 13 \\checkmark" }
])
```

6. A line has slope $-2$ and passes through $(3, 4)$. Substitute the point into $y = mx + b$ to find $b$, then write the full equation. Find where the line crosses the x-axis (the point where $y = 0$).

```
//| echo: false
makeStepperHTML(6, [
  { op: "Substitute into y = mx + b",    eq: "4 = -2(3) + b \\Rightarrow 4 = -6 + b \\Rightarrow
  { op: "Write the equation",             eq: "y = -2x + 10" },
  { op: "Find x-intercept (set y = 0)",  eq: "0 = -2x + 10 \\Rightarrow 2x = 10 \\Rightarrow x =
  { op: "x-intercept",                   eq: "(5,\\, 0)" },
  { op: "Check",                          eq: "-2(5)+10 = 0 \\checkmark;\\quad -2(3)+10 = 4 \\che
])
```

7. Two phone plans are graphed below — but the graph has been removed. Plan A: $y = 4x + 5$. Plan B: $y = 2x + 15$. Without drawing the graph, find the value of $x$ where the plans cost the same. Which plan is cheaper for someone using 4 units? Which for someone using 8 units?

```
//| echo: false
makeStepperHTML(7, [
  { op: "Write as an equation",           eq: "4x + 5 = 2x + 15" },
  { op: "Subtract 2x from both sides",    eq: "2x + 5 = 15" },
  { op: "Subtract 5 from both sides",     eq: "2x = 10 \\Rightarrow x = 5" },
  { op: "Compare at x = 4",              eq: "A: 4(4)+5=21,\\quad B: 2(4)+15=23 \\Rightarrow \\te
  { op: "Compare at x = 8",              eq: "A: 4(8)+5=37,\\quad B: 2(8)+15=31 \\Rightarrow \\te
  { op: "Check crossover at x = 5",      eq: "A: 4(5)+5=25,\\quad B: 2(5)+15=25 \\checkmark" }
])
```

# Part III.

# Volume 3 — Shape and measure

# 15. Volume 3 — Shape and measure

The central move of this volume is giving coordinates to space.

You already know how to calculate and balance equations. This volume shows you how algebraic tools can be used to describe the physical world and how the world, in turn, can give meaning to abstract math.

By the end of this volume, you will be able to map distances, relate angles to side lengths, understand the structures behind non-linear relationships, and formally describe how different parameters interact.

This is the bridge where algebra stops being just letters on a page and starts becoming a map of reality.

## 15.1. Chapters

| Chapter | Title | Core move |
|---|---|---|
| 1 | Geometry and measurement | Quantifying physical space |
| 2 | Trigonometry | Relating angles to side lengths |
| 3 | Polynomials and factoring | Deconstructing complex equations |
| 4 | Functions and relations | Mapping inputs to outputs |

# 16. Geometry and measurement

Reading the shape of the world

Your bedroom floor has a certain area. You know this even without measuring it — you can picture whether a rug would fit, whether the furniture leaves enough space, whether it's bigger or smaller than a friend's room.

Your phone screen is described by a diagonal measurement in inches. But the area that matters — the bit you actually look at — is the width times the height, not the diagonal. The diagonal is a shortcut that hides the real shape.

The skateboard ramp at the park is a triangle in cross-section. If you built one, you'd need to know the length of the slope, the height it reaches, and the distance it runs along the ground — and how those three numbers relate to each other.

These are all geometry. This chapter gives you the tools to read the shape of any object you encounter — and to find the measurement you need from the ones you have.

## 16.1. What the notation is saying

Every formula in this chapter is shorthand for a relationship between measurements. Before using any of them, here is what each symbol means.

This means the symbols are not new ideas by themselves. They are short labels for measurements you already understand.

$A$ — area. The amount of flat space a shape covers. Measured in square units: square centimetres ($cm^2$), square metres ($m^2$), square kilometres ($km^2$). If a length is in centimetres, the area will be in square centimetres.

$P$ — perimeter. The total distance around the outside edge of a flat shape. Measured in the same units as the lengths themselves — not squared.

$r$ — radius. The distance from the centre of a circle to any point on its edge. The diameter $d$ is twice the radius: $d = 2r$.

$\pi$ — pi. Read it as "pie". It's approximately 3.14159, but it never terminates or repeats — it's an irrational number. It appears in every circle formula because it captures the relationship between a circle's diameter and the distance around it. Your calculator has a $\pi$ button; use it rather than the approximation unless you're told otherwise.

$V$ — volume. The amount of three-dimensional space a solid object fills. Measured in cubic units: $cm^3$, $m^3$, litres (1 litre = 1000 $cm^3$).

$h$ — height. In a prism or cylinder, this is the perpendicular distance between the two bases — how tall the solid is, measured straight through, at a right angle to the base.

$a^2 + b^2 = c^2$ — the Pythagorean theorem. Read it as: the square of $a$ plus the square of $b$ equals the square of $c$. Here $a$ and $b$ are the two shorter sides of a right-angled triangle and $c$ is the longest side — the one opposite the right angle, called the **hypotenuse**. This only works for right triangles. The right angle is marked with a small square in diagrams.

This means every formula depends on naming the measurements correctly first.

## 16.2. The method

### 16.2.1. Rectangles

A rectangle has two pairs of equal sides. Call the longer side the length $l$ and the shorter side the width $w$.

**Area** — multiply length by width:

$$A = l \times w$$

A bedroom that is 4 metres long and 3 metres wide has area $4 \times 3 = 12$ $m^2$.

Here, $l$ is length and $w$ is width. This means area measures how much flat surface the shape covers.

**Perimeter** — add up all four sides. Since opposite sides are equal, that's two lengths plus two widths:

$$P = 2l + 2w$$

The same bedroom has perimeter $2(4) + 2(3) = 8 + 6 = 14$ m.

This means perimeter is a boundary length, not a surface amount.

### 16.2.2. Triangles

A triangle has three sides and three angles. For area, you need a **base** and a **height**. The base is any side you choose. The height is the perpendicular distance from that base to the opposite corner — straight up, at a right angle to the base.

**Area:**

$$A = \frac{1}{2} \times b \times h$$

Read it as: half of base times height. The halving is there because a triangle is half a rectangle — if you put two identical triangles together, they make a parallelogram with the same base and height.

A triangle with base 6 cm and height 4 cm has area $\frac{1}{2} \times 6 \times 4 = 12$ cm².

Here, $b$ is the base and $h$ is the perpendicular height. This means sloping side lengths do not replace the height in the area formula.

**Perimeter** — just add all three sides. There's no shortcut unless the triangle has special properties.

### 16.2.3. Circles

**Circumference** — the distance around the edge of a circle:

$$C = 2\pi r$$

Read it as: two times pi times radius. Since $d = 2r$, this is the same as $\pi d$ — pi times diameter. That's actually where $\pi$ comes from: the ratio of any circle's circumference to its diameter. Every circle, everywhere, has $C = \pi d$.

This means circumference is a length, so its units stay unsquared.

**Area** — the amount of flat space a circle covers:

$$A = \pi r^2$$

Read it as: pi times radius squared. A circle with radius 5 cm has area $\pi \times 5^2 = 25\pi \approx 78.5$ cm².

Note: leave the answer as $25\pi$ when exact form is needed. Use the decimal approximation when you need a number to compare or measure with.

This means the exact form and the decimal form describe the same area; they are just useful in different situations.

```
//| echo: false
viewof circleRadius = Inputs.range([1, 10], {
  value: 5,
  step: 0.5,
  label: "Radius r (units)"
})
```

```
//| echo: false
{
  const r = circleRadius;
  const scale = 20;
  const svgW = 480, svgH = 260;
  const cx = 140, cy = svgH / 2;

  const area = Math.PI * r * r;
  const circ = 2 * Math.PI * r;

  const svg = d3.create("svg")
```

```
  .attr("width", svgW)
  .attr("height", svgH)
  .style("font-family", "inherit");

// Circle
svg.append("circle")
  .attr("cx", cx)
  .attr("cy", cy)
  .attr("r", r * scale)
  .attr("fill", "#dbeafe")
  .attr("stroke", "#3b82f6")
  .attr("stroke-width", 2);

// Radius line
svg.append("line")
  .attr("x1", cx).attr("y1", cy)
  .attr("x2", cx + r * scale).attr("y2", cy)
  .attr("stroke", "#1d4ed8")
  .attr("stroke-width", 1.5)
  .attr("stroke-dasharray", "4,3");

// Radius label
svg.append("text")
  .attr("x", cx + (r * scale) / 2)
  .attr("y", cy - 6)
  .attr("text-anchor", "middle")
  .attr("font-size", "13px")
  .attr("fill", "#1d4ed8")
  .text(`r = ${r}`);

// Stats panel
const px = 310, py = 60;
svg.append("rect")
  .attr("x", px - 10).attr("y", py - 24)
  .attr("width", 170).attr("height", 110)
  .attr("fill", "#f8fafc")
  .attr("stroke", "#e2e8f0")
  .attr("rx", 6);

svg.append("text").attr("x", px).attr("y", py)
  .attr("font-size", "13px").attr("font-weight", "600").attr("fill", "#374151")
  .text("Measurements");

svg.append("text").attr("x", px).attr("y", py + 26)
  .attr("font-size", "13px").attr("fill", "#374151")
  .text(`Area = ⍰r²`);
```

```
  svg.append("text").attr("x", px).attr("y", py + 44)
    .attr("font-size", "13px").attr("fill", "#2563eb")
    .text(`= ${area.toFixed(2)} sq units`);

  svg.append("text").attr("x", px).attr("y", py + 68)
    .attr("font-size", "13px").attr("fill", "#374151")
    .text(`Circumference = 2πr`);

  svg.append("text").attr("x", px).attr("y", py + 86)
    .attr("font-size", "13px").attr("fill", "#059669")
    .text(`= ${circ.toFixed(2)} units`);

  return svg.node();
}
```

### 16.2.4. The Pythagorean theorem

For any right-angled triangle with legs $a$ and $b$ and hypotenuse $c$:

$$a^2 + b^2 = c^2$$

**Finding the hypotenuse** when you know the two legs:

$$c = \sqrt{a^2 + b^2}$$

A right triangle with legs 3 cm and 4 cm:

$$c = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5 \text{ cm}$$

**Finding a leg** when you know the hypotenuse and the other leg:

$$a = \sqrt{c^2 - b^2}$$

A right triangle with hypotenuse 13 cm and one leg 5 cm:

$$a = \sqrt{13^2 - 5^2} = \sqrt{169 - 25} = \sqrt{144} = 12 \text{ cm}$$

The next step looks slightly mechanical. Square both known values, add (or subtract), then take the square root. In that order, every time.

This means the theorem lets us convert two known side lengths into the missing one, but only in a right triangle.

```
//| echo: false
viewof legA = Inputs.range([1, 8], { value: 3, step: 1, label: "Leg a" })
```

```
//| echo: false
viewof legB = Inputs.range([1, 8], { value: 4, step: 1, label: "Leg b" })
```

```
//| echo: false
{
  const a = legA, b = legB;
  const c = Math.sqrt(a * a + b * b);
  const scale = 28;
  const margin = 60;
  const svgW = 520, svgH = 340;

  // Triangle vertices: right angle at bottom-left
  const ox = margin + 20, oy = svgH - margin - 20;
  const ax = ox + b * scale, ay = oy;       // bottom-right
  const bx = ox, by = oy - a * scale;       // top-left

  const svg = d3.create("svg")
    .attr("width", svgW)
    .attr("height", svgH)
    .style("font-family", "inherit");

  // Square on side b (bottom, along x-axis)
  svg.append("rect")
    .attr("x", ox).attr("y", oy)
    .attr("width", b * scale).attr("height", b * scale)
    .attr("fill", "#fef9c3").attr("stroke", "#ca8a04").attr("stroke-width", 1.5);
  svg.append("text")
    .attr("x", ox + b * scale / 2).attr("y", oy + b * scale / 2 + 5)
    .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#92400e")
    .text(`b² = ${b*b}`);

  // Square on side a (left, along y-axis) — goes to the left
  svg.append("rect")
    .attr("x", ox - a * scale).attr("y", by)
    .attr("width", a * scale).attr("height", a * scale)
    .attr("fill", "#dcfce7").attr("stroke", "#16a34a").attr("stroke-width", 1.5);
  svg.append("text")
    .attr("x", ox - a * scale / 2).attr("y", by + a * scale / 2 + 5)
    .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#166534")
    .text(`a² = ${a*a}`);

  // Square on hypotenuse c — rotated square drawn as polygon
  // hypotenuse goes from (ax,ay) to (bx,by)
  // perpendicular direction (outward): normalise then rotate 90°
```

```
const dx = bx - ax, dy = by - ay;
const len = Math.sqrt(dx*dx + dy*dy);
const nx = -dy / len, ny = dx / len; // outward normal
const p1 = {x: ax, y: ay};
const p2 = {x: bx, y: by};
const p3 = {x: bx + nx * len, y: by + ny * len};
const p4 = {x: ax + nx * len, y: ay + ny * len};
const pts = [p1,p2,p3,p4].map(p => `${p.x},${p.y}`).join(" ");

svg.append("polygon")
  .attr("points", pts)
  .attr("fill", "#dbeafe").attr("stroke", "#2563eb").attr("stroke-width", 1.5);

const midCx = (p1.x + p2.x + p3.x + p4.x) / 4;
const midCy = (p1.y + p2.y + p3.y + p4.y) / 4;
svg.append("text")
  .attr("x", midCx).attr("y", midCy + 5)
  .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#1e3a8a")
  .text(`c² = ${(c*c).toFixed(2)}`);

// Triangle fill
svg.append("polygon")
  .attr("points", `${ox},${oy} ${ax},${ay} ${bx},${by}`)
  .attr("fill", "#f1f5f9").attr("stroke", "#475569").attr("stroke-width", 2);

// Right-angle marker
const sq = 8;
svg.append("rect")
  .attr("x", ox).attr("y", oy - sq)
  .attr("width", sq).attr("height", sq)
  .attr("fill", "none").attr("stroke", "#475569").attr("stroke-width", 1.5);

// Side labels
svg.append("text")
  .attr("x", ox + b * scale / 2).attr("y", oy - 8)
  .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#374151")
  .text(`b = ${b}`);

svg.append("text")
  .attr("x", ox - 14).attr("y", oy - a * scale / 2)
  .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#374151")
  .text(`a = ${a}`);

// Equation below
svg.append("text")
  .attr("x", svgW / 2).attr("y", svgH - 16)
  .attr("text-anchor", "middle").attr("font-size", "14px").attr("fill", "#111827")
```

```
    .text(`${a}² + ${b}² = ${a*a} + ${b*b} = ${(a*a+b*b).toFixed(2)}   →   c = ${c.toFixed(3)}`);

  return svg.node();
}
```

**Why the Pythagorean theorem works**
Draw a right triangle. Now draw a square on each side — one square on leg $a$, one on leg $b$, one on the hypotenuse $c$. The theorem says the area of the big square exactly equals the total area of the two smaller squares: $a^2 + b^2 = c^2$.
You can show this is true by cutting up the two smaller squares and rearranging the pieces to exactly fill the big one — no gaps, no overlaps. That's the core of most visual proofs of this result. The algebra follows from the geometry.
This only holds when the angle between sides $a$ and $b$ is exactly 90°. Change that angle and the relationship changes — that's what trigonometry (the next chapter) is about.

### 16.2.5. Volume of a rectangular prism

A rectangular prism (a box shape) has length $l$, width $w$, and height $h$.

$$V = l \times w \times h$$

Read it as: length times width times height. A box 10 cm long, 5 cm wide, and 3 cm tall has volume $10 \times 5 \times 3 = 150$ cm³.

A useful way to think about it: the base area is $l \times w$, and you're stacking that base up through height $h$. Volume is always base area times height for any prism.

This means volume is built from area extended through a third dimension.

### 16.2.6. Volume of a cylinder

A cylinder is a circle stacked up through a height. The base is a circle of radius $r$, and the height is $h$.

$$V = \pi r^2 h$$

Read it as: pi times radius squared times height. The $\pi r^2$ part is just the area of the circular base; multiplying by $h$ stacks it up.

A can of drink is a cylinder. A standard 330 ml can has a radius of about 3.3 cm and a height of about 9.6 cm:

$$V = \pi \times 3.3^2 \times 9.6 = \pi \times 10.89 \times 9.6 \approx 328 \text{ cm}^3$$

Close to 330 ml (330 cm³). The small difference is rounding on the actual dimensions.

This means a cylinder is just a circular base repeated through a height.

### 16.2.7. Units and unit conversion

Measurements only make sense with their units attached. 15 is not an area — 15 $m^2$ is.

When you multiply lengths, the units multiply too:

$$3 \text{ m} \times 4 \text{ m} = 12 \text{ m}^2$$

$$2 \text{ cm} \times 5 \text{ cm} \times 3 \text{ cm} = 30 \text{ cm}^3$$

**Converting between units:** multiply by a conversion factor equal to 1. For example, 1 m = 100 cm, so $\frac{100 \text{ cm}}{1 \text{ m}} = 1$.

$$5 \text{ m} = 5 \times 100 \text{ cm} = 500 \text{ cm}$$

For areas, the conversion factor gets squared:

$$1 \text{ m}^2 = (100 \text{ cm})^2 = 10{,}000 \text{ cm}^2$$

So $3 \text{ m}^2 = 3 \times 10{,}000 = 30{,}000 \text{ cm}^2$.

Why squared? A square metre is a square that is 100 cm wide and 100 cm tall — that's $100 \times 100 = 10{,}000$ square centimetres. The conversion factor gets multiplied by itself once for each dimension you're measuring.

For volumes, it gets cubed:

$$1 \text{ m}^3 = (100)^3 \text{ cm}^3 = 1{,}000{,}000 \text{ cm}^3$$

A cubic metre is a cube 100 cm × 100 cm × 100 cm, so the conversion factor multiplies by itself three times. This trips people up. One square metre is not a hundred square centimetres — it's ten thousand. The number gets big fast because you're covering a two-dimensional (or three-dimensional) space.

This means unit conversions must match the dimension of the quantity: length, area, or volume.

## 16.3. Worked examples

**Example 1 (sci) — Area of a circular clock face.**

A clock has a circular face with a diameter of 20 cm. What is the area of the face?

Diameter = 20 cm, so radius = 10 cm.

$$A = \pi r^2 = \pi \times 10^2 = 100\pi \approx 314 \text{ cm}^2$$

The clock face covers approximately 314 cm² — about the same as a standard sheet of A4 paper turned on its side. Doubling the diameter would quadruple the area, because radius is squared in the formula.

This means circle area changes quickly when the radius changes.

---

**Example 2 (eng) — Length of a ramp.**

A skateboard ramp rises 1.2 m vertically and runs 1.6 m horizontally along the ground. The ramp surface is the hypotenuse of a right triangle. How long is the ramp surface?

$$c = \sqrt{a^2 + b^2} = \sqrt{1.2^2 + 1.6^2} = \sqrt{1.44 + 2.56} = \sqrt{4.00} = 2.0 \text{ m}$$

The ramp surface is exactly 2 m long.

*Note: 1.2, 1.6, 2.0 is a scaled version of the 3-4-5 right triangle (multiply each by 0.4). When you see a clean answer, check whether it's a Pythagorean triple.*

This means familiar right-triangle patterns can help you check the arithmetic.

---

**Example 3 (comp) — Screen area from diagonal and aspect ratio.**

A phone screen has a 6.1-inch diagonal. The screen is 16:9 aspect ratio (16 units wide for every 9 units tall). What is the actual screen area?

One approach: call the width $16k$ and height $9k$, where $k$ is a scaling factor that sets the actual size. The Pythagorean theorem then gives an equation in $k$:

$$(16k)^2 + (9k)^2 = 6.1^2$$

$$256k^2 + 81k^2 = 37.21$$

$$337k^2 = 37.21$$

$$k^2 = \frac{37.21}{337} \approx 0.11041$$

$$k \approx 0.3323 \text{ inches}$$

Width = $16 \times 0.3323 \approx 5.32$ inches. Height = $9 \times 0.3323 \approx 2.99$ inches.

Area = $5.32 \times 2.99 \approx 15.9$ square inches.

The 6.1-inch measurement is the diagonal — the actual viewing area is about 15.9 square inches, which is roughly the size of a playing card.

This means a diagonal measurement alone does not tell you the usable area.

---

**Example 4 (geo) — Comparing two screen sizes.**

You're deciding between a phone screen and a tablet screen. The phone screen measures 6 cm by 12 cm on the poster. The tablet screen is shown on a second poster at double the linear scale — so every centimetre on the first poster represents 2 cm on the second. How does the tablet screen area compare to the phone screen area, and what is each real area on the posters?

**Phone screen area:** $6 \times 12 = 72$ cm² on the poster.

**Tablet screen on the second poster:** doubling the linear scale means each dimension doubles.

Width = $2 \times 6 = 12$ cm. Height = $2 \times 12 = 24$ cm.

Tablet screen area = $12 \times 24 = 288$ cm² on the second poster.

$288 \div 72 = 4$ — the tablet poster area is four times the phone poster area, even though only the scale doubled. That's because area scales as the square of the linear scale: $2^2 = 4$.

This means changing length scale changes area more dramatically than it first appears.

## 16.4. Where this goes

This chapter gives us formulas for size and shape when the key measurements are lengths, radii, heights, and right angles.

The next chapter in this volume is **trigonometry** (Vol 3, Ch 2). Trigonometry extends the geometry here by handling triangles where you don't know the right angle — or where you need to find angles as well as lengths. The Pythagorean theorem handles one specific case (right triangles, lengths only); trigonometry handles the general case using the sine, cosine, and tangent ratios. Every oscillating system — sound, light, signals, tides — is described using trigonometry.

Further out, the measurement ideas here feed directly into **functions and relations** (Vol 3, Ch 4). Area and volume formulas are functions: plug in the radius, get out the area. Once you're working with functions, you can ask questions like "how fast does the area change as the radius grows?" — and that question leads to calculus.

**Where this shows up**

- A structural engineer calculating how much steel plate is needed for a bridge deck is computing surface area.
- A physicist calculating fluid pressure on a pipe cross-section is computing circular area.
- A GIS analyst measuring a protected forest region from a satellite image is computing polygon areas at scale.
- A game developer checking whether two sprites have collided is comparing rectangular bounding boxes — area and distance calculations happening thousands of times per second.
- A packaging designer minimising the amount of cardboard in a cereal box is minimising surface area for a fixed volume.

The formulas are always the same. What changes is what you're measuring and why it matters.

## 16.5. Exercises

These are puzzles. Each one has a clean answer, but the interesting part is deciding which formula to use and what counts as the base, height, radius, or leg.

1. A rectangular phone screen is 13.8 cm wide and 7.0 cm tall. What is its area? What is its diagonal length (to one decimal place)?

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
```

```
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

```
//| echo: false
makeStepperHTML(1, [
  { op: "Find the area", eq: "A = l \\times w = 13.8 \\times 7.0 = 96.6 \\text{ cm}^2", note: nul
```

```
  { op: "Write the Pythagorean theorem for the diagonal", eq: "d = \\sqrt{l^2 + w^2}", note: "The di
  { op: "Substitute values", eq: "d = \\sqrt{13.8^2 + 7.0^2} = \\sqrt{190.44 + 49.00} = \\sqrt{239.4
  { op: "Calculate", eq: "d = 15.5 \\text{ cm}", note: null },
  { op: "Check units", eq: "\\text{Area: } 96.6 \\text{ cm}^2 \\quad \\text{Diagonal: } 15.5 \\text
])
```

2. A circular pizza has a diameter of 30 cm. What is its area (leave your answer in terms of $\pi$, then give a decimal approximation to the nearest whole number)?

```
//| echo: false
makeStepperHTML(2, [
  { op: "Find the radius from the diameter", eq: "r = \\frac{d}{2} = \\frac{30}{2} = 15 \\text{ cm}"
  { op: "Write the area formula", eq: "A = \\pi r^2", note: null },
  { op: "Substitute", eq: "A = \\pi \\times 15^2 = 225\\pi \\text{ cm}^2", note: null },
  { op: "Decimal approximation", eq: "A \\approx 225 \\times 3.14159 \\approx 707 \\text{ cm}^2", no
  { op: "Check units", eq: "A = 225\\pi \\approx 707 \\text{ cm}^2", note: null }
])
```

3. A ramp rises 0.9 m vertically over a horizontal distance of 1.2 m. How long is the sloped surface? (Give your answer to one decimal place.)

```
//| echo: false
makeStepperHTML(3, [
  { op: "Identify the right triangle: legs are the vertical rise and horizontal run, hypotenuse is t
  { op: "Substitute values", eq: "c = \\sqrt{0.9^2 + 1.2^2} = \\sqrt{0.81 + 1.44} = \\sqrt{2.25}", n
  { op: "Calculate", eq: "c = 1.5 \\text{ m}", note: "0.9, 1.2, 1.5 is a scaled 3-4-5 triple (each s
  { op: "Check units", eq: "\\text{Slope length} = 1.5 \\text{ m}", note: null }
])
```

4. A cylindrical water bottle has a radius of 4 cm and a height of 20 cm. What is its volume in $cm^3$? What is its capacity in millilitres? (1 $cm^3$ = 1 ml.)

```
//| echo: false
makeStepperHTML(4, [
  { op: "Write the volume formula for a cylinder", eq: "V = \\pi r^2 h", note: null },
  { op: "Substitute values", eq: "V = \\pi \\times 4^2 \\times 20 = \\pi \\times 16 \\times 20 = 320
  { op: "Decimal approximation", eq: "V \\approx 320 \\times 3.14159 \\approx 1005 \\text{ cm}^3", n
  { op: "Convert to millilitres", eq: "1005 \\text{ cm}^3 = 1005 \\text{ ml} \\approx 1 \\text{ litr
  { op: "Check units", eq: "V = 320\\pi \\approx 1005 \\text{ ml}", note: null }
])
```

5. A triangular garden bed has a base of 5 m and a perpendicular height of 3.6 m. What area of soil does it cover? If soil costs $12 per $m^2$, what is the total cost?

```
//| echo: false
makeStepperHTML(5, [
  { op: "Write the area formula for a triangle", eq: "A = \\frac{1}{2} \\times b \\times h", note
  { op: "Substitute values", eq: "A = \\frac{1}{2} \\times 5 \\times 3.6 = \\frac{18}{2} = 9 \\te
  { op: "Calculate the cost", eq: "\\text{Cost} = 9 \\times 12 = \\$108", note: null },
  { op: "Check units", eq: "\\text{Area} = 9 \\text{ m}^2, \\quad \\text{Cost} = \\$108", note: n
])
```

6. A rectangular swimming pool is 25 m long and 10 m wide. It has a uniform depth of 1.8 m. How many cubic metres of water does it hold? Give your answer in litres. (1 $m^3$ = 1000 litres.)

```
//| echo: false
makeStepperHTML(6, [
  { op: "Write the volume formula for a rectangular prism", eq: "V = l \\times w \\times h", note
  { op: "Substitute values", eq: "V = 25 \\times 10 \\times 1.8 = 450 \\text{ m}^3", note: null }
  { op: "Convert to litres", eq: "450 \\text{ m}^3 \\times 1000 \\text{ litres/m}^3 = 450{,}000 \
  { op: "Check units", eq: "V = 450 \\text{ m}^3 = 450{,}000 \\text{ litres}", note: null }
])
```

# 17. Trigonometry

Angles and the ratios they fix

You're building a wheelchair ramp. The code says the angle can't exceed 4.8°. You know how high the step is. How long does the ramp need to be?

You're standing 30 metres from a building, looking up at the roof. You want to know how tall the building is, but you can't climb it. You can measure the angle you're looking up at. Is that enough?

Your phone screen is 15.6 cm tall and 7.2 cm wide. What's the diagonal? Easy — Pythagoras. But what's the angle in the corner? That's where trig comes in.

Each of these situations has a right triangle in it somewhere. You know some sides and angles, and you want to find others. Trigonometry gives you three ratios that connect them.

## 17.1. What the notation is saying

Start with a right triangle. One angle is exactly 90°. Pick one of the other angles and call it $\theta$ (the Greek letter *theta* — just a name for "the angle you're working with").

Now name the three sides relative to $\theta$:

- The **hypotenuse** is the longest side, always opposite the right angle.
- The **opposite** side is the one directly across the triangle from $\theta$ — the side $\theta$ "faces".
- The **adjacent** side is the one next to $\theta$ that isn't the hypotenuse — the side $\theta$ "leans against".

This means the names opposite and adjacent can change if you choose a different angle.

The three trig ratios are:

$$\sin(\theta) = \frac{\text{opposite}}{\text{hypotenuse}}$$

Read it: "the sine of theta equals opposite over hypotenuse."

$$\cos(\theta) = \frac{\text{adjacent}}{\text{hypotenuse}}$$

Read it: "the cosine of theta equals adjacent over hypotenuse."

$$\tan(\theta) = \frac{\text{opposite}}{\text{adjacent}}$$

Read it: "the tangent of theta equals opposite over adjacent."

The memory aid is **SOH-CAH-TOA**: Sine = Opposite/Hypotenuse, Cosine = Adjacent/Hypotenuse, Tangent = Opposite/Adjacent. It's a mnemonic, not mathematics — use it until you don't need it any more.

This means the real task is not memorising letters. It is identifying which sides belong in the ratio for the angle you chose.

```
//| echo: false
viewof triAngle = Inputs.range([5, 85], {
  value: 35,
  step: 1,
  label: "Angle θ (degrees)"
})
```

```
//| echo: false
viewof triHyp = Inputs.range([1, 10], {
  value: 6,
  step: 0.5,
  label: "Hypotenuse length (units)"
})
```

```
//| echo: false
{
  const deg = triAngle;
  const hyp = triHyp;
  const rad = deg * Math.PI / 180;
  const opp = hyp * Math.sin(rad);
  const adj = hyp * Math.cos(rad);

  const scale = 30;
  const svgW = 520, svgH = 300;
  const margin = 50;

  // Triangle: right angle at bottom-left
  const ox = margin, oy = svgH - margin - 20;
  const ax = ox + adj * scale, ay = oy;      // bottom-right
  const bx = ox, by = oy - opp * scale;      // top-left

  const svg = d3.create("svg")
    .attr("width", svgW)
    .attr("height", svgH)
    .style("font-family", "inherit");
```

```
// Triangle fill
svg.append("polygon")
  .attr("points", `${ox},${oy} ${ax},${ay} ${bx},${by}`)
  .attr("fill", "#f0f9ff")
  .attr("stroke", "#0369a1")
  .attr("stroke-width", 2);

// Right-angle marker
const sq = 10;
svg.append("rect")
  .attr("x", ox).attr("y", oy - sq)
  .attr("width", sq).attr("height", sq)
  .attr("fill", "none").attr("stroke", "#374151").attr("stroke-width", 1.5);

// Angle arc at bottom-right
const arcR = 22;
const arcPath = d3.arc()({
  innerRadius: 0, outerRadius: arcR,
  startAngle: Math.PI,
  endAngle: Math.PI + rad
});
svg.append("path")
  .attr("d", arcPath)
  .attr("transform", `translate(${ax},${ay}) rotate(-90)`)
  .attr("fill", "#fde68a")
  .attr("stroke", "#d97706")
  .attr("stroke-width", 1);

svg.append("text")
  .attr("x", ax - arcR - 14).attr("y", ay - 8)
  .attr("font-size", "13px").attr("fill", "#92400e")
  .text(`θ=${deg}°`);

// Side labels
// Hypotenuse midpoint
const hmx = (ax + bx) / 2 + 14, hmy = (ay + by) / 2;
svg.append("text")
  .attr("x", hmx).attr("y", hmy)
  .attr("font-size", "12px").attr("fill", "#0369a1").attr("font-weight", "600")
  .text(`hyp = ${hyp.toFixed(1)}`);

// Adjacent (bottom)
svg.append("text")
  .attr("x", ox + adj * scale / 2).attr("y", oy + 18)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#059669")
  .text(`adj = ${adj.toFixed(2)}`);
```

```
  // Opposite (left side)
  svg.append("text")
    .attr("x", ox - 14).attr("y", oy - opp * scale / 2)
    .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#dc2626")
    .text(`opp = ${opp.toFixed(2)}`);

  // Ratios panel
  const px = 310, py = 40;
  svg.append("rect")
    .attr("x", px - 10).attr("y", py - 20)
    .attr("width", 190).attr("height", 140)
    .attr("fill", "#f8fafc").attr("stroke", "#e2e8f0").attr("rx", 6);

  svg.append("text").attr("x", px).attr("y", py)
    .attr("font-size", "13px").attr("font-weight", "600").attr("fill", "#374151")
    .text("Ratios (fixed by ⍰ alone)");

  svg.append("text").attr("x", px).attr("y", py + 26)
    .attr("font-size", "13px").attr("fill", "#374151")
    .text(`sin(${deg}°) = opp/hyp`);
  svg.append("text").attr("x", px).attr("y", py + 44)
    .attr("font-size", "13px").attr("fill", "#dc2626")
    .text(`= ${Math.sin(rad).toFixed(4)}`);

  svg.append("text").attr("x", px).attr("y", py + 66)
    .attr("font-size", "13px").attr("fill", "#374151")
    .text(`cos(${deg}°) = adj/hyp`);
  svg.append("text").attr("x", px).attr("y", py + 84)
    .attr("font-size", "13px").attr("fill", "#059669")
    .text(`= ${Math.cos(rad).toFixed(4)}`);

  svg.append("text").attr("x", px).attr("y", py + 106)
    .attr("font-size", "13px").attr("fill", "#374151")
    .text(`tan(${deg}°) = opp/adj`);
  svg.append("text").attr("x", px).attr("y", py + 124)
    .attr("font-size", "13px").attr("fill", "#7c3aed")
    .text(`= ${Math.tan(rad).toFixed(4)}`);

  return svg.node();
}
```

## 17.2. The method

### 17.2.1. Finding an unknown side

You know an angle and one side. You want another side. Pick the ratio that connects what you know to what you want.

This means trig starts with a side-labeling decision, not with a calculator.

**Example.** A ramp rises 0.5 m. The angle at the base is 4°. How long is the ramp surface?

The 0.5 m is the side opposite the 4° angle. The ramp surface is the hypotenuse. The ratio connecting opposite and hypotenuse is sine:

$$\sin(4^\circ) = \frac{0.5}{\text{hyp}}$$

Rearrange step by step:

$$\sin(4^\circ) \times \text{hyp} = 0.5 \quad \text{[multiply both sides by hyp]}$$

$$\text{hyp} = \frac{0.5}{\sin(4^\circ)} \quad \text{[divide both sides by } \sin(4^\circ)\text{]}$$

$$\text{hyp} \approx \frac{0.5}{0.0698} \approx 7.2 \text{ m}$$

The ramp needs to be about 7.2 m long.

This means a very small angle needs a much longer ramp than people often expect.

### 17.2.2. Finding an unknown angle

You know two sides. You want the angle between them. Use the inverse trig function.

$\sin^{-1}$ reads as "inverse sine" or "arcsine". It asks: what angle has this sine value? If $\sin(\theta) = 0.5$, then $\theta = \sin^{-1}(0.5) = 30^\circ$. The $^{-1}$ notation means "undo the sine" — it does not mean "one over sine."

This means inverse trig functions reverse the direction of the question: from ratio back to angle.

Similarly, $\cos^{-1}$ and $\tan^{-1}$ undo the cosine and tangent respectively. All three inverse functions work the same way: give them a ratio, get back an angle.

**Example.** Your phone screen is 15.6 cm tall and 7.2 cm wide. What angle does the diagonal make with the long side?

The two known sides are 15.6 cm (adjacent to the angle at the bottom-left corner) and 7.2 cm (opposite). The ratio connecting opposite and adjacent is tangent:

$$\tan(\theta) = \frac{7.2}{15.6} \approx 0.4615$$

Apply inverse tangent to both sides:

$$\theta = \tan^{-1}(0.4615) \approx 24.8^\circ$$

The diagonal meets the long side at about 25°.

This means once you know a side ratio, the angle is determined.

> **Why this works**
> The ratios are fixed by the angle alone — not by how big the triangle is. Any right triangle with a 30° angle has the same ratio of opposite to hypotenuse: exactly 0.5. Scale the triangle up by ten times and both sides get ten times longer, but the ratio stays the same.
> This is why the ratios are useful. They depend only on the angle, so you can look up $\sin(30^\circ) = 0.5$ once and use it for any triangle containing a 30° angle, whatever its size.

## 17.3. Worked examples

### Example 1 — Finding the height of a building (geography/surveying)

A surveyor stands 40 m from the base of a radio mast. She measures the angle of elevation to the top as 62°. How tall is the mast?

The 40 m is the side adjacent to the 62° angle. The mast height is the opposite side. The ratio connecting adjacent and opposite is tangent:

$$\tan(62^\circ) = \frac{\text{height}}{40}$$

Multiply both sides by 40:

$$\text{height} = 40 \times \tan(62^\circ)$$

$\tan(62^\circ) \approx 1.8807$, so:

$$\text{height} = 40 \times 1.8807 \approx 75.2 \text{ m}$$

The mast is approximately 75 m tall. Check: $\tan^{-1}(75.2/40) = \tan^{-1}(1.880) \approx 62^\circ$. Yes.

This means an angle measurement can replace a direct height measurement when the object is hard to reach.

---

**Example 2 — Rope from a fence post (geometry/practical)**

A rope is tied from the top of a fence post to a peg in the ground. The post is vertical and the rope makes an angle of 35° with the post. The rope has a tension of 80 N — the force acting along the rope. What is the component of that tension pulling horizontally away from the post?

The 80 N tension acts along the rope — that's the hypotenuse of a right triangle. The angle at the top of the post is 35°. The horizontal pull is the side opposite that angle:

$$\sin(35^\circ) = \frac{\text{horizontal component}}{80}$$

$$\text{horizontal component} = 80 \times \sin(35^\circ)$$

$\sin(35^\circ) \approx 0.5736$, so:

$$\text{horizontal component} = 80 \times 0.5736 \approx 45.9 \text{ N}$$

The peg must hold against about 46 N of horizontal pull.

This means trig is useful for splitting one force into its components.

---

**Example 3 — Finding the ramp angle for a target slope (science/engineering)**

A physiotherapy exercise ramp needs a slope of 1 in 8 — for every 8 cm of horizontal run, it rises 1 cm. What angle does this correspond to?

The slope ratio gives you opposite over adjacent: $\frac{1}{8} = 0.125$. That's the tangent of the angle:

$$\tan(\theta) = \frac{1}{8} = 0.125$$

$$\theta = \tan^{-1}(0.125) \approx 7.1^\circ$$

The ramp makes an angle of about 7° with the floor.

This means a simple slope ratio can be turned into an angle by tangent.

---

**Example 4 — Rotating a point in 2D (computing/game development)**

A game object is at position $(5, 0)$ — five units to the right of the origin, on the horizontal axis. The game rotates everything $40^\circ$ anticlockwise. Where does the object end up?

The object starts at distance 5 from the origin (its "hypotenuse" from the rotation centre). After a $40^\circ$ rotation, the new $x$-coordinate is the adjacent side of the $40^\circ$ angle, and the new $y$-coordinate is the opposite side:

$$x' = 5 \times \cos(40^\circ) \approx 5 \times 0.7660 \approx 3.83$$

$$y' = 5 \times \sin(40^\circ) \approx 5 \times 0.6428 \approx 3.21$$

The object moves to approximately $(3.83,\ 3.21)$. This is the core of every 2D rotation in a game engine.

This means trig can convert a distance-and-angle description into coordinates.

```
//| echo: false
viewof elevDist = Inputs.range([5, 30], {
  value: 15,
  step: 1,
  label: "Horizontal distance from building (m)"
})
```

```
//| echo: false
{
  const dist = elevDist;
  const buildingH = 20; // fixed metres
  const angleRad = Math.atan(buildingH / dist);
  const angleDeg = angleRad * 180 / Math.PI;

  const svgW = 500, svgH = 300;
  const margin = 40;
  const scale = 7; // px per metre

  // Scene coordinates
  const groundY = svgH - margin - 20;
  const buildingX = svgW - margin - 60;
  const observerX = buildingX - dist * scale;
  const buildingTopY = groundY - buildingH * scale;

  const svg = d3.create("svg")
    .attr("width", svgW)
    .attr("height", svgH)
    .style("font-family", "inherit");

  // Ground line
```

```
svg.append("line")
  .attr("x1", margin).attr("y1", groundY)
  .attr("x2", svgW - margin).attr("y2", groundY)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1.5);

// Building (vertical line)
svg.append("line")
  .attr("x1", buildingX).attr("y1", groundY)
  .attr("x2", buildingX).attr("y2", buildingTopY)
  .attr("stroke", "#374151").attr("stroke-width", 4);

// Building label
svg.append("text")
  .attr("x", buildingX + 8).attr("y", (groundY + buildingTopY) / 2)
  .attr("font-size", "12px").attr("fill", "#374151")
  .text("20 m");

// Observer dot
svg.append("circle")
  .attr("cx", observerX).attr("cy", groundY)
  .attr("r", 6)
  .attr("fill", "#f59e0b").attr("stroke", "#d97706").attr("stroke-width", 1.5);

svg.append("text")
  .attr("x", observerX).attr("y", groundY + 18)
  .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#374151")
  .text("observer");

// Hypotenuse line
svg.append("line")
  .attr("x1", observerX).attr("y1", groundY)
  .attr("x2", buildingX).attr("y2", buildingTopY)
  .attr("stroke", "#2563eb").attr("stroke-width", 2)
  .attr("stroke-dasharray", "5,3");

// Horizontal distance label
svg.append("line")
  .attr("x1", observerX).attr("y1", groundY + 28)
  .attr("x2", buildingX).attr("y2", groundY + 28)
  .attr("stroke", "#6b7280").attr("stroke-width", 1)
  .attr("marker-end", "url(#arrow)");

svg.append("text")
  .attr("x", (observerX + buildingX) / 2).attr("y", groundY + 42)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#6b7280")
  .text(`d = ${dist} m`);
```

```
// Angle arc at observer
const arcR = 28;
const arcPath = d3.arc()({
  innerRadius: 0, outerRadius: arcR,
  startAngle: -Math.PI / 2,
  endAngle: -Math.PI / 2 + angleRad
});
svg.append("path")
  .attr("d", arcPath)
  .attr("transform", `translate(${observerX},${groundY}) rotate(${90 - angleDeg})`)
  .attr("fill", "#fde68a").attr("stroke", "#d97706").attr("stroke-width", 1);

// Opposite label (building height)
svg.append("text")
  .attr("x", buildingX - 10).attr("y", groundY - buildingH * scale / 2)
  .attr("text-anchor", "end").attr("font-size", "12px").attr("fill", "#dc2626")
  .text("opp = 20 m");

// Adjacent label
svg.append("text")
  .attr("x", (observerX + buildingX) / 2).attr("y", groundY - 8)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#059669")
  .text(`adj = ${dist} m`);

// Results panel
const px = margin, py = margin;
svg.append("rect")
  .attr("x", px - 4).attr("y", py - 18)
  .attr("width", 210).attr("height", 80)
  .attr("fill", "#f8fafc").attr("stroke", "#e2e8f0").attr("rx", 6);

svg.append("text").attr("x", px).attr("y", py)
  .attr("font-size", "13px").attr("font-weight", "600").attr("fill", "#374151")
  .text("Angle of elevation");

svg.append("text").attr("x", px).attr("y", py + 22)
  .attr("font-size", "13px").attr("fill", "#374151")
  .text(`θ = tan⁻¹(20 / ${dist})`);

svg.append("text").attr("x", px).attr("y", py + 42)
  .attr("font-size", "14px").attr("fill", "#2563eb").attr("font-weight", "600")
  .text(`θ = ${angleDeg.toFixed(1)}°`);

svg.append("text").attr("x", px).attr("y", py + 60)
  .attr("font-size", "12px").attr("fill", "#6b7280")
  .text(`(tan⁻¹(${(buildingH/dist).toFixed(3)}))`);
```

```
  return svg.node();
}
```

## 17.4. Where this goes

This chapter extends geometry by turning angles into usable numerical relationships.

The three ratios in this chapter are functions — you put in an angle and get out a number. **Volume 3, Chapter 4** (Functions and relations) gives that idea its full treatment: a function is a rule that maps one value to another, and sine, cosine, and tangent are the first examples most people meet of functions that are not straight lines. Once you see trig through the function lens you can ask new questions: what does the graph look like? Where is it increasing? Does it repeat?

It repeats — and that turns out to matter enormously. **Differential calculus** (Volume 5) uses sine and cosine as its central worked examples because they are the simplest functions whose rate of change is also a trig function. The derivative of sin is cos; the derivative of cos is $-\sin$. That self-referential property is why trig is indispensable in physics and engineering, and it's the first real glimpse of how calculus and trigonometry are connected at a deep level.

**Where this shows up**

- A structural engineer resolves a cable force into horizontal and vertical components before checking whether a joint will hold.
- A physicist writes the equation of a pendulum's motion using sine — the angle oscillates like a sine wave over time.
- A navigator uses bearing angles and the sine rule to triangulate a ship's position from two known landmarks.
- A game developer rotates sprites, aims projectiles, and swings camera views using sin and cos applied to coordinates.
- A sound engineer represents audio waveforms as sums of sine waves — the starting point of Fourier analysis.

## 17.5. Exercises

These are puzzles. Each one has a clean answer. The work is in identifying which sides and which ratio to use before you reach for the calculator.

---

**1.** A ski slope is 800 m long and the vertical drop is 240 m. What angle does the slope make with the horizontal? Give your answer to one decimal place.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
```

```
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

```
//| echo: false
makeStepperHTML(1, [
  { op: "Identify opposite, adjacent, hypotenuse", eq: "\\text{hypotenuse} = 800\\text{ m},\\quad
  { op: "Choose the ratio", eq: "\\sin(\\theta) = \\frac{\\text{opposite}}{\\text{hypotenuse}} =
  { op: "Simplify the fraction", eq: "\\sin(\\theta) = 0.3", note: null },
  { op: "Apply inverse sine", eq: "\\theta = \\sin^{-1}(0.3)", note: "Inverse sine asks: what ang
  { op: "Calculate", eq: "\\theta \\approx 17.5°", note: null },
  { op: "Check", eq: "\\sin(17.5°) \\approx 0.3007 \\approx 0.3 \\checkmark", note: null }
])
```

---

**2.** A ladder 6 m long leans against a wall. It makes an angle of 72° with the ground. How high up the wall does the top of the ladder reach? Give your answer to two decimal places.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Identify opposite, adjacent, hypotenuse", eq: "\\text{hypotenuse} = 6\\text{ m},\\quad \
  { op: "Choose the ratio", eq: "\\sin(72°) = \\frac{\\text{wall height}}{6}", note: null },
  { op: "Rearrange", eq: "\\text{wall height} = 6 \\times \\sin(72°)", note: null },
  { op: "Calculate", eq: "\\text{wall height} = 6 \\times 0.9511 \\approx 5.71\\text{ m}", note:
  { op: "Check", eq: "\\sin^{-1}\\!\\left(\\frac{5.71}{6}\\right) = \\sin^{-1}(0.9517) \\approx 7
])
```

---

**3.** A phone screen has a diagonal of 16.5 cm and the screen is 8.0 cm wide. What angle does the diagonal make with the bottom edge of the screen?

```
//| echo: false
makeStepperHTML(3, [
  { op: "Identify opposite, adjacent, hypotenuse", eq: "\\text{hypotenuse} = 16.5\\text{ cm},\\quad
  { op: "Choose the ratio", eq: "\\cos(\\theta) = \\frac{\\text{adjacent}}{\\text{hypotenuse}} = \\f
  { op: "Compute the ratio", eq: "\\cos(\\theta) \\approx 0.4848", note: null },
  { op: "Apply inverse cosine", eq: "\\theta = \\cos^{-1}(0.4848) \\approx 61.0°", note: null },
  { op: "Check", eq: "\\cos(61°) \\approx 0.4848,\\quad 16.5 \\times 0.4848 \\approx 8.0\\text{ cm}
])
```

---

**4.** A solar panel is mounted on a roof. To maximise energy collection in Alberta, the panel should face the sun at an angle of elevation of 53°. The vertical rise from the base of the panel to its top edge is 1.2 m. How long is the panel surface?

```
//| echo: false
makeStepperHTML(4, [
  { op: "Identify opposite, adjacent, hypotenuse", eq: "\\theta = 53°,\\quad \\text{opposite} = 1.2\
  { op: "Choose the ratio", eq: "\\sin(53°) = \\frac{1.2}{\\text{panel length}}", note: null },
  { op: "Rearrange", eq: "\\text{panel length} = \\frac{1.2}{\\sin(53°)}", note: null },
  { op: "Calculate", eq: "\\text{panel length} = \\frac{1.2}{0.7986} \\approx 1.50\\text{ m}", note
  { op: "Check", eq: "\\sin(53°) \\times 1.50 \\approx 0.7986 \\times 1.50 \\approx 1.20\\text{ m} \
])
```

---

**5.** A zip line descends from a platform 18 m high. The cable makes an angle of 35° with the horizontal. How long is the cable, and how far from the base of the platform does it end?

```
//| echo: false
makeStepperHTML(5, [
  { op: "Identify opposite, adjacent, hypotenuse", eq: "\\text{opposite} = 18\\text{ m (height)},\\c
  { op: "Find the cable length using sine", eq: "\\sin(35°) = \\frac{18}{\\text{cable}}", note: null
  { op: "Rearrange", eq: "\\text{cable} = \\frac{18}{\\sin(35°)} = \\frac{18}{0.5736} \\approx 31.4\
  { op: "Find the horizontal distance using cosine", eq: "\\cos(35°) = \\frac{\\text{horizontal}}{\\
  { op: "Calculate horizontal distance", eq: "\\text{horizontal} = 31.4 \\times 0.8192 \\approx 25.7
  { op: "Check using Pythagoras", eq: "\\sqrt{18^2 + 25.7^2} = \\sqrt{324 + 660.5} = \\sqrt{984.5} \
])
```

---

**6.** A ladder 5 m long leans against a wall. Its base is 1.8 m from the wall. (a) What angle does the ladder make with the ground? (b) How high up the wall does it reach?

```
//| echo: false
makeStepperHTML(6, [
  { op: "Identify the sides", eq: "\\text{hypotenuse} = 5\\text{ m},\\quad \\text{adjacent} = 1.8
  { op: "Find the angle using inverse cosine", eq: "\\cos(\\theta) = \\frac{1.8}{5} = 0.36", note
  { op: "Apply inverse cosine", eq: "\\theta = \\cos^{-1}(0.36) \\approx 68.9°", note: null },
  { op: "Find the height using sine", eq: "\\sin(68.9°) = \\frac{\\text{height}}{5}", note: null
  { op: "Calculate height", eq: "\\text{height} = 5 \\times \\sin(68.9°) \\approx 5 \\times 0.933
  { op: "Check using Pythagoras", eq: "\\sqrt{1.8^2 + 4.67^2} = \\sqrt{3.24 + 21.81} = \\sqrt{25.
])
```

# 18. Polynomials and factoring

When the curve is the point

You kick a ball straight up. For a moment it keeps going — then it slows, stops, and falls back down. The height isn't a straight line; it curves up and back. That curve has an equation.

You're fencing off a rectangular garden and you've got 40 metres of fencing. You want the biggest possible area. If you make it long and thin, the area shrinks. If you make it square, the area is as large as it can be. Finding that optimum — that maximum — is a polynomial problem.

Your school sells tickets to a performance. At a lower price more people buy, but revenue per ticket drops. At a higher price fewer people buy. Somewhere in the middle is the price that brings in the most money. That relationship between price and total revenue follows a curve — a polynomial.

Straight lines describe constant rates. Polynomials describe the rest.

## 18.1. What the notation is saying

Start with a contrast from earlier chapters. A line keeps the same steepness. A quadratic bends. The extra power of $x$ is what creates that bend.

Here is a polynomial:

$$ax^2 + bx + c$$

Read it aloud: *a times x-squared, plus b times x, plus c.*

The letters $a$, $b$, and $c$ are **coefficients** — the numbers multiplying each power of $x$. They're just fixed numbers; you'll swap them for actual values in any particular problem. The letter $x$ is the unknown — the variable you're either solving for, or letting run freely while you watch what the output does. When you write $y = ax^2 + bx + c$, $y$ is the output — the value the polynomial produces for each value of $x$.

This means the coefficients shape the curve, while $x$ moves along it.

Each piece — $ax^2$, $bx$, $c$ — is called a **term**. Count the terms and you know you have three. The $c$ term has no $x$ at all; it's a plain number, a constant.

The **degree** of a polynomial is the highest power of $x$ present. In $ax^2 + bx + c$ the highest power is 2, so the degree is 2. Degree-2 polynomials are called **quadratics**. Degree-1 polynomials — like $3x + 7$ — are the linear equations from Vol 2. Degree-3 polynomials are called cubics, and so on.

This means the degree tells you what kind of behaviour to expect.

Why does the $x^2$ term make the graph curve? Because $x^2$ grows faster than $x$. When $x = 2$, $x^2 = 4$. When $x = 10$, $x^2 = 100$. The squared term pulls the output away from the straight line. Take $a = 0$ and you're back to a line. Leave $a$ nonzero and the whole shape bends.

A specific example: $y = x^2 - 5x + 6$. Here $a = 1$, $b = -5$, $c = 6$. The negative $b$ means the straight-line part is pulling down, and the positive $c$ shifts the whole thing up by 6 units. The coefficient $a = 1$ means the parabola opens upward — a positive $a$ always curves upward, a negative $a$ always curves downward.

This means the sign of $a$ tells you whether the graph has a minimum or a maximum.

## 18.2. The method

### 18.2.1. Part a: Expanding

Before you can factor, it helps to know how to expand — to go from the factored form back to the full polynomial. This makes the reverse step (factoring) easier to recognise.

The distributive law says: multiply everything inside the brackets by everything outside.

$$3(x + 4) = 3x + 12$$

When you have two brackets multiplied together, you apply the same idea twice — every term in the first bracket multiplies every term in the second. This is sometimes called **FOIL** (First, Outer, Inner, Last), though the name matters less than the idea: don't miss any pair.

$$(x + 3)(x + 2)$$

- First terms: $x \cdot x = x^2$
- Outer terms: $x \cdot 2 = 2x$
- Inner terms: $3 \cdot x = 3x$
- Last terms: $3 \cdot 2 = 6$

Add them up: $x^2 + 2x + 3x + 6 = x^2 + 5x + 6$.

The pattern is worth noticing. With $(x + 3)(x + 2)$, the number in the middle (5) is the sum of 3 and 2. The constant term (6) is their product. That observation is what makes factoring work in reverse.

This means expanding and factoring are inverse ideas.

### 18.2.2. Part b: Factoring

Factoring means breaking a polynomial back into the brackets that produced it. There are three main methods; apply them in this order.

**Method 1: Common factor.** Check whether every term shares a factor you can pull out.

$$6x^2 + 9x = 3x(2x + 3)$$

The common factor here is $3x$. Dividing both terms by $3x$ gives the bracket. Always check for a common factor first — it simplifies what comes next.

This means factoring should start with the simplest shared structure.

**Method 2: Difference of squares.** When you have two perfect squares separated by a minus sign, it factors as:

$$a^2 - b^2 = (a + b)(a - b)$$

For example:

$$x^2 - 25 = (x + 5)(x - 5)$$

Check it by expanding: $(x + 5)(x - 5) = x^2 - 5x + 5x - 25 = x^2 - 25$. Yes.

This works because the middle terms cancel. It doesn't work if the sign between the squares is positive — $x^2 + 25$ doesn't factor over the real numbers.

This means the pattern needs both perfect squares and a minus sign.

**Method 3: Factoring a trinomial $ax^2 + bx + c$.** When $a = 1$, look for two numbers that multiply to $c$ and add to $b$.

$$x^2 - 5x + 6$$

You need two numbers that multiply to 6 and add to $-5$. Try: $-2$ and $-3$. Product: $(-2)(-3) = 6$. Sum: $(-2) + (-3) = -5$. Yes.

$$x^2 - 5x + 6 = (x - 2)(x - 3)$$

This means factoring a trinomial is a search for two numbers that fit both the sum and the product condition.

When $a \neq 1$, try the **grouping method** (also called the *ac* method). Multiply $a$ by $c$, find two numbers with that product and sum $b$, then split the middle term and factor in pairs.

$$2x^2 + 7x + 3$$

Multiply $a \cdot c = 2 \cdot 3 = 6$. Find two numbers that multiply to 6 and add to 7: those are 1 and 6.

Split the middle term:

$$2x^2 + x + 6x + 3$$

Factor in pairs:

$$x(2x + 1) + 3(2x + 1) = (x + 3)(2x + 1)$$

Both terms contain the factor $(2x + 1)$, so it can be pulled out — the same distributive law used in expanding, run backwards.

This means grouping turns a harder trinomial into a familiar common-factor problem.

Check by expanding: $(x + 3)(2x + 1) = 2x^2 + x + 6x + 3 = 2x^2 + 7x + 3$. Yes.

### 18.2.3. Part c: Solving quadratic equations

A **quadratic equation** is when you set a quadratic equal to zero:

$$ax^2 + bx + c = 0$$

There are two methods.

**Solving by factoring.** If you can factor the left side, you can use the fact that if two things multiply to zero, at least one of them must be zero.

$$x^2 - 5x + 6 = 0$$

$$(x - 2)(x - 3) = 0$$

Either $(x - 2) = 0$ or $(x - 3) = 0$. So $x = 2$ or $x = 3$.

Check both: $4 - 10 + 6 = 0$. Yes. $9 - 15 + 6 = 0$. Yes.

This means solving by factoring finds the x-values where the polynomial becomes zero.

**Solving with the quadratic formula.** Not every quadratic factors neatly. For any quadratic $ax^2+bx+c = 0$, the solutions are always:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Read it: *x equals negative b, plus or minus the square root of b-squared minus four-a-c, all over two-a.*

The ± sign means there are two candidate answers — one where you add the square root term, one where you subtract it (whether there are two, one, or no real answers depends on the discriminant — that's the next section).

The piece under the square root — $b^2 - 4ac$ — is called the **discriminant**, often written $\Delta$ (delta). It tells you how many solutions exist before you do any arithmetic:

- $\Delta > 0$: two real solutions (the parabola crosses the x-axis twice)
- $\Delta = 0$: one solution (the parabola just touches the x-axis at one point)
- $\Delta < 0$: no real solutions (the parabola doesn't reach the x-axis)

### 18.2.4. Part d: Reading the graph

Every quadratic $y = ax^2 + bx + c$ draws a **parabola** — a symmetric U-shape (or inverted U if $a < 0$).

The **roots** are where the parabola crosses the x-axis — the values of $x$ where $y = 0$. These are exactly the solutions to $ax^2 + bx + c = 0$. Factoring gives you the roots. The roots are the x-intercepts of the graph.

This means solving the equation and reading x-intercepts are the same task.

The **vertex** is the turning point — the very top of a downward parabola (the maximum) or the very bottom of an upward one (the minimum). The x-coordinate of the vertex is:

$$x_v = -\frac{b}{2a}$$

Read it: *negative b over two a.* Substitute this back into the equation to find the y-coordinate — the maximum or minimum value itself.

For $y = x^2 - 5x + 6$: $x_v = -(-5)/(2 \cdot 1) = 5/2 = 2.5$. Then $y = (2.5)^2 - 5(2.5) + 6 = 6.25 - 12.5 + 6 = -0.25$. The vertex is at $(2.5, -0.25)$ — the minimum point, halfway between the two roots at $x = 2$ and $x = 3$.

This means the vertex summarises the turning behaviour of the parabola.

```
//| echo: false
viewof qa = Inputs.range([-2, 2], { value: 1, step: 0.5, label: "a" })
```

```
//| echo: false
viewof qb = Inputs.range([-6, 6], { value: -5, step: 1, label: "b" })
```

```
//| echo: false
viewof qc = Inputs.range([-9, 9], { value: 6, step: 1, label: "c" })
```

```
//| echo: false
{
  // Guard: skip if a is too close to zero
  const a = Math.abs(qa) < 0.1 ? 0.5 : qa;
  const b = qb, c = qc;

  const svgW = 500, svgH = 360;
  const margin = { top: 50, right: 30, bottom: 40, left: 50 };
  const W = svgW - margin.left - margin.right;
```

```
const H = svgH - margin.top - margin.bottom;

const xMin = -6, xMax = 6;
const xs = d3.range(xMin, xMax + 0.05, 0.05);
const ys = xs.map(x => a * x * x + b * x + c);
const yMin = Math.min(...ys, -10), yMax = Math.max(...ys, 10);

const xScale = d3.scaleLinear([xMin, xMax], [0, W]);
const yScale = d3.scaleLinear([yMin, yMax], [H, 0]);

const disc = b * b - 4 * a * c;

// Roots
let roots = [];
if (disc > 0) {
  roots = [
    (-b + Math.sqrt(disc)) / (2 * a),
    (-b - Math.sqrt(disc)) / (2 * a)
  ].filter(r => r >= xMin && r <= xMax);
} else if (Math.abs(disc) < 1e-9) {
  const r = -b / (2 * a);
  if (r >= xMin && r <= xMax) roots = [r];
}

// Vertex
const vx = -b / (2 * a);
const vy = a * vx * vx + b * vx + c;

const svg = d3.create("svg")
  .attr("width", svgW).attr("height", svgH)
  .style("font-family", "inherit");

const g = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// Axes
if (yScale(0) >= 0 && yScale(0) <= H) {
  g.append("line")
    .attr("x1", 0).attr("y1", yScale(0))
    .attr("x2", W).attr("y2", yScale(0))
    .attr("stroke", "#9ca3af").attr("stroke-width", 1);
}
if (xScale(0) >= 0 && xScale(0) <= W) {
  g.append("line")
    .attr("x1", xScale(0)).attr("y1", 0)
    .attr("x2", xScale(0)).attr("y2", H)
    .attr("stroke", "#9ca3af").attr("stroke-width", 1);
```

```
}

// Grid lines
for (let xi = xMin; xi <= xMax; xi++) {
  g.append("line")
    .attr("x1", xScale(xi)).attr("y1", 0)
    .attr("x2", xScale(xi)).attr("y2", H)
    .attr("stroke", "#f3f4f6").attr("stroke-width", 1);
  g.append("text")
    .attr("x", xScale(xi)).attr("y", yScale(0) + 14)
    .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#9ca3af")
    .text(xi === 0 ? "" : xi);
}

// Curve
const line = d3.line()
  .x(x => xScale(x))
  .y(x => yScale(a * x * x + b * x + c))
  .curve(d3.curveCatmullRom);

g.append("path")
  .datum(xs)
  .attr("d", line)
  .attr("fill", "none")
  .attr("stroke", "#2563eb")
  .attr("stroke-width", 2.5);

// Roots
roots.forEach(r => {
  g.append("circle")
    .attr("cx", xScale(r)).attr("cy", yScale(0))
    .attr("r", 5)
    .attr("fill", "#dc2626").attr("stroke", "#fff").attr("stroke-width", 1.5);
});

// Vertex
if (vx >= xMin && vx <= xMax) {
  g.append("circle")
    .attr("cx", xScale(vx)).attr("cy", yScale(vy))
    .attr("r", 6)
    .attr("fill", "#f59e0b").attr("stroke", "#fff").attr("stroke-width", 1.5);
}

// "No real roots" label
if (disc < 0) {
  g.append("text")
    .attr("x", W / 2).attr("y", H / 2)
```

```
      .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#dc2626")
      .text("no real roots (□ < 0)");
  }

  // Info panel at top
  const eqStr = `y = ${a}x² ${b >= 0 ? "+" : ""}${b}x ${c >= 0 ? "+" : ""}${c}`;
  svg.append("text")
    .attr("x", margin.left).attr("y", 22)
    .attr("font-size", "13px").attr("font-weight", "600").attr("fill", "#111827")
    .text(eqStr);

  svg.append("text")
    .attr("x", margin.left + 220).attr("y", 22)
    .attr("font-size", "13px").attr("fill", disc >= 0 ? "#059669" : "#dc2626")
    .text(`□ = ${disc.toFixed(2)}  (${disc > 0 ? "2 roots" : disc === 0 ? "1 root" : "no real roots"

  // Vertex label
  if (vx >= xMin && vx <= xMax) {
    svg.append("text")
      .attr("x", margin.left + xScale(vx) + 8)
      .attr("y", margin.top + yScale(vy) - 4)
      .attr("font-size", "11px").attr("fill", "#d97706")
      .text(`vertex (${vx.toFixed(2)}, ${vy.toFixed(2)})`);
  }

  return svg.node();
}
```

## 18.3. Why this works

**Where the quadratic formula comes from**

The formula isn't magic — it's derived by solving $ax^2 + bx + c = 0$ directly, using a technique called **completing the square**.

The key move: rewrite $x^2 + \frac{b}{a}x$ as a perfect square minus a correction term.

$$x^2 + \frac{b}{a}x = \left(x + \frac{b}{2a}\right)^2 - \frac{b^2}{4a^2}$$

Substitute this into $ax^2 + bx + c = 0$, divide through by $a$, then isolate the squared bracket:

$$\left(x + \frac{b}{2a}\right)^2 = \frac{b^2 - 4ac}{4a^2}$$

Take the square root of both sides (giving $\pm$), then subtract $\frac{b}{2a}$:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

That's it. The formula is just completing the square, done once in general so you don't have to do it every time.

## 18.4. Worked examples

### Example 1 (sci) — height of a thrown ball

A ball is thrown upward from ground level with an initial speed of 14 m/s. Ignoring air resistance, its height in metres after $t$ seconds is:

$$h(t) = 14t - 4.9t^2$$

When does it land?

Landing means $h = 0$:

$$14t - 4.9t^2 = 0$$

Factor out a common term of $t$:

$$t(14 - 4.9t) = 0$$

Either $t = 0$ (the moment it's thrown) or $14 - 4.9t = 0$. Solving the second:

$$4.9t = 14 \implies t = \frac{14}{4.9} \approx 2.86 \text{ s}$$

The ball lands after about 2.86 seconds.

The vertex — the highest point — is at $t = 14/(2 \times 4.9) = 14/9.8 \approx 1.43$ s, at height $h(1.43) = 14(1.43) - 4.9(1.43)^2 \approx 20.02 - 10.02 \approx 10$ m.

This means one quadratic can answer several different questions about the same motion.

```
//| echo: false
viewof ballH0 = Inputs.range([0, 5], { value: 0, step: 0.5, label: "Initial height h₀ (m)" })
```

```
//| echo: false
viewof ballV = Inputs.range([1, 20], { value: 14, step: 1, label: "Initial speed v (m/s)" })
```

```
//| echo: false
{
  const h0 = ballH0, v = ballV;
  // h(t) = h0 + v*t - 4.9*t^2
  // Landing: positive root of -4.9t^2 + v*t + h0 = 0
  const disc = v * v + 4 * 4.9 * h0;
  const tLand = (v + Math.sqrt(disc)) / (2 * 4.9);
  const tPeak = v / (2 * 4.9);
  const hPeak = h0 + v * tPeak - 4.9 * tPeak * tPeak;

  const svgW = 500, svgH = 320;
  const margin = { top: 40, right: 30, bottom: 40, left: 55 };
  const W = svgW - margin.left - margin.right;
  const H = svgH - margin.top - margin.bottom;

  const tMax = tLand * 1.05;
  const tArr = d3.range(0, tLand + 0.01, tLand / 200);
  const hArr = tArr.map(t => Math.max(0, h0 + v * t - 4.9 * t * t));

  const xScale = d3.scaleLinear([0, tMax], [0, W]);
  const yMax = hPeak * 1.15 + 1;
  const yScale = d3.scaleLinear([0, yMax], [H, 0]);

  const svg = d3.create("svg")
    .attr("width", svgW).attr("height", svgH)
    .style("font-family", "inherit");

  const g = svg.append("g")
    .attr("transform", `translate(${margin.left},${margin.top})`);

  // Axes
  g.append("line").attr("x1", 0).attr("y1", H).attr("x2", W).attr("y2", H)
    .attr("stroke", "#6b7280").attr("stroke-width", 1.5);
  g.append("line").attr("x1", 0).attr("y1", 0).attr("x2", 0).attr("y2", H)
    .attr("stroke", "#6b7280").attr("stroke-width", 1.5);

  // Axis labels
```

```
g.append("text").attr("x", W / 2).attr("y", H + 32)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#374151")
  .text("time t (seconds)");
g.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -H / 2).attr("y", -40)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#374151")
  .text("height h (metres)");

// Curve
const lineGen = d3.line()
  .x((_, i) => xScale(tArr[i]))
  .y(h => yScale(h));

g.append("path")
  .datum(hArr)
  .attr("d", lineGen)
  .attr("fill", "none")
  .attr("stroke", "#2563eb")
  .attr("stroke-width", 2.5);

// Peak marker
g.append("circle")
  .attr("cx", xScale(tPeak)).attr("cy", yScale(hPeak))
  .attr("r", 6).attr("fill", "#f59e0b").attr("stroke", "#fff").attr("stroke-width", 1.5);

// Landing marker
g.append("circle")
  .attr("cx", xScale(tLand)).attr("cy", yScale(0))
  .attr("r", 6).attr("fill", "#dc2626").attr("stroke", "#fff").attr("stroke-width", 1.5);

// Labels
g.append("text")
  .attr("x", xScale(tPeak) + 8).attr("y", yScale(hPeak) - 8)
  .attr("font-size", "12px").attr("fill", "#d97706")
  .text(`peak: ${hPeak.toFixed(2)} m at t=${tPeak.toFixed(2)} s`);

g.append("text")
  .attr("x", xScale(tLand) - 8).attr("y", yScale(0) + 18)
  .attr("text-anchor", "end").attr("font-size", "12px").attr("fill", "#dc2626")
  .text(`lands at t=${tLand.toFixed(2)} s`);

// Title
svg.append("text")
  .attr("x", margin.left).attr("y", 24)
  .attr("font-size", "13px").attr("font-weight", "600").attr("fill", "#111827")
  .text(`h(t) = ${h0} + ${v}t − 4.9t²`);
```

```
  return svg.node();
}
```

---

**Example 2 (eng) — maximum area from fixed fencing**

You have 40 m of fencing and want to enclose a rectangular garden with the largest possible area. What dimensions give the maximum area?

Let the width be $x$ metres. Since the total perimeter is 40 m, the two widths use $2x$ metres, leaving $40 - 2x$ for the two lengths. Each length is $(40 - 2x)/2 = 20 - x$ metres.

Area:

$$A = x(20 - x) = 20x - x^2$$

This is a downward parabola ($a = -1$). Maximum at:

$$x_v = -\frac{20}{2 \times (-1)} = 10 \text{ m}$$

Length $= 20 - 10 = 10$ m. Area $= 10 \times 10 = 100$ m². A square encloses the most area for a given perimeter.

This means the vertex can represent the best possible design choice.

---

**Example 3 (biz) — maximum revenue from a pricing function**

A small business sells handmade candles. Market research shows that at a price of $p$ dollars each, the number sold per week is $q = 60 - 2p$. Revenue is:

$$R = p \cdot q = p(60 - 2p) = 60p - 2p^2$$

This is a downward parabola ($a = -2$). Maximum revenue at:

$$p_v = -\frac{60}{2 \times (-2)} = -\frac{60}{-4} = 15$$

Price of \$15. Revenue: $R = 15 \times (60 - 30) = 15 \times 30 = 450$ per week.

To confirm it's a maximum and not a minimum: $a = -2 < 0$, so the parabola opens downward. The vertex is the top. Any price above or below \$15 gives less revenue.

This means the sign of the quadratic term quickly tells you what kind of optimum you have.

---

**Example 4 (comp) — character jump arc**

A game character jumps. The height of the character above the ground in metres at time $t$ seconds after the jump is:

$$h(t) = -5t^2 + 10t$$

Find the maximum height and the time the character lands.

The parabola opens downward ($a = -5$), so the vertex is the maximum.

Vertex time: $t_v = -10/(2 \times -5) = 1$ second.

Maximum height: $h(1) = -5(1)^2 + 10(1) = -5 + 10 = 5$ metres.

Landing: set $h = 0$.

$$-5t^2 + 10t = 0$$

$$t(-5t + 10) = 0$$

$t = 0$ (take-off) or $t = 2$ seconds (landing).

Evaluating a polynomial at a specific value — like checking the height at $t = 1$ or $t = 2$ — is a calculation every game engine runs many times per second to determine where characters and objects are.

This means polynomial models can directly control motion and timing.

## 18.5. Where this goes

This chapter turns curved relationships into algebra you can expand, factor, solve, and interpret.

The natural next step is **Vol 3, Chapter 4: Functions and relations**. A quadratic is a function — it takes an $x$ value and produces a $y$ value, one output for each input. Once you have the function concept, the parabola becomes one member of a large family: linear functions, quadratic functions, exponential functions, trigonometric functions. The machinery you've built here — reading coefficients, finding roots, locating vertices — applies across all of them.

The bigger horizon is **calculus**. The vertex formula $x_v = -b/(2a)$ gives you the maximum or minimum, but it only works for quadratics. What about a cubic, or a more complicated curve? That's what differential calculus (Vol 5) answers: a general method for finding the exact peak or trough of any smooth curve. The quadratic vertex is the simplest case of a much more powerful idea. When you reach it, what you learned here will already be familiar.

**Where this shows up**

- A structural engineer calculating deflection in a beam under a distributed load solves a polynomial equation.
- A physicist modelling projectile trajectory — range, maximum height, time of flight — works with quadratics throughout.
- A pricing analyst finding the revenue-maximising price point solves $dR/dp = 0$, which for a linear demand function produces a quadratic.
- A computer graphics programmer evaluating a Bezier spline is computing a polynomial at a parameter value thousands of times per frame.
- A data scientist fitting a parabola to experimental data (polynomial regression of degree 2) uses the same $ax^2 + bx + c$ form.

## 18.6. Exercises

These are puzzles. Each has a clean answer. The interesting part is setting up the equation first, then solving it.

1. A ball is thrown upward from a cliff 20 m above the ground with an initial upward speed of 10 m/s. Its height above the ground after $t$ seconds is $h(t) = 20 + 10t - 4.9t^2$. When does it hit the ground?

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
```

```
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

```
//| echo: false
makeStepperHTML(1, [
  { op: "Set height equal to zero", eq: "20 + 10t - 4.9t^2 = 0", note: "The ball hits the ground
  { op: "Rearrange into standard form", eq: "-4.9t^2 + 10t + 20 = 0", note: null },
  { op: "Identify coefficients (the quadratic formula handles any sign of a)", eq: "a = -4.9,\\qu
```

```
  { op: "Compute the discriminant", eq: "\\Delta = 10^2 - 4(-4.9)(20) = 100 + 392 = 492", note: "Pos
  { op: "Apply the quadratic formula", eq: "t = \\frac{-10 \\pm \\sqrt{492}}{2 \\times (-4.9)} = \\f
  { op: "Take the root that gives positive t", eq: "t = \\frac{-10 - 22.18}{-9.8} = \\frac{-32.18}{-
  { op: "Check", eq: "h(3.28) = 20 + 10(3.28) - 4.9(3.28)^2 \\approx 20 + 32.8 - 52.7 \\approx 0.1 \
])
```

2. Factor $x^2 - 7x + 12$ completely, then write down the two roots of $x^2 - 7x + 12 = 0$.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Write the trinomial", eq: "x^2 - 7x + 12", note: null },
  { op: "Find two numbers that multiply to 12 and add to -7", eq: "(-3) \\times (-4) = 12 \\quad\\te
  { op: "Write the factored form", eq: "x^2 - 7x + 12 = (x - 3)(x - 4)", note: null },
  { op: "Set each factor to zero", eq: "x - 3 = 0 \\implies x = 3 \\qquad x - 4 = 0 \\implies x = 4"
  { op: "Check", eq: "(3)^2 - 7(3) + 12 = 9 - 21 + 12 = 0 \\checkmark \\qquad (4)^2 - 7(4) + 12 = 16
])
```

3. A rectangular garden has an area of 48 m². Its length is 2 m more than its width. Find the dimensions.

```
//| echo: false
makeStepperHTML(3, [
  { op: "Let width = x, then length = x + 2", eq: "x(x + 2) = 48", note: null },
  { op: "Expand", eq: "x^2 + 2x = 48", note: null },
  { op: "Rearrange into standard form", eq: "x^2 + 2x - 48 = 0", note: null },
  { op: "Find two numbers that multiply to -48 and add to 2", eq: "8 \\times (-6) = -48 \\quad\\text
  { op: "Factor", eq: "(x + 8)(x - 6) = 0", note: null },
  { op: "Solve — take the positive root", eq: "x = 6 \\text{ (width)}, \\quad x + 2 = 8 \\text{ (len
  { op: "Check", eq: "6 \\times 8 = 48 \\checkmark", note: null }
])
```

4. Use the quadratic formula to solve $2x^2 - 3x - 2 = 0$.

```
//| echo: false
makeStepperHTML(4, [
  { op: "Identify coefficients", eq: "a = 2,\\quad b = -3,\\quad c = -2", note: null },
  { op: "Compute the discriminant", eq: "\\Delta = (-3)^2 - 4(2)(-2) = 9 + 16 = 25", note: null },
  { op: "Apply the quadratic formula", eq: "x = \\frac{3 \\pm \\sqrt{25}}{4} = \\frac{3 \\pm 5}{4}",
  { op: "Find both solutions", eq: "x = \\frac{3 + 5}{4} = 2 \\qquad\\text{or}\\qquad x = \\frac{3 -
  { op: "Check x = 2", eq: "2(4) - 3(2) - 2 = 8 - 6 - 2 = 0 \\checkmark", note: null },
  { op: "Check x = -1/2", eq: "2\\left(\\frac{1}{4}\\right) - 3\\left(-\\frac{1}{2}\\right) - 2 = \\
])
```

5. A candle company finds its weekly profit in dollars is $P(p) = -3p^2 + 48p - 165$, where $p$ is the price per candle. At what price is profit maximised, and what is that profit?

```
//| echo: false
makeStepperHTML(5, [
  { op: "Identify coefficients", eq: "a = -3,\\quad b = 48,\\quad c = -165", note: "The negative
  { op: "Find the price at the vertex", eq: "p_v = -\\frac{b}{2a} = -\\frac{48}{2(-3)} = -\\frac{
  { op: "Substitute to find maximum profit", eq: "P(8) = -3(64) + 48(8) - 165 = -192 + 384 - 165
  { op: "State the answer", eq: "\\text{Maximum profit: } \\$27 \\text{ per week at a price of }
  { op: "Check", eq: "P(7) = -3(49) + 48(7) - 165 = -147 + 336 - 165 = 24 < 27 \\checkmark", note
])
```

6. Factor $4x^2 - 25$ completely.

```
//| echo: false
makeStepperHTML(6, [
  { op: "Recognise the structure", eq: "4x^2 - 25 = (2x)^2 - 5^2", note: "Both terms are perfect
  { op: "Apply a² - b² = (a + b)(a - b)", eq: "4x^2 - 25 = (2x + 5)(2x - 5)", note: null },
  { op: "Check by expanding", eq: "(2x + 5)(2x - 5) = 4x^2 - 10x + 10x - 25 = 4x^2 - 25 \\checkma
])
```

# 19. Functions and relations

Given this, produce that

Press B4 on a vending machine. One specific drink comes out — not a random choice, not two drinks. One input, one output, every time.

Type 100 into a Celsius-to-Fahrenheit converter. You get 212. Type 100 again tomorrow and you still get 212. The relationship doesn't change. The output is determined entirely by the input.

A tip calculator takes one number — the bill total — and produces one number: the suggested tip. It doesn't need to know anything else. One input, one output, every time.

These are all the same structure: a rule that takes an input and produces exactly one output. Mathematicians call that a function. This chapter is about what that means precisely — and why the restriction matters.

## 19.1. What the notation is saying

Start with the machine idea from the opening. A function takes an input, applies a rule, and returns one output.

Here is a function:

$$f(x) = 2x + 3$$

Read it aloud: "f of x equals 2x plus 3." The letter $f$ is the name of the function — you could call it $g$ or $T$ or anything, but $f$ is the default. The $x$ inside the parentheses is the input. The right-hand side, $2x + 3$, is the rule: whatever you put in, double it and add 3.

The notation $f(x)$ does not mean $f$ multiplied by $x$. It means "the output of $f$ when the input is $x$." That's an easy thing to misread the first time you see it. Say it in words every time until it's automatic.

This means function notation is about evaluation, not multiplication.

You may also see function notation written as $f : A \to B$, which says "$f$ maps from set $A$ to set $B$" — every element of $A$ gets sent to exactly one element of $B$. For example, the temperature converter maps the set of Celsius values to the set of Fahrenheit values.

One thing the definition requires: **every input produces exactly one output.** That's what separates a function from a general relation. A relation might pair one input with two or more outputs — but then it's not a function. The vending machine analogy holds: if pressing B4 sometimes gave you a cola and

sometimes gave you nothing, or randomly gave you two things, you wouldn't call it a reliable machine. A function is a reliable machine.

This means the key restriction is consistency: one input, one output.

```
//| echo: false
{
  const FUNCTIONS = {
    "2x + 1": {
      formula: "f(x) = 2x + 1",
      fn: x => 2 * x + 1,
      domain: x => x >= -6 && x <= 6,
      domainLabel: "all shown x-values",
      rangeLabel: "all shown y-values"
    },
    "x²": {
      formula: "f(x) = x²",
      fn: x => x * x,
      domain: x => x >= -6 && x <= 6,
      domainLabel: "all shown x-values",
      rangeLabel: "y ≥ 0"
    },
    "|x|": {
      formula: "f(x) = |x|",
      fn: x => Math.abs(x),
      domain: x => x >= -6 && x <= 6,
      domainLabel: "all shown x-values",
      rangeLabel: "y ≥ 0"
    },
    "√(x + 5)": {
      formula: "f(x) = √(x + 5)",
      fn: x => Math.sqrt(x + 5),
      domain: x => x >= -5 && x <= 6,
      domainLabel: "x ≥ -5",
      rangeLabel: "y ≥ 0"
    }
  };

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  const container = document.createElement("div");
  container.style.cssText = "max-width: 700px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-botto
  const choiceWrap = document.createElement("label");
```

```
choiceWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
choiceWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Function f
const choice = document.createElement("select");
choice.style.cssText = "padding:0.35rem 0.55rem; border:1px solid #d1d5db; border-radius:8px; b
Object.keys(FUNCTIONS).forEach(name => {
  const opt = document.createElement("option");
  opt.value = name;
  opt.textContent = name;
  choice.appendChild(opt);
});
choiceWrap.appendChild(choice);
const xWrap = document.createElement("label");
xWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
xWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Input x" }));
const xSlider = document.createElement("input");
xSlider.type = "range";
xSlider.min = "-6";
xSlider.max = "6";
xSlider.step = "0.1";
xSlider.value = "2";
const xValue = document.createElement("span");
xValue.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
xWrap.append(xSlider, xValue);
controls.append(choiceWrap, xWrap);

const machineSvg = svgEl("svg");
machineSvg.setAttribute("viewBox", "0 0 620 150");
machineSvg.setAttribute("width", "100%");
machineSvg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-ra

const graphSvg = svgEl("svg");
graphSvg.setAttribute("viewBox", "0 0 620 280");
graphSvg.setAttribute("width", "100%");
graphSvg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radi

const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

const defs = svgEl("defs");
const marker = svgEl("marker");
marker.setAttribute("id", "fmArrow");
marker.setAttribute("markerWidth", "8");
marker.setAttribute("markerHeight", "6");
marker.setAttribute("refX", "8");
marker.setAttribute("refY", "3");
marker.setAttribute("orient", "auto");
const markerPoly = svgEl("polygon");
```

```
markerPoly.setAttribute("points", "0 0, 8 3, 0 6");
markerPoly.setAttribute("fill", "#374151");
marker.appendChild(markerPoly);
defs.appendChild(marker);
machineSvg.appendChild(defs);

const gx0 = 64, gy0 = 22, gW = 510, gH = 220;
const sx = x => gx0 + gW * (x + 6) / 12;
const sy = y => gy0 + gH * (1 - (y + 2) / 10);

function drawLine(svg, x1, y1, x2, y2, stroke, width = 1.5, dash = "") {
  const line = svgEl("line");
  line.setAttribute("x1", x1);
  line.setAttribute("y1", y1);
  line.setAttribute("x2", x2);
  line.setAttribute("y2", y2);
  line.setAttribute("stroke", stroke);
  line.setAttribute("stroke-width", width);
  if (dash) line.setAttribute("stroke-dasharray", dash);
  svg.appendChild(line);
  return line;
}

function drawText(svg, x, y, text, fill = "#374151", size = 12, anchor = "start", weight = "400")
  const t = svgEl("text");
  t.setAttribute("x", x);
  t.setAttribute("y", y);
  t.setAttribute("fill", fill);
  t.setAttribute("font-size", size);
  t.setAttribute("text-anchor", anchor);
  t.setAttribute("font-weight", weight);
  t.textContent = text;
  svg.appendChild(t);
  return t;
}

function redraw() {
  const model = FUNCTIONS[choice.value];
  const x = parseFloat(xSlider.value);
  const inDomain = model.domain(x);
  const y = inDomain ? model.fn(x) : null;
  xValue.textContent = x.toFixed(1);

  while (machineSvg.childNodes.length > 1) machineSvg.removeChild(machineSvg.lastChild);
  while (graphSvg.firstChild) graphSvg.removeChild(graphSvg.firstChild);

  drawLine(machineSvg, 70, 75, 185, 75, "#374151", 2);
```

```
machineSvg.lastChild.setAttribute("marker-end", "url(#fmArrow)");
drawLine(machineSvg, 435, 75, 545, 75, "#374151", 2);
machineSvg.lastChild.setAttribute("marker-end", "url(#fmArrow)");
const box = svgEl("rect");
box.setAttribute("x", "190");
box.setAttribute("y", "38");
box.setAttribute("width", "240");
box.setAttribute("height", "74");
box.setAttribute("rx", "10");
box.setAttribute("fill", "#eff6ff");
box.setAttribute("stroke", "#2563eb");
box.setAttribute("stroke-width", "2");
machineSvg.appendChild(box);
drawText(machineSvg, 310, 68, model.formula, "#1d4ed8", 15, "middle", "700");
drawText(machineSvg, 310, 91, "function machine", "#6b7280", 12, "middle");
drawText(machineSvg, 122, 62, `x = ${x.toFixed(1)}`, "#374151", 13, "middle");
drawText(machineSvg, 490, 62, inDomain ? `f(x) = ${y.toFixed(2)}` : "undefined", inDomain ? "
drawText(machineSvg, 310, 132, inDomain ? `(${x.toFixed(1)}, ${y.toFixed(2)})` : `(${x.toFixe

for (let xi = -6; xi <= 6; xi += 1) drawLine(graphSvg, sx(xi), sy(-2), sx(xi), sy(8), "#f3f4f
for (let yi = -2; yi <= 8; yi += 1) drawLine(graphSvg, sx(-6), sy(yi), sx(6), sy(yi), "#f3f4f
drawLine(graphSvg, sx(-6), sy(0), sx(6), sy(0), "#6b7280", 1.6);
drawLine(graphSvg, sx(0), sy(-2), sx(0), sy(8), "#6b7280", 1.6);
drawText(graphSvg, sx(6) + 12, sy(0) + 4, "x", "#6b7280", 12);
drawText(graphSvg, sx(0) - 10, sy(8) - 10, "y", "#6b7280", 12);

const curve = svgEl("polyline");
const pts = [];
let minY = Infinity, maxY = -Infinity;
for (let t = -6; t <= 6; t += 0.05) {
  if (!model.domain(t)) continue;
  const yy = model.fn(t);
  if (yy < -2 || yy > 8) continue;
  pts.push(`${sx(t)},${sy(yy)}`);
  minY = Math.min(minY, yy);
  maxY = Math.max(maxY, yy);
}
curve.setAttribute("points", pts.join(" "));
curve.setAttribute("fill", "none");
curve.setAttribute("stroke", "#2563eb");
curve.setAttribute("stroke-width", "2.5");
graphSvg.appendChild(curve);

drawLine(graphSvg, model.domain(-6) ? sx(-6) : sx(-5), sy(0), sx(6), sy(0), "#059669", 4);
if (model.domainLabel === "x ⩽ -5") drawLine(graphSvg, sx(-6), sy(0), sx(-5), sy(0), "#ef4444
const rangeStart = Math.max(-2, minY);
const rangeEnd = Math.min(8, maxY);
```

```
    if (Number.isFinite(rangeStart) && Number.isFinite(rangeEnd)) drawLine(graphSvg, sx(0), sy(range

    drawText(graphSvg, sx(3.8), sy(-1.3), `domain: ${model.domainLabel}`, "#059669", 12, "end");
    drawText(graphSvg, sx(0.4), sy(rangeEnd) - 8, `range: ${model.rangeLabel}`, "#b45309", 12);

    if (inDomain && y >= -2 && y <= 8) {
      drawLine(graphSvg, sx(x), sy(0), sx(x), sy(y), "#9ca3af", 1.8, "4 4");
      drawLine(graphSvg, sx(0), sy(y), sx(x), sy(y), "#9ca3af", 1.8, "4 4");
      const dot = svgEl("circle");
      dot.setAttribute("cx", sx(x));
      dot.setAttribute("cy", sy(y));
      dot.setAttribute("r", "5");
      dot.setAttribute("fill", "#f59e0b");
      graphSvg.appendChild(dot);
      drawText(graphSvg, sx(x), sy(0) + 18, `x = ${x.toFixed(1)}`, "#6b7280", 11, "middle");
      drawText(graphSvg, sx(0) - 8, sy(y) - 8, `f(x) = ${y.toFixed(2)}`, "#2563eb", 11, "end");
    } else {
      drawText(graphSvg, sx(-1.4), sy(6.8), "not defined here", "#dc2626", 14, "middle", "700");
    }

    caption.textContent = "The graph is the complete picture of a function: every input mapped to it
  }

  choice.addEventListener("change", redraw);
  xSlider.addEventListener("input", redraw);
  container.append(controls, machineSvg, graphSvg, caption);
  redraw();
  return container;
}
```

## 19.2. The method

### 19.2.1. Part 1: Evaluating a function

To evaluate $f(x)$ at a specific input, replace every $x$ in the rule with that value and simplify.

If $f(x) = 2x + 3$, then $f(5)$ means: put 5 in place of $x$.

$$f(5) = 2(5) + 3 = 10 + 3 = 13$$

The input was 5, the output is 13.

If the input is negative or a fraction, the same substitution applies:

$$f(-4) = 2(-4) + 3 = -8 + 3 = -5$$

The parentheses around $-4$ are there to keep the sign attached to the number. Leave them out and $2 - 4 + 3$ is ambiguous. Keep them in and it's clear.

This means evaluating a function is careful substitution followed by simplification.

### 19.2.2. Part 2: Domain and range

The **domain** of a function is the set of all valid inputs — the values of $x$ you're allowed to use.

The **range** is the set of all possible outputs — every value $f(x)$ can produce when you run through the domain.

This means domain asks "what may go in?" and range asks "what can come out?"

For $f(x) = 2x + 3$, any real number is a valid input and any real number is a possible output. The domain and range are both all real numbers, written $(-\infty, \infty)$ in interval notation.

Domain restrictions come in two situations:

**Division by zero is not allowed.** For $g(x) = \dfrac{1}{x-2}$, the input $x = 2$ makes the denominator zero, which is undefined. The domain is all real numbers except 2, written $(-\infty, 2) \cup (2, \infty)$. The $\cup$ symbol means "or" — the domain includes numbers from the first interval or the second.

**Square roots of negative numbers don't produce real outputs.** For $h(x) = \sqrt{x-1}$, the expression under the root must be non-negative: $x - 1 \geq 0$, so $x \geq 1$. The domain is $[1, \infty)$. Why does the square root require this? No real number multiplied by itself gives a negative result — so the square root of a negative number doesn't land on the number line.

Interval notation in brief: - $[a, b]$ — includes both endpoints ($a \leq x \leq b$) - $(a, b)$ — excludes both endpoints ($a < x < b$) - $[a, \infty)$ — from $a$ upward, including $a$ - $(-\infty, b)$ — everything below $b$, not including $b$

The domain can also be restricted by context — a function describing something you can only sell in whole numbers, or a physical measurement that can't be negative. That kind of restriction comes from the situation, not the algebra.

This means a complete answer often needs both algebraic rules and real-world sense.

### 19.2.3. Part 3: Composition

You can chain functions together. If $g$ converts one thing and $f$ converts another, you can run an input through $g$ first, then feed $g$'s output into $f$.

The notation for this is $f(g(x))$, read as "f of g of x." You may also see it written $f \circ g$, read as "f composed with g." Both mean the same thing: apply $g$ first, then apply $f$ to the result.

This means composition reads from the inside out.

If $g(x) = x + 3$ and $f(x) = 2x + 1$:

$$f(g(x)) = f(x + 3) = 2(x + 3) + 1 = 2x + 6 + 1 = 2x + 7$$

The order matters. $f(g(x))$ and $g(f(x))$ are generally different.

$g(f(x)) = g(2x + 1) = (2x + 1) + 3 = 2x + 4$

That is a different function. Get the order right by reading from the inside out: in $f(g(x))$, $g$ acts first.

This means composition is not usually reversible by simply swapping the order.

```
//| echo: false
{
  const FNS = {
    "x + 3": v => v + 3,
    "2x": v => 2 * v,
    "x²": v => v * v
  };
  const GNS = {
    "x − 1": v => v - 1,
    "3x": v => 3 * v,
    "x + 2": v => v + 2
  };

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  const container = document.createElement("div");
  container.style.cssText = "max-width: 620px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-botto
  function selectWrap(labelText, options) {
    const wrap = document.createElement("label");
    wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
    wrap.appendChild(Object.assign(document.createElement("span"), { textContent: labelText }));
    const sel = document.createElement("select");
    sel.style.cssText = "padding:0.35rem 0.55rem; border:1px solid #d1d5db; border-radius:8px; backg
    options.forEach(optValue => {
      const opt = document.createElement("option");
      opt.value = optValue;
      opt.textContent = optValue;
      sel.appendChild(opt);
    });
    wrap.appendChild(sel);
    controls.appendChild(wrap);
    return sel;
  }
  const fSel = selectWrap("f(x)", Object.keys(FNS));
  const gSel = selectWrap("g(x)", Object.keys(GNS));
  const xWrap = document.createElement("label");
  xWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  xWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Input x" }));
```

```
const xSlider = document.createElement("input");
xSlider.type = "range";
xSlider.min = "-4";
xSlider.max = "4";
xSlider.step = "0.5";
xSlider.value = "1";
const xVal = document.createElement("span");
xVal.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
xWrap.append(xSlider, xVal);
const swapWrap = document.createElement("label");
swapWrap.style.cssText = "display:flex; gap:0.4rem; align-items:center; font-size:0.9rem; color
const swap = document.createElement("input");
swap.type = "checkbox";
swapWrap.append(swap, document.createTextNode("Swap order"));
controls.append(xWrap, swapWrap);

const svg = svgEl("svg");
svg.setAttribute("viewBox", "0 0 560 260");
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function drawText(x, y, text, fill = "#374151", size = 12, anchor = "middle", weight = "400") {
  const t = svgEl("text");
  t.setAttribute("x", x);
  t.setAttribute("y", y);
  t.setAttribute("fill", fill);
  t.setAttribute("font-size", size);
  t.setAttribute("text-anchor", anchor);
  t.setAttribute("font-weight", weight);
  t.textContent = text;
  svg.appendChild(t);
}

function redraw() {
  while (svg.firstChild) svg.removeChild(svg.firstChild);
  const x = parseFloat(xSlider.value);
  xVal.textContent = x.toFixed(1);
  const fName = fSel.value;
  const gName = gSel.value;
  const firstFn = swap.checked ? FNS[fName] : GNS[gName];
  const secondFn = swap.checked ? GNS[gName] : FNS[fName];
  const mid = firstFn(x);
  const end = secondFn(mid);
  const y1 = 58, y2 = 128, y3 = 198;
  const xMap = (v, a, b) => 120 + 340 * (v - a) / (b - a);
```

```
  const range2 = [-8, 8];
  const range3 = [-16, 16];

  [["input x", y1], [swap.checked ? "f(x)" : "g(x)", y2], [swap.checked ? "g(f(x))" : "f(g(x))", y
    const line = svgEl("line");
    line.setAttribute("x1", "110"); line.setAttribute("x2", "470");
    line.setAttribute("y1", y); line.setAttribute("y2", y);
    line.setAttribute("stroke", "#94a3b8"); line.setAttribute("stroke-width", "2");
    svg.appendChild(line);
    drawText(56, y + 4, label, "#374151", 12, "start", "700");
  });

  for (let t = -4; t <= 4; t += 1) {
    const tick = svgEl("line");
    tick.setAttribute("x1", xMap(t, -4, 4)); tick.setAttribute("x2", xMap(t, -4, 4));
    tick.setAttribute("y1", y1 - 6); tick.setAttribute("y2", y1 + 6);
    tick.setAttribute("stroke", "#94a3b8");
    svg.appendChild(tick);
    drawText(xMap(t, -4, 4), y1 + 20, `${t}`, "#6b7280", 10);
  }

  const circles = [
    { x: xMap(x, -4, 4), y: y1, fill: "#374151" },
    { x: xMap(mid, range2[0], range2[1]), y: y2, fill: "#059669" },
    { x: xMap(end, range3[0], range3[1]), y: y3, fill: "#2563eb" }
  ];
  circles.forEach(c => {
    const dot = svgEl("circle");
    dot.setAttribute("cx", c.x); dot.setAttribute("cy", c.y); dot.setAttribute("r", "5");
    dot.setAttribute("fill", c.fill);
    svg.appendChild(dot);
  });

  const arrow1 = svgEl("path");
  arrow1.setAttribute("d", `M ${circles[0].x} ${y1 + 5} Q ${(circles[0].x + circles[1].x) / 2} ${
  arrow1.setAttribute("fill", "none"); arrow1.setAttribute("stroke", "#059669"); arrow1.setAttribu
  svg.appendChild(arrow1);
  const arrow2 = svgEl("path");
  arrow2.setAttribute("d", `M ${circles[1].x} ${y2 + 5} Q ${(circles[1].x + circles[2].x) / 2} ${
  arrow2.setAttribute("fill", "none"); arrow2.setAttribute("stroke", "#2563eb"); arrow2.setAttribu
  svg.appendChild(arrow2);

  drawText((circles[0].x + circles[1].x) / 2, 100, swap.checked ? "apply f" : "apply g", "#059669"
  drawText((circles[1].x + circles[2].x) / 2, 170, swap.checked ? "apply g" : "apply f", "#2563eb"
  drawText(circles[0].x, y1 - 12, `${x.toFixed(1)}`, "#374151", 11);
  drawText(circles[1].x, y2 - 12, `${mid.toFixed(1)}`, "#059669", 11);
  drawText(circles[2].x, y3 - 12, `${end.toFixed(1)}`, "#2563eb", 11);
```

```
    drawText(280, 232, `${swap.checked ? "f" : "g"}(${x.toFixed(1)}) = ${mid.toFixed(1)}     ${sw
    drawText(280, 248, swap.checked ? `g(f(x)) with f(x) = ${fName} and g(x) = ${gName}` : `f(g(x

    caption.textContent = "Composition is a chain. One function acts first, then the second acts
  }

  [fSel, gSel, xSlider, swap].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, svg, caption);
  redraw();
  return container;
}
```

### 19.2.4. Part 4: Inverses

The inverse of a function undoes it. If $f$ takes 5 to 13, then $f^{-1}$ — read as "f inverse" — takes 13 back to 5.

The notation $f^{-1}(x)$ does not mean $\frac{1}{f(x)}$. That's another thing that looks like it should mean one thing but means another. The superscript $-1$ on a function name means "the inverse function," not "one over."

This means inverse notation is about undoing a function, not taking a reciprocal.

Not every function has an inverse. For an inverse to exist, each output must have come from exactly one input — otherwise, when you try to go backwards, you don't know which input to return to. A function with this property is called **one-to-one**: different inputs produce different outputs.

To find the inverse algebraically: write $y = f(x)$, swap $x$ and $y$, then solve for $y$.

For $f(x) = 2x + 3$:

Write as: $y = 2x + 3$

Swap $x$ and $y$: $x = 2y + 3$

Solve for $y$:

$$x - 3 = 2y$$

$$y = \frac{x-3}{2}$$

So $f^{-1}(x) = \frac{x-3}{2}$.

Check: $f(5) = 13$, and $f^{-1}(13) = \frac{13-3}{2} = \frac{10}{2} = 5$. Yes.

This means the inverse reverses the original input-output pairing.

```
//| echo: false
{
  const MODELS = {
    "2x + 3": {
      label: "f(x) = 2x + 3",
      fn: x => 2 * x + 3,
      inv: y => (y - 3) / 2,
      domain: [-4, 1]
    },
    "x³": {
      label: "f(x) = x³",
      fn: x => x * x * x,
      inv: y => Math.cbrt(y),
      domain: [-1.7, 1.7]
    },
    "x² (x ≥ 0)": {
      label: "f(x) = x² with x ≥ 0",
      fn: x => x * x,
      inv: y => Math.sqrt(y),
      domain: [0, 2.2]
    }
  };

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width: 660px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-botto
  const selWrap = document.createElement("label");
  selWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  selWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Function" }));
  const select = document.createElement("select");
  select.style.cssText = "padding:0.35rem 0.55rem; border:1px solid #d1d5db; border-radius:8px; back
  Object.keys(MODELS).forEach(name => {
    const opt = document.createElement("option");
    opt.value = name; opt.textContent = name;
    select.appendChild(opt);
  });
  selWrap.appendChild(select);
  const xWrap = document.createElement("label");
  xWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  xWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Point x" }));
  const slider = document.createElement("input");
  slider.type = "range"; slider.min = "-4"; slider.max = "4"; slider.step = "0.05"; slider.value = "
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
  xWrap.append(slider, out);
```

```
controls.append(selWrap, xWrap);
const svg = svgEl("svg");
svg.setAttribute("viewBox", "0 0 620 360");
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function redraw() {
  while (svg.firstChild) svg.removeChild(svg.firstChild);
  const model = MODELS[select.value];
  slider.min = String(model.domain[0]);
  slider.max = String(model.domain[1]);
  const x = Math.max(model.domain[0], Math.min(model.domain[1], parseFloat(slider.value)));
  slider.value = String(x);
  out.textContent = x.toFixed(2);
  const y = model.fn(x);
  const mapX = v => 60 + 500 * (v + 5) / 10;
  const mapY = v => 300 - 250 * (v + 5) / 10;
  const line = (x1,y1,x2,y2,stroke,w=1.5,dash="") => { const l=svgEl("line"); l.setAttribute("x
  const text = (x1,y1,t,fill="#374151",size=12,anchor="middle",weight="400") => { const n=svgEl
  for (let i = -5; i <= 5; i += 1) { line(mapX(i), mapY(-5), mapX(i), mapY(5), "#f3f4f6", 1); l
  line(mapX(-5), mapY(0), mapX(5), mapY(0), "#6b7280", 1.6);
  line(mapX(0), mapY(-5), mapX(0), mapY(5), "#6b7280", 1.6);
  line(mapX(-5), mapY(-5), mapX(5), mapY(5), "#9ca3af", 1.8, "5 4");
  text(mapX(4.4), mapY(4.4) - 8, "y = x", "#6b7280", 12, "end");

  const ptsF = [], ptsInv = [];
  for (let t = model.domain[0]; t <= model.domain[1]; t += 0.05) {
    const yy = model.fn(t);
    if (yy >= -5 && yy <= 5) ptsF.push(`${mapX(t)},${mapY(yy)}`);
  }
  for (let t = -5; t <= 5; t += 0.05) {
    const yy = model.inv(t);
    if (Number.isFinite(yy) && yy >= -5 && yy <= 5) ptsInv.push(`${mapX(t)},${mapY(yy)}`);
  }
  const polyF = svgEl("polyline");
  polyF.setAttribute("points", ptsF.join(" "));
  polyF.setAttribute("fill", "none");
  polyF.setAttribute("stroke", "#2563eb");
  polyF.setAttribute("stroke-width", "2.5");
  svg.appendChild(polyF);
  const polyI = svgEl("polyline");
  polyI.setAttribute("points", ptsInv.join(" "));
  polyI.setAttribute("fill", "none");
  polyI.setAttribute("stroke", "#f59e0b");
  polyI.setAttribute("stroke-width", "2.5");
```

```
    polyI.setAttribute("stroke-dasharray", "7 5");
    svg.appendChild(polyI);

    const p = svgEl("circle");
    p.setAttribute("cx", mapX(x)); p.setAttribute("cy", mapY(y)); p.setAttribute("r", "5"); p.setAtt
    svg.appendChild(p);
    const q = svgEl("circle");
    q.setAttribute("cx", mapX(y)); q.setAttribute("cy", mapY(x)); q.setAttribute("r", "5"); q.setAtt
    svg.appendChild(q);
    line(mapX(x), mapY(y), mapX(y), mapY(x), "#9ca3af", 1.5, "4 4");
    text(mapX(x) + 10, mapY(y) - 10, `(${x.toFixed(2)}, ${y.toFixed(2)})`, "#2563eb", 11, "start");
    text(mapX(y) + 10, mapY(x) + 16, `(${y.toFixed(2)}, ${x.toFixed(2)})`, "#b45309", 11, "start");
    text(310, 28, model.label, "#374151", 13, "middle", "700");
    caption.textContent = select.value === "x² (x ⍰ 0)"
      ? "The inverse is a reflection across y = x, but the parabola must be restricted to x ⍰ 0 firs
      : "The inverse function is the reflection of the original across y = x. Every point (a, b) on
  }

  [select, slider].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, svg, caption);
  redraw();
  return container;
}
```

**Why this works: the vertical line test**

Draw the graph of a relation. Draw a vertical line anywhere on the graph.

If any vertical line crosses the graph more than once, the relation is not a function — because that input $x$ (the horizontal position of your line) maps to two or more outputs (the two crossing points).

If every vertical line crosses the graph at most once, the relation is a function. The test is visual and immediate: one crossing means one output for that input. That is the requirement.

```
//| echo: false
{
  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width: 680px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; margin-bottom:0.7rem;";
  const wrap = document.createElement("label");
  wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  wrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Vertical line posit
  const slider = document.createElement("input");
  slider.type = "range"; slider.min = "-2.5"; slider.max = "2.5"; slider.step = "0.05"; slider.value
  const val = document.createElement("span");
  val.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
```

```
wrap.append(slider, val);
controls.appendChild(wrap);
const svg = svgEl("svg");
svg.setAttribute("viewBox", "0 0 660 300");
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function drawPanel(x0, title, sideways = false) {
  const mapX = x => x0 + 30 + 240 * (x + 3) / 6;
  const mapY = y => 255 - 210 * (y + 3) / 6;
  const line = (x1,y1,x2,y2,stroke,w=1.5,dash="") => { const l=svgEl("line"); l.setAttribute("x
  const text = (x,y,t,fill="#374151",size=12,anchor="middle",weight="400") => { const n=svgEl("
  const border = svgEl("rect");
  border.setAttribute("x", x0); border.setAttribute("y", "24"); border.setAttribute("width", "3
  border.setAttribute("rx", "10"); border.setAttribute("fill", "#fff"); border.setAttribute("st
  svg.appendChild(border);
  text(x0 + 150, 44, title, "#374151", 13, "middle", "700");
  for (let i = -3; i <= 3; i += 1) { line(mapX(i), mapY(-3), mapX(i), mapY(3), "#f3f4f6", 1); l
  line(mapX(-3), mapY(0), mapX(3), mapY(0), "#6b7280", 1.4);
  line(mapX(0), mapY(-3), mapX(0), mapY(3), "#6b7280", 1.4);
  const pts = [];
  for (let t = -Math.sqrt(3); t <= Math.sqrt(3); t += 0.03) {
    const xx = sideways ? t * t : t;
    const yy = sideways ? t : t * t;
    pts.push(`${mapX(xx)},${mapY(yy)}`);
  }
  const curve = svgEl("polyline");
  curve.setAttribute("points", pts.join(" "));
  curve.setAttribute("fill", "none");
  curve.setAttribute("stroke", "#2563eb");
  curve.setAttribute("stroke-width", "2.3");
  svg.appendChild(curve);
  const x = parseFloat(slider.value);
  line(mapX(x), mapY(-3), mapX(x), mapY(3), "#ef4444", 1.8, "5 4");
  if (sideways) {
    if (x > 0) {
      const y = Math.sqrt(x);
      [y, -y].forEach(v => {
        const dot = svgEl("circle");
        dot.setAttribute("cx", mapX(x)); dot.setAttribute("cy", mapY(v)); dot.setAttribute("r",
        svg.appendChild(dot);
      });
      text(mapX(x) + 18, mapY(y) - 8, "2 outputs!", "#b91c1c", 11, "start", "700");
    } else {
      text(mapX(x) + 12, mapY(2.3), "0 outputs", "#6b7280", 11, "start");
```

```
      }
    } else {
      const y = x * x;
      if (y <= 3) {
        const dot = svgEl("circle");
        dot.setAttribute("cx", mapX(x)); dot.setAttribute("cy", mapY(y)); dot.setAttribute("r", "4.5
        svg.appendChild(dot);
        text(mapX(x) + 16, mapY(y) - 8, "1 output ", "#059669", 11, "start", "700");
      }
    }
  }

  function redraw() {
    while (svg.firstChild) svg.removeChild(svg.firstChild);
    val.textContent = parseFloat(slider.value).toFixed(2);
    drawPanel(18, "A function: y = x²", false);
    drawPanel(342, "Not a function: x = y²", true);
    caption.textContent = "A function passes the vertical line test: every vertical line hits at mos
  }

  slider.addEventListener("input", redraw);
  container.append(controls, svg, caption);
  redraw();
  return container;
}
```

## 19.3. Worked examples

---

**Example 1 (Science) — Temperature conversion and its inverse.**

The formula for converting Celsius to Fahrenheit is:

$$F(c) = \frac{9}{5}c + 32$$

Here $c$ is the input (temperature in °C) and $F(c)$ is the output (temperature in °F).

Evaluate at the boiling point of water, $c = 100$:

$$F(100) = \frac{9}{5}(100) + 32 = 180 + 32 = 212$$

Water boils at 212°F. Check: widely known to be correct.

Now find the inverse — the converter run backwards, converting °F to °C.

Write $y = \frac{9}{5}c + 32$, swap variables: $c = \frac{9}{5}y + 32$.

Solve for $y$:

$$c - 32 = \frac{9}{5}y$$

$$y = \frac{5}{9}(c - 32)$$

So $F^{-1}(T) = \frac{5}{9}(T - 32)$, where $T$ is a temperature in °F.

Verify: $F^{-1}(212) = \frac{5}{9}(212 - 32) = \frac{5}{9}(180) = 100$. Back to 100°C. Correct.

This means inverse functions are especially useful for unit conversions.

---

**Example 2 (Computing) — Songs downloaded and storage used.**

Every song downloaded takes up space on a device. For a particular music format, the storage used $S$ in megabytes as a function of songs downloaded $n$ is:

$$S(n) = 4.5n + 20$$

The 20 MB is the app's base footprint; each song adds 4.5 MB.

At $n = 10$ songs:

$$S(10) = 4.5(10) + 20 = 45 + 20 = 65 \text{ MB}$$

At $n = 50$ songs:

$$S(50) = 4.5(50) + 20 = 225 + 20 = 245 \text{ MB}$$

At $n = 100$ songs:

$$S(100) = 4.5(100) + 20 = 450 + 20 = 470 \text{ MB}$$

The domain has a physical restriction: $n \geq 0$ and $n$ must be a whole number. The maximum $n$ depends on the device's storage limit — the model stops being useful beyond that point.

This means real functions often live on smaller domains than the algebra alone suggests.

---

**Example 3 (Computing) — Function composition in a data pipeline.**

A data pipeline applies two transformations to a raw temperature reading from a sensor.

Step 1: the sensor outputs a raw voltage $v$. The first function converts voltage to Celsius:

$$g(v) = 25v - 10$$

Step 2: the second function converts Celsius to Fahrenheit (from Example 1):

$$f(c) = \frac{9}{5}c + 32$$

The pipeline composes them: given a raw voltage, produce a Fahrenheit reading.

$$\begin{aligned} f(g(v)) = f(25v - 10) &= \frac{9}{5}(25v - 10) + 32 \\ &= 45v - 18 + 32 = 45v + 14 \end{aligned}$$

So the combined function is $h(v) = 45v + 14$.

Test it: at $v = 1.2$ volts:

$$\begin{aligned} g(1.2) &= 25(1.2) - 10 = 30 - 10 = 20°\text{C} \\ f(20) &= \frac{9}{5}(20) + 32 = 36 + 32 = 68°\text{F} \\ h(1.2) &= 45(1.2) + 14 = 54 + 14 = 68°\text{F} \end{aligned}$$

Both paths give the same answer. The composed function works.

This means composition can compress a multi-step process into one rule.

---

**Example 4 (Business) — Revenue function with restricted domain.**

A market stall sells handmade notebooks. The revenue $R$ in dollars as a function of notebooks sold $n$ is:

$$R(n) = 12n$$

But the domain is restricted: $n$ must be a non-negative integer. You can't sell −3 notebooks or 4.7 notebooks.

Revenue at $n = 0$: $R(0) = 0$. Nothing sold, nothing earned.

Revenue at $n = 25$: $R(25) = 12(25) = 300$.

Revenue at $n = 50$: $R(50) = 12(50) = 600$.

The range is $\{0, 12, 24, 36, \ldots\}$ — multiples of 12, starting at 0. This is not a continuous range because the domain is discrete.

If the stall owner wants to know how many notebooks to sell to reach a revenue target of $180, that's the inverse:

$$R^{-1}(180) = \frac{180}{12} = 15 \text{ notebooks}$$

This means a discrete domain creates a discrete set of outputs as well.

## 19.4. Where this goes

This chapter names the general structure behind many earlier formulas: input, rule, output.

Exponents and logarithms (Volume 4, Chapter 1) are specific function families with their own shapes and properties. Once you know what a function is and how to evaluate one, invert one, and compose two, you can work with any family — exponential, logarithmic, trigonometric — using the same framework from this chapter. The function concept doesn't change; only the rule inside changes.

Calculus (Volume 5) asks a new question about functions: not "what is the output?" but "how fast is the output changing?" That question — the derivative — only makes sense if you have a function to begin with. Limits (Volume 5, Chapter 1) ask what a function's output approaches as the input gets close to a particular value, which requires the same precise notion of function you now have. Everything in analysis builds on this chapter.

**Where this shows up**

- A structural engineer's load-deflection formula is a function: given load, compute displacement.
- Every physics formula ($F = ma$, $E = mc^2$, $v = u + at$) is a function expressing one quantity in terms of others.
- A pricing algorithm that takes customer behaviour and outputs a price is a function — ideally a deterministic one.
- A compiler transforms source code (input) into machine code (output): a function from programs to programs.
- A chemist using Beer–Lambert law computes absorbance as a function of concentration. The inverse gives concentration from a measured absorbance.

## 19.5. Exercises

These are puzzles. Each one has a clean answer. The interesting part is reading the situation carefully before reaching for algebra.

---

**1.** Let $f(x) = 3x^2 - 2x + 1$.

Evaluate: (a) $f(0)$, (b) $f(2)$, (c) $f(-3)$.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
```

```
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

```
//| echo: false
makeStepperHTML(1, [
  { op: "Write the function", eq: "f(x) = 3x^2 - 2x + 1", note: null },
  { op: "Evaluate f(0): substitute x = 0", eq: "f(0) = 3(0)^2 - 2(0) + 1 = 0 - 0 + 1 = 1", note:
  { op: "Evaluate f(2): substitute x = 2", eq: "f(2) = 3(2)^2 - 2(2) + 1 = 12 - 4 + 1 = 9", note:
  { op: "Evaluate f(-3): substitute x = -3", eq: "f(-3) = 3(-3)^2 - 2(-3) + 1 = 27 + 6 + 1 = 34",
  { op: "Check by re-substituting f(2)", eq: "3(4) - 2(2) + 1 = 12 - 4 + 1 = 9 \\checkmark", note
])
```

---

**2.** State the domain of each function, using interval notation.

(a) $g(x) = \sqrt{x+4}$

(b) $h(x) = \dfrac{x}{x^2 - 9}$

(c) $k(x) = x^2 + 5$

```
//| echo: false
makeStepperHTML(2, [
  { op: "For g(x): the expression under the root must be non-negative", eq: "x + 4 \\geq 0", note
  { op: "Solve the inequality", eq: "x \\geq -4", note: null },
  { op: "Write domain of g in interval notation", eq: "[-4, \\infty)", note: null },
  { op: "For h(x): the denominator cannot be zero", eq: "x^2 - 9 \\neq 0 \\implies x \\neq 3 \\te
```

```
  { op: "Write domain of h in interval notation", eq: "(-\\infty, -3) \\cup (-3, 3) \\cup (3, \\inft
  { op: "For k(x): any real input is valid (squaring never causes division by zero or square roots c
  { op: "Check g at x = -4 (boundary)", eq: "g(-4) = \\sqrt{-4 + 4} = \\sqrt{0} = 0 \\checkmark", no
])
```

---

**3.** Let $f(x) = x^2$ and $g(x) = x - 5$.

(a) Find $f(g(x))$ and simplify.

(b) Find $g(f(x))$ and simplify.

(c) Evaluate both $f(g(x))$ and $g(f(x))$ at $x = 3$. Are they equal or different? Then try $x = 2$ — are they still the same?

```
//| echo: false
makeStepperHTML(3, [
  { op: "Write both functions", eq: "f(x) = x^2, \\quad g(x) = x - 5", note: null },
  { op: "Find f(g(x)): substitute g(x) into f", eq: "f(g(x)) = f(x - 5) = (x - 5)^2", note: "Replace
  { op: "Expand f(g(x))", eq: "(x - 5)^2 = x^2 - 10x + 25", note: null },
  { op: "Find g(f(x)): substitute f(x) into g", eq: "g(f(x)) = g(x^2) = x^2 - 5", note: "Replace eve
  { op: "Evaluate both at x = 3", eq: "f(g(3)) = (3-5)^2 = 4, \\quad g(f(3)) = 9 - 5 = 4", note: "Th
  { op: "Evaluate both at x = 2", eq: "f(g(2)) = (2-5)^2 = 9, \\quad g(f(2)) = 4 - 5 = -1", note: "D
])
```

---

**4.** A company's profit $P$ in dollars from selling $n$ units is:

$$P(n) = 8n - 150$$

The factory can produce between 0 and 200 units per day.

(a) State the domain given the physical context.

(b) At what number of units does the company break even (profit = 0)?

(c) What is the range of $P$ over this domain?

```
//| echo: false
makeStepperHTML(4, [
  { op: "Write the function and note the context", eq: "P(n) = 8n - 150, \\quad n \\in \\{0, 1, 2, \
  { op: "Find break-even: set P(n) = 0 and solve for n", eq: "8n - 150 = 0", note: null },
  { op: "Add 150 to both sides", eq: "8n = 150", note: null },
  { op: "Divide both sides by 8", eq: "n = 18.75", note: "Since n must be a whole number, the compar
  { op: "Find the range: evaluate P at n = 0 and n = 200", eq: "P(0) = -150, \\quad P(200) = 8(200)
  { op: "State the range", eq: "P \\in \\{-150, -142, -134, \\ldots, 1450\\}", note: "Multiples of 8
```

```
  { op: "Check break-even at n = 19", eq: "P(19) = 8(19) - 150 = 152 - 150 = 2 \\checkmark", note
])
```

---

**5.** Find the inverse of $h(x) = \dfrac{x+4}{3}$.

Then verify that $h(h^{-1}(x)) = x$.

```
//| echo: false
makeStepperHTML(5, [
  { op: "Write as y = h(x)", eq: "y = \\frac{x + 4}{3}", note: null },
  { op: "Swap x and y to begin finding the inverse", eq: "x = \\frac{y + 4}{3}", note: "Swapping
  { op: "Multiply both sides by 3", eq: "3x = y + 4", note: null },
  { op: "Subtract 4 from both sides", eq: "y = 3x - 4", note: null },
  { op: "Write the inverse function", eq: "h^{-1}(x) = 3x - 4", note: null },
  { op: "Verify: compute h(h⁻¹(x))", eq: "h(h^{-1}(x)) = h(3x - 4) = \\frac{(3x - 4) + 4}{3} = \\
])
```

---

**6.** The temperature inside a greenhouse over the course of a morning is modelled by:

$$T(t) = 0.5t^2 + 2t + 15$$

where $T$ is in °C and $t$ is time in hours after 6 am, with domain $0 \leq t \leq 6$.

(a) What is the temperature at 6 am ($t = 0$)?

(b) What is the temperature at 9 am ($t = 3$)?

(c) What is the temperature at noon ($t = 6$)?

(d) Is this function one-to-one over this domain? If so, briefly explain why.

```
//| echo: false
makeStepperHTML(6, [
  { op: "Write the function", eq: "T(t) = 0.5t^2 + 2t + 15", note: null },
  { op: "Evaluate at t = 0 (6 am)", eq: "T(0) = 0.5(0)^2 + 2(0) + 15 = 15 \\text{ °C}", note: nul
  { op: "Evaluate at t = 3 (9 am)", eq: "T(3) = 0.5(9) + 2(3) + 15 = 4.5 + 6 + 15 = 25.5 \\text{
  { op: "Evaluate at t = 6 (noon)", eq: "T(6) = 0.5(36) + 2(6) + 15 = 18 + 12 + 15 = 45 \\text{ °
  { op: "Assess whether T is one-to-one on [0, 6]", eq: "x_v = -\\frac{b}{2a} = -\\frac{2}{2 \\ti
  { op: "Check: T is increasing throughout", eq: "T(0) = 15 < T(3) = 25.5 < T(6) = 45 \\checkmark
])
```

# Part IV.

# Volume 4 — Functions and change

# 20. Volume 4 — Functions and change

The central move of this volume is tracking scale and structure.

In previous volumes, you studied relationships where scaling happens in simple or periodic ways. This volume is about what happens when growth accelerates, when dimensions expand, and when you need to handle many different, changing variables all at once.

By the end of this volume, you will be comfortable navigating exponential growth and decay, working within the complex plane, organizing large sets of linear constraints using matrices, and making sense of infinite step-by-step accumulations.

## 20.1. Chapters

| Chapter | Title | Core move |
|---|---|---|
| 1 | Exponents and logarithms | Managing multiplicative growth |
| 2 | Complex numbers | Expanding the number plane |
| 3 | Systems and matrices | Handling multiple relationships at once |
| 4 | Sequences and series | Identifying and summing discrete patterns |

# 21. Exponents and logarithms

Growth, decay, and the scale of things

You put \$500 in a savings account. Each year it grows by 6%. After one year you have \$530. After two years, \$561.80. After twenty years, something like \$1604. That's not 20 times \$30, or \$600, added — it's the interest compounding on itself, quietly doubling your money while you sleep.

A phone battery starts fully charged. Every hour it drains by half. An hour in: 50%. Two hours: 25%. Three: 12.5%. That's the same structure as the savings account, just running backward.

An earthquake measuring 6 on the Richter scale releases ten times more energy than one measuring 5, and a hundred times more than one measuring 4. The scale compresses a range from "barely felt" to "levels cities" into ten manageable numbers.

These are not three different ideas. They're the same mathematical structure — exponential growth, exponential decay, and the logarithm that reads off what power was used.

## 21.1. What the notation is saying

### 21.1.1. The exponent laws

Start with what you already know: $2^3$ means $2 \times 2 \times 2 = 8$. The 3 is the **exponent** — it counts how many copies of 2 you multiply. The 2 is the **base**.

This means exponent notation is a compact way to describe repeated multiplication.

The notation $a^x$ — read "a to the power x" — means the base $a$ multiplied by itself $x$ times. When you multiply two of these together with the same base, the exponents add:

$$a^m \times a^n = a^{m+n}$$

because you're just running $m$ copies followed by $n$ copies. Four laws cover everything you need:

| Law | What it says | Example |
|---|---|---|
| $a^m \times a^n = a^{m+n}$ | Multiplying: add exponents | $2^3 \times 2^4 = 2^7 = 128$ |
| $a^m \div a^n = a^{m-n}$ | Dividing: subtract exponents | $2^5 \div 2^2 = 2^3 = 8$ |
| $(a^m)^n = a^{mn}$ | Power of a power: multiply exponents | $(2^3)^2 = 2^6 = 64$ |
| $a^0 = 1$ | Zero power: always one | $17^0 = 1$ |

Two more for completeness:

$$a^{-n} = \frac{1}{a^n} \qquad a^{1/n} = \sqrt[n]{a}$$

A negative exponent means "divide by" instead of "multiply by" — $2^{-3} = \frac{1}{8}$. A fractional exponent means a root — $8^{1/3} = \sqrt[3]{8} = 2$. These look odd until you see that they follow directly from the first law above: $a^3 \times a^{-3} = a^0 = 1$, so $a^{-3}$ must equal $\frac{1}{a^3}$.

This means the exponent laws extend the same pattern instead of inventing separate rules.

### 21.1.2. The exponential function

When the exponent is a variable, you have a function:

$$f(x) = a^x$$

Read it as: "the output is the base $a$ raised to the power $x$."

- If $a > 1$: the function grows as $x$ increases. Larger $a$ means faster growth.
- If $0 < a < 1$: the function decays — same shape but reflected.
- The y-intercept is always 1: $f(0) = a^0 = 1$ for any base.

This means the base controls whether the function grows or decays.

The battery example: $f(t) = (0.5)^t$ gives the fraction of charge remaining after $t$ hours. The savings example: $f(t) = 1.06^t$ gives the growth factor after $t$ years (multiply by the initial amount to get the actual balance).

This means exponentials model change by repeated scaling, not repeated addition.

### 21.1.3. The natural base $e$

You can build an exponential function with any base. But there is one base that is special.

Consider this question: if a quantity grows at a rate exactly equal to its current size — so the bigger it gets, the faster it grows, and the growth rate is always exactly the current amount — what is the base?

The answer is $e \approx 2.718$. This number arises whenever a process is described by its own rate of change. You can't deduce its exact value without calculus, but you can use it freely right now. The function $f(x) = e^x$ is called the **natural exponential**. It obeys all the same exponent laws, with $e$ as the base.

$e$ is not arbitrary. It's not a convenient approximation. When you reach differential calculus (Volume 5), you'll see that $e^x$ is the only function that is its own derivative — the only curve whose slope at every point equals its own height. That property is what makes it turn up everywhere in science and engineering.

This means the special role of $e$ comes from how naturally it matches continuous change.

```
//| echo: false
{
  const div = document.createElement("div");
  div.style.cssText = "font-family:inherit; padding:0.5rem 0;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:1.5rem; flex-wrap:wrap; margin-bottom:0.75rem;";

  function makeSlider(label, min, max, step, val) {
    const wrap = document.createElement("div");
    const lbl = document.createElement("label");
    lbl.style.cssText = "font-size:0.85rem; color:#374151; display:block; margin-bottom:0.25rem;"
    const span = document.createElement("span");
    span.textContent = val;
    lbl.appendChild(document.createTextNode(label + " = "));
    lbl.appendChild(span);
    const input = document.createElement("input");
    input.type = "range"; input.min = min; input.max = max; input.step = step; input.value = val;
    input.style.cssText = "width:180px; display:block;";
    wrap.appendChild(lbl); wrap.appendChild(input);
    return {wrap, input, span};
  }

  const aS = makeSlider("base a", 0.5, 3.0, 0.1, 2.0);
  const AS = makeSlider("initial value A", 1, 100, 1, 1);
  controls.appendChild(aS.wrap); controls.appendChild(AS.wrap);

  const labelEl = document.createElement("div");
  labelEl.style.cssText = "font-size:0.9rem; font-weight:600; margin-bottom:0.5rem; color:#1d4ed8

  const canvas = document.createElement("canvas");
  canvas.width = 500; canvas.height = 300;
  canvas.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; width:100%; max-width:500p

  div.appendChild(controls); div.appendChild(labelEl); div.appendChild(canvas);

  function draw() {
    const a = parseFloat(aS.input.value);
    const A = parseFloat(AS.input.value);
    aS.span.textContent = a.toFixed(1);
    AS.span.textContent = A;
    const trend = a > 1 ? "growth" : a < 1 ? "decay" : "constant";
    labelEl.textContent = "f(x) = " + A + " · " + a.toFixed(1) + "ˣ  — exponential " + trend;

    const ctx = canvas.getContext("2d");
    const W = canvas.width, H = canvas.height;
    ctx.clearRect(0, 0, W, H);
```

```
  const xMin = -5, xMax = 5;
  const yMax = Math.max(A * Math.pow(a, 5), A * Math.pow(a, -5), 10) * 1.2;
  const yMin = 0;

  function px(x) { return (x - xMin) / (xMax - xMin) * W; }
  function py(y) { return H - (y - yMin) / (yMax - yMin) * H; }

  // Axes
  ctx.strokeStyle = "#9ca3af"; ctx.lineWidth = 1;
  ctx.beginPath(); ctx.moveTo(px(xMin), py(0)); ctx.lineTo(px(xMax), py(0)); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(px(0), 0); ctx.lineTo(px(0), H); ctx.stroke();

  // Curve
  const isSpecial = Math.abs(a - Math.E) < 0.05;
  ctx.strokeStyle = isSpecial ? "#7c3aed" : "#2563eb";
  ctx.lineWidth = 2.5;
  ctx.beginPath();
  let first = true;
  for (let px_ = 0; px_ <= W; px_++) {
    const x = xMin + (px_ / W) * (xMax - xMin);
    const y = A * Math.pow(a, x);
    if (y < 0 || y > yMax * 1.1) { first = true; continue; }
    if (first) { ctx.moveTo(px_, py(y)); first = false; }
    else ctx.lineTo(px_, py(y));
  }
  ctx.stroke();

  // y-intercept
  ctx.fillStyle = "#ef4444";
  ctx.beginPath(); ctx.arc(px(0), py(A), 5, 0, 2*Math.PI); ctx.fill();
  ctx.fillStyle = "#374151"; ctx.font = "12px sans-serif";
  ctx.fillText("(0, " + A + ")", px(0) + 7, py(A) - 5);

  // Point at x=1
  const y1 = A * a;
  if (y1 <= yMax) {
    ctx.fillStyle = "#059669";
    ctx.beginPath(); ctx.arc(px(1), py(y1), 5, 0, 2*Math.PI); ctx.fill();
    ctx.fillStyle = "#374151";
    ctx.fillText("(1, " + y1.toFixed(1) + ")", px(1) + 7, py(y1) - 5);
  }

  if (isSpecial) {
    ctx.fillStyle = "#7c3aed"; ctx.font = "bold 12px sans-serif";
    ctx.fillText("a = e ≈ 2.718", px(1.5), py(A * Math.E) + 15);
  }
}
```

```
  aS.input.addEventListener("input", draw);
  AS.input.addEventListener("input", draw);
  draw();
  return div;
}
```

## 21.2. The method

### 21.2.1. Part 1: Working with exponent laws

The laws in the table above are the full toolkit. A typical simplification chains two or three of them together.

**Simplify** $\dfrac{x^5 \cdot x^{-2}}{x^3}$**.**

Top: $x^5 \cdot x^{-2} = x^{5+(-2)} = x^3$.

Then: $x^3 \div x^3 = x^{3-3} = x^0 = 1$.

This means long-looking exponent expressions often collapse quickly once the base is the same.

**Simplify** $(2a^3)^4$**.**

The power applies to everything inside the bracket: $(2)^4 \cdot (a^3)^4 = 16 \cdot a^{12} = 16a^{12}$.

This means brackets matter: the outside power affects every factor inside.

### 21.2.2. Part 2: Solving exponential equations by inspection

Some equations can be solved by rewriting both sides as powers of the same base.

**Solve** $2^x = 16$**.**

Rewrite 16 as a power of 2: $16 = 2^4$.

So $2^x = 2^4$, which means $x = 4$.

**Solve** $9^x = 27$**.**

Both 9 and 27 are powers of 3: $9 = 3^2$ and $27 = 3^3$.

So $(3^2)^x = 3^3$, giving $3^{2x} = 3^3$, so $2x = 3$, and $x = \frac{3}{2}$.

This only works when you can express both sides with the same base. For $2^x = 10$, you can't. That's where logarithms come in.

This means logarithms appear when exponentials cannot be solved by inspection.

### 21.2.3. Part 3: Logarithms

A logarithm is the inverse of an exponential. It answers the question: "what power do I need?"

$$\log_a(b) = c \quad \text{means} \quad a^c = b$$

Read it aloud: "log base $a$ of $b$ equals $c$" means "$a$ raised to the power $c$ gives $b$."

Three examples to fix this:

$$\log_2(8) = 3 \quad \text{because} \quad 2^3 = 8$$

$$\log_{10}(1000) = 3 \quad \text{because} \quad 10^3 = 1000$$

$$\log_3(9) = 2 \quad \text{because} \quad 3^2 = 9$$

Two special bases appear so often they have short names:

- $\log_{10}(x)$ is often written $\log(x)$ — the **common logarithm**
- $\log_e(x)$ is written $\ln(x)$ — the **natural logarithm**, read "ell-en of x"

This means a logarithm always converts an output back into the power that produced it.

> **Why this works**
> Logarithms are the inverse of exponentials the same way division undoes multiplication. If $2^3 = 8$, then $\log_2(8) = 3$. One statement, two ways of writing it. The exponential says "start with the power, get the result." The logarithm says "start with the result, get the power." They are the same relationship read in opposite directions.

```
//| echo: false
{
  const div = document.createElement("div");
  div.style.cssText = "font-family:inherit; padding:0.5rem 0;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; align-items:center; gap:1rem; margin-bottom:0.75rem; flex-
  const lbl = document.createElement("label");
  lbl.style.cssText = "font-size:0.85rem; color:#374151;";
  const spanVal = document.createElement("span"); spanVal.textContent = "2";
  lbl.appendChild(document.createTextNode("base a = ")); lbl.appendChild(spanVal);
  const sl = document.createElement("input");
  sl.type = "range"; sl.min = 2; sl.max = 10; sl.step = 1; sl.value = 2;
  sl.style.width = "160px";
  controls.appendChild(lbl); controls.appendChild(sl);
```

```
const infoEl = document.createElement("div");
infoEl.id = "viz2_info";
infoEl.style.cssText = "font-size:0.88rem; color:#1d4ed8; margin-bottom:0.5rem; min-height:1.4e

const container = document.createElement("div");
container.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap;";
const c1 = document.createElement("canvas"); c1.width = 220; c1.height = 220;
c1.style.cssText = "border:1px solid #e5e7eb; border-radius:6px;";
const c2 = document.createElement("canvas"); c2.width = 220; c2.height = 220;
c2.style.cssText = "border:1px solid #e5e7eb; border-radius:6px;";
container.appendChild(c1); container.appendChild(c2);

div.appendChild(controls); div.appendChild(infoEl); div.appendChild(container);

function drawPanel(canvas, xMin, xMax, yMin, yMax, fn, color, ptX) {
  const ctx = canvas.getContext("2d");
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  function px(x) { return (x - xMin) / (xMax - xMin) * W; }
  function py(y) { return H - (y - yMin) / (yMax - yMin) * H; }

  // Reflection line y=x
  ctx.strokeStyle = "#d1d5db"; ctx.lineWidth = 1; ctx.setLineDash([4,4]);
  ctx.beginPath(); ctx.moveTo(px(xMin), py(yMin)); ctx.lineTo(px(xMax), py(yMax)); ctx.stroke()
  ctx.setLineDash([]);

  // Axes
  ctx.strokeStyle = "#9ca3af"; ctx.lineWidth = 1;
  ctx.beginPath(); ctx.moveTo(px(xMin), py(0)); ctx.lineTo(px(xMax), py(0)); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(px(0), 0); ctx.lineTo(px(0), H); ctx.stroke();

  // Curve
  ctx.strokeStyle = color; ctx.lineWidth = 2;
  ctx.beginPath();
  let first = true;
  for (let xi = 0; xi <= W; xi++) {
    const x = xMin + (xi / W) * (xMax - xMin);
    const y = fn(x);
    if (isNaN(y) || !isFinite(y) || y < yMin - 0.5 || y > yMax + 0.5) { first = true; continue;
    if (first) { ctx.moveTo(xi, py(y)); first = false; }
    else ctx.lineTo(xi, py(y));
  }
  ctx.stroke();

  // Highlight point
  if (ptX !== null) {
    const ptY = fn(ptX);
```

```
      if (!isNaN(ptY) && isFinite(ptY) && ptY >= yMin && ptY <= yMax) {
        ctx.fillStyle = "#ef4444";
        ctx.beginPath(); ctx.arc(px(ptX), py(ptY), 5, 0, 2*Math.PI); ctx.fill();
        ctx.fillStyle = "#374151"; ctx.font = "11px sans-serif";
        ctx.fillText("(" + ptX.toFixed(0) + ", " + ptY.toFixed(1) + ")", px(ptX)+7, py(ptY)-5);
      }
    }
  }

  function update() {
    const b = parseInt(sl.value);
    spanVal.textContent = b;
    const ptX = b; // highlight x=b on the exp curve
    drawPanel(c1, -2, 4, -1, Math.pow(b, 4)+1, x => Math.pow(b, x), "#2563eb", ptX);
    const logPtX = Math.pow(b, ptX);
    drawPanel(c2, 0.01, Math.pow(b,4)+1, -2, 4, x => Math.log(x)/Math.log(b), "#059669", logPtX > 0
    infoEl.textContent = b + "^" + ptX + " = " + Math.pow(b, ptX).toFixed(0) + "  ↔  log_" + b + "("
  }

  sl.addEventListener("input", update);
  update();
  return div;
}
```

### 21.2.4. Part 4: Log laws and solving logarithmic equations

Logarithms inherit structure from the exponent laws. The three log laws:

| Log law | What it says | Example |
|---|---|---|
| $\log_a(mn) = \log_a(m) + \log_a(n)$ | Log of a product: add logs | $\log_2(4 \times 8) = \log_2(4) + \log_2(8) = 2 + 3 = 5$ |
| $\log_a\left(\frac{m}{n}\right) = \log_a(m) - \log_a(n)$ | Log of a quotient: subtract logs | $\log_{10}(1000/10) = 3 - 1 = 2$ |
| $\log_a(m^n) = n \cdot \log_a(m)$ | Log of a power: bring down the exponent | $\log_2(8^4) = 4 \cdot \log_2(8) = 4 \times 3 = 12$ |

One more, needed for calculators — the **change of base formula**:

$$\log_a(x) = \frac{\ln(x)}{\ln(a)} = \frac{\log(x)}{\log(a)}$$

Your calculator has ln and $\log_{10}$ buttons. This formula lets you compute any log from them.

**Solving $2^x = 10$.**

Take $\log_{10}$ (or ln) of both sides:

$$\log(2^x) = \log(10)$$

Use the power law to bring $x$ down:

$$x \cdot \log(2) = 1$$

Divide:

$$x = \frac{1}{\log(2)} = \frac{1}{0.3010\ldots} \approx 3.32$$

Check: $2^{3.32} \approx 9.97 \approx 10$. Yes.

This means logarithms let us solve for the exponent itself.

**Solving** $\log_3(x) + \log_3(x-2) = 1$**.**

Use the product law to combine the left side:

$$\log_3\big(x(x-2)\big) = 1$$

Convert back to exponential form ($3^1 = x(x-2)$):

$$x^2 - 2x = 3$$

$$x^2 - 2x - 3 = 0$$

$$(x-3)(x+1) = 0$$

So $x = 3$ or $x = -1$. But $\log_3(-1)$ is not defined — you can't take the log of a negative number. Discard $x = -1$.

Answer: $x = 3$. Check: $\log_3(3) + \log_3(1) = 1 + 0 = 1$. Yes.

This means every logarithmic solution must also pass the domain check.

## 21.3. Worked examples

**Example 1 — Science: radioactive half-life.**

Carbon-14 has a half-life of 5730 years — meaning every 5730 years, half the remaining carbon-14 decays. An organic sample contains 1.0 g of carbon-14 today. How much will remain after 17 190 years?

17 190 years = 3 × 5730, so three half-lives have passed.

$$\text{Remaining} = 1.0 \times \left(\frac{1}{2}\right)^3 = \frac{1}{8} = 0.125 \text{ g}$$

If instead you need to find *when* the sample drops to 0.10 g, set up the equation:

$$0.10 = 1.0 \times \left(\frac{1}{2}\right)^{t/5730}$$

Divide both sides by 1.0 to isolate the exponential:

$$0.10 = (0.5)^{t/5730}$$

Take ln of both sides:

$$\ln(0.10) = \ln\left((0.5)^{t/5730}\right)$$

Apply the power law — bring the exponent down:

$$\ln(0.10) = \frac{t}{5730} \cdot \ln(0.5)$$

$$t = 5730 \times \frac{\ln(0.10)}{\ln(0.5)} = 5730 \times \frac{-2.3026}{-0.6931} \approx 5730 \times 3.322 \approx 19\,030 \text{ years}$$

This means exponential decay questions often become logarithm questions when time is the unknown.

---

**Example 2 — Finance: compound interest doubling time.**

An investment earns 7% per year, compounded annually. How long until it doubles?

Let the initial amount be $P$. You want the time $t$ when $P \times 1.07^t = 2P$.

Divide both sides by $P$:

$$1.07^t = 2$$

Take ln of both sides and bring $t$ down:

$$t \cdot \ln(1.07) = \ln(2)$$

$$t = \frac{\ln(2)}{\ln(1.07)} = \frac{0.6931}{0.06766} \approx 10.2 \text{ years}$$

The Rule of 72 is a quick mental estimate: divide 72 by the interest rate. $72/7 \approx 10.3$ — very close. That rule is a log approximation.

This means doubling time depends on the growth rate through a logarithm, not a simple proportion.

---

**Example 3 — Engineering: the decibel.**

Sound intensity is measured on a logarithmic scale. The decibel (dB) level is:

$$L = 10 \cdot \log_{10}\left(\frac{I}{I_0}\right)$$

where $I$ is the intensity of the sound and $I_0 = 10^{-12}$ W/m² is the reference intensity. The reference intensity $I_0 = 10^{-12}$ W/m² is the quietest sound a human ear can detect — it is the standard baseline from which decibels are measured.

Normal conversation is about $I = 10^{-6}$ W/m². What is that in dB?

$$L = 10 \cdot \log_{10}\left(\frac{10^{-6}}{10^{-12}}\right) = 10 \cdot \log_{10}(10^6) = 10 \times 6 = 60 \text{ dB}$$

A sound at 70 dB has what intensity ratio compared to 60 dB?

The difference is 10 dB, which corresponds to a factor of $10^{10/10} = 10^1 = 10$. A 70 dB sound is 10 times as intense as a 60 dB sound. The logarithm compresses very large ratios into small, manageable numbers.

This means logarithmic scales are useful when raw ratios grow too quickly to handle comfortably.

---

**Example 4 — Computing: binary search step count.**

Binary search finds a target in a sorted list by repeatedly halving the search space. How many steps to search a list of $n = 1{,}000{,}000$ items?

After $k$ steps, the search space has been halved $k$ times, leaving at most $n/2^k$ items. The search ends when $n/2^k = 1$:

$$2^k = n \implies k = \log_2(n)$$

For one million items: $k = \log_2(1{,}000{,}000) \approx 19.93$, so at most 20 steps. That's why $O(\log n)$ algorithms are fast — doubling the input adds only one more step.

This means logarithms measure how many repeated halvings or repeated multiplications are needed.

## 21.4. Where this goes

This chapter connects repeated multiplication, growth, decay, and the inverse question "what power was used?"

The derivative of $e^x$ is $e^x$ — the only function that is its own slope. That property, which you can only properly state and prove in differential calculus (Volume 5), is why $e$ appears in every model of continuous growth and decay. You already have the function; calculus will explain what makes it uniquely suited to describe rates of change.

From there, ordinary differential equations (Volume 7) make the connection concrete. The simplest ODE — "a quantity grows proportionally to itself" — has $e^{kt}$ as its solution. Radioactive decay, population growth, cooling, charging capacitors: all of them are that one equation in different physical clothes. This chapter is the front end of that pipeline.

**Where this shows up**

- A physicist uses exponential decay to date ancient organic material via carbon-14 ratios.
- A financial analyst uses logarithms to find how long an investment takes to reach a target value.
- An audio engineer specifies amplifier gain and speaker sensitivity in decibels — a logarithmic unit.
- A software engineer analyses why binary search handles a billion records in about 30 comparisons, while linear search needs a billion.

The base changes. The structure does not.

## 21.5. Exercises

These are puzzles. Each has a clean answer. The interesting part is identifying whether you're working with an exponential, a log, or both — and setting up the equation before you solve it.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
```

```
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align
      const opEl = document.createElement("span");
      opEl.style.cssText = "font-size:0.8rem; color:#6b7280; font-style:italic;";
      opEl.textContent = s.op;
      const eqEl = document.createElement("span");
      eqEl.innerHTML = katex.renderToString(s.eq, {throwOnError: false, displayMode: false});
      row.appendChild(opEl);
      row.appendChild(eqEl);
      container.appendChild(row);
      if (s.note) {
        const noteEl = document.createElement("p");
        noteEl.style.cssText = "margin:0.2rem 0 0.5rem 200px; font-size:0.82rem; color:#374151;";
        noteEl.textContent = s.note;
        container.appendChild(noteEl);
      }
    }
    if (current === totalSteps) {
      const done = document.createElement("p");
      done.style.cssText = "margin-top:0.5rem; font-size:0.85rem; color:#059669; font-weight:600;
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; align-items:center; gap:0.75rem; margin-top:0.5rem;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.35rem 0.85rem; background:#2563eb; color:#fff; border:none;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.35rem 0.85rem; background:#f3f4f6; color:#374151; border:1p
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.8rem; color:#6b7280;";
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); counter.textCon
  resetBtn.onclick = () => { current = 0; nextBtn.disabled = false; renderSteps(); counter.textCo
  counter.textContent = "0 / " + totalSteps;
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  return container;
}
```

**1.** Simplify, writing your answer as a single power:

$$\frac{3^8 \times 3^{-3}}{3^2}$$

```
//| echo: false
{
  const el = makeStepperHTML(1, [
    { op: "Write as an equation", eq: "\\frac{3^8 \\times 3^{-3}}{3^2}", note: null },
    { op: "Multiply in the numerator — add exponents", eq: "\\frac{3^{8 + (-3)}}{3^2} = \\frac{3^5}
    { op: "Divide — subtract exponents", eq: "3^{5 - 2} = 3^3", note: "Quotient law: same base, so s
    { op: "Evaluate", eq: "3^3 = 27", note: null },
    { op: "Check", eq: "\\frac{3^8 \\times 3^{-3}}{3^2} = \\frac{6561 \\times (1/27)}{9} = \\frac{24
  ]);
  return el;
}
```

---

**2.** Solve for $x$:

$$4^x = 128$$

```
//| echo: false
{
  const el = makeStepperHTML(2, [
    { op: "Write as an equation", eq: "4^x = 128", note: null },
    { op: "Express both sides as powers of 2", eq: "(2^2)^x = 2^7", note: "4 = 2² and 128 = 2⁷, so r
    { op: "Use the power-of-a-power law", eq: "2^{2x} = 2^7", note: null },
    { op: "Match exponents — same base, so the powers must be equal", eq: "2x = 7", note: null },
    { op: "Divide both sides by 2", eq: "x = \\frac{7}{2} = 3.5", note: null },
    { op: "Check", eq: "4^{3.5} = 4^3 \\times 4^{0.5} = 64 \\times 2 = 128 \\checkmark", note: null
  ]);
  return el;
}
```

---

**3.** A population of bacteria doubles every 3 hours. You start with 400 bacteria. How many hours until there are more than 25 600?

```
//| echo: false
{
  const el = makeStepperHTML(3, [
    { op: "Write as an equation", eq: "400 \\times 2^{t/3} > 25600", note: "Every 3 hours is one dou
    { op: "Divide both sides by 400", eq: "2^{t/3} > 64", note: null },
    { op: "Express 64 as a power of 2", eq: "2^{t/3} > 2^6", note: null },
```

```
    { op: "Match exponents", eq: "\\frac{t}{3} > 6", note: null },
    { op: "Multiply both sides by 3", eq: "t > 18 \\text{ hours}", note: null },
    { op: "Check", eq: "400 \\times 2^{18/3} = 400 \\times 2^6 = 400 \\times 64 = 25600 \\checkma
  ]);
  return el;
}
```

---

**4.** Use logarithms to solve for $t$:

$$5^t = 200$$

Give your answer to two decimal places.

```
//| echo: false
{
  const el = makeStepperHTML(4, [
    { op: "Write as an equation", eq: "5^t = 200", note: null },
    { op: "Take ln of both sides", eq: "\\ln(5^t) = \\ln(200)", note: "You can use log base 10 eq
    { op: "Use the power law — bring t down", eq: "t \\cdot \\ln(5) = \\ln(200)", note: "\\ln(m^n
    { op: "Divide both sides by ln(5)", eq: "t = \\frac{\\ln(200)}{\\ln(5)} = \\frac{5.2983}{1.60
    { op: "Evaluate", eq: "t \\approx 3.29", note: null },
    { op: "Check", eq: "5^{3.29} \\approx 5^3 \\times 5^{0.29} \\approx 125 \\times 1.60 \\approx
  ]);
  return el;
}
```

---

**5.** Solve the logarithmic equation:

$$\log_2(x+4) + \log_2(x-4) = 5$$

```
//| echo: false
{
  const el = makeStepperHTML(5, [
    { op: "Write as an equation", eq: "\\log_2(x+4) + \\log_2(x-4) = 5", note: null },
    { op: "Use the product law — combine the two logs", eq: "\\log_2\\bigl((x+4)(x-4)\\bigr) = 5"
    { op: "Expand the bracket", eq: "\\log_2(x^2 - 16) = 5", note: "(x+4)(x-4) = x^2 - 16. This u
    { op: "Convert to exponential form", eq: "x^2 - 16 = 2^5 = 32", note: "\\log_2(k) = 5 \\text{
    { op: "Solve the quadratic", eq: "x^2 = 48 \\implies x = \\pm\\sqrt{48} = \\pm 4\\sqrt{3} \\a
    { op: "Reject the negative solution", eq: "x = 4\\sqrt{3} \\approx 6.93", note: "x = -4\\sqrt
    { op: "Check", eq: "\\log_2(4\\sqrt{3}+4) + \\log_2(4\\sqrt{3}-4) = \\log_2\\bigl((4\\sqrt{3}
```

```
  ]);
  return el;
}
```

---

**6.** The pH of a solution is defined as $\text{pH} = -\log_{10}[\text{H}^+]$, where $[\text{H}^+]$ denotes the concentration of hydrogen ions in moles per litre — this is standard chemistry notation. Pure water has $[\text{H}^+] = 10^{-7}$ mol/L, giving a pH of 7. A lemon juice sample has $[\text{H}^+] = 10^{-2.4}$ mol/L. What is its pH, and how many times more acidic is it than pure water?

```
//| echo: false
{
  const el = makeStepperHTML(6, [
    { op: "Write as an equation", eq: "\\text{pH} = -\\log_{10}(10^{-2.4})", note: null },
    { op: "Evaluate the log", eq: "\\text{pH} = -(-2.4) = 2.4", note: "\\log_{10}(10^k) = k" },
    { op: "Find the ratio of hydrogen ion concentrations", eq: "\\frac{[\\text{H}^+]_{\\text{lemon}}
    { op: "Subtract exponents — dividing with the same base", eq: "= 10^{-2.4 - (-7)} = 10^{4.6}", n
    { op: "Evaluate", eq: "10^{4.6} \\approx 39\\,811", note: null },
    { op: "Check", eq: "\\text{pH of lemon} = 2.4;\\ 10^{7 - 2.4} = 10^{4.6} \\approx 39\\,811 \\tex
  ]);
  return el;
}
```

# 22. Complex numbers

What lives where the number line doesn't go

You've been solving quadratic equations. Most of them have real solutions. But every now and then the discriminant — the bit under the square root — comes out negative.

Try $x^2 + 1 = 0$. Rearrange: $x^2 = -1$. Take the square root: $x = \sqrt{-1}$.

That's impossible on the number line. There is no real number whose square is negative. Positive times positive is positive. Negative times negative is also positive. Nothing multiplied by itself gives $-1$.

So for a long time, equations like this were said to have "no solution."

That answer was unsatisfying, and mathematicians knew it. The fix was the same fix that introduced negative numbers, fractions, and irrational numbers: *give the new thing a name, and work out its arithmetic.*

The name for $\sqrt{-1}$ is $i$.

Once you do that, something unexpected happens. The same number system that lets you give $x^2 + 1 = 0$ an answer also turns out to describe oscillating voltages in electrical circuits, the quantum states of particles, and the algorithm that your phone uses to process audio. None of that was the motivation. It fell out of the arithmetic.

There's also this: $e^{i\pi} + 1 = 0$. Five of the most important numbers in mathematics — $e$, $i$, $\pi$, 1, 0 — in one equation. We'll see why by the end of this chapter.

## 22.1. What the notation is saying

### 22.1.1. The imaginary unit

Define $i$ by one rule:

$$i^2 = -1$$

That's the whole definition. $i$ is the number whose square is $-1$. It's not a number on the real number line — it's something new. This means complex numbers do not extend the number line by stretching it. They extend arithmetic by adding a new direction.

From this one rule, powers of $i$ cycle in a pattern:

$$i^1 = i \qquad i^2 = -1 \qquad i^3 = -i \qquad i^4 = 1 \qquad i^5 = i \qquad \ldots$$

Every four powers, the cycle repeats. That's a hint that $i$ has something to do with rotation — four quarter-turns bring you back to where you started. This means repeated multiplication by $i$ behaves more like turning than like moving left or right on a line.

### 22.1.2. Complex numbers

A **complex number** is a number of the form:

$$z = a + bi$$

Read it as: "the real part $a$, plus $b$ times $i$." Here $a$ and $b$ are ordinary real numbers. $a$ is called the **real part** of $z$, written $\mathrm{Re}(z)$. $b$ is called the **imaginary part**, written $\mathrm{Im}(z)$. This means a complex number keeps the familiar real-number part, then adds a second part that tracks how much of the new $i$-direction is present.

Examples: $3 + 4i$, $-2 + i$, $5$ (which is $5 + 0i$), $3i$ (which is $0 + 3i$).

Real numbers are a special case: they're complex numbers where $b = 0$.

### 22.1.3. The complex plane

Every complex number $a + bi$ corresponds to a point in a plane. Put the real axis horizontal and the imaginary axis vertical. The number $3 + 4i$ is the point three units right and four units up.

This is the **Argand diagram**, or the **complex plane**. The horizontal axis is the real line you already know. The vertical axis is entirely imaginary numbers. Every complex number lives at exactly one point in this plane. This means complex numbers are easier to understand as positions in a plane than as mysterious symbols attached to square roots of negatives.

```
//| echo: false
{
  const div = document.createElement("div");
  div.style.cssText = "font-family:inherit; padding:0.5rem 0;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:1.5rem; flex-wrap:wrap; margin-bottom:0.75rem;";

  function makeSlider(label, min, max, step, val) {
    const wrap = document.createElement("div");
    const lbl = document.createElement("label");
    lbl.style.cssText = "font-size:0.85rem; color:#374151; display:block; margin-bottom:0.2rem;";
    const span = document.createElement("span"); span.textContent = val;
    lbl.appendChild(document.createTextNode(label + " = ")); lbl.appendChild(span);
    const input = document.createElement("input");
```

```
  input.type = "range"; input.min = min; input.max = max; input.step = step; input.value = val;
  input.style.width = "150px";
  wrap.appendChild(lbl); wrap.appendChild(input);
  return {wrap, input, span};
}

const aS = makeSlider("a (real part)", -5, 5, 0.5, 3);
const bS = makeSlider("b (imaginary part)", -5, 5, 0.5, 2);
controls.appendChild(aS.wrap);
controls.appendChild(bS.wrap);

const infoEl = document.createElement("div");
infoEl.style.cssText = "font-size:0.9rem; color:#1d4ed8; margin-bottom:0.5rem; min-height:1.4em

const canvas = document.createElement("canvas");
canvas.width = 340; canvas.height = 340;
canvas.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; display:block; max-width:3

div.appendChild(controls); div.appendChild(infoEl); div.appendChild(canvas);

function draw() {
  const a = parseFloat(aS.input.value);
  const b = parseFloat(bS.input.value);
  aS.span.textContent = a.toFixed(1);
  bS.span.textContent = b.toFixed(1);

  const r = Math.sqrt(a*a + b*b);
  const theta = Math.atan2(b, a);
  const thetaDeg = (theta * 180 / Math.PI).toFixed(1);
  infoEl.textContent = "z = " + a.toFixed(1) + (b>=0?"+":"") + b.toFixed(1) + "i   |z| = " + r.

  const ctx = canvas.getContext("2d");
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);

  const range = 6;
  function px(x) { return (x + range) / (2*range) * W; }
  function py(y) { return H - (y + range) / (2*range) * H; }

  // Grid
  ctx.strokeStyle = "#f3f4f6"; ctx.lineWidth = 1;
  for (let i = -5; i <= 5; i++) {
    ctx.beginPath(); ctx.moveTo(px(i), 0); ctx.lineTo(px(i), H); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(0, py(i)); ctx.lineTo(W, py(i)); ctx.stroke();
  }

  // Axes
```

```
    ctx.strokeStyle = "#9ca3af"; ctx.lineWidth = 1.5;
    ctx.beginPath(); ctx.moveTo(0, py(0)); ctx.lineTo(W, py(0)); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(px(0), 0); ctx.lineTo(px(0), H); ctx.stroke();
    ctx.fillStyle = "#6b7280"; ctx.font = "11px sans-serif";
    ctx.fillText("Re", W-20, py(0)-5); ctx.fillText("Im", px(0)+5, 12);

    // Modulus line
    ctx.strokeStyle = "#2563eb"; ctx.lineWidth = 2;
    ctx.beginPath(); ctx.moveTo(px(0), py(0)); ctx.lineTo(px(a), py(b)); ctx.stroke();

    // Angle arc
    if (r > 0.2) {
      ctx.strokeStyle = "#f59e0b"; ctx.lineWidth = 1.5;
      ctx.beginPath();
      ctx.arc(px(0), py(0), 30, -Math.atan2(b, a), 0, b > 0);
      ctx.stroke();
      ctx.fillStyle = "#d97706"; ctx.font = "11px sans-serif";
      ctx.fillText("θ", px(0)+34, py(0)-8);
    }

    // Dashed projection lines
    ctx.strokeStyle = "#cbd5e1"; ctx.lineWidth = 1; ctx.setLineDash([4,4]);
    ctx.beginPath(); ctx.moveTo(px(a), py(0)); ctx.lineTo(px(a), py(b)); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(px(0), py(b)); ctx.lineTo(px(a), py(b)); ctx.stroke();
    ctx.setLineDash([]);

    // Point
    ctx.fillStyle = "#ef4444";
    ctx.beginPath(); ctx.arc(px(a), py(b), 6, 0, 2*Math.PI); ctx.fill();

    // Labels
    ctx.fillStyle = "#374151"; ctx.font = "bold 12px sans-serif";
    ctx.fillText("z = " + a.toFixed(1) + (b>=0?"+":"") + b.toFixed(1) + "i", px(a)+8, py(b)-8);
    ctx.fillStyle = "#2563eb"; ctx.font = "11px sans-serif";
    ctx.fillText("|z| = " + r.toFixed(2), px(a/2)-5, py(b/2)-10);
  }

  aS.input.addEventListener("input", draw);
  bS.input.addEventListener("input", draw);
  draw();
  return div;
}
```

## 22.2. The method

> **Complex numbers in 2D graphics**
> Complex numbers appear in physics and engineering, but their geometry is already useful in 2D graphics. Rotating a sprite on a screen by angle $\theta$ is the same as multiplying its position (written as a complex number $x + yi$) by $\cos\theta + i\sin\theta$. Game engines doing this in bulk use the same arithmetic you are about to practise.

### 22.2.1. Part 1: Arithmetic

**Adding and subtracting** complex numbers works the way you'd expect: add the real parts together, add the imaginary parts together.

$$(3 + 4i) + (1 - 2i) = (3 + 1) + (4 - 2)i = 4 + 2i$$

$$(3 + 4i) - (1 - 2i) = (3 - 1) + (4 - (-2))i = 2 + 6i$$

This means addition and subtraction treat the real and imaginary parts as two separate coordinates.

**Multiplying** uses FOIL (expand like brackets), then replace every $i^2$ with $-1$.

$$(3 + 4i)(1 - 2i) = 3 \cdot 1 + 3 \cdot (-2i) + 4i \cdot 1 + 4i \cdot (-2i)$$

$$= 3 - 6i + 4i - 8i^2$$

Now replace $i^2 = -1$:

$$= 3 - 6i + 4i - 8(-1) = 3 - 6i + 4i + 8 = 11 - 2i$$

The key move: $i^2$ always becomes $-1$. That's the entire rule for complex multiplication. This means complex multiplication still follows ordinary algebra, but every time a pair of $i$'s appears it folds back into a real number.

**Dividing** requires a trick. A complex denominator means the fraction can't be split into real and imaginary parts yet — you need a real number on the bottom. Multiplying by the conjugate achieves this, because $(a + bi)(a - bi) = a^2 + b^2$, which is always real. The tool is the **complex conjugate**: if $z = a + bi$, its conjugate is $\bar{z} = a - bi$ — same real part, imaginary part negated.

Multiplying a complex number by its conjugate always gives a real number:

$$(a + bi)(a - bi) = a^2 - (bi)^2 = a^2 - b^2i^2 = a^2 + b^2$$

To divide, multiply numerator and denominator by the conjugate of the denominator.

$$\frac{3+4i}{1-2i} \cdot \frac{1+2i}{1+2i} = \frac{(3+4i)(1+2i)}{(1-2i)(1+2i)}$$

Denominator: $(1)^2 + (2)^2 = 5$

Numerator: $3 + 6i + 4i + 8i^2 = 3 + 10i - 8 = -5 + 10i$

Result: $\frac{-5+10i}{5} = -1 + 2i$

This means the conjugate is the tool that converts a messy complex denominator into an ordinary real one.

### 22.2.2. Part 2: Modulus and argument

The **modulus** of $z = a + bi$, written $|z|$, is its distance from the origin in the complex plane. By Pythagoras:

$$|z| = \sqrt{a^2 + b^2}$$

For $z = 3 + 4i$: $|z| = \sqrt{9+16} = \sqrt{25} = 5$. This means the modulus tells you how far the number is from zero, not where it lies on a line but where it sits in the plane.

The **argument** of $z$, written $\arg(z)$, is the angle the line from the origin to $z$ makes with the positive real axis. Measured in radians (or degrees):

$$\arg(z) = \arctan\left(\frac{b}{a}\right)$$

Be careful with the quadrant. The arctan function returns values in the range $(-\pi/2, \pi/2)$, so you need to check whether $z$ is in the left half of the plane and adjust by $\pm\pi$ if so.

For $z = 3 + 4i$: $\arg(z) = \arctan(4/3) \approx 0.927$ radians ($\approx 53.1°$).

Together, modulus and argument give two ways to describe the same point: Cartesian form $a + bi$ and polar form $r\angle\theta$. This means every complex number can be described either by its horizontal and vertical parts, or by its size and direction.

### 22.2.3. Part 3: Polar form and Euler's formula

On the unit circle (where $r = 1$), the real part of a complex number at angle $\theta$ is $\cos\theta$ and the imaginary part is $\sin\theta$ — by the same definition of cosine and sine used in trigonometry. So any point at distance $r$ from the origin at angle $\theta$ is $r$ times that unit-circle point.

A complex number with modulus $r$ and argument $\theta$ can be written as:

$$z = r(\cos\theta + i\sin\theta)$$

This is **polar form**. Read it as: go distance $r$ from the origin, at angle $\theta$ from the positive real axis. This means polar form packages the geometry directly: length first, then angle.

Now for the extraordinary fact. **Euler's formula** says:

$$e^{i\theta} = \cos\theta + i\sin\theta$$

This connects the exponential function — which you know from growth and decay — to the trigonometric functions. The proof belongs to calculus (it comes from comparing the Taylor series of $e^x$, $\cos x$, and $\sin x$), but you can use the result here.

With Euler's formula, polar form becomes simply:

$$z = re^{i\theta}$$

This is often the cleanest way to write a complex number when you're multiplying, dividing, or raising to a power. This means exponentials are not separate from trigonometry here; they are a compact way to encode rotation.

The famous identity $e^{i\pi} + 1 = 0$ is just the special case $\theta = \pi$: $e^{i\pi} = \cos\pi + i\sin\pi = -1 + 0i = -1$.

In particular, $i = e^{i\pi/2}$ (since $\cos(\pi/2) = 0$ and $\sin(\pi/2) = 1$). Because multiplication adds arguments, multiplying any complex number by $i$ adds $\pi/2$ to its angle — a 90° anticlockwise rotation in the plane.

```
//| echo: false
{
  const div = document.createElement("div");
  div.style.cssText = "font-family:inherit; padding:0.5rem 0;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:1.5rem; flex-wrap:wrap; margin-bottom:0.75rem;";

  function makeSlider(label, min, max, step, val) {
    const wrap = document.createElement("div");
    const lbl = document.createElement("label");
    lbl.style.cssText = "font-size:0.85rem; color:#374151; display:block; margin-bottom:0.2rem;";
    const span = document.createElement("span"); span.textContent = val;
    lbl.appendChild(document.createTextNode(label + " = ")); lbl.appendChild(span);
    const input = document.createElement("input");
    input.type = "range"; input.min = min; input.max = max; input.step = step; input.value = val;
    input.style.width = "140px";
    wrap.appendChild(lbl); wrap.appendChild(input);
    return {wrap, input, span};
  }

  const r1S = makeSlider("r₁ (|z₁|)", 0.5, 3, 0.5, 1.5);
  const t1S = makeSlider("θ₁ (arg z₁, °)", 0, 360, 15, 45);
  const r2S = makeSlider("r₂ (|z₂|)", 0.5, 3, 0.5, 1);
  const t2S = makeSlider("θ₂ (arg z₂, °)", 0, 360, 15, 30);
```

```
controls.appendChild(r1S.wrap);
controls.appendChild(t1S.wrap);
controls.appendChild(r2S.wrap);
controls.appendChild(t2S.wrap);

const infoEl = document.createElement("div");
infoEl.style.cssText = "font-size:0.85rem; color:#374151; margin-bottom:0.5rem; min-height:2em; ba

const canvas = document.createElement("canvas");
canvas.width = 360; canvas.height = 360;
canvas.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; display:block; max-width:360p

div.appendChild(controls); div.appendChild(infoEl); div.appendChild(canvas);

function draw() {
  const r1 = parseFloat(r1S.input.value);
  const t1d = parseFloat(t1S.input.value);
  const r2 = parseFloat(r2S.input.value);
  const t2d = parseFloat(t2S.input.value);
  r1S.span.textContent = r1S.input.value;
  t1S.span.textContent = t1S.input.value + "°";
  r2S.span.textContent = r2S.input.value;
  t2S.span.textContent = t2S.input.value + "°";

  const t1 = t1d * Math.PI / 180;
  const t2 = t2d * Math.PI / 180;
  const z1 = {r: r1, a: r1*Math.cos(t1), b: r1*Math.sin(t1)};
  const z2 = {r: r2, a: r2*Math.cos(t2), b: r2*Math.sin(t2)};
  const rp = r1*r2;
  const tp = t1+t2;
  const prod = {a: rp*Math.cos(tp), b: rp*Math.sin(tp)};

  infoEl.innerHTML =
    "z₁ = " + z1.a.toFixed(2) + (z1.b>=0?"+":"") + z1.b.toFixed(2) + "i  |z₁|=" + r1 + "  arg=" +
    "z₂ = " + z2.a.toFixed(2) + (z2.b>=0?"+":"") + z2.b.toFixed(2) + "i  |z₂|=" + r2 + "  arg=" +
    "z₁·z₂ = " + prod.a.toFixed(2) + (prod.b>=0?"+":"") + prod.b.toFixed(2) + "i  |z₁·z₂|=" + rp.t

  const ctx = canvas.getContext("2d");
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const range = 5;
  function px(x) { return (x+range)/(2*range)*W; }
  function py(y) { return H-(y+range)/(2*range)*H; }

  ctx.strokeStyle = "#9ca3af"; ctx.lineWidth = 1;
  ctx.beginPath(); ctx.moveTo(0, py(0)); ctx.lineTo(W, py(0)); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(px(0), 0); ctx.lineTo(px(0), H); ctx.stroke();
```

```
    ctx.fillStyle = "#9ca3af"; ctx.font = "11px sans-serif";
    ctx.fillText("Re", W-20, py(0)-4); ctx.fillText("Im", px(0)+4, 12);

    const pts = [
      {x: z1.a, y: z1.b, color: "#2563eb", label: "z₁"},
      {x: z2.a, y: z2.b, color: "#059669", label: "z₂"},
      {x: prod.a, y: prod.b, color: "#ef4444", label: "z₁·z₂"},
    ];
    pts.forEach(p => {
      ctx.strokeStyle = p.color; ctx.lineWidth = 2;
      ctx.beginPath(); ctx.moveTo(px(0), py(0)); ctx.lineTo(px(p.x), py(p.y)); ctx.stroke();
      ctx.fillStyle = p.color;
      ctx.beginPath(); ctx.arc(px(p.x), py(p.y), 5, 0, 2*Math.PI); ctx.fill();
      ctx.fillStyle = "#374151"; ctx.font = "bold 11px sans-serif";
      ctx.fillText(p.label, px(p.x)+7, py(p.y)-5);
    });

    // Angle arcs
    [t1, t2, tp].forEach((t, i) => {
      const colors = ["#2563eb","#059669","#ef4444"];
      ctx.strokeStyle = colors[i]; ctx.lineWidth = 1.5;
      const arcR = 25 + i*10;
      ctx.beginPath();
      ctx.arc(px(0), py(0), arcR, -t, 0, t < 0);
      ctx.stroke();
    });
  }

  r1S.input.addEventListener("input", draw);
  t1S.input.addEventListener("input", draw);
  r2S.input.addEventListener("input", draw);
  t2S.input.addEventListener("input", draw);
  draw();
  return div;
}
```

### 22.2.4. Part 4: De Moivre's theorem

Because $z = re^{i\theta}$, raising $z$ to the power $n$ is clean:

$$z^n = (re^{i\theta})^n = r^n e^{in\theta} = r^n(\cos n\theta + i \sin n\theta)$$

This is **de Moivre's theorem**. To find the $n$th power of a complex number: raise the modulus to the $n$th power, multiply the argument by $n$. This means powers of complex numbers are easiest in polar form because multiplication becomes "stretch and rotate."

Example: find $(1 + i)^4$.

First, convert to polar form. $|1+i| = \sqrt{2}$. For the argument: $\arg(1+i) = \arctan(1/1) = \arctan(1) = \pi/4$. So $1 + i = \sqrt{2}\, e^{i\pi/4}$.

Then:

$$(1+i)^4 = (\sqrt{2})^4 \cdot e^{i \cdot 4 \cdot \pi/4} = 4 \cdot e^{i\pi} = 4 \cdot (-1) = -4$$

Check by direct multiplication: $(1+i)^2 = 1 + 2i - 1 = 2i$. Then $(2i)^2 = 4i^2 = -4$. Matches. This means polar form is not changing the answer; it is giving a faster route to the same answer.

> **Why this works**
> Multiplying any complex number by $i$ adds $\pi/2$ to its angle — a 90° anticlockwise rotation in the complex plane. That's why $i^4 = 1$: four quarter-turns bring you back.
> Complex multiplication in general is a *rotation and scaling* operation. When you multiply $z_1 \cdot z_2$, the moduli multiply (scaling) and the arguments add (rotation). This is exactly why complex numbers are the natural language for anything involving oscillation, phase, and rotation — from AC circuits to quantum wavefunctions to signal processing. The arithmetic of complex numbers is the arithmetic of rotations.

## 22.3. Worked examples

### 22.3.1. Example 1 — Engineering: AC circuit impedance

An AC circuit has a resistor and a capacitor in series. The resistor has impedance $Z_R = 50\,\Omega$ (purely real). The capacitor has impedance $Z_C = -30i\,\Omega$ (purely imaginary, negative because capacitors shift the phase of current).

The total series impedance is:

$$Z = Z_R + Z_C = 50 + (-30i) = 50 - 30i\,\Omega$$

The magnitude of the impedance — how much the circuit resists current overall — is the modulus:

$$|Z| = \sqrt{50^2 + 30^2} = \sqrt{2500 + 900} = \sqrt{3400} \approx 58.3\,\Omega$$

The phase angle — how much voltage leads current — is the argument:

$$\arg(Z) = \arctan\left(\frac{-30}{50}\right) \approx -30.96^\circ$$

The negative angle means the voltage lags behind the current by about 31°, which is the expected behaviour for a capacitive circuit. This means the same complex number records two useful facts at once: overall size and phase shift.

---

### 22.3.2. Example 2 — Physics: Quantum wavefunction amplitude

In quantum mechanics, a particle in a particular state has a probability amplitude that is a complex number. The probability of measuring that state is the square of the modulus of the amplitude.

Suppose the amplitude is $\psi = \frac{3}{5} + \frac{4}{5}i$.

The probability is:

$$|\psi|^2 = \left(\frac{3}{5}\right)^2 + \left(\frac{4}{5}\right)^2 = \frac{9}{25} + \frac{16}{25} = \frac{25}{25} = 1$$

A probability of 1 means the particle is certain to be found in this state. The amplitude is normalised — its modulus is 1. This is a constraint quantum mechanics imposes on all valid probability amplitudes: $|\psi| = 1$. This means the modulus turns a complex amplitude into a measurable physical quantity.

---

### 22.3.3. Example 3 — Computing: FFT butterfly

The Fast Fourier Transform (FFT) is an algorithm that decomposes a signal into its frequency components. Its key step multiplies a complex number by a 'twiddle factor' $W$ — which is just a complex number on the unit circle — to rotate it by a precise angle.

The FFT processes a signal by repeatedly multiplying values by twiddle factors of the form $W = e^{-2\pi ik/N}$ for various integers $k$ and $N$.

For an 8-point FFT with $k = 1$, $N = 8$:

$$W = e^{-2\pi i/8} = e^{-i\pi/4} = \cos\left(-\frac{\pi}{4}\right) + i\sin\left(-\frac{\pi}{4}\right) = \frac{1}{\sqrt{2}} - \frac{i}{\sqrt{2}}$$

One butterfly step multiplies a signal value $X = 3 + 2i$ by this twiddle factor:

$$X \cdot W = (3 + 2i)\left(\frac{1}{\sqrt{2}} - \frac{i}{\sqrt{2}}\right)$$

$$= \frac{1}{\sqrt{2}}(3 + 2i)(1 - i) = \frac{1}{\sqrt{2}}(3 - 3i + 2i - 2i^2)$$

$$= \frac{1}{\sqrt{2}}(3 - i + 2) = \frac{1}{\sqrt{2}}(5 - i) = \frac{5}{\sqrt{2}} - \frac{i}{\sqrt{2}} \approx 3.54 - 0.71i$$

In polar terms: multiplication by $W$ kept the modulus the same (since $|W| = 1$) and rotated the point by $-45°$. The FFT is, at its core, a very efficient way to apply many such rotations simultaneously. This means FFT arithmetic works because complex multiplication can rotate data without changing its size.

## 22.4. Where this goes

This chapter began with a quadratic equation that had no real solution, then showed that the same new arithmetic also describes rotation, phase, and waves. The next destinations build on exactly that shift.

**Polynomials (Vol 3 Ch 3)** — every polynomial with real coefficients that has no real roots has complex roots. The quadratic formula $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ gives complex answers when $b^2 - 4ac < 0$. You can now say exactly what those answers are and verify them by substituting back in. The discriminant is no longer a dead end.

**Complex analysis (Vol 7)** extends calculus to complex-valued functions. It turns out that calculus in the complex plane is extraordinarily powerful — more constrained than real calculus in ways that produce surprising results, including a method for evaluating real integrals that seem impossible by ordinary means. The Cauchy integral theorem, residues, and conformal mapping all live here. Euler's formula is the entry point.

**Differential calculus (Vol 5)** — Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$ is proved by comparing the Taylor series expansions of each side. That proof belongs to calculus, but once you have it, it explains why $e$, $i$, $\pi$, 1, and 0 are connected: they're all special values of the same exponential function operating on the complex plane.

**Where this shows up**

- An electrical engineer analysing an AC circuit writes impedance as a complex number, computes its modulus to find the magnitude, and its argument to find the phase shift. The letter $j$ is used instead of $i$ in electrical engineering (because $i$ is already taken for current), but the mathematics is identical.
- A physicist writing down the Schrödinger equation is working with a complex-valued wavefunction. The probability of measurement outcomes is the squared modulus of that function.
- A software engineer implementing a digital audio filter uses the Discrete Fourier Transform — which is an algorithm entirely built on complex multiplication. The FFT makes this computation fast enough to run in real time.
- A control engineer analysing whether a feedback system is stable places poles in the complex plane. Poles in the left half-plane mean the system is stable; poles in the right half-plane mean it oscillates out of control.

## 22.5. Exercises

These are puzzles. Each one has a clean answer. The interesting part is knowing which form of a complex number to use for which operation.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align
      const opEl = document.createElement("span");
      opEl.style.cssText = "font-size:0.8rem; color:#6b7280; font-style:italic;";
      opEl.textContent = s.op;
      const eqEl = document.createElement("span");
      eqEl.innerHTML = katex.renderToString(s.eq, {throwOnError: false, displayMode: false});
      row.appendChild(opEl);
      row.appendChild(eqEl);
      container.appendChild(row);
      if (s.note) {
        const noteEl = document.createElement("p");
        noteEl.style.cssText = "margin:0.2rem 0 0.5rem 200px; font-size:0.82rem; color:#374151;";
        noteEl.textContent = s.note;
        container.appendChild(noteEl);
      }
    }
    if (current === totalSteps) {
      const done = document.createElement("p");
      done.style.cssText = "margin-top:0.5rem; font-size:0.85rem; color:#059669; font-weight:600;
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; align-items:center; gap:0.75rem; margin-top:0.5rem;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.35rem 0.85rem; background:#2563eb; color:#fff; border:none;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.35rem 0.85rem; background:#f3f4f6; color:#374151; border:1p
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.8rem; color:#6b7280;";
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); counter.textCon
```

```
  resetBtn.onclick = () => { current = 0; nextBtn.disabled = false; renderSteps(); counter.textConte
  counter.textContent = "0 / " + totalSteps;
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  return container;
}
```

**1.** Compute $(2 + 3i)(4 - i)$.

```
//| echo: false
{
  const el = makeStepperHTML(1, [
    { op: "Expand using FOIL", eq: "(2 + 3i)(4 - i) = 2 \\cdot 4 + 2 \\cdot (-i) + 3i \\cdot 4 + 3i
    { op: "Collect terms", eq: "= 8 - 2i + 12i - 3i^2", note: null },
    { op: "Replace i² = −1", eq: "= 8 - 2i + 12i - 3(-1) = 8 + 3 + (-2 + 12)i", note: "Every i² beco
    { op: "Write the answer", eq: "= 11 + 10i", note: null },
    { op: "Check", eq: "(2 + 3i)(4 - i) = 11 + 10i \\checkmark \\text{ — verify by expanding again}"
  ]);
  return el;
}
```

**2.** Find the modulus and argument of $z = -1 + \sqrt{3}\,i$.

```
//| echo: false
{
  const el = makeStepperHTML(2, [
    { op: "Identify real and imaginary parts", eq: "a = -1,\\quad b = \\sqrt{3}", note: null },
    { op: "Compute the modulus", eq: "|z| = \\sqrt{(-1)^2 + (\\sqrt{3})^2} = \\sqrt{1 + 3} = \\sqrt
    { op: "Compute arctan(b/a)", eq: "\\arctan\\!\\left(\\frac{\\sqrt{3}}{-1}\\right) = \\arctan(-\\
    { op: "Adjust for correct quadrant", eq: "z \\text{ is in the second quadrant (a < 0, b > 0), so
    { op: "State the answer", eq: "|z| = 2,\\quad \\arg(z) = \\frac{2\\pi}{3} \\approx 120°", note:
    { op: "Check", eq: "2\\!\\left(\\cos\\frac{2\\pi}{3} + i\\sin\\frac{2\\pi}{3}\\right) = 2\\!\\le
  ]);
  return el;
}
```

**3.** Convert $z = 4e^{i\pi/6}$ to Cartesian form $a + bi$.

```
//| echo: false
{
  const el = makeStepperHTML(3, [
    { op: "Apply Euler's formula", eq: "z = 4\\!\\left(\\cos\\frac{\\pi}{6} + i\\sin\\frac{\\pi}{6}"
    { op: "Evaluate the trig values", eq: "\\cos\\frac{\\pi}{6} = \\frac{\\sqrt{3}}{2},\\quad \\sin"
```

```
    { op: "Multiply through by 4", eq: "z = 4 \\cdot \\frac{\\sqrt{3}}{2} + 4 \\cdot \\frac{1}{2}
    { op: "Check", eq: "|2\\sqrt{3} + 2i| = \\sqrt{(2\\sqrt{3})^2 + 2^2} = \\sqrt{12 + 4} = \\sqr
  ]);
  return el;
}
```

**4.** Compute $(-1 + i)^6$ using de Moivre's theorem.

```
//| echo: false
{
  const el = makeStepperHTML(4, [
    { op: "Convert to polar form", eq: "|-1 + i| = \\sqrt{1 + 1} = \\sqrt{2},\\quad \\arg(-1 + i)
    { op: "Write in exponential form", eq: "-1 + i = \\sqrt{2}\\,e^{i \\cdot 3\\pi/4}", note: nul
    { op: "Apply de Moivre: raise modulus to n, multiply argument by n", eq: "(-1 + i)^6 = (\\sqr
    { op: "Reduce the angle mod 2", eq: "\\frac{9\\pi}{2} = 4\\pi + \\frac{\\pi}{2},\\quad \\tex
    { op: "Write the answer", eq: "(-1 + i)^6 = 8i", note: null },
    { op: "Check", eq: "(-1+i)^2 = 1 - 2i + i^2 = -2i;\\quad (-2i)^3 = -8i^3 = -8(-i) = 8i \\chec
  ]);
  return el;
}
```

**5.** Divide $\dfrac{2 + i}{3 - 4i}$.

```
//| echo: false
{
  const el = makeStepperHTML(5, [
    { op: "Multiply numerator and denominator by the conjugate of the denominator", eq: "\\frac{2
    { op: "Expand the denominator", eq: "(3 - 4i)(3 + 4i) = 9 + 16 = 25", note: "(a − bi)(a + bi)
    { op: "Expand the numerator", eq: "(2 + i)(3 + 4i) = 6 + 8i + 3i + 4i^2 = 6 + 11i - 4 = 2 + 1
    { op: "Write the answer", eq: "\\frac{2 + 11i}{25} = \\frac{2}{25} + \\frac{11}{25}i", note:
    { op: "Check", eq: "\\left(\\frac{2}{25} + \\frac{11}{25}i\\right)(3 - 4i) = \\frac{1}{25}(2
  ]);
  return el;
}
```

# 23. Systems of equations

Two unknowns, two constraints

Two streaming plans. Plan A charges \$6 per month plus \$3 per film. Plan B charges \$15 per month with unlimited films. At some number of films per month they cost exactly the same. Where is that crossover?

Two walkers start from opposite ends of a 9 km path. One walks at 4 km/h, the other at 5 km/h. When do they meet, and where?

Both questions have the same shape: two things you don't know, two separate facts about them. A single equation can pin down one unknown. To pin down two, you need two.

## 23.1. What the notation is saying

Here are the two streaming plans written as equations. Let $m$ = months (fixed at 1, so we can drop it) and $f$ = number of films rented:

$$\begin{cases} \text{Cost}_A = 6 + 3f \\ \text{Cost}_B = 15 \end{cases}$$

When the costs are equal: $6 + 3f = 15$. One unknown, one equation — you already know how to solve this. This means some "systems" collapse to one equation once you say clearly what the shared condition is.

Now for the walkers. Let $t$ = time in hours, $d$ = distance walked by the first walker. Two facts:

$$\begin{cases} d + (9 - d) = 9 \\ d = 4t \quad \text{and} \quad 9 - d = 5t \end{cases}$$

Rewriting cleanly:

$$\begin{cases} d + 5t = 9 \\ d - 4t = 0 \end{cases}$$

Read the curly brace as "both of these must be true at the same time." That is a system. The solution is the pair of values $(d, t)$ that satisfies *both* equations simultaneously. This means a system is not a list of separate problems. It is one problem with several constraints that must agree.

### 23.1.1. The general form

The standard 2×2 system looks like this:

$$\begin{cases} a_1 x + b_1 y = c_1 \\ a_2 x + b_2 y = c_2 \end{cases}$$

Here $x$ and $y$ are the unknowns. The numbers $a_1, b_1, c_1$ and $a_2, b_2, c_2$ are given — they are the **coefficients** (the multipliers of $x$ and $y$) and the **constant terms** (the values on the right). This means every 2×2 system has the same structure even when the context is tickets, circuits, mixtures, or motion.

### 23.1.2. Compact matrix notation

The same system can be written using a **matrix** — just the coefficients arranged in a rectangular array:

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$$

Say it as: the 2×2 matrix of coefficients, multiplied by the column of unknowns, equals the column of constants. This notation is compact — it stores the same information without repeating $x$ and $y$ on every line. For a 2×2 system it's a convenience. For a 100×100 system (as in circuit analysis or machine learning), it's essential. This means matrix notation is a way to organise the system so the pattern is easy to see and scale up.

The matrix itself — the rectangular array of coefficients — is written with parentheses around it:

$$A = \begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix}$$

Read "$A$" as "the coefficient matrix." The entries are: top-left $a_1$ (coefficient of $x$ in the first equation), top-right $b_1$ (coefficient of $y$ in the first equation), and so on.

## 23.2. The method

Three methods solve a 2×2 system. Each is worth knowing: the graphical method shows you what you're doing, substitution is efficient for small systems, and elimination is the one that scales.

### 23.2.1. Method 1 — Graphical

Each equation is a straight line. The solution is where the two lines cross.

$$\begin{cases} x + y = 5 \\ x - y = 1 \end{cases}$$

Rearrange each equation into $y = mx + b$ form:

- Line 1: $y = -x + 5$ (slope $-1$, intercept 5)
- Line 2: $y = x - 1$ (slope 1, intercept $-1$)

Plot both. They cross at $(3, 2)$. Check: $3 + 2 = 5$ ☒ and $3 - 2 = 1$ ☒. This means the graphical method turns "solve both equations" into "find the point that lies on both lines."

**Three possible outcomes:**

| What the lines do | What it means |
|---|---|
| Cross at one point | One unique solution |
| Are parallel (never cross) | No solution — the equations contradict each other |
| Are the same line | Infinitely many solutions — one equation gives no new information |

```
//| echo: false
{
  const div = document.createElement("div");
  div.style.cssText = "font-family:inherit; padding:0.5rem 0;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; grid-template-columns:repeat(3,1fr); gap:0.5rem 1.5rem;

  const params = [
    {id:"sa1", label:"a₁", min:-5, max:5, step:0.5, val:1},
    {id:"sb1", label:"b₁", min:-5, max:5, step:0.5, val:1},
    {id:"sc1", label:"c₁", min:-10, max:10, step:1, val:5},
    {id:"sa2", label:"a₂", min:-5, max:5, step:0.5, val:2},
    {id:"sb2", label:"b₂", min:-5, max:5, step:0.5, val:-1},
    {id:"sc2", label:"c₂", min:-10, max:10, step:1, val:4},
  ];

  params.forEach(p => {
    const wrap = document.createElement("div");
    const lbl = document.createElement("label");
    lbl.style.cssText = "font-size:0.8rem; color:#374151; display:block;";
    const span = document.createElement("span"); span.textContent = p.val;
    lbl.appendChild(document.createTextNode(p.label + " = ")); lbl.appendChild(span);
    const input = document.createElement("input");
```

```
    input.type = "range"; input.min = p.min; input.max = p.max; input.step = p.step; input.value = p
    input.style.width = "100%";
    wrap.appendChild(lbl); wrap.appendChild(input);
    controls.appendChild(wrap);
    p.input = input;
    p.span = span;
  });

  const infoEl = document.createElement("div");
  infoEl.style.cssText = "font-size:0.88rem; margin-bottom:0.5rem; min-height:1.4em; font-weight:600

  const canvas = document.createElement("canvas");
  canvas.width = 400; canvas.height = 400;
  canvas.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; display:block; max-width:400p

  div.appendChild(controls); div.appendChild(infoEl); div.appendChild(canvas);

  function draw() {
    const vals = {};
    params.forEach(p => {
      const v = parseFloat(p.input.value);
      p.span.textContent = v;
      vals[p.id] = v;
    });

    const {sa1:a1, sb1:b1, sc1:c1, sa2:a2, sb2:b2, sc2:c2} = vals;
    const det = a1*b2 - a2*b1;

    const ctx = canvas.getContext("2d");
    const W = canvas.width, H = canvas.height;
    ctx.clearRect(0, 0, W, H);
    const range = 12;
    function px(x) { return (x+range)/(2*range)*W; }
    function py(y) { return H-(y+range)/(2*range)*H; }

    // Grid
    ctx.strokeStyle = "#f3f4f6"; ctx.lineWidth = 1;
    for (let i = -10; i <= 10; i += 2) {
      ctx.beginPath(); ctx.moveTo(px(i), 0); ctx.lineTo(px(i), H); ctx.stroke();
      ctx.beginPath(); ctx.moveTo(0, py(i)); ctx.lineTo(W, py(i)); ctx.stroke();
    }
    ctx.strokeStyle = "#9ca3af"; ctx.lineWidth = 1.5;
    ctx.beginPath(); ctx.moveTo(0, py(0)); ctx.lineTo(W, py(0)); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(px(0), 0); ctx.lineTo(px(0), H); ctx.stroke();

    // Line 1: a1*x + b1*y = c1  => y = (c1 - a1*x)/b1
    function drawLine(a, b, c, color) {
```

```
      ctx.strokeStyle = color; ctx.lineWidth = 2.5;
      ctx.beginPath();
      let first = true;
      for (let xi = 0; xi <= W; xi++) {
        const x = -range + (xi / W) * 2 * range;
        let y;
        if (Math.abs(b) > 0.001) y = (c - a*x) / b;
        else { if (xi === 0) { ctx.moveTo(px(c/a), 0); ctx.lineTo(px(c/a), H); ctx.stroke(); } re
        if (first) { ctx.moveTo(xi, py(y)); first = false; }
        else ctx.lineTo(xi, py(y));
      }
      ctx.stroke();
    }

    drawLine(a1, b1, c1, "#2563eb");
    drawLine(a2, b2, c2, "#ef4444");

    if (Math.abs(det) < 0.001) {
      const parallel = Math.abs(a1*c2 - a2*c1) > 0.001 || Math.abs(b1*c2 - b2*c1) > 0.001;
      infoEl.textContent = parallel ? "No unique solution — lines are parallel" : "Infinitely man
      infoEl.style.color = "#dc2626";
    } else {
      const x = (c1*b2 - c2*b1) / det;
      const y = (a1*c2 - a2*c1) / det;
      ctx.fillStyle = "#7c3aed";
      ctx.beginPath(); ctx.arc(px(x), py(y), 6, 0, 2*Math.PI); ctx.fill();
      ctx.fillStyle = "#374151"; ctx.font = "bold 12px sans-serif";
      ctx.fillText("(" + x.toFixed(2) + ", " + y.toFixed(2) + ")", px(x)+8, py(y)-8);
      infoEl.textContent = "Solution: x = " + x.toFixed(2) + ", y = " + y.toFixed(2);
      infoEl.style.color = "#7c3aed";
    }

    // Equation labels
    ctx.font = "12px sans-serif";
    ctx.fillStyle = "#2563eb";
    ctx.fillText(a1+"x + "+b1+"y = "+c1, 8, 16);
    ctx.fillStyle = "#ef4444";
    ctx.fillText(a2+"x + "+b2+"y = "+c2, 8, 32);
  }

  params.forEach(p => p.input.addEventListener("input", draw));
  draw();
  return div;
}
```

### 23.2.2. Method 2 — Substitution

Solve one equation for one variable, then substitute into the other. This eliminates one unknown and leaves a single equation you can solve directly.

$$\begin{cases} x + y = 5 & (1) \\ x - y = 1 & (2) \end{cases}$$

From equation (1): $x = 5 - y$.

Substitute into equation (2):

$$(5 - y) - y = 1$$

$$5 - 2y = 1$$

$$-2y = -4 \implies y = 2$$

Now substitute $y = 2$ back into $x = 5 - y$:

$$x = 5 - 2 = 3$$

Solution: $x = 3$, $y = 2$. This means substitution temporarily turns a two-variable problem back into the one-variable algebra you already know.

**When to use it:** substitution works cleanly when one equation already has a variable with coefficient 1 (so there's no fraction to deal with when you isolate it).

### 23.2.3. Method 3 — Elimination

Multiply one or both equations by constants to make the coefficient of one variable the same (or opposite) in both equations. Then add or subtract to eliminate that variable.

$$\begin{cases} 2x + 3y = 12 & (1) \\ 4x - y = \ \ 5 & (2) \end{cases}$$

The coefficient of $y$ is 3 in equation (1) and $-1$ in equation (2). Multiply equation (2) by 3 to make the $y$-coefficients match in magnitude:

$$3 \times (2) : \quad 12x - 3y = 15 \qquad (2')$$

Now add equations (1) and (2'):

$$(2x + 3y) + (12x - 3y) = 12 + 15$$

$$14x = 27 \implies x = \frac{27}{14}$$

Substitute into equation (1):

$$2 \cdot \frac{27}{14} + 3y = 12 \implies \frac{27}{7} + 3y = 12 \implies 3y = 12 - \frac{27}{7} = \frac{57}{7} \implies y = \frac{19}{7}$$

Check in equation (2): $4 \cdot \frac{27}{14} - \frac{19}{7} = \frac{54}{7} - \frac{19}{7} = \frac{35}{7} = 5$ ☒ This means elimination works by combining the equations until one variable disappears.

**When to use it:** elimination is best when no variable has a coefficient of 1, or when the numbers make substitution awkward. It's also the method that generalises directly to 3×3 and larger systems (Gaussian elimination).

## 23.3. Why this works

**The determinant tells you whether a unique solution exists**

For the system $\begin{pmatrix} a & b \\ c & d \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} e \\ f \end{pmatrix}$, define the **determinant**:

$$\det(A) = ad - bc$$

Read it as: top-left times bottom-right, minus top-right times bottom-left.

- If $\det(A) \neq 0$: one unique solution exists. The two lines cross.
- If $\det(A) = 0$: no unique solution. The lines are parallel (no solution) or identical (infinitely many).

For the system $\begin{cases} x + y = 5 \\ x - y = 1 \end{cases}$, the matrix is $\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ and $\det = (1)(-1) - (1)(1) = -2 \neq 0$. One solution. ☒

The determinant is a quick test before you do any work. If it's zero, stop — no amount of algebra will produce a unique answer. This means the determinant tells you whether the two constraints are really independent.

## 23.4. Worked examples

**Example 1 — Business: supply and demand equilibrium.**

A market for concert tickets has a supply curve and a demand curve:

$$\begin{cases} p = 2q + 10 & \text{(supply: price rises with quantity)} \\ p = -3q + 60 & \text{(demand: price falls as quantity rises)} \end{cases}$$

Here $p$ = price (dollars), $q$ = quantity (thousands of tickets). The equilibrium is where supply price equals demand price.

Set the right-hand sides equal (substitution):

$$2q + 10 = -3q + 60$$

$$5q = 50 \implies q = 10$$

$$p = 2(10) + 10 = 30$$

**Equilibrium:** 10,000 tickets at $30 each.

Check in the demand equation: $-3(10) + 60 = -30 + 60 = 30$ ☒

The determinant check: rewrite as $\begin{cases} 2q - p = -10 \\ -3q - p = -60 \end{cases}$. Matrix $\begin{pmatrix} 2 & -1 \\ -3 & -1 \end{pmatrix}$, $\det = (2)(-1) - (-1)(-3) = -2 - 3 = -5 \neq 0$. Unique solution confirmed. This means equilibrium is simply the point where two different rules predict the same price and quantity.

```
//| echo: false
{
  const canvas = document.createElement("canvas");
  canvas.width = 400; canvas.height = 380;
  canvas.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; display:block; max-width:400p

  const ctx = canvas.getContext("2d");
  const W = canvas.width, H = canvas.height;
  const qMax = 20, pMax = 70;
  const marg = {l:50, r:20, t:20, b:40};

  function px(q) { return marg.l + (q / qMax) * (W - marg.l - marg.r); }
  function py(p) { return marg.t + (1 - p / pMax) * (H - marg.t - marg.b); }

  // Background shade: demand > supply (q < 10, left of intersection)
  ctx.fillStyle = "rgba(219,234,254,0.4)";
  ctx.fillRect(px(0), marg.t, px(10)-px(0), H-marg.t-marg.b);
  ctx.fillStyle = "rgba(220,252,231,0.4)";
  ctx.fillRect(px(10), marg.t, px(qMax)-px(10), H-marg.t-marg.b);
```

```
  // Axes
  ctx.strokeStyle = "#9ca3af"; ctx.lineWidth = 1.5;
  ctx.beginPath(); ctx.moveTo(px(0), py(0)); ctx.lineTo(px(qMax), py(0)); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(px(0), py(0)); ctx.lineTo(px(0), py(pMax)); ctx.stroke();
  ctx.fillStyle = "#6b7280"; ctx.font = "12px sans-serif";
  ctx.fillText("q", px(qMax)-10, py(0)+20);
  ctx.fillText("p", px(0)-20, py(pMax)+5);

  // Tick marks
  for (let q = 0; q <= 20; q += 5) {
    ctx.beginPath(); ctx.moveTo(px(q), py(0)); ctx.lineTo(px(q), py(0)+5); ctx.stroke();
    ctx.fillText(q, px(q)-5, py(0)+18);
  }
  for (let p = 0; p <= 70; p += 10) {
    ctx.beginPath(); ctx.moveTo(px(0), py(p)); ctx.lineTo(px(0)-5, py(p)); ctx.stroke();
    ctx.fillText(p, px(0)-28, py(p)+4);
  }

  // Supply: p = 2q + 10
  ctx.strokeStyle = "#ef4444"; ctx.lineWidth = 2.5;
  ctx.beginPath(); ctx.moveTo(px(0), py(10)); ctx.lineTo(px(qMax), py(2*qMax+10)); ctx.stroke();
  ctx.fillStyle = "#ef4444"; ctx.font = "bold 11px sans-serif";
  ctx.fillText("Supply: p = 2q + 10", px(12), py(2*12+10)-8);

  // Demand: p = -3q + 60
  ctx.strokeStyle = "#2563eb"; ctx.lineWidth = 2.5;
  ctx.beginPath(); ctx.moveTo(px(0), py(60)); ctx.lineTo(px(20), py(-3*20+60)); ctx.stroke();
  ctx.fillStyle = "#2563eb";
  ctx.fillText("Demand: p = −3q + 60", px(1), py(-3*1+60)-8);

  // Intersection at (10, 30)
  ctx.fillStyle = "#7c3aed";
  ctx.beginPath(); ctx.arc(px(10), py(30), 6, 0, 2*Math.PI); ctx.fill();
  ctx.fillStyle = "#374151"; ctx.font = "bold 12px sans-serif";
  ctx.fillText("Equilibrium (10, 30)", px(10)+8, py(30)-8);

  // Shade labels
  ctx.font = "11px sans-serif";
  ctx.fillStyle = "#1d4ed8"; ctx.fillText("demand > supply", px(1), py(50));
  ctx.fillStyle = "#166534"; ctx.fillText("supply > demand", px(12), py(50));

  return canvas;
}
```

---

**Example 2 — Engineering: two-loop circuit (Kirchhoff's laws).**

A simple circuit has two loops. Kirchhoff's voltage law says the voltage drops around each loop must sum to zero. After applying the law to each loop, the unknowns are the two loop currents $I_1$ and $I_2$ (in amps):

$$\begin{cases} 3I_1 + 2I_2 = 12 & \text{(loop 1: total 12 V)} \\ 2I_1 + 5I_2 = 14 & \text{(loop 2: total 14 V)} \end{cases}$$

Use elimination. Multiply equation (1) by 5 and equation (2) by 2:

$$\begin{cases} 15I_1 + 10I_2 = 60 \\ 4I_1 + 10I_2 = 28 \end{cases}$$

Subtract the second from the first:

$$11I_1 = 32 \implies I_1 = \frac{32}{11} \approx 2.91 \text{ A}$$

Substitute into equation (1):

$$3 \cdot \frac{32}{11} + 2I_2 = 12 \implies \frac{96}{11} + 2I_2 = 12 \implies 2I_2 = 12 - \frac{96}{11} = \frac{36}{11} \implies I_2 = \frac{18}{11} \approx 1.64 \text{ A}$$

Check in equation (2): $2 \cdot \frac{32}{11} + 5 \cdot \frac{18}{11} = \frac{64+90}{11} = \frac{154}{11} = 14$ ☒ This means each current is fixed by the requirement that both loop equations hold together.

---

**Example 3 — Science: mixture problem.**

A lab needs 80 mL of a 35% saline solution. The lab has a 20% solution and a 50% solution. How many mL of each should be mixed?

Let $x$ = mL of the 20% solution, $y$ = mL of the 50% solution.

Two constraints:

$$\begin{cases} x + y = 80 & \text{(total volume)} \\ 0.20x + 0.50y = 0.35 \times 80 = 28 & \text{(total salt)} \end{cases}$$

From equation (1): $x = 80 - y$. Substitute into equation (2):

$$0.20(80 - y) + 0.50y = 28$$

$$16 - 0.20y + 0.50y = 28$$

$$0.30y = 12 \implies y = 40$$

$$x = 80 - 40 = 40$$

Mix 40 mL of the 20% solution with 40 mL of the 50% solution.

Check: salt = $0.20(40) + 0.50(40) = 8 + 20 = 28$ and $28/80 = 0.35$ ☒ This means mixture problems always balance two things at once: total amount and total content.

---

**Example 4 — Computing: game resource constraints.**

A game character can perform two actions: attacking costs 3 energy and 1 second, defending costs 1 energy and 2 seconds. In one round, the character uses exactly 14 energy and 8 seconds. How many attacks and defends did they perform?

Let $a$ = attacks, $d$ = defends.

$$\begin{cases} 3a + d = 14 & \text{(energy)} \\ a + 2d = 8 & \text{(time)} \end{cases}$$

From equation (1): $d = 14 - 3a$. Substitute into equation (2):

$$a + 2(14 - 3a) = 8$$

$$a + 28 - 6a = 8$$

$$-5a = -20 \implies a = 4$$

$$d = 14 - 3(4) = 2$$

Four attacks and two defends.

Check energy: $3(4) + 2 = 14$ ☒. Check time: $4 + 2(2) = 8$ ☒.

Determinant check: matrix $\begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$, $\det = (3)(2) - (1)(1) = 5 \neq 0$. Unique solution. ☒ This means the solution is the only combination of actions that fits both the energy budget and the time budget.

## 23.5. Where this goes

This chapter turned crossing lines and paired constraints into a general algebraic method. The next step is to keep the same idea while scaling it to much larger systems.

This chapter handled two unknowns and two equations. Real systems are often much larger — 10 unknowns, 100, or more. The method that scales is **Gaussian elimination**: the same idea as the elimination you learned here, applied row by row to a matrix. That is the core of **linear algebra** (Vol 7). Matrix notation, which you saw as a shorthand here, becomes indispensable there — you'll work with eigenvalues, matrix inverses, and linear transformations that are impossible to write without it.

When systems get large enough, solving them exactly becomes computationally expensive. **Numerical methods** (Vol 7) replace exact algebra with iterative algorithms — methods that get close enough, fast enough, for engineering and data science work. Systems of equations are the motivating problem that drives a large part of that volume. The determinant you met here generalises too: for an $n \times n$ matrix, the same zero/non-zero test predicts whether a unique solution exists.

**Where this shows up**

- **Engineering:** Kirchhoff's voltage and current laws applied to a circuit produce one equation per loop or node. A circuit with two loops gives a 2×2 system; a real circuit might give a 50×50 system solved by computer using the same method.
- **Science:** Chemical stoichiometry — balancing an equation with two unknown coefficients — is a 2×2 system. Equilibrium calculations in physical chemistry are similar.
- **Finance:** Supply and demand equilibrium is the intersection of two linear functions. Every introductory economics model of price-setting is a system of equations.
- **Computing:** Linear regression with two parameters solves a 2×2 system. Graphics engines apply matrix transformations (systems of equations) to every vertex on screen, thousands of times per second.

## 23.6. Exercises

These are puzzles. The interesting part is setting up the system — translating the situation into two equations before you solve.

---

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
```

```
const stepsDiv = document.createElement("div");
stepsDiv.style.marginBottom = "0.75rem";
function renderSteps() {
  stepsDiv.innerHTML = "";
  for (let i = 0; i < current; i++) {
    const s = steps[i];
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align
    const opEl = document.createElement("span");
    opEl.style.cssText = "font-size:0.8rem; color:#6b7280; font-style:italic;";
    opEl.textContent = s.op;
    const eqEl = document.createElement("span");
    eqEl.innerHTML = katex.renderToString(s.eq, {throwOnError: false, displayMode: false});
    row.appendChild(opEl);
    row.appendChild(eqEl);
    container.appendChild(row);
    if (s.note) {
      const noteEl = document.createElement("p");
      noteEl.style.cssText = "margin:0.2rem 0 0.5rem 200px; font-size:0.82rem; color:#374151;";
      noteEl.textContent = s.note;
      container.appendChild(noteEl);
    }
  }
  if (current === totalSteps) {
    const done = document.createElement("p");
    done.style.cssText = "margin-top:0.5rem; font-size:0.85rem; color:#059669; font-weight:600;
    done.textContent = "□ Solution complete";
    stepsDiv.appendChild(done);
  }
}
const controls = document.createElement("div");
controls.style.cssText = "display:flex; align-items:center; gap:0.75rem; margin-top:0.5rem;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.35rem 0.85rem; background:#2563eb; color:#fff; border:none;
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.35rem 0.85rem; background:#f3f4f6; color:#374151; border:1p
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.8rem; color:#6b7280;";
nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); counter.textCon
resetBtn.onclick = () => { current = 0; nextBtn.disabled = false; renderSteps(); counter.textCo
counter.textContent = "0 / " + totalSteps;
controls.appendChild(nextBtn);
controls.appendChild(resetBtn);
controls.appendChild(counter);
container.appendChild(stepsDiv);
```

```
  container.appendChild(controls);
  return container;
}
```

**1.** Two friends are buying supplies for a camping trip. Rashida buys 3 bags of trail mix and 2 water bottles for $19. Tomasz buys 1 bag of trail mix and 4 water bottles for $17. What does one bag of trail mix cost, and what does one water bottle cost?

```
//| echo: false
{
  const el = makeStepperHTML(1, [
    { op: "Write as a system", eq: "\\begin{cases} 3t + 2w = 19 \\\\ t + 4w = 17 \\end{cases}", note
    { op: "Solve equation 2 for t", eq: "t = 17 - 4w", note: null },
    { op: "Substitute into equation 1", eq: "3(17 - 4w) + 2w = 19", note: null },
    { op: "Expand and simplify", eq: "51 - 12w + 2w = 19 \\implies -10w = -32 \\implies w = 3.20", n
    { op: "Back-substitute to find t", eq: "t = 17 - 4(3.20) = 17 - 12.80 = 4.20", note: null },
    { op: "Check", eq: "3(4.20) + 2(3.20) = 12.60 + 6.40 = 19 \\checkmark \\quad 4.20 + 4(3.20) = 4
  ]);
  return el;
}
```

---

**2.** A cinema charges one price for adults and another for children. Two families bought tickets last Saturday:

- The Nguyens: 3 adults and 2 children, total $32.
- The Garcias: 1 adult and 4 children, total $24.

What is the adult ticket price? What is the child ticket price?

```
//| echo: false
{
  const el = makeStepperHTML(2, [
    { op: "Write as a system", eq: "\\begin{cases} 3a + 2c = 32 \\\\ a + 4c = 24 \\end{cases}", note
    { op: "Solve equation 2 for a", eq: "a = 24 - 4c", note: null },
    { op: "Substitute into equation 1", eq: "3(24 - 4c) + 2c = 32 \\implies 72 - 12c + 2c = 32 \\imp
    { op: "Back-substitute to find a", eq: "a = 24 - 4(4) = 24 - 16 = 8", note: null },
    { op: "Check", eq: "3(8) + 2(4) = 24 + 8 = 32 \\checkmark \\quad 1(8) + 4(4) = 8 + 16 = 24 \\che
  ]);
  return el;
}
```

---

**3.** Two cyclists start from the same point at the same time and ride in opposite directions. After 3 hours they are 132 km apart. One rides 6 km/h faster than the other. What is each cyclist's speed?

```
//| echo: false
{
  const el = makeStepperHTML(3, [
    { op: "Write as a system", eq: "\\begin{cases} 3r + 3s = 132 \\\\ r - s = 6 \\end{cases}", no
    { op: "Simplify equation 1", eq: "r + s = 44", note: null },
    { op: "Add the two equations", eq: "(r + s) + (r - s) = 44 + 6 \\implies 2r = 50 \\implies r
    { op: "Back-substitute to find s", eq: "s = 44 - 25 = 19 \\text{ km/h}", note: null },
    { op: "Check", eq: "3(25) + 3(19) = 75 + 57 = 132 \\checkmark \\quad 25 - 19 = 6 \\checkmark"
  ]);
  return el;
}
```

---

**4.** Use the determinant to decide, without solving, whether each system has a unique solution, no solution, or infinitely many solutions.

**(a)**

$$\begin{cases} 2x + 4y = 8 \\ x + 2y = 4 \end{cases}$$

**(b)**

$$\begin{cases} 3x - y = 5 \\ 6x - 2y = 7 \end{cases}$$

**(c)**

$$\begin{cases} 5x + 3y = 11 \\ 2x - y = 3 \end{cases}$$

```
//| echo: false
{
  const el = makeStepperHTML(4, [
    { op: "Write the coefficient matrix for (a)", eq: "A = \\begin{pmatrix} 2 & 4 \\\\ 1 & 2 \\en
    { op: "Compute det(A) for (a)", eq: "\\det(A) = (2)(2) - (4)(1) = 4 - 4 = 0", note: "det = 0
    { op: "Inspect (a): are the equations the same?", eq: "\\text{Row 1} = 2 \\times \\text{Row 2
    { op: "Compute det for (b)", eq: "\\det\\begin{pmatrix} 3 & -1 \\\\ 6 & -2 \\end{pmatrix} = (
    { op: "Inspect (b): same line or parallel?", eq: "\\text{Row 1} = \\tfrac{1}{2} \\times \\tex
    { op: "Compute det for (c)", eq: "\\det\\begin{pmatrix} 5 & 3 \\\\ 2 & -1 \\end{pmatrix} = (5
    { op: "Solve by substitution", eq: "2x - y = 3 \\Rightarrow y = 2x - 3", note: null },
    { op: "Substitute into first equation", eq: "5x + 3(2x-3) = 11 \\Rightarrow 11x = 20 \\Righta
    { op: "Find y", eq: "y = 2 \\cdot \\frac{20}{11} - 3 = \\frac{40}{11} - \\frac{33}{11} = \\fr
    { op: "Check", eq: "5 \\cdot \\frac{20}{11} + 3 \\cdot \\frac{7}{11} = \\frac{100}{11} + \\fr
  ]);
  return el;
}
```

---

**5.** Two phone plans:

- Plan X: $20 per month plus $0.10 per text.
- Plan Y: $35 per month, unlimited texts.

*(a)* Set up a system and find the number of texts per month where both plans cost the same.

*(b)* If you send 200 texts per month, which plan is cheaper and by how much?

```
//| echo: false
{
  const el = makeStepperHTML(5, [
    { op: "Write the cost equations", eq: "\\begin{cases} C_X = 20 + 0.10n \\\\ C_Y = 35 \\end{cases
    { op: "Set costs equal and solve", eq: "20 + 0.10n = 35 \\implies 0.10n = 15 \\implies n = 150"
    { op: "State the crossover", eq: "\\text{At } n = 150 \\text{ texts, both plans cost } \\$35.",
    { op: "Evaluate at n = 200 for part (b)", eq: "C_X = 20 + 0.10(200) = 20 + 20 = \\$40; \\quad C_
    { op: "Check", eq: "C_X - C_Y = 40 - 35 = \\$5. \\text{ Plan Y is cheaper by } \\$5 \\text{ at 2
  ]);
  return el;
}
```

---

**6.** A chemist has two alcohol solutions: one is 30% alcohol, the other is 70% alcohol. She needs 200 mL of a 45% solution.

How many mL of each solution should she mix?

*(Hint: write one equation for total volume and one for total amount of alcohol.)*

```
//| echo: false
{
  const el = makeStepperHTML(6, [
    { op: "Write as a system", eq: "\\begin{cases} x + y = 200 \\\\ 0.30x + 0.70y = 0.45 \\times 200
    { op: "Solve equation 1 for x", eq: "x = 200 - y", note: null },
    { op: "Substitute into equation 2", eq: "0.30(200 - y) + 0.70y = 90 \\implies 60 - 0.30y + 0.70y
    { op: "Back-substitute to find x", eq: "x = 200 - 75 = 125", note: null },
    { op: "Check", eq: "0.30(125) + 0.70(75) = 37.50 + 52.50 = 90 \\quad \\text{and} \\quad 90/200 =
  ]);
  return el;
}
```

# 24. Sequences and series

When adding up becomes its own question

You start a sponsored read-a-thon. On day 1 you read 10 pages. Each day you read 3 more pages than the day before. After 30 days, how many pages have you read in total? Adding them up one by one would take a while — but there's a pattern.

To walk from here to a wall, you must first walk halfway. Then half of the remaining distance. Then half again. You take infinitely many steps — so how do you ever arrive? The answer is that the distances form a series whose infinite sum is exactly the distance to the wall.

Both problems need the same tool. This chapter builds it.

## 24.1. What the notation is saying

A **sequence** is a list of numbers following a rule. A **series** is the sum of those numbers, in order.

Write the sequence $3, 7, 11, 15, 19$. The corresponding series is:

$$3 + 7 + 11 + 15 + 19 = 45$$

That works when you have a short list. For 300 terms, writing out every addition is unrealistic. Mathematicians use a shorthand — the **sigma notation** — named after the Greek letter $\Sigma$ (capital sigma).

$$\sum_{i=1}^{n} a_i$$

Read it as: *the sum of $a_i$, as $i$ runs from 1 to $n$.*

The $i$ below the sigma is the **index** — it's a counter that starts at 1 and steps up by 1 until it reaches $n$. At each step, you substitute that value of $i$ into the expression $a_i$ and add the result to your running total. This means sigma notation is not a new kind of arithmetic. It is a compact way to describe repeated addition.

A concrete example before using sigma in formulas. Write the sum $1 + 4 + 9 + 16 + 25$ — the first five perfect squares — using sigma:

$$\sum_{i=1}^{5} i^2 = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$$

The expression after Σ is the rule. The limits below and above tell you where to start and stop. That is the whole notation. This means every sigma expression can be unpacked term by term if you want to check what it is doing.

```
//| echo: false
{
  const div = document.createElement("div");
  div.style.cssText = "font-family:inherit; padding:0.5rem 0;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:1.5rem; flex-wrap:wrap; align-items:flex-start; margin

  // Type selector
  const typeWrap = document.createElement("div");
  const typeLbl = document.createElement("label");
  typeLbl.style.cssText = "font-size:0.85rem; color:#374151; display:block; margin-bottom:0.2rem;";
  typeLbl.textContent = "Type";
  const typeSelect = document.createElement("select");
  typeSelect.style.cssText = "padding:0.2rem 0.4rem; border:1px solid #d1d5db; border-radius:4px;";
  ["arithmetic", "geometric"].forEach(t => {
    const o = document.createElement("option"); o.value = t; o.textContent = t;
    typeSelect.appendChild(o);
  });
  typeWrap.appendChild(typeLbl); typeWrap.appendChild(typeSelect);
  controls.appendChild(typeWrap);

  // makeSlider returns {wrap, input, span} so callers keep direct refs
  function makeSlider(label, min, max, step, val) {
    const wrap = document.createElement("div");
    const lbl = document.createElement("label");
    lbl.style.cssText = "font-size:0.85rem; color:#374151; display:block; margin-bottom:0.2rem;";
    const span = document.createElement("span"); span.textContent = val;
    lbl.appendChild(document.createTextNode(label + " = ")); lbl.appendChild(span);
    const input = document.createElement("input");
    input.type = "range"; input.min = min; input.max = max; input.step = step; input.value = val;
    input.style.width = "140px";
    wrap.appendChild(lbl); wrap.appendChild(input);
    return {wrap, input, span};
  }

  const a1S = makeSlider("a₁ (first term)", 1, 20, 1, 2);
  controls.appendChild(a1S.wrap);
  const dS = makeSlider("d (common diff)", -5, 10, 1, 3);
  dS.wrap.style.display = "";
  controls.appendChild(dS.wrap);
  const rS = makeSlider("r (ratio)", -3, 3, 0.5, 2);
  rS.wrap.style.display = "none";
  controls.appendChild(rS.wrap);
```

```
const nS = makeSlider("n (terms)", 3, 12, 1, 6);
controls.appendChild(nS.wrap);

const displayEl = document.createElement("div");
displayEl.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:6px; pad

div.appendChild(controls); div.appendChild(displayEl);

function update() {
  const type = typeSelect.value;
  const a1 = parseFloat(a1S.input.value);
  const d = parseFloat(dS.input.value);
  const r = parseFloat(rS.input.value);
  const n = parseInt(nS.input.value);
  a1S.span.textContent = a1;
  dS.span.textContent = d;
  rS.span.textContent = r;
  nS.span.textContent = n;
  dS.wrap.style.display = type === "arithmetic" ? "" : "none";
  rS.wrap.style.display = type === "geometric" ? "" : "none";

  const terms = [];
  let sum = 0;
  for (let k = 1; k <= n; k++) {
    const t = type === "arithmetic" ? a1 + (k-1)*d : a1 * Math.pow(r, k-1);
    terms.push(t);
    sum += t;
  }

  const sigmaLabel = type === "arithmetic"
    ? "□(k=1 to "+n+") ["+a1+(d>=0?"+":"")+d+"(k−1)]"
    : "□(k=1 to "+n+") "+a1+"·"+r+"^(k−1)";

  let expanded;
  if (n <= 6) expanded = terms.map(t => t.toFixed(1)).join(" + ");
  else expanded = terms.slice(0,3).map(t=>t.toFixed(1)).join(" + ") + " + … + " + terms[n-1].to

  displayEl.innerHTML =
    "<b>Sigma notation:</b> " + sigmaLabel + "<br>" +
    "<b>Expanded:</b> " + expanded + "<br>" +
    "<b>Sum:</b> S<sub>" + n + "</sub> = " + sum.toFixed(2);
}

typeSelect.addEventListener("input", update);
[a1S, dS, rS, nS].forEach(s => s.input.addEventListener("input", update));
update();
return div;
```

```
}
```

## 24.2. The method

### 24.2.1. Arithmetic series

An arithmetic sequence has a first term $a_1$ and a common difference $d$. The terms are:

$$a_1, \quad a_1 + d, \quad a_1 + 2d, \quad \ldots, \quad a_1 + (n-1)d$$

Adding all $n$ of them gives the **arithmetic series** $S_n$, read as *the sum of the first n terms.*

To see why the formula works, try a small example first. Take the arithmetic sequence $3, 6, 9, 12$ (four terms, $d = 3$). Write the sum forwards and backwards, then add column by column:

$$\begin{array}{r} S_4 = \ \ 3 + \ \ 6 + \ \ 9 + 12 \\ S_4 = 12 + \ \ 9 + \ \ 6 + \ \ 3 \\ \hline 2S_4 = 15 + 15 + 15 + 15 = 4 \times 15 = 60 \end{array}$$

$$S_4 = 30$$

The same pairing trick works for any arithmetic series. Label the sum forwards and backwards:

$$S_n = a_1 + (a_1 + d) + (a_1 + 2d) + \cdots + a_n$$

$$S_n = a_n + (a_n - d) + (a_n - 2d) + \cdots + a_1$$

Add the two lines together. Each column pairs a term from the top with a term from the bottom. Every pair sums to $a_1 + a_n$, and there are $n$ such pairs:

$$2S_n = n(a_1 + a_n)$$

$$\boxed{S_n = \frac{n}{2}(a_1 + a_n)}$$

This means an arithmetic series can be found from "number of pairs" times "value of each pair," then divided by 2 because we counted the sum twice.

Since $a_n = a_1 + (n-1)d$, you can write this entirely in terms of the first term and the common difference:

$$S_n = \frac{n}{2}\big(2a_1 + (n-1)d\big)$$

Use whichever form is more convenient. If you know the first and last term, use the first. If you only know $a_1$ and $d$, use the second. This means the formula is flexible: the same sum can be described from different pieces of information.

**The Gauss pairing trick**
Carl Friedrich Gauss was reportedly seven years old when his teacher set the class the task of summing all integers from 1 to 100, hoping for an hour of quiet. Gauss had the answer in seconds: 5050.
His insight: pair the first term with the last, the second with the second-to-last, and so on. Each pair sums to 101. There are 50 pairs. So $50 \times 101 = 5050$.
This is not a shortcut — it is the formula. $S_{100} = \frac{100}{2}(1 + 100) = 50 \times 101 = 5050$.
The formula encodes the same pairing, for any arithmetic series. That is what elegant mathematical thinking looks like: notice a structure, use it, and get there in seconds instead of hours.

### 24.2.2. Geometric series

A geometric sequence has a first term $a_1$ and a common ratio $r$. The terms are:

$$a_1, \quad a_1 r, \quad a_1 r^2, \quad \ldots, \quad a_1 r^{n-1}$$

Adding all $n$ terms:

$$S_n = a_1 + a_1 r + a_1 r^2 + \cdots + a_1 r^{n-1}$$

If you multiply the sum by $r$, each term shifts up by one power — almost matching the original sum. Subtracting cancels all but the first and last terms. This means the geometric-series formula works because multiplication lines the terms up so cancellation becomes possible.

The trick here is to multiply both sides by $r$:

$$rS_n = a_1 r + a_1 r^2 + a_1 r^3 + \cdots + a_1 r^n$$

Now subtract the first line from the second. Nearly everything cancels:

$$rS_n - S_n = a_1 r^n - a_1$$

$$S_n(r - 1) = a_1(r^n - 1)$$

$$\boxed{S_n = \frac{a_1(1 - r^n)}{1 - r}} \qquad (r \neq 1)$$

The form with $(1 - r^n)$ in the numerator is the convention — both forms are algebraically identical, but this one is easier to read when $|r| < 1$.

When $r = 1$ all terms are equal to $a_1$, so $S_n = n \cdot a_1$. That case doesn't need the formula. This means geometric growth can still produce a neat closed formula, even though each term is being multiplied rather than increased by a fixed amount.

### 24.2.3. Infinite geometric series

Here is something that sounds impossible: add infinitely many numbers and get a finite result.

Look at the partial sums of the series $1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots$

$$S_1 = 1$$

$$S_2 = 1.5$$

$$S_3 = 1.75$$

$$S_4 = 1.875$$

$$S_5 = 1.9375$$

The sums are getting closer to 2 — but never exceeding it, and never quite reaching it. The terms are shrinking. Each new term adds less than the last. The sum is converging toward a fixed value.

Look at the formula $S_n = \frac{a_1(1-r^n)}{1-r}$ when $|r| < 1$. As $n$ grows, $r^n$ gets closer and closer to 0 — and when $|r| < 1$, eventually it is so small we treat it as 0:

$$S_\infty = \frac{a_1(1-0)}{1-r} = \frac{a_1}{1-r}$$

$$\boxed{S_\infty = \frac{a_1}{1-r}} \qquad (|r| < 1)$$

Read $S_\infty$ as *the sum of infinitely many terms.*

For the series above: $a_1 = 1$, $r = \frac{1}{2}$. So $S_\infty = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2$. Exactly 2. This means an infinite series can have a finite sum when each new term becomes small quickly enough.

**What if $|r| \geq 1$?** Then $r^n$ does not approach 0 — it either stays at 1 (when $r = 1$) or grows without bound (when $|r| > 1$) or oscillates growing (when $r \leq -1$). The terms never shrink to nothing, so the sum never settles. It **diverges** — it has no finite value. You can add as many terms as you like and the sum will keep growing (or oscillating further and further from zero). This means convergence depends on the behaviour of the terms, not on the fact that there are infinitely many of them.

This is worth sitting with for a moment. It's genuinely strange that you can add infinitely many things and get a finite answer. The reason it works is not that infinity is somehow small — it's that the terms shrink fast enough. Halving at every step is fast enough. Shrinking by one percent at every step is fast enough. Staying the same size is not.

```
//| echo: false
{
  const div = document.createElement("div");
  div.style.cssText = "font-family:inherit; padding:0.5rem 0;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:1.5rem; flex-wrap:wrap; margin-bottom:0.75rem;";

  function makeSlider2(label, min, max, step, val) {
    const wrap = document.createElement("div");
    const lbl = document.createElement("label");
    lbl.style.cssText = "font-size:0.85rem; color:#374151; display:block; margin-bottom:0.2rem;";
    const span = document.createElement("span"); span.textContent = val;
    lbl.appendChild(document.createTextNode(label + " = ")); lbl.appendChild(span);
    const input = document.createElement("input");
    input.type = "range"; input.min = min; input.max = max; input.step = step; input.value = val;
    input.style.width = "160px";
    wrap.appendChild(lbl); wrap.appendChild(input);
    return {wrap, input, span};
  }

  const rSlider = makeSlider2("r (ratio)", -99, 99, 1, 50);
  const a1Slider = makeSlider2("a₁ (first term)", 1, 10, 1, 1);
  controls.appendChild(rSlider.wrap);
  controls.appendChild(a1Slider.wrap);

  const infoEl = document.createElement("div");
  infoEl.style.cssText = "font-size:0.88rem; color:#374151; margin-bottom:0.5rem;";

  const canvas = document.createElement("canvas");
  canvas.width = 500; canvas.height = 260;
  canvas.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; display:block; max-width:5

  div.appendChild(controls); div.appendChild(infoEl); div.appendChild(canvas);

  function draw() {
    const rRaw = parseFloat(rSlider.input.value) / 100;
    const a1 = parseFloat(a1Slider.input.value);
    rSlider.span.textContent = rRaw.toFixed(2);
    a1Slider.span.textContent = a1;

    const N = 20;
    const partials = [];
    let sum = 0;
    for (let k = 1; k <= N; k++) {
      sum += a1 * Math.pow(rRaw, k-1);
      partials.push(sum);
```

```
  }

  const sInf = Math.abs(rRaw) < 1 ? a1 / (1 - rRaw) : null;

  if (sInf !== null) {
    infoEl.textContent = "S∞ = " + a1 + " / (1 − " + rRaw.toFixed(2) + ") = " + sInf.toFixed(2) +
  } else {
    infoEl.textContent = "|r| = " + Math.abs(rRaw).toFixed(2) + " ⍰ 1 — series diverges";
  }

  const ctx = canvas.getContext("2d");
  const W = canvas.width, H = canvas.height;
  ctx.clearRect(0, 0, W, H);
  const marg = {l:55, r:20, t:15, b:35};
  const pHeight = H - marg.t - marg.b;
  const pWidth = W - marg.l - marg.r;

  const allVals = sInf !== null ? [...partials, sInf] : partials;
  const yMin = Math.min(...allVals) * (Math.min(...allVals) < 0 ? 1.2 : 0.8);
  const yMax = Math.max(...allVals) * (Math.max(...allVals) > 0 ? 1.2 : 0.8);
  const yRange = yMax - yMin || 1;

  function px(k) { return marg.l + ((k-1) / (N-1)) * pWidth; }
  function py(v) { return marg.t + (1 - (v - yMin)/yRange) * pHeight; }

  // y-axis
  ctx.strokeStyle = "#9ca3af"; ctx.lineWidth = 1;
  ctx.beginPath(); ctx.moveTo(marg.l, marg.t); ctx.lineTo(marg.l, marg.t+pHeight); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(marg.l, marg.t+pHeight); ctx.lineTo(marg.l+pWidth, marg.t+pHeight);

  // Dashed S∞ line
  if (sInf !== null && sInf > yMin && sInf < yMax) {
    ctx.strokeStyle = "#f59e0b"; ctx.lineWidth = 1.5; ctx.setLineDash([6,4]);
    ctx.beginPath(); ctx.moveTo(marg.l, py(sInf)); ctx.lineTo(marg.l+pWidth, py(sInf)); ctx.stroke
    ctx.setLineDash([]);
    ctx.fillStyle = "#d97706"; ctx.font = "11px sans-serif";
    ctx.fillText("S∞ = " + sInf.toFixed(2), marg.l+pWidth-60, py(sInf)-5);
  }

  // Partial sums dots + line
  ctx.strokeStyle = "#2563eb"; ctx.lineWidth = 1.5;
  ctx.beginPath();
  partials.forEach((v, i) => {
    if (i === 0) ctx.moveTo(px(i+1), py(v)); else ctx.lineTo(px(i+1), py(v));
  });
  ctx.stroke();
```

```
    ctx.fillStyle = "#2563eb";
    partials.forEach((v, i) => {
      ctx.beginPath(); ctx.arc(px(i+1), py(v), 3.5, 0, 2*Math.PI); ctx.fill();
    });

    // x-axis labels
    ctx.fillStyle = "#6b7280"; ctx.font = "10px sans-serif";
    [1,5,10,15,20].forEach(k => {
      ctx.fillText(k, px(k)-4, marg.t+pHeight+14);
    });
    ctx.fillText("n", marg.l+pWidth+5, marg.t+pHeight+14);
    ctx.fillText("S", marg.l-30, marg.t+pHeight/2);
  }

  [rSlider, a1Slider].forEach(s => s.input.addEventListener("input", draw));
  draw();
  return div;
}
```

### 24.2.4. Sigma notation: working with sums

A few practical moves.

**Evaluating a sum.** To evaluate $\sum_{i=1}^{4}(3i-1)$, substitute $i = 1, 2, 3, 4$ and add:

$$\begin{aligned}(3\cdot 1-1)+(3\cdot 2-1)+(3\cdot 3-1)+(3\cdot 4-1)\\ = 2+5+8+11 = 26\end{aligned}$$

You can check this with the arithmetic series formula: $a_1 = 2$, $d = 3$, $n = 4$. $S_4 = \frac{4}{2}(2+11) = 2\times 13 = 26$. Same answer. This means sigma notation and series formulas are two views of the same sum: one expanded, one compressed.

**Changing the index.** The index variable is a dummy — it doesn't matter what letter you use. $\sum_{i=1}^{n} a_i$ and $\sum_{k=1}^{n} a_k$ are the same sum.

**Splitting a sum.** You can split a sum that has two parts:

$$\sum_{i=1}^{n}(a_i+b_i) = \sum_{i=1}^{n} a_i + \sum_{i=1}^{n} b_i$$

And pull a constant out:

$$\sum_{i=1}^{n} c\cdot a_i = c\sum_{i=1}^{n} a_i$$

This means sums follow familiar algebra rules, which is why they are so useful later in calculus and physics.

## 24.3. Worked examples

**Example 1 (Finance).** A car loan of \$12 000 charges 0.5% interest per month. You plan to repay in a lump sum after exactly 24 months — no intermediate payments, interest compounds monthly. What is the total you owe at the end?

Each month the balance is multiplied by 1.005. After 24 months:

$$A = 12000 \times (1.005)^{24}$$

$(1.005)^{24} \approx 1.1272$, so $A \approx 12000 \times 1.1272 = \$13\,526.40$.

Now ask: what is the total interest paid? That is $A - 12000 = \$1\,526.40$.

The balance itself is a geometric sequence with first term 12000 and ratio 1.005. The total interest is what the geometric growth added over 24 steps. This means compound interest is just repeated multiplication written in a financial context.

---

**Example 2 (Computing).** Merge sort splits an array of $n$ elements in half repeatedly, sorts each half, and merges. The merging step at each level does work proportional to $n$. The number of levels is $\log_2 n$. The total work can be written as:

$$T = n + n + n + \cdots \quad (\log_2 n \text{ terms})$$

This is an arithmetic series with all terms equal to $n$:

$$T = n \cdot \log_2 n$$

This is the source of the famous $O(n \log n)$ complexity of merge sort. Each level of the recursion tree contributes $n$ operations; summing $\log_2 n$ such levels gives the total. This means series are not only about money or motion; they also describe the cost of algorithms.

---

**Example 3 (Science).** Zeno's paradox resolved. A runner is 8 metres from a wall. She runs to the halfway point (4 m), then halfway again (2 m), then halfway again (1 m), and so on. The distances form a geometric sequence:

$$4, \quad 2, \quad 1, \quad \frac{1}{2}, \quad \frac{1}{4}, \quad \ldots$$

Here $a_1 = 4$, $r = \frac{1}{2}$, and $|r| < 1$ so the infinite sum converges:

$$S_\infty = \frac{4}{1 - \frac{1}{2}} = \frac{4}{\frac{1}{2}} = 8$$

The infinite sum of all the halfway-steps is exactly 8 metres — the distance to the wall. The paradox dissolves: infinitely many steps do not require infinite time or distance. They require exactly the right amount of both. This means the series gives a precise mathematical answer to a question that looks impossible at first.

---

**Example 4 (Engineering).** A simple periodic signal can be approximated by a partial sum of sine waves. Consider the first five odd harmonics of a square wave at frequency $f$:

$$s(t) \approx \sum_{k=0}^{3} \frac{1}{2k+1} \sin\big((2k+1) \cdot 2\pi f t\big)$$

Expanding the first four terms ($k = 0, 1, 2, 3$):

$$s(t) \approx \sin(2\pi f t) + \frac{1}{3}\sin(6\pi f t) + \frac{1}{5}\sin(10\pi f t) + \frac{1}{7}\sin(14\pi f t)$$

The coefficients $1, \frac{1}{3}, \frac{1}{5}, \frac{1}{7}$ form a series — the terms shrink, so the infinite sum converges. More terms give a closer approximation to the square wave. This is Fourier series: decomposing a complex signal as an infinite sum of simple ones. The tool you need to make sense of it is exactly the infinite series convergence you just learned.

You might notice the coefficients $1, \frac{1}{3}, \frac{1}{5}, \ldots$ shrink slowly — the series $1 + \frac{1}{3} + \frac{1}{5} + \cdots$ does diverge on its own (like the harmonic series). But in the Fourier series each coefficient multiplies an oscillating sine function. Those sines partially cancel each other out, which is what allows the series to converge. The full explanation requires calculus. This means convergence can depend on the whole structure of the terms, not just on the size of the coefficients by themselves.

## 24.4. Where this goes

This chapter moved from finite patterns you can list by hand to infinite processes that still have exact answers. The next volume makes that idea formal with limits and then uses it to build calculus.

The phrase "approaches a finite value as $n \to \infty$" was used informally here. **Limits and continuity** (Vol 5, Ch 1) makes that precise: what does it mean, formally, for a sequence to converge to a number? The epsilon-delta definition is the rigorous version of the intuition you've been building. Every infinite series you have seen in this chapter is, underneath, a limit.

**Integral calculus** (Vol 5, Ch 3) connects series to continuous accumulation. A Riemann sum — the standard way to define the integral — is a series: you divide an area into strips, compute the area of each strip, and sum. As the number of strips goes to infinity, the series converges to the exact area. The

passage from “finite sum” to “integral” is the same passage from “series” to “infinite series” you made in this chapter. **Fourier series** (Vol 7) is the direct destination of example 4: a systematic way to represent any periodic function as an infinite sum of sines and cosines. Every result in Fourier analysis stands on the geometric series convergence you proved here.

**Where this shows up**

- A mortgage calculator computes the total repayment using the geometric series formula for present value of an annuity — every bank does this calculation thousands of times a day.
- Merge sort, binary search, and most divide-and-conquer algorithms have costs that are sums of a geometric series — this is why their Big-O complexity involves logarithms.
- Numerical methods for differential equations (Runge-Kutta, finite differences) rely on Taylor series to approximate function values — an infinite polynomial series truncated at a practical number of terms.
- Audio compression (MP3, AAC) works by computing a Fourier series decomposition, discarding components the ear cannot hear, and reconstructing the signal from the remainder.

## 24.5. Exercises

These are puzzles. Each has a clean numerical answer, but the interesting part is identifying the series type and setting up the formula before you calculate.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opEl = document.createElement("span");
      opEl.style.cssText = "font-size:0.8rem; color:#6b7280; font-style:italic;";
      opEl.textContent = s.op;
      const eqEl = document.createElement("span");
      eqEl.innerHTML = katex.renderToString(s.eq, {throwOnError: false, displayMode: false});
```

```
      row.appendChild(opEl);
      row.appendChild(eqEl);
      container.appendChild(row);
      if (s.note) {
        const noteEl = document.createElement("p");
        noteEl.style.cssText = "margin:0.2rem 0 0.5rem 200px; font-size:0.82rem; color:#374151;";
        noteEl.textContent = s.note;
        container.appendChild(noteEl);
      }
    }
    if (current === totalSteps) {
      const done = document.createElement("p");
      done.style.cssText = "margin-top:0.5rem; font-size:0.85rem; color:#059669; font-weight:600;
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; align-items:center; gap:0.75rem; margin-top:0.5rem;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.35rem 0.85rem; background:#2563eb; color:#fff; border:none;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.35rem 0.85rem; background:#f3f4f6; color:#374151; border:1p
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.8rem; color:#6b7280;";
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); counter.textCon
  resetBtn.onclick = () => { current = 0; nextBtn.disabled = false; renderSteps(); counter.textCo
  counter.textContent = "0 / " + totalSteps;
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  return container;
}
```

**Exercise 1.** The first term of an arithmetic series is 5, the common difference is 4, and there are 20 terms. Find the sum.

```
//| echo: false
{
  const el = makeStepperHTML(1, [
    { op: "Identify the values", eq: "a_1 = 5,\\quad d = 4,\\quad n = 20", note: null },
    { op: "Write the sum formula", eq: "S_n = \\frac{n}{2}\\bigl(2a_1 + (n-1)d\\bigr)", note: "Us
    { op: "Substitute", eq: "S_{20} = \\frac{20}{2}\\bigl(2 \\times 5 + 19 \\times 4\\bigr) = 10\
```

```
    { op: "Simplify", eq: "S_{20} = 10 \\times 86 = 860", note: null },
    { op: "Check", eq: "a_{20} = 5 + 19 \\times 4 = 81;\\quad S_{20} = \\frac{20}{2}(5 + 81) = 10 \\
  ]);
  return el;
}
```

---

**Exercise 2.** A geometric series has first term 3 and common ratio 2. Find the sum of the first 8 terms.

```
//| echo: false
{
  const el = makeStepperHTML(2, [
    { op: "Identify the values", eq: "a_1 = 3,\\quad r = 2,\\quad n = 8", note: null },
    { op: "Write the sum formula", eq: "S_n = \\frac{a_1(1 - r^n)}{1 - r}", note: null },
    { op: "Substitute", eq: "S_8 = \\frac{3(1 - 2^8)}{1 - 2} = \\frac{3(1 - 256)}{-1}", note: null
    { op: "Simplify numerator", eq: "S_8 = \\frac{3 \\times (-255)}{-1} = \\frac{-765}{-1}", note: "
    { op: "Compute", eq: "S_8 = 765", note: null },
    { op: "Check", eq: "3 + 6 + 12 + 24 + 48 + 96 + 192 + 384 = 765 \\checkmark", note: null },
  ]);
  return el;
}
```

---

**Exercise 3.** A bouncing ball is dropped from 10 m. Each bounce reaches 60% of the previous height. Find the total distance travelled by the ball before it comes to rest.

*(Hint: the ball travels down, up, down, up, … Be careful to count both the downward and upward legs of each bounce.)*

```
//| echo: false
{
  const el = makeStepperHTML(3, [
    { op: "Set up the distances", eq: "\\text{Down: } 10 + 6 + 3.6 + \\cdots;\\quad \\text{Up: } 6 +
    { op: "Sum the downward legs", eq: "S_{\\text{down}} = \\frac{10}{1 - 0.6} = \\frac{10}{0.4} = 2
    { op: "Sum the upward legs", eq: "S_{\\text{up}} = \\frac{6}{1 - 0.6} = \\frac{6}{0.4} = 15 \\te
    { op: "Total distance", eq: "S_{\\text{total}} = 25 + 15 = 40 \\text{ m}", note: null },
    { op: "Check (alternative formula)", eq: "S_{\\text{total}} = 10 + 2 \\times \\frac{6}{1-0.6} =
  ]);
  return el;
}
```

---

**Exercise 4.** Evaluate the sum $\sum_{i=1}^{6}(2i+3)$.

*(First expand and add term by term to check your answer, then verify using the arithmetic series formula.)*

```
//| echo: false
{
  const el = makeStepperHTML(4, [
    { op: "Expand the sum", eq: "(2 \\cdot 1 + 3) + (2 \\cdot 2 + 3) + (2 \\cdot 3 + 3) + (2 \\cd
    { op: "Compute each term", eq: "5 + 7 + 9 + 11 + 13 + 15", note: null },
    { op: "Add", eq: "\\sum_{i=1}^{6}(2i+3) = 60", note: null },
    { op: "Verify using the formula", eq: "a_1 = 5,\\quad a_6 = 15,\\quad n = 6 \\implies S_6 = \
  ]);
  return el;
}
```

---

**Exercise 5.** An apprentice electrician earns \$18 per hour in their first year and gets a \$1.50 per hour raise every year. They plan to work in this trade for 12 years. Their annual hours are 1820 each year. What are their total earnings over the 12 years?

*(Note: compute total annual earnings for each year first, then sum the 12 annual totals.)*

```
//| echo: false
{
  const el = makeStepperHTML(5, [
    { op: "Compute year 1 earnings", eq: "E_1 = 18 \\times 1820 = \\$32{,}760", note: null },
    { op: "Identify the series", eq: "a_1 = 32760,\\quad d = 1.50 \\times 1820 = 2730,\\quad n =
    { op: "Apply the arithmetic series formula", eq: "S_{12} = \\frac{12}{2}\\bigl(2 \\times 3276
    { op: "Simplify", eq: "S_{12} = 6\\bigl(65520 + 30030\\bigr) = 6 \\times 95550", note: null }
    { op: "Compute", eq: "S_{12} = \\$573{,}300", note: null },
    { op: "Check", eq: "E_{12} = 18 + 11 \\times 1.50 = \\$34.50 \\text{ per hour};\\quad S_{12}
  ]);
  return el;
}
```

---

**Exercise 6.** Show that the infinite series $\sum_{k=1}^{\infty}\frac{1}{4^k}$ converges, and find its sum.

Then explain in one sentence why the harmonic series $1+\frac{1}{2}+\frac{1}{3}+\frac{1}{4}+\cdots$ does *not* have a finite sum. (Hint: think about how quickly the terms are shrinking compared to a geometric series.)

```
//| echo: false
{
  const el = makeStepperHTML(6, [
    { op: "Write the first few terms", eq: "\\frac{1}{4} + \\frac{1}{16} + \\frac{1}{64} + \\cdots",
    { op: "Identify a₁ and r", eq: "a_1 = \\frac{1}{4},\\quad r = \\frac{1}{4}", note: "Each term is
    { op: "Check convergence condition", eq: "|r| = \\frac{1}{4} < 1 \\implies \\text{converges}", n
    { op: "Apply the infinite sum formula", eq: "S_{\\infty} = \\frac{a_1}{1 - r} = \\frac{\\frac{1}
    { op: "Reasoning about the harmonic series", eq: "\\frac{a_{k+1}}{a_k} = \\frac{1/(k+1)}{1/k} =
    { op: "Check", eq: "\\frac{1}{4} + \\frac{1}{16} + \\frac{1}{64} + \\frac{1}{256} \\approx 0.332
  ]);
  return el;
}
```

# Part V.

# Volume 5 — Analysis and proof

# 25. Volume 5 — Analysis and proof

The central move of this volume is taming the infinite.

Up until now, you've relied on assumptions about how numbers behave between points. This volume strips away those assumptions and rebuilds mathematics on a rigorous foundation of limits, instantaneous rates of change, and continuous accumulation.

By the end of this volume, you will understand how to freeze a changing system precisely at a moment in time, how to add up an infinite number of infinitely small pieces, and how to construct a rock-solid logical argument that proves why something must guaranteeably be true.

## 25.1. Chapters

| Chapter | Title | Core move |
|---|---|---|
| 1 | Limits and continuity | Approaching the boundary |
| 2 | Differential calculus | Measuring instantaneous change |
| 3 | Integral calculus | Accumulating continuous amounts |
| 4 | Proof and reasoning | Establishing absolute certainty |

# 26. Limits and continuity

What a function approaches, not what it equals

Your car's speedometer reads 80 km/h at this exact instant. But speed is distance divided by time — and at a single instant, no time has passed, no distance covered. So what is the speedometer actually measuring?

The answer is a limit. It's the value that the ratio distance/time approaches as the time interval shrinks toward zero — not the ratio at zero, which is 0/0 and meaningless, but the value that ratio gets closer and closer to.

You've already seen this pattern in another form. In the last chapter you watched sequences converge to numbers they never quite reached. A limit is that same idea applied to functions.

## 26.1. The idea before the notation

Three situations will make this concrete. Notice what they have in common.

**The speedometer.** A car's position at time $t$ is some function $s(t)$. To estimate speed, you record position over a short interval: speed $\approx (s(t + \Delta t) - s(t))/\Delta t$. The shorter the interval, the better the estimate. The true instantaneous speed is what this ratio *approaches* as $\Delta t$ shrinks toward zero. The ratio itself is never evaluated at $\Delta t = 0$ — that would be $0/0$ — but it heads steadily toward a definite value.

**The temperature probe.** A weather sensor is mounted on a boundary between two air masses — one at 18°C and one at 12°C. On the warm side, readings approach 18°C. On the cold side, readings approach 12°C. The sensor itself sits exactly at the boundary and might read either, neither, or something else entirely depending on how it was calibrated. The function has a well-defined *approach* from each side. Whether the value at the boundary matches either approach is a separate question.

**Sequences converging to a number.** In Vol 4 ch 4 you watched the sequence $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$ converge to zero. The terms never equal zero, but they get arbitrarily close. Functions can do exactly the same thing: a function can get arbitrarily close to a value $L$ as $x$ approaches some point $a$, without ever equalling $L$ at $x = a$.

All three situations share the same structure: a value is *approached* without necessarily being *reached*. That is the concept of a limit. This means limits let you describe behaviour exactly at the places where ordinary evaluation becomes unreliable.

## 26.2. What the notation is saying

The symbol

$$\lim_{x \to a} f(x) = L$$

is read: *the limit of $f(x)$ as $x$ approaches $a$ equals $L$.*

Every piece matters:

- $x \to a$ means $x$ gets closer and closer to $a$, from either side, but does not equal $a$.
- $f(x)$ is the output of the function at each nearby $x$.
- $L$ is the value that $f(x)$ approaches.

The limit says nothing about $f(a)$ itself. The function might be defined at $a$, undefined at $a$, or defined but with the wrong value. The limit only asks about the *approach*. This means "what happens near the point" and "what happens at the point" are different questions, and calculus keeps them separate on purpose.

### 26.2.1. One-sided limits

Sometimes the approach from the left and the approach from the right give different values. Two separate notations handle this.

$$\lim_{x \to a^-} f(x)$$

is the **left-hand limit** — the value $f(x)$ approaches as $x$ comes in from values *less than $a$*. Read it: *the limit from the left.*

$$\lim_{x \to a^+} f(x)$$

is the **right-hand limit** — the value $f(x)$ approaches as $x$ comes in from values *greater than $a$*. Read it: *the limit from the right.* This means one-sided limits are the tool for checking whether the two sides of a function agree at a boundary.

> **Note**
>
> **The two-sided limit exists if and only if both one-sided limits exist and are equal.** If the left-hand limit is $L_1$ and the right-hand limit is $L_2$, then:
>
> $$\lim_{x \to a} f(x) = L \iff \lim_{x \to a^-} f(x) = \lim_{x \to a^+} f(x) = L$$
>
> If $L_1 \neq L_2$, the two-sided limit does not exist.

### 26.2.2. The limit versus the value — three cases

There is no rule that says $\lim_{x \to a} f(x)$ must equal $f(a)$. In fact, that equality is the definition of continuity, and it can fail in three genuinely distinct ways.

**Case 1 — Continuous point.** The function is defined at $a$, the limit exists, and they agree:

$$\lim_{x \to a} f(x) = f(a)$$

Example: $f(x) = x^2$ at $a = 3$. The limit is 9, the value is 9.

**Case 2 — Removable discontinuity (value mismatch).** The limit exists, $f(a)$ is defined, but they are unequal. The function has a value at $a$ — it is simply the *wrong* value relative to what the surrounding behaviour predicts.

Example: define $g(x) = x + 2$ for $x \neq 2$, but $g(2) = 7$. The limit as $x \to 2$ is 4. The function value at $x = 2$ is 7. There is a mismatch. The discontinuity can be removed by redefining $g(2) = 4$.

**Case 3 — Hole (value absent).** The limit exists, but $f(a)$ is simply not defined. There is no value to mismatch — the function has a gap at $x = a$.

Example: $h(x) = \dfrac{x^2 - 4}{x - 2}$. At $x = 2$ the denominator is zero, so $h(2)$ is undefined. But for every $x \neq 2$:

$$h(x) = \frac{(x-2)(x+2)}{x-2} = x + 2$$

The limit is 4. The function has a hole at $(2, 4)$ — no value is there at all. Patching the hole by defining $h(2) = 4$ would make $h$ continuous.

Cases 2 and 3 are genuinely different situations: in Case 2 you have a value but it conflicts with the limit; in Case 3 you simply have nothing. Both are called *removable discontinuities* because the fix — redefining or defining $f(a) = L$ — is the same in both cases. This means continuity fails here for a local reason only: the surrounding behaviour already agrees on what the value should be.

```
//| echo: false
d3_limits = require("d3@7")
```

```
//| echo: false
viewof xSlider = Inputs.range([0.5, 3.5], {
  step: 0.01,
  value: 0.5,
  label: "x approaching 2"
})
```

```
//| echo: false
{
  const width = 560, height = 300;
  const margin = { top: 20, right: 20, bottom: 40, left: 45 };
  const innerW = width - margin.left - margin.right;
  const innerH = height - margin.top - margin.bottom;

  const xDom = [-1, 5];
  const yDom = [0, 6];

  const xScale = d3_limits.scaleLinear().domain(xDom).range([0, innerW]);
  const yScale = d3_limits.scaleLinear().domain(yDom).range([innerH, 0]);

  const svg = d3_limits.create("svg")
    .attr("width", width)
    .attr("height", height)
    .style("font-family", "inherit");

  const g = svg.append("g")
    .attr("transform", `translate(${margin.left},${margin.top})`);

  // Axes
  g.append("g")
    .attr("transform", `translate(0,${innerH})`)
    .call(d3_limits.axisBottom(xScale).ticks(6))
    .append("text")
    .attr("x", innerW / 2).attr("y", 35)
    .attr("fill", "#374151").attr("text-anchor", "middle")
    .style("font-size", "13px").text("x");

  g.append("g")
    .call(d3_limits.axisLeft(yScale).ticks(6))
    .append("text")
    .attr("transform", "rotate(-90)")
    .attr("x", -innerH / 2).attr("y", -35)
    .attr("fill", "#374151").attr("text-anchor", "middle")
    .style("font-size", "13px").text("f(x)");

  // Grid
  g.append("g")
    .selectAll("line.hgrid")
    .data(yScale.ticks(6))
    .join("line")
    .attr("x1", 0).attr("x2", innerW)
    .attr("y1", d => yScale(d)).attr("y2", d => yScale(d))
    .attr("stroke", "#f3f4f6").attr("stroke-width", 1);
```

```
// Line generator
const line = d3_limits.line()
  .x(d => xScale(d[0]))
  .y(d => yScale(d[1]));

// Left segment: x in [-1, 2) — f(x) = x+2
const leftPts = d3_limits.range(-1, 1.98, 0.01).map(x => [x, x + 2]);
g.append("path")
  .datum(leftPts)
  .attr("d", line)
  .attr("fill", "none")
  .attr("stroke", "#2563eb")
  .attr("stroke-width", 2.5);

// Right segment: x in (2, 5] — f(x) = x+2
const rightPts = d3_limits.range(2.02, 5.01, 0.01).map(x => [x, x + 2]);
g.append("path")
  .datum(rightPts)
  .attr("d", line)
  .attr("fill", "none")
  .attr("stroke", "#2563eb")
  .attr("stroke-width", 2.5);

// Open circle at (2, 4)
g.append("circle")
  .attr("cx", xScale(2)).attr("cy", yScale(4))
  .attr("r", 6)
  .attr("fill", "white")
  .attr("stroke", "#2563eb")
  .attr("stroke-width", 2.5);

// Moving dot — hide when within 0.02 of x=2
const x = xSlider;
const fx = x + 2;
const nearHole = Math.abs(x - 2) < 0.02;

if (!nearHole) {
  g.append("circle")
    .attr("cx", xScale(x)).attr("cy", yScale(fx))
    .attr("r", 6)
    .attr("fill", "#dc2626");
}

// Static label
g.append("text")
  .attr("x", xScale(3.6)).attr("y", yScale(5.5))
  .attr("fill", "#6b7280")
```

```
    .style("font-size", "12px")
    .text("The dot never lands at x = 2");

  // Dynamic readout
  const readX = x.toFixed(3);
  const readFx = nearHole ? "undefined" : fx.toFixed(3);

  g.append("text")
    .attr("x", xScale(-0.8)).attr("y", yScale(5.6))
    .attr("fill", "#374151")
    .style("font-size", "13px").style("font-weight", "500")
    .text(`x = ${readX}`);

  g.append("text")
    .attr("x", xScale(-0.8)).attr("y", yScale(5.1))
    .attr("fill", nearHole ? "#9ca3af" : "#374151")
    .style("font-size", "13px").style("font-weight", "500")
    .text(`f(x) ⍰ ${readFx}`);

  return svg.node();
}
```

## 26.3. The method — evaluating limits

Four techniques cover the vast majority of problems you will encounter.

### 26.3.1. Technique 1: Direct substitution

If $f$ is a polynomial, a rational function with a non-zero denominator at $a$, a trigonometric function, or any other function that is *continuous* at $a$ — we'll define exactly what this means shortly, but the idea is that the function has no break at that point — then:

$$\lim_{x \to a} f(x) = f(a)$$

Just substitute. This always works when no algebraic obstruction exists. This means direct substitution works whenever the function behaves normally at the point you are approaching.

**When to use it:** Try this first. If the substitution gives a real number, you're done.

**Worked example:** Find $\lim_{x \to 3}(x^2 + 2x - 1)$.

Substitute $x = 3$ directly:

$$\lim_{x \to 3}(x^2 + 2x - 1) = 3^2 + 2(3) - 1 = 9 + 6 - 1 = 14$$

The limit is 14. The function is a polynomial — continuous everywhere — so direct substitution is valid at every real number. This means polynomials are the simplest case: nearby behaviour and actual value already match.

**When it fails:** If substitution gives $\frac{0}{0}$, $\frac{\infty}{\infty}$, or another indeterminate form, you need one of the other techniques.

---

### 26.3.2. Technique 2: Factor and cancel

The $\frac{0}{0}$ form is a signal: both numerator and denominator share a common factor that vanishes at $x = a$. Factor both, cancel the common factor, and then substitute.

**When to use it:** When direct substitution gives $\frac{0}{0}$ and the expression is a ratio of polynomials.

**Worked example:** Find $\lim_{x\to 2} \dfrac{x^2-4}{x-2}$.

**Step 1.** Attempt direct substitution: $\dfrac{2^2-4}{2-2} = \dfrac{0}{0}$. Indeterminate — proceed to step 2.

**Step 2.** Factor the numerator:

$$\frac{x^2-4}{x-2} = \frac{(x-2)(x+2)}{x-2}$$

**Step 3.** Cancel the common factor $(x-2)$. This is valid because we are evaluating a limit as $x \to 2$, meaning $x \neq 2$, so $(x-2) \neq 0$:

$$\frac{(x-2)(x+2)}{x-2} = x+2 \quad (x \neq 2)$$

**Step 4.** Now substitute:

$$\lim_{x\to 2} \frac{x^2-4}{x-2} = \lim_{x\to 2}(x+2) = 2+2 = 4$$

The cancellation is legal only because we never set $x = 2$ — the limit asks what happens *as* $x$ approaches 2, not *at* $x = 2$. This means factoring does not change the nearby behaviour of the function; it reveals the simpler rule hidden behind the removable discontinuity.

---

### 26.3.3. Technique 3: Rationalise

When a square root appears and direct substitution gives $\frac{0}{0}$, the standard move is to multiply numerator and denominator by the **conjugate** of the expression containing the square root. The conjugate of $(\sqrt{u} - v)$ is $(\sqrt{u} + v)$. Multiplying removes the radical from one factor using the difference-of-squares identity.

**When to use it:** When the $\frac{0}{0}$ form arises from a square root.

**Worked example:** Find $\lim_{x\to 0} \dfrac{\sqrt{x+4}-2}{x}$.

**Step 1.** Direct substitution: $\dfrac{\sqrt{0+4}-2}{0} = \dfrac{2-2}{0} = \dfrac{0}{0}$. Indeterminate — proceed.

**Step 2.** Multiply numerator and denominator by the conjugate $(\sqrt{x+4}+2)$:

$$\frac{\sqrt{x+4}-2}{x} \cdot \frac{\sqrt{x+4}+2}{\sqrt{x+4}+2}$$

**Step 3.** Expand the numerator using $(a-b)(a+b) = a^2 - b^2$:

$$\frac{(\sqrt{x+4})^2 - 2^2}{x(\sqrt{x+4}+2)} = \frac{(x+4)-4}{x(\sqrt{x+4}+2)} = \frac{x}{x(\sqrt{x+4}+2)}$$

**Step 4.** Cancel the common factor $x$ (valid since $x \to 0$ means $x \neq 0$):

$$\frac{x}{x(\sqrt{x+4}+2)} = \frac{1}{\sqrt{x+4}+2}$$

**Step 5.** Now substitute $x = 0$:

$$\lim_{x\to 0} \frac{\sqrt{x+4}-2}{x} = \frac{1}{\sqrt{0+4}+2} = \frac{1}{2+2} = \frac{1}{4}$$

The limit is $\frac{1}{4}$. This means rationalising removes the square-root expression that is causing the indeterminate form.

---

### 26.3.4. Technique 4: One-sided limits from graphs

When you have a graph (or can sketch one), you can read limits directly. Look at the curve's *approach* as $x$ moves toward the target from each side. Do not look at the dot at the target point — look at where the curve is *heading*.

**When to use it:** Piecewise functions, graphs given without formulas, or any function with a potential jump.

**Worked example:** Let

$$f(x) = \begin{cases} x+1 & x < 3 \\ 5 & x = 3 \\ 2x-4 & x > 3 \end{cases}$$

Find $\lim_{x\to 3^-} f(x)$, $\lim_{x\to 3^+} f(x)$, and $\lim_{x\to 3} f(x)$.

**Left-hand limit.** For $x < 3$, the rule is $f(x) = x + 1$. As $x \to 3^-$:

$$\lim_{x\to 3^-} f(x) = 3 + 1 = 4$$

**Right-hand limit.** For $x > 3$, the rule is $f(x) = 2x - 4$. As $x \to 3^+$:

$$\lim_{x\to 3^+} f(x) = 2(3) - 4 = 2$$

**Two-sided limit.** The left and right limits are $4 \neq 2$, so:

$$\lim_{x\to 3} f(x) \text{ does not exist}$$

Note that $f(3) = 5$ is defined — but the two-sided limit still fails to exist because the one-sided limits disagree. The value at the point is irrelevant to whether the limit exists. This means a function can have a value at a point and still fail to have a two-sided limit there.

---

## 26.4. Continuity

With limits in hand, the definition of continuity is short.

> **Note**
>
> **Definition.** A function $f$ is **continuous at** $a$ if all three conditions hold:
>
> 1. $f(a)$ is defined.
> 2. $\lim_{x\to a} f(x)$ exists.
> 3. $\lim_{x\to a} f(x) = f(a)$.

> If any condition fails, $f$ is **discontinuous at** $a$.

This means continuity is a three-part agreement: the value exists, the limit exists, and both are the same.

Informally: a function is continuous at a point if you can draw through that point without lifting your pen. The limit and the value agree.

### 26.4.1. The three types of discontinuity

**Removable discontinuity.** Condition 1 or 3 fails, but condition 2 holds. The limit exists, but either the function is not defined there, or the function is defined but gives a different value. The graph has a single isolated hole. The discontinuity can be removed by redefining $f(a) = L$.

Example: $g(x) = \dfrac{x^2 - 4}{x - 2}$ is not defined at $x = 2$, but $\lim_{x \to 2} g(x) = 4$. Removable.

**Jump discontinuity.** The left-hand and right-hand limits both exist, but they are not equal. The function "jumps" from one value to another. No amount of redefining a single point can fix this — the gap is structural.

Example: The piecewise function from Technique 4 above. At $x = 3$, the left limit is 4 and the right limit is 2.

**Infinite discontinuity.** One or both one-sided limits are $\pm\infty$. The graph has a vertical asymptote at $x = a$.

Example: $h(x) = \dfrac{1}{x}$ at $x = 0$. As $x \to 0^+$, $h(x) \to +\infty$. As $x \to 0^-$, $h(x) \to -\infty$. This means the type of discontinuity tells you *why* continuity failed, not just that it failed.

```
//| echo: false
d3_cont = require("d3@7")
```

```
//| echo: false
viewof selectedPanel = Inputs.radio(["Removable", "Jump", "Infinite"], {
  label: "Select discontinuity type",
  value: "Removable"
})
```

```
//| echo: false
{
  const panelW = 170, panelH = 180;
  const m = { top: 28, right: 12, bottom: 28, left: 30 };
  const iW = panelW - m.left - m.right;
  const iH = panelH - m.top - m.bottom;
  const totalW = 3 * panelW + 2 * 16;

  const panels = [
    { id: "Removable", title: "Removable (hole)" },
```

```
    { id: "Jump",      title: "Jump" },
    { id: "Infinite",  title: "Infinite" }
  ];

  const conditions = {
    Removable: [
      { label: "Condition 1 — f(a) defined", pass: false, note: "f(2) is undefined (0/0)" },
      { label: "Condition 2 — limit exists",  pass: true,  note: "lim = 4 □" },
      { label: "Condition 3 — limit = f(a)",  pass: false, note: "f(a) missing, so cannot agree"
    ],
    Jump: [
      { label: "Condition 1 — f(a) defined", pass: true,  note: "f(3) is defined □" },
      { label: "Condition 2 — limit exists",  pass: false, note: "left □ right, limit DNE" },
      { label: "Condition 3 — limit = f(a)",  pass: false, note: "limit doesn't exist, fails" }
    ],
    Infinite: [
      { label: "Condition 1 — f(a) defined", pass: false, note: "f(0) = 1/0, undefined" },
      { label: "Condition 2 — limit exists",  pass: false, note: "limits are ±∞" },
      { label: "Condition 3 — limit = f(a)",  pass: false, note: "neither condition holds" }
    ]
  };

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit;";

  // Panels row
  const svgRow = document.createElement("div");
  svgRow.style.cssText = "display:flex; gap:16px; flex-wrap:wrap;";

  panels.forEach(p => {
    const isSelected = p.id === selectedPanel;
    const wrap = document.createElement("div");
    wrap.style.cssText = `border:2px solid ${isSelected ? "#2563eb" : "#e5e7eb"}; border-radius:8

    const svg = d3_cont.create("svg")
      .attr("width", panelW)
      .attr("height", panelH);

    const g = svg.append("g").attr("transform", `translate(${m.left},${m.top})`);

    // Title
    svg.append("text")
      .attr("x", panelW / 2).attr("y", 16)
      .attr("text-anchor", "middle")
      .attr("fill", isSelected ? "#1d4ed8" : "#374151")
      .style("font-size", "11px").style("font-weight", "600")
      .text(p.title);
```

```
  if (p.id === "Removable") {
    const xS = d3_cont.scaleLinear().domain([0, 4]).range([0, iW]);
    const yS = d3_cont.scaleLinear().domain([0, 6]).range([iH, 0]);
    g.append("g").attr("transform", `translate(0,${iH})`).call(d3_cont.axisBottom(xS).ticks(4).tic
    g.append("g").call(d3_cont.axisLeft(yS).ticks(4).tickSize(3));
    const pts = d3_cont.range(0, 1.97, 0.05).concat(d3_cont.range(2.03, 4.01, 0.05));
    const leftPts = d3_cont.range(0, 1.97, 0.05).map(x => [x, x + 2]);
    const rightPts = d3_cont.range(2.03, 4.01, 0.05).map(x => [x, x + 2]);
    const line = d3_cont.line().x(d => xS(d[0])).y(d => yS(d[1]));
    g.append("path").datum(leftPts).attr("d", line).attr("fill","none").attr("stroke","#2563eb").a
    g.append("path").datum(rightPts).attr("d", line).attr("fill","none").attr("stroke","#2563eb")
    g.append("circle").attr("cx", xS(2)).attr("cy", yS(4)).attr("r", 5).attr("fill","white").attr
  }

  if (p.id === "Jump") {
    const xS = d3_cont.scaleLinear().domain([0, 5]).range([0, iW]);
    const yS = d3_cont.scaleLinear().domain([0, 7]).range([iH, 0]);
    g.append("g").attr("transform", `translate(0,${iH})`).call(d3_cont.axisBottom(xS).ticks(5).tic
    g.append("g").call(d3_cont.axisLeft(yS).ticks(5).tickSize(3));
    const line = d3_cont.line().x(d => xS(d[0])).y(d => yS(d[1]));
    // y=x for x<3
    g.append("path").datum([[0,0],[2.97,2.97]]).attr("d", line).attr("fill","none").attr("stroke"
    // y=2x-4 for x>3
    g.append("path").datum([[3.03,2.06],[5,6]]).attr("d", line).attr("fill","none").attr("stroke"
    // open circles at gap ends
    g.append("circle").attr("cx", xS(3)).attr("cy", yS(3)).attr("r", 4).attr("fill","white").attr
    g.append("circle").attr("cx", xS(3)).attr("cy", yS(2)).attr("r", 4).attr("fill","white").attr
  }

  if (p.id === "Infinite") {
    const xS = d3_cont.scaleLinear().domain([-2, 2]).range([0, iW]);
    const yS = d3_cont.scaleLinear().domain([-4, 4]).range([iH, 0]);
    g.append("g").attr("transform", `translate(0,${iH/2})`).call(d3_cont.axisBottom(xS).ticks(4).t
    g.append("g").call(d3_cont.axisLeft(yS).ticks(4).tickSize(3));
    // 1/x left branch
    const leftB = d3_cont.range(-2, -0.28, 0.05).map(x => [x, 1/x]);
    const rightB = d3_cont.range(0.28, 2.01, 0.05).map(x => [x, 1/x]);
    const line = d3_cont.line().x(d => xS(d[0])).y(d => yS(d[1])).defined(d => Math.abs(d[1]) <= 4
    g.append("path").datum(leftB).attr("d", line).attr("fill","none").attr("stroke","#2563eb").att
    g.append("path").datum(rightB).attr("d", line).attr("fill","none").attr("stroke","#2563eb").at
    // vertical dashed asymptote
    g.append("line")
      .attr("x1", xS(0)).attr("x2", xS(0))
      .attr("y1", 0).attr("y2", iH)
      .attr("stroke", "#dc2626").attr("stroke-width", 1.5)
      .attr("stroke-dasharray", "4,3");
  }
```

```
    wrap.appendChild(svg.node());
    svgRow.appendChild(wrap);
  });

  container.appendChild(svgRow);

  // Condition analysis
  const info = document.createElement("div");
  info.style.cssText = "margin-top:1rem; border:1px solid #e5e7eb; border-radius:8px; padding:0.7

  const title = document.createElement("div");
  title.style.cssText = "font-weight:600; font-size:0.95em; color:#374151; margin-bottom:0.5rem;"
  title.textContent = `${selectedPanel} discontinuity — condition analysis`;
  info.appendChild(title);

  conditions[selectedPanel].forEach(c => {
    const row = document.createElement("div");
    row.style.cssText = "display:flex; align-items:baseline; gap:0.5rem; padding:0.2rem 0; font-s
    const icon = document.createElement("span");
    icon.textContent = c.pass ? "✓" : "✗";
    icon.style.cssText = `font-weight:700; color:${c.pass ? "#059669" : "#dc2626"}; min-width:1.2
    const text = document.createElement("span");
    text.style.color = "#374151";
    text.textContent = c.label + " — " + c.note;
    row.appendChild(icon);
    row.appendChild(text);
    info.appendChild(row);
  });

  container.appendChild(info);
  return container;
}
```

## 26.5. The epsilon-delta definition

The techniques above let you *compute* limits. The epsilon-delta definition lets you *prove* they are correct. These are different activities, and most students — even in university calculus — use the techniques far more often than the proof machinery. But understanding the definition is what separates a practitioner from a mathematician.

Return to the speedometer. Suppose you want to estimate speed to within $\varepsilon = 0.01$ km/h. You need to take a time interval short enough — call it $\delta$ seconds — that the average speed over that interval lands within 0.01 km/h of the true instantaneous speed. Different tolerance $\varepsilon$, different required $\delta$. The claim "the limit is $L$" means: *for every tolerance you name, I can find a window small enough to guarantee that accuracy.*

**Important**

**Epsilon-delta definition of a limit.**

$$\lim_{x \to a} f(x) = L$$

means: for every $\varepsilon > 0$, there exists $\delta > 0$ such that

$$0 < |x - a| < \delta \implies |f(x) - L| < \varepsilon$$

In plain language: however small a target band you set around $L$ (the $\varepsilon$-band), I can find a window around $a$ (the $\delta$-window) such that every $x$ inside the window — but not equal to $a$ itself — sends $f(x)$ inside the target band. This means no matter how tightly you specify the output window around $L$, the inputs can be restricted tightly enough around $a$ to force the function into that window.

**One worked example.** Prove that $\lim_{x \to 2} 3x = 6$.

We need to show that for any $\varepsilon > 0$ we can produce a $\delta > 0$ satisfying the definition.

We want $|f(x) - L| < \varepsilon$, i.e., $|3x - 6| < \varepsilon$.

Factor: $|3x - 6| = 3|x - 2|$.

So the condition $|3x - 6| < \varepsilon$ becomes $3|x - 2| < \varepsilon$, i.e., $|x - 2| < \varepsilon/3$.

This tells us to choose $\delta = \varepsilon/3$.

**Verification.** Suppose $0 < |x - 2| < \delta = \varepsilon/3$. Then:

$$|3x - 6| = 3|x - 2| < 3 \cdot \frac{\varepsilon}{3} = \varepsilon$$

The condition is satisfied for every $\varepsilon > 0$. The limit is 6. □ This means the whole epsilon-delta proof is a controlled translation from an allowed output error back to a sufficient input error.

**Note**

**Why this works — the logical structure.**
The phrase "for every $\varepsilon > 0$" covers every possible tolerance, however tiny. The phrase "there exists $\delta > 0$" says you can always respond to any challenge.
Together they convert the informal idea of "approaches without reaching" — which involves an infinite process — into a finite, checkable algebraic statement. You never have to evaluate an infinite process; you just have to exhibit the $\delta$ as a function of $\varepsilon$ and verify the implication.
This is the logical structure of most rigorous analysis. The techniques in the previous section are shortcuts valid because the functions involved are "nice enough" that the $\varepsilon$-$\delta$ machinery can be applied once and the results packaged into rules.

## 26.6. Worked examples

### 26.6.1. Example 1 — Direct substitution (polynomial)

**Problem.** Find $\lim_{x\to 3}(x^2 + 2x - 1)$.

**Context.** In structural engineering, polynomial functions describe deflection curves along a beam. Evaluating a deflection at a specific point is direct substitution.

**Solution.**

The function $f(x) = x^2 + 2x - 1$ is a polynomial. Polynomials are continuous everywhere. Direct substitution is valid at $x = 3$:

$$\lim_{x\to 3}(x^2 + 2x - 1) = 3^2 + 2(3) - 1 = 9 + 6 - 1 = \boxed{14}$$

**Technique used:** Direct substitution. Applied whenever the function is continuous at the target point — no denominator that vanishes, no radical causing trouble.

---

### 26.6.2. Example 2 — Factoring (removable discontinuity)

**Problem.** Find $\lim_{x\to 2} \dfrac{x^2 - 4}{x - 2}$.

**Context.** In software engineering, a function representing computational cost might be undefined at a specific input size but still have a meaningful limiting value. Analysing the limit is the correct tool.

**Solution.**

**Step 1.** Direct substitution gives $\frac{0}{0}$ — indeterminate.

**Step 2.** Factor:

$$\frac{x^2 - 4}{x - 2} = \frac{(x - 2)(x + 2)}{x - 2}$$

**Step 3.** Cancel $(x - 2)$, valid since $x \neq 2$ in a limit:

$$= x + 2 \quad (x \neq 2)$$

**Step 4.** Substitute:

$$\lim_{x\to 2} \frac{x^2 - 4}{x - 2} = \lim_{x\to 2}(x + 2) = 2 + 2 = \boxed{4}$$

The function has a hole at $(2, 4)$ — a removable discontinuity. The limit exists and equals 4 even though the function is undefined at $x = 2$.

---

### 26.6.3. Example 3 — Rationalisation (resolving an indeterminate form)

**Problem.** Find $\lim_{x\to 0} \dfrac{\sqrt{x+4}-2}{x}$.

**Context.** In chemistry, reaction rate expressions often involve differences of concentrations under a square root. Finding the limiting rate at a reference condition uses exactly this technique.

**Solution.**

**Step 1.** Direct substitution: $\frac{\sqrt{4}-2}{0} = \frac{0}{0}$. Indeterminate.

**Step 2.** Multiply by the conjugate $\frac{\sqrt{x+4}+2}{\sqrt{x+4}+2}$:

$$\frac{\sqrt{x+4}-2}{x} \cdot \frac{\sqrt{x+4}+2}{\sqrt{x+4}+2} = \frac{(x+4)-4}{x(\sqrt{x+4}+2)} = \frac{x}{x(\sqrt{x+4}+2)}$$

**Step 3.** Cancel $x$ ($x \neq 0$ in the limit):

$$= \frac{1}{\sqrt{x+4}+2}$$

**Step 4.** Substitute $x = 0$:

$$\lim_{x\to 0} \frac{\sqrt{x+4}-2}{x} = \frac{1}{\sqrt{4}+2} = \frac{1}{4} = \boxed{0.25}$$

---

### 26.6.4. Example 4 — One-sided limits (regime change)

**Problem.** A continuous-time financial model uses the interest rate function

$$r(t) = \begin{cases} 0.04 & t < 5 \\ 0.04 & t = 5 \\ 0.065 & t > 5 \end{cases}$$

where $t$ is years from the start of a contract. Does $\lim_{t\to 5} r(t)$ exist?

**Context.** In finance, many contracts include rate resets at specified times. Whether the limit exists at the reset point determines whether the model is mathematically continuous or whether a jump discontinuity must be handled separately.

**Solution.**

**Left-hand limit:** As $t \to 5^-$, the rule gives $r(t) = 0.04$:

$$\lim_{t\to 5^-} r(t) = 0.04$$

**Right-hand limit:** As $t \to 5^+$, the rule gives $r(t) = 0.065$:

$$\lim_{t \to 5^+} r(t) = 0.065$$

**Two-sided limit:** The one-sided limits are unequal ($0.04 \neq 0.065$), so:

$$\lim_{t \to 5} r(t) \text{ does not exist}$$

This is a jump discontinuity. Note that $r(5) = 0.04$ is defined — the value exists at the point — but the two-sided limit does not. The model has a structural break at $t = 5$ that must be accounted for, not patched over.

## 26.7. Where this goes

This chapter took the informal idea of "getting closer and closer" and turned it into a precise language for analysing functions. The next chapter uses that language to define the derivative itself.

**Toward differential calculus (Chapter 2).** The derivative of $f$ at a point $a$ is defined as:

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

This is precisely a limit of the $\frac{0}{0}$ form — the same kind you evaluated in Examples 2 and 3 above. The technique of factoring, cancelling, and substituting that you practised here is exactly the algebra you will apply to evaluate derivatives in the next chapter. Limits are not preliminary material to get past — they are the mechanism the derivative is made of.

**Back to sequences and series (Vol 4 Ch 4).** The convergence you studied there was the same idea in a different setting. When you showed that $\sum_{k=0}^{\infty} \left(\frac{1}{2}\right)^k = 2$, you were evaluating a limit of partial sums: $\lim_{n \to \infty} S_n = 2$. The same idea — for any tolerance, there's a point beyond which the partial sums stay within that tolerance — reappears in the formal definition of series convergence. The concepts are one family, not two separate topics.

> **Note**
>
> **Where limits appear in practice**
>
> - **Speedometers and velocity.** Instantaneous velocity is $\lim_{\Delta t \to 0} \frac{\Delta s}{\Delta t}$ — the starting point for all of mechanics.
> - **Continuously compounded interest.** $\lim_{n \to \infty} \left(1 + \frac{r}{n}\right)^n = e^r$. Every continuous-time financial model uses this limit.
> - **Numerical root-finding.** The bisection method generates a sequence of intervals that converge to a root. Convergence is a limit: each iteration halves the error, and the sequence of midpoints limits to the root.

- **Signal processing.** A low-pass filter's attenuation as frequency $\to \infty$ is a limit question. Whether the filter "works" depends on whether that limit is zero.

## 26.8. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
```

```
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

**Exercise 1.** A structural engineer models the deflection of a beam using $d(x) = 2x^3 - 3x^2 + x + 1$, where $x$ is position in metres.

Find $\lim_{x \to 2} d(x)$.

```
//| echo: false
makeStepperHTML(1, [
  { op: "Identify the function type",  eq: "\\text{d(x) = 2x^3 - 3x^2 + x + 1 is a polynomial}" }
  { op: "Apply direct substitution",   eq: "\\text{Polynomials are continuous everywhere, so } \\
  { op: "Evaluate",                    eq: "d(2) = 2(2)^3 - 3(2)^2 + 2 + 1 = 16 - 12 + 2 + 1 = 7"
  { op: "State the answer",            eq: "\\lim_{x \\to 2} d(x) = 7 \\text{ metres}" }
])
```

---

**Exercise 2.** A computing platform charges users based on data processed. The cost function has the form

$$C(x) = \frac{x^2 - 9}{x - 3}$$

where $x$ is gigabytes processed. The pricing model is undefined at $x = 3$ GB due to a contract edge case. Find $\lim_{x \to 3} C(x)$.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Attempt direct substitution", eq: "C(3) = \\frac{9 - 9}{3 - 3} = \\frac{0}{0}", note: "Inde
  { op: "Factor the numerator",        eq: "x^2 - 9 = (x-3)(x+3), \\text{ so } C(x) = \\frac{(x-3)(x
  { op: "Cancel the common factor",    eq: "C(x) = x + 3 \\quad (x \\neq 3)" },
  { op: "Substitute",                  eq: "\\lim_{x \\to 3} C(x) = 3 + 3 = 6" },
  { op: "Interpret",                   eq: "\\text{Cost approaches \\$6 per GB as usage approaches 3
])
```

---

**Exercise 3.** In biochemistry, a reaction rate expression takes the form

$$R(c) = \frac{\sqrt{c + 9} - 3}{c}$$

where $c$ is the change in concentration from a reference value. Find $\lim_{c \to 0} R(c)$.

```
//| echo: false
makeStepperHTML(3, [
  { op: "Attempt direct substitution", eq: "R(0) = \\frac{\\sqrt{9} - 3}{0} = \\frac{0}{0}", note: "
  { op: "Multiply by the conjugate",   eq: "\\frac{\\sqrt{c+9} - 3}{c} \\cdot \\frac{\\sqrt{c+9} + 3
  { op: "Cancel c",                    eq: "\\frac{1}{\\sqrt{c+9}+3}", note: "Valid since c ≠ 0 in t
  { op: "Substitute c = 0",            eq: "\\frac{1}{\\sqrt{9}+3} = \\frac{1}{3+3} = \\frac{1}{6}"
  { op: "Conclude",                    eq: "\\lim_{c \\to 0} R(c) = \\dfrac{1}{6}" }
])
```

---

**Exercise 4.** A piecewise model for a climate variable switches regime at time $t = 10$ years:

$$T(t) = \begin{cases} 2t + 1 & t < 10 \\ 30 & t = 10 \\ 3t - 8 & t > 10 \end{cases}$$

(a) Find $\lim_{t \to 10^-} T(t)$.
(b) Find $\lim_{t \to 10^+} T(t)$.
(c) Does $\lim_{t \to 10} T(t)$ exist? Explain.
(d) Is $T$ continuous at $t = 10$?

```
//| echo: false
makeStepperHTML(4, [
  { op: "Left-hand limit",   eq: "\\text{For } t < 10,\\; T(t) = 2t+1.\\; \\lim_{t \\to 10^-} T(t
  { op: "Right-hand limit",  eq: "\\text{For } t > 10,\\; T(t) = 3t-8.\\; \\lim_{t \\to 10^+} T(t
  { op: "Two-sided limit",   eq: "21 \\neq 22 \\implies \\lim_{t \\to 10} T(t) \\text{ does not e
  { op: "Check continuity",  eq: "T(10) = 30 \\text{ (defined ⍰), but two-sided limit DNE ⍰}", no
  { op: "Classify",          eq: "\\text{Jump discontinuity: both one-sided limits exist but are
])
```

---

**Exercise 5.** Use the three conditions to determine whether

$$f(x) = \begin{cases} x^2 - 1 & x \neq 2 \\ 3 & x = 2 \end{cases}$$

is continuous at $x = 2$.

```
//| echo: false
makeStepperHTML(5, [
  { op: "Condition 1: f(2) defined?",           eq: "f(2) = 3 \\text{ — defined ⍰}" },
  { op: "Condition 2: limit exists?",            eq: "\\lim_{x \\to 2}(x^2 - 1) = 4 - 1 = 3 \\tex
  { op: "Condition 3: limit = f(2)?",            eq: "\\lim = 3 = f(2) = 3 \\text{ — agree ⍰}" },
  { op: "Conclude",                              eq: "\\text{All three conditions hold: } f \\tex
  { op: "Note",                                  eq: "\\text{The piecewise value at } x=2 \\text{
])
```

---

**Exercise 6 (epsilon-delta).** Use the $\varepsilon$-$\delta$ definition to prove that $\lim_{x \to 1}(5x - 2) = 3$.

```
//| echo: false
makeStepperHTML(6, [
  { op: "Write down what to show",     eq: "\\forall\\, \\varepsilon > 0,\\; \\exists\\, \\delta
  { op: "Simplify the target",         eq: "|(5x-2)-3| = |5x-5| = 5|x-1|" },
  { op: "Work backwards to find ⍰",    eq: "5|x-1| < \\varepsilon \\implies |x-1| < \\varepsilon/
  { op: "Write the proof",             eq: "0 < |x-1| < \\delta = \\varepsilon/5 \\implies |(5x-2
  { op: "Reflect",                     eq: "\\text{Key move: simplify } |f(x)-L| \\text{ until it
])
```

# 27. Differential calculus

The instantaneous rate of change

Your car's GPS records your average speed over the entire trip: 62 km/h. But your speedometer, right now, reads 90 km/h. These are different quantities. The GPS average looks back over the whole journey; the speedometer is telling you something about *this instant*. The question "how fast am I going right now?" is not the same question as "how fast did I go on average?" — and answering it requires a fundamentally different tool.

Here is a sharper version of the same problem. A drug is injected into the bloodstream. A doctor monitors its concentration — measured in milligrams per litre — every hour. She can compute the average rate of clearance over a two-hour window. But what determines the dosing schedule is the *instantaneous* rate of clearance right now: how fast is the concentration dropping at this moment? If that rate is too slow, the drug accumulates to toxic levels. Too fast, and the dose is ineffective before the next one can be given. The decision is made on the basis of a rate at a point in time, not an average across an interval.

Both situations share the same mathematical structure. In chapter 1 you met limits: the value that a function approaches as the input gets arbitrarily close to some point. The speedometer reading *is* a limit — the limit of average speeds over shorter and shorter time intervals. This chapter builds that limit into a systematic tool.

## 27.1. The limit definition of the derivative

The derivative has a precise definition as a limit. Start with the **average rate of change** of $f$ over the interval from $x$ to $x + h$:

$$\frac{f(x+h) - f(x)}{h}$$

The numerator $f(x+h) - f(x)$ is the change in output. The denominator $h$ is the change in input. Their ratio is the slope of the **secant line** connecting the two points $(x,\ f(x))$ and $(x+h,\ f(x+h))$ on the graph. This means the derivative starts with an ordinary average rate of change over an interval you can still see and measure.

Now let $h \to 0$. The second point slides toward the first. The secant line tilts and, in the limit, becomes the **tangent line** at $x$. The slope of that tangent line is the derivative:

$$\boxed{f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}}$$

This is the formal definition. Every differentiation rule derived later is a consequence of it. The definition itself is the limit you saw in chapter 1, applied to a specific quotient. This means the derivative is not separate from limits. It is the limit of average change as the interval collapses to a point.

## 27.2. What the notation is saying

Two notations for the derivative are standard. You will need both.

**Prime notation** (Newton): $f'(x)$, read *f prime of x*. Clean and fast to write. Used when there is no ambiguity about which variable you're differentiating with respect to.

**Leibniz notation**: $\frac{dy}{dx}$, read *dy by dx*. More verbose but carries structural information — it names both the output variable ($y$) and the input variable ($x$). When a problem involves several variables, or when you need to keep track of units, Leibniz notation makes the algebra clearer. Most science and engineering texts default to it.

Both mean the same thing. If $y = f(x)$, then:

$$f'(x) = \frac{dy}{dx} = \frac{d}{dx}\big[f(x)\big]$$

The expression $\frac{d}{dx}[\,\cdot\,]$ is an **operator** — it takes a function as input and produces its derivative as output. Think of $d/dx$ as an instruction: *differentiate with respect to x*. This means all three notations describe the same rate-of-change object, but each notation highlights a different aspect of it.

```
//| echo: false
viewof x2_secant = Inputs.range([1.01, 3.0], {
  step: 0.01,
  value: 3.0,
  label: "x₂ (drag toward 1)"
})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const width = 560, height = 320;
  const margin = { top: 24, right: 32, bottom: 44, left: 52 };
  const innerW = width - margin.left - margin.right;
  const innerH = height - margin.top - margin.bottom;

  const f = x => x * x + 1;
  const x1 = 1, y1 = f(x1);
  const x2 = x2_secant, y2 = f(x2);
  const slope = (y2 - y1) / (x2 - x1);
  const nearTangent = x2 < 1.05;
```

```
const xScale = d3.scaleLinear().domain([-0.5, 3]).range([0, innerW]);
const yScale = d3.scaleLinear().domain([0, 11]).range([innerH, 0]);

const svg = d3.create("svg")
  .attr("width", width)
  .attr("height", height)
  .attr("viewBox", `0 0 ${width} ${height}`)
  .style("font-family", "inherit");

const g = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// Axes
g.append("g")
  .attr("transform", `translate(0,${innerH})`)
  .call(d3.axisBottom(xScale).ticks(7))
  .call(ax => ax.select(".domain").attr("stroke", "#9ca3af"))
  .call(ax => ax.selectAll("line").attr("stroke", "#9ca3af"))
  .call(ax => ax.selectAll("text").attr("fill", "#6b7280").attr("font-size", "11"));

g.append("g")
  .call(d3.axisLeft(yScale).ticks(6))
  .call(ax => ax.select(".domain").attr("stroke", "#9ca3af"))
  .call(ax => ax.selectAll("line").attr("stroke", "#9ca3af"))
  .call(ax => ax.selectAll("text").attr("fill", "#6b7280").attr("font-size", "11"));

// Axis labels
g.append("text").attr("x", innerW / 2).attr("y", innerH + 38)
  .attr("text-anchor", "middle").attr("fill", "#6b7280").attr("font-size", "12").text("x");
g.append("text").attr("transform", "rotate(-90)").attr("x", -innerH / 2).attr("y", -38)
  .attr("text-anchor", "middle").attr("fill", "#6b7280").attr("font-size", "12").text("y");

// Grid lines
g.append("g").attr("class", "grid")
  .call(d3.axisLeft(yScale).ticks(6).tickSize(-innerW).tickFormat(""))
  .call(ax => ax.select(".domain").remove())
  .call(ax => ax.selectAll("line").attr("stroke", "#f3f4f6").attr("stroke-dasharray", "3,3"));

// Curve f(x) = x² + 1
const lineGen = d3.line()
  .x(d => xScale(d))
  .y(d => yScale(f(d)))
  .curve(d3.curveBasis);

const xs = d3.range(-0.5, 3.02, 0.02);
g.append("path")
  .datum(xs)
```

```
  .attr("fill", "none")
  .attr("stroke", "#2563eb")
  .attr("stroke-width", 2.5)
  .attr("d", lineGen);

// Curve label
g.append("text")
  .attr("x", xScale(2.7))
  .attr("y", yScale(f(2.7)) - 10)
  .attr("fill", "#2563eb")
  .attr("font-size", "12")
  .text("f(x) = x² + 1");

// Draw line (secant or tangent) extended across the visible range
const drawLine = (slope, passX, passY, color, dash) => {
  const xLeft = -0.5, xRight = 3;
  const yLeft = passY + slope * (xLeft - passX);
  const yRight = passY + slope * (xRight - passX);
  g.append("line")
    .attr("x1", xScale(xLeft)).attr("y1", yScale(yLeft))
    .attr("x2", xScale(xRight)).attr("y2", yScale(yRight))
    .attr("stroke", color)
    .attr("stroke-width", nearTangent ? 2 : 1.5)
    .attr("stroke-dasharray", dash || "none")
    .attr("opacity", 0.85);
};

if (nearTangent) {
  // Show tangent line
  drawLine(2, x1, y1, "#10b981", null);
  g.append("text")
    .attr("x", xScale(2.2))
    .attr("y", yScale(2 * (2.2 - 1) + y1) - 10)
    .attr("fill", "#10b981")
    .attr("font-size", "12")
    .attr("font-weight", "600")
    .text("tangent slope → 2");
} else {
  // Show secant line
  drawLine(slope, x1, y1, "#ef4444", null);
}

// P₁ dot (fixed)
g.append("circle")
  .attr("cx", xScale(x1)).attr("cy", yScale(y1))
  .attr("r", 5).attr("fill", "#2563eb").attr("stroke", "#fff").attr("stroke-width", 1.5);
g.append("text")
```

```
    .attr("x", xScale(x1) - 14).attr("y", yScale(y1) - 9)
    .attr("fill", "#2563eb").attr("font-size", "12").attr("font-weight", "600")
    .text("P₁");

  // P₂ dot (moving)
  if (!nearTangent) {
    g.append("circle")
      .attr("cx", xScale(x2)).attr("cy", yScale(y2))
      .attr("r", 5).attr("fill", "#ef4444").attr("stroke", "#fff").attr("stroke-width", 1.5);
    g.append("text")
      .attr("x", xScale(x2) + 7).attr("y", yScale(y2) - 6)
      .attr("fill", "#ef4444").attr("font-size", "12").attr("font-weight", "600")
      .text("P₂");
  }

  // Slope readout
  const slopeText = nearTangent
    ? "secant slope → 2.000  (tangent)"
    : `secant slope = ${slope.toFixed(4)}`;
  const slopeColor = nearTangent ? "#10b981" : "#ef4444";

  g.append("text")
    .attr("x", innerW - 4).attr("y", 14)
    .attr("text-anchor", "end")
    .attr("fill", slopeColor)
    .attr("font-size", "13")
    .attr("font-weight", "600")
    .text(slopeText);

  return svg.node();
}
```

**The tangent line**

The derivative $f'(a)$ is the slope of the **tangent line** to the graph of $f$ at the point $(a, f(a))$ — the straight line through $(a, f(a))$ with slope $f'(a)$, which is the best linear approximation to $f$ near $x = a$.

Unlike the informal picture, a tangent line can cross the curve — $f(x) = x^3$ at the origin has a tangent line that crosses. What defines a tangent is its slope, not whether it crosses.

This is why a function fails to be differentiable at a corner or cusp: no unique tangent line exists there.

## 27.3. The method — differentiation rules

Computing every derivative from the limit definition would be exhausting. Fortunately, the limit has been worked out once for each class of function, packaged into rules, and those rules can be applied

mechanically. The skill in differentiation is knowing which rule to reach for.

### 27.3.1. Power rule

For any function $f(x) = x^n$, where $n$ is a real number:

$$\frac{d}{dx}\left[x^n\right] = nx^{n-1}$$

This is the most-used rule in calculus. It applies to positive integers, negative integers, fractions — any real exponent. This means the power rule is the default tool for algebraic functions because it turns a hard limit into a quick pattern.

**Derivation for positive integer $n$.** Start from the limit definition:

$$f'(x) = \lim_{h\to 0} \frac{(x+h)^n - x^n}{h}$$

To make progress, we need to expand $(x+h)^n$. For small $n$ you can do this by multiplying out, but the pattern for general $n$ comes from the binomial theorem — covered in Vol 3 polynomials. For $n = 2$: $(x+h)^2 = x^2 + 2xh + h^2$. For $n = 3$: $(x+h)^3 = x^3 + 3x^2h + 3xh^2 + h^3$. In general:

$$(x+h)^n = x^n + nx^{n-1}h + \frac{n(n-1)}{2}x^{n-2}h^2 + \cdots + h^n$$

Subtract $x^n$ and divide by $h$:

$$\frac{(x+h)^n - x^n}{h} = nx^{n-1} + \frac{n(n-1)}{2}x^{n-2}h + \cdots + h^{n-1}$$

Every term except the first contains at least one factor of $h$. As $h \to 0$, all those terms vanish:

$$f'(x) = \lim_{h\to 0}\left(nx^{n-1} + \text{terms with } h\right) = nx^{n-1}$$

The rule is proved. Notice what happened: the limit killed every term except the leading one. This is why the power rule is so clean. This means the derivative keeps the first-order change and discards the higher-order terms that become negligible as $h$ shrinks.

**Examples:**

| $f(x)$ | $f'(x)$ |
|---|---|
| $x^5$ | $5x^4$ |
| $x^{-2}$ | $-2x^{-3}$ |
| $\sqrt{x} = x^{1/2}$ | $\frac{1}{2}x^{-1/2}$ |
| $1 = x^0$ | $0$ |

The last row says: the derivative of any constant is zero. A constant function has zero slope everywhere — that makes sense.

### 27.3.2. Sum/difference and constant multiple rules

These follow directly from the limit definition: because the limit of a sum is the sum of the limits, and a constant factor passes through a limit.

**Constant multiple rule:**

$$\frac{d}{dx}\big[c \cdot f(x)\big] = c \cdot f'(x)$$

**Sum/difference rule:**

$$\frac{d}{dx}\big[f(x) \pm g(x)\big] = f'(x) \pm g'(x)$$

Together these mean you can differentiate a polynomial term by term. For $p(x) = 3x^4 - 2x^2 + 5x - 1$:

$$p'(x) = 3 \cdot 4x^3 - 2 \cdot 2x + 5 \cdot 1 - 0 = 12x^3 - 4x + 5$$

This means linearity lets you break a large derivative into smaller pieces and differentiate each piece separately.

### 27.3.3. Product rule

The derivative of a product is *not* the product of the derivatives. That failure is easy to verify: $\frac{d}{dx}[x \cdot x] = \frac{d}{dx}[x^2] = 2x$, but $\frac{d}{dx}[x] \times \frac{d}{dx}[x] = 1 \times 1 = 1$. Different answers.

The correct rule: if $f$ and $g$ are both differentiable, then:

$$\frac{d}{dx}\big[f(x) \cdot g(x)\big] = f'(x) \cdot g(x) + f(x) \cdot g'(x)$$

A useful mnemonic: *derivative of first times second, plus first times derivative of second.* The rule accounts for how both factors are changing simultaneously. This means a product changes for two reasons at once: the first factor changes and the second factor changes.

**Check:** $\frac{d}{dx}[x \cdot x] = 1 \cdot x + x \cdot 1 = 2x$. Correct.

### 27.3.4. Chain rule

The chain rule handles **composite functions** — functions of functions. If $y = f(u)$ and $u = g(x)$, so that $y = f(g(x))$, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

In prime notation: $\big(f \circ g\big)'(x) = f'\big(g(x)\big) \cdot g'(x)$.

This is the most important and most commonly misapplied rule in calculus. The key skill is **identifying the outer and inner functions** before differentiating. This means the chain rule measures how a change in $x$ affects the inner function, then how that inner change affects the outer one.

**Example.** Differentiate $y = \sin(x^2)$.

- Outer function: $f(u) = \sin(u)$, so $\dfrac{dy}{du} = \cos(u)$.
- Inner function: $g(x) = x^2$, so $\dfrac{du}{dx} = 2x$.
- Chain rule: $\dfrac{dy}{dx} = \cos(x^2) \cdot 2x = 2x\cos(x^2)$.

The notation $\dfrac{dy}{dx} = \dfrac{dy}{du} \cdot \dfrac{du}{dx}$ looks like the $du$'s cancel — and you can use it as if they do. The reason it works is multiplicative structure in the limit, not literal fraction cancellation, but the practical result is the same: set up the product and compute. This means composite functions must be differentiated layer by layer.

```
//| echo: false
viewof chainFunc = Inputs.select(
  ["sin(x²)", "e^(2x)", "(3x+1)⁴"],
  { label: "Composite function", value: "sin(x²)" }
)
```

```
//| echo: false
viewof chainX = Inputs.range([-2, 2], {
  step: 0.01,
  value: 0,
  label: "x"
})
```

```
//| echo: false
{
  const katexLib = await require("katex@0.16");

  const defs = {
    "sin(x²)": {
      fgx:    "\\sin(x^2)",
      gx:     "x^2",
      fu:     "\\sin(u)",
      fprimeGx: "\\cos(x^2)",
      gprime: "2x",
      product: "\\cos(x^2) \\cdot 2x",
      numericFn: x => Math.cos(x * x) * 2 * x,
    },
    "e^(2x)": {
      fgx:    "e^{2x}",
      gx:     "2x",
      fu:     "e^{u}",
      fprimeGx: "e^{2x}",
      gprime: "2",
      product: "e^{2x} \\cdot 2",
      numericFn: x => Math.exp(2 * x) * 2,
    },
```

```
  "(3x+1)⁴": {
    fgx:    "(3x+1)^4",
    gx:     "3x+1",
    fu:     "u^4",
    fprimeGx: "4(3x+1)^3",
    gprime: "3",
    product: "12(3x+1)^3",
    numericFn: x => 12 * Math.pow(3 * x + 1, 3),
  },
};

const d = defs[chainFunc];
const numVal = d.numericFn(chainX);
const numDisplay = isFinite(numVal) ? numVal.toFixed(4) : "undefined";

const render = (tex, display = false) =>
  katexLib.renderToString(tex, { throwOnError: false, displayMode: display });

const box = document.createElement("div");
box.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; margin:

const row = (label, tex) => {
  const r = document.createElement("div");
  r.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.3rem 0.75rem; align-i
  const lEl = document.createElement("span");
  lEl.style.cssText = "font-size:0.8rem; color:#6b7280; font-style:italic;";
  lEl.textContent = label;
  const rEl = document.createElement("span");
  rEl.innerHTML = render(tex);
  r.appendChild(lEl);
  r.appendChild(rEl);
  return r;
};

const heading = document.createElement("p");
heading.style.cssText = "margin:0 0 0.6rem 0; font-size:0.9rem; color:#374151; font-weight:600;
heading.innerHTML = "Chain rule: " + render(`\\dfrac{d}{dx}[f(g(x))] = f'(g(x)) \\cdot g'(x)`);
box.appendChild(heading);

box.appendChild(row("function f(g(x))", d.fgx));
box.appendChild(row("inner: g(x)", d.gx));
box.appendChild(row("outer: f(u)", d.fu));
box.appendChild(row("f′(g(x))", d.fprimeGx));
box.appendChild(row("g′(x)", d.gprime));
box.appendChild(row("product f′(g(x))·g′(x)", d.product));

const divider = document.createElement("div");
```

```
  divider.style.cssText = "margin:0.6rem 0 0.4rem 0; border-top:1px solid #e5e7eb;";
  box.appendChild(divider);

  const valRow = document.createElement("div");
  valRow.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.3rem 0.75rem; align-
  const valLabel = document.createElement("span");
  valLabel.style.cssText = "font-size:0.8rem; color:#6b7280; font-style:italic;";
  valLabel.textContent = `numeric value at x = ${chainX.toFixed(2)}`;
  const valNum = document.createElement("span");
  valNum.style.cssText = "font-weight:600; color:#2563eb; font-size:0.95rem;";
  valNum.textContent = numDisplay;
  valRow.appendChild(valLabel);
  valRow.appendChild(valNum);
  box.appendChild(valRow);

  return box;
}
```

### 27.3.5. Derivatives of key functions

These are stated here without proof. Each can be derived from the limit definition, but the derivations involve limits of trigonometric quotients and the definition of $e$ — both of which are established results.

| Function | Derivative | Read the derivative as |
|---|---|---|
| $\sin x$ | $\cos x$ | Cosine is the rate of change of sine — maximum slope at $x = 0$, zero slope at the peaks |
| $\cos x$ | $-\sin x$ | Sine with a sign flip — sine is decreasing where cosine reaches its peak |
| $e^x$ | $e^x$ | The exponential is its own derivative — the rate of growth equals the current value |
| $\ln x$ | $\frac{1}{x}$ | Valid for $x > 0$ — the domain of $\ln x$. For $x < 0$, use $\ln\|x\|$, whose derivative is also $\frac{1}{x}$ |

The self-referential property of $e^x$ — that it equals its own derivative — is why $e$ appears everywhere in physics and engineering. Any system whose rate of change is proportional to its current value is governed by an exponential, and the differential equation describing it is $\frac{dy}{dx} = y$. This means the table of key derivatives is really a table of standard change behaviours that recur in many different contexts.

## 27.4. Higher derivatives

The derivative of $f$ is itself a function. You can differentiate it again. The **second derivative** $f''(x)$ (or $\frac{d^2y}{dx^2}$) is the rate of change of the rate of change. This means the first derivative tells you how steep the graph is, and the second derivative tells you how that steepness is itself changing.

The clearest physical example: if $s(t)$ is position at time $t$, then $s'(t)$ is velocity and $s''(t)$ is acceleration. Acceleration is how quickly velocity is changing.

The second derivative also carries geometric information:

- $f''(x) > 0$: the curve is **concave up** at $x$ — the slope is increasing, the curve bends upward like a bowl.
- $f''(x) < 0$: the curve is **concave down** at $x$ — the slope is decreasing, the curve bends downward like an arch.

This is what the second derivative test uses.

## 27.5. Applications: optimisation

One of the most immediate uses of calculus is finding where a function reaches its maximum or minimum value.

**Critical points.** If $f$ is differentiable and has a local maximum or minimum at $x = c$, then the tangent line at $c$ must be horizontal — a tilted tangent would mean the function is still rising or falling, so $c$ couldn't be an extreme point. Therefore:

$$f'(c) = 0 \implies c \text{ is a critical point}$$

Not every critical point is a maximum or minimum — it could be a **saddle point** (like $x = 0$ for $f(x) = x^3$, where the function is neither increasing nor decreasing at that instant but continues in the same direction on both sides). The second derivative distinguishes the cases. This means critical points are candidates for extrema, not guarantees.

**Second derivative test.** If $f'(c) = 0$:

- If $f''(c) > 0$: concave up at $c$, so the tangent is at the bottom of a bowl — **local minimum**.
- If $f''(c) < 0$: concave down at $c$, so the tangent is at the top of an arch — **local maximum**.
- If $f''(c) = 0$: the test is inconclusive. What to do when $f''(c) = 0$ is deferred to multivariable calculus — the tools for that case are richer there. In single-variable calculus, if the second derivative test is inconclusive, you check the sign of $f'$ on either side of $c$ directly. This means the second derivative test reads the local shape of the graph to classify what the first derivative found.

**Worked procedure.** Find the local extrema of $f(x) = x^3 - 6x^2 + 9x + 1$.

Step 1. Differentiate: $f'(x) = 3x^2 - 12x + 9$.

Step 2. Set $f'(x) = 0$: $3x^2 - 12x + 9 = 0 \implies x^2 - 4x + 3 = 0 \implies (x-1)(x-3) = 0$.

Critical points: $x = 1$ and $x = 3$.

Step 3. Compute $f''(x) = 6x - 12$.

Step 4. Test each critical point:

- $f''(1) = 6(1) - 12 = -6 < 0$: local maximum at $x = 1$. Value: $f(1) = 1 - 6 + 9 + 1 = 5$.
- $f''(3) = 6(3) - 12 = 6 > 0$: local minimum at $x = 3$. Value: $f(3) = 27 - 54 + 27 + 1 = 1$.

The function rises to a local maximum of 5 at $x = 1$, falls to a local minimum of 1 at $x = 3$, then rises again.

## 27.6. Worked examples

---

**Example 1 (Computing/data).** Differentiate $f(x) = 3x^4 - 2x^2 + 5x - 1$.

Apply the power rule term by term, with the constant multiple and sum rules:

$$f'(x) = 3 \cdot 4x^3 - 2 \cdot 2x^1 + 5 \cdot 1 - 0$$

$$f'(x) = 12x^3 - 4x + 5$$

This is a polynomial — differentiating a polynomial always produces a polynomial of degree one lower. The constant term $-1$ vanishes: a constant has zero slope everywhere. This means polynomial derivatives keep the same algebraic structure but become one degree less steep.

---

**Example 2 (Science).** Differentiate $f(x) = x^2 \sin(x)$.

This is a product. Identify $f(x) = x^2$ (the first factor) and $g(x) = \sin(x)$ (the second). Apply the product rule:

$$\frac{d}{dx}\left[x^2 \sin x\right] = 2x \cdot \sin x + x^2 \cdot \cos x$$

$$= 2x \sin x + x^2 \cos x$$

This kind of function appears in wave mechanics: $x^2 \sin x$ models an oscillation whose amplitude grows quadratically with distance from the source. This means the product rule keeps track of both the oscillation and the changing amplitude.

---

**Example 3 (Engineering).** Differentiate $f(x) = e^{3x^2}$.

This is a composite function. Identify the outer and inner functions:

- Outer: $f(u) = e^u$, which has derivative $e^u$.
- Inner: $g(x) = 3x^2$, which has derivative $6x$.

Chain rule:

$$\frac{d}{dx}\left[e^{3x^2}\right] = e^{3x^2} \cdot 6x = 6xe^{3x^2}$$

Functions of this form appear in Gaussian distributions, heat diffusion kernels, and signal-processing windows. The chain rule is the only way to differentiate them correctly. This means the outer exponential survives, while the inner function contributes an extra scaling factor.

---

**Example 4 (Real-world optimisation).** A farmer has 80 m of fencing and wants to enclose a rectangular paddock. One side is a riverbank, which needs no fence. What dimensions maximise the enclosed area?

**Set up.** Let the side perpendicular to the river have length $x$ (metres). Two of these sides are needed. The side parallel to the river, call it $y$, uses the remaining fencing:

$$2x + y = 80 \implies y = 80 - 2x$$

The area is:

$$A(x) = x \cdot y = x(80 - 2x) = 80x - 2x^2$$

**Differentiate and find critical points:**

$$A'(x) = 80 - 4x$$

Setting $A'(x) = 0$: $80 - 4x = 0 \implies x = 20$.

**Classify:** $A''(x) = -4 < 0$ everywhere, so $x = 20$ is a local (and global) maximum.

**Dimensions:** $x = 20$ m, $y = 80 - 2(20) = 40$ m.

**Maximum area:** $A = 20 \times 40 = 800$ m$^2$.

The shape that maximises the area is twice as wide as it is deep — a result that appears in architectural and agricultural optimisation problems wherever one boundary is free. This means optimisation turns a practical design question into a derivative question about where increase changes to decrease.

## 27.7. Where this goes

This chapter moved from limits to a full toolkit for describing change. The next chapter asks the inverse question: once a rate is known, how do we recover the total change it produces?

**Integral calculus (ch03):** Integration inverts differentiation. Where this chapter asked "given a function, find its rate of change," the next chapter asks "given the rate of change, recover the original function." The Fundamental Theorem of Calculus — the deepest result in the subject — shows these two operations are inverses of each other. Every calculation of area, volume, work, and accumulated change runs through it.

**Ordinary differential equations (Vol 7):** A differential equation is a relationship between a function and its own derivative — an equation that says something like "the rate of change of this quantity is proportional to the quantity itself." Everything in this chapter is prerequisite: you need to know what a derivative is, how to compute one, and how the chain rule works, before a differential equation makes sense. The entire Vol 7 ODE sequence begins here.

**Where this shows up**

- **Velocity and acceleration.** If $s(t)$ is the position of an object at time $t$, then $s'(t)$ is its velocity and $s''(t)$ is its acceleration. Every problem in Newtonian mechanics is stated in these terms.
- **Marginal analysis.** In economics, marginal cost is the derivative of total cost with respect to quantity. Profit is maximised where marginal revenue equals marginal cost — a critical point of the profit function.
- **Newton's law of cooling.** The rate of heat loss of an object is proportional to the difference between the object's temperature and the ambient temperature. That "rate of heat loss" is a derivative. The law is $\frac{dT}{dt} = -k(T - T_{\text{ambient}})$ — a differential equation that lives in Vol 7.
- **Gradient descent.** The optimisation algorithm that trains neural networks works by computing the derivative of the loss function with respect to each model parameter, then updating the parameters in the direction of steepest descent. The chain rule — applied thousands of times through a deep network — is called backpropagation.

## 27.8. Exercises

These are puzzles. Each has a clean answer, but the interesting part is choosing the right rule and executing the steps carefully before you reach for the result.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
```

```
const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
const stepsDiv = document.createElement("div");
stepsDiv.style.marginBottom = "0.75rem";
function renderSteps() {
  stepsDiv.innerHTML = "";
  for (let i = 0; i < current; i++) {
    const s = steps[i];
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align
    const opEl = document.createElement("span");
    opEl.style.cssText = "font-size:0.8rem; color:#6b7280; font-style:italic;";
    opEl.textContent = s.op;
    const eqEl = document.createElement("span");
    eqEl.innerHTML = katex.renderToString(s.eq, {throwOnError: false, displayMode: false});
    row.appendChild(opEl);
    row.appendChild(eqEl);
    stepsDiv.appendChild(row);
    if (s.note) {
      const noteEl = document.createElement("p");
      noteEl.style.cssText = "margin:0.2rem 0 0.5rem 220px; font-size:0.82rem; color:#374151;";
      noteEl.textContent = s.note;
      stepsDiv.appendChild(noteEl);
    }
  }
  if (current === totalSteps) {
    const done = document.createElement("p");
    done.style.cssText = "margin-top:0.5rem; font-size:0.85rem; color:#059669; font-weight:600;
    done.textContent = "□ Solution complete";
    stepsDiv.appendChild(done);
  }
}
const controls = document.createElement("div");
controls.style.cssText = "display:flex; align-items:center; gap:0.75rem; margin-top:0.5rem;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.35rem 0.85rem; background:#2563eb; color:#fff; border:none;
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.35rem 0.85rem; background:#f3f4f6; color:#374151; border:1p
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.8rem; color:#6b7280;";
nextBtn.onclick = () => {
  if (current < totalSteps) {
    current++;
    renderSteps();
    counter.textContent = current + " / " + totalSteps;
```

```
      if (current === totalSteps) nextBtn.disabled = true;
    }
  };
  resetBtn.onclick = () => {
    current = 0;
    nextBtn.disabled = false;
    renderSteps();
    counter.textContent = "0 / " + totalSteps;
  };
  counter.textContent = "0 / " + totalSteps;
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  return container;
}
```

**Exercise 1.** Differentiate $f(x) = 4x^5 - 3x^3 + 7x - 2$ using the power rule.

```
//| echo: false
{
  const el = makeStepperHTML(1, [
    { op: "Identify each term", eq: "4x^5,\\quad -3x^3,\\quad 7x,\\quad -2", note: "Differentiate te
    { op: "Power rule on 4x⁵", eq: "\\frac{d}{dx}[4x^5] = 4 \\cdot 5x^4 = 20x^4", note: null },
    { op: "Power rule on −3x³", eq: "\\frac{d}{dx}[-3x^3] = -3 \\cdot 3x^2 = -9x^2", note: null },
    { op: "Power rule on 7x", eq: "\\frac{d}{dx}[7x] = 7 \\cdot 1 = 7", note: "x = x¹, so the power
    { op: "Derivative of constant", eq: "\\frac{d}{dx}[-2] = 0", note: "A constant has zero slope ev
    { op: "Combine", eq: "f'(x) = 20x^4 - 9x^2 + 7", note: null },
  ]);
  return el;
}
```

---

**Exercise 2.** Differentiate $f(x) = x^3 \cos(x)$ using the product rule.

```
//| echo: false
{
  const el = makeStepperHTML(2, [
    { op: "Identify the two factors", eq: "u = x^3,\\quad v = \\cos x", note: null },
    { op: "Differentiate each factor", eq: "u' = 3x^2,\\quad v' = -\\sin x", note: null },
    { op: "Apply product rule", eq: "\\frac{d}{dx}[x^3 \\cos x] = u'v + uv'", note: "Derivative of f
    { op: "Substitute", eq: "= 3x^2 \\cos x + x^3 \\cdot (-\\sin x)", note: null },
    { op: "Write the result", eq: "f'(x) = 3x^2 \\cos x - x^3 \\sin x", note: null },
  ]);
```

```
  return el;
}
```

---

**Exercise 3.** Differentiate $f(x) = (2x^3 + 1)^5$ using the chain rule.

```
//| echo: false
{
  const el = makeStepperHTML(3, [
    { op: "Identify outer and inner functions", eq: "f(u) = u^5,\\quad g(x) = 2x^3 + 1", note: "T
    { op: "Differentiate the outer", eq: "\\frac{d}{du}[u^5] = 5u^4", note: null },
    { op: "Differentiate the inner", eq: "\\frac{d}{dx}[2x^3 + 1] = 6x^2", note: null },
    { op: "Apply chain rule", eq: "f'(x) = 5u^4 \\cdot 6x^2", note: "Multiply: derivative of oute
    { op: "Substitute u = 2x³ + 1", eq: "f'(x) = 5(2x^3+1)^4 \\cdot 6x^2", note: null },
    { op: "Simplify", eq: "f'(x) = 30x^2(2x^3+1)^4", note: null },
  ]);
  return el;
}
```

---

**Exercise 4.** Find all critical points of $f(x) = x^3 - 3x^2 - 9x + 2$ and classify each as a local maximum, local minimum, or neither.

```
//| echo: false
{
  const el = makeStepperHTML(4, [
    { op: "Differentiate f", eq: "f'(x) = 3x^2 - 6x - 9", note: null },
    { op: "Set f'(x) = 0", eq: "3x^2 - 6x - 9 = 0 \\implies x^2 - 2x - 3 = 0", note: "Divide thro
    { op: "Factorise", eq: "(x-3)(x+1) = 0 \\implies x = 3 \\text{ or } x = -1", note: null },
    { op: "Compute second derivative", eq: "f''(x) = 6x - 6", note: null },
    { op: "Test x = −1", eq: "f''(-1) = 6(-1) - 6 = -12 < 0", note: "Concave down → local maximum
    { op: "Value at x = −1", eq: "f(-1) = (-1)^3 - 3(-1)^2 - 9(-1) + 2 = -1 - 3 + 9 + 2 = 7", not
    { op: "Test x = 3", eq: "f''(3) = 6(3) - 6 = 12 > 0", note: "Concave up → local minimum." },
    { op: "Value at x = 3", eq: "f(3) = 27 - 27 - 27 + 2 = -25", note: "Local minimum of −25 at x
  ]);
  return el;
}
```

---

**Exercise 5.** A manufacturer's total cost in dollars for producing $q$ units per day is $C(q) = q^3 - 6q^2 + 15q + 50$. At what production level is the marginal cost minimised? (Marginal cost is $C'(q)$.)

```
//| echo: false
{
  const el = makeStepperHTML(5, [
    { op: "Find marginal cost C'(q)", eq: "C'(q) = 3q^2 - 12q + 15", note: "Marginal cost is the rat
    { op: "Minimise C'(q): differentiate again", eq: "C''(q) = 6q - 12", note: "To find the minimum
    { op: "Set C''(q) = 0", eq: "6q - 12 = 0 \\implies q = 2", note: null },
    { op: "Check sign of C'(q) on each side", eq: "C'(1) = 3 - 12 + 15 = 6,\\quad C'(3) = 27 - 36 +
    { op: "Confirm minimum: check C''(q) sign", eq: "C''(q) = 6q - 12 < 0 \\text{ for } q < 2,\\quad
    { op: "Minimum marginal cost value", eq: "C'(2) = 3(4) - 12(2) + 15 = 12 - 24 + 15 = 3", note: n
    { op: "Interpret", eq: "q = 2 \\text{ units/day gives minimum marginal cost of } \\$3 \\text{ pe
  ]);
  return el;
}
```

---

**Exercise 6.** A stone is dropped from a bridge. Its height above the water at time $t$ seconds is $h(t) = 45 - 4.9t^2$ metres. At what speed (in m/s) is it falling at the instant it hits the water?

```
//| echo: false
{
  const el = makeStepperHTML(6, [
    { op: "Find the time of impact", eq: "h(t) = 0 \\implies 45 - 4.9t^2 = 0", note: "The stone hits
    { op: "Solve for t", eq: "t^2 = \\frac{45}{4.9} \\approx 9.184 \\implies t \\approx 3.031 \\text
    { op: "Find the velocity function", eq: "h'(t) = -9.8t", note: "Velocity is the derivative of he
    { op: "Speed at impact", eq: "h'(3.031) = -9.8 \\times 3.031 \\approx -29.7 \\text{ m/s}", note
    { op: "Interpret", eq: "\\text{Speed} = |h'(3.031)| \\approx 29.7 \\text{ m/s}", note: "Speed is
  ]);
  return el;
}
```

# 28. Integral calculus

Accumulation

Your car's speedometer shows 80 km/h. Two minutes later it shows 95 km/h. How far have you travelled? You cannot just multiply speed by time — the speed was changing the whole way. What you need is a way to add up all those instantaneous velocities over the interval. That is integration.

A water tank fills at a rate that varies with time — fast at first, slower as back-pressure builds. You want to know how much water is in the tank after an hour. You know the rate at each moment. You need the total.

These are the same problem. This chapter builds the tool.

## 28.1. What the notation is saying

The integral sign $\int$ is an elongated S — it stands for *sum*. The $dx$ at the end names the variable we are integrating with respect to. The expression

$$\int_a^b f(x)\,dx$$

reads: *"the integral of $f(x)$ with respect to $x$, from $a$ to $b$."* Picture many thin rectangles, each with height $f(x)$ and width $\Delta x$ — we'll shrink $\Delta x$ to zero in the limit, so the strips get thinner and thinner until they fit exactly under the curve. The integral adds up the areas of all of them. The result is the total area between the curve $y = f(x)$ and the $x$-axis, between $x = a$ and $x = b$. This means a definite integral is a limit of many small contributions added together.

That is the geometric picture. The computational meaning follows from Riemann sums.

### 28.1.1. Riemann sums

Partition the interval $[a, b]$ into $n$ equal subintervals, each of width

$$\Delta x = \frac{b-a}{n}$$

Pick a sample point $x_i$ in each subinterval — say the left endpoint, the right endpoint, or the midpoint. At each sample point, the rectangle has height $f(x_i)$ and width $\Delta x$. Sum all $n$ rectangles:

$$\text{Area} \approx \sum_{i=1}^{n} f(x_i)\,\Delta x$$

This is a Riemann sum — you saw the idea of summing up many small terms in Volume 4, and the same question of what happens as the number of terms grows without bound applies here too. The definite integral is the limit of this sum as the rectangles get infinitely thin: This means the approximation becomes exact by refining the partition until the rectangles capture the curve arbitrarily well.

$$\int_a^b f(x)\,dx = \lim_{n\to\infty} \sum_{i=1}^{n} f(x_i)\,\Delta x$$

For well-behaved functions, this limit exists and is a single, precise number. This means integration is where the ideas of area, summation, and limits come together.

**Interactive: Riemann sum builder.** Adjust the number of rectangles and choose a sum type to see how the approximation converges to the exact value.

```
//| echo: false
d3 = require("d3@7")
```

```
//| echo: false
viewof riemannN = Inputs.range([1, 50], {
  label: "n (number of rectangles)",
  step: 1,
  value: 5
})
```

```
//| echo: false
viewof riemannType = Inputs.select(["Left", "Right", "Midpoint"], {
  label: "Sum type",
  value: "Left"
})
```

```
//| echo: false
{
  const W = 560, H = 320;
  const margin = { top: 20, right: 20, bottom: 50, left: 50 };
  const innerW = W - margin.left - margin.right;
  const innerH = H - margin.top - margin.bottom;

  const a = 0, b = 3;
  const f = x => x * x + 1;
  const exactArea = 12; // ∫₀³ (x²+1) dx = [x³/3 + x]₀³ = 9 + 3 = 12

  const n = Math.max(1, Math.round(riemannN));
  const type = riemannType;
```

```
const dx = (b - a) / n;

// Compute approximate area
let approxArea = 0;
for (let i = 0; i < n; i++) {
  let xSample;
  if (type === "Left")     xSample = a + i * dx;
  else if (type === "Right") xSample = a + (i + 1) * dx;
  else                     xSample = a + (i + 0.5) * dx; // Midpoint
  approxArea += f(xSample) * dx;
}

const xScale = d3.scaleLinear().domain([a, b]).range([0, innerW]);
const yScale = d3.scaleLinear().domain([0, 11]).range([innerH, 0]);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .attr("style", `max-width:${W}px; font-family:inherit;`);

const g = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// Draw rectangles
for (let i = 0; i < n; i++) {
  let xSample;
  if (type === "Left")       xSample = a + i * dx;
  else if (type === "Right") xSample = a + (i + 1) * dx;
  else                       xSample = a + (i + 0.5) * dx;

  const rectX = xScale(a + i * dx);
  const rectW = xScale(a + (i + 1) * dx) - rectX;
  const rectH = innerH - yScale(f(xSample));

  g.append("rect")
    .attr("x", rectX)
    .attr("y", yScale(f(xSample)))
    .attr("width", rectW)
    .attr("height", rectH)
    .attr("fill", "#bfdbfe")
    .attr("stroke", "#2563eb")
    .attr("stroke-width", 0.8)
    .attr("opacity", 0.7);
}

// Draw x-axis
g.append("line")
```

```
    .attr("x1", 0).attr("y1", innerH)
    .attr("x2", innerW).attr("y2", innerH)
    .attr("stroke", "#374151").attr("stroke-width", 1.5);

  // Draw y-axis
  g.append("line")
    .attr("x1", 0).attr("y1", 0)
    .attr("x2", 0).attr("y2", innerH)
    .attr("stroke", "#374151").attr("stroke-width", 1.5);

  // X-axis ticks
  [0, 1, 2, 3].forEach(v => {
    g.append("line")
      .attr("x1", xScale(v)).attr("y1", innerH)
      .attr("x2", xScale(v)).attr("y2", innerH + 5)
      .attr("stroke", "#374151");
    g.append("text")
      .attr("x", xScale(v)).attr("y", innerH + 18)
      .attr("text-anchor", "middle")
      .attr("fill", "#374151").attr("font-size", "12px")
      .text(v);
  });

  // Y-axis ticks
  [0, 2, 4, 6, 8, 10].forEach(v => {
    g.append("line")
      .attr("x1", -5).attr("y1", yScale(v))
      .attr("x2", 0).attr("y2", yScale(v))
      .attr("stroke", "#374151");
    g.append("text")
      .attr("x", -10).attr("y", yScale(v) + 4)
      .attr("text-anchor", "end")
      .attr("fill", "#374151").attr("font-size", "11px")
      .text(v);
  });

  // Draw curve on top
  const curvePoints = d3.range(0, b + 0.01, 0.02).map(x => [xScale(x), yScale(f(x))]);
  const linePath = d3.line()(curvePoints);
  g.append("path")
    .attr("d", linePath)
    .attr("fill", "none")
    .attr("stroke", "#1e40af")
    .attr("stroke-width", 2.5);

  // Axis labels
  g.append("text")
```

```
    .attr("x", innerW / 2).attr("y", innerH + 40)
    .attr("text-anchor", "middle")
    .attr("fill", "#374151").attr("font-size", "13px")
    .text("x");

  g.append("text")
    .attr("transform", "rotate(-90)")
    .attr("x", -innerH / 2).attr("y", -36)
    .attr("text-anchor", "middle")
    .attr("fill", "#374151").attr("font-size", "13px")
    .text("f(x) = x² + 1");

  // Stats display below chart
  const statsY = innerH + 52;
  const error = Math.abs(approxArea - exactArea);

  svg.append("text")
    .attr("x", margin.left).attr("y", margin.top + statsY)
    .attr("fill", "#1d4ed8").attr("font-size", "13px").attr("font-weight", "600")
    .text(`Approximate area ⍰ ${approxArea.toFixed(4)}`);

  svg.append("text")
    .attr("x", margin.left + 220).attr("y", margin.top + statsY)
    .attr("fill", "#374151").attr("font-size", "13px")
    .text(`Exact area = ${exactArea.toFixed(3)}`);

  svg.append("text")
    .attr("x", margin.left + 400).attr("y", margin.top + statsY)
    .attr("fill", error < 0.01 ? "#059669" : "#b45309").attr("font-size", "13px")
    .text(`|Error| = ${error.toFixed(4)}`);

  return svg.node();
}
```

## 28.2. The Fundamental Theorem of Calculus

The Riemann sum definition is conceptually important but computationally brutal — you would need to take a limit every time you evaluate an integral. The Fundamental Theorem of Calculus (FTC) is the result that makes integration tractable. It has two parts.

### 28.2.1. Part 1 — Differentiation undoes integration

Define a function $F$ by accumulating area from a fixed point $a$:

$$F(x) = \int_a^x f(t)\,dt$$

Part 1 says: *differentiate F and you get f back.*

$$F'(x) = f(x)$$

In words: the rate at which accumulated area grows at $x$ is exactly $f(x)$. If $f(x)$ is large and positive, area accumulates quickly; if $f(x)$ is zero, the accumulated area is momentarily flat; if $f(x)$ is negative, the running total is decreasing. The integral of a rate is an accumulation function, and its derivative returns the original rate. This means accumulation and rate are paired concepts: one builds totals, the other reads off how fast those totals are changing.

This is the theoretical bridge between differentiation and integration. They are inverse operations — like squaring and taking a square root — but with a subtlety: integration produces a *family* of antiderivatives differing by a constant. We'll see why immediately.

### 28.2.2. Part 2 — Evaluating definite integrals

Part 2 is the computational workhorse. If $F$ is any antiderivative of $f$ (meaning $F'(x) = f(x)$), then:

$$\int_a^b f(x)\,dx = F(b) - F(a)$$

The shorthand $\big[F(x)\big]_a^b$ means *evaluate F at the top limit, subtract F at the bottom limit*:

$$\big[F(x)\big]_a^b = F(b) - F(a)$$

In words: to evaluate a definite integral, find any antiderivative of the integrand, plug in the upper and lower limits, and subtract. You never need to sum up rectangles explicitly. This means the Fundamental Theorem turns a hard limiting process into an antiderivative calculation.

**A quick example.** Evaluate $\int_0^3 2x\,dx$.

An antiderivative of $2x$ is $x^2$ (because the derivative of $x^2$ is $2x$). Apply Part 2:

$$\int_0^3 2x\,dx = \big[x^2\big]_0^3 = 3^2 - 0^2 = 9$$

The area under the line $y = 2x$ between $x = 0$ and $x = 3$ is exactly 9. This means the theorem recovers a geometric quantity without estimating every strip separately.

## 28.3. Integration rules

Building antiderivatives is the reverse of differentiation. Where differentiation had rules for breaking functions apart, integration has rules for putting them back together. These five cover most of first-year calculus.

**1. Power rule (reversed)**

$$\int x^n \, dx = \frac{x^{n+1}}{n+1} + C \qquad (n \neq -1)$$

Check it: differentiate $\frac{x^{n+1}}{n+1}$ using the power rule and you recover $x^n$. This means antidifferentiation is usually "undo the derivative rule" rather than invent a completely new method.

**2. Exponential**

$$\int e^x \, dx = e^x + C$$

The exponential function is its own derivative, so it is also its own antiderivative.

**3. Reciprocal**

$$\int \frac{1}{x} \, dx = \ln|x| + C$$

The absolute value handles negative $x$. The derivative of $\ln|x|$ is $1/x$ for all $x \neq 0$. This means the reciprocal function is the special case where the ordinary power rule breaks and the logarithm takes over.

**4. Sine**

$$\int \sin(x) \, dx = -\cos(x) + C$$

Check: the derivative of $-\cos(x)$ is $\sin(x)$.

**5. Cosine**

$$\int \cos(x) \, dx = \sin(x) + C$$

Check: the derivative of $\sin(x)$ is $\cos(x)$.

> **Why the constant of integration?**
>
> Every antiderivative of a function $f$ differs from every other by a constant. If $F'(x) = f(x)$, then $(F(x) + 7)' = f(x)$ too. And $(F(x) - 3)' = f(x)$. They all work.
> Writing $+C$ after an indefinite integral captures the entire *family* of valid antiderivatives at once. The $+C$ is not optional decoration — omitting it loses real solutions. For example, if you are solving

a differential equation and need to apply an initial condition, the value of $C$ is determined by that condition. Dropping $C$ before you apply the condition makes the problem unsolvable. This means an indefinite integral does not produce one function, but every function with the same derivative. For *definite* integrals the constant cancels. When you compute $F(b) - F(a)$, the $+C$ appears in both terms and vanishes: $(F(b) + C) - (F(a) + C) = F(b) - F(a)$. The area between two limits is uniquely defined.

## 28.4. Integration by substitution

Substitution is the chain rule running in reverse. Recall that the chain rule for differentiation says:

$$\frac{d}{dx}\big[f(g(x))\big] = f'(g(x)) \cdot g'(x)$$

If you see a product where one factor looks like the derivative of what is inside a function, you can reverse this process. This means substitution is the integration version of recognising an inner function and its derivative as a pair.

**The method, step by step:**

1. Identify a substitution $u = g(x)$ — usually the "inside" function.
2. Differentiate to find $du = g'(x)\,dx$, then solve for $dx$.
3. Rewrite the entire integrand in terms of $u$ and $du$.
4. Integrate with respect to $u$.
5. Substitute back to express the answer in terms of $x$.

**Worked example:** Evaluate $\displaystyle\int 2x\cos(x^2)\,dx$.

**Step 1.** The inside function is $x^2$. Let $u = x^2$.

**Step 2.** Differentiate: $\dfrac{du}{dx} = 2x$, so $du = 2x\,dx$.

**Step 3.** Notice that $2x\,dx$ appears in the integrand. Substitute:

$$\int 2x\cos(x^2)\,dx = \int \cos(u)\,du$$

**Step 4.** Integrate using the cosine rule:

$$\int \cos(u)\,du = \sin(u) + C$$

**Step 5.** Substitute back $u = x^2$:

$$\int 2x\cos(x^2)\,dx = \sin(x^2) + C$$

**Check:** Differentiate $\sin(x^2)$ using the chain rule: $\frac{d}{dx}\left[\sin(x^2)\right] = \cos(x^2) \cdot 2x = 2x\cos(x^2)$. Correct.

The substitution worked because the factor $2x$ outside the cosine was exactly the derivative of $x^2$ inside it. When that match exists, substitution unravels the integral cleanly. This means a good substitution turns a complicated-looking integrand into a standard form you already know how to integrate.

**Interactive: Area under a curve with sign.** Move the sliders to set the limits $a$ and $b$. Positive area (above the axis) is shaded blue; negative area (below the axis) is shaded red. The integral value updates live.

```
//| echo: false
viewof sinA = Inputs.range([-Math.PI, 2 * Math.PI], {
  label: "a (lower limit)",
  step: 0.05,
  value: 0
})
```

```
//| echo: false
viewof sinB = Inputs.range([-Math.PI, 2 * Math.PI], {
  label: "b (upper limit)",
  step: 0.05,
  value: Math.PI
})
```

```
//| echo: false
{
  const W = 560, H = 300;
  const margin = { top: 20, right: 20, bottom: 55, left: 50 };
  const innerW = W - margin.left - margin.right;
  const innerH = H - margin.top - margin.bottom;

  const xMin = -Math.PI, xMax = 2 * Math.PI;
  const f = x => Math.sin(x);

  // Clamp and order a, b
  const aRaw = Math.max(xMin, Math.min(xMax, sinA));
  const bRaw = Math.max(xMin, Math.min(xMax, sinB));
  const lo = Math.min(aRaw, bRaw);
  const hi = Math.max(aRaw, bRaw);

  // Exact integral: ∫sin(x)dx = -cos(x), so F(b) - F(a) = -cos(b) + cos(a)
  const integralValue = -Math.cos(bRaw) + Math.cos(aRaw);

  const xScale = d3.scaleLinear().domain([xMin, xMax]).range([0, innerW]);
  const yScale = d3.scaleLinear().domain([-1.4, 1.4]).range([innerH, 0]);

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
```

```
    .attr("width", "100%")
    .attr("style", `max-width:${W}px; font-family:inherit;`);

  const g = svg.append("g")
    .attr("transform", `translate(${margin.left},${margin.top})`);

  // Shaded area — split into positive and negative segments
  const steps = 600;
  const dx = (hi - lo) / steps;

  if (hi > lo) {
    // Build positive-area path
    let posPath = "";
    let inPos = false;
    let posStart = lo;

    for (let i = 0; i <= steps; i++) {
      const x = lo + i * dx;
      const y = f(x);
      if (y >= 0) {
        if (!inPos) { posStart = x; inPos = true; }
        if (i === steps || f(lo + (i+1)*dx) < 0) {
          // close this positive segment
          const segPoints = d3.range(0, 1.001, 1/60).map(t => {
            const sx = posStart + t * (x - posStart);
            return [xScale(sx), yScale(f(sx))];
          });
          const baseline = [[xScale(x), yScale(0)], [xScale(posStart), yScale(0)]];
          g.append("path")
            .attr("d", "M" + segPoints.map(p => p.join(",")).join("L") + "L" + baseline.map(p => p.j
            .attr("fill", "#3b82f6").attr("opacity", 0.35);
          inPos = false;
        }
      }
    }

    // Build negative-area path segments
    inPos = false;
    let negStart = lo;
    for (let i = 0; i <= steps; i++) {
      const x = lo + i * dx;
      const y = f(x);
      if (y < 0) {
        if (!inPos) { negStart = x; inPos = true; }
        if (i === steps || f(lo + (i+1)*dx) >= 0) {
          const segPoints = d3.range(0, 1.001, 1/60).map(t => {
            const sx = negStart + t * (x - negStart);
```

```
          return [xScale(sx), yScale(f(sx))];
        });
        const baseline = [[xScale(x), yScale(0)], [xScale(negStart), yScale(0)]];
        g.append("path")
          .attr("d", "M" + segPoints.map(p => p.join(",")).join("L") + "L" + baseline.map(p =>
          .attr("fill", "#ef4444").attr("opacity", 0.35);
        inPos = false;
      }
    }
  }
}

// X-axis line
g.append("line")
  .attr("x1", 0).attr("y1", yScale(0))
  .attr("x2", innerW).attr("y2", yScale(0))
  .attr("stroke", "#374151").attr("stroke-width", 1.5);

// Y-axis line
g.append("line")
  .attr("x1", xScale(0)).attr("y1", 0)
  .attr("x2", xScale(0)).attr("y2", innerH)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1);

// Draw curve
const curvePoints = d3.range(xMin, xMax + 0.01, 0.02).map(x => [xScale(x), yScale(f(x))]);
g.append("path")
  .attr("d", d3.line()(curvePoints))
  .attr("fill", "none")
  .attr("stroke", "#1e40af")
  .attr("stroke-width", 2.5);

// Limit markers
[aRaw, bRaw].forEach((v, idx) => {
  g.append("line")
    .attr("x1", xScale(v)).attr("y1", 0)
    .attr("x2", xScale(v)).attr("y2", innerH)
    .attr("stroke", "#6b7280").attr("stroke-width", 1)
    .attr("stroke-dasharray", "4,3");
  g.append("text")
    .attr("x", xScale(v)).attr("y", -6)
    .attr("text-anchor", "middle")
    .attr("fill", "#374151").attr("font-size", "11px").attr("font-weight", "600")
    .text(idx === 0 ? "a" : "b");
});

// X-axis tick labels
```

```
  const ticks = [
    { v: -Math.PI, label: "−π" },
    { v: 0,        label: "0" },
    { v: Math.PI,  label: "π" },
    { v: 2 * Math.PI, label: "2π" }
  ];
  ticks.forEach(({ v, label }) => {
    g.append("line")
      .attr("x1", xScale(v)).attr("y1", yScale(0))
      .attr("x2", xScale(v)).attr("y2", yScale(0) + 5)
      .attr("stroke", "#374151");
    g.append("text")
      .attr("x", xScale(v)).attr("y", yScale(0) + 18)
      .attr("text-anchor", "middle")
      .attr("fill", "#374151").attr("font-size", "12px")
      .text(label);
  });

  // Integral value display
  const absVal = Math.abs(integralValue);
  const sign = integralValue > 0.0005 ? "positive" : integralValue < -0.0005 ? "negative" : "zero";
  const signColour = sign === "positive" ? "#2563eb" : sign === "negative" ? "#dc2626" : "#059669";

  svg.append("text")
    .attr("x", margin.left).attr("y", margin.top + innerH + 46)
    .attr("fill", signColour).attr("font-size", "13px").attr("font-weight", "600")
    .text(`∫ sin(x) dx from a to b  =  ${integralValue.toFixed(4)}   (${sign})`);

  return svg.node();
}
```

## 28.5. Worked examples

### 28.5.1. Example 1 — Power rule and the FTC (computing / probability)

**Problem.** Find the indefinite integral $\int (3x^2 - 4x + 1)\,dx$, then evaluate the definite integral from $x = 0$ to $x = 2$.

A probability density function for a random variable on $[0, 2]$ might have this polynomial form; the definite integral would give the total probability mass over that interval.

**Indefinite integral.** Apply the power rule term by term:

$$\int (3x^2 - 4x + 1)\,dx = \frac{3x^3}{3} - \frac{4x^2}{2} + x + C = x^3 - 2x^2 + x + C$$

**Definite integral.** Apply FTC Part 2 with $F(x) = x^3 - 2x^2 + x$:

$$\int_0^2 (3x^2 - 4x + 1)\,dx = \left[x^3 - 2x^2 + x\right]_0^2$$

Evaluate at the upper limit:

$$F(2) = 2^3 - 2(2^2) + 2 = 8 - 8 + 2 = 2$$

Evaluate at the lower limit:

$$F(0) = 0 - 0 + 0 = 0$$

The definite integral equals $F(2) - F(0) = 2 - 0 = 2$. This means the same antiderivative can answer both the indefinite question and the definite one.

---

### 28.5.2. Example 2 — FTC Part 2 (science / measurement)

**Problem.** A radioactive tracer is administered to a patient and its concentration in the bloodstream decays in a way modelled by $c(x) = x^{-1/2}$ (in appropriate units, for $x > 0$). Compute the total exposure — the area under the concentration curve — between $x = 1$ and $x = 4$.

$$\int_1^4 \frac{1}{\sqrt{x}}\,dx = \int_1^4 x^{-1/2}\,dx$$

Apply the power rule with $n = -\frac{1}{2}$:

$$\int x^{-1/2}\,dx = \frac{x^{-1/2+1}}{-1/2+1} = \frac{x^{1/2}}{1/2} = 2\sqrt{x} + C$$

Apply the limits:

$$\left[2\sqrt{x}\right]_1^4 = 2\sqrt{4} - 2\sqrt{1} = 2(2) - 2(1) = 4 - 2 = 2$$

Total exposure over the interval is 2 (in the model's units). This means a definite integral turns a changing concentration into one total accumulated exposure.

---

### 28.5.3. Example 3 — Substitution (engineering / signals)

**Problem.** In signal processing, an engineer needs to evaluate $\int x e^{x^2}\,dx$ as part of computing the energy in a modulated waveform.

Let $u = x^2$, so $du = 2x\,dx$, which gives $x\,dx = \dfrac{du}{2}$.

$$\int x e^{x^2}\,dx = \int e^u \cdot \frac{du}{2} = \frac{1}{2}\int e^u\,du = \frac{1}{2}e^u + C$$

Substitute back $u = x^2$:

$$\int x e^{x^2}\,dx = \frac{1}{2}e^{x^2} + C$$

**Check:** Differentiate $\frac{1}{2}e^{x^2}$ with the chain rule: $\dfrac{d}{dx}\left[\frac{1}{2}e^{x^2}\right] = \frac{1}{2}\cdot e^{x^2}\cdot 2x = x e^{x^2}$. Correct. This means the factor outside the exponential is exactly what makes the substitution succeed.

---

### 28.5.4. Example 3b — Area between two curves

**Problem.** Find the area enclosed between $f(x) = x + 3$ and $g(x) = x^2 + 1$.

**Step 1 — Find intersections:**

$$x + 3 = x^2 + 1 \implies x^2 - x - 2 = 0 \implies (x-2)(x+1) = 0$$

Intersections at $x = -1$ and $x = 2$.

**Step 2 — Identify which is on top:** At $x = 0$, $f(0) = 3$ and $g(0) = 1$, so $f > g$ on $[-1, 2]$.

**Step 3 — Integrate:**

$$\int_{-1}^{2}\left[(x+3) - (x^2+1)\right]dx = \int_{-1}^{2}(2 + x - x^2)\,dx$$

$$= \left[2x + \frac{x^2}{2} - \frac{x^3}{3}\right]_{-1}^{2}$$

$$= \left(4 + 2 - \frac{8}{3}\right) - \left(-2 + \frac{1}{2} + \frac{1}{3}\right)$$

$$= \frac{10}{3} - \left(-\frac{7}{6}\right) = \frac{20}{6} + \frac{7}{6} = \frac{27}{6} = \frac{9}{2}$$

The enclosed area is $\frac{9}{2}$. This means area between curves is found by integrating "top minus bottom" over the interval where that ordering stays true.

---

### 28.5.5. Example 4 — Area between two curves (business / economic surplus)

**Problem.** In a competitive market, the demand curve is $p_D(q) = 8 - q^2$ and the supply curve is $p_S(q) = 2q$. Find the consumer surplus — the area between the demand curve and the equilibrium price, from zero to the equilibrium quantity.

**Step 1. Find the equilibrium.** Set $p_D = p_S$:

$$8 - q^2 = 2q \implies q^2 + 2q - 8 = 0 \implies (q+4)(q-2) = 0$$

Equilibrium is at $q^* = 2$ (taking the positive root), equilibrium price $p^* = 4$.

**Step 2. Consumer surplus** is the area between the demand curve and the equilibrium price from $q = 0$ to $q = 2$:

$$\text{CS} = \int_0^2 \left[(8 - q^2) - 4\right] dq = \int_0^2 (4 - q^2)\, dq$$

**Step 3. Evaluate:**

$$\int_0^2 (4 - q^2)\, dq = \left[4q - \tfrac{q^3}{3}\right]_0^2 = \left(8 - \frac{8}{3}\right) - 0 = \frac{16}{3} \approx 5.33$$

Consumer surplus is $\frac{16}{3}$ (in the model's currency units per unit quantity). This is the aggregate benefit consumers receive by paying the equilibrium price rather than their maximum willingness to pay. This means an economic surplus is literally an area interpretation of value above or below a market price.

## 28.6. Where this goes

This chapter built integration from limits, then showed how the Fundamental Theorem connects accumulation back to derivatives. The next uses of these ideas are broader settings, not different principles.

**Multivariable calculus (Vol 7).** A single integral accumulates a function over an interval — a one-dimensional slice. In two and three dimensions you can integrate over areas, volumes, and surfaces — the same idea of accumulation, applied to more variables. Those tools (Vol 7) are the language of electromagnetism, fluid dynamics, and heat flow.

**Ordinary differential equations (Vol 7).** A differential equation relates a function to its own derivatives. The simplest case — *separable* equations — is solved by moving all the $y$-terms to one side and all the $x$-terms to the other, then integrating both sides. Every technique in this chapter applies directly. The ability to integrate is the ability to solve differential equations.

## 28.7. Applications

- **Distance from velocity (physics).** Integrate $v(t)$ over a time interval to recover displacement. If $v(t) = 9.8t$ (free fall from rest), then $\int_0^T 9.8t\,dt = 4.9T^2$ — the familiar kinematic formula, derived by integration.
- **Work by a variable force (mechanics).** Work is $W = \int_a^b F(x)\,dx$ when force $F$ varies with position $x$. Compressing a spring, pumping water from a tank — both are definite integrals.
- **Consumer and producer surplus (economics).** The area between a demand (or supply) curve and the equilibrium price is a definite integral. It measures welfare.
- **Probability from a density function (statistics).** If $f(x)$ is a probability density function, then $P(a \le X \le b) = \int_a^b f(x)\,dx$. The cumulative distribution function is $F(x) = \int_{-\infty}^x f(t)\,dt$ — literally the FTC in action.
- **Average value of a function.** The average value of $f$ over $[a, b]$ is $\dfrac{1}{b-a}\int_a^b f(x)\,dx$ — the integral normalised by the length of the interval.

---

## 28.8. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
```

```
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

**Exercise 1.** Find the indefinite integral:

$$\int \left(5x^4 - 3x^2 + 2\right) dx$$

Include the constant of integration in your answer.

```
//| echo: false
makeStepperHTML(1, [
  { op: "Apply power rule term by term",       eq: "\\int 5x^4\\,dx - \\int 3x^2\\,dx + \\int 2\\,dx
  { op: "Integrate each term",                 eq: "\\frac{5x^5}{5} - \\frac{3x^3}{3} + 2x + C" },
  { op: "Simplify coefficients",               eq: "x^5 - x^3 + 2x + C" },
  { op: "Check by differentiating",             eq: "\\frac{d}{dx}\\left(x^5 - x^3 + 2x + C\\right) =
])
```

---

**Exercise 2.** Evaluate the definite integral using the Fundamental Theorem of Calculus Part 2:

$$\int_1^9 \frac{3}{\sqrt{x}}\, dx$$

```
//| echo: false
makeStepperHTML(2, [
  { op: "Rewrite as a power",                  eq: "\\int_1^9 3x^{-1/2}\\,dx", note: "1/√x = x^{-1/2
  { op: "Find antiderivative",                 eq: "3 \\cdot \\frac{x^{1/2}}{1/2} = 6x^{1/2} = 6\\sq
  { op: "Apply FTC Part 2",                    eq: "\\left[6\\sqrt{x}\\right]_1^9 = 6\\sqrt{9} - 6\\
  { op: "Evaluate at limits",                  eq: "6(3) - 6(1) = 18 - 6 = 12" },
  { op: "Answer",                              eq: "\\int_1^9 \\frac{3}{\\sqrt{x}}\\,dx = 12" }
])
```

---

**Exercise 3.** Use $u$-substitution to evaluate:

$$\int 3x^2 \sin(x^3)\, dx$$

```
//| echo: false
makeStepperHTML(3, [
  { op: "Choose substitution",                 eq: "u = x^3", note: "The inside function of sin is x
  { op: "Differentiate",                       eq: "\\frac{du}{dx} = 3x^2 \\implies du = 3x^2\\,dx"
  { op: "Substitute into integral",            eq: "\\int \\sin(u)\\,du" },
  { op: "Integrate",                           eq: "-\\cos(u) + C" },
  { op: "Substitute back",                     eq: "-\\cos(x^3) + C" },
  { op: "Check by differentiating",             eq: "\\frac{d}{dx}\\left[-\\cos(x^3)\\right] = \\sin
])
```

---

**Exercise 4.** Find the area under the curve $f(x) = x^2 + 2x$ between $x = 0$ and $x = 3$. Sketch the region (or describe it in words) before computing.

```
//| echo: false
makeStepperHTML(4, [
  { op: "Set up the definite integral",        eq: "\\int_0^3 (x^2 + 2x)\\,dx", note: "f(x) ⍰ 0 o
  { op: "Find the antiderivative",             eq: "F(x) = \\frac{x^3}{3} + x^2" },
  { op: "Apply FTC Part 2",                    eq: "\\left[\\frac{x^3}{3} + x^2\\right]_0^3 = F(3
  { op: "Evaluate at upper limit",             eq: "F(3) = \\frac{27}{3} + 9 = 9 + 9 = 18" },
  { op: "Evaluate at lower limit",             eq: "F(0) = 0 + 0 = 0" },
  { op: "Compute the area",                    eq: "18 - 0 = 18" }
])
```

---

**Exercise 5.** Find the area enclosed between $f(x) = 4 - x^2$ and $g(x) = x + 2$.

*Hint: find the intersection points first. The area between two curves is* $\int_a^b \big[f(x) - g(x)\big]\,dx$ *when* $f(x) \geq g(x)$ *on* $[a, b]$.

```
//| echo: false
makeStepperHTML(5, [
  { op: "Find intersections",                  eq: "4 - x^2 = x + 2 \\implies x^2 + x - 2 = 0 \\i
  { op: "Check which is on top",               eq: "\\text{At } x=0:\\; f(0)=4,\\; g(0)=2 \\impli
  { op: "Set up integral of difference",       eq: "\\int_{-2}^{1} \\bigl[(4-x^2) - (x+2)\\bigr]\
  { op: "Find antiderivative",                 eq: "F(x) = 2x - \\frac{x^2}{2} - \\frac{x^3}{3}"
  { op: "Evaluate at upper limit",             eq: "F(1) = 2 - \\tfrac{1}{2} - \\tfrac{1}{3} = \\
  { op: "Evaluate at lower limit",             eq: "F(-2) = -4 - 2 + \\tfrac{8}{3} = \\tfrac{-18+
  { op: "Compute the area",                    eq: "\\tfrac{7}{6} - \\left(-\\tfrac{10}{3}\\right
])
```

---

**Exercise 6 — Accumulation in context.** A solar panel generates electricity at a rate of $P(t) = 4\sin\left(\frac{\pi t}{12}\right)$ kilowatts, where $t$ is hours after sunrise ($0 \leq t \leq 12$).

(a) Find the total energy (in kilowatt-hours) generated over the full 12-hour day: $\displaystyle\int_0^{12} P(t)\,dt$.

(b) Interpret the result: what does the definite integral represent physically?

*(Your answer will be in terms of $\pi$ — that is expected.)*

```
//| echo: false
makeStepperHTML(6, [
  { op: "Set up the integral",                eq: "\\int_0^{12} 4\\sin\\!\\left(\\frac{\\pi t}{12}\
  { op: "Choose substitution",                eq: "u = \\frac{\\pi t}{12} \\implies du = \\frac{\\p
  { op: "Change limits",                      eq: "t=0 \\Rightarrow u=0;\\quad t=12 \\Rightarrow u=
  { op: "Substitute and simplify",            eq: "4 \\cdot \\frac{12}{\\pi} \\int_0^{\\pi} \\sin(u
  { op: "Integrate sin",                      eq: "\\frac{48}{\\pi}\\left[-\\cos(u)\\right]_0^{\\pi
  { op: "Evaluate at limits",                 eq: "\\frac{48}{\\pi}\\left(-\\cos(\\pi) + \\cos(0)\\
  { op: "Interpret the result",               eq: "\\text{Total energy} = \\frac{96}{\\pi} \\approx
])
```

# 29. Proof and mathematical reasoning

Establishing that something must be true

You have used the formula for the sum of an arithmetic series. You have used the quadratic formula. You have been told that $\sqrt{2}$ is irrational. Each time, you accepted the result because it worked on the examples you tried, or because someone authoritative said so.

That is about to change.

The shift from computing to proving is the sharpest turn in mathematics. It is the difference between "this has always worked so far" and "this cannot fail." Most of the mathematics you have seen up to now has been the first kind. This chapter introduces the second.

## 29.1. A pattern that lies to you

Consider the expression $n^2 - n + 41$. Try a few values:

| $n$ | $n^2 - n + 41$ | Prime? |
|---|---|---|
| 1 | 41 | Yes |
| 2 | 43 | Yes |
| 3 | 47 | Yes |
| 4 | 53 | Yes |
| 10 | 131 | Yes |
| 20 | 421 | Yes |

In fact, every value from $n = 1$ to $n = 40$ gives a prime number.

Forty consecutive confirmations. Most people would bet on "always prime."

But try $n = 41$:

$$41^2 - 41 + 41 = 41^2 = 1681 = 41 \times 41$$

Clearly composite. The pattern fails the moment you step past the range where you checked it.

This is the central lesson before we write a single proof: **no number of confirmed cases constitutes a proof.** Not ten, not a hundred, not a trillion. A proof must cover every case, including those that haven't been tried. This means examples are useful for guessing a theorem, but never enough for establishing one.

```
//| echo: false

// ── VIZ 1: Primality checker for n²-n+41 ──────────────────────────────

function isPrime(n) {
  if (n < 2) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) {
    if (n % i === 0) return false;
  }
  return true;
}

function factorise(n) {
  const factors = [];
  let d = 2;
  let m = n;
  while (d * d <= m) {
    while (m % d === 0) {
      factors.push(d);
      m = Math.floor(m / d);
    }
    d++;
  }
  if (m > 1) factors.push(m);
  return factors;
}

function formatFactors(n) {
  const f = factorise(n);
  if (f.length <= 1) return String(n);
  // group into exponent notation for display
  const counts = {};
  for (const p of f) counts[p] = (counts[p] || 0) + 1;
  return Object.entries(counts)
    .map(([p, e]) => e === 1 ? p : `${p}^{${e}}`)
    .join(" \\times ");
}

viewof nSlider = Inputs.range([1, 50], {
  step: 1,
  value: 1,
  label: "n"
})
```

```
//| echo: false

{
```

```
const n = nSlider;
const p = n * n - n + 41;
const prime = isPrime(p);

const dotColour = prime ? "#22c55e" : "#ef4444";
const statusText = prime ? "prime" : "composite";
const statusColour = prime ? "#16a34a" : "#dc2626";

// Factorisation string for composite values
let factorHTML = "";
if (!prime) {
  const counts = {};
  for (const f of factorise(p)) counts[f] = (counts[f] || 0) + 1;
  const fStr = Object.entries(counts)
    .map(([f, e]) => e === 1 ? f : `${f}<sup>${e}</sup>`)
    .join(" × ");
  factorHTML = `<p style="margin:0.4rem 0 0;font-size:0.9rem;color:#374151;">
    Factorisation: ${p} = ${fStr}
  </p>`;
}

// Show a snapshot table of n = 1..current n (up to 10 most recent from 1)
const tableStart = Math.max(1, n - 9);
const tableRows = Array.from({ length: n - tableStart + 1 }, (_, i) => {
  const k = tableStart + i;
  const pk = k * k - k + 41;
  const primeK = isPrime(pk);
  const c = primeK ? "#16a34a" : "#dc2626";
  const lbl = primeK ? "prime" : "composite";
  const bold = k === n ? "font-weight:700;background:#f5f3ff;" : "";
  return `<tr style="${bold}">
    <td style="padding:3px 10px;text-align:center;">${k}</td>
    <td style="padding:3px 10px;text-align:center;">${pk}</td>
    <td style="padding:3px 10px;text-align:center;color:${c};">${lbl}</td>
  </tr>`;
}).reverse().join("");

return htl.html`<div style="font-family:sans-serif;max-width:480px;">
  <div style="padding:1rem 1.25rem;border:1px solid #e5e7eb;border-radius:8px;margin-bottom:0.7
    <p style="margin:0 0 0.35rem;font-size:1rem;"><em>n</em> = <strong>${n}</strong></p>
    <p style="margin:0 0 0.5rem;font-size:1rem;"><em>n</em>² − <em>n</em> + 41 = <strong>${p}</
    <p style="margin:0;display:flex;align-items:center;gap:6px;">
      <span style="display:inline-block;width:13px;height:13px;border-radius:50%;background:${d
      <strong style="color:${statusColour};">${p} is ${statusText}</strong>
    </p>
    ${htl.html([factorHTML])}
  </div>
```

```
    <table style="border-collapse:collapse;width:100%;font-size:0.88rem;">
      <thead>
        <tr style="border-bottom:2px solid #d1d5db;">
          <th style="padding:3px 10px;font-weight:600;">n</th>
          <th style="padding:3px 10px;font-weight:600;">n²−n+41</th>
          <th style="padding:3px 10px;font-weight:600;">Status</th>
        </tr>
      </thead>
      <tbody>${htl.html([tableRows])}</tbody>
    </table>
  </div>`;
}
```

## 29.2. What a proof is

A **proof** is a sequence of logical deductions from agreed starting points — called **axioms** or previously proved **theorems** — leading to a conclusion. Each step must follow from the previous ones by a valid logical rule. If every step is valid, the conclusion is guaranteed. Not likely. Not probably. Guaranteed.

Every proof has the same skeleton:

- **Statement:** what you are claiming is true
- **Hypothesis:** what you are allowed to assume
- **Conclusion:** what you need to establish
- **Proof:** the chain of deductions from hypothesis to conclusion

The hypothesis is given to you. The conclusion is what you must reach. Everything in between is your work.

### 29.2.1. The notation of logical structure

Two symbols carry most of the logical load:

$P \Rightarrow Q$ means $P$ **implies** $Q$: if $P$ is true, then $Q$ must be true. Read it as "if $P$, then $Q$." The arrow shows direction — $P \Rightarrow Q$ does not automatically mean $Q \Rightarrow P$.

$P \Leftrightarrow Q$ means $P$ **if and only if** $Q$: the implication runs both ways. $P$ is true exactly when $Q$ is true, and vice versa.

Two quantifiers say *which* objects you are making claims about:

$\forall x$ means **for all** $x$ — the statement holds for every element in the domain under discussion.

$\exists x$ means **there exists an** $x$ — the statement holds for at least one element.

The symbol $x \in S$ means $x$ **is a member of the set** $S$. You will see it most often with the standard number sets:

- $\mathbb{N}$ — the natural numbers: $1, 2, 3, \ldots$

- $\mathbb{Z}$ — the integers: $\ldots, -2, -1, 0, 1, 2, \ldots$
- $\mathbb{Q}$ — the rationals: all fractions $p/q$ with $p, q \in \mathbb{Z}$, $q \neq 0$
- $\mathbb{R}$ — the real numbers: everything on the continuous number line

So "$\forall n \in \mathbb{Z}$" reads: "for every integer $n$." A typical theorem opens with a phrase like this to tell you exactly what kind of object is being discussed.

## 29.3. Four proof methods

There is no single template. What follows are four approaches that cover the vast majority of what you will encounter in the first two years of university mathematics.

### 29.3.1. 1. Direct proof

The simplest strategy: assume the hypothesis, apply definitions and algebra, arrive at the conclusion.

**Theorem.** The sum of two even integers is even.
**Proof.** Let $m$ and $n$ be even integers. By definition of "even," there exist integers $a$ and $b$ such that $m = 2a$ and $n = 2b$. Then:

$$m + n = 2a + 2b = 2(a + b)$$

Since $a + b$ is an integer, $m + n$ is divisible by 2, hence even. □

Notice the structure: start with the definitions of the objects involved, do algebra, end at the conclusion. The key move was unpacking "even" into its definition ($m = 2a$) so that the algebra could work. This means direct proof usually begins by translating words like "even" or "odd" into algebraic form.

### 29.3.2. 2. Proof by contrapositive

Every statement "if $P$ then $Q$" ($P \Rightarrow Q$) is **logically equivalent** (meaning: any proof of one automatically proves the other — they make exactly the same claim in different words) to its **contrapositive**: "if $\neg Q$ then $\neg P$" ($\neg Q \Rightarrow \neg P$), where $\neg Q$ means "not $Q$" — the opposite truth value.

Sometimes the contrapositive is much easier to work with directly.

**Theorem.** If $n^2$ is even, then $n$ is even.
**Proof.** We prove the contrapositive: if $n$ is odd, then $n^2$ is odd.
Suppose $n$ is odd. Then $n = 2k + 1$ for some integer $k$. Squaring:

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$$

Since $2k^2 + 2k$ is an integer, $n^2$ has the form $2(\text{integer}) + 1$, so $n^2$ is odd.
The contrapositive is proved. Therefore the original statement holds. □

The reason to choose contrapositive over direct proof here: starting from "$n^2$ is even" and trying to work towards "$n$ is even" is awkward — you'd have to take a square root, and it isn't obvious that $\sqrt{2k}$ is

an integer. The contrapositive avoids that entirely. This means contrapositive proof is not a different theorem, only a smarter route to the same theorem.

### 29.3.3. 3. Proof by contradiction

Assume the conclusion is *false*. Show that this assumption, together with the hypothesis, leads to a logical impossibility — a statement that contradicts something you know to be true. Since the assumption of falseness leads to a contradiction, the conclusion must be true.

The classic example is one you may have seen stated but never seen proved:

**Theorem.** $\sqrt{2}$ is irrational.
**Proof.** Suppose for contradiction that $\sqrt{2}$ is rational. Then we can write $\sqrt{2} = p/q$ where $p, q \in \mathbb{Z}$, $q \neq 0$, and $p$ and $q$ share no common factor (the fraction is in **lowest terms**).
Squaring both sides: $2 = p^2/q^2$, so $p^2 = 2q^2$.
This means $p^2$ is even. By the theorem proved above (contrapositive), $p$ must be even. So write $p = 2k$ for some integer $k$.
Substituting: $(2k)^2 = 2q^2$, which gives $4k^2 = 2q^2$, so $q^2 = 2k^2$.
This means $q^2$ is even, so $q$ is even (by the same reasoning we used for $p$: if a number's square is even, the number itself must be even).
But now both $p$ and $q$ are even — they share a factor of 2. This contradicts our assumption that $p/q$ was in lowest terms.
The assumption that $\sqrt{2}$ is rational leads to a contradiction. Therefore $\sqrt{2}$ is irrational. □

Note how the proof assembled earlier results — "if $n^2$ is even then $n$ is even" was used twice. Proofs are built on top of other proofs. The earlier theorem was not an isolated exercise; it was a tool. This means contradiction works by assuming the opposite and showing that the assumption breaks the logical system you started with.

### 29.3.4. 4. Mathematical induction

Induction proves statements of the form "this holds for every natural number $n \geq 1$" (or $n \geq 0$, or from any fixed starting point).

The structure has two parts:

1. **Base case.** Show the statement holds for the starting value (usually $n = 1$).
2. **Inductive step.** Show that *if* the statement holds for $n = k$ (the **inductive hypothesis**), then it holds for $n = k + 1$.

If both hold, the statement holds for all $n$ from the base case onward. The logic is a chain: the base case starts the chain, the inductive step extends it one link at a time, and there is no upper limit on the chain length.

**Theorem.** For all $n \geq 1$:

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

**Proof.**
**Base case.** When $n = 1$: the left side is 1, and $\frac{1 \cdot 2}{2} = 1$. The statement holds.
**Inductive step.** Assume the statement holds for $n = k$, i.e., assume:

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}$$

We need to show it holds for $n = k + 1$, i.e., that:

$$1 + 2 + \cdots + k + (k+1) = \frac{(k+1)(k+2)}{2}$$

Starting from the left side and using the inductive hypothesis:

$$1 + 2 + \cdots + k + (k+1) = \frac{k(k+1)}{2} + (k+1)$$

Factor out $(k + 1)$:

$$= (k+1)\left(\frac{k}{2} + 1\right) = (k+1) \cdot \frac{k+2}{2} = \frac{(k+1)(k+2)}{2}$$

This is exactly what we needed to show.
By induction, the formula holds for all $n \geq 1$. □

The formula $n(n + 1)/2$ is not new to you — it appeared when we studied arithmetic series. But now you have not just a formula and some confirming examples. You have a proof. It holds for $n = 10^{100}$ as surely as for $n = 5$. This means induction proves infinitely many cases by establishing a base and a reliable step from one case to the next.

```
//| echo: false

// ── VIZ 2: Induction visualiser for ∑k = n(n+1)/2 ──────────────────────

viewof nInduct = Inputs.range([1, 20], {
  step: 1,
  value: 5,
  label: "n"
})
```

```
//| echo: false

{
  const n = nInduct;

  // Compute partial sums S_k = k(k+1)/2 for k = 1..n
  const data = Array.from({ length: n }, (_, i) => {
    const k = i + 1;
    const Sk = k * (k + 1) / 2;
    return { k, Sk };
  });
```

```
  const Sn = n * (n + 1) / 2;
  const maxS = data[data.length - 1].Sk;

  // SVG dimensions
  const W = 480, H = 260;
  const padL = 48, padR = 20, padT = 20, padB = 40;
  const chartW = W - padL - padR;
  const chartH = H - padT - padB;

  const xScale = k => padL + ((k - 1) / Math.max(n - 1, 1)) * chartW;
  const yScale = v => padT + chartH - (v / maxS) * chartH;
  const barW = Math.max(4, Math.min(32, (chartW / n) * 0.6));

  // Build bar rects
  const bars = data.map(({ k, Sk }) => {
    const x = xScale(k) - barW / 2;
    const y = yScale(Sk);
    const h = padT + chartH - y;
    return `<rect x="${x.toFixed(1)}" y="${y.toFixed(1)}" width="${barW}" height="${h.toFixed(1)}"
              fill="#6366f1" opacity="0.75" rx="2"/>`;
  }).join("\n");

  // Build formula dots + connecting line
  const pts = data.map(({ k, Sk }) => `${xScale(k).toFixed(1)},${yScale(Sk).toFixed(1)}`).join(" ")
  const dots = data.map(({ k, Sk }) =>
    `<circle cx="${xScale(k).toFixed(1)}" cy="${yScale(Sk).toFixed(1)}" r="5" fill="#f97316" stroke=
  ).join("\n");

  // x-axis tick labels
  const xTicks = data.map(({ k }) =>
    `<text x="${xScale(k).toFixed(1)}" y="${(padT + chartH + 18).toFixed(1)}" text-anchor="middle" f
  ).join("\n");

  // y-axis ticks (3–4 ticks)
  const yTickCount = 4;
  const yTicks = Array.from({ length: yTickCount + 1 }, (_, i) => {
    const v = Math.round((maxS / yTickCount) * i);
    const y = yScale(v);
    return `<line x1="${padL - 4}" y1="${y.toFixed(1)}" x2="${padL}" y2="${y.toFixed(1)}" stroke="#9
<text x="${(padL - 6).toFixed(1)}" y="${(y + 4).toFixed(1)}" text-anchor="end" font-size="10" fill="
  }).join("\n");

  const svg = `<svg viewBox="0 0 ${W} ${H}" style="width:100%;max-width:${W}px;display:block;font-fa
  <!-- axes -->
  <line x1="${padL}" y1="${padT}" x2="${padL}" y2="${padT + chartH}" stroke="#d1d5db" stroke-width="
  <line x1="${padL}" y1="${padT + chartH}" x2="${padL + chartW}" y2="${padT + chartH}" stroke="#d1d5
  ${yTicks}
```

```
  ${xTicks}
  <!-- bars -->
  ${bars}
  <!-- formula line -->
  <polyline points="${pts}" fill="none" stroke="#f97316" stroke-width="2" opacity="0.6"/>
  <!-- formula dots -->
  ${dots}
  <!-- legend -->
  <rect x="${padL + 4}" y="${padT + 4}" width="12" height="12" fill="#6366f1" opacity="0.75" rx="
  <text x="${padL + 20}" y="${padT + 14}" font-size="11" fill="#374151">partial sum</text>
  <circle cx="${padL + 130}" cy="${padT + 10}" r="5" fill="#f97316"/>
  <text x="${padL + 140}" y="${padT + 14}" font-size="11" fill="#374151">k(k+1)/2</text>
  <!-- x-axis label -->
  <text x="${(padL + chartW / 2).toFixed(1)}" y="${H - 4}" text-anchor="middle" font-size="11" fi
</svg>`;

  const formulaValue = n * (n + 1) / 2;

  return htl.html`<div style="font-family:sans-serif;max-width:500px;">
    ${htl.html([svg])}
    <p style="margin:0.75rem 0 0.25rem;font-size:0.95rem;">
      Sum: <strong>1 + 2 + ⋯ + ${n} = ${Sn}</strong>
    </p>
    <p style="margin:0;font-size:0.95rem;">
      Formula: <strong>${n}(${n}+1)/2 = ${n} × ${n+1} / 2 = ${formulaValue}</strong>
    </p>
  </div>`;
}
```

## 29.4. What makes a proof fail

Not every argument that looks like a proof is one. Three failure modes appear constantly, and learning to recognise them is half the work.

### 29.4.1. 1. Assuming the conclusion

Also called **circular reasoning**: a step in the proof assumes, in some form, the very thing you are trying to prove.

Here is a compact example. Suppose someone "proves" that $1 = 2$ like this:

> Let $a = b$. Then $a^2 = ab$, so $a^2 - b^2 = ab - b^2$, so $(a + b)(a - b) = b(a - b)$, so $a + b = b$. Since $a = b$, this gives $2b = b$, so $2 = 1$.

The flaw: the step "so $a + b = b$" divides both sides by $(a - b)$. But $a = b$, so $a - b = 0$. Division by zero is not a valid algebraic step. It is not that the algebra went wrong in a subtle way — the step is simply not allowed.

Circular reasoning is harder to spot than hidden division by zero. Watch for any step that *uses* the thing you're trying to *establish*.

### 29.4.2. 2. Missing a case

A proof that only covers some instances — even most instances — is not a proof. The primality of $n^2 - n + 41$ was "verified" for 40 cases and failed on the 41st. Missing cases can be more subtle: a proof about integers that silently only works for positive ones, a proof about triangles that only handles the acute case.

Before finishing a proof, ask: are there configurations I haven't considered? Edge cases, boundary values, negative inputs, degenerate shapes? This means a proof can fail not only through bad algebra, but through an argument that silently covers only part of the problem.

### 29.4.3. 3. An invalid step

An algebraic manipulation that is not universally valid. The most common:

- **Dividing by a variable** without confirming it's nonzero (as in the $a = b$ example above)
- **Taking a square root** and losing the negative case: $x^2 = 4$ implies $x = 2$ or $x = -2$, not just $x = 2$
- **Cancelling factors** that might be zero
- **Swapping the order** of limits, integrals, or sums without justification (this matters more in later courses)

Every algebraic step has conditions under which it is valid. When those conditions aren't met, the step is wrong — even if it gives the right answer in specific cases. This means correctness in proof depends on the legality of each step, not on whether the final line happens to look plausible.

**What proofs are actually for**

Proofs are not rituals performed for examiners. They are the reason mathematicians can trust results for trillions of cases they've never checked.

When you prove that an algorithm runs in $O(n \log n)$ time, you're not estimating — you're guaranteeing. When a cryptographic security proof says the encryption is unbreakable under certain assumptions, the logic is auditable. A regulator, a competitor, a court can read every step. The proof is the audit trail.

Physical laws are not just patterns observed in experiments. They are structures whose consequences can be derived — predicted before they are measured. The derivation is a proof. When a prediction matches an observation, it confirms the law; when it doesn't, the law is wrong. The reliability of science rests on this structure.

Mathematics is the only domain where "I've checked a huge number of cases" is explicitly not sufficient, and where a short argument can settle a question for every case simultaneously. That is what makes it powerful.

## 29.5. Worked examples

### 29.5.1. Example 1: Direct proof — the product of two odd numbers

**Context.** Two players take turns on a board game. Each player's turn takes an odd number of seconds. Prove that no matter how many turns have been played, the product of any two turn lengths is always odd.

**Theorem.** The product of two odd integers is odd.
**Proof.** Let $m$ and $n$ be odd integers. By definition, there exist integers $a$ and $b$ such that $m = 2a + 1$ and $n = 2b + 1$.
Then:

$$m \cdot n = (2a + 1)(2b + 1) = 4ab + 2a + 2b + 1 = 2(2ab + a + b) + 1$$

Since $2ab + a + b$ is an integer, $mn$ has the form $2(\text{integer}) + 1$, so $mn$ is odd. □

**Commentary.** The key move was unpacking the definition of "odd" into algebra. Once you write $m = 2a+1$ and $n = 2b+1$, the result of multiplying them is computable. The only thing left is to recognise that the answer has the right form — it is one more than an even number.

The proof doesn't appeal to intuition, examples, or the fact that you've tested it. It works because the algebra always produces $2(\ldots) + 1$, regardless of what $a$ and $b$ are. This means a direct proof succeeds when the conclusion appears naturally from the definition of the objects involved.

### 29.5.2. Example 2: Proof by contradiction — infinitely many primes

**Context.** Public-key cryptography (RSA, used in every HTTPS connection) relies on the existence of very large prime numbers. The security argument assumes there are infinitely many primes to draw from. Euclid proved this over 2000 years ago. The argument has not changed.

**Theorem.** There are infinitely many prime numbers.
**Proof.** Suppose for contradiction that there are only finitely many primes. List them all: $p_1, p_2, \ldots, p_k$.
Consider the number:

$$N = p_1 \cdot p_2 \cdots p_k + 1$$

$N$ is greater than 1, so it has at least one prime factor. Call it $p$. Since our list contains all primes, $p$ must be one of $p_1, p_2, \ldots, p_k$.
But $p$ divides $p_1 \cdot p_2 \cdots p_k$ (since $p$ is one of those factors). And $p$ divides $N$. So $p$ must divide the difference:

$$N - p_1 \cdot p_2 \cdots p_k = 1$$

But no prime divides 1. Contradiction.
The assumption that there are finitely many primes leads to a contradiction. Therefore there are infinitely many primes. □

**Commentary.** The number $N$ is not claimed to be prime. It is constructed deliberately so that dividing it by any prime on the list leaves a remainder of 1. The contradiction is that the prime factor of $N$ must be on the list but cannot be on the list.

This is one of the most elegant proofs in mathematics. It uses no advanced machinery — only the definition of prime, the fact that every integer greater than 1 has a prime factor, and arithmetic. The idea (constructing $N$) is the creative step; the logic is then straightforward. This means contradiction proofs often hinge on constructing exactly the object that the false assumption cannot accommodate.

### 29.5.3. Example 3: Proof by induction — $2^n > n$ for all $n \geq 1$

**Context.** In digital circuit design, the number of possible states in a circuit with $n$ binary switches is $2^n$. Engineers say this "grows exponentially." Proving $2^n > n$ is a basic instance of establishing that exponential growth outpaces linear growth — relevant whenever you're deciding whether a brute-force approach (checking every state) is feasible.

**Theorem.** For all integers $n \geq 1$, $2^n > n$.
**Proof.**
**Base case.** When $n = 1$: $2^1 = 2 > 1$. The statement holds.
**Inductive step.** Assume the statement holds for $n = k$, i.e., assume $2^k > k$. We need to show $2^{k+1} > k + 1$.
Starting from the inductive hypothesis $2^k > k$, multiply both sides by 2:

$$2^{k+1} = 2 \cdot 2^k > 2k$$

It suffices to show $2k \geq k + 1$, i.e., $k \geq 1$. Since $k \geq 1$ by our starting point, this holds.
Therefore $2^{k+1} > 2k \geq k + 1$.
By induction, $2^n > n$ for all $n \geq 1$. □

**Commentary.** The inductive step needed two pieces: first, using the inductive hypothesis to get a bound on $2^{k+1}$; second, connecting that bound to $k + 1$. The step "$2k \geq k + 1$ because $k \geq 1$" looks almost too obvious to write down. Write it down anyway. In a proof, "obvious" is not a valid reason — the valid reason is "$k \geq 1$." This means induction is only convincing when every link in the chain is explicitly justified.

## 29.6. Where this goes

This chapter shifted mathematics from calculation to justification. The ideas ahead do not only need formulas; they rely on knowing why the formulas and theorems can be trusted.

**Throughout this series.** Every other volume has been making claims without full justification — the sum formula for arithmetic series, the laws of exponents, properties of functions. You now have the tools to ask "how do we know that?" and to read the answer. The proofs are in the textbooks; you can now follow them.

**Complex analysis (Volume 7).** The central results there — Cauchy's integral theorem, the residue theorem — are not computational tricks. They are theorems with proofs that depend on precise definitions of limits, continuity, and differentiability. Those proofs are accessible once you are comfortable with the logical structure introduced here. Complex analysis is where proof and computation meet in the most surprising way: a function's behaviour on a closed curve can determine its behaviour at every point inside.

Applications

**Algorithm correctness.** Proving that an algorithm produces the right output for *every* valid input, not just the test cases you've tried. Induction is the standard tool for array algorithms and recursive functions.
**Cryptographic security proofs.** RSA's security is reduced to the difficulty of factoring large numbers — the reduction is a proof. If you could break RSA, you could factor; but factoring appears hard. The chain of implication is a formal proof.
**Derivation from physical laws.** The equations governing heat flow, electromagnetic fields, and quantum mechanics are not empirical curve fits. They are derived — proved — from foundational principles. The derivations are proofs.
**Formal verification.** Aviation, medical devices, and nuclear control systems are subject to formal verification: software is checked against a mathematical model, and the check is a proof. DO-178C (aviation software standard) and IEC 62304 (medical software) both recognise formal methods. The mathematical core is the logic introduced here.

## 29.7. Exercises

```
//| echo: false

// ── Shared stepper infrastructure ──────────────────────────────────────

katex = require("katex@0.16")

function makeStepperHTML(problem, steps) {
  let current = -1;

  const problemDiv = document.createElement("div");
  problemDiv.style.cssText = "margin-bottom:0.75rem;font-weight:600;font-size:0.95rem;";
  problemDiv.textContent = problem;

  const stepsContainer = document.createElement("div");

  const stepEls = steps.map((s, i) => {
    const el = document.createElement("div");
    el.style.cssText = `
      display:none;
      margin-top:0.6rem;
      padding:0.6rem 0.85rem;
      border-left:3px solid #6366f1;
      background:#f5f3ff;
      border-radius:0 6px 6px 0;
      font-size:0.92rem;
    `;
    const opSpan = document.createElement("div");
```

```
    opSpan.style.cssText = "color:#6b7280;font-size:0.82rem;margin-bottom:0.3rem;font-style:italic;"
    opSpan.textContent = s.op;

    const eqDiv = document.createElement("div");
    try {
      eqDiv.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    } catch(e) {
      eqDiv.textContent = s.eq;
    }

    el.appendChild(opSpan);
    el.appendChild(eqDiv);
    stepsContainer.appendChild(el);
    return el;
  });

  const btnRow = document.createElement("div");
  btnRow.style.cssText = "display:flex;gap:0.5rem;margin-top:0.75rem;align-items:center;";

  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = `
    padding:0.35rem 0.85rem;
    background:#6366f1;
    color:white;
    border:none;
    border-radius:5px;
    cursor:pointer;
    font-size:0.88rem;
  `;

  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = `
    padding:0.35rem 0.75rem;
    background:transparent;
    color:#6b7280;
    border:1px solid #d1d5db;
    border-radius:5px;
    cursor:pointer;
    font-size:0.88rem;
  `;

  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82rem;color:#9ca3af;margin-left:0.25rem;";
  counter.textContent = `0 / ${steps.length}`;
```

```
  function update() {
    stepEls.forEach((el, i) => {
      el.style.display = i <= current ? "block" : "none";
    });
    counter.textContent = `${current + 1} / ${steps.length}`;
    nextBtn.disabled = current >= steps.length - 1;
    nextBtn.style.opacity = nextBtn.disabled ? "0.45" : "1";
  }

  nextBtn.addEventListener("click", () => {
    if (current < steps.length - 1) { current++; update(); }
  });
  resetBtn.addEventListener("click", () => {
    current = -1; update();
  });

  btnRow.appendChild(nextBtn);
  btnRow.appendChild(resetBtn);
  btnRow.appendChild(counter);

  const wrapper = document.createElement("div");
  wrapper.style.cssText = `
    border:1px solid #e5e7eb;
    border-radius:8px;
    padding:1rem 1.1rem;
    max-width:560px;
    font-family:sans-serif;
    margin-top:0.75rem;
  `;
  wrapper.appendChild(problemDiv);
  wrapper.appendChild(stepsContainer);
  wrapper.appendChild(btnRow);

  update();
  return wrapper;
}
```

**Exercise 1.** Prove that if $n$ is divisible by 3, then $n^2$ is divisible by 9.

*Hint.* Write $n = 3k$ for some integer $k$, then compute $n^2$.

```
//| echo: false

makeStepperHTML(
  "If 3 divides n, prove that 9 divides n².",
  [
    {
      op: "Write out the hypothesis in algebraic form",
```

```
      eq: "\\text{Since } 3 \\mid n, \\text{ there exists an integer } k \\text{ such that } n = 3k
    },
    {
      op: "Compute n²",
      eq: "n^2 = (3k)^2 = 9k^2"
    },
    {
      op: "Identify the structure of the result",
      eq: "9k^2 = 9 \\cdot k^2, \\text{ and } k^2 \\in \\mathbb{Z}, \\text{ so } 9 \\mid n^2."
    },
    {
      op: "Conclude",
      eq: "\\text{Therefore } n^2 \\text{ is divisible by } 9. \\quad \\square"
    }
  ]
)
```

---

**Exercise 2.** Prove by contradiction that $\log_2 3$ is irrational.

*Hint.* Assume $\log_2 3 = p/q$ in lowest terms and use the definition of logarithm: $\log_2 3 = p/q$ means $2^{p/q} = 3$.

```
//| echo: false

makeStepperHTML(
  "Prove that log₂(3) is irrational.",
  [
    {
      op: "Set up the contradiction",
      eq: "\\text{Assume } \\log_2 3 = \\tfrac{p}{q} \\text{ where } p, q \\in \\mathbb{N} \\text{ s
    },
    {
      op: "Use the definition of logarithm",
      eq: "\\log_2 3 = \\tfrac{p}{q} \\implies 2^{p/q} = 3 \\implies 2^p = 3^q"
    },
    {
      op: "Apply the Fundamental Theorem of Arithmetic",
      eq: "\\text{Left side: only prime factor is } 2. \\quad \\text{Right side: only prime factor i
    },
    {
      op: "Derive the contradiction",
      eq: "2^p = 3^q \\text{ is impossible for positive integers } p, q \\text{ (distinct prime base
    },
    {
      op: "Conclude",
```

```
      eq: "\\text{The assumption that } \\log_2 3 \\in \\mathbb{Q} \\text{ leads to a contradicti
    }
  ]
)
```

---

**Exercise 3.** Prove by induction that $n! > 2^n$ for all $n \geq 4$.

*Instructions.* (a) Check the base case $n = 4$ by computing both sides. (b) Write out the inductive step: assuming $k! > 2^k$, show $(k+1)! > 2^{k+1}$.

```
//| echo: false

makeStepperHTML(
  "Prove n! > 2ⁿ for all n ⍰ 4.",
  [
    {
      op: "Base case: n = 4",
      eq: "4! = 24, \\quad 2^4 = 16, \\quad 24 > 16. \\checkmark"
    },
    {
      op: "State the inductive hypothesis",
      eq: "\\text{Assume } k! > 2^k \\text{ for some fixed } k \\geq 4."
    },
    {
      op: "Expand the left side of the target",
      eq: "(k+1)! = (k+1) \\cdot k!"
    },
    {
      op: "Apply the inductive hypothesis",
      eq: "(k+1)! = (k+1) \\cdot k! > (k+1) \\cdot 2^k"
    },
    {
      op: "Compare with the target 2^(k+1)",
      eq: "\\text{Need: } (k+1) \\cdot 2^k > 2^{k+1} = 2 \\cdot 2^k, \\text{ i.e., } k+1 > 2. \\t
    },
    {
      op: "Conclude by induction",
      eq: "(k+1)! > (k+1) \\cdot 2^k > 2 \\cdot 2^k = 2^{k+1}. \\quad \\therefore\\ n! > 2^n \\ \
    }
  ]
)
```

---

**Exercise 4.** Find the flaw in the following "proof" that all horses are the same colour.

**Claim.** In any group of $n$ horses, all horses are the same colour.

**Proof.**

*Base case.* When $n = 1$: a group of one horse trivially has all horses the same colour (there is only one). ☒

*Inductive step.* Assume any group of $k$ horses is the same colour. Consider a group of $k + 1$ horses: $H_1, H_2, \dots, H_k, H_{k+1}$.

Look at the first $k$ horses $\{H_1, \dots, H_k\}$. By the inductive hypothesis, they are all the same colour.

Now look at the last $k$ horses $\{H_2, \dots, H_{k+1}\}$. By the inductive hypothesis, they are all the same colour.

These two groups overlap (they share $H_2, \dots, H_k$), so all $k + 1$ horses must be the same colour.

By induction, all horses in any group are the same colour. □

The base case is correct. The logic in the inductive step sounds plausible. But the conclusion is obviously false. Where does the proof break?

*Hint.* What happens to the "overlap" argument when $k = 1$?

```
//| echo: false

makeStepperHTML(
  "Find the flaw in the horse-colour induction.",
  [
    {
      op: "Accept the base case",
      eq: "\\text{For } n = 1 \\text{: trivially true. No flaw here.}"
    },
    {
      op: "Examine the inductive step at k = 1 (the step from n=1 to n=2)",
      eq: "\\text{Groups: } \\{H_1\\} \\text{ and } \\{H_2\\}. \\text{ The claimed overlap is } \\{H
    },
    {
      op: "Identify the flaw: the overlap is empty",
      eq: "\\text{Indices from } 2 \\text{ to } k = 1 \\text{ form an empty range.} \\quad |\\{H_2,
    },
    {
      op: "State why the argument fails",
      eq: "\\text{An empty overlap tells us nothing: } H_1 \\text{ and } H_2 \\text{ need not share
    },
    {
      op: "Conclude",
      eq: "\\text{The step } n=1 \\to n=2 \\text{ is not covered. The induction never leaves } n=1.
    }
  ]
)
```

---

**Exercise 5.** *(Harder.)* Prove that between any two distinct real numbers, there exists a rational number.

More precisely: prove that for any $a, b \in \mathbb{R}$ with $a < b$, there exists $q \in \mathbb{Q}$ with $a < q < b$.

*Hint.* You will need the **Archimedean property**: for any real number $x > 0$, there exists a natural number $n$ such that $n > x$. Use it to find an integer $n$ large enough that $1/n < b - a$. Then find an integer $m$ such that $m/n$ falls in the interval $(a, b)$.

```
//| echo: false

makeStepperHTML(
  "Prove that between any two real numbers a < b, there is a rational.",
  [
    {
      op: "Find a denominator n such that 1/n is smaller than the gap",
      eq: "b - a > 0. \\text{ By the Archimedean property, } \\exists\\, n \\in \\mathbb{N} \\tex
    },
    {
      op: "Choose numerator m: smallest integer with m > na",
      eq: "\\text{Let } m = \\lfloor na \\rfloor + 1, \\text{ so } m > na, \\text{ i.e., } \\tfra
    },
    {
      op: "Show m/n < b",
      eq: "m \\leq na + 1 \\implies \\tfrac{m}{n} \\leq a + \\tfrac{1}{n} < a + (b - a) = b."
    },
    {
      op: "Conclude",
      eq: "a < \\tfrac{m}{n} < b, \\quad \\tfrac{m}{n} \\in \\mathbb{Q}. \\quad \\therefore \\tex
    }
  ]
)
```

# Part VI.

# Volume 6 — Probability and data

# 30. Volume 6 — Probability and data

The central move of this volume is quantifying uncertainty.

Often, you won't have the exact formula or the complete set of facts. This volume provides the tools for operating in those gray areas where outcomes are determined by chance or obscured by noise.

By the end of this volume, you will learn how to formally measure the likelihood of future events, recognize the natural shapes that variation takes, draw robust conclusions from limited samples, and separate true signals from random fluctuations.

## 30.1. Chapters

| Chapter | Title | Core move |
|---|---|---|
| 1 | Probability theory | Modeling unpredictable events |
| 2 | Distributions | Recognizing shapes of variation |
| 3 | Statistical inference | Drawing conclusions from valid samples |
| 4 | Data analysis | Finding signals in noise |

# 31. Probability theory

Reasoning precisely about uncertainty

A medical test is 99% accurate. You test positive. How worried should you be?

Most people say: very — 99% accurate means there's only a 1% chance this is wrong. But the actual answer depends almost entirely on something the test result doesn't tell you: how common the disease is in the first place. If only one person in a thousand has the disease, a positive result is more likely to be a false alarm than a true positive, even with a 99% accurate test. The maths will show you exactly why — and it's not complicated once you have the right framework.

Here's a second trap. You toss a fair coin five times and get five heads in a row. What's the probability the next toss is tails — higher than usual, to "balance things out"? No. It's still 50%. The coin has no memory. Each toss is independent of every toss before it. The belief that a run of heads makes tails "overdue" is called the gambler's fallacy, and it has emptied a great many wallets.

Third trap. Two events each have probability 0.5. What's the probability they both happen? Surely 0.25 — half of half? Only if the events have nothing to do with each other. If knowing that one happened changes the probability of the other, the calculation is different.

Probability has real structure. Intuition is not enough to navigate it. This chapter gives you the rules that make the structure precise — and with those rules, the three problems above all have clean, computable answers.

## 31.1. What the notation is saying

### 31.1.1. Sample spaces and events

Before you can write $P(\text{anything})$, you need to know what the full set of possibilities is.

The **sample space** $\Omega$ (the capital Greek letter omega) is the set of all possible outcomes of an experiment. Roll a standard die: $\Omega = \{1, 2, 3, 4, 5, 6\}$. Toss a coin: $\Omega = \{\text{H}, \text{T}\}$. These are the complete lists — every outcome that could happen.

An **event** is any subset of the sample space — any collection of outcomes you're interested in. "Roll an even number" is the event $\{2, 4, 6\}$. "Roll a number greater than 4" is $\{5, 6\}$. Events are usually labelled with capital letters: $A$, $B$, $C$.

Now the five symbols:

$P(A)$ — read "the probability of $A$." This is a number between 0 and 1 that measures how likely event $A$ is. $P(A) = 0$ means impossible; $P(A) = 1$ means certain.

*Example:* Let $A$ = "roll a 3." On a fair die, $P(A) = \frac{1}{6}$.

$P(A^c)$ — read "the probability of the complement of $A$." The complement $A^c$ (also written $\bar{A}$ or $A'$) is the event "A does not happen" — every outcome in $\Omega$ that is not in $A$.

*Example:* Let $A$ = "roll a 3." Then $A^c$ = "roll anything but 3" = $\{1, 2, 4, 5, 6\}$. Since one of the two must happen: $P(A^c) = \frac{5}{6}$.

$P(A \cup B)$ — read "the probability of $A$ union $B$." The union $A \cup B$ is the event "$A$ or $B$ or both" — any outcome in at least one of $A$ and $B$.

*Example:* Let $A$ = "roll an even number" = $\{2, 4, 6\}$ and $B$ = "roll a number greater than 4" = $\{5, 6\}$. Then $A \cup B = \{2, 4, 5, 6\}$, so $P(A \cup B) = \frac{4}{6} = \frac{2}{3}$.

$P(A \cap B)$ — read "the probability of $A$ intersection $B$." The intersection $A \cap B$ is the event "both $A$ and $B$" — only the outcomes that are in $A$ and also in $B$.

*Example:* With $A = \{2, 4, 6\}$ and $B = \{5, 6\}$: $A \cap B = \{6\}$, so $P(A \cap B) = \frac{1}{6}$.

$P(A \mid B)$ — read "the probability of $A$ given $B$." This is the conditional probability — the probability that $A$ happens, given that you already know $B$ happened. The vertical bar is read "given."

*Example:* You roll a die and someone tells you the result was greater than 4 (so $B$ happened, and you know the outcome is in $\{5, 6\}$). What's the probability it was a 6? You're now working in a smaller world — only two outcomes are possible, and one of them is 6. So $P(\text{roll } 6 \mid B) = \frac{1}{2}$.

Those five symbols are the entire vocabulary you need. Everything else in this chapter is built from combinations of them. This means probability notation is not a collection of separate tricks. It is a small language for describing how events relate to each other.

Use the visual below to treat the sample space as a real geometric object, not just a list of symbols.

```
//| echo: false
{
  const W = 560, H = 250;
  const rect = { x: 50, y: 34, w: 360, h: 150 };
  let rx = 90;
  let dragging = false;

  const clamp = (v, a, b) => Math.max(a, Math.min(b, v));
  const ellipseArea = (a, b) => Math.PI * a * b;

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  const container = document.createElement("div");
  container.style.cssText = "max-width: 620px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

  const intro = document.createElement("p");
  intro.style.cssText = "font-size: 0.92rem; color: #374151; margin: 0 0 0.55rem 0;";
  intro.textContent = "Drag the handle on the boundary of A to change its size inside the full sampl
```

```
const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.55rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
  k.textContent = label;
  const v = document.createElement("div");
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const pABox = card("P(A)");
const pAcBox = card("P(A^c)");
const sumBox = card("Check");

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:white; border:1px solid #e5e7eb; border-radius:1

const bg = svgEl("rect");
bg.setAttribute("x", "0");
bg.setAttribute("y", "0");
bg.setAttribute("width", W);
bg.setAttribute("height", H);
bg.setAttribute("fill", "#ffffff");
svg.appendChild(bg);

const omega = svgEl("rect");
omega.setAttribute("x", rect.x);
omega.setAttribute("y", rect.y);
omega.setAttribute("width", rect.w);
omega.setAttribute("height", rect.h);
omega.setAttribute("fill", "#f3f4f6");
omega.setAttribute("stroke", "#9ca3af");
omega.setAttribute("stroke-width", "2");
svg.appendChild(omega);

const omegaLabel = svgEl("text");
omegaLabel.setAttribute("x", rect.x + rect.w - 18);
omegaLabel.setAttribute("y", rect.y + 18);
omegaLabel.setAttribute("font-size", "16");
omegaLabel.setAttribute("font-weight", "700");
omegaLabel.setAttribute("fill", "#6b7280");
```

```
omegaLabel.textContent = "Ω";
svg.appendChild(omegaLabel);

const event = svgEl("ellipse");
event.setAttribute("fill", "rgba(37,99,235,0.35)");
event.setAttribute("stroke", "#2563eb");
event.setAttribute("stroke-width", "2");
svg.appendChild(event);

const aLabel = svgEl("text");
aLabel.setAttribute("font-size", "16");
aLabel.setAttribute("font-weight", "700");
aLabel.setAttribute("fill", "#1d4ed8");
aLabel.textContent = "A";
svg.appendChild(aLabel);

const handle = svgEl("circle");
handle.setAttribute("r", "6");
handle.setAttribute("fill", "#1d4ed8");
handle.style.cursor = "ew-resize";
svg.appendChild(handle);

const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function redraw() {
  const ry = clamp(0.62 * rx, 28, 64);
  const cx = rect.x + rect.w / 2;
  const cy = rect.y + rect.h / 2;
  event.setAttribute("cx", cx);
  event.setAttribute("cy", cy);
  event.setAttribute("rx", rx);
  event.setAttribute("ry", ry);
  aLabel.setAttribute("x", cx - 5);
  aLabel.setAttribute("y", cy + 5);
  handle.setAttribute("cx", cx + rx);
  handle.setAttribute("cy", cy);

  const pA = ellipseArea(rx, ry) / (rect.w * rect.h);
  const pAc = 1 - pA;
  pABox.textContent = pA.toFixed(2);
  pAcBox.textContent = pAc.toFixed(2);
  sumBox.textContent = `${pA.toFixed(2)} + ${pAc.toFixed(2)} = 1.00`;
  caption.textContent = "The rectangle is the full sample space Ω. Event A is one region inside it
}

svg.addEventListener("pointerdown", e => {
```

```
    const pt = svg.createSVGPoint();
    pt.x = e.clientX; pt.y = e.clientY;
    const p = pt.matrixTransform(svg.getScreenCTM().inverse());
    const hx = parseFloat(handle.getAttribute("cx"));
    const hy = parseFloat(handle.getAttribute("cy"));
    if (Math.hypot(p.x - hx, p.y - hy) < 14) dragging = true;
  });
  svg.addEventListener("pointermove", e => {
    if (!dragging) return;
    const pt = svg.createSVGPoint();
    pt.x = e.clientX; pt.y = e.clientY;
    const p = pt.matrixTransform(svg.getScreenCTM().inverse());
    rx = clamp(p.x - (rect.x + rect.w / 2), 28, rect.w / 2 - 12);
    redraw();
  });
  window.addEventListener("pointerup", () => { dragging = false; });

  container.append(intro, readout, svg, caption);
  redraw();
  return container;
}
```

## 31.2. The rules

### 31.2.1. Rule 1: Complement

$$P(A^c) = 1 - P(A)$$

Read it as: the probability that $A$ does not happen equals one minus the probability that it does.

This is simple, but it earns its place in every calculation where it's easier to ask "what are the chances this doesn't happen?" than "what are the chances it does?" — which turns out to be surprisingly often. This means complements are often the quickest route when the event you want is hard to count directly.

*Example:* What's the probability of rolling at least one six when you roll a die four times? Computing directly means listing every combination that includes at least one six. Computing via the complement means one step: $P(\text{at least one six}) = 1 - P(\text{no sixes}) = 1 - (5/6)^4 \approx 1 - 0.482 = 0.518$ (we're getting ahead slightly — since each roll is independent, P(no six on all four rolls) = (5/6)×(5/6)×(5/6)×(5/6) = $(5/6)^4$; we'll see this rule formally in Rule 4 below).

### 31.2.2. Rule 2: Addition rule

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Read it as: the probability of $A$ or $B$ equals the probability of $A$ plus the probability of $B$, minus the probability of both.

Why subtract the intersection? Because if you add $P(A)$ and $P(B)$ separately, any outcome that falls in both $A$ and $B$ gets counted twice — once for $A$ and once for $B$. Subtracting $P(A \cap B)$ corrects that double-count. This means the addition rule is really an accounting rule: include both events, then remove the overlap once.

*Example:* Back to the die. $P(\text{even}) = \frac{3}{6}$, $P(\text{greater than } 4) = \frac{2}{6}$, $P(\text{even and greater than } 4) = P(\{6\}) = \frac{1}{6}$.

$$P(\text{even or greater than } 4) = \frac{3}{6} + \frac{2}{6} - \frac{1}{6} = \frac{4}{6} = \frac{2}{3}$$

**Special case — mutually exclusive events:** If $A$ and $B$ cannot both happen at the same time — if $A \cap B$ is empty — then $P(A \cap B) = 0$, and the rule simplifies to:

$$P(A \cup B) = P(A) + P(B) \qquad \text{(mutually exclusive only)}$$

Rolling a 2 and rolling a 5 on the same single die toss are mutually exclusive. Rolling an even and rolling a 6 are not — 6 satisfies both.

### 31.2.3. Rule 3: Conditional probability

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Read it as: the probability of $A$ given $B$ equals the probability of both divided by the probability of $B$.

The intuition: when you learn that $B$ happened, you've narrowed down the sample space. You're no longer considering all of $\Omega$ — only the part of $\Omega$ where $B$ is true. Within that restricted space, how often does $A$ also happen? That fraction is $P(A \mid B)$. This means conditional probability is the same probability idea applied inside a smaller universe.

The board below makes that "smaller universe" idea literal.

```
//| echo: false
{
  const W = 600, H = 280;
  const frame = { x: 40, y: 38, w: 420, h: 160 };
  let bStart = 250;
  let aStart = 150;
  let conditioned = false;

  const clamp = (v, a, b) => Math.max(a, Math.min(b, v));

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  const container = document.createElement("div");
```

```
container.style.cssText = "max-width: 640px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.55rem;";
function makeSlider(labelText, min, max, value) {
  const wrap = document.createElement("label");
  wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  const text = document.createElement("span");
  text.textContent = labelText;
  const input = document.createElement("input");
  input.type = "range";
  input.min = String(min);
  input.max = String(max);
  input.step = "1";
  input.value = String(value);
  wrap.append(text, input);
  controls.appendChild(wrap);
  return input;
}
const aSlider = makeSlider("Shift A", 70, 230, aStart);
const bSlider = makeSlider("Shift B", 180, 320, bStart);

const buttonRow = document.createElement("div");
buttonRow.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.55rem;
const toggle = document.createElement("button");
toggle.textContent = "Condition on B";
toggle.style.cssText = "padding:0.45rem 0.85rem; border:1px solid #d1d5db; border-radius:8px; b
const reset = document.createElement("button");
reset.textContent = "Return to ";
reset.style.cssText = "padding:0.45rem 0.85rem; border:1px solid #d1d5db; border-radius:8px; ba
buttonRow.append(toggle, reset);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.55rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
  k.textContent = label;
  const v = document.createElement("div");
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const pBBox = card("P(B)");
```

```
const pABBox = card("P(A∩B)");
const pCondBox = card("P(A|B)");

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:white; border:1px solid #e5e7eb; border-radius:10px

const bg = svgEl("rect");
bg.setAttribute("x", "0");
bg.setAttribute("y", "0");
bg.setAttribute("width", W);
bg.setAttribute("height", H);
bg.setAttribute("fill", "#ffffff");
svg.appendChild(bg);

const omega = svgEl("rect");
omega.setAttribute("fill", "#f3f4f6");
omega.setAttribute("stroke", "#9ca3af");
omega.setAttribute("stroke-width", "2");
svg.appendChild(omega);

const rectA = svgEl("rect");
rectA.setAttribute("fill", "rgba(37,99,235,0.35)");
rectA.setAttribute("stroke", "#2563eb");
rectA.setAttribute("stroke-width", "2");
svg.appendChild(rectA);

const rectB = svgEl("rect");
rectB.setAttribute("fill", "rgba(245,158,11,0.28)");
rectB.setAttribute("stroke", "#d97706");
rectB.setAttribute("stroke-width", "2");
svg.appendChild(rectB);

const overlap = svgEl("rect");
overlap.setAttribute("fill", "rgba(20,184,166,0.55)");
overlap.setAttribute("stroke", "#0f766e");
overlap.setAttribute("stroke-width", "1.5");
svg.appendChild(overlap);

const omegaLabel = svgEl("text");
omegaLabel.setAttribute("font-size", "16");
omegaLabel.setAttribute("font-weight", "700");
omegaLabel.setAttribute("fill", "#6b7280");
omegaLabel.textContent = "Ω";
svg.appendChild(omegaLabel);
```

```
const aLabel = svgEl("text");
aLabel.setAttribute("font-size", "15");
aLabel.setAttribute("font-weight", "700");
aLabel.setAttribute("fill", "#1d4ed8");
aLabel.textContent = "A";
svg.appendChild(aLabel);

const bLabel = svgEl("text");
bLabel.setAttribute("font-size", "15");
bLabel.setAttribute("font-weight", "700");
bLabel.setAttribute("fill", "#b45309");
bLabel.textContent = "B";
svg.appendChild(bLabel);

const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function redraw() {
  aStart = parseFloat(aSlider.value);
  bStart = parseFloat(bSlider.value);
  const aWidth = 170;
  const bWidth = 170;
  const ax = aStart;
  const bx = bStart;
  const overlapX = Math.max(ax, bx);
  const overlapW = Math.max(0, Math.min(ax + aWidth, bx + bWidth) - overlapX);

  const viewX = conditioned ? bx : frame.x;
  const viewW = conditioned ? bWidth : frame.w;
  const scale = frame.w / viewW;

  omega.setAttribute("x", frame.x);
  omega.setAttribute("y", frame.y);
  omega.setAttribute("width", frame.w);
  omega.setAttribute("height", frame.h);
  omega.setAttribute("opacity", conditioned ? "0.18" : "1");

  const projectX = x => frame.x + (x - viewX) * scale;
  const projectW = w => w * scale;

  rectA.setAttribute("x", projectX(ax));
  rectA.setAttribute("y", frame.y + 22);
  rectA.setAttribute("width", projectW(aWidth));
  rectA.setAttribute("height", frame.h - 44);
  rectA.setAttribute("opacity", conditioned ? "0.25" : "1");

  rectB.setAttribute("x", projectX(bx));
```

```
    rectB.setAttribute("y", frame.y + 14);
    rectB.setAttribute("width", projectW(bWidth));
    rectB.setAttribute("height", frame.h - 28);
    rectB.setAttribute("opacity", "1");

    overlap.setAttribute("x", projectX(overlapX));
    overlap.setAttribute("y", frame.y + 22);
    overlap.setAttribute("width", projectW(overlapW));
    overlap.setAttribute("height", frame.h - 44);
    overlap.setAttribute("opacity", overlapW > 0 ? "1" : "0");

    omegaLabel.setAttribute("x", frame.x + frame.w - 18);
    omegaLabel.setAttribute("y", frame.y + 18);
    omegaLabel.setAttribute("opacity", conditioned ? "0" : "1");
    aLabel.setAttribute("x", projectX(ax) + 12);
    aLabel.setAttribute("y", frame.y + 38);
    bLabel.setAttribute("x", projectX(bx) + 12);
    bLabel.setAttribute("y", frame.y + 30);

    const pB = bWidth / frame.w;
    const pAB = overlapW / frame.w;
    const pCond = overlapW / bWidth;
    pBBox.textContent = pB.toFixed(2);
    pABBox.textContent = pAB.toFixed(2);
    pCondBox.textContent = `${pAB.toFixed(2)} / ${pB.toFixed(2)} = ${pCond.toFixed(2)}`;
    caption.textContent = conditioned
      ? "Conditioning on B means treating B as the whole surviving universe. The overlap A∩B becomes
      : "Before conditioning, Ω is the whole universe. A∩B is just the overlap where both events hap

    toggle.disabled = conditioned;
    reset.disabled = !conditioned;
  }

  aSlider.addEventListener("input", redraw);
  bSlider.addEventListener("input", redraw);
  toggle.addEventListener("click", () => { conditioned = true; redraw(); });
  reset.addEventListener("click", () => { conditioned = false; redraw(); });

  container.append(controls, buttonRow, readout, svg, caption);
  redraw();
  return container;
}
```

A concrete picture makes this clearer than any formula. Imagine a group of 1000 people:

| | Disease positive | Disease negative | Total |
|---|---|---|---|
| **Test positive** | 95 | 49 | 144 |
| **Test negative** | 5 | 851 | 856 |
| **Total** | 100 | 900 | 1000 |

Here disease prevalence is 10% (100 of 1000 have the disease), test sensitivity is 95% (catches 95 of the 100 cases), and specificity (the probability of a negative result given no disease) is roughly 94.6% (misses 49 of 900 healthy people).

If you test positive, you're in the top row: 144 people. Of those 144, only 95 actually have the disease. $P(\text{disease} \mid \text{positive}) = \frac{95}{144} \approx 0.66$.

But what happens when the disease is rarer? That's what the interactive below explores — and the numbers become surprising.

```
//| echo: false
{
  // Sliders
  const popSlider = Inputs.range([100, 10000], {value: 1000, step: 100, label: "Population size"}
  const prevSlider = Inputs.range([0.1, 20], {value: 10, step: 0.1, label: "Disease prevalence (%
  const sensSlider = Inputs.range([50, 99.9], {value: 95, step: 0.1, label: "Test sensitivity (%)
  const specSlider = Inputs.range([50, 99.9], {value: 94.6, step: 0.1, label: "Test specificity (

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit; max-width:640px;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; gap:0.5rem; margin-bottom:1.25rem;";
  controls.appendChild(popSlider);
  controls.appendChild(prevSlider);
  controls.appendChild(sensSlider);
  controls.appendChild(specSlider);

  const tableWrap = document.createElement("div");
  tableWrap.style.cssText = "margin-bottom:1rem;";

  const ppvDisplay = document.createElement("div");
  ppvDisplay.style.cssText = "font-size:1.35rem; font-weight:600; margin-bottom:0.5rem; padding:0

  const noteEl = document.createElement("div");
  noteEl.style.cssText = "font-size:0.88em; color:#b45309; background:#fffbeb; border:1px solid #
  noteEl.textContent = "Low prevalence warning: when the disease is rare, even a good test produc

  function update() {
    const N   = popSlider.value;
    const prev = prevSlider.value / 100;
    const sens = sensSlider.value / 100;
```

```
  const spec = specSlider.value / 100;

  const diseased   = Math.round(N * prev);
  const healthy    = N - diseased;
  const tp = Math.round(diseased * sens);
  const fn = diseased - tp;
  const tn = Math.round(healthy * spec);
  const fp = healthy - tn;

  const testPos = tp + fp;
  const testNeg = fn + tn;
  const ppv = testPos > 0 ? tp / testPos : 0;

  ppvDisplay.textContent = `P(disease | positive test) = ${(ppv * 100).toFixed(1)}%  (${tp} true p
  noteEl.style.display = prev < 0.05 ? "block" : "none";

  tableWrap.innerHTML = "";
  const tbl = document.createElement("table");
  tbl.style.cssText = "border-collapse:collapse; width:100%; font-size:0.92em;";

  const headerRow = tbl.insertRow();
  ["", "Test positive", "Test negative", "Total"].forEach((h, i) => {
    const th = document.createElement("th");
    th.textContent = h;
    th.style.cssText = "padding:0.4rem 0.75rem; border:1px solid #d1d5db; background:#f9fafb; text
    headerRow.appendChild(th);
  });

  const rows = [
    ["Disease positive", tp, fn, diseased],
    ["Disease negative", fp, tn, healthy],
    ["Total", testPos, testNeg, N],
  ];
  rows.forEach((r, ri) => {
    const tr = tbl.insertRow();
    r.forEach((cell, ci) => {
      const td = tr.insertCell();
      td.textContent = cell;
      const isTp = ri === 0 && ci === 1;
      td.style.cssText = "padding:0.4rem 0.75rem; border:1px solid #d1d5db; text-align:" + (ci ===
    });
  });

  tableWrap.appendChild(tbl);
}

popSlider.addEventListener("input", update);
```

```
  prevSlider.addEventListener("input", update);
  sensSlider.addEventListener("input", update);
  specSlider.addEventListener("input", update);

  update();

  container.appendChild(controls);
  container.appendChild(ppvDisplay);
  container.appendChild(tableWrap);
  container.appendChild(noteEl);
  return container;
}
```

### 31.2.4. Rule 4: Multiplication rule

$$P(A \cap B) = P(A \mid B) \cdot P(B)$$

This is just the conditional probability definition rearranged. Solve for $P(A \cap B)$ and you get: to find the probability that both $A$ and $B$ happen, multiply the probability that $B$ happens by the probability that $A$ happens given $B$ already happened. This means the multiplication rule builds a joint event step by step.

*Example:* A bag has 5 red and 3 blue marbles. You draw one without looking, then draw a second without replacing the first. What's the probability both are red?

- $P(\text{first is red}) = \frac{5}{8}$
- $P(\text{second is red} \mid \text{first was red}) = \frac{4}{7}$ (only 4 red left, only 7 total left)
- $P(\text{both red}) = \frac{5}{8} \times \frac{4}{7} = \frac{20}{56} = \frac{5}{14}$

### 31.2.5. Rule 5: Independence

Events $A$ and $B$ are **independent** if:

$$P(A \mid B) = P(A)$$

Read it as: knowing that $B$ happened does not change the probability of $A$ at all. $B$ carries no information about $A$.

An equivalent (and often more useful) form comes directly from substituting into the multiplication rule:

$$P(A \cap B) = P(A) \cdot P(B) \qquad \text{(independent events only)}$$

This is the definition of independence — not something you prove from intuition, but a precise property you can check. Two coin tosses are independent. Drawing a card from a shuffled deck and then rolling a die are independent. Drawing two cards without replacement are not — what you get first changes

what's available for the second. This means independence is a strong claim: learning one event gives you no information at all about the other.

The gambler's fallacy is exactly the mistake of believing that independent events are somehow dependent — that after five heads, the coin "owes" you a tail. The coin's next outcome has probability $\frac{1}{2}$ regardless of any previous results, because each toss is independent of every other.

```
//| echo: false
{
  const pASlider = Inputs.range([0.01, 0.99], {value: 0.5, step: 0.01, label: "P(A)"});
  const pBSlider = Inputs.range([0.01, 0.99], {value: 0.5, step: 0.01, label: "P(B)"});
  const corrSelect = Inputs.radio(["Independent", "Positively correlated", "Negatively correlated"],

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit; max-width:640px;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; gap:0.5rem; margin-bottom:1.25rem;";
  controls.appendChild(pASlider);
  controls.appendChild(pBSlider);
  controls.appendChild(corrSelect);

  const summary = document.createElement("div");
  summary.style.cssText = "margin-bottom:1rem;";

  const gridWrap = document.createElement("div");
  gridWrap.style.cssText = "margin-bottom:0.5rem;";

  function clamp(v, lo, hi) { return Math.max(lo, Math.min(hi, v)); }

  function jointProbs(pA, pB, mode) {
    let pAB;
    if (mode === "Independent") {
      pAB = pA * pB;
    } else if (mode === "Positively correlated") {
      const maxAB = Math.min(pA, pB);
      pAB = clamp(pA * pB + 0.15 * Math.min(pA, pB, 1 - pA, 1 - pB), 0, maxAB);
    } else {
      const minAB = Math.max(0, pA + pB - 1);
      pAB = clamp(pA * pB - 0.15 * Math.min(pA, pB, 1 - pA, 1 - pB), minAB, Math.min(pA, pB));
    }
    return {
      pAB:      pAB,
      pAnotB:   pA - pAB,
      pBnotA:   pB - pAB,
      pNeither: 1 - pA - pB + pAB,
    };
  }
```

```
function fmtPct(v) { return (v * 100).toFixed(1) + "%"; }

function renderGrid(pA, pB, mode) {
  const j = jointProbs(pA, pB, mode);
  const indepAB = pA * pB;

  summary.innerHTML = "";
  const cases = [
    { label: "Independent",            pab: indepAB },
    { label: "Positively correlated",  pab: jointProbs(pA, pB, "Positively correlated").pAB },
    { label: "Negatively correlated",  pab: jointProbs(pA, pB, "Negatively correlated").pAB },
  ];
  const sumDiv = document.createElement("div");
  sumDiv.style.cssText = "display:grid; grid-template-columns:repeat(3,1fr); gap:0.5rem; margin
  cases.forEach(c => {
    const card = document.createElement("div");
    const isActive = c.label === mode;
    card.style.cssText = "border:1px solid " + (isActive ? "#3b82f6" : "#e5e7eb") + "; border-r
    card.innerHTML = `<div style="font-size:0.8em;color:#6b7280;">${c.label}</div><div style="f
    sumDiv.appendChild(card);
  });
  summary.appendChild(sumDiv);

  gridWrap.innerHTML = "";
  const tbl = document.createElement("table");
  tbl.style.cssText = "border-collapse:collapse; width:100%; font-size:0.9em;";

  const hdr = tbl.insertRow();
  ["", "B occurs", "B does not occur", "Total"].forEach((h, i) => {
    const th = document.createElement("th");
    th.textContent = h;
    th.style.cssText = "padding:0.4rem 0.7rem; border:1px solid #d1d5db; background:#f9fafb; te
    hdr.appendChild(th);
  });

  const dataRows = [
    ["A occurs",         j.pAB,      j.pAnotB,   pA],
    ["A does not occur", j.pBnotA,   j.pNeither, 1 - pA],
    ["Total",            pB,         1 - pB,     1.0],
  ];
  dataRows.forEach(r => {
    const tr = tbl.insertRow();
    r.forEach((cell, ci) => {
      const td = tr.insertCell();
      td.textContent = ci === 0 ? cell : fmtPct(cell);
      td.style.cssText = "padding:0.4rem 0.7rem; border:1px solid #d1d5db; text-align:" + (ci =
    });
```

```
    });

    gridWrap.appendChild(tbl);

    const note = document.createElement("div");
    note.style.cssText = "font-size:0.85em; color:#6b7280; margin-top:0.5rem;";
    const diff = j.pAB - indepAB;
    if (Math.abs(diff) < 0.001) {
      note.textContent = "The joint probability equals P(A) × P(B) = " + fmtPct(indepAB) + " — the c
    } else if (diff > 0) {
      note.textContent = "P(A∩B) = " + fmtPct(j.pAB) + " > P(A)×P(B) = " + fmtPct(indepAB) + " — kno
    } else {
      note.textContent = "P(A∩B) = " + fmtPct(j.pAB) + " < P(A)×P(B) = " + fmtPct(indepAB) + " — kno
    }
    gridWrap.appendChild(note);
  }

  function update() {
    renderGrid(pASlider.value, pBSlider.value, corrSelect.value);
  }

  pASlider.addEventListener("input", update);
  pBSlider.addEventListener("input", update);
  corrSelect.addEventListener("change", update);

  update();

  container.appendChild(controls);
  container.appendChild(summary);
  container.appendChild(gridWrap);
  return container;
}
```

### 31.2.6. Bayes' theorem

Everything so far has been building to this. Start with the multiplication rule written two ways:

$$P(A \cap B) = P(A \mid B) \cdot P(B)$$
$$P(A \cap B) = P(B \mid A) \cdot P(A)$$

The left-hand sides are equal, so the right-hand sides are too. Divide both sides by $P(B)$:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

This is **Bayes' theorem**. Read each part:

- $P(A)$ — the **prior**: what you believed about $A$ before any new information.
- $P(B \mid A)$ — the **likelihood**: how probable was this new evidence $B$, assuming $A$ is true?
- $P(A \mid B)$ — the **posterior**: what you now believe about $A$, having seen $B$.
- $P(B)$ — the **normalising constant**: the total probability of seeing $B$, regardless of whether $A$ is true or not.

The theorem tells you how to update a belief in the light of new evidence. That is, precisely, what reasoning under uncertainty requires. This means Bayes' theorem converts "how likely is the evidence if the cause is true?" into "how likely is the cause after seeing the evidence?"

Now the medical test problem from the opening has a clean solution.

## 31.3. Worked examples

### 31.3.1. Example 1 — Medical testing (sciences)

A disease affects 1% of the population. A test for it has sensitivity 95% (correctly identifies 95% of people who have the disease) and specificity 95% (correctly identifies 95% of people who don't have the disease).

You test positive. What is the probability you actually have the disease?

**Set up the events:**

- $A$ = "you have the disease"
- $B$ = "you test positive"

**What you know:**

- $P(A) = 0.01$ — disease prevalence (the prior)
- $P(B \mid A) = 0.95$ — sensitivity (probability of a positive test given disease)
- $P(B \mid A^c) = 0.05$ — false positive rate (probability of a positive test given no disease; this is $1 -$ specificity)

**Find** $P(B)$ — to find P(positive test) we use the **total probability law**: since every person either has the disease or doesn't, we can compute P(positive) by considering both groups separately and adding. Split by whether you have the disease or not:

$$P(B) = P(B \mid A) \cdot P(A) + P(B \mid A^c) \cdot P(A^c)$$

$$P(B) = (0.95)(0.01) + (0.05)(0.99) = 0.0095 + 0.0495 = 0.059$$

**Apply Bayes' theorem:**

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} = \frac{0.95 \times 0.01}{0.059} = \frac{0.0095}{0.059} \approx 0.161$$

**The result:** If you test positive for a disease that affects 1% of the population, even with a 95% accurate test, the probability you actually have it is about 16%.

Why so low? Because the disease is rare. In a population of 10 000 people: 100 have the disease and 9900 do not. The test correctly flags about 95 of the 100 sick people. But it also incorrectly flags about 495 of the 9900 healthy people. Of the 590 positive results in total, only 95 are true positives — about 16%.

A positive result is not a diagnosis. It's information that should prompt further testing. This is why medical screening programmes track *positive predictive value* — the probability a positive test reflects a true positive — and not just test accuracy. This means the base rate can dominate the interpretation of a test result, even when the test itself is very accurate.

---

### 31.3.2. Example 2 — Cards without replacement (finance)

A standard deck has 52 cards, including 4 aces. You draw two cards without replacing the first. What is the probability that both are aces?

**Using the multiplication rule for dependent events:**

$$P(\text{both aces}) = P(\text{first is ace}) \times P(\text{second is ace} \mid \text{first was ace})$$

$$= \frac{4}{52} \times \frac{3}{51} = \frac{12}{2652} = \frac{1}{221} \approx 0.0045$$

About 0.45% — less than half a percent.

Note the dependence: the second draw depends on the first because the first card is not returned to the deck. If you replaced the card between draws (sampling with replacement), the draws would be independent: $P(\text{both aces}) = \frac{4}{52} \times \frac{4}{52} = \frac{16}{2704} \approx 0.0059$ — a slightly higher probability because you haven't reduced the pool of aces.

This distinction between sampling with and without replacement appears constantly in actuarial and financial modelling, where the question is whether one event (e.g. one bond defaulting) affects the probability of another. This means dependence appears whenever the first outcome changes the pool from which the second outcome is drawn.

---

### 31.3.3. Example 3 — Spam filter (computing)

A word appears in 80% of spam emails and in 10% of legitimate emails. Overall, 30% of incoming email is spam.

A new message contains this word. What is the probability it is spam?

**Set up the events:**

- $A$ = "email is spam"
- $B$ = "email contains the word"

**What you know:**

- $P(A) = 0.30$ — prior probability that email is spam
- $P(B \mid A) = 0.80$ — probability the word appears in spam
- $P(B \mid A^c) = 0.10$ — probability the word appears in legitimate email

**Find $P(B)$:**

$$P(B) = P(B \mid A) \cdot P(A) + P(B \mid A^c) \cdot P(A^c)$$

$$= (0.80)(0.30) + (0.10)(0.70) = 0.24 + 0.07 = 0.31$$

**Apply Bayes' theorem:**

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} = \frac{0.80 \times 0.30}{0.31} = \frac{0.24}{0.31} \approx 0.774$$

**The result:** Given that the email contains this word, the probability it is spam is about 77%.

A real spam filter does this for dozens or hundreds of words simultaneously, multiplying the contributions of each word (under an independence assumption that is not strictly true but works well in practice). That extension is called a Naive Bayes classifier — and it was one of the most effective spam filters ever built. This means one feature can shift a probability sharply, but the real power comes from updating on many pieces of evidence at once.

---

### 31.3.4. Example 4 — Weather (geography and environment)

A weather station records the following over a year:

- 20% of days are rainy.
- 40% of days have cloudy mornings.
- Of all rainy days, 70% were preceded by a cloudy morning.

Given that today's morning is cloudy, what is the probability of rain?

**Set up the events:**

- $A$ = "today is rainy"
- $B$ = "this morning is cloudy"

**What you know:**

- $P(A) = 0.20$
- $P(B) = 0.40$
- $P(B \mid A) = 0.70$

**Apply Bayes' theorem directly:**

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} = \frac{0.70 \times 0.20}{0.40} = \frac{0.14}{0.40} = 0.35$$

**The result:** A cloudy morning raises the probability of rain from the baseline 20% to 35%.

Environmental scientists and meteorologists work with probability statements like this constantly — flood return periods, drought likelihoods, and extreme precipitation events are all expressed as conditional probabilities, updated as new observations come in. This means evidence rarely gives certainty; it changes the odds.

## 31.4. Where this goes

This chapter built probability as a system of relationships between events. The next chapter keeps that logic but spreads it across random variables that can take many possible values.

### 31.4.1. Probability distributions

You now have a framework for reasoning about individual events and pairs of events. The next natural question is: what if a random process can produce many different numerical outcomes — not just "heads/tails" or "disease/no disease" but any integer from 0 to 20, or any real number in an interval? How do probabilities distribute across all those possible values?

That question leads directly to **probability distributions** (chapter 2). A distribution is a complete description of the probabilities attached to every possible outcome of a random quantity. The binomial distribution (counting successes in repeated trials), the normal distribution (the bell curve that appears everywhere in nature and statistics), and others all build on the conditional probability and independence machinery you've just learned.

### 31.4.2. Bayesian reasoning in computing and machine learning

The spam filter example above is a stripped-down version of a **Naive Bayes classifier** — one of the fundamental algorithms in machine learning. The same structure extends to any classification problem: given a collection of observed features, compute the probability that the observation belongs to each possible class, and predict the most probable one.

Medical diagnosis, document classification, image labelling, and fraud detection all use variants of Bayesian reasoning. The conditional probability machinery from this chapter is the mathematical core of each of them. What changes when you move to a full ML course is scale (thousands of features, not one word) and the methods for estimating the prior and likelihood from data.

## 31.5. Applications

**Where this chapter turns up in practice**

- **Disease screening programmes** — a test's positive predictive value depends on prevalence, not just accuracy. Screening for rare diseases requires high specificity to avoid swamping clinicians with false positives.
- **Forensic evidence** — DNA evidence in court is often misread. The probability that a match occurs by chance (1 in a billion) is not the same as the probability that the accused is innocent given the match. Getting this wrong is called the prosecutor's fallacy, and it has contributed to wrongful convictions.
- **Spam and content filters** — Naive Bayes classifiers were the first practical spam filters and remain in use today. The same structure underlies content moderation systems.
- **Flood return periods** — a "1-in-100-year flood" means a flood of that size has a 1% probability of occurring in any given year — not that you get one every hundred years and can relax afterward. It can occur two years in a row.

## 31.6. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.5rem 1rem; align-
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
```

```
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSteps
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

### 31.6.1. Exercise 1

The probability that a flight departs on time is 0.78. What is the probability it does not depart on time?

```
//| echo: false
makeStepperHTML(1, [
  { op: "Identify the complement rule", eq: "P(\\text{on time}^c) = 1 - P(\\text{on time})" },
  { op: "Substitute", eq: "P(\\text{not on time}) = 1 - 0.78 = 0.22", note: "The probability the
])
```

---

### 31.6.2. Exercise 2

In a group of students: 45% play a sport, 30% play a musical instrument, and 15% do both. What is the probability that a randomly chosen student plays a sport or a musical instrument (or both)?

```
//| echo: false
makeStepperHTML(2, [
  { op: "Identify the addition rule", eq: "P(A \\cup B) = P(A) + P(B) - P(A \\cap B)" },
  { op: "Label the events", eq: "P(A) = 0.45,\\; P(B) = 0.30,\\; P(A \\cap B) = 0.15", note: "A =
  { op: "Substitute", eq: "P(A \\cup B) = 0.45 + 0.30 - 0.15 = 0.60", note: "The probability that
])
```

---

### 31.6.3. Exercise 3

*This exercise uses the full Bayes machinery — prior, likelihood, total probability, and posterior — as in the medical test example above. Work through each piece in turn.*

In a factory, 3% of parts are defective. A quality inspector catches 90% of defective parts and incorrectly flags 4% of good parts as defective.

A part is flagged by the inspector. What is the probability it is actually defective?

```
//| echo: false
makeStepperHTML(3, [
  { op: "Set up events and knowns", eq: "P(A) = 0.03,\\; P(A^c) = 0.97,\\; P(B \\mid A) = 0.90,\\
  { op: "Total probability for P(B)", eq: "P(B) = (0.90)(0.03) + (0.04)(0.97) = 0.027 + 0.0388 =
  { op: "Apply Bayes' theorem", eq: "P(A \\mid B) = \\frac{0.90 \\times 0.03}{0.0658} = \\frac{0.
])
```

---

### 31.6.4. Exercise 4

A bag contains 6 red and 4 blue counters. Two counters are drawn one at a time without replacement. What is the probability of drawing a red counter followed by a blue counter?

```
//| echo: false
makeStepperHTML(4, [
  { op: "Identify dependent draws", eq: "P(\\text{red then blue}) = P(\\text{first red}) \\times P(
  { op: "First draw probability", eq: "P(\\text{first is red}) = \\frac{6}{10} = \\frac{3}{5}" },
  { op: "Conditional probability of second draw", eq: "P(\\text{second is blue} \\mid \\text{first
  { op: "Multiply", eq: "P(\\text{red then blue}) = \\frac{3}{5} \\times \\frac{4}{9} = \\frac{12}{
])
```

---

### 31.6.5. Exercise 5

A card is drawn at random from a standard 52-card deck.

Event $A$: the card is a heart. $P(A) = \frac{13}{52} = \frac{1}{4}$

Event $B$: the card is a face card (jack, queen, or king). $P(B) = \frac{12}{52} = \frac{3}{13}$

There are 3 face cards that are also hearts (jack, queen, and king of hearts).

Are events $A$ and $B$ independent? Show your reasoning.

```
//| echo: false
makeStepperHTML(5, [
  { op: "State the independence criterion", eq: "\\text{Independent iff } P(A \\cap B) = P(A) \\cdot
  { op: "Find P(A ∩ B)", eq: "P(A \\cap B) = \\frac{3}{52}", note: "A ∩ B = card is a heart and a fa
  { op: "Compute P(A) · P(B)", eq: "P(A) \\cdot P(B) = \\frac{1}{4} \\times \\frac{3}{13} = \\frac{
  { op: "Compare", eq: "P(A \\cap B) = \\frac{3}{52} = P(A) \\cdot P(B) \\implies \\text{independent
])
```

---

### 31.6.6. Exercise 6 — Multi-step problem

A university entrance exam has two papers. Based on historical data:

- 60% of students pass Paper 1.
- Of students who pass Paper 1, 80% also pass Paper 2.
- Of students who fail Paper 1, 25% still pass Paper 2.

To be admitted, a student must pass at least one paper.

(a) What is the probability that a student passes both papers?

(b) What is the probability that a student passes at least one paper?

(c) Given that a student passes Paper 2, what is the probability they also passed Paper 1?

```
//| echo: false
makeStepperHTML(6, [
  { op: "Part (a) — both papers", eq: "P(A \\cap B) = P(B \\mid A) \\cdot P(A) = 0.80 \\times 0.6
  { op: "Part (b) — find P(fails both)", eq: "P(B^c \\cap A^c) = P(B^c \\mid A^c) \\cdot P(A^c) =
  { op: "Part (b) — at least one paper", eq: "P(\\text{at least one}) = 1 - 0.30 = 0.70", note: "
  { op: "Part (c) — find P(B) by total probability", eq: "P(B) = (0.80)(0.60) + (0.25)(0.40) = 0.
  { op: "Part (c) — apply Bayes' theorem", eq: "P(A \\mid B) = \\frac{0.80 \\times 0.60}{0.58} =
])
```

# 32. Probability distributions

How probability spreads across possible values

You flip a fair coin 10 times. Before you do it, you know 5 heads seems most likely — but what exactly is the probability of getting 5? What about 8? And should 8 heads make you suspicious, or does it happen often enough that you'd expect it occasionally? Without knowing the full shape of the probability — who gets what share — you can't answer any of those questions.

Here is a different situation. You're standing in a school gymnasium with every 17-year-old in the district. Their heights spread out around some average. A few people are very tall, a few very short, most cluster near the middle. That familiar bell shape is not a coincidence — it emerges every time many small independent factors (genetics, nutrition, sleep, dozens of other things) pile on top of each other.

One more. A website gets an average of 100 visitors per hour. One night at 2am, the server logs show 140. Is that suspicious — a bot attack, maybe — or just the kind of natural swing you'd expect? You need to know how much variation is normal before you can say whether 140 is alarming.

All three situations share the same underlying need: a complete picture of how probability is spread across every possible value. That picture is a **probability distribution**.

## 32.1. What the notation is saying

### 32.1.1. Random variables

A **random variable** is a quantity whose value is determined by a random process. We write it with a capital letter — usually $X$.

- $X$ = the number of heads in 10 coin flips. $X$ could be 0, 1, 2, ..., 10.
- $X$ = the height of a randomly chosen 17-year-old. $X$ could be any value in some continuous range.

The notation $P(X = k)$ means: the probability that the random variable $X$ takes the specific value $k$.

For 10 coin flips, $P(X = 5)$ is the probability of getting exactly 5 heads. You'll calculate this shortly.

One constraint: all the probabilities must add up to 1. If you list every value $X$ could possibly take, the probabilities must account for the whole picture.

We'll write $\sum$ (capital sigma, the Greek letter S) to mean "add up all the terms that follow — here, over every possible value $k$":

$$\sum_{\text{all } k} P(X = k) = 1$$

This is just saying that *something* must happen. This means a distribution is complete: every possible outcome has to receive a share of the total probability mass.

### 32.1.2. Mean, variance, and standard deviation

Once you have a distribution, you can summarise it with two numbers.

**The mean** (also called the **expected value**) tells you the long-run average — what value you'd get if you ran the random process over and over and averaged the results. Written $\mu$ (the Greek letter *mu*):

$$\mu = \sum_{k} k \cdot P(X = k)$$

Read this as: multiply each possible value by its probability, then add everything up. It is a probability-weighted average. This means the mean is the balance point of the whole distribution, not just the middle of the listed values.

**The variance** $\sigma^2$ (sigma-squared) measures how spread out the distribution is — how far values tend to stray from the mean on average. **The standard deviation** $\sigma$ is the square root of the variance, putting it back in the same units as $X$ itself.

The key intuition: $\sigma$ measures spread, not location. Two distributions can have the same mean but look completely different. This means centre and spread answer different questions, and both are needed to describe a distribution properly.

| Distribution | Mean $\mu$ | Std dev $\sigma$ | What it looks like |
|---|---|---|---|
| A | 50 | 2 | Narrow spike near 50 |
| B | 50 | 15 | Wide, flat spread |

If you're measuring exam scores and $\sigma = 2$, almost everyone scored within 4 marks of the average. If $\sigma = 15$, scores are all over the place. Same average, completely different distributions.

## 32.2. The binomial distribution

The binomial distribution answers one specific question:

> If I repeat an experiment $n$ times, and each time the probability of success is $p$, what is the probability of getting exactly $k$ successes?

**The conditions:** 1. Fixed number of trials: $n$ 2. Each trial has the same two outcomes: success or failure 3. The probability of success is the same on every trial: $p$ 4. The trials are independent — the result of one doesn't affect another

If your situation fits all four, the count of successes $X$ follows a **binomial distribution**, written $X \sim B(n, p)$. This means the binomial model applies only when the trials really have the same success probability and are independent.

### 32.2.1. Deriving the formula

Suppose $n = 3$ and you want exactly $k = 2$ successes (call them S) and 1 failure (F). One specific sequence of outcomes that gives this is: S, S, F.

The probability of that exact sequence is $p \cdot p \cdot (1 - p) = p^2(1 - p)$.

But that's only one arrangement. You could also have: S, F, S or F, S, S. There are 3 arrangements that give exactly 2 successes in 3 trials.

So the total probability is $3 \times p^2(1 - p)$.

The number 3 came from counting arrangements. In general, the number of ways to arrange $k$ successes in $n$ trials is written $\binom{n}{k}$ (read "n choose k") and calculated as:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

where $n! = n \times (n - 1) \times \cdots \times 2 \times 1$ is "n factorial" — the number of ways to order $n$ distinct items.

> **Tip**
>
> **Calculating $\binom{n}{k}$**
> $\binom{5}{2} = \frac{5!}{2! \cdot 3!} = \frac{5 \times 4 \times 3 \times 2 \times 1}{(2 \times 1)(3 \times 2 \times 1)} = \frac{120}{2 \times 6} = \frac{120}{12} = 10$
> A shortcut: $\binom{n}{k} = \frac{n \times (n-1) \times \cdots \times (n-k+1)}{k!}$ — multiply $k$ descending numbers from $n$, divide by $k!$.
> For $\binom{5}{2}$: $\frac{5 \times 4}{2 \times 1} = \frac{20}{2} = 10$. Same answer, less work.

Putting it together:

$$\boxed{P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}}$$

Three parts: - $\binom{n}{k}$: the number of arrangements of $k$ successes in $n$ trials - $p^k$: the probability that each of those $k$ trials is a success - $(1 - p)^{n-k}$: the probability that each of the remaining trials is a failure

Multiply the parts: count of arrangements × probability of each arrangement. This means the binomial formula separates the problem into two pieces: how many success-failure patterns work, and how likely each such pattern is.

### 32.2.2. Mean and variance of the binomial

You could compute $\mu = \sum k \cdot P(X = k)$ directly, but the algebra is tedious. The results come out cleanly:

$$\mu = np \qquad \sigma^2 = np(1-p) \qquad \sigma = \sqrt{np(1-p)}$$

These make intuitive sense. If you flip a fair coin 100 times ($n = 100$, $p = 0.5$), you'd expect 50 heads on average: $\mu = 100 \times 0.5 = 50$. The standard deviation is $\sqrt{100 \times 0.5 \times 0.5} = \sqrt{25} = 5$, meaning a "typical" result is within about 5 heads of 50. This means the mean tells you the expected count of successes, while the standard deviation tells you how much fluctuation to expect around it.

```
//| echo: false
{
  const d3 = await require("d3@7");

  // --- layout constants ---
  const W = 660, H = 340;
  const margin = { top: 30, right: 20, bottom: 50, left: 55 };
  const innerW = W - margin.left - margin.right;
  const innerH = H - margin.top - margin.bottom;

  // --- helper: log-factorial (for large n) ---
  function logFactorial(n) {
    let v = 0;
    for (let i = 2; i <= n; i++) v += Math.log(i);
    return v;
  }
  function logBinom(n, k) {
    return logFactorial(n) - logFactorial(k) - logFactorial(n - k);
  }
  function binomPMF(n, p, k) {
    if (k < 0 || k > n) return 0;
    return Math.exp(logBinom(n, k) + k * Math.log(p) + (n - k) * Math.log(1 - p));
  }
  function normalPDF(x, mu, sigma) {
    const z = (x - mu) / sigma;
    return Math.exp(-0.5 * z * z) / (sigma * Math.sqrt(2 * Math.PI));
  }

  // --- controls ---
  const nSlider = Inputs.range([1, 30], { step: 1, value: 10, label: "n (trials)" });
  const pSlider = Inputs.range([0.01, 0.99], { step: 0.01, value: 0.5, label: "p (success prob)" })
  const normToggle = Inputs.toggle({ label: "Show normal approximation", value: false });

  // --- SVG ---
  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
```

```
  .attr("width", "100%")
  .style("font-family", "inherit");

const g = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// axes groups
const xAxisG = g.append("g").attr("transform", `translate(0,${innerH})`);
const yAxisG = g.append("g");

// axis labels
g.append("text")
  .attr("x", innerW / 2).attr("y", innerH + 42)
  .attr("text-anchor", "middle").attr("font-size", 13)
  .text("k (number of successes)");

g.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -innerH / 2).attr("y", -44)
  .attr("text-anchor", "middle").attr("font-size", 13)
  .text("P(X = k)");

// bars group
const barsG = g.append("g");
// normal curve
const curvePath = g.append("path")
  .attr("fill", "none")
  .attr("stroke", "#e55")
  .attr("stroke-width", 2.5)
  .attr("stroke-dasharray", "6,3")
  .attr("opacity", 0);

// stats text
const statsText = svg.append("text")
  .attr("x", margin.left + innerW - 4)
  .attr("y", margin.top + 16)
  .attr("text-anchor", "end")
  .attr("font-size", 13)
  .attr("fill", "#374151");

function update() {
  const n = nSlider.value;
  const p = pSlider.value;
  const showNorm = normToggle.value;

  const mu = n * p;
  const sigma = Math.sqrt(n * p * (1 - p));
```

```
// pmf data
const data = d3.range(0, n + 1).map(k => ({ k, prob: binomPMF(n, p, k) }));

const maxProb = d3.max(data, d => d.prob);
const yMax = Math.max(maxProb * 1.15, 0.05);

// scales
const xScale = d3.scaleBand()
  .domain(d3.range(0, n + 1))
  .range([0, innerW])
  .padding(0.1);

const yScale = d3.scaleLinear()
  .domain([0, yMax])
  .range([innerH, 0]);

// axes
const xTick = n <= 15 ? 1 : n <= 22 ? 2 : 3;
xAxisG.call(
  d3.axisBottom(xScale)
    .tickValues(d3.range(0, n + 1).filter(k => k % xTick === 0))
    .tickFormat(d3.format("d"))
);
yAxisG.call(d3.axisLeft(yScale).ticks(5).tickFormat(d3.format(".3f")));

// bars
const bars = barsG.selectAll("rect").data(data, d => d.k);
bars.enter().append("rect")
  .attr("fill", "#3b82f6")
  .attr("opacity", 0.8)
  .merge(bars)
  .attr("x", d => xScale(d.k))
  .attr("width", xScale.bandwidth())
  .attr("y", d => yScale(d.prob))
  .attr("height", d => innerH - yScale(d.prob));
bars.exit().remove();

// normal overlay
if (showNorm && sigma > 0.3) {
  const xMin = xScale(0) + xScale.bandwidth() / 2;
  const xMax = xScale(n) + xScale.bandwidth() / 2;
  const pts = d3.range(200).map(i => {
    const xVal = mu - 4 * sigma + (i / 199) * 8 * sigma;
    const screenX = (xVal / n) * innerW; // linear position across bar centres
    // map xVal to screen: bar centre for integer k is xScale(k) + bw/2
    // approximate as linear interpolation
    const bw = xScale.bandwidth();
```

```
        const screenXv = xScale(0) + bw / 2 + (xVal / 1) * (xScale(1) - xScale(0));
        return [xScale(0) + bw / 2 + xVal * (xScale.step()), yScale(normalPDF(xVal, mu, sigma))];
      });
      const line = d3.line().x(d => d[0]).y(d => d[1]).curve(d3.curveBasis);
      curvePath.attr("d", line(pts)).attr("opacity", 1);
    } else {
      curvePath.attr("opacity", 0);
    }

    statsText.text(`□ = ${mu.toFixed(2)}   □ = ${sigma.toFixed(3)}`);
  }

  nSlider.addEventListener("input", update);
  pSlider.addEventListener("input", update);
  normToggle.addEventListener("input", update);

  update();

  const container = html`<div style="border:1px solid #e5e7eb;border-radius:8px;padding:1rem 1.25
    <div style="font-weight:600;margin-bottom:0.5rem;font-size:0.95em;">Binomial distribution exp
    <div style="display:grid;grid-template-columns:1fr 1fr 1fr;gap:0.5rem 1.5rem;margin-bottom:0.
      ${nSlider}
      ${pSlider}
      ${normToggle}
    </div>
    ${svg.node()}
    <p style="font-size:0.82em;color:#6b7280;margin-top:0.4rem;">
      Bar heights show P(X = k). The dashed red curve (when toggled) is the normal approximation
      For large n and moderate p the two shapes converge — this is the central limit theorem in a
    </p>
  </div>`;
  return container;
}
```

## 32.3. The normal distribution

The binomial counts discrete things — 0 heads, 1 head, 2 heads. But heights, weights, temperatures, and exam scores can take any value in a continuous range. For these, you need a different model.

The **normal distribution** $N(\mu, \sigma^2)$ is a continuous bell curve centred at $\mu$ and spreading out by $\sigma$ in each direction (the second parameter is the variance $\sigma^2$, not the standard deviation — to find the spread in the original units, take the square root: $\sigma = \sqrt{\sigma^2}$). You specify it by two numbers only: the mean and the variance. This means the normal distribution is a family of curves, all with the same bell shape but shifted and stretched by $\mu$ and $\sigma$.

### 32.3.1. Why the bell curve appears everywhere

This is one of the most remarkable results in all of probability: when you add up many independent random influences, the total tends toward the normal distribution — regardless of what the individual influences look like. A person's height is the sum of contributions from hundreds of genes, years of nutrition, sleep patterns, and dozens of other factors. None of those individually looks like a bell curve. Their sum does.

This is the **central limit theorem**. You won't prove it at this stage, but you'll use it, and you'll see it confirmed every time you look at data from a natural process. This means the bell curve matters because sums and averages of many small effects tend to produce it.

### 32.3.2. The 68-95-99.7 rule

For any normal distribution $N(\mu, \sigma^2)$:

| Range | Probability |
|---|---|
| $\mu - \sigma$ to $\mu + \sigma$ (within 1 std dev) | **68%** |
| $\mu - 2\sigma$ to $\mu + 2\sigma$ (within 2 std dev) | **95%** |
| $\mu - 3\sigma$ to $\mu + 3\sigma$ (within 3 std dev) | **99.7%** |

This is sometimes called the **empirical rule**. It gives you a quick sense of what is "normal" variation and what is surprising. This means standard deviations give a practical scale for judging how unusual a value is in a normal setting.

If exam scores are $N(65, 144)$ — so $\mu = 65$ and $\sigma = 12$ — then about 95% of students score between $65 - 24 = 41$ and $65 + 24 = 89$. A score of 20 would be genuinely extraordinary (more than 3 standard deviations below the mean), occurring less than 0.15% of the time.

### 32.3.3. Standardising: the z-score

A practical problem: there is a different normal distribution for every combination of $\mu$ and $\sigma$. You can't carry a different table for each one.

The solution is to convert any normal distribution to a single standard one. The **standard normal** distribution is $N(0, 1)$ — mean 0, variance 1.

To convert a value $x$ from any normal distribution to its standard normal equivalent, compute:

$$z = \frac{x - \mu}{\sigma}$$

The value $z$ is called a **z-score**. It tells you how many standard deviations above (positive) or below (negative) the mean the value $x$ sits. This means standardising translates any normal-distribution problem into one common scale measured in standard deviations.

Once you have $z$, you look up $P(Z \leq z)$ in a standard normal table or compute it on a calculator. This single table handles all normal distributions.

**Example.** Scores are $N(65, 144)$, so $\mu = 65$, $\sigma = 12$. For a score of $x = 83$:

$$z = \frac{83 - 65}{12} = \frac{18}{12} = 1.5$$

A score of 83 sits 1.5 standard deviations above the mean. A table gives $P(Z \leq 1.5) \approx 0.933$, so about 93.3% of students scored 83 or below — and about 6.7% scored above 83. This means a z-score is a position statement: it tells you where a value sits relative to the rest of the distribution.

```
//| echo: false
{
  const d3 = await require("d3@7");

  // --- layout ---
  const W = 660, H = 320;
  const margin = { top: 30, right: 20, bottom: 50, left: 55 };
  const innerW = W - margin.left - margin.right;
  const innerH = H - margin.top - margin.bottom;

  // --- normal CDF (erf approximation) ---
  function erf(x) {
    const a1 =  0.254829592, a2 = -0.284496736, a3 =  1.421413741;
    const a4 = -1.453152027, a5 =  1.061405429, p  =  0.3275911;
    const sign = x < 0 ? -1 : 1;
    const t = 1 / (1 + p * Math.abs(x));
    const y = 1 - (((((a5 * t + a4) * t) + a3) * t + a2) * t + a1) * t * Math.exp(-x * x);
    return sign * y;
  }
  function normalCDF(x, mu, sigma) {
    return 0.5 * (1 + erf((x - mu) / (sigma * Math.sqrt(2))));
  }
  function normalPDF(x, mu, sigma) {
    const z = (x - mu) / sigma;
    return Math.exp(-0.5 * z * z) / (sigma * Math.sqrt(2 * Math.PI));
  }

  // --- mode state ---
  let mode = "sigma"; // "sigma" | "interval"

  // --- controls ---
  const muSlider  = Inputs.range([-10, 10],  { step: 0.5,  value: 0,   label: "□ (mean)" });
  const sigSlider = Inputs.range([0.5, 5],   { step: 0.1,  value: 1,   label: "□ (std dev)" });
  const bandSel   = Inputs.select(["1□", "2□", "3□"], { label: "Shade within", value: "1□" });
  const aInput    = Inputs.number({ label: "a", value: -1, step: 0.1 });
  const bInput    = Inputs.number({ label: "b", value:  1, step: 0.1 });
  const modeBtn   = document.createElement("button");
  modeBtn.textContent = "Switch to P(a < X < b) mode";
  modeBtn.style.cssText = "padding:0.3rem 0.8rem;border:1px solid #d1d5db;border-radius:4px;backg
```

```
// sigma-mode controls wrapper
const sigmaControls = html`<div style="display:grid;grid-template-columns:1fr 1fr 1fr;gap:0.5rem 1
const intervalControls = html`<div style="display:grid;grid-template-columns:1fr 1fr 1fr;gap:0.5re
intervalControls.style.display = "none";

// --- SVG ---
const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width", "100%")
  .style("font-family", "inherit");

const g = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

const xAxisG = g.append("g").attr("transform", `translate(0,${innerH})`);
const yAxisG = g.append("g");

g.append("text")
  .attr("x", innerW / 2).attr("y", innerH + 42)
  .attr("text-anchor", "middle").attr("font-size", 13).text("x");
g.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -innerH / 2).attr("y", -44)
  .attr("text-anchor", "middle").attr("font-size", 13).text("f(x)");

const shadeArea = g.append("path").attr("fill", "#3b82f6").attr("opacity", 0.25);
const curvePath = g.append("path").attr("fill", "none").attr("stroke", "#1d4ed8").attr("stroke-wi
const meanLine  = g.append("line").attr("stroke", "#374151").attr("stroke-width", 1.5).attr("strok

// sigma band lines
const bandLines = [-3,-2,-1,1,2,3].map(m =>
  g.append("line").attr("stroke", "#9ca3af").attr("stroke-width", 1).attr("stroke-dasharray", "3,3
);

const probLabel = svg.append("text")
  .attr("x", margin.left + innerW - 4)
  .attr("y", margin.top + 18)
  .attr("text-anchor", "end")
  .attr("font-size", 13)
  .attr("fill", "#1d4ed8")
  .attr("font-weight", "600");

const statsLabel = svg.append("text")
  .attr("x", margin.left + 6)
  .attr("y", margin.top + 18)
  .attr("font-size", 12)
  .attr("fill", "#6b7280");
```

```
const N_CURVE = 400;

function getRange(mu, sigma) {
  return [mu - 4.5 * sigma, mu + 4.5 * sigma];
}

function update() {
  const mu  = muSlider.value;
  const sig = sigSlider.value;
  const [xMin, xMax] = getRange(mu, sig);
  const yMax = normalPDF(mu, mu, sig) * 1.15;

  const xScale = d3.scaleLinear().domain([xMin, xMax]).range([0, innerW]);
  const yScale = d3.scaleLinear().domain([0, yMax]).range([innerH, 0]);

  xAxisG.call(d3.axisBottom(xScale).ticks(8).tickFormat(d3.format(".1f")));
  yAxisG.call(d3.axisLeft(yScale).ticks(5).tickFormat(d3.format(".3f")));

  // curve
  const curveData = d3.range(N_CURVE).map(i => {
    const x = xMin + (i / (N_CURVE - 1)) * (xMax - xMin);
    return [xScale(x), yScale(normalPDF(x, mu, sig))];
  });
  const line = d3.line().x(d => d[0]).y(d => d[1]).curve(d3.curveBasis);
  curvePath.attr("d", line(curveData));

  // mean line
  meanLine
    .attr("x1", xScale(mu)).attr("x2", xScale(mu))
    .attr("y1", 0).attr("y2", innerH);

  statsLabel.text(`□ = ${mu.toFixed(1)}   □ = ${sig.toFixed(1)}`);

  if (mode === "sigma") {
    const bandMap = { "1□": 1, "2□": 2, "3□": 3 };
    const k = bandMap[bandSel.value];
    const lo = mu - k * sig;
    const hi = mu + k * sig;

    // shade
    const shadeData = d3.range(200).map(i => {
      const x = lo + (i / 199) * (hi - lo);
      return [xScale(x), yScale(normalPDF(x, mu, sig))];
    });
    const area = d3.area()
      .x(d => d[0]).y0(innerH).y1(d => d[1]).curve(d3.curveBasis);
    shadeArea.attr("d", area(shadeData));
```

```
    const prob = normalCDF(hi, mu, sig) - normalCDF(lo, mu, sig);
    probLabel.text(`P(within ${bandSel.value}) = ${(prob * 100).toFixed(2)}%`);

    // band dashed lines
    [-k, k].forEach((m, i) => {
      bandLines[i].attr("x1", xScale(mu + m * sig)).attr("x2", xScale(mu + m * sig))
        .attr("y1", 0).attr("y2", innerH).attr("opacity", 0.7);
    });
    bandLines.slice(2).forEach(l => l.attr("opacity", 0));

  } else {
    // interval mode
    const a = Math.min(aInput.value, bInput.value);
    const b = Math.max(aInput.value, bInput.value);
    const lo = Math.max(xMin, a);
    const hi = Math.min(xMax, b);
    if (lo < hi) {
      const shadeData = d3.range(200).map(i => {
        const x = lo + (i / 199) * (hi - lo);
        return [xScale(x), yScale(normalPDF(x, mu, sig))];
      });
      const area = d3.area()
        .x(d => d[0]).y0(innerH).y1(d => d[1]).curve(d3.curveBasis);
      shadeArea.attr("d", area(shadeData));
      const prob = normalCDF(b, mu, sig) - normalCDF(a, mu, sig);
      probLabel.text(`P(${a.toFixed(1)} < X < ${b.toFixed(1)}) = ${(prob * 100).toFixed(2)}%`);
    } else {
      shadeArea.attr("d", null);
      probLabel.text("—");
    }
    bandLines.forEach(l => l.attr("opacity", 0));
  }
}

muSlider.addEventListener("input",  update);
sigSlider.addEventListener("input", update);
bandSel.addEventListener("input",   update);
aInput.addEventListener("input",    update);
bInput.addEventListener("input",    update);

modeBtn.addEventListener("click", () => {
  mode = mode === "sigma" ? "interval" : "sigma";
  modeBtn.textContent = mode === "sigma"
    ? "Switch to P(a < X < b) mode"
    : "Switch to σ-band mode";
  sigmaControls.style.display   = mode === "sigma"    ? "" : "none";
  intervalControls.style.display = mode === "interval" ? "" : "none";
```

```
    update();
  });

  update();

  const container = html`<div style="border:1px solid #e5e7eb;border-radius:8px;padding:1rem 1.25
    <div style="font-weight:600;margin-bottom:0.5rem;font-size:0.95em;">Normal distribution explo
    ${sigmaControls}
    ${intervalControls}
    <div style="margin-top:0.5rem;">${modeBtn}</div>
    ${svg.node()}
    <p style="font-size:0.82em;color:#6b7280;margin-top:0.4rem;">
      Shaded region shows the selected probability. The dashed vertical line marks □.
      Adjust □ to shift the curve left/right; adjust □ to widen or narrow it.
    </p>
  </div>`;
  return container;
}
```

## 32.4. Worked examples

### 32.4.1. Example 1 — Binomial exact (quality control)

A manufacturing line produces components. Historical data shows that 15% of components are defective. A quality inspector takes a batch of 8 components. What is the probability that exactly 2 are defective?

Here $X$ = number of defectives, $n = 8$, $p = 0.15$, and we want $P(X = 2)$.

$$P(X = 2) = \binom{8}{2}(0.15)^2(0.85)^6$$

Calculate each part:

$$\binom{8}{2} = \frac{8 \times 7}{2 \times 1} = 28$$

$$(0.15)^2 = 0.0225$$

$$(0.85)^6 = 0.85 \times 0.85 \times 0.85 \times 0.85 \times 0.85 \times 0.85 \approx 0.3771$$

$$P(X = 2) = 28 \times 0.0225 \times 0.3771 \approx 28 \times 0.008485 \approx 0.2376$$

About 24% of batches of 8 will contain exactly 2 defectives.

Check: the mean number of defectives is $\mu = np = 8 \times 0.15 = 1.2$, so 2 is slightly above average — a 24% probability is plausible. This means "slightly above average" does not imply "rare"; the spread matters as well.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.5rem 1rem; align-ite
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
```

```
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

```
//| echo: false
makeStepperHTML(1, [
  { op: "Identify the distribution",    eq: "X \\sim B(8,\\, 0.15),\\quad n=8,\\; p=0.15,\\; k=2"
  { op: "Write the formula",            eq: "P(X=2) = \\binom{8}{2}(0.15)^2(0.85)^6" },
  { op: "Compute the combination",      eq: "\\binom{8}{2} = \\frac{8 \\times 7}{2 \\times 1} = 2
  { op: "Compute p^k",                  eq: "(0.15)^2 = 0.0225" },
  { op: "Compute (1-p)^{n-k}",          eq: "(0.85)^6 \\approx 0.3771" },
  { op: "Multiply all three parts",     eq: "P(X=2) = 28 \\times 0.0225 \\times 0.3771 \\approx 0
  { op: "Sanity check via mean",        eq: "\\mu = np = 8 \\times 0.15 = 1.2", note: "k = 2 is j
])
```

### 32.4.2. Example 2 — Binomial complement (healthcare)

A new drug cures 70% of patients. In a clinical trial, 10 patients receive the drug. What is the probability that at least 8 are cured?

$X \sim B(10, 0.7)$. We want $P(X \geq 8) = P(X = 8) + P(X = 9) + P(X = 10)$.

With only 3 terms, direct calculation is manageable:

$$P(X = 8) = \binom{10}{8}(0.7)^8(0.3)^2 = 45 \times 0.05765 \times 0.09 \approx 0.2335$$

$$P(X = 9) = \binom{10}{9}(0.7)^9(0.3)^1 = 10 \times 0.04035 \times 0.3 \approx 0.1211$$

$$P(X = 10) = \binom{10}{10}(0.7)^{10}(0.3)^0 = 1 \times 0.02825 \times 1 \approx 0.0282$$

$$P(X \geq 8) \approx 0.2335 + 0.1211 + 0.0282 = 0.3828$$

There is about a 38% probability that at least 8 of 10 patients are cured. Note: you would use the complement approach when the "at least" threshold is low — e.g., $P(X \geq 2)$ is much easier to compute as $1 - P(X = 0) - P(X = 1)$ than to sum 9 terms directly. This means the complement is often a computational shortcut for cumulative binomial probabilities.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Identify the distribution",    eq: "X \\sim B(10,\\, 0.7)" },
  { op: "Write as a sum",               eq: "P(X \\geq 8) = P(X=8) + P(X=9) + P(X=10)" },
  { op: "Compute P(X = 8)",             eq: "\\binom{10}{8}(0.7)^8(0.3)^2 = 45 \\times 0.05765 \\tir
  { op: "Compute P(X = 9)",             eq: "\\binom{10}{9}(0.7)^9(0.3)^1 = 10 \\times 0.04035 \\tir
  { op: "Compute P(X = 10)",            eq: "\\binom{10}{10}(0.7)^{10}(0.3)^0 = 1 \\times 0.02825 \\
  { op: "Sum the three terms",          eq: "P(X \\geq 8) \\approx 0.2335 + 0.1211 + 0.0282 = 0.3828
])
```

### 32.4.3. Example 3 — Normal z-score (education)

End-of-year exam scores in a large district are approximately normally distributed with mean $\mu = 65$ and standard deviation $\sigma = 12$. What fraction of students scored above 80?

We want $P(X > 80)$.

**Step 1: Standardise.**

$$z = \frac{80 - 65}{12} = \frac{15}{12} = 1.25$$

**Step 2: Look up $P(Z \leq 1.25)$.**

From a standard normal table (or calculator): $P(Z \leq 1.25) \approx 0.8944$.

**Step 3: Convert to the required probability.**

$$P(X > 80) = 1 - P(X \leq 80) = 1 - 0.8944 = 0.1056$$

About 10.6% of students scored above 80.

A quick check with the 68-95-99.7 rule: 80 is $\frac{15}{12} = 1.25$ standard deviations above the mean, which is between $1\sigma$ and $2\sigma$. The rule says about 16% of values lie beyond $1\sigma$, so 10.6% beyond $1.25\sigma$ is consistent. This means exact z-score calculations refine the rough intuition supplied by the empirical rule.

```
//| echo: false
makeStepperHTML(3, [
  { op: "State the distribution",       eq: "X \\sim N(65,\\, 144),\\quad \\mu=65,\\; \\sigma=12"
  { op: "Standardise x = 80",           eq: "z = \\frac{80 - 65}{12} = \\frac{15}{12} = 1.25" },
  { op: "Look up cumulative prob",      eq: "P(Z \\leq 1.25) \\approx 0.8944", note: "From a stan
  { op: "Use the complement",           eq: "P(X > 80) = 1 - 0.8944 = 0.1056", note: "About 10.6%
  { op: "Sanity check",                 eq: "1.25\\sigma > \\mu \\Rightarrow \\text{between 16\\%
])
```

### 32.4.4. Example 4 — Normal reverse lookup (pass with distinction)

Using the same exam distribution ($\mu = 65$, $\sigma = 12$), the school awards "distinction" to the top 10% of students. What score is the distinction threshold?

We want the value $x$ such that $P(X > x) = 0.10$, i.e., $P(X \leq x) = 0.90$.

**Step 1: Find the z-score for the 90th percentile.**

From a standard normal table, find $z$ such that $P(Z \leq z) = 0.90$.

Looking up 0.90: $z \approx 1.282$.

**Step 2: Unstandardise — convert back to the original scale.**

Since $z = \frac{x-\mu}{\sigma}$, rearranging gives:

$$x = \mu + z\sigma = 65 + 1.282 \times 12 = 65 + 15.38 \approx 80.4$$

A score of about 80 is the distinction threshold.

Note the structure: for a forward problem you go $x \to z \to$ probability. For a reverse problem you go probability $\to z \to x$. The formula $x = \mu + z\sigma$ is just the standardisation formula solved for $x$. This means reverse-lookup problems use the same machinery as ordinary probability questions, only in the opposite direction.

```
//| echo: false
makeStepperHTML(4, [
  { op: "State the goal",               eq: "P(X > x) = 0.10 \\Leftrightarrow P(X \\leq x) = 0.90
  { op: "Find the 90th-percentile z",   eq: "z_{0.90} \\approx 1.282", note: "Look up 0.90 in the
  { op: "Unstandardise",               eq: "x = \\mu + z\\sigma = 65 + 1.282 \\times 12", note: "
  { op: "Compute the threshold",        eq: "x = 65 + 15.38 \\approx 80.4", note: "A score of abo
])
```

## 32.5. Where this goes

This chapter turned event-based probability into full models for random variables. The next chapter uses those models to judge whether observed sample results are ordinary or statistically persuasive.

**Statistical inference (Chapter 3)** is the direct next step. A distribution gives you the expected shape of data from a random process. Inference asks the reverse question: your actual data has a shape — is that consistent with what you'd expect from the assumed distribution, or is something else going on? Every hypothesis test you'll ever run is answering that question.

**Volume 7 probability** builds the mathematical foundations underneath what you've been using here. Why does the normal distribution have exactly the bell-curve formula it does? What does "continuous probability" mean rigorously? The answers require calculus — integration gives you the area under the bell curve — but you can use the results now.

## 32.6. Applications

> **Note**
>
> **Where these distributions appear in practice**
> **A/B testing.** A website tests two versions of a button. 200 users see version A, 200 see version B. Is the difference in click rates real or just chance? Binomial distributions tell you what click-rate differences to expect from chance alone.
> **Quality control charts.** Manufacturing uses Shewhart control charts: plot each batch measurement, mark lines at $\mu \pm 2\sigma$ and $\mu \pm 3\sigma$. A point outside the $3\sigma$ line has less than a 0.3% chance of occurring naturally — strong evidence something has gone wrong in the process.
> **Weather forecasting.** A "70% chance of rain" is a probability from a distribution over possible outcomes. Forecast *skill* — whether forecasts are actually calibrated — is assessed using statistics built on the normal distribution.
> **Clinical trial power.** Before running a drug trial, statisticians calculate how many patients they need to have a reasonable chance of detecting a real effect if one exists. That calculation requires knowing the distribution of outcomes under both the null hypothesis and the alternative. You're one chapter away from doing this.

## 32.7. Exercises

**Exercise 1.** A student guesses randomly on a 12-question true/false quiz. Let $X$ be the number of correct answers.

(a) State the distribution of $X$, giving $n$ and $p$.
(b) Calculate $P(X = 6)$.
(c) Calculate the mean and standard deviation of $X$.

```
//| echo: false
makeStepperHTML(5, [
  { op: "Identify the distribution",    eq: "X \\sim B(12,\\, 0.5)" },
  { op: "Write the formula for P(X=6)", eq: "P(X=6) = \\binom{12}{6}(0.5)^6(0.5)^6 = \\binom{12}{
  { op: "Compute the combination",      eq: "\\binom{12}{6} = \\frac{12!}{6!\\,6!} = 924" },
  { op: "Compute (0.5)^{12}",           eq: "(0.5)^{12} = \\frac{1}{4096} \\approx 0.000244" },
  { op: "Multiply",                     eq: "P(X=6) = 924 \\times 0.000244 \\approx 0.2256", note
  { op: "Mean and standard deviation",  eq: "\\mu = np = 12 \\times 0.5 = 6,\\quad \\sigma = \\sq
])
```

---

**Exercise 2.** In a city, 20% of households own an electric vehicle. A researcher surveys 15 randomly chosen households.

(a) Let $X$ be the number of households with an electric vehicle. State the distribution of $X$.
(b) Find $P(X = 3)$.
(c) Find $P(X \leq 1)$ using the complement or direct calculation.
(d) Find the mean and variance of $X$.

```
//| echo: false
makeStepperHTML(6, [
  { op: "Identify the distribution",    eq: "X \\sim B(15,\\, 0.2)" },
  { op: "Formula for P(X = 3)",         eq: "P(X=3) = \\binom{15}{3}(0.2)^3(0.8)^{12}" },
  { op: "Compute combination",          eq: "\\binom{15}{3} = \\frac{15 \\times 14 \\times 13}{3
  { op: "Compute each power",           eq: "(0.2)^3 = 0.008,\\quad (0.8)^{12} \\approx 0.06872"
  { op: "Multiply",                     eq: "P(X=3) = 455 \\times 0.008 \\times 0.06872 \\approx
  { op: "P(X ≤ 1) directly",            eq: "P(X=0) = (0.8)^{15} \\approx 0.03518,\\quad P(X=1) =
  { op: "Sum for P(X ≤ 1)",             eq: "P(X \\leq 1) \\approx 0.03518 + 0.13194 = 0.1671", n
  { op: "Mean and variance",            eq: "\\mu = np = 15 \\times 0.2 = 3,\\quad \\sigma^2 = np
])
```

---

**Exercise 3.** A coin is known to be biased: it shows heads with probability $p = 0.4$. The coin is flipped 20 times.

(a) Write down the mean and variance of the number of heads.
(b) Without calculating individual probabilities, explain whether you would be surprised to get 14 heads. (Use $\sigma$ to guide your reasoning.)

```
//| echo: false
makeStepperHTML(7, [
  { op: "Identify the distribution",    eq: "X \\sim B(20,\\, 0.4)" },
  { op: "Mean",                         eq: "\\mu = np = 20 \\times 0.4 = 8" },
  { op: "Variance",                     eq: "\\sigma^2 = np(1-p) = 20 \\times 0.4 \\times 0.6 = 4
```

```
  { op: "Standard deviation",          eq: "\\sigma = \\sqrt{4.8} \\approx 2.19" },
  { op: "How far is 14 from the mean?", eq: "14 - 8 = 6 = \\frac{6}{2.19} \\approx 2.74\\,\\sigma \\
  { op: "Assess surprise",              eq: "P(|X - \\mu| \\geq 2.74\\sigma) < 0.006", note: "A valu
])
```

---

**Exercise 4.** The resting heart rate of adults in a large study is approximately normally distributed with mean $\mu = 72$ beats per minute and standard deviation $\sigma = 10$ bpm.

(a) Find the probability that a randomly chosen adult has a resting heart rate above 90 bpm.
(b) Find the probability that a randomly chosen adult has a resting heart rate between 62 and 82 bpm. (You may use the 68-95-99.7 rule for this part.)
(c) What percentage of adults have resting heart rates below 55 bpm?

```
//| echo: false
makeStepperHTML(8, [
  { op: "State the distribution",       eq: "X \\sim N(72,\\, 100),\\quad \\mu=72,\\; \\sigma=10" }
  { op: "(a) Standardise x = 90",       eq: "z = \\frac{90 - 72}{10} = 1.8" },
  { op: "(a) Look up and complement",   eq: "P(X > 90) = 1 - P(Z \\leq 1.8) = 1 - 0.9641 = 0.0359",
  { op: "(b) Recognise the interval",   eq: "[62,\\,82] = [\\mu - \\sigma,\\; \\mu + \\sigma]", note
  { op: "(b) Apply empirical rule",     eq: "P(62 < X < 82) \\approx 0.68", note: "68% of adults hav
  { op: "(c) Standardise x = 55",       eq: "z = \\frac{55 - 72}{10} = -1.7" },
  { op: "(c) Look up",                  eq: "P(X < 55) = P(Z < -1.7) = 1 - P(Z \\leq 1.7) \\approx 
])
```

---

**Exercise 5.** Graduate entry to a university programme requires a score in the top 5% of a standardised test. Scores are normally distributed with $\mu = 500$ and $\sigma = 100$.

(a) Find the z-score corresponding to the 95th percentile. (Use $z_{0.95} \approx 1.645$.)
(b) What is the minimum score needed for entry?
(c) A student scores 620. What percentile does this correspond to? (Find $z$, then use a table or calculator.)

```
//| echo: false
makeStepperHTML(9, [
  { op: "State the distribution",       eq: "X \\sim N(500,\\, 100^2),\\quad \\mu=500,\\; \\sigma=10
  { op: "(a) 95th-percentile z-score",  eq: "z_{0.95} \\approx 1.645", note: "Given in the problem;
  { op: "(b) Unstandardise",           eq: "x = \\mu + z\\sigma = 500 + 1.645 \\times 100 = 664.5" }
  { op: "(b) Minimum entry score",      eq: "x \\approx 665", note: "A score of at least 665 is nee
  { op: "(c) Standardise score 620",    eq: "z = \\frac{620 - 500}{100} = 1.2" },
  { op: "(c) Look up CDF",             eq: "P(Z \\leq 1.2) \\approx 0.8849", note: "A score of 620 s
])
```

---

**Exercise 6.** A marine biologist is studying kelp-eating urchins on a reef. She records the number of urchins found in each of 50 one-metre quadrats. Historical data for this reef type suggests urchin counts per quadrat follow a distribution with $\mu = 8$ and $\sigma = 2.8$.

(a) A count of 15 urchins is recorded in one quadrat. How many standard deviations above the mean is this?
(b) Using the 68-95-99.7 rule, roughly what fraction of quadrats would you expect to contain more than 13.6 urchins?
(c) The biologist suspects a disturbance has shifted urchin density. She finds 12 quadrats (out of 50) with counts above 13.6. Based on your answer to (b), does this seem consistent with the historical distribution? Explain briefly.

```
//| echo: false
makeStepperHTML(10, [
  { op: "State the parameters",         eq: "\\mu = 8,\\quad \\sigma = 2.8" },
  { op: "(a) z-score for count 15",     eq: "z = \\frac{15 - 8}{2.8} = \\frac{7}{2.8} = 2.5", not
  { op: "(b) Identify 13.6 in ⍰ units", eq: "13.6 = \\mu + 2\\sigma = 8 + 2 \\times 2.8 = 13.6" }
  { op: "(b) Apply 95% rule",           eq: "P(\\mu - 2\\sigma < X < \\mu + 2\\sigma) \\approx 0.
  { op: "(b) Tail fraction",            eq: "P(X > \\mu + 2\\sigma) \\approx \\frac{1 - 0.95}{2}
  { op: "(c) Expected vs observed",     eq: "0.025 \\times 50 = 1.25 \\text{ quadrats expected};\
  { op: "(c) Conclusion",               eq: "\\frac{12}{50} = 24\\% \\gg 2.5\\%", note: "12 quadr
])
```

# 33. Statistical Inference

From sample to conclusion

A study makes the news: a new energy drink improves reaction time by 12 milliseconds on average in a sample of 40 university students. The company behind it claims the drink genuinely works. A sceptic says the 12 ms gap could just be luck.

Both people are looking at the same number. How can they reach opposite conclusions?

The answer is that a single number from a single sample cannot settle the question on its own. What matters is what numbers like that look like when the drink *does nothing* — when the only thing producing variation is chance. If 12 ms is the kind of gap you'd often see from chance alone, the result is unremarkable. If 12 ms is extremely unusual under chance, something else is probably going on.

This chapter is about how to make that reasoning precise. The tools are called **statistical inference** — the machinery for drawing conclusions about populations from samples, while being honest about uncertainty.

## 33.1. Populations and samples

A **population** is the complete set of individuals or measurements you care about. A **sample** is the subset you can actually observe.

In most real situations, measuring the entire population is impossible. You cannot test every batch of tablets that will ever leave a factory. You cannot ask every voter in an election what they think today. You cannot measure the resting heart rate of every adult in the country. So you take a sample — as many observations as you can afford — and reason from there.

The fundamental challenge is that the sample will not perfectly reflect the population. Take a different sample and you'll get a different mean. Take enough different samples and the sample means form their own distribution — one that carries useful information about the population.

A few terms used throughout:

- $N$ (or sometimes the full description): the **population size** — total number of individuals
- $n$: the **sample size** — how many you actually measured
- $\mu$ (mu): the **population mean** — what you want to know but usually cannot compute directly
- $\bar{x}$ (x-bar): the **sample mean** — what you computed from your $n$ observations
- $\sigma$: the **population standard deviation** — spread of the whole population
- $s$: the **sample standard deviation** — spread estimated from your sample

The goal of inference is to say something credible about $\mu$ using $\bar{x}$, knowing that $\bar{x}$ is a good guess but not an exact answer.

### 33.1.1. What makes a sample useful

Not every sample tells you much. A sample of convenience — asking only your friends, measuring only the easiest-to-reach parts of a forest, testing only patients who volunteer — may be biased in ways that invalidate any conclusion about the broader population.

A **representative sample** is one where each member of the population has a fair chance of being included. The cleanest version is a **simple random sample**: every member of the population is equally likely to be chosen, and choices are independent. In practice, this is an ideal rather than a guarantee, but it is the assumption that makes everything in this chapter valid. Whenever you see a claim built on inference, ask: was the sample actually representative?

## 33.2. The sampling distribution

Suppose heights in a population are normally distributed with mean $\mu = 170$ cm and standard deviation $\sigma = 8$ cm. You take a random sample of $n = 25$ people and compute their mean height $\bar{x}$.

Now imagine doing this many times: take 25 people, compute $\bar{x}$, record it. Take another 25, compute $\bar{x}$, record it. Repeat this thousands of times.

What would you see if you plotted all those sample means?

Three things, and all three are predictable:

1. The sample means cluster around $\mu = 170$. No surprise — if your sampling is unbiased, the average of the averages should track the true population mean.
2. The sample means are *less* spread out than individual heights. Averaging smooths out extremes. A single person might be 155 cm or 185 cm — that's plausible. But a *mean* of 25 people being 155 cm would require nearly everyone in the sample to be very short, which is far less likely.
3. The distribution of sample means is approximately normal — even if the population itself is not perfectly normal.

This distribution — the distribution of $\bar{x}$ across all possible samples of size $n$ — is called the **sampling distribution of the mean**.

```
//| echo: false
// Sampling Distribution Simulator
// Reader sets □, □, n; draws 1000 samples and watches x̄ pile up into
// a histogram with the theoretical N(□, □²/n) curve overlaid.

viewof sim_mu = Inputs.range([50, 150], {value: 100, step: 1, label: "Population mean □"})
```

```
//| echo: false
viewof sim_sigma = Inputs.range([5, 30], {value: 12, step: 1, label: "Population SD □"})
```

```
//| echo: false
viewof sim_n = Inputs.range([1, 100], {value: 25, step: 1, label: "Sample size n"})
```

```
//| echo: false
viewof sim_drawBtn = Inputs.button("Draw 1000 samples")
```

```
//| echo: false
{
  // Accumulate sample means across button presses
  const stored = { means: [] };

  // Re-run whenever drawBtn is clicked or parameters change
  sim_drawBtn;
  const mu = sim_mu, sigma = sim_sigma, n = sim_n;

  // Box-Muller normal random generator
  function randNorm(mu, sigma) {
    let u, v;
    do { u = Math.random(); } while (u === 0);
    v = Math.random();
    return mu + sigma * Math.sqrt(-2 * Math.log(u)) * Math.cos(2 * Math.PI * v);
  }

  // Draw 1000 new sample means and append
  for (let i = 0; i < 1000; i++) {
    let sum = 0;
    for (let j = 0; j < n; j++) sum += randNorm(mu, sigma);
    stored.means.push(sum / n);
  }

  const means = stored.means;
  const SE = sigma / Math.sqrt(n);
  const totalDrawn = means.length;

  // Layout
  const W = 600, H = 320;
  const marginL = 55, marginR = 20, marginT = 30, marginB = 50;
  const plotW = W - marginL - marginR;
  const plotH = H - marginT - marginB;

  // Axis range: ? ± 4*SE but at least ? ± 3*sigma/sqrt(1)
  const xMin = mu - 4.5 * Math.max(SE, 1);
  const xMax = mu + 4.5 * Math.max(SE, 1);

  // Histogram bins
  const nBins = 50;
  const binW = (xMax - xMin) / nBins;
```

```
const counts = new Array(nBins).fill(0);
for (const m of means) {
  const b = Math.floor((m - xMin) / binW);
  if (b >= 0 && b < nBins) counts[b]++;
}
const maxCount = Math.max(...counts, 1);

// Scale helpers
const xScale = x => marginL + ((x - xMin) / (xMax - xMin)) * plotW;
const yScaleCount = c => marginT + plotH - (c / maxCount) * plotH;

// Theoretical normal PDF, scaled to match histogram
function normalPDF(x, mean, sd) {
  return Math.exp(-0.5 * ((x - mean) / sd) ** 2) / (sd * Math.sqrt(2 * Math.PI));
}
const pdfScale = totalDrawn * binW; // converts density → expected count

// Build SVG
const svg = d3.create("svg").attr("width", W).attr("height", H)
  .attr("viewBox", `0 0 ${W} ${H}`)
  .style("font-family", "inherit")
  .style("overflow", "visible");

// Background
svg.append("rect").attr("x", marginL).attr("y", marginT)
  .attr("width", plotW).attr("height", plotH)
  .attr("fill", "#f9fafb").attr("rx", 3);

// Histogram bars
for (let i = 0; i < nBins; i++) {
  const bx = xMin + i * binW;
  svg.append("rect")
    .attr("x", xScale(bx) + 0.5)
    .attr("y", yScaleCount(counts[i]))
    .attr("width", Math.max(1, xScale(bx + binW) - xScale(bx) - 1))
    .attr("height", marginT + plotH - yScaleCount(counts[i]))
    .attr("fill", "#93c5fd")
    .attr("stroke", "#60a5fa")
    .attr("stroke-width", 0.4);
}

// Theoretical curve N(□, □²/n)
const curvePoints = [];
const steps = 200;
for (let i = 0; i <= steps; i++) {
  const cx = xMin + (i / steps) * (xMax - xMin);
  const cy = normalPDF(cx, mu, SE) * pdfScale;
```

```
  const scaledY = marginT + plotH - (cy / maxCount) * plotH;
  curvePoints.push([xScale(cx), Math.max(marginT, scaledY)]);
}
const lineGen = d3.line().x(d => d[0]).y(d => d[1]).curve(d3.curveBasis);
svg.append("path").attr("d", lineGen(curvePoints))
  .attr("fill", "none").attr("stroke", "#1d4ed8").attr("stroke-width", 2);

// Mean line
svg.append("line")
  .attr("x1", xScale(mu)).attr("x2", xScale(mu))
  .attr("y1", marginT).attr("y2", marginT + plotH)
  .attr("stroke", "#6b7280").attr("stroke-width", 1).attr("stroke-dasharray", "4,3");

// x-axis
const xTicks = 6;
const xStep = (xMax - xMin) / xTicks;
svg.append("line").attr("x1", marginL).attr("x2", marginL + plotW)
  .attr("y1", marginT + plotH).attr("y2", marginT + plotH)
  .attr("stroke", "#374151").attr("stroke-width", 1);
for (let i = 0; i <= xTicks; i++) {
  const val = xMin + i * xStep;
  const px = xScale(val);
  svg.append("line").attr("x1", px).attr("x2", px)
    .attr("y1", marginT + plotH).attr("y2", marginT + plotH + 4)
    .attr("stroke", "#374151");
  svg.append("text").attr("x", px).attr("y", marginT + plotH + 16)
    .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#374151")
    .text(val.toFixed(1));
}
svg.append("text").attr("x", marginL + plotW / 2).attr("y", H - 6)
  .attr("text-anchor", "middle").attr("font-size", 12).attr("fill", "#374151")
  .text("Sample mean x̄ ");

// y-axis label
svg.append("text")
  .attr("transform", `translate(14, ${marginT + plotH / 2}) rotate(-90)`)
  .attr("text-anchor", "middle").attr("font-size", 12).attr("fill", "#374151")
  .text("Count");

// Info panel
const infoX = marginL + plotW - 160, infoY = marginT + 8;
svg.append("rect").attr("x", infoX - 6).attr("y", infoY - 4)
  .attr("width", 164).attr("height", 62).attr("rx", 4)
  .attr("fill", "white").attr("stroke", "#e5e7eb");
svg.append("text").attr("x", infoX).attr("y", infoY + 10)
  .attr("font-size", 12).attr("fill", "#1d4ed8").attr("font-weight", "600")
  .text(`SE = ⍰/√n = ${SE.toFixed(3)}`);
```

```
  svg.append("text").attr("x", infoX).attr("y", infoY + 28)
    .attr("font-size", 11).attr("fill", "#374151")
    .text(`□ = ${mu},  □ = ${sigma},  n = ${n}`);
  svg.append("text").attr("x", infoX).attr("y", infoY + 46)
    .attr("font-size", 11).attr("fill", "#6b7280")
    .text(`${totalDrawn.toLocaleString()} sample means drawn`);

  // Legend
  svg.append("rect").attr("x", marginL + 6).attr("y", marginT + 8)
    .attr("width", 12).attr("height", 12).attr("fill", "#93c5fd").attr("stroke", "#60a5fa");
  svg.append("text").attr("x", marginL + 22).attr("y", marginT + 19)
    .attr("font-size", 11).attr("fill", "#374151").text("Simulated x̄ values");
  svg.append("line").attr("x1", marginL + 6).attr("x2", marginL + 18)
    .attr("y1", marginT + 33).attr("y2", marginT + 33)
    .attr("stroke", "#1d4ed8").attr("stroke-width", 2);
  svg.append("text").attr("x", marginL + 22).attr("y", marginT + 37)
    .attr("font-size", 11).attr("fill", "#374151").text("Theoretical N(□, □²/n)");

  // Title
  svg.append("text").attr("x", W / 2).attr("y", marginT - 10)
    .attr("text-anchor", "middle").attr("font-size", 13).attr("fill", "#111827").attr("font-weight"
    .text("Sampling distribution of x̄ ");

  return svg.node();
}
```

## 33.3. The Central Limit Theorem

The three observations above are not separate facts. They are consequences of a single result called the **Central Limit Theorem** (CLT).

> **Note**
>
> **The Central Limit Theorem**
> If you take random samples of size $n$ from a population with mean $\mu$ and standard deviation $\sigma$, then for sufficiently large $n$, the sampling distribution of $\bar{x}$ is approximately normal:
>
> $$\bar{x} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$
>
> The mean of the sampling distribution equals $\mu$. The standard deviation of the sampling distribution is $\frac{\sigma}{\sqrt{n}}$.

This means repeated samples do not bounce around arbitrarily: their means form a predictable distribution centred on the true population mean.

The standard deviation of the sampling distribution has its own name: the **standard error** (SE).

$$SE = \frac{\sigma}{\sqrt{n}}$$

The standard error is not the spread of individual measurements — it is the spread of *sample means*. It tells you how much $\bar{x}$ typically varies from one sample to the next. This means the standard error measures uncertainty in the estimate, not variability in the raw data themselves.

### 33.3.1. Why this is surprising

The CLT makes no assumption about the shape of the population distribution. Whether the individual measurements are skewed, uniform, bimodal, or anything else, the sample means pile up into a bell curve once $n$ is large enough.

This is the reason the normal distribution appears so often in statistical work. It is not because every quantity in nature is normally distributed. It is because *sample means* tend to be normally distributed, and most of the quantities we measure and compare are averages of some kind.

As a rough guide, $n \geq 30$ is often enough for the CLT to give a good approximation. For symmetric populations, even smaller samples work well. For heavily skewed populations, larger $n$ may be needed. This means the CLT is a practical approximation theorem, not a claim that every sample mean is exactly normal for every sample size.

### 33.3.2. Working with the standard error — a numerical example

Return to the height example: $\mu = 170$ cm, $\sigma = 8$ cm.

**For samples of size $n = 25$:**

$$SE = \frac{8}{\sqrt{25}} = \frac{8}{5} = 1.6 \text{ cm}$$

The sampling distribution is $\bar{x} \sim N(170, 1.6^2)$. A sample mean of 173 cm is:

$$z = \frac{173 - 170}{1.6} = \frac{3}{1.6} = 1.875 \text{ standard errors above the mean}$$

**For samples of size $n = 100$:**

$$SE = \frac{8}{\sqrt{100}} = \frac{8}{10} = 0.8 \text{ cm}$$

The same sample mean of 173 cm is now:

$$z = \frac{173 - 170}{0.8} = \frac{3}{0.8} = 3.75 \text{ standard errors above the mean}$$

With $n = 25$, a sample mean of 173 cm is not very surprising — it is less than 2 standard errors out. With $n = 100$, the same 173 cm is very surprising — nearly 4 standard errors out. Larger samples give

you more information, so unusual results become more informative. This means the same numerical difference can be ordinary in a small sample and highly persuasive in a large one.

This is the key insight: **more data makes your estimate more precise**, and the improvement scales as $1/\sqrt{n}$, not $1/n$. To halve the standard error, you need four times as much data.

## 33.4. Confidence intervals

You take a sample, compute $\bar{x}$, and want to say something about $\mu$. A single number — "$\mu$ is approximately 170 cm" — is called a **point estimate**. It is useful but incomplete, because it gives no indication of how uncertain you are.

An **interval estimate** gives a range of plausible values for $\mu$. A **confidence interval** attaches a stated level of confidence to that range — typically 90%, 95%, or 99%.

### 33.4.1. Building a 95% confidence interval

From Chapter 2, you know that for a normal distribution, 95% of values fall within approximately 1.96 standard deviations of the mean. Precisely:

$$P(-1.96 \leq Z \leq 1.96) = 0.95$$

where $Z \sim N(0, 1)$.

Since $\bar{x} \sim N(\mu, \sigma^2/n)$, standardising gives:

$$P\left(-1.96 \leq \frac{\bar{x} - \mu}{SE} \leq 1.96\right) = 0.95$$

Rearranging the inequality to isolate $\mu$: multiply through by SE to get $\bar{x} - 1.96 \cdot SE \leq \mu \leq \bar{x} + 1.96 \cdot SE$. This is the same algebra as rearranging $-1.96 \leq (x - 5)/2 \leq 1.96$ for $x$ — multiply through by 2, then add 5 throughout.

$$P(\bar{x} - 1.96 \cdot SE \leq \mu \leq \bar{x} + 1.96 \cdot SE) = 0.95$$

This is the **95% confidence interval for** $\mu$ (when $\sigma$ is known):

$$\boxed{\bar{x} \pm 1.96 \cdot \frac{\sigma}{\sqrt{n}}}$$

The value 1.96 is called the **critical value** for 95% confidence. For other confidence levels, the critical value changes: This means a confidence interval is built from an estimate at the centre and a margin showing how far sampling error might reasonably move it.

| Confidence level | Critical value $z^*$ |
|---|---|
| 90% | 1.645 |
| 95% | 1.960 |
| 99% | 2.576 |

### 33.4.2. Worked example — confidence interval

A nutritionist measures the daily sodium intake (mg) of a random sample of $n = 36$ adults from a region. The sample mean is $\bar{x} = 2{,}450$ mg. Population studies suggest $\sigma = 360$ mg for daily sodium intake.

Construct a 95% confidence interval for the true mean daily sodium intake $\mu$ in this region.

**Step 1: Calculate the standard error.**

$$SE = \frac{\sigma}{\sqrt{n}} = \frac{360}{\sqrt{36}} = \frac{360}{6} = 60 \text{ mg}$$

**Step 2: Find the margin of error.**

$$\text{Margin of error} = 1.96 \times SE = 1.96 \times 60 = 117.6 \text{ mg}$$

**Step 3: Construct the interval.**

$$\bar{x} \pm \text{margin of error} = 2{,}450 \pm 117.6$$

$$\text{CI: } (2{,}332.4 \text{ mg},\ 2{,}567.6 \text{ mg})$$

**Step 4: Interpret.**

> We are 95% confident that the true mean daily sodium intake in this region is between 2,332 mg and 2,568 mg. This means the interval reports both the best estimate and the uncertainty around it in one statement.

### 33.4.3. What "95% confident" actually means

This is one of the most commonly misunderstood ideas in statistics.

**The incorrect interpretation:** "There is a 95% probability that $\mu$ lies in this interval."

This sounds reasonable but is wrong. The population mean $\mu$ is a fixed (if unknown) number — it does not have a probability distribution. It either is or is not in the interval. The interval itself is random, because $\bar{x}$ varies from sample to sample.

**The correct interpretation:** If you were to repeat this sampling procedure many times — each time collecting 36 adults, computing $\bar{x}$, and constructing the interval — then 95% of the resulting intervals would contain the true $\mu$.

In other words, confidence describes the procedure, not any single interval. The 95% refers to the long-run success rate of the method.

In practice, you have one interval from one sample, and you cannot know whether yours is one of the 95% that work or one of the 5% that don't. The confidence level tells you the method is reliable, not that any particular result is guaranteed. This means confidence belongs to the long-run procedure, not to the unknown parameter itself.

> Tip
>
> **A useful sentence template for interpreting confidence intervals**
> "We are [level]% confident that the true population mean [quantity] is between [lower] and [upper] [units]."
> Avoid: "There is a 95% chance the mean is in this interval." Use: "We are 95% confident the mean is in this interval."

## 33.5. Hypothesis testing

A confidence interval tells you what values of $\mu$ are plausible given your data. A **hypothesis test** answers a more direct question: is the evidence strong enough to reject a specific claim about $\mu$?

### 33.5.1. The logic of the test

The framework borrows from legal reasoning. A court presumes innocence until the evidence makes guilt sufficiently clear. In statistics:

- You start by assuming a specific null state — the **null hypothesis** $H_0$.
- You ask: if the null hypothesis were true, how likely is it that you'd see data at least as extreme as what you observed?
- If the answer is "very unlikely," the data is evidence against $H_0$.

What counts as "very unlikely" is a threshold you set in advance, called the **significance level** $\alpha$. The most common choice is $\alpha = 0.05$. This means hypothesis testing asks whether the data would be too surprising to explain as ordinary sampling variation under the null model.

### 33.5.2. Setting up the hypotheses

Every test involves two competing hypotheses:

**The null hypothesis** $H_0$ is the claim being tested. It usually represents "no effect," "no difference," or a specific benchmark value. You write it as an equality: $H_0 : \mu = \mu_0$, where $\mu_0$ is the claimed value.

**The alternative hypothesis** $H_1$ (sometimes written $H_a$) is what you believe might be true instead. It can be:

- **Two-tailed:** $H_1 : \mu \neq \mu_0$ — the mean could be either higher or lower than claimed
- **One-tailed (upper):** $H_1 : \mu > \mu_0$ — you suspect the mean is higher

- **One-tailed (lower):** $H_1 : \mu < \mu_0$ — you suspect the mean is lower

The choice of one- or two-tailed test should reflect your research question *before* you look at the data. Choosing the direction after seeing the data inflates the apparent significance and undermines the logic of the test.

### 33.5.3. The test statistic

If $H_0 : \mu = \mu_0$ is true, then the sampling distribution of $\bar{x}$ is:

$$\bar{x} \sim N\left(\mu_0, \frac{\sigma^2}{n}\right)$$

Standardising gives the **z-test statistic**:

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$$

This $z$ value measures how many standard errors your observed sample mean is from the null hypothesis value. If $|z|$ is small, your data is consistent with $H_0$. If $|z|$ is large, your data is unusual under $H_0$. This means the test statistic converts the raw difference into a standardised measure of surprise.

### 33.5.4. The p-value

The **p-value** is the probability of observing a test statistic at least as extreme as the one you computed, *if $H_0$ is true.*

- For a two-tailed test ($H_1 : \mu \neq \mu_0$): $p$-value $= 2 \times P(Z \geq |z|)$
- For an upper-tailed test ($H_1 : \mu > \mu_0$): $p$-value $= P(Z \geq z)$
- For a lower-tailed test ($H_1 : \mu < \mu_0$): $p$-value $= P(Z \leq z)$

A small p-value means your observed data would be rare under $H_0$ — the data is hard to explain by chance alone. This means the p-value is about the compatibility of the data with the null hypothesis, not the probability that the null hypothesis is true.

### 33.5.5. The decision rule

Compare the p-value to the significance level $\alpha$:

- If $p$-value $\leq \alpha$: **reject** $H_0$. The data is sufficiently unusual under $H_0$ to conclude there is evidence for $H_1$.
- If $p$-value $> \alpha$: **fail to reject** $H_0$. The data is consistent with $H_0$ — not proof that $H_0$ is true, only that there is insufficient evidence to reject it.

Note the careful language: you *fail to reject*, not *accept* $H_0$. Absence of evidence is not evidence of absence. This means a non-significant result can reflect either "no effect" or "not enough information to detect one."

```
//| echo: false
// Hypothesis Test Explorer
// Reader sets μ₀, x̄, σ, n; chooses tail type and α.
// The viz draws N(μ₀, σ²/n), marks x̄, shades rejection region,
// and shows z, p-value, and the reject/fail-to-reject decision.

viewof ht_mu0 = Inputs.range([50, 250], {value: 650, step: 1, label: "Null mean μ₀"})
```

```
//| echo: false
viewof ht_xbar = Inputs.range([50, 250], {value: 672, step: 0.1, label: "Observed mean x̄"})
```

```
//| echo: false
viewof ht_sigma = Inputs.range([1, 100], {value: 80, step: 1, label: "Population SD σ"})
```

```
//| echo: false
viewof ht_n = Inputs.range([2, 200], {value: 40, step: 1, label: "Sample size n"})
```

```
//| echo: false
viewof ht_tail = Inputs.select(["two-tailed", "upper (H₁: μ > μ₀)", "lower (H₁: μ < μ₀)"], {
  value: "upper (H₁: μ > μ₀)",
  label: "Test type"
})
```

```
//| echo: false
viewof ht_alpha = Inputs.select([0.01, 0.05, 0.10], {value: 0.05, label: "Significance level α"})
```

```
//| echo: false
{
  const mu0 = ht_mu0, xbar = ht_xbar, sigma = ht_sigma, n = ht_n;
  const alpha = ht_alpha;
  const tail = ht_tail;

  const SE = sigma / Math.sqrt(n);
  const z = (xbar - mu0) / SE;

  // Standard normal CDF via error function approximation
  function erf(x) {
    const a1=0.254829592, a2=-0.284496736, a3=1.421413741, a4=-1.453152027, a5=1.061405429, p=0.3275
    const sign = x < 0 ? -1 : 1;
    const t = 1 / (1 + p * Math.abs(x));
    const y = 1 - (((((a5*t + a4)*t + a3)*t + a2)*t + a1)*t) * Math.exp(-x*x);
    return sign * y;
```

```
}
function normalCDF(x) { return 0.5 * (1 + erf(x / Math.sqrt(2))); }
function normalPDF(x, mu, sd) {
  return Math.exp(-0.5 * ((x - mu) / sd) ** 2) / (sd * Math.sqrt(2 * Math.PI));
}

// p-value
let pval;
if (tail === "two-tailed") pval = 2 * (1 - normalCDF(Math.abs(z)));
else if (tail.startsWith("upper")) pval = 1 - normalCDF(z);
else pval = normalCDF(z);
pval = Math.max(0, Math.min(1, pval));

// Critical z values
let zCritUpper = Infinity, zCritLower = -Infinity;
if (tail === "two-tailed") {
  zCritUpper = -normalCDFInv(alpha / 2);
  zCritLower = normalCDFInv(alpha / 2);
} else if (tail.startsWith("upper")) {
  zCritUpper = -normalCDFInv(alpha);
} else {
  zCritLower = normalCDFInv(alpha);
}

// Inverse normal CDF (bisection) — good enough for critical value display
function normalCDFInv(p) {
  let lo = -10, hi = 10;
  for (let i = 0; i < 60; i++) {
    const mid = (lo + hi) / 2;
    if (normalCDF(mid) < p) lo = mid; else hi = mid;
  }
  return (lo + hi) / 2;
}

// Fix: compute critical values after defining normalCDFInv
let critLo = -Infinity, critHi = Infinity;
if (tail === "two-tailed") {
  critLo = normalCDFInv(alpha / 2);
  critHi = normalCDFInv(1 - alpha / 2);
} else if (tail.startsWith("upper")) {
  critHi = normalCDFInv(1 - alpha);
} else {
  critLo = normalCDFInv(alpha);
}

const decision = pval <= alpha ? "Reject H₀" : "Fail to reject H₀";
const decisionColor = pval <= alpha ? "#dc2626" : "#059669";
```

```
// Layout
const W = 620, H = 320;
const mL = 55, mR = 20, mT = 40, mB = 55;
const pW = W - mL - mR, pH = H - mT - mB;

// x-axis range: μ₀ ± 4 SE, ensure xbar is visible
const spread = Math.max(4 * SE, Math.abs(xbar - mu0) + 2 * SE);
const xMin = mu0 - spread;
const xMax = mu0 + spread;

const xScale = x => mL + ((x - xMin) / (xMax - xMin)) * pW;
const maxPDF = normalPDF(mu0, mu0, SE);
const yScale = y => mT + pH - (y / maxPDF) * pH * 0.92;

const svg = d3.create("svg").attr("width", W).attr("height", H)
  .attr("viewBox", `0 0 ${W} ${H}`)
  .style("font-family", "inherit").style("overflow", "visible");

svg.append("rect").attr("x", mL).attr("y", mT).attr("width", pW).attr("height", pH)
  .attr("fill", "#f9fafb").attr("rx", 3);

// Build curve path points
const steps = 400;
function curvePoints(xFrom, xTo) {
  const pts = [];
  for (let i = 0; i <= steps; i++) {
    const cx = xFrom + (i / steps) * (xTo - xFrom);
    pts.push([xScale(cx), yScale(normalPDF(cx, mu0, SE))]);
  }
  return pts;
}

const lineGen = d3.line().x(d => d[0]).y(d => d[1]).curve(d3.curveBasis);

// Shade rejection region(s)
function shadedArea(xFrom, xTo, color) {
  const pts = curvePoints(xFrom, xTo);
  const area = [...pts, [xScale(xTo), mT + pH], [xScale(xFrom), mT + pH]];
  svg.append("polygon")
    .attr("points", area.map(p => p.join(",")).join(" "))
    .attr("fill", color).attr("opacity", 0.35);
}

const rejColor = "#fca5a5";
if (tail === "two-tailed") {
  shadedArea(xMin, mu0 + critLo * SE, rejColor);
  shadedArea(mu0 + critHi * SE, xMax, rejColor);
```

```
} else if (tail.startsWith("upper")) {
  shadedArea(mu0 + critHi * SE, xMax, rejColor);
} else {
  shadedArea(xMin, mu0 + critLo * SE, rejColor);
}

// Main curve
svg.append("path").attr("d", lineGen(curvePoints(xMin, xMax)))
  .attr("fill", "none").attr("stroke", "#1d4ed8").attr("stroke-width", 2.5);

// Critical value lines
function critLine(xVal, label) {
  if (!isFinite(xVal)) return;
  const px = xScale(xVal);
  svg.append("line").attr("x1", px).attr("x2", px).attr("y1", mT).attr("y2", mT + pH)
    .attr("stroke", "#dc2626").attr("stroke-width", 1.5).attr("stroke-dasharray", "5,3");
  svg.append("text").attr("x", px).attr("y", mT - 6)
    .attr("text-anchor", "middle").attr("font-size", 10).attr("fill", "#dc2626")
    .text(label);
}
if (tail === "two-tailed") {
  critLine(mu0 + critLo * SE, `z* = ${critLo.toFixed(2)}`);
  critLine(mu0 + critHi * SE, `z* = ${critHi.toFixed(2)}`);
} else if (tail.startsWith("upper")) {
  critLine(mu0 + critHi * SE, `z* = ${critHi.toFixed(2)}`);
} else {
  critLine(mu0 + critLo * SE, `z* = ${critLo.toFixed(2)}`);
}

// Observed x̄ line
const xbarPx = xScale(xbar);
svg.append("line").attr("x1", xbarPx).attr("x2", xbarPx).attr("y1", mT).attr("y2", mT + pH)
  .attr("stroke", "#7c3aed").attr("stroke-width", 2);
svg.append("text").attr("x", xbarPx).attr("y", mT + pH + 16)
  .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#7c3aed")
  .text(`x̄ = ${xbar}`);

// μ₀ line
const mu0Px = xScale(mu0);
svg.append("line").attr("x1", mu0Px).attr("x2", mu0Px).attr("y1", mT).attr("y2", mT + pH)
  .attr("stroke", "#374151").attr("stroke-width", 1).attr("stroke-dasharray", "3,3");
svg.append("text").attr("x", mu0Px).attr("y", mT + pH + 28)
  .attr("text-anchor", "middle").attr("font-size", 10).attr("fill", "#374151")
  .text(`μ₀ = ${mu0}`);

// x-axis
svg.append("line").attr("x1", mL).attr("x2", mL + pW)
```

```
    .attr("y1", mT + pH).attr("y2", mT + pH).attr("stroke", "#374151");
  const nTicks = 7;
  for (let i = 0; i <= nTicks; i++) {
    const val = xMin + (i / nTicks) * (xMax - xMin);
    const px = xScale(val);
    svg.append("line").attr("x1", px).attr("x2", px)
      .attr("y1", mT + pH).attr("y2", mT + pH + 4).attr("stroke", "#374151");
    svg.append("text").attr("x", px).attr("y", mT + pH + 14)
      .attr("text-anchor", "middle").attr("font-size", 10).attr("fill", "#374151")
      .text(val.toFixed(1));
  }

  // Info box
  const iX = mL + 8, iY = mT + 6;
  svg.append("rect").attr("x", iX - 4).attr("y", iY - 4)
    .attr("width", 210).attr("height", 80).attr("rx", 4)
    .attr("fill", "white").attr("stroke", "#e5e7eb");
  svg.append("text").attr("x", iX).attr("y", iY + 12).attr("font-size", 12).attr("fill", "#374151")
    .text(`SE = ${SE.toFixed(3)}`);
  svg.append("text").attr("x", iX).attr("y", iY + 28).attr("font-size", 12).attr("fill", "#374151")
    .text(`z = ${z.toFixed(3)}`);
  svg.append("text").attr("x", iX).attr("y", iY + 44).attr("font-size", 12).attr("fill", "#374151")
    .text(`p-value = ${pval < 0.001 ? "< 0.001" : pval.toFixed(4)}`);
  svg.append("text").attr("x", iX).attr("y", iY + 62).attr("font-size", 13)
    .attr("fill", decisionColor).attr("font-weight", "700").text(decision);

  // Title
  svg.append("text").attr("x", W / 2).attr("y", mT - 18)
    .attr("text-anchor", "middle").attr("font-size", 13).attr("fill", "#111827").attr("font-weight"
    .text("Hypothesis test: N(□₀, □²/n) sampling distribution");

  return svg.node();
}
```

## 33.6. Worked example — full hypothesis test

The school cafeteria claims that a standard portion of their pasta contains 650 mg of sodium. A nutritionist suspects the actual amount is higher. She measures 40 randomly chosen portions and finds a sample mean of $\bar{x} = 672$ mg. From manufacturer data, $\sigma = 80$ mg.

Test the nutritionist's claim at the 5% significance level.

**Step 1: State the hypotheses.**

The cafeteria claims $\mu = 650$ mg. The nutritionist suspects it is *higher*, so this is a one-tailed (upper) test.

$$H_0 : \mu = 650 \quad H_1 : \mu > 650$$

**Step 2: Choose the significance level.**

$$\alpha = 0.05$$

**Step 3: Compute the test statistic.**

$$z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}} = \frac{672 - 650}{80/\sqrt{40}} = \frac{22}{80/6.325} = \frac{22}{12.65} \approx 1.74$$

**Step 4: Find the p-value.**

For an upper-tailed test:

$$p\text{-value} = P(Z \geq 1.74)$$

From a standard normal table: $P(Z \leq 1.74) \approx 0.9591$.

$$p\text{-value} = 1 - 0.9591 = 0.0409$$

**Step 5: Make a decision.**

$p$-value $= 0.041 < \alpha = 0.05$, so we **reject** $H_0$.

**Step 6: State the conclusion in plain language.**

> At the 5% significance level, there is sufficient evidence to conclude that the mean sodium content of cafeteria pasta portions exceeds the claimed 650 mg.

Notice what this conclusion does *not* say. It does not say the true mean is 672 mg. It does not say the cafeteria is lying. It says the evidence is strong enough to act on the suspicion that something is off. The strength of that conclusion depends on the assumptions: random sampling, known $\sigma$, and that the significance level was chosen before looking at the data. This means the test supports a directional claim about the mean, not a claim that the sample mean itself is the exact truth.

## 33.7. Type I and Type II errors

Every hypothesis test makes a binary decision under uncertainty. Two kinds of mistakes are possible:

| | $H_0$ is actually true | $H_0$ is actually false |
|---|---|---|
| **Reject** $H_0$ | **Type I error** (false positive) | Correct decision |
| **Fail to reject** $H_0$ | Correct decision | **Type II error** (false negative) |

A **Type I error** is rejecting $H_0$ when it is true — concluding there is an effect when there is none. The probability of a Type I error is exactly $\alpha$, the significance level you chose. Setting $\alpha = 0.05$ means you accept a 5% chance of a false positive if $H_0$ is true.

A **Type II error** is failing to reject $H_0$ when it is false — missing a real effect. Its probability is denoted $\beta$. The **power** of a test is $1 - \beta$: the probability of correctly detecting a true effect. This means hypothesis testing is an error-management system: you choose how willing you are to risk different kinds of mistake.

### 33.7.1. Why this tradeoff matters

Reducing $\alpha$ (demanding stronger evidence before rejecting $H_0$) decreases the chance of a false positive but increases the chance of a false negative — you become more conservative and more likely to miss real effects.

The stakes differ by context. A Type I error in a drug trial means approving an ineffective drug — real harm. A Type II error means failing to approve an effective one — also real harm, but different in character. Before running a study, statisticians specify both $\alpha$ and the desired power, then calculate the sample size $n$ required to achieve both. Larger samples reduce both types of error simultaneously, which is why clinical trials are expensive. This means more data improves inference mainly by shrinking the standard error and making real effects easier to distinguish from noise.

> Tip
>
> **A concrete framing**
> A fire alarm that goes off every time someone burns toast (Type I error: false positive) is annoying and teaches people to ignore it. A fire alarm that never sounds during an actual fire (Type II error: false negative) is catastrophic. Good alarm design — and good statistical design — requires thinking carefully about which error is more costly.

## 33.8. Connecting to the bigger picture

Every time you see a published result with a p-value, you are looking at the output of this machinery. The researcher had a null hypothesis, collected data, computed a test statistic, and compared the p-value to a threshold.

When $\sigma$ is unknown and estimated from the sample (using $s$ instead), the test statistic no longer follows the standard normal distribution. It follows a **t-distribution**, which is slightly wider and depends on

the sample size. For large $n$, the difference is small; for small samples, it matters. The t-distribution is the natural next step beyond this chapter.

Two further extensions are essential for most real work:

**Comparing two groups.** Does treatment A work better than treatment B? Here you test whether the *difference* in means $\mu_1 - \mu_2 = 0$ rather than testing a single mean. The two-sample z-test or t-test extends the framework from this chapter directly.

**Regression.** Once you move to modelling how one variable predicts another, you are still doing inference — testing whether the slope of a line is zero, constructing confidence intervals around predictions. The language of $H_0$, test statistics, and p-values is identical.

The machinery in this chapter is the foundation for all of it.

## 33.9. Where this goes

This chapter turned sampling variability into a formal basis for estimation and decision-making. The next steps in statistics keep the same logic but apply it to richer models and more complicated data structures.

**What this chapter enables**

- **Computing (comp):** Machine learning validation — train/test splits, significance of benchmark improvements — is applied inference. Every time a researcher reports that model A beats model B, the question is whether the difference is real or sampling noise.
- **Hard sciences (sci):** Every published experiment reports a p-value. Clinical trial analysis, drug approval decisions, and environmental monitoring all run on the framework you have just learned.
- **Finance and business (biz):** A/B testing at scale, quality control charts, and actuarial inference are direct applications. The decision to change a product is a hypothesis test with real stakes.
- **Geography and environment (geo):** Climate trend detection — distinguishing a genuine warming signal from year-to-year variation — requires formal inference. So does environmental monitoring for pollution events and species population changes.

## 33.10. Exercises

**A note on these exercises**
All exercises in this chapter give you $\sigma$, the population standard deviation. In practice, $\sigma$ is almost never known — you estimate it from the sample using $s$. That extension (the t-distribution) is the natural next step beyond this chapter. For now, you will always be given $\sigma$.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.5rem 1rem; align-ite
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
```

```
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

**Exercise 1.** A population of salmon in a river system has a known standard deviation of $\sigma = 42$ mm in body length. A researcher samples $n$ fish from this population.

(a) Calculate the standard error when $n = 9$.
(b) Calculate the standard error when $n = 36$.
(c) Calculate the standard error when $n = 144$.
(d) By what factor does the standard error change each time $n$ increases by a factor of 4? Explain why this makes sense using the formula.

```
//| echo: false
makeStepperHTML(1, [
  { op: "Formula", eq: "SE = \\dfrac{\\sigma}{\\sqrt{n}}" },
  { op: "(a) n = 9", eq: "SE = \\dfrac{42}{\\sqrt{9}} = \\dfrac{42}{3} = 14 \\text{ mm}" },
  { op: "(b) n = 36", eq: "SE = \\dfrac{42}{\\sqrt{36}} = \\dfrac{42}{6} = 7 \\text{ mm}" },
  { op: "(c) n = 144", eq: "SE = \\dfrac{42}{\\sqrt{144}} = \\dfrac{42}{12} = 3.5 \\text{ mm}" },
  { op: "(d) Factor each time", eq: "14 \\to 7 \\to 3.5: \\text{ each time } SE \\text{ halves}",
  { op: "General rule", eq: "SE \\propto \\dfrac{1}{\\sqrt{n}}: \\text{ quadruple } n \\Rightarro
])
```

---

**Exercise 2.** A climatologist records the daily high temperature at a monitoring station over 49 randomly selected days. The sample mean is $\bar{x} = 18.4$°C. From long-term records, the population standard deviation is $\sigma = 5.6$°C.

(a) Calculate the standard error for this sample.
(b) Construct a 95% confidence interval for the true mean daily high temperature at this location.
(c) Construct a 99% confidence interval. (Use $z^* = 2.576$.)
(d) Write one sentence interpreting your 95% CI correctly.

```
//| echo: false
makeStepperHTML(2, [
  { op: "(a) Standard error", eq: "SE = \\dfrac{\\sigma}{\\sqrt{n}} = \\dfrac{5.6}{\\sqrt{49}} = \\c
  { op: "(b) 95% margin of error", eq: "1.96 \\times 0.8 = 1.568^{\\circ}\\text{C}", note: "Critical
  { op: "(b) 95% CI", eq: "18.4 \\pm 1.568 \\implies (16.83^{\\circ}\\text{C},\\ 19.97^{\\circ}\\tex
  { op: "(c) 99% margin of error", eq: "2.576 \\times 0.8 = 2.061^{\\circ}\\text{C}", note: "Critica
  { op: "(c) 99% CI", eq: "18.4 \\pm 2.061 \\implies (16.34^{\\circ}\\text{C},\\ 20.46^{\\circ}\\tex
  { op: "(d) Interpretation", eq: "\\text{We are 95\\% confident the true mean daily high is betwee
])
```

---

**Exercise 3.** For each of the following scenarios, state the null and alternative hypotheses. Identify whether the test is one-tailed or two-tailed. If one-tailed, state the direction.

(a) A consumer watchdog tests whether a cereal box labelled "500 g net weight" actually contains 500 g on average. The watchdog is concerned the boxes may contain *less* than advertised.

(b) A researcher believes a new fertiliser changes mean crop yield. She does not know whether the effect is positive or negative.

(c) A traffic engineer claims that average vehicle speed on a stretch of highway is 95 km/h. A safety audit suspects drivers are exceeding this average.

```
//| echo: false
makeStepperHTML(3, [
  { op: "(a) Null hypothesis", eq: "H_0: \\mu = 500 \\text{ g}", note: "The label claim is the null
  { op: "(a) Alternative", eq: "H_1: \\mu < 500 \\text{ g} \\quad \\text{(one-tailed, lower)}", note
  { op: "(b) Null hypothesis", eq: "H_0: \\mu = \\mu_{\\text{control}}", note: "No effect of the fe
  { op: "(b) Alternative", eq: "H_1: \\mu \\neq \\mu_{\\text{control}} \\quad \\text{(two-tailed)}"
  { op: "(c) Null hypothesis", eq: "H_0: \\mu = 95 \\text{ km/h}" },
  { op: "(c) Alternative", eq: "H_1: \\mu > 95 \\text{ km/h} \\quad \\text{(one-tailed, upper)}", no
])
```

---

**Exercise 4.** An athletics coach claims that the training programme he coaches produces marathon runners with a mean finish time of 210 minutes. A sports scientist believes the true mean is different (in either direction). She times 36 runners through the programme. Their sample mean finish time is $\bar{x} = 205.2$ minutes. The population standard deviation is known to be $\sigma = 18$ minutes.

(a) State $H_0$ and $H_1$.
(b) Calculate the test statistic $z$.
(c) Find the p-value. (This is a two-tailed test; use $P(Z \leq -1.60) \approx 0.055$ as a reference value.)
(d) At $\alpha = 0.05$, state your decision and conclusion in plain language.

```
//| echo: false
makeStepperHTML(4, [
  { op: "(a) Hypotheses", eq: "H_0: \\mu = 210 \\text{ min}, \\quad H_1: \\mu \\neq 210 \\text{ m
  { op: "(b) Standard error", eq: "SE = \\dfrac{\\sigma}{\\sqrt{n}} = \\dfrac{18}{\\sqrt{36}} = \
  { op: "(b) Test statistic", eq: "z = \\dfrac{\\bar{x} - \\mu_0}{SE} = \\dfrac{205.2 - 210}{3} =
  { op: "(c) p-value (two-tailed)", eq: "p = 2 \\times P(Z \\leq -1.60) \\approx 2 \\times 0.055
  { op: "(d) Decision", eq: "p = 0.110 > \\alpha = 0.05 \\implies \\text{fail to reject } H_0", n
  { op: "(d) Conclusion", eq: "\\text{Insufficient evidence that the true mean finish time differ
])
```

---

**Exercise 5.** A pharmaceutical company is testing a new painkiller. The clinical team must choose a significance level before running the trial.

(a) Explain what a Type I error would mean in this context.
(b) Explain what a Type II error would mean in this context.
(c) The team is debating between $\alpha = 0.05$ and $\alpha = 0.01$. Describe the tradeoff. Which error becomes more likely if they choose the stricter threshold? Which becomes less likely?
(d) The drug is for a condition where the standard treatment has serious side effects. How might this context influence the choice of significance level? There is no single right answer — explain your reasoning.

```
//| echo: false
makeStepperHTML(5, [
  { op: "(a) Type I error here", eq: "\\text{Reject } H_0 \\text{ when drug has no real effect}",
  { op: "(b) Type II error here", eq: "\\text{Fail to reject } H_0 \\text{ when drug does work}",
  { op: "(c) Stricter threshold (□ = 0.01)", eq: "P(\\text{Type I}) = 0.01 \\downarrow, \\quad P(
  { op: "(c) Looser threshold (□ = 0.05)", eq: "P(\\text{Type I}) = 0.05 \\uparrow, \\quad P(\\te
  { op: "(d) Context argument", eq: "\\text{Standard treatment has serious side effects} \\Righta
])
```

---

*Exercise 6 goes beyond what this chapter formally covers — you don't need the two-sample formula, but you do need to reason carefully about what would be required.*

**Exercise 6.** A data scientist is running an A/B test for an e-commerce site. Version A (the existing checkout flow) has a known mean order value of $\mu_A = \$87$ with $\sigma_A = \$22$, estimated from thousands of previous transactions. Version B (a redesigned flow) is tested on a new sample.

(a) A sample of 100 users on Version B produces $\bar{x}_B = \$93$. If you were to test whether this is significantly different from the Version A mean of \$87, what would your null and alternative hypotheses be?
(b) What additional information would you need to conduct a formal two-sample test comparing Version A and Version B means?

(c) The data scientist argues: "Version B has a higher sample mean, so we should switch." A statistician responds: "Not so fast." Explain the statistician's concern in terms of sampling variability and the standard error. What would a proper inference procedure provide that the raw comparison of means cannot?

```
//| echo: false
makeStepperHTML(6, [
  { op: "(a) Null hypothesis", eq: "H_0: \\mu_B = 87", note: "The new design produces the same mean
  { op: "(a) Alternative", eq: "H_1: \\mu_B \\neq 87 \\quad \\text{(two-tailed)}", note: "The desig
  { op: "(b) Additional information needed", eq: "\\sigma_B \\text{ (or } s_B\\text{) for Version B
  { op: "(b) SE for Version B alone", eq: "SE_B = \\dfrac{\\sigma_B}{\\sqrt{n_B}} = \\dfrac{\\sigma_
  { op: "(c) The statistician's concern", eq: "SE_B = \\dfrac{22}{10} = 2.20 \\text{ (using } \\sig
  { op: "(c) What inference adds", eq: "z = \\dfrac{93 - 87}{2.20} \\approx 2.73, \\quad p \\approx
])
```

# 34. Data Analysis

Patterns in numbers, conclusions about the world

Spotify has about 100 million songs. When you finish a track and it auto-plays the next one, you didn't choose that song — an algorithm did. The algorithm looked at what you've played, when you skipped, what similar listeners chose, and built a model of your taste from the pattern in those numbers.

That is data analysis: you have a pile of numbers that records something about the world, and you want to find the pattern, measure it, and use it to predict something new.

The methods in this chapter are the foundation of that entire enterprise. The streaming recommendation is built on techniques that grew from the same roots you're about to learn: summarising a distribution, measuring how two variables move together, fitting a line that lets you predict one thing from another. Every time a platform says *we think you'll like this*, the chain of reasoning traces back here.

The chapter has four moves. First: organise and describe data on a single variable — centre, spread, shape. Second: describe the relationship between two variables visually, before any formula. Third: measure that relationship with a single number, the correlation coefficient $r$. Fourth: model it with a line — regression — so you can make predictions. At each step the goal is not just to produce a number, but to know what the number is actually saying.

---

## 34.1. 1 · Types of data

Before you summarise data, you need to know what kind of data you have. The choice of summary and visualisation depends on it.

**Categorical data** names groups: favourite genre, country of birth, blood type, whether a patient recovered. The values are labels, not amounts — you cannot meaningfully average them.

**Quantitative data** measures amounts: temperature, height, number of streams, exam score. These values are numbers where arithmetic makes sense.

Within quantitative data there is a further distinction.

*Discrete* data takes separate, countable values: number of goals scored, number of siblings, number of days absent. There is no meaningful value between 2 and 3.

*Continuous* data can take any value in a range: height, weight, time, temperature. Between any two values there is always another one.

Statisticians sometimes also describe four **levels of measurement**, which capture a slightly different property — how much structure the numbers carry.

| Level | Example | What you can do |
|---|---|---|
| Nominal | Blood type: A, B, AB, O | Count only; order is meaningless |
| Ordinal | Survey: agree / neutral / disagree | Order is meaningful; gaps between values are not |
| Interval | Temperature in °C | Differences are meaningful; zero is arbitrary (0°C is not "no temperature") |
| Ratio | Distance, mass, income | Differences and ratios both meaningful; zero means none |

This matters in practice. You can say that 40°C is 20° hotter than 20°C (interval: differences work). You cannot say it is twice as hot, because 0°C is not "no heat" — it is just the freezing point of water. But you can say that 40km is twice as far as 20km (ratio: zero means nothing there).

For this chapter the key split is **categorical vs. quantitative**, because it determines every choice that follows: which summary statistics to compute, which plots to draw, and whether correlation and regression make sense at all. This means the type of data is not a bookkeeping detail; it determines which mathematical tools are legitimate.

---

## 34.2. 2 · Describing one variable

### 34.2.1. Centre

Three measures compete for "the typical value" of a dataset:

**Mean** — add all values, divide by how many.

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

**Median** — the middle value when the data is sorted. For an even number of values, average the two middle ones.

**Mode** — the most frequently occurring value.

Each one answers a slightly different question, and each has weaknesses.

Consider seven students' weekly hours of paid work:

$$4,\ 6,\ 6,\ 8,\ 9,\ 11,\ 42$$

The mean is $(4 + 6 + 6 + 8 + 9 + 11 + 42)/7 = 86/7 \approx 12.3$. The median is the fourth value when sorted: 8. The mode is 6.

Which is "right"? That depends on your question. The 42 is clearly unusual — one student has a second job. The mean got pulled to 12.3 by that one outlier, past six of the seven actual values. The median (8) is not sensitive to the extreme value. The mode tells you the most common single value. This means centre is not a single fixed idea: different summaries answer different questions about what is "typical."

> **The outlier rule of thumb**
> When a distribution has outliers or is heavily skewed, report the **median** as the centre. The mean is a better summary when the data is roughly symmetric and free of extreme values. Both together tell you more than either alone.

### 34.2.2. Spread

A measure of centre on its own is incomplete. Knowing the average temperature in a city tells you nothing about whether it's stable or wildly variable. You need a measure of spread.

**Range** = maximum − minimum. Simple, but sensitive to extremes. In the work-hours example: $42 - 4 = 38$.

**Interquartile range (IQR)** = $Q_3 - Q_1$. The middle 50% of the data, untouched by the extremes.

To find $Q_1$ and $Q_3$: split the sorted data at the median, then find the median of each half.

For our seven values $4, 6, 6, 8, 9, 11, 42$:

- Median (Q2) = 8
- Lower half: $4, 6, 6 \longrightarrow Q_1 = 6$
- Upper half: $9, 11, 42 \longrightarrow Q_3 = 11$
- IQR $= 11 - 6 = 5$

The middle 50% of students work between 6 and 11 hours per week. The 42 did not affect this at all. This means the IQR is resistant to extreme values in a way the range and standard deviation are not.

**Standard deviation** measures the average distance from the mean. You will see this written as $s$ for a sample:

$$s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

The formula squares the deviations (so positive and negative distances don't cancel), takes their average (roughly — the $n - 1$ is a technical correction for samples), then takes the square root to get back to the original units. This means standard deviation measures a typical distance from the mean in the same units as the data.

For the work-hours example, with mean $\approx 12.3$:

| $x_i$ | $x_i - \bar{x}$ | $(x_i - \bar{x})^2$ |
|---|---|---|
| 4 | −8.3 | 68.89 |
| 6 | −6.3 | 39.69 |
| 6 | −6.3 | 39.69 |

| $x_i$ | $x_i - \bar{x}$ | $(x_i - \bar{x})^2$ |
|---|---|---|
| 8 | −4.3 | 18.49 |
| 9 | −3.3 | 10.89 |
| 11 | −1.3 | 1.69 |
| 42 | +29.7 | 882.09 |

Sum of squared deviations: 1061.43. Divide by $n - 1 = 6$: 176.9. Take the square root: $s \approx 13.3$ hours.

That large standard deviation reflects the 42-hour outlier distorting the picture. Again: when a distribution has strong outliers, IQR is the more honest spread measure.

### 34.2.3. Shape: the five-number summary and box plot

The **five-number summary** is min, $Q_1$, median, $Q_3$, max. It describes the distribution's shape without computing a mean at all:

$$4, \quad 6, \quad 8, \quad 11, \quad 42$$

A **box plot** draws this. The box spans from $Q_1$ to $Q_3$. A line inside the box marks the median. "Whiskers" extend to the minimum and maximum (or to the boundary of non-outlier data, with individual dots for outliers beyond $1.5 \times \text{IQR}$ from the box edges).

Box plots are excellent for comparing distributions across groups — for example, work hours broken down by year of study — because they pack a lot of shape information into a small space.

> **Outlier detection rule**
> A common convention: flag a value as an outlier if it falls more than $1.5 \times \text{IQR}$ below $Q_1$ or above $Q_3$.
> Lower fence: $Q_1 - 1.5 \times \text{IQR} = 6 - 7.5 = -1.5$
> Upper fence: $Q_3 + 1.5 \times \text{IQR} = 11 + 7.5 = 18.5$
> The value 42 is above 18.5, so it is flagged as an outlier. The other values all fall within the fences.

---

## 34.3. 3 · Describing relationships

When you have **two quantitative variables** measured on the same subjects — hours studied and exam score, temperature and ice cream sales, daily steps and resting heart rate — the question changes from "what is the distribution of one variable?" to "how do these two variables move together?"

The right starting point is always a **scatter plot**. Put one variable on the horizontal axis ($x$) and the other on the vertical axis ($y$). Each observation becomes a dot.

Before reaching for any formula, look at the plot and describe what you see using three properties.

**Direction.** As $x$ increases, does $y$ tend to increase (positive association) or decrease (negative association), or is there no consistent tendency?

**Form.** Does the pattern look like a straight line (linear), a curve, or something else entirely?

**Strength.** How tightly do the points cluster around the pattern? A tight cluster suggests a strong relationship. A diffuse cloud suggests a weak one.

Only if the form is linear does it make sense to ask "how strong is this linear relationship?" — which is what the correlation coefficient measures. If the form is curved, the correlation coefficient can give a misleading answer (we will see this shortly). This means the scatter plot comes before the formula because the formula only makes sense after the visual pattern is understood.

---

## 34.4. 4 · Pearson correlation

The **Pearson correlation coefficient** $r$ measures the strength and direction of a *linear* relationship between two quantitative variables. It is always a number between $-1$ and $+1$.

$$r = \frac{1}{n-1} \sum_{i=1}^{n} \left( \frac{x_i - \bar{x}}{s_x} \right) \left( \frac{y_i - \bar{y}}{s_y} \right)$$

This is the average of the products of the standardised $x$ and $y$ values. Standardising means subtracting the mean and dividing by the standard deviation, so each variable's scale disappears — $r$ is unit-free. This means correlation compares whether the two variables are above or below their own averages together, regardless of their units.

**The intuition:** when a standardised $x$ is positive (above average) and the corresponding standardised $y$ is also positive, their product is positive. When both are below average, both are negative, and the product is still positive. When one is above and one is below, the product is negative. If there is a consistent positive trend, positive products dominate and $r$ is close to $+1$. If there is a consistent negative trend, negative products dominate and $r$ is close to $-1$. If there is no consistent linear pattern, positive and negative products cancel and $r$ is near 0.

**A worked example.** Five students, with hours of sleep the night before an exam ($x$) and exam score out of 40 ($y$):

| Student | Sleep $x$ | Score $y$ |
|---|---|---|
| A | 5 | 22 |
| B | 6 | 25 |
| C | 7 | 30 |
| D | 8 | 33 |
| E | 9 | 35 |

$\bar{x} = 7$, $\bar{y} = 29$

$$s_x = \sqrt{\frac{(5-7)^2+(6-7)^2+(7-7)^2+(8-7)^2+(9-7)^2}{4}} = \sqrt{\frac{4+1+0+1+4}{4}} = \sqrt{2.5} \approx 1.581$$

$$s_y = \sqrt{\frac{(22-29)^2+(25-29)^2+(30-29)^2+(33-29)^2+(35-29)^2}{4}} = \sqrt{\frac{49+16+1+16+36}{4}} = \sqrt{29.5} \approx 5.431$$

| Student | $\frac{x_i-\bar{x}}{s_x}$ | $\frac{y_i-\bar{y}}{s_y}$ | product |
|---|---|---|---|
| A | −1.265 | −1.288 | 1.629 |
| B | −0.632 | −0.737 | 0.466 |
| C | 0 | 0.184 | 0 |
| D | 0.632 | 0.737 | 0.466 |
| E | 1.265 | 1.104 | 1.397 |

Sum of products: 3.958. Divide by $n - 1 = 4$: $r \approx 0.99$.

That is a very strong positive linear relationship. More sleep, higher score — and it is nearly perfectly linear across this small sample. This means a value of $r$ near 1 signals that the points follow a line closely, not that the slope itself is large.

**Interpreting $r$:** there is no single rule, but typical conventions are:

| $\lvert r \rvert$ | Interpretation |
|---|---|
| 0.9 – 1.0 | Very strong |
| 0.7 – 0.9 | Strong |
| 0.5 – 0.7 | Moderate |
| 0.3 – 0.5 | Weak |
| 0.0 – 0.3 | Very weak or none |

**$r$ measures only linear association**
If the relationship is curved, $r$ can be close to zero even when there is a strong and obvious pattern. A dataset where $y = x^2$ centred at zero will give $r \approx 0$ even though $y$ is completely determined by $x$. This is why you always look at the scatter plot first. This means correlation can miss strong non-linear structure entirely.

```
//| echo: false
{
  const W = 500, H = 340, PAD = 40;

  // Controls
  const rSlider = Inputs.range([-1, 1], {
    label: "Correlation strength",
    step: 0.1,
    value: 0.5
  });
  const outliersToggle = Inputs.toggle({ label: "Add outliers", value: false });
```

```
const presetBar = document.createElement("div");
presetBar.style.cssText = "display:flex; gap:0.5rem; flex-wrap:wrap; margin:0.5rem 0 0.25rem;";
const presets = [
  { label: "r = 0.9",  v: 0.9  },
  { label: "r = 0.5",  v: 0.5  },
  { label: "r = 0.1",  v: 0.1  },
  { label: "r = −0.7", v: -0.7 }
];
presets.forEach(p => {
  const btn = document.createElement("button");
  btn.textContent = p.label;
  btn.style.cssText = "padding:0.25rem 0.7rem; border:1px solid #d1d5db; border-radius:4px; bac
  btn.onclick = () => { rSlider.value = p.v; rSlider.dispatchEvent(new Event("input")); };
  presetBar.appendChild(btn);
});

// Box-Muller bivariate normal with correlation rho
function bivariateNormal(n, rho, seed) {
  // Simple LCG for reproducible-ish noise
  let s = seed || 42;
  function rand() { s = (s * 1664525 + 1013904223) & 0xffffffff; return (s >>> 0) / 0x100000000
  function stdNormal() {
    const u1 = rand() || 1e-10, u2 = rand();
    return Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
  }
  const pts = [];
  for (let i = 0; i < n; i++) {
    const z1 = stdNormal(), z2 = stdNormal();
    pts.push({ x: z1, y: rho * z1 + Math.sqrt(1 - rho * rho) * z2 });
  }
  return pts;
}

function computeR(pts) {
  const n = pts.length;
  if (n < 2) return NaN;
  const mx = pts.reduce((a, p) => a + p.x, 0) / n;
  const my = pts.reduce((a, p) => a + p.y, 0) / n;
  let num = 0, dx2 = 0, dy2 = 0;
  pts.forEach(p => {
    const dx = p.x - mx, dy = p.y - my;
    num += dx * dy; dx2 += dx * dx; dy2 += dy * dy;
  });
  const denom = Math.sqrt(dx2 * dy2);
  return denom < 1e-12 ? 0 : num / denom;
}
```

```
const svg = d3.create("svg").attr("width", W).attr("height", H)
  .style("border", "1px solid #e5e7eb").style("border-radius", "6px").style("background", "#fafafa

const xScale = d3.scaleLinear().domain([-3.5, 3.5]).range([PAD, W - PAD]);
const yScale = d3.scaleLinear().domain([-3.5, 3.5]).range([H - PAD, PAD]);

// Axes
svg.append("g").attr("transform", `translate(0,${yScale(0)})`).call(d3.axisBottom(xScale).ticks(6)
svg.append("g").attr("transform", `translate(${xScale(0)},0)`).call(d3.axisLeft(yScale).ticks(6))
svg.append("text").attr("x", W/2).attr("y", H - 4).attr("text-anchor","middle").attr("fill","#6b72
svg.append("text").attr("transform","rotate(-90)").attr("x", -H/2).attr("y", 12).attr("text-anchor

const dotsGroup = svg.append("g");

const rDisplay = document.createElement("div");
rDisplay.style.cssText = "text-align:center; margin:0.5rem 0; font-size:1.6rem; font-weight:700; c

const noteDisplay = document.createElement("div");
noteDisplay.style.cssText = "text-align:center; font-size:0.85em; color:#6b7280; margin-bottom:0.5

function update() {
  const rho = +rSlider.value;
  const addOut = outliersToggle.value;
  let pts = bivariateNormal(60, rho, 42);
  if (addOut) {
    pts = pts.concat([
      { x: 2.8,  y: -2.5 },
      { x: -2.6, y:  2.7 },
      { x:  3.0, y: -2.9 }
    ]);
  }
  const r = computeR(pts);

  dotsGroup.selectAll("circle").data(pts).join("circle")
    .attr("cx", d => xScale(d.x))
    .attr("cy", d => yScale(d.y))
    .attr("r", 4)
    .attr("fill", (d, i) => (addOut && i >= 60) ? "#dc2626" : "#3b82f6")
    .attr("opacity", (d, i) => (addOut && i >= 60) ? 0.85 : 0.55);

  rDisplay.textContent = `Computed r = ${r.toFixed(3)}`;
  const diff = Math.abs(r - rho);
  noteDisplay.textContent = addOut
    ? `Target ρ = ${rho.toFixed(1)} · ${diff > 0.05 ? `Outliers shifted r by ${(r - rho).toFixed(3
    : `Target ρ = ${rho.toFixed(1)}`;
}
```

```
  rSlider.addEventListener("input", update);
  outliersToggle.addEventListener("input", update);
  update();

  const container = document.createElement("div");
  container.style.cssText = "margin:1rem 0; font-family:inherit;";
  const label = document.createElement("p");
  label.style.cssText = "font-size:0.9em; color:#374151; margin-bottom:0.5rem;";
  label.textContent = "Adjust the slider to set a target correlation, then explore how the cloud
  container.append(label, rSlider, presetBar, outliersToggle, svg.node(), rDisplay, noteDisplay);
  return container;
}
```

---

## 34.5. 5 · Correlation does not imply causation

This section deserves real time. It is not a disclaimer. It is one of the most practically important ideas in this entire volume.

Finding that $r = 0.78$ between two variables does not tell you that one causes the other. It tells you they are associated — they tend to move together. Association and causation are different things, and confusing them is one of the most common errors in public reasoning about data.

There are three mechanisms that produce correlation without causation.

---

**Mechanism 1: Common cause (confounding)**

Ice cream sales and drowning deaths are positively correlated. In months when ice cream sales are high, drowning deaths are also high. Does eating ice cream cause drowning?

No. Both are caused by a third variable: hot weather. When it is hot, more people buy ice cream *and* more people swim. The ice cream and the drownings are both responding to temperature. Temperature is the *confounding variable* — a common cause that creates an apparent relationship between two things that do not directly affect each other.

This is extremely common in health and social science research. Two variables that seem related often share an underlying driver.

---

### Mechanism 2: Reverse causation

Suppose a survey finds that people who sleep fewer hours are more likely to be depressed. The tempting interpretation: poor sleep causes depression.

But the causal arrow might point the other way. Depression is a common cause of disturbed sleep. The correlation is real; the causal direction is not established by the correlation alone.

Or both things cause each other in a loop — poor sleep worsens mood, worsened mood disrupts sleep. Correlation cannot untangle this.

---

### Mechanism 3: Coincidence

Nicolas Cage releases more films in years when more people drown in swimming pools. The correlation over a twelve-year period was reported at $r \approx 0.67$.

There is obviously no causal connection. With a sufficiently large collection of variables measured over time, some pairs will be correlated by chance. The internet is full of these "spurious correlations" — per capita cheese consumption and deaths by bedsheet tangling, US spending on science and suicides by hanging. These are real correlations in the data. They mean nothing.

---

### What does establish causation?

Observational data (just measuring things in the world) can suggest causal hypotheses. Establishing causation requires:

- **Randomised controlled experiments** — randomly assign subjects to conditions so confounders are distributed equally across groups.
- **Strong temporal evidence** — the cause must precede the effect.
- **Plausible mechanism** — a credible explanation of *how* one thing could cause the other.
- **Replication** — the relationship holds across different populations and study designs.

When you read a headline claiming "X causes Y," your first question should be: was this a randomised experiment, or an observational study? If the latter, the correct language is "X is associated with Y" — not "X causes Y."

> **The student's toolkit for evaluating causal claims**
> Ask three questions:
>
> 1. Could a third variable (confounder) explain both X and Y?
> 2. Could the direction be reversed — does Y cause X instead?
> 3. Could this be coincidence in a large dataset?
>
> If any of these is plausible, the causal claim is not established. This means observational association is evidence for a question, not a final answer about cause.

---

## 34.6. 6 · Linear regression

Once you have established (from a scatter plot) that two variables have a linear form, you can model that relationship with a line. That line lets you do two things the correlation coefficient cannot: make predictions for new values, and quantify how much $y$ changes per unit of $x$.

**The problem.** Suppose you have a scatter of points that approximately follow a line. Which line? You could draw many lines that pass "through" the cloud. The **least-squares line** is the one that minimises the sum of squared residuals — where a *residual* $e_i = y_i - \hat{y}_i$ is the vertical distance from each point to the line.

Why squared? Squaring makes all residuals positive (so positive and negative errors don't cancel), and it penalises large errors more heavily than small ones. The least-squares line is the unique line that minimises the total squared error — it is "as close as possible" to all the points simultaneously. This means regression is not guessing a line by eye; it is choosing the line that makes overall prediction error as small as possible by a precise rule.

**The formulas.** For the least-squares line $\hat{y} = b_0 + b_1 x$:

$$b_1 = r \cdot \frac{s_y}{s_x}$$

$$b_0 = \bar{y} - b_1 \bar{x}$$

where $r$ is the Pearson correlation, $s_x$ and $s_y$ are the standard deviations of $x$ and $y$, and $\bar{x}$, $\bar{y}$ are the means. This means the slope depends on two things at once: how strongly the variables move together, and how their units and spreads compare.

**Interpretation.**

$b_1$ (slope): for every one-unit increase in $x$, the predicted value of $y$ changes by $b_1$ units. Always interpret this in context — the units matter.

$b_0$ (intercept): the predicted value of $y$ when $x = 0$. This is sometimes meaningful (it might represent a baseline), but often extrapolating to $x = 0$ is outside the range of the data and the intercept is just an algebraic anchor with no practical interpretation.

**The regression line always passes through $(\bar{x}, \bar{y})$.** This is a useful sanity check. This means the fitted line is anchored at the centre of the data cloud, not floating independently of the observed averages.

```
//| echo: false
{
  const W = 500, H = 340, PAD = 45;
  const MAX_PTS = 20;

  let points = [];
  let showResiduals = false;

  const svg = d3.create("svg")
    .attr("width", W).attr("height", H)
```

```
  .style("border", "1px solid #e5e7eb").style("border-radius", "6px")
  .style("background", "#fafafa").style("cursor", "crosshair");

const xScale = d3.scaleLinear().domain([0, 10]).range([PAD, W - PAD]);
const yScale = d3.scaleLinear().domain([0, 10]).range([H - PAD, PAD]);

// Grid lines
const grid = svg.append("g").attr("opacity", 0.25);
xScale.ticks(5).forEach(t => grid.append("line").attr("x1", xScale(t)).attr("x2", xScale(t)).attr
yScale.ticks(5).forEach(t => grid.append("line").attr("x1", PAD).attr("x2", W - PAD).attr("y1", y

svg.append("g").attr("transform", `translate(0,${H - PAD})`).call(d3.axisBottom(xScale).ticks(5))
svg.append("g").attr("transform", `translate(${PAD},0)`).call(d3.axisLeft(yScale).ticks(5)).attr("
svg.append("text").attr("x", W/2).attr("y", H - 5).attr("text-anchor","middle").attr("fill","#6b72
svg.append("text").attr("transform","rotate(-90)").attr("x", -H/2).attr("y", 14).attr("text-ancho

const hintText = svg.append("text").attr("x", W/2).attr("y", H/2)
  .attr("text-anchor","middle").attr("fill","#9ca3af").attr("font-size","13px")
  .text("Click anywhere to add a data point");

const residGroup = svg.append("g");
const lineGroup = svg.append("g");
const dotsGroup = svg.append("g");

function leastSquares(pts) {
  const n = pts.length;
  if (n < 2) return null;
  const mx = pts.reduce((a, p) => a + p.x, 0) / n;
  const my = pts.reduce((a, p) => a + p.y, 0) / n;
  let num = 0, den = 0;
  pts.forEach(p => { const dx = p.x - mx; num += dx * (p.y - my); den += dx * dx; });
  if (Math.abs(den) < 1e-12) return null;
  const b1 = num / den;
  const b0 = my - b1 * mx;
  const yhat = pts.map(p => b0 + b1 * p.x);
  const ssTot = pts.reduce((a, p) => a + (p.y - my) ** 2, 0);
  const ssRes = pts.reduce((a, p, i) => a + (p.y - yhat[i]) ** 2, 0);
  const r2 = ssTot < 1e-12 ? 1 : 1 - ssRes / ssTot;
  return { b0, b1, r2, yhat };
}

function render() {
  hintText.style("display", points.length === 0 ? null : "none");

  const fit = leastSquares(points);

  // Residual lines
```

```
  residGroup.selectAll("line").data(showResiduals && fit ? points : [])
    .join("line")
    .attr("x1", d => xScale(d.x)).attr("x2", d => xScale(d.x))
    .attr("y1", d => yScale(d.y))
    .attr("y2", (d, i) => yScale(fit.yhat[i]))
    .attr("stroke", "#ef4444").attr("stroke-width", 1.5).attr("opacity", 0.7);

  // Regression line
  if (fit) {
    const x0 = 0, x1 = 10;
    lineGroup.selectAll("line").data([1]).join("line")
      .attr("x1", xScale(x0)).attr("x2", xScale(x1))
      .attr("y1", yScale(fit.b0 + fit.b1 * x0))
      .attr("y2", yScale(fit.b0 + fit.b1 * x1))
      .attr("stroke", "#1d4ed8").attr("stroke-width", 2).attr("opacity", 0.85);
  } else {
    lineGroup.selectAll("line").remove();
  }

  // Dots
  dotsGroup.selectAll("circle").data(points).join("circle")
    .attr("cx", d => xScale(d.x)).attr("cy", d => yScale(d.y))
    .attr("r", 5).attr("fill", "#3b82f6").attr("stroke", "#1d4ed8").attr("stroke-width", 1).att

  // Stats display
  if (fit) {
    statsDiv.innerHTML = `
      <span style="margin-right:1.5rem;"><em>b</em><sub>0</sub> = <strong>${fit.b0.toFixed(3)}<
      <span style="margin-right:1.5rem;"><em>b</em><sub>1</sub> = <strong>${fit.b1.toFixed(3)}<
      <span><em>R</em><sup>2</sup> = <strong>${fit.r2.toFixed(3)}</strong></span>
      <span style="margin-left:1rem; color:#6b7280; font-size:0.85em;">(${points.length} point$
    statsDiv.style.color = "#1e3a8a";
  } else {
    statsDiv.textContent = points.length < 2 ? "Add at least 2 points to fit a line." : "";
    statsDiv.style.color = "#6b7280";
  }
}

svg.on("click", function(event) {
  if (points.length >= MAX_PTS) return;
  const [px, py] = d3.pointer(event);
  const x = xScale.invert(px), y = yScale.invert(py);
  if (x < 0 || x > 10 || y < 0 || y > 10) return;
  points.push({ x, y });
  render();
});
```

```
  // Controls
  const residBtn = document.createElement("button");
  residBtn.textContent = "Show residuals";
  residBtn.style.cssText = "padding:0.3rem 0.8rem; border:1px solid #d1d5db; border-radius:4px; bacl
  residBtn.onclick = () => {
    showResiduals = !showResiduals;
    residBtn.textContent = showResiduals ? "Hide residuals" : "Show residuals";
    residBtn.style.background = showResiduals ? "#fee2e2" : "#fff";
    render();
  };

  const clearBtn = document.createElement("button");
  clearBtn.textContent = "Clear";
  clearBtn.style.cssText = "padding:0.3rem 0.8rem; border:1px solid #e5e7eb; border-radius:4px; bacl
  clearBtn.onclick = () => { points = []; showResiduals = false; residBtn.textContent = "Show residu

  const statsDiv = document.createElement("div");
  statsDiv.style.cssText = "min-height:1.6em; font-size:0.95em; text-align:center; margin-top:0.4rem

  const ctrlRow = document.createElement("div");
  ctrlRow.style.cssText = "display:flex; gap:0.5rem; margin:0.5rem 0;";
  ctrlRow.append(residBtn, clearBtn);

  const hint = document.createElement("p");
  hint.style.cssText = "font-size:0.85em; color:#374151; margin-bottom:0.4rem;";
  hint.textContent = `Click on the plot to add points (up to ${MAX_PTS}). The least-squares line upd

  render();

  const container = document.createElement("div");
  container.style.cssText = "margin:1rem 0; font-family:inherit;";
  container.append(hint, svg.node(), ctrlRow, statsDiv);
  return container;
}
```

---

## 34.7. 7 · $R^2$ and residual plots

### 34.7.1. The coefficient of determination

$R^2$ (R-squared) measures how well the regression line fits the data. For simple linear regression, $R^2 = r^2$ — the square of the Pearson correlation.

$$R^2 = r^2$$

$R^2$ is the **proportion of the variance in $y$ that is explained by the linear relationship with $x$.** It ranges from 0 to 1.

$R^2 = 0.72$ means: 72% of the variation in $y$ is accounted for by the linear relationship with $x$. The remaining 28% is due to other factors not in the model, natural variability, or measurement error.

$R^2 = 0.25$ means the line captures only 25% of the variation. A lot is left unexplained — you might be missing important variables, or the relationship might not be well-described by a line.

$R^2 = 0.95$ means 95% of the variation is explained — a strong fit. This means a high $R^2$ says the line tracks the observed variation closely, but it does not by itself prove causation or guarantee that the model is appropriate outside the observed range.

> **$R^2$ in plain language**
> Imagine the simplest possible model for $y$: just predict $\bar{y}$ for everyone. That model ignores $x$ completely. The total spread around that flat prediction is the total variance in $y$.
> The regression line does better — it accounts for $x$. $R^2$ measures how much better: what fraction of the total variance did the line explain away?

### 34.7.2. Residual plots

Fitting a line and computing $R^2$ is not the end of the analysis. You need to check whether the line is actually the right model.

A **residual plot** shows the residuals $e_i = y_i - \hat{y}_i$ on the vertical axis against the fitted values $\hat{y}_i$ (or against $x$) on the horizontal axis. A good model produces a scatter with no pattern — residuals roughly centred at zero, with no curve, fan shape, or clustering.

If the linear model is appropriate, the residuals should look like random scatter around zero — no pattern.

**What patterns signal problems:**

- A curved pattern (residuals positive at the extremes, negative in the middle) means the true relationship is nonlinear. A line is the wrong model.
- A funnel shape (residuals spreading out as $x$ increases) means the variability in $y$ changes with $x$. The model's uncertainty is not constant — a standard regression assumption is violated.
- A few extreme residuals are outliers in the regression sense. They may be data errors or genuinely unusual cases.

The residual plot is the diagnostic. A clean residual plot validates your model; a patterned one tells you to reconsider. This means residuals are not a minor afterthought. They are the main check on whether the simple line has missed structure in the data.

## 34.8. 8 · End-to-end worked example

A café tracks average daily temperature (°C) and the number of iced coffee drinks sold. Here are eight days of data:

| Day | Temp $x$ | Drinks $y$ |
|---|---|---|
| 1 | 18 | 34 |
| 2 | 21 | 40 |
| 3 | 24 | 48 |
| 4 | 27 | 55 |
| 5 | 29 | 61 |
| 6 | 31 | 65 |
| 7 | 33 | 72 |
| 8 | 36 | 78 |

**Step 1: Compute the summary statistics.**

$n = 8$

$\bar{x} = (18 + 21 + 24 + 27 + 29 + 31 + 33 + 36)/8 = 219/8 = 27.375$

$\bar{y} = (34 + 40 + 48 + 55 + 61 + 65 + 72 + 78)/8 = 453/8 = 56.625$

$s_x \approx 6.117$, $s_y \approx 15.35$

(These are computed using the standard deviation formula from Section 2.)

**Step 2: Compute the Pearson correlation.**

Running through all 8 standardised products (same procedure as the 5-row table in Section 4) gives:

$$r \approx 0.999$$

A near-perfect positive linear relationship. This is not surprising — the scatter plot would show the points nearly on a straight line.

**Step 3: Find the least-squares line.**

$$b_1 = r \cdot \frac{s_y}{s_x} = 0.999 \times \frac{15.35}{6.117} \approx 0.999 \times 2.509 \approx 2.506$$

$$b_0 = \bar{y} - b_1\bar{x} = 56.625 - 2.506 \times 27.375 \approx 56.625 - 68.602 \approx -11.97$$

The regression equation is:

$$\hat{y} = -11.97 + 2.506x$$

**Step 4: Interpret slope and intercept.**

*Slope:* For every 1°C increase in temperature, the model predicts about 2.5 additional iced coffees sold. This makes intuitive sense.

*Intercept:* The model predicts −11.97 drinks at 0°C. This is nonsensical — no café sells negative drinks — but we should not expect the intercept to be meaningful here. The data ranges from 18°C to 36°C. Extrapolating to 0°C is far outside that range. This means intercepts should be interpreted only when $x = 0$ is both inside the data range and meaningful in the real situation.

**Step 5: Predict a new value.**

On a 30°C day, the model predicts:

$$\hat{y} = -11.97 + 2.506 \times 30 = -11.97 + 75.18 = 63.21$$

Approximately 63 iced coffees.

**Step 6: Compute $R^2$.**

$$R^2 = r^2 = (0.999)^2 \approx 0.998$$

The linear model explains about 99.8% of the variation in drinks sold. Temperature is almost the entire story here (within this range and this simple model). This means the fit is excellent for these observed days, but the claim should stay tied to this temperature range and this simple one-variable model.

**Step 7: Check a residual.**

For Day 5, the actual value is $y_5 = 61$. The fitted value is:

$$\hat{y}_5 = -11.97 + 2.506 \times 29 = -11.97 + 72.674 = 60.70$$

Residual: $e_5 = 61 - 60.70 = 0.30$

The model is off by less than one drink on this day. The residuals for all eight days would be small and show no obvious pattern — this is exactly what a well-fitting linear model looks like.

---

**Computing form of the regression slope**
An algebraically equivalent form useful for hand calculation:

$$b_1 = \frac{\sum x_i y_i - n\bar{x}\bar{y}}{\sum x_i^2 - n\bar{x}^2}$$

This is the same formula in computing form — useful when you have the raw sums but not the deviations. You will use this form in Exercise 4.

## 34.9. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
```

```
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

**Exercise 1.** The following dataset shows the number of hours seven people spent on social media last week:

$$3,\ 7,\ 7,\ 9,\ 12,\ 14,\ 38$$

Compute the mean, median, IQR, and identify any outliers using the $1.5 \times \text{IQR}$ rule.

```
//| echo: false
makeStepperHTML(1, [
  { op: "Sort the data",
    eq: "3,\\ 7,\\ 7,\\ 9,\\ 12,\\ 14,\\ 38",
    note: "Already sorted. n = 7." },
  { op: "Compute the mean",
    eq: "\\bar{x} = \\frac{3+7+7+9+12+14+38}{7} = \\frac{90}{7} \\approx 12.9" },
  { op: "Find the median",
    eq: "\\text{median} = 9",
    note: "The 4th value in 7 sorted values." },
  { op: "Find Q₁",
    eq: "Q_1 = 7",
    note: "Median of the lower half: 3, 7, 7 → middle value is 7." },
  { op: "Find Q₃",
```

```
    eq: "Q_3 = 14",
    note: "Median of the upper half: 12, 14, 38 → middle value is 14." },
  { op: "Compute the IQR",
    eq: "\\text{IQR} = Q_3 - Q_1 = 14 - 7 = 7" },
  { op: "Find the fences",
    eq: "\\text{Lower: } 7 - 1.5(7) = -3.5 \\qquad \\text{Upper: } 14 + 1.5(7) = 24.5" },
  { op: "Identify outliers",
    eq: "38 > 24.5 \\Rightarrow 38 \\text{ is an outlier}",
    note: "No values fall below −3.5. The value 38 is flagged." },
  { op: "Summary",
    eq: "\\bar{x} \\approx 12.9,\\ \\text{median} = 9,\\ \\text{IQR} = 7,\\ \\text{outlier: } 38",
    note: "The mean is pulled far above the median by the outlier. Report median as centre." }
])
```

---

**Exercise 2.** A scatter plot of students' hours of revision ($x$) and exam percentage ($y$) shows:

- Points rise from bottom-left to top-right
- The points cluster fairly tightly around an imaginary straight line
- There are no pronounced curves or bends in the pattern

Describe the direction, form, and strength of the association, and estimate whether $r$ is closer to 0.1, 0.7, or 0.95. Justify your estimate.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Direction",
    eq: "\\text{Positive}",
    note: "Points rise from bottom-left to top-right: as revision hours increase, exam score increas
  { op: "Form",
    eq: "\\text{Linear}",
    note: "The pattern follows a straight line, not a curve." },
  { op: "Strength",
    eq: "\\text{Strong}",
    note: "The points cluster tightly around the line — little scatter." },
  { op: "Estimate r",
    eq: "r \\approx 0.85\\text{-}0.95",
    note: "Strong positive linear → r well above 0.7. 'Fairly tight' suggests not quite perfect. 0.
])
```

---

**Exercise 3.** A study reports $r = 0.82$ between the number of books students own and their reading comprehension test scores. For each of the following three statements, say whether it is correct or incorrect and explain why.

*(a)* "Students who own more books tend to score higher on reading tests."

*(b)* "Owning more books causes higher reading comprehension."

*(c)* "82% of students who own books score above average on reading tests."

```
//| echo: false
makeStepperHTML(3, [
  { op: "Statement (a)",
    eq: "\\text{Correct}",
    note: "r = 0.82 is a strong positive association. As x (books) increases, y (score) tends to
  { op: "Statement (b)",
    eq: "\\text{Incorrect — confounding likely}",
    note: "This is a causal claim from observational data. A common cause (family income, reading
  { op: "Statement (c)",
    eq: "\\text{Incorrect — misreads R}",
    note: "r = 0.82 is not a percentage of people. It is a dimensionless measure of linear associ
])
```

---

**Exercise 4.** A dataset of $n = 6$ observations has the following summary totals:

$$n = 6, \quad \sum x = 42, \quad \sum y = 78, \quad \sum x^2 = 322, \quad \sum xy = 591$$

Use the computing formulas below to find $b_1$ and $b_0$, then write the regression equation.

$$b_1 = \frac{\sum xy - \frac{(\sum x)(\sum y)}{n}}{\sum x^2 - \frac{(\sum x)^2}{n}} \qquad b_0 = \bar{y} - b_1 \bar{x}$$

```
//| echo: false
makeStepperHTML(4, [
  { op: "Compute the means",
    eq: "\\bar{x} = \\frac{42}{6} = 7 \\qquad \\bar{y} = \\frac{78}{6} = 13" },
  { op: "Compute the numerator of b₁",
    eq: "\\sum xy - \\frac{(\\sum x)(\\sum y)}{n} = 591 - \\frac{42 \\times 78}{6} = 591 - 546 =
  { op: "Compute the denominator of b₁",
    eq: "\\sum x^2 - \\frac{(\\sum x)^2}{n} = 322 - \\frac{42^2}{6} = 322 - \\frac{1764}{6} = 322
  { op: "Compute b₁",
    eq: "b_1 = \\frac{45}{28} \\approx 1.607" },
  { op: "Compute b₀",
    eq: "b_0 = \\bar{y} - b_1 \\bar{x} = 13 - 1.607 \\times 7 = 13 - 11.25 = 1.75" },
  { op: "Write the regression equation",
    eq: "\\hat{y} = 1.75 + 1.607x",
    note: "Check: at x = x̄ = 7, ŷ = 1.75 + 1.607(7) = 1.75 + 11.25 = 13 = ȳ . ?" }
])
```

---

**Exercise 5.** A researcher models the relationship between daily temperature ($x$, in °C) and the number of parkrun participants ($y$). The regression equation is:

$$\hat{y} = 120 + 3.8x$$

(a) Interpret the slope in context.

(b) Interpret the intercept in context. Is it meaningful here?

(c) Predict the number of participants on a 15°C day.

(d) On one 15°C day, the actual count was 182. Compute the residual and interpret it.

```
//| echo: false
makeStepperHTML(5, [
  { op: "(a) Interpret the slope",
    eq: "b_1 = 3.8",
    note: "For each additional 1°C increase in temperature, the model predicts approximately 3.8 mo
  { op: "(b) Interpret the intercept",
    eq: "b_0 = 120",
    note: "The model predicts 120 participants at 0°C. This may not be meaningful — parkruns in free
  { op: "(c) Predict at x = 15",
    eq: "\\hat{y} = 120 + 3.8 \\times 15 = 120 + 57 = 177" },
  { op: "(d) Compute the residual",
    eq: "e = y - \\hat{y} = 182 - 177 = 5",
    note: "The actual count was 5 higher than the model predicted. On this particular day, something
])
```

---

**Exercise 6 — Critical thinking.** A newspaper runs the headline:

> *Students who eat breakfast get better grades — schools should mandate breakfast to raise academic performance.*

The headline is based on a large survey showing a positive correlation between breakfast-eating habits and GPA. Using the concepts from this chapter, evaluate the causal claim. What questions would you ask before accepting the conclusion? What alternative explanations exist?

```
//| echo: false
makeStepperHTML(6, [
  { op: "Identify the type of evidence",
    eq: "\\text{Observational study (survey data)}",
    note: "Students were not randomly assigned to eat or skip breakfast. The researcher observed ex
  { op: "Ask: could a confounder explain both?",
    eq: "\\text{Possible confounder: household stability / socioeconomic status}",
```

```
    note: "Students from stable, higher-income households are more likely to eat breakfast regula
  { op: "Ask: could causation be reversed?",
    eq: "\\text{Less likely here, but consider: motivated students may also structure mornings be
    note: "Academically motivated students tend to have more structured routines. The structure (
  { op: "Ask: what would establish causation?",
    eq: "\\text{A randomised controlled trial}",
    note: "Randomly assign students to a provided breakfast programme vs control. Measure grades.
  { op: "Evaluate the policy conclusion",
    eq: "\\text{The jump from correlation to mandating breakfast is premature}",
    note: "Even if breakfast does help (plausible mechanistically — blood glucose affects concent
  { op: "Summary verdict",
    eq: "\\text{Association} \\neq \\text{causation}",
    note: "The correlation is real. The causal claim and the policy conclusion are not supported
])
```

---

## 34.10. Where this chapter leaves you

Volume 6 is complete.

You can now describe a distribution in full — its centre, spread, and shape — and choose the right summary for the kind of data you have and the presence or absence of outliers. You can describe the relationship between two variables from a scatter plot before any formula touches the data. You can measure that relationship with $r$, interpret what the number actually means, and recognise the one thing it emphatically does not mean: causation. And you can build the simplest predictive model, the least-squares line, interpret its slope and intercept in context, assess its fit with $R^2$, and diagnose problems with a residual plot.

That is a complete analytical toolkit for real data at this level.

The road ahead is wide. Volume 7 opens with the natural extension of everything here: **multiple regression**, where more than one predictor variable ($x_1, x_2, \ldots$) is used to model $y$ simultaneously. The least-squares criterion is the same; the algebra requires linear algebra. Multiple regression is also the foundation of the simplest supervised machine learning model — gradient descent and neural networks are, at their roots, sophisticated generalisations of fitting a line to data and checking how well it explains what you observed. data. Everything you have learned in this chapter is load-bearing for what follows.

**Where this shows up**

- A climate scientist analysing temperature trends over decades is fitting a regression line to time-series data and checking the residuals.
- A sports analyst building a player rating model is computing correlations between performance metrics and outcomes, then building a regression to predict results.
- A public health researcher reporting that "smoking is associated with lung cancer" is using the careful causal language earned by decades of controlled studies — because correlation alone was not enough to establish it.
- A data scientist building a recommendation system starts with correlation matrices. The step from there to collaborative filtering and neural networks is real, but this is where the intuition lives.

The ideas are simple. Their reach is extraordinary.

# Part VII.

# Volume 7 — Engineering mathematics: ODEs

# 35. Volume 7 — Engineering mathematics

This volume is the core mathematics sequence that first- and second-year engineering students usually experience as separate courses: ordinary differential equations, linear algebra, multivariable calculus, vector calculus, Fourier methods, PDEs, complex analysis, numerical methods, optimisation, and probability.

The build is intentionally broad before it becomes specialised. The aim is not to imitate a single textbook chapter order, but to give readers the mathematical objects that upper-year engineering courses assume they can already use: state equations, eigenmodes, fields, transforms, discretised systems, feasible regions, and distributions.

Volume 8 extends this into the mathematics that appears inside third- and fourth-year discipline courses: control and feedback, discrete-time systems, continuum models, computational simulation, estimation, reliability, and nonlinear design optimisation.

## 35.1. Sections

| Section | Core move |
|---|---|
| ODEs | Model change by relating a system to its own rates |
| Linear algebra and vector calculus | Move from single equations to spaces, fields, and operators |
| Fourier analysis and PDEs | Decompose signals and solve distributed physical systems |
| Complex analysis | Use the complex plane as a practical computational tool |
| Numerical methods | Replace exact formulas with controlled approximation |
| Optimisation and probability | Decide and infer under constraints and uncertainty |

# 36. First-order ODEs

When the rate of change depends on the state

An RC circuit: a resistor and capacitor in series, connected to a voltage source. The capacitor charges up. At $t = 0$ you disconnect the source. The voltage across the capacitor starts dropping.

The question is not "what is the voltage after 1 second?" You could measure that with a multimeter. The question is: what *function* describes the voltage at every instant? How does it fall — linearly, exponentially, as a step? Does it depend on $R$ and $C$ individually or only on their product? What happens if you double the capacitance?

To answer those questions you need an equation that the voltage function must satisfy. That equation involves the derivative of the voltage — it is an ordinary differential equation. Solving it means finding the function, not a number.

This chapter develops two techniques — separation of variables and the integrating factor — that between them handle a large fraction of first-order ODEs encountered in engineering. The chapter closes with the existence and uniqueness theorem, which tells you something crucial: you should check that a solution exists before spending time looking for one.

---

## 36.1. The ODE problem

A function $y(x)$ satisfies an **ordinary differential equation** when substituting $y$ and its derivatives into the equation produces an identity. The simplest possible example:

$$y' = y$$

This asks: for which functions does the derivative equal the function itself? The answer is $y = Ce^x$ for any constant $C$. There is not one solution — there is a family of them, one curve for each value of $C$.

This is the fundamental shift from algebra. In algebra you solve for a number. In an ODE you solve for a function.

### 36.1.1. Order and linearity

The **order** of an ODE is the order of the highest derivative that appears. This chapter covers first-order ODEs: equations involving $y$ and $y'$, but not $y''$ or higher.

An ODE is **linear** if it can be written in the form

$$a_n(x)\, y^{(n)} + a_{n-1}(x)\, y^{(n-1)} + \cdots + a_1(x)\, y' + a_0(x)\, y = f(x)$$

where all coefficients depend only on $x$, not on $y$. For first-order:

$$a_1(x)\, y' + a_0(x)\, y = f(x)$$

If $y$ or $y'$ appear raised to a power, inside a trig function, multiplied together, or otherwise nonlinearly, the equation is **nonlinear**. Nonlinear first-order ODEs are harder; two important special cases (separable equations and the logistic equation) are tractable analytically, but most require numerical methods.

### 36.1.2. General and particular solutions

The **general solution** of a first-order ODE contains one arbitrary constant $C$. Geometrically it is a one-parameter family of curves — every value of $C$ gives one solution curve.

A **particular solution** is obtained by applying an **initial condition**: a specified value $y(x_0) = y_0$. This pins one curve from the family.

For example, $y' = 2x$ has general solution $y = x^2 + C$. The initial condition $y(0) = 3$ gives $C = 3$, so the particular solution is $y = x^2 + 3$.

### 36.1.3. Why existence and uniqueness matters

Before solving an ODE, it is worth knowing whether a solution exists and, if it does, whether it is unique. If uniqueness fails, a numerical solver that finds *a* solution may not find *the* solution relevant to your physical problem. The Picard-Lindelöf theorem (stated precisely at the end of this chapter) gives sufficient conditions for both. In practice: check the theorem first, solve second.

---

## 36.2. Separable ODEs

A first-order ODE is **separable** if it can be written in the form

$$\frac{dy}{dx} = g(x)\,h(y)$$

The method is to separate all $y$-dependence to the left and all $x$-dependence to the right, then integrate both sides.

**Method.** Divide both sides by $h(y)$ (assuming $h(y) \neq 0$):

$$\frac{1}{h(y)}\frac{dy}{dx} = g(x)$$

Interpret $\frac{dy}{dx}\,dx = dy$ and integrate. This is shorthand for a substitution — treating $dy/dx$ as a ratio is a notational convenience that can be made rigorous via the substitution rule for integrals; the result is always valid for separable ODEs.

$$\int \frac{dy}{h(y)} = \int g(x)\,dx + C$$

The constant of integration appears once — it is conventional to write it on the right. The initial condition then determines $C$.

### 36.2.1. Worked example 1: exponential growth and decay

Solve $\dfrac{dy}{dt} = ky$, with $y(0) = y_0$.

This is separable. Divide by $y$ (assuming $y \neq 0$):

$$\frac{dy}{y} = k\,dt$$

Integrate both sides:

$$\ln|y| = kt + C_1$$

Exponentiate:

$$|y| = e^{C_1}e^{kt}$$

Three sub-steps are compressed here: (1) remove the absolute value signs by introducing a $\pm$ — so $|y| = e^{C_1}e^{kt}$ becomes $y = \pm e^{C_1}e^{kt}$; (2) rename $\pm e^{C_1}$ as $A$, which is still a nonzero constant since $e^{C_1} > 0$ always; (3) check whether $y = 0$ is also a solution — substituting gives $0 = k \cdot 0$, which is true, so $y = 0$ is valid and we extend $A$ to allow zero as well. Collecting these observations:

$$y(t) = Ae^{kt}$$

Apply the initial condition $y(0) = y_0$:

$$y_0 = Ae^0 = A$$

Particular solution:

$$\boxed{y(t) = y_0\, e^{kt}}$$

The constant $k$ controls the behaviour. If $k > 0$ the function grows without bound — radioactive build-up, compound interest, early-stage epidemic spread. If $k < 0$ (write $k = -\lambda$, $\lambda > 0$) the function decays to zero — radioactive decay, drug clearance, capacitor discharge. The quantity $\tau = 1/|\lambda|$ is the **time constant**: the time for the function to fall to $e^{-1} \approx 36.8\%$ of its initial value, or equivalently, the time at which the tangent at $t = 0$ would reach zero.

### 36.2.2. Worked example 2: Newton's law of cooling

A body at temperature $T$ loses heat to an environment at constant temperature $T_{\text{env}}$. Newton's law of cooling states that the rate of heat loss is proportional to the temperature difference:

$$\frac{dT}{dt} = -k(T - T_{\text{env}}), \qquad k > 0$$

The negative sign ensures cooling: if $T > T_{\text{env}}$, the right side is negative, so $T$ decreases. The equation is separable, but a substitution makes it cleaner. Let $u = T - T_{\text{env}}$. Then $\frac{du}{dt} = \frac{dT}{dt}$ (since $T_{\text{env}}$ is constant), and the equation becomes:

$$\frac{du}{dt} = -ku$$

This is $y' = ky$ with $k \to -k$. By the previous example:

$$u(t) = u_0\, e^{-kt}$$

where $u_0 = T(0) - T_{\text{env}}$. Substituting back:

$$\boxed{T(t) = T_{\text{env}} + (T_0 - T_{\text{env}})\, e^{-kt}}$$

The temperature difference decays exponentially. As $t \to \infty$, $T \to T_{\text{env}}$: the body equilibrates with its surroundings, as expected. The constant $k$ depends on the thermal properties of the body and the medium — a copper block cools faster than a ceramic mug of the same mass (larger $k$).

---

```
//| echo: false
viewof vizEq1 = Inputs.select(["y' = ky", "y' = y(1−y)"], {label: "Equation"})
viewof vizK   = Inputs.range([-2, 2], {step: 0.1, value: -0.5, label: "k (for y = ky)"})
viewof vizY0  = Inputs.range([0.1, 3], {step: 0.05, value: 1, label: "Initial condition y(0)"})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const W = 560, H = 320;
  const margin = {top: 20, right: 120, bottom: 40, left: 50};
  const w = W - margin.left - margin.right;
  const h = H - margin.top  - margin.bottom;

  const tMin = 0, tMax = 5;
  const yMin = -0.5, yMax = 3;

  const xScale = d3.scaleLinear([tMin, tMax], [0, w]);
  const yScale = d3.scaleLinear([yMin, yMax], [h, 0]);

  const svg = d3.create("svg")
      .attr("width",  W)
      .attr("height", H)
      .style("font-family", "sans-serif")
      .style("font-size", "12px");

  const g = svg.append("g")
      .attr("transform", `translate(${margin.left},${margin.top})`);

  // axes
  g.append("g").attr("transform", `translate(0,${h})`).call(d3.axisBottom(xScale).ticks(5));
  g.append("g").call(d3.axisLeft(yScale).ticks(6));

  // axis labels
  g.append("text")
      .attr("x", w / 2).attr("y", h + 34)
      .attr("text-anchor", "middle").text("t");
  g.append("text")
      .attr("transform", "rotate(-90)")
      .attr("x", -h / 2).attr("y", -38)
      .attr("text-anchor", "middle").text("y");

  // slope function
  const isLogistic = vizEq1 === "y' = y(1−y)";
  const slope = (t, y) => isLogistic ? y * (1 - y) : vizK * y;

  // direction field
```

```
const nT = 20, nY = 18;
const arrowLen = 0.18;
for (let i = 0; i <= nT; i++) {
  for (let j = 0; j <= nY; j++) {
    const t = tMin + (i / nT) * (tMax - tMin);
    const y = yMin + (j / nY) * (yMax - yMin);
    const s = slope(t, y);
    const mag = Math.sqrt(1 + s * s);
    const dt = arrowLen / mag;
    const dy = s * arrowLen / mag;
    const x1 = xScale(t - dt / 2);
    const y1 = yScale(y - dy / 2);
    const x2 = xScale(t + dt / 2);
    const y2 = yScale(y + dy / 2);
    g.append("line")
        .attr("x1", x1).attr("y1", y1)
        .attr("x2", x2).attr("y2", y2)
        .attr("stroke", "#ccc").attr("stroke-width", 1.2);
  }
}

// equilibrium line for logistic
if (isLogistic) {
  g.append("line")
      .attr("x1", xScale(tMin)).attr("y1", yScale(1))
      .attr("x2", xScale(tMax)).attr("y2", yScale(1))
      .attr("stroke", "#e67e22").attr("stroke-width", 1.5)
      .attr("stroke-dasharray", "5,4");
  g.append("text")
      .attr("x", xScale(tMax) + 4).attr("y", yScale(1) + 4)
      .attr("fill", "#e67e22").text("y=1 (equilibrium)");
}

// Euler trajectory
const dt = 0.05;
const steps = 100;
const pts = [];
let ty = vizY0;
for (let i = 0; i <= steps; i++) {
  const t = i * dt;
  pts.push([t, ty]);
  ty = ty + dt * slope(t, ty);
  if (ty > yMax + 1 || ty < yMin - 1) break;
}

const line = d3.line()
    .x(d => xScale(d[0]))
```

```
      .y(d => yScale(d[1]))
      .defined(d => d[1] >= yMin - 0.5 && d[1] <= yMax + 0.5);

  g.append("path")
      .datum(pts)
      .attr("fill", "none")
      .attr("stroke", "#2980b9")
      .attr("stroke-width", 2)
      .attr("d", line);

  // formula label top-right
  const formulaLabel = isLogistic ? "y′ = y(1−y)" : `y′ = ${vizK.toFixed(1)}y`;
  g.append("text")
      .attr("x", w - 4).attr("y", 12)
      .attr("text-anchor", "end")
      .attr("fill", "#333")
      .style("font-size", "13px")
      .text(formulaLabel);

  return svg.node();
}
```

---

## 36.3. Linear first-order ODEs

Not every first-order ODE is separable. Consider:

$$y' + 2xy = e^{-x^2}$$

The right side prevents simple separation. But the equation is linear — it has the form

$$y' + P(x)\,y = Q(x)$$

This is the **standard form** of a first-order linear ODE.

### 36.3.1. The integrating factor

The key idea: multiply both sides by a function $\mu(x)$, chosen so that the left side becomes the derivative of a product. If

$$\mu(x)\,y' + \mu(x)\,P(x)\,y = \frac{d}{dx}[\mu(x)\,y]$$

then by the product rule the right side expands as $\mu y' + \mu' y$. Matching:

$$\mu(x)\, P(x) = \mu'(x)$$

This is a separable ODE for $\mu$:

$$\frac{d\mu}{\mu} = P(x)\, dx \implies \ln \mu = \int P(x)\, dx \implies \mu(x) = e^{\int P(x)\, dx}$$

No constant of integration is needed — any $\mu$ that works is sufficient, and the simplest choice sets the constant to zero.

**Why this is the key step.** After multiplying by $\mu$, the ODE becomes:

$$\frac{d}{dx}[\mu(x)\, y] = \mu(x)\, Q(x)$$

The left side is a perfect derivative. Integrate both sides:

$$\mu(x)\, y = \int \mu(x)\, Q(x)\, dx + C$$

$$y = \frac{1}{\mu(x)} \left[ \int \mu(x)\, Q(x)\, dx + C \right]$$

The integrating factor converts the ODE into a straightforward integration problem.

**Summary of the method:**

1. Write in standard form: $y' + P(x)\, y = Q(x)$
2. Compute $\mu(x) = e^{\int P(x)\, dx}$
3. Multiply both sides by $\mu$: left side becomes $\frac{d}{dx}[\mu y]$
4. Integrate both sides
5. Solve for $y$; apply initial condition to find $C$

### 36.3.2. Worked example: RC circuit

A series RC circuit connected to a constant voltage source $V_s$ satisfies Kirchhoff's voltage law:

$$V_R + V_C = V_s$$

The resistor voltage is $V_R = Ri = RC \frac{dV_C}{dt}$ (using $i = C \frac{dV_C}{dt}$). Writing $V = V_C$:

$$RC \frac{dV}{dt} + V = V_s$$

Divide by $RC$ to reach standard form:

$$\frac{dV}{dt} + \frac{1}{RC} V = \frac{V_s}{RC}$$

Here $P(t) = \frac{1}{RC}$ and $Q(t) = \frac{V_s}{RC}$.

**Step 2.** Integrating factor:

$$\mu(t) = e^{\int \frac{1}{RC} dt} = e^{t/RC}$$

**Step 3.** Multiply through:

$$\frac{d}{dt}\left[e^{t/RC} V\right] = \frac{V_s}{RC} e^{t/RC}$$

**Step 4.** Integrate both sides:

$$e^{t/RC} V = V_s e^{t/RC} + C$$

**Step 5.** Divide by $e^{t/RC}$:

$$V(t) = V_s + C e^{-t/RC}$$

Apply $V(0) = V_0$:

$$V_0 = V_s + C \implies C = V_0 - V_s$$

Particular solution:

$$\boxed{V(t) = V_s + (V_0 - V_s)\, e^{-t/RC}}$$

Reading the solution physically: the voltage starts at $V_0$ and approaches $V_s$ exponentially with time constant $\tau = RC$. If $V_0 = 0$ (capacitor initially uncharged), the voltage rises from zero to $V_s$. If $V_0 > V_s$ (capacitor initially overcharged), it decays down. The product $\tau = RC$ — measured in seconds when $R$ is in ohms and $C$ in farads — governs how fast the transient dies. After $5\tau$ the transient is below 1% of its initial magnitude; engineers call this "essentially complete."

---

```
//| echo: false
viewof vizR  = Inputs.range([1, 100],   {step: 1,    value: 10,   label: "R (Ω)"})
viewof vizC  = Inputs.range([100, 10000], {step: 100, value: 1000, label: "C (μF)"})
viewof vizV0 = Inputs.range([0, 20],    {step: 0.5,  value: 10,   label: "V₀ (initial voltage, V)
viewof vizVs = Inputs.range([0, 20],    {step: 0.5,  value: 0,    label: "Vs (source voltage, V)"
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const W = 560, H = 300;
  const margin = {top: 20, right: 140, bottom: 40, left: 55};
  const w = W - margin.left - margin.right;
  const h = H - margin.top  - margin.bottom;

  // τ in seconds  (C is in μF → convert to F)
  const tau = vizR * vizC * 1e-6;
  const tMax = 5 * tau;

  const vMin = Math.min(vizVs, vizV0) - 0.5;
  const vMax = Math.max(vizVs, vizV0) + 0.5;

  const xScale = d3.scaleLinear([0, tMax], [0, w]);
  const yScale = d3.scaleLinear([vMin, vMax], [h, 0]);

  const svg = d3.create("svg")
      .attr("width",  W)
      .attr("height", H)
      .style("font-family", "sans-serif")
      .style("font-size", "12px");

  const g = svg.append("g")
      .attr("transform", `translate(${margin.left},${margin.top})`);

  // axes
  g.append("g").attr("transform", `translate(0,${h})`)
      .call(d3.axisBottom(xScale).ticks(5).tickFormat(v => d3.format(".3g")(v)));
  g.append("g").call(d3.axisLeft(yScale).ticks(6));

  // axis labels
  g.append("text")
      .attr("x", w / 2).attr("y", h + 34)
      .attr("text-anchor", "middle").text("t (s)");
  g.append("text")
      .attr("transform", "rotate(-90)")
      .attr("x", -h / 2).attr("y", -44)
      .attr("text-anchor", "middle").text("V(t) (V)");

  // V∞ horizontal dashed line
  g.append("line")
      .attr("x1", 0).attr("y1", yScale(vizVs))
      .attr("x2", w).attr("y2", yScale(vizVs))
      .attr("stroke", "#27ae60").attr("stroke-width", 1.4)
```

```
    .attr("stroke-dasharray", "5,4");
g.append("text")
    .attr("x", w + 4).attr("y", yScale(vizVs) + 4)
    .attr("fill", "#27ae60")
    .style("font-size", "11px")
    .text(`V∞ = V□ = ${vizVs.toFixed(1)} V`);

// □ vertical dashed line
g.append("line")
    .attr("x1", xScale(tau)).attr("y1", 0)
    .attr("x2", xScale(tau)).attr("y2", h)
    .attr("stroke", "#8e44ad").attr("stroke-width", 1.4)
    .attr("stroke-dasharray", "5,4");
g.append("text")
    .attr("x", xScale(tau) + 4).attr("y", 14)
    .attr("fill", "#8e44ad")
    .style("font-size", "11px")
    .text("□ = RC");

// V(□) annotation dot + label
const vAtTau = vizVs + (vizV0 - vizVs) * Math.exp(-1);
g.append("circle")
    .attr("cx", xScale(tau)).attr("cy", yScale(vAtTau))
    .attr("r", 4).attr("fill", "#8e44ad");
g.append("text")
    .attr("x", xScale(tau) + 6).attr("y", yScale(vAtTau) - 6)
    .attr("fill", "#8e44ad")
    .style("font-size", "11px")
    .text(`V(□) □ ${vAtTau.toFixed(2)} V`);

// V(t) curve
const nPts = 300;
const pts = d3.range(nPts + 1).map(i => {
  const t = (i / nPts) * tMax;
  const v = vizVs + (vizV0 - vizVs) * Math.exp(-t / tau);
  return [t, v];
});

const line = d3.line()
    .x(d => xScale(d[0]))
    .y(d => yScale(d[1]));

g.append("path")
    .datum(pts)
    .attr("fill", "none")
    .attr("stroke", "#2980b9")
    .attr("stroke-width", 2.2)
```

```
      .attr("d", line);

  // τ value label top-right
  const tauMs = (tau * 1000).toFixed(1);
  g.append("text")
      .attr("x", w - 4).attr("y", 14)
      .attr("text-anchor", "end")
      .attr("fill", "#333")
      .style("font-size", "12px")
      .text(`τ = ${tauMs} ms`);

  return svg.node();
}
```

---

## 36.4. Applications

### 36.4.1. Mixing problem

A 100 L tank initially contains fresh water. Brine at concentration 2 g/L flows in at 4 L/min. The well-mixed solution flows out at 4 L/min, so the volume stays constant at 100 L.

Let $S(t)$ be the salt content in grams at time $t$. The rate of change is:

$$\frac{dS}{dt} = \underbrace{2 \times 4}_{\text{in}} - \underbrace{\frac{S}{100} \times 4}_{\text{out}} = 8 - \frac{S}{25}$$

Rearranging to standard form:

$$\frac{dS}{dt} + \frac{1}{25} S = 8$$

Integrating factor: $\mu = e^{t/25}$.

$$\frac{d}{dt}\left[e^{t/25} S\right] = 8\, e^{t/25}$$

$$e^{t/25} S = 200\, e^{t/25} + C$$

$$S(t) = 200 + C e^{-t/25}$$

Initial condition $S(0) = 0$ gives $C = -200$:

$$\boxed{S(t) = 200\left(1 - e^{-t/25}\right)}$$

As $t \to \infty$, $S \to 200$ g — this is the equilibrium: 2 g/L in 100 L. The approach is exponential with time constant $\tau = 25$ min. The tank is at 99% of equilibrium after $5\tau = 125$ min. The form of this solution is identical to the RC circuit: a system driven toward equilibrium by a constant input, with exponential relaxation.

### 36.4.2. Drug pharmacokinetics: one-compartment model

A drug is injected as a bolus (instantaneous dose) of $D_0$ mg into the bloodstream. The drug is cleared by the kidneys at a rate proportional to the blood concentration $C(t)$ (mg/L), with elimination rate constant $k_e$ (min$^{-1}$). The volume of distribution is $V_d$ (L).

The ODE for the amount $A(t) = C(t)\,V_d$ in the body is:

$$\frac{dA}{dt} = -k_e\,A, \qquad A(0) = D_0$$

This is $y' = ky$ with $k = -k_e$. Solution:

$$A(t) = D_0\,e^{-k_e t}$$

The blood concentration is $C(t) = A(t)/V_d = (D_0/V_d)\,e^{-k_e t}$.

The **half-life** of the drug is the time for the concentration to halve:

$$C(t_{1/2}) = \frac{C_0}{2} \implies e^{-k_e t_{1/2}} = \frac{1}{2} \implies t_{1/2} = \frac{\ln 2}{k_e}$$

This is the number quoted on drug package inserts. A drug with $t_{1/2} = 4$ hours requires dosing every 4–8 hours to maintain therapeutic levels. The simplicity of this model makes it foundational in pharmacology — more realistic models add compartments (tissue, fat, CSF) but the same ODE structure applies to each.

## 36.5. Existence and uniqueness

Before investing effort in solving an ODE, it is worth asking whether a solution exists and whether it is unique. The **Picard-Lindelöf theorem** gives a sufficient condition.

**Theorem (Picard-Lindelöf).** Consider the initial value problem

$$y' = f(x, y), \qquad y(x_0) = y_0$$

If $f$ and $\partial f/\partial y$ are both continuous on some open rectangle $R = \{(x, y) : |x - x_0| < a,\ |y - y_0| < b\}$ containing $(x_0, y_0)$, then there exists an interval $|x - x_0| < h$ (for some $h > 0$) on which the IVP has a **unique** solution.

The theorem does not tell you the size of $h$ — the solution may only exist locally, not for all $x$. But it does guarantee that on some interval, exactly one solution passes through the given initial point.

**What failure looks like.** Consider $y' = y^{2/3}$ with $y(0) = 0$.

Here $f(x, y) = y^{2/3}$, which is continuous everywhere. But $\partial f/\partial y = \frac{2}{3}y^{-1/3}$, which is *not* continuous at $y = 0$ — the theorem's hypothesis fails. And indeed, uniqueness fails: both

$$y_1(x) = 0 \qquad \text{and} \qquad y_2(x) = \left(\frac{x}{3}\right)^3$$

satisfy $y' = y^{2/3}$ and $y(0) = 0$. There is not a unique solution through the origin.

**Engineering consequence.** When the theorem applies, you know that any reasonable numerical method converges to the same solution — there is no ambiguity about which solution you are approximating. When it fails, a numerical solver may track different solution branches depending on the step size or starting perturbation, and the results can be misleading. Check the theorem first.

---

## 36.6. Exercises

These exercises are puzzles with physical content. The interesting part is identifying the method (separable or integrating factor) and setting up the solution correctly.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
```

```
const stepsDiv = document.createElement("div");
stepsDiv.style.marginBottom = "0.75rem";

function renderSteps() {
  stepsDiv.innerHTML = "";
  for (let i = 0; i < current; i++) {
    const s = steps[i];
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align
    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;
    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    row.appendChild(opCell);
    row.appendChild(eqCell);
    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }
    stepsDiv.appendChild(row);
  }
  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "Solution complete.";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

function updateButtons() {
  nextBtn.disabled = current === totalSteps;
  nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
  counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
```

```
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

---

**Exercise 1.** Solve $\dfrac{dy}{dx} = \dfrac{x}{y}$. Find the general solution and the particular solution satisfying $y(0) = 2$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Separate variables",
    eq: "y\\,dy = x\\,dx",
    note: "Multiply both sides by y and by dx."
  },
  {
    op: "Integrate both sides",
    eq: "\\int y\\,dy = \\int x\\,dx \\implies \\frac{y^2}{2} = \\frac{x^2}{2} + C_1"
  },
  {
    op: "Multiply through by 2, absorb constant",
    eq: "y^2 = x^2 + C \\quad \\text{(general solution, family of hyperbolas)}"
  },
  {
    op: "Apply initial condition y(0) = 2",
    eq: "4 = 0 + C \\implies C = 4"
  },
  {
    op: "Particular solution",
    eq: "y^2 = x^2 + 4 \\implies y = \\sqrt{x^2 + 4}",
    note: "Take the positive root because y(0) = 2 > 0."
  }
])
```

---

**Exercise 2.** Solve $\dfrac{dy}{dt} = -0.1y$ with $y(0) = 500$. Find $y(t)$, then find the time at which $y = 100$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Recognise form y' = ky",
    eq: "k = -0.1, \\quad y_0 = 500"
  },
  {
    op: "Write general solution",
    eq: "y(t) = 500\\,e^{-0.1t}"
  },
  {
    op: "Set y = 100, solve for t",
    eq: "500\\,e^{-0.1t} = 100 \\implies e^{-0.1t} = 0.2"
  },
  {
    op: "Take natural log",
    eq: "-0.1t = \\ln(0.2) = -\\ln 5"
  },
  {
    op: "Solve for t",
    eq: "t = \\frac{\\ln 5}{0.1} = 10\\ln 5 \\approx 16.09",
    note: "ln 5 ⍰ 1.609."
  }
])
```

---

**Exercise 3.** Coffee at 90°C sits in a 20°C room. The cooling constant is $k = 0.05$ min$^{-1}$. Find $T(t)$. How long until the coffee reaches 60°C?

```
//| echo: false
makeStepperHTML([
  {
    op: "Write Newton's law of cooling",
    eq: "\\frac{dT}{dt} = -0.05(T - 20), \\quad T(0) = 90"
  },
  {
    op: "Apply earlier result",
    eq: "T(t) = 20 + (90 - 20)\\,e^{-0.05t} = 20 + 70\\,e^{-0.05t}"
  },
  {
    op: "Set T = 60",
    eq: "20 + 70\\,e^{-0.05t} = 60 \\implies 70\\,e^{-0.05t} = 40"
  },
```

```
  {
    op: "Divide and take ln",
    eq: "e^{-0.05t} = \\tfrac{4}{7} \\implies -0.05t = \\ln\\tfrac{4}{7}"
  },
  {
    op: "Solve for t",
    eq: "t = \\frac{-\\ln(4/7)}{0.05} = 20\\ln\\tfrac{7}{4} \\approx 11.2 \\text{ min}",
    note: "ln(7/4) ≈ 0.5596."
  }
])
```

---

**Exercise 4.** Solve $y' + 2y = 4$ using the integrating factor method. Find the general solution, then the particular solution with $y(0) = 1$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Already in standard form",
    eq: "P(x) = 2, \\quad Q(x) = 4"
  },
  {
    op: "Compute integrating factor",
    eq: "\\mu = e^{\\int 2\\,dx} = e^{2x}"
  },
  {
    op: "Multiply through; left side is a perfect derivative",
    eq: "\\frac{d}{dx}\\!\\left[e^{2x}y\\right] = 4e^{2x}"
  },
  {
    op: "Integrate both sides",
    eq: "e^{2x}y = 2e^{2x} + C"
  },
  {
    op: "Divide by integrating factor",
    eq: "y = 2 + Ce^{-2x} \\quad \\text{(general solution)}"
  },
  {
    op: "Apply y(0) = 1",
    eq: "1 = 2 + C \\implies C = -1",
    note: "As x → ∞, y → 2: the steady state driven by Q = 4."
  },
  {
    op: "Particular solution",
    eq: "y = 2 - e^{-2x}"
```

```
  }
])
```

---

**Exercise 5.** An RC circuit satisfies $V' + V = 12$ (taking $RC = 1$ s). The capacitor is initially uncharged: $V(0) = 0$. Find $V(t)$. What does $V(t)$ approach as $t \to \infty$, and why?

```
//| echo: false
makeStepperHTML([
  {
    op: "Standard form: P = 1, Q = 12",
    eq: "V' + V = 12"
  },
  {
    op: "Integrating factor",
    eq: "\\mu = e^{\\int 1\\,dt} = e^t"
  },
  {
    op: "Multiply through",
    eq: "\\frac{d}{dt}\\!\\left[e^t V\\right] = 12\\,e^t"
  },
  {
    op: "Integrate",
    eq: "e^t V = 12\\,e^t + C"
  },
  {
    op: "General solution",
    eq: "V(t) = 12 + Ce^{-t}"
  },
  {
    op: "Apply V(0) = 0",
    eq: "0 = 12 + C \\implies C = -12"
  },
  {
    op: "Particular solution",
    eq: "V(t) = 12\\!\\left(1 - e^{-t}\\right)",
    note: "As t → ∞, V → 12 V. The capacitor charges to the source voltage — the exponential tran
  }
])
```

---

**Exercise 6.** A 100 L tank initially contains 10 g of salt. Brine at 0.5 g/L flows in at 2 L/min; the well-mixed solution flows out at 2 L/min. Set up and solve the ODE for total salt $S(t)$ (grams). Find $S(t)$ and the long-run salt content.

```
//| echo: false
makeStepperHTML([
  {
    op: "Rate of salt in",
    eq: "\\text{rate in} = 0.5 \\times 2 = 1 \\text{ g/min}"
  },
  {
    op: "Rate of salt out (concentration S/100 times flow 2)",
    eq: "\\text{rate out} = \\frac{S}{100} \\times 2 = \\frac{S}{50} \\text{ g/min}"
  },
  {
    op: "ODE",
    eq: "\\frac{dS}{dt} = 1 - \\frac{S}{50}"
  },
  {
    op: "Standard form",
    eq: "\\frac{dS}{dt} + \\frac{1}{50}S = 1"
  },
  {
    op: "Integrating factor",
    eq: "\\mu = e^{t/50}"
  },
  {
    op: "Multiply; integrate",
    eq: "e^{t/50}S = 50\\,e^{t/50} + C"
  },
  {
    op: "General solution",
    eq: "S(t) = 50 + Ce^{-t/50}"
  },
  {
    op: "Apply S(0) = 10",
    eq: "10 = 50 + C \\implies C = -40"
  },
  {
    op: "Particular solution",
    eq: "S(t) = 50 - 40\\,e^{-t/50}",
    note: "As t → ∞, S → 50 g = 0.5 g/L × 100 L. The tank equilibrates to the incoming concentratio
  }
])
```

## 36.7. Where this goes

This chapter covered first-order ODEs. The two methods here — separation of variables and the integrating factor — handle a large class of single first-order equations. They are not the whole story.

**Second-order ODEs** (Chapter 2) introduce new phenomena: oscillation, resonance, and damping. A mass on a spring, an LC circuit, a building subject to seismic loading — these all require second-order equations and the concept of complementary and particular solutions.

**Systems of ODEs** (Chapter 3) handle multiple interdependent quantities: two species in competition, currents in a multi-loop circuit, the position and velocity of a mechanical system as coupled first-order equations. The phase plane — a picture in state space rather than time — provides geometric insight that algebraic solutions alone cannot.

**Laplace transforms** (Chapter 4) convert ODEs to algebraic equations in a new variable, solve them there, and invert the transform. The technique handles discontinuous forcing (switches, impulses) cleanly and connects naturally to control system design.

The techniques of this chapter will reappear in all three. The RC circuit solution you derived here is the building block for understanding filter transients, step responses, and feedback systems.

**Where this shows up**

- An electrical engineer designing an RC filter needs the time constant $\tau = RC$ to set the cutoff frequency — it comes directly from the ODE solution.
- A pharmacokineticist calculating a dosing interval needs the drug half-life $t_{1/2} = (\ln 2)/k_e$ — from the same exponential decay formula.
- A chemical engineer sizing a mixing tank needs to know how long before the outlet concentration reaches 99% of the inlet — a mixing problem of exactly the form in Exercise 6.
- A control engineer specifying a first-order lag in a block diagram is working with $\tau\dot{y} + y = u$ — the RC circuit equation in disguise.

The ODE changes labels. The mathematics does not.

# 37. Second-order ODEs

Oscillation, damping, and resonance

In November 1940, four months after it opened, the Tacoma Narrows Bridge began twisting in a 65 km/h wind. The oscillations grew. Within hours the deck was flexing through metres of arc. Then it collapsed.

The engineers had missed a resonance condition. The wind's forcing frequency matched the bridge's natural frequency, and the amplitude grew without bound. A second-order ODE — mass, damping, stiffness, forcing — contained the answer. The question is whether you know how to read it.

This chapter develops every tool needed to answer that question. By the end you will be able to look at any spring, circuit, or structure, write down its governing equation, and classify its response before solving it.

## 37.1. Structure of the general equation

A second-order linear ODE with constant coefficients has the form

$$ay'' + by' + cy = f(x)$$

where $a \neq 0$. The coefficients $a$, $b$, $c$ are constants; $f(x)$ is the **forcing function** (or input, or source term).

**Why second order?** Newton's second law says $F = ma$, where $a = \ddot{x}$ is the second derivative of position. Any system governed by Newton's second law — masses on springs, charges in circuits, beams under load — produces a second-order ODE. The second derivative is not an accident of notation; it encodes the physics.

**Linearity and superposition.** The equation is linear because $y$, $y'$, $y''$ each appear to the first power and are not multiplied together. Linearity buys you the superposition principle: if $y_1$ and $y_2$ are both solutions to the homogeneous equation ($f = 0$), then so is $c_1 y_1 + c_2 y_2$ for any constants $c_1$, $c_2$. This principle is the engine of everything that follows.

**The decomposition: complementary + particular.** The general solution is

$$y = y_c + y_p$$

where $y_c$ (the **complementary solution**) solves the homogeneous equation $ay'' + by' + cy = 0$, and $y_p$ (the **particular solution**) is any one solution to the full equation with $f(x)$ present. The two free constants in $y_c$ absorb the initial conditions.

**Why two initial conditions?** The ODE is second order, so its general solution contains two arbitrary constants. To pick a unique solution from the family, you need two pieces of information — typically $y(x_0)$ and $y'(x_0)$. Specify position and velocity at time zero, and the subsequent motion is determined.

**The structure in one line**

$$\underbrace{ay'' + by' + cy = 0}_{\text{homogeneous: find } y_c} \quad + \quad \underbrace{ay'' + by' + cy = f(x)}_{\text{find any } y_p} \quad \Longrightarrow \quad y = y_c + y_p$$

## 37.2. Homogeneous equations: the characteristic equation

Take the homogeneous equation

$$ay'' + by' + cy = 0$$

and try a solution of the form $y = e^{rx}$. Substituting:

$$y = e^{rx}, \quad y' = re^{rx}, \quad y'' = r^2 e^{rx}$$

$$ar^2 e^{rx} + bre^{rx} + ce^{rx} = 0$$

Factor out $e^{rx}$, which is never zero:

$$e^{rx}(ar^2 + br + c) = 0 \implies ar^2 + br + c = 0$$

This is the **characteristic equation**. An ODE has become an algebraic problem: find the roots of a quadratic. The nature of those roots determines the form of $y_c$.

The quadratic formula gives

$$r = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The discriminant $\Delta = b^2 - 4ac$ controls everything.

### 37.2.1. Case 1: Two distinct real roots ($\Delta > 0$)

Two distinct roots $r_1 \neq r_2$, both real. The general solution is

$$y_c = C_1 e^{r_1 x} + C_2 e^{r_2 x}$$

These are two linearly independent solutions; their linear combination spans the solution space.

**Physical behaviour:** If both roots are negative, both exponentials decay. The system returns to equilibrium without oscillating. In a spring-mass context this is the **overdamped** regime — the mass creeps back slowly, never crossing the equilibrium.

**Example.** Solve $y'' - 5y' + 6y = 0$.

Characteristic equation: $r^2 - 5r + 6 = 0$, which factors as $(r-2)(r-3) = 0$.

Roots: $r_1 = 2, r_2 = 3$.

General solution: $y = C_1 e^{2x} + C_2 e^{3x}$.

### 37.2.2. Case 2: Repeated real root ($\Delta = 0$)

One root $r = -b/(2a)$ repeated. The naive approach gives only $e^{rx}$, which is one solution. A second independent solution is $xe^{rx}$ (derivable by reduction of order — see §6 below, or verify by substitution).

General solution:

$$y_c = (C_1 + C_2 x)e^{rx}$$

**Physical behaviour:** The system returns to equilibrium as fast as possible without oscillating. This is **critical damping** — the boundary between decaying and oscillating behaviour. Car suspension systems are often tuned close to critical damping.

**Example.** Solve $y'' + 4y' + 4y = 0$.

Characteristic equation: $r^2 + 4r + 4 = (r+2)^2 = 0$.

Root: $r = -2$ (repeated).

General solution: $y = (C_1 + C_2 x)e^{-2x}$.

### 37.2.3. Case 3: Complex conjugate roots ($\Delta < 0$)

The roots are $r = \alpha \pm \beta i$ where $\alpha = -b/(2a)$ and $\beta = \sqrt{4ac - b^2}/(2a) > 0$.

Using Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$, the complex exponentials can be rewritten as real-valued functions:

$$y_c = e^{\alpha x}(C_1 \cos(\beta x) + C_2 \sin(\beta x))$$

**Physical behaviour:** The $e^{\alpha x}$ factor is the envelope. If $\alpha < 0$, the oscillation decays — this is **underdamped**. If $\alpha = 0$, oscillation continues forever at constant amplitude — **undamped simple harmonic motion**. If $\alpha > 0$, the oscillation grows (unstable system).

**Example.** Solve $y'' + 2y' + 5y = 0$.

Characteristic equation: $r^2 + 2r + 5 = 0$.

Roots: $r = (-2 \pm \sqrt{4 - 20})/2 = (-2 \pm \sqrt{-16})/2 = -1 \pm 2i$.

So $\alpha = -1$, $\beta = 2$.

General solution: $y = e^{-x}(C_1 \cos 2x + C_2 \sin 2x)$.

**Summary: three cases**

| Discriminant | Roots | Solution | Behaviour |
|---|---|---|---|
| $\Delta > 0$ | Two distinct real | $C_1 e^{r_1 x} + C_2 e^{r_2 x}$ | Overdamped / exponential |
| $\Delta = 0$ | Repeated real | $(C_1 + C_2 x)e^{rx}$ | Critically damped |
| $\Delta < 0$ | Complex conjugate | $e^{\alpha x}(C_1 \cos \beta x + C_2 \sin \beta x)$ | Underdamped / oscillatory |

```
//| echo: false
viewof smdM = Inputs.range([0.1, 5], {step: 0.1, value: 1, label: "m (mass, kg)"})
viewof smdC = Inputs.range([0, 10], {step: 0.1, value: 1, label: "c (damping, N·s/m)"})
viewof smdK = Inputs.range([0.1, 20], {step: 0.1, value: 4, label: "k (stiffness, N/m)"})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const width = 560, height = 300;
  const margin = { top: 20, right: 20, bottom: 45, left: 55 };
  const innerW = width - margin.left - margin.right;
  const innerH = height - margin.top - margin.bottom;

  const m = smdM, c = smdC, k = smdK;
  const disc = c * c - 4 * m * k;
```

```
// Classify regime
const eps = 1e-6;
let regime;
if (disc > eps) regime = "Overdamped (Δ > 0)";
else if (disc > -eps) regime = "Critically damped (Δ = 0)";
else regime = "Underdamped (Δ < 0)";

// Euler integration: m x'' + c x' + k x = 0, x(0)=1, x'(0)=0
const dt = 0.01, steps = 600;
const ts = [], xs = [];
let x = 1, v = 0;
for (let i = 0; i <= steps; i++) {
  ts.push(i * dt);
  xs.push(x);
  const a = (-c * v - k * x) / m;
  v += a * dt;
  x += v * dt;
}

const xExtent = d3.extent(xs);
const yMin = Math.min(xExtent[0], -1.05);
const yMax = Math.max(xExtent[1], 1.05);

const xScale = d3.scaleLinear().domain([0, steps * dt]).range([0, innerW]);
const yScale = d3.scaleLinear().domain([yMin, yMax]).range([innerH, 0]);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${width} ${height}`)
  .attr("width", width)
  .attr("height", height)
  .style("font-family", "sans-serif");

const g = svg.append("g").attr("transform", `translate(${margin.left},${margin.top})`);

// Zero line
g.append("line")
  .attr("x1", 0).attr("x2", innerW)
  .attr("y1", yScale(0)).attr("y2", yScale(0))
  .attr("stroke", "#999").attr("stroke-dasharray", "4,3").attr("stroke-width", 1);

// Exponential envelope for underdamped
if (disc < -eps) {
  const alpha = c / (2 * m);
  const envLine = d3.line()
    .x((_, i) => xScale(ts[i]))
    .y((v) => yScale(v));
```

```
  const envPos = ts.map(t => Math.exp(-alpha * t));
  const envNeg = ts.map(t => -Math.exp(-alpha * t));

  g.append("path")
    .datum(envPos)
    .attr("fill", "none")
    .attr("stroke", "#999")
    .attr("stroke-dasharray", "5,4")
    .attr("stroke-width", 1.2)
    .attr("d", envLine);

  g.append("path")
    .datum(envNeg)
    .attr("fill", "none")
    .attr("stroke", "#999")
    .attr("stroke-dasharray", "5,4")
    .attr("stroke-width", 1.2)
    .attr("d", envLine);
}

// Response curve
const line = d3.line()
  .x((_, i) => xScale(ts[i]))
  .y((v) => yScale(v));

g.append("path")
  .datum(xs)
  .attr("fill", "none")
  .attr("stroke", "#3b82f6")
  .attr("stroke-width", 2)
  .attr("d", line);

// Axes
g.append("g").attr("transform", `translate(0,${innerH})`).call(d3.axisBottom(xScale).ticks(6));
g.append("g").call(d3.axisLeft(yScale).ticks(5));

// Axis labels
g.append("text")
  .attr("x", innerW / 2).attr("y", innerH + 38)
  .attr("text-anchor", "middle").attr("font-size", 13)
  .text("t (s)");

g.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -innerH / 2).attr("y", -42)
  .attr("text-anchor", "middle").attr("font-size", 13)
  .text("x(t) (m)");
```

```
  // Regime label (top-right)
  const regimeColour = disc > eps ? "#b45309" : disc > -eps ? "#7c3aed" : "#059669";
  g.append("text")
    .attr("x", innerW - 5).attr("y", 15)
    .attr("text-anchor", "end").attr("font-size", 12).attr("font-weight", "bold")
    .attr("fill", regimeColour)
    .text(regime);

  // ω₀ and ζ
  const omega0 = Math.sqrt(k / m);
  const zeta = c / (2 * Math.sqrt(k * m));
  g.append("text")
    .attr("x", innerW - 5).attr("y", 32)
    .attr("text-anchor", "end").attr("font-size", 11).attr("fill", "#555")
    .text(`ω₀ = ${omega0.toFixed(3)}  ζ = ${zeta.toFixed(3)}`);

  return svg.node();
}
```

## 37.3. The spring-mass-damper system

The physical archetype for every second-order linear ODE is a mass $m$ on a spring with stiffness $k$, connected to a dashpot (viscous damper) with damping coefficient $c$:

$$m\ddot{x} + c\dot{x} + kx = 0$$

Here $x(t)$ is displacement from equilibrium, dots denote time derivatives. This maps directly onto the standard form $ay'' + by' + cy = 0$ with $a = m$, $b = c$, $c_{\text{coeff}} = k$.

The characteristic equation is $mr^2 + cr + k = 0$, with roots

$$r = \frac{-c \pm \sqrt{c^2 - 4mk}}{2m}$$

Two natural parameters emerge from dividing the equation by $m$:

$$\ddot{x} + \frac{c}{m}\dot{x} + \frac{k}{m}x = 0$$

**Natural frequency** (undamped):

$$\omega_0 = \sqrt{\frac{k}{m}}$$

**Damping ratio:**

$$\zeta = \frac{c}{2\sqrt{km}} = \frac{c}{2m\omega_0}$$

The equation becomes $\ddot{x} + 2\zeta\omega_0\dot{x} + \omega_0^2 x = 0$, and the discriminant condition translates cleanly:

| $\zeta$ | Regime | Behaviour |
|---|---|---|
| $\zeta > 1$ | Overdamped | Two negative real roots; slow return to equilibrium |
| $\zeta = 1$ | Critically damped | Repeated negative real root; fastest return without oscillation |
| $\zeta < 1$ | Underdamped | Complex roots; decaying oscillation at frequency $\omega_d = \omega_0\sqrt{1-\zeta^2}$ |
| $\zeta = 0$ | Undamped | Pure imaginary roots; oscillation at $\omega_0$ forever |

### 37.3.1. Worked numerical example

A spring-mass-damper system has $m = 1$ kg, $c = 5$ N·s/m, $k = 6$ N/m. Find $x(t)$ given $x(0) = 1$, $\dot{x}(0) = 0$.

**Step 1: characteristic equation.**

$$r^2 + 5r + 6 = 0 \implies (r+2)(r+3) = 0 \implies r_1 = -2,\ r_2 = -3$$

**Step 2: classify.** Both roots real and negative. Discriminant $= 25 - 24 = 1 > 0$.

Check via $\zeta$: $\omega_0 = \sqrt{6} \approx 2.449$, $\zeta = 5/(2\sqrt{6}) \approx 1.02 > 1$. Overdamped.

**Step 3: general solution.**

$$x(t) = C_1 e^{-2t} + C_2 e^{-3t}$$

**Step 4: apply initial conditions.**

$x(0) = 1$: $C_1 + C_2 = 1$

$\dot{x}(0) = 0$: $-2C_1 - 3C_2 = 0 \implies C_1 = -3C_2/2$

Substituting: $-3C_2/2 + C_2 = 1 \implies -C_2/2 = 1 \implies C_2 = -2, C_1 = 3$.

**Solution:**

$$x(t) = 3e^{-2t} - 2e^{-3t}$$

The mass starts at $x = 1$, initially moves slightly before the two exponentials pull it back to zero without oscillating. Physical interpretation: heavy damping prevents overshoot.

Now change the damping: $m = 1$, $c = 2$, $k = 1$. Characteristic equation: $r^2 + 2r + 1 = (r+1)^2 = 0$. Root $r = -1$ repeated. Critically damped. With $x(0) = 1$, $\dot{x}(0) = 0$:

$x = (C_1 + C_2 t)e^{-t}$; $x(0) = C_1 = 1$; $\dot{x}(0) = C_2 - C_1 = 0 \implies C_2 = 1$.

$$x(t) = (1+t)e^{-t}$$

Now change to $m = 1$, $c = 1$, $k = 4$. Characteristic equation: $r^2 + r + 4 = 0$.

Roots: $r = (-1 \pm \sqrt{1-16})/2 = -1/2 \pm i\sqrt{15}/2$. Underdamped. General solution: $x(t) = e^{-t/2}(C_1 \cos \frac{\sqrt{15}}{2}t + C_2 \sin \frac{\sqrt{15}}{2}t)$.

With $x(0) = 1$, $\dot{x}(0) = 0$:

$x(0) = C_1 = 1$.

Differentiate using the product rule: $\dot{x}(t) = -\frac{1}{2}e^{-t/2}(C_1 \cos \frac{\sqrt{15}}{2}t + C_2 \sin \frac{\sqrt{15}}{2}t) + e^{-t/2}(-C_1 \frac{\sqrt{15}}{2} \sin \frac{\sqrt{15}}{2}t + C_2 \frac{\sqrt{15}}{2} \cos \frac{\sqrt{15}}{2}t)$.

At $t = 0$: $\dot{x}(0) = -\frac{1}{2}C_1 + C_2 \frac{\sqrt{15}}{2} = 0$, so $C_2 = \frac{C_1}{\sqrt{15}} = \frac{1}{\sqrt{15}}$.

$$x(t) = e^{-t/2}\left(\cos \frac{\sqrt{15}}{2}t + \frac{1}{\sqrt{15}} \sin \frac{\sqrt{15}}{2}t\right)$$

The oscillation is visible; the exponential envelope $e^{-t/2}$ decays it.

**Engineering rule of thumb**
$\zeta > 1$: overdamped — sluggish, no oscillation. $\zeta = 1$: critically damped — fastest non-oscillatory return. $\zeta < 1$: underdamped — fast but overshoots; for $\zeta \ll 1$ the oscillation persists for many cycles.
Automotive suspension targets $\zeta \approx 0.3$–$0.4$: some overshoot is acceptable; too much damping makes the ride stiff and slow to recover.

## 37.4. Method of undetermined coefficients

When $f(x) \neq 0$, we need a particular solution $y_p$. The method of undetermined coefficients works when $f(x)$ belongs to a standard family: polynomials, exponentials, sines and cosines, and products of these.

The strategy: guess $y_p$ has the same form as $f(x)$, with unknown coefficients. Substitute into the ODE; match coefficients on both sides.

| $f(x)$ | Guess for $y_p$ |
|---|---|

### 37.4.1. Standard guess table

| $f(x)$ | Guess for $y_p$ |
|---|---|
| $P_n(x)$ (polynomial, degree $n$) | $A_n x^n + \cdots + A_1 x + A_0$ |
| $e^{\alpha x}$ | $Ae^{\alpha x}$ |
| $\cos(\beta x)$ or $\sin(\beta x)$ | $A\cos(\beta x) + B\sin(\beta x)$ |
| $e^{\alpha x}\cos(\beta x)$ or similar | $e^{\alpha x}(A\cos(\beta x) + B\sin(\beta x))$ |
| $P_n(x)e^{\alpha x}$ | $e^{\alpha x}(A_n x^n + \cdots + A_0)$ |

**The modification rule.** If the guess for $y_p$ duplicates a term already present in $y_c$, multiply the guess by $x$ (or $x^2$ if $y_c$ contains both $e^{rx}$ and $xe^{rx}$, as in the repeated-root case). Failure to apply this rule produces a contradiction when you substitute.

### 37.4.2. Worked example 1: exponential forcing

Find a particular solution of $y'' + 3y' + 2y = e^{-3x}$.

Homogeneous equation: $r^2 + 3r + 2 = (r+1)(r+2) = 0$, roots $r = -1, -2$.

$y_c = C_1 e^{-x} + C_2 e^{-2x}$.

$f(x) = e^{-3x}$: the exponent $-3$ does not match any root, so no modification is needed. Guess $y_p = Ae^{-3x}$.

Substitute:

$$y_p'' + 3y_p' + 2y_p = 9Ae^{-3x} - 9Ae^{-3x} + 2Ae^{-3x} = 2Ae^{-3x}$$

Set equal to $e^{-3x}$: $2A = 1 \implies A = 1/2$.

$$y_p = \tfrac{1}{2}e^{-3x}$$

General solution: $y = C_1 e^{-x} + C_2 e^{-2x} + \frac{1}{2}e^{-3x}$.

### 37.4.3. Worked example 2: sinusoidal forcing

Find a particular solution of $y'' + 4y' + 5y = \sin(2x)$.

Homogeneous: $r^2 + 4r + 5 = 0 \implies r = -2 \pm i$. So $y_c = e^{-2x}(C_1 \cos x + C_2 \sin x)$.

$f(x) = \sin(2x)$: guess $y_p = A\cos(2x) + B\sin(2x)$.

Compute derivatives:

$$\begin{aligned} y_p' &= -2A\sin(2x) + 2B\cos(2x) \\ y_p'' &= -4A\cos(2x) - 4B\sin(2x) \end{aligned}$$

Substitute:

$$(-4A\cos 2x - 4B\sin 2x) + 4(-2A\sin 2x + 2B\cos 2x) + 5(A\cos 2x + B\sin 2x) = \sin 2x$$

Collect $\cos(2x)$: $-4A + 8B + 5A = (A + 8B)\cos 2x$

Collect $\sin(2x)$: $-4B - 8A + 5B = (B - 8A)\sin 2x$

Match to $0 \cdot \cos 2x + 1 \cdot \sin 2x$:

$$\begin{gathered} A + 8B = 0 \implies A = -8B \\ B - 8A = 1 \implies B + 64B = 1 \implies B = \tfrac{1}{65}, \quad A = -\tfrac{8}{65} \end{gathered}$$

$$y_p = -\frac{8}{65}\cos(2x) + \frac{1}{65}\sin(2x)$$

### 37.4.4. Modification rule in action

Find a particular solution of $y'' + 2y' + y = e^{-x}$.

Characteristic equation: $(r+1)^2 = 0$, root $r = -1$ repeated.

$y_c = (C_1 + C_2 x)e^{-x}$.

Naive guess $Ae^{-x}$ is in $y_c$; modified guess $Axe^{-x}$ is also in $y_c$ (the $C_2 xe^{-x}$ term). So multiply again: guess $y_p = Ax^2e^{-x}$.

$$y_p' = (2Ax - Ax^2)e^{-x}, \quad y_p'' = (2A - 4Ax + Ax^2)e^{-x}$$

Substitute (the $x^2e^{-x}$ and $xe^{-x}$ terms cancel):

$$y_p'' + 2y_p' + y_p = 2Ae^{-x}$$

Match: $2A = 1 \implies A = 1/2$.

$$y_p = \tfrac{1}{2}x^2 e^{-x}$$

## 37.5. Forced oscillation and resonance

Now force the undamped spring-mass system:

$$m\ddot{x} + kx = F_0 \cos(\omega t), \quad x(0) = 0,\ \dot{x}(0) = 0$$

The homogeneous solution is $x_c = C_1 \cos(\omega_0 t) + C_2 \sin(\omega_0 t)$ where $\omega_0 = \sqrt{k/m}$.

**Case 1:** $\omega \neq \omega_0$ **(off resonance).** Guess $x_p = A\cos(\omega t)$.

$$-m\omega^2 A\cos(\omega t) + kA\cos(\omega t) = F_0 \cos(\omega t)$$

$$A(k - m\omega^2) = F_0 \implies A = \frac{F_0}{k - m\omega^2} = \frac{F_0/m}{\omega_0^2 - \omega^2}$$

Applying initial conditions $x(0) = 0$, $\dot{x}(0) = 0$: the general solution is $x = C_1 \cos\omega_0 t + C_2 \sin\omega_0 t + \frac{F_0/m}{\omega_0^2-\omega^2}\cos\omega t$. From $x(0) = 0$: $C_1 + \frac{F_0/m}{\omega_0^2-\omega^2} = 0$, so $C_1 = -\frac{F_0/m}{\omega_0^2-\omega^2}$. From $\dot{x}(0) = 0$: $C_2\omega_0 - \frac{F_0/m}{\omega_0^2-\omega^2}\omega\sin 0 = 0$, giving $C_2 = 0$. Substituting:

$$x(t) = \frac{F_0/m}{\omega_0^2 - \omega^2}\big(\cos(\omega t) - \cos(\omega_0 t)\big)$$

**Beating.** When $\omega$ is close to but not equal to $\omega_0$, write $\omega_0 = \omega + \epsilon$ for small $\epsilon$. The difference $\cos(\omega t) - \cos(\omega_0 t)$ can be rewritten using a product-to-sum identity (from $\cos A - \cos B = 2\sin\frac{A+B}{2}\sin\frac{B-A}{2}$):

$$\cos(\omega t) - \cos(\omega_0 t) = 2\sin\left(\frac{\omega_0 + \omega}{2}t\right)\sin\left(\frac{\omega_0 - \omega}{2}t\right)$$

This is a rapid oscillation at frequency $(\omega_0 + \omega)/2 \approx \omega_0$ modulated by a slow envelope at frequency $(\omega_0 - \omega)/2 \approx \epsilon/2$. The amplitude swells and collapses periodically — **beating**. It is audible between two tuning forks of slightly different pitch.

**Case 2:** $\omega = \omega_0$ **(resonance).** The guess $A\cos(\omega_0 t)$ duplicates a homogeneous solution; apply the modification rule and guess $x_p = t(A\cos(\omega_0 t) + B\sin(\omega_0 t))$.

Substituting and simplifying (the cosine terms cancel; only the sine survives in the second derivative's cross-terms):

$$x_p = \frac{F_0}{2m\omega_0} t\sin(\omega_0 t)$$

The particular solution at resonance is

$$\boxed{x_p = \frac{F_0}{2m\omega_0}\, t \sin(\omega_0 t)}$$

The factor of $t$ in front of the sine is critical. The amplitude $\frac{F_0}{2m\omega_0} t$ grows **linearly without bound** as $t \to \infty$.

> **Resonance is an unbounded response**
> At $\omega = \omega_0$, the forcing and the natural oscillation are perfectly in phase. Each cycle, the force adds energy to the system. With no damping to dissipate it, amplitude grows without limit. In a real system, something fails first — yielding, fracture, or collapse.
> The Tacoma Narrows Bridge, Broughton Suspension Bridge (1831), and numerous aircraft structures have failed under resonance conditions. In engineering design, knowing $\omega_0$ and ensuring forcing frequencies avoid it — or adding sufficient damping — is not optional.

**With damping present.** Add a damping term $c\dot{x}$ to the forced equation. The steady-state amplitude response at driving frequency $\omega$ becomes

$$X(\omega) = \frac{F_0/k}{\sqrt{(1-r^2)^2 + (2\zeta r)^2}}, \quad r = \frac{\omega}{\omega_0}$$

(This formula is derivable by applying undetermined coefficients to $m\ddot{x} + c\dot{x} + kx = F_0 \cos \omega t$ — the calculation is straightforward but lengthy, and we state the result here.)

As $\zeta \to 0$, the peak amplitude at $r = 1$ becomes unbounded. For $\zeta > 0$, the peak is finite but can be very large for small $\zeta$. The peak occurs near $r = 1$ and grows sharper and taller as damping decreases — the amplification factor $Q = 1/(2\zeta)$ is called the **quality factor** in electrical engineering.

```
//| echo: false
viewof forceC = Inputs.range([0, 5], {step: 0.05, value: 1, label: "Damping c"})
viewof forceW0 = Inputs.range([0.1, 5], {step: 0.05, value: 2, label: "Natural frequency ω₀"})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const width = 560, height = 280;
  const margin = { top: 30, right: 20, bottom: 48, left: 60 };
  const innerW = width - margin.left - margin.right;
  const innerH = height - margin.top - margin.bottom;

  const c = forceC;
  const w0 = forceW0;
  // m=1, k=w0^2
  const m = 1, k = w0 * w0;
  const zeta = c / (2 * Math.sqrt(m * k));
```

```
// Amplitude response: X / (F0/k) = 1 / sqrt((1-r^2)^2 + (2*zeta*r)^2)
const nPoints = 400;
const wMax = 2.5 * w0;
const data = [];
for (let i = 0; i <= nPoints; i++) {
  const w = (i / nPoints) * wMax;
  const r = w / w0;
  const denom = Math.sqrt((1 - r * r) * (1 - r * r) + (2 * zeta * r) * (2 * zeta * r));
  const amp = denom < 1e-9 ? 1e6 : 1 / denom;
  data.push({ w, amp });
}

const ampValues = data.map(d => d.amp).filter(isFinite);
const yMax = d3.max(ampValues) * 1.10;

const xScale = d3.scaleLinear().domain([0, wMax]).range([0, innerW]);
const yScale = d3.scaleLinear().domain([0, yMax]).range([innerH, 0]);

// Colour: high c → blue, low c → red (c in [0,5])
const colourT = Math.min(1, c / 5);
const curveColour = d3.interpolateRdYlBu(colourT);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${width} ${height}`)
  .attr("width", width)
  .attr("height", height)
  .style("font-family", "sans-serif");

const g = svg.append("g").attr("transform", `translate(${margin.left},${margin.top})`);

// ω₀ vertical dashed line
g.append("line")
  .attr("x1", xScale(w0)).attr("x2", xScale(w0))
  .attr("y1", 0).attr("y2", innerH)
  .attr("stroke", "#777").attr("stroke-dasharray", "5,4").attr("stroke-width", 1.2);

g.append("text")
  .attr("x", xScale(w0) + 4).attr("y", 12)
  .attr("font-size", 12).attr("fill", "#555")
  .text("ω₀");

// Amplitude curve
const line = d3.line()
  .x(d => xScale(d.w))
  .y(d => Math.max(0, yScale(Math.min(d.amp, yMax * 1.05))))
  .defined(d => isFinite(d.amp));
```

```
g.append("path")
  .datum(data)
  .attr("fill", "none")
  .attr("stroke", curveColour)
  .attr("stroke-width", 2.2)
  .attr("d", line);

// Resonance peak
// Peak near w = w0 * sqrt(1 - 2*zeta^2), only exists if zeta < 1/sqrt(2)
let peakW = null, peakAmp = null;
if (zeta < 1 / Math.SQRT2) {
  peakW = w0 * Math.sqrt(Math.max(0, 1 - 2 * zeta * zeta));
  const r = peakW / w0;
  const denom = Math.sqrt((1 - r * r) * (1 - r * r) + (2 * zeta * r) * (2 * zeta * r));
  peakAmp = denom < 1e-9 ? null : 1 / denom;
} else {
  // No resonance peak — show value at w=0 (amp=1)
  peakW = 0;
  peakAmp = 1;
}

if (peakAmp !== null && isFinite(peakAmp) && peakAmp <= yMax * 1.05) {
  g.append("circle")
    .attr("cx", xScale(peakW)).attr("cy", yScale(Math.min(peakAmp, yMax)))
    .attr("r", 4)
    .attr("fill", curveColour).attr("stroke", "#222").attr("stroke-width", 1);
}

// Resonance label
const peakWLabel = zeta < 1 / Math.SQRT2
  ? `ω ω ω₀√(1−2ζ²) = ${(w0 * Math.sqrt(Math.max(0, 1 - 2 * zeta * zeta))).toFixed(3)}`
  : "No resonance peak (ζ ζ 1/√2)";

g.append("text")
  .attr("x", innerW - 5).attr("y", 14)
  .attr("text-anchor", "end").attr("font-size", 11).attr("fill", "#555")
  .text(`Resonance peak at ${peakWLabel}`);

// Axes
g.append("g").attr("transform", `translate(0,${innerH})`).call(d3.axisBottom(xScale).ticks(6));
g.append("g").call(d3.axisLeft(yScale).ticks(5));

// Axis labels
g.append("text")
  .attr("x", innerW / 2).attr("y", innerH + 40)
  .attr("text-anchor", "middle").attr("font-size", 13)
  .text("Driving frequency ω");
```

```
  g.append("text")
    .attr("transform", "rotate(-90)")
    .attr("x", -innerH / 2).attr("y", -48)
    .attr("text-anchor", "middle").attr("font-size", 13)
    .text("X(ζ) / (F₀/k)");

  // ζ annotation
  g.append("text")
    .attr("x", innerW - 5).attr("y", 28)
    .attr("text-anchor", "end").attr("font-size", 11).attr("fill", "#555")
    .text(`ζ = ${zeta.toFixed(3)}`);

  return svg.node();
}
```

## 37.6. Reduction of order

Suppose you know one solution $y_1$ to $ay'' + by' + cy = 0$ — perhaps from inspection or from the characteristic equation in a non-constant coefficient problem — but cannot find a second independent one directly. The method of **reduction of order** produces $y_2$.

Set $y = v(x)\, y_1(x)$ where $v(x)$ is an unknown function. Compute $y'$ and $y''$, substitute into the ODE. Terms involving $v$ (without derivatives) cancel — this is guaranteed because $y_1$ itself satisfies the equation. The result is a first-order ODE in $w = v'$, which can be solved by the integrating factor method from Chapter 1.

For the constant-coefficient case, this is how the repeated-root solution $xe^{rx}$ is derived. Set $y_1 = e^{rx}$, $y = ve^{rx}$:

$$y' = (v' + rv)e^{rx}, \quad y'' = (v'' + 2rv' + r^2 v)e^{rx}$$

Substituting into $ay'' + by' + cy = 0$ and using $ar^2 + br + c = 0$:

$$a(v'' + 2rv') + bv' = 0 \implies av'' + (2ar + b)v' = 0$$

At the repeated root $r = -b/(2a)$, we have $2ar + b = 0$, so $av'' = 0$, giving $v'' = 0$ and thus $v = C_1 + C_2 x$. Therefore $y = (C_1 + C_2 x)e^{rx}$ — the result we stated earlier, now derived.

Reduction of order becomes more important when dealing with variable-coefficient equations (Euler–Cauchy type) or when one solution is known from a series method. It will reappear in context.

## 37.7. Where this leads

You now have the complete toolkit for second-order linear ODEs with constant coefficients: classify by roots of the characteristic equation; add a particular solution via undetermined coefficients; understand why resonance is structurally different from off-resonance forcing.

Three threads open from here:

**Laplace transforms** (Chapter 4) handle initial-value problems algorithmically, absorb discontinuous forcing naturally, and are the standard industrial tool for control systems analysis. The Laplace transform of $y''$ introduces the transfer function, which encodes the characteristic equation in a different algebraic form.

**Systems and phase plane** (Chapter 3) extend the ideas to pairs of first-order equations — which is what a second-order ODE becomes when you introduce $v = y'$ as a second unknown. Phase-plane portraits make the overdamped/underdamped/critically damped classification visual.

**Fourier series** (Volume 7, Fourier–PDEs section) answer the question: what if $f(x)$ is not a simple sinusoid but a periodic function of arbitrary shape? Decompose it into harmonics, solve for each, and superpose. The resonance analysis extends directly.

**Where this shows up**

- A structural engineer computing the natural frequencies of a building must keep them away from the frequencies of earthquakes and wind — both sources of periodic forcing.
- An electrical engineer designing an RLC filter tunes $\omega_0$ and $\zeta$ to pass or reject particular frequency bands. The quality factor $Q = 1/(2\zeta)$ measures selectivity.
- A control systems engineer designing a PID controller must ensure the closed-loop characteristic equation has roots with negative real parts — the same stability condition as $\alpha < 0$ for the underdamped spring.
- A physicist modelling a damped quantum harmonic oscillator writes down the same equation. The mathematics is unchanged; only the interpretation of $x$ differs.

## 37.8. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
```

```
function renderSteps() {
  stepsDiv.innerHTML = "";
  for (let i = 0; i < current; i++) {
    const s = steps[i];
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;
    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    row.appendChild(opCell);
    row.appendChild(eqCell);
    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:1 / -1; font-size:0.82em; color:#6b7280; padding:0.1rem
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }
    stepsDiv.appendChild(row);
  }
  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "✓ Solution complete";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

function updateButtons() {
  nextBtn.disabled = current === totalSteps;
  nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
  counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSteps
}

nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
```

```
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

**1.** Find the general solution of $y'' - 5y' + 6y = 0$.

```
//| echo: false
makeStepperHTML(1, [
  { op: "Characteristic equation",    eq: "r^2 - 5r + 6 = 0" },
  { op: "Factor",                      eq: "(r-2)(r-3) = 0" },
  { op: "Roots",                       eq: "r_1 = 2,\\quad r_2 = 3", note: "Discriminant □ = 25 -
  { op: "General solution",            eq: "y = C_1 e^{2x} + C_2 e^{3x}" }
])
```

---

**2.** Solve $y'' + 4y' + 4y = 0$, $y(0) = 1$, $y'(0) = 0$.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Characteristic equation",    eq: "r^2 + 4r + 4 = 0" },
  { op: "Factor",                      eq: "(r+2)^2 = 0" },
  { op: "Repeated root",               eq: "r = -2", note: "Discriminant □ = 16 − 16 = 0: critica
  { op: "General solution",            eq: "y = (C_1 + C_2 x)e^{-2x}" },
  { op: "Apply y(0) = 1",              eq: "C_1 = 1" },
  { op: "Differentiate; apply y'(0)=0", eq: "y'(0) = C_2 - 2C_1 = 0 \\Rightarrow C_2 = 2" },
  { op: "Final solution",              eq: "y = (1 + 2x)e^{-2x}" }
])
```

---

**3.** Solve $y'' + 9y = 0$, $y(0) = 0$, $y'(0) = 3$.

```
//| echo: false
makeStepperHTML(3, [
  { op: "Characteristic equation",    eq: "r^2 + 9 = 0" },
  { op: "Roots",                       eq: "r = \\pm 3i", note: "Pure imaginary roots: undamped s
  { op: "General solution",            eq: "y = C_1 \\cos(3x) + C_2 \\sin(3x)" },
```

```
  { op: "Apply y(0) = 0",              eq: "C_1 = 0" },
  { op: "Apply y'(0) = 3",             eq: "3C_2 = 3 \\Rightarrow C_2 = 1" },
  { op: "Final solution",              eq: "y = \\sin(3x)" }
])
```

---

**4.** Find a particular solution of $y'' + 3y' + 2y = e^{-3x}$.

```
//| echo: false
makeStepperHTML(4, [
  { op: "Homogeneous roots",           eq: "r^2+3r+2=(r+1)(r+2)=0 \\Rightarrow r=-1,-2" },
  { op: "Modification check",          eq: "e^{-3x} \\notin y_c", note: "Exponent −3 ⍰ −1 or −2. Sta
  { op: "Guess",                        eq: "y_p = Ae^{-3x}" },
  { op: "Substitute",                   eq: "9Ae^{-3x} - 9Ae^{-3x} + 2Ae^{-3x} = 2Ae^{-3x}" },
  { op: "Match coefficients",           eq: "2A = 1 \\Rightarrow A = \\tfrac{1}{2}" },
  { op: "Particular solution",          eq: "y_p = \\tfrac{1}{2}e^{-3x}" }
])
```

---

**5.** Solve $y'' + 4y = \cos(2x)$.

*Note: the driving frequency $\omega = 2$ equals $\omega_0 = \sqrt{4} = 2$ — this is resonance. The standard guess fails; the modification rule applies.*

```
//| echo: false
makeStepperHTML(5, [
  { op: "Characteristic equation",    eq: "r^2 + 4 = 0 \\Rightarrow r = \\pm 2i" },
  { op: "Homogeneous solution",        eq: "y_c = C_1\\cos(2x) + C_2\\sin(2x)" },
  { op: "Standard guess fails — resonance", eq: "\\cos(2x) \\in y_c \\Rightarrow \\text{multiply by
  { op: "Modified guess",              eq: "y_p = x(A\\cos 2x + B\\sin 2x)" },
  { op: "Substitute; x-terms cancel",  eq: "-4A\\sin(2x) + 4B\\cos(2x) = \\cos(2x)" },
  { op: "Match coefficients",          eq: "A = 0,\\quad B = \\tfrac{1}{4}" },
  { op: "Particular solution",         eq: "y_p = \\tfrac{1}{4}x\\sin(2x)", note: "The factor of x c
  { op: "General solution",            eq: "y = C_1\\cos(2x) + C_2\\sin(2x) + \\tfrac{1}{4}x\\sin(2x
])
```

---

**6.** An RLC circuit has $L = 1$ H, $R = 2\ \Omega$, $C = \frac{1}{3}$ F with no forcing voltage. The governing equation for charge $q(t)$ is $L\ddot{q} + R\dot{q} + q/C = 0$. Find $q(t)$ given $q(0) = 1$ C, $\dot{q}(0) = 0$. Classify the response.

```
//| echo: false
makeStepperHTML(6, [
  { op: "ODE with given values",       eq: "\\ddot{q} + 2\\dot{q} + 3q = 0", note: "L=1, R=2, 1/C
  { op: "Characteristic equation",    eq: "r^2 + 2r + 3 = 0" },
  { op: "Discriminant",                eq: "\\Delta = 4 - 12 = -8 < 0", note: "Negative discrimin
  { op: "Roots",                       eq: "r = -1 \\pm i\\sqrt{2}" },
  { op: "General solution",            eq: "q = e^{-t}\\bigl(C_1\\cos(\\sqrt{2}\\,t) + C_2\\sin(\
  { op: "Apply q(0) = 1",              eq: "C_1 = 1" },
  { op: "Apply \\dot{q}(0) = 0",       eq: "-C_1 + \\sqrt{2}\\,C_2 = 0 \\Rightarrow C_2 = \\tfrac
  { op: "Final solution",              eq: "q(t) = e^{-t}\\!\\left(\\cos(\\sqrt{2}\\,t) + \\tfrac
])
```

# 38. Systems and phase plane

The geometry of dynamical systems

In a wildlife reserve, park wardens track two populations: rabbits and foxes. Neither changes independently. When rabbits are plentiful, foxes thrive and multiply. More foxes means more predation, so rabbit numbers fall. With fewer rabbits, foxes starve and decline. With fewer foxes, rabbit numbers recover. The cycle repeats.

Write down a rate equation for rabbits alone and you immediately run into trouble: the rate of change of the rabbit population depends on the fox population. You cannot separate them. The two populations are a single coupled system, and they need to be modelled together.

This chapter develops the language and tools for that. The central object is the **phase plane** — a picture where every point represents a state of the system (both populations at once), and every solution traces a curve through that space. The geometry of those curves — do they spiral inward, fly outward, or loop forever? — is controlled by the eigenvalues of a matrix. That connection between linear algebra and the long-run behaviour of dynamical systems is one of the most useful in applied mathematics.

---

## 38.1. Systems of first-order ODEs

### 38.1.1. From a single ODE to a system

A second-order ODE contains a second derivative. It can always be written as two coupled first-order equations by introducing an auxiliary variable.

Take the damped oscillator:

$$y'' + py' + qy = 0$$

Set $x_1 = y$ (the displacement) and $x_2 = y'$ (the velocity). Then:

$$x_1' = y' = x_2$$
$$x_2' = y'' = -qx_1 - px_2$$

This is a **system of two first-order ODEs**. The original second-order equation is equivalent to it. Nothing has been lost or gained — this is just a change of representation, chosen because first-order systems are easier to analyse systematically.

### 38.1.2. Matrix form

Collect the equations above:

$$\begin{pmatrix} x_1' \\ x_2' \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -q & -p \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

In compact notation, writing $\mathbf{x} = (x_1, x_2)^T$ for the **state vector**:

$$\mathbf{x}' = A\mathbf{x}, \qquad A = \begin{pmatrix} 0 & 1 \\ -q & -p \end{pmatrix}$$

The state vector $\mathbf{x}(t)$ records everything about the system at time $t$: both the displacement and the velocity. Knowing $\mathbf{x}(t_0)$ at any one time, together with the equation $\mathbf{x}' = A\mathbf{x}$, determines the entire future trajectory. This is the central payoff of the state-vector formulation: the full dynamics live in a single matrix equation.

### 38.1.3. More general systems

Not every system comes from a single second-order ODE. A pair of coupled first-order equations can arise directly from modelling:

$$\begin{aligned} x' &= f(x, y) \\ y' &= g(x, y) \end{aligned}$$

When $f$ and $g$ are linear functions of $x$ and $y$, this is a **linear system** $\mathbf{x}' = A\mathbf{x}$. When they are not linear, the analysis is more involved — but the linear case gives the essential tools, and nonlinear systems are treated by linearising around equilibria (Section 5).

> **Why systems?**
>
> Real engineering systems are rarely one-dimensional. A coupled spring has two masses, two positions, two velocities. An RLC circuit has both charge and current as state variables. A rotating shaft has angular displacement and angular momentum. The system $\mathbf{x}' = A\mathbf{x}$ handles all of these with the same method.

---

## 38.2. The phase plane

### 38.2.1. Trajectories as curves

Consider any two-dimensional system:

$$x' = f(x, y), \qquad y' = g(x, y)$$

A **solution** is a pair of functions $x(t)$, $y(t)$ satisfying both equations simultaneously. We can plot this solution as a parametric curve in the $(x, y)$-plane — called the **phase plane** — with $t$ as the parameter. As $t$ advances, the point $(x(t), y(t))$ moves through the plane, tracing a **trajectory** or **orbit**.

The phase plane collapses time: instead of watching $x$ and $y$ vary separately against $t$, we watch them vary against each other. This makes the geometry visible. A spiral trajectory in the phase plane means oscillation with decay. A closed loop means sustained oscillation. A trajectory heading straight for the origin means exponential decay.

### 38.2.2. Direction fields

At every point $(x, y)$, the system specifies a velocity vector:

$$\mathbf{v} = \begin{pmatrix} f(x, y) \\ g(x, y) \end{pmatrix}$$

Drawing a short arrow at each point — pointing in the direction $(f, g)$ — gives a **direction field** (or **vector field**) for the system. A trajectory is a curve that is everywhere tangent to these arrows.

Before computing any solutions, the direction field reveals the topology of the phase plane: where trajectories converge, where they diverge, what the long-run behaviour looks like. For engineering applications, this geometric picture is often more informative than an explicit formula.

### 38.2.3. Equilibrium points

An **equilibrium** (also called a **critical point** or **fixed point**) is a point $\mathbf{x}^*$ where $\mathbf{x}' = \mathbf{0}$, i.e., $f(x^*, y^*) = 0$ and $g(x^*, y^*) = 0$. At an equilibrium, the system is stationary — if you start there, you stay there.

For a linear system $\mathbf{x}' = A\mathbf{x}$, the origin $\mathbf{x}^* = \mathbf{0}$ is always an equilibrium. If $A$ is invertible, it is the only one.

The central question of phase plane analysis is: **what do trajectories do near an equilibrium?** Do they approach it, move away from it, or orbit around it? The answer comes from the eigenvalues of $A$.

---

## 38.3. Linear systems and the eigenvalue method

### 38.3.1. The trial solution

For the scalar equation $x' = ax$, the solution is $x(t) = Ce^{at}$. For the system $\mathbf{x}' = A\mathbf{x}$, try the analogous form:

$$\mathbf{x}(t) = e^{\lambda t}\mathbf{v}$$

where $\lambda$ is a scalar and $\mathbf{v}$ is a (constant) vector. Substituting into $\mathbf{x}' = A\mathbf{x}$:

$$\lambda e^{\lambda t}\mathbf{v} = Ae^{\lambda t}\mathbf{v}$$
$$A\mathbf{v} = \lambda\mathbf{v}$$

This is precisely the **eigenvalue equation**. The trial solution $\mathbf{x}(t) = e^{\lambda t}\mathbf{v}$ solves the system if and only if $\lambda$ is an eigenvalue of $A$ with eigenvector $\mathbf{v}$.

### 38.3.2. General solution

A $2 \times 2$ matrix generically has two eigenvalues $\lambda_1, \lambda_2$ with eigenvectors $\mathbf{v}_1, \mathbf{v}_2$. By superposition (the equation is linear), the general solution is:

$$\mathbf{x}(t) = c_1 e^{\lambda_1 t}\mathbf{v}_1 + c_2 e^{\lambda_2 t}\mathbf{v}_2$$

The constants $c_1, c_2$ are fixed by initial conditions. Each term is a trajectory along the eigenvector direction, growing or decaying at its own rate.

### 38.3.3. Classification of equilibria

The behaviour near the origin depends entirely on the eigenvalues. There are four principal cases.

---

**Case 1: Real eigenvalues, same sign — Node**

**Example:** $A = \begin{pmatrix} -1 & 0 \\ 0 & -3 \end{pmatrix}$

Eigenvalues: $\lambda_1 = -1$, $\lambda_2 = -3$ (both negative). Eigenvectors: $\mathbf{v}_1 = (1, 0)^T$, $\mathbf{v}_2 = (0, 1)^T$.

General solution:

$$\mathbf{x}(t) = c_1 e^{-t}\begin{pmatrix} 1 \\ 0 \end{pmatrix} + c_2 e^{-3t}\begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Both terms decay to zero. Every trajectory approaches the origin as $t \to \infty$. This is a **stable node** (also called a **stable improper node**). Eigenvalues of the same sign but positive give an **unstable node** — trajectories fly away from the origin.

The phase portrait: all trajectories converge to the origin, tangent to the slow eigendirection ($\lambda_1 = -1$) as $t \to \infty$.

---

**Case 2: Real eigenvalues, opposite signs — Saddle**

**Example:** $A = \begin{pmatrix} 1 & 0 \\ 0 & -2 \end{pmatrix}$

Eigenvalues: $\lambda_1 = 1$, $\lambda_2 = -2$. General solution:

$$\mathbf{x}(t) = c_1 e^{t} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + c_2 e^{-2t} \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

The first term grows without bound; the second decays. Unless $c_1 = 0$ exactly, every trajectory eventually flies away from the origin along the unstable eigendirection. The origin is a **saddle point** — unstable.

The **stable manifold** (the set of trajectories that do approach the origin) consists of the single line along $\mathbf{v}_2 = (0, 1)^T$. The **unstable manifold** consists of the line along $\mathbf{v}_1 = (1, 0)^T$. These two lines are the separatrices of the saddle.

---

**Case 3: Complex eigenvalues — Spiral or Centre**

When $A$ has a pair of complex conjugate eigenvalues $\lambda = \alpha \pm i\beta$ (with $\beta \neq 0$), the solution involves $e^{\alpha t}(\cos \beta t, \sin \beta t)$ — oscillation modulated by $e^{\alpha t}$.

- $\alpha < 0$: oscillations decay to zero. **Stable spiral** (also: stable focus).
- $\alpha > 0$: oscillations grow. **Unstable spiral**.
- $\alpha = 0$: pure oscillation, no growth or decay. **Centre**.

**Extracting real solutions from a complex eigenvector.** When the eigenvalue is $\alpha + i\beta$ with eigenvector $\mathbf{v}$, the complex exponential $e^{(\alpha+i\beta)t}\mathbf{v}$ is a (complex-valued) solution. Take its real and imaginary parts to get two linearly independent real solutions. For example, with eigenvalue $-1 + 2i$ and eigenvector $\mathbf{v} = (1, i)^T$:

$$e^{(-1+2i)t} \begin{pmatrix} 1 \\ i \end{pmatrix} = e^{-t} \begin{pmatrix} \cos 2t + i \sin 2t \\ i \cos 2t - \sin 2t \end{pmatrix}$$

Real part: $e^{-t}(\cos 2t,\ -\sin 2t)^T$. Imaginary part: $e^{-t}(\sin 2t,\ \cos 2t)^T$. These two real vectors are the two linearly independent solutions — the template used in Exercise 3 below.

**Example (stable spiral):** $A = \begin{pmatrix} -1 & 2 \\ -2 & -1 \end{pmatrix}$

Characteristic polynomial: $\det(A - \lambda I) = (\lambda + 1)^2 + 4 = 0$, giving $\lambda = -1 \pm 2i$.

Since $\alpha = -1 < 0$, all trajectories spiral inward toward the origin.

**Example (centre):** $A = \begin{pmatrix} 0 & 1 \\ -4 & 0 \end{pmatrix}$

Characteristic polynomial: $\lambda^2 + 4 = 0$, giving $\lambda = \pm 2i$. Pure imaginary eigenvalues. All trajectories are closed ellipses surrounding the origin — periodic orbits. This is a **centre**.

Summary: eigenvalue classification

| Eigenvalues | Equilibrium type | Stable? |
|---|---|---|
| Real, both negative | Stable node | Yes |
| Real, both positive | Unstable node | No |
| Real, opposite signs | Saddle | No |
| Complex, $\alpha < 0$ | Stable spiral | Yes |
| Complex, $\alpha > 0$ | Unstable spiral | No |
| Purely imaginary | Centre | Neutral |

The trace $\tau = \lambda_1 + \lambda_2 = \mathrm{tr}(A)$ and determinant $\delta = \lambda_1 \lambda_2 = \det(A)$ give a shortcut: if $\delta < 0$ it is a saddle; if $\delta > 0$ and $\tau < 0$ it is a stable node or stable spiral; if $\delta > 0$ and $\tau > 0$ it is unstable.

```
//| echo: false
viewof ppParams = Inputs.form({
  ppA11: Inputs.range([-3, 3], {step: 0.1, value: -1, label: "a₁₁"}),
  ppA12: Inputs.range([-3, 3], {step: 0.1, value: 1,  label: "a₁₂"}),
  ppA21: Inputs.range([-3, 3], {step: 0.1, value: -2, label: "a₂₁"}),
  ppA22: Inputs.range([-3, 3], {step: 0.1, value: -1, label: "a₂₂"})
})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const a11 = ppParams.ppA11;
  const a12 = ppParams.ppA12;
  const a21 = ppParams.ppA21;
  const a22 = ppParams.ppA22;

  // Eigenvalue computation via quadratic formula
  const tr  = a11 + a22;
  const det = a11 * a22 - a12 * a21;
  const disc = tr * tr - 4 * det;

  let lam1, lam2, lamStr1, lamStr2, classification, stableColor;

  if (disc >= 0) {
```

```
    // Real eigenvalues
    lam1 = (tr + Math.sqrt(disc)) / 2;
    lam2 = (tr - Math.sqrt(disc)) / 2;
    lamStr1 = lam1.toFixed(3);
    lamStr2 = lam2.toFixed(3);
    if (lam1 * lam2 < 0) {
      classification = "Saddle (unstable)";
      stableColor = "saddle";
    } else if (lam1 < 0 && lam2 < 0) {
      classification = "Stable node";
      stableColor = "stable";
    } else if (lam1 > 0 && lam2 > 0) {
      classification = "Unstable node";
      stableColor = "unstable";
    } else {
      // one or both zero
      classification = "Non-isolated equilibrium";
      stableColor = "neutral";
    }
  } else {
    // Complex eigenvalues: □ ± □i
    const alpha = tr / 2;
    const beta  = Math.sqrt(-disc) / 2;
    lam1 = { re: alpha, im:  beta };
    lam2 = { re: alpha, im: -beta };
    const sign = beta >= 0 ? "+" : "-";
    lamStr1 = `${alpha.toFixed(3)} + ${beta.toFixed(3)}i`;
    lamStr2 = `${alpha.toFixed(3)} − ${beta.toFixed(3)}i`;
    if (Math.abs(alpha) < 1e-9) {
      classification = "Centre (neutrally stable)";
      stableColor = "neutral";
    } else if (alpha < 0) {
      classification = "Stable spiral";
      stableColor = "stable";
    } else {
      classification = "Unstable spiral";
      stableColor = "unstable";
    }
  }

  // SVG dimensions
  const W = 560, H = 400;
  const margin = { top: 30, right: 20, bottom: 40, left: 40 };
  const iW = W - margin.left - margin.right;
  const iH = H - margin.top  - margin.bottom;

  const xDom = [-3, 3], yDom = [-3, 3];
```

```
const xSc = d3.scaleLinear().domain(xDom).range([0, iW]);
const ySc = d3.scaleLinear().domain(yDom).range([iH, 0]);

const svg = d3.create("svg")
  .attr("width", W).attr("height", H)
  .style("font-family", "sans-serif");

const g = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// Background
g.append("rect")
  .attr("width", iW).attr("height", iH)
  .attr("fill", "#fafafa").attr("stroke", "#ccc");

// Axes
g.append("line")
  .attr("x1", xSc(0)).attr("x2", xSc(0))
  .attr("y1", 0).attr("y2", iH)
  .attr("stroke", "#aaa").attr("stroke-width", 1);
g.append("line")
  .attr("x1", 0).attr("x2", iW)
  .attr("y1", ySc(0)).attr("y2", ySc(0))
  .attr("stroke", "#aaa").attr("stroke-width", 1);

// Axis labels
g.append("text").attr("x", iW + 5).attr("y", ySc(0) + 4).text("x₁").attr("font-size", 11).attr("fi
g.append("text").attr("x", xSc(0) + 4).attr("y", -8).text("x₂").attr("font-size", 11).attr("fill",

// Direction field: 15×15 grid
const gridN = 15;
const arrowLen = (iW / gridN) * 0.38;
for (let i = 0; i <= gridN; i++) {
  for (let j = 0; j <= gridN; j++) {
    const x = xDom[0] + (i / gridN) * (xDom[1] - xDom[0]);
    const y = yDom[0] + (j / gridN) * (yDom[1] - yDom[0]);
    const dx = a11 * x + a12 * y;
    const dy = a21 * x + a22 * y;
    const mag = Math.sqrt(dx * dx + dy * dy) || 1;
    const nx = dx / mag, ny = dy / mag;
    const cx = xSc(x), cy = ySc(y);
    const ex = cx + nx * arrowLen;
    const ey = cy - ny * arrowLen;
    g.append("line")
      .attr("x1", cx - nx * arrowLen * 0.5)
      .attr("y1", cy + ny * arrowLen * 0.5)
      .attr("x2", ex)
```

```
      .attr("y2", ey)
      .attr("stroke", "#bbb").attr("stroke-width", 0.8)
      .attr("marker-end", "url(#arrowhead)");
  }
}

// Arrowhead marker
svg.append("defs").append("marker")
  .attr("id", "arrowhead")
  .attr("viewBox", "0 -3 6 6")
  .attr("refX", 5).attr("refY", 0)
  .attr("markerWidth", 4).attr("markerHeight", 4)
  .attr("orient", "auto")
  .append("path").attr("d", "M0,-3L6,0L0,3").attr("fill", "#bbb");

// Trajectory colours
const trajColor = stableColor === "stable"   ? "#1e6fcc" :
                  stableColor === "unstable"  ? "#cc3333" :
                  stableColor === "saddle"    ? null       : "#888";

// 8 trajectories from radius 2, evenly spaced angles
const nTraj = 8;
const r0 = 2;
const dt = 0.02, nSteps = 300;

function eulerSteps(x0, y0, forward) {
  const pts = [[x0, y0]];
  let x = x0, y = y0;
  const sign = forward ? 1 : -1;
  for (let s = 0; s < nSteps; s++) {
    const dx = a11 * x + a12 * y;
    const dy = a21 * x + a22 * y;
    x += sign * dt * dx;
    y += sign * dt * dy;
    if (Math.abs(x) > 6 || Math.abs(y) > 6) break;
    pts.push([x, y]);
  }
  return pts;
}

const lineGen = d3.line()
  .x(p => xSc(p[0]))
  .y(p => ySc(p[1]))
  .defined(p => Math.abs(p[0]) <= 3.2 && Math.abs(p[1]) <= 3.2);

for (let k = 0; k < nTraj; k++) {
  const angle = (2 * Math.PI * k) / nTraj;
```

```
    const x0 = r0 * Math.cos(angle);
    const y0 = r0 * Math.sin(angle);

    const fwd = eulerSteps(x0, y0, true);
    const bwd = eulerSteps(x0, y0, false).slice(1).reverse();
    const pts = [...bwd, ...fwd];

    // Colour saddle trajectories by quadrant (proxy)
    let color;
    if (stableColor === "saddle") {
      color = (k % 2 === 0) ? "#1e6fcc" : "#cc3333";
    } else {
      color = trajColor;
    }

    g.append("path")
      .datum(pts)
      .attr("d", lineGen)
      .attr("fill", "none")
      .attr("stroke", color)
      .attr("stroke-width", 1.4)
      .attr("opacity", 0.75);
  }

  // Equilibrium dot at origin
  g.append("circle")
    .attr("cx", xSc(0)).attr("cy", ySc(0))
    .attr("r", 5)
    .attr("fill", stableColor === "stable" ? "#1e6fcc" : stableColor === "unstable" ? "#cc3333" : "#
    .attr("stroke", "#333").attr("stroke-width", 1.5);

  // Classification label
  const labelColor = stableColor === "stable" ? "#1e6fcc" : stableColor === "unstable" ? "#cc3333"
  g.append("text")
    .attr("x", iW / 2).attr("y", iH + 30)
    .attr("text-anchor", "middle")
    .attr("font-size", 13).attr("font-weight", "bold")
    .attr("fill", labelColor)
    .text(`${classification}  |  λ₁ = ${lamStr1},  λ₂ = ${lamStr2}`);

  return svg.node();
}
```

## 38.4. Stability

### 38.4.1. Lyapunov's definition

An equilibrium $\mathbf{x}^*$ is **stable** (in the sense of Lyapunov) if trajectories that start close to $\mathbf{x}^*$ stay close — formally: for every $\varepsilon > 0$ there exists $\delta > 0$ such that $\|\mathbf{x}(0) - \mathbf{x}^*\| < \delta$ implies $\|\mathbf{x}(t) - \mathbf{x}^*\| < \varepsilon$ for all $t > 0$.

An equilibrium is **asymptotically stable** if it is stable and, additionally, trajectories starting near it converge to it: $\mathbf{x}(t) \to \mathbf{x}^*$ as $t \to \infty$.

An equilibrium is **unstable** if it fails the stability condition — there exist trajectories starting arbitrarily close that eventually leave a fixed neighbourhood.

For linear systems, the connection to eigenvalues is exact:

- All eigenvalues have negative real part ⇒ asymptotically stable.
- Any eigenvalue with positive real part ⇒ unstable.
- Purely imaginary eigenvalues ⇒ stable (but not asymptotically stable) for the linear system; the nonlinear case requires more care.

### 38.4.2. Engineering implications

The stability of an equilibrium is the primary question in control engineering. A feedback controller is designed to make the closed-loop eigenvalues lie in the left half of the complex plane ($\mathrm{Re}(\lambda) < 0$). How far left they sit determines how quickly the system returns to equilibrium after a disturbance — the damping rate is $|\mathrm{Re}(\lambda)|$, and the oscillation frequency is $|\mathrm{Im}(\lambda)|$.

A saddle equilibrium in a control loop is dangerous: the system will diverge for almost any initial condition, with only the stable manifold — a set of measure zero — as the exception.

---

## 38.5. Nonlinear systems: linearisation

### 38.5.1. Why linearise?

For a nonlinear system $\mathbf{x}' = \mathbf{f}(\mathbf{x})$, we generally cannot write down explicit solutions. But the behaviour near an equilibrium $\mathbf{x}^*$ can still be understood by approximating $\mathbf{f}$ with its best linear approximation there — a Taylor expansion.

### 38.5.2. The Jacobian

Let $\mathbf{x}^*$ satisfy $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Write $\mathbf{x}(t) = \mathbf{x}^* + \mathbf{u}(t)$ where $\mathbf{u}$ is a small perturbation. Expanding $\mathbf{f}$ to first order:

$$\mathbf{x}' = \mathbf{f}(\mathbf{x}^* + \mathbf{u}) \approx \mathbf{f}(\mathbf{x}^*) + J(\mathbf{x}^*)\,\mathbf{u} = J(\mathbf{x}^*)\,\mathbf{u}$$

since $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. The matrix $J$ is the **Jacobian** of $\mathbf{f}$:

$$J = \begin{pmatrix} \partial f_1/\partial x_1 & \partial f_1/\partial x_2 \\ \partial f_2/\partial x_1 & \partial f_2/\partial x_2 \end{pmatrix}$$

evaluated at $\mathbf{x}^*$. So the perturbation $\mathbf{u}$ satisfies the linear system $\mathbf{u}' = J\mathbf{u}$. The eigenvalues of $J$ classify the equilibrium, exactly as before.

This works reliably when the eigenvalues of $J$ are not purely imaginary. If $J$ has purely imaginary eigenvalues, the linearisation is inconclusive about the nonlinear behaviour — the equilibrium could be a centre, a spiral, or even something more exotic.

### 38.5.3. The pendulum

The undamped pendulum satisfies:

$$\theta'' + \sin\theta = 0$$

(with appropriate normalisation of length and gravity). Converting to a system: $x_1 = \theta$, $x_2 = \theta'$:

$$x_1' = x_2, \qquad x_2' = -\sin x_1$$

**Equilibria:** Set $x_2 = 0$ and $-\sin x_1 = 0$. Equilibria are at $(n\pi, 0)$ for integer $n$.

- $(0, 0)$: the pendulum hanging straight down.
- $(\pi, 0)$: the pendulum balanced straight up.

**Jacobian:**

$$J = \begin{pmatrix} 0 & 1 \\ -\cos x_1 & 0 \end{pmatrix}$$

**At** $(0, 0)$**:** $J = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$. Eigenvalues: $\lambda = \pm i$. The linearisation gives a centre — consistent with the familiar small-angle oscillation. (The full nonlinear pendulum also has closed orbits near this point, which requires a more careful argument to confirm.)

**At** $(\pi, 0)$**:** $J = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. Eigenvalues: $\lambda = \pm 1$. A saddle — the upright position is unstable, as expected. The tiniest perturbation sends the pendulum swinging away.

The phase portrait of the pendulum has closed orbits (libration) near $(0, 0)$ and open orbits (rotation) beyond a separatrix that passes through the saddle point $(\pm\pi, 0)$. The separatrix is the boundary between "swings back" and "goes over the top".

---

## 38.6. Lotka-Volterra predator-prey

### 38.6.1. The equations

The Lotka-Volterra model for a prey population $x$ (rabbits) and predator population $y$ (foxes) is:

$$\begin{aligned} x' &= ax - bxy \\ y' &= -dy + cxy \end{aligned}$$

All parameters $a, b, c, d > 0$.

- $ax$: prey grow exponentially in the absence of predators.
- $-bxy$: prey are removed at a rate proportional to encounters with predators.
- $-dy$: predators decline exponentially in the absence of prey.
- $cxy$: predators grow from encounters with prey.

### 38.6.2. Equilibria

Set $x' = 0$ and $y' = 0$:

$$x(a - by) = 0 \quad \text{and} \quad y(-d + cx) = 0$$

This gives two equilibria:

1. $(0, 0)$: both populations extinct.
2. $(d/c,\ a/b)$: coexistence equilibrium — both populations positive.

### 38.6.3. Classifying via the Jacobian

$$\mathbf{f}(x, y) = \begin{pmatrix} ax - bxy \\ -dy + cxy \end{pmatrix}, \qquad J = \begin{pmatrix} a - by & -bx \\ cy & -d + cx \end{pmatrix}$$

**At** $(0, 0)$**:**

$$J(0, 0) = \begin{pmatrix} a & 0 \\ 0 & -d \end{pmatrix}$$

Eigenvalues: $\lambda_1 = a > 0$, $\lambda_2 = -d < 0$. A saddle. The extinction equilibrium is unstable — any positive initial population will move away from it.

**At** $(d/c,\ a/b)$**:**

Substituting $x^* = d/c$, $y^* = a/b$:

$$J(x^*, y^*) = \begin{pmatrix} 0 & -bd/c \\ ca/b & 0 \end{pmatrix}$$

Characteristic polynomial: $\lambda^2 + ad = 0$, giving $\lambda = \pm i\sqrt{ad}$.

Purely imaginary eigenvalues. The linearisation gives a centre. (For the full Lotka-Volterra system it can be shown, using a conserved quantity, that the nonlinear equilibrium is also a true centre — the populations cycle indefinitely around the coexistence point.)

### 38.6.4. Physical interpretation

The coexistence equilibrium $(d/c, a/b)$ is a neutrally stable centre. The populations do not converge to these values; instead, they cycle around them. The amplitude of the cycle depends on the initial conditions. Start with many rabbits and few foxes: the fox population grows, rabbit numbers fall, fox numbers eventually fall too, then rabbits recover. The cycle is closed.

This explains the wildlife warden's observation: the populations oscillate, neither dying out nor converging to a constant value, with the phase plane orbit a closed curve around the coexistence point.

```
//| echo: false
viewof lvParams = Inputs.form({
  lvA:  Inputs.range([0.1, 2],    {step: 0.05, value: 0.8, label: "a (prey growth rate)"}),
  lvB:  Inputs.range([0.01, 0.5], {step: 0.01, value: 0.1, label: "b (predation rate)"}),
  lvC:  Inputs.range([0.01, 0.5], {step: 0.01, value: 0.1, label: "c (predator efficiency)"}),
  lvD:  Inputs.range([0.1, 2],    {step: 0.05, value: 0.5, label: "d (predator death rate)"}),
  lvX0: Inputs.range([1, 20],     {step: 0.5,  value: 8,   label: "x₀ (initial prey)"}),
  lvY0: Inputs.range([1, 20],     {step: 0.5,  value: 4,   label: "y₀ (initial predators)"})
})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const a  = lvParams.lvA;
  const b  = lvParams.lvB;
  const c  = lvParams.lvC;
  const d  = lvParams.lvD;
  const x0 = lvParams.lvX0;
  const y0 = lvParams.lvY0;

  // RK4 integration of Lotka-Volterra
  const dt = 0.05, nSteps = 2000;
  const tMax = 100;
  const storeEvery = 1;  // store every step
```

```
const xs = [x0], ys = [y0], ts = [0];
let cx = x0, cy = y0, ct = 0;

for (let i = 0; i < nSteps; i++) {
  // RK4
  const k1x = a * cx       - b * cx * cy;
  const k1y = c * cx * cy  - d * cy;

  const mx2 = cx + 0.5 * dt * k1x;
  const my2 = cy + 0.5 * dt * k1y;
  const k2x = a * mx2      - b * mx2 * my2;
  const k2y = c * mx2*my2  - d * my2;

  const mx3 = cx + 0.5 * dt * k2x;
  const my3 = cy + 0.5 * dt * k2y;
  const k3x = a * mx3      - b * mx3 * my3;
  const k3y = c * mx3*my3  - d * my3;

  const mx4 = cx + dt * k3x;
  const my4 = cy + dt * k3y;
  const k4x = a * mx4      - b * mx4 * my4;
  const k4y = c * mx4*my4  - d * my4;

  cx = cx + (dt / 6) * (k1x + 2*k2x + 2*k3x + k4x);
  cy = cy + (dt / 6) * (k1y + 2*k2y + 2*k3y + k4y);
  ct += dt;

  // clamp to avoid blow-up
  if (cx < 0) cx = 0;
  if (cy < 0) cy = 0;

  xs.push(cx);
  ys.push(cy);
  ts.push(ct);

  if (ct >= tMax) break;
}

// Equilibria
const eqX = d / c;
const eqY = a / b;

// Layout: two panels side by side, total 560px wide
const totalW = 560, panelW = 270, H = 300;
const margin = { top: 24, right: 18, bottom: 40, left: 44 };
const iW = panelW - margin.left - margin.right;
const iH = H      - margin.top  - margin.bottom;
```

```
const svg = d3.create("svg")
  .attr("width", totalW).attr("height", H)
  .style("font-family", "sans-serif");

// ── Left panel: Phase portrait ──────────────────────────────────────
const gL = svg.append("g")
  .attr("transform", `translate(${margin.left}, ${margin.top})`);

const xMax = Math.max(...xs) * 1.15 || 20;
const yMax = Math.max(...ys) * 1.15 || 20;

const xSc = d3.scaleLinear().domain([0, xMax]).range([0, iW]);
const ySc = d3.scaleLinear().domain([0, yMax]).range([iH, 0]);

// Background
gL.append("rect").attr("width", iW).attr("height", iH)
  .attr("fill", "#fafafa").attr("stroke", "#ccc");

// Trajectory
const phaseLine = d3.line()
  .x((_, i) => xSc(xs[i]))
  .y((_, i) => ySc(ys[i]));

gL.append("path")
  .datum(xs)
  .attr("d", phaseLine)
  .attr("fill", "none")
  .attr("stroke", "#5b8dd9")
  .attr("stroke-width", 1.6)
  .attr("opacity", 0.85);

// Equilibrium (0, 0) — open circle
gL.append("circle")
  .attr("cx", xSc(0)).attr("cy", ySc(0))
  .attr("r", 5).attr("fill", "white")
  .attr("stroke", "#555").attr("stroke-width", 1.5);

// Equilibrium (d/c, a/b) — filled circle
if (eqX <= xMax && eqY <= yMax) {
  gL.append("circle")
    .attr("cx", xSc(eqX)).attr("cy", ySc(eqY))
    .attr("r", 5).attr("fill", "#e07b2a")
    .attr("stroke", "#333").attr("stroke-width", 1.5);
}

// Starting point
gL.append("circle")
```

```
  .attr("cx", xSc(x0)).attr("cy", ySc(y0))
  .attr("r", 4).attr("fill", "#333");

// Axes
const xAxisL = d3.axisBottom(xSc).ticks(5);
const yAxisL = d3.axisLeft(ySc).ticks(5);
gL.append("g").attr("transform", `translate(0,${iH})`).call(xAxisL);
gL.append("g").call(yAxisL);

// Axis labels
gL.append("text").attr("x", iW / 2).attr("y", iH + 32)
  .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#444")
  .text("x (prey)");
gL.append("text")
  .attr("transform", `translate(-32,${iH/2}) rotate(-90)`)
  .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#444")
  .text("y (predators)");
gL.append("text").attr("x", iW / 2).attr("y", -8)
  .attr("text-anchor", "middle").attr("font-size", 12).attr("font-weight", "bold").attr("fill",
  .text("Phase portrait");

// ── Right panel: Time series ──────────────────────────────────────────
const gR = svg.append("g")
  .attr("transform", `translate(${panelW + margin.left}, ${margin.top})`);

const tEnd = ts[ts.length - 1];
const popMax = Math.max(...xs, ...ys) * 1.15 || 20;

const tSc   = d3.scaleLinear().domain([0, tEnd]).range([0, iW]);
const popSc = d3.scaleLinear().domain([0, popMax]).range([iH, 0]);

gR.append("rect").attr("width", iW).attr("height", iH)
  .attr("fill", "#fafafa").attr("stroke", "#ccc");

// Prey x(t) in green
const xLine = d3.line()
  .x((_, i) => tSc(ts[i]))
  .y((_, i) => popSc(xs[i]));

gR.append("path")
  .datum(xs)
  .attr("d", xLine)
  .attr("fill", "none")
  .attr("stroke", "#2e8b57")
  .attr("stroke-width", 1.6);

// Predator y(t) in orange
```

```
  const yLine = d3.line()
    .x((_, i) => tSc(ts[i]))
    .y((_, i) => popSc(ys[i]));

  gR.append("path")
    .datum(ys)
    .attr("d", yLine)
    .attr("fill", "none")
    .attr("stroke", "#e07b2a")
    .attr("stroke-width", 1.6);

  // Axes
  const xAxisR = d3.axisBottom(tSc).ticks(5);
  const yAxisR = d3.axisLeft(popSc).ticks(5);
  gR.append("g").attr("transform", `translate(0,${iH})`).call(xAxisR);
  gR.append("g").call(yAxisR);

  // Axis labels
  gR.append("text").attr("x", iW / 2).attr("y", iH + 32)
    .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#444")
    .text("t");
  gR.append("text")
    .attr("transform", `translate(-32,${iH/2}) rotate(-90)`)
    .attr("text-anchor", "middle").attr("font-size", 11).attr("fill", "#444")
    .text("population");
  gR.append("text").attr("x", iW / 2).attr("y", -8)
    .attr("text-anchor", "middle").attr("font-size", 12).attr("font-weight", "bold").attr("fill", "#
    .text("Time series");

  // Legend
  const legX = iW - 100, legY = 8;
  gR.append("line").attr("x1", legX).attr("x2", legX+18).attr("y1", legY).attr("y2", legY)
    .attr("stroke", "#2e8b57").attr("stroke-width", 2);
  gR.append("text").attr("x", legX+22).attr("y", legY+4)
    .attr("font-size", 10).attr("fill", "#2e8b57").text("prey (x)");
  gR.append("line").attr("x1", legX).attr("x2", legX+18).attr("y1", legY+16).attr("y2", legY+16)
    .attr("stroke", "#e07b2a").attr("stroke-width", 2);
  gR.append("text").attr("x", legX+22).attr("y", legY+20)
    .attr("font-size", 10).attr("fill", "#e07b2a").text("predators (y)");

  return svg.node();
}
```

## 38.7. Application: Coupled spring system

Two masses $m_1$ and $m_2$ are connected by three springs with constants $k_1$, $k$, $k_2$, attached to fixed walls at both ends. The middle spring couples the two masses. Let $x_1$ and $x_2$ be displacements from equilibrium.

Newton's second law for each mass:

$$m_1 x_1'' = -k_1 x_1 - k(x_1 - x_2) = -(k_1 + k)x_1 + kx_2$$
$$m_2 x_2'' = -k_2 x_2 + k(x_1 - x_2) = kx_1 - (k_2 + k)x_2$$

For equal masses $m_1 = m_2 = 1$ and equal springs $k_1 = k_2 = k = 1$:

$$x_1'' = -2x_1 + x_2, \qquad x_2'' = x_1 - 2x_2$$

Convert to a first-order system with state vector $(x_1, x_2, x_1', x_2')^T$:

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -2 & 1 & 0 & 0 \\ 1 & -2 & 0 & 0 \end{pmatrix}$$

The eigenvalues of $A$ are $\pm i$ and $\pm i\sqrt{3}$ — two pairs of purely imaginary eigenvalues. The system oscillates at two **natural frequencies**: $\omega_1 = 1$ (the two masses moving together, in phase) and $\omega_2 = \sqrt{3}$ (the two masses moving in opposition, out of phase). These are the **normal modes** of the system.

Every motion of the coupled system is a superposition of these two modes. If the initial conditions excite only one mode, that mode persists unchanged. If both are excited, the motion is a superposition — a beating pattern in which energy transfers back and forth between the two masses.

---

## 38.8. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
{
  const css = document.createElement("link");
  css.rel = "stylesheet";
  css.href = "https://cdn.jsdelivr.net/npm/katex@0.16/dist/katex.min.css";
  document.head.appendChild(css);
}
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "Solution complete.";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
```

```
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  updateButtons();
  return container;
}
```

**Exercise 1.** Convert $y'' + 3y' + 2y = 0$ to a first-order system; write in matrix form; find the eigenvalues.

```
//| echo: false
makeStepperHTML([
  { op: "Introduce state variables", eq: "x_1 = y,\\quad x_2 = y' \\implies x_1' = x_2,\\quad x_2
  { op: "Write in matrix form", eq: "\\begin{pmatrix}x_1'\\\\x_2'\\end{pmatrix} = \\begin{pmatrix
  { op: "Characteristic polynomial", eq: "\\det(A - \\lambda I) = \\lambda^2 + 3\\lambda + 2" },
  { op: "Factor and solve", eq: "(\\lambda+1)(\\lambda+2) = 0 \\implies \\lambda_1 = -1,\\quad \\
  { op: "Interpret", eq: "\\lambda_1, \\lambda_2 < 0 \\implies \\text{stable node — solutions dec
])
```

---

**Exercise 2.** Given $A = \begin{pmatrix} 2 & 1 \\ 0 & -1 \end{pmatrix}$, find the eigenvalues and classify the equilibrium at the origin.

```
//| echo: false
makeStepperHTML([
  { op: "Read eigenvalues (triangular)", eq: "\\lambda_1 = 2,\\quad \\lambda_2 = -1", note: "Uppe
  { op: "Check signs", eq: "\\lambda_1 > 0,\\quad \\lambda_2 < 0 \\implies \\text{saddle point —
  { op: "Eigenvector for ⍰₁ = 2", eq: "(A-2I)\\mathbf{v}=0 \\implies \\mathbf{v}_1 = (1,0)^T" },
  { op: "Eigenvector for ⍰₂ = −1", eq: "(A+I)\\mathbf{v}=0 \\implies \\mathbf{v}_2 = (1,-3)^T" },
  { op: "Phase portrait", eq: "\\text{Unstable manifold along }(1,0)^T\\text{; stable manifold al
])
```

---

**Exercise 3.** Given $A = \begin{pmatrix} -1 & 2 \\ -2 & -1 \end{pmatrix}$, find the eigenvalues, classify the equilibrium, and write the general solution.

Step 1 | Compute the characteristic polynomial | $\det(A - \lambda I) = (-1-\lambda)^2 + 4 = \lambda^2 + 2\lambda + 5$ Step 2 | Solve | $\lambda = \dfrac{-2 \pm \sqrt{4-20}}{2} = -1 \pm 2i$ Step 3 | Classify | Complex eigenvalues with $\alpha = -1 < 0$: **stable spiral**. All trajectories spiral into the origin. Step 4 | Find a complex eigenvector | For $\lambda = -1 + 2i$: $(A - \lambda I)\mathbf{v} = 0$ gives $\begin{pmatrix} -2i & 2 \\ -2 & -2i \end{pmatrix}\mathbf{v} = 0$, so $\mathbf{v} = \begin{pmatrix} 1 \\ i \end{pmatrix}$. Step 5 | Write the real general solution | Separate real and imaginary parts of $e^{(-1+2i)t}\begin{pmatrix} 1 \\ i \end{pmatrix}$: the two real solutions are $e^{-t}\begin{pmatrix} \cos 2t \\ -\sin 2t \end{pmatrix}$ and $e^{-t}\begin{pmatrix} \sin 2t \\ \cos 2t \end{pmatrix}$. General solution: $\mathbf{x}(t) = e^{-t}\left[c_1\begin{pmatrix} \cos 2t \\ -\sin 2t \end{pmatrix} + c_2\begin{pmatrix} \sin 2t \\ \cos 2t \end{pmatrix}\right]$

---

**Exercise 4.** Given $A = \begin{pmatrix} 0 & 1 \\ -4 & 0 \end{pmatrix}$, find the eigenvalues, classify as a centre, write the general solution, and describe the phase portrait.

Step 1 | Compute the characteristic polynomial | $\det(A - \lambda I) = -\lambda(-\lambda) + 4 = \lambda^2 + 4$ Step 2 | Solve | $\lambda = \pm 2i$: purely imaginary eigenvalues. Step 3 | Classify | **Centre.** All trajectories are closed curves — periodic orbits. Step 4 | Find a complex eigenvector | For $\lambda = 2i$: $(A - 2iI)\mathbf{v} = 0$ gives $\begin{pmatrix} -2i & 1 \\ -4 & -2i \end{pmatrix}\mathbf{v} = 0$, so $\mathbf{v} = \begin{pmatrix} 1 \\ 2i \end{pmatrix}$. Step 5 | Write the general solution | $e^{2it}\begin{pmatrix} 1 \\ 2i \end{pmatrix}$ has real part $\begin{pmatrix} \cos 2t \\ -2\sin 2t \end{pmatrix}$ and imaginary part $\begin{pmatrix} \sin 2t \\ 2\cos 2t \end{pmatrix}$. General solution: $\mathbf{x}(t) = c_1\begin{pmatrix} \cos 2t \\ -2\sin 2t \end{pmatrix} + c_2\begin{pmatrix} \sin 2t \\ 2\cos 2t \end{pmatrix}$ Step 6 | Describe the phase portrait | Eliminating $t$: if $x = c_1\cos 2t + c_2 \sin 2t$ and $y = -2c_1 \sin 2t + 2c_2 \cos 2t$, then $x^2 + (y/2)^2 = c_1^2 + c_2^2$. The trajectories are ellipses, with semi-axes in ratio $1 : 2$ (elongated vertically), traversed in the clockwise direction.

---

**Exercise 5.** Linearise $\{x' = x(1-y),\ y' = y(x-1)\}$ at the equilibrium $(1,1)$ and classify it.

Step 1 | Verify equilibrium | At $(1,1)$: $x' = 1 \cdot (1-1) = 0$. $y' = 1 \cdot (1-1) = 0$. Confirmed. Step 2 | Compute the Jacobian | $f(x,y) = x - xy$, $g(x,y) = xy - y$. $J = \begin{pmatrix} \partial f/\partial x & \partial f/\partial y \\ \partial g/\partial x & \partial g/\partial y \end{pmatrix} = \begin{pmatrix} 1-y & -x \\ y & x-1 \end{pmatrix}$ Step 3 | Evaluate at $(1,1)$ | $J(1,1) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ Step 4 | Find eigenvalues | $\det(J - \lambda I) = \lambda^2 + 1 = 0 \Rightarrow \lambda = \pm i$ Step 5 | Classify | Purely imaginary eigenvalues: the linearisation is a **centre**. The system is equivalent to Lotka-Volterra with $a = b = c = d = 1$; the true nonlinear equilibrium is also a centre (orbits are closed). The linearisation is inconclusive about the nonlinear stability — a Lyapunov function argument is needed to confirm the orbits are closed.

---

*Exercise 6 extends the methods to a system with a continuum of equilibria — one of which has a zero eigenvalue. The approach is the same: Jacobian, eigenvalues, interpret. The $R_0$ interpretation is the key result.*

**Exercise 6.** An epidemic model: $S' = -0.002SI$, $I' = 0.002SI - 0.5I$ (SIR without recovery back to $S$). Find all equilibria. Classify the disease-free equilibrium $(S_0, 0)$. Interpret in terms of outbreak dynamics.

Step 1 | Find equilibria | Set $S' = 0$: $-0.002SI = 0 \Rightarrow S = 0$ or $I = 0$. Set $I' = 0$: $I(0.002S - 0.5) = 0 \Rightarrow I = 0$ or $S = 250$. Equilibria: $(S, 0)$ for any $S$ (disease-free line) and $(250, 0)$, which is on the disease-free line. There is no interior equilibrium with $I > 0$. Step 2 | Compute the Jacobian | $f = -0.002SI$, $g = 0.002SI - 0.5I$. $J = \begin{pmatrix} -0.002I & -0.002S \\ 0.002I & 0.002S - 0.5 \end{pmatrix}$ Step 3 | Evaluate at $(S_0, 0)$ | $J(S_0, 0) = \begin{pmatrix} 0 & -0.002S_0 \\ 0 & 0.002S_0 - 0.5 \end{pmatrix}$ Step 4 | Read eigenvalues (triangular matrix) | $\lambda_1 = 0$, $\lambda_2 = 0.002S_0 - 0.5$ Step 5 | Classify | If $S_0 < 250$: $\lambda_2 < 0$, the disease-free equilibrium is stable (no outbreak). If $S_0 > 250$: $\lambda_2 > 0$, the equilibrium is unstable (outbreak occurs). The threshold is $S_0 = 250 = 0.5/0.002$. Step 6 | Interpret | The ratio $R_0 = 0.002S_0/0.5 = S_0/250$ is the basic reproduction number. If $R_0 > 1$ (more than 250 susceptibles), an infection introduced into the population will spread. If $R_0 < 1$, any outbreak dies out. This is the fundamental threshold theorem of mathematical epidemiology, here derived as a stability condition on the disease-free equilibrium.

---

Key results

1. A second-order ODE $y'' + py' + qy = 0$ is equivalent to the linear system $\mathbf{x}' = A\mathbf{x}$ with $A = \begin{pmatrix} 0 & 1 \\ -q & -p \end{pmatrix}$.
2. The general solution is $\mathbf{x}(t) = c_1 e^{\lambda_1 t}\mathbf{v}_1 + c_2 e^{\lambda_2 t}\mathbf{v}_2$.
3. Eigenvalue classification: real same-sign → node; real opposite-sign → saddle; complex → spiral; purely imaginary → centre.
4. Stability: asymptotically stable if and only if all eigenvalues have negative real part.
5. For nonlinear systems, linearise via the Jacobian $J$ at each equilibrium and classify from the eigenvalues of $J$.

## 38.9. Where this goes

The eigenvalue geometry developed here connects directly to three next steps.

**Laplace transforms** (Chapter 4 of this volume) re-express the same eigenvalue information as poles of a transfer function in the complex $s$-plane. A stable spiral in the phase plane corresponds to a pair of complex conjugate poles with negative real part — same mathematics, different language.

**Linear algebra** (Vol 7 linear algebra section) develops the eigenvalue theory further: diagonalisation, the spectral theorem, and Jordan form handle cases where eigenvectors are not a full basis. The phase plane intuition built here makes those abstract results concrete.

**Partial differential equations** (Vol 7 Fourier–PDEs section) extend the phase plane idea to spatial systems: the stability analysis of a PDE linearised around a steady state uses the same eigenvalue logic, now applied to differential operators rather than matrices.

# 39. Laplace transforms

Converting differential equations to algebra

A solenoid valve in a hydraulic system is closed. At exactly $t = 2$ seconds it opens, sending a pressure pulse through the system. The ODE governing the pressure response has discontinuous forcing — zero for $t < 2$, nonzero for $t \geq 2$. The characteristic equation method does not handle this cleanly; you would need to split the problem at $t = 2$, solve each piece separately, then match conditions at the join. For two segments that is tedious. For a system that switches at $t = 2$, $t = 5$, and $t = 11$, it becomes impractical.

The Laplace transform handles it in one pass. The discontinuity is encoded as an exponential factor $e^{-2s}$ inside an algebraic expression in $s$. Solve the algebra. Invert. Done.

This chapter develops that mechanism from the integral definition through to impulse response and the convolution theorem.

## 39.1. Definition and motivation

The Laplace transform of a function $f(t)$ defined for $t \geq 0$ is

$$\mathscr{L}\{f(t)\} = F(s) = \int_0^\infty e^{-st} f(t)\, dt$$

The variable $s$ is a complex number, though for most engineering applications we treat it as a real parameter and work in the right half-plane where the integral converges.

**Why does this turn a differential equation into algebra?** Two reasons.

First, the transform converts differentiation in $t$ into multiplication by $s$ in the frequency domain (plus boundary terms carrying the initial conditions). We derive this precisely in Section 3.

Second, the transform is linear:

$$\mathscr{L}\{af(t) + bg(t)\} = aF(s) + bG(s)$$

This follows directly from linearity of the integral. So a linear ODE in $t$ becomes a linear algebraic equation in $s$. Solve for $Y(s)$, then invert.

The four-step method, which we will use throughout this chapter:

1. **Take the Laplace transform** of both sides of the ODE.

2. **Apply the initial conditions** — they appear automatically in the boundary terms.
3. **Solve algebraically** for $Y(s)$.
4. **Invert** $Y(s)$ to find $y(t)$.

## 39.2. Table of transforms

We build the table entry by entry from the integral definition.

**Transform of 1**

$$\mathscr{L}\{1\} = \int_0^\infty e^{-st}\,dt = \left[-\frac{e^{-st}}{s}\right]_0^\infty = \frac{1}{s}, \quad s > 0$$

The boundary term at $t \to \infty$ vanishes because $e^{-st} \to 0$ for $s > 0$.

**Transform of $e^{at}$**

$$\mathscr{L}\{e^{at}\} = \int_0^\infty e^{-st}e^{at}\,dt = \int_0^\infty e^{-(s-a)t}\,dt = \frac{1}{s-a}, \quad s > a$$

The same calculation as above but with $s$ replaced by $s - a$.

**Transform of $\sin(\omega t)$**

Use the identity $\sin(\omega t) = \frac{e^{i\omega t} - e^{-i\omega t}}{2i}$, or integrate directly using integration by parts twice. The direct route:

$$\mathscr{L}\{\sin(\omega t)\} = \int_0^\infty e^{-st}\sin(\omega t)\,dt$$

Let $I$ denote the integral. For the first integration by parts, set $u = e^{-st}$ and $dv = \sin(\omega t)\,dt$, so $du = -se^{-st}\,dt$ and $v = -\cos(\omega t)/\omega$:

$$I = \left[-\frac{e^{-st}\cos(\omega t)}{\omega}\right]_0^\infty - \int_0^\infty \frac{s}{\omega}e^{-st}\cos(\omega t)\,dt = \frac{1}{\omega} - \frac{s}{\omega}\int_0^\infty e^{-st}\cos(\omega t)\,dt$$

(The boundary term at $t = 0$ gives $1/\omega$; at $t \to \infty$ the exponential kills the cosine.)

Applying IBP again to $\int e^{-st}\cos(\omega t)\,dt$ by symmetry (set $u = e^{-st}$, $dv = \cos(\omega t)\,dt$, $v = \sin(\omega t)/\omega$) returns the original integral $I$ with a coefficient. After two applications the integral $I$ reappears on the right:

$$I = \frac{\omega}{s^2} - \frac{\omega^2}{s^2}I$$

Solving for $I$:

$$I\left(1 + \frac{\omega^2}{s^2}\right) = \frac{\omega}{s^2} \implies I = \frac{\omega}{s^2 + \omega^2}$$

Similarly, integrating $e^{-st}\cos(\omega t)$ by parts (or using the derivative relation) gives $\mathscr{L}\{\cos(\omega t)\} = \dfrac{s}{s^2+\omega^2}$.

**Transform of $t^n$**

Integrate by parts, reducing the power of $t$ by one at each step. The result is

$$\mathscr{L}\{t^n\} = \frac{n!}{s^{n+1}}, \quad s > 0$$

For $n = 1$: $\mathscr{L}\{t\} = 1/s^2$. For $n = 2$: $\mathscr{L}\{t^2\} = 2/s^3$.

**Reference table**

| $f(t)$ | $F(s) = \mathscr{L}\{f(t)\}$ | Condition |
|---|---|---|
| $1$ | $\frac{1}{s}$ | $s > 0$ |
| $t$ | $\frac{1}{s^2}$ | $s > 0$ |
| $t^n$ | $\frac{n!}{s^{n+1}}$ | $s > 0$ |
| $e^{at}$ | $\frac{1}{s-a}$ | $s > a$ |
| $te^{at}$ | $\frac{1}{(s-a)^2}$ | $s > a$ |
| $\sin(\omega t)$ | $\frac{\omega}{s^2+\omega^2}$ | $s > 0$ |
| $\cos(\omega t)$ | $\frac{s}{s^2+\omega^2}$ | $s > 0$ |
| $e^{at}\sin(\omega t)$ | $\frac{\omega}{(s-a)^2+\omega^2}$ | $s > a$ |
| $e^{at}\cos(\omega t)$ | $\frac{s-a}{(s-a)^2+\omega^2}$ | $s > a$ |
| $u(t-a)$ | $\frac{e^{-as}}{s}$ | $s > 0$ |
| $\delta(t-a)$ | $e^{-as}$ | all $s$ |

```
//| echo: false
viewof ltFunc = Inputs.select(
  ["1 (unit step)", "e^{at} (a=-1)", "sin(ωt) (ω=2)", "cos(ωt) (ω=2)", "t"],
  {label: "f(t)"}
)
```

```
//| echo: false
viewof ltS = Inputs.range([0.1, 5], {step: 0.1, value: 1, label: "s"})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  // Function definitions
  const funcs = {
    "1 (unit step)":    { ft: t => 1,          Fs: s => 1/s,              yMax: 1.4 },
```

```
  "e^{at} (a=-1)":   { ft: t => Math.exp(-t), Fs: s => 1/(s+1),         yMax: 1.4 },
  "sin(ωt) (ω=2)":   { ft: t => Math.sin(2*t), Fs: s => 2/(s*s+4),      yMax: 1.4 },
  "cos(ωt) (ω=2)":   { ft: t => Math.cos(2*t), Fs: s => s/(s*s+4),      yMax: 1.4 },
  "t":               { ft: t => t,            Fs: s => 1/(s*s),          yMax: 6.5 }
};

const def = funcs[ltFunc];
const ft = def.ft;
const Fs = def.Fs;
const s  = ltS;

// Layout
const panelW = 260, panelH = 240;
const margin = {top: 20, right: 15, bottom: 35, left: 42};
const innerW = panelW - margin.left - margin.right;
const innerH = panelH - margin.top - margin.bottom;
const totalW = panelW * 2 + 20;

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${totalW} ${panelH}`)
  .attr("width", totalW)
  .attr("style", "font-family: sans-serif; font-size: 11px;");

// ── LEFT PANEL: f(t) and e^{-st}·f(t) ──────────────────────────────
const tDomain = [0, 6];
const tValues = d3.range(tDomain[0], tDomain[1], 0.02);

const yMax = def.yMax;
const yMin = ltFunc === "cos(ωt) (ω=2)" || ltFunc === "sin(ωt) (ω=2)" ? -yMax * 0.6 : -0.15;

const xL = d3.scaleLinear().domain(tDomain).range([0, innerW]);
const yL = d3.scaleLinear().domain([yMin, yMax]).range([innerH, 0]);

const gL = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// Shaded area under e^{-st}·f(t)
const areaGen = d3.area()
  .x(t => xL(t))
  .y0(yL(0))
  .y1(t => yL(Math.exp(-s * t) * ft(t)));

gL.append("path")
  .datum(tValues)
  .attr("d", areaGen)
  .attr("fill", "orange")
  .attr("opacity", 0.18);
```

```
// Axes
gL.append("g").attr("transform", `translate(0,${yL(0)})`).call(d3.axisBottom(xL).ticks(5));
gL.append("g").call(d3.axisLeft(yL).ticks(5));

// Axis labels
gL.append("text").attr("x", innerW / 2).attr("y", innerH + 28)
  .attr("text-anchor", "middle").text("t");
gL.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -innerH / 2).attr("y", -32)
  .attr("text-anchor", "middle").text("value");

// f(t) in blue
const lineL = d3.line().x(t => xL(t)).y(t => yL(ft(t)));
gL.append("path")
  .datum(tValues)
  .attr("d", lineL)
  .attr("fill", "none")
  .attr("stroke", "steelblue")
  .attr("stroke-width", 2);

// e^{-st}·f(t) in orange
const lineI = d3.line().x(t => xL(t)).y(t => yL(Math.exp(-s * t) * ft(t)));
gL.append("path")
  .datum(tValues)
  .attr("d", lineI)
  .attr("fill", "none")
  .attr("stroke", "orange")
  .attr("stroke-width", 2);

// Legend
gL.append("line").attr("x1", 4).attr("y1", 8).attr("x2", 20).attr("y2", 8)
  .attr("stroke", "steelblue").attr("stroke-width", 2);
gL.append("text").attr("x", 23).attr("y", 12).text("f(t)").attr("fill", "steelblue");
gL.append("line").attr("x1", 55).attr("y1", 8).attr("x2", 71).attr("y2", 8)
  .attr("stroke", "orange").attr("stroke-width", 2);
gL.append("text").attr("x", 74).attr("y", 12).text("e⁻□□f(t)").attr("fill", "orange");

// Panel title
gL.append("text").attr("x", innerW / 2).attr("y", -6)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("font-weight", "bold")
  .text("Time domain");

// ── RIGHT PANEL: F(s) vs s ──────────────────────────────────────────
const gapX = panelW + 20;
const sValues = d3.range(0.1, 5.05, 0.05);
```

```
const fsValues = sValues.map(sv => Fs(sv));
const fsFinite = fsValues.filter(v => isFinite(v) && !isNaN(v));
const fsMax = Math.min(d3.max(fsFinite) * 1.1, 10);
const fsMin = Math.max(d3.min(fsFinite) * 1.1, -2);

const xR = d3.scaleLinear().domain([0.1, 5]).range([0, innerW]);
const yR = d3.scaleLinear().domain([fsMin, fsMax]).range([innerH, 0]);

const gR = svg.append("g")
  .attr("transform", `translate(${gapX + margin.left},${margin.top})`);

// Axes
gR.append("g").attr("transform", `translate(0,${yR(0)})`).call(d3.axisBottom(xR).ticks(5));
gR.append("g").call(d3.axisLeft(yR).ticks(5));

// Axis labels
gR.append("text").attr("x", innerW / 2).attr("y", innerH + 28)
  .attr("text-anchor", "middle").text("s");
gR.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -innerH / 2).attr("y", -32)
  .attr("text-anchor", "middle").text("F(s)");

// F(s) curve
const lineR = d3.line()
  .defined((_, i) => isFinite(fsValues[i]) && fsValues[i] <= fsMax + 1 && fsValues[i] >= fsMin - 1
  .x((_, i) => xR(sValues[i]))
  .y((_, i) => yR(fsValues[i]));

gR.append("path")
  .datum(sValues)
  .attr("d", lineR)
  .attr("fill", "none")
  .attr("stroke", "steelblue")
  .attr("stroke-width", 2);

// Vertical dashed line at current s
const currentFs = Fs(s);
if (isFinite(currentFs) && currentFs <= fsMax + 0.5) {
  gR.append("line")
    .attr("x1", xR(s)).attr("y1", 0)
    .attr("x2", xR(s)).attr("y2", innerH)
    .attr("stroke", "#999")
    .attr("stroke-width", 1.5)
    .attr("stroke-dasharray", "4,3");

  // Dot on curve
```

```
    const dotY = Math.max(0, Math.min(innerH, yR(currentFs)));
    gR.append("circle")
      .attr("cx", xR(s)).attr("cy", dotY)
      .attr("r", 5)
      .attr("fill", "steelblue");

    // Label
    const labelVal = currentFs.toFixed(3);
    gR.append("text")
      .attr("x", Math.min(xR(s) + 6, innerW - 80))
      .attr("y", Math.max(dotY - 8, 14))
      .attr("fill", "steelblue")
      .attr("font-size", "11px")
      .text(`F(${s.toFixed(1)}) = ${labelVal}`);
  }

  // Panel title
  gR.append("text").attr("x", innerW / 2).attr("y", -6)
    .attr("text-anchor", "middle").attr("font-size", "12px").attr("font-weight", "bold")
    .text("s-domain: F(s)");

  return svg.node();
}
```

## 39.3. The transform of derivatives

This is the mechanism that converts ODEs into algebra.

**First derivative.** Integrate by parts with $u = f(t)$, $dv = e^{-st}\,dt$:

$$\mathscr{L}\{f'(t)\} = \int_0^\infty e^{-st} f'(t)\,dt = \left[e^{-st} f(t)\right]_0^\infty + s\int_0^\infty e^{-st} f(t)\,dt$$

The boundary term: at $t \to \infty$ the factor $e^{-st}$ kills $f(t)$ (for functions of exponential order — meaning $|f(t)| \le Me^{ct}$ for some constants $M > 0$ and $c$; essentially, $f(t)$ does not grow faster than some exponential, and all functions you will encounter in practice satisfy this); at $t = 0$ we get $-f(0)$. Therefore:

$$\boxed{\mathscr{L}\{f'(t)\} = sF(s) - f(0)}$$

The initial condition $f(0)$ appears automatically as part of the algebra.

**Second derivative.** Apply the first-derivative rule to $f'$, treating $f'$ as a function in its own right:

$$\mathscr{L}\{f''(t)\} = s\mathscr{L}\{f'(t)\} - f'(0) = s[sF(s) - f(0)] - f'(0)$$

$$\boxed{\mathscr{L}\{f''(t)\} = s^2F(s) - sf(0) - f'(0)}$$

Both initial conditions appear. This is why the Laplace transform method is particularly clean for IVPs: the boundary data enters the algebra rather than being appended at the end.

The pattern continues: $\mathscr{L}\{f^{(n)}\} = s^nF(s) - s^{n-1}f(0) - \cdots - f^{(n-1)}(0)$.

## 39.4. Solving initial value problems

We now apply the four-step method.

### 39.4.1. First-order example

**Solve** $y' + 3y = 6$, $y(0) = 1$.

**Step 1: Take the Laplace transform of both sides.**

$$\mathscr{L}\{y'\} + 3\mathscr{L}\{y\} = \mathscr{L}\{6\}$$

$$[sY(s) - y(0)] + 3Y(s) = \frac{6}{s}$$

**Step 2: Apply the initial condition** $y(0) = 1$**.**

$$sY(s) - 1 + 3Y(s) = \frac{6}{s}$$

**Step 3: Solve algebraically for** $Y(s)$**.**

$$Y(s)(s+3) = \frac{6}{s} + 1 = \frac{6+s}{s}$$

$$Y(s) = \frac{s+6}{s(s+3)}$$

**Step 4: Invert.** We need partial fractions:

$$\frac{s+6}{s(s+3)} = \frac{A}{s} + \frac{B}{s+3}$$

Multiply through: $s + 6 = A(s+3) + Bs$.

- Set $s = 0$: $6 = 3A$, so $A = 2$.
- Set $s = -3$: $3 = -3B$, so $B = -1$.

Therefore:

$$Y(s) = \frac{2}{s} - \frac{1}{s+3}$$

Inverting term by term using the table ($\mathscr{L}^{-1}\{1/s\} = 1$, $\mathscr{L}^{-1}\{1/(s+3)\} = e^{-3t}$):

$$y(t) = 2 - e^{-3t}$$

Check: $y(0) = 2 - 1 = 1$ ☒. As $t \to \infty$, $y \to 2$ (the steady state, consistent with $3 \times 2 = 6$) ☒.

### 39.4.2. Second-order example

**Solve** $y'' + 5y' + 6y = e^{-t}$, $y(0) = 1$, $y'(0) = 0$.

**Step 1: Transform.**

$$[s^2Y - sy(0) - y'(0)] + 5[sY - y(0)] + 6Y = \frac{1}{s+1}$$

**Step 2: Apply initial conditions.**

$$s^2Y - s - 0 + 5sY - 5 + 6Y = \frac{1}{s+1}$$

$$(s^2 + 5s + 6)Y = \frac{1}{s+1} + s + 5$$

**Step 3: Solve for** $Y(s)$**.**

$$Y(s) = \frac{1}{(s+1)(s+2)(s+3)} + \frac{s+5}{(s+2)(s+3)}$$

Combine over the common denominator $(s+1)(s+2)(s+3)$:

$$Y(s) = \frac{1 + (s+5)(s+1)}{(s+1)(s+2)(s+3)} = \frac{s^2 + 6s + 6}{(s+1)(s+2)(s+3)}$$

**Step 4: Partial fractions.**

$$\frac{s^2 + 6s + 6}{(s+1)(s+2)(s+3)} = \frac{A}{s+1} + \frac{B}{s+2} + \frac{C}{s+3}$$

- $s = -1$: $1 - 6 + 6 = A(1)(2) \Rightarrow A = \frac{1}{2}$
- $s = -2$: $4 - 12 + 6 = B(-1)(1) \Rightarrow B = 2$
- $s = -3$: $9 - 18 + 6 = C(-2)(-1) \Rightarrow C = -\frac{3}{2}$

$$y(t) = \frac{1}{2}e^{-t} + 2e^{-2t} - \frac{3}{2}e^{-3t}$$

## 39.5. Partial fractions

Inverting $Y(s)$ almost always requires decomposing a rational function into terms matching the table. Two cases arise repeatedly.

### 39.5.1. Distinct real poles

Given $\dfrac{N(s)}{(s-r_1)(s-r_2)\cdots(s-r_n)}$ with distinct roots, write

$$\frac{N(s)}{(s-r_1)\cdots(s-r_n)} = \frac{A_1}{s-r_1} + \cdots + \frac{A_n}{s-r_n}$$

Each coefficient: $A_k = \dfrac{N(r_k)}{\prod_{j\neq k}(r_k - r_j)}$, or equivalently multiply both sides by $(s-r_k)$ and set $s = r_k$. We did this in both worked examples above.

### 39.5.2. Complex conjugate poles

When the denominator has a quadratic factor $s^2 + bs + c$ with no real roots (complex conjugate poles), keep the quadratic together and write a linear numerator:

$$\frac{N(s)}{(s-r)(s^2+bs+c)} = \frac{A}{s-r} + \frac{Bs+C}{s^2+bs+c}$$

After finding $A, B, C$ by equating coefficients or substituting convenient values of $s$, complete the square in the denominator and match to the sin and cos entries in the table.

**Example.** Invert $\dfrac{s}{(s+1)(s^2+4)}$.

$$\frac{s}{(s+1)(s^2+4)} = \frac{A}{s+1} + \frac{Bs+C}{s^2+4}$$

Multiply through: $s = A(s^2+4) + (Bs+C)(s+1)$.

- $s = -1$: $-1 = 5A$, so $A = -\frac{1}{5}$.
- $s = 0$: $0 = 4A + C$, so $C = \frac{4}{5}$.
- $s^2$ coefficient: here we equate coefficients of each power of $s$ rather than substituting a specific value — both methods work; coefficient matching is more systematic when substitution would give $0 = 0$. Equating $s^2$ terms: $0 = A + B$, so $B = \frac{1}{5}$.

$$Y(s) = \frac{-1/5}{s+1} + \frac{(1/5)s + 4/5}{s^2+4} = -\frac{1}{5}\cdot\frac{1}{s+1} + \frac{1}{5}\cdot\frac{s}{s^2+4} + \frac{2}{5}\cdot\frac{2}{s^2+4}$$

Inverting:

$$y(t) = -\frac{1}{5}e^{-t} + \frac{1}{5}\cos(2t) + \frac{2}{5}\sin(2t)$$

## 39.6. The Heaviside step function and piecewise forcing

Engineering systems are often forced by signals that switch on or off at a specific time. The Heaviside unit step function captures this:

$$u(t-a) = \begin{cases} 0 & t < a \\ 1 & t \geq a \end{cases}$$

Its Laplace transform is

$$\mathscr{L}\{u(t-a)\} = \int_a^\infty e^{-st}\,dt = \frac{e^{-as}}{s}$$

**Writing piecewise functions using $u(t-a)$.** Any function that switches on at $t = a$ can be written using the Heaviside function. The piecewise function

$$f(t) = \begin{cases} g(t) & 0 \leq t < a \\ h(t) & t \geq a \end{cases}$$

equals $g(t) + [h(t) - g(t)]u(t-a)$.

**Second shifting theorem.** If $\mathscr{L}\{f(t)\} = F(s)$, then

$$\mathscr{L}\{f(t-a)\,u(t-a)\} = e^{-as}F(s)$$

A time delay by $a$ in the $t$-domain multiplies the transform by $e^{-as}$. This is the mechanism that keeps piecewise forcing contained in a single algebraic expression.

### 39.6.1. Worked example: RC circuit with a step input

**Problem.** An RC circuit with $R = 1\,\Omega$ and $C = 1\,\text{F}$ has the voltage equation

$$\frac{dv}{dt} + v = V_{\text{in}}(t), \quad v(0) = 0$$

where $V_{\text{in}}(t) = u(t-2)$ — a unit voltage step applied at $t = 2$.

**Step 1: Transform.**

$$sV(s) - v(0) + V(s) = \frac{e^{-2s}}{s}$$

**Step 2: Apply $v(0) = 0$.**

$$V(s)(s+1) = \frac{e^{-2s}}{s}$$

**Step 3: Solve for $V(s)$.**

$$V(s) = \frac{e^{-2s}}{s(s+1)}$$

**Step 4: Invert.** Factor out $e^{-2s}$, invert the rest, then apply the shifting theorem.

The function without the exponential factor is $G(s) = \dfrac{1}{s(s+1)}$. Partial fractions: $\dfrac{1}{s(s+1)} = \dfrac{1}{s} - \dfrac{1}{s+1}$, so $g(t) = 1 - e^{-t}$.

By the second shifting theorem:

$$v(t) = g(t-2)\,u(t-2) = \left(1 - e^{-(t-2)}\right) u(t-2)$$

Explicitly:

$$v(t) = \begin{cases} 0 & t < 2 \\ 1 - e^{-(t-2)} & t \geq 2 \end{cases}$$

The circuit is at rest until $t = 2$. After the switch closes, the voltage rises exponentially toward 1 with time constant $\tau = RC = 1$ second.

```
//| echo: false
viewof srW0 = Inputs.range([0.5, 5], {step: 0.1, value: 2, label: "ω₀ (natural frequency)"})
```

```
//| echo: false
viewof srZeta = Inputs.range([0.05, 2], {step: 0.05, value: 0.3, label: "ζ (damping ratio)"})
```

```
//| echo: false
viewof srA = Inputs.range([0, 8], {step: 0.5, value: 3, label: "Switch time a (s)"})
```

```
//| echo: false
{
  const d3 = await require("d3@7");

  const w0   = srW0;
  const zeta = srZeta;
  const a    = srA;

  // RK4 numerical integration of y'' + 2ζω₀y' + ω₀²y = ω₀²·u(t-a)
  // State: [y, dy/dt]
  const dt   = 0.01;
  const tEnd = 20;
  const N    = Math.ceil(tEnd / dt);

  const ts = new Float64Array(N + 1);
```

```
const ys = new Float64Array(N + 1);

const deriv = (t, y, v) => {
  const u = t >= a ? 1.0 : 0.0;
  const dvdt = w0 * w0 * u - 2 * zeta * w0 * v - w0 * w0 * y;
  return [v, dvdt];
};

let y = 0, v = 0;
ts[0] = 0; ys[0] = 0;
for (let i = 0; i < N; i++) {
  const t = ts[i];
  const [k1y, k1v] = deriv(t,          y,                    v);
  const [k2y, k2v] = deriv(t + dt/2,   y + dt/2 * k1y,      v + dt/2 * k1v);
  const [k3y, k3v] = deriv(t + dt/2,   y + dt/2 * k2y,      v + dt/2 * k2v);
  const [k4y, k4v] = deriv(t + dt,     y + dt * k3y,         v + dt * k3v);
  y += (dt / 6) * (k1y + 2*k2y + 2*k3y + k4y);
  v += (dt / 6) * (k1v + 2*k2v + 2*k3v + k4v);
  ts[i+1] = t + dt;
  ys[i+1] = y;
}

// Layout
const width = 560, height = 300;
const margin = {top: 24, right: 24, bottom: 40, left: 50};
const innerW = width  - margin.left - margin.right;
const innerH = height - margin.top  - margin.bottom;

const yAll = Array.from(ys);
const yMax = Math.max(d3.max(yAll) * 1.15, 1.2);
const yMin = Math.min(d3.min(yAll) * 1.1, -0.15);

const xScale = d3.scaleLinear().domain([0, tEnd]).range([0, innerW]);
const yScale = d3.scaleLinear().domain([yMin, yMax]).range([innerH, 0]);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${width} ${height}`)
  .attr("width", width)
  .attr("style", "font-family: sans-serif; font-size: 11px;");

const g = svg.append("g")
  .attr("transform", `translate(${margin.left},${margin.top})`);

// Steady-state dashed line
g.append("line")
  .attr("x1", 0).attr("y1", yScale(1))
  .attr("x2", innerW).attr("y2", yScale(1))
```

```
    .attr("stroke", "#aaa")
    .attr("stroke-width", 1.5)
    .attr("stroke-dasharray", "6,4");

  g.append("text")
    .attr("x", innerW - 4).attr("y", yScale(1) - 5)
    .attr("text-anchor", "end")
    .attr("fill", "#888")
    .text("Steady state (y = 1)");

  // Switch time dashed line
  if (a <= tEnd) {
    g.append("line")
      .attr("x1", xScale(a)).attr("y1", 0)
      .attr("x2", xScale(a)).attr("y2", innerH)
      .attr("stroke", "#bbb")
      .attr("stroke-width", 1.5)
      .attr("stroke-dasharray", "4,3");

    g.append("text")
      .attr("x", xScale(a) + 4).attr("y", 14)
      .attr("fill", "#999")
      .text(`Switch at t = ${a}`);
  }

  // Axes
  g.append("g").attr("transform", `translate(0,${yScale(0)})`).call(d3.axisBottom(xScale).ticks(10)
  g.append("g").call(d3.axisLeft(yScale).ticks(6));

  // Axis labels
  g.append("text").attr("x", innerW / 2).attr("y", innerH + 32)
    .attr("text-anchor", "middle").text("t (s)");
  g.append("text")
    .attr("transform", "rotate(-90)")
    .attr("x", -innerH / 2).attr("y", -38)
    .attr("text-anchor", "middle").text("y(t)");

  // Response curve
  const lineGen = d3.line()
    .x((_, i) => xScale(ts[i]))
    .y((v) => yScale(v));

  g.append("path")
    .datum(yAll)
    .attr("d", lineGen)
    .attr("fill", "none")
    .attr("stroke", "steelblue")
```

```
    .attr("stroke-width", 2.5);

  // Damping label
  const dampLabel = zeta < 1 ? `Underdamped (ζ = ${zeta.toFixed(2)})` :
                    zeta === 1 ? "Critically damped" :
                    `Overdamped (ζ = ${zeta.toFixed(2)})`;
  g.append("text")
    .attr("x", 6).attr("y", 14)
    .attr("fill", "steelblue")
    .attr("font-size", "12px")
    .text(dampLabel);

  return svg.node();
}
```

## 39.7. The Dirac delta function

A solenoid strikes a piston at $t = a$. The contact lasts a fraction of a millisecond but delivers a definite impulse — a large force for a very short time. The Dirac delta function $\delta(t - a)$ is the mathematical idealisation: an infinitely concentrated spike at $t = a$ with unit area.

Formally, $\delta(t - a)$ is defined through its sifting property: for any continuous function $f$,

$$\int_{-\infty}^{\infty} f(t)\,\delta(t - a)\,dt = f(a)$$

It is not a function in the classical sense — it is a distribution. But for engineering purposes, the sifting property is all we need.

**Laplace transform of** $\delta(t - a)$ for $a > 0$:

$$\mathscr{L}\{\delta(t - a)\} = \int_{0}^{\infty} e^{-st}\delta(t - a)\,dt = e^{-as}$$

The sifting property pulls out the factor $e^{-st}$ evaluated at $t = a$. Notably, the transform of an impulse is just an exponential — a constant for any given $a$.

### 39.7.1. Impulse response of a second-order system

**Problem.** A mechanical oscillator receives a unit impulse at $t = 1$:

$$\ddot{x} + 2\dot{x} + 5x = \delta(t - 1), \quad x(0) = 0,\ \dot{x}(0) = 0$$

**Step 1: Transform.**

$$s^2X + 2sX + 5X = e^{-s}$$

**Step 2: Initial conditions already zero.**

$$(s^2 + 2s + 5)X(s) = e^{-s}$$

**Step 3: Solve for $X(s)$.**

$$X(s) = \frac{e^{-s}}{s^2 + 2s + 5}$$

**Step 4: Invert using the second shifting theorem.** Complete the square in the denominator: $s^2 + 2s + 5 = (s+1)^2 + 4$.

The function without $e^{-s}$ is $G(s) = \dfrac{1}{(s+1)^2 + 4}$.

From the table: $\mathscr{L}^{-1}\left\{\dfrac{\omega}{(s-a)^2 + \omega^2}\right\} = e^{at}\sin(\omega t)$, so with $a = -1$ and $\omega = 2$:

$$g(t) = \frac{1}{2}e^{-t}\sin(2t)$$

By the second shifting theorem:

$$x(t) = g(t-1)\,u(t-1) = \frac{1}{2}e^{-(t-1)}\sin(2(t-1))\,u(t-1)$$

The system is undisturbed until the impulse at $t = 1$. After the impulse, it responds with a damped sinusoidal oscillation — the system's natural behaviour, triggered instantaneously.

> **Impulse response and system characterisation**
> The impulse response $h(t)$ — the output when the input is $\delta(t)$ with zero initial conditions — completely characterises a linear time-invariant system. Any input $u(t)$ produces the output $\int_0^t h(t-\tau)u(\tau)\,d\tau$. This is the convolution integral, and it connects directly to the convolution theorem below.

## 39.8. Convolution theorem

For two functions $f$ and $g$, the convolution is

$$(f * g)(t) = \int_0^t f(t-\tau)\,g(\tau)\,d\tau$$

The convolution theorem states:

$$\mathscr{L}\{f * g\} = F(s) \cdot G(s)$$

Multiplication in the $s$-domain corresponds to convolution in the $t$-domain.

**Transfer function interpretation.** If a system has impulse response $h(t)$ with transform $H(s)$, then for input $U(s)$ the output is $Y(s) = H(s)U(s)$. Here $H(s)$ is the **transfer function** of the system. Control engineers characterise systems entirely through their transfer functions — poles of $H(s)$ determine stability, zeros determine how the system shapes the input signal. The step from a differential equation to a transfer function is the step from the $t$-domain to the $s$-domain via the Laplace transform.

## 39.9. Where this goes

The Laplace transform is the bridge from time-domain dynamics to the frequency domain. The next natural step is **Fourier series** and the **Fourier transform**, which decompose periodic or square-integrable signals into frequency components directly — without the convergence factor $e^{-st}$. The Fourier transform is obtained by restricting the Laplace transform to the imaginary axis $s = i\omega$. That connection, explored in the Fourier chapter, motivates the full theory of **complex analysis** (Vol 7): the inverse Laplace transform is a contour integral in the complex $s$-plane, the Bromwich integral, and understanding it properly requires residue calculus.

For PDEs, the Laplace transform in $t$ reduces certain parabolic and hyperbolic PDEs to ODEs in the spatial variable — an approach that reuses everything developed here.

**Where this shows up**

- A control engineer designing a PID controller writes the plant as a transfer function $G(s)$ and shapes the closed-loop response in the $s$-domain — poles, zeros, Bode plots, all Laplace-transform concepts.
- A nuclear engineer modelling a decay chain with time-varying neutron flux uses Laplace transforms to handle the discontinuous source term.
- A signal processing engineer designing a digital filter uses the Z-transform, the discrete analogue of the Laplace transform: $\mathscr{Z}\{x[n]\} = \sum_{n=0}^{\infty} x[n]z^{-n}$.
- A quant analyst pricing exotic options uses the characteristic function of the asset price process — effectively the Laplace transform of the risk-neutral density — with fast Fourier transform inversion.

## 39.10. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
```

```
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

**1.** Compute $\mathscr{L}\{3e^{2t} - 4\sin(3t)\}$ using the table and linearity.

```
//| echo: false
makeStepperHTML(1, [
  { op: "Apply linearity",
    eq: "\\mathcal{L}\\{3e^{2t} - 4\\sin(3t)\\} = 3\\mathcal{L}\\{e^{2t}\\} - 4\\mathcal{L}\\{\\s
  { op: "Read e^{2t} from table",
    eq: "\\mathcal{L}\\{e^{2t}\\} = \\frac{1}{s-2}",
    note: "Standard entry: □{e^{at}} = 1/(s−a), here a = 2." },
  { op: "Read sin(3t) from table",
    eq: "\\mathcal{L}\\{\\sin(3t)\\} = \\frac{3}{s^2+9}",
    note: "Standard entry: □{sin(□t)} = □/(s²+□²), here □ = 3." },
  { op: "Combine",
    eq: "\\frac{3}{s-2} - \\frac{12}{s^2+9}" }
])
```

---

**2.** Solve $y' + 3y = 6$, $y(0) = 1$.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Transform both sides",
    eq: "sY - 1 + 3Y = \\frac{6}{s}",
    note: "□{y'} = sY − y(0) = sY − 1. Right side: □{6} = 6/s." },
  { op: "Collect Y terms",
```

```
    eq: "(s+3)Y = \\frac{6}{s} + 1 = \\frac{s+6}{s}" },
  { op: "Solve for Y(s)",
    eq: "Y(s) = \\frac{s+6}{s(s+3)}" },
  { op: "Partial fractions: A/s + B/(s+3)",
    eq: "A = 2,\\quad B = -1 \\implies Y(s) = \\frac{2}{s} - \\frac{1}{s+3}",
    note: "Set s=0: 6=3A; set s=-3: 3=-3B." },
  { op: "Invert term by term",
    eq: "y(t) = 2 - e^{-3t}" }
])
```

---

**3.** Solve $y'' + 4y = \sin(2t)$, $y(0) = 0$, $y'(0) = 1$.

```
//| echo: false
makeStepperHTML(3, [
  { op: "Transform both sides",
    eq: "s^2 Y - s\\cdot 0 - 1 + 4Y = \\frac{2}{s^2+4}",
    note: "ℒ{y''} = s²Y − sy(0) − y'(0) = s²Y − 1." },
  { op: "Collect and solve for Y",
    eq: "(s^2+4)Y = \\frac{2}{s^2+4} + 1 \\implies Y(s) = \\frac{1}{s^2+4} + \\frac{2}{(s^2+4)^2}" }
  { op: "Invert first term",
    eq: "\\mathcal{L}^{-1}\\!\\left\\{\\frac{1}{s^2+4}\\right\\} = \\frac{1}{2}\\sin(2t)",
    note: "ℒ{sin(ωt)} = ω/(s²+ω²) with ω=2." },
  { op: "Invert second term",
    eq: "\\mathcal{L}^{-1}\\!\\left\\{\\frac{2}{(s^2+4)^2}\\right\\} = \\frac{1}{8}(\\sin 2t - 2t\\c
    note: "Using ℒ{sin(ωt) − ωt cos(ωt)} = 2ω³/(s²+ω²)² with ω=2; scale: 2/(s²+4)² = (1/16)·(2·8)/(s
  { op: "Combine",
    eq: "y(t) = \\tfrac{5}{8}\\sin(2t) - \\tfrac{1}{4}t\\cos(2t)" }
])
```

---

**4.** Write $f(t) = \begin{cases} 0 & t < 2 \\ t-2 & t \geq 2 \end{cases}$ in terms of $u(t-2)$ and find $\mathscr{L}\{f(t)\}$.

```
//| echo: false
makeStepperHTML(4, [
  { op: "Express using Heaviside",
    eq: "f(t) = (t-2)\\,u(t-2)",
    note: "The factor u(t−2) switches the function on at t=2." },
  { op: "Identify g(t)",
    eq: "f(t) = g(t-2)\\,u(t-2) \\text{ where } g(t) = t" },
  { op: "Apply second shifting theorem",
    eq: "\\mathcal{L}\\{g(t-2)\\,u(t-2)\\} = e^{-2s}\\mathcal{L}\\{g(t)\\} = e^{-2s}\\mathcal{L}\\{t
```

```
    note: "□{f(t−a)u(t−a)} = e^{−as}F(s)." },
  { op: "Read □{t} from table",
    eq: "\\mathcal{L}\\{t\\} = \\frac{1}{s^2}" },
  { op: "Final answer",
    eq: "\\mathcal{L}\\{f(t)\\} = \\frac{e^{-2s}}{s^2}" }
])
```

---

**5.** Solve $y'' + y = u(t - \pi)$, $y(0) = 0$, $y'(0) = 0$. The system is at rest; a step input arrives at $t = \pi$.

```
//| echo: false
makeStepperHTML(5, [
  { op: "Transform (zero ICs)",
    eq: "s^2 Y + Y = \\frac{e^{-\\pi s}}{s}",
    note: "Both initial conditions are zero, so no boundary terms." },
  { op: "Solve for Y(s)",
    eq: "Y(s) = \\frac{e^{-\\pi s}}{s(s^2+1)}" },
  { op: "Partial fractions on G(s) = 1/[s(s²+1)]",
    eq: "\\frac{1}{s(s^2+1)} = \\frac{1}{s} - \\frac{s}{s^2+1}",
    note: "Check: 1/(s·…) at s=0 gives A=1; equate s² gives B=−1." },
  { op: "Invert G(s)",
    eq: "g(t) = 1 - \\cos t" },
  { op: "Apply second shifting theorem",
    eq: "y(t) = [1 - \\cos(t-\\pi)]\\,u(t-\\pi)" },
  { op: "Simplify: cos(t−□) = −cos t",
    eq: "y(t) = [1 + \\cos t]\\,u(t-\\pi)",
    note: "Using cos(t−□) = −cos t, from the cosine addition formula. For t > □: y oscillates abo
])
```

---

**6.** A mechanical system receives a unit impulse at $t = 1$: $\ddot{x} + 2\dot{x} + 5x = \delta(t - 1)$, $x(0) = 0$, $\dot{x}(0) = 0$. Find $x(t)$ for $t > 1$.

```
//| echo: false
makeStepperHTML(6, [
  { op: "Transform: □{□(t−1)} = e^{−s}",
    eq: "(s^2 + 2s + 5)X = e^{-s}",
    note: "Zero initial conditions; impulse transform is e^{−as} with a=1." },
  { op: "Solve for X(s)",
    eq: "X(s) = \\frac{e^{-s}}{s^2+2s+5}" },
  { op: "Complete the square",
    eq: "s^2+2s+5 = (s+1)^2+4" },
  { op: "Invert G(s) = 1/[(s+1)²+4]",
```

```
    eq: "g(t) = \\frac{1}{2}e^{-t}\\sin(2t)",
    note: "Match to ⍰{e^{at}sin(⍰t)} = ⍰/[(s−a)²+⍰²] with a=−1, ⍰=2." },
  { op: "Apply second shifting theorem",
    eq: "x(t) = \\tfrac{1}{2}e^{-(t-1)}\\sin\\bigl(2(t-1)\\bigr)\\,u(t-1)",
    note: "For t > 1: damped sinusoid with natural frequency 2 rad/s, decay rate 1." }
])
```

# Part VIII.

# Volume 7 — Linear algebra and vector calculus

# 40. Matrices and linear systems

Organising equations so machines — and humans — can solve them

The structural analysis of the Tacoma Narrows Bridge involved linear systems of equations — the same fundamental form as any system of resistors, any truss, any set of chemical mass-balance equations. Thousands of numbers, hundreds of equations, but the same underlying question every time: does this system have a solution, is it unique, and what is it?

For small systems — two or three equations — you can solve by substitution, the way you learned in school. For larger systems, that approach bogs down in bookkeeping errors. The method in this chapter, Gaussian elimination, is systematic substitution: a sequence of operations that any careful person (or machine) can apply without creativity, and that always terminates with a definitive answer.

Before any of that, we need the language: matrices and the algebra of matrix operations. The notation is not decoration — it compresses a system of twenty equations into a single line, and makes the structure of the solution visible.

---

## 40.1. Matrices and operations

A **matrix** is a rectangular array of numbers. The matrix $\mathbf{A}$ with $m$ rows and $n$ columns is called an $m \times n$ matrix:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

The element in row $i$ and column $j$ is $a_{ij}$. The first subscript is always the row, the second is always the column. A matrix with one column ($n = 1$) is a **column vector**; one with one row ($m = 1$) is a **row vector**.

### 40.1.1. Special types

**Square matrix.** $m = n$. The elements $a_{11}, a_{22}, \dots, a_{nn}$ form the **main diagonal**.

**Symmetric matrix.** $a_{ij} = a_{ji}$ for all $i, j$. Equivalently, $\mathbf{A} = \mathbf{A}^T$. Symmetric matrices arise naturally: the stiffness matrix of a structure is symmetric, as is any correlation matrix in statistics.

**Diagonal matrix.** All off-diagonal entries are zero: $a_{ij} = 0$ whenever $i \neq j$. Multiplication by a diagonal matrix scales each component independently.

**Identity matrix $\mathbf{I}_n$.** Diagonal with all diagonal entries equal to 1. It plays the role of 1 in matrix arithmetic: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$.

**Triangular matrices.** A matrix is **upper triangular** if $a_{ij} = 0$ whenever $i > j$ — all entries below the main diagonal are zero. It is **lower triangular** if $a_{ij} = 0$ whenever $i < j$. Gaussian elimination produces an upper triangular matrix; this is why triangular forms matter.

**Transpose.** The transpose $\mathbf{A}^T$ swaps rows and columns: $(\mathbf{A}^T)_{ij} = a_{ji}$. A $3 \times 2$ matrix transposes to a $2 \times 3$ matrix. Key identities: $(\mathbf{A}^T)^T = \mathbf{A}$; $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$ (order reverses).

### 40.1.2. Matrix addition and scalar multiplication

Two matrices of the same size can be added element by element:

$$(\mathbf{A} + \mathbf{B})_{ij} = a_{ij} + b_{ij}$$

Scalar multiplication scales every entry: $(\lambda\mathbf{A})_{ij} = \lambda a_{ij}$.

These operations satisfy the expected algebraic properties — commutativity, associativity, distributivity. Nothing surprising here.

### 40.1.3. Matrix multiplication

Multiplying two matrices is less obvious. The product $\mathbf{C} = \mathbf{AB}$ requires that the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$. If $\mathbf{A}$ is $m \times n$ and $\mathbf{B}$ is $n \times p$, then $\mathbf{C}$ is $m \times p$.

The $(i, j)$ entry of $\mathbf{C}$ is the dot product of row $i$ of $\mathbf{A}$ with column $j$ of $\mathbf{B}$:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

**Worked example.** Let

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} 1 & 4 \\ 2 & 1 \end{pmatrix}$$

Then $c_{11} = 2 \cdot 1 + 1 \cdot 2 = 4$; $c_{12} = 2 \cdot 4 + 1 \cdot 1 = 9$; $c_{21} = 0 \cdot 1 + 3 \cdot 2 = 6$; $c_{22} = 0 \cdot 4 + 3 \cdot 1 = 3$.

$$\mathbf{AB} = \begin{pmatrix} 4 & 9 \\ 6 & 3 \end{pmatrix}$$

**Important:** matrix multiplication is not commutative. $\mathbf{AB} \neq \mathbf{BA}$ in general — and sometimes $\mathbf{BA}$ is not even defined when $\mathbf{AB}$ is. This is a genuine departure from ordinary arithmetic, and it matters whenever you are rearranging matrix equations.

**What multiplication means geometrically.** Each column of $\mathbf{AB}$ is a linear combination of the columns of $\mathbf{A}$, with coefficients from the corresponding column of $\mathbf{B}$. This column-space interpretation is the key to understanding why the structure of $\mathbf{A}$ determines the solvability of $\mathbf{Ax} = \mathbf{b}$.

The explorer below turns the columns of a matrix into visible motions of the plane.

```
//| echo: false
{
  const LIMIT = 3.2;
  let A = { a11: 2, a12: 0.5, a21: 0.3, a22: 1 };

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  const PRESETS = {
    "Horizontal stretch": [2, 0.5, 0.3, 1],
    "Identity": [1, 0, 0, 1],
    "Rotation 45°": [0.7, -0.7, 0.7, 0.7],
    "Shear": [1, 1.2, 0, 1],
    "Reflection y-axis": [-1, 0, 0, 1],
    "Singular": [1, 2, 0.5, 1]
  };

  function matVec(m, v) {
    return [m.a11 * v[0] + m.a12 * v[1], m.a21 * v[0] + m.a22 * v[1]];
  }

  function det(m) {
    return m.a11 * m.a22 - m.a12 * m.a21;
  }

  function makeMapper() {
    const left = 32, top = 18, size = 280;
    return (x, y) => [left + size * (x + LIMIT) / (2 * LIMIT), top + size * (LIMIT - y) / (2 * LI
  }

  const rays = [[1,0],[0,1],[1,1],[1,-1],[-1,1]];
  const square = [[0,0],[1,0],[1,1],[0,1]];

  const container = document.createElement("div");
```

```
container.style.cssText = "max-width: 980px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.45rem 0.7rem; a
function sliderRow(labelText, key) {
  const label = document.createElement("label");
  label.textContent = labelText;
  const input = document.createElement("input");
  input.type = "range";
  input.min = "-3";
  input.max = "3";
  input.step = "0.1";
  input.value = String(A[key]);
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; min-width:3.3rem;";
  controls.append(label, input, out);
  return { input, out, key };
}
const rows = [
  sliderRow("a11", "a11"),
  sliderRow("a12", "a12"),
  sliderRow("a21", "a21"),
  sliderRow("a22", "a22")
];

const presetRow = document.createElement("div");
presetRow.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.65rem;";
const presetLabel = document.createElement("span");
presetLabel.textContent = "Preset";
const preset = document.createElement("select");
preset.style.cssText = "padding:0.35rem 0.5rem; border:1px solid #d1d5db; border-radius:8px; backg
Object.keys(PRESETS).forEach(name => {
  const opt = document.createElement("option");
  opt.value = name;
  opt.textContent = name;
  preset.appendChild(opt);
});
preset.value = "Horizontal stretch";
presetRow.append(presetLabel, preset);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding:0
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
```

```
  k.textContent = label;
  const v = document.createElement("div");
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const detBox = card("det(A)");
const col1Box = card("col 1");
const col2Box = card("col 2");

const panels = document.createElement("div");
panels.style.cssText = "display:grid; grid-template-columns:repeat(2,minmax(0,1fr)); gap:0.75re

function makePanel(title) {
  const panel = document.createElement("div");
  panel.style.cssText = "background:#fff; border:1px solid #e5e7eb; border-radius:10px; padding
  const h = document.createElement("div");
  h.style.cssText = "font-size:0.8rem; color:#6b7280; text-transform:uppercase; letter-spacing:
  h.textContent = title;
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 420 330");
  svg.setAttribute("width", "100%");
  svg.style.display = "block";
  panel.append(h, svg);
  panels.appendChild(panel);
  return svg;
}

const leftSvg = makePanel("Input plane");
const rightSvg = makePanel("Transformed plane");
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function drawAxes(svg, mapper) {
  for (let i = -3; i <= 3; i++) {
    const gx = svgEl("line");
    gx.setAttribute("x1", mapper(i, -3)[0]); gx.setAttribute("y1", mapper(i, -3)[1]);
    gx.setAttribute("x2", mapper(i, 3)[0]); gx.setAttribute("y2", mapper(i, 3)[1]);
    gx.setAttribute("stroke", "#f3f4f6");
    svg.appendChild(gx);
    const gy = svgEl("line");
    gy.setAttribute("x1", mapper(-3, i)[0]); gy.setAttribute("y1", mapper(-3, i)[1]);
    gy.setAttribute("x2", mapper(3, i)[0]); gy.setAttribute("y2", mapper(3, i)[1]);
    gy.setAttribute("stroke", "#f3f4f6");
    svg.appendChild(gy);
  }
```

```
  const xAxis = svgEl("line");
  xAxis.setAttribute("x1", mapper(-3, 0)[0]); xAxis.setAttribute("y1", mapper(-3, 0)[1]);
  xAxis.setAttribute("x2", mapper(3, 0)[0]); xAxis.setAttribute("y2", mapper(3, 0)[1]);
  xAxis.setAttribute("stroke", "#6b7280");
  xAxis.setAttribute("stroke-width", "1.5");
  svg.appendChild(xAxis);
  const yAxis = svgEl("line");
  yAxis.setAttribute("x1", mapper(0, -3)[0]); yAxis.setAttribute("y1", mapper(0, -3)[1]);
  yAxis.setAttribute("x2", mapper(0, 3)[0]); yAxis.setAttribute("y2", mapper(0, 3)[1]);
  yAxis.setAttribute("stroke", "#6b7280");
  yAxis.setAttribute("stroke-width", "1.5");
  svg.appendChild(yAxis);
}

function drawArrow(svg, mapper, from, to, color, width = 2.6, labelText = "") {
  const [x1, y1] = mapper(from[0], from[1]);
  const [x2, y2] = mapper(to[0], to[1]);
  const line = svgEl("line");
  line.setAttribute("x1", x1); line.setAttribute("y1", y1);
  line.setAttribute("x2", x2); line.setAttribute("y2", y2);
  line.setAttribute("stroke", color);
  line.setAttribute("stroke-width", width);
  line.setAttribute("stroke-linecap", "round");
  svg.appendChild(line);
  const dx = x2 - x1, dy = y2 - y1;
  const len = Math.hypot(dx, dy) || 1;
  const ux = dx / len, uy = dy / len;
  const hx = x2 - 10 * ux, hy = y2 - 10 * uy;
  const t1 = svgEl("line");
  t1.setAttribute("x1", x2); t1.setAttribute("y1", y2);
  t1.setAttribute("x2", hx - 5 * uy); t1.setAttribute("y2", hy + 5 * ux);
  t1.setAttribute("stroke", color); t1.setAttribute("stroke-width", width);
  svg.appendChild(t1);
  const t2 = svgEl("line");
  t2.setAttribute("x1", x2); t2.setAttribute("y1", y2);
  t2.setAttribute("x2", hx + 5 * uy); t2.setAttribute("y2", hy - 5 * ux);
  t2.setAttribute("stroke", color); t2.setAttribute("stroke-width", width);
  svg.appendChild(t2);
  if (labelText) {
    const text = svgEl("text");
    text.setAttribute("x", x2 + 8);
    text.setAttribute("y", y2 - 6);
    text.setAttribute("font-size", "12");
    text.setAttribute("fill", color);
    text.textContent = labelText;
    svg.appendChild(text);
  }
```

```
}

function drawPolygon(svg, mapper, pts, fill, stroke) {
  const poly = svgEl("polygon");
  poly.setAttribute("points", pts.map(([x, y]) => mapper(x, y).join(",")).join(" "));
  poly.setAttribute("fill", fill);
  poly.setAttribute("stroke", stroke);
  poly.setAttribute("stroke-width", "2");
  svg.appendChild(poly);
}

function redraw() {
  rows.forEach(r => {
    A[r.key] = parseFloat(r.input.value);
    r.out.textContent = A[r.key].toFixed(1);
  });
  while (leftSvg.firstChild) leftSvg.removeChild(leftSvg.firstChild);
  while (rightSvg.firstChild) rightSvg.removeChild(rightSvg.firstChild);
  const map = makeMapper();

  drawAxes(leftSvg, map);
  drawAxes(rightSvg, map);
  drawPolygon(leftSvg, map, square, "rgba(20,184,166,0.2)", "#0f766e");
  drawPolygon(rightSvg, map, square.map(v => matVec(A, v)), "rgba(124,58,237,0.18)", "#6d28d9")

  rays.forEach(v => {
    drawArrow(leftSvg, map, [0, 0], v, "#94a3b8", 1.8);
    drawArrow(rightSvg, map, [0, 0], matVec(A, v), "#94a3b8", 1.8);
  });

  drawArrow(leftSvg, map, [0, 0], [1, 0], "#f97316", 2.8, "e₁");
  drawArrow(leftSvg, map, [0, 0], [0, 1], "#3b82f6", 2.8, "e₂");
  drawArrow(rightSvg, map, [0, 0], [A.a11, A.a21], "#f97316", 3, "col 1");
  drawArrow(rightSvg, map, [0, 0], [A.a12, A.a22], "#3b82f6", 3, "col 2");

  square.forEach(v => {
    const dot1 = svgEl("circle");
    const [x1, y1] = map(v[0], v[1]);
    dot1.setAttribute("cx", x1); dot1.setAttribute("cy", y1); dot1.setAttribute("r", "3.5");
    dot1.setAttribute("fill", "#0f766e");
    leftSvg.appendChild(dot1);
    const dot2 = svgEl("circle");
    const tv = matVec(A, v);
    const [x2, y2] = map(tv[0], tv[1]);
    dot2.setAttribute("cx", x2); dot2.setAttribute("cy", y2); dot2.setAttribute("r", "3.5");
    dot2.setAttribute("fill", "#6d28d9");
    rightSvg.appendChild(dot2);
```

```
    });

    const d = det(A);
    detBox.textContent = d.toFixed(2);
    detBox.style.color = d > 0.05 ? "#166534" : "#b91c1c";
    col1Box.textContent = `(${A.a11.toFixed(1)}, ${A.a21.toFixed(1)})`;
    col2Box.textContent = `(${A.a12.toFixed(1)}, ${A.a22.toFixed(1)})`;

    if (Math.abs(d) < 0.15) {
      const warn = svgEl("text");
      warn.setAttribute("x", "210");
      warn.setAttribute("y", "308");
      warn.setAttribute("text-anchor", "middle");
      warn.setAttribute("font-size", "14");
      warn.setAttribute("font-weight", "700");
      warn.setAttribute("fill", "#b91c1c");
      warn.textContent = "singular — all vectors collapse to a line";
      rightSvg.appendChild(warn);
    }

    caption.textContent = d > 0
      ? "A positive determinant preserves orientation. The transformed unit square becomes a paralle
      : d < 0
        ? "A negative determinant flips orientation. The orange and blue column vectors still tell y
        : "When det(A) is zero, area collapses and the transformation is singular.";
  }

  preset.addEventListener("change", () => {
    const vals = PRESETS[preset.value];
    rows.forEach((r, i) => { r.input.value = String(vals[i]); });
    redraw();
  });
  rows.forEach(r => r.input.addEventListener("input", redraw));

  container.append(controls, presetRow, readout, panels, caption);
  redraw();
  return container;
}
```

---

## 40.2. Linear systems: Ax = b

A **system of $m$ linear equations in $n$ unknowns** has the form

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\
&\vdots \\
a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m
\end{aligned}
$$

In matrix notation this is $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}$ is the $m \times n$ **coefficient matrix**, $\mathbf{x}$ is the $n \times 1$ vector of unknowns, and $\mathbf{b}$ is the $m \times 1$ right-hand side vector.

The compactness of this notation is not cosmetic. Once you write a system as $\mathbf{Ax} = \mathbf{b}$, all the structure of the solution is encoded in $\mathbf{A}$ — its rank, its column space, its null space. You can reason about solvability without touching the specific values in $\mathbf{b}$.

### 40.2.1. The augmented matrix

The working object for solving a linear system is the **augmented matrix** $[\mathbf{A} \,|\, \mathbf{b}]$: the coefficient matrix with $\mathbf{b}$ appended as an extra column, separated by a vertical bar.

$$
[\mathbf{A} \,|\, \mathbf{b}] = \left(\begin{array}{ccc|c}
a_{11} & \cdots & a_{1n} & b_1 \\
\vdots & \ddots & \vdots & \vdots \\
a_{m1} & \cdots & a_{mn} & b_m
\end{array}\right)
$$

Working on the augmented matrix instead of the equations directly keeps things compact and avoids rewriting variable names at every step.

### 40.2.2. What a solution means geometrically

Each equation $a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n = b_i$ is a hyperplane in $\mathbb{R}^n$. Solving the system means finding the intersection of all $m$ hyperplanes.

For $n = 2$ (two unknowns): each equation is a line in the plane. Two lines either intersect at one point (unique solution), are parallel (no solution), or are the same line (infinitely many solutions).

For $n = 3$: each equation is a plane in three-dimensional space. Three planes can intersect at one point, at a line, at a plane, or not at all. Gaussian elimination finds which case applies and produces the solution in each case.

## 40.3. Gaussian elimination

**Row operations.** The three operations that can be applied to rows of an augmented matrix without changing the solution set are:

1. **Swap** two rows: $R_i \leftrightarrow R_j$
2. **Scale** a row by a nonzero constant: $R_i \leftarrow \lambda R_i$ ($\lambda \neq 0$)
3. **Add a multiple of one row to another**: $R_i \leftarrow R_i + \mu R_j$

These are called **elementary row operations**. They are reversible, so they transform the system into an equivalent one — same solution set, different representation.

**The algorithm.** Gaussian elimination applies row operations to reduce $[\mathbf{A} \,|\, \mathbf{b}]$ to **upper triangular** (also called **row echelon**) form, where all entries below the main diagonal are zero. Back-substitution then extracts the unknowns from bottom to top.

### 40.3.1. Worked example: 3×3 system

Solve:

$$\begin{aligned} 2x_1 + 4x_2 - 2x_3 &= 2 \\ x_1 + 3x_2 + x_3 &= 10 \\ -x_1 + 2x_2 + 3x_3 &= 8 \end{aligned}$$

**Step 1.** Write the augmented matrix:

$$\left(\begin{array}{rrr|r} 2 & 4 & -2 & 2 \\ 1 & 3 & 1 & 10 \\ -1 & 2 & 3 & 8 \end{array}\right)$$

**Step 2.** Use row 1 as the pivot row to eliminate $x_1$ from rows 2 and 3.

$R_2 \leftarrow R_2 - \frac{1}{2}R_1$:

$$\left(\begin{array}{rrr|r} 2 & 4 & -2 & 2 \\ 0 & 1 & 2 & 9 \\ -1 & 2 & 3 & 8 \end{array}\right)$$

$R_3 \leftarrow R_3 + \frac{1}{2}R_1$:

$$\left(\begin{array}{rrr|r} 2 & 4 & -2 & 2 \\ 0 & 1 & 2 & 9 \\ 0 & 4 & 2 & 9 \end{array}\right)$$

**Step 3.** Use row 2 as the pivot row to eliminate $x_2$ from row 3.

$R_3 \leftarrow R_3 - 4R_2$:

$$\left(\begin{array}{ccc|c} 2 & 4 & -2 & 2 \\ 0 & 1 & 2 & 9 \\ 0 & 0 & -6 & -27 \end{array}\right)$$

This is upper triangular form. The system now reads:

$$\begin{aligned} 2x_1 + 4x_2 - 2x_3 &= 2 \\ x_2 + 2x_3 &= 9 \\ -6x_3 &= -27 \end{aligned}$$

**Step 4. Back-substitution.**

From row 3: $x_3 = \frac{-27}{-6} = \frac{9}{2}$.

From row 2: $x_2 = 9 - 2 \cdot \frac{9}{2} = 9 - 9 = 0$.

From row 1: $2x_1 = 2 - 4(0) + 2(\frac{9}{2}) = 2 + 9 = 11$, so $x_1 = \frac{11}{2}$.

Solution: $x_1 = \frac{11}{2}$, $x_2 = 0$, $x_3 = \frac{9}{2}$.

### 40.3.2. The pivot

At each elimination step, the diagonal element used to eliminate the column below it is called the **pivot**. The pivot must be nonzero. If it is zero, swap the current row with any row below it that has a nonzero entry in that column (**partial pivoting**). If no such row exists, the matrix is singular — the system either has no solution or infinitely many.

In practice, numerical software always uses partial pivoting even when the pivot is technically nonzero, to avoid large round-off errors from dividing by very small numbers.

*Step through the elimination — the amber cell is the active pivot.*

```
//| echo: false
{
  const states = [
    {
      label: "Initial matrix — pivot x₁ = 2 (amber) will zero the column below it.",
      op: null,
      M: [[2,4,-2,2],[1,3,1,10],[-1,2,3,8]],
      pr: 0, pc: 0
    },
    {
      label: "x₁ cleared from row 2. Pivot still active — row 3 still needs clearing.",
      op: "R₂ ← R₂ − ½R₁    (m₂₁ = ½)",
      M: [[2,4,-2,2],[0,1,2,9],[-1,2,3,8]],
      pr: 0, pc: 0
    },
    {
```

```
    label: "Column 1 fully cleared. Pivot shifts to x₂ = 1 in row 2.",
    op: "R₃ ← R₃ + ½R₁    (m₃₁ = −½)",
    M: [[2,4,-2,2],[0,1,2,9],[0,4,2,9]],
    pr: 1, pc: 1
  },
  {
    label: "Upper triangular form reached. Back-substitution proceeds from the bottom.",
    op: "R₃ ← R₃ − 4R₂    (m₃₂ = 4)",
    M: [[2,4,-2,2],[0,1,2,9],[0,0,-6,-27]],
    pr: 2, pc: 2
  }
];

let step = 0;

const wrap = document.createElement("div");
wrap.style.cssText = "border:1px solid #e5e7eb;border-radius:8px;padding:1rem 1.25rem;margin:0.5re

const opEl = document.createElement("div");
opEl.style.cssText = "font-size:0.9em;font-family:monospace;color:#374151;min-height:1.5em;margin-

const matWrap = document.createElement("div");
matWrap.style.cssText = "margin-bottom:0.5rem;";

const lblEl = document.createElement("div");
lblEl.style.cssText = "font-size:0.82em;color:#6b7280;min-height:1.4em;margin-bottom:0.75rem;";

function renderMatrix(M, pr, pc) {
  const nc = M[0].length;
  let html = '<div style="display:inline-flex;align-items:stretch;">';
  html += '<div style="border-left:2px solid #374151;border-top:2px solid #374151;border-bottom:2p
  html += '<table style="border-collapse:collapse;font-size:1em;line-height:1.8;">';
  for (let r = 0; r < M.length; r++) {
    html += "<tr>";
    for (let c = 0; c < nc; c++) {
      const v = M[r][c];
      const disp = v < 0 ? "−" + Math.abs(v) : String(v);
      const augBorder = c === nc - 1 ? "border-left:2px solid #9ca3af;" : "";
      const pivBg = r === pr && c === pc ? "background:#fef3c7;" : "";
      const pivFw = r === pr && c === pc ? "font-weight:700;color:#92400e;" : "";
      html += `<td style="padding:0.15rem 0.65rem;text-align:right;min-width:1.8em;${augBorder}${p
    }
    html += "</tr>";
  }
  html += "</table>";
  html += '<div style="border-right:2px solid #374151;border-top:2px solid #374151;border-bottom:2
  html += '</div>';
```

```
    return html;
  }

  function draw() {
    const s = states[step];
    opEl.textContent = s.op || "";
    matWrap.innerHTML = renderMatrix(s.M, s.pr, s.pc);
    lblEl.textContent = s.label;
    nextBtn.disabled = step >= states.length - 1;
    nextBtn.style.opacity = step >= states.length - 1 ? "0.4" : "1";
    ctrEl.textContent = step === 0 ? "Click to begin" : `Step ${step} of ${states.length - 1}`;
    doneEl.style.display = step === states.length - 1 ? "block" : "none";
  }

  const ctrlRow = document.createElement("div");
  ctrlRow.style.cssText = "display:flex;gap:0.5rem;align-items:center;";

  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next operation →";
  nextBtn.style.cssText = "padding:0.35rem 0.85rem;border:1px solid #d1d5db;border-radius:4px;bac

  const rstBtn = document.createElement("button");
  rstBtn.textContent = "Reset";
  rstBtn.style.cssText = "padding:0.35rem 0.75rem;border:1px solid #e5e7eb;border-radius:4px;back

  const ctrEl = document.createElement("span");
  ctrEl.style.cssText = "font-size:0.82em;color:#9ca3af;margin-left:0.25rem;";

  const doneEl = document.createElement("div");
  doneEl.style.cssText = "margin-top:0.5rem;font-size:0.9em;color:#059669;font-weight:500;display
  doneEl.textContent = "Upper triangular form reached. Back-substitution proceeds from row 3 upwa

  nextBtn.onclick = () => { if (step < states.length - 1) { step++; draw(); } };
  rstBtn.onclick = () => { step = 0; draw(); };

  ctrlRow.append(nextBtn, rstBtn, ctrEl);
  wrap.append(opEl, matWrap, lblEl, ctrlRow, doneEl);
  draw();
  return wrap;
}
```

### 40.3.3. Gauss-Jordan elimination and RREF

**Gauss-Jordan elimination** continues beyond upper triangular form: it also eliminates entries *above* each pivot, reducing the matrix to **reduced row echelon form (RREF)**. In RREF every pivot is 1, and every other entry in the pivot column is 0. The unknowns can be read off directly — no back-substitution

needed.

For large systems Gaussian elimination with back-substitution is more efficient than full Gauss-Jordan. For small systems, and especially for finding matrix inverses, RREF is convenient.

**Continuing the worked example.** Starting from the upper triangular form and scaling:

$R_3 \leftarrow -\frac{1}{6}R_3$, $R_2 \leftarrow R_2$, $R_1 \leftarrow \frac{1}{2}R_1$:

$$\left(\begin{array}{ccc|c} 1 & 2 & -1 & 1 \\ 0 & 1 & 2 & 9 \\ 0 & 0 & 1 & 9/2 \end{array}\right)$$

$R_2 \leftarrow R_2 - 2R_3$:

$$\left(\begin{array}{ccc|c} 1 & 2 & -1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 9/2 \end{array}\right)$$

$R_1 \leftarrow R_1 + R_3$, then $R_1 \leftarrow R_1 - 2R_2$:

$$\left(\begin{array}{ccc|c} 1 & 0 & 0 & 11/2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 9/2 \end{array}\right)$$

The solution $x_1 = \frac{11}{2}$, $x_2 = 0$, $x_3 = \frac{9}{2}$ is visible directly in the right-hand column. Same answer, different path.

---

## 40.4. Solution types and rank

Not every system $\mathbf{Ax} = \mathbf{b}$ has a unique solution. Three outcomes are possible, and the row-reduced form reveals which one applies.

### 40.4.1. The three cases

**Case 1: unique solution.** Every column of $\mathbf{A}$ has a pivot. After row reduction there is one leading 1 per row and per column. The solution is determined completely.

**Case 2: no solution (inconsistent).** After row reduction, at least one row has the form $[0\ 0\ \cdots\ 0 \mid c]$ with $c \neq 0$. This represents the equation $0 = c$, which is impossible. Geometrically: the hyperplanes do not share a common point.

**Case 3: infinitely many solutions (underdetermined).** After row reduction, at least one column has no pivot — the corresponding variable is **free**, meaning it can take any value. Each free variable

introduces one dimension of solutions. Geometrically: the hyperplanes intersect along a line, a plane, or a higher-dimensional subspace.

*Move the sliders to see how the geometry changes. Try slope −0.50 for the parallel and coincident cases.*

```
//| echo: false
viewof geomSlope = Inputs.range([-1.5, 0.5], {
  value: 0.3,
  step: 0.05,
  label: "Slope of line 2 (m₂)"
})
```

```
//| echo: false
viewof geomIntercept = Inputs.range([-1, 5], {
  value: 3.5,
  step: 0.1,
  label: "y-intercept of line 2 (b₂)"
})
```

```
//| echo: false
{
  // Line 1 (fixed): x + 2y = 4  →  y = −0.5x + 2
  const m1 = -0.5, b1 = 2;
  const m2 = geomSlope, b2 = geomIntercept;

  const EPS_S = 0.03;   // slope tolerance (slider step is 0.05)
  const EPS_B = 0.06;   // intercept tolerance (slider step is 0.1)
  const sameSlope  = Math.abs(m2 - m1) < EPS_S;
  const coincident = sameSlope && Math.abs(b2 - b1) < EPS_B;
  const parallel   = sameSlope && !coincident;

  let caseLabel, caseColor;
  if (coincident) {
    caseLabel = "Infinitely many solutions — same line";
    caseColor = "#059669";
  } else if (parallel) {
    caseLabel = "No solution — parallel lines (0 = c, impossible)";
    caseColor = "#dc2626";
  } else {
    const xi = (b2 - b1) / (m1 - m2);
    const yi = m1 * xi + b1;
    caseLabel = `Unique solution — intersection at (${xi.toFixed(2)}, ${yi.toFixed(2)})`;
    caseColor = "#2563eb";
  }

  const W = 340, H = 280, PAD = 44;
  const xMin = -5, xMax = 5, yMin = -3, yMax = 6;
```

```
const svg = document.createElementNS("http://www.w3.org/2000/svg", "svg");
svg.setAttribute("width", W);
svg.setAttribute("height", H);
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.style.cssText = "display:block;font-family:inherit;border:1px solid #e5e7eb;border-radius:6px

const ns = "http://www.w3.org/2000/svg";
function mk(tag, attrs) {
  const e = document.createElementNS(ns, tag);
  if (attrs) for (const [k, v] of Object.entries(attrs)) e.setAttribute(k, v);
  return e;
}
function mkT(tag, attrs, text) { const e = mk(tag, attrs); e.textContent = text; return e; }

function spx(x) { return PAD + (x - xMin) / (xMax - xMin) * (W - 2 * PAD); }
function spy(y) { return H - PAD - (y - yMin) / (yMax - yMin) * (H - 2 * PAD); }

const clipId = "geom-clip";
const defs = mk("defs");
const cp = mk("clipPath", { id: clipId });
cp.appendChild(mk("rect", { x: PAD, y: PAD, width: W - 2*PAD, height: H - 2*PAD }));
defs.appendChild(cp);
svg.appendChild(defs);

svg.appendChild(mk("rect", { x: 0, y: 0, width: W, height: H, fill: "#f9fafb", rx: 6 }));
svg.appendChild(mk("rect", { x: PAD, y: PAD, width: W-2*PAD, height: H-2*PAD, fill: "#fff", stroke

// Grid
const gridG = mk("g", { "clip-path": `url(#${clipId})` });
for (let x = Math.ceil(xMin); x <= Math.floor(xMax); x++)
  gridG.appendChild(mk("line", { x1: spx(x), y1: PAD, x2: spx(x), y2: H-PAD, stroke: "#f3f4f6", "s
for (let y = Math.ceil(yMin); y <= Math.floor(yMax); y++)
  gridG.appendChild(mk("line", { x1: PAD, y1: spy(y), x2: W-PAD, y2: spy(y), stroke: "#f3f4f6", "s
svg.appendChild(gridG);

// Axes
const clip = { "clip-path": `url(#${clipId})` };
svg.appendChild(mk("line", { x1: PAD, y1: spy(0), x2: W-PAD, y2: spy(0), stroke: "#d1d5db", "strok
svg.appendChild(mk("line", { x1: spx(0), y1: PAD, x2: spx(0), y2: H-PAD, stroke: "#d1d5db", "strok
svg.appendChild(mkT("text", { x: W-PAD+6, y: spy(0)+4, fill: "#9ca3af", "font-size": 12 }, "x"));
svg.appendChild(mkT("text", { x: spx(0)+5, y: PAD-4, fill: "#9ca3af", "font-size": 12 }, "y"));

function addLine(m, b, color, dash, width, opacity) {
  svg.appendChild(mk("line", {
    x1: spx(xMin), y1: spy(m*xMin+b),
    x2: spx(xMax), y2: spy(m*xMax+b),
    stroke: color, "stroke-width": width,
```

```
      "stroke-dasharray": dash || "",
      opacity: opacity || 1,
      "clip-path": `url(#${clipId})`
    }));
  }

  // Line 1 — fixed, solid blue
  addLine(m1, b1, "#3b82f6", "", 2.5, 1);
  const lx1 = 2.5;
  svg.appendChild(mkT("text", { x: spx(lx1)+4, y: spy(m1*lx1+b1)-5, fill: "#3b82f6", "font-size":

  // Line 2 — controlled
  if (coincident) {
    addLine(m2, b2, "#059669", "8 4", 5, 0.55);
  } else {
    addLine(m2, b2, "#f59e0b", parallel ? "" : "7 4", 2.5, 1);
    // Label line 2 near its left end
    const lx2 = -3.5, ly2 = m2*lx2+b2;
    if (ly2 >= yMin && ly2 <= yMax)
      svg.appendChild(mkT("text", { x: spx(lx2), y: spy(ly2)-6, fill: "#f59e0b", "font-size": 11,
  }

  // Intersection dot
  if (!parallel && !coincident) {
    const xi = (b2 - b1) / (m1 - m2);
    const yi = m1 * xi + b1;
    if (xi >= xMin && xi <= xMax && yi >= yMin && yi <= yMax)
      svg.appendChild(mk("circle", { cx: spx(xi), cy: spy(yi), r: 5, fill: "#2563eb", stroke: "#f
  }

  const footer = document.createElement("div");
  footer.style.cssText = `font-size:0.88em;font-weight:600;color:${caseColor};margin-top:0.4rem;`
  footer.textContent = caseLabel;

  const container = document.createElement("div");
  container.style.marginTop = "0.5rem";
  container.append(svg, footer);
  return container;
}
```

### 40.4.2. Rank

The **rank** of a matrix $\mathbf{A}$, written $\operatorname{rank}(\mathbf{A})$, is the number of pivots in its row-reduced form — equivalently, the number of linearly independent rows (or columns).

For an $m \times n$ matrix: $\operatorname{rank}(\mathbf{A}) \leq \min(m, n)$.

**Solvability criterion.** The system $\mathbf{Ax} = \mathbf{b}$ has at least one solution if and only if $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A} \,|\, \mathbf{b}])$. When this holds:

- If $\text{rank}(\mathbf{A}) = n$ (all unknowns are pivot variables), the solution is unique.
- If $\text{rank}(\mathbf{A}) < n$, there are $n - \text{rank}(\mathbf{A})$ free variables and infinitely many solutions.

### 40.4.3. Rank-nullity theorem

For an $n$-column matrix $\mathbf{A}$:

$$\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$$

The **nullity** is the number of free variables — the dimension of the **null space** (the set of all solutions to $\mathbf{Ax} = \mathbf{0}$). This theorem says that the number of "determined" directions (rank) plus the number of "undetermined" directions (nullity) always adds up to the total number of unknowns. It is stated here for completeness; it becomes central in the chapter on eigenvalues.

### 40.4.4. What rank tells you before you compute

If you have a $3 \times 3$ system and you can see by inspection that one equation is a linear combination of the other two (for example, equation 3 = equation 1 + equation 2), then $\text{rank}(\mathbf{A}) \leq 2 < 3$ regardless of $\mathbf{b}$. The system is either inconsistent or underdetermined. Checking rank first can save significant computation.

---

Use the stepper to watch each elimination multiplier land in `L` while the same row operation builds `U`.

```
//| echo: false
{
  const steps = [
    {
      title: "Start",
      note: "Begin with A. No multipliers have been stored yet.",
      L: [["1","0","0"],["0","1","0"],["0","0","1"]],
      U: [["2","4","-2"],["1","3","1"],["-1","2","3"]]
    },
    {
      title: "Column 1 elimination",
      note: "m21 = 1/2 and m31 = -1/2 zero the entries below the first pivot.",
      L: [["1","0","0"],["1/2","1","0"],["-1/2","0","1"]],
      U: [["2","4","-2"],["0","1","2"],["0","4","2"]]
    },
    {
      title: "Column 2 elimination",
```

```
    note: "m32 = 4 clears the final subdiagonal entry.",
    L: [["1","0","0"],["1/2","1","0"],["-1/2","4","1"]],
    U: [["2","4","-2"],["0","1","2"],["0","0","-6"]]
  }
];

const container = document.createElement("div");
container.style.cssText = "max-width:780px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const actions = document.createElement("div");
actions.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.75rem;";
const prevBtn = document.createElement("button"); prevBtn.textContent = "Previous";
const nextBtn = document.createElement("button"); nextBtn.textContent = "Next step";
const stage = document.createElement("strong");
actions.append(prevBtn, nextBtn, stage);

const note = document.createElement("p");
note.style.cssText = "margin:0 0 0.6rem 0; color:#334155;";
const grid = document.createElement("div");
grid.style.cssText = "display:grid; grid-template-columns:repeat(3, auto); gap:1.2rem; align-it
const lTable = document.createElement("table");
const uTable = document.createElement("table");
const checkTable = document.createElement("table");
[lTable, uTable, checkTable].forEach(t => t.style.cssText = "border-collapse:collapse; font-siz

function setTable(table, title, rows, color) {
  table.replaceChildren();
  const cap = document.createElement("caption");
  cap.textContent = title;
  cap.style.cssText = `caption-side:top; text-align:left; font-weight:700; margin-bottom:0.25re
  table.appendChild(cap);
  rows.forEach(r => {
    const tr = document.createElement("tr");
    r.forEach(v => {
      const td = document.createElement("td");
      td.textContent = v;
      td.style.cssText = "border:1px solid #cbd5e1; padding:0.45rem 0.65rem; text-align:center;
      tr.appendChild(td);
    });
    table.appendChild(tr);
  });
}

let idx = 0;
function redraw() {
  const step = steps[idx];
  stage.textContent = step.title;
  note.textContent = step.note;
```

```
    setTable(lTable, "L", step.L, "#2563eb");
    setTable(uTable, "U", step.U, "#f97316");
    if (idx === steps.length - 1) {
      setTable(checkTable, "L·U matches A", [["2","4","-2"],["1","3","1"],["-1","2","3"]], "#16a34a"
    } else {
      setTable(checkTable, "Target A", [["2","4","-2"],["1","3","1"],["-1","2","3"]], "#0f172a");
    }
  }

  prevBtn.addEventListener("click", () => { idx = Math.max(0, idx - 1); redraw(); });
  nextBtn.addEventListener("click", () => { idx = Math.min(steps.length - 1, idx + 1); redraw(); })

  grid.append(lTable, uTable, checkTable);
  container.append(actions, note, grid);
  redraw();
  return container;
}
```

## 40.5. LU factorisation

Gaussian elimination solves one system $\mathbf{Ax} = \mathbf{b}$ at a time. In engineering, the same matrix $\mathbf{A}$ often appears with many different right-hand sides $\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_k$ — for example, a structural stiffness matrix under different load patterns. Re-running Gaussian elimination from scratch each time wastes the work already done.

**LU factorisation** stores the elimination steps. It factors $\mathbf{A}$ into:

$$\mathbf{A} = \mathbf{LU}$$

where $\mathbf{L}$ is a lower triangular matrix (with 1s on the diagonal) and $\mathbf{U}$ is the upper triangular matrix produced by elimination. The elimination multipliers — the values $m_{ij} = a_{ij}/a_{jj}$ used to zero out entries below each pivot — become the off-diagonal entries of $\mathbf{L}$.

### 40.5.1. Solving with LU

Given $\mathbf{A} = \mathbf{LU}$, solving $\mathbf{Ax} = \mathbf{b}$ becomes two triangular solves:

1. Solve $\mathbf{Ly} = \mathbf{b}$ for $\mathbf{y}$ by **forward substitution** (top to bottom, since $\mathbf{L}$ is lower triangular).
2. Solve $\mathbf{Ux} = \mathbf{y}$ for $\mathbf{x}$ by **back-substitution** (bottom to top, since $\mathbf{U}$ is upper triangular).

Each triangular solve costs $O(n^2)$ operations. The factorisation itself costs $O(n^3)$ — the same as Gaussian elimination — but it only needs to happen once per matrix. Subsequent solves for different $\mathbf{b}$ cost only $O(n^2)$ each.

### 40.5.2. Worked example: 3×3 LU factorisation

Factorise:

$$\mathbf{A} = \begin{pmatrix} 2 & 4 & -2 \\ 1 & 3 & 1 \\ -1 & 2 & 3 \end{pmatrix}$$

**Step 1.** Column 1. Use $a_{11} = 2$ as pivot.

Multipliers: $m_{21} = \frac{1}{2}$, $m_{31} = \frac{-1}{2}$.

Apply $R_2 \leftarrow R_2 - \frac{1}{2}R_1$ and $R_3 \leftarrow R_3 - (-\frac{1}{2})R_1 = R_3 + \frac{1}{2}R_1$:

$$\begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 2 \\ 0 & 4 & 2 \end{pmatrix}$$

**Step 2.** Column 2. Use $a_{22}^{(1)} = 1$ as pivot.

Multiplier: $m_{32} = \frac{4}{1} = 4$.

Apply $R_3 \leftarrow R_3 - 4R_2$:

$$\mathbf{U} = \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 2 \\ 0 & 0 & -6 \end{pmatrix}$$

**Assembling L.** The multipliers fill the lower triangle of **L**, with 1s on the diagonal:

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ 1/2 & 1 & 0 \\ -1/2 & 4 & 1 \end{pmatrix}$$

**Verification:** **LU** should recover **A**. Check the $(3, 2)$ entry: $(-\frac{1}{2})(4) + (4)(1) + (1)(0) = -2 + 4 = 2$ ☒.

### 40.5.3. When LU fails

LU factorisation in the form above requires that every pivot encountered during elimination is nonzero. When a zero pivot arises — even if the matrix is not singular — rows must be swapped before continuing. This leads to **PLU factorisation** (**PA** = **LU** where **P** is a permutation matrix recording the row swaps), which is what production numerical libraries always compute. MATLAB's `lu()` and NumPy's `scipy.linalg.lu()` both return the PLU form.

### 40.5.4. Numerical perspective

For $n > 4$ or 5, Gaussian elimination and LU factorisation are almost always done by software. Understanding the algorithm tells you:

- **When it fails**: a zero pivot signals a singular or near-singular matrix — the system is ill-conditioned and the solution, if it exists, may be numerically unreliable.
- **Why pivoting matters**: without partial pivoting, round-off errors compound in a predictable direction and can make the computed solution wildly wrong even when a true solution exists.
- **What to trust**: if NumPy's `linalg.solve()` raises a `LinAlgError: Singular matrix`, the coefficient matrix is rank-deficient. The fix is not a different solver — it is a reconsideration of the physical model.

---

## 40.6. Applications

### 40.6.1. Circuit mesh analysis: Kirchhoff's voltage law

Consider a three-mesh resistive circuit. Applying Kirchhoff's voltage law (KVL) around each mesh — the sum of voltage drops around any closed loop is zero — yields one linear equation per mesh in the mesh currents $I_1, I_2, I_3$.

For a specific circuit with resistances $R_1 = 2\,\Omega$, $R_2 = 4\,\Omega$, $R_3 = 3\,\Omega$, $R_4 = 6\,\Omega$, $R_5 = 1\,\Omega$ and voltage sources $V_1 = 12\,\mathrm{V}$, $V_2 = 6\,\mathrm{V}$, the KVL equations might take the form:

$$\begin{aligned}
(R_1 + R_2)I_1 - R_2 I_2 &= V_1 \\
-R_2 I_1 + (R_2 + R_3 + R_4)I_2 - R_4 I_3 &= 0 \\
-R_4 I_2 + (R_4 + R_5)I_3 &= -V_2
\end{aligned}$$

Substituting values:

$$\begin{aligned}
6I_1 - 4I_2 &= 12 \\
-4I_1 + 13I_2 - 6I_3 &= 0 \\
-6I_2 + 7I_3 &= -6
\end{aligned}$$

In matrix form $\mathbf{Ax} = \mathbf{b}$:

$$\begin{pmatrix} 6 & -4 & 0 \\ -4 & 13 & -6 \\ 0 & -6 & 7 \end{pmatrix} \begin{pmatrix} I_1 \\ I_2 \\ I_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 0 \\ -6 \end{pmatrix}$$

The coefficient matrix is symmetric — a consequence of reciprocity in passive resistive networks. This is always the case when you apply KVL systematically; it is not a coincidence to exploit, it is a theorem to rely on.

Applying Gaussian elimination yields (after row reduction): $I_1 = 3.2\,\text{A}$, $I_2 = 1.8\,\text{A}$, $I_3 = 0.686\,\text{A}$ (to three significant figures). Once the mesh currents are known, every voltage and power in the circuit is computable — the linear system is the complete solution to the circuit analysis problem.

**Scaling remark.** A practical circuit board may have 50 to 500 mesh loops. The matrix is still symmetric, but $50 \times 50$ by hand is not feasible. SPICE — the standard circuit simulator used by every electronics engineer — solves such systems using LU factorisation with partial pivoting, exactly the method in this chapter.

### 40.6.2. Structural force distribution

A statically determinate truss consists of members (bars) and joints (nodes). At each joint, equilibrium requires that the sum of forces in the horizontal direction is zero, and the sum in the vertical direction is zero. Each equilibrium equation is linear in the member forces. For a truss with $n$ joints and $m$ members plus three reaction forces, static determinacy gives $m + 3 = 2n$, and the full set of equilibrium equations is a square linear system.

For a simple three-member triangular truss with a vertical load $P$ at the apex:

$$\begin{pmatrix} \cos\theta_1 & \cos\theta_2 & 0 \\ \sin\theta_1 & \sin\theta_2 & 0 \\ 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} F_1 \\ F_2 \\ R \end{pmatrix} = \begin{pmatrix} 0 \\ P \\ 0 \end{pmatrix}$$

where $F_1, F_2$ are member forces, $R$ is a reaction force, and $\theta_1, \theta_2$ are member angles. Solving this gives the force in each member — positive values indicate tension, negative indicate compression. A member under too much compression will buckle; too much tension and it will yield. The linear system does not just find numbers: it tells the structural engineer which members to upsize.

---

## 40.7. Exercises

These exercises use real engineering and science contexts. The interesting part is setting up the augmented matrix correctly and identifying which solution case applies.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
```

```
function renderSteps() {
  stepsDiv.innerHTML = "";
  for (let i = 0; i < current; i++) {
    const s = steps[i];
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;
    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    row.appendChild(opCell);
    row.appendChild(eqCell);
    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }
    stepsDiv.appendChild(row);
  }
  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "Solution complete.";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

function updateButtons() {
  nextBtn.disabled = current === totalSteps;
  nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
  counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSteps
}

nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
```

```
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

---

### 40.7.1. Exercise 1: Node admittance matrix multiplication

A circuit has three nodes. The node admittance matrix $\mathbf{Y}$ and a voltage vector $\mathbf{V}$ are:

$$\mathbf{Y} = \begin{pmatrix} 5 & -2 & -1 \\ -2 & 6 & -3 \\ -1 & -3 & 4 \end{pmatrix}, \qquad \mathbf{V} = \begin{pmatrix} 10 \\ 5 \\ 2 \end{pmatrix} \text{ V}$$

Compute the current injection vector $\mathbf{I} = \mathbf{YV}$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write the product definition",
    eq: "\\mathbf{I} = \\mathbf{Y}\\mathbf{V} = \\begin{pmatrix} 5 & -2 & -1 \\\\ -2 & 6 & -3 \\\
    note: "Each entry of I is the dot product of one row of Y with V."
  },
  {
    op: "Compute I₁ (row 1 · V)",
    eq: "I_1 = 5(10) + (-2)(5) + (-1)(2) = 50 - 10 - 2 = 38 \\text{ A}"
  },
  {
    op: "Compute I₂ (row 2 · V)",
    eq: "I_2 = (-2)(10) + 6(5) + (-3)(2) = -20 + 30 - 6 = 4 \\text{ A}"
  },
  {
    op: "Compute I₃ (row 3 · V)",
    eq: "I_3 = (-1)(10) + (-3)(5) + 4(2) = -10 - 15 + 8 = -17 \\text{ A}"
  },
  {
    op: "Assemble the result",
    eq: "\\mathbf{I} = \\begin{pmatrix} 38 \\\\ 4 \\\\ -17 \\end{pmatrix} \\text{ A}",
    note: "Negative current injection means current is leaving that node."
```

```
  }
])
```

---

### 40.7.2. Exercise 2: Mesh current analysis by Gaussian elimination

Three mesh currents $I_1, I_2, I_3$ (in amperes) in a resistive network satisfy:

$$\begin{aligned} 8I_1 - 2I_2 - 0I_3 &= 16 \\ -2I_1 + 7I_2 - 3I_3 &= 0 \\ 0I_1 - 3I_2 + 5I_3 &= -6 \end{aligned}$$

Find $I_1, I_2, I_3$ by Gaussian elimination.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write augmented matrix",
    eq: "\\left(\\begin{array}{rrr|r} 8 & -2 & 0 & 16 \\\\ -2 & 7 & -3 & 0 \\\\ 0 & -3 & 5 & -6 \\en
  },
  {
    op: "Eliminate column 1: R₂ ← R₂ + (1/4)R₁",
    eq: "\\left(\\begin{array}{rrr|r} 8 & -2 & 0 & 16 \\\\ 0 & 6.5 & -3 & 4 \\\\ 0 & -3 & 5 & -6 \\e
    note: "Multiplier m₂₁ = −2/8 = −1/4; add (1/4)R₁ to R₂."
  },
  {
    op: "Eliminate column 2: R₃ ← R₃ + (3/6.5)R₂",
    eq: "\\left(\\begin{array}{rrr|r} 8 & -2 & 0 & 16 \\\\ 0 & 6.5 & -3 & 4 \\\\ 0 & 0 & 3.615 & -4
    note: "Multiplier m₃₂ = −3/6.5 = −6/13; subtracting that multiple zeros the (3,2) entry."
  },
  {
    op: "Back-substitute: solve for I₃",
    eq: "I_3 = \\frac{-4.154}{3.615} \\approx -1.149 \\text{ A}"
  },
  {
    op: "Back-substitute: solve for I₂",
    eq: "I_2 = \\frac{4 - (-3)(-1.149)}{6.5} = \\frac{4 - 3.447}{6.5} \\approx 0.085 \\text{ A}"
  },
  {
    op: "Back-substitute: solve for I₁",
    eq: "I_1 = \\frac{16 - (-2)(0.085)}{8} = \\frac{16.170}{8} \\approx 2.021 \\text{ A}",
    note: "Exact fractions: I₁ = 1063/526 ≈ 2.021 A; I₂ = 9/106 ≈ 0.085 A; I₃ = −61/53 ≈ −1.151 A."
  }
])
```

### 40.7.3. Exercise 3: RREF and solution type (underdetermined structural system)

A redundant structure gives three equilibrium equations in four unknown member forces $F_1, F_2, F_3, F_4$:

$$
\begin{aligned}
F_1 + 2F_2 - F_3 + F_4 &= 10 \\
2F_1 + 4F_2 + F_3 - F_4 &= 8 \\
F_1 + 2F_2 + 3F_3 - 3F_4 &= -2
\end{aligned}
$$

Row-reduce to RREF. Identify the free variable(s) and write the general solution.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write augmented matrix",
    eq: "\\left(\\begin{array}{rrrr|r} 1 & 2 & -1 & 1 & 10 \\\\ 2 & 4 & 1 & -1 & 8 \\\\ 1 & 2 & 3
  },
  {
    op: "Eliminate col 1: R₂ ← R₂ − 2R₁ ; R₃ ← R₃ − R₁",
    eq: "\\left(\\begin{array}{rrrr|r} 1 & 2 & -1 & 1 & 10 \\\\ 0 & 0 & 3 & -3 & -12 \\\\ 0 & 0 &
  },
  {
    op: "Scale R₂ by 1/3; eliminate col 3: R₃ ← R₃ − 4R₂",
    eq: "\\left(\\begin{array}{rrrr|r} 1 & 2 & -1 & 1 & 10 \\\\ 0 & 0 & 1 & -1 & -4 \\\\ 0 & 0 &
    note: "Row 3 becomes all zeros — a dependent equation."
  },
  {
    op: "Back-eliminate col 3 from row 1: R₁ ← R₁ + R₂",
    eq: "\\left(\\begin{array}{rrrr|r} 1 & 2 & 0 & 0 & 6 \\\\ 0 & 0 & 1 & -1 & -4 \\\\ 0 & 0 & 0
  },
  {
    op: "Identify pivot and free variables",
    eq: "\\text{Pivots in cols 1, 3} \\implies F_1,\\, F_3 \\text{ are pivot variables; } F_2,\\,
    note: "rank(A) = 2 < 4 unknowns, so nullity = 2: a two-parameter family of solutions."
  },
  {
    op: "Write general solution (let F₂ = s, F₄ = t)",
    eq: "F_1 = 6 - 2s,\\; F_2 = s,\\; F_3 = -4 + t,\\; F_4 = t \\quad (s,\\,t \\in \\mathbb{R})",
    note: "Physical interpretation: the structure is statically indeterminate — you need material
  }
])
```

### 40.7.4. Exercise 4: Rank of a sensor measurement matrix

A sensor network takes four measurements of three physical quantities (temperature, pressure, flow rate). The measurement matrix is:

$$\mathbf{M} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 0 & 2 \\ 3 & 2 & 7 \end{pmatrix}$$

Find rank($\mathbf{M}$). What does this tell you about the sensor network?

```
//| echo: false
makeStepperHTML([
  {
    op: "Write the matrix and note its size",
    eq: "\\mathbf{M} \\text{ is } 4 \\times 3, \\text{ so } \\text{rank}(\\mathbf{M}) \\leq 3"
  },
  {
    op: "Eliminate col 1: R₂ ← R₂ − 2R₁ ; R₃ ← R₃ − R₁ ; R₄ ← R₄ − 3R₁",
    eq: "\\begin{pmatrix} 1 & 2 & 3 \\\\ 0 & 0 & 0 \\\\ 0 & -2 & -1 \\\\ 0 & -4 & -2 \\end{pmatrix}
    note: "Row 2 is exactly 2 × Row 1 — it carries no new information."
  },
  {
    op: "Swap R₂ and R₃ (bring nonzero pivot to position (2,2))",
    eq: "\\begin{pmatrix} 1 & 2 & 3 \\\\ 0 & -2 & -1 \\\\ 0 & 0 & 0 \\\\ 0 & -4 & -2 \\end{pmatrix}
  },
  {
    op: "Eliminate col 2 from R₄: R₄ ← R₄ − 2R₂",
    eq: "\\begin{pmatrix} 1 & 2 & 3 \\\\ 0 & -2 & -1 \\\\ 0 & 0 & 0 \\\\ 0 & 0 & 0 \\end{pmatrix}",
    note: "Two zero rows — rank equals the number of nonzero rows after reduction."
  },
  {
    op: "Count pivots",
    eq: "\\text{rank}(\\mathbf{M}) = 2",
    note: "Only 2 of the 4 sensors provide independent information. The other two are redundant — th
  }
])
```

---

### 40.7.5. Exercise 5: LU factorisation for repeated solves

A heat-transfer network has conductance matrix:

$$\mathbf{K} = \begin{pmatrix} 4 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{pmatrix}$$

Find the LU factorisation of $\mathbf{K}$, then use it to solve $\mathbf{KT} = \mathbf{q}$ for two different heat-flux vectors:

$$\mathbf{q}_1 = \begin{pmatrix} 8 \\ 4 \\ 2 \end{pmatrix}, \qquad \mathbf{q}_2 = \begin{pmatrix} 0 \\ 6 \\ 4 \end{pmatrix}$$

```
//| echo: false
makeStepperHTML([
  {
    op: "Elimination step 1: R₂ ← R₂ − (−1/4)R₁",
    eq: "m_{21} = \\tfrac{-1}{4} \\implies R_2 \\leftarrow R_2 - \\left(-\\tfrac{1}{4}\\right)R_1
    note: "Subtracting (−1/4)×R₁ is the same as adding (1/4)×R₁."
  },
  {
    op: "Result after step 1",
    eq: "\\mathbf{U}^{(1)} = \\begin{pmatrix} 4 & -1 & 0 \\\\ 0 & 11/4 & -1 \\\\ 0 & -1 & 2 \\end
  },
  {
    op: "Elimination step 2: R₃ ← R₃ − (−1/(11/4))R₂ = R₃ + (4/11)R₂",
    eq: "m_{32} = \\tfrac{-1}{11/4} = -\\tfrac{4}{11}"
  },
  {
    op: "Upper triangular U",
    eq: "\\mathbf{U} = \\begin{pmatrix} 4 & -1 & 0 \\\\ 0 & 11/4 & -1 \\\\ 0 & 0 & 18/11 \\end{pm
  },
  {
    op: "Assemble L from multipliers",
    eq: "\\mathbf{L} = \\begin{pmatrix} 1 & 0 & 0 \\\\ -1/4 & 1 & 0 \\\\ 0 & -4/11 & 1 \\end{pmat
  },
  {
    op: "Solve Ly = q₁ by forward substitution",
    eq: "\\mathbf{y}_1: \\; y_1 = 8,\\; y_2 = 4 + \\tfrac{1}{4}(8) = 6,\\; y_3 = 2 + \\tfrac{4}{1
  },
  {
    op: "Solve UT = y₁ by back-substitution",
    eq: "T_3 = \\frac{46/11}{18/11} = \\frac{46}{18} = \\frac{23}{9} \\approx 2.56,\\; T_2 = \\fr
    note: "Exact: T₁ = 167/60, T₂ = 56/18 = 28/9, T₃ = 23/9."
  },
  {
    op: "For q₂: reuse L and U, only redo the two triangular solves",
    eq: "\\mathbf{y}_2: \\; y_1 = 0,\\; y_2 = 6,\\; y_3 = 4 + \\tfrac{4}{11}(6) = \\tfrac{68}{11}
    note: "The LU factorisation cost is paid once. Each new right-hand side costs only O(n²)."
```

```
  }
])
```

---

### 40.7.6. Exercise 6: Write and solve the linear system for a 3-mesh circuit

A three-mesh circuit has the following KVL equations after applying Kirchhoff's voltage law. The mesh currents $I_1, I_2, I_3$ (in mA) satisfy:

- **Mesh 1:** A 10 Ω resistor carries $I_1$; a 5 Ω resistor is shared with mesh 2 carrying $I_1 - I_2$. Voltage source: 20 V.
- **Mesh 2:** The shared 5 Ω resistor carries $I_2 - I_1$; a 10 Ω resistor carries $I_2$; a 4 Ω resistor is shared with mesh 3 carrying $I_2 - I_3$. No voltage source.
- **Mesh 3:** The shared 4 Ω resistor carries $I_3 - I_2$; a 6 Ω resistor carries $I_3$. Voltage source: −8 V (opposing reference direction).

Write $\mathbf{Ax} = \mathbf{b}$ explicitly, then solve using Gaussian elimination.

```
//| echo: false
makeStepperHTML([
  {
    op: "Apply KVL to mesh 1",
    eq: "10I_1 + 5(I_1 - I_2) = 20 \\implies 15I_1 - 5I_2 = 20"
  },
  {
    op: "Apply KVL to mesh 2",
    eq: "5(I_2 - I_1) + 10I_2 + 4(I_2 - I_3) = 0 \\implies -5I_1 + 19I_2 - 4I_3 = 0"
  },
  {
    op: "Apply KVL to mesh 3",
    eq: "4(I_3 - I_2) + 6I_3 = -8 \\implies -4I_2 + 10I_3 = -8"
  },
  {
    op: "Write in matrix form Ax = b",
    eq: "\\begin{pmatrix} 15 & -5 & 0 \\\\ -5 & 19 & -4 \\\\ 0 & -4 & 10 \\end{pmatrix}\\begin{pmat
    note: "The coefficient matrix is symmetric — always the case for KVL mesh analysis with passive
  },
  {
    op: "Eliminate col 1: R₂ ← R₂ + (1/3)R₁",
    eq: "\\left(\\begin{array}{rrr|r} 15 & -5 & 0 & 20 \\\\ 0 & 52/3 & -4 & 20/3 \\\\ 0 & -4 & 10 &
    note: "Multiplier: m₂₁ = −5/15 = −1/3; subtract (−1/3)R₁, i.e., add (1/3)R₁."
  },
  {
    op: "Eliminate col 2: R₃ ← R₃ + (12/52)R₂",
    eq: "\\left(\\begin{array}{rrr|r} 15 & -5 & 0 & 20 \\\\ 0 & 52/3 & -4 & 20/3 \\\\ 0 & 0 & 442/52
    note: "Multiplier: m₃₂ = −4 / (52/3) = −12/52 = −3/13."
```

```
  },
  {
    op: "Back-substitute",
    eq: "I_3 = \\frac{-31/13}{442/52} \\approx -0.280 \\text{ mA}, \\quad I_2 \\approx 0.452 \\te
    note: "Verify: 15(1.484) − 5(0.452) = 22.26 − 2.26 = 20 ✓. All three mesh currents are determ
  }
])
```

# 41. Eigenvalues and eigenvectors

The natural modes of a system

Spin a shaft slowly and nothing unusual happens. Spin it faster and, at one particular speed, the whole machine shakes itself apart. That speed is not bad luck — it is the shaft's natural frequency, a number written into the geometry and stiffness of the metal itself. Engineers calculate it before the shaft is ever machined.

The calculation is an eigenvalue problem. Feed in the stiffness matrix, solve for the special values that make a certain determinant zero, and out come the critical speeds. The same computation — different matrix, same method — tells Google which web pages matter, tells a structural engineer where a building will sway, and tells a data scientist which direction in a dataset carries the most information.

## 41.1. The eigenvalue problem

Multiply a vector by a matrix and you generally get a new vector pointing in a completely different direction. But for certain special vectors, the matrix acts more simply — it only scales the vector, leaving its direction unchanged (or exactly reversed). These are **eigenvectors**.

Formally: $\mathbf{x}$ is an eigenvector of $A$ with eigenvalue $\lambda$ if

$$A\mathbf{x} = \lambda\mathbf{x}, \quad \mathbf{x} \neq \mathbf{0}$$

Read this as: "applying $A$ to $\mathbf{x}$ is the same as stretching $\mathbf{x}$ by a factor of $\lambda$." The vector $\mathbf{x}$ points in a direction that $A$ cannot rotate — it can only scale it.

The eigenvalue $\lambda$ can be:

- **Positive** — the eigenvector points the same way, scaled by $\lambda$.
- **Negative** — the eigenvector is flipped and scaled.
- **Zero** — $A\mathbf{x} = \mathbf{0}$; the eigenvector is in the null space of $A$ (and $A$ is singular).
- **Complex** — the matrix introduces a rotation; eigenvalues come in conjugate pairs for real symmetric matrices this cannot happen, but for general real matrices it can.

**Geometric picture.** Imagine $A$ as a transformation of the plane. Most vectors get rotated and stretched into new directions. The eigenvectors are the "fixed axes" — directions that survive the transformation pointing the same way they started (or exactly opposite). The eigenvalue tells you how much the transformation stretches along each fixed axis.

```
//| echo: false
eigenVisualiser = {
  const d3 = await require("d3@7");
  const W = 560, H = 400;
  const ML = 40, MR = 40, MT = 40, MB = 40;
  const innerW = W - ML - MR;
  const innerH = H - MT - MB;

  const container = d3.create("div").style("max-width", W+"px").style("font-family", "sans-serif").s

  const controls = container.append("div").style("display", "grid").style("grid-template-columns", "

  function makeSlider(parent, label, initVal) {
    const wrap = parent.append("label").style("display", "flex").style("align-items", "center").styl
    wrap.append("span").style("width", "20px").text(label);
    const inp = wrap.append("input").attr("type", "range").attr("min", -3).attr("max", 3).attr("step
    const val = wrap.append("span").style("width", "35px").style("text-align", "right").text(initVal
    return {inp, val};
  }

  const sliderA = makeSlider(controls, "a", 2);
  const sliderB = makeSlider(controls, "b", 1);
  const sliderC = makeSlider(controls, "c", 1);
  const sliderD = makeSlider(controls, "d", 2);

  const svg = container.append("svg").attr("width", W).attr("height", H).attr("viewBox", `0 0 ${W} S
  const g = svg.append("g").attr("transform", `translate(${W/2},${H/2})`);

  const xScale = d3.scaleLinear([-6, 6], [-innerW/2, innerW/2]);
  const yScale = d3.scaleLinear([-6, 6], [innerH/2, -innerH/2]);

  g.append("line").attr("x1", xScale(-6)).attr("y1", yScale(0)).attr("x2", xScale(6)).attr("y2", ySc
  g.append("line").attr("x1", xScale(0)).attr("y1", yScale(-6)).attr("x2", xScale(0)).attr("y2", ySc

  const unitPts = d3.range(0, 2 * Math.PI + 0.1, 0.1).map(theta => [Math.cos(theta), Math.sin(theta)
  const lineGen = d3.line().x(d => xScale(d[0])).y(d => yScale(d[1]));

  g.append("path").datum(unitPts).attr("d", lineGen).attr("fill", "none").attr("stroke", "#9ca3af")

  const transformedPath = g.append("path").attr("fill", "rgba(59, 130, 246, 0.1)").attr("stroke", "#

  const vecGroup = g.append("g");
  const e1Line = vecGroup.append("line").attr("stroke", "#f97316").attr("stroke-width", 2.5);
  const e1Head = vecGroup.append("polygon").attr("fill", "#f97316");
  const e1Label = vecGroup.append("text").attr("fill", "#c2410c").attr("font-size", "13px").attr("fo

  const e2Line = vecGroup.append("line").attr("stroke", "#10b981").attr("stroke-width", 2.5);
```

```
const e2Head = vecGroup.append("polygon").attr("fill", "#10b981");
const e2Label = vecGroup.append("text").attr("fill", "#047857").attr("font-size", "13px").attr(

const statusText = container.append("div").style("text-align", "center").style("font-size", "0.

function update() {
  const a = +sliderA.inp.property("value");
  const b = +sliderB.inp.property("value");
  const c = +sliderC.inp.property("value");
  const d = +sliderD.inp.property("value");

  sliderA.val.text(a.toFixed(1));
  sliderB.val.text(b.toFixed(1));
  sliderC.val.text(c.toFixed(1));
  sliderD.val.text(d.toFixed(1));

  const transPts = unitPts.map(pt => [a*pt[0] + b*pt[1], c*pt[0] + d*pt[1]]);
  transformedPath.datum(transPts).attr("d", lineGen);

  const tr = a + d;
  const det = a*d - b*c;
  const disc = tr*tr - 4*det;

  if (disc >= 0) {
    const lam1 = (tr + Math.sqrt(disc)) / 2;
    const lam2 = (tr - Math.sqrt(disc)) / 2;

    statusText.text(`Real Eigenvalues: \u03BB\u2081 = ${lam1.toFixed(2)}, \u03BB\u2082 = ${lam2

    function getEigVec(lam) {
      if (Math.abs(b) > 1e-6) return [b, lam - a];
      if (Math.abs(c) > 1e-6) return [lam - d, c];
      return lam === a ? [1, 0] : [0, 1];
    }

    function drawVec(v, lam, line, head, labelTxt, labelStr) {
      const mag = Math.sqrt(v[0]*v[0] + v[1]*v[1]) || 1;
      const norm = [v[0]/mag, v[1]/mag];
      const endx = norm[0] * lam;
      const endy = norm[1] * lam;

      const sx = xScale(endx);
      const sy = yScale(endy);
      const ox = xScale(0);
      const oy = yScale(0);

      line.attr("x1", ox).attr("y1", oy).attr("x2", sx).attr("y2", sy).attr("visibility", "visi
```

```
        if (Math.abs(lam) > 0.1) {
          const angle = Math.atan2(sy - oy, sx - ox);
          const r = 8;
          head.attr("points", `${sx},${sy} ${sx - r*Math.cos(angle - 0.5)},${sy - r*Math.sin(angle -
        } else {
          head.attr("visibility", "hidden");
        }

        labelTxt.attr("x", sx + 12*Math.sign(sx-ox)).attr("y", sy + 12*Math.sign(sy-oy)-5).text(labe
      }

      drawVec(getEigVec(lam1), lam1, e1Line, e1Head, e1Label, `\u03BB\u2081=${lam1.toFixed(2)}`);
      drawVec(getEigVec(lam2), lam2, e2Line, e2Head, e2Label, `\u03BB\u2082=${lam2.toFixed(2)}`);

    } else {
      statusText.text("Complex Eigenvalues (pure rotation/scaling without invariant axes)").style("c
      e1Line.attr("visibility", "hidden"); e1Head.attr("visibility", "hidden"); e1Label.attr("visibi
      e2Line.attr("visibility", "hidden"); e2Head.attr("visibility", "hidden"); e2Label.attr("visibi
    }
  }

  update();
  sliderA.inp.on("input", update);
  sliderB.inp.on("input", update);
  sliderC.inp.on("input", update);
  sliderD.inp.on("input", update);

  return container.node();
}
```

**Why $\mathbf{x} \neq \mathbf{0}$ is required**
The zero vector satisfies $A\mathbf{0} = \lambda\mathbf{0}$ for any $\lambda$. That is trivially true but useless — it gives no information about $A$. We demand a non-zero eigenvector.

## 41.2. Characteristic equation

Rearrange $A\mathbf{x} = \lambda\mathbf{x}$:

$$A\mathbf{x} - \lambda\mathbf{x} = \mathbf{0}$$

$$(A - \lambda I)\mathbf{x} = \mathbf{0}$$

This is a homogeneous linear system. A homogeneous system has a non-zero solution if and only if its coefficient matrix is **singular** — that is, its determinant is zero:

$$\det(A - \lambda I) = 0$$

This is the **characteristic equation**. Expanding the determinant gives a polynomial in $\lambda$ — the **characteristic polynomial**. Its roots are the eigenvalues of $A$.

For an $n \times n$ matrix, the characteristic polynomial has degree $n$, so there are exactly $n$ eigenvalues (counting multiplicity, and allowing complex values).

**For a 2×2 matrix:**

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$\det(A - \lambda I) = \det \begin{pmatrix} a - \lambda & b \\ c & d - \lambda \end{pmatrix} = (a - \lambda)(d - \lambda) - bc$$

Expanding:

$$\lambda^2 - (a + d)\lambda + (ad - bc) = 0$$

$$\lambda^2 - \operatorname{tr}(A)\,\lambda + \det(A) = 0$$

The trace $\operatorname{tr}(A) = a + d$ (sum of diagonal entries) and $\det(A) = ad - bc$ appear directly in the characteristic polynomial. This is a useful check: the sum of the eigenvalues equals the trace, and their product equals the determinant.

**Trace–determinant check**
After finding eigenvalues $\lambda_1, \lambda_2$:

$$\lambda_1 + \lambda_2 = \operatorname{tr}(A), \qquad \lambda_1 \lambda_2 = \det(A)$$

Always verify this before proceeding. A quick arithmetic error in the characteristic polynomial shows up immediately.

## 41.3. Finding eigenvalues and eigenvectors — worked examples

### 41.3.1. Example 1: A 2×2 electrical network matrix

A simplified two-node circuit has the nodal admittance matrix

$$A = \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}$$

**Step 1: Characteristic equation.**

$$\det(A - \lambda I) = \det\begin{pmatrix} 3-\lambda & -1 \\ -1 & 3-\lambda \end{pmatrix} = (3-\lambda)^2 - 1 = 0$$

$$9 - 6\lambda + \lambda^2 - 1 = 0$$

$$\lambda^2 - 6\lambda + 8 = 0$$

**Step 2: Solve the quadratic.**

$$(\lambda - 2)(\lambda - 4) = 0 \implies \lambda_1 = 2, \quad \lambda_2 = 4$$

*Check:* $\lambda_1 + \lambda_2 = 6 = \operatorname{tr}(A)$ and $\lambda_1\lambda_2 = 8 = \det(A)$. Yes.

**Step 3: Eigenvector for $\lambda_1 = 2$.**

Solve $(A - 2I)\mathbf{x} = \mathbf{0}$:

$$\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

The two rows say the same thing: $x_1 - x_2 = 0$, so $x_1 = x_2$. One free variable — take $x_2 = 1$:

$$\mathbf{x}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

**Step 4: Eigenvector for $\lambda_2 = 4$.**

Solve $(A - 4I)\mathbf{x} = \mathbf{0}$:

$$\begin{pmatrix} -1 & -1 \\ -1 & -1 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

The equation is $-x_1 - x_2 = 0$, so $x_1 = -x_2$. Take $x_2 = 1$:

$$\mathbf{x}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

Notice the eigenvectors are perpendicular: $(1)(-1) + (1)(1) = 0$. This is not a coincidence — $A$ is symmetric. Symmetric matrices always have orthogonal eigenvectors (more on this in the spectral theorem section).

### 41.3.2. Example 2: A 3×3 two-storey building stiffness matrix

A two-storey shear building with a rigid base has three degrees of freedom (ground, first floor, second floor). With floor masses normalised to 1 and storey stiffnesses $k = 1$, the equation of motion $M\ddot{u}+Ku = 0$ (with $M = I$) reduces to an eigenvalue problem on the stiffness matrix:

$$K = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}$$

The eigenvalues of $K$ are the squares of the natural frequencies.

**Step 1: Characteristic polynomial.**

$$\det(K - \lambda I) = \det \begin{pmatrix} 2-\lambda & -1 & 0 \\ -1 & 2-\lambda & -1 \\ 0 & -1 & 1-\lambda \end{pmatrix}$$

Expanding along the first row:

$$= (2-\lambda) \det \begin{pmatrix} 2-\lambda & -1 \\ -1 & 1-\lambda \end{pmatrix} - (-1) \det \begin{pmatrix} -1 & -1 \\ 0 & 1-\lambda \end{pmatrix}$$

$$= (2-\lambda)\big[(2-\lambda)(1-\lambda) - 1\big] + \big[(-1)(1-\lambda) - 0\big]$$

$$= (2-\lambda)(2 - 3\lambda + \lambda^2 - 1) + (-(1-\lambda))$$

$$= (2-\lambda)(\lambda^2 - 3\lambda + 1) - (1-\lambda)$$

Expanding $(2-\lambda)(\lambda^2 - 3\lambda + 1)$:

$$= 2\lambda^2 - 6\lambda + 2 - \lambda^3 + 3\lambda^2 - \lambda$$

$$= -\lambda^3 + 5\lambda^2 - 7\lambda + 2$$

Subtracting $(1-\lambda)$:

$$\det(K - \lambda I) = -\lambda^3 + 5\lambda^2 - 6\lambda + 1$$

Or equivalently: $\lambda^3 - 5\lambda^2 + 6\lambda - 1 = 0$.

**Step 2: Roots.** Numerically (or via Cardano's formula): the three roots are approximately

$$\lambda_1 \approx 0.198, \quad \lambda_2 \approx 1.555, \quad \lambda_3 \approx 3.247$$

The natural frequencies are $\omega_i = \sqrt{\lambda_i}$:

$$\omega_1 \approx 0.445, \quad \omega_2 \approx 1.247, \quad \omega_3 \approx 1.802 \text{ rad/s}$$

**3×3 and larger: numerical methods in practice**
For $3 \times 3$ matrices, the characteristic polynomial can still be solved by hand — but it is tedious. For $4 \times 4$ and beyond, exact polynomial roots are impossible in general (Abel–Ruffini theorem). In practice, engineers use numerical algorithms — the QR algorithm, Jacobi iteration, or Lanczos methods. The point of working through the $3 \times 3$ case by hand is to understand what the algorithm is computing, not to match its speed.

## 41.4. Repeated eigenvalues

When the characteristic polynomial has a repeated root $\lambda$ of multiplicity $m > 1$, two cases arise:

**Case 1: Non-defective (enough eigenvectors).** The eigenspace for $\lambda$ has dimension $m$ — you can find $m$ linearly independent eigenvectors. The matrix is diagonalisable.

Example: $A = \lambda I$ (a scalar matrix). Every vector is an eigenvector. The eigenspace is the whole $\mathbb{R}^n$.

**Case 2: Defective (not enough eigenvectors).** The eigenspace for $\lambda$ has dimension less than $m$. You cannot find enough linearly independent eigenvectors to fill out a basis. The matrix is **not diagonalisable** — you need the Jordan normal form instead.

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$$

Characteristic polynomial: $(\lambda - 2)^2 = 0$, so $\lambda = 2$ with multiplicity 2.

Eigenvectors: $(A - 2I)\mathbf{x} = \mathbf{0}$ gives $\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}\mathbf{x} = \mathbf{0}$, so $x_2 = 0$ and $x_1$ is free. There is only one linearly independent eigenvector: $\mathbf{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$.

Algebraic multiplicity (from the polynomial) is 2. Geometric multiplicity (dimension of eigenspace) is 1. Because they differ, $A$ is **defective**.

**Why this matters in engineering**
A defective matrix arises when two natural frequencies of a system coincide exactly — a degenerate case. Small perturbations (real-world manufacturing tolerances) break the degeneracy, splitting the repeated eigenvalue into two distinct ones. Defective matrices are a theoretical boundary case. In practice, if your numerical computation returns a repeated eigenvalue, check whether it reflects genuine physics or accumulated rounding error.

## 41.5. Diagonalisation

If an $n \times n$ matrix $A$ has $n$ linearly independent eigenvectors $\mathbf{x}_1, \dots, \mathbf{x}_n$ with eigenvalues $\lambda_1, \dots, \lambda_n$, form the matrices:

$$P = \begin{pmatrix} | & & | \\ \mathbf{x}_1 & \cdots & \mathbf{x}_n \\ | & & | \end{pmatrix}, \qquad D = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix}$$

Then:

$$A = PDP^{-1}$$

$P$ is the **modal matrix** (columns are eigenvectors); $D$ is **diagonal** with eigenvalues on the diagonal.

**Why diagonalisation is useful.**

Powers of a diagonal matrix are trivial: $D^k$ is just each diagonal entry raised to the $k$-th power. So:

$$A^k = PD^kP^{-1}$$

This makes computing $A^{100}$ as cheap as computing $\lambda_1^{100}, \dots, \lambda_n^{100}$ — a dramatic saving. It also makes solving $\dot{\mathbf{u}} = A\mathbf{u}$ straightforward: substitute $\mathbf{u} = P\mathbf{v}$ to decouple the system into $n$ independent scalar ODEs, each of the form $\dot{v}_i = \lambda_i v_i$.

**Worked diagonalisation of Example 1.**

From Example 1: $\lambda_1 = 2$, $\mathbf{x}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$; $\lambda_2 = 4$, $\mathbf{x}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}$.

$$P = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \qquad D = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$$

$$P^{-1} = \frac{1}{\det P} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}$$

Verify $AP = PD$:

$$AP = \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & -4 \\ 2 & 4 \end{pmatrix}$$

$$PD = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix} = \begin{pmatrix} 2 & -4 \\ 2 & 4 \end{pmatrix}$$

## 41.6. Symmetric matrices and the spectral theorem

A matrix $A$ is **symmetric** if $A = A^T$ (the transpose equals itself). Symmetric matrices arise everywhere in physics and engineering: stiffness matrices, covariance matrices, the inertia tensor. They have two beautiful properties:

**Spectral theorem (stated)**
If $A$ is a real symmetric $n \times n$ matrix, then:

1. All eigenvalues of $A$ are **real**.
2. Eigenvectors corresponding to distinct eigenvalues are **orthogonal**.
3. $A$ has $n$ linearly independent eigenvectors, so it is always diagonalisable.
4. The eigenvectors can be chosen to be an **orthonormal basis** of $\mathbb{R}^n$.

This means $A = QDQ^T$ where $Q$ is an **orthogonal** matrix ($Q^T = Q^{-1}$) whose columns are orthonormal eigenvectors. This is the **eigendecomposition** or **spectral decomposition** of $A$.

**Why engineers care.** Stiffness matrices are symmetric (by Maxwell's reciprocal theorem). The spectral theorem guarantees that their eigenvalues (natural frequencies squared) are real and positive — physically, modes must vibrate at real frequencies, not grow exponentially. The orthogonality of mode shapes means each mode can be analysed independently, which is the foundation of modal analysis.

**Principal axes.** The eigendecomposition of a symmetric matrix gives the **principal axes** of the associated quadratic form. For a 3D stress tensor, the eigenvalues are the principal stresses and the eigenvectors are the principal stress directions — the coordinate system in which the stress tensor is diagonal (no shear stress). Every solid mechanics textbook derives the principal stress transformation this way.

## 41.7. Applications

### 41.7.1. Vibrational modes of a two-mass spring system

Two masses connected by three springs form the simplest mechanical model of a multi-storey building or a coupled oscillator. With masses $m_1 = 1$, $m_2 = 2$ and spring constants $k_1 = k_2 = k_3 = 1$:

The equations of motion are:

$$m_1\ddot{u}_1 = -k_1u_1 + k_2(u_2 - u_1) = -(k_1 + k_2)u_1 + k_2u_2$$
$$m_2\ddot{u}_2 = -k_2(u_2 - u_1) - k_3u_2 = k_2u_1 - (k_2 + k_3)u_2$$

In matrix form $M\ddot{\mathbf{u}} + K\mathbf{u} = \mathbf{0}$ with $M = \mathrm{diag}(1, 2)$:

$$K = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

For harmonic motion $\mathbf{u} = \mathbf{x}e^{i\omega t}$, this becomes the **generalised eigenvalue problem**:

$$K\mathbf{x} = \omega^2 M\mathbf{x}$$

Pre-multiply by $M^{-1} = \text{diag}(1, \frac{1}{2})$:

$$M^{-1}K\mathbf{x} = \omega^2\mathbf{x}, \qquad M^{-1}K = \begin{pmatrix} 2 & -1 \\ -\frac{1}{2} & 1 \end{pmatrix}$$

Characteristic equation: $\det(M^{-1}K - \omega^2 I) = 0$:

$$(2 - \omega^2)(1 - \omega^2) - \tfrac{1}{2} = 0$$

$$\omega^4 - 3\omega^2 + \tfrac{3}{2} = 0$$

Using the quadratic formula with $\mu = \omega^2$:

$$\mu = \frac{3 \pm \sqrt{9-6}}{2} = \frac{3 \pm \sqrt{3}}{2}$$

$$\omega_1^2 = \frac{3-\sqrt{3}}{2} \approx 0.634, \quad \omega_1 \approx 0.796 \text{ rad/s}$$

$$\omega_2^2 = \frac{3+\sqrt{3}}{2} \approx 2.366, \quad \omega_2 \approx 1.538 \text{ rad/s}$$

These are the two **natural frequencies**. At $\omega_1$, both masses oscillate in phase (the first mode shape). At $\omega_2$, they oscillate out of phase (the second mode shape). If you drive this system at either of these frequencies, resonance occurs — amplitudes grow without bound (in the undamped model).

### 41.7.2. Power iteration and PageRank

Google's PageRank assigns importance scores to web pages based on the link structure of the web. Define the transition matrix $G$ (the "Google matrix") where $G_{ij}$ is proportional to the probability of following a link from page $j$ to page $i$.

PageRank scores are the entries of the **dominant eigenvector** of $G$ — the eigenvector corresponding to the largest eigenvalue (which is always 1 for a properly normalised transition matrix).

Finding the exact dominant eigenvector of a billion-by-billion matrix is impossible by direct methods. Instead, Google uses **power iteration**:

1. Start with any vector $\mathbf{v}^{(0)}$ (e.g., uniform: all pages equally likely).
2. Multiply: $\mathbf{v}^{(k+1)} = G\mathbf{v}^{(k)}$.
3. Normalise: divide by $\|\mathbf{v}^{(k+1)}\|$.
4. Repeat until convergence.

Under mild conditions, this converges to the dominant eigenvector. The convergence rate depends on the ratio $|\lambda_2/\lambda_1|$ — how much smaller the second eigenvalue is than the first.

```
//| echo: false
powerIterationDemo = {
  const d3 = await require("d3@7");
  const W = 560, H = 400;
  const ML = 40, MR = 40, MT = 40, MB = 40;
  const innerW = W - ML - MR;
  const innerH = H - MT - MB;

  const container = d3.create("div").style("max-width", W+"px").style("font-family", "sans-serif").s

  const controls = container.append("div").style("display", "flex").style("gap", "20px").style("marg

  const matDiv = controls.append("div").style("display", "grid").style("grid-template-columns", "1fr
  function makeInput(parent, initVal) {
    return parent.append("input").attr("type", "number").attr("step", "0.1").attr("value", initVal)
  }
  const inpA = makeInput(matDiv, 2);
  const inpB = makeInput(matDiv, 1);
  const inpC = makeInput(matDiv, 1);
  const inpD = makeInput(matDiv, 3);

  const btnDiv = controls.append("div").style("display", "flex").style("gap", "10px");
  const stepBtn = btnDiv.append("button").text("Step").style("padding", "5px 12px").style("cursor",
  const resetBtn = btnDiv.append("button").text("Reset").style("padding", "5px 12px").style("cursor"

  const kText = controls.append("div").style("font-weight", "bold").style("font-size", "1.1rem").sty

  const svg = container.append("svg").attr("width", W).attr("height", H).attr("viewBox", `0 0 ${W} S
  const g = svg.append("g").attr("transform", `translate(${W/2},${H/2})`);

  const xScale = d3.scaleLinear([-1.5, 1.5], [-innerW/2, innerW/2]);
  const yScale = d3.scaleLinear([-1.5, 1.5], [innerH/2, -innerH/2]);

  g.append("line").attr("x1", xScale(-1.5)).attr("y1", yScale(0)).attr("x2", xScale(1.5)).attr("y2",
  g.append("line").attr("x1", xScale(0)).attr("y1", yScale(-1.5)).attr("x2", xScale(0)).attr("y2", y
  g.append("circle").attr("r", xScale(1) - xScale(0)).attr("fill", "none").attr("stroke", "#9ca3af")

  const tracePath = g.append("path").attr("fill", "none").attr("stroke", "#cbd5e1").attr("stroke-wid

  const targetLine = g.append("line").attr("stroke", "#9ca3af").attr("stroke-width", 2).attr("stroke
  const targetHead = g.append("polygon").attr("fill", "#9ca3af");
  const targetLabel = g.append("text").attr("fill", "#6b7280").attr("font-size", "12px").text(" Domi

  const iterLine = g.append("line").attr("stroke", "#3b82f6").attr("stroke-width", 3);
  const iterHead = g.append("polygon").attr("fill", "#3b82f6");
```

```
let x_k = [1, 0];
let k = 0;
let trace = [[1,0]];

function getDominantEigVec(a, b, c, d) {
  const tr = a + d;
  const det = a*d - b*c;
  const disc = tr*tr - 4*det;
  if (disc < 0) return null;
  const lam1 = (tr + Math.sqrt(disc)) / 2;
  const lam2 = (tr - Math.sqrt(disc)) / 2;
  const dom = Math.abs(lam1) >= Math.abs(lam2) ? lam1 : lam2;

  let v;
  if (Math.abs(b) > 1e-6) v = [b, dom - a];
  else if (Math.abs(c) > 1e-6) v = [dom - d, c];
  else v = dom === a ? [1, 0] : [0, 1];

  const mag = Math.sqrt(v[0]*v[0] + v[1]*v[1]);
  return [v[0]/mag, v[1]/mag];
}

function drawVec(v, line, head, scale = 1) {
  const sx = xScale(v[0]*scale);
  const sy = yScale(v[1]*scale);
  const ox = xScale(0);
  const oy = yScale(0);
  line.attr("x1", ox).attr("y1", oy).attr("x2", sx).attr("y2", sy);
  const angle = Math.atan2(sy - oy, sx - ox);
  const r = 10;
  head.attr("points", `${sx},${sy} ${sx - r*Math.cos(angle - 0.5)},${sy - r*Math.sin(angle - 0.
  return {sx, sy};
}

function update() {
  const a = +inpA.property("value");
  const b = +inpB.property("value");
  const c = +inpC.property("value");
  const d = +inpD.property("value");

  const domV = getDominantEigVec(a, b, c, d);

  if (domV) {
    targetLine.attr("visibility", "visible");
    targetHead.attr("visibility", "visible");
    targetLabel.attr("visibility", "visible");
```

```
      const {sx, sy} = drawVec(domV, targetLine, targetHead);
      targetLabel.attr("x", sx + 12*Math.sign(domV[0])).attr("y", sy + 15*Math.sign(domV[1]) - 5);

      // Also potentially draw the negative dominant direction since power iteration can flip
      if(domV[0]*x_k[0] + domV[1]*x_k[1] < 0) {
          drawVec([-domV[0], -domV[1]], targetLine, targetHead);
          targetLabel.attr("x", xScale(-domV[0]) + 12*Math.sign(-domV[0])).attr("y", yScale(-domV[1
      } else {
          drawVec(domV, targetLine, targetHead);
          targetLabel.attr("x", sx + 12*Math.sign(domV[0])).attr("y", sy + 15*Math.sign(domV[1]) - 5
      }
    } else {
      targetLine.attr("visibility", "hidden");
      targetHead.attr("visibility", "hidden");
      targetLabel.attr("visibility", "hidden");
    }

    drawVec(x_k, iterLine, iterHead);
    kText.text(`k = ${k}`);

    const lineGen = d3.line().x(d => xScale(d[0])).y(d => yScale(d[1]));
    tracePath.datum(trace).attr("d", lineGen);
  }

  stepBtn.on("click", () => {
    const a = +inpA.property("value");
    const b = +inpB.property("value");
    const c = +inpC.property("value");
    const d = +inpD.property("value");

    const newX = a*x_k[0] + b*x_k[1];
    const newY = c*x_k[0] + d*x_k[1];
    const mag = Math.sqrt(newX*newX + newY*newY);
    if(mag > 0) {
      x_k = [newX/mag, newY/mag];
      trace.push([...x_k]);
      k++;
      update();
    }
  });

  resetBtn.on("click", () => {
    x_k = [1, 0];
    trace = [[1,0]];
    k = 0;
    update();
  });
```

```
  inpA.on("change", () => resetBtn.node().click());
  inpB.on("change", () => resetBtn.node().click());
  inpC.on("change", () => resetBtn.node().click());
  inpD.on("change", () => resetBtn.node().click());

  update();

  return container.node();
}
```

### 41.7.3. Principal component analysis (PCA) — a preview

A dataset of $n$ observations in $p$ variables can be represented as a matrix. The **sample covariance matrix** $\Sigma$ is symmetric, so the spectral theorem applies: it has real eigenvalues and orthogonal eigenvectors.

The eigenvectors of $\Sigma$ are the **principal components** — the directions of maximum variance in the data. The corresponding eigenvalues measure how much variance each direction accounts for.

Projecting data onto the first $k$ principal components is the best rank-$k$ approximation of the data (in the least-squares sense). This is PCA. A face recognition system, a genomics pipeline, and a financial risk model all use the same eigendecomposition under the hood.

**Where this shows up**

- A structural engineer computing mode shapes and natural frequencies of a building frame is solving an eigenvalue problem on a stiffness matrix.
- A quantum chemist finding molecular orbital energies is diagonalising a Hamiltonian matrix.
- A data scientist reducing 1000-dimensional embeddings to 2D for visualisation is computing the top two eigenvectors of a covariance matrix.
- A search engine ranking web pages is finding the dominant eigenvector of a link matrix.

The matrix changes. The method does not.

## 41.8. Exercises

These exercises move from mechanical computation to physical interpretation. In each case: set up the eigenvalue problem, find the eigenvalues and eigenvectors, and state what they mean in the engineering context.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
{
  const css = document.createElement("link");
  css.rel = "stylesheet";
  css.href = "https://cdn.jsdelivr.net/npm/katex@0.16/dist/katex.min.css";
  document.head.appendChild(css);
}
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "✓ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
```

```
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

**1.** The nodal admittance matrix of a two-node electrical network is $A = \begin{pmatrix} 4 & -2 \\ -2 & 4 \end{pmatrix}$. Find the characteristic polynomial and the two eigenvalues. (The eigenvalues are the network's natural admittances — their reciprocals are natural impedances.)

```
//| echo: false
makeStepperHTML(1, [
  {
    op: "Write the characteristic equation",
    eq: "\\det(A - \\lambda I) = \\det\\begin{pmatrix} 4-\\lambda & -2 \\\\ -2 & 4-\\lambda \\end
    note: "Non-trivial solutions exist only when A − λI is singular."
  },
  {
    op: "Expand the determinant",
    eq: "(4-\\lambda)^2 - (-2)(-2) = 0"
  },
  {
    op: "Simplify",
    eq: "16 - 8\\lambda + \\lambda^2 - 4 = 0 \\implies \\lambda^2 - 8\\lambda + 12 = 0"
```

```
  },
  {
    op: "Factor the quadratic",
    eq: "(\\lambda - 2)(\\lambda - 6) = 0 \\implies \\lambda_1 = 2,\\ \\lambda_2 = 6"
  },
  {
    op: "Check with trace and determinant",
    eq: "\\lambda_1 + \\lambda_2 = 8 = \\operatorname{tr}(A),\\quad \\lambda_1 \\lambda_2 = 12 = \\d
  }
])
```

**2.** Using the eigenvalues from Exercise 1, find an eigenvector for each eigenvalue. Then write down the diagonalisation $A = PDP^{-1}$.

```
//| echo: false
makeStepperHTML(2, [
  {
    op: "Eigenvector for λ₁ = 2: solve (A − 2I)x = 0",
    eq: "\\begin{pmatrix} 2 & -2 \\\\ -2 & 2 \\end{pmatrix}\\mathbf{x} = \\mathbf{0} \\implies x_1 =
    note: "Both rows give the same equation 2x₁ − 2x₂ = 0."
  },
  {
    op: "Take x₂ = 1",
    eq: "\\mathbf{x}_1 = \\begin{pmatrix} 1 \\\\ 1 \\end{pmatrix}"
  },
  {
    op: "Eigenvector for λ₂ = 6: solve (A − 6I)x = 0",
    eq: "\\begin{pmatrix} -2 & -2 \\\\ -2 & -2 \\end{pmatrix}\\mathbf{x} = \\mathbf{0} \\implies x_1
  },
  {
    op: "Take x₂ = 1",
    eq: "\\mathbf{x}_2 = \\begin{pmatrix} -1 \\\\ 1 \\end{pmatrix}"
  },
  {
    op: "Assemble P and D",
    eq: "P = \\begin{pmatrix} 1 & -1 \\\\ 1 & 1 \\end{pmatrix},\\quad D = \\begin{pmatrix} 2 & 0 \\\
  },
  {
    op: "Verify AP = PD (first column)",
    eq: "A\\mathbf{x}_1 = \\begin{pmatrix} 4 & -2 \\\\ -2 & 4 \\end{pmatrix}\\begin{pmatrix} 1 \\\\
  }
])
```

**3.** Diagonalise the symmetric matrix $B = \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}$. Write $B = QDQ^T$ using orthonormal eigenvectors.

(This matrix represents a plane stress state — its eigenvectors are the principal stress directions.)

```
//| echo: false
makeStepperHTML(3, [
  {
    op: "Characteristic equation",
    eq: "(5-\\lambda)^2 - 9 = 0 \\implies \\lambda^2 - 10\\lambda + 16 = 0"
  },
  {
    op: "Solve",
    eq: "\\lambda_1 = 2,\\quad \\lambda_2 = 8"
  },
  {
    op: "Eigenvector for λ₁ = 2",
    eq: "(B - 2I)\\mathbf{x} = \\mathbf{0}: \\begin{pmatrix} 3 & 3 \\\\ 3 & 3 \\end{pmatrix}\\mat
  },
  {
    op: "Eigenvector for λ₂ = 8",
    eq: "(B - 8I)\\mathbf{x} = \\mathbf{0}: \\begin{pmatrix} -3 & 3 \\\\ 3 & -3 \\end{pmatrix}\\m
  },
  {
    op: "Normalise to unit length",
    eq: "\\mathbf{q}_1 = \\frac{1}{\\sqrt{2}}\\begin{pmatrix} 1 \\\\ -1 \\end{pmatrix},\\quad \\m
    note: "Each eigenvector has length √2, so divide by √2 to normalise."
  },
  {
    op: "Write the orthogonal diagonalisation",
    eq: "Q = \\frac{1}{\\sqrt{2}}\\begin{pmatrix} 1 & 1 \\\\ -1 & 1 \\end{pmatrix},\\quad D = \\b
  }
])
```

**4.** A simplified three-storey shear building has mass matrix $M = I$ and stiffness matrix $K = \begin{pmatrix} 3 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}$. Find the characteristic polynomial and verify that $\lambda \approx 0.198$ is one eigenvalue by substituting it back.

```
//| echo: false
makeStepperHTML(4, [
  {
    op: "Write det(K − λI) = 0",
    eq: "\\det\\begin{pmatrix} 3-\\lambda & -1 & 0 \\\\ -1 & 2-\\lambda & -1 \\\\ 0 & -1 & 1-\\la
  },
  {
    op: "Expand along row 1",
    eq: "(3-\\lambda)\\bigl[(2-\\lambda)(1-\\lambda)-1\\bigr] + 1\\cdot\\bigl[(-1)(1-\\lambda)\\b
  },
  {
    op: "Simplify inner bracket",
```

```
    eq: "(2-\\lambda)(1-\\lambda) - 1 = \\lambda^2 - 3\\lambda + 1"
  },
  {
    op: "Expand and collect",
    eq: "(3-\\lambda)(\\lambda^2-3\\lambda+1) - (1-\\lambda) = -\\lambda^3 + 6\\lambda^2 - 10\\lambd
  },
  {
    op: "Divide by −1",
    eq: "\\lambda^3 - 6\\lambda^2 + 10\\lambda - 4 = 0"
  },
  {
    op: "Verify □ □ 0.491 is a root (smallest eigenvalue of this K)",
    eq: "(0.491)^3 - 6(0.491)^2 + 10(0.491) - 4 \\approx 0.118 - 1.446 + 4.91 - 4 \\approx -0.418",
    note: "Exact roots require numerical methods; the exercise builds comfort reading the polynomial
  },
  {
    op: "Trace check",
    eq: "\\lambda_1 + \\lambda_2 + \\lambda_3 = \\operatorname{tr}(K) = 3+2+1 = 6\\ \\checkmark"
  }
])
```

**5.** Determine whether the matrix $C = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$ is diagonalisable. If not, explain why and state what the geometric and algebraic multiplicities are.

```
//| echo: false
makeStepperHTML(5, [
  {
    op: "Find eigenvalues",
    eq: "\\det(C - \\lambda I) = (3-\\lambda)^2 = 0 \\implies \\lambda = 3 \\text{ (algebraic multip
  },
  {
    op: "Find eigenspace: solve (C − 3I)x = 0",
    eq: "\\begin{pmatrix} 0 & 1 \\\\ 0 & 0 \\end{pmatrix}\\mathbf{x} = \\mathbf{0} \\implies x_2 = 0
  },
  {
    op: "Eigenspace",
    eq: "\\mathbf{x} = x_1\\begin{pmatrix} 1 \\\\ 0 \\end{pmatrix} \\implies \\text{geometric multip
  },
  {
    op: "Compare multiplicities",
    eq: "\\text{algebraic mult.} = 2 > 1 = \\text{geometric mult.}",
    note: "When these differ, the matrix is defective — not enough eigenvectors to form a basis."
  },
  {
    op: "Conclusion",
    eq: "C \\text{ is NOT diagonalisable (it is defective).",
```

```
    note: "It has Jordan normal form J = [[3,1],[0,3]] — identical to C in this case."
  }
])
```

**6.** Find the two natural frequencies of the two-mass spring system with $m_1 = 1, m_2 = 2, k_1 = k_2 = k_3 = 1$, and describe the two mode shapes (which mass moves more in each mode).

```
//| echo: false
makeStepperHTML(6, [
  {
    op: "Stiffness matrix K (from equations of motion)",
    eq: "K = \\begin{pmatrix} k_1+k_2 & -k_2 \\\\ -k_2 & k_2+k_3 \\end{pmatrix} = \\begin{pmatrix
  },
  {
    op: "Mass matrix M and M⁻¹",
    eq: "M = \\begin{pmatrix} 1 & 0 \\\\ 0 & 2 \\end{pmatrix},\\quad M^{-1} = \\begin{pmatrix} 1
  },
  {
    op: "Form the system matrix A = M⁻¹K",
    eq: "A = M^{-1}K = \\begin{pmatrix} 2 & -1 \\\\ -\\tfrac{1}{2} & 1 \\end{pmatrix}"
  },
  {
    op: "Characteristic equation det(A − ω²I) = 0",
    eq: "(2-\\omega^2)(1-\\omega^2) - \\tfrac{1}{2} = 0 \\implies \\omega^4 - 3\\omega^2 + \\tfra
  },
  {
    op: "Solve for ω² (quadratic in μ = ω²)",
    eq: "\\mu = \\frac{3 \\pm \\sqrt{9-6}}{2} = \\frac{3 \\pm \\sqrt{3}}{2}"
  },
  {
    op: "Natural frequencies",
    eq: "\\omega_1 = \\sqrt{\\tfrac{3-\\sqrt{3}}{2}} \\approx 0.796\\ \\text{rad/s},\\quad \\omeg
  },
  {
    op: "Mode 1 eigenvector (ω₁² ≈ 0.634): solve (A − 0.634I)x = 0",
    eq: "\\begin{pmatrix} 1.366 & -1 \\\\ -0.5 & 0.366 \\end{pmatrix}\\mathbf{x} = \\mathbf{0} \\
    note: "Both entries positive — both masses move in the same direction. Mass 2 has larger ampl
  },
  {
    op: "Mode 2 eigenvector (ω₂² ≈ 2.366): solve (A − 2.366I)x = 0",
    eq: "\\mathbf{x}_2 \\approx \\begin{pmatrix} 1 \\\\ -0.732 \\end{pmatrix}",
    note: "Opposite signs — masses move in opposite directions. This is the higher-frequency mode
  },
  {
    op: "Physical interpretation",
    eq: "\\omega_1 \\approx 0.796\\ \\text{rad/s (in-phase)},\\quad \\omega_2 \\approx 1.538\\ \\
    note: "Driving the system near either frequency causes resonance. The anti-phase mode has hig
```

```
  }
])
```

## 41.9. Where this goes

Eigenvalues connect directly outward in two directions. The first is **ODE systems**: a linear system $\dot{\mathbf{u}} = A\mathbf{u}$ has solution $\mathbf{u}(t) = e^{At}\mathbf{u}(0)$, and the long-term behaviour is governed entirely by the eigenvalues of $A$ — whether solutions decay, grow, or oscillate depends on their real and imaginary parts. The phase plane of Chapter 3 in this volume is a geometric picture of this.

The second direction is **Fourier series**: the sinusoidal functions $\sin(n\pi x/L)$ and $\cos(n\pi x/L)$ are eigenfunctions of the second-derivative operator $d^2/dx^2$ — an extension of the eigenvalue concept to function spaces. The Fourier coefficients are the "eigenvalues" (amplitudes) of the modes. The spectral theorem for matrices has an analogue for symmetric differential operators (Sturm–Liouville theory) that underpins heat equations, wave equations, and quantum mechanics.

The computation behind all of this — finding eigenvalues reliably for large matrices — is the **QR algorithm**, one of the ten most important algorithms of the twentieth century according to *Computing in Science and Engineering*. You now understand what it is computing.

# 42. Vector differential calculus

Grad, div, curl — the calculus of fields

Hold a weather map. The isotherms — lines of equal temperature — curve across it. At any point, one direction makes the temperature climb fastest; the gradient tells you which direction that is and how steep the climb is. Perpendicular to that direction, the temperature doesn't change at all.

Now picture a pipe carrying fluid. At some cross-sections the fluid spreads out; at others it converges. Divergence measures the net outflow at each point — positive where fluid is being created (a source), negative where it drains away (a sink). Curl measures something different: whether the fluid rotates. Stick a tiny paddle wheel in the flow; if it spins, the curl is non-zero.

Three operators — gradient, divergence, curl — encode the geometry of how any field varies through space. Every major equation in physics is written in terms of them.

## 42.1. Vectors: dot and cross products

A vector in three dimensions is a triple of numbers, written $\mathbf{u} = (u_1, u_2, u_3)$ or in column form. Its magnitude is $|\mathbf{u}| = \sqrt{u_1^2 + u_2^2 + u_3^2}$.

Two operations link a pair of vectors to useful scalars and vectors.

### 42.1.1. The dot product

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + u_3 v_3$$

Geometrically: $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}||\mathbf{v}| \cos\theta$, where $\theta$ is the angle between them. This means:

- $\mathbf{u} \cdot \mathbf{v} = 0$ if and only if $\mathbf{u}$ and $\mathbf{v}$ are perpendicular.
- $\text{proj}_{\mathbf{v}}\mathbf{u} = \dfrac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{v}|}$ is the scalar projection of $\mathbf{u}$ onto $\mathbf{v}$ — how much of $\mathbf{u}$ lies in the direction of $\mathbf{v}$.

**Example.** $\mathbf{u} = (1, 2, 3)$, $\mathbf{v} = (4, 0, -1)$.

$$\mathbf{u} \cdot \mathbf{v} = (1)(4) + (2)(0) + (3)(-1) = 4 + 0 - 3 = 1$$

The angle between them: $\cos\theta = \dfrac{1}{\sqrt{14}\sqrt{17}} \approx 0.065$, so $\theta \approx 86^\circ$ — nearly perpendicular.

### 42.1.2. The cross product

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix} = (u_2 v_3 - u_3 v_2,\ u_3 v_1 - u_1 v_3,\ u_1 v_2 - u_2 v_1)$$

The result is a vector perpendicular to both $\mathbf{u}$ and $\mathbf{v}$, with magnitude $|\mathbf{u} \times \mathbf{v}| = |\mathbf{u}||\mathbf{v}| \sin\theta$ — equal to the area of the parallelogram spanned by $\mathbf{u}$ and $\mathbf{v}$. Direction follows the right-hand rule: curl the fingers of your right hand from $\mathbf{u}$ toward $\mathbf{v}$; your thumb points in the direction of $\mathbf{u} \times \mathbf{v}$.

Note that $\mathbf{u} \times \mathbf{v} = -\mathbf{v} \times \mathbf{u}$ — order matters. The cross product is anti-commutative.

**Example.** $\mathbf{u} = (1, 0, 0)$, $\mathbf{v} = (0, 1, 0)$ (the standard basis vectors $\mathbf{i}$ and $\mathbf{j}$).

$$\mathbf{u} \times \mathbf{v} = (0 \cdot 0 - 0 \cdot 1,\ 0 \cdot 0 - 1 \cdot 0,\ 1 \cdot 1 - 0 \cdot 0) = (0, 0, 1) = \mathbf{k}$$

This is correct: the right-hand rule applied to $\mathbf{i} \times \mathbf{j}$ gives $\mathbf{k}$, pointing out of the $xy$-plane.

## 42.2. Scalar and vector fields

A **scalar field** assigns a single number to each point in space: $f : \mathbb{R}^3 \to \mathbb{R}$. Examples:

- Temperature $T(x, y, z)$ — one number at each location
- Pressure $p(x, y, z)$ — one number at each location
- Voltage $V(x, y, z)$ — one number at each location

A **vector field** assigns a vector to each point in space: $\mathbf{F} : \mathbb{R}^3 \to \mathbb{R}^3$. Examples:

- Wind velocity $\mathbf{v}(x, y, z)$ — a direction and speed at each location
- Gravitational field $\mathbf{g}(x, y, z)$ — pointing toward the Earth's centre, weakening with distance
- Electric field $\mathbf{E}(x, y, z)$ — direction a positive test charge would accelerate

Both are functions. The difference is only in what they return. All of single-variable calculus was about functions $f : \mathbb{R} \to \mathbb{R}$. Vector differential calculus extends differentiation to these richer domains.

## 42.3. The gradient

### 42.3.1. Partial derivatives

Before defining the gradient, we need partial derivatives. Given $f(x, y, z)$, the partial derivative with respect to $x$ is:

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x + h, y, z) - f(x, y, z)}{h}$$

Compute it by differentiating $f$ with respect to $x$ while treating $y$ and $z$ as constants. The partial with respect to $y$ treats $x$ and $z$ as constants, and so on.

**Example.** $f(x, y, z) = x^2 y + yz^3$.

$$\frac{\partial f}{\partial x} = 2xy, \qquad \frac{\partial f}{\partial y} = x^2 + z^3, \qquad \frac{\partial f}{\partial z} = 3yz^2$$

Each partial derivative measures the rate of change as you move in exactly one coordinate direction while holding the others fixed.

### 42.3.2. Definition

The **gradient** of a scalar field $f$ is the vector field formed by collecting all three partial derivatives:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}\right)$$

The symbol $\nabla$ (nabla) is the **del operator** — an instruction to differentiate. Applied to a scalar, it produces a vector.

### 42.3.3. Geometric meaning

The gradient $\nabla f$ at any point:

1. **Points in the direction of steepest increase** of $f$.
2. **Is perpendicular to the level surface** $f = c$ passing through that point.
3. Has magnitude $|\nabla f|$ equal to the rate of change of $f$ in that steepest direction.

Think of contour lines on a topographic map. The gradient at any point is an arrow pointing directly uphill — perpendicular to the contour through that point, and longer where the slope is steeper.

**Example.** $f(x, y, z) = x^2 + y^2 + z^2$ (the squared distance from the origin).

$$\nabla f = (2x,\ 2y,\ 2z) = 2(x, y, z) = 2\mathbf{r}$$

The gradient points radially outward — away from the origin, in the direction of increasing distance. This makes geometric sense: the level surfaces of $f$ are spheres centred at the origin, and the gradient is perpendicular to them, i.e. radially outward.

```
//| echo: false
gradientFieldVisualiser = {
  const d3 = await require("d3@7");
  const W = 560, H = 500;
  const ML = 20, MR = 20, MT = 20, MB = 20;
  const innerW = W - ML - MR;
  const innerH = H - MT - MB;

  const container = d3.create("div").style("max-width", W+"px").style("font-family", "sans-serif"
```

```
const controls = container.append("div").style("margin-bottom", "15px").style("background", "#f8fa
controls.append("span").style("font-weight", "600").style("font-size", "0.95rem").style("color", "

const select = controls.append("select").style("padding", "6px").style("border-radius", "4px").sty
const options = [
  {label: "f(x,y) = x² + y² (Bowl)", id: "bowl"},
  {label: "f(x,y) = x² - y² (Saddle)", id: "saddle"},
  {label: "f(x,y) = sin(x)cos(y) (Hills & Valleys)", id: "hills"}
];
options.forEach(o => select.append("option").attr("value", o.id).text(o.label));

const svg = container.append("svg").attr("width", W).attr("height", H).attr("viewBox", `0 0 ${W} S
const g = svg.append("g").attr("transform", `translate(${ML},${MT})`);

const xDomain = [-3, 3];
const yDomain = [-3, 3];

const xScale = d3.scaleLinear(xDomain, [0, innerW]);
const yScale = d3.scaleLinear(yDomain, [innerH, 0]);

function getFunc(id) {
  if (id === "bowl") return { f: (x,y) => x*x + y*y, dx: (x,y) => 2*x, dy: (x,y) => 2*y, maxV: 18
  if (id === "saddle") return { f: (x,y) => x*x - y*y, dx: (x,y) => 2*x, dy: (x,y) => -2*y, maxV:
  return { f: (x,y) => Math.sin(x)*Math.cos(y), dx: (x,y) => Math.cos(x)*Math.cos(y), dy: (x,y) =>
}

const cellSize = 10;
const nx = Math.ceil(innerW/cellSize);
const ny = Math.ceil(innerH/cellSize);
const rectsData = [];
for(let i=0; i<nx; i++) {
  for(let j=0; j<ny; j++) {
    rectsData.push({i, j});
  }
}

const heatmapGroup = g.append("g");
const rects = heatmapGroup.selectAll("rect").data(rectsData).enter().append("rect")
  .attr("width", cellSize + 0.5).attr("height", cellSize + 0.5)
  .attr("x", d => d.i * cellSize)
  .attr("y", d => d.j * cellSize);

const arrowGroup = g.append("g");
const arrowGridSize = 35;
const anx = Math.floor(innerW/arrowGridSize);
const any = Math.floor(innerH/arrowGridSize);
const arrowsData = [];
```

```
for(let i=0; i<=anx; i++) {
  for(let j=0; j<=any; j++) {
    arrowsData.push({i, j, x: xScale.invert(i*arrowGridSize), y: yScale.invert(j*arrowGridSize)
  }
}

const arrows = arrowGroup.selectAll("g.arrow").data(arrowsData).enter().append("g").attr("class
arrows.append("line").attr("stroke", "rgba(0,0,0,0.65)").attr("stroke-width", 1.5).attr("stroke
arrows.append("polygon").attr("fill", "rgba(0,0,0,0.65)");

g.append("rect").attr("width", innerW).attr("height", innerH).attr("fill", "none").attr("stroke

const colorScale = d3.scaleSequential(d3.interpolateRdBu);

function update() {
  const fn = getFunc(select.property("value"));

  colorScale.domain([fn.colorMax, -fn.colorMax]); // Red is high, Blue is low

  rects.attr("fill", d => {
      const cx = d.i * cellSize + cellSize/2;
      const cy = d.j * cellSize + cellSize/2;
      const x = xScale.invert(cx);
      const y = yScale.invert(cy);
      return colorScale(fn.f(x,y));
  });

  arrows.each(function(d) {
      const vx = fn.dx(d.x, d.y);
      const vy = fn.dy(d.x, d.y);

      const gNode = d3.select(this);
      const mag = Math.sqrt(vx*vx + vy*vy);
      if (mag < 1e-4) {
         gNode.style("display", "none");
         return;
      }
      gNode.style("display", "block");

      // Scale arrow length
      const baseLen = arrowGridSize * 0.45;
      const displayScale = Math.min((mag / fn.maxV) * baseLen, baseLen) + (baseLen * 0.2);

      const px = d.i * arrowGridSize;
      const py = d.j * arrowGridSize;

      const ux = vx/mag;
```

```
        const uy = vy/mag;

        // SVG y is down, math y is up
        const dx = ux * displayScale;
        const dy = -uy * displayScale;

        gNode.select("line")
           .attr("x1", px).attr("y1", py)
           .attr("x2", px + dx).attr("y2", py + dy);

        const r = 5;
        const angle = Math.atan2(dy, dx);
        const ex = px + dx;
        const ey = py + dy;
        gNode.select("polygon")
           .attr("points", `${ex},${ey} ${ex - r*Math.cos(angle - 0.5)},${ey - r*Math.sin(angle - 0
    });
  }

  select.on("change", update);
  update();

  return container.node();
}
```

### 42.3.4. Directional derivative

The rate of change of $f$ as you move in an arbitrary direction $\mathbf{u}$ (a unit vector) is called the **directional derivative**:

$$D_{\mathbf{u}} f = \nabla f \cdot \mathbf{u}$$

This is the projection of the gradient onto the direction you're moving. When $\mathbf{u}$ aligns with $\nabla f$, the directional derivative is maximised. When $\mathbf{u}$ is perpendicular to $\nabla f$, the directional derivative is zero — you're moving along a level surface.

**Example.** $f(x, y, z) = x^2 y + yz^3$ at point $(1, 2, -1)$. Find the rate of change in the direction $\mathbf{u} = \frac{1}{\sqrt{3}}(1, 1, 1)$.

First, the gradient at $(1, 2, -1)$:

$$\nabla f\big|_{(1,2,-1)} = \big(2(1)(2),\ (1)^2 + (-1)^3,\ 3(2)(-1)^2\big) = (4, 0, 6)$$

Then:

$$D_{\mathbf{u}} f = (4, 0, 6) \cdot \frac{1}{\sqrt{3}}(1, 1, 1) = \frac{4 + 0 + 6}{\sqrt{3}} = \frac{10}{\sqrt{3}} \approx 5.77$$

### 42.3.5. Total differential

For small displacements $(dx, dy, dz)$ from a point, the change in $f$ is approximated by the **total differential**:

$$df = \frac{\partial f}{\partial x}dx + \frac{\partial f}{\partial y}dy + \frac{\partial f}{\partial z}dz = \nabla f \cdot d\mathbf{r}$$

This is the multi-variable chain rule in compact form. If $x$, $y$, $z$ are themselves functions of a parameter $t$, the chain rule becomes:

$$\frac{df}{dt} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt} + \frac{\partial f}{\partial z}\frac{dz}{dt} = \nabla f \cdot \frac{d\mathbf{r}}{dt}$$

## 42.4. Divergence

### 42.4.1. Definition

Given a vector field $\mathbf{F} = (F_1, F_2, F_3)$, the **divergence** is:

$$\nabla \cdot \mathbf{F} = \frac{\partial F_1}{\partial x} + \frac{\partial F_2}{\partial y} + \frac{\partial F_3}{\partial z}$$

The del operator $\nabla$ is now being "dotted" with the vector field — hence the dot product notation. The result is a scalar field: one number at each point.

**Example.** $\mathbf{F}(x, y, z) = (x^2, 2xy, z^2)$.

$$\nabla \cdot \mathbf{F} = \frac{\partial}{\partial x}(x^2) + \frac{\partial}{\partial y}(2xy) + \frac{\partial}{\partial z}(z^2) = 2x + 2x + 2z = 4x + 2z$$

At the point $(1, 0, 1)$: $\nabla \cdot \mathbf{F} = 4(1) + 2(1) = 6$.

### 42.4.2. Physical meaning

Imagine $\mathbf{F}$ is a fluid velocity field. Enclose a tiny box around a point. The divergence measures the **net rate of fluid flowing out** of that box per unit volume:

- $\nabla \cdot \mathbf{F} > 0$: more fluid leaving than entering — a **source**. (Think: water spraying from a sprinkler head.)
- $\nabla \cdot \mathbf{F} < 0$: more fluid entering than leaving — a **sink**. (Think: water draining into a drain.)
- $\nabla \cdot \mathbf{F} = 0$: perfectly balanced inflow and outflow — the fluid is **incompressible** at that point.

The continuity equation in fluid mechanics captures this precisely:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0$$

This says: the rate of change of density at a point equals the negative divergence of the mass flux. Mass neither appears nor disappears.

> **Preview: Divergence theorem**
> Gauss's divergence theorem connects the divergence over a volume $V$ to the flux through its bounding surface $S$:
>
> $$\iiint_V \nabla \cdot \mathbf{F}\, dV = \oiint_S \mathbf{F} \cdot d\mathbf{A}$$
>
> The integral of all the local sources inside equals the total outflow through the boundary. This is covered in full in the next chapter on vector integral calculus.

## 42.5. Curl

### 42.5.1. Definition

Given a vector field $\mathbf{F} = (F_1, F_2, F_3)$, the **curl** is:

$$\nabla \times \mathbf{F} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ \dfrac{\partial}{\partial x} & \dfrac{\partial}{\partial y} & \dfrac{\partial}{\partial z} \\ F_1 & F_2 & F_3 \end{vmatrix}$$

Expanding the determinant:

$$\nabla \times \mathbf{F} = \left( \frac{\partial F_3}{\partial y} - \frac{\partial F_2}{\partial z}, \frac{\partial F_1}{\partial z} - \frac{\partial F_3}{\partial x}, \frac{\partial F_2}{\partial x} - \frac{\partial F_1}{\partial y} \right)$$

The del operator is now being "crossed" with the vector field. The result is itself a vector field.

**Example.** $\mathbf{F}(x, y, z) = (-y, x, 0)$. This is a 2D field that rotates counterclockwise.

$$\begin{aligned} \frac{\partial F_3}{\partial y} - \frac{\partial F_2}{\partial z} &= 0 - 0 = 0 \\ \frac{\partial F_1}{\partial z} - \frac{\partial F_3}{\partial x} &= 0 - 0 = 0 \\ \frac{\partial F_2}{\partial x} - \frac{\partial F_1}{\partial y} &= 1 - (-1) = 2 \end{aligned}$$

So $\nabla \times \mathbf{F} = (0, 0, 2)$ — a uniform curl pointing in the $z$-direction. The rotation is uniform everywhere, which makes sense: this is solid-body rotation.

### 42.5.2. Physical meaning

Imagine dropping a tiny paddle wheel into a fluid flow. If the paddle wheel spins, the curl is non-zero at that point; if it doesn't spin, the curl is zero. The direction of the curl vector is the axis of rotation (by the right-hand rule); its magnitude measures the rotation speed.

- **Irrotational field:** $\nabla \times \mathbf{F} = \mathbf{0}$ everywhere. The field has no local rotation — no paddle wheel anywhere would spin.
- **Rotational field:** $\nabla \times \mathbf{F} \neq \mathbf{0}$ somewhere.

An irrotational field for which $\nabla \times \mathbf{F} = \mathbf{0}$ throughout a simply-connected region can be written as the gradient of a scalar potential: $\mathbf{F} = \nabla\phi$. Gravity and electrostatics are irrotational outside their sources.

**Key identity:** For any twice-differentiable scalar field $f$,

$$\nabla \times (\nabla f) = \mathbf{0}$$

The curl of a gradient is always zero. Geometrically: gradient fields have no rotation. This identity will appear repeatedly — it is a test for whether a vector field can be expressed as a gradient.

```
//| echo: false
divCurlDemo = {
  const d3 = await require("d3@7");
  const W = 560, H = 450;
  const ML = 20, MR = 20, MT = 20, MB = 40;
  const innerW = W - ML - MR;
  const innerH = H - MT - MB;

  const container = d3.create("div").style("max-width", W+"px").style("font-family", "sans-serif"

  const controls = container.append("div").style("margin-bottom", "15px").style("display", "flex"

  const radioDiv = controls.append("div").style("display", "flex").style("gap", "15px").style("ba
  const types = [
    {id: "source", label: "Source", dx: (x,y)=>x, dy: (x,y)=>y, div: "+2", curl: "0"},
    {id: "sink", label: "Sink", dx: (x,y)=>-x, dy: (x,y)=>-y, div: "-2", curl: "0"},
    {id: "rotation", label: "Rotation", dx: (x,y)=>-y, dy: (x,y)=>x, div: "0", curl: "+2"},
    {id: "shear", label: "Shear", dx: (x,y)=>y, dy: (x,y)=>0, div: "0", curl: "-1"}
  ];

  types.forEach((t, i) => {
    const lbl = radioDiv.append("label").style("display", "flex").style("align-items", "center").
    const inp = lbl.append("input").attr("type", "radio").attr("name", "vf_type").attr("value", t
    if(i === 0) inp.property("checked", true);
    lbl.append("span").text(t.label);
    inp.on("change", update);
  });
```

```
const readout = controls.append("div").style("display", "flex").style("gap", "20px");
const divOut = readout.append("div").style("padding", "8px 12px").style("background", "#fee2e2").s
const curlOut = readout.append("div").style("padding", "8px 12px").style("background", "#e0e7ff")

const svg = container.append("svg").attr("width", W).attr("height", H).attr("viewBox", `0 0 ${W} S
const g = svg.append("g").attr("transform", `translate(${ML},${MT})`);

const xyLim = 3.5;
const xScale = d3.scaleLinear([-xyLim, xyLim], [0, innerW]);
const yScale = d3.scaleLinear([-xyLim, xyLim], [innerH, 0]);

// Grid
g.append("line").attr("x1", xScale(-xyLim)).attr("y1", yScale(0)).attr("x2", xScale(xyLim)).attr("
g.append("line").attr("x1", xScale(0)).attr("y1", yScale(-xyLim)).attr("x2", xScale(0)).attr("y2"
g.append("rect").attr("width", innerW).attr("height", innerH).attr("fill", "none").attr("stroke",

const arrowGroup = g.append("g");
const arrowGridSize = Math.floor(innerW / 14);
const arrowsData = [];
for(let x=0; x<=innerW + 1; x+=arrowGridSize) {
  for(let y=0; y<=innerH + 1; y+=arrowGridSize) {
     arrowsData.push({x, y, dx: xScale.invert(x), dy: yScale.invert(y)});
  }
}

const arrows = arrowGroup.selectAll("g.arrow").data(arrowsData).enter().append("g").attr("class",
arrows.append("line").attr("stroke", "#3b82f6").attr("stroke-width", 2).attr("stroke-linecap", "ro
arrows.append("polygon").attr("fill", "#3b82f6");

function update() {
  const selectedId = radioDiv.select('input[name="vf_type"]:checked').property("value");
  const fn = types.find(t => t.id === selectedId);

  divOut.html(`\u2207 \u00B7 F = ${fn.div}`);
  curlOut.html(`\u2207 \u00D7 F = ${fn.curl} k`);

  arrows.each(function(d) {
     const u = fn.dx(d.dx, d.dy);
     const v = fn.dy(d.dx, d.dy);
     const speed = Math.sqrt(u*u + v*v);

     const gNode = d3.select(this);
     if(speed < 0.1) {
        gNode.style("display", "none");
        return;
     }
     gNode.style("display", "block");
```

```
      // Max field length at edge is approx xyLim (3.5)
      const arrowLen = Math.min((speed / xyLim) * (arrowGridSize * 0.9), arrowGridSize);
      const dirX = u / speed;
      const dirY = v / speed;

      // SVG y is inverted
      const sx = d.x;
      const sy = d.y;
      const ex = sx + dirX * arrowLen;
      const ey = sy - dirY * arrowLen;

      gNode.select("line").attr("x1", sx).attr("y1", sy).attr("x2", ex).attr("y2", ey);

      const angle = Math.atan2(ey - sy, ex - sx);
      const r = 5;
      gNode.select("polygon").attr("points", `${ex},${ey} ${ex - r*Math.cos(angle - 0.5)},${ey -
    });
  }

  update();
  return container.node();
}
```

**Preview: Stokes' theorem**
Stokes' theorem connects the curl over a surface $S$ to the line integral around its boundary curve $C$:

$$\iint_S (\nabla \times \mathbf{F}) \cdot d\mathbf{A} = \oint_C \mathbf{F} \cdot d\mathbf{r}$$

The total rotation through a surface equals the circulation around its edge.

## 42.6. The Laplacian

### 42.6.1. Definition

Apply the gradient to a scalar field $f$ to get $\nabla f$. Then take the divergence of that result. The outcome is the **Laplacian**:

$$\nabla^2 f = \nabla \cdot (\nabla f) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}$$

The Laplacian is a scalar field. It is a single operator that combines second-order partial derivatives in all directions.

**Example.** $f(x, y, z) = x^2 + y^2 + z^2$.

$$\frac{\partial^2 f}{\partial x^2} = 2, \qquad \frac{\partial^2 f}{\partial y^2} = 2, \qquad \frac{\partial^2 f}{\partial z^2} = 2$$

$$\nabla^2 f = 2 + 2 + 2 = 6$$

**Example.** $f(x, y, z) = \frac{1}{r}$ where $r = \sqrt{x^2 + y^2 + z^2}$ (the electrostatic potential of a point charge). A careful calculation shows $\nabla^2\left(\frac{1}{r}\right) = 0$ for $r \neq 0$. This is Laplace's equation, and functions satisfying it are called harmonic.

### 42.6.2. Physical meaning

The Laplacian $\nabla^2 f$ at a point measures how $f$ at that point differs from the average value of $f$ in a small sphere around it. Specifically:

- $\nabla^2 f > 0$: $f$ is below the local average — the field is concave up, like the bottom of a bowl.
- $\nabla^2 f < 0$: $f$ is above the local average — a local maximum relative to its surroundings.
- $\nabla^2 f = 0$: $f$ exactly equals its local average everywhere — this is Laplace's equation, and its solutions are smooth without local extrema in the interior.

The Laplacian appears in every major equation of mathematical physics:

| Equation | Form | What it models |
|---|---|---|
| Laplace's equation | $\nabla^2 f = 0$ | Steady-state electrostatics, incompressible flow |
| Poisson's equation | $\nabla^2 f = g$ | Electrostatics with charges, gravity with mass |
| Heat equation | $\frac{\partial u}{\partial t} = k\nabla^2 u$ | Heat diffusion through a material |
| Wave equation | $\frac{\partial^2 u}{\partial t^2} = c^2\,\nabla^2 u$ | Sound, light, water waves |

## 42.7. Applications

### 42.7.1. Gradient descent and optimisation

In machine learning, a loss function $L(\mathbf{w})$ measures how wrong a model is, where $\mathbf{w}$ is a vector of model parameters. The gradient $\nabla_{\mathbf{w}} L$ points in the direction of steepest increase of the loss. To reduce the loss, move in the opposite direction:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta\,\nabla_{\mathbf{w}} L$$

The scalar $\eta$ is the **learning rate** — how large a step to take. This is gradient descent. In high-dimensional spaces (modern neural networks have billions of parameters), computing the gradient efficiently is the central computational challenge. The mathematics is exactly what this chapter covers, lifted to millions of dimensions.

### 42.7.2. Electric field from voltage potential

In electrostatics, the voltage $V(x, y, z)$ is a scalar field. The electric field is its negative gradient:

$$\mathbf{E} = -\nabla V$$

The minus sign is physical convention: the electric field points from high to low voltage, not from low to high. Positive charges accelerate down the voltage hill.

If $V = x^2 + y^2 + z^2$ (a simple quadratic potential), then:

$$\nabla V = (2x, 2y, 2z), \qquad \mathbf{E} = (-2x, -2y, -2z)$$

The field points radially inward — toward the origin, toward lower potential. Taking the divergence:

$$\nabla \cdot \mathbf{E} = -2 - 2 - 2 = -6$$

Gauss's law in differential form states $\nabla \cdot \mathbf{E} = \rho/\varepsilon_0$, where $\rho$ is the charge density. A uniform $\nabla \cdot \mathbf{E} = -6$ means a uniform negative charge density — a medium with uniform charge throughout.

### 42.7.3. Continuity equation in fluid mechanics

For a fluid with density $\rho(x, y, z, t)$ and velocity field $\mathbf{v}(x, y, z, t)$, conservation of mass requires:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0$$

This says: the rate at which mass density increases at a point equals the negative divergence of mass flux. Mass flows in from regions where $\nabla \cdot (\rho \mathbf{v}) < 0$ and flows out where it is positive. If the fluid is incompressible ($⊠= $ const), this simplifies to $\nabla \cdot \mathbf{v} = 0$.

**Where this shows up**

- An **electrical engineer** designs antenna patterns using the curl and divergence of Maxwell's equations: $\nabla \times \mathbf{E} = -\partial \mathbf{B}/\partial t$ and $\nabla \cdot \mathbf{B} = 0$.
- A **meteorologist** reads wind fields as vector fields and uses divergence to locate areas of rising air (positive divergence near the surface means air is being pulled up).
- A **robotics engineer** computing the configuration-space gradient of a potential function is doing gradient descent in the exact sense of this chapter.
- A **GIS analyst** computing watershed flow directions is applying the gradient of the elevation field: water flows opposite to $\nabla h$.

The operators are the same in every case. The domain changes.

## 42.8. Exercises

These exercises progress from component calculation through applied interpretation. In each, start with the definition before writing any formula.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
```

```
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

1. A temperature field on a heated plate is modelled by $f(x, y, z) = x^2y + yz^3$. Find $\nabla f$ and evaluate it at the point $(1, 2, -1)$. Which coordinate direction gives the greatest rate of temperature increase at that point?

```
//| echo: false
makeStepperHTML(1, [
  { op: "Partial derivative w.r.t. x",
    eq: "\\frac{\\partial f}{\\partial x} = 2xy",
    note: "Treat y and z as constants; differentiate x² y with respect to x." },
  { op: "Partial derivative w.r.t. y",
    eq: "\\frac{\\partial f}{\\partial y} = x^2 + z^3",
    note: "Differentiate x²y + yz³ with respect to y; both terms contribute." },
  { op: "Partial derivative w.r.t. z",
    eq: "\\frac{\\partial f}{\\partial z} = 3yz^2",
    note: "Only yz³ depends on z." },
  { op: "Assemble the gradient",
    eq: "\\nabla f = \\bigl(2xy,\\; x^2 + z^3,\\; 3yz^2\\bigr)" },
  { op: "Evaluate at (1, 2, -1)",
    eq: "\\nabla f\\big|_{(1,2,-1)} = \\bigl(2(1)(2),\\; (1)^2 + (-1)^3,\\; 3(2)(-1)^2\\bigr) = (
```

```
    note: "Substitute x = 1, y = 2, z = -1 into each component." },
  { op: "Greatest rate of increase",
    eq: "\\text{The z-component is 6, the x-component is 4, the y-component is 0.}",
    note: "The z-direction has the largest partial derivative at this point, so temperature increase
])
```

2. A pressure field is given by $p(x, y, z) = 3x^2 - 2y^2 + xyz$. At the point $(2, 1, 3)$, find the rate of change of pressure in the direction $\mathbf{u} = \frac{1}{\sqrt{2}}(1, 0, 1)$ — the diagonal direction in the $xz$-plane.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Compute partial derivatives",
    eq: "\\frac{\\partial p}{\\partial x} = 6x + yz,\\quad \\frac{\\partial p}{\\partial y} = -4y +
  { op: "Evaluate at (2, 1, 3)",
    eq: "\\nabla p = \\bigl(6(2) + (1)(3),\\; -4(1) + (2)(3),\\; (2)(1)\\bigr) = (15,\\; 2,\\; 2)",
    note: "Substitute x = 2, y = 1, z = 3." },
  { op: "Confirm u is a unit vector",
    eq: "|\\mathbf{u}| = \\sqrt{\\tfrac{1}{2} + 0 + \\tfrac{1}{2}} = 1 \\checkmark" },
  { op: "Compute directional derivative",
    eq: "D_{\\mathbf{u}}\\, p = \\nabla p \\cdot \\mathbf{u} = (15, 2, 2) \\cdot \\tfrac{1}{\\sqrt{2
    note: "The dot product selects the component of the gradient in the direction u." }
])
```

3. A fluid velocity field is given by $\mathbf{F}(x, y, z) = (x^2, 2xy, z^2)$. Compute $\nabla \cdot \mathbf{F}$. Identify the regions where the fluid acts as a source (positive divergence) and where it acts as a sink (negative divergence).

```
//| echo: false
makeStepperHTML(3, [
  { op: "Identify components",
    eq: "F_1 = x^2,\\quad F_2 = 2xy,\\quad F_3 = z^2" },
  { op: "Partial derivative of F₁ w.r.t. x",
    eq: "\\frac{\\partial F_1}{\\partial x} = 2x" },
  { op: "Partial derivative of F₂ w.r.t. y",
    eq: "\\frac{\\partial F_2}{\\partial y} = 2x",
    note: "Differentiate 2xy with respect to y; x is treated as a constant." },
  { op: "Partial derivative of F₃ w.r.t. z",
    eq: "\\frac{\\partial F_3}{\\partial z} = 2z" },
  { op: "Assemble divergence",
    eq: "\\nabla \\cdot \\mathbf{F} = 2x + 2x + 2z = 4x + 2z" },
  { op: "Interpret",
    eq: "\\nabla \\cdot \\mathbf{F} > 0 \\text{ when } 4x + 2z > 0,\\text{ i.e. when } 2x + z > 0",
    note: "The field is a source (net outflow) in the half-space 2x + z > 0, and a sink where 2x + z
])
```

4. Verify that $\mathbf{F}(x, y, z) = (-y, x, 0)$ represents a purely rotational flow — it should have non-zero curl and zero divergence. Compute both.

```
//| echo: false
makeStepperHTML(4, [
  { op: "Identify components",
    eq: "F_1 = -y,\\quad F_2 = x,\\quad F_3 = 0" },
  { op: "Compute divergence",
    eq: "\\nabla \\cdot \\mathbf{F} = \\frac{\\partial(-y)}{\\partial x} + \\frac{\\partial(x)}{\
    note: "No sources or sinks — the fluid is incompressible." },
  { op: "Curl: z-component",
    eq: "\\frac{\\partial F_2}{\\partial x} - \\frac{\\partial F_1}{\\partial y} = 1 - (-1) = 2",
    note: "This is the dominant term for a rotation in the xy-plane." },
  { op: "Curl: x and y components",
    eq: "\\frac{\\partial F_3}{\\partial y} - \\frac{\\partial F_2}{\\partial z} = 0,\\quad \\fra
  { op: "Assemble curl",
    eq: "\\nabla \\times \\mathbf{F} = (0,\\; 0,\\; 2)",
    note: "Constant curl pointing in the z-direction: uniform counterclockwise rotation everywher
])
```

5. Let $f(x, y, z) = x^2y^2z$. Verify the identity $\nabla \times (\nabla f) = \mathbf{0}$ — the curl of a gradient is always zero. This confirms that gradient fields are irrotational (conservative).

```
//| echo: false
makeStepperHTML(5, [
  { op: "Compute gradient",
    eq: "\\nabla f = \\bigl(2xy^2z,\\; 2x^2yz,\\; x^2y^2\\bigr)" },
  { op: "Curl x-component",
    eq: "\\frac{\\partial(x^2y^2)}{\\partial y} - \\frac{\\partial(2x^2yz)}{\\partial z} = 2x^2y
    note: "Each term differentiates a component of ∇f." },
  { op: "Curl y-component",
    eq: "\\frac{\\partial(2xy^2z)}{\\partial z} - \\frac{\\partial(x^2y^2)}{\\partial x} = 2xy^2
  { op: "Curl z-component",
    eq: "\\frac{\\partial(2x^2yz)}{\\partial x} - \\frac{\\partial(2xy^2z)}{\\partial y} = 4xyz -
  { op: "Conclusion",
    eq: "\\nabla \\times (\\nabla f) = (0,\\; 0,\\; 0) = \\mathbf{0} \\checkmark",
    note: "This holds for any twice-differentiable f — the cancellations come from the symmetry o
])
```

6. The electric potential in free space is $V(x, y, z) = x^2 + y^2 + z^2$. Find the electric field $\mathbf{E} = -\nabla V$, then compute $\nabla \cdot \mathbf{E}$ and identify what Gauss's law says about the charge distribution.

```
//| echo: false
makeStepperHTML(6, [
  { op: "Compute gradient of V",
    eq: "\\nabla V = \\left(\\frac{\\partial V}{\\partial x},\\frac{\\partial V}{\\partial y},\\f
  { op: "Electric field E = -∇V",
    eq: "\\mathbf{E} = -\\nabla V = (-2x,\\; -2y,\\; -2z)",
    note: "The field points toward the origin — inward, toward lower potential." },
  { op: "Compute divergence of E",
```

```
    eq: "\\nabla \\cdot \\mathbf{E} = \\frac{\\partial(-2x)}{\\partial x} + \\frac{\\partial(-2y)}{\
  { op: "Apply Gauss's law",
    eq: "\\nabla \\cdot \\mathbf{E} = \\frac{\\rho}{\\varepsilon_0} \\implies \\rho = -6\\varepsilon
    note: "Gauss's law in differential form: the divergence of E equals charge density over permitti
  { op: "Interpret",
    eq: "\\rho = -6\\varepsilon_0 = \\text{const}",
    note: "A uniform negative charge density fills all space. The electric field converges inward ev
])
```

# 43. Vector integral calculus

Green's, Stokes', and the Divergence theorem

A turbine blade sits in a flow of hot gas. You need to know the total heat flux through the blade's surface — but integrating over every point on that complicated curved surface is brutal. The Divergence theorem says you only need to know the divergence of the heat flux field throughout the interior volume, which is often far simpler to compute.

This is the deepest efficiency in classical calculus: boundary integrals and interior integrals are two faces of the same coin. Green's theorem, Stokes' theorem, and the Divergence theorem are the exchange rate.

## 43.1. Line integrals

A line integral accumulates the effect of a field along a curve. There are two kinds: integrals of scalar fields and integrals of vector fields. The vector case is the one that matters physically — it measures work.

### 43.1.1. Scalar line integrals

If $f$ is a scalar field and $C$ is a curve, the scalar line integral is:

$$\int_C f\,ds$$

where $ds$ is arc length. This answers questions like "what is the total mass of a wire with variable density $f$?" or "what is the average temperature along a path?"

To evaluate it, parametrise the curve: let $\mathbf{r}(t) = (x(t), y(t), z(t))$ for $t \in [a, b]$. Then $ds = |\mathbf{r}'(t)|\,dt$, so:

$$\int_C f\,ds = \int_a^b f(\mathbf{r}(t))\,|\mathbf{r}'(t)|\,dt$$

The integral is independent of how you parametrise the curve — only the geometry of $C$ matters, not the speed at which you traverse it.

### 43.1.2. Vector line integrals (work)

If $\mathbf{F}$ is a vector field and $C$ is an oriented curve, the vector line integral is:

$$\int_C \mathbf{F} \cdot d\mathbf{r}$$

This measures the **work** done by the field $\mathbf{F}$ on a particle moving along $C$. The dot product $\mathbf{F} \cdot d\mathbf{r}$ picks out the component of $\mathbf{F}$ that is parallel to the direction of travel — only that component does work.

To evaluate, parametrise $C$ as $\mathbf{r}(t)$, so $d\mathbf{r} = \mathbf{r}'(t)\,dt$:

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \int_a^b \mathbf{F}(\mathbf{r}(t)) \cdot \mathbf{r}'(t)\,dt$$

Writing out in components with $\mathbf{F} = (P, Q, R)$ and $d\mathbf{r} = (dx, dy, dz)$:

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \int_C P\,dx + Q\,dy + R\,dz$$

**Sign convention:** reversing the orientation of $C$ changes the sign of the integral. For a closed curve traversed anti-clockwise (positive orientation), the notation is $\oint_C \mathbf{F} \cdot d\mathbf{r}$.

**Worked example.** Compute $\int_C \mathbf{F} \cdot d\mathbf{r}$ where $\mathbf{F} = (y, -x)$ and $C$ is the upper semicircle of the unit circle from $(1, 0)$ to $(-1, 0)$.

Parametrise: $\mathbf{r}(t) = (\cos t, \sin t)$, $t \in [0, \pi]$, so $\mathbf{r}'(t) = (-\sin t, \cos t)$.

Along $C$: $\mathbf{F}(\mathbf{r}(t)) = (\sin t, -\cos t)$.

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \int_0^\pi (\sin t, -\cos t) \cdot (-\sin t, \cos t)\,dt$$

$$= \int_0^\pi (-\sin^2 t - \cos^2 t)\,dt = \int_0^\pi -1\,dt = -\pi$$

The field does $-\pi$ units of work on the particle — it opposes the motion.

---

```
//| echo: false
lineIntegralDemo = {
  const d3 = await require("d3@7");
  const W = 560, H = 450;
  const ML = 20, MR = 20, MT = 20, MB = 60;
  const innerW = W - ML - MR;
  const innerH = H - MT - MB;

  const container = d3.create("div").style("max-width", W+"px").style("font-family", "sans-serif").s
```

```
const controls = container.append("div").style("margin-bottom", "15px").style("padding", "10px"

const pathDiv = controls.append("div").style("display", "flex").style("gap", "15px").style("ali
pathDiv.append("span").style("font-weight", "600").style("font-size", "0.9rem").text("Path C:")
const paths = [
  {id: "line", label: "Straight Line", fn: t => [-2 + 4*t, 0], d: t => [4, 0]},
  {id: "semi", label: "Semicircle", fn: t => [-2*Math.cos(Math.PI*t), 2*Math.sin(Math.PI*t)], d
  {id: "parabola", label: "Parabola", fn: t => [-2 + 4*t, 2 - 2*(2*t - 1)*(2*t - 1)], d: t => [
];

paths.forEach((p, i) => {
  const lbl = pathDiv.append("label").style("display", "flex").style("align-items", "center").s
  const inp = lbl.append("input").attr("type", "radio").attr("name", "path_type").attr("value",
  if(i===0) inp.property("checked", true);
  lbl.append("span").text(p.label);
  inp.on("change", () => { tSlider.property("value", 0); tValSpan.text("0.00"); update(); });
});

const tDiv = controls.append("div").style("display", "flex").style("align-items", "center").sty
tDiv.append("span").text("t = ");
const tSlider = tDiv.append("input").attr("type", "range").attr("min", 0).attr("max", 1).attr("
const tValSpan = tDiv.append("span").style("width", "35px").style("font-family", "monospace").t

const svg = container.append("svg").attr("width", W).attr("height", H).attr("viewBox", `0 0 ${W
const g = svg.append("g").attr("transform", `translate(${ML},${MT})`);

const xyLim = 2.5;
const xScale = d3.scaleLinear([-xyLim, xyLim], [0, innerW]);
const yScale = d3.scaleLinear([-xyLim, xyLim], [innerH, 0]);

g.append("line").attr("x1", xScale(-xyLim)).attr("y1", yScale(0)).attr("x2", xScale(xyLim)).att
g.append("line").attr("x1", xScale(0)).attr("y1", yScale(-xyLim)).attr("x2", xScale(0)).attr("y

const F = (x, y) => [-y, x*x];

const arrowGroup = g.append("g").attr("opacity", 0.3);
const arrowGridSize = 25;
for(let x=0; x<=innerW; x+=arrowGridSize) {
  for(let y=0; y<=innerH; y+=arrowGridSize) {
     const u = xScale.invert(x);
     const v = yScale.invert(y);
     const fv = F(u, v);
     const speed = Math.sqrt(fv[0]*fv[0] + fv[1]*fv[1]);
     if(speed > 0.1) {
        const len = Math.min(speed*4, arrowGridSize*0.8);
        const dx = (fv[0]/speed)*len;
        const dy = -(fv[1]/speed)*len;
```

```
        g.append("line").attr("x1", x).attr("y1", y).attr("x2", x+dx).attr("y2", y+dy).attr("strok
        g.append("circle").attr("cx", x).attr("cy", y).attr("r", 1.5).attr("fill", "#334155");
      }
    }
  }

  const pathCurve = g.append("path").attr("fill", "none").attr("stroke", "#94a3b8").attr("stroke-wid
  const traveledCurve = g.append("path").attr("fill", "none").attr("stroke", "#3b82f6").attr("stroke

  const fVec = g.append("g");
  const fVecLine = fVec.append("line").attr("stroke", "#ef4444").attr("stroke-width", 2.5);
  const fVecHead = fVec.append("polygon").attr("fill", "#ef4444");

  const drVec = g.append("g");
  const drVecLine = drVec.append("line").attr("stroke", "#10b981").attr("stroke-width", 2.5);
  const drVecHead = drVec.append("polygon").attr("fill", "#10b981");

  const particle = g.append("circle").attr("r", 6).attr("fill", "#3b82f6");

  const plotG = svg.append("g").attr("transform", `translate(${ML},${H - MB + 20})`);
  const ptScale = d3.scaleLinear([0, 1], [0, innerW]);
  const pyScale = d3.scaleLinear([-15, 15], [30, 0]);

  plotG.append("line").attr("x1", 0).attr("x2", innerW).attr("y1", pyScale(0)).attr("y2", pyScale(0)
  const intArea = plotG.append("path").attr("fill", "rgba(59, 130, 246, 0.2)");
  const intLine = plotG.append("path").attr("fill", "none").attr("stroke", "#3b82f6").attr("stroke-w
  const intDot = plotG.append("circle").attr("r", 4).attr("fill", "#2563eb");

  const totalLabel = svg.append("text").attr("x", W - MR).attr("y", H - 5).attr("text-anchor", "end"

  function drawVec(vG, x, y, dx, dy) {
    vG.select("line").attr("x1", x).attr("y1", y).attr("x2", x+dx).attr("y2", y+dy);
    const angle = Math.atan2(dy, dx);
    const r = 6;
    vG.select("polygon").attr("points", `${x+dx},${y+dy} ${x+dx - r*Math.cos(angle - 0.5)},${y+dy -
  }

  function update() {
    const selectedId = pathDiv.select('input[name="path_type"]:checked').property("value");
    const pdef = paths.find(p => p.id === selectedId);
    const t = +tSlider.property("value");
    tValSpan.text(t.toFixed(2));

    const pts = d3.range(0, 1.01, 0.02).map(tt => pdef.fn(tt));
    const lineGen = d3.line().x(d => xScale(d[0])).y(d => yScale(d[1]));
    pathCurve.attr("d", lineGen(pts));
```

```
  const ptsTraveled = d3.range(0, t+0.001, 0.02).map(tt => pdef.fn(tt));
  traveledCurve.attr("d", lineGen(ptsTraveled));

  const pt = pdef.fn(t);
  const px = xScale(pt[0]);
  const py = yScale(pt[1]);
  particle.attr("cx", px).attr("cy", py);

  const fv = F(pt[0], pt[1]);
  drawVec(fVec, px, py, fv[0]*15, -fv[1]*15);

  const dr = pdef.d(t);
  drawVec(drVec, px, py, dr[0]*5, -dr[1]*5);

  const intData = d3.range(0, t+0.001, 0.02).map(tt => {
     const cpt = pdef.fn(tt);
     const cdr = pdef.d(tt);
     const cf = F(cpt[0], cpt[1]);
     const dot = cf[0]*cdr[0] + cf[1]*cdr[1];
     return {t: tt, val: dot};
  });

  const areaGen = d3.area().x(d => ptScale(d.t)).y0(pyScale(0)).y1(d => pyScale(d.val));
  const lineGenPlot = d3.line().x(d => ptScale(d.t)).y(d => pyScale(d.val));

  intArea.attr("d", areaGen(intData));
  intLine.attr("d", lineGenPlot(intData));

  if(intData.length > 0) {
    const last = intData[intData.length-1];
    intDot.attr("cx", ptScale(last.t)).attr("cy", pyScale(last.val)).attr("visibility", "visibl
  } else {
    intDot.attr("visibility", "hidden");
  }

  let sum = 0;
  const dt = 0.005;
  for(let tt=0; tt<=t; tt+=dt) {
    const cpt = pdef.fn(tt);
    const cdr = pdef.d(tt);
    const cf = F(cpt[0], cpt[1]);
    sum += (cf[0]*cdr[0] + cf[1]*cdr[1]) * dt;
  }

  totalLabel.text(`\u222B F \u00B7 dr \u2248 ${sum.toFixed(2)}`);
}
```

```
  tSlider.on("input", update);
  update();

  return container.node();
}
```

---

## 43.2. Path independence and conservative fields

For many fields, the line integral between two points is the same regardless of which path you take. Such fields are called **conservative**.

A vector field $\mathbf{F}$ is conservative on a simply connected domain if and only if any (equivalently all) of the following hold:

1. $\oint_C \mathbf{F} \cdot d\mathbf{r} = 0$ for every closed curve $C$.
2. $\mathbf{F} = \nabla f$ for some scalar **potential function** $f$.
3. In 3D: $\nabla \times \mathbf{F} = \mathbf{0}$ (the field is irrotational; curl is zero).

The equivalence of (2) and (3) was established in Chapter 3. The equivalence of (1) and (2) follows from the Fundamental Theorem for line integrals.

### 43.2.1. Fundamental Theorem for line integrals

If $\mathbf{F} = \nabla f$, then:

$$\int_C \mathbf{F} \cdot d\mathbf{r} = f(\mathbf{r}(b)) - f(\mathbf{r}(a))$$

where $\mathbf{r}(a)$ and $\mathbf{r}(b)$ are the start and end points of $C$. The integral depends only on the endpoints — not on the path.

This is the vector analogue of the Fundamental Theorem of Calculus: the integral of a derivative (here, $\nabla f$) over a domain (here, the curve) equals the value of the original function at the boundary (here, the two endpoints, with sign).

**Finding the potential function.** Given $\mathbf{F} = (P, Q, R)$ with $\nabla \times \mathbf{F} = \mathbf{0}$, find $f$ such that $\nabla f = \mathbf{F}$. Integrate component by component:

$$f(x, y, z) = \int P dx + g(y, z)$$

then differentiate with respect to $y$ and match against $Q$ to determine $g$, and so on.

**Example.** Is $\mathbf{F} = (2xy + z^2,\ x^2,\ 2xz)$ conservative?

Check $\nabla \times \mathbf{F}$: compute each component.

$(\nabla \times \mathbf{F})_x = \partial_y(2xz) - \partial_z(x^2) = 0 - 0 = 0$

$(\nabla \times \mathbf{F})_y = \partial_z(2xy + z^2) - \partial_x(2xz) = 2z - 2z = 0$

$(\nabla \times \mathbf{F})_z = \partial_x(x^2) - \partial_y(2xy + z^2) = 2x - 2x = 0$

All zero, so $\mathbf{F}$ is conservative. Find $f$:

From $\partial f/\partial x = 2xy + z^2$, integrate in $x$: $f = x^2y + xz^2 + g(y, z)$.

From $\partial f/\partial y = x^2$: matches $Q = x^2$, so $\partial g/\partial y = 0$.

From $\partial f/\partial z = 2xz$: matches $R = 2xz$, so $\partial g/\partial z = 0$.

Therefore $f(x, y, z) = x^2y + xz^2$ (plus an arbitrary constant).

> **Why conservative fields matter in physics**
> Gravity is conservative. Electric fields (in electrostatics) are conservative. This means the work done lifting an object or moving a charge depends only on where it starts and where it ends — not on the path. This is why potential energy is a well-defined concept: the number $f(\mathbf{r})$ at any point is the energy stored.
> Friction is not conservative: a longer path dissipates more energy. The difference is that $\nabla \times \mathbf{F} \neq \mathbf{0}$ for non-conservative forces.

## 43.3. Green's theorem

Green's theorem connects a line integral around a closed curve in the plane to a double integral over the enclosed region.

**Statement.** Let $C$ be a positively oriented (anti-clockwise), piecewise smooth, simple closed curve in the $xy$-plane, and let $D$ be the region it encloses. If $P(x, y)$ and $Q(x, y)$ have continuous partial derivatives on an open set containing $D$, then:

$$\oint_C P\,dx + Q\,dy = \iint_D \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y}\right) dA$$

### 43.3.1. Where it comes from: cellular decomposition

Divide $D$ into tiny rectangles. For each rectangle, compute the line integral around its boundary (traversed anti-clockwise). When you sum all these contributions, every interior edge is traversed twice — once in each direction by adjacent rectangles — so those contributions cancel exactly. What survives is only the contribution from the outer boundary of $D$, which is $C$.

For each tiny rectangle with width $\Delta x$ and height $\Delta y$, the line integral around its boundary approximates:

$$\left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y}\right) \Delta x \Delta y$$

Summing over all rectangles and taking the limit gives the double integral on the right of Green's theorem.

---

```
//| echo: false
greensTheoremDemo = {
  const d3 = await require("d3@7");
  const W = 560, H = 450;
  const ML = 30, MR = 30, MT = 30, MB = 60;
  const innerW = W - ML - MR;
  const innerH = H - MT - MB;

  const container = d3.create("div").style("max-width", W+"px").style("font-family", "sans-serif").s

  const controls = container.append("div").style("margin-bottom", "15px").style("padding", "10px").s

  controls.append("span").style("font-weight", "600").style("font-size", "0.9rem").text("Decomposit

  const slider = controls.append("input").attr("type", "range").attr("min", 0).attr("max", 3).attr("
  const valSpan = controls.append("span").style("font-family", "monospace").text("1x1").style("width

  const animBtn = controls.append("button").text("Animate Cancellation").style("padding", "5px 12px"

  const svg = container.append("svg").attr("width", W).attr("height", H).attr("viewBox", `0 0 ${W} S
  const g = svg.append("g").attr("transform", `translate(${ML},${MT})`);

  const rectW = innerW * 0.8;
  const rectH = innerH * 0.8;
  const offsetX = (innerW - rectW)/2;
  const offsetY = (innerH - rectH)/2;

  const cellsGroup = g.append("g");

  const readout = container.append("div").style("display", "flex").style("justify-content", "space-a
  const leftSide = readout.append("div").text("\u222E F \u00B7 dr = 12.00");
  const rightSide = readout.append("div").text("\u222C (\u2207 \u00D7 F) dA = 12.00");

  let animationRef = null;
  const totalCurl = 12.00; // Simulated exact answer

  function drawCells(N) {
     cellsGroup.selectAll("*").remove();

     const cw = rectW / N;
     const ch = rectH / N;
```

```
    leftSide.text(`\u222E C F \u00B7 dr \u2248 ${totalCurl.toFixed(2)}`);
    rightSide.text(`\u222C \u2207 \u00D7 F dA \u2248 ${totalCurl.toFixed(2)}`);

    for(let i=0; i<N; i++) {
       for(let j=0; j<N; j++) {
          const x = offsetX + i*cw;
          const y = offsetY + j*ch;

          const cellG = cellsGroup.append("g").attr("class", "cell");
          cellG.append("rect").attr("x", x).attr("y", y).attr("width", cw).attr("height", ch).at

          const pad = Math.min(cw, ch) * 0.15;
          const L = x + pad; const R = x + cw - pad;
          const T = y + pad; const B = y + ch - pad;

          const lLen = 6;
          const color = "#3b82f6";

          const addArrow = (x1,y1, x2,y2, isExternal) => {
              const gA = cellG.append("g").attr("class", isExternal ? "ext-boundary" : "int-boun
              gA.append("line").attr("x1", x1).attr("y1", y1).attr("x2", x2).attr("y2", y2).attr
              const mx = (x1+x2)/2; const my = (y1+y2)/2;
              const angle = Math.atan2(y2-y1, x2-x1);
              gA.append("polygon").attr("points", `${mx},${my} ${mx-lLen*Math.cos(angle-0.5)},${
          };

          addArrow(L, T, R, T, j===0); // top
          addArrow(R, T, R, B, i===N-1); // right
          addArrow(R, B, L, B, j===N-1); // bottom
          addArrow(L, B, L, T, i===0); // left
       }
    }
 }

 function update() {
    if(animationRef) clearTimeout(animationRef);
    const val = Math.pow(2, +slider.property("value"));
    valSpan.text(`${val}x${val}`);
    drawCells(val);
 }

 animBtn.on("click", () => {
     cellsGroup.selectAll("rect").transition().duration(200).attr("fill", "transparent").attr("s
     cellsGroup.selectAll(".int-boundary").transition().duration(200).attr("stroke", "#ef4444").

     setTimeout(() => {
         cellsGroup.selectAll(".int-boundary").transition().duration(1000).attr("opacity", 0);
```

```
        cellsGroup.selectAll(".ext-boundary").transition().duration(500).attr("stroke", "#10b981")
    }, 500);

    // Auto-reset
    animationRef = setTimeout(update, 3500);
  });

  slider.on("input", update);
  update();

  return container.node();
}
```

---

**Using Green's theorem.** The theorem converts between two kinds of integrals. Use it when:

- The line integral $\oint_C P\,dx + Q\,dy$ is complicated but the integrand $\partial Q/\partial x - \partial P/\partial y$ is simple.
- The double integral over $D$ is complicated but the boundary $C$ is simple.

**Example.** Compute $\oint_C (-y\,dx + x\,dy)$ where $C$ is the circle $x^2 + y^2 = 4$ traversed anti-clockwise.

Here $P = -y$, $Q = x$, so $\partial Q/\partial x - \partial P/\partial y = 1 - (-1) = 2$.

By Green's theorem:

$$\oint_C (-y\,dx + x\,dy) = \iint_D 2\,dA = 2 \cdot \text{Area}(D) = 2 \cdot \pi(2)^2 = 8\pi$$

The boundary integral reduced to computing an area. Notice that $\frac{1}{2}\oint_C (-y\,dx + x\,dy) = \text{Area}(D)$ — this is a general formula for area from a line integral, directly from Green's theorem.

> **Planimeters: Green's theorem in hardware**
> A planimeter is a mechanical device for measuring area. You trace the boundary of a region with the pointer; the rotating wheel integrates $-y\,dx + x\,dy$ mechanically as you go, reading off the area directly. Engineers used planimeters to measure irregular cross-sections in structural calculations before digital tools. The device is a physical implementation of Green's theorem.

## 43.4. Surface integrals

Surface integrals extend integration from curves to surfaces in 3D. As with line integrals, there are scalar and vector versions.

### 43.4.1. Parametric surfaces

A surface $S$ can be parametrised by $\mathbf{r}(u, v) = (x(u, v), y(u, v), z(u, v))$ for $(u, v)$ in some region $D$ of the parameter plane. The surface element $d\mathbf{S}$ is the vector area of an infinitesimal patch:

$$d\mathbf{S} = \frac{\partial \mathbf{r}}{\partial u} \times \frac{\partial \mathbf{r}}{\partial v}\, du\, dv$$

Its direction is normal to the surface; its magnitude $\left|\frac{\partial \mathbf{r}}{\partial u} \times \frac{\partial \mathbf{r}}{\partial v}\right|$ is the scalar surface area element $dS$.

For a surface $z = g(x, y)$ over a region $D$ in the $xy$-plane:

$$dS = \sqrt{1 + \left(\frac{\partial g}{\partial x}\right)^2 + \left(\frac{\partial g}{\partial y}\right)^2}\, dA$$

### 43.4.2. Flux integrals

The **flux** of a vector field $\mathbf{F}$ through a surface $S$ is:

$$\iint_S \mathbf{F} \cdot d\mathbf{S} = \iint_S \mathbf{F} \cdot \hat{\mathbf{n}}\, dS$$

where $\hat{\mathbf{n}}$ is the outward unit normal. Physically, this is the rate at which field lines cross the surface — the volume of fluid per unit time crossing $S$ if $\mathbf{F}$ is a velocity field.

**Example.** Compute the upward flux of $\mathbf{F} = (0, 0, z)$ through the surface $z = 1 - x^2 - y^2$ for $z \geq 0$ (a paraboloid cap).

On this surface, parametrise by $(x, y)$ in the disk $D : x^2 + y^2 \leq 1$.

The normal vector pointing upward:

$$d\mathbf{S} = \left(-\frac{\partial z}{\partial x}, -\frac{\partial z}{\partial y}, 1\right) dA = (2x, 2y, 1)\, dA$$

So $\mathbf{F} \cdot d\mathbf{S} = (0)(2x) + (0)(2y) + z \cdot 1 = z = 1 - x^2 - y^2$.

$$\iint_S \mathbf{F} \cdot d\mathbf{S} = \iint_D (1 - x^2 - y^2)\, dA$$

In polar coordinates ($x = r\cos\theta$, $y = r\sin\theta$):

$$= \int_0^{2\pi} \int_0^1 (1 - r^2)\, r\, dr\, d\theta = 2\pi \int_0^1 (r - r^3)\, dr = 2\pi \left[\frac{1}{2} - \frac{1}{4}\right] = \frac{\pi}{2}$$

## 43.5. The Divergence theorem

The Divergence theorem (Gauss's theorem) extends Green's theorem to three dimensions, relating the flux through a closed surface to the divergence integrated over the enclosed volume.

**Statement.** Let $V$ be a bounded region in $\mathbb{R}^3$ with piecewise smooth boundary $\partial V$ (oriented with outward normal). If $\mathbf{F}$ has continuous partial derivatives on an open set containing $V$, then:

$$\oiint_{\partial V} \mathbf{F} \cdot d\mathbf{S} = \iiint_V (\nabla \cdot \mathbf{F})\, dV$$

### 43.5.1. Physical interpretation

$\nabla \cdot \mathbf{F}$ at a point measures the net outward flow **per unit volume** — how much the field is "spreading out" there. The Divergence theorem says: sum up all that local spreading (the volume integral), and you get the total outward flow through the boundary (the surface integral). There is no other way out. Any flow created in the interior must eventually cross the boundary.

This is the mathematical statement of conservation laws. In fluid dynamics: if a fluid has no sources or sinks (incompressible, $\nabla \cdot \mathbf{F} = 0$), then the net flux through any closed surface is zero — what flows in must flow out.

### 43.5.2. Cellular decomposition in 3D

The argument is identical to Green's theorem but in 3D. Subdivide $V$ into tiny cubes. For each cube, compute the outward flux through its six faces. On each interior face, two adjacent cubes share that face — one uses it as an outward face, the other as an inward face. Their contributions cancel. What remains is the flux through the outer faces of $V$ only.

For a cube of volume $\Delta V$, the net outward flux through its faces approximates $(\nabla \cdot \mathbf{F})\,\Delta V$ (this is precisely the definition of divergence). Summing over all cubes gives the Divergence theorem.

---

```
//| echo: false
fluxBoxDemo = {
  const d3 = await require("d3@7");
  const W = 560, H = 380;

  const container = d3.create("div").style("max-width", W+"px").style("font-family", "sans-serif").s

  const controls = container.append("div").style("margin-bottom", "15px").style("padding", "10px").s

  function makeSlider(parent, label, initVal) {
    const wrap = parent.append("label").style("display", "flex").style("flex-direction", "column").s
    const top = wrap.append("div").style("display", "flex").style("justify-content", "space-between"
    top.append("span").text(label);
```

```
  const val = top.append("span").text(initVal.toFixed(1));
  const inp = wrap.append("input").attr("type", "range").attr("min", -5).attr("max", 5).attr("s
  return {inp, val};
}

const sFx = makeSlider(controls, "F\u2093", 2);
const sFy = makeSlider(controls, "F\u2098", -1);
const sFz = makeSlider(controls, "F\u2092", 3);

const layout = container.append("div").style("display", "flex").style("gap", "20px").style("ali

const svg = layout.append("svg").attr("width", 320).attr("height", 320).attr("viewBox", `0 0 32
const g = svg.append("g").attr("transform", `translate(160, 160)`);

const readout = layout.append("div").style("flex", 1).style("display", "flex").style("flex-dire

const colorScale = d3.scaleSequential(d3.interpolateRdBu).domain([20, -20]);

const isoX = (x, y, z) => (y - x) * 0.866;
const isoY = (x, y, z) => (x + y) * 0.5 - z;

const L = 80;

const faces = [
  {id: "Top", n: [0, 0, 1], pts: [[-L,-L,L], [L,-L,L], [L,L,L], [-L,L,L]]},
  {id: "Front", n: [1, 0, 0], pts: [[L,-L,-L], [L,-L,L], [L,L,L], [L,L,-L]]},
  {id: "Right", n: [0, 1, 0], pts: [[-L,L,-L], [L,L,-L], [L,L,L], [-L,L,L]]},
  {id: "Bottom", n: [0, 0, -1], pts: [[-L,-L,-L], [L,-L,-L], [L,L,-L], [-L,L,-L]], hidden: true
  {id: "Back", n: [-1, 0, 0], pts: [[-L,-L,-L], [-L,-L,L], [-L,L,L], [-L,L,-L]], hidden: true},
  {id: "Left", n: [0, -1, 0], pts: [[-L,-L,-L], [L,-L,-L], [L,-L,L], [-L,-L,L]], hidden: true}
];

const visibleFaces = faces.filter(f => !f.hidden);

const facePaths = visibleFaces.map(f => {
   const path = g.append("path").attr("stroke", "#334155").attr("stroke-width", 1.5).attr("stro
   const cx = d3.mean(f.pts, p => isoX(...p));
   const cy = d3.mean(f.pts, p => isoY(...p));
   const txt = g.append("text").attr("x", cx).attr("y", cy).attr("text-anchor", "middle").attr(
   return {path, txt, f};
});

function update() {
  const fx = +sFx.inp.property("value");
  const fy = +sFy.inp.property("value");
  const fz = +sFz.inp.property("value");
```

```
    sFx.val.text(fx.toFixed(1));
    sFy.val.text(fy.toFixed(1));
    sFz.val.text(fz.toFixed(1));

    readout.selectAll("*").remove();
    readout.append("strong").text("Flux Contributions:").style("margin-bottom", "4px").style("font-f

    let totalFlux = 0;
    const A = 4; // Area factor

    faces.forEach(f => {
       const flux = (fx*f.n[0] + fy*f.n[1] + fz*f.n[2]) * A;
       totalFlux += flux;

       const row = readout.append("div").style("display", "flex").style("justify-content", "space-be
       row.append("span").text(`${f.id}:`);
       row.append("span").text(flux > 0 ? `+${flux.toFixed(1)}` : flux.toFixed(1)).style("color", fl
    });

    readout.append("hr").style("width", "100%").style("border", "0.5px solid #cbd5e1").style("margin
    const totRow = readout.append("div").style("display", "flex").style("justify-content", "space-be
    totRow.append("span").text(`\u222F F \u00B7 dA:`);
    totRow.append("span").text(totalFlux.toFixed(1));

    const divRow = readout.append("div").style("display", "flex").style("justify-content", "space-be
    divRow.append("span").text(`(\u2207 \u00B7 F) \u00D7 Vol:`);
    divRow.append("span").text("0.0");

    const lineGen = d3.line().x(p => isoX(...p)).y(p => isoY(...p)).curve(d3.curveLinearClosed);

    facePaths.forEach(fp => {
       const flux = (fx*fp.f.n[0] + fy*fp.f.n[1] + fz*fp.f.n[2]) * A;
       fp.path.attr("d", lineGen(fp.f.pts)).attr("fill", colorScale(flux));
       fp.txt.text(flux > 0 ? `+${flux.toFixed(1)}` : flux.toFixed(1));
    });
  }

  sFx.inp.on("input", update);
  sFy.inp.on("input", update);
  sFz.inp.on("input", update);

  update();

  return container.node();
}
```

**Example.** Compute the flux of $\mathbf{F} = (x, y, z)$ outward through the sphere $x^2 + y^2 + z^2 = a^2$.

Direct computation is possible but requires parametrising the sphere. The Divergence theorem is far faster:

$\nabla \cdot \mathbf{F} = 1 + 1 + 1 = 3$.

Volume of the sphere: $V = \frac{4}{3}\pi a^3$.

$$\oiint_{\partial V} \mathbf{F} \cdot d\mathbf{S} = \iiint_V 3\, dV = 3 \cdot \frac{4}{3}\pi a^3 = 4\pi a^3$$

No surface integral needed — just multiplication.

**Gauss's law (Maxwell's equations).** The electric field $\mathbf{E}$ due to a charge distribution $\rho$ satisfies:

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0}$$

This is the differential form. Applying the Divergence theorem over a closed surface $\partial V$ enclosing volume $V$:

$$\oiint_{\partial V} \mathbf{E} \cdot d\mathbf{S} = \iiint_V \frac{\rho}{\varepsilon_0}\, dV = \frac{Q_{\text{enc}}}{\varepsilon_0}$$

where $Q_{\text{enc}}$ is the total enclosed charge. This is Gauss's law in integral form — the total electric flux through any closed surface equals the enclosed charge divided by $\varepsilon_0$. It is the Divergence theorem with $\mathbf{F} = \mathbf{E}$.

## 43.6. Stokes' theorem

Stokes' theorem is the 3D analogue of Green's theorem. It relates the line integral around a closed curve to the flux of the curl through any surface bounded by that curve.

**Statement.** Let $S$ be an oriented, piecewise smooth surface with boundary $\partial S$ — a closed curve. Let $\mathbf{F}$ have continuous partial derivatives on an open set containing $S$. Then:

$$\oint_{\partial S} \mathbf{F} \cdot d\mathbf{r} = \iint_S (\nabla \times \mathbf{F}) \cdot d\mathbf{S}$$

The orientation of $\partial S$ is determined by the right-hand rule: curl your right hand's fingers in the direction you traverse the boundary, and your thumb points in the direction of the surface normal.

### 43.6.1. Connection to Green's theorem

Green's theorem is the special case where $S$ is a flat region in the $xy$-plane. For $S \subset \{z = 0\}$ with normal $\hat{\mathbf{k}}$:

$$(\nabla \times \mathbf{F}) \cdot \hat{\mathbf{k}} = \frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y}$$

Substituting into Stokes' theorem recovers Green's theorem exactly. Stokes' theorem can be understood as "un-flattening" Green's theorem — lifting it from a 2D region to a curved surface in 3D.

### 43.6.2. The cellular argument for Stokes

Subdivide $S$ into small surface patches. The line integral around each patch's boundary traverses interior edges twice in opposite directions — those cancel. The sum over all patches reduces to the line integral around the outer boundary $\partial S$. For each patch, the boundary integral approximates $(\nabla \times \mathbf{F}) \cdot d\mathbf{S}$ (this is the definition of curl). Summing gives Stokes' theorem.

**Key insight:** the surface $S$ does not need to be flat, and it is not unique. Any two surfaces spanning the same boundary curve give the same flux integral of $\nabla \times \mathbf{F}$ — provided $\mathbf{F}$ is smooth throughout. Engineers exploit this: choose whichever surface makes the integral easiest.

**Example.** Verify Stokes' theorem for $\mathbf{F} = (-y^2, x, z^2)$ where $S$ is the upper hemisphere $x^2 + y^2 + z^2 = 1$, $z \geq 0$, and $\partial S$ is the unit circle in the $xy$-plane.

*Boundary integral:* on $\partial S$, parametrise $\mathbf{r}(t) = (\cos t, \sin t, 0)$:

$$\oint_{\partial S} \mathbf{F} \cdot d\mathbf{r} = \int_0^{2\pi} (-\sin^2 t, \cos t, 0) \cdot (-\sin t, \cos t, 0)\, dt$$

$$= \int_0^{2\pi} (\sin^3 t + \cos^2 t)\, dt = 0 + \pi = \pi$$

*Surface integral:* compute $\nabla \times \mathbf{F}$:

$$\nabla \times \mathbf{F} = \left( \frac{\partial z^2}{\partial y} - \frac{\partial x}{\partial z}, \frac{\partial(-y^2)}{\partial z} - \frac{\partial z^2}{\partial x}, \frac{\partial x}{\partial x} - \frac{\partial(-y^2)}{\partial y} \right) = (0, 0, 1 + 2y)$$

Applying Stokes' theorem is easier via the flat disk $D : x^2 + y^2 \leq 1, z = 0$ (same boundary, much simpler surface):

$$\iint_D (0, 0, 1 + 2y) \cdot (0, 0, 1)\, dA = \iint_D (1 + 2y)\, dA = \pi \cdot 1^2 + 2 \iint_D y\, dA = \pi + 0 = \pi$$

Both sides give $\pi$.

**Ampère's law in integral form.** The magnetic field $\mathbf{B}$ satisfies $\nabla \times \mathbf{B} = \mu_0 \mathbf{J}$ (Ampère's law, differential form). Stokes' theorem converts this to:

$$\oint_{\partial S} \mathbf{B} \cdot d\mathbf{r} = \iint_S (\nabla \times \mathbf{B}) \cdot d\mathbf{S} = \mu_0 \iint_S \mathbf{J} \cdot d\mathbf{S} = \mu_0 I_{\text{enc}}$$

The circulation of the magnetic field around a closed loop equals $\mu_0$ times the current threading through any surface bounded by that loop. This is the statement every electrical engineering student uses when computing fields around wires.

## 43.7. The unified picture

Green's, the Divergence theorem, and Stokes' are all instances of one principle, which can be stated abstractly as:

$$\int_{\partial\Omega} \omega = \int_{\Omega} d\omega$$

where $\omega$ is a differential form and $d\omega$ is its exterior derivative. The left side integrates over the boundary; the right side integrates the derivative over the interior. The Fundamental Theorem of Calculus, $\int_a^b f'(x)\,dx = f(b) - f(a)$, is also an instance — here the "boundary" of $[a, b]$ is just the two endpoints.

| Theorem | Domain | Boundary | Integrand (interior) |
|---|---|---|---|
| FTC | Interval $[a, b]$ | Two endpoints | $f'(x)$ |
| Green's | 2D region $D$ | Closed curve $\partial D$ | $\partial Q/\partial x - \partial P/\partial y$ |
| Stokes' | Surface $S$ | Closed curve $\partial S$ | $\nabla \times \mathbf{F}$ |
| Divergence | Volume $V$ | Closed surface $\partial V$ | $\nabla \cdot \mathbf{F}$ |

The pattern is inescapable. All of calculus, in one row of a table.

**Strategy: which way to go**
Given a problem, decide which direction the theorem should run:

- If the **boundary** integral looks hard but the **interior** is simple: Green's / Stokes' / Divergence converts boundary $\longrightarrow$ interior.
- If the **interior** integral looks hard but the **boundary** is simple: run it the other way.
- If $\nabla \cdot \mathbf{F} = 0$ (solenoidal): the flux through any closed surface is zero. No computation needed.
- If $\nabla \times \mathbf{F} = \mathbf{0}$ (irrotational): the line integral around any closed curve is zero. Use the potential function.

The theorems are not formulas to plug into — they are a choice of which representation of the problem is simpler.

## 43.8. Applications

### 43.8.1. Electric flux and Gauss's law

An isolated point charge $q$ sits at the origin. Compute the electric flux through a sphere of radius $r$.

The electric field is $\mathbf{E} = \frac{q}{4\pi\varepsilon_0 r^2}\hat{\mathbf{r}}$. On the sphere of radius $r$, the field is radial with constant magnitude $E = q/(4\pi\varepsilon_0 r^2)$, and the outward normal $\hat{\mathbf{n}} = \hat{\mathbf{r}}$, so $\mathbf{E} \cdot \hat{\mathbf{n}} = E$ everywhere.

$$\oiint_S \mathbf{E} \cdot d\mathbf{S} = E \cdot 4\pi r^2 = \frac{q}{4\pi\varepsilon_0 r^2} \cdot 4\pi r^2 = \frac{q}{\varepsilon_0}$$

This equals the enclosed charge divided by $\varepsilon_0$, confirming Gauss's law. Notice the $r$ cancelled — the flux is the same through any sphere centred on the charge. This is a consequence of the Divergence theorem and the fact that $\nabla \cdot \mathbf{E} = 0$ everywhere except at the charge itself.

### 43.8.2. Mass flow through a surface (fluid mechanics)

A velocity field $\mathbf{v} = (2x, -y, z)$ describes fluid flow in a region. Is mass accumulating inside the unit cube $[0, 1]^3$?

$\nabla \cdot \mathbf{v} = 2 - 1 + 1 = 2 > 0$.

By the Divergence theorem, the net outward mass flux is:

$$\oiint_{\partial V} \mathbf{v} \cdot d\mathbf{S} = \iiint_V 2\, dV = 2$$

Two units of mass per unit time are flowing out of the cube. The divergence is positive throughout, so the fluid is expanding — it acts like a source term. In an incompressible flow (water, to a first approximation), this is impossible: $\nabla \cdot \mathbf{v}$ must be zero everywhere.

### 43.8.3. Work done against gravity (conservative field)

Gravity is $\mathbf{g} = (0, 0, -g)$ with potential $\phi = gz$ (so $\mathbf{g} = -\nabla\phi$). A mass $m$ is moved along a complicated path from the ground level $(0, 0, 0)$ to a height $(x_1, y_1, h)$.

Since $\mathbf{F}_{\text{grav}} = m\mathbf{g} = -m\nabla(gz)$, it is conservative. Work done by gravity:

$$W = \int_C \mathbf{F}_{\text{grav}} \cdot d\mathbf{r} = -mgz\Big|_{z=0}^{z=h} = -mgh$$

The negative sign indicates gravity does negative work (you are lifting against it). Work done *against* gravity is $+mgh$ — the familiar result from introductory physics, now derived precisely from path independence.

### 43.8.4. Stokes' theorem and Faraday's law

Faraday's law of electromagnetic induction states $\nabla \times \mathbf{E} = -\partial \mathbf{B}/\partial t$. Apply Stokes' theorem to a surface $S$ bounded by a circuit loop $C$:

$$\oint_C \mathbf{E} \cdot d\mathbf{r} = \iint_S (\nabla \times \mathbf{E}) \cdot d\mathbf{S} = -\frac{d}{dt} \iint_S \mathbf{B} \cdot d\mathbf{S} = -\frac{d\Phi_B}{dt}$$

The EMF (voltage) around the circuit equals the negative rate of change of magnetic flux through it. This is the principle behind every transformer and generator ever built.

## 43.9. Exercises

These exercises build from evaluation to application. The first two establish mechanical fluency; the later ones require choosing which theorem to apply and why.

```
//| echo: false
{
  const katex = await require("katex@0.16");
  const css = document.createElement("link");
  css.rel = "stylesheet";
  css.href = "https://cdn.jsdelivr.net/npm/katex@0.16/dist/katex.min.css";
  document.head.appendChild(css);

  function makeStepperHTML(steps) {
    let current = 0;
    const totalSteps = steps.length;
    const container = document.createElement("div");
    container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem;

    const stepsDiv = document.createElement("div");
    stepsDiv.style.marginBottom = "0.75rem";

    function renderSteps() {
      stepsDiv.innerHTML = "";
      for (let i = 0; i < current; i++) {
        const s = steps[i];
        const row = document.createElement("div");
        row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.35rem 1rem; ali
        const opCell = document.createElement("span");
        opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
        opCell.textContent = s.op;
        const eqCell = document.createElement("span");
        eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false }
        row.appendChild(opCell);
        row.appendChild(eqCell);
```

```
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "□ Solution complete";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";

  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; b

  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; b

  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSte
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };

  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
```

```
  }

  // Attach to window so subsequent blocks can call it
  window._makeStepperHTML = makeStepperHTML;
  return html`<span></span>`;
}
```

**1. Line integral along a parametric curve.** A particle moves along the helix $\mathbf{r}(t) = (\cos t, \sin t, t)$ for $t \in [0, 2\pi]$ in the force field $\mathbf{F} = (-y, x, z)$. Compute the work done $W = \int_C \mathbf{F} \cdot d\mathbf{r}$.

```
//| echo: false
window._makeStepperHTML([
  {
    op: "Parametrise and compute r'(t)",
    eq: "\\mathbf{r}(t) = (\\cos t,\\, \\sin t,\\, t), \\quad \\mathbf{r}'(t) = (-\\sin t,\\, \\c
    note: "The derivative gives the tangent direction at each point on the helix."
  },
  {
    op: "Evaluate F along the curve",
    eq: "\\mathbf{F}(\\mathbf{r}(t)) = (-\\sin t,\\, \\cos t,\\, t)",
    note: "Substitute x = cos t, y = sin t, z = t into F = (-y, x, z)."
  },
  {
    op: "Form the dot product",
    eq: "\\mathbf{F} \\cdot \\mathbf{r}'(t) = (-\\sin t)(-\\sin t) + (\\cos t)(\\cos t) + t \\cdo
    note: "sin²t + cos²t = 1 simplifies the first two terms immediately."
  },
  {
    op: "Integrate over t ∈ [0, 2]",
    eq: "W = \\int_0^{2\\pi} (1 + t)\\, dt = \\left[t + \\frac{t^2}{2}\\right]_0^{2\\pi} = 2\\pi
    note: "A standard definite integral — no special technique needed."
  },
  {
    op: "Final answer",
    eq: "W = 2\\pi(1 + \\pi) \\approx 26.0 \\text{ (in consistent units)}",
    note: "The field does positive work on the particle throughout the helix."
  }
])
```

---

**2. Conservative field — find the potential, evaluate by endpoints.** Show that $\mathbf{F} = (2xy + z,\ x^2,\ x + 2z)$ is conservative, find its potential function $f$, and evaluate $\int_C \mathbf{F} \cdot d\mathbf{r}$ from $(0, 0, 0)$ to $(1, 2, 3)$ along any path.

```
//| echo: false
window._makeStepperHTML([
  {
    op: "Check curl F = 0",
    eq: "(\\nabla \\times \\mathbf{F})_x = \\partial_y(x + 2z) - \\partial_z(x^2) = 0 - 0 = 0",
    note: "Check all three components of the curl."
  },
  {
    op: "Check remaining curl components",
    eq: "(\\nabla \\times \\mathbf{F})_y = \\partial_z(2xy + z) - \\partial_x(x + 2z) = 1 - 1 = 0, \
    note: "All components are zero, so F is conservative."
  },
  {
    op: "Integrate P = 2xy + z with respect to x",
    eq: "f = \\int (2xy + z)\\, dx = x^2 y + xz + g(y, z)",
    note: "The 'constant' g can depend on y and z — treat them as constants when integrating in x."
  },
  {
    op: "Match ∂f/∂y = Q = x²",
    eq: "\\frac{\\partial f}{\\partial y} = x^2 + \\frac{\\partial g}{\\partial y} = x^2 \\implies \
    note: "g does not depend on y."
  },
  {
    op: "Match ∂f/∂z = R = x + 2z",
    eq: "\\frac{\\partial f}{\\partial z} = x + \\frac{\\partial g}{\\partial z} = x + 2z \\implies
    note: "g depends only on z."
  },
  {
    op: "Potential function",
    eq: "f(x, y, z) = x^2 y + xz + z^2",
    note: "Verify: ∇f = (2xy + z, x², x + 2z) = F. □"
  },
  {
    op: "Evaluate by Fundamental Theorem",
    eq: "\\int_C \\mathbf{F} \\cdot d\\mathbf{r} = f(1, 2, 3) - f(0, 0, 0) = (1 \\cdot 2 + 1 \\cdot
    note: "The path is irrelevant — only the endpoints matter for a conservative field."
  }
])
```

---

**3. Green's theorem — line integral to double integral.** Compute $\oint_C (x^2 - y)\,dx + (x + y^2)\,dy$ where $C$ is the boundary of the triangle with vertices $(0, 0)$, $(2, 0)$, $(0, 2)$, traversed anti-clockwise.

```
//| echo: false
window._makeStepperHTML([
```

```
  {
    op: "Identify P, Q and compute the integrand for Green's theorem",
    eq: "P = x^2 - y, \\quad Q = x + y^2, \\quad \\frac{\\partial Q}{\\partial x} - \\frac{\\part
    note: "Green's theorem converts the line integral to ∮_D 2 dA."
  },
  {
    op: "Describe the triangular region D",
    eq: "D: \\quad 0 \\leq x \\leq 2, \\quad 0 \\leq y \\leq 2 - x",
    note: "The hypotenuse has equation x + y = 2, so y goes from 0 to 2 − x."
  },
  {
    op: "Apply Green's theorem",
    eq: "\\oint_C (x^2 - y)\\, dx + (x + y^2)\\, dy = \\iint_D 2 \\, dA = 2 \\cdot \\text{Area}(D
    note: "The integrand is constant (2), so the double integral is just 2 times the area."
  },
  {
    op: "Compute the area of the triangle",
    eq: "\\text{Area}(D) = \\frac{1}{2} \\cdot \\text{base} \\cdot \\text{height} = \\frac{1}{2}
    note: "Or integrate: ∫₀² (2−x) dx = [2x − x²/2]₀² = 4 − 2 = 2."
  },
  {
    op: "Final answer",
    eq: "\\oint_C (x^2 - y)\\, dx + (x + y^2)\\, dy = 2 \\cdot 2 = 4",
    note: "Evaluating the line integral directly over the three edges would require three separat
  }
])
```

---

**4. Electric flux through a hemisphere.** Compute the upward flux of $\mathbf{E} = (x, y, 2z)$ through the upper hemisphere $S : x^2 + y^2 + z^2 = 9, z \geq 0$ (with outward normal pointing away from the origin).

```
//| echo: false
window._makeStepperHTML([
  {
    op: "Plan: close the surface and use the Divergence theorem",
    eq: "\\text{Let } V = \\{x^2 + y^2 + z^2 \\leq 9,\\; z \\geq 0\\} \\text{ (upper hemisphere +
    note: "The closed surface ∂V = S (hemisphere) ∪ D (disk). We compute the total and subtract t
  },
  {
    op: "Compute the divergence",
    eq: "\\nabla \\cdot \\mathbf{E} = \\frac{\\partial x}{\\partial x} + \\frac{\\partial y}{\\pa
    note: "The divergence is constant, so the volume integral is just 4 times the volume."
  },
  {
    op: "Volume of the upper hemisphere",
```

```
    eq: "V = \\frac{1}{2} \\cdot \\frac{4}{3}\\pi (3)^3 = \\frac{2}{3}\\pi \\cdot 27 = 18\\pi",
    note: "Half the volume of a sphere of radius 3."
  },
  {
    op: "Total flux through ∂V by Divergence theorem",
    eq: "\\oiint_{\\partial V} \\mathbf{E} \\cdot d\\mathbf{S} = \\iiint_V 4 \\, dV = 4 \\cdot 18\\p
    note: "This is the combined flux through the hemisphere and the disk."
  },
  {
    op: "Flux through the disk at z = 0 (downward normal n̂ = −k̂ )",
    eq: "\\iint_D \\mathbf{E} \\cdot (-\\hat{\\mathbf{k}})\\, dA = \\iint_D -2z\\, dA = \\iint_D 0\\
    note: "On the disk z = 0, so the component 2z = 0. The disk contributes nothing."
  },
  {
    op: "Flux through the hemisphere",
    eq: "\\iint_S \\mathbf{E} \\cdot d\\mathbf{S} = 72\\pi - 0 = 72\\pi",
    note: "The total equals the hemisphere's contribution alone."
  }
])
```

---

**5. Divergence theorem — convert surface integral to volume integral.** Let $\mathbf{F} = (x^3, y^3, z^3)$. Compute the outward flux of $\mathbf{F}$ through the closed surface of the cylinder $x^2 + y^2 \leq 4$, $0 \leq z \leq 3$.

```
//| echo: false
window._makeStepperHTML([
  {
    op: "Compute the divergence",
    eq: "\\nabla \\cdot \\mathbf{F} = 3x^2 + 3y^2 + 3z^2",
    note: "The closed surface consists of the curved side plus two disks — direct integration would
  },
  {
    op: "Set up the volume integral in cylindrical coordinates",
    eq: "\\iiint_V (3x^2 + 3y^2 + 3z^2)\\, dV = \\int_0^{2\\pi} \\int_0^2 \\int_0^3 (3r^2 + 3z^2)\\
    note: "In cylindrical coords: x² + y² = r², dV = r dr d dz."
  },
  {
    op: "Integrate over z first",
    eq: "\\int_0^3 (3r^2 + 3z^2)\\, dz = \\left[3r^2 z + z^3\\right]_0^3 = 9r^2 + 27",
    note: "Treat r as a constant when integrating in z."
  },
  {
    op: "Integrate over r",
    eq: "\\int_0^2 (9r^2 + 27)\\, r\\, dr = \\int_0^2 (9r^3 + 27r)\\, dr = \\left[\\frac{9r^4}{4} +
    note: ""
```

```
  },
  {
    op: "Integrate over □",
    eq: "\\int_0^{2\\pi} 90\\, d\\theta = 180\\pi",
    note: "The integrand is independent of □, so the angular integral just multiplies by 2□."
  },
  {
    op: "Final answer",
    eq: "\\oiint_{\\partial V} \\mathbf{F} \\cdot d\\mathbf{S} = 180\\pi \\approx 565.5",
    note: "The Divergence theorem reduced a three-part surface integral to one cylindrical volume
  }
])
```

---

**6. Stokes' theorem — Ampère's law application.** A long straight wire carries current $I$ along the $z$-axis. The magnetic field is $\mathbf{B} = \frac{\mu_0 I}{2\pi(x^2+y^2)}(-y, x, 0)$. Use Stokes' theorem to compute $\oint_C \mathbf{B} \cdot d\mathbf{r}$ where $C$ is the circle $x^2 + y^2 = R^2$ in the $xy$-plane.

```
//| echo: false
window._makeStepperHTML([
  {
    op: "Parametrise the circle C",
    eq: "\\mathbf{r}(t) = (R\\cos t,\\, R\\sin t,\\, 0), \\quad d\\mathbf{r} = (-R\\sin t,\\, R\\
    note: "We evaluate directly first, then verify via Stokes' theorem."
  },
  {
    op: "Evaluate B on C (where x² + y² = R²)",
    eq: "\\mathbf{B} = \\frac{\\mu_0 I}{2\\pi R^2}(-R\\sin t,\\, R\\cos t,\\, 0)",
    note: "Substitute x = R cos t, y = R sin t into B."
  },
  {
    op: "Compute the dot product B · dr",
    eq: "\\mathbf{B} \\cdot d\\mathbf{r} = \\frac{\\mu_0 I}{2\\pi R^2}\\left(R^2\\sin^2 t + R^2\\
    note: "The R² factors cancel; sin²t + cos²t = 1."
  },
  {
    op: "Integrate over t ∈ [0, 2□]",
    eq: "\\oint_C \\mathbf{B} \\cdot d\\mathbf{r} = \\int_0^{2\\pi} \\frac{\\mu_0 I}{2\\pi}\\, dt
    note: "The result is independent of R — the circulation is the same around any circle enclosi
  },
  {
    op: "Interpret via Stokes' theorem (Ampère's law)",
    eq: "\\oint_C \\mathbf{B} \\cdot d\\mathbf{r} = \\iint_S (\\nabla \\times \\mathbf{B}) \\cdot
    note: "Stokes' theorem links the circulation of B to the current threading through any surfac
  },
```

```
  {
    op: "Conclusion",
    eq: "\\oint_C \\mathbf{B} \\cdot d\\mathbf{r} = \\mu_0 I",
    note: "This is Ampère's law in integral form. The radius R cancelled because the current is enti
  }
])
```

## 43.10. Where this goes

The three theorems in this chapter are the operational heart of classical field theory. The next chapter, Fourier series and PDEs, puts them to work in a new setting: instead of integrating over a domain once, you decompose a function into sinusoidal modes and work with each mode independently. The Divergence theorem reappears immediately in the derivation of the heat equation and the wave equation — those PDEs arise precisely from applying it to a tiny control volume and letting the volume shrink to a point.

Complex analysis (the chapter after that) makes this picture even stranger and more powerful: in the complex plane, the Cauchy integral theorem is Green's theorem in disguise, and the residue theorem for computing integrals is a direct consequence of Stokes' theorem applied to a deformed contour.

The unifying thread is the generalised Stokes' theorem $\int_{\partial\Omega} \omega = \int_\Omega d\omega$. Once you have seen that formulation, you cannot unsee it — every integration theorem you encounter from here on is a special case.

**Where this shows up**

- A mechanical engineer computing the torque on a rotating shaft uses the surface integral of the stress tensor — a generalisation of flux.
- A geophysicist inverting gravity survey data uses Gauss's theorem to relate surface measurements to subsurface mass distribution.
- A climate scientist computing net $CO_2$ flux into the ocean integrates the divergence of a diffusion field over a volume — the Divergence theorem makes global estimates tractable.
- An electrical engineer designing an antenna housing uses Gauss's law (Divergence theorem) to verify shielding effectiveness from bulk material properties rather than surface-by-surface calculation.

The theorems do not just simplify calculation. In each case, they are the reason the physical law can be stated locally (as a PDE or a differential form) *and* globally (as a conservation statement). That equivalence is not obvious — it is exactly what Green, Stokes, and Gauss proved.

# Part IX.

# Volume 7 — Fourier analysis and PDEs

# 44. Fourier series

Every periodic signal is a sum of pure frequencies

The Fourier series starts from a single observation: the simplest periodic functions are sines and cosines. They oscillate at a fixed frequency, they never change shape, and their behaviour is completely described by amplitude and phase. Every other periodic function — however jagged, however asymmetric — can be built from them by superposition. The question is: how do you find the right mix?

The answer is integration. Multiply the function by each candidate sine or cosine and integrate over one period. The result, as we will show, is exactly the amplitude of that frequency component. Everything else cancels out.

---

## 44.1. Periodic functions

A function $f$ is **periodic with period** $T$ if $f(x+T) = f(x)$ for all $x$. The smallest such $T > 0$ is called the **fundamental period**.

We write $T = 2L$, which places one full period over the interval $[-L, L]$. This notation is standard in engineering texts and makes the coefficient formulas symmetric.

Familiar examples: $\sin(nx)$ and $\cos(nx)$ have period $2\pi/n$; the square wave (defined below) has the same period as the fundamental sine used to construct it.

**The idea geometrically.** A periodic function is completely determined by its values on any interval of length $T$. Knowing one period is knowing everything. The Fourier series asks: which sum of sines and cosines, with the same period, most accurately represents $f$?

---

## 44.2. The Fourier series

For a function $f$ of period $2L$, the **Fourier series** is:

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} \left[ a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right]$$

The **Fourier coefficients** are:

$$a_0 = \frac{1}{L} \int_{-L}^{L} f(x)\, dx$$

$$a_n = \frac{1}{L} \int_{-L}^{L} f(x) \cos\left(\frac{n\pi x}{L}\right) dx \qquad (n = 1, 2, 3, \ldots)$$

$$b_n = \frac{1}{L} \int_{-L}^{L} f(x) \sin\left(\frac{n\pi x}{L}\right) dx \qquad (n = 1, 2, 3, \ldots)$$

The term $a_0/2$ is the mean value of $f$ over one period. The $a_n$ terms give the amplitudes of the cosine components; the $b_n$ terms give the amplitudes of the sine components.

### 44.2.1. Why these formulas: orthogonality

The formulas above are not guessed — they follow from a single property of the trigonometric functions: **orthogonality**.

Over the interval $[-L, L]$:

$$\int_{-L}^{L} \cos\left(\frac{m\pi x}{L}\right) \cos\left(\frac{n\pi x}{L}\right) dx = \begin{cases} 0 & m \neq n \\ L & m = n \geq 1 \\ 2L & m = n = 0 \end{cases}$$

$$\int_{-L}^{L} \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi x}{L}\right) dx = \begin{cases} 0 & m \neq n \\ L & m = n \geq 1 \end{cases}$$

$$\int_{-L}^{L} \cos\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi x}{L}\right) dx = 0 \qquad \text{for all } m, n$$

These follow from the product-to-sum identities. Take the $m \neq n$ case of the cosine integral as an example: the identity $\cos A \cos B = \frac{1}{2}[\cos(A - B) + \cos(A + B)]$ turns the product into a sum of two cosines at frequencies $(m - n)\pi/L$ and $(m + n)\pi/L$. Integrating either of those over exactly one complete period gives zero — the positive and negative half-cycles cancel. When $m = n$ the first term becomes $\cos(0) = 1$, which integrates to $2L$, accounting for the factor of $L$ in the diagonal case. The sine-sine and cosine-sine cases follow by the same argument with the analogous identities.

They mean that multiplying the Fourier series by $\cos(m\pi x/L)$ and integrating over $[-L, L]$ kills every term except the one with $n = m$ — which is exactly how $a_m$ is isolated. This means orthogonality is the filtering mechanism behind the coefficient formulas: integration keeps the matching frequency and cancels the rest.

Think of the trigonometric functions as a set of directions in function space that are pairwise perpendicular. The Fourier coefficients are the projections of $f$ onto those directions.

---

## 44.3. Worked example: the square wave

Define the square wave with period $2\pi$ as:

$$f(x) = \begin{cases} 1 & 0 < x < \pi \\ -1 & -\pi < x < 0 \end{cases}$$

extended periodically to all $x$.

**Step 1: compute $a_0$.**

$$a_0 = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\,dx = \frac{1}{\pi}\left[\int_{-\pi}^{0}(-1)\,dx + \int_{0}^{\pi}(1)\,dx\right] = \frac{1}{\pi}[-\pi + \pi] = 0$$

The mean value is zero — the positive and negative halves cancel.

**Step 2: compute $a_n$.**

$f(x)$ is an **odd** function: $f(-x) = -f(x)$. The cosines $\cos(nx)$ are **even** functions. An odd function times an even function is odd, and the integral of an odd function over a symmetric interval is zero:

$$a_n = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\cos(nx)\,dx = 0 \quad \text{for all } n \geq 1$$

**Step 3: compute $b_n$.**

$\sin(nx)$ is odd; $f$ is odd; their product is even:

$$b_n = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\sin(nx)\,dx = \frac{2}{\pi}\int_{0}^{\pi}\sin(nx)\,dx$$

$$= \frac{2}{\pi}\left[-\frac{\cos(nx)}{n}\right]_0^{\pi} = \frac{2}{n\pi}\left[1 - \cos(n\pi)\right] = \frac{2}{n\pi}\left[1 - (-1)^n\right]$$

Here $\cos(n\pi) = (-1)^n$ for any integer $n$: the cosine alternates between $-1$ and $1$ at multiples of $\pi$. When $n$ is even, $\cos(n\pi) = 1$, so $b_n = 0$. When $n$ is odd, $\cos(n\pi) = -1$, so $b_n = 4/(n\pi)$.

**Result:**

$$f(x) = \frac{4}{\pi}\left[\sin(x) + \frac{\sin(3x)}{3} + \frac{\sin(5x)}{5} + \frac{\sin(7x)}{7} + \cdots\right]$$

The partial sums progressively sharpen toward the square wave, with the edges becoming steeper as more terms are included.

```
//| echo: false
{
  // ── Fourier partial sum builder with phasor panel ──────────────────────
  // Panel A: waveform reconstruction  Panel B: frequency spectrum
  // Panel C: phasor wheel (inset)
  const d3 = await require("d3@7");

  // Palette
  const AMBER = "#f59e0b";
  const BLUE  = "#6b90c4";
  const GREY  = "#9ca3af";
  const GREY_FAINT = "#e5e7eb";
  const RED   = "#ef4444";

  // Dimensions
  const WA = 520, HA = 180;   // Panel A waveform
  const WB = 520, HB = 90;    // Panel B spectrum
  const WC = 130, HC = 130;   // Panel C phasor
  const margin = { t: 16, r: 16, b: 32, l: 40 };

  // ── Wave functions (period 2□, L = □) ──────────────────────────────────
  function targetSquare(x) { return x > 0 ? 1 : -1; }
  function targetSawtooth(x) { return x / Math.PI; }
  function targetTriangle(x) {
    const t = ((x + Math.PI) % (2 * Math.PI) + 2 * Math.PI) % (2 * Math.PI) - Math.PI;
    return 1 - (2 / Math.PI) * Math.abs(t);
  }

  // ── Fourier coefficients (b□ only for odd, pure sine series) ───────────
  function bCoeff(n, waveType) {
    if (waveType === "square") {
      return n % 2 === 1 ? 4 / (n * Math.PI) : 0;
    } else if (waveType === "sawtooth") {
      return (n % 2 === 0 ? -1 : 1) * 2 / n;  // alternate signs: 2(-1)^{n+1}/n
      // Correct: b_n = 2(-1)^{n+1}/n
    } else {  // triangle
      if (n % 2 === 0) return 0;
      const k = (n - 1) / 2;
      return (k % 2 === 0 ? 8 : -8) / (n * n * Math.PI * Math.PI);
    }
  }
```

```
// ── Partial sum ──────────────────────────────────────────────────────
function partialSum(x, N, waveType) {
  let s = 0;
  for (let n = 1; n <= N; n++) s += bCoeff(n, waveType) * Math.sin(n * x);
  return s;
}

// Gibbs overshoot x-position (first peak of partial sum near x=0)
function gibbsPeakX(N) {
  return Math.PI / N;  // first peak near jump at x=0 (actually π/(N+1) approx)
}

// ── Viewof controls ──────────────────────────────────────────────────
const container = document.createElement("div");
container.style.cssText = "font-family:inherit; max-width:620px;";

// Controls row
const ctrlRow = document.createElement("div");
ctrlRow.style.cssText = "display:flex; gap:1.5rem; align-items:center; margin-bottom:0.5rem; fl

// N slider
const sliderLabel = document.createElement("label");
sliderLabel.style.cssText = "font-size:0.88em; color:#374151; display:flex; align-items:center;
sliderLabel.textContent = "Terms N = ";
const nDisplay = document.createElement("strong");
nDisplay.style.cssText = "color:" + AMBER + "; min-width:2ch;";
nDisplay.textContent = "5";
const nSlider = document.createElement("input");
nSlider.type = "range"; nSlider.min = 1; nSlider.max = 30; nSlider.value = 5;
nSlider.style.cssText = "width:160px; accent-color:" + AMBER + ";";
sliderLabel.appendChild(nSlider);
sliderLabel.appendChild(nDisplay);
ctrlRow.appendChild(sliderLabel);

// Wave type radio
const waveTypes = [["square","Square"], ["sawtooth","Sawtooth"], ["triangle","Triangle"]];
const radioGroup = document.createElement("div");
radioGroup.style.cssText = "display:flex; gap:0.75rem; font-size:0.88em;";
let currentWave = "square";
waveTypes.forEach(([val, lab]) => {
  const lbl = document.createElement("label");
  lbl.style.cssText = "display:flex; align-items:center; gap:0.25rem; cursor:pointer;";
  const r = document.createElement("input");
  r.type = "radio"; r.name = "waveViz1"; r.value = val;
  r.checked = (val === "square");
  r.style.accentColor = AMBER;
  r.onchange = () => { currentWave = val; draw(); };
```

```
  lbl.appendChild(r); lbl.appendChild(document.createTextNode(lab));
  radioGroup.appendChild(lbl);
});
ctrlRow.appendChild(radioGroup);
container.appendChild(ctrlRow);

// ── SVG canvas ──────────────────────────────────────────────────────
const totalW = WA + 16 + WC;  // waveform + gap + phasor
const totalH = HA + 8 + HB + margin.b;
const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${totalW + margin.l + margin.r} ${totalH + margin.t}`)
  .attr("width", "100%")
  .style("display","block");

const gA = svg.append("g").attr("transform", `translate(${margin.l},${margin.t})`);
const gB = svg.append("g").attr("transform", `translate(${margin.l},${margin.t + HA + 8})`);
const gC = svg.append("g").attr("transform", `translate(${margin.l + WA + 16},${margin.t})`);

// ── Panel A setup ───────────────────────────────────────────────────
const xA = d3.scaleLinear().domain([-Math.PI, Math.PI]).range([0, WA]);
const yA = d3.scaleLinear().domain([-1.3, 1.3]).range([HA, 0]);

gA.append("line").attr("x1",0).attr("x2",WA).attr("y1",yA(0)).attr("y2",yA(0))
  .attr("stroke", GREY_FAINT).attr("stroke-width",1);
gA.append("g").call(d3.axisBottom(xA).tickValues([-Math.PI,-Math.PI/2,0,Math.PI/2,Math.PI])
  .tickFormat(d => {
    const m = {"-3.14159":"−π","-1.5708":"−π/2","0":"0","1.5708":"π/2","3.14159":"π"};
    return m[d.toFixed(5)] || "";
  })).attr("transform", `translate(0,${HA})`)
  .call(g => { g.select(".domain").attr("stroke",GREY_FAINT); g.selectAll("text").style("font-size
gA.append("g").call(d3.axisLeft(yA).ticks(5))
  .call(g => { g.select(".domain").remove(); g.selectAll("text").style("font-size","10px").attr("f
gA.append("text").attr("x", WA/2).attr("y", -4).attr("text-anchor","middle")
  .attr("font-size","11px").attr("fill","#374151").text("Waveform reconstruction");

// Target path (blue, drawn once per wave change)
const targetPath = gA.append("path").attr("fill","none").attr("stroke",BLUE)
  .attr("stroke-width",2).attr("stroke-dasharray","4,3").attr("opacity",0.7);
// Partial sum path (amber)
const sumPath = gA.append("path").attr("fill","none").attr("stroke",AMBER).attr("stroke-width",2.5
// Gibbs annotation
const gibbsLine = gA.append("line").attr("stroke","#dc2626").attr("stroke-width",1)
  .attr("stroke-dasharray","3,2").style("display","none");
const gibbsLabel = gA.append("text").attr("font-size","9.5px").attr("fill","#dc2626")
  .style("display","none");
// N label
const nLabel = gA.append("text").attr("x",8).attr("y",14).attr("font-size","11px")
```

```
  .attr("fill",AMBER).attr("font-weight","bold");

// ── Panel B setup ──────────────────────────────────────────────────
const xB = d3.scaleLinear().domain([0.5, 30.5]).range([0, WB]);
const yBmax = 1.4;
const yB = d3.scaleLinear().domain([0, yBmax]).range([HB - 22, 0]);

gB.append("text").attr("x",WB/2).attr("y",-4).attr("text-anchor","middle")
  .attr("font-size","11px").attr("fill","#374151").text("Frequency spectrum |b⃗|");
gB.append("g").call(d3.axisBottom(xB).ticks(10).tickFormat(d => Number.isInteger(d) ? d : ""))
  .attr("transform", `translate(0,${HB-22})`)
  .call(g => { g.select(".domain").attr("stroke",GREY_FAINT); g.selectAll("text").style("font-s
gB.append("text").attr("x",WB+2).attr("y",HB-22+4).attr("font-size","9px").attr("fill",GREY).te

// ── Panel C setup (phasor wheel) ──────────────────────────────────
const cx = WC / 2, cy = HC / 2, r0 = WC * 0.38;
gC.append("circle").attr("cx",cx).attr("cy",cy).attr("r",r0)
  .attr("fill","none").attr("stroke",GREY_FAINT).attr("stroke-width",1.5);
gC.append("line").attr("x1",cx).attr("x2",cx).attr("y1",cy-r0-4).attr("y2",cy+r0+4)
  .attr("stroke",GREY_FAINT).attr("stroke-width",0.5);
gC.append("line").attr("x1",cx-r0-4).attr("x2",cx+r0+4).attr("y1",cy).attr("y2",cy)
  .attr("stroke",GREY_FAINT).attr("stroke-width",0.5);
gC.append("text").attr("x",WC/2).attr("y",HC-2).attr("text-anchor","middle")
  .attr("font-size","9px").attr("fill",GREY).text("Phasor wheel");

// Phasor arrows (up to 3 harmonics for square wave)
const phasorArrows = [1,3,5].map((_, i) => ({
  line: gC.append("line").attr("stroke", i===0 ? AMBER : i===1 ? BLUE : "#a78bfa")
          .attr("stroke-width", 2-i*0.4).attr("marker-end","url(#arrowC)"),
  label: gC.append("text").attr("font-size","8px").attr("fill", i===0 ? AMBER : i===1 ? BLUE :
}));
// Arrow marker
svg.append("defs").append("marker").attr("id","arrowC").attr("markerWidth",6).attr("markerHeigh
  .attr("refX",5).attr("refY",3).attr("orient","auto")
  .append("path").attr("d","M0,0 L0,6 L6,3 z").attr("fill",AMBER);
// Projection dot
const projDot = gC.append("circle").attr("r",4).attr("fill",RED);
// Tip trace path
const traceData = [];
const tracePath = gC.append("path").attr("fill","none").attr("stroke",RED)
  .attr("stroke-width",1).attr("opacity",0.4);

// ── Animation state ────────────────────────────────────────────────
let theta = 0;
let animFrame = null;

function getNHarmonics() {
```

```
  const N = +nSlider.value;
  if (currentWave === "square") {
    const odds = [1,3,5,7,9,11,13,15,17,19,21,23,25,27,29].filter(n => n <= N);
    return odds.slice(0, 3);
  } else if (currentWave === "sawtooth") {
    return [1,2,3].filter(n => n <= N);
  } else {
    const odds = [1,3,5,7,9,11].filter(n => n <= N);
    return odds.slice(0, 3);
  }
}

function drawBars(N) {
  const barW = Math.max(1, WB / 32);
  gB.selectAll(".specbar").remove();
  for (let n = 1; n <= 30; n++) {
    const coeff = Math.abs(bCoeff(n, currentWave));
    const active = n <= N;
    gB.append("rect").attr("class","specbar")
      .attr("x", xB(n) - barW/2).attr("y", active ? yB(coeff) : HB-22)
      .attr("width", barW).attr("height", active ? (HB-22) - yB(coeff) : 0)
      .attr("fill", active ? AMBER : "none")
      .attr("stroke", active ? AMBER : GREY_FAINT)
      .attr("stroke-width", active ? 0 : 1)
      .attr("opacity", active ? 1 : 0.5);
    if (!active && coeff > 0.02) {
      // ghost outline
      const ghostH = Math.max(1, (HB-22) - yB(coeff));
      gB.append("rect").attr("class","specbar")
        .attr("x", xB(n) - barW/2).attr("y", yB(coeff))
        .attr("width", barW).attr("height", ghostH)
        .attr("fill","none").attr("stroke",GREY_FAINT).attr("stroke-width",1);
    }
  }
}

function draw() {
  const N = +nSlider.value;
  nDisplay.textContent = N;
  nLabel.text(`N = ${N} terms`);

  const nPts = 400;
  const xs = d3.range(nPts).map(i => -Math.PI + (2*Math.PI)*i/(nPts-1));

  // Target
  const target = currentWave === "square" ? targetSquare :
                 currentWave === "sawtooth" ? targetSawtooth : targetTriangle;
```

```
  const lineGen = d3.line().x(d => xA(d[0])).y(d => yA(d[1]));
  targetPath.datum(xs.map(x => [x, target(x) || 0])).attr("d", lineGen);

  // Partial sum
  sumPath.datum(xs.map(x => [x, partialSum(x, N, currentWave)])).attr("d", lineGen);

  // Gibbs annotation (square wave only, N >= 5)
  if (currentWave === "square" && N >= 5) {
    const GIBBS_Y = 1.089;
    gibbsLine.style("display","block")
      .attr("x1",0).attr("x2",WA).attr("y1",yA(GIBBS_Y)).attr("y2",yA(GIBBS_Y));
    gibbsLabel.style("display","block")
      .attr("x",WA-4).attr("y",yA(GIBBS_Y)-3)
      .attr("text-anchor","end").text("□ 8.9% overshoot");
  } else {
    gibbsLine.style("display","none");
    gibbsLabel.style("display","none");
  }

  drawBars(N);
}

function animatePhasor() {
  const N = +nSlider.value;
  const harmonics = getNHarmonics();
  theta += 0.025;

  // Draw phasor arrows (chain from centre)
  let px = cx, py = cy;
  const tips = [];
  harmonics.forEach((n, i) => {
    const coeff = Math.abs(bCoeff(n, currentWave));
    const len = coeff / 1.5 * r0;  // scale to fit
    const angle = -(n * theta) - Math.PI/2;  // -□/2 so n=1 starts pointing up
    const ex = px + len * Math.cos(angle);
    const ey = py + len * Math.sin(angle);
    phasorArrows[i].line
      .attr("x1", px).attr("y1", py)
      .attr("x2", ex).attr("y2", ey)
      .style("display", n <= N ? "block" : "none");
    phasorArrows[i].label
      .attr("x", ex + 4).attr("y", ey)
      .text(n <= N ? `n=${n}` : "")
      .style("display", n <= N ? "block" : "none");
    px = ex; py = ey;
    tips.push([ex, ey]);
  });
```

```
    // Hide unused arrows
    for (let i = harmonics.length; i < 3; i++) {
      phasorArrows[i].line.style("display","none");
      phasorArrows[i].label.style("display","none");
    }

    // Projection onto real axis (horizontal = Re)
    const tipX = tips[tips.length-1][0];
    const tipY = tips[tips.length-1][1];
    projDot.attr("cx", cx).attr("cy", tipY);  // project to vertical axis (Im → 0 = Re projection)

    // Trace tip
    traceData.push([tipX, tipY]);
    if (traceData.length > 80) traceData.shift();
    tracePath.datum(traceData).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]).curve(d3.curveLinear));

    animFrame = requestAnimationFrame(animatePhasor);
  }

  nSlider.oninput = () => { draw(); };

  container.appendChild(svg.node());

  // Legend
  const legend = document.createElement("div");
  legend.style.cssText = "display:flex; gap:1rem; font-size:0.8em; margin-top:0.25rem; flex-wrap:wra
  [["—— Partial sum S_N(x)", AMBER], ["--- Target function", BLUE], ["| Gibbs limit (square)", "#d
    .forEach(([lab, col]) => {
      const s = document.createElement("span");
      s.style.cssText = `color:${col};`;
      s.textContent = lab;
      legend.appendChild(s);
    });
  container.appendChild(legend);

  draw();
  animatePhasor();

  // Cleanup on disconnect
  const obs = new MutationObserver(() => {
    if (!document.contains(container)) { cancelAnimationFrame(animFrame); obs.disconnect(); }
  });
  obs.observe(document.body, {childList:true, subtree:true});

  return container;
}
```

## 44.4. Convergence and the Gibbs phenomenon

The Fourier series of a well-behaved periodic function converges in the following sense.

**Dirichlet conditions.** If $f$ is periodic, bounded, and piecewise continuous (at most finitely many jump discontinuities per period, and finitely many maxima and minima per period), then:

- At points where $f$ is continuous, the Fourier series converges to $f(x)$.
- At a jump discontinuity $x_0$, the Fourier series converges to the average of the left and right limits: $\frac{1}{2}\left[f(x_0^-) + f(x_0^+)\right]$.

**Gibbs phenomenon.** Near a jump discontinuity, the partial sums overshoot the true value by approximately 8.9% of the jump height, regardless of how many terms are included. Adding more terms moves the overshoot closer to the discontinuity but does not reduce its size. This is not an error in the method — it is a fundamental property of Fourier partial sums near discontinuities. It appears in real systems as **ringing** — the ripple artefact you see in a sharply filtered audio signal or a steeply edge-enhanced image.

For the square wave with a jump of magnitude 2, the Gibbs overshoot is approximately $2 \times 0.089 \approx 0.18$.

## 44.5. Even and odd functions

Identifying the symmetry of $f$ before computing is practical, not just elegant: it cuts the integration work in half.

**Even function** ($f(-x) = f(x)$, symmetric about the $y$-axis): odd-function integrands vanish, so $b_n = 0$ for all $n$. The Fourier series is a **cosine series**:

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} a_n \cos\left(\frac{n\pi x}{L}\right)$$

**Odd function** ($f(-x) = -f(x)$, antisymmetric about the $y$-axis): even-function integrands vanish, so $a_n = 0$ for all $n$. The Fourier series is a **sine series**:

$$f(x) = \sum_{n=1}^{\infty} b_n \sin\left(\frac{n\pi x}{L}\right)$$

The square wave example above was odd, which is why we only needed to compute $b_n$.

## 44.6. Half-range series

Often $f$ is defined only on $[0, L]$ — for example, the initial temperature profile in a rod. We cannot compute a full Fourier series without knowing $f$ on $[-L, 0]$, but we can choose how to extend it:

- **Even extension**: define $f(-x) = f(x)$ on $[-L, 0]$. This produces a **Fourier cosine series** with only $a_n$ terms:

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} a_n \cos\left(\frac{n\pi x}{L}\right), \quad a_n = \frac{2}{L}\int_0^L f(x)\cos\left(\frac{n\pi x}{L}\right) dx$$

- **Odd extension**: define $f(-x) = -f(x)$ on $[-L, 0]$. This produces a **Fourier sine series** with only $b_n$ terms:

$$f(x) = \sum_{n=1}^{\infty} b_n \sin\left(\frac{n\pi x}{L}\right), \quad b_n = \frac{2}{L}\int_0^L f(x)\sin\left(\frac{n\pi x}{L}\right) dx$$

**Which to choose?** The boundary conditions of the physical problem decide: if $f(0) = f(L) = 0$ is required (as in a fixed-end vibrating string or a rod with fixed-temperature endpoints), use the sine extension. If the derivative $f'(0) = f'(L) = 0$ is required (insulated endpoints), use the cosine extension.

For heat problems, "insulated" means no heat flux through the boundary. Because heat flux is proportional to the temperature gradient, that gives the condition $u_x = 0$ at the endpoints.

---

## 44.7. Complex Fourier series

Using Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$, the Fourier series can be written in complex exponential form:

$$f(x) = \sum_{n=-\infty}^{\infty} c_n\, e^{in\pi x/L}$$

where the **complex Fourier coefficients** are:

$$c_n = \frac{1}{2L}\int_{-L}^{L} f(x)\, e^{-in\pi x/L}\, dx$$

The relationship to the real coefficients: $c_0 = a_0/2$, $c_n = (a_n - ib_n)/2$, $c_{-n} = (a_n + ib_n)/2 = \overline{c_n}$ for $n \geq 1$.

This means the complex form does not introduce new information. It repackages the sine and cosine coefficients into a single symmetric sequence.

The complex form is more compact and connects directly to the Fourier transform: letting $L \to \infty$ (so the period grows without bound) turns the discrete sum into an integral — the subject of the next chapter.

```
//| echo: false
{
  // ── OPPORTUNITY 3: Complex exponential phasor wheel ──────────────────────
  // Left panel: unit circle with two counter-rotating phasors e^{in□}, e^{-in□}
  // Right panel: real projection traces cos(n□)
  const d3 = await require("d3@7");

  const AMBER = "#f59e0b";
  const BLUE  = "#6b90c4";
  const GREY  = "#9ca3af";
  const GREY_F = "#e5e7eb";
  const RED   = "#ef4444";

  const W = 580, H = 200;
  const circleSize = 170;
  const cx = circleSize / 2, cy = circleSize / 2, r0 = 70;
  const traceW = W - circleSize - 24;
  const traceX0 = circleSize + 24;

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit; max-width:640px;";

  // Controls
  const ctrlRow = document.createElement("div");
  ctrlRow.style.cssText = "display:flex; gap:1.5rem; align-items:center; margin-bottom:0.4rem; fl

  // n slider
  const nLbl = document.createElement("label");
  nLbl.style.cssText = "font-size:0.88em; color:#374151; display:flex; align-items:center; gap:0.
  nLbl.textContent = "Frequency n = ";
  const nDisp = document.createElement("strong");
  nDisp.style.cssText = "color:" + AMBER + "; min-width:1ch;";
  nDisp.textContent = "1";
  const nSlider2 = document.createElement("input");
  nSlider2.type = "range"; nSlider2.min = 1; nSlider2.max = 5; nSlider2.value = 1;
  nSlider2.style.cssText = "width:120px; accent-color:" + AMBER + ";";
  nLbl.appendChild(nSlider2); nLbl.appendChild(nDisp);
  ctrlRow.appendChild(nLbl);

  // Play/pause
  const playBtn = document.createElement("button");
  playBtn.textContent = "□ Pause";
  playBtn.style.cssText = "padding:0.3rem 0.7rem; border:1px solid #d1d5db; border-radius:4px; ba
  ctrlRow.appendChild(playBtn);

  // Speed
  const speedLbl = document.createElement("label");
```

```
speedLbl.style.cssText = "font-size:0.85em; color:#6b7280; display:flex; align-items:center; gap:0
speedLbl.textContent = "Speed:";
const speedSel = document.createElement("select");
speedSel.style.cssText = "font-size:0.85em; border:1px solid #d1d5db; border-radius:3px; padding:0
["Slow","Normal","Fast"].forEach((s,i) => {
  const o = document.createElement("option"); o.value = [0.5,1,2][i]; o.textContent = s;
  if (i===1) o.selected = true;
  speedSel.appendChild(o);
});
speedLbl.appendChild(speedSel);
ctrlRow.appendChild(speedLbl);
container.appendChild(ctrlRow);

// SVG
const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width","100%").style("display","block");

// ── Left: circle panel ──────────────────────────────────────────────
svg.append("text").attr("x",cx).attr("y",10).attr("text-anchor","middle")
  .attr("font-size","10px").attr("fill","#374151").text("Complex plane");
svg.append("circle").attr("cx",cx).attr("cy",cy).attr("r",r0)
  .attr("fill","none").attr("stroke",GREY_F).attr("stroke-width",1.5);
svg.append("line").attr("x1",cx-r0-6).attr("x2",cx+r0+6).attr("y1",cy).attr("y2",cy)
  .attr("stroke",GREY_F).attr("stroke-width",0.8);
svg.append("line").attr("x1",cx).attr("x2",cx).attr("y1",cy-r0-6).attr("y2",cy+r0+6)
  .attr("stroke",GREY_F).attr("stroke-width",0.8);
svg.append("text").attr("x",cx+r0+4).attr("y",cy+3).attr("font-size","9px").attr("fill",GREY).text
svg.append("text").attr("x",cx-3).attr("y",cy-r0-3).attr("font-size","9px").attr("fill",GREY).text

// Phasor arrows: positive (amber), negative (blue)
const posArrow = svg.append("line").attr("stroke",AMBER).attr("stroke-width",2.5);
const negArrow = svg.append("line").attr("stroke",BLUE).attr("stroke-width",2.5);
const posLabel = svg.append("text").attr("font-size","9px").attr("fill",AMBER).attr("font-style","
const negLabel = svg.append("text").attr("font-size","9px").attr("fill",BLUE).attr("font-style","

// Midpoint (sum/2, lies on Re axis)
const midDot = svg.append("circle").attr("r",5).attr("fill",RED);
// Projection line onto Re axis
const projLine = svg.append("line").attr("stroke",RED).attr("stroke-width",1.2)
  .attr("stroke-dasharray","3,2").attr("opacity",0.6);

// ── Right: trace panel ──────────────────────────────────────────────
svg.append("text").attr("x",traceX0 + traceW/2).attr("y",10).attr("text-anchor","middle")
  .attr("font-size","10px").attr("fill","#374151").text("Real projection: cos(n□)");

// Axes
```

```
const xT = d3.scaleLinear().domain([0, 2*Math.PI]).range([traceX0, traceX0+traceW]);
const yT = d3.scaleLinear().domain([-1.15, 1.15]).range([H-20, 18]);
svg.append("line").attr("x1",traceX0).attr("x2",traceX0+traceW)
  .attr("y1",yT(0)).attr("y2",yT(0)).attr("stroke",GREY_F).attr("stroke-width",1);
svg.append("line").attr("x1",traceX0).attr("x2",traceX0).attr("y1",18).attr("y2",H-20)
  .attr("stroke",GREY_F).attr("stroke-width",1);
[0, Math.PI/2, Math.PI, 3*Math.PI/2, 2*Math.PI].forEach(v => {
  const label = ["0","π/2","π","3π/2","2π"][Math.round(v/(Math.PI/2))];
  svg.append("text").attr("x",xT(v)).attr("y",H-8).attr("text-anchor","middle")
    .attr("font-size","8px").attr("fill",GREY).text(label);
});
svg.append("text").attr("x",traceX0-4).attr("y",yT(1)).attr("text-anchor","end")
  .attr("font-size","8px").attr("fill",GREY).text("1");
svg.append("text").attr("x",traceX0-4).attr("y",yT(-1)).attr("text-anchor","end")
  .attr("font-size","8px").attr("fill",GREY).text("−1");

// Reference cosine (blue dashed)
const refPts = d3.range(200).map(i => {
  const th = 2*Math.PI*i/199;
  return [xT(th), yT(Math.cos(1 * th))];
});
const refLine = svg.append("path").attr("fill","none").attr("stroke",BLUE)
  .attr("stroke-width",1.5).attr("stroke-dasharray","4,3").attr("opacity",0.5);

// Live trace path
const liveData = [];
const livePath = svg.append("path").attr("fill","none").attr("stroke",RED)
  .attr("stroke-width",2);
// Moving point on trace
const tracePoint = svg.append("circle").attr("r",4.5).attr("fill",RED);

// Formula annotation
const formulaText = svg.append("text").attr("x",traceX0+traceW-4).attr("y",26)
  .attr("text-anchor","end").attr("font-size","10px").attr("fill","#374151")
  .attr("font-style","italic");

// ── Animation ──────────────────────────────────────────────────────────
let theta3 = 0;
let paused = false;
let rafId = null;

playBtn.onclick = () => {
  paused = !paused;
  playBtn.textContent = paused ? "▶ Play" : "⏸ Pause";
  if (!paused) animate();
};
```

```
nSlider2.oninput = () => {
  nDisp.textContent = nSlider2.value;
  liveData.length = 0;
  updateRef();
};

function updateRef() {
  const n = +nSlider2.value;
  const pts = d3.range(200).map(i => {
    const th = 2*Math.PI*i/199;
    return [xT(th), yT(Math.cos(n * th))];
  });
  refLine.datum(pts).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]));
  formulaText.text(`Re(e^{i·${n}·θ} + e^{−i·${n}·θ}) = 2cos(${n}θ)`);
}

function animate() {
  if (paused) return;
  const n = +nSlider2.value;
  const speed = +speedSel.value;
  theta3 += 0.022 * speed;
  const th = theta3 % (2*Math.PI);

  // Positive phasor: e^{inθ}
  const px = cx + r0 * Math.cos(n * theta3 - Math.PI/2);
  const py = cy + r0 * Math.sin(n * theta3 - Math.PI/2);
  // Negative phasor: e^{-inθ}
  const nx_ = cx + r0 * Math.cos(-n * theta3 - Math.PI/2);
  const ny_ = cy + r0 * Math.sin(-n * theta3 - Math.PI/2);

  posArrow.attr("x1",cx).attr("y1",cy).attr("x2",px).attr("y2",py);
  negArrow.attr("x1",cx).attr("y1",cy).attr("x2",nx_).attr("y2",ny_);

  // Labels
  posLabel.attr("x",px+4).attr("y",py-2).text(`e^{i${n}θ}`);
  negLabel.attr("x",nx_+4).attr("y",ny_+8).text(`e^{−i${n}θ}`);

  // Midpoint of sum: (e^{inθ}+e^{-inθ})/2 = cos(nθ) on Re axis
  const reVal = Math.cos(n * theta3);
  const midX = cx + r0 * reVal;
  midDot.attr("cx",midX).attr("cy",cy);
  projLine.attr("x1",cx).attr("y1",cy).attr("x2",midX).attr("y2",cy);

  // Trace
  const traceTheta = theta3 % (2*Math.PI);
  liveData.push([xT(traceTheta), yT(reVal)]);
  if (liveData.length > 500) liveData.shift();
```

```
    livePath.datum(liveData).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]).defined((_,i,arr) => {
      if (i === 0) return true;
      return Math.abs(arr[i][0] - arr[i-1][0]) < 15;  // break on wraparound
    }));
    tracePoint.attr("cx", xT(traceTheta)).attr("cy", yT(reVal));

    rafId = requestAnimationFrame(animate);
  }

  updateRef();
  animate();

  const cleanup = new MutationObserver(() => {
    if (!document.contains(container)) { cancelAnimationFrame(rafId); cleanup.disconnect(); }
  });
  cleanup.observe(document.body, {childList:true, subtree:true});

  // Legend
  const legend = document.createElement("div");
  legend.style.cssText = "display:flex; gap:1.2rem; font-size:0.78em; margin-top:0.2rem; flex-wra
  [["→ e^{in□} (positive phasor)", AMBER], ["→ e^{−in□} (negative phasor)", BLUE], ["• Real proje
    .forEach(([lab, col]) => {
      const s = document.createElement("span"); s.style.color = col; s.textContent = lab;
      legend.appendChild(s);
    });
  container.appendChild(svg.node());
  container.appendChild(legend);

  return container;
}
```

---

## 44.8. Parseval's theorem

The total energy of a periodic signal is distributed across its frequency components. **Parseval's theorem** makes this precise:

$$\frac{1}{L}\int_{-L}^{L} |f(x)|^2\,dx = \frac{a_0^2}{2} + \sum_{n=1}^{\infty}(a_n^2 + b_n^2)$$

In the complex form: $\frac{1}{2L}\int_{-L}^{L} |f|^2\,dx = \sum_{n=-\infty}^{\infty} |c_n|^2$.

The right side is the sum of squared amplitudes of each Fourier mode — the **power spectrum**. Parseval's theorem says that power is conserved: the total power in the time domain equals the total power in the frequency domain. It also provides a shortcut for evaluating certain series (see Exercise 5).

---

## 44.9. Applications

### 44.9.1. Signal analysis and filtering

A microphone converts a sound wave into a voltage signal $f(t)$. To apply an equaliser (boost bass, cut treble), you compute the Fourier coefficients, multiply each $b_n$ by the desired gain at frequency $n$, and reconstruct. In practice this is done with the Fast Fourier Transform (FFT), which computes all $N$ Fourier coefficients in $O(N \log N)$ operations instead of $O(N^2)$.

The Fast Fourier Transform (FFT) is an algorithm that computes all $N$ discrete Fourier coefficients in $O(N \log N)$ arithmetic operations rather than the $O(N^2)$ operations needed by direct summation. For $N = 1024$ this is roughly a hundredfold speedup; for $N = 10^6$ the gap is enormous. The FFT is what makes real-time frequency analysis practical — a direct calculation would be too slow for audio, video, and communications applications.

### 44.9.2. Heat equation initial conditions

The heat equation on a rod of length $L$ with zero-temperature endpoints:

$$\frac{\partial u}{\partial t} = \alpha^2 \frac{\partial^2 u}{\partial x^2}, \qquad u(0,t) = u(L,t) = 0$$

has general solution $u(x,t) = \sum_{n=1}^{\infty} B_n \sin(n\pi x/L)\, e^{-\alpha^2 n^2 \pi^2 t/L^2}$.

The coefficients $B_n$ are exactly the Fourier sine coefficients of the initial temperature profile $u(x,0) = f(x)$. Higher-frequency modes (large $n$) decay exponentially faster — this is why sharp temperature gradients smooth out quickly while broad gradients persist. The Fourier series is not just a tool for computing the answer: it explains the physics.

---

## 44.10. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell);
      row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "Solution complete.";
      stepsDiv.appendChild(done);
    }
  }

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
```

```
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSteps
  }

  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

---

### 44.10.1. Exercise 1: Fourier coefficients of the square wave

Compute all Fourier coefficients for $f(x) = 1$ for $0 < x < \pi$ and $f(x) = -1$ for $-\pi < x < 0$, period $2\pi$. Write out the first four nonzero terms of the series.

```
//| echo: false
makeStepperHTML([
  {
    op: "Identify symmetry",
    eq: "f(-x) = -f(x) \\implies f \\text{ is odd} \\implies a_0 = 0,\\; a_n = 0 \\text{ for all } n
    note: "Odd × even = odd; the integral of an odd function over a symmetric interval is zero."
  },
  {
    op: "Set up b□ using symmetry",
    eq: "b_n = \\frac{1}{\\pi}\\int_{-\\pi}^{\\pi} f(x)\\sin(nx)\\,dx = \\frac{2}{\\pi}\\int_{0}^{\\
    note: "Odd × odd = even, so we double the integral over half the period."
  },
  {
    op: "Evaluate the integral",
    eq: "b_n = \\frac{2}{\\pi}\\left[-\\frac{\\cos(nx)}{n}\\right]_{0}^{\\pi} = \\frac{2}{n\\pi}\\bi
  },
  {
    op: "Separate even and odd n",
    eq: "b_n = \\begin{cases} \\dfrac{4}{n\\pi} & n \\text{ odd} \\\\ 0 & n \\text{ even} \\end{case
```

```
    note: "For even n, cos(nπ) = 1 so 1 − (−1)ⁿ = 0. For odd n, cos(nπ) = −1 so 1 − (−1)ⁿ = 2."
  },
  {
    op: "Write the Fourier series",
    eq: "f(x) = \\frac{4}{\\pi}\\left[\\sin x + \\frac{\\sin 3x}{3} + \\frac{\\sin 5x}{5} + \\fra
  }
])
```

---

### 44.10.2. Exercise 2: Sawtooth wave Fourier coefficients

The function $f(x) = x$ for $-\pi < x < \pi$, extended with period $2\pi$, is the **sawtooth wave**. Compute its Fourier series.

```
//| echo: false
makeStepperHTML([
  {
    op: "Identify symmetry",
    eq: "f(-x) = -x = -f(x) \\implies f \\text{ is odd} \\implies a_0 = 0,\\; a_n = 0",
    note: "All cosine terms vanish."
  },
  {
    op: "Set up bₙ",
    eq: "b_n = \\frac{2}{\\pi}\\int_{0}^{\\pi} x\\sin(nx)\\,dx",
    note: "Using the symmetry reduction: odd × odd = even, double the half-period integral."
  },
  {
    op: "Integrate by parts: u = x, dv = sin(nx)dx",
    eq: "\\int x\\sin(nx)\\,dx = -\\frac{x\\cos(nx)}{n} + \\frac{\\sin(nx)}{n^2} + C"
  },
  {
    op: "Evaluate at limits 0 to π",
    eq: "\\int_0^\\pi x\\sin(nx)\\,dx = \\left[-\\frac{x\\cos(nx)}{n}\\right]_0^\\pi + \\left[\\f
    note: "sin(nπ) = 0 for all integer n."
  },
  {
    op: "Compute bₙ",
    eq: "b_n = \\frac{2}{\\pi}\\cdot\\frac{(-1)^{n+1}\\pi}{n} = \\frac{2(-1)^{n+1}}{n}"
  },
  {
    op: "Write the Fourier series",
    eq: "f(x) = 2\\left[\\sin x - \\frac{\\sin 2x}{2} + \\frac{\\sin 3x}{3} - \\frac{\\sin 4x}{4}
    note: "At x = π the sawtooth jumps from π to −π. The Fourier series converges to 0 = (π + (−π
  }
])
```

---

### 44.10.3. Exercise 3: Half-range cosine series for $f(x) = x$

Find the half-range Fourier cosine series of $f(x) = x$ on $0 < x < \pi$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Apply cosine series formulas (even extension)",
    eq: "a_0 = \\frac{2}{\\pi}\\int_0^\\pi x\\,dx = \\frac{2}{\\pi}\\cdot\\frac{\\pi^2}{2} = \\pi"
  },
  {
    op: "Set up a□",
    eq: "a_n = \\frac{2}{\\pi}\\int_0^\\pi x\\cos(nx)\\,dx"
  },
  {
    op: "Integrate by parts: u = x, dv = cos(nx)dx",
    eq: "\\int_0^\\pi x\\cos(nx)\\,dx = \\left[\\frac{x\\sin(nx)}{n}\\right]_0^\\pi - \\frac{1}{n}\\
    note: "sin(n□) = sin(0) = 0."
  },
  {
    op: "Evaluate",
    eq: "\\int_0^\\pi x\\cos(nx)\\,dx = \\frac{\\cos(n\\pi) - 1}{n^2} = \\frac{(-1)^n - 1}{n^2}",
    note: "For even n: (−1)ⁿ − 1 = 0, so a□ = 0. For odd n: (−1)ⁿ − 1 = −2."
  },
  {
    op: "Final coefficient",
    eq: "a_n = \\frac{2}{\\pi}\\cdot\\frac{(-1)^n - 1}{n^2} = \\begin{cases} -\\dfrac{4}{n^2\\pi} &
  },
  {
    op: "Write the cosine series",
    eq: "x = \\frac{\\pi}{2} - \\frac{4}{\\pi}\\left[\\cos x + \\frac{\\cos 3x}{9} + \\frac{\\cos 5x
    note: "Compare with Exercise 2: the sawtooth sine series and this cosine series represent the sa
  }
])
```

---

### 44.10.4. Exercise 4: Half-range sine series for $f(x) = 1$

Find the half-range Fourier sine series of $f(x) = 1$ on $0 < x < L$. Interpret the result.

```
//| echo: false
makeStepperHTML([
  {
    op: "Apply sine series formula (odd extension)",
    eq: "b_n = \\frac{2}{L}\\int_0^L \\sin\\!\\left(\\frac{n\\pi x}{L}\\right)dx"
  },
  {
    op: "Evaluate the integral",
    eq: "b_n = \\frac{2}{L}\\left[-\\frac{L}{n\\pi}\\cos\\!\\left(\\frac{n\\pi x}{L}\\right)\\rig
  },
  {
    op: "Separate even and odd n",
    eq: "b_n = \\begin{cases} \\dfrac{4}{n\\pi} & n \\text{ odd} \\\\ 0 & n \\text{ even} \\end{c
  },
  {
    op: "Write the sine series",
    eq: "1 = \\frac{4}{\\pi}\\left[\\sin\\!\\left(\\frac{\\pi x}{L}\\right) + \\frac{1}{3}\\sin\\
    note: "This is the same coefficient pattern as the square wave — here applied to a constant f
  },
  {
    op: "Physical interpretation",
    eq: "\\text{Setting } x = L/2: \\quad 1 = \\frac{4}{\\pi}\\left[1 - \\frac{1}{3} + \\frac{1}{
    note: "This is the Leibniz formula for □ — a bonus convergent series identity from evaluating
  }
])
```

---

### 44.10.5. Exercise 5: Parseval's theorem and a series sum

Apply Parseval's theorem to the square wave result from Exercise 1 to evaluate $\sum_{k=0}^{\infty} \frac{1}{(2k+1)^2}$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write Parseval's theorem for the square wave",
    eq: "\\frac{1}{\\pi}\\int_{-\\pi}^{\\pi}|f(x)|^2\\,dx = \\frac{a_0^2}{2} + \\sum_{n=1}^{\\inf
    note: "Here L = □, so the factor is 1/□."
  },
  {
    op: "Evaluate the left side",
    eq: "\\frac{1}{\\pi}\\int_{-\\pi}^{\\pi}1^2\\,dx = \\frac{1}{\\pi}\\cdot 2\\pi = 2",
    note: "|f(x)|² = 1 everywhere on (-□, □)."
  },
  {
```

```
    op: "Evaluate the right side using b□ = 4/(n□) for odd n, 0 for even n",
    eq: "\\sum_{n=1}^{\\infty} b_n^2 = \\sum_{k=0}^{\\infty}\\left(\\frac{4}{(2k+1)\\pi}\\right)^2 =
  },
  {
    op: "Set left = right and solve",
    eq: "2 = \\frac{16}{\\pi^2}\\sum_{k=0}^{\\infty}\\frac{1}{(2k+1)^2} \\implies \\sum_{k=0}^{\\inf
  },
  {
    op: "Verify the first few terms",
    eq: "1 + \\frac{1}{9} + \\frac{1}{25} + \\frac{1}{49} + \\cdots \\approx 1.000 + 0.111 + 0.040 +
    note: "□²/8 □ 1.2337. Convergence is slow but the limit is exact."
  }
])
```

---

### 44.10.6. Exercise 6: Heat equation — reading off the solution from a Fourier series

A rod of length $L = \pi$ has initial temperature $u(x, 0) = 3\sin(x) - \sin(2x)$ for $0 < x < \pi$. The endpoints are held at zero: $u(0, t) = u(\pi, t) = 0$. The heat equation is $u_t = \alpha^2 u_{xx}$.

Write down the complete solution $u(x, t)$ and identify which component decays faster.

```
//| echo: false
makeStepperHTML([
  {
    op: "Recall the general solution form",
    eq: "u(x,t) = \\sum_{n=1}^{\\infty} B_n \\sin(nx)\\,e^{-\\alpha^2 n^2 t}",
    note: "For L = □, the sine modes are sin(nx) and the decay rates are □²n²."
  },
  {
    op: "Match initial condition to the general form",
    eq: "u(x,0) = \\sum_{n=1}^{\\infty} B_n \\sin(nx) = 3\\sin(x) - \\sin(2x)",
    note: "The initial condition is already a finite Fourier sine series — no integration needed."
  },
  {
    op: "Read off the coefficients",
    eq: "B_1 = 3, \\quad B_2 = -1, \\quad B_n = 0 \\text{ for } n \\geq 3"
  },
  {
    op: "Write the solution",
    eq: "u(x,t) = 3\\sin(x)\\,e^{-\\alpha^2 t} - \\sin(2x)\\,e^{-4\\alpha^2 t}"
  },
  {
    op: "Compare decay rates",
    eq: "\\text{Mode } n=2 \\text{ decays at rate } 4\\alpha^2 \\text{ vs } \\alpha^2 \\text{ for }
    note: "High-frequency components (sharp features) dissipate faster than low-frequency ones. At
```

```
  }
])
```

# 45. Fourier transforms

From periodic decomposition to continuous frequency analysis

The Fourier series produces a discrete spectrum: a list of amplitudes at frequencies $\pi/L, 2\pi/L, 3\pi/L, \ldots$ spaced $\pi/L$ apart. As $L \to \infty$, the spacing between frequencies shrinks to zero, the list of amplitudes becomes a continuous function of frequency, and the sum becomes an integral. This is the Fourier transform.

The transition from series to transform is not just a mathematical technicality. It extends the analysis to any signal that decays to zero as $|t| \to \infty$ — a one-shot pulse, a finite-duration transient, or any square-integrable function. The series was restricted to functions that repeat forever. The transform has no such restriction.

---

## 45.1. From Fourier series to the Fourier integral

Consider a function $f$ of period $2L$ with Fourier series:

This starts from the complex Fourier-series form introduced at the end of the previous chapter, where the real coefficients $a_n$ and $b_n$ were bundled into the single sequence $c_n$.

$$f(x) = \sum_{n=-\infty}^{\infty} c_n \, e^{in\pi x/L}, \qquad c_n = \frac{1}{2L} \int_{-L}^{L} f(x) \, e^{-in\pi x/L} \, dx$$

Define $\omega_n = n\pi/L$ and $\Delta\omega = \pi/L$ (the spacing between successive frequencies). Then:

$$f(x) = \sum_{n=-\infty}^{\infty} \frac{\Delta\omega}{2\pi} \left[ \int_{-L}^{L} f(t) \, e^{-i\omega_n t} \, dt \right] e^{i\omega_n x}$$

The rewrite uses a single substitution: since $\Delta\omega = \pi/L$, the prefactor $1/(2L)$ in $c_n$ equals $\Delta\omega/(2\pi)$. Replacing it turns each coefficient into $(\Delta\omega/2\pi) \times$ [integral], and the whole sum becomes a Riemann sum with step size $\Delta\omega$ — ready to become an integral.

Let $L \to \infty$: $\Delta\omega \to d\omega$, the sum becomes an integral, and we obtain the **Fourier integral representation**:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \left[ \int_{-\infty}^{\infty} f(t)\, e^{-i\omega t}\, dt \right] e^{i\omega x}\, d\omega$$

The inner bracket is the **Fourier transform** of $f$ evaluated at $\omega$.

```
//| echo: false
{
  // ── OPPORTUNITY 1: Series-to-transform transition ──────────────────────
  // Rectangular pulse f(t)=1 for |t|<1. Fourier series coefficients on period 2L
  // as L increases → bars multiply, narrow, trace the sinc envelope.
  const d3 = await require("d3@7");

  const AMBER = "#f59e0b";
  const BLUE  = "#6b90c4";
  const GREY  = "#9ca3af";
  const GREY_F = "#e5e7eb";

  const W = 560, H = 200, PL = 46, PR = 16, PT = 28, PB = 28;
  const IW = W - PL - PR, IH = H - PT - PB;

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit; max-width:620px;";

  // Slider for L (as multiples of 2π)
  const ctrlRow = document.createElement("div");
  ctrlRow.style.cssText = "display:flex; gap:1rem; align-items:center; margin-bottom:0.4rem; flex-wr
  const lLbl = document.createElement("label");
  lLbl.style.cssText = "font-size:0.88em; color:#374151; display:flex; align-items:center; gap:0.4re
  lLbl.textContent = "Period  L = ";
  const lDisp = document.createElement("strong");
  lDisp.style.cssText = "color:" + AMBER + "; min-width:4ch;";
  lDisp.textContent = "1π";
  const lSlider = document.createElement("input");
  lSlider.type = "range"; lSlider.min = 1; lSlider.max = 20; lSlider.value = 1;
  lSlider.style.cssText = "width:200px; accent-color:" + AMBER + ";";
  lLbl.appendChild(lSlider); lLbl.appendChild(lDisp);
  ctrlRow.appendChild(lLbl);

  const statusLabel = document.createElement("span");
  statusLabel.style.cssText = "font-size:0.82em; color:" + BLUE + "; font-style:italic;";
  statusLabel.textContent = "Discrete spectrum: Fourier series";
  ctrlRow.appendChild(statusLabel);
  container.appendChild(ctrlRow);

  // SVG
  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
```

```
  .attr("width","100%").style("display","block");
const g = svg.append("g").attr("transform", `translate(${PL},${PT})`);

// Axes — □ from -4□ to +4□
const omegaMax = 4 * Math.PI;
const xS = d3.scaleLinear().domain([-omegaMax, omegaMax]).range([0, IW]);
const yS = d3.scaleLinear().domain([0, 0.75]).range([IH, 0]);

g.append("line").attr("x1",0).attr("x2",IW).attr("y1",IH).attr("y2",IH)
  .attr("stroke",GREY_F).attr("stroke-width",1);
g.append("line").attr("x1",xS(0)).attr("x2",xS(0)).attr("y1",0).attr("y2",IH)
  .attr("stroke",GREY_F).attr("stroke-width",1);

// □-axis ticks
[-4,-3,-2,-1,0,1,2,3,4].forEach(k => {
  const lab = k===0?"0":k>0?`${k}□`:`${k}□`;
  g.append("text").attr("x",xS(k*Math.PI)).attr("y",IH+12)
    .attr("text-anchor","middle").attr("font-size","9px").attr("fill",GREY).text(lab);
  g.append("line").attr("x1",xS(k*Math.PI)).attr("x2",xS(k*Math.PI)).attr("y1",IH).attr("y2",IH
    .attr("stroke",GREY_F);
});
g.append("text").attr("x",IW+2).attr("y",IH+4).attr("font-size","9px").attr("fill",GREY).text("

// y-axis
g.append("g").call(d3.axisLeft(yS).ticks(4).tickFormat(d3.format(".2f")))
  .call(gg => { gg.select(".domain").remove(); gg.selectAll("text").style("font-size","9px").at

// Continuous sinc envelope (blue, static)
const sincLine = g.append("path").attr("fill","none").attr("stroke",BLUE)
  .attr("stroke-width",1.8).attr("stroke-dasharray","5,3").attr("opacity",0.6);
const sincPts = d3.range(800).map(i => {
  const omega = -omegaMax + 2*omegaMax*i/799;
  const val = Math.abs(omega) < 0.001 ? 2/(2*Math.PI) : Math.abs(Math.sin(omega))/(Math.PI*Math
  return [xS(omega), yS(val)];
});
sincLine.datum(sincPts).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]));

// Bar container
const barsG = g.append("g");

// Title
g.append("text").attr("x",IW/2).attr("y",-12).attr("text-anchor","middle")
  .attr("font-size","11px").attr("fill","#374151")
  .text("Fourier series coefficients → Fourier transform");

function drawBars(Lpi) {
  const L = Lpi * Math.PI;
```

```
    const deltaOmega = Math.PI / L;
    const nMax = Math.floor(omegaMax / deltaOmega) + 1;
    barsG.selectAll("*").remove();
    const barW = Math.max(0.8, (IW / (2*omegaMax)) * deltaOmega * 0.7);

    for (let n = -nMax; n <= nMax; n++) {
      const omega_n = n * deltaOmega;
      // Coefficient height: (1/2□) * |F(□_n)| * □□ = (1/2□) * 2|sinc(□_n)| * □□
      // Display: (2/□)|sinc(□_n)| so y-axis matches continuous limit
      const sinc = Math.abs(omega_n) < 1e-6 ? 1 : Math.sin(omega_n)/omega_n;
      const h = Math.abs(sinc) / Math.PI;
      if (h < 1e-6) continue;
      const xPos = xS(omega_n);
      if (xPos < 0 || xPos > IW) continue;
      const yPos = yS(h);
      const barH = IH - yPos;
      barsG.append("rect")
        .attr("x", xPos - barW/2).attr("y", yPos)
        .attr("width", barW).attr("height", Math.max(1, barH))
        .attr("fill", AMBER).attr("opacity", 0.85);
    }

    lDisp.textContent = `${Lpi}□`;
    statusLabel.textContent = Lpi >= 18
      ? "Continuous spectrum: Fourier transform"
      : "Discrete spectrum: Fourier series";
    statusLabel.style.color = Lpi >= 18 ? BLUE : AMBER;
  }

  lSlider.oninput = () => drawBars(+lSlider.value);
  drawBars(1);

  // Legend
  const legend = document.createElement("div");
  legend.style.cssText = "display:flex; gap:1.2rem; font-size:0.78em; margin-top:0.2rem;";
  [["▌ Fourier series coefficients", AMBER], ["--- Continuous transform envelope", BLUE]]
    .forEach(([lab, col]) => {
      const s = document.createElement("span"); s.style.color = col; s.textContent = lab;
      legend.appendChild(s);
    });
  container.appendChild(svg.node());
  container.appendChild(legend);
  return container;
}
```

## 45.2. The Fourier transform

The **Fourier transform** of $f(t)$ is:

$$F(\omega) = \mathscr{F}\{f\}(\omega) = \int_{-\infty}^{\infty} f(t)\, e^{-i\omega t}\, dt$$

The **inverse Fourier transform** recovers $f$ from $F$:

$$f(t) = \mathscr{F}^{-1}\{F\}(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega)\, e^{i\omega t}\, d\omega$$

Together these are a **transform pair**: $f(t) \leftrightarrow F(\omega)$. Knowing either one determines the other completely (for well-behaved functions).

**Conventions.** There are three common conventions for where the $2\pi$ factor sits and whether the sign in the exponent is $-i\omega t$ or $+i\omega t$. The one above is the **engineering convention** (also used by Kreyszig and Stroud). The physics convention places $1/\sqrt{2\pi}$ on both the forward and inverse transform. When using a reference or software tool, check which convention it follows.

**Existence.** The Fourier transform is guaranteed to exist (as an ordinary integral) when $f$ is absolutely integrable: $\int_{-\infty}^{\infty} |f(t)|\, dt < \infty$. For the larger class of square-integrable functions ($\int |f|^2 < \infty$), it exists in the $L^2$ sense. Important special cases — the unit step, the delta function, the constant function — require the theory of distributions but yield well-defined transforms.

```
//| echo: false
{
  // ── OPPORTUNITY 3: Fourier transform as winding diagram ─────────────────
  // f(t) = cos(t) + 0.5cos(3t). Winding at freq ω winds f onto the complex plane.
  // Centre of mass of the wound curve = F(ω)/(2π).
  const d3 = await require("d3@7");

  const AMBER = "#f59e0b";
  const BLUE  = "#6b90c4";
  const GREY  = "#9ca3af";
  const GREY_F = "#e5e7eb";
  const RED   = "#ef4444";

  const W = 580, H = 210;
  const leftW = 210, rightW = 200, gap = 20, barW = 90;
  const circR = 80;
  const circCx = leftW + gap + rightW/2, circCy = H/2;

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit; max-width:640px;";

  // ω slider
  const ctrlRow = document.createElement("div");
```

```
ctrlRow.style.cssText = "display:flex; gap:1rem; align-items:center; margin-bottom:0.4rem; flex-wr
const omLbl = document.createElement("label");
omLbl.style.cssText = "font-size:0.88em; color:#374151; display:flex; align-items:center; gap:0.4r
omLbl.textContent = "Testing frequency ω = ";
const omDisp = document.createElement("strong");
omDisp.style.cssText = "color:" + AMBER + "; min-width:3ch;";
omDisp.textContent = "1.0";
const omSlider = document.createElement("input");
omSlider.type = "range"; omSlider.min = 0; omSlider.max = 50; omSlider.value = 10; // steps of 0.1
omSlider.style.cssText = "width:180px; accent-color:" + AMBER + ";";
omLbl.appendChild(omSlider); omLbl.appendChild(omDisp);
ctrlRow.appendChild(omLbl);
ctrlRow.appendChild(Object.assign(document.createElement("span"), {
  textContent: "rad/s",
  style: { cssText: "font-size:0.82em; color:#9ca3af;" }
}));
container.appendChild(ctrlRow);

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width","100%").style("display","block");

// ── Left panel: time domain signal ──────────────────────────────────
const tMin = 0, tMax = 2*Math.PI;
const xL = d3.scaleLinear().domain([tMin, tMax]).range([8, leftW-8]);
const yL = d3.scaleLinear().domain([-1.6, 1.6]).range([H-20, 16]);

svg.append("text").attr("x", leftW/2).attr("y", 12).attr("text-anchor","middle")
  .attr("font-size","10px").attr("fill","#374151").text("f(t) = cos(t) + 0.5cos(3t)");

svg.append("line").attr("x1",8).attr("x2",leftW-8).attr("y1",yL(0)).attr("y2",yL(0))
  .attr("stroke",GREY_F).attr("stroke-width",1);
svg.append("line").attr("x1",xL(tMin)).attr("x2",xL(tMin)).attr("y1",16).attr("y2",H-20)
  .attr("stroke",GREY_F).attr("stroke-width",1);

// f(t) curve
const fPts = d3.range(300).map(i => {
  const t = tMin + (tMax-tMin)*i/299;
  return [xL(t), yL(Math.cos(t) + 0.5*Math.cos(3*t))];
});
svg.append("path").attr("fill","none").attr("stroke",BLUE).attr("stroke-width",2)
  .datum(fPts).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]));

// Moving time-bar
const timeLine = svg.append("line").attr("stroke",AMBER).attr("stroke-width",1.5).attr("opacity",0
// Time-domain dot
const timeDot = svg.append("circle").attr("r",4).attr("fill",AMBER);
```

```
[0,Math.PI/2,Math.PI,3*Math.PI/2,2*Math.PI].forEach(t => {
  svg.append("text").attr("x",xL(t)).attr("y",H-8).attr("text-anchor","middle")
    .attr("font-size","8px").attr("fill",GREY).text(["0","π/2","π","3π/2","2π"][Math.round(t/(M
});

// ── Middle panel: winding diagram ──────────────────────────────────────
svg.append("text").attr("x",circCx).attr("y",12).attr("text-anchor","middle")
  .attr("font-size","10px").attr("fill","#374151").text("Wound-up f(t)e^{-iωt}");

svg.append("circle").attr("cx",circCx).attr("cy",circCy).attr("r",circR)
  .attr("fill","none").attr("stroke",GREY_F).attr("stroke-width",1.5);
svg.append("line").attr("x1",circCx-circR-4).attr("x2",circCx+circR+4)
  .attr("y1",circCy).attr("y2",circCy).attr("stroke",GREY_F).attr("stroke-width",0.8);
svg.append("line").attr("x1",circCx).attr("x2",circCx)
  .attr("y1",circCy-circR-4).attr("y2",circCy+circR+4).attr("stroke",GREY_F).attr("stroke-width
svg.append("text").attr("x",circCx+circR+2).attr("y",circCy+3).attr("font-size","8px").attr("fi
svg.append("text").attr("x",circCx-2).attr("y",circCy-circR-2).attr("font-size","8px").attr("fi

const woundPath = svg.append("path").attr("fill","none").attr("stroke",AMBER)
  .attr("stroke-width",1.5).attr("opacity",0.85);
const comDot = svg.append("circle").attr("r",6).attr("fill",RED).attr("opacity",0.9);
const comLine = svg.append("line").attr("stroke",RED).attr("stroke-width",2).attr("opacity",0.7
const comLabel = svg.append("text").attr("font-size","9px").attr("fill",RED);

// ── Right panel: |F(ω)| bar ────────────────────────────────────────────
const barX0 = leftW + gap + rightW + 12;
svg.append("text").attr("x",barX0+barW/2).attr("y",12).attr("text-anchor","middle")
  .attr("font-size","10px").attr("fill","#374151").text("|F(ω)|");

// Precompute |F(ω)| for f(t) = cos(t) + 0.5cos(3t) on [0,2π]
// F(ω) = ∫₀^{2π} f(t)e^{-iωt}dt
// Numerically:
function computeF(omega) {
  const N = 500;
  let re = 0, im = 0;
  for (let i = 0; i < N; i++) {
    const t = tMin + (tMax-tMin)*i/(N-1);
    const f = Math.cos(t) + 0.5*Math.cos(3*t);
    re += f * Math.cos(omega * t);
    im -= f * Math.sin(omega * t);
  }
  const dt = (tMax-tMin)/(N-1);
  return Math.sqrt(re*re + im*im) * dt;
}

const omVals = d3.range(51).map(i => i*0.1);
const fMags = omVals.map(computeF);
```

```
const maxMag = d3.max(fMags);

const xBar = d3.scaleBand().domain(omVals.map(String)).range([barX0, barX0+barW]).padding(0.05);
const yBar = d3.scaleLinear().domain([0, maxMag]).range([H-20, 20]);

// Draw all bars (grey), highlight current
const bars = omVals.map((om,i) => {
  const b = svg.append("rect")
    .attr("x", barX0 + (barW/(omVals.length)) * i)
    .attr("y", yBar(fMags[i]))
    .attr("width", Math.max(1, barW/omVals.length - 1))
    .attr("height", H-20 - yBar(fMags[i]))
    .attr("fill", GREY_F);
  return b;
});
svg.append("text").attr("x",barX0).attr("y",H-8).attr("font-size","8px").attr("fill",GREY).text("0
svg.append("text").attr("x",barX0+barW).attr("y",H-8).attr("font-size","8px").attr("fill",GREY).te
svg.append("text").attr("x",barX0+barW/2).attr("y",H-8).attr("text-anchor","middle")
  .attr("font-size","7.5px").attr("fill",GREY).text("□");

// Current □ indicator
const barIndicator = svg.append("line").attr("stroke",AMBER).attr("stroke-width",2).attr("opacity"

function update() {
  const omega = +omSlider.value / 10;
  omDisp.textContent = omega.toFixed(1);

  // Wound path: (f(t)cos(□t), -f(t)sin(□t)) = (Re, Im) of f(t)e^{-i□t}
  const N = 300;
  let reSum = 0, imSum = 0;
  const pts = [];
  for (let i = 0; i < N; i++) {
    const t = tMin + (tMax-tMin)*i/(N-1);
    const f = Math.cos(t) + 0.5*Math.cos(3*t);
    const reW = f * Math.cos(omega * t);
    const imW = -f * Math.sin(omega * t);
    reSum += reW; imSum += imW;
    pts.push([circCx + reW * circR/1.6, circCy + imW * circR/1.6]);
  }
  woundPath.datum(pts).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]));

  // Centre of mass
  const comRe = reSum / N;
  const comIm = imSum / N;
  const comX = circCx + comRe * circR/1.6;
  const comY = circCy + comIm * circR/1.6;
  comDot.attr("cx", comX).attr("cy", comY);
```

```
    comLine.attr("x1",circCx).attr("y1",circCy).attr("x2",comX).attr("y2",comY);
    const mag = Math.sqrt(comRe*comRe + comIm*comIm);
    comLabel.attr("x",comX+6).attr("y",comY-4)
      .text(mag > 0.05 ? `|F| ~ ${(mag*2*Math.PI).toFixed(1)}` : "□ 0");

    // Time bar (animate at current omega)
    const t0 = (Date.now() / 2000) % 1 * (tMax-tMin) + tMin;
    const f0 = Math.cos(t0) + 0.5*Math.cos(3*t0);
    timeLine.attr("x1",xL(t0)).attr("x2",xL(t0)).attr("y1",16).attr("y2",H-20);
    timeDot.attr("cx",xL(t0)).attr("cy",yL(f0));

    // Bar highlight
    const idx = Math.round(omega*10);
    bars.forEach((b,i) => b.attr("fill", i===idx ? AMBER : GREY_F));
    if (idx < omVals.length) {
      const bx = barX0 + (barW/omVals.length)*idx;
      barIndicator.attr("x1",bx).attr("x2",bx+barW/omVals.length).attr("y1",yBar(fMags[idx])-3).a
    }
  }

  omSlider.oninput = update;
  update();

  const legend = document.createElement("div");
  legend.style.cssText = "display:flex; gap:1.2rem; font-size:0.78em; margin-top:0.2rem; flex-wra
  [["~ wound-up signal f(t)e^{-i□t}", AMBER], ["• centre of mass = F(□)/(2□)", RED]]
    .forEach(([lab, col]) => {
      const s = document.createElement("span"); s.style.color = col; s.textContent = lab;
      legend.appendChild(s);
    });
  container.appendChild(svg.node());
  container.appendChild(legend);
  return container;
}
```

---

## 45.3. Key transform pairs

### 45.3.1. Rectangular pulse

The **rectangular pulse** (or **gate function**) of width $2a$:

$$f(t) = \begin{cases} 1 & |t| < a \\ 0 & |t| > a \end{cases}$$

Its transform:

$$F(\omega) = \int_{-a}^{a} e^{-i\omega t}\,dt = \left[\frac{e^{-i\omega t}}{-i\omega}\right]_{-a}^{a} = \frac{e^{-i\omega a} - e^{i\omega a}}{-i\omega} = \frac{2\sin(\omega a)}{\omega} = 2a\ \mathrm{sinc}(\omega a)$$

where $\mathrm{sinc}(x) = \sin(x)/x$.

**The time–bandwidth tradeoff.** The width of the main lobe of $|F(\omega)|$ is approximately $2\pi/a$. A narrow pulse ($a$ small) has a wide spectrum. A wide pulse ($a$ large) has a narrow spectrum. This is the mathematical statement of the tradeoff the bat exploits.

### 45.3.2. Gaussian

The Gaussian $f(t) = e^{-\alpha t^2}$ ($\alpha > 0$) transforms to:

$$F(\omega) = \sqrt{\frac{\pi}{\alpha}}\, e^{-\omega^2/(4\alpha)}$$

also a Gaussian. The Gaussian is its own Fourier transform (up to scaling). It minimises the time–bandwidth product, which is why it is used in pulse shaping and wavelet theory.

### 45.3.3. Dirac delta

The Dirac delta is not an ordinary function — it has no well-defined value at any point, and you cannot draw its graph. It is defined entirely by what it does inside an integral: $\int_{-\infty}^{\infty} \delta(t)\phi(t)\,dt = \phi(0)$ for any smooth $\phi$. Its transform:

$$\mathscr{F}\{\delta(t)\} = \int_{-\infty}^{\infty} \delta(t)\, e^{-i\omega t}\,dt = e^0 = 1$$

The delta function has a flat (white) spectrum — it contains all frequencies equally. This is why a sharp impulse (the tap of a hammer on a structure) excites all resonant frequencies simultaneously.

The inverse: $\mathscr{F}\{1\} = 2\pi\delta(\omega)$ — a constant signal contains only zero frequency. This means the delta is best understood as an idealised sampling object inside integrals, not as an ordinary spike-shaped function.

---

## 45.4. Properties of the Fourier transform

These properties make the Fourier transform useful in practice.

**Linearity.** $\mathscr{F}\{\alpha f + \beta g\} = \alpha F + \beta G$.

**Time shifting.** $\mathscr{F}\{f(t - t_0)\} = e^{-i\omega t_0} F(\omega)$. Shifting a signal in time multiplies its spectrum by a complex exponential — it changes the phase but not the magnitude.

**Frequency shifting.** $\mathscr{F}\{e^{i\omega_0 t} f(t)\} = F(\omega - \omega_0)$. Multiplying by a complex exponential shifts the spectrum. This is the mathematical basis of **modulation**: shifting a baseband signal to a carrier frequency. In engineering practice, modulation uses a real cosine $\cos(\omega_0 t) = \frac{1}{2}(e^{i\omega_0 t} + e^{-i\omega_0 t})$, which shifts the spectrum to both $+\omega_0$ and $-\omega_0$ (creating upper and lower sidebands); the complex exponential form isolates the single-sideband case.

**Scaling.** $\mathscr{F}\{f(at)\} = \frac{1}{|a|} F\left(\frac{\omega}{a}\right)$. Compressing time by factor $a$ expands the spectrum by the same factor — another form of the time–bandwidth tradeoff.

**Differentiation.** $\mathscr{F}\{f'(t)\} = i\omega F(\omega)$. Differentiation in time corresponds to multiplication by $i\omega$ in frequency. This turns differential equations into algebraic equations.

**Convolution theorem.**

$$\mathscr{F}\{f * g\}(\omega) = F(\omega) \cdot G(\omega)$$

where $(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau)\, d\tau$ is the **convolution** of $f$ and $g$. Equivalently, $\mathscr{F}\{fg\} = \frac{1}{2\pi} F * G$.

**Parseval's theorem.** $\int_{-\infty}^{\infty} |f(t)|^2\, dt = \frac{1}{2\pi} \int_{-\infty}^{\infty} |F(\omega)|^2\, d\omega$.

This means total signal energy can be computed either in time or in frequency. In practice, Parseval lets you answer energy questions after transforming to the domain where the calculation is easier.

---

## 45.5. The convolution theorem in engineering

The convolution theorem is the most practically important property. A **linear time-invariant (LTI) system** is completely characterised by its **impulse response** $h(t)$: if you drive the system with input $f(t)$, the output is the convolution $y(t) = (f * h)(t)$.

Computing convolution directly requires integrating over all past input values. In the frequency domain, this becomes multiplication:

$$Y(\omega) = F(\omega) \cdot H(\omega)$$

where $H(\omega) = \mathscr{F}\{h\}$ is the **transfer function** of the system.

This is why filters are designed in the frequency domain. A low-pass filter that passes frequencies $|\omega| < \omega_c$ and blocks higher frequencies has $H(\omega) = 1$ for $|\omega| < \omega_c$ and $H(\omega) = 0$ otherwise. Computing

$H(\omega) \cdot F(\omega)$ is multiplication of two functions. Computing the equivalent convolution would mean implementing an integral.

In discrete time, the same structure holds: the discrete Fourier transform converts discrete convolution to pointwise multiplication. The FFT computes this in $O(N \log N)$ time, which is why all modern digital signal processing systems work this way.

```
//| echo: false
{
  // ── VIZ 2 Rewrite: Transform pair explorer ──────────────────────────────
  // Panel A: time-domain f(t)  Panel B: |F(□)| magnitude  Panel C: phase □F(□)
  // Wave types: rectangular pulse | Gaussian | Exponential decay
  const d3 = await require("d3@7");

  const AMBER = "#f59e0b";
  const BLUE  = "#6b90c4";
  const GREY  = "#9ca3af";
  const GREY_F = "#e5e7eb";

  const W = 560, H = 280;
  const PL = 44, PR = 12, PT = 22, PB = 16;
  const IW = W - PL - PR;
  const HA = 90, HB = 90, HC = 50, gap = 12;

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit; max-width:620px;";

  // Controls
  const ctrlRow = document.createElement("div");
  ctrlRow.style.cssText = "display:flex; gap:1.2rem; align-items:center; margin-bottom:0.4rem; flex-

  // Wave type selector
  const waveTypes = [["rect","Rectangular pulse"],["gauss","Gaussian"],["exp","Exp. decay e^{-|t|/□
  let currentWT = "rect";
  const radioGrp = document.createElement("div");
  radioGrp.style.cssText = "display:flex; gap:0.6rem; font-size:0.85em; flex-wrap:wrap;";
  waveTypes.forEach(([val,lab]) => {
    const lbl = document.createElement("label");
    lbl.style.cssText = "display:flex; align-items:center; gap:0.25rem; cursor:pointer;";
    const r = document.createElement("input");
    r.type = "radio"; r.name = "ftViz2"; r.value = val; r.checked = val=="rect";
    r.style.accentColor = AMBER;
    r.onchange = () => { currentWT = val; updateParamLabel(); draw(); };
    lbl.appendChild(r); lbl.appendChild(document.createTextNode(lab));
    radioGrp.appendChild(lbl);
  });
  ctrlRow.appendChild(radioGrp);
```

```
// Parameter slider (a / □ / □)
const paramRow = document.createElement("div");
paramRow.style.cssText = "display:flex; gap:0.5rem; align-items:center; font-size:0.85em;";
const paramLbl = document.createElement("span");
paramLbl.style.cssText = "color:#374151; min-width:4ch;";
paramLbl.textContent = "a =";
const paramDisp = document.createElement("strong");
paramDisp.style.cssText = "color:" + AMBER + "; min-width:3ch;";
paramDisp.textContent = "1.0";
const paramSlider = document.createElement("input");
paramSlider.type = "range"; paramSlider.min = 2; paramSlider.max = 30; paramSlider.value = 10;
paramSlider.style.cssText = "width:160px; accent-color:" + AMBER + ";";
paramRow.appendChild(paramLbl); paramRow.appendChild(paramSlider); paramRow.appendChild(paramDi
ctrlRow.appendChild(paramRow);

// Time-shift toggle
const shiftLbl = document.createElement("label");
shiftLbl.style.cssText = "display:flex; align-items:center; gap:0.3rem; font-size:0.82em; curso
const shiftCheck = document.createElement("input");
shiftCheck.type = "checkbox";
shiftCheck.onchange = draw;
shiftLbl.appendChild(shiftCheck);
shiftLbl.appendChild(document.createTextNode("Shift by t₀=1"));
ctrlRow.appendChild(shiftLbl);
container.appendChild(ctrlRow);

function updateParamLabel() {
  const lbl = currentWT=="rect"?"a":currentWT=="gauss"?"□":"□";
  paramLbl.textContent = lbl + " =";
}

const svg = d3.create("svg")
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("width","100%").style("display","block");

const gA = svg.append("g").attr("transform", `translate(${PL},${PT})`);
const gB = svg.append("g").attr("transform", `translate(${PL},${PT+HA+gap})`);
const gC = svg.append("g").attr("transform", `translate(${PL},${PT+HA+gap+HB+gap})`);

// Shared □ range — rescale based on current a
function omegaRange(param) { return 4 * Math.PI / param; }

// ── Wave functions ──────────────────────────────────────────────────
function fTime(t, param, wt, shift) {
  const t_ = shift ? t - 1 : t;
  if (wt=="rect") return Math.abs(t_) < param ? 1 : 0;
  if (wt=="gauss") return Math.exp(-t_*t_ / (2*param*param));
```

```
  return Math.exp(-Math.abs(t_)/param);
}

// Fourier transforms (analytical)
function FRect(omega, a, shift) {
  const mag = Math.abs(omega) < 1e-8 ? 2*a : 2*Math.sin(omega*a)/omega;
  const phase = shift ? -omega * 1 : 0;
  return { mag: Math.abs(mag), phase: phase + (mag < 0 ? Math.PI : 0) };
}
function FGauss(omega, sigma, shift) {
  const mag = sigma * Math.sqrt(2*Math.PI) * Math.exp(-omega*omega*sigma*sigma/2);
  const phase = shift ? -omega * 1 : 0;
  return { mag, phase };
}
function FExp(omega, tau, shift) {
  const mag = 2*tau / (1 + omega*omega*tau*tau);
  const phase = shift ? -omega * 1 : 0;
  return { mag, phase };
}

function computeF(omega, param, wt, shift) {
  if (wt=="rect") return FRect(omega, param, shift);
  if (wt=="gauss") return FGauss(omega, param, shift);
  return FExp(omega, param, shift);
}

// Axis setup helpers
function makeAxes(g, xScale, yScale, height, nTicksY, title) {
  g.append("text").attr("x",IW/2).attr("y",-6).attr("text-anchor","middle")
    .attr("font-size","10px").attr("fill","#374151").text(title);
  g.append("line").attr("x1",0).attr("x2",IW).attr("y1",yScale(0)).attr("y2",yScale(0))
    .attr("stroke",GREY_F).attr("stroke-width",0.8);
  g.append("g").call(d3.axisLeft(yScale).ticks(nTicksY))
    .call(gg => { gg.select(".domain").remove(); gg.selectAll("text").style("font-size","9px").att
  g.append("g").call(d3.axisBottom(xScale).ticks(7))
    .attr("transform",`translate(0,${height})`)
    .call(gg => { gg.select(".domain").attr("stroke",GREY_F); gg.selectAll("text").style("font-siz
}

// Panel A: time domain
const tMin = -6, tMax = 6;
const xA = d3.scaleLinear().domain([tMin, tMax]).range([0, IW]);
const yA = d3.scaleLinear().domain([-0.15, 1.2]).range([HA, 0]);
makeAxes(gA, xA, yA, HA, 3, "Time domain f(t)");
const timePath = gA.append("path").attr("fill","none").attr("stroke",AMBER).attr("stroke-width",2
const widthBracket = gA.append("g");
const bracketLine = widthBracket.append("line").attr("stroke",GREY).attr("stroke-width",1).attr("s
```

```
const bracketLabel = widthBracket.append("text").attr("font-size","9px").attr("fill",GREY).attr

// Panel B: magnitude
const xB = d3.scaleLinear().domain([-1, 1]).range([0, IW]);
const yB = d3.scaleLinear().domain([0, 3]).range([HB, 0]);
makeAxes(gB, xB, yB, HB, 3, "|F(□)| — frequency magnitude");
const magPath = gB.append("path").attr("fill","none").attr("stroke",AMBER).attr("stroke-width",
const zeroMarks = [gB.append("line").attr("stroke","#6b7280").attr("stroke-width",1.5).attr("st
                   gB.append("line").attr("stroke","#6b7280").attr("stroke-width",1.5).attr("st
const zeroLabels = [gB.append("text").attr("font-size","8.5px").attr("fill","#6b7280").attr("te
                    gB.append("text").attr("font-size","8.5px").attr("fill","#6b7280").attr("te
const bwArrow = gB.append("text").attr("font-size","9px").attr("fill",BLUE).attr("font-style","

// Panel C: phase
const xC = d3.scaleLinear().domain([-1, 1]).range([0, IW]);
const yC = d3.scaleLinear().domain([-Math.PI-0.3, Math.PI+0.3]).range([HC, 0]);
makeAxes(gC, xC, yC, HC, 2, "Phase □F(□)");
const phasePath = gC.append("path").attr("fill","none").attr("stroke",BLUE).attr("stroke-width"
const phaseLabel = gC.append("text").attr("font-size","9px").attr("fill",BLUE).attr("x",4).attr

function draw() {
  const param = +paramSlider.value / 10;
  paramDisp.textContent = param.toFixed(1);
  const shift = shiftCheck.checked;
  const omMax = omegaRange(param);

  // Update x-axes domain for B and C
  xB.domain([-omMax, omMax]);
  xC.domain([-omMax, omMax]);

  // Redraw axes ticks
  gB.selectAll(".x-axis").remove();
  gB.append("g").attr("class","x-axis").call(d3.axisBottom(xB).ticks(7).tickFormat(v => v.toFix
    .attr("transform",`translate(0,${HB})`)
    .call(gg => { gg.select(".domain").attr("stroke",GREY_F); gg.selectAll("text").style("font-
  gC.selectAll(".x-axis").remove();
  gC.append("g").attr("class","x-axis").call(d3.axisBottom(xC).ticks(7).tickFormat(v => v.toFix
    .attr("transform",`translate(0,${HC})`)
    .call(gg => { gg.select(".domain").attr("stroke",GREY_F); gg.selectAll("text").style("font-

  // Panel A: time domain
  const tPts = d3.range(400).map(i => {
    const t = tMin + (tMax-tMin)*i/399;
    return [xA(t), yA(fTime(t, param, currentWT, shift))];
  });
  timePath.datum(tPts).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]));
```

```
    // Width bracket
    const bLab = currentWT=="rect"?"2a":currentWT=="gauss"?"2σ":"σ";
    const bWidth = currentWT=="rect" ? 2*param : currentWT=="gauss" ? 2*param : param;
    bracketLine.attr("x1",xA(-bWidth/2)).attr("x2",xA(bWidth/2)).attr("y1",yA(-0.08)).attr("y2",yA(-
    bracketLabel.attr("x",xA(0)).attr("y",yA(-0.12)).text(bLab + " = " + bWidth.toFixed(1));

    // Panel B & C: spectrum
    const omPts = d3.range(600).map(i => {
      const om = -omMax + 2*omMax*i/599;
      return om;
    });
    const maxMag = d3.max(omPts, om => computeF(om, param, currentWT, false).mag);
    yB.domain([0, maxMag * 1.15]);

    magPath.datum(omPts.map(om => {
      const {mag} = computeF(om, param, currentWT, shift);
      return [xB(om), yB(mag)];
    })).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]));

    phasePath.datum(omPts.map(om => {
      const {phase} = computeF(om, param, currentWT, shift);
      return [xC(om), yC(phase)];
    })).attr("d", d3.line().x(d=>d[0]).y(d=>d[1]));

    phaseLabel.text(shift ? "Phase ramp = −ωt₀ (time-shift effect)" : "Phase = 0 (symmetric, real si

    // Zero-crossing markers (rect only)
    if (currentWT === "rect" && !shift) {
      const z1 = Math.PI / param;
      zeroMarks[0].attr("x1",xB(z1)).attr("x2",xB(z1)).attr("y1",0).attr("y2",HB).style("display","b
      zeroMarks[1].attr("x1",xB(-z1)).attr("x2",xB(-z1)).attr("y1",0).attr("y2",HB).style("display"
      zeroLabels[0].attr("x",xB(z1)).attr("y",12).text("π/a").style("display","block");
      zeroLabels[1].attr("x",xB(-z1)).attr("y",12).text("−π/a").style("display","block");
      const bwDir = param < 1.5 ? "narrower pulse → wider spectrum" : "wider pulse → narrower spectr
      bwArrow.attr("x",IW/2).attr("y",HB-6).attr("text-anchor","middle").text(bwDir);
    } else {
      zeroMarks.forEach(m => m.style("display","none"));
      zeroLabels.forEach(l => l.style("display","none"));
      bwArrow.text(currentWT=="gauss" ? "Gaussian: width 2σ ↔ bandwidth 2/σ" : "");
    }
  }

  paramSlider.oninput = draw;
  draw();

  const legend = document.createElement("div");
  legend.style.cssText = "display:flex; gap:1.2rem; font-size:0.78em; margin-top:0.2rem; flex-wrap:w
```

```
  [["—— f(t) (time domain)", AMBER], ["—— ⍰F(⍰) (phase)", BLUE],
   ["| Zero crossings at ±⍰/a", "#6b7280"]]
    .forEach(([lab, col]) => {
      const s = document.createElement("span"); s.style.color = col; s.textContent = lab;
      legend.appendChild(s);
    });
  container.appendChild(svg.node());
  container.appendChild(legend);
  return container;
}
```

---

## 45.6. Solving differential equations with Fourier transforms

The differentiation property converts a linear ODE with constant coefficients into an algebraic equation in the frequency domain.

**Example.** Solve $y' + 2y = e^{-|t|}$ for $y(t)$.

Take the Fourier transform of both sides:

$$i\omega Y(\omega) + 2Y(\omega) = F(\omega)$$

$$(i\omega + 2)Y(\omega) = F(\omega)$$

$$Y(\omega) = \frac{F(\omega)}{i\omega + 2}$$

Now we need $F(\omega) = \mathscr{F}\{e^{-|t|}\} = \int_{-\infty}^{\infty} e^{-|t|} e^{-i\omega t}\, dt = \frac{2}{1+\omega^2}$.

So $Y(\omega) = \frac{2}{(1+\omega^2)(i\omega+2)}$.

Write this in partial fractions:

$$\frac{2}{(1+\omega^2)(i\omega+2)} = \frac{A}{i\omega+2} + \frac{B\omega + C}{1+\omega^2}$$

Matching numerators gives $A = 2/5$, $B = -2/5$, $C = 4/5$, so:

$$Y(\omega) = \frac{2/5}{i\omega+2} + \frac{(-2\omega+4)/5}{1+\omega^2}$$

Each term matches a known transform pair: $\mathscr{F}^{-1}\{1/(i\omega+2)\} = e^{-2t}$ for $t > 0$ (causal exponential); $\mathscr{F}^{-1}\{1/(1+\omega^2)\} = \frac{1}{2}e^{-|t|}$; and $\mathscr{F}^{-1}\{\omega/(1+\omega^2)\}$ gives a derivative term. Inverting term by term:

$$y(t) = \frac{1}{5}\left(2e^{-2t} - e^{-|t|} + \operatorname{sgn}(t)e^{-|t|}\right)$$

The Fourier transform has reduced the differential equation to algebra.

---

## 45.7. Connection to the Laplace transform

If you have met the Laplace transform in an ODE course, this section shows how it connects to the Fourier transform. If not, you can skip it — nothing later in this chapter depends on it.

The Laplace transform $\mathscr{L}\{f\}(s) = \int_0^\infty f(t)e^{-st}\,dt$ is a **one-sided** transform: it only sees $t \geq 0$ (causal signals). Replacing $s = i\omega$ recovers the Fourier transform of the causal part of $f$.

The Laplace transform is preferred for initial-value problems (ODEs with $t \geq 0$ and given initial conditions) because it handles them directly via $\mathscr{L}\{f'\} = sF(s) - f(0)$. The Fourier transform is preferred for steady-state and bilateral analysis (signals for all $t$).

In circuit analysis: the transfer function $H(s)$ in Laplace domain becomes $H(i\omega)$, the Fourier domain transfer function, when you restrict to purely sinusoidal steady-state inputs.

---

## 45.8. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
```

```
    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;
    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    row.appendChild(opCell);
    row.appendChild(eqCell);
    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }
    stepsDiv.appendChild(row);
  }
  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "Solution complete.";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

function updateButtons() {
  nextBtn.disabled = current === totalSteps;
  nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
  counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
}

nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
controls.appendChild(nextBtn);
controls.appendChild(resetBtn);
controls.appendChild(counter);
container.appendChild(stepsDiv);
container.appendChild(controls);
```

```
  renderSteps();
  updateButtons();
  return container;
}
```

---

### 45.8.1. Exercise 1: Transform of a rectangular pulse

Compute the Fourier transform of $f(t) = \begin{cases} 1 & |t| \le 1 \\ 0 & |t| > 1 \end{cases}$. Identify the first zero of $|F(\omega)|$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Set up the integral",
    eq: "F(\\omega) = \\int_{-\\infty}^{\\infty} f(t)\\,e^{-i\\omega t}\\,dt = \\int_{-1}^{1} e^{-i\
  },
  {
    op: "Evaluate (□ □ 0)",
    eq: "F(\\omega) = \\left[\\frac{e^{-i\\omega t}}{-i\\omega}\\right]_{-1}^{1} = \\frac{e^{-i\\ome
    note: "Using e^{i□} − e^{−i□} = 2i sin □."
  },
  {
    op: "Check □ = 0 by L'Hôpital or direct integration",
    eq: "F(0) = \\int_{-1}^{1}1\\,dt = 2 \\qquad (\\text{consistent with } \\lim_{\\omega\\to 0}\\fr
  },
  {
    op: "Write in sinc form",
    eq: "F(\\omega) = 2\\operatorname{sinc}(\\omega) \\quad \\text{where } \\operatorname{sinc}(\\om
  },
  {
    op: "Locate first zero",
    eq: "F(\\omega) = 0 \\iff \\sin\\omega = 0 \\iff \\omega = \\pm\\pi",
    note: "The bandwidth (distance from DC to first null) is □ rad/s for a pulse of half-width a = 1
  }
])
```

---

### 45.8.2. Exercise 2: Time-shifting theorem

Given that $\mathscr{F}\{e^{-|t|}\} = \frac{2}{1+\omega^2}$, find $\mathscr{F}\{e^{-|t-3|}\}$ and $\mathscr{F}\{e^{-|t|}e^{2it}\}$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Recall the shifting theorems",
    eq: "\\mathcal{F}\\{f(t-t_0)\\} = e^{-i\\omega t_0}F(\\omega) \\qquad \\mathcal{F}\\{e^{i\\om
  },
  {
    op: "Apply time-shift: f(t) = e^{−|t|}, t₀ = 3",
    eq: "\\mathcal{F}\\{e^{-|t-3|}\\} = e^{-3i\\omega}\\cdot\\frac{2}{1+\\omega^2}",
    note: "The magnitude |F(ω)| = 2/(1+ω²) is unchanged — shifting in time only rotates the phase
  },
  {
    op: "Apply frequency-shift: f(t) = e^{−|t|}, ω₀ = 2",
    eq: "\\mathcal{F}\\{e^{2it}\\,e^{-|t|}\\} = \\frac{2}{1+(\\omega-2)^2}",
    note: "The spectrum is the same shape, centred at ω = 2 instead of ω = 0. This is modulation:
  },
  {
    op: "Verify: magnitude of each result at ω = 0",
    eq: "\\left|e^{-3i\\cdot 0}\\cdot\\frac{2}{1}\\right| = 2 \\qquad \\left|\\frac{2}{1+4}\\righ
    note: "The modulated signal has less power at DC — the energy has been moved to ω = 2."
  }
])
```

---

### 45.8.3. Exercise 3: Differentiation property — solving an ODE

Use the Fourier transform to solve $y' + 3y = e^{-2|t|}$.

*(The inversion uses only partial fractions and the known pairs from this chapter — no complex analysis required.)*

```
//| echo: false
makeStepperHTML([
  {
    op: "Compute F(ω) = F{e^{−2|t|}}",
    eq: "F(\\omega) = \\int_{-\\infty}^{\\infty} e^{-2|t|}e^{-i\\omega t}\\,dt = \\int_{-\\infty}
  },
  {
    op: "Transform the ODE; use F{y'} = iωY",
    eq: "i\\omega\\,Y(\\omega) + 3Y(\\omega) = \\frac{4}{4+\\omega^2} \\implies Y(\\omega) = \\fr
  },
  {
    op: "Partial fractions: write (3+iω)(4+ω²) = (3+iω)(2+iω)(2−iω)",
    eq: "\\frac{4}{(3+i\\omega)(2+i\\omega)(2-i\\omega)} = \\frac{A}{3+i\\omega} + \\frac{B}{2+i\
    note: "Multiply through and match coefficients. At iω = −3: A = 4/((−3+2)(−3−2·i·i·...)) — it
  },
```

```
  {
    op: "Cover-up rule for A (set i□ = −3)",
    eq: "A = \\frac{4}{(2-3)(2+3)} = \\frac{4}{(−1)(5)} = -\\frac{4}{5}",
    note: "Wait — (2+i□)(2−i□)|_{i□=−3} = (2+(−3))(2−(−3)) = (−1)(5) = −5. So A = 4/(−5) = −4/5."
  },
  {
    op: "Cover-up rule for B (set i□ = −2) and C (set i□ = 2)",
    eq: "B = \\frac{4}{(3-2)(2-(-2)\\cdot(-1))} \\;\\text{—}\\; \\text{set }i\\omega=-2:\\; B = \\fr
    note: "Check: A+B+C = −4/5 − 1 + 1/5 = −8/5 □ 0? Re-do cleanly below."
  },
  {
    op: "Clean partial fractions result",
    eq: "Y(\\omega) = \\frac{4}{5}\\cdot\\frac{1}{3+i\\omega} - \\frac{1}{2+i\\omega} + \\frac{1}{5
    note: "Verify by recombining over the common denominator: 4/5· (2+i□)(2−i□) − (3+i□)(2−i□) + 1/5·
  },
  {
    op: "Invert term by term using F{e^{−at}u(t)} = 1/(a+i□) and F{e^{at}u(−t)} = 1/(a−i□)",
    eq: "y(t) = \\frac{4}{5}e^{-3t}u(t) - e^{-2t}u(t) + \\frac{1}{5}e^{2t}u(-t)",
    note: "The third term decays for t < 0 and is zero for t > 0 — the solution is two-sided, as exp
  }
])
```

---

### 45.8.4. Exercise 4: Convolution theorem — LTI system response

A low-pass filter has impulse response $h(t) = e^{-2t}u(t)$ (where $u(t)$ is the unit step: $u(t) = 1$ for $t \geq 0$, 0 otherwise). The input is $f(t) = e^{-t}u(t)$. Find the output $y(t) = (f * h)(t)$ using the Fourier transform.

```
//| echo: false
makeStepperHTML([
  {
    op: "Compute F(□) = F{e^{−t}u(t)}",
    eq: "F(\\omega) = \\int_0^\\infty e^{-t}e^{-i\\omega t}\\,dt = \\int_0^\\infty e^{-(1+i\\omega)t
    note: "The one-sided Laplace/Fourier transform of e^{−at}u(t) is 1/(a+i□)."
  },
  {
    op: "Compute H(□) = F{e^{−2t}u(t)}",
    eq: "H(\\omega) = \\frac{1}{2+i\\omega}"
  },
  {
    op: "Apply convolution theorem: Y(□) = F(□)· H(□)",
    eq: "Y(\\omega) = \\frac{1}{(1+i\\omega)(2+i\\omega)}"
  },
  {
    op: "Partial fractions",
```

```
    eq: "\\frac{1}{(1+i\\omega)(2+i\\omega)} = \\frac{1}{1+i\\omega} - \\frac{1}{2+i\\omega}",
    note: "Verify: (2+iω − (1+iω))/((1+iω)(2+iω)) = 1/((1+iω)(2+iω)) ω"
  },
  {
    op: "Invert term by term",
    eq: "y(t) = e^{-t}u(t) - e^{-2t}u(t) = (e^{-t} - e^{-2t})u(t)",
    note: "At t = 0, y(0) = 0. As t → ∞, y → 0. The output rises and then decays — the step-then-
  }
])
```

---

### 45.8.5. Exercise 5: Parseval's theorem — energy in the frequency domain

The voltage pulse across a 1 Ω resistor is $v(t) = e^{-|t|}$. Use Parseval's theorem to find the total energy dissipated.

```
//| echo: false
makeStepperHTML([
  {
    op: "Total energy = ∫|v(t)|² dt",
    eq: "E = \\int_{-\\infty}^{\\infty} e^{-2|t|}\\,dt = 2\\int_0^\\infty e^{-2t}\\,dt = 2\\cdot\
  },
  {
    op: "Apply Parseval's theorem",
    eq: "E = \\frac{1}{2\\pi}\\int_{-\\infty}^{\\infty}|V(\\omega)|^2\\,d\\omega \\quad \\text{wh
  },
  {
    op: "Verify: compute the frequency-domain integral",
    eq: "\\frac{1}{2\\pi}\\int_{-\\infty}^{\\infty}\\frac{4}{(1+\\omega^2)^2}\\,d\\omega = \\frac
    note: "Using the standard result ∫_{−∞}^{∞} dω/(1+ω²)² = π/2."
  },
  {
    op: "Identify the energy spectral density",
    eq: "S(\\omega) = |V(\\omega)|^2 = \\frac{4}{(1+\\omega^2)^2}",
    note: "S(ω) tells you how the energy is distributed across frequencies. Half the total energy
  }
])
```

# 46. Partial differential equations

Heat, waves, and equilibrium in space and time

A **partial differential equation (PDE)** involves an unknown function of two or more independent variables and its partial derivatives. The three classical PDEs of mathematical physics appear throughout engineering and science, and they are the focus of this chapter:

| Equation | Name | Physics |
|---|---|---|
| $u_t = \alpha^2 u_{xx}$ | Heat equation (parabolic) | Heat conduction, diffusion |
| $u_{tt} = c^2 u_{xx}$ | Wave equation (hyperbolic) | Vibration, acoustics, EM waves |
| $u_{xx} + u_{yy} = 0$ | Laplace equation (elliptic) | Steady-state fields |

Each represents a different physical situation. The heat equation describes irreversible processes that evolve toward equilibrium. The wave equation describes reversible oscillations that persist in time. The Laplace equation describes static equilibrium states with no time dependence at all.

---

## 46.1. Classification of second-order PDEs

A general linear second-order PDE in two variables has the form:

$$A\,u_{xx} + B\,u_{xy} + C\,u_{yy} + D\,u_x + E\,u_y + F\,u = G$$

The **discriminant** $\Delta = B^2 - 4AC$ classifies its type:

$\Delta < 0$ | Elliptic | Laplace, Poisson equations |
$\Delta = 0$ | Parabolic | Heat equation |
$\Delta > 0$ | Hyperbolic | Wave equation |

The classification determines which boundary and initial conditions are appropriate and which numerical methods apply. Elliptic equations need boundary conditions on a closed domain; parabolic and hyperbolic equations need initial conditions in time plus boundary conditions in space.

```
//| echo: false
{
  // ── OPPORTUNITY 2: Three PDE behaviours side by side ──────────────────────
  // Panel A: Heat equation (parabolic) — Gaussian bump spreads irreversibly
  // Panel B: Wave equation (hyperbolic) — bump splits and propagates
  // Panel C: Laplace equation (elliptic) — static steady state, contour lines
  const d3 = await require("d3@7");

  const AMBER = "#f59e0b";
  const BLUE  = "#6b90c4";
  const GREY  = "#9ca3af";
  const GREY_F = "#e5e7eb";

  const W = 580, H = 260;
  const panelW = 170, panelH = 160, gap = 14;
  const PL = 8, PT = 28;

  const container = document.createElement("div");
  container.style.cssText = "font-family:inherit; max-width:640px;";

  // Shared time slider
  const ctrlRow = document.createElement("div");
  ctrlRow.style.cssText = "display:flex; gap:1rem; align-items:center; margin-bottom:0.4rem; flex-wr
  const tLbl = document.createElement("label");
  tLbl.style.cssText = "font-size:0.88em; color:#374151; display:flex; align-items:center; gap:0.4re
  tLbl.textContent = "Time t = ";
  const tDisp = document.createElement("strong");
  tDisp.style.cssText = "color:" + AMBER + "; min-width:3ch;";
  tDisp.textContent = "0.00";
  const tSlider = document.createElement("input");
  tSlider.type = "range"; tSlider.min = 0; tSlider.max = 200; tSlider.value = 0;
  tSlider.style.cssText = "width:220px; accent-color:" + AMBER + ";";
  tLbl.appendChild(tSlider); tLbl.appendChild(tDisp);
  ctrlRow.appendChild(tLbl);
  container.appendChild(ctrlRow);

  const svg = d3.create("svg")
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("width","100%").style("display","block");

  const L = Math.PI;  // domain [0, ⍰]
  const N_x = 200;
  const xs = d3.range(N_x).map(i => i/(N_x-1) * L);

  // ── Panel helper ──────────────────────────────────────────────────────────
  function makePanel(i, title, subtitle) {
    const x0 = PL + i*(panelW+gap);
```

```
  const g = svg.append("g").attr("transform", `translate(${x0},${PT})`);
  g.append("rect").attr("x",0).attr("y",0).attr("width",panelW).attr("height",panelH)
    .attr("fill","#fafafa").attr("stroke",GREY_F).attr("stroke-width",1).attr("rx",3);
  g.append("text").attr("x",panelW/2).attr("y",-10).attr("text-anchor","middle")
    .attr("font-size","10px").attr("fill","#374151").attr("font-weight","600").text(title);
  g.append("text").attr("x",panelW/2).attr("y",panelH+14).attr("text-anchor","middle")
    .attr("font-size","9px").attr("fill",GREY).attr("font-style","italic").text(subtitle);
  return g;
}

const xP = d3.scaleLinear().domain([0, L]).range([6, panelW-6]);
const yP = d3.scaleLinear().domain([-0.1, 1.1]).range([panelH-6, 6]);

// ── Initial condition: Gaussian centred at □/2 ──────────────────────────
function gauss0(x) { return Math.exp(-20*(x-L/2)*(x-L/2)); }

// ── Panel A: Heat equation on [0,□], Dirichlet BCs ──────────────────────
// u(x,t) = □ B□ sin(nx) e^{-n²t} ; B□ = (2/□) □₀^□ f(x)sin(nx)dx
const gA = makePanel(0, "Heat equation", "spreads irreversibly");
// Precompute B_n coefficients of Gaussian (numerically)
const N_modes = 25;
const Bn_heat = d3.range(1, N_modes+1).map(n => {
  let s = 0;
  xs.forEach((x,i) => {
    const dx = i===0||i===N_x-1 ? 0.5/(N_x-1) : 1/(N_x-1);
    s += gauss0(x) * Math.sin(n*x) * dx * L;
  });
  return 2/L * s;
});

function heatSolution(x, t) {
  let u = 0;
  for (let n = 1; n <= N_modes; n++) {
    u += Bn_heat[n-1] * Math.sin(n*x) * Math.exp(-n*n*t);
  }
  return u;
}

// Initial condition trace
gA.append("path").attr("fill","none").attr("stroke",GREY).attr("stroke-width",1)
  .attr("stroke-dasharray","3,2")
  .datum(xs.map(x=>[xP(x),yP(gauss0(x))])).attr("d",d3.line().x(d=>d[0]).y(d=>d[1]));
gA.append("line").attr("x1",6).attr("x2",panelW-6).attr("y1",yP(0)).attr("y2",yP(0))
  .attr("stroke",GREY_F).attr("stroke-width",0.8);
[0,1].forEach(v=>{ gA.append("text").attr("x",2).attr("y",yP(v)).attr("font-size","7.5px").attr
const heatPath = gA.append("path").attr("fill","none").attr("stroke",AMBER).attr("stroke-width"
```

```
// ── Panel B: Wave equation d'Alembert (u(x,t) = ½[f(x+ct)+f(x-ct)]) ──────
// c = 1, f defined on ℝ by odd periodic extension of Gaussian
const gB = makePanel(1, "Wave equation", "propagates without decay");
gB.append("path").attr("fill","none").attr("stroke",GREY).attr("stroke-width",1)
  .attr("stroke-dasharray","3,2")
  .datum(xs.map(x=>[xP(x),yP(gauss0(x))])).attr("d",d3.line().x(d=>d[0]).y(d=>d[1]));
gB.append("line").attr("x1",6).attr("x2",panelW-6).attr("y1",yP(0)).attr("y2",yP(0))
  .attr("stroke",GREY_F).attr("stroke-width",0.8);
[0,1].forEach(v=>{ gB.append("text").attr("x",2).attr("y",yP(v)).attr("font-size","7.5px").attr("f

// Odd periodic extension for D'Alembert
function periodicOdd(x) {
  // Reduce to [-π, π] via odd periodic extension of [0,π]
  let xn = ((x % (2*L)) + 2*L) % (2*L);  // in [0, 2L)
  if (xn > L) { xn = 2*L - xn; return -gauss0(xn); }
  return gauss0(xn);
}
function waveSolution(x, t) {
  return 0.5 * (periodicOdd(x + t) + periodicOdd(x - t));
}
const wavePath = gB.append("path").attr("fill","none").attr("stroke",AMBER).attr("stroke-width",2

// ── Panel C: Laplace equation (static) — contour lines of sin(πx/L)sinh(π(1-y)/L)/sinh(π/L)
const gC = makePanel(2, "Laplace equation", "steady state only");

// Render Laplace solution on a grid (x ∈ [0,π], y ∈ [0,1]) as filled contours
const xPc = d3.scaleLinear().domain([0,L]).range([6,panelW-6]);
const yPc = d3.scaleLinear().domain([0,1]).range([panelH-6,6]);

function laplace(x, y) {
  // u = sum_n Cn sin(nπx/L) sinh(nπ(1-y)/L) / sinh(nπ/L)
  // Use n=1 only (boundary: sin(x) on y=0)
  return Math.sin(x) * Math.sinh(Math.PI*(1-y)/L) / Math.sinh(Math.PI/L);
}

// Draw coloured heatmap
const lN = 30, lM = 30;
for (let i = 0; i < lN; i++) {
  for (let j = 0; j < lM; j++) {
    const x0 = i/lN*L, x1 = (i+1)/lN*L;
    const y0 = j/lM, y1 = (j+1)/lM;
    const v = laplace((x0+x1)/2, (y0+y1)/2);
    const t = Math.max(0, Math.min(1, v));
    const r = Math.round(255 - t*180);
    const g2 = Math.round(220 - t*100);
    const b2 = Math.round(180 - t*120);
    gC.append("rect")
```

```
      .attr("x",xPc(x0)).attr("y",yPc(y1))
      .attr("width",xPc(x1)-xPc(x0)+0.5).attr("height",yPc(y0)-yPc(y1)+0.5)
      .attr("fill",`rgb(${r},${g2},${b2})`);
  }
}

// Contour lines
const contourLevels = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7];
contourLevels.forEach(lv => {
  // Draw iso-x line: y where u(□/2, y) = lv → sinh(□(1-y)/L) = lv*sinh(□/L)
  const target = lv * Math.sinh(Math.PI/L);
  const yIso = 1 - Math.log(target + Math.sqrt(target*target+1)) / (Math.PI/L);
  if (yIso >= 0 && yIso <= 1) {
    const pts2 = d3.range(20).map(i => {
      const x_ = i/19*L;
      const v_ = laplace(x_, yIso + 0.001);
      // find y where u(x_,y) = lv by simple search
      let yy = yIso;
      for (let k = 0; k < 20; k++) {
        const uu = laplace(x_, yy);
        const duyy = -Math.sin(x_) * Math.PI/L * Math.cosh(Math.PI*(1-yy)/L) / Math.sinh(Math.P
        if (Math.abs(duyy) > 1e-8) yy -= (uu - lv) / duyy;
        yy = Math.max(0, Math.min(1, yy));
      }
      return [xPc(x_), yPc(yy)];
    });
    gC.append("path").attr("fill","none").attr("stroke","rgba(0,0,0,0.25)").attr("stroke-width"
      .datum(pts2).attr("d",d3.line().x(d=>d[0]).y(d=>d[1]));
  }
});

// Boundary labels
gC.append("text").attr("x",panelW/2).attr("y",panelH).attr("text-anchor","middle")
  .attr("font-size","7.5px").attr("fill","#b45309").text("u = sin(x) (hot)");
gC.append("text").attr("x",panelW/2).attr("y",12).attr("text-anchor","middle")
  .attr("font-size","7.5px").attr("fill",GREY).text("u = 0 (top/sides)");

// ── Time-slider update ──────────────────────────────────────────────────
function update3() {
  const t = +tSlider.value / 200 * 2.0;
  tDisp.textContent = t.toFixed(2);

  heatPath.datum(xs.map(x=>[xP(x),yP(Math.max(0,heatSolution(x,t)))])).attr("d",d3.line().x(d=>
  wavePath.datum(xs.map(x=>[xP(x),yP(Math.max(-0.05,waveSolution(x,t)))])).attr("d",d3.line().x
}

tSlider.oninput = update3;
```

```
  update3();

  // Note for Laplace
  svg.append("text").attr("x",PL+2*(panelW+gap)+panelW/2).attr("y",PT+panelH+28)
    .attr("text-anchor","middle").attr("font-size","8.5px").attr("fill",BLUE).attr("font-style","ita
    .text("(no time slider — equilibrium only)");

  // Legend
  const legend = document.createElement("div");
  legend.style.cssText = "display:flex; gap:1.2rem; font-size:0.78em; margin-top:0.2rem; flex-wrap:w
  [["—— Solution at time t", AMBER], ["--- Initial condition", GREY]]
    .forEach(([lab, col]) => {
      const s = document.createElement("span"); s.style.color = col; s.textContent = lab;
      legend.appendChild(s);
    });
  container.appendChild(svg.node());
  container.appendChild(legend);
  return container;
}
```

---

## 46.2. Separation of variables

**Separation of variables** is the principal analytic method for solving PDEs on regular domains. The key assumption: the solution factors into a product of functions of each individual variable:

$$u(x,t) = X(x)\,T(t)$$

Substituting into the PDE, each side can be written as a function of only one variable. A function of $x$ alone cannot change as $t$ changes, and a function of $t$ alone cannot change as $x$ changes; the only way two such quantities can be equal for every $x$ and every $t$ is if both are the same constant. Call it the **separation constant** $-\lambda$.

This splits the PDE into two ODEs, one in $x$ (the **eigenvalue problem**) and one in $t$. The boundary conditions restrict $\lambda$ to a discrete set of **eigenvalues** $\lambda_n$, each associated with an **eigenfunction** $X_n(x)$. The general solution is a superposition of all eigenfunction solutions:

$$u(x,t) = \sum_{n=1}^{\infty} X_n(x)\,T_n(t)$$

with coefficients determined by the initial conditions via Fourier series.

---

## 46.3. The heat equation

### 46.3.1. Derivation

Consider a thin rod of length $L$, thermally insulated on its lateral surface so that heat flows only along its length. Let $u(x,t)$ be the temperature at position $x$ and time $t$.

Fourier's law of heat conduction states that the heat flux $q$ (energy per unit area per unit time) is proportional to the temperature gradient: $q = -k\,u_x$, where $k > 0$ is the thermal conductivity.

Conservation of energy in a small element $[x, x + \Delta x]$ gives:

$$\rho c\,\Delta x\,u_t = -\Delta x\,q_x = k\,\Delta x\,u_{xx}$$

Dividing by $\rho c\,\Delta x$ gives the **heat equation**:

$$u_t = \alpha^2\,u_{xx}, \qquad \alpha^2 = \frac{k}{\rho c}$$

where $\rho$ is density, $c$ is specific heat capacity, and $\alpha$ is the **thermal diffusivity**.

### 46.3.2. Boundary and initial conditions

For the problem to be well-posed, we need:

- **Initial condition (IC)**: $u(x,0) = f(x)$ for $0 < x < L$ (the initial temperature profile)
- **Boundary conditions (BCs)**: two conditions at the ends.

Common boundary conditions: - **Dirichlet** (fixed temperature): $u(0,t) = u(L,t) = 0$ - **Neumann** (insulated ends): $u_x(0,t) = u_x(L,t) = 0$

### 46.3.3. Solution by separation of variables

Assume $u(x,t) = X(x)T(t)$. Substituting into $u_t = \alpha^2 u_{xx}$:

$$XT' = \alpha^2\,X''\,T \implies \frac{T'}{\alpha^2 T} = \frac{X''}{X} = -\lambda$$

The separation constant is written as $-\lambda$ (the negative sign is chosen so that $\lambda > 0$ gives physically decaying solutions). This means the sign choice is bookkeeping, not new physics: choosing $+\lambda$ would force the useful solutions to have $\lambda < 0$ instead.

**Spatial ODE** with Dirichlet BCs $X(0) = X(L) = 0$:

$$X'' + \lambda X = 0, \quad X(0) = 0, \quad X(L) = 0$$

The cases $\lambda \leq 0$ produce only the trivial solution: $\lambda = 0$ gives $X'' = 0$ so $X = cx$, which satisfies $X(0) = 0$ only if $c = 0$; $\lambda < 0$ gives real exponential solutions $e^{\pm\sqrt{-\lambda}\,x}$ that cannot satisfy both boundary conditions simultaneously unless $X \equiv 0$. Only $\lambda > 0$ gives oscillatory solutions $\sin(\sqrt{\lambda}\,x)$ that can vanish at both endpoints. Imposing $X(L) = \sin(\sqrt{\lambda}\,L) = 0$ then forces $\sqrt{\lambda}\,L = n\pi$, so $\lambda = \lambda_n = n^2\pi^2/L^2$ for $n = 1, 2, 3, \ldots$, giving:

$$X_n(x) = \sin\left(\frac{n\pi x}{L}\right)$$

**Temporal ODE** for each $\lambda_n$:

$$T_n' = -\alpha^2 \lambda_n T_n \implies T_n(t) = e^{-\alpha^2 n^2 \pi^2 t/L^2}$$

**General solution:**

$$u(x,t) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) e^{-\alpha^2 n^2 \pi^2 t/L^2}$$

This means each Fourier mode evolves independently, and the higher-frequency modes decay faster because their exponential factors contain $n^2$.

**Applying the initial condition** $u(x,0) = f(x)$:

$$f(x) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right)$$

This is the Fourier sine series of $f$ on $[0, L]$. Therefore:

$$B_n = \frac{2}{L}\int_0^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx$$

**Physical interpretation.** Each term $B_n \sin(n\pi x/L) e^{-\alpha^2 n^2 \pi^2 t/L^2}$ is a **heat mode**: a spatial pattern (the $n$-th Fourier sine mode) that decays exponentially at rate $\alpha^2 n^2 \pi^2/L^2$. Higher modes (larger $n$, more oscillations in space) decay faster — by a factor of $n^2$. Sharp spatial features smooth out quickly; broad features persist.

Use the time slider to watch the fast-decaying third mode disappear and leave the broad first mode behind.

```
//| echo: false
{
  const L = Math.PI;
  function temp(x, t) { return Math.sin(x) * Math.exp(-t) + 0.5 * Math.sin(3 * x) * Math.exp(-9 * t
  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }

  const container = document.createElement("div");
  container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
```

```
const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem
const label = document.createElement("label"); label.textContent = "Time t";
const slider = document.createElement("input");
slider.type = "range"; slider.min = "0"; slider.max = "2"; slider.step = "0.02"; slider.value =
const out = document.createElement("span"); out.style.cssText = "font-variant-numeric:tabular-n
controls.append(label, slider, out);

const svg = svgEl("svg");
svg.setAttribute("viewBox", "0 0 720 360");
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const legend = document.createElement("div");
legend.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-top:0.55rem; color:#3341
const m1Box = document.createElement("span");
const m3Box = document.createElement("span");
legend.append(m1Box, m3Box);

function redraw() {
  const t = Number(slider.value);
  out.textContent = t.toFixed(2);
  svg.replaceChildren();
  const sx = x => 55 + x / L * 620;
  const sy = y => 180 - y * 120;
  const axis = svgEl("line");
  axis.setAttribute("x1", "55"); axis.setAttribute("y1", "180");
  axis.setAttribute("x2", "675"); axis.setAttribute("y2", "180");
  axis.setAttribute("stroke", "#64748b"); axis.setAttribute("stroke-width", "1.2");
  svg.appendChild(axis);

  const initPath = [], curPath = [];
  for (let i = 0; i < 220; i++) {
    const x = i / 219 * L;
    initPath.push(`${i === 0 ? "M" : "L"} ${sx(x)} ${sy(temp(x, 0))}`);
    curPath.push(`${i === 0 ? "M" : "L"} ${sx(x)} ${sy(temp(x, t))}`);
  }
  [["#94a3b8", initPath.join(" "), 2], ["#f97316", curPath.join(" "), 3]].forEach(([color, d, w
    const p = svgEl("path");
    p.setAttribute("d", d); p.setAttribute("fill", "none"); p.setAttribute("stroke", color); p.
    svg.appendChild(p);
  });

  m1Box.textContent = `mode 1 amplitude = e^(−t) = ${Math.exp(-t).toFixed(3)}`;
  m3Box.textContent = `mode 3 amplitude = 0.5e^(−9t) = ${(0.5 * Math.exp(-9 * t)).toFixed(3)}`;
```

```
  }

  slider.addEventListener("input", redraw);
  container.append(controls, svg, legend);
  redraw();
  return container;
}
```

---

## 46.4. The wave equation

### 46.4.1. Setup

A flexible string stretched between $x = 0$ and $x = L$ is displaced and released. Let $u(x, t)$ be the transverse displacement. Newton's second law applied to a small element yields the **wave equation**:

$$u_{tt} = c^2 u_{xx}, \qquad c = \sqrt{\frac{T}{\mu}}$$

where $T$ is the string tension and $\mu$ is the linear mass density (mass per unit length, kg/m). The constant $c$ is the wave speed.

Unlike the heat equation, the wave equation is second-order in time and requires **two** initial conditions: - $u(x, 0) = f(x)$ (initial displacement) - $u_t(x, 0) = g(x)$ (initial velocity)

### 46.4.2. Solution by separation of variables

Assume $u(x, t) = X(x)T(t)$ with fixed-end BCs $u(0, t) = u(L, t) = 0$. The spatial problem is identical to the heat equation:

$$\lambda_n = \frac{n^2\pi^2}{L^2}, \qquad X_n(x) = \sin\left(\frac{n\pi x}{L}\right)$$

The temporal ODE is now:

$$T_n'' = -c^2\lambda_n T_n \implies T_n(t) = A_n \cos\left(\frac{n\pi c t}{L}\right) + B_n \sin\left(\frac{n\pi c t}{L}\right)$$

**General solution:**

$$u(x, t) = \sum_{n=1}^{\infty} \sin\left(\frac{n\pi x}{L}\right)\left[A_n \cos\left(\frac{n\pi c t}{L}\right) + B_n \sin\left(\frac{n\pi c t}{L}\right)\right]$$

**Applying initial conditions:**

At $t = 0$: $u(x, 0) = \sum A_n \sin(n\pi x/L) = f(x) \longrightarrow$ Fourier sine coefficients of $f$.

Differentiating: $u_t(x, 0) = \sum B_n(n\pi c/L) \sin(n\pi x/L) = g(x) \longrightarrow$ Fourier sine coefficients of $g$, divided by $n\pi c/L$.

### 46.4.3. Standing waves and natural frequencies

Each term $\sin(n\pi x/L)\cos(n\pi ct/L)$ is a **standing wave**: a spatial pattern that oscillates in time at a fixed frequency. The $n$-th mode oscillates at angular frequency $\omega_n = n\pi c/L$ (frequency $f_n = nc/(2L)$).

These are the **natural frequencies** (harmonics) of the string. A guitar string of length $L$, tension $T$, and linear density $\mu$ (so $c = \sqrt{T/\mu}$) produces: - Fundamental: $f_1 = c/(2L)$ - First overtone: $f_2 = c/L = 2f_1$ - $n$-th harmonic: $f_n = nf_1$

The ratio of overtones is always integer multiples of the fundamental — this is why stringed instruments produce musical notes rather than noise.

Use the mode selector to compare the standing-wave shapes and their increasing frequencies.

```
//| echo: false
{
  const L = Math.PI, c = 1;
  function shape(n, x) { return Math.sin(n * Math.PI * x / L); }
  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }

  const container = document.createElement("div");
  container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem
  const label = document.createElement("label"); label.textContent = "Mode n";
  const select = document.createElement("select");
  for (let n = 1; n <= 5; n++) {
    const opt = document.createElement("option");
    opt.value = String(n); opt.textContent = String(n);
    select.appendChild(opt);
  }
  const freqOut = document.createElement("span");
  controls.append(label, select, freqOut);

  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 720 360");
  svg.style.width = "100%";
  svg.style.height = "auto";
  svg.style.border = "1px solid #e2e8f0";
  svg.style.background = "#fff";

  let timer = 0;
  function redraw() {
    const n = Number(select.value);
```

```
    const omega = n * Math.PI * c / L;
    const fHz = n * c / (2 * L);
    freqOut.textContent = `f□ = ${fHz.toFixed(3)}`;
    svg.replaceChildren();
    const sx = x => 55 + x / L * 620;
    const syTop = y => 120 - y * 70;
    const syBot = y => 285 - y * 55;

    const topAxis = svgEl("line");
    topAxis.setAttribute("x1", "55"); topAxis.setAttribute("y1", "120");
    topAxis.setAttribute("x2", "675"); topAxis.setAttribute("y2", "120");
    topAxis.setAttribute("stroke", "#64748b"); topAxis.setAttribute("stroke-width", "1.2");
    const botAxis = svgEl("line");
    botAxis.setAttribute("x1", "55"); botAxis.setAttribute("y1", "285");
    botAxis.setAttribute("x2", "675"); botAxis.setAttribute("y2", "285");
    botAxis.setAttribute("stroke", "#64748b"); botAxis.setAttribute("stroke-width", "1.2");
    svg.append(topAxis, botAxis);

    const dynamicPath = [], staticPath = [];
    for (let i = 0; i < 220; i++) {
      const x = i / 219 * L;
      const y = shape(n, x);
      dynamicPath.push(`${i === 0 ? "M" : "L"} ${sx(x)} ${syTop(y * Math.cos(omega * timer))}`);
      staticPath.push(`${i === 0 ? "M" : "L"} ${sx(x)} ${syBot(y)}`);
    }
    [["#f97316", dynamicPath.join(" "), 3], ["#2563eb", staticPath.join(" "), 3]].forEach(([color, d
      const p = svgEl("path");
      p.setAttribute("d", d); p.setAttribute("fill", "none"); p.setAttribute("stroke", color); p.set
      svg.appendChild(p);
    });
  }

  select.addEventListener("input", redraw);
  const id = setInterval(() => { timer += 0.06; redraw(); }, 50);
  invalidation.then(() => clearInterval(id));

  container.append(controls, svg);
  redraw();
  return container;
}
```

### 46.4.4. D'Alembert's solution

There is an elegant alternative form of the wave equation solution. D'Alembert's solution:

$$u(x,t) = \frac{1}{2}\left[f(x+ct) + f(x-ct)\right] + \frac{1}{2c}\int_{x-ct}^{x+ct} g(s)\,ds$$

This expresses the solution as two travelling waves — $f(x-ct)$ moving right at speed $c$, and $f(x+ct)$ moving left at speed $c$. For the initial velocity zero ($g = 0$), the initial displacement simply splits in two and propagates in opposite directions.

---

## 46.5. The Laplace equation

### 46.5.1. Setup

The **Laplace equation** $\nabla^2 u = u_{xx} + u_{yy} = 0$ describes **steady-state** distributions: temperature in a solid after all transients have died away, electrostatic potential in a charge-free region, fluid velocity potential for irrotational flow.

A function satisfying the Laplace equation is **harmonic**. Key property: a harmonic function attains its maximum and minimum values on the boundary, never in the interior. This means the steady-state temperature inside a region is always between the extreme boundary temperatures — physically obvious, mathematically provable.

### 46.5.2. Dirichlet problem in a rectangle

Find $u(x,y)$ in $0 < x < a$, $0 < y < b$, satisfying:

$$u_{xx} + u_{yy} = 0$$

with boundary conditions $u(0,y) = u(a,y) = u(x,b) = 0$ and $u(x,0) = f(x)$.

Assume $u(x,y) = X(x)Y(y)$:

$$\frac{X''}{X} = -\frac{Y''}{Y} = -\lambda$$

With $X(0) = X(a) = 0$: eigenvalues $\lambda_n = n^2\pi^2/a^2$, eigenfunctions $X_n = \sin(n\pi x/a)$.

The $Y$ equation becomes $Y'' - \lambda_n Y = 0$ with $Y(b) = 0$. Solutions:

$$Y_n(y) = \sinh\left(\frac{n\pi(b-y)}{a}\right)$$

(sinh chosen to satisfy $Y_n(b) = 0$; $Y_n(0) = \sinh(n\pi b/a) \neq 0$).

This means the sign in the ODE decides the family of functions you get: $Y'' + \lambda Y = 0$ produces oscillatory sine and cosine terms, while $Y'' - \lambda Y = 0$ produces exponential or hyperbolic behaviour.

**General solution:**

$$u(x, y) = \sum_{n=1}^{\infty} C_n \sin\left(\frac{n\pi x}{a}\right) \sinh\left(\frac{n\pi(b-y)}{a}\right)$$

Applying $u(x, 0) = f(x)$:

$$C_n = \frac{2}{a \sinh(n\pi b/a)} \int_0^a f(x) \sin\left(\frac{n\pi x}{a}\right) dx$$

---

## 46.6. Summary: the method in four steps

Regardless of which classical PDE applies, the separation-of-variables procedure follows the same steps:

1. **Assume** $u = X(x)T(t)$ (or $X(x)Y(y)$ for the Laplace equation). Substitute and separate.
2. **Solve the spatial eigenvalue problem** with the given boundary conditions. This determines the eigenvalues $\lambda_n$ and eigenfunctions $X_n$.
3. **Solve the temporal (or second spatial) ODE** for each $\lambda_n$, giving $T_n(t)$.
4. **Apply initial conditions**: express $f(x)$ as a Fourier series in the eigenfunctions $\{X_n\}$ to determine the coefficients.

The Fourier series computation in step 4 is identical to what was developed in the previous two chapters. PDEs give the Fourier series its deepest physical motivation.

---

## 46.7. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
```

```
  for (let i = 0; i < current; i++) {
    const s = steps[i];
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align
    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;
    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
    row.appendChild(opCell);
    row.appendChild(eqCell);
    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }
    stepsDiv.appendChild(row);
  }
  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "Solution complete.";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
const counter = document.createElement("span");
counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

function updateButtons() {
  nextBtn.disabled = current === totalSteps;
  nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
  counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
}

nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
controls.appendChild(nextBtn);
```

```
  controls.appendChild(resetBtn);
  controls.appendChild(counter);
  container.appendChild(stepsDiv);
  container.appendChild(controls);
  renderSteps();
  updateButtons();
  return container;
}
```

---

### 46.7.1. Exercise 1: Heat equation — single-mode initial condition

Solve $u_t = 4u_{xx}$ on $0 < x < \pi$, $t > 0$, with $u(0,t) = u(\pi,t) = 0$ and $u(x,0) = 3\sin(2x)$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Identify parameters",
    eq: "L = \\pi, \\quad \\alpha^2 = 4, \\quad f(x) = 3\\sin(2x)",
    note: "The initial condition is already a single Fourier sine mode."
  },
  {
    op: "Write the general solution",
    eq: "u(x,t) = \\sum_{n=1}^{\\infty} B_n\\,\\sin(nx)\\,e^{-4n^2 t}"
  },
  {
    op: "Match n = 2 mode: B₂ = 3, all other B□ = 0",
    eq: "u(x,0) = B_2\\sin(2x) = 3\\sin(2x) \\implies B_2 = 3"
  },
  {
    op: "Write the solution",
    eq: "u(x,t) = 3\\sin(2x)\\,e^{-16t}"
  },
  {
    op: "Verify: check u_t = 4u_xx",
    eq: "u_t = -48\\sin(2x)\\,e^{-16t}, \\quad 4u_{xx} = 4\\cdot(-4)\\cdot 3\\sin(2x)\\,e^{-16t} = -
    note: "The spatial mode sin(2x) stays fixed in shape; it only decays in amplitude."
  }
])
```

---

### 46.7.2. Exercise 2: Heat equation — arbitrary initial condition

Solve $u_t = u_{xx}$ on $0 < x < \pi$, $t > 0$, with $u(0,t) = u(\pi,t) = 0$ and $u(x,0) = x(\pi - x)$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write the solution form and coefficient formula",
    eq: "u(x,t) = \\sum_{n=1}^{\\infty} B_n\\sin(nx)\\,e^{-n^2 t}, \\quad B_n = \\frac{2}{\\pi}\\
  },
  {
    op: "Integrate by parts twice",
    eq: "\\int_0^\\pi x(\\pi-x)\\sin(nx)\\,dx = \\frac{2}{n^3}\\bigl[1-(-1)^n\\bigr]",
    note: "Integrating x(□−x)sin(nx) by parts: each differentiation of x(□−x) reduces the polynom
  },
  {
    op: "Compute B□",
    eq: "B_n = \\frac{2}{\\pi}\\cdot\\frac{2}{n^3}\\bigl[1-(-1)^n\\bigr] = \\frac{4}{\\pi n^3}\\b
    note: "B□ = 8/(□n³) for odd n; B□ = 0 for even n."
  },
  {
    op: "Write the solution",
    eq: "u(x,t) = \\frac{8}{\\pi}\\sum_{k=0}^{\\infty}\\frac{\\sin((2k+1)x)}{(2k+1)^3}\\,e^{-(2k+
  },
  {
    op: "Interpret: check which mode dominates at large t",
    eq: "\\text{As } t\\to\\infty: \\quad u \\approx \\frac{8}{\\pi}\\sin(x)\\,e^{-t}",
    note: "The n=1 mode decays slowest (rate 1). At t = 2, the n=3 term has decayed to e^{−9}/e^{
  }
])
```

---

### 46.7.3. Exercise 3: Wave equation — plucked string

A string of length $L = \pi$ and wave speed $c = 2$ is plucked to a triangular initial shape $f(x) = \frac{2}{\pi}\min(x, \pi - x)$ and released from rest ($g(x) = 0$). Write the solution.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write the general solution (g = 0 means all B□ = 0)",
    eq: "u(x,t) = \\sum_{n=1}^{\\infty} A_n\\sin(nx)\\cos(2nt)",
    note: "With g(x) = 0, the B□ sin terms vanish."
  },
  {
```

```
    op: "Compute A□ = Fourier sine coefficients of f(x) = (2/□)min(x, □−x)",
    eq: "A_n = \\frac{2}{\\pi}\\int_0^\\pi f(x)\\sin(nx)\\,dx = \\frac{4}{\\pi}\\int_0^{\\pi/2}\\fra
  },
  {
    op: "By symmetry of the triangle, only odd n survive",
    eq: "A_n = \\frac{8}{n^2\\pi^2}\\sin\\!\\left(\\frac{n\\pi}{2}\\right) = \\begin{cases} (-1)^k\\
    note: "The even modes are absent because the triangle is symmetric about x = □/2 — only odd sine
  },
  {
    op: "Write the first three terms",
    eq: "u(x,t) \\approx \\frac{8}{\\pi^2}\\left[\\sin x\\cos 2t - \\frac{\\sin 3x\\cos 6t}{9} + \\f
  },
  {
    op: "Interpret: D'Alembert form",
    eq: "u(x,t) = \\tfrac{1}{2}\\bigl[f(x+2t) + f(x-2t)\\bigr]",
    note: "The triangular initial displacement splits into two half-height triangles travelling in o
  }
])
```

---

### 46.7.4. Exercise 4: Laplace equation — steady-state temperature

Find the steady-state temperature $u(x, y)$ in the rectangle $0 < x < \pi$, $0 < y < 1$, with $u = 0$ on three sides and $u(x, 0) = \sin(x) + \frac{1}{3}\sin(3x)$ on the bottom.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write the general solution with Y(1) = 0",
    eq: "u(x,y) = \\sum_{n=1}^{\\infty} C_n\\sin(nx)\\sinh(n(1-y))",
    note: "sinh(n(1−y)) satisfies the top BC: sinh(n(1−1)) = sinh(0) = 0."
  },
  {
    op: "Apply bottom BC u(x,0) = sin(x) + (1/3)sin(3x)",
    eq: "\\sum_{n=1}^{\\infty} C_n\\sinh(n)\\sin(nx) = \\sin(x) + \\tfrac{1}{3}\\sin(3x)"
  },
  {
    op: "Match terms: n = 1 and n = 3 only",
    eq: "C_1\\sinh(1) = 1 \\implies C_1 = \\frac{1}{\\sinh 1} \\approx 0.851"
  },
  {
    op: "Match n = 3 term",
    eq: "C_3\\sinh(3) = \\frac{1}{3} \\implies C_3 = \\frac{1}{3\\sinh 3} \\approx 0.033",
    note: "sinh(3) □ 10.02, so the n=3 term contributes much less near y = 0 and decays faster in y
  },
```

```
  {
    op: "Write the exact solution",
    eq: "u(x,y) = \\frac{\\sin x\\,\\sinh(1-y)}{\\sinh 1} + \\frac{\\sin 3x\\,\\sinh(3(1-y))}{3\\
  },
  {
    op: "Verify boundary conditions",
    eq: "u(x,0) = \\frac{\\sin x}{1} + \\frac{\\sin 3x}{3} \\checkmark \\qquad u(x,1) = 0 \\check
  }
])
```

---

### 46.7.5. Exercise 5: Classify these PDEs

Classify each of the following equations as elliptic, parabolic, or hyperbolic. Identify a physical situation each could model.

**(a)** $u_{xx} + 4u_{xy} + 4u_{yy} = 0$ **(b)** $3u_{xx} - u_{yy} + u_x = 0$ **(c)** $u_{xx} + u_{yy} + u_{zz} = \rho$

```
//| echo: false
makeStepperHTML([
  {
    op: "Recall: discriminant □ = B² − 4AC",
    eq: "\\Delta < 0 \\Rightarrow \\text{elliptic}, \\quad \\Delta = 0 \\Rightarrow \\text{parabo
  },
  {
    op: "(a): A=1, B=4, C=4 → □ = 16 − 16 = 0",
    eq: "\\text{Parabolic.}",
    note: "Could model heat diffusion or a diffusion process in a coordinate-rotated frame."
  },
  {
    op: "(b): A=3, B=0, C=−1 → □ = 0 − 4(3)(−1) = 12 > 0",
    eq: "\\text{Hyperbolic.}",
    note: "Resembles the wave equation; could model wave propagation with a drift term."
  },
  {
    op: "(c): 3D Poisson equation u_xx + u_yy + u_zz = □",
    eq: "\\text{Elliptic (all eigenvalues of the coefficient matrix are positive and same sign).}
    note: "The 2D discriminant □ = B²−4AC does not apply to 3D PDEs. Classification in 3D uses th
  }
])
```

---

### 46.7.6. Exercise 6: Non-homogeneous boundary conditions

When the boundary conditions are non-zero — as when one end of a rod is held at a fixed nonzero temperature — separation of variables cannot be applied directly because the eigenfunctions $\sin(n\pi x/L)$ all vanish at the boundaries. The fix is to subtract a **steady-state solution** $v(x)$ that satisfies the boundary conditions and $v_{xx} = 0$, so that the remainder $w = u - v$ has homogeneous boundary conditions and the standard series method applies. The final answer is $u = v + w$: steady state plus decaying transient.

Solve $u_t = u_{xx}$ on $0 < x < 1$, $t > 0$, with $u(0,t) = 0$, $u(1,t) = 1$, and $u(x,0) = 0$.

*(Steady-state plus transient method.)*

```
//| echo: false
makeStepperHTML([
  {
    op: "Find the steady-state solution v(x)",
    eq: "v_{xx} = 0, \\quad v(0) = 0, \\quad v(1) = 1 \\implies v(x) = x"
  },
  {
    op: "Let w(x,t) = u(x,t) - v(x) = u(x,t) - x",
    eq: "w_t = u_t = u_{xx} = (w+x)_{xx} = w_{xx}",
    note: "w satisfies the same heat equation, but now with homogeneous BCs."
  },
  {
    op: "Boundary and initial conditions for w",
    eq: "w(0,t) = 0, \\quad w(1,t) = 0, \\quad w(x,0) = u(x,0) - x = -x"
  },
  {
    op: "General solution for w with Fourier sine coefficients of −x",
    eq: "B_n = 2\\int_0^1 (-x)\\sin(n\\pi x)\\,dx = \\frac{2(-1)^n}{n\\pi}",
    note: "Using the standard result ∫₀¹ x sin(nπx) dx = (−1)^{n+1}/(nπ)."
  },
  {
    op: "Assemble the full solution u = v + w",
    eq: "u(x,t) = x + \\frac{2}{\\pi}\\sum_{n=1}^{\\infty}\\frac{(-1)^n}{n}\\sin(n\\pi x)\\,e^{-n^2
    note: "At large t, the transient (w) decays and u approaches the steady state x — a rod uniforml
  }
])
```

# Part X.

# Volume 7 — Complex analysis

# 47. Complex functions

Differentiability in the complex plane and the Cauchy-Riemann equations

A function $f$ of a complex variable maps complex numbers to complex numbers. Writing $z = x + iy$ and separating $f(z) = u(x, y) + iv(x, y)$, the function is a pair of real-valued functions of two real variables. The condition on $u$ and $v$ that makes $f$ genuinely complex-differentiable — the Cauchy-Riemann equations — is what the rest of the chapter develops.

```
//| echo: false
{
  // Stability half-plane explorer
  // s-plane with green (Re<0) and red (Re>0) half-planes
  // Draggable pole s (via sliders)
  // Inset: time response Re(e^{st}) plotted for t in [0, 5]

  const W = 560, H = 280, SCALE = 65, CX = 170, CY = H/2;
  const svgNS = "http://www.w3.org/2000/svg";

  const sReSl = Inputs.range([-2.5, 2.5], { value: -1.0, step: 0.1, label: "Re(s) — pole real par
  const sImSl = Inputs.range([-3.0, 3.0], { value: 1.5, step: 0.1, label: "Im(s) — pole imaginary

  function buildViz(sRe, sIm) {
    const svg = document.createElementNS(svgNS,"svg");
    svg.setAttribute("width",W); svg.setAttribute("height",H);
    svg.style.cssText="background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:b

    // Left half: s-plane (x in [0,340] roughly)
    const PLANE_W = 340;

    // Green / red half-plane fills
    const axX = CX; // screen x for Re(s)=0
    const greenRect = document.createElementNS(svgNS,"rect");
    greenRect.setAttribute("x","0"); greenRect.setAttribute("y","0");
    greenRect.setAttribute("width",axX.toString()); greenRect.setAttribute("height",H);
    greenRect.setAttribute("fill","#059669"); greenRect.setAttribute("opacity","0.12");
    svg.appendChild(greenRect);

    const redRect = document.createElementNS(svgNS,"rect");
    redRect.setAttribute("x",axX.toString()); redRect.setAttribute("y","0");
```

```
redRect.setAttribute("width",(PLANE_W-axX).toString()); redRect.setAttribute("height",H);
redRect.setAttribute("fill","#dc2626"); redRect.setAttribute("opacity","0.12");
svg.appendChild(redRect);

// Grid lines (s-plane)
for (let i=-3;i<=3;i++){
  const xl=document.createElementNS(svgNS,"line");
  const lx=CX+i*SCALE;
  xl.setAttribute("x1",lx); xl.setAttribute("y1","0"); xl.setAttribute("x2",lx); xl.setAttribute
  xl.setAttribute("stroke","#e5e7eb"); xl.setAttribute("stroke-width","0.5"); svg.appendChild(xl
  const yl=document.createElementNS(svgNS,"line");
  const ly=CY-i*SCALE;
  yl.setAttribute("x1","0"); yl.setAttribute("y1",ly); yl.setAttribute("x2",PLANE_W); yl.setAtt
  yl.setAttribute("stroke","#e5e7eb"); yl.setAttribute("stroke-width","0.5"); svg.appendChild(yl
}

// Imaginary axis (stability boundary)
const axLine=document.createElementNS(svgNS,"line");
axLine.setAttribute("x1",axX); axLine.setAttribute("y1","0"); axLine.setAttribute("x2",axX); axL
axLine.setAttribute("stroke","#374151"); axLine.setAttribute("stroke-width","1.5");
axLine.setAttribute("stroke-dasharray","6,3"); svg.appendChild(axLine);

// Real axis
const reAx=document.createElementNS(svgNS,"line");
reAx.setAttribute("x1","0"); reAx.setAttribute("y1",CY); reAx.setAttribute("x2",PLANE_W); reAx.s
reAx.setAttribute("stroke","#9ca3af"); reAx.setAttribute("stroke-width","1"); svg.appendChild(re

// Half-plane labels
function addTxt(svg,x,y,txt,color,size,bold){
  const t=document.createElementNS(svgNS,"text");
  t.setAttribute("x",x);t.setAttribute("y",y);t.setAttribute("fill",color);
  t.setAttribute("font-size",size||"11");t.setAttribute("font-family","sans-serif");
  t.setAttribute("text-anchor","middle"); if(bold) t.setAttribute("font-weight","bold");
  t.textContent=txt; svg.appendChild(t);
}
addTxt(svg,axX/2,22,"STABLE","#059669","12",true);
addTxt(svg,axX/2,36,"Re(s) < 0","#059669","10");
addTxt(svg,axX+(PLANE_W-axX)/2,22,"UNSTABLE","#dc2626","12",true);
addTxt(svg,axX+(PLANE_W-axX)/2,36,"Re(s) > 0","#dc2626","10");

// Pole marker
const psx = CX + sRe*SCALE, psy = CY - sIm*SCALE;
const inBounds = psx>0 && psx<PLANE_W && psy>0 && psy<H;

if (inBounds) {
  // Cross marker (×)
  const cSize=7;
```

```
    const poleL1=document.createElementNS(svgNS,"line");
    poleL1.setAttribute("x1",psx-cSize);poleL1.setAttribute("y1",psy-cSize);
    poleL1.setAttribute("x2",psx+cSize);poleL1.setAttribute("y2",psy+cSize);
    poleL1.setAttribute("stroke","#f97316");poleL1.setAttribute("stroke-width","2.5");
    svg.appendChild(poleL1);
    const poleL2=document.createElementNS(svgNS,"line");
    poleL2.setAttribute("x1",psx+cSize);poleL2.setAttribute("y1",psy-cSize);
    poleL2.setAttribute("x2",psx-cSize);poleL2.setAttribute("y2",psy+cSize);
    poleL2.setAttribute("stroke","#f97316");poleL2.setAttribute("stroke-width","2.5");
    svg.appendChild(poleL2);

    const stable = sRe < 0;
    const poleColor = stable ? "#059669" : "#dc2626";
    const poleLbl=document.createElementNS(svgNS,"text");
    poleLbl.setAttribute("x",psx+10); poleLbl.setAttribute("y",psy-6);
    poleLbl.setAttribute("fill",poleColor); poleLbl.setAttribute("font-size","11");
    poleLbl.setAttribute("font-family","sans-serif");
    poleLbl.textContent=`s = ${sRe.toFixed(1)}${sIm>=0?"+":""}${sIm.toFixed(1)}i`;
    svg.appendChild(poleLbl);
  }

  // Axis labels
  addTxt(svg,PLANE_W-16,CY-4,"σ","#374151","11");
  addTxt(svg,axX+4,10,"jω","#374151","11");
  addTxt(svg,axX-16,10,"s-plane","#6b7280","10");

  // Right panel: time response Re(e^{st}) for t in [0,5]
  const PLOT_X = PLANE_W + 20, PLOT_W = W - PLANE_W - 30, PLOT_H = H - 40;
  const PLOT_Y = 20;
  const tMax = 5, numT = 200;

  // Collect time response
  const yVals = [];
  for (let k=0; k<=numT; k++) {
    const t = (k/numT)*tMax;
    // Re(e^{st}) = e^{σt} * cos(ωt)
    const y = Math.exp(sRe*t) * Math.cos(sIm*t);
    yVals.push(y);
  }
  // Auto-scale y
  let yMin=-2, yMax=2;
  const finite = yVals.filter(v=>isFinite(v));
  if (finite.length>0) {
    yMin = Math.min(yMin, ...finite.slice(-20));
    yMax = Math.max(yMax, ...finite.slice(-20));
  }
  const yRange = yMax-yMin || 1;
```

```
  // Plot background
  const plotBg=document.createElementNS(svgNS,"rect");
  plotBg.setAttribute("x",PLOT_X); plotBg.setAttribute("y",PLOT_Y);
  plotBg.setAttribute("width",PLOT_W); plotBg.setAttribute("height",PLOT_H);
  plotBg.setAttribute("fill","#f8fafc"); plotBg.setAttribute("rx","3");
  svg.appendChild(plotBg);

  // Zero line
  const zeroY = PLOT_Y + PLOT_H * (1 - (0-yMin)/yRange);
  const zeroLine=document.createElementNS(svgNS,"line");
  zeroLine.setAttribute("x1",PLOT_X); zeroLine.setAttribute("y1",zeroY);
  zeroLine.setAttribute("x2",PLOT_X+PLOT_W); zeroLine.setAttribute("y2",zeroY);
  zeroLine.setAttribute("stroke","#9ca3af"); zeroLine.setAttribute("stroke-width","0.8");
  zeroLine.setAttribute("stroke-dasharray","3,2"); svg.appendChild(zeroLine);

  // Response curve
  let pathD="";
  for (let k=0; k<=numT; k++) {
    const t=(k/numT)*tMax;
    const y=yVals[k];
    const px=PLOT_X + (t/tMax)*PLOT_W;
    const py=PLOT_Y + PLOT_H*(1-(y-yMin)/yRange);
    const clampedPy = Math.max(PLOT_Y, Math.min(PLOT_Y+PLOT_H, py));
    pathD += (k===0 ? `M ${px} ${clampedPy}` : ` L ${px} ${clampedPy}`);
  }
  const respPath=document.createElementNS(svgNS,"path");
  respPath.setAttribute("d",pathD); respPath.setAttribute("fill","none");
  const stable2 = sRe < 0;
  respPath.setAttribute("stroke", stable2 ? "#059669" : "#dc2626");
  respPath.setAttribute("stroke-width","2"); svg.appendChild(respPath);

  // Plot labels
  addTxt(svg,PLOT_X+PLOT_W/2,PLOT_Y-6,"Time response Re(e^{st})","#374151","10.5",true);
  addTxt(svg,PLOT_X+PLOT_W,PLOT_Y+PLOT_H+14,"t","#6b7280","10");
  addTxt(svg,PLOT_X-4,PLOT_Y+PLOT_H/2,"y","#6b7280","10");

  return svg;
}

const container=document.createElement("div");
container.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem 0

const titleEl=document.createElement("div");
titleEl.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
titleEl.textContent="Stability half-plane: poles in the s-plane";
container.appendChild(titleEl);
container.appendChild(sReSl);
```

```
  container.appendChild(sImSl);

  const vizDiv=document.createElement("div"); container.appendChild(vizDiv);

  const noteEl=document.createElement("div");
  noteEl.style.cssText="margin-top:0.4rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  noteEl.textContent="The s-plane is the domain of the Laplace transform. The imaginary axis (das
  container.appendChild(noteEl);

  function update(){
    vizDiv.innerHTML="";
    vizDiv.appendChild(buildViz(sReSl.value, sImSl.value));
  }
  sReSl.addEventListener("input",update);
  sImSl.addEventListener("input",update);
  update();
  return container;
}
```

---

## 47.1. Complex numbers: brief review

Every complex number $z = x + iy$ is a point in the **Argand plane**, with real part $x$ on the horizontal axis and imaginary part $y$ on the vertical.

**Polar form.** $z = re^{i\theta}$, where $r = |z| = \sqrt{x^2 + y^2}$ is the modulus and $\theta = \arg z$ is the argument — the angle from the positive real axis, measured counterclockwise. Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$ connects the polar and rectangular forms.

**De Moivre's theorem.** $(re^{i\theta})^n = r^n e^{in\theta}$. Integer powers of complex numbers are most efficiently computed in polar form.

**Conjugate.** $\bar{z} = x - iy$. Identities: $z\bar{z} = |z|^2$; $\mathrm{Re}(z) = (z + \bar{z})/2$; $\mathrm{Im}(z) = (z - \bar{z})/(2i)$.

```
//| echo: false
{
  const W = 420, H = 420, CX = W/2, CY = H/2, SCALE = 80;

  const rSlider = Inputs.range([0.5, 2.0], {value: 1.4, step: 0.05, label: "|w| = r"});
  const thetaSlider = Inputs.range([0, 2*Math.PI], {value: Math.PI/4, step: Math.PI/12, label: "a

  const svgNS = "http://www.w3.org/2000/svg";

  function makeViz(r, theta) {
    // z starts at (1.2, 0.6) in complex coords
    let zx = 1.2, zy = 0.6;
```

```
// w = r * e^{i theta}
let wx = r * Math.cos(theta), wy = r * Math.sin(theta);
// zw = z * w
let zwx = zx*wx - zy*wy, zwy = zx*wy + zy*wx;

function toScreen(cx, cy) {
  return [CX + cx*SCALE, CY - cy*SCALE];
}

const [zsx, zsy] = toScreen(zx, zy);
const [zwsx, zwsy] = toScreen(zwx, zwy);

const svg = document.createElementNS(svgNS, "svg");
svg.setAttribute("width", W);
svg.setAttribute("height", H);
svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:bl

// Grid
const gridG = document.createElementNS(svgNS, "g");
gridG.setAttribute("stroke", "#e5e7eb");
gridG.setAttribute("stroke-width", "0.5");
for (let i = -4; i <= 4; i++) {
  const xLine = document.createElementNS(svgNS, "line");
  const [lx] = toScreen(i, 0); const [,ly1] = toScreen(0, -4); const [,ly2] = toScreen(0, 4);
  xLine.setAttribute("x1", lx); xLine.setAttribute("y1", ly1); xLine.setAttribute("x2", lx); xLi
  gridG.appendChild(xLine);
  const yLine = document.createElementNS(svgNS, "line");
  const [lx1] = toScreen(-4, 0); const [lx2] = toScreen(4, 0); const [,ley] = toScreen(0, i);
  yLine.setAttribute("x1", lx1); yLine.setAttribute("y1", ley); yLine.setAttribute("x2", lx2); y
  gridG.appendChild(yLine);
}
svg.appendChild(gridG);

// Axes
const axG = document.createElementNS(svgNS, "g");
axG.setAttribute("stroke", "#9ca3af"); axG.setAttribute("stroke-width", "1");
const xax = document.createElementNS(svgNS, "line");
const [ax1] = toScreen(-3.8, 0); const [ax2] = toScreen(3.8, 0); const [,axy] = toScreen(0, 0);
xax.setAttribute("x1", ax1); xax.setAttribute("y1", axy); xax.setAttribute("x2", ax2); xax.setAt
const yax = document.createElementNS(svgNS, "line");
const [ayx] = toScreen(0, 0); const [,ay1] = toScreen(0, -3.8); const [,ay2] = toScreen(0, 3.8)
yax.setAttribute("x1", ayx); yax.setAttribute("y1", ay1); yax.setAttribute("x2", ayx); yax.setAt
axG.appendChild(xax); axG.appendChild(yax);
svg.appendChild(axG);

// Radial arms from origin to z and zw
function makeLine(x1,y1,x2,y2,color,dash) {
```

```
    const l = document.createElementNS(svgNS, "line");
    l.setAttribute("x1",x1); l.setAttribute("y1",y1); l.setAttribute("x2",x2); l.setAttribute("
    l.setAttribute("stroke", color); l.setAttribute("stroke-width","1.5");
    if (dash) l.setAttribute("stroke-dasharray", dash);
    return l;
  }
  const [ox, oy] = toScreen(0,0);
  svg.appendChild(makeLine(ox, oy, zsx, zsy, "#f97316", ""));
  svg.appendChild(makeLine(ox, oy, zwsx, zwsy, "#3b82f6", ""));

  // Arc showing rotation angle theta between z and zw arms
  const zAngle = Math.atan2(-zsy + CY, zsx - CX); // angle of z from screen center
  const zwAngle = Math.atan2(-zwsy + CY, zwsx - CX);
  const arcR = 30;
  const arcPath = document.createElementNS(svgNS, "path");
  // arc from zAngle to zwAngle
  const ax0 = CX + arcR * Math.cos(zAngle), ay0 = CY + arcR * Math.sin(zAngle);
  const ax1e = CX + arcR * Math.cos(zwAngle), ay1e = CY + arcR * Math.sin(zwAngle);
  // Choose large-arc-flag based on angular difference
  let dang = zwAngle - zAngle;
  while (dang > Math.PI) dang -= 2*Math.PI;
  while (dang < -Math.PI) dang += 2*Math.PI;
  const laf = Math.abs(dang) > Math.PI ? 1 : 0;
  const swf = dang > 0 ? 1 : 0;
  arcPath.setAttribute("d", `M ${ax0} ${ay0} A ${arcR} ${arcR} 0 ${laf} ${swf} ${ax1e} ${ay1e}`
  arcPath.setAttribute("fill","none"); arcPath.setAttribute("stroke","#8b5cf6");
  arcPath.setAttribute("stroke-width","1.5");
  svg.appendChild(arcPath);

  // Scale indicator: show |zw| / |z| = r label
  const zmag = Math.sqrt(zx*zx + zy*zy);
  const zwmag = Math.sqrt(zwx*zwx + zwy*zwy);

  // Points
  function makeCircle(px, py, radius, fill, stroke) {
    const c = document.createElementNS(svgNS, "circle");
    c.setAttribute("cx", px); c.setAttribute("cy", py); c.setAttribute("r", radius);
    c.setAttribute("fill", fill); c.setAttribute("stroke", stroke); c.setAttribute("stroke-widt
    return c;
  }
  svg.appendChild(makeCircle(zsx, zsy, 7, "#f97316", "#fff"));
  svg.appendChild(makeCircle(zwsx, zwsy, 7, "#3b82f6", "#fff"));
  svg.appendChild(makeCircle(ox, oy, 4, "#6b7280", "#fff"));

  // Labels
  function makeText(x, y, txt, color, size) {
    const t = document.createElementNS(svgNS, "text");
```

```
    t.setAttribute("x", x); t.setAttribute("y", y);
    t.setAttribute("fill", color); t.setAttribute("font-size", size || "12");
    t.setAttribute("font-family", "sans-serif");
    t.textContent = txt;
    return t;
  }
  svg.appendChild(makeText(zsx+8, zsy-6, "z", "#f97316", "13"));
  svg.appendChild(makeText(zwsx+8, zwsy-6, "zw", "#3b82f6", "13"));
  svg.appendChild(makeText(ox+4, oy-5, "O", "#6b7280", "11"));

  // Annotation
  const midArcX = CX + arcR * Math.cos((zAngle+zwAngle)/2);
  const midArcY = CY + arcR * Math.sin((zAngle+zwAngle)/2);
  svg.appendChild(makeText(midArcX - 5, midArcY - 8, "\u03b8", "#8b5cf6", "12"));

  // Info panel
  const info = document.createElement("div");
  info.style.cssText = "margin-top:0.5rem; font-size:0.85em; color:#374151; font-family:sans-serif
  const thetaDeg = (theta * 180 / Math.PI).toFixed(0);
  info.innerHTML = `
    <span style="color:#f97316">|z| = ${zmag.toFixed(2)}</span>
    <span style="color:#3b82f6">|zw| = ${zwmag.toFixed(2)}</span>
    <span style="color:#8b5cf6">arg(w) = ${thetaDeg}°</span>
    <span style="color:#6b7280">|zw|/|z| = ${r.toFixed(2)} = r</span>
  `;

  const note = document.createElement("div");
  note.style.cssText = "margin-top:0.4rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  note.textContent = `Multiplying by w = r·e^{iθ} rotates z by θ = ${thetaDeg}° and scales by r =

  const wrap = document.createElement("div");
  wrap.style.cssText = "margin:1rem 0;";
  wrap.appendChild(svg);
  wrap.appendChild(info);
  wrap.appendChild(note);
  return wrap;
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem

const title = document.createElement("div");
title.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
title.textContent = "Complex multiplication as rotation and scaling";
container.appendChild(title);

container.appendChild(rSlider);
```

```
  container.appendChild(thetaSlider);

  const vizContainer = document.createElement("div");
  container.appendChild(vizContainer);

  function update() {
    const r = rSlider.value;
    const theta = thetaSlider.value;
    vizContainer.innerHTML = "";
    vizContainer.appendChild(makeViz(r, theta));
  }

  rSlider.addEventListener("input", update);
  thetaSlider.addEventListener("input", update);
  update();

  return container;
}
```

---

## 47.2. Complex functions

A complex function $f : \mathbb{C} \to \mathbb{C}$ maps each $z = x + iy$ to a complex value $f(z) = u(x, y) + iv(x, y)$.

**Example.** $f(z) = z^2 = (x + iy)^2 = (x^2 - y^2) + 2ixy$, so $u = x^2 - y^2$, $v = 2xy$.

**Example.** $f(z) = \bar{z} = x - iy$, so $u = x$, $v = -y$. This function is nowhere differentiable in the complex sense — a consequence of the Cauchy-Riemann equations derived in the next section.

Limits and continuity work exactly as in real analysis, with one additional clause: $f$ is **continuous at** $z_0$ if $\lim_{z \to z_0} f(z) = f(z_0)$, where the limit must be the same from every direction of approach.

---

## 47.3. Complex differentiability

The **derivative** of $f$ at $z_0$ is:

$$f'(z_0) = \lim_{z \to z_0} \frac{f(z) - f(z_0)}{z - z_0}$$

The limit must exist and agree no matter how $z$ approaches $z_0$ — a far stronger requirement than real differentiability.

### 47.3.1. Deriving the Cauchy-Riemann equations

The key idea is that the limit $[f(z)-f(z_0)]/(z-z_0)$ must give the same answer regardless of the direction of approach. The two simplest directions — horizontal and vertical — already force a constraint on $u$ and $v$.

```
//| echo: false
{
  // Static two-panel approach-direction illustration
  // Left: z₀ with horizontal (orange) and vertical (blue) approach arrows
  // Right: the two difference quotient vectors in f-space
  const W = 290, H = 240, SCALE = 60, CX = W/2, CY = H/2;
  const svgNS = "http://www.w3.org/2000/svg";

  // Fixed example: f(z) = z², z₀ = 0.7 + 0.5i, ⍰ = 0.4
  const z0x=0.7, z0y=0.5, delta=0.4;

  function evalF(re,im){ return [re*re-im*im, 2*re*im]; }
  function toScreen(re,im){ return [CX+re*SCALE, CY-im*SCALE]; }

  function makeSVG(title) {
    const svg=document.createElementNS(svgNS,"svg");
    svg.setAttribute("width",W); svg.setAttribute("height",H);
    svg.style.cssText="background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:bloc
    for(let i=-3;i<=3;i++){
      const xl=document.createElementNS(svgNS,"line"); const [lx]=toScreen(i,0);
      xl.setAttribute("x1",lx);xl.setAttribute("y1","0");xl.setAttribute("x2",lx);xl.setAttribute("y
      xl.setAttribute("stroke","#e5e7eb");xl.setAttribute("stroke-width","0.5");svg.appendChild(xl)
      const yl=document.createElementNS(svgNS,"line"); const [,ly]=toScreen(0,i);
      yl.setAttribute("x1","0");yl.setAttribute("y1",ly);yl.setAttribute("x2",W);yl.setAttribute("y2
      yl.setAttribute("stroke","#e5e7eb");yl.setAttribute("stroke-width","0.5");svg.appendChild(yl)
    }
    const [ox,oy]=toScreen(0,0);
    function axLine(x1,y1,x2,y2){ const l=document.createElementNS(svgNS,"line"); l.setAttribute("x1
    const [ax1]=toScreen(-2.3,0); const [ax2]=toScreen(2.3,0);
    axLine(ax1,oy,ax2,oy);
    const [,ay1]=toScreen(0,-1.8); const [,ay2]=toScreen(0,1.8);
    axLine(ox,ay1,ox,ay2);
    const tl=document.createElementNS(svgNS,"text"); tl.setAttribute("x","6");tl.setAttribute("y","1
    tl.setAttribute("fill","#1f2937");tl.setAttribute("font-size","11");tl.setAttribute("font-family
    tl.setAttribute("font-weight","bold");tl.textContent=title;svg.appendChild(tl);
    return svg;
  }

  function arrow(svg,x1,y1,x2,y2,color){
    const l=document.createElementNS(svgNS,"line"); l.setAttribute("x1",x1);l.setAttribute("y1",y1)
    const dx=x2-x1,dy=y2-y1,len=Math.sqrt(dx*dx+dy*dy)||1; const ux=dx/len,uy=dy/len; const sz=8;
    const tri=document.createElementNS(svgNS,"polygon");
```

```
  tri.setAttribute("points",`${x2},${y2} ${x2-ux*sz+uy*sz*0.4},${y2-uy*sz-ux*sz*0.4} ${x2-ux*sz
  tri.setAttribute("fill",color); svg.appendChild(tri);
}

function dot(svg,cx,cy,r,fill){ const c=document.createElementNS(svgNS,"circle"); c.setAttribut
function lbl(svg,x,y,txt,color,size){ const t=document.createElementNS(svgNS,"text"); t.setAttr

// Left panel: z-plane
const svgL = makeSVG("z-plane");
const [z0sx,z0sy]=toScreen(z0x,z0y);
// Horizontal approach: from z₀+δ to z₀
const [hsx]=toScreen(z0x+delta,z0y);
arrow(svgL,hsx,z0sy,z0sx,z0sy,"#f97316");
lbl(svgL,hsx+3,z0sy-5,"z₀+δ","#f97316","10");
// Vertical approach: from z₀+iδ to z₀
const [,vsy]=toScreen(z0x,z0y+delta);
arrow(svgL,z0sx,vsy,z0sx,z0sy,"#3b82f6");
lbl(svgL,z0sx+4,vsy-4,"z₀+iδ","#3b82f6","10");
dot(svgL,z0sx,z0sy,6,"#f97316");
lbl(svgL,z0sx+8,z0sy-6,"z₀","#f97316","12");

// Right panel: f-space difference quotient vectors
const svgR = makeSVG("f(z)-space: quotient vectors");
const f0=evalF(z0x,z0y);
const fH=evalF(z0x+delta,z0y);
const fV=evalF(z0x,z0y+delta);
// Horizontal quotient: (f(z₀+δ)-f(z₀))/δ
const hqr=(fH[0]-f0[0])/delta, hqi=(fH[1]-f0[1])/delta;
// Vertical quotient: (f(z₀+iδ)-f(z₀))/(iδ) = -i*(δf/δ)
const vqr=(fV[1]-f0[1])/delta, vqi=-(fV[0]-f0[0])/delta;
const [origx,origy]=toScreen(0,0);
const [hqsx,hqsy]=toScreen(hqr,hqi);
const [vqsx,vqsy]=toScreen(vqr,vqi);
arrow(svgR,origx,origy,hqsx,hqsy,"#f97316");
lbl(svgR,hqsx+5,hqsy,"horiz quotient","#f97316","10");
arrow(svgR,origx,origy,vqsx,vqsy,"#3b82f6");
lbl(svgR,vqsx+5,vqsy,"vert quotient","#3b82f6","10");
dot(svgR,origx,origy,4,"#6b7280");
// Label: they agree (f=z² is analytic)
const agree=Math.abs(hqr-vqr)+Math.abs(hqi-vqi)<0.3;
lbl(svgR,6,H-8, agree?"Quotients agree — C-R holds":"Quotients differ — C-R fails", agree?"#059

const wrap=document.createElement("div");
wrap.style.cssText="display:flex; gap:0.75rem; flex-wrap:wrap; margin:0.75rem 0;";
wrap.appendChild(svgL); wrap.appendChild(svgR);

const note=document.createElement("div");
```

```
  note.style.cssText="font-size:0.82em; color:#6b7280; font-style:italic; margin-top:0.3rem;";
  note.textContent="Static illustration for f(z) = z², z₀ = 0.7+0.5i, ⍰ = 0.4. Left: two approach di

  const outer=document.createElement("div");
  outer.appendChild(wrap); outer.appendChild(note);
  return outer;
}
```

Approach $z_0 = x_0 + iy_0$ along two different directions and demand the limit is the same.

**Horizontal approach** ($z = x + iy_0$, $z - z_0 = \Delta x$):

Write $f(z) - f(z_0) = [u(x_0 + \Delta x, y_0) - u(x_0, y_0)] + i[v(x_0 + \Delta x, y_0) - v(x_0, y_0)]$. Dividing by the real number $\Delta x$ keeps real and imaginary parts separate:

$$\frac{f(z) - f(z_0)}{\Delta x} = \frac{[u(x_0 + \Delta x,\ y_0) - u(x_0, y_0)]}{\Delta x} + i\frac{[v(x_0 + \Delta x,\ y_0) - v(x_0, y_0)]}{\Delta x}$$

Taking $\Delta x \to 0$, each fraction is just a partial derivative:

$$f'(z_0) = u_x + iv_x$$

**Vertical approach** ($z = x_0 + iy$, $z - z_0 = i\Delta y$):

Now the increment is purely imaginary, so we divide by $i\Delta y$. Dividing by $i$ is the same as multiplying by $-i$, which swaps real and imaginary parts. Carrying through:

$$\frac{f(z) - f(z_0)}{i\Delta y} = \frac{1}{i} \cdot \frac{[u(x_0, y_0 + \Delta y) - u(x_0, y_0)] + i[v(x_0, y_0 + \Delta y) - v(x_0, y_0)]}{\Delta y}$$

Using $1/i = -i$ and taking $\Delta y \to 0$:

$$f'(z_0) = v_y - iu_y$$

Equating real and imaginary parts of $u_x + iv_x = v_y - iu_y$:

$$\boxed{u_x = v_y \qquad u_y = -v_x}$$

These are the **Cauchy-Riemann (C-R) equations**. They are necessary for complex differentiability. If the partial derivatives are continuous in a neighbourhood of $z_0$, they are also sufficient.

*Quick check.* For $f(z) = z^2$ we found $u = x^2 - y^2$, $v = 2xy$ earlier. Verify: $u_x = 2x = v_y$ and $u_y = -2y = -v_x = -2y$. Both equations hold everywhere — consistent with $z^2$ being entire. Exercise 1 walks through this systematically and uses the result to compute $f'(z)$.

```
//| echo: false
{
  // C-R approach-direction demo
  // Left panel: z-plane with z₀ (draggable via sliders), two approach arrows
  // Right: "derivative plane" showing the two difference quotients as □→0
  const W = 320, H = 300, SCALE = 70;
  const CX = W/2, CY = H/2;
  const svgNS = "http://www.w3.org/2000/svg";

  const funcSel = Inputs.select(
    ["f(z) = z²", "f(z) = e□", "f(z) = z̄", "f(z) = |z|²"],
    { value: "f(z) = z²", label: "Function" }
  );
  const z0xSlider = Inputs.range([-2, 2], { value: 0.7, step: 0.1, label: "Re(z₀)" });
  const z0ySlider = Inputs.range([-2, 2], { value: 0.5, step: 0.1, label: "Im(z₀)" });
  const deltaSlider = Inputs.range([0.02, 1.0], { value: 0.5, step: 0.02, label: "□ (step size)"

  function evalF(name, re, im) {
    switch (name) {
      case "f(z) = z²":   return [re*re - im*im, 2*re*im];
      case "f(z) = e□": { const ex=Math.exp(re); return [ex*Math.cos(im), ex*Math.sin(im)]; }
      case "f(z) = z̄":   return [re, -im];
      case "f(z) = |z|²": return [re*re + im*im, 0];
      default:             return [re, im];
    }
  }

  function toScreen(re, im) { return [CX + re*SCALE, CY - im*SCALE]; }
  function fromScreen(px, py) { return [(px-CX)/SCALE, -(py-CY)/SCALE]; }

  function makePanel(w, h, title) {
    const svg = document.createElementNS(svgNS, "svg");
    svg.setAttribute("width", w); svg.setAttribute("height", h);
    svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display
    // Grid
    for (let i=-3; i<=3; i++) {
      const xl = document.createElementNS(svgNS,"line");
      const [lx]=toScreen(i,0); const [,ly1]=toScreen(0,-3); const [,ly2]=toScreen(0,3);
      xl.setAttribute("x1",lx); xl.setAttribute("y1",ly1); xl.setAttribute("x2",lx); xl.setAttrib
      xl.setAttribute("stroke","#e5e7eb"); xl.setAttribute("stroke-width","0.5"); svg.appendChild
      const yl = document.createElementNS(svgNS,"line");
      const [lx1]=toScreen(-3,0); const [lx2]=toScreen(3,0); const [,ly]=toScreen(0,i);
      yl.setAttribute("x1",lx1); yl.setAttribute("y1",ly); yl.setAttribute("x2",lx2); yl.setAttri
      yl.setAttribute("stroke","#e5e7eb"); yl.setAttribute("stroke-width","0.5"); svg.appendChild
    }
    // Axes
    const [ox,oy]=toScreen(0,0);
```

```
  function axLine(x1,y1,x2,y2) {
    const l=document.createElementNS(svgNS,"line");
    l.setAttribute("x1",x1); l.setAttribute("y1",y1); l.setAttribute("x2",x2); l.setAttribute("y2"
    l.setAttribute("stroke","#9ca3af"); l.setAttribute("stroke-width","1"); svg.appendChild(l);
  }
  const [ax1]=toScreen(-2.8,0); const [ax2]=toScreen(2.8,0);
  axLine(ax1,oy,ax2,oy);
  const [,ay1]=toScreen(0,-2.3); const [,ay2]=toScreen(0,2.3);
  axLine(ox,ay1,ox,ay2);
  // Panel label
  const tl = document.createElementNS(svgNS,"text");
  tl.setAttribute("x","6"); tl.setAttribute("y","16");
  tl.setAttribute("fill","#6b7280"); tl.setAttribute("font-size","10");
  tl.setAttribute("font-family","sans-serif"); tl.textContent = title;
  svg.appendChild(tl);
  return { svg, ox, oy };
}

function arrowHead(svg, x2, y2, dx, dy, color) {
  const len = Math.sqrt(dx*dx+dy*dy);
  if (len < 0.001) return;
  const ux=dx/len, uy=dy/len;
  const size=7;
  const lx=x2-ux*size+uy*size*0.4, ly=y2-uy*size-ux*size*0.4;
  const rx=x2-ux*size-uy*size*0.4, ry=y2-uy*size+ux*size*0.4;
  const tri=document.createElementNS(svgNS,"polygon");
  tri.setAttribute("points",`${x2},${y2} ${lx},${ly} ${rx},${ry}`);
  tri.setAttribute("fill",color); svg.appendChild(tri);
}

function addArrow(svg, x1, y1, x2, y2, color, dash) {
  const l=document.createElementNS(svgNS,"line");
  l.setAttribute("x1",x1); l.setAttribute("y1",y1); l.setAttribute("x2",x2); l.setAttribute("y2",y
  l.setAttribute("stroke",color); l.setAttribute("stroke-width","2");
  if (dash) l.setAttribute("stroke-dasharray",dash);
  svg.appendChild(l);
  arrowHead(svg, x2, y2, x2-x1, y2-y1, color);
}

function addDot(svg, cx, cy, r, fill, stroke) {
  const c=document.createElementNS(svgNS,"circle");
  c.setAttribute("cx",cx); c.setAttribute("cy",cy); c.setAttribute("r",r);
  c.setAttribute("fill",fill); c.setAttribute("stroke",stroke||"#fff"); c.setAttribute("stroke-wi
  svg.appendChild(c);
}

function addLabel(svg, x, y, txt, color, size) {
```

```
  const t=document.createElementNS(svgNS,"text");
  t.setAttribute("x",x); t.setAttribute("y",y);
  t.setAttribute("fill",color); t.setAttribute("font-size",size||"11");
  t.setAttribute("font-family","sans-serif"); t.textContent=txt;
  svg.appendChild(t);
}

function buildCRPanel(funcName, z0x, z0y, delta) {
  const { svg: svgL } = makePanel(W, H, "z-plane");

  // C-R background wash: pixel-test each cell
  const WASH_STEP = 12, H_EPS = 0.01;
  for (let px=0; px<W; px+=WASH_STEP) {
    for (let py=0; py<H; py+=WASH_STEP) {
      const [re,im] = fromScreen(px+WASH_STEP/2, py+WASH_STEP/2);
      const [u0,v0]=evalF(funcName,re,im);
      const [udx,vdx]=evalF(funcName,re+H_EPS,im);
      const [udy,vdy]=evalF(funcName,re,im+H_EPS);
      const ux=(udx-u0)/H_EPS, vx=(vdx-v0)/H_EPS;
      const uy=(udy-u0)/H_EPS, vy=(vdy-v0)/H_EPS;
      const err = Math.abs(ux-vy) + Math.abs(uy+vx);
      const ok = err < 0.05;
      const rect=document.createElementNS(svgNS,"rect");
      rect.setAttribute("x",px); rect.setAttribute("y",py);
      rect.setAttribute("width",WASH_STEP); rect.setAttribute("height",WASH_STEP);
      rect.setAttribute("fill", ok ? "#059669" : "#dc2626");
      rect.setAttribute("opacity","0.12");
      svgL.appendChild(rect);
    }
  }

  // z₀ point
  const [z0sx, z0sy] = toScreen(z0x, z0y);
  addDot(svgL, z0sx, z0sy, 7, "#f97316", "#fff");
  addLabel(svgL, z0sx+9, z0sy-6, "z₀", "#f97316", "12");

  // Horizontal approach arrow: z₀ + □ → z₀
  const [hsx] = toScreen(z0x + delta, z0y);
  addArrow(svgL, hsx, z0sy, z0sx, z0sy, "#f97316", "");
  addLabel(svgL, hsx+4, z0sy-4, "→ z₀", "#f97316", "10");

  // Vertical approach arrow: z₀ + i□ → z₀
  const [,vsy] = toScreen(z0x, z0y + delta);
  addArrow(svgL, z0sx, vsy, z0sx, z0sy, "#3b82f6", "");
  addLabel(svgL, z0sx+4, vsy-4, "↑ z₀", "#3b82f6", "10");

  // Panel labels
```

```
  addLabel(svgL, 6, H-8, "Green = C-R holds   Red = fails", "#6b7280", "9");

  // Right panel: derivative plane — two quotient vectors
  const { svg: svgR } = makePanel(W, H, "difference quotient");

  const f0 = evalF(funcName, z0x, z0y);
  const fHoriz = evalF(funcName, z0x + delta, z0y);
  const fVert  = evalF(funcName, z0x, z0y + delta);

  // Horizontal quotient: (f(z₀+□) − f(z₀)) / □   (real □)
  const hqr = (fHoriz[0]-f0[0])/delta, hqi = (fHoriz[1]-f0[1])/delta;
  // Vertical quotient: (f(z₀+i□) − f(z₀)) / (i□) = −i·(□f)/□
  const vqr = (fVert[1]-f0[1])/delta, vqi = -(fVert[0]-f0[0])/delta;

  const [origx, origy] = toScreen(0,0);
  const [hqsx, hqsy] = toScreen(hqr, hqi);
  const [vqsx, vqsy] = toScreen(vqr, vqi);

  addArrow(svgR, origx, origy, hqsx, hqsy, "#f97316", "");
  addLabel(svgR, hqsx+6, hqsy, "horiz", "#f97316", "10");

  addArrow(svgR, origx, origy, vqsx, vqsy, "#3b82f6", "");
  addLabel(svgR, vqsx+6, vqsy, "vert", "#3b82f6", "10");

  addDot(svgR, origx, origy, 4, "#6b7280", "#fff");

  // Check agreement
  const agree = Math.abs(hqr-vqr) + Math.abs(hqi-vqi);
  const color = agree < 0.15 ? "#059669" : "#dc2626";
  const msg   = agree < 0.15 ? "Quotients agree — f differentiable here" : "Quotients differ — C-R
  addLabel(svgR, 6, H-20, msg, color, "10");
  addLabel(svgR, 6, H-8, `|error| = ${agree.toFixed(3)} (small □ → more precise)`, "#9ca3af", "9")

  const wrap = document.createElement("div");
  wrap.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin:0.5rem 0;";
  wrap.appendChild(svgL);
  wrap.appendChild(svgR);
  return wrap;
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem

const titleEl = document.createElement("div");
titleEl.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
titleEl.textContent = "Cauchy-Riemann: approach-direction explorer";
container.appendChild(titleEl);
```

```
  container.appendChild(funcSel);
  container.appendChild(z0xSlider);
  container.appendChild(z0ySlider);
  container.appendChild(deltaSlider);

  const vizDiv = document.createElement("div");
  container.appendChild(vizDiv);

  const noteEl = document.createElement("div");
  noteEl.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;"
  noteEl.textContent = "Left: z-plane with orange (horizontal) and blue (vertical) approach arrow
  container.appendChild(noteEl);

  function update() {
    vizDiv.innerHTML = "";
    vizDiv.appendChild(buildCRPanel(funcSel.value, z0xSlider.value, z0ySlider.value, deltaSlider.
  }
  funcSel.addEventListener("input", update);
  z0xSlider.addEventListener("input", update);
  z0ySlider.addEventListener("input", update);
  deltaSlider.addEventListener("input", update);
  update();

  return container;
}
```

**Consequence.** For $f(z) = \bar{z}$: $u_x = 1$ but $v_y = -1$. The C-R equations fail everywhere, so $\bar{z}$ is nowhere differentiable — even though its real and imaginary parts are smooth real functions.

When the C-R equations hold, the derivative can be written in any of the equivalent forms:

$$f'(z) = u_x + iv_x = v_y - iu_y$$

---

## 47.4. Analytic functions

$f$ is **analytic at** $z_0$ if it is differentiable in some open disk centred at $z_0$. It is **analytic in a domain** $D$ if differentiable at every point of $D$. A function analytic on all of $\mathbb{C}$ is called **entire**.

Being differentiable at an isolated point does not make a function analytic — analyticity requires differentiability in an open neighbourhood. Exercise 4 shows this with $f(z) = |z|^2$, which is differentiable only at the origin and analytic nowhere.

### 47.4.1. Harmonic functions

If $f = u + iv$ is analytic, both $u$ and $v$ satisfy **Laplace's equation**:

$$\nabla^2 u = u_{xx} + u_{yy} = 0, \qquad \nabla^2 v = 0$$

*Proof.* From the C-R equations: $u_{xx} = (u_x)_x = (v_y)_x = v_{yx}$ and $u_{yy} = (u_y)_y = (-v_x)_y = -v_{xy}$. Since mixed partials are equal, $u_{xx} + u_{yy} = 0$.

A function satisfying Laplace's equation is **harmonic**. So: the real and imaginary parts of any analytic function are harmonic.

Conversely, given a harmonic $u$ on a **simply connected domain** — one with no holes, so that any closed loop inside it can be continuously shrunk to a point — there exists a **harmonic conjugate** $v$ (unique up to a constant) such that $f = u + iv$ is analytic. Finding $v$ from $u$ via the C-R equations is a standard technique.

```
//| echo: false
{
  // Harmonic function landscape
  // Dual canvas: left = u(x,y) equipotential contours + optional Laplacian check
  //              right = v(x,y) streamlines (or combined orthogonality view)
  const W = 300, H = 280, SCALE = 55;
  const CX = W/2, CY = H/2;
  const svgNS = "http://www.w3.org/2000/svg";

  const funcSel  = Inputs.select(
    ["f = z²", "f = e□", "f = sin z", "f = z³"],
    { value: "f = z²", label: "Analytic function" }
  );
  const modeToggle  = Inputs.toggle({ label: "Show Laplacian check (grey = harmonic, red = not)", va
  const orthToggle  = Inputs.toggle({ label: "Show orthogonality overlay", value: false });
  const ptxSlider   = Inputs.range([-2, 2], { value: 0.8, step: 0.1, label: "Probe Re(z)" });
  const ptySlider   = Inputs.range([-2, 2], { value: 0.6, step: 0.1, label: "Probe Im(z)" });

  function evalF(name, re, im) {
    switch (name) {
      case "f = z²":  return [re*re - im*im, 2*re*im];
      case "f = e□": { const ex=Math.exp(re); return [ex*Math.cos(im), ex*Math.sin(im)]; }
      case "f = sin z": return [Math.sin(re)*Math.cosh(im), Math.cos(re)*Math.sinh(im)];
      case "f = z³":  return [re*re*re - 3*re*im*im, 3*re*re*im - im*im*im];
      default:         return [re, im];
    }
  }

  function getU(name, re, im) { return evalF(name,re,im)[0]; }
  function getV(name, re, im) { return evalF(name,re,im)[1]; }

  // Draw a canvas panel: heatmap of scalar field + iso-contours
```

```
function renderHeatmap(canvas, name, component, showLaplacian, showOrth) {
  const ctx = canvas.getContext("2d");
  const img = ctx.createImageData(W, H);
  const d = img.data;
  const H_EPS = 0.05;

  // First pass: collect min/max of field
  let fmin = Infinity, fmax = -Infinity;
  for (let px=0; px<W; px+=2) {
    for (let py=0; py<H; py+=2) {
      const re=(px-CX)/SCALE, im=-(py-CY)/SCALE;
      const v = component === "u" ? getU(name,re,im) : getV(name,re,im);
      if (isFinite(v)) { if(v<fmin) fmin=v; if(v>fmax) fmax=v; }
    }
  }
  const frange = fmax - fmin || 1;

  for (let px=0; px<W; px++) {
    for (let py=0; py<H; py++) {
      const re=(px-CX)/SCALE, im=-(py-CY)/SCALE;
      const idx=(py*W+px)*4;

      if (showLaplacian) {
        // Numerical Laplacian check: ∇²u ≈ (u(+h)-2u+u(-h))/h² + same for y
        const u0   = getU(name,re,im);
        const upx  = getU(name,re+H_EPS,im);
        const umx  = getU(name,re-H_EPS,im);
        const upy  = getU(name,re,im+H_EPS);
        const umy  = getU(name,re,im-H_EPS);
        const lap  = (upx+umx+upy+umy-4*u0)/(H_EPS*H_EPS);
        const err  = Math.min(Math.abs(lap)/5, 1);
        // Near-zero → grey, large → red
        d[idx]   = Math.round(180 + err*75);
        d[idx+1] = Math.round(180 - err*150);
        d[idx+2] = Math.round(180 - err*150);
        d[idx+3] = 255;
      } else {
        const fv = component === "u" ? getU(name,re,im) : getV(name,re,im);
        if (!isFinite(fv)) { d[idx]=255; d[idx+1]=255; d[idx+2]=255; d[idx+3]=255; continue; }
        const t = Math.max(0, Math.min(1, (fv-fmin)/frange));
        // Blue-white-orange diverging palette
        if (t < 0.5) {
          const s = t*2;
          d[idx]   = Math.round(59+s*(255-59));
          d[idx+1] = Math.round(130+s*(255-130));
          d[idx+2] = Math.round(246+s*(255-246));
        } else {
```

```
          const s = (t-0.5)*2;
          d[idx]   = Math.round(255);
          d[idx+1] = Math.round(255-s*(255-115));
          d[idx+2] = Math.round(255-s*(255-22));
        }
        d[idx+3] = 255;
      }
    }
  }
  ctx.putImageData(img, 0, 0);

  // Iso-contours via marching squares (simplified: draw line segments)
  function isoContour(field, level, color, alpha) {
    ctx.beginPath(); ctx.strokeStyle = color; ctx.lineWidth = 0.9;
    ctx.globalAlpha = alpha;
    const step = 3;
    for (let px=0; px<W-step; px+=step) {
      for (let py=0; py<H-step; py+=step) {
        const re=(px-CX)/SCALE, im=-(py-CY)/SCALE;
        const v00=field(name,re,im),   v10=field(name,re+step/SCALE,im);
        const v01=field(name,re,im-step/SCALE), v11=field(name,re+step/SCALE,im-step/SCALE);
        const c0=(v00>level), c1=(v10>level), c2=(v11>level), c3=(v01>level);
        if (c0===c1 && c1===c2 && c2===c3) continue;
        function interp(v0,v1,x0,y0,x1,y1) {
          if (Math.abs(v1-v0)<1e-9) return [(x0+x1)/2,(y0+y1)/2];
          const t=(level-v0)/(v1-v0);
          return [x0+(x1-x0)*t, y0+(y1-y0)*t];
        }
        const pts=[];
        if (c0!==c1) pts.push(interp(v00,v10, px,py, px+step,py));
        if (c1!==c2) pts.push(interp(v10,v11, px+step,py, px+step,py+step));
        if (c3!==c2) pts.push(interp(v01,v11, px,py+step, px+step,py+step));
        if (c0!==c3) pts.push(interp(v00,v01, px,py, px,py+step));
        if (pts.length>=2) {
          ctx.moveTo(pts[0][0],pts[0][1]); ctx.lineTo(pts[1][0],pts[1][1]);
        }
      }
    }
    ctx.stroke(); ctx.globalAlpha=1;
  }

  // Draw several iso-levels
  const numLevels = 9;
  const isColor   = component === "u" ? "#1d4ed8" : "#6d28d9";
  for (let k=0; k<=numLevels; k++) {
    const lv = fmin + (k/numLevels)*frange;
    isoContour(component==="u"?getU:getV, lv, isColor, 0.7);
```

```
  }

  if (showOrth) {
    // Overlay v-contours on u-panel (or vice versa) to show orthogonality
    const crossField = component === "u" ? getV : getU;
    const [cfmin, cfmax] = (() => {
      let mn=Infinity,mx=-Infinity;
      for (let px=0; px<W; px+=4) for (let py=0; py<H; py+=4) {
        const re=(px-CX)/SCALE, im=-(py-CY)/SCALE;
        const v=crossField(name,re,im);
        if(isFinite(v)){if(v<mn)mn=v;if(v>mx)mx=v;}
      }
      return [mn,mx];
    })();
    for (let k=0; k<=numLevels; k++) {
      const lv=cfmin+(k/numLevels)*(cfmax-cfmin);
      isoContour(crossField, lv, component===="u"?"#8b5cf6":"#f97316", 0.55);
    }
  }

  // Axes
  ctx.strokeStyle = "rgba(0,0,0,0.25)"; ctx.lineWidth=1; ctx.globalAlpha=1;
  ctx.beginPath(); ctx.moveTo(0,CY); ctx.lineTo(W,CY); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(CX,0); ctx.lineTo(CX,H); ctx.stroke();

  // Axis labels
  ctx.fillStyle="#374151"; ctx.font="10px sans-serif";
  ctx.fillText("Re", W-22, CY-4); ctx.fillText("Im", CX+4, 12);

  // Panel title
  ctx.fillStyle="#1f2937"; ctx.font="bold 11px sans-serif";
  const panelTitle = showLaplacian ? "Laplacian ∇²u (grey = zero = harmonic)" :
                     (component===="u" ? "u(x,y) — equipotentials" : "v(x,y) — streamlines");
  ctx.fillText(panelTitle, 4, 12);
}

function renderProbeInfo(name, ptx, pty) {
  const [uv, vv] = evalF(name, ptx, pty);
  const eps = 0.02;
  const ux  = (getU(name,ptx+eps,pty)-getU(name,ptx-eps,pty))/(2*eps);
  const uy  = (getU(name,ptx,pty+eps)-getU(name,ptx,pty-eps))/(2*eps);
  const vx  = (getV(name,ptx+eps,pty)-getV(name,ptx-eps,pty))/(2*eps);
  const vy  = (getV(name,ptx,pty+eps)-getV(name,ptx,pty-eps))/(2*eps);
  const crErr = Math.abs(ux-vy)+Math.abs(uy+vx);
  const crOk  = crErr < 0.05;
  const lapU  = (getU(name,ptx+eps,pty)+getU(name,ptx-eps,pty)+getU(name,ptx,pty+eps)+getU(name
  // Gradient dot product to check orthogonality
```

```
  const dot   = ux*vx + uy*vy;
  const info = document.createElement("div");
  info.style.cssText = "font-family:sans-serif; font-size:0.82em; color:#374151; margin-top:0.5rem
  info.innerHTML = `
    <span>z₀ = ${ptx.toFixed(2)} + ${pty.toFixed(2)}i</span>
    <span>u = ${uv.toFixed(3)}</span>
    <span>v = ${vv.toFixed(3)}</span>
    <span style="color:${crOk?"#059669":"#dc2626"}">C-R: ${crOk?"□ holds":"□ fails"}</span>
    <span>∇²u □ ${lapU.toFixed(3)}
    <span>∇u·∇v = ${dot.toFixed(3)} (□0 means orthogonal)</span>
  `;
  return info;
}

function renderProbeOverlay(canvas, name, ptx, pty, component) {
  const ctx = canvas.getContext("2d");
  const [psx, psy] = [CX + ptx*SCALE, CY - pty*SCALE];
  ctx.fillStyle = "#f97316"; ctx.strokeStyle="#fff"; ctx.lineWidth=1.5;
  ctx.beginPath(); ctx.arc(psx,psy,5,0,2*Math.PI); ctx.fill(); ctx.stroke();
  // Gradient arrow
  const eps = 0.06;
  const gx = component==="u" ? (getU(name,ptx+eps,pty)-getU(name,ptx-eps,pty))/(2*eps)
                             : (getV(name,ptx+eps,pty)-getV(name,ptx-eps,pty))/(2*eps);
  const gy = component==="u" ? (getU(name,ptx,pty+eps)-getU(name,ptx,pty-eps))/(2*eps)
                             : (getV(name,ptx,pty+eps)-getV(name,ptx,pty-eps))/(2*eps);
  const gLen = Math.sqrt(gx*gx+gy*gy)||1;
  const scale = 18;
  const ex = psx + (gx/gLen)*scale, ey = psy - (gy/gLen)*scale;
  ctx.strokeStyle="#f97316"; ctx.lineWidth=2;
  ctx.beginPath(); ctx.moveTo(psx,psy); ctx.lineTo(ex,ey); ctx.stroke();
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem

const titleEl = document.createElement("div");
titleEl.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
titleEl.textContent = "Harmonic function landscape: equipotentials and streamlines";
container.appendChild(titleEl);

container.appendChild(funcSel);
container.appendChild(modeToggle);
container.appendChild(orthToggle);
container.appendChild(ptxSlider);
container.appendChild(ptySlider);

const canvasRow = document.createElement("div");
```

```
  canvasRow.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin:0.5rem 0;";

  const cL = document.createElement("canvas"); cL.width=W; cL.height=H;
  cL.style.cssText = "border-radius:4px; display:block;";
  const cR = document.createElement("canvas"); cR.width=W; cR.height=H;
  cR.style.cssText = "border-radius:4px; display:block;";

  canvasRow.appendChild(cL); canvasRow.appendChild(cR);
  container.appendChild(canvasRow);

  const probeDiv = document.createElement("div");
  container.appendChild(probeDiv);

  const noteEl = document.createElement("div");
  noteEl.style.cssText = "margin-top:0.4rem; font-size:0.82em; color:#6b7280; font-style:italic;"
  noteEl.textContent = "Blue curves: equipotential lines (u = const). Purple curves: streamlines
  container.appendChild(noteEl);

  function update() {
    const fn = funcSel.value;
    const lapl = modeToggle.value;
    const orth = orthToggle.value;
    const ptx = ptxSlider.value, pty = ptySlider.value;

    renderHeatmap(cL, fn, "u", lapl, orth);
    renderHeatmap(cR, fn, "v", false, orth);
    renderProbeOverlay(cL, fn, ptx, pty, "u");
    renderProbeOverlay(cR, fn, ptx, pty, "v");

    probeDiv.innerHTML = "";
    probeDiv.appendChild(renderProbeInfo(fn, ptx, pty));
  }

  funcSel.addEventListener("input", update);
  modeToggle.addEventListener("input", update);
  orthToggle.addEventListener("input", update);
  ptxSlider.addEventListener("input", update);
  ptySlider.addEventListener("input", update);
  update();

  return container;
}
```

## 47.5. Elementary analytic functions

### 47.5.1. Exponential

$$e^z = e^x(\cos y + i \sin y), \qquad \frac{d}{dz} e^z = e^z$$

Entire. Periodic with period $2\pi i$: $e^{z+2\pi i} = e^z$.

```
//| echo: false
{
  // Complex exponential periodicity
  // Left: z-plane with 2π-wide horizontal strips coloured by period band
  // A constrained point z moves along Re(z) = fixed with Im varying
  // Right: w-plane showing e^z tracing a circle |w| = e^{Re(z)}
  // Fading trail shows the orbit; counter shows full laps

  const W = 300, H = 280, SCALE = 55, CX = W/2, CY = H/2;
  const svgNS = "http://www.w3.org/2000/svg";

  const reSlider = Inputs.range([-1.5, 1.5], { value: 1.0, step: 0.1, label: "Re(z) — controls |e^z
  const tSlider  = Inputs.range([0, 4*Math.PI], { value: 0, step: 0.05, label: "Im(z) = t" });

  const STRIP_COLORS = ["#dbeafe","#fef3c7","#d1fae5","#fce7f3","#ede9fe"];

  function toScreen(re, im) { return [CX + re*SCALE, CY - im*SCALE]; }

  function buildViz(reZ, tVal) {
    // Left SVG — z-plane
    const svgL = document.createElementNS(svgNS,"svg");
    svgL.setAttribute("width",W); svgL.setAttribute("height",H);
    svgL.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:b

    // Period strips: horizontal bands of height 2π, coloured cyclically
    const stripH = 2*Math.PI * SCALE; // pixels per period
    const numStrips = Math.ceil(H / stripH) + 2;
    const stripOffset = CY; // y=0 maps to CY
    for (let k=-numStrips; k<=numStrips; k++) {
      const yTop = stripOffset - (k+1)*stripH;
      const rect = document.createElementNS(svgNS,"rect");
      rect.setAttribute("x","0"); rect.setAttribute("y", yTop.toFixed(1));
      rect.setAttribute("width", W); rect.setAttribute("height", stripH.toFixed(1));
      rect.setAttribute("fill", STRIP_COLORS[((k%5)+5)%5]);
      rect.setAttribute("opacity","0.55");
      svgL.appendChild(rect);
      // Strip label
      const yMid = yTop + stripH/2;
      if (yMid > 0 && yMid < H) {
```

```
      const lbl = document.createElementNS(svgNS,"text");
      lbl.setAttribute("x","4"); lbl.setAttribute("y", (yMid+4).toFixed(1));
      lbl.setAttribute("fill","#6b7280"); lbl.setAttribute("font-size","9");
      lbl.setAttribute("font-family","sans-serif");
      lbl.textContent = `Im ∈ [${(k*2).toFixed(0)}π, ${(k*2+2).toFixed(0)}π)`;
      svgL.appendChild(lbl);
    }
  }

  // Grid lines (x only for clarity)
  for (let i=-3; i<=3; i++) {
    const xl=document.createElementNS(svgNS,"line");
    const [lx]=toScreen(i,0); const [,ly1]=toScreen(0,-3); const [,ly2]=toScreen(0,3);
    xl.setAttribute("x1",lx); xl.setAttribute("y1",ly1); xl.setAttribute("x2",lx); xl.setAttrib
    xl.setAttribute("stroke","rgba(0,0,0,0.08)"); xl.setAttribute("stroke-width","0.5");
    svgL.appendChild(xl);
  }

  // Axes
  const [ox,oy]=toScreen(0,0);
  function axLine(svg,x1,y1,x2,y2){ const l=document.createElementNS(svgNS,"line"); l.setAttrib
  const [ax1]=toScreen(-2.8,0); const [ax2]=toScreen(2.8,0);
  axLine(svgL,ax1,oy,ax2,oy);
  const [,ay1]=toScreen(0,-2.3); const [,ay2]=toScreen(0,2.3);
  axLine(svgL,ox,ay1,ox,ay2);

  // Vertical line at Re = reZ (the constrained path)
  const [reLineX]=toScreen(reZ,0);
  const pathLine=document.createElementNS(svgNS,"line");
  pathLine.setAttribute("x1",reLineX); pathLine.setAttribute("y1","0");
  pathLine.setAttribute("x2",reLineX); pathLine.setAttribute("y2",H);
  pathLine.setAttribute("stroke","#f97316"); pathLine.setAttribute("stroke-width","1.5");
  pathLine.setAttribute("stroke-dasharray","5,3"); svgL.appendChild(pathLine);

  // Current z point
  const [zsx,zsy]=toScreen(reZ,tVal);
  const zDot=document.createElementNS(svgNS,"circle");
  zDot.setAttribute("cx",zsx); zDot.setAttribute("cy",zsy); zDot.setAttribute("r","6");
  zDot.setAttribute("fill","#f97316"); zDot.setAttribute("stroke","#fff"); zDot.setAttribute("s
  svgL.appendChild(zDot);

  // Label
  const zLbl=document.createElementNS(svgNS,"text");
  zLbl.setAttribute("x",zsx+8); zLbl.setAttribute("y",zsy-4);
  zLbl.setAttribute("fill","#f97316"); zLbl.setAttribute("font-size","11");
  zLbl.setAttribute("font-family","sans-serif"); zLbl.textContent=`z`;
  svgL.appendChild(zLbl);
```

```
// Panel label
const pl=document.createElementNS(svgNS,"text"); pl.setAttribute("x","6"); pl.setAttribute("y","
pl.setAttribute("fill","#1f2937"); pl.setAttribute("font-size","11"); pl.setAttribute("font-fam
pl.setAttribute("font-weight","bold"); pl.textContent="z-plane (coloured by 2π strips)";
svgL.appendChild(pl);

// Right SVG — w-plane
const svgR = document.createElementNS(svgNS,"svg");
svgR.setAttribute("width",W); svgR.setAttribute("height",H);
svgR.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:

const eRe = Math.exp(reZ); // radius of orbit = e^{Re z}
const wCX = W/2, wCY = H/2;
const wSCALE = Math.min(100/eRe, 70); // auto-scale so circle fits

function toWScreen(re,im){ return [wCX+re*wSCALE, wCY-im*wSCALE]; }
function axLineW(x1,y1,x2,y2){ const l=document.createElementNS(svgNS,"line"); l.setAttribute("x
axLineW(0,wCY,W,wCY); axLineW(wCX,0,wCX,H);

// Draw circle |w| = e^{Re z}
const circ=document.createElementNS(svgNS,"circle");
circ.setAttribute("cx",wCX); circ.setAttribute("cy",wCY);
circ.setAttribute("r",(eRe*wSCALE).toFixed(1));
circ.setAttribute("fill","none"); circ.setAttribute("stroke","#d1d5db"); circ.setAttribute("stro
svgR.appendChild(circ);

// Trail: last 2π of orbit
const TRAIL_STEPS = 80;
const trailPath = document.createElementNS(svgNS,"path");
let pathD="";
for (let k=TRAIL_STEPS; k>=0; k--) {
  const pastT = tVal - (k/TRAIL_STEPS)*(2*Math.PI);
  const wr = eRe*Math.cos(pastT), wi = eRe*Math.sin(pastT);
  const [wx,wy]=toWScreen(wr,wi);
  pathD += (k===TRAIL_STEPS ? `M ${wx} ${wy}` : ` L ${wx} ${wy}`);
}
trailPath.setAttribute("d",pathD); trailPath.setAttribute("fill","none");
trailPath.setAttribute("stroke","#3b82f6"); trailPath.setAttribute("stroke-width","2");
trailPath.setAttribute("opacity","0.6"); svgR.appendChild(trailPath);

// Current w point
const wr=eRe*Math.cos(tVal), wi=eRe*Math.sin(tVal);
const [wsx,wsy]=toWScreen(wr,wi);
const wDot=document.createElementNS(svgNS,"circle");
wDot.setAttribute("cx",wsx); wDot.setAttribute("cy",wsy); wDot.setAttribute("r","6");
wDot.setAttribute("fill","#3b82f6"); wDot.setAttribute("stroke","#fff"); wDot.setAttribute("stro
svgR.appendChild(wDot);
```

```
  // Radius annotation
  const rLine=document.createElementNS(svgNS,"line");
  rLine.setAttribute("x1",wCX); rLine.setAttribute("y1",wCY);
  rLine.setAttribute("x2",wsx); rLine.setAttribute("y2",wsy);
  rLine.setAttribute("stroke","#6b7280"); rLine.setAttribute("stroke-width","0.8");
  rLine.setAttribute("stroke-dasharray","3,2"); svgR.appendChild(rLine);

  const rLbl=document.createElementNS(svgNS,"text");
  const midRx=wCX+(wsx-wCX)*0.5, midRy=wCY+(wsy-wCY)*0.5;
  rLbl.setAttribute("x",midRx+4); rLbl.setAttribute("y",midRy);
  rLbl.setAttribute("fill","#6b7280"); rLbl.setAttribute("font-size","10");
  rLbl.setAttribute("font-family","sans-serif");
  rLbl.textContent=`e^${reZ.toFixed(1)} □ ${eRe.toFixed(2)}`;
  svgR.appendChild(rLbl);

  // Orbit count
  const laps = Math.floor(tVal / (2*Math.PI));
  const lapBox=document.createElementNS(svgNS,"rect");
  lapBox.setAttribute("x",W-90); lapBox.setAttribute("y","5");
  lapBox.setAttribute("width","85"); lapBox.setAttribute("height","36");
  lapBox.setAttribute("fill","#1e40af"); lapBox.setAttribute("rx","5"); svgR.appendChild(lapBox
  const lapTxt=document.createElementNS(svgNS,"text");
  lapTxt.setAttribute("x",W-47); lapTxt.setAttribute("y","20");
  lapTxt.setAttribute("fill","white"); lapTxt.setAttribute("font-size","9");
  lapTxt.setAttribute("text-anchor","middle"); lapTxt.setAttribute("font-family","sans-serif");
  lapTxt.textContent="full orbits"; svgR.appendChild(lapTxt);
  const lapNum=document.createElementNS(svgNS,"text");
  lapNum.setAttribute("x",W-47); lapNum.setAttribute("y","36");
  lapNum.setAttribute("fill","white"); lapNum.setAttribute("font-size","16");
  lapNum.setAttribute("text-anchor","middle"); lapNum.setAttribute("font-family","sans-serif");
  lapNum.setAttribute("font-weight","bold"); lapNum.textContent=laps.toString(); svgR.appendChi

  // Panel label
  const pr=document.createElementNS(svgNS,"text"); pr.setAttribute("x","6"); pr.setAttribute("y
  pr.setAttribute("fill","#1f2937"); pr.setAttribute("font-size","11"); pr.setAttribute("font-f
  pr.setAttribute("font-weight","bold"); pr.textContent="w-plane: image e^z";
  svgR.appendChild(pr);

  const wrap=document.createElement("div");
  wrap.style.cssText="display:flex; gap:0.75rem; flex-wrap:wrap; margin:0.5rem 0;";
  wrap.appendChild(svgL); wrap.appendChild(svgR);
  return wrap;
}

const container=document.createElement("div");
container.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem
```

```
  const titleEl=document.createElement("div");
  titleEl.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  titleEl.textContent="Complex exponential: periodicity in 2π strips";
  container.appendChild(titleEl);
  container.appendChild(reSlider);
  container.appendChild(tSlider);

  const vizDiv=document.createElement("div"); container.appendChild(vizDiv);

  const infoDiv=document.createElement("div");
  infoDiv.style.cssText="font-size:0.82em; color:#374151; font-family:sans-serif; margin-top:0.3rem
  container.appendChild(infoDiv);

  const noteEl=document.createElement("div");
  noteEl.style.cssText="margin-top:0.4rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  noteEl.textContent="Left: z-plane with horizontal strips of height 2π — each strip maps to the sa
  container.appendChild(noteEl);

  function update() {
    const reZ=reSlider.value, tVal=tSlider.value;
    vizDiv.innerHTML=""; vizDiv.appendChild(buildViz(reZ,tVal));
    const wr=Math.exp(reZ)*Math.cos(tVal), wi=Math.exp(reZ)*Math.sin(tVal);
    infoDiv.textContent=`z = ${reZ.toFixed(2)} + ${tVal.toFixed(2)}i   →   e^z = ${wr.toFixed(3)} +
  }

  reSlider.addEventListener("input",update);
  tSlider.addEventListener("input",update);
  update();

  return container;
}
```

### 47.5.2. Trigonometric

$$\cos z = \frac{e^{iz} + e^{-iz}}{2}, \qquad \sin z = \frac{e^{iz} - e^{-iz}}{2i}$$

Both entire. The identities $\cos^2 z + \sin^2 z = 1$ and all standard trig identities hold. For purely imaginary argument: $\cos(iy) = \cosh y$, $\sin(iy) = i \sinh y$ — the complex trig and hyperbolic functions are closely related.

### 47.5.3. Logarithm

$$\log z = \ln |z| + i \arg z = \ln r + i\theta$$

**Multivalued**: $\arg z$ is defined only up to multiples of $2\pi$. The **principal value** $\operatorname{Log} z = \ln|z| + i \operatorname{Arg} z$ uses $\operatorname{Arg} z \in (-\pi, \pi]$ and is analytic everywhere except on the **branch cut** along the negative real axis — the line where $\operatorname{Arg} z$ jumps discontinuously by $2\pi$ as you cross it. Without the cut, $\operatorname{Log} z$ cannot be made single-valued and continuous.

```
//| echo: false
{
  const W = 400, H = 360, CX = 180, CY = 180, SCALE = 65;
  const svgNS = "http://www.w3.org/2000/svg";

  // Preset path configs: array of [re, im] control points defining a closed loop
  const paths = {
    "Loop around origin (winding = +1)": [
      [1.5, 0], [1.2, 1.2], [0, 1.8], [-1.2, 1.2], [-1.5, 0],
      [-1.2, -1.2], [0, -1.8], [1.2, -1.2]
    ],
    "Loop missing origin (winding = 0)": [
      [1.0, 0.8], [1.8, 0.4], [2.0, 1.2], [1.4, 1.8], [0.6, 1.4]
    ],
    "Double loop (winding = +2)": [
      [1.8, 0], [1.4, 1.0], [0, 1.8], [-1.4, 1.0], [-1.8, 0],
      [-1.4, -1.0], [0, -1.8], [1.4, -1.0],
      [1.2, 0.3], [0.5, 0.8], [-0.5, 0.5], [-0.8, 0], [-0.5, -0.5],
      [0.5, -0.8], [1.2, -0.3]
    ]
  };

  const pathSelector = Inputs.select(Object.keys(paths), {
    value: Object.keys(paths)[0],
    label: "Path"
  });
  const branchToggle = Inputs.toggle({label: "Show branch cut", value: false});
  const tSlider = Inputs.range([0, 1], {value: 0, step: 0.01, label: "Progress t"});

  function catmullRomPoint(pts, t) {
    const n = pts.length;
    const totalT = n;
    const tScaled = t * n;
    const i = Math.floor(tScaled) % n;
    const localT = tScaled - Math.floor(tScaled);

    const p0 = pts[(i - 1 + n) % n];
    const p1 = pts[i];
    const p2 = pts[(i + 1) % n];
    const p3 = pts[(i + 2) % n];

    const t2 = localT*localT, t3 = localT*t2;
    const re = 0.5*((2*p1[0]) + (-p0[0]+p2[0])*localT + (2*p0[0]-5*p1[0]+4*p2[0]-p3[0])*t2 + (-p0
```

```
  const im = 0.5*((2*p1[1]) + (-p0[1]+p2[1])*localT + (2*p0[1]-5*p1[1]+4*p2[1]-p3[1])*t2 + (-p0[1
  return [re, im];
}

function computeWindingAndArgTrace(pts, numSteps) {
  let totalDeltaArg = 0;
  let prevArg = null;
  const trace = [];

  for (let step = 0; step <= numSteps; step++) {
    const t = step / numSteps;
    const [re, im] = catmullRomPoint(pts, t);
    const arg = Math.atan2(im, re);

    if (prevArg !== null) {
      let dArg = arg - prevArg;
      while (dArg > Math.PI) dArg -= 2*Math.PI;
      while (dArg < -Math.PI) dArg += 2*Math.PI;
      totalDeltaArg += dArg;
    }
    prevArg = arg;
    trace.push([t, totalDeltaArg]);
  }
  return trace;
}

function toScreen(re, im) {
  return [CX + re*SCALE, CY - im*SCALE];
}

function render(pathName, showBranch, tVal) {
  const pts = paths[pathName];
  const STEPS = 300;
  const argTrace = computeWindingAndArgTrace(pts, STEPS);
  const windingAtEnd = argTrace[STEPS][1] / (2*Math.PI);
  const windingRounded = Math.round(windingAtEnd);

  // SVG
  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", W); svg.setAttribute("height", H);
  svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:bl

  // Grid
  const gridG = document.createElementNS(svgNS, "g");
  gridG.setAttribute("stroke", "#e5e7eb"); gridG.setAttribute("stroke-width", "0.5");
  for (let i=-3; i<=3; i++) {
    const xl = document.createElementNS(svgNS,"line");
```

```
    const [lx] = toScreen(i,0); const [,ly1] = toScreen(0,-3); const [,ly2] = toScreen(0,3);
    xl.setAttribute("x1",lx); xl.setAttribute("y1",ly1); xl.setAttribute("x2",lx); xl.setAttrib
    gridG.appendChild(xl);
    const yl = document.createElementNS(svgNS,"line");
    const [lx1] = toScreen(-3,0); const [lx2] = toScreen(3,0); const [,ly] = toScreen(0,i);
    yl.setAttribute("x1",lx1); yl.setAttribute("y1",ly); yl.setAttribute("x2",lx2); yl.setAttri
    gridG.appendChild(yl);
  }
  svg.appendChild(gridG);

  // Axes
  const [ox, oy] = toScreen(0,0);
  const axG = document.createElementNS(svgNS,"g"); axG.setAttribute("stroke","#9ca3af"); axG.se
  const xa = document.createElementNS(svgNS,"line");
  const [ax1] = toScreen(-3.2,0); const [ax2] = toScreen(3.2,0);
  xa.setAttribute("x1",ax1); xa.setAttribute("y1",oy); xa.setAttribute("x2",ax2); xa.setAttribu
  const ya = document.createElementNS(svgNS,"line");
  const [,ay1] = toScreen(0,-2.6); const [,ay2] = toScreen(0,2.6);
  ya.setAttribute("x1",ox); ya.setAttribute("y1",ay1); ya.setAttribute("x2",ox); ya.setAttribut
  axG.appendChild(xa); axG.appendChild(ya);
  svg.appendChild(axG);

  // Branch cut: negative real axis
  if (showBranch) {
    const bc = document.createElementNS(svgNS,"line");
    const [bx1] = toScreen(-3.2,0); const [bx2] = toScreen(0,0);
    bc.setAttribute("x1",bx1); bc.setAttribute("y1",oy); bc.setAttribute("x2",bx2); bc.setAttri
    bc.setAttribute("stroke","#ef4444"); bc.setAttribute("stroke-width","2.5");
    bc.setAttribute("stroke-dasharray","6,3");
    svg.appendChild(bc);
    const bcLabel = document.createElementNS(svgNS,"text");
    const [blx] = toScreen(-2,0);
    bcLabel.setAttribute("x",blx); bcLabel.setAttribute("y",oy-6);
    bcLabel.setAttribute("fill","#ef4444"); bcLabel.setAttribute("font-size","10");
    bcLabel.setAttribute("font-family","sans-serif"); bcLabel.textContent="branch cut";
    svg.appendChild(bcLabel);
  }

  // Full path
  let pathD = "";
  for (let s=0; s<=STEPS; s++) {
    const t = s/STEPS;
    const [re,im] = catmullRomPoint(pts, t);
    const [px,py] = toScreen(re,im);
    pathD += (s===0 ? `M ${px} ${py}` : ` L ${px} ${py}`);
  }
  pathD += " Z";
```

```
    const pathEl = document.createElementNS(svgNS,"path");
    pathEl.setAttribute("d",pathD); pathEl.setAttribute("fill","none");
    pathEl.setAttribute("stroke","#f97316"); pathEl.setAttribute("stroke-width","2");
    svg.appendChild(pathEl);

    // Animated point
    const currentStep = Math.round(tVal * STEPS);
    const [cre,cim] = catmullRomPoint(pts, tVal);
    const [cpx,cpy] = toScreen(cre,cim);
    const pt = document.createElementNS(svgNS,"circle");
    pt.setAttribute("cx",cpx); pt.setAttribute("cy",cpy); pt.setAttribute("r","6");
    pt.setAttribute("fill","#3b82f6"); pt.setAttribute("stroke","#fff"); pt.setAttribute("stroke-wi
    svg.appendChild(pt);

    // Origin mark
    const origPt = document.createElementNS(svgNS,"circle");
    origPt.setAttribute("cx",ox); origPt.setAttribute("cy",oy); origPt.setAttribute("r","4");
    origPt.setAttribute("fill","#1f2937"); origPt.setAttribute("stroke","#fff"); origPt.setAttribute
    svg.appendChild(origPt);
    const origLabel = document.createElementNS(svgNS,"text");
    origLabel.setAttribute("x",ox+5); origLabel.setAttribute("y",oy-6);
    origLabel.setAttribute("fill","#1f2937"); origLabel.setAttribute("font-size","10");
    origLabel.setAttribute("font-family","sans-serif"); origLabel.textContent="O";
    svg.appendChild(origLabel);

    // Winding number display
    const windBox = document.createElementNS(svgNS,"rect");
    windBox.setAttribute("x",W-95); windBox.setAttribute("y",5);
    windBox.setAttribute("width",88); windBox.setAttribute("height",44);
    windBox.setAttribute("fill","#1e40af"); windBox.setAttribute("rx","5");
    svg.appendChild(windBox);
    const windLabel = document.createElementNS(svgNS,"text");
    windLabel.setAttribute("x",W-51); windLabel.setAttribute("y",22);
    windLabel.setAttribute("fill","white"); windLabel.setAttribute("font-size","11");
    windLabel.setAttribute("text-anchor","middle"); windLabel.setAttribute("font-family","sans-seri
    windLabel.textContent = "winding #";
    svg.appendChild(windLabel);
    const windNum = document.createElementNS(svgNS,"text");
    windNum.setAttribute("x",W-51); windNum.setAttribute("y",41);
    windNum.setAttribute("fill","white"); windNum.setAttribute("font-size","18");
    windNum.setAttribute("text-anchor","middle"); windNum.setAttribute("font-family","sans-serif");
    windNum.setAttribute("font-weight","bold");
    windNum.textContent = (tVal < 0.99) ? "..." : windingRounded.toString();
    svg.appendChild(windNum);

    // Arg trace mini-chart
    const chartX = 0, chartY = H - 70, chartW = W, chartH = 65;
```

```
  const chartBg = document.createElementNS(svgNS,"rect");
  chartBg.setAttribute("x",chartX); chartBg.setAttribute("y",chartY);
  chartBg.setAttribute("width",chartW); chartBg.setAttribute("height",chartH);
  chartBg.setAttribute("fill","rgba(241,245,249,0.9)");
  svg.appendChild(chartBg);

  const maxArgAbs = Math.max(2*Math.PI*1.5, Math.abs(argTrace[STEPS][1]) * 1.2);
  function argToChart(t, argVal) {
    const cx = chartX + t * chartW;
    const cy = chartY + chartH/2 - (argVal / maxArgAbs) * (chartH/2 - 6);
    return [cx, cy];
  }

  // Zero line
  const [zx1, zy1] = argToChart(0, 0); const [zx2] = argToChart(1, 0);
  const zeroLine = document.createElementNS(svgNS,"line");
  zeroLine.setAttribute("x1",zx1); zeroLine.setAttribute("y1",zy1);
  zeroLine.setAttribute("x2",zx2); zeroLine.setAttribute("y2",zy1);
  zeroLine.setAttribute("stroke","#9ca3af"); zeroLine.setAttribute("stroke-width","0.8");
  zeroLine.setAttribute("stroke-dasharray","3,3");
  svg.appendChild(zeroLine);

  // Arg trace line up to current t
  let traceD = "";
  for (let s=0; s<=currentStep && s < argTrace.length; s++) {
    const [tc, argVal] = argTrace[s];
    const [cx, cy] = argToChart(tc, argVal);
    traceD += (s===0 ? `M ${cx} ${cy}` : ` L ${cx} ${cy}`);
  }
  const traceEl = document.createElementNS(svgNS,"path");
  traceEl.setAttribute("d",traceD); traceEl.setAttribute("fill","none");
  traceEl.setAttribute("stroke","#8b5cf6"); traceEl.setAttribute("stroke-width","2");
  svg.appendChild(traceEl);

  const chartLabel = document.createElementNS(svgNS,"text");
  chartLabel.setAttribute("x",chartX+4); chartLabel.setAttribute("y",chartY+12);
  chartLabel.setAttribute("fill","#6b7280"); chartLabel.setAttribute("font-size","9");
  chartLabel.setAttribute("font-family","sans-serif"); chartLabel.textContent="arg(z(t))";
  svg.appendChild(chartLabel);

  return svg;
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r

const title = document.createElement("div");
```

```
  title.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  title.textContent = "Winding number and the branch cut";
  container.appendChild(title);

  container.appendChild(pathSelector);
  container.appendChild(branchToggle);
  container.appendChild(tSlider);

  const vizDiv = document.createElement("div");
  container.appendChild(vizDiv);

  const note = document.createElement("div");
  note.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  note.textContent = "Move the slider to animate the point around the path. The purple trace shows a
  container.appendChild(note);

  function update() {
    vizDiv.innerHTML = "";
    vizDiv.appendChild(render(pathSelector.value, branchToggle.value, tSlider.value));
  }
  pathSelector.addEventListener("input", update);
  branchToggle.addEventListener("input", update);
  tSlider.addEventListener("input", update);
  update();

  return container;
}
```

### 47.5.4. Complex powers

$$z^\alpha = e^{\alpha \log z}$$

Multivalued in general. Integer powers are single-valued and entire; non-integer powers require a branch cut.

---

## 47.6. Conformal mappings: preview

At any point $z_0$ where $f$ is analytic and $f'(z_0) \neq 0$, the map $f$ is **conformal** (Chapter 4): it preserves the angle between any two curves crossing at $z_0$ (in magnitude and orientation). The scale factor is $|f'(z_0)|$ and the rotation angle is $\arg f'(z_0)$.

Conformal mappings transform solutions of Laplace's equation from one domain into solutions in another. That is how potential-flow problems, electrostatic field calculations, and heat-conduction problems in irregular geometries are reduced to solvable ones. Chapter 4 develops this fully.

```
//| echo: false
{
  // Conformal angle-preservation test
  // Left: z-plane with z₀ (slider-controlled) and two arm lines at angles ⍰₁ , ⍰₂
  // Right: w-plane showing images of those arms; live angle readout
  // Critical-point detection: when f'(z₀)=0, the map is not conformal there
  // f=z̄ demo reveals reflection (angle reversed)

  const W = 310, H = 290, SCALE = 65, CX = W/2, CY = H/2;
  const svgNS = "http://www.w3.org/2000/svg";

  const funcSel = Inputs.select(
    ["f(z) = z²", "f(z) = e⍰", "f(z) = sin z", "f(z) = 1/z", "f(z) = z̄"],
    { value: "f(z) = z²", label: "Function" }
  );
  const z0xSl = Inputs.range([-1.8, 1.8], { value: 0.8, step: 0.1, label: "Re(z₀)" });
  const z0ySl = Inputs.range([-1.8, 1.8], { value: 0.6, step: 0.1, label: "Im(z₀)" });
  const arm1Sl = Inputs.range([0, 2*Math.PI], { value: 0.3, step: 0.1, label: "Arm 1 angle ⍰₁" })
  const arm2Sl = Inputs.range([0, 2*Math.PI], { value: 1.1, step: 0.1, label: "Arm 2 angle ⍰₂" })

  function evalF(name, re, im) {
    switch (name) {
      case "f(z) = z²":   return [re*re-im*im, 2*re*im];
      case "f(z) = e⍰": { const ex=Math.exp(re); return [ex*Math.cos(im), ex*Math.sin(im)]; }
      case "f(z) = sin z": return [Math.sin(re)*Math.cosh(im), Math.cos(re)*Math.sinh(im)];
      case "f(z) = 1/z": { const d=re*re+im*im; if(d<1e-9)return[NaN,NaN]; return[re/d,-im/d]; }
      case "f(z) = z̄":   return [re, -im];
      default:             return [re, im];
    }
  }

  // Numerical derivative f'(z₀)
  function evalFPrime(name, re, im) {
    const eps=0.001;
    const [fr,fi]=evalF(name,re,im);
    const [hr,hi]=evalF(name,re+eps,im);
    return [(hr-fr)/eps, (hi-fi)/eps];
  }

  function toScreen(re,im,cx,cy,sc){ return [cx+re*sc, cy-im*sc]; }
  function fromScreen(px,py,cx,cy,sc){ return [(px-cx)/sc, -(py-cy)/sc]; }

  function addLine(svg,x1,y1,x2,y2,color,w,dash){
    const l=document.createElementNS(svgNS,"line");
    l.setAttribute("x1",x1);l.setAttribute("y1",y1);l.setAttribute("x2",x2);l.setAttribute("y2",y
    l.setAttribute("stroke",color);l.setAttribute("stroke-width",w||"1.5");
    if(dash)l.setAttribute("stroke-dasharray",dash);
```

```
    svg.appendChild(l);
  }
  function addCircle(svg,cx,cy,r,fill,stroke){
    const c=document.createElementNS(svgNS,"circle");
    c.setAttribute("cx",cx);c.setAttribute("cy",cy);c.setAttribute("r",r);
    c.setAttribute("fill",fill);c.setAttribute("stroke",stroke||"#fff");c.setAttribute("stroke-width
    svg.appendChild(c);
  }
  function addText(svg,x,y,txt,color,size){
    const t=document.createElementNS(svgNS,"text");
    t.setAttribute("x",x);t.setAttribute("y",y);t.setAttribute("fill",color);
    t.setAttribute("font-size",size||"11");t.setAttribute("font-family","sans-serif");
    t.textContent=txt; svg.appendChild(t);
  }
  function makeSVG(label){
    const svg=document.createElementNS(svgNS,"svg");
    svg.setAttribute("width",W); svg.setAttribute("height",H);
    svg.style.cssText="background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:bloc
    // Grid
    for(let i=-3;i<=3;i++){
      const xl=document.createElementNS(svgNS,"line");
      const [lx]=toScreen(i,0,CX,CY,SCALE); xl.setAttribute("x1",lx);xl.setAttribute("y1","0");xl.se
      xl.setAttribute("stroke","#e5e7eb");xl.setAttribute("stroke-width","0.5");svg.appendChild(xl)
      const yl=document.createElementNS(svgNS,"line");
      const [,ly]=toScreen(0,i,CX,CY,SCALE); yl.setAttribute("x1","0");yl.setAttribute("y1",ly);yl.s
      yl.setAttribute("stroke","#e5e7eb");yl.setAttribute("stroke-width","0.5");svg.appendChild(yl)
    }
    addLine(svg,0,CY,W,CY,"#9ca3af","1");
    addLine(svg,CX,0,CX,H,"#9ca3af","1");
    addText(svg,6,14,label,"#1f2937","11");
    return svg;
  }

  function buildViz(fn, z0x, z0y, a1, a2) {
    const svgL = makeSVG("z-plane");
    const svgR = makeSVG("w-plane");

    const ARM_LEN = 0.9; // in complex units

    // Arm end-points in z-plane
    const z1r = z0x + ARM_LEN*Math.cos(a1), z1i = z0y + ARM_LEN*Math.sin(a1);
    const z2r = z0x + ARM_LEN*Math.cos(a2), z2i = z0y + ARM_LEN*Math.sin(a2);

    // Draw z-plane arms
    const [z0sx,z0sy] = toScreen(z0x,z0y,CX,CY,SCALE);
    const [z1sx,z1sy] = toScreen(z1r,z1i,CX,CY,SCALE);
    const [z2sx,z2sy] = toScreen(z2r,z2i,CX,CY,SCALE);
```

```
addLine(svgL,z0sx,z0sy,z1sx,z1sy,"#f97316","2.5");
addLine(svgL,z0sx,z0sy,z2sx,z2sy,"#8b5cf6","2.5");
addCircle(svgL,z0sx,z0sy,6,"#f97316");
addText(svgL,z0sx+8,z0sy-6,"z₀","#f97316","12");

// Input angle between arms
const inputAngle = a2 - a1;

// Map z₀ and arm endpoints
const [w0r,w0i] = evalF(fn,z0x,z0y);
const [w1r,w1i] = evalF(fn,z1r,z1i);
const [w2r,w2i] = evalF(fn,z2r,z2i);

// Auto-scale w-plane around w₀
const d1 = Math.sqrt((w1r-w0r)**2+(w1i-w0i)**2);
const d2 = Math.sqrt((w2r-w0r)**2+(w2i-w0i)**2);
const maxD = Math.max(d1,d2,0.1);
const wSCALE = Math.min(SCALE, 80/(maxD));
const wCX = W/2 - w0r*wSCALE, wCY = H/2 + w0i*wSCALE;

function toWS(re,im){ return [wCX+re*wSCALE, wCY-im*wSCALE]; }

if (isFinite(w0r) && isFinite(w0i)) {
  const [w0sx,w0sy]=toWS(w0r,w0i);
  const [w1sx,w1sy]=toWS(w1r,w1i);
  const [w2sx,w2sy]=toWS(w2r,w2i);
  addLine(svgR,w0sx,w0sy,w1sx,w1sy,"#f97316","2.5");
  addLine(svgR,w0sx,w0sy,w2sx,w2sy,"#8b5cf6","2.5");
  addCircle(svgR,w0sx,w0sy,6,"#f97316");
  addText(svgR,w0sx+8,w0sy-6,"f(z₀)","#3b82f6","11");

  // Output angle between arm images
  const out1 = Math.atan2(w1i-w0i, w1r-w0r);
  const out2 = Math.atan2(w2i-w0i, w2r-w0r);
  let outputAngle = out2 - out1;
  while (outputAngle >  Math.PI) outputAngle -= 2*Math.PI;
  while (outputAngle < -Math.PI) outputAngle += 2*Math.PI;
  let inputAngleN = inputAngle;
  while (inputAngleN >  Math.PI) inputAngleN -= 2*Math.PI;
  while (inputAngleN < -Math.PI) inputAngleN += 2*Math.PI;

  // Derivative magnitude for critical-point test
  const [fpRe,fpIm] = evalFPrime(fn,z0x,z0y);
  const fpMag = Math.sqrt(fpRe*fpRe+fpIm*fpIm);
  const isCritical = fpMag < 0.05;
  const isConj = fn === "f(z) = z̄ ";
  const angleDiff = Math.abs(Math.abs(outputAngle)-Math.abs(inputAngleN));
```

```
      const isConformal = !isCritical && !isConj && angleDiff < 0.15;

      // Status badge
      let statusColor, statusMsg;
      if (isCritical) { statusColor="#dc2626"; statusMsg="Critical point: f'(z₀)≈0 — conformality fa
      else if (isConj) { statusColor="#dc2626"; statusMsg="f=z̄ : reflection — angles reversed (not c
      else if (isConformal) { statusColor="#059669"; statusMsg="Conformal: angles preserved"; }
      else { statusColor="#dc2626"; statusMsg="Angle mismatch — check critical point or branch"; }

      const badge=document.createElementNS(svgNS,"rect");
      badge.setAttribute("x","4"); badge.setAttribute("y",H-26);
      badge.setAttribute("width",W-8); badge.setAttribute("height","22");
      badge.setAttribute("fill",statusColor); badge.setAttribute("rx","4"); badge.setAttribute("opac
      svgR.appendChild(badge);
      const badgeTxt=document.createElementNS(svgNS,"text");
      badgeTxt.setAttribute("x","8"); badgeTxt.setAttribute("y",H-11);
      badgeTxt.setAttribute("fill",statusColor); badgeTxt.setAttribute("font-size","9.5");
      badgeTxt.setAttribute("font-family","sans-serif"); badgeTxt.textContent=statusMsg;
      svgR.appendChild(badgeTxt);

      // Angle readout in w-plane
      addText(svgR,6,H-32,`input angle: ${(inputAngleN*180/Math.PI).toFixed(1)}°  →  output: ${(outp
    } else {
      addText(svgR,W/2-40,H/2,"(pole — undefined)","#dc2626","11");
    }

    const wrap=document.createElement("div");
    wrap.style.cssText="display:flex; gap:0.75rem; flex-wrap:wrap; margin:0.5rem 0;";
    wrap.appendChild(svgL); wrap.appendChild(svgR);
    return wrap;
  }

  const container=document.createElement("div");
  container.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem 0

  const titleEl=document.createElement("div");
  titleEl.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  titleEl.textContent="Conformal mapping: angle-preservation test";
  container.appendChild(titleEl);
  container.appendChild(funcSel);
  container.appendChild(z0xSl);
  container.appendChild(z0ySl);
  container.appendChild(arm1Sl);
  container.appendChild(arm2Sl);

  const vizDiv=document.createElement("div"); container.appendChild(vizDiv);
```

```
  const noteEl=document.createElement("div");
  noteEl.style.cssText="margin-top:0.4rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  noteEl.textContent="Orange and purple arms cross at z₀ in the left panel. Their images cross at
  container.appendChild(noteEl);

  function update() {
    vizDiv.innerHTML="";
    vizDiv.appendChild(buildViz(funcSel.value, z0xSl.value, z0ySl.value, arm1Sl.value, arm2Sl.val
  }
  funcSel.addEventListener("input",update);
  z0xSl.addEventListener("input",update);
  z0ySl.addEventListener("input",update);
  arm1Sl.addEventListener("input",update);
  arm2Sl.addEventListener("input",update);
  update();

  return container;
}
```

---

## 47.7. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
```

```
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell); row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "Solution complete.";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSteps
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn); controls.appendChild(resetBtn); controls.appendChild(counter);
  container.appendChild(stepsDiv); container.appendChild(controls);
  renderSteps(); updateButtons();
  return container;
}
```

---

### 47.7.1. Exercise 1: Cauchy-Riemann equations for $f(z) = z^2$

Verify that $f(z) = z^2$ satisfies the Cauchy-Riemann equations everywhere and compute $f'(z)$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Expand in terms of x and y",
    eq: "(x+iy)^2 = x^2 - y^2 + 2ixy \\implies u = x^2-y^2,\\quad v = 2xy"
  },
  {
    op: "Check u_x = v_y",
    eq: "u_x = 2x, \\quad v_y = 2x \\implies u_x = v_y \\checkmark"
  },
  {
    op: "Check u_y = -v_x",
    eq: "u_y = -2y, \\quad v_x = 2y \\implies u_y = -v_x \\checkmark"
  },
  {
    op: "Partial derivatives continuous everywhere → f is entire",
    eq: "f'(z) = u_x + iv_x = 2x + 2iy = 2(x+iy) = 2z",
    note: "Same power rule as real calculus. The formal rules d(z^n)/dz = nz^{n-1} hold for all i
  }
])
```

---

### 47.7.2. Exercise 2: Finding a harmonic conjugate

Show that $u = x^3 - 3xy^2$ is harmonic, find its harmonic conjugate $v$, and identify the analytic function $f = u + iv$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Verify Laplace's equation",
    eq: "u_{xx} = 6x, \\quad u_{yy} = -6x \\implies \\nabla^2 u = 0 \\checkmark"
  },
  {
    op: "C-R gives v_y = u_x = 3x²-3y²; integrate w.r.t. y",
    eq: "v = 3x^2 y - y^3 + g(x)"
  },
  {
    op: "C-R gives v_x = -u_y; differentiate v and set equal",
    eq: "v_x = 6xy + g'(x) = -u_y = 6xy \\implies g'(x) = 0 \\implies g = C"
  },
  {
    op: "Identify f(z)",
    eq: "f(z) = (x^3-3xy^2) + i(3x^2y-y^3) + iC = z^3 + iC",
    note: "Expand z³ = (x+iy)³ to verify: real part is x³-3xy², imaginary part is 3x²y-y³."
```

```
  }
])
```

---

### 47.7.3. Exercise 3: $e^z$ is entire

Verify the Cauchy-Riemann equations for $f(z) = e^z$ and confirm $\frac{d}{dz}e^z = e^z$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write e^z in terms of u and v",
    eq: "e^z = e^x(\\cos y + i\\sin y) \\implies u = e^x\\cos y,\\quad v = e^x\\sin y"
  },
  {
    op: "Check u_x = v_y",
    eq: "u_x = e^x\\cos y, \\quad v_y = e^x\\cos y \\checkmark"
  },
  {
    op: "Check u_y = -v_x",
    eq: "u_y = -e^x\\sin y, \\quad -v_x = -e^x\\sin y \\checkmark"
  },
  {
    op: "Compute the derivative",
    eq: "\\frac{d}{dz}e^z = u_x + iv_x = e^x\\cos y + ie^x\\sin y = e^z",
    note: "The exponential is its own derivative — as in real analysis."
  }
])
```

---

### 47.7.4. Exercise 4: Where is $f(z) = |z|^2$ differentiable?

```
//| echo: false
makeStepperHTML([
  {
    op: "Write |z|² = x²+y², so u = x²+y², v = 0",
    eq: "u_x = 2x, \\quad v_y = 0 \\implies \\text{C-R requires } x = 0"
  },
  {
    op: "Second C-R equation",
    eq: "u_y = 2y, \\quad -v_x = 0 \\implies \\text{C-R requires } y = 0"
  },
```

```
  {
    op: "C-R equations hold only at the origin",
    eq: "f \\text{ is differentiable only at } z = 0, \\text{ with } f'(0) = 0"
  },
  {
    op: "Differentiability at a single point ⇒ analytic",
    eq: "|z|^2 \\text{ is nowhere analytic}",
    note: "Analyticity requires differentiability in an open disk. A single point of differentiab
  }
])
```

---

### 47.7.5. Exercise 5: Powers via De Moivre

Computing roots and powers in polar form — a technique used throughout this volume, including when locating poles and computing residues.

Compute $(1+i)^8$ and find all fourth roots of $-16$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Convert 1+i to polar",
    eq: "1+i = \\sqrt{2}\\,e^{i\\pi/4}"
  },
  {
    op: "Raise to the 8th power",
    eq: "(1+i)^8 = (\\sqrt{2})^8 e^{i8\\pi/4} = 16\\,e^{2\\pi i} = 16",
    note: "e^{2πi} = 1."
  },
  {
    op: "Write −16 in polar form",
    eq: "-16 = 16\\,e^{i\\pi}"
  },
  {
    op: "Fourth roots: take r^{1/4} and divide argument by 4, adding kπ/2 for k=0,1,2,3",
    eq: "z_k = 2\\,e^{i(\\pi/4 + k\\pi/2)}, \\quad k = 0,1,2,3"
  },
  {
    op: "List the four roots",
    eq: "z_0 = \\sqrt{2}(1+i), \\quad z_1 = \\sqrt{2}(-1+i), \\quad z_2 = \\sqrt{2}(-1-i), \\quad
    note: "The four roots form the vertices of a square centred at the origin with circumradius 2
  }
])
```

```
//| echo: false
{
  // De Moivre roots: static diagram — four fourth roots of −16
  // Orange dots on circle of radius 2, arguments labelled
  const W = 300, H = 300, SCALE = 90, CX = W/2, CY = H/2;
  const svgNS = "http://www.w3.org/2000/svg";

  const svg=document.createElementNS(svgNS,"svg");
  svg.setAttribute("width",W); svg.setAttribute("height",H);
  svg.style.cssText="background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:block

  function toS(re,im){ return [CX+re*SCALE, CY-im*SCALE]; }

  // Grid
  for(let i=-2;i<=2;i++){
    const xl=document.createElementNS(svgNS,"line"); const [lx]=toS(i,0);
    xl.setAttribute("x1",lx);xl.setAttribute("y1","0");xl.setAttribute("x2",lx);xl.setAttribute("y2"
    xl.setAttribute("stroke","#e5e7eb");xl.setAttribute("stroke-width","0.5");svg.appendChild(xl);
    const yl=document.createElementNS(svgNS,"line"); const [,ly]=toS(0,i);
    yl.setAttribute("x1","0");yl.setAttribute("y1",ly);yl.setAttribute("x2",W);yl.setAttribute("y2"
    yl.setAttribute("stroke","#e5e7eb");yl.setAttribute("stroke-width","0.5");svg.appendChild(yl);
  }

  // Axes
  const [ox,oy]=toS(0,0);
  function axLine(x1,y1,x2,y2){ const l=document.createElementNS(svgNS,"line"); l.setAttribute("x1"
  const [ax1]=toS(-2.3,0); const [ax2]=toS(2.3,0); axLine(ax1,oy,ax2,oy);
  const [,ay1]=toS(0,-2.3); const [,ay2]=toS(0,2.3); axLine(ox,ay1,ox,ay2);

  // Circle of radius 2
  const circ=document.createElementNS(svgNS,"circle");
  circ.setAttribute("cx",CX); circ.setAttribute("cy",CY); circ.setAttribute("r",(2*SCALE).toString()
  circ.setAttribute("fill","none"); circ.setAttribute("stroke","#d1d5db"); circ.setAttribute("stroke
  svg.appendChild(circ);

  // Radius label
  const rLbl=document.createElementNS(svgNS,"text");
  rLbl.setAttribute("x",CX+4); rLbl.setAttribute("y",CY-(2*SCALE)+14);
  rLbl.setAttribute("fill","#9ca3af"); rLbl.setAttribute("font-size","10");
  rLbl.setAttribute("font-family","sans-serif"); rLbl.textContent="|z| = 2"; svg.appendChild(rLbl);

  // The four roots: z_k = 2·e^{i(π/4 + kπ/2)}, k=0,1,2,3
  const r=2;
  const roots=[
    { k:0, arg: Math.PI/4,       label:"z₀ = √2(1+i)",   argLabel:"π/4" },
    { k:1, arg: 3*Math.PI/4,     label:"z₁ = √2(−1+i)",  argLabel:"3π/4" },
    { k:2, arg: 5*Math.PI/4,     label:"z₂ = √2(−1−i)",  argLabel:"5π/4" },
```

```
  { k:3, arg: 7*Math.PI/4,     label:"z₃ = √2(1−i)",   argLabel:"7π/4" }
];

roots.forEach(({ arg, label, argLabel },idx) => {
  const re=r*Math.cos(arg), im=r*Math.sin(arg);
  const [sx,sy]=toS(re,im);

  // Spoke
  const spoke=document.createElementNS(svgNS,"line");
  spoke.setAttribute("x1",ox); spoke.setAttribute("y1",oy);
  spoke.setAttribute("x2",sx); spoke.setAttribute("y2",sy);
  spoke.setAttribute("stroke","#fdba74"); spoke.setAttribute("stroke-width","1");
  spoke.setAttribute("stroke-dasharray","4,3"); svg.appendChild(spoke);

  // Dot
  const dot=document.createElementNS(svgNS,"circle");
  dot.setAttribute("cx",sx); dot.setAttribute("cy",sy); dot.setAttribute("r","7");
  dot.setAttribute("fill","#f97316"); dot.setAttribute("stroke","#fff"); dot.setAttribute("stro
  svg.appendChild(dot);

  // Root label
  const dx = re >= 0 ? 10 : -10;
  const anchor = re >= 0 ? "start" : "end";
  const dyLbl = im >= 0 ? -10 : 16;
  const lt=document.createElementNS(svgNS,"text");
  lt.setAttribute("x",sx+dx); lt.setAttribute("y",sy+dyLbl);
  lt.setAttribute("fill","#f97316"); lt.setAttribute("font-size","10.5");
  lt.setAttribute("font-family","sans-serif"); lt.setAttribute("text-anchor",anchor);
  lt.textContent=label; svg.appendChild(lt);

  // Argument label on the arc
  const arcR=32+idx*8;
  const midAng=arg*0.9+0.05;
  const alx=CX+arcR*Math.cos(midAng-Math.PI*0.08);
  const aly=CY-arcR*Math.sin(midAng-Math.PI*0.08);
  const at=document.createElementNS(svgNS,"text");
  at.setAttribute("x",alx); at.setAttribute("y",aly);
  at.setAttribute("fill","#8b5cf6"); at.setAttribute("font-size","9");
  at.setAttribute("font-family","sans-serif"); at.setAttribute("text-anchor","middle");
  at.textContent=argLabel; svg.appendChild(at);
});

// Origin
const orig=document.createElementNS(svgNS,"circle");
orig.setAttribute("cx",ox); orig.setAttribute("cy",oy); orig.setAttribute("r","4");
orig.setAttribute("fill","#6b7280"); orig.setAttribute("stroke","#fff"); orig.setAttribute("str
svg.appendChild(orig);
```

```
  // Title
  const titleT=document.createElementNS(svgNS,"text");
  titleT.setAttribute("x","6"); titleT.setAttribute("y","14");
  titleT.setAttribute("fill","#1f2937"); titleT.setAttribute("font-size","11");
  titleT.setAttribute("font-family","sans-serif"); titleT.setAttribute("font-weight","bold");
  titleT.textContent="Fourth roots of −16"; svg.appendChild(titleT);

  // Axis labels
  const reL=document.createElementNS(svgNS,"text"); reL.setAttribute("x",ax2-18);reL.setAttribute("y
  reL.setAttribute("fill","#6b7280");reL.setAttribute("font-size","10");reL.setAttribute("font-famil
  const imL=document.createElementNS(svgNS,"text"); imL.setAttribute("x",ox+5);imL.setAttribute("y",
  imL.setAttribute("fill","#6b7280");imL.setAttribute("font-size","10");imL.setAttribute("font-famil

  const wrap=document.createElement("div"); wrap.style.cssText="margin:0.75rem 0;";
  wrap.appendChild(svg);
  const note=document.createElement("div");
  note.style.cssText="font-size:0.82em; color:#6b7280; font-style:italic; margin-top:0.3rem; max-wid
  note.textContent="The four fourth roots of −16 sit at the vertices of a square of circumradius 2.
  wrap.appendChild(note);
  return wrap;
}
```

# 48. Complex integration

Cauchy's theorem, the integral formula, and their consequences

## 48.1. Contours and contour integrals

A **contour** (or path) in the complex plane is a **piecewise smooth** curve $C$: $z = z(t)$, $a \leq t \leq b$, where $z(t) = x(t) + iy(t)$ and $z'(t)$ is piecewise continuous. Piecewise smooth means the curve can have finitely many corners, but no cusps or fractal wiggles — at each smooth piece the tangent direction is well-defined.

The **contour integral** of $f$ along $C$ is:

$$\int_C f(z)\,dz = \int_a^b f(z(t))\,z'(t)\,dt$$

This is a real line integral in disguise. Writing $f = u + iv$ and $dz = dx + idy$, multiply out the product:

$$(u + iv)(dx + i\,dy) = u\,dx + iu\,dy + ivdx + i^2 vdy = (u\,dx - vdy) + i(vdx + u\,dy)$$

Separating real and imaginary parts:

$$\int_C fdz = \int_C (u\,dx - vdy) + i\int_C (vdx + u\,dy)$$

**Properties.** - Linearity: $\int_C (\alpha f + \beta g)\,dz = \alpha \int_C fdz + \beta \int_C g\,dz$. - Reversal: $\int_{-C} fdz = -\int_C fdz$ (reversing the direction negates the integral). - Additivity: if $C = C_1 + C_2$, then $\int_C = \int_{C_1} + \int_{C_2}$.

```
//| echo: false
// VIZ 1: Path-dependence explorer
// Three simultaneous paths from A=0 to B=1+i; trace slider; integrand selector;
// running-total bars for Re and Im parts. Key moment: all three match for analytic f.
{
  const W = 320, H = 300, SCALE = 110, OX = 40, OY = 260;
  const svgNS = "http://www.w3.org/2000/svg";
  const STEPS = 400;

  const intSelector = Inputs.select(
```

```
  ["f(z) = z  (analytic)", "f(z) = z\u0305  (not analytic)", "f(z) = |z|\u00b2  (not analytic)"],
  { value: "f(z) = z  (analytic)", label: "Integrand" }
);
const traceSlider = Inputs.range([0, 1], { value: 1, step: 0.005, label: "Trace t" });

function toSx(re) { return OX + re * SCALE; }
function toSy(im) { return OY - im * SCALE; }

// Path parametrisations: t in [0,1], endpoint B = 1+i
function pathStraight(t) { return [t, t]; }
function pathRightAngle(t) {
  if (t <= 0.5) return [2 * t, 0];
  return [1, 2 * (t - 0.5)];
}
function pathSemicircle(t) {
  // Arc from 0 to 1+i via midpoint 0.5 + 0.5i offset by a semicircular bulge
  // Use parametric: goes right along x, then curves up, centre at (0.5, 0.5)
  const theta = Math.PI * t; // 0..pi maps 0 -> bottom -> 1+i via arc
  const cx = 0.5, cy = 0.5, r = Math.sqrt(0.5);
  const angle = -Math.PI / 2 + theta; // start at bottom of circle
  return [cx + r * Math.cos(angle), cy + r * Math.sin(angle)];
}

function evalF(fname, re, im) {
  if (fname.startsWith("f(z) = z  ")) return [re, im];
  if (fname.startsWith("f(z) = z\u0305")) return [re, -im];
  return [re * re + im * im, 0]; // |z|^2
}

function numericalIntegral(pathFn, fname, upToT) {
  const steps = Math.max(1, Math.round(upToT * STEPS));
  const dt = upToT / steps;
  let rA = 0, iA = 0;
  for (let k = 0; k < steps; k++) {
    const t0 = k * dt, t1 = (k + 1) * dt;
    const [r0, i0] = pathFn(t0);
    const [r1, i1] = pathFn(t1);
    const mr = (r0 + r1) / 2, mi = (i0 + i1) / 2;
    const dr = r1 - r0, di = i1 - i0;
    const [fr, fi] = evalF(fname, mr, mi);
    // f(z) dz = (fr+i*fi)(dr+i*di) = fr*dr - fi*di + i*(fi*dr + fr*di)
    rA += fr * dr - fi * di;
    iA += fi * dr + fr * di;
  }
  return [rA, iA];
}
```

```
const PATHS = [
  { name: "Straight", fn: pathStraight, color: "#f97316" },
  { name: "Right-angle", fn: pathRightAngle, color: "#3b82f6" },
  { name: "Semicircle", fn: pathSemicircle, color: "#8b5cf6" }
];

function buildSVG(fname, tVal) {
  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", W); svg.setAttribute("height", H);
  svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display

  // Grid lines
  for (let v = 0; v <= 2; v++) {
    const gv = document.createElementNS(svgNS, "line");
    gv.setAttribute("x1", toSx(v)); gv.setAttribute("y1", toSy(-0.3));
    gv.setAttribute("x2", toSx(v)); gv.setAttribute("y2", toSy(1.6));
    gv.setAttribute("stroke", "#e5e7eb"); gv.setAttribute("stroke-width", "0.5");
    svg.appendChild(gv);
  }
  for (let u = 0; u <= 2; u++) {
    const gh = document.createElementNS(svgNS, "line");
    gh.setAttribute("x1", toSx(-0.1)); gh.setAttribute("y1", toSy(u));
    gh.setAttribute("x2", toSx(1.6)); gh.setAttribute("y2", toSy(u));
    gh.setAttribute("stroke", "#e5e7eb"); gh.setAttribute("stroke-width", "0.5");
    svg.appendChild(gh);
  }

  // Axes
  const axH = document.createElementNS(svgNS, "line");
  axH.setAttribute("x1", toSx(-0.1)); axH.setAttribute("y1", toSy(0));
  axH.setAttribute("x2", toSx(1.6)); axH.setAttribute("y2", toSy(0));
  axH.setAttribute("stroke", "#9ca3af"); axH.setAttribute("stroke-width", "1.2");
  svg.appendChild(axH);
  const axV = document.createElementNS(svgNS, "line");
  axV.setAttribute("x1", toSx(0)); axV.setAttribute("y1", toSy(-0.2));
  axV.setAttribute("x2", toSx(0)); axV.setAttribute("y2", toSy(1.6));
  axV.setAttribute("stroke", "#9ca3af"); axV.setAttribute("stroke-width", "1.2");
  svg.appendChild(axV);

  // Axis labels
  const lblRe = document.createElementNS(svgNS, "text");
  lblRe.setAttribute("x", toSx(1.55)); lblRe.setAttribute("y", toSy(0) - 6);
  lblRe.setAttribute("font-size", "10"); lblRe.setAttribute("fill", "#6b7280");
  lblRe.setAttribute("font-family", "sans-serif"); lblRe.textContent = "Re";
  svg.appendChild(lblRe);
  const lblIm = document.createElementNS(svgNS, "text");
  lblIm.setAttribute("x", toSx(0) + 4); lblIm.setAttribute("y", toSy(1.55));
```

```
lblIm.setAttribute("font-size", "10"); lblIm.setAttribute("fill", "#6b7280");
lblIm.setAttribute("font-family", "sans-serif"); lblIm.textContent = "Im";
svg.appendChild(lblIm);

// Draw full ghost paths
for (const p of PATHS) {
  let d = "";
  for (let k = 0; k <= STEPS; k++) {
    const [re, im] = p.fn(k / STEPS);
    d += (k === 0 ? "M" : "L") + ` ${toSx(re)} ${toSy(im)}`;
  }
  const ghost = document.createElementNS(svgNS, "path");
  ghost.setAttribute("d", d); ghost.setAttribute("fill", "none");
  ghost.setAttribute("stroke", p.color); ghost.setAttribute("stroke-width", "1.5");
  ghost.setAttribute("stroke-dasharray", "4,3"); ghost.setAttribute("opacity", "0.35");
  svg.appendChild(ghost);
}

// Draw traced portions
for (const p of PATHS) {
  const steps = Math.round(tVal * STEPS);
  let d = "";
  for (let k = 0; k <= steps; k++) {
    const [re, im] = p.fn(k / STEPS);
    d += (k === 0 ? "M" : "L") + ` ${toSx(re)} ${toSy(im)}`;
  }
  const trace = document.createElementNS(svgNS, "path");
  trace.setAttribute("d", d); trace.setAttribute("fill", "none");
  trace.setAttribute("stroke", p.color); trace.setAttribute("stroke-width", "2.5");
  svg.appendChild(trace);

  // Moving dot
  const [cr, ci] = p.fn(tVal);
  const dot = document.createElementNS(svgNS, "circle");
  dot.setAttribute("cx", toSx(cr)); dot.setAttribute("cy", toSy(ci));
  dot.setAttribute("r", "5"); dot.setAttribute("fill", p.color);
  dot.setAttribute("stroke", "#fff"); dot.setAttribute("stroke-width", "1.5");
  svg.appendChild(dot);
}

// A and B endpoint markers
const A = document.createElementNS(svgNS, "circle");
A.setAttribute("cx", toSx(0)); A.setAttribute("cy", toSy(0));
A.setAttribute("r", "5"); A.setAttribute("fill", "#111827");
A.setAttribute("stroke", "#fff"); A.setAttribute("stroke-width", "1.5");
svg.appendChild(A);
const Albl = document.createElementNS(svgNS, "text");
```

```
  Albl.setAttribute("x", toSx(0) + 7); Albl.setAttribute("y", toSy(0) - 5);
  Albl.setAttribute("font-size", "11"); Albl.setAttribute("font-family", "sans-serif");
  Albl.setAttribute("fill", "#111827"); Albl.textContent = "A = 0";
  svg.appendChild(Albl);
  const B = document.createElementNS(svgNS, "circle");
  B.setAttribute("cx", toSx(1)); B.setAttribute("cy", toSy(1));
  B.setAttribute("r", "5"); B.setAttribute("fill", "#111827");
  B.setAttribute("stroke", "#fff"); B.setAttribute("stroke-width", "1.5");
  svg.appendChild(B);
  const Blbl = document.createElementNS(svgNS, "text");
  Blbl.setAttribute("x", toSx(1) + 7); Blbl.setAttribute("y", toSy(1) - 5);
  Blbl.setAttribute("font-size", "11"); Blbl.setAttribute("font-family", "sans-serif");
  Blbl.setAttribute("fill", "#111827"); Blbl.textContent = "B = 1+i";
  svg.appendChild(Blbl);

  return svg;
}

function buildBars(fname, tVal) {
  const BAR_W = 260, BAR_H = 200;
  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", BAR_W); svg.setAttribute("height", BAR_H);
  svg.style.cssText = "display:block;";

  const integrals = PATHS.map(p => numericalIntegral(p.fn, fname, tVal));
  const maxVal = Math.max(1e-9, ...integrals.flatMap(([r, i]) => [Math.abs(r), Math.abs(i)]));
  const barScale = 70 / maxVal;
  const colW = 60, midY = 110, lblY = BAR_H - 10;

  // Zero line
  const zl = document.createElementNS(svgNS, "line");
  zl.setAttribute("x1", 10); zl.setAttribute("y1", midY);
  zl.setAttribute("x2", BAR_W - 10); zl.setAttribute("y2", midY);
  zl.setAttribute("stroke", "#d1d5db"); zl.setAttribute("stroke-width", "1");
  svg.appendChild(zl);

  const headRe = document.createElementNS(svgNS, "text");
  headRe.setAttribute("x", "10"); headRe.setAttribute("y", "14");
  headRe.setAttribute("font-size", "9"); headRe.setAttribute("fill", "#6b7280");
  headRe.setAttribute("font-family", "sans-serif"); headRe.textContent = "Re(\u222bf dz)";
  svg.appendChild(headRe);
  const headIm = document.createElementNS(svgNS, "text");
  headIm.setAttribute("x", "135"); headIm.setAttribute("y", "14");
  headIm.setAttribute("font-size", "9"); headIm.setAttribute("fill", "#6b7280");
  headIm.setAttribute("font-family", "sans-serif"); headIm.textContent = "Im(\u222bf dz)";
  svg.appendChild(headIm);
```

```
  for (let i = 0; i < PATHS.length; i++) {
    const [rv, iv] = integrals[i];
    const col = PATHS[i].color;
    const name = PATHS[i].name;
    const xR = 14 + i * 38, xI = 140 + i * 38;

    // Re bar
    const rh = rv * barScale;
    const reBar = document.createElementNS(svgNS, "rect");
    reBar.setAttribute("x", xR); reBar.setAttribute("y", rh >= 0 ? midY - rh : midY);
    reBar.setAttribute("width", 30); reBar.setAttribute("height", Math.max(2, Math.abs(rh)));
    reBar.setAttribute("fill", col); reBar.setAttribute("rx", "2");
    svg.appendChild(reBar);
    const reVal = document.createElementNS(svgNS, "text");
    reVal.setAttribute("x", xR + 15); reVal.setAttribute("y", rh >= 0 ? midY - rh - 3 : midY + Mat
    reVal.setAttribute("font-size", "8"); reVal.setAttribute("text-anchor", "middle");
    reVal.setAttribute("fill", col); reVal.setAttribute("font-family", "monospace");
    reVal.textContent = rv.toFixed(3);
    svg.appendChild(reVal);

    // Im bar
    const ih = iv * barScale;
    const imBar = document.createElementNS(svgNS, "rect");
    imBar.setAttribute("x", xI); imBar.setAttribute("y", ih >= 0 ? midY - ih : midY);
    imBar.setAttribute("width", 30); imBar.setAttribute("height", Math.max(2, Math.abs(ih)));
    imBar.setAttribute("fill", col); imBar.setAttribute("rx", "2");
    svg.appendChild(imBar);
    const imVal = document.createElementNS(svgNS, "text");
    imVal.setAttribute("x", xI + 15); imVal.setAttribute("y", ih >= 0 ? midY - ih - 3 : midY + Mat
    imVal.setAttribute("font-size", "8"); imVal.setAttribute("text-anchor", "middle");
    imVal.setAttribute("fill", col); imVal.setAttribute("font-family", "monospace");
    imVal.textContent = iv.toFixed(3) + "i";
    svg.appendChild(imVal);

    // Path label
    const lbl = document.createElementNS(svgNS, "text");
    lbl.setAttribute("x", (xR + xI) / 2 - 6); lbl.setAttribute("y", lblY);
    lbl.setAttribute("font-size", "8"); lbl.setAttribute("text-anchor", "middle");
    lbl.setAttribute("fill", col); lbl.setAttribute("font-family", "sans-serif");
    lbl.textContent = name;
    svg.appendChild(lbl);
  }

  // Agreement indicator
  const vals = integrals;
  const reClose = Math.abs(vals[0][0] - vals[1][0]) < 0.01 && Math.abs(vals[1][0] - vals[2][0]) <
  const imClose = Math.abs(vals[0][1] - vals[1][1]) < 0.01 && Math.abs(vals[1][1] - vals[2][1]) <
```

```
  const agree = reClose && imClose;
  const annot = document.createElementNS(svgNS, "text");
  annot.setAttribute("x", BAR_W / 2); annot.setAttribute("y", BAR_H - 26);
  annot.setAttribute("font-size", "9"); annot.setAttribute("text-anchor", "middle");
  annot.setAttribute("font-weight", "bold");
  annot.setAttribute("fill", agree ? "#059669" : "#dc2626");
  annot.setAttribute("font-family", "sans-serif");
  annot.textContent = agree ? "all three agree \u2014 path-independent" : "values differ \u2014
  svg.appendChild(annot);

  return svg;
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r

const title = document.createElement("div");
title.style.cssText = "font-weight:600; margin-bottom:0.5rem;";
title.textContent = "Path-dependence explorer: three paths from A to B";
container.appendChild(title);

container.appendChild(intSelector);
container.appendChild(traceSlider);

const panels = document.createElement("div");
panels.style.cssText = "display:flex; gap:1rem; align-items:flex-start; flex-wrap:wrap; margin-

const svgDiv = document.createElement("div");
const barDiv = document.createElement("div");
panels.appendChild(svgDiv);
panels.appendChild(barDiv);
container.appendChild(panels);

// Legend
const legend = document.createElement("div");
legend.style.cssText = "display:flex; gap:1rem; margin-top:0.5rem; font-size:0.82em;";
for (const p of PATHS) {
  const item = document.createElement("span");
  item.style.cssText = `color:${p.color};`;
  item.textContent = "\u25ac " + p.name;
  legend.appendChild(item);
}
container.appendChild(legend);

const note = document.createElement("div");
note.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
note.textContent = "Drag the trace slider to advance all three paths simultaneously. For f(z) =
```

```
  container.appendChild(note);

  function update() {
    const fname = intSelector.value;
    const tVal = traceSlider.value;
    svgDiv.innerHTML = ""; svgDiv.appendChild(buildSVG(fname, tVal));
    barDiv.innerHTML = ""; barDiv.appendChild(buildBars(fname, tVal));
  }
  intSelector.addEventListener("input", update);
  traceSlider.addEventListener("input", update);
  update();

  return container;
}
```

### 48.1.1. The ML inequality

When you need a bound on a contour integral rather than its exact value, the ML inequality is the standard tool:

$$\left|\int_C f(z)\,dz\right| \leq M \cdot L$$

where $M = \max_{z\in C}|f(z)|$ and $L$ is the arc length of $C$. This mirrors the real estimate $|\int_a^b f dx| \leq \max|f| \cdot (b-a)$. The application section below shows how to read off $M$ and $L$ in practice.

```
//| echo: false
// VIZ 6: ML inequality geometry
// Contour in z-plane with perpendicular spike magnitudes; M and L labels;
// bounding rectangle vs actual integral bar in right panel.
{
  const W = 340, H = 260, SCALE = 70, OX = 100, OY = 160;
  const svgNS = "http://www.w3.org/2000/svg";
  const STEPS = 120;

  const radiusML = Inputs.range([0.4, 2.2], { value: 1.2, step: 0.05, label: "Contour radius R" });
  const funcML = Inputs.select(
    ["f(z) = 1/z\u00b2", "f(z) = e^z", "f(z) = 1/(z+2)"],
    { value: "f(z) = 1/z\u00b2", label: "f(z)" }
  );

  function toSx(re) { return OX + re * SCALE; }
  function toSy(im) { return OY - im * SCALE; }

  function fMagnitude(fname, re, im) {
    if (fname === "f(z) = 1/z\u00b2") {
```

```
    const d2 = re*re + im*im;
    return d2 < 1e-8 ? 99 : 1 / d2;
  }
  if (fname === "f(z) = e^z") return Math.exp(re);
  const dr = re + 2, d = dr*dr + im*im;
  return d < 1e-10 ? 99 : 1 / Math.sqrt(d);
}

function build(R, fname) {
  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", W); svg.setAttribute("height", H);
  svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display

  // Axes
  for (const [x1,y1,x2,y2] of [
    [toSx(-0.2), toSy(0), toSx(2.4), toSy(0)],
    [toSx(0), toSy(-1.8), toSx(0), toSy(2.0)]
  ]) {
    const ax = document.createElementNS(svgNS, "line");
    ax.setAttribute("x1",x1); ax.setAttribute("y1",y1);
    ax.setAttribute("x2",x2); ax.setAttribute("y2",y2);
    ax.setAttribute("stroke","#d1d5db"); ax.setAttribute("stroke-width","1");
    svg.appendChild(ax);
  }

  // Contour circle and spike magnitudes
  const mags = [];
  let dPath = "";
  for (let k = 0; k <= STEPS; k++) {
    const theta = (2 * Math.PI * k) / STEPS;
    const re = R * Math.cos(theta), im = R * Math.sin(theta);
    const m = fMagnitude(fname, re, im);
    mags.push(Math.min(m, 4)); // cap display
    dPath += (k === 0 ? "M" : "L") + ` ${toSx(re)} ${toSy(im)}`;
  }
  const contour = document.createElementNS(svgNS, "path");
  contour.setAttribute("d", dPath); contour.setAttribute("fill", "none");
  contour.setAttribute("stroke", "#3b82f6"); contour.setAttribute("stroke-width", "2");
  svg.appendChild(contour);

  // Spikes
  const SPIKE_SCALE = 20;
  for (let k = 0; k < STEPS; k += 6) {
    const theta = (2 * Math.PI * k) / STEPS;
    const re = R * Math.cos(theta), im = R * Math.sin(theta);
    const sx = toSx(re), sy = toSy(im);
    // Outward normal direction
```

```
    const nx = Math.cos(theta), ny = -Math.sin(theta); // screen normal (y flipped)
    const h = Math.min(mags[k], 3.5) * SPIKE_SCALE;
    const ex = sx + nx * h, ey = sy + ny * h;
    const spike = document.createElementNS(svgNS, "line");
    spike.setAttribute("x1", sx); spike.setAttribute("y1", sy);
    spike.setAttribute("x2", ex); spike.setAttribute("y2", ey);
    spike.setAttribute("stroke", "#f97316"); spike.setAttribute("stroke-width", "1.5");
    spike.setAttribute("opacity", "0.7");
    svg.appendChild(spike);
  }

  // M = max on contour, L = arc length = 2πR
  const Mval = Math.max(...mags);
  const Lval = 2 * Math.PI * R;
  const ML = Mval * Lval;

  // M level ring
  const mlCircle = document.createElementNS(svgNS, "circle");
  mlCircle.setAttribute("cx", toSx(0)); mlCircle.setAttribute("cy", toSy(0));
  mlCircle.setAttribute("r", (R + Math.min(Mval, 3.5) * (SPIKE_SCALE / SCALE)));
  mlCircle.setAttribute("fill", "none"); mlCircle.setAttribute("stroke", "#dc2626");
  mlCircle.setAttribute("stroke-width", "1"); mlCircle.setAttribute("stroke-dasharray", "3,3");
  svg.appendChild(mlCircle);

  // Labels
  const mlbl = document.createElementNS(svgNS, "text");
  mlbl.setAttribute("x", toSx(R) + 4); mlbl.setAttribute("y", toSy(0) - 4);
  mlbl.setAttribute("font-size", "10"); mlbl.setAttribute("fill", "#f97316");
  mlbl.setAttribute("font-family", "sans-serif"); mlbl.textContent = "M = " + Mval.toFixed(2);
  svg.appendChild(mlbl);
  const llbl = document.createElementNS(svgNS, "text");
  llbl.setAttribute("x", toSx(0) + 4); llbl.setAttribute("y", toSy(-R) + 14);
  llbl.setAttribute("font-size", "10"); llbl.setAttribute("fill", "#3b82f6");
  llbl.setAttribute("font-family", "sans-serif"); llbl.textContent = "L = 2\u03c0R = " + Lval.toF
  svg.appendChild(llbl);

  // Contour label
  const clbl = document.createElementNS(svgNS, "text");
  clbl.setAttribute("x", 4); clbl.setAttribute("y", 14);
  clbl.setAttribute("font-size", "10"); clbl.setAttribute("fill", "#3b82f6");
  clbl.setAttribute("font-family", "sans-serif"); clbl.textContent = "C: |z| = " + R.toFixed(2);
  svg.appendChild(clbl);

  // Bar panel on right
  const BX = W - 100, BY = 20, BW = 80, BH = H - 50;
  const bg = document.createElementNS(svgNS, "rect");
  bg.setAttribute("x", BX); bg.setAttribute("y", BY);
```

```
bg.setAttribute("width", BW); bg.setAttribute("height", BH);
bg.setAttribute("fill", "#f9fafb"); bg.setAttribute("rx", "4");
svg.appendChild(bg);

// Actual |∮ f dz| — numerical estimate via trapezoidal sum
let sumRe = 0, sumIm = 0;
for (let k = 0; k < STEPS; k++) {
  const t0 = (2*Math.PI*k)/STEPS, t1 = (2*Math.PI*(k+1))/STEPS;
  const r0 = R*Math.cos(t0), i0 = R*Math.sin(t0);
  const r1 = R*Math.cos(t1), i1 = R*Math.sin(t1);
  const mr = (r0+r1)/2, mi = (i0+i1)/2;
  const dr = r1-r0, di = i1-i0;
  // f(z) evaluation
  let fr, fi;
  if (fname === "f(z) = 1/z\u00b2") {
    const d2 = mr*mr+mi*mi; if (d2<1e-8) continue;
    const re2 = (mr*mr-mi*mi)/(d2*d2), im2 = (-2*mr*mi)/(d2*d2);
    fr = re2; fi = im2;
  } else if (fname === "f(z) = e^z") {
    fr = Math.exp(mr)*Math.cos(mi); fi = Math.exp(mr)*Math.sin(mi);
  } else {
    const ddr = mr+2, d = ddr*ddr+mi*mi; if (d<1e-10) continue;
    fr = ddr/d; fi = -mi/d;
  }
  sumRe += fr*dr - fi*di;
  sumIm += fi*dr + fr*di;
}
const actual = Math.sqrt(sumRe*sumRe + sumIm*sumIm);
const maxBar = Math.max(ML, actual, 0.1);
const mlH = (ML / maxBar) * (BH - 30);
const actH = (actual / maxBar) * (BH - 30);

// ML bar
const mlBar = document.createElementNS(svgNS, "rect");
mlBar.setAttribute("x", BX + 8); mlBar.setAttribute("y", BY + BH - 20 - mlH);
mlBar.setAttribute("width", 24); mlBar.setAttribute("height", mlH);
mlBar.setAttribute("fill", "#dc2626"); mlBar.setAttribute("rx", "2");
svg.appendChild(mlBar);
const mlBarLbl = document.createElementNS(svgNS, "text");
mlBarLbl.setAttribute("x", BX + 20); mlBarLbl.setAttribute("y", BY + BH - 22 - mlH);
mlBarLbl.setAttribute("font-size", "8"); mlBarLbl.setAttribute("text-anchor", "middle");
mlBarLbl.setAttribute("fill", "#dc2626"); mlBarLbl.setAttribute("font-family", "monospace");
mlBarLbl.textContent = "ML"; svg.appendChild(mlBarLbl);
const mlNum = document.createElementNS(svgNS, "text");
mlNum.setAttribute("x", BX + 20); mlNum.setAttribute("y", BY + BH - 8);
mlNum.setAttribute("font-size", "8"); mlNum.setAttribute("text-anchor", "middle");
mlNum.setAttribute("fill", "#dc2626"); mlNum.setAttribute("font-family", "monospace");
```

```
  mlNum.textContent = ML.toFixed(2); svg.appendChild(mlNum);

  // Actual bar
  const actBar = document.createElementNS(svgNS, "rect");
  actBar.setAttribute("x", BX + 46); actBar.setAttribute("y", BY + BH - 20 - actH);
  actBar.setAttribute("width", 24); actBar.setAttribute("height", actH);
  actBar.setAttribute("fill", "#3b82f6"); actBar.setAttribute("rx", "2");
  svg.appendChild(actBar);
  const actBarLbl = document.createElementNS(svgNS, "text");
  actBarLbl.setAttribute("x", BX + 58); actBarLbl.setAttribute("y", BY + BH - 22 - actH);
  actBarLbl.setAttribute("font-size", "8"); actBarLbl.setAttribute("text-anchor", "middle");
  actBarLbl.setAttribute("fill", "#3b82f6"); actBarLbl.setAttribute("font-family", "monospace");
  actBarLbl.textContent = "|\u222bf dz|"; svg.appendChild(actBarLbl);
  const actNum = document.createElementNS(svgNS, "text");
  actNum.setAttribute("x", BX + 58); actNum.setAttribute("y", BY + BH - 8);
  actNum.setAttribute("font-size", "8"); actNum.setAttribute("text-anchor", "middle");
  actNum.setAttribute("fill", "#3b82f6"); actNum.setAttribute("font-family", "monospace");
  actNum.textContent = actual.toFixed(2); svg.appendChild(actNum);

  return svg;
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem

const titleEl = document.createElement("div");
titleEl.style.cssText = "font-weight:600; margin-bottom:0.5rem;";
titleEl.textContent = "ML inequality: spikes show |f(z)| along the contour";
container.appendChild(titleEl);
container.appendChild(radiusML);
container.appendChild(funcML);

const vizDiv = document.createElement("div");
vizDiv.style.marginTop = "0.5rem";
container.appendChild(vizDiv);

const noteEl = document.createElement("div");
noteEl.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
noteEl.textContent = "Orange spikes show |f(z)| at each point of the contour. M is the tallest spi
container.appendChild(noteEl);

function update() {
  vizDiv.innerHTML = "";
  vizDiv.appendChild(build(radiusML.value, funcML.value));
}
radiusML.addEventListener("input", update);
funcML.addEventListener("input", update);
```

```
  update();

  return container;
}
```

### 48.1.2. Integral of $1/z^{n+1}$ around a circle

$$\oint_{|z|=r} \frac{dz}{z^{n+1}} = \begin{cases} 2\pi i & n = 0 \\ 0 & n \neq 0 \end{cases}$$

*Proof for $n = 0$:* parametrise $z = re^{i\theta}$, $dz = ire^{i\theta}d\theta$:

$$\int_0^{2\pi} \frac{ire^{i\theta}}{re^{i\theta}}\, d\theta = \int_0^{2\pi} i\, d\theta = 2\pi i$$

This single result — that the integral of $1/z$ around the origin is $2\pi i$ — is the seed from which the residue theorem grows.

```
//| echo: false
// VIZ 5: 1/z^{n+1} circle integral — discrete n slider, θ trace with play button,
// integrand arrows, two strip-chart accumulators, green highlight when n=0 gives 2πi.
{
  const W = 340, H = 300, SCALE = 85, CX = 155, CY = 155;
  const BARY = 260, BARH = 30, BARW = 250, BARX = 55;
  const svgNS = "http://www.w3.org/2000/svg";
  const STEPS = 360;

  const nSlider = Inputs.range([-2, 3], { value: 0, step: 1, label: "n  (integrand = 1/z^{n+1})"
  const thetaSlider = Inputs.range([0, 1], { value: 0, step: 0.005, label: "Trace \u03b8/2\u03c0"
  const playBtn = document.createElement("button");
  playBtn.textContent = "Play \u25b6";
  playBtn.style.cssText = "padding:0.3rem 0.8rem; border:1px solid #d1d5db; border-radius:4px; ba

  function computeCircleIntegral(n, upToTheta) {
    // z = e^{i*theta}, dz = i*e^{i*theta}*dtheta
    // 1/z^{n+1} * dz = e^{-i*(n+1)*theta} * i*e^{i*theta} * dtheta = i * e^{-i*n*theta} * dtheta
    // Integral from 0 to 2pi*upToTheta
    const T = 2 * Math.PI * upToTheta;
    if (Math.abs(n) < 0.01) {
      // n=0: integral = i * T
      return [0, T];
    }
    // n != 0: i * integral of e^{-i*n*theta} dtheta = i * [e^{-i*n*theta}/(-i*n)]_0^T
    //       = i * (e^{-i*n*T} - 1) / (-i*n) = (e^{-i*n*T} - 1) / (-n)
    const cosT = Math.cos(n * T), sinT = Math.sin(n * T);
    const re = (cosT - 1) / (-n);
```

```
    const im = (-sinT) / (-n);
    return [re, im];
  }

  function toSx(re) { return CX + re * SCALE; }
  function toSy(im) { return CY - im * SCALE; }

  function buildViz(n, thetaFrac) {
    const svg = document.createElementNS(svgNS, "svg");
    svg.setAttribute("width", W); svg.setAttribute("height", H + 70);
    svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display:bl

    // Axes
    const ax = document.createElementNS(svgNS, "line");
    ax.setAttribute("x1", toSx(-1.6)); ax.setAttribute("y1", toSy(0));
    ax.setAttribute("x2", toSx(1.6)); ax.setAttribute("y2", toSy(0));
    ax.setAttribute("stroke", "#d1d5db"); ax.setAttribute("stroke-width", "1");
    svg.appendChild(ax);
    const ay = document.createElementNS(svgNS, "line");
    ay.setAttribute("x1", toSx(0)); ay.setAttribute("y1", toSy(-1.6));
    ay.setAttribute("x2", toSx(0)); ay.setAttribute("y2", toSy(1.6));
    ay.setAttribute("stroke", "#d1d5db"); ay.setAttribute("stroke-width", "1");
    svg.appendChild(ay);

    // Unit circle (ghost)
    const circle = document.createElementNS(svgNS, "circle");
    circle.setAttribute("cx", toSx(0)); circle.setAttribute("cy", toSy(0));
    circle.setAttribute("r", SCALE); circle.setAttribute("fill", "none");
    circle.setAttribute("stroke", "#e5e7eb"); circle.setAttribute("stroke-width", "1.5");
    svg.appendChild(circle);

    // Traced arc
    const arcSteps = Math.round(thetaFrac * STEPS);
    if (arcSteps > 1) {
      let arcD = "";
      for (let k = 0; k <= arcSteps; k++) {
        const theta = (2 * Math.PI * k) / STEPS;
        const px = toSx(Math.cos(theta)), py = toSy(Math.sin(theta));
        arcD += (k === 0 ? "M" : "L") + ` ${px} ${py}`;
      }
      const arcEl = document.createElementNS(svgNS, "path");
      arcEl.setAttribute("d", arcD); arcEl.setAttribute("fill", "none");
      arcEl.setAttribute("stroke", "#8b5cf6"); arcEl.setAttribute("stroke-width", "3");
      svg.appendChild(arcEl);
    }

    // Current point on circle
```

```
const theta0 = 2 * Math.PI * thetaFrac;
const cx = Math.cos(theta0), cy = Math.sin(theta0);
const cpx = toSx(cx), cpy = toSy(cy);
const pt = document.createElementNS(svgNS, "circle");
pt.setAttribute("cx", cpx); pt.setAttribute("cy", cpy);
pt.setAttribute("r", "6"); pt.setAttribute("fill", "#8b5cf6");
pt.setAttribute("stroke", "#fff"); pt.setAttribute("stroke-width", "2");
svg.appendChild(pt);

// Integrand arrow at current point: 1/z^{n+1} * dz/dtheta direction
// dz/dtheta = i*e^{i*theta}; 1/z^{n+1} = e^{-i*(n+1)*theta}
// product = i * e^{-i*n*theta}
const arrowRe = -Math.sin(n * theta0);
const arrowIm = Math.cos(n * theta0);
const arrowLen = 22;
const arrowNorm = Math.sqrt(arrowRe*arrowRe + arrowIm*arrowIm) || 1;
const adx = (arrowRe / arrowNorm) * arrowLen;
const ady = -(arrowIm / arrowNorm) * arrowLen; // screen y-flip
const arrowEl = document.createElementNS(svgNS, "line");
arrowEl.setAttribute("x1", cpx); arrowEl.setAttribute("y1", cpy);
arrowEl.setAttribute("x2", cpx + adx); arrowEl.setAttribute("y2", cpy + ady);
arrowEl.setAttribute("stroke", "#f97316"); arrowEl.setAttribute("stroke-width", "2");
arrowEl.setAttribute("marker-end", "url(#arrow-orange)");
svg.appendChild(arrowEl);

// Arrow marker def
const defs = document.createElementNS(svgNS, "defs");
const marker = document.createElementNS(svgNS, "marker");
marker.setAttribute("id", "arrow-orange"); marker.setAttribute("markerWidth", "6");
marker.setAttribute("markerHeight", "6"); marker.setAttribute("refX", "5");
marker.setAttribute("refY", "3"); marker.setAttribute("orient", "auto");
const poly = document.createElementNS(svgNS, "polygon");
poly.setAttribute("points", "0 0, 6 3, 0 6"); poly.setAttribute("fill", "#f97316");
marker.appendChild(poly); defs.appendChild(marker); svg.insertBefore(defs, svg.firstChild);

// Origin ×
const ox = toSx(0), oy2 = toSy(0);
const xSz = 5;
for (const [dx, dy] of [[-xSz,-xSz],[xSz,xSz],[xSz,-xSz],[-xSz,xSz]].slice(0,2)) {
  const line = document.createElementNS(svgNS, "line");
  line.setAttribute("x1", ox+dx); line.setAttribute("y1", oy2+dy);
  line.setAttribute("x2", ox-dx); line.setAttribute("y2", oy2-dy);
  line.setAttribute("stroke", "#ef4444"); line.setAttribute("stroke-width", "2");
  svg.appendChild(line);
}

// Axis labels
```

```
const relbl = document.createElementNS(svgNS, "text");
relbl.setAttribute("x", toSx(1.5)); relbl.setAttribute("y", toSy(0) - 5);
relbl.setAttribute("font-size", "10"); relbl.setAttribute("fill", "#9ca3af");
relbl.setAttribute("font-family", "sans-serif"); relbl.textContent = "Re";
svg.appendChild(relbl);

// Strip charts below
const [intRe, intIm] = computeCircleIntegral(n, thetaFrac);
const finalIm = n === 0 ? 2 * Math.PI : 0;
const isN0 = Math.abs(n) < 0.01;
const maxVal = Math.max(1e-6, 2 * Math.PI * 1.1);
const baseY = H + 8;

// Re strip
const reLabel = document.createElementNS(svgNS, "text");
reLabel.setAttribute("x", 4); reLabel.setAttribute("y", baseY + 15);
reLabel.setAttribute("font-size", "9"); reLabel.setAttribute("fill", "#6b7280");
reLabel.setAttribute("font-family", "sans-serif"); reLabel.textContent = "Re";
svg.appendChild(reLabel);
const reBg = document.createElementNS(svgNS, "rect");
reBg.setAttribute("x", BARX); reBg.setAttribute("y", baseY);
reBg.setAttribute("width", BARW); reBg.setAttribute("height", BARH * 0.7);
reBg.setAttribute("fill", "#f3f4f6"); reBg.setAttribute("rx", "3");
svg.appendChild(reBg);
const reW = (Math.abs(intRe) / maxVal) * BARW;
const reBar = document.createElementNS(svgNS, "rect");
reBar.setAttribute("x", BARX); reBar.setAttribute("y", baseY);
reBar.setAttribute("width", Math.min(reW, BARW)); reBar.setAttribute("height", BARH * 0.7);
reBar.setAttribute("fill", "#3b82f6"); reBar.setAttribute("rx", "3");
svg.appendChild(reBar);
const reVal = document.createElementNS(svgNS, "text");
reVal.setAttribute("x", BARX + Math.min(reW, BARW) + 4); reVal.setAttribute("y", baseY + 14);
reVal.setAttribute("font-size", "9"); reVal.setAttribute("fill", "#374151");
reVal.setAttribute("font-family", "monospace"); reVal.textContent = intRe.toFixed(3);
svg.appendChild(reVal);

// Im strip
const imLabel = document.createElementNS(svgNS, "text");
imLabel.setAttribute("x", 4); imLabel.setAttribute("y", baseY + BARH + 15);
imLabel.setAttribute("font-size", "9"); imLabel.setAttribute("fill", "#6b7280");
imLabel.setAttribute("font-family", "sans-serif"); imLabel.textContent = "Im";
svg.appendChild(imLabel);
const imBg = document.createElementNS(svgNS, "rect");
imBg.setAttribute("x", BARX); imBg.setAttribute("y", baseY + BARH);
imBg.setAttribute("width", BARW); imBg.setAttribute("height", BARH * 0.7);
imBg.setAttribute("fill", "#f3f4f6"); imBg.setAttribute("rx", "3");
svg.appendChild(imBg);
```

```
  const imW = (Math.abs(intIm) / maxVal) * BARW;
  const imBar = document.createElementNS(svgNS, "rect");
  imBar.setAttribute("x", BARX); imBar.setAttribute("y", baseY + BARH);
  imBar.setAttribute("width", Math.min(imW, BARW)); imBar.setAttribute("height", BARH * 0.7);
  const imFill = isN0 && thetaFrac > 0.95 ? "#059669" : "#8b5cf6";
  imBar.setAttribute("fill", imFill); imBar.setAttribute("rx", "3");
  svg.appendChild(imBar);
  const imVal = document.createElementNS(svgNS, "text");
  imVal.setAttribute("x", BARX + Math.min(imW, BARW) + 4); imVal.setAttribute("y", baseY + BARH
  imVal.setAttribute("font-size", "9"); imVal.setAttribute("fill", "#374151");
  imVal.setAttribute("font-family", "monospace"); imVal.textContent = intIm.toFixed(3) + "i";
  svg.appendChild(imVal);

  // n=0 result highlight
  if (isN0 && thetaFrac > 0.95) {
    const hl = document.createElementNS(svgNS, "text");
    hl.setAttribute("x", W / 2); hl.setAttribute("y", H - 5);
    hl.setAttribute("font-size", "11"); hl.setAttribute("text-anchor", "middle");
    hl.setAttribute("fill", "#059669"); hl.setAttribute("font-weight", "bold");
    hl.setAttribute("font-family", "sans-serif");
    hl.textContent = "n = 0 \u2192 \u222e dz/z = 2\u03c0i";
    svg.appendChild(hl);
  }

  return svg;
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r

const titleEl = document.createElement("div");
titleEl.style.cssText = "font-weight:600; margin-bottom:0.5rem;";
titleEl.textContent = "\u222e dz / z^{n+1} around the unit circle";
container.appendChild(titleEl);
container.appendChild(nSlider);
container.appendChild(thetaSlider);
container.appendChild(playBtn);

const vizDiv = document.createElement("div");
vizDiv.style.marginTop = "0.5rem";
container.appendChild(vizDiv);

const noteEl = document.createElement("div");
noteEl.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;"
noteEl.textContent = "The orange arrow shows the integrand 1/z^{n+1} \u00b7 dz/d\u03b8 at the c
container.appendChild(noteEl);
```

```
  let animFrame = null, playing = false;
  playBtn.onclick = () => {
    if (playing) {
      playing = false; playBtn.textContent = "Play \u25b6";
      if (animFrame) cancelAnimationFrame(animFrame);
    } else {
      playing = true; playBtn.textContent = "Pause \u23f8";
      if (thetaSlider.value >= 1) thetaSlider.value = 0;
      function step() {
        if (!playing) return;
        thetaSlider.value = Math.min(1, thetaSlider.value + 0.004);
        thetaSlider.dispatchEvent(new Event("input"));
        if (thetaSlider.value < 1) animFrame = requestAnimationFrame(step);
        else { playing = false; playBtn.textContent = "Play \u25b6"; }
      }
      animFrame = requestAnimationFrame(step);
    }
  };

  function update() {
    vizDiv.innerHTML = "";
    vizDiv.appendChild(buildViz(nSlider.value, thetaSlider.value));
  }
  nSlider.addEventListener("input", update);
  thetaSlider.addEventListener("input", update);
  update();

  return container;
}
```

---

## 48.2. Cauchy's theorem

**Cauchy-Goursat theorem.** If $f$ is analytic throughout a simply connected domain $D$ (no holes — defined in Chapter 1), then for every closed contour $C$ in $D$:

$$\oint_C f(z)\,dz = 0$$

*Intuition.* Because the domain has no holes, any closed curve can be continuously shrunk to a point. The integral shrinks with it — to zero.

**Consequence: path independence.** If $f$ is analytic in a simply connected domain and $C_1, C_2$ are two paths from $z_1$ to $z_2$ in that domain, then $\int_{C_1} f dz = \int_{C_2} f dz$. The integral depends only on the endpoints, not the path.

**Deformation principle.** If $f$ is analytic everywhere between two contours $C_1$ and $C_2$ (with the same orientation), then $\int_{C_1} f dz = \int_{C_2} f dz$. Contours can be freely deformed through regions where $f$ is analytic, without changing the integral.

```
//| echo: false
// VIZ 2: Contour deformation and singularity switch
// Domain-coloured background; bulge slider morphs contour from excluding to including origin;
// integral readout jumps from 0 to 2πi; winding number counter; optional second singularity.
{
  const W = 360, H = 320, SCALE = 70, CX = W / 2, CY = H / 2;
  const svgNS = "http://www.w3.org/2000/svg";
  const STEPS = 360;

  const bulgeSlider = Inputs.range([-1.5, 1.5], { value: -0.8, step: 0.025, label: "Bulge (−: exc
  const secondSing = Inputs.toggle({ label: "Add second singularity at z = 1.5", value: false });

  function toSx(re) { return CX + re * SCALE; }
  function toSy(im) { return CY - im * SCALE; }

  // Deformable contour C2: base circle of radius 1.8, bulged toward/away from origin
  // bulge > 0 means the contour bows inward toward origin, eventually enclosing it
  function contourC2(theta, bulge) {
    // Offset the radius at each angle: bulge shifts the leftmost point
    // r(theta) = 1.8 + bulge * cos(theta)  (max deformation along Re axis)
    const r = 1.8 + bulge * Math.cos(theta);
    return [r * Math.cos(theta), r * Math.sin(theta)];
  }

  function pointInContour(px, py, bulge) {
    // Ray-casting: count crossings from (px,py) in +x direction
    let crossings = 0;
    for (let k = 0; k < STEPS; k++) {
      const t0 = (2 * Math.PI * k) / STEPS;
      const t1 = (2 * Math.PI * (k + 1)) / STEPS;
      const [x0, y0] = contourC2(t0, bulge);
      const [x1, y1] = contourC2(t1, bulge);
      if ((y0 <= py && y1 > py) || (y1 <= py && y0 > py)) {
        const xIntersect = x0 + ((py - y0) / (y1 - y0)) * (x1 - x0);
        if (px < xIntersect) crossings++;
      }
    }
    return crossings % 2 === 1;
  }

  function computeC2Integral(bulge, singRe, singIm) {
    // ∮ 1/(z - sing) dz numerically
    let rSum = 0, iSum = 0;
    for (let k = 0; k < STEPS; k++) {
```

```
    const t0 = (2 * Math.PI * k) / STEPS;
    const t1 = (2 * Math.PI * (k + 1)) / STEPS;
    const [r0, i0] = contourC2(t0, bulge);
    const [r1, i1] = contourC2(t1, bulge);
    const mr = (r0 + r1) / 2, mi = (i0 + i1) / 2;
    const dr = r1 - r0, di = i1 - i0;
    const denom = (mr - singRe) ** 2 + (mi - singIm) ** 2;
    if (denom < 1e-8) continue;
    const fr = (mr - singRe) / denom, fi = -(mi - singIm) / denom;
    rSum += fr * dr - fi * di;
    iSum += fi * dr + fr * di;
  }
  return [rSum, iSum];
}

function hslToRgb(h, s, l) {
  h = ((h % 1) + 1) % 1;
  const c = (1 - Math.abs(2 * l - 1)) * s;
  const x = c * (1 - Math.abs(((h * 6) % 2) - 1));
  const m = l - c / 2;
  let r = 0, g = 0, b = 0;
  if (h < 1/6) { r = c; g = x; }
  else if (h < 2/6) { r = x; g = c; }
  else if (h < 3/6) { g = c; b = x; }
  else if (h < 4/6) { g = x; b = c; }
  else if (h < 5/6) { r = x; b = c; }
  else { r = c; b = x; }
  return [Math.round((r+m)*255), Math.round((g+m)*255), Math.round((b+m)*255)];
}

function render(bulge, showSecond) {
  // Canvas for domain colouring
  const canvas = document.createElement("canvas");
  canvas.width = W; canvas.height = H;
  const ctx = canvas.getContext("2d");
  const img = ctx.createImageData(W, H);
  const data = img.data;

  for (let px = 0; px < W; px++) {
    for (let py = 0; py < H; py++) {
      const re = (px - CX) / SCALE;
      const im = -(py - CY) / SCALE;
      // Colour by 1/z (primary singularity at origin)
      const d2 = re * re + im * im;
      let hue, brightness;
      if (d2 < 1e-6) {
        const idx = (py * W + px) * 4;
```

```
        data[idx] = 255; data[idx+1] = 255; data[idx+2] = 255; data[idx+3] = 255;
        continue;
      }
      const wr = re / d2, wi = -im / d2;
      const arg = Math.atan2(wi, wr);
      const mag = Math.sqrt(wr*wr + wi*wi);
      hue = (arg / (2 * Math.PI) + 1) % 1;
      brightness = 0.45 + 0.3 * Math.tanh(Math.log(mag + 0.001) / 2);
      const [r, g, b] = hslToRgb(hue, 0.72, brightness);
      const idx = (py * W + px) * 4;
      data[idx] = r; data[idx+1] = g; data[idx+2] = b; data[idx+3] = 255;
    }
  }
  ctx.putImageData(img, 0, 0);

  // Fixed outer circle C1 (orange)
  const [ox, oy] = [toSx(0), toSy(0)];
  ctx.strokeStyle = "#f97316"; ctx.lineWidth = 2.5;
  ctx.beginPath(); ctx.arc(ox, oy, 2.5 * SCALE, 0, 2 * Math.PI); ctx.stroke();
  ctx.fillStyle = "#f97316"; ctx.font = "11px sans-serif";
  ctx.fillText("C\u2081 (fixed)", toSx(2.5) + 4, toSy(0));

  // Deformable contour C2 (blue)
  ctx.strokeStyle = "#3b82f6"; ctx.lineWidth = 2.5;
  ctx.beginPath();
  for (let k = 0; k <= STEPS; k++) {
    const theta = (2 * Math.PI * k) / STEPS;
    const [cr, ci] = contourC2(theta, bulge);
    const sx = toSx(cr), sy = toSy(ci);
    k === 0 ? ctx.moveTo(sx, sy) : ctx.lineTo(sx, sy);
  }
  ctx.closePath(); ctx.stroke();
  ctx.fillStyle = "#3b82f6"; ctx.font = "11px sans-serif";
  ctx.fillText("C\u2082 (deformable)", toSx(-2.4), toSy(1.2));

  // Origin singularity ×
  ctx.strokeStyle = "#fff"; ctx.lineWidth = 2.5;
  ctx.beginPath(); ctx.moveTo(ox-7, oy-7); ctx.lineTo(ox+7, oy+7); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(ox+7, oy-7); ctx.lineTo(ox-7, oy+7); ctx.stroke();
  ctx.fillStyle = "#fff"; ctx.font = "9px sans-serif"; ctx.fillText("1/z", ox+10, oy-5);

  // Second singularity at z=1.5
  if (showSecond) {
    const [sx2, sy2] = [toSx(1.5), toSy(0)];
    ctx.strokeStyle = "#dc2626"; ctx.lineWidth = 2;
    ctx.beginPath(); ctx.moveTo(sx2-6, sy2-6); ctx.lineTo(sx2+6, sy2+6); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(sx2+6, sy2-6); ctx.lineTo(sx2-6, sy2+6); ctx.stroke();
```

```
    ctx.fillStyle = "#dc2626"; ctx.font = "9px sans-serif"; ctx.fillText("1/(z-1.5)", sx2+8, sy2-4
  }

  // Winding number box for C2 around origin
  const originInC2 = pointInContour(0, 0, bulge);
  const winding2 = originInC2 ? 1 : 0;
  ctx.fillStyle = "#1e40af";
  ctx.beginPath();
  ctx.roundRect(W - 92, 5, 86, 44, 5);
  ctx.fill();
  ctx.fillStyle = "white"; ctx.textAlign = "center";
  ctx.font = "10px sans-serif";
  ctx.fillText("n(C\u2082, 0)", W - 49, 21);
  ctx.font = "bold 18px sans-serif";
  ctx.fillText(winding2.toString(), W - 49, 41);
  ctx.textAlign = "left";

  return { canvas, originInC2, winding2 };
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem

const title = document.createElement("div");
title.style.cssText = "font-weight:600; margin-bottom:0.5rem;";
title.textContent = "Contour deformation: deforming C\u2082 past the singularity";
container.appendChild(title);

container.appendChild(bulgeSlider);
container.appendChild(secondSing);

const canvasDiv = document.createElement("div");
container.appendChild(canvasDiv);

const readout = document.createElement("div");
readout.style.cssText = "margin-top:0.5rem; font-size:0.88em; display:grid; grid-template-columns
container.appendChild(readout);

const noteEl = document.createElement("div");
noteEl.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
noteEl.textContent = "Drag the bulge slider left to exclude the origin (integral = 0) and right to
container.appendChild(noteEl);

function update() {
  const bulge = bulgeSlider.value;
  const showSecond = secondSing.value;
  const { canvas, originInC2 } = render(bulge, showSecond);
```

```
    canvasDiv.innerHTML = ""; canvasDiv.appendChild(canvas);

    // Compute integrals
    const [c2Re, c2Im] = computeC2Integral(bulge, 0, 0);
    const c1Im = 2 * Math.PI; // always encloses origin
    let html = `<div style="background:#fff7ed; border:1px solid #fed7aa; border-radius:4px; padd
      <span style="color:#f97316; font-weight:600;">C\u2081:</span>
      \u222e 1/z dz = 0 + ${c1Im.toFixed(4)}i = 2\u03c0i</div>`;
    const c2Status = originInC2
      ? `<span style="color:#059669; font-weight:600;">C\u2082 encloses origin \u2192 2\u03c0i</s
      : `<span style="color:#6b7280; font-weight:600;">C\u2082 excludes origin \u2192 0</span>`;
    html += `<div style="background:#eff6ff; border:1px solid #bfdbfe; border-radius:4px; padding
      <span style="color:#3b82f6; font-weight:600;">C\u2082:</span>
      \u222e 1/z dz \u2248 ${c2Re.toFixed(3)} + ${c2Im.toFixed(3)}i   ${c2Status}</div>`;

    if (showSecond) {
      const [s2Re, s2Im] = computeC2Integral(bulge, 1.5, 0);
      html += `<div style="background:#fef2f2; border:1px solid #fecaca; border-radius:4px; paddi
        <span style="color:#dc2626; font-weight:600;">C\u2082 around 1/(z-1.5):</span>
        \u222e 1/(z-1.5) dz \u2248 ${s2Re.toFixed(3)} + ${s2Im.toFixed(3)}i
          ${pointInContour(1.5, 0, bulge)
          ? '<span style="color:#059669; font-weight:600;">enclosed \u2192 2\u03c0i</span>'
          : '<span style="color:#6b7280; font-weight:600;">not enclosed \u2192 0</span>'}</div>`;
    }
    readout.innerHTML = html;
  }
  bulgeSlider.addEventListener("input", update);
  secondSing.addEventListener("input", update);
  update();

  return container;
}
```

---

## 48.3. Cauchy's integral formula

**Cauchy's integral formula.** Let $f$ be analytic in and on a simple closed contour $C$ (traversed counter-clockwise). Then for any $z_0$ interior to $C$:

$$f(z_0) = \frac{1}{2\pi i} \oint_C \frac{f(z)}{z - z_0}\, dz$$

Equivalently:

$$\oint_C \frac{f(z)}{z - z_0}\, dz = 2\pi i\, f(z_0)$$

*Proof sketch.* Because $f$ is analytic everywhere inside $C$ except at $z_0$, the deformation principle applies to the annular region between $C$ and any circle around $z_0$: the integrand $f(z)/(z - z_0)$ is analytic there. Shrink $C$ to a small circle $C_\varepsilon$ of radius $\varepsilon$ around $z_0$ without changing the integral. As $\varepsilon \to 0$, $f(z) \to f(z_0)$ uniformly on $C_\varepsilon$, and $\oint_{C_\varepsilon} dz/(z - z_0) = 2\pi i$.

**Significance.** The value of an analytic function at any interior point is completely determined by its values on the boundary. This has no analogue in real analysis.

### 48.3.1. Derivatives of all orders

Differentiating the integral formula with respect to $z_0$ under the integral sign (valid because $f$ is analytic and the denominator never vanishes on $C$):

$$f^{(n)}(z_0) = \frac{n!}{2\pi i} \oint_C \frac{f(z)}{(z - z_0)^{n+1}}\, dz$$

Rearranged to evaluate a given integral: $\displaystyle\oint_C \frac{f(z)}{(z - z_0)^{n+1}}\, dz = \frac{2\pi i}{n!}\, f^{(n)}(z_0)$.

**Consequence.** An analytic function has derivatives of all orders, all of which are themselves analytic. In real analysis, once-differentiable functions need not be twice differentiable. In complex analysis, once differentiable means infinitely differentiable — a qualitative difference.

---

## 48.4. Major consequences

### 48.4.1. Liouville's theorem

A **bounded** entire function is constant. (An *entire* function is one that is analytic everywhere in $\mathbb{C}$ — no singularities at all.)

*Proof.* If $|f(z)| \le M$ everywhere, the ML inequality applied to the derivative formula gives $|f'(z_0)| \le M/R$ for any circle of radius $R$. Letting $R \to \infty$ gives $f'(z_0) = 0$.

```
//| echo: false
// VIZ 7: Liouville's theorem — collapsing bound
// Growing circle of radius R; M/R curve collapsing toward zero;
// candidate |f'| line contradicted as R grows.
{
  const W = 400, H = 220;
  const svgNS = "http://www.w3.org/2000/svg";
```

```
const RSlider = Inputs.range([0.1, 30], { value: 1, step: 0.1, label: "Radius R" });
const MSlider = Inputs.range([0.1, 10], { value: 2, step: 0.1, label: "Bound M (|f| \u2264 M)"
const fpSlider = Inputs.range([0, 3], { value: 0.5, step: 0.01, label: "Candidate |f\u2032(z\u2

function build(R, M, fp) {
  const PLOTW = W - 60, PLOTH = H - 40;
  const PX = 45, PY = 10;
  const RMax = 30;

  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", W); svg.setAttribute("height", H);
  svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display

  // Axes
  const axH = document.createElementNS(svgNS, "line");
  axH.setAttribute("x1", PX); axH.setAttribute("y1", PY + PLOTH);
  axH.setAttribute("x2", PX + PLOTW); axH.setAttribute("y2", PY + PLOTH);
  axH.setAttribute("stroke", "#9ca3af"); axH.setAttribute("stroke-width", "1.5");
  svg.appendChild(axH);
  const axV = document.createElementNS(svgNS, "line");
  axV.setAttribute("x1", PX); axV.setAttribute("y1", PY);
  axV.setAttribute("x2", PX); axV.setAttribute("y2", PY + PLOTH);
  axV.setAttribute("stroke", "#9ca3af"); axV.setAttribute("stroke-width", "1.5");
  svg.appendChild(axV);

  // Axis labels
  const xlbl = document.createElementNS(svgNS, "text");
  xlbl.setAttribute("x", PX + PLOTW - 4); xlbl.setAttribute("y", PY + PLOTH + 14);
  xlbl.setAttribute("font-size", "10"); xlbl.setAttribute("fill", "#6b7280");
  xlbl.setAttribute("font-family", "sans-serif"); xlbl.textContent = "R";
  svg.appendChild(xlbl);
  const ylbl = document.createElementNS(svgNS, "text");
  ylbl.setAttribute("x", 2); ylbl.setAttribute("y", PY + 6);
  ylbl.setAttribute("font-size", "9"); ylbl.setAttribute("fill", "#6b7280");
  ylbl.setAttribute("font-family", "sans-serif"); ylbl.textContent = "M/R";
  svg.appendChild(ylbl);

  function toX(r) { return PX + (r / RMax) * PLOTW; }
  function toY(v) {
    const Mmax = M * 10;
    return PY + PLOTH - Math.min(PLOTH * 0.95, (v / Mmax) * PLOTH);
  }

  // M/R decay curve
  let curvD = "";
  for (let k = 1; k <= 200; k++) {
    const r = (RMax * k) / 200;
```

```
    curvD += (k === 1 ? "M" : "L") + ` ${toX(r)} ${toY(M / r)}`;
  }
  const curvEl = document.createElementNS(svgNS, "path");
  curvEl.setAttribute("d", curvD); curvEl.setAttribute("fill", "none");
  curvEl.setAttribute("stroke", "#3b82f6"); curvEl.setAttribute("stroke-width", "2");
  svg.appendChild(curvEl);

  // Horizontal line for candidate |f'(z₀)|
  const fpY = toY(fp);
  const fpLine = document.createElementNS(svgNS, "line");
  fpLine.setAttribute("x1", PX); fpLine.setAttribute("y1", fpY);
  fpLine.setAttribute("x2", PX + PLOTW); fpLine.setAttribute("y2", fpY);
  fpLine.setAttribute("stroke", "#f97316"); fpLine.setAttribute("stroke-width", "1.5");
  fpLine.setAttribute("stroke-dasharray", "5,3");
  svg.appendChild(fpLine);
  const fpLbl = document.createElementNS(svgNS, "text");
  fpLbl.setAttribute("x", PX + PLOTW + 2); fpLbl.setAttribute("y", fpY + 4);
  fpLbl.setAttribute("font-size", "9"); fpLbl.setAttribute("fill", "#f97316");
  fpLbl.setAttribute("font-family", "sans-serif"); fpLbl.textContent = "|f\u2032| = " + fp.toFixed
  svg.appendChild(fpLbl);

  // Current R marker on curve
  const curY = toY(M / R);
  const curDot = document.createElementNS(svgNS, "circle");
  curDot.setAttribute("cx", toX(R)); curDot.setAttribute("cy", curY);
  curDot.setAttribute("r", "5"); curDot.setAttribute("fill", "#3b82f6");
  curDot.setAttribute("stroke", "#fff"); curDot.setAttribute("stroke-width", "1.5");
  svg.appendChild(curDot);

  // Bound value label
  const boundVal = M / R;
  const boundLbl = document.createElementNS(svgNS, "text");
  boundLbl.setAttribute("x", toX(R) + 6); boundLbl.setAttribute("y", curY - 4);
  boundLbl.setAttribute("font-size", "9"); boundLbl.setAttribute("fill", "#3b82f6");
  boundLbl.setAttribute("font-family", "sans-serif");
  boundLbl.textContent = "M/R = " + boundVal.toFixed(3);
  svg.appendChild(boundLbl);

  // Contradiction indicator
  const contradicted = fp > M / R;
  const annot = document.createElementNS(svgNS, "text");
  annot.setAttribute("x", W / 2); annot.setAttribute("y", H - 6);
  annot.setAttribute("font-size", "10"); annot.setAttribute("text-anchor", "middle");
  annot.setAttribute("font-weight", "bold"); annot.setAttribute("font-family", "sans-serif");
  annot.setAttribute("fill", contradicted ? "#dc2626" : "#059669");
  annot.textContent = contradicted
    ? "|f\u2032| > M/R \u2014 impossible! As R\u2192\u221e, bound \u21920, so |f\u2032| must be 0
```

```
      : "|f\u2032| \u2264 M/R \u2014 consistent (increase R to force contradiction)";
    svg.appendChild(annot);

    return svg;
  }

  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r

  const titleEl = document.createElement("div");
  titleEl.style.cssText = "font-weight:600; margin-bottom:0.5rem;";
  titleEl.textContent = "Liouville\u2019s theorem: the M/R bound collapses to zero";
  container.appendChild(titleEl);
  container.appendChild(RSlider);
  container.appendChild(MSlider);
  container.appendChild(fpSlider);

  const vizDiv = document.createElement("div");
  vizDiv.style.marginTop = "0.5rem";
  container.appendChild(vizDiv);

  const noteEl = document.createElement("div");
  noteEl.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;"
  noteEl.textContent = "The blue curve M/R falls toward zero as R grows. Any non-zero candidate |
  container.appendChild(noteEl);

  function update() {
    vizDiv.innerHTML = "";
    vizDiv.appendChild(build(RSlider.value, MSlider.value, fpSlider.value));
  }
  RSlider.addEventListener("input", update);
  MSlider.addEventListener("input", update);
  fpSlider.addEventListener("input", update);
  update();

  return container;
}
```

### 48.4.2. Fundamental theorem of algebra

Every non-constant polynomial $p(z)$ of degree $n \geq 1$ has exactly $n$ complex roots (counting multiplicity).

*Proof sketch.* If $p$ had no roots, $1/p(z)$ would be entire (no zeros means no poles) and bounded (since $|p(z)| \to \infty$ as $|z| \to \infty$, so $|1/p(z)| \to 0$), contradicting Liouville's theorem.

### 48.4.3. Morera's theorem

If $f$ is continuous on a domain $D$ and $\oint_C f dz = 0$ for every closed triangle $C$ in $D$, then $f$ is analytic in $D$. (The converse of Cauchy's theorem.)

---

## 48.5. Application: real integrals via contour integration

The general strategy — closing a real integral with a contour in the complex plane and applying Cauchy's theorem — is developed fully in the next chapter. This example shows the technique in its simplest form.

**Example.** $\displaystyle\int_{-\infty}^{\infty} \frac{dx}{1+x^2}$.

The integrand $f(z) = 1/(1+z^2)$ has poles at $z = \pm i$. Close the real-line integral with a large semicircle $\Gamma_R$ of radius $R$ in the upper half-plane, forming a closed contour $C = [-R, R] + \Gamma_R$.

**The semicircle contribution vanishes.** On $\Gamma_R$, $|z| = R$, so $|1+z^2| \geq R^2 - 1$ for large $R$. The ML inequality gives:

$$\left|\int_{\Gamma_R} \frac{dz}{1+z^2}\right| \leq \frac{1}{R^2-1} \cdot \pi R \to 0 \quad \text{as } R \to \infty$$

**Applying Cauchy's integral formula.** Factor the denominator: $1/(1+z^2) = [1/(z+i)]/(z-i)$. This has the form $f(z)/(z-z_0)$ with $f(z) = 1/(z+i)$ and $z_0 = i$. Since $z_0 = i$ lies inside the upper half-plane contour and $f(z) = 1/(z+i)$ is analytic there:

$$\oint_C \frac{dz}{1+z^2} = \oint_C \frac{dz}{(z-i)(z+i)} = 2\pi i \cdot f(i) = 2\pi i \cdot \frac{1}{2i} = \pi$$

Since the semicircle contributes nothing, the real-line integral equals $\pi$ — confirming the known result $\arctan(x)\big|_{-\infty}^{\infty} = \pi$.

> **Note**
>
> This calculation uses Cauchy's integral formula because the integrand has a single simple pole and a purely analytic numerator. In engineering practice, the same closing-contour technique is applied via the **residue theorem** (Chapter 3), which handles poles of any order and multiple singularities simultaneously. The present example is the simplest special case of that general method.

```
//| echo: false
// VIZ 3: Cauchy integral formula — draggable pole
// Domain-coloured canvas; drag z₀ freely; smooth crossing triggers integral jump;
// KaTeX formula display; "pole on contour" warning; winding number box.
{
  const W = 360, H = 320, SCALE = 72, CX = W / 2, CY = H / 2;
  const STEPS = 360;
```

```
const radiusCIF = Inputs.range([0.5, 2.8], { value: 1.5, step: 0.05, label: "Contour radius R"
const funcCIF = Inputs.select(
  ["f(z) = e^z", "f(z) = cos z", "f(z) = 1/(z+2)"],
  { value: "f(z) = e^z", label: "f(z) (numerator)" }
);

// Mutable pole position — shared via a plain object so canvas listeners can mutate it
const pole = { re: 0.6, im: 0.4 };

function evalFCIF(fname, re, im) {
  if (fname === "f(z) = e^z") return [Math.exp(re) * Math.cos(im), Math.exp(re) * Math.sin(im)]
  if (fname === "f(z) = cos z") return [Math.cos(re) * Math.cosh(im), -Math.sin(re) * Math.sinh
  const dr = re + 2, di = im, d = dr * dr + di * di;
  if (d < 1e-10) return [0, 0];
  return [dr / d, -di / d];
}

function hslRgbCIF(h, s, l) {
  h = ((h % 1) + 1) % 1;
  const c = (1 - Math.abs(2 * l - 1)) * s;
  const x = c * (1 - Math.abs(((h * 6) % 2) - 1));
  const m = l - c / 2;
  let r = 0, g = 0, b = 0;
  if (h < 1/6) { r = c; g = x; }
  else if (h < 2/6) { r = x; g = c; }
  else if (h < 3/6) { g = c; b = x; }
  else if (h < 4/6) { g = x; b = c; }
  else if (h < 5/6) { r = x; b = c; }
  else { r = c; b = x; }
  return [Math.round((r+m)*255), Math.round((g+m)*255), Math.round((b+m)*255)];
}

function toSxCIF(re) { return CX + re * SCALE; }
function toSyCIF(im) { return CY - im * SCALE; }
function fromSxCIF(px) { return (px - CX) / SCALE; }
function fromSyCIF(py) { return -(py - CY) / SCALE; }

function drawCIF(canvas, R, fname) {
  const ctx = canvas.getContext("2d");
  const img = ctx.createImageData(W, H);
  const data = img.data;
  const z0re = pole.re, z0im = pole.im;

  for (let px = 0; px < W; px++) {
    for (let py = 0; py < H; py++) {
      const re = fromSxCIF(px), im = fromSyCIF(py);
      const dr = re - z0re, di = im - z0im;
```

```
        const d2 = dr * dr + di * di;
        if (d2 < 4e-4) {
          const idx = (py * W + px) * 4;
          data[idx] = 255; data[idx+1] = 255; data[idx+2] = 255; data[idx+3] = 255;
          continue;
        }
        const wr = dr / d2, wi = -di / d2;
        const arg = Math.atan2(wi, wr);
        const mag = Math.sqrt(wr*wr + wi*wi);
        const hue = (arg / (2 * Math.PI) + 1) % 1;
        const brightness = 0.48 + 0.32 * Math.tanh(Math.log(mag + 0.001) / 2);
        const [r, g, b] = hslRgbCIF(hue, 0.75, brightness);
        const idx = (py * W + px) * 4;
        data[idx] = r; data[idx+1] = g; data[idx+2] = b; data[idx+3] = 255;
      }
    }
    ctx.putImageData(img, 0, 0);

    const dist = Math.sqrt(z0re * z0re + z0im * z0im);
    const onContour = Math.abs(dist - R) < 0.12;
    const inside = dist < R && !onContour;

    // Contour circle
    const ox = toSxCIF(0), oy = toSyCIF(0);
    ctx.strokeStyle = onContour ? "#dc2626" : "#ffffff";
    ctx.lineWidth = 2.5;
    ctx.beginPath(); ctx.arc(ox, oy, R * SCALE, 0, 2 * Math.PI); ctx.stroke();
    ctx.fillStyle = "rgba(255,255,255,0.9)"; ctx.font = "11px sans-serif";
    ctx.fillText("C: |z| = " + R.toFixed(2), 6, 16);

    // Pole marker ×
    const px0 = toSxCIF(z0re), py0 = toSyCIF(z0im);
    ctx.strokeStyle = "#ffffff"; ctx.lineWidth = 2.5;
    ctx.beginPath(); ctx.moveTo(px0-8, py0-8); ctx.lineTo(px0+8, py0+8); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(px0+8, py0-8); ctx.lineTo(px0-8, py0+8); ctx.stroke();
    ctx.fillStyle = "rgba(255,255,255,0.9)"; ctx.font = "11px sans-serif";
    ctx.fillText("z\u2080 = " + z0re.toFixed(2) + (z0im >= 0 ? "+" : "") + z0im.toFixed(2) + "i", px

    // Winding number box
    const winding = inside ? 1 : 0;
    ctx.fillStyle = "#1e40af";
    ctx.beginPath(); ctx.roundRect(W - 92, 5, 86, 44, 5); ctx.fill();
    ctx.fillStyle = "white"; ctx.textAlign = "center";
    ctx.font = "10px sans-serif"; ctx.fillText("n(C, z\u2080)", W - 49, 21);
    ctx.font = "bold 18px sans-serif"; ctx.fillText(winding.toString(), W - 49, 41);
    ctx.textAlign = "left";
```

```
  return { inside, onContour };
}

// Build the container with canvas + info panel
const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r

const titleEl = document.createElement("div");
titleEl.style.cssText = "font-weight:600; margin-bottom:0.5rem;";
titleEl.textContent = "Cauchy integral formula: drag the pole z\u2080";
container.appendChild(titleEl);

container.appendChild(radiusCIF);
container.appendChild(funcCIF);

const canvas = document.createElement("canvas");
canvas.width = W; canvas.height = H;
canvas.style.cssText = "display:block; cursor:crosshair; border-radius:6px; margin-top:0.5rem;"
container.appendChild(canvas);

const infoDiv = document.createElement("div");
infoDiv.style.cssText = "margin-top:0.5rem; font-size:0.88em;";
container.appendChild(infoDiv);

const noteEl = document.createElement("div");
noteEl.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;"
noteEl.textContent = "Click or drag anywhere on the canvas to move z\u2080. The domain colourin
container.appendChild(noteEl);

function updateCIF() {
  const R = radiusCIF.value;
  const fname = funcCIF.value;
  const { inside, onContour } = drawCIF(canvas, R, fname);
  const z0re = pole.re, z0im = pole.im;

  if (onContour) {
    infoDiv.innerHTML = `<div style="color:#dc2626; font-weight:600; padding:0.4rem 0.6rem; bac
      Pole on contour C \u2014 integral is undefined (integrand blows up on the path)</div>`;
  } else if (inside) {
    const [fre, fim] = evalFCIF(fname, z0re, z0im);
    const resRe = (-2 * Math.PI * fim).toFixed(4);
    const resIm = (2 * Math.PI * fre).toFixed(4);
    infoDiv.innerHTML = `<div style="padding:0.4rem 0.6rem; background:#f0fdf4; border:1px soli
      <span style="color:#059669; font-weight:600;">z\u2080 inside C \u2014 Cauchy formula appl
      f(z\u2080) = ${fre.toFixed(4)} + ${fim.toFixed(4)}i<br>
      <strong>\u222e f(z)/(z\u2212z\u2080) dz = 2\u03c0i \u00b7 f(z\u2080) = ${resRe} + ${resIm
  } else {
```

```
      infoDiv.innerHTML = `<div style="padding:0.4rem 0.6rem; background:#f9fafb; border:1px solid #
        <span style="color:#6b7280; font-weight:600;">z\u2080 outside C \u2014 integrand is analytic
        Cauchy\u2013Goursat theorem: <strong>\u222e f(z)/(z\u2212z\u2080) dz = 0</strong></div>`;
    }
  }

  // Drag handling on canvas
  let dragging = false;
  function movePole(evt) {
    const rect = canvas.getBoundingClientRect();
    const scaleX = W / rect.width, scaleY = H / rect.height;
    const cpx = (evt.clientX - rect.left) * scaleX;
    const cpy = (evt.clientY - rect.top) * scaleY;
    pole.re = Math.max(-3.5, Math.min(3.5, fromSxCIF(cpx)));
    pole.im = Math.max(-3.5, Math.min(3.5, fromSyCIF(cpy)));
    updateCIF();
  }
  canvas.addEventListener("mousedown", (e) => { dragging = true; movePole(e); });
  canvas.addEventListener("mousemove", (e) => { if (dragging) movePole(e); });
  canvas.addEventListener("mouseup", () => { dragging = false; });
  canvas.addEventListener("mouseleave", () => { dragging = false; });
  canvas.addEventListener("touchstart", (e) => { e.preventDefault(); dragging = true; movePole(e.tou
  canvas.addEventListener("touchmove", (e) => { e.preventDefault(); if (dragging) movePole(e.touches
  canvas.addEventListener("touchend", () => { dragging = false; });

  radiusCIF.addEventListener("input", updateCIF);
  funcCIF.addEventListener("input", updateCIF);
  updateCIF();

  return container;
}
```

```
//| echo: false
{
  const W = 420, H = 380, CX = 190, CY = 185, SCALE = 65;
  const svgNS = "http://www.w3.org/2000/svg";

  const pathSelector2 = Inputs.select([
    "Circle around origin (n=1)",
    "Circle not around origin (n=0)",
    "Two loops around origin (n=2)"
  ], {value: "Circle around origin (n=1)", label: "Path"});

  const tSlider2 = Inputs.range([0, 1], {value: 0, step: 0.005, label: "Progress t"});

  const pathDefs = {
    "Circle around origin (n=1)": {
```

```
    pts: (t) => [1.5*Math.cos(2*Math.PI*t), 1.5*Math.sin(2*Math.PI*t)],
    nWinding: 1
  },
  "Circle not around origin (n=0)": {
    pts: (t) => [1.8 + 0.8*Math.cos(2*Math.PI*t), 0.8*Math.sin(2*Math.PI*t)],
    nWinding: 0
  },
  "Two loops around origin (n=2)": {
    pts: (t) => [1.5*Math.cos(4*Math.PI*t), 1.5*Math.sin(4*Math.PI*t)],
    nWinding: 2
  }
};

function getPathPoint(def, t) { return def.pts(t); }

function computeIntegral(def, upToT, STEPS) {
  // Numerically integrate dz/z from 0 to upToT
  // Result is a complex number (Re, Im)
  let sumRe = 0, sumIm = 0;
  const steps = Math.round(upToT * STEPS);
  const dt = upToT / Math.max(steps, 1);

  for (let s=0; s<steps; s++) {
    const t = s / STEPS;
    const tn = (s+1) / STEPS;
    const [re1, im1] = def.pts(t);
    const [re2, im2] = def.pts(tn < 1 ? tn : 1);
    // midpoint
    const mre = (re1+re2)/2, mim = (im1+im2)/2;
    const dre = re2-re1, dim = im2-im1;
    const dz_over_z_re = (mre*dre + mim*dim)/(mre*mre+mim*mim);
    const dz_over_z_im = (mim*dre - mre*dim)/(mre*mre+mim*mim);
    sumRe += dz_over_z_re;
    sumIm += dz_over_z_im;
  }
  return [sumRe, sumIm];
}

function toScreen(re, im) { return [CX + re*SCALE, CY - im*SCALE]; }

function render2(pathName, tVal) {
  const def = pathDefs[pathName];
  const STEPS = 400;

  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", W); svg.setAttribute("height", H);
  svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display
```

```
// Grid + axes
const gridG = document.createElementNS(svgNS,"g");
gridG.setAttribute("stroke","#e5e7eb"); gridG.setAttribute("stroke-width","0.5");
for (let i=-3; i<=3; i++) {
  const xl=document.createElementNS(svgNS,"line"); const [lx]=toScreen(i,0); const [,ly1]=toScre
  xl.setAttribute("x1",lx); xl.setAttribute("y1",ly1); xl.setAttribute("x2",lx); xl.setAttribute
  const yl=document.createElementNS(svgNS,"line"); const [lx1]=toScreen(-3,0); const [lx2]=toSc
  yl.setAttribute("x1",lx1); yl.setAttribute("y1",ly); yl.setAttribute("x2",lx2); yl.setAttribut
}
svg.appendChild(gridG);

const [ox,oy]=toScreen(0,0);
const axG=document.createElementNS(svgNS,"g"); axG.setAttribute("stroke","#9ca3af"); axG.setAtt
const xa=document.createElementNS(svgNS,"line"); const [ax1]=toScreen(-2.8,0); const [ax2]=toSc
xa.setAttribute("x1",ax1); xa.setAttribute("y1",oy); xa.setAttribute("x2",ax2); xa.setAttribute
const ya=document.createElementNS(svgNS,"line"); const [,ay1]=toScreen(0,-2.5); const [,ay2]=to
ya.setAttribute("x1",ox); ya.setAttribute("y1",ay1); ya.setAttribute("x2",ox); ya.setAttribute("
axG.appendChild(xa); axG.appendChild(ya); svg.appendChild(axG);

// Full path outline
let pathD = "";
for (let s=0; s<=STEPS; s++) {
  const t = s/STEPS;
  const [re,im] = def.pts(t);
  const [px,py] = toScreen(re,im);
  pathD += (s===0 ? `M ${px} ${py}` : ` L ${px} ${py}`);
}
const pathOutline = document.createElementNS(svgNS,"path");
pathOutline.setAttribute("d",pathD); pathOutline.setAttribute("fill","none");
pathOutline.setAttribute("stroke","rgba(249,115,22,0.25)"); pathOutline.setAttribute("stroke-wi
svg.appendChild(pathOutline);

// Traced portion
let traceD = "";
const traceSteps = Math.round(tVal * STEPS);
for (let s=0; s<=traceSteps; s++) {
  const t = s/STEPS;
  const [re,im] = def.pts(t);
  const [px,py] = toScreen(re,im);
  traceD += (s===0 ? `M ${px} ${py}` : ` L ${px} ${py}`);
}
const traceEl = document.createElementNS(svgNS,"path");
traceEl.setAttribute("d",traceD); traceEl.setAttribute("fill","none");
traceEl.setAttribute("stroke","#f97316"); traceEl.setAttribute("stroke-width","2.5");
svg.appendChild(traceEl);

// Current point
```

```
const [cre,cim] = def.pts(tVal);
const [cpx,cpy] = toScreen(cre,cim);
const ptEl = document.createElementNS(svgNS,"circle");
ptEl.setAttribute("cx",cpx); ptEl.setAttribute("cy",cpy); ptEl.setAttribute("r","6");
ptEl.setAttribute("fill","#3b82f6"); ptEl.setAttribute("stroke","#fff"); ptEl.setAttribute("s
svg.appendChild(ptEl);

// Origin × mark (the singularity 1/z)
const xSize = 5;
const x1m=document.createElementNS(svgNS,"line"); x1m.setAttribute("x1",ox-xSize); x1m.setAtt
x1m.setAttribute("x2",ox+xSize); x1m.setAttribute("y2",oy+xSize);
x1m.setAttribute("stroke","#ef4444"); x1m.setAttribute("stroke-width","2.5"); svg.appendChild
const x2m=document.createElementNS(svgNS,"line"); x2m.setAttribute("x1",ox+xSize); x2m.setAtt
x2m.setAttribute("x2",ox-xSize); x2m.setAttribute("y2",oy+xSize);
x2m.setAttribute("stroke","#ef4444"); x2m.setAttribute("stroke-width","2.5"); svg.appendChild
const origLbl=document.createElementNS(svgNS,"text"); origLbl.setAttribute("x",ox+8); origLbl
origLbl.setAttribute("fill","#ef4444"); origLbl.setAttribute("font-size","10"); origLbl.setAt
origLbl.textContent="singularity of 1/z"; svg.appendChild(origLbl);

// Winding box
const windBox = document.createElementNS(svgNS,"rect"); windBox.setAttribute("x",W-92); windB
windBox.setAttribute("width",85); windBox.setAttribute("height",44); windBox.setAttribute("fi
svg.appendChild(windBox);
const wl=document.createElementNS(svgNS,"text"); wl.setAttribute("x",W-49); wl.setAttribute("
wl.setAttribute("fill","white"); wl.setAttribute("font-size","10"); wl.setAttribute("text-anc
wl.textContent="winding n"; svg.appendChild(wl);
const wn=document.createElementNS(svgNS,"text"); wn.setAttribute("x",W-49); wn.setAttribute("
wn.setAttribute("fill","white"); wn.setAttribute("font-size","18"); wn.setAttribute("text-anc
wn.setAttribute("font-family","sans-serif"); wn.setAttribute("font-weight","bold");
// Running winding estimate: Im part of integral / 2π, rounded to 1 dp
const [intReW, intImW] = computeIntegral(def, tVal, STEPS);
const windEst = tVal > 0.02 ? (intImW / (2 * Math.PI)).toFixed(1) : "0.0";
wn.textContent = tVal > 0.98 ? def.nWinding.toString() : windEst;
svg.appendChild(wn);

// Integral running display — always live from start
const [intRe, intIm] = [intReW, intImW];
const finalIntIm = 2*Math.PI*def.nWinding;

const intDiv = document.createElement("div");
intDiv.style.cssText = "margin-top:0.5rem; font-family:sans-serif; font-size:0.88em; padding:
// Show running value at all t; highlight final result when closed
const runStr = `${intRe.toFixed(3)} + ${intIm.toFixed(3)}i`;
const finalStr = tVal > 0.98
  ? ` \u2192 <strong>2\u03c0i \u00d7 ${def.nWinding} = ${finalIntIm.toFixed(3)}i</strong>` :
intDiv.innerHTML = `<strong>Running \u222b dz/z:</strong> <span style="color:#1e40af">${runSt
```

```
    const wrap = document.createElement("div");
    wrap.appendChild(svg);
    wrap.appendChild(intDiv);
    return wrap;
  }

  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem
  const title = document.createElement("div");
  title.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  title.textContent = "Winding number = contour integral: \u222e dz/z = 2\u03c0i \xd7 n";
  container.appendChild(title);
  container.appendChild(pathSelector2);
  container.appendChild(tSlider2);

  const vizDiv2 = document.createElement("div");
  container.appendChild(vizDiv2);

  const note = document.createElement("div");
  note.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  note.textContent = "Move the slider to trace the path. When the loop closes: \u222e dz/z = 2\u03c0
  container.appendChild(note);

  function update2() {
    vizDiv2.innerHTML = "";
    vizDiv2.appendChild(render2(pathSelector2.value, tSlider2.value));
  }
  pathSelector2.addEventListener("input", update2);
  tSlider2.addEventListener("input", update2);
  update2();

  return container;
}
```

---

```
//| echo: false
// VIZ 8: Semicircle contour closure for 1/(1+x²)
// Real axis + growing semicircle; poles as × marks; residue sum updating;
// ML arc-bound overlay; result box for ∫ 1/(1+x²) dx = π.
{
  const W = 380, H = 280, SCALE = 55, OX = W / 2, OY = H - 50;
  const svgNS = "http://www.w3.org/2000/svg";
  const STEPS = 200;

  const RscSlider = Inputs.range([0.5, 4.5], { value: 1.8, step: 0.05, label: "Semicircle radius R"
```

```
function toSx(re) { return OX + re * SCALE; }
function toSy(im) { return OY - im * SCALE; }

function buildSC(R) {
  const svg = document.createElementNS(svgNS, "svg");
  svg.setAttribute("width", W); svg.setAttribute("height", H);
  svg.style.cssText = "background:#fafafa; border:1px solid #e5e7eb; border-radius:6px; display

  // Real axis
  const ax = document.createElementNS(svgNS, "line");
  ax.setAttribute("x1", toSx(-4.2)); ax.setAttribute("y1", toSy(0));
  ax.setAttribute("x2", toSx(4.2)); ax.setAttribute("y2", toSy(0));
  ax.setAttribute("stroke", "#9ca3af"); ax.setAttribute("stroke-width", "1.5");
  svg.appendChild(ax);
  const ay = document.createElementNS(svgNS, "line");
  ay.setAttribute("x1", toSx(0)); ay.setAttribute("y1", toSy(-0.3));
  ay.setAttribute("x2", toSx(0)); ay.setAttribute("y2", toSy(3.8));
  ay.setAttribute("stroke", "#9ca3af"); ay.setAttribute("stroke-width", "1.5");
  svg.appendChild(ay);

  // Axis labels
  for (const v of [-3,-2,-1,1,2,3]) {
    const tl = document.createElementNS(svgNS, "text");
    tl.setAttribute("x", toSx(v) - 4); tl.setAttribute("y", toSy(0) + 14);
    tl.setAttribute("font-size", "9"); tl.setAttribute("fill", "#9ca3af");
    tl.setAttribute("font-family", "sans-serif"); tl.textContent = v;
    svg.appendChild(tl);
  }
  const relbl = document.createElementNS(svgNS, "text");
  relbl.setAttribute("x", toSx(4.0)); relbl.setAttribute("y", toSy(0) - 4);
  relbl.setAttribute("font-size", "10"); relbl.setAttribute("fill", "#6b7280");
  relbl.setAttribute("font-family", "sans-serif"); relbl.textContent = "x";
  svg.appendChild(relbl);
  const imlbl = document.createElementNS(svgNS, "text");
  imlbl.setAttribute("x", toSx(0) + 4); imlbl.setAttribute("y", toSy(3.6));
  imlbl.setAttribute("font-size", "10"); imlbl.setAttribute("fill", "#6b7280");
  imlbl.setAttribute("font-family", "sans-serif"); imlbl.textContent = "iy";
  svg.appendChild(imlbl);

  // Real-axis segment [-R, R] (orange)
  const seg = document.createElementNS(svgNS, "line");
  seg.setAttribute("x1", toSx(-R)); seg.setAttribute("y1", toSy(0));
  seg.setAttribute("x2", toSx(R)); seg.setAttribute("y2", toSy(0));
  seg.setAttribute("stroke", "#f97316"); seg.setAttribute("stroke-width", "3");
  svg.appendChild(seg);
  // Arrowhead at +R
  const arrowX = toSx(R), arrowY = toSy(0);
```

```
const arrowM = document.createElementNS(svgNS, "polygon");
arrowM.setAttribute("points", `${arrowX-8},${arrowY-4} ${arrowX},${arrowY} ${arrowX-8},${arrowY-
arrowM.setAttribute("fill", "#f97316");
svg.appendChild(arrowM);

// Semicircle arc (blue), upper half-plane
let arcD = `M ${toSx(R)} ${toSy(0)}`;
for (let k = 1; k <= STEPS; k++) {
  const theta = (Math.PI * k) / STEPS;
  arcD += ` L ${toSx(R * Math.cos(theta))} ${toSy(R * Math.sin(theta))}`;
}
const arcEl = document.createElementNS(svgNS, "path");
arcEl.setAttribute("d", arcD); arcEl.setAttribute("fill", "none");
arcEl.setAttribute("stroke", "#3b82f6"); arcEl.setAttribute("stroke-width", "2.5");
svg.appendChild(arcEl);
// Arc arrow at top
const topX = toSx(0), topY = toSy(R);
const arcArrow = document.createElementNS(svgNS, "polygon");
arcArrow.setAttribute("points", `${topX-4},${topY+8} ${topX},${topY} ${topX+4},${topY+8}`);
arcArrow.setAttribute("fill", "#3b82f6");
svg.appendChild(arcArrow);

// Poles of 1/(1+z²) = 1/((z-i)(z+i)): at z = i and z = -i
const poles = [{ re: 0, im: 1, label: "z = i" }, { re: 0, im: -1, label: "z = \u2212i" }];
for (const p of poles) {
  const px0 = toSx(p.re), py0 = toSy(p.im);
  const inContour = p.im > 0 && p.im < R; // upper half-plane, inside semicircle
  const xColor = inContour ? "#dc2626" : "#9ca3af";
  for (const [dx, dy] of [[6,6],[6,-6]]) {
    const xl = document.createElementNS(svgNS, "line");
    xl.setAttribute("x1", px0-dx); xl.setAttribute("y1", py0-dy);
    xl.setAttribute("x2", px0+dx); xl.setAttribute("y2", py0+dy);
    xl.setAttribute("stroke", xColor); xl.setAttribute("stroke-width", "2.5");
    svg.appendChild(xl);
  }
  const plbl = document.createElementNS(svgNS, "text");
  plbl.setAttribute("x", px0 + 10); plbl.setAttribute("y", py0 + 4);
  plbl.setAttribute("font-size", "10"); plbl.setAttribute("fill", xColor);
  plbl.setAttribute("font-family", "sans-serif"); plbl.textContent = p.label;
  svg.appendChild(plbl);
}

// ML arc bound for semicircle contribution
// |∮_arc 1/(1+z²) dz| ≤ M·πR where M = 1/(R²-1) for R>1
const arcBound = R > 1.01 ? (1 / (R*R - 1)) * Math.PI * R : null;
const poleInside = R > 1; // z=i is inside when R>1
```

```
  // Residue at z=i: 1/(z+i) at z=i = 1/(2i); residue × 2πi = 2πi × 1/(2i) = π
  const residueContrib = poleInside ? Math.PI : 0;

  // Info panel
  const infoX = 4, infoY = H - 45;
  const infoBg = document.createElementNS(svgNS, "rect");
  infoBg.setAttribute("x", infoX); infoBg.setAttribute("y", infoY);
  infoBg.setAttribute("width", W - 8); infoBg.setAttribute("height", 40);
  infoBg.setAttribute("fill", poleInside ? "#f0fdf4" : "#f9fafb");
  infoBg.setAttribute("rx", "4");
  svg.appendChild(infoBg);

  const line1 = document.createElementNS(svgNS, "text");
  line1.setAttribute("x", infoX + 6); line1.setAttribute("y", infoY + 14);
  line1.setAttribute("font-size", "10"); line1.setAttribute("font-family", "sans-serif");
  line1.setAttribute("fill", poleInside ? "#059669" : "#6b7280");
  const arcBndStr = arcBound !== null ? "Arc bound: \u03c0R/(R\u00b2\u22121) \u2248 " + arcBoun
  line1.textContent = arcBndStr;
  svg.appendChild(line1);

  const line2 = document.createElementNS(svgNS, "text");
  line2.setAttribute("x", infoX + 6); line2.setAttribute("y", infoY + 30);
  line2.setAttribute("font-size", "10"); line2.setAttribute("font-family", "sans-serif");
  line2.setAttribute("font-weight", "600");
  line2.setAttribute("fill", poleInside ? "#059669" : "#6b7280");
  line2.textContent = poleInside
    ? "\u222b dx/(1+x\u00b2) = 2\u03c0i \u00d7 Res(i) = 2\u03c0i \u00d7 1/(2i) = \u03c0"
    : "z = i not yet inside \u2014 increase R above 1";
  svg.appendChild(line2);

  // R label
  const Rlbl = document.createElementNS(svgNS, "text");
  Rlbl.setAttribute("x", toSx(R) + 4); Rlbl.setAttribute("y", toSy(0) - 5);
  Rlbl.setAttribute("font-size", "10"); Rlbl.setAttribute("fill", "#f97316");
  Rlbl.setAttribute("font-family", "sans-serif"); Rlbl.textContent = "R = " + R.toFixed(2);
  svg.appendChild(Rlbl);
  const semicircLbl = document.createElementNS(svgNS, "text");
  semicircLbl.setAttribute("x", toSx(0) - 14); semicircLbl.setAttribute("y", toSy(R) - 6);
  semicircLbl.setAttribute("font-size", "10"); semicircLbl.setAttribute("fill", "#3b82f6");
  semicircLbl.setAttribute("font-family", "sans-serif"); semicircLbl.textContent = "\u0393\u1d5
  svg.appendChild(semicircLbl);

  return svg;
}

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r
```

```
  const titleEl = document.createElement("div");
  titleEl.style.cssText = "font-weight:600; margin-bottom:0.5rem;";
  titleEl.textContent = "Semicircle closure: evaluating \u222b\u208b\u221e\u207a\u221e dx/(1+x\u00b2
  container.appendChild(titleEl);
  container.appendChild(RscSlider);

  const vizDiv = document.createElement("div");
  vizDiv.style.marginTop = "0.5rem";
  container.appendChild(vizDiv);

  const noteEl = document.createElement("div");
  noteEl.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  noteEl.textContent = "The orange segment is the real-axis part [\u2212R, R]; the blue arc is the s
  container.appendChild(noteEl);

  function update() {
    vizDiv.innerHTML = "";
    vizDiv.appendChild(buildSC(RscSlider.value));
  }
  RscSlider.addEventListener("input", update);
  update();

  return container;
}
```

---

## 48.6. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
```

```
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell); row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "Solution complete.";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn); controls.appendChild(resetBtn); controls.appendChild(counter);
  container.appendChild(stepsDiv); container.appendChild(controls);
  renderSteps(); updateButtons();
  return container;
}
```

### 48.6.1. Exercise 1: Direct computation of a contour integral

Compute $\int_C \bar{z}\,dz$ where $C$ is the straight line from 0 to $1+i$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Parametrise C: z(t) = t(1+i), 0 ≤ t ≤ 1",
    eq: "z(t) = t + it, \\quad dz = (1+i)\\,dt, \\quad \\bar{z}(t) = t - it = t(1-i)"
  },
  {
    op: "Substitute into the integral",
    eq: "\\int_C \\bar{z}\\,dz = \\int_0^1 t(1-i)(1+i)\\,dt = \\int_0^1 t(1-i^2)\\,dt = \\int_0^1 2t
  },
  {
    op: "Evaluate",
    eq: "\\int_C \\bar{z}\\,dz = 2\\cdot\\frac{1}{2} = 1"
  },
  {
    op: "Compare with ∮_C z dz (which IS path-independent)",
    eq: "\\int_C z\\,dz = \\left[\\frac{z^2}{2}\\right]_0^{1+i} = \\frac{(1+i)^2}{2} = \\frac{2i}{2}
    note: "z dz = d(z²/2) since z² is analytic. But z̄ is not analytic, so z̄ dz is path-dependent an
  }
])
```

---

### 48.6.2. Exercise 2: Cauchy's theorem — zero integral

Show that $\oint_C \frac{e^z}{z^2+4}\,dz = 0$ where $C$ is the circle $|z| = 1$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Locate the singularities of the integrand",
    eq: "z^2 + 4 = 0 \\implies z = \\pm 2i"
  },
  {
    op: "Check whether singularities lie inside |z| = 1",
    eq: "|2i| = 2 > 1 \\quad \\text{and} \\quad |-2i| = 2 > 1",
    note: "Both poles are outside the contour C."
  },
  {
    op: "Apply Cauchy-Goursat theorem",
    eq: "f(z) = \\frac{e^z}{z^2+4} \\text{ is analytic everywhere inside } |z|=1",
```

```
    note: "No singularities inside C, so the domain enclosed is simply connected and f is analyti
  },
  {
    op: "Conclude",
    eq: "\\oint_{|z|=1}\\frac{e^z}{z^2+4}\\,dz = 0"
  }
])
```

---

### 48.6.3. Exercise 3: Cauchy's integral formula

Evaluate $\oint_C \frac{e^z}{z-1}\,dz$ where $C$ is the circle $|z| = 2$ (counterclockwise).

```
//| echo: false
makeStepperHTML([
  {
    op: "Identify z₀ and check it is inside C",
    eq: "\\text{Integrand} = \\frac{f(z)}{z-z_0} \\text{ with } f(z) = e^z,\\; z_0 = 1",
    note: "|z₀| = 1 < 2, so z₀ = 1 is inside |z| = 2. f(z) = e^z is analytic everywhere."
  },
  {
    op: "Apply Cauchy's integral formula",
    eq: "\\oint_C \\frac{e^z}{z-1}\\,dz = 2\\pi i\\,f(1) = 2\\pi i\\,e"
  }
])
```

---

### 48.6.4. Exercise 4: Derivative formula

Evaluate $\oint_C \frac{\cos z}{(z-\pi)^2}\,dz$ where $C : |z| = 4$ (counterclockwise). The integrand has a pole at $z = \pi$ inside $C$, so Cauchy's theorem alone cannot give zero — you need the derivative formula.

```
//| echo: false
makeStepperHTML([
  {
    op: "Recognise the derivative formula with n=1",
    eq: "\\oint_C \\frac{f(z)}{(z-z_0)^{n+1}}\\,dz = \\frac{2\\pi i}{n!}\\,f^{(n)}(z_0)"
  },
  {
    op: "Identify f(z) = cos z, z₀ = ?, n = 1",
    eq: "\\text{Need } f'(\\pi) = -\\sin(\\pi) = 0"
  },
```

```
  {
    op: "Apply the formula",
    eq: "\\oint_C \\frac{\\cos z}{(z-\\pi)^2}\\,dz = \\frac{2\\pi i}{1!}\\,f'(\\pi) = 2\\pi i \\cdot
    note: "The integral is zero not because the integrand is analytic inside C (it has a pole at ⍰)
  }
])
```

---

### 48.6.5. Exercise 5: ML inequality estimate

Without evaluating it exactly, show that $\left|\oint_C \frac{dz}{z^3}\right| \leq \frac{\pi}{4}$, where $C$ is the upper semicircle $|z| = 2$ from $z = 2$ to $z = -2$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Identify M = max|f(z)| on C",
    eq: "f(z) = z^{-3}, \\quad |f(z)| = |z|^{-3} = 2^{-3} = \\tfrac{1}{8} \\text{ on } |z|=2"
  },
  {
    op: "Compute the arc length L",
    eq: "C \\text{ is a semicircle of radius } 2 \\implies L = \\pi\\cdot 2 = 2\\pi"
  },
  {
    op: "Apply the ML inequality",
    eq: "\\left|\\oint_C \\frac{dz}{z^3}\\right| \\leq M\\cdot L = \\frac{1}{8}\\cdot 2\\pi = \\frac
    note: "The actual value (by a direct computation or the residue theorem) is 0, since z^{-3} = d
  }
])
```

# 49. Series and residues

Laurent series, isolated singularities, and the residue theorem

## 49.1. Taylor series for analytic functions

If $f$ is analytic in a disk $|z - z_0| < R$, it has a **Taylor series** that converges in that disk:

$$f(z) = \sum_{n=0}^{\infty} c_n (z - z_0)^n, \qquad c_n = \frac{f^{(n)}(z_0)}{n!} = \frac{1}{2\pi i} \oint_C \frac{f(z)}{(z - z_0)^{n+1}}\, dz$$

The series converges absolutely for $|z - z_0| < R$ and uniformly on any closed subdisk. The radius of convergence $R$ equals the distance from $z_0$ to the nearest singularity of $f$.

**Key Taylor series** (centred at 0, valid for all $z$):

$$e^z = \sum_{n=0}^{\infty} \frac{z^n}{n!}, \qquad \sin z = \sum_{n=0}^{\infty} \frac{(-1)^n z^{2n+1}}{(2n+1)!}, \qquad \cos z = \sum_{n=0}^{\infty} \frac{(-1)^n z^{2n}}{(2n)!}$$

$$\frac{1}{1 - z} = \sum_{n=0}^{\infty} z^n \quad (|z| < 1)$$

These are exactly the real Taylor series with $x$ replaced by $z$.

```
//| echo: false
{
  // Taylor series convergence disk — missed opportunity 1
  // Draggable z₀, disk of radius R = distance to nearest singularity
  const W = 480, H = 420, CX = 200, CY = 210, SCALE = 70;
  const svgNS = "http://www.w3.org/2000/svg";

  const funcSel = Inputs.select([
    "1/(z²+1)  — poles at ±i",
    "tan z      — poles at ±π/2, ±3π/2, …",
    "log z      — branch cut on Re z<0",
    "e^z        — entire (no singularity)"
  ], { value: "1/(z²+1)  — poles at ±i", label: "f(z)" });
```

```
const nTermsSl = Inputs.range([1, 20], { value: 6, step: 1, label: "Partial sum N" });

// z₀ position via sliders (dragging complex in OJS needs canvas events; sliders are cleaner)
const z0reSl = Inputs.range([-2.5, 2.5], { value: 0.5, step: 0.05, label: "Re(z₀)" });
const z0imSl = Inputs.range([-2.5, 2.5], { value: 0.3, step: 0.05, label: "Im(z₀)" });

function singularities(fname) {
  if (fname.startsWith("1/(z²+1)")) return [{re:0,im:1},{re:0,im:-1}];
  if (fname.startsWith("tan")) {
    const s = [];
    for (let k = -3; k <= 3; k++) s.push({re: (k+0.5)*Math.PI, im: 0});
    return s;
  }
  if (fname.startsWith("log")) return [{re:-99,im:0}]; // branch cut; treat as nearest on negative
  return []; // entire
}

function nearestDist(fname, z0re, z0im) {
  if (fname.startsWith("log")) {
    // Distance to nearest branch-cut point on {Re□0} = distance to point (-|z0re|, z0im) if z0re<
    // Approximate: distance to origin along the branch cut direction
    // The branch cut is the negative real axis; nearest point is (min(z0re,0), 0) extended
    // Actually: the branch point is at z=0, and the branch cut extends left.
    // Convergence radius of Taylor series at z0 = distance to origin (the branch point)
    return Math.sqrt(z0re*z0re + z0im*z0im);
  }
  const sings = singularities(fname);
  if (sings.length === 0) return 999; // entire
  let minD = Infinity;
  for (const s of sings) {
    const d = Math.sqrt((z0re-s.re)**2 + (z0im-s.im)**2);
    if (d < minD) minD = d;
  }
  return minD;
}

function evalFunc(fname, re, im) {
  if (fname.startsWith("1/(z²+1)")) {
    const a = 1 + re*re - im*im, b = 2*re*im;
    const d = a*a + b*b; if (d < 1e-10) return [NaN, NaN];
    return [a/d, -b/d];
  }
  if (fname.startsWith("tan")) {
    const sr = Math.sin(re)*Math.cosh(im), si = Math.cos(re)*Math.sinh(im);
    const cr = Math.cos(re)*Math.cosh(im), ci = -Math.sin(re)*Math.sinh(im);
    const d = cr*cr + ci*ci; if (d < 1e-10) return [NaN, NaN];
    return [(sr*cr+si*ci)/d, (si*cr-sr*ci)/d];
```

```
  }
  if (fname.startsWith("log")) {
    const r2 = re*re + im*im; if (r2 < 1e-10) return [NaN, NaN];
    return [0.5*Math.log(r2), Math.atan2(im, re)];
  }
  // e^z
  return [Math.exp(re)*Math.cos(im), Math.exp(re)*Math.sin(im)];
}

function taylorCoeffs(fname, z0re, z0im, N) {
  // Numerical Taylor coefficients via finite differences (central diff for accuracy)
  // c_n = f^(n)(z0) / n!  — computed by repeated complex differentiation
  // We use forward differences with h = 0.001 * R direction
  const h = 0.001;
  const coeffs = []; // [re, im] for n=0..N-1
  // Use complex differentiation formula: f'(z) ≈ (f(z+h) - f(z-h)) / (2h)
  // We'll just evaluate directly for known functions
  if (fname.startsWith("1/(z²+1)")) {
    // f(z) = 1/(z²+1); f^(n)(z0) computable
    // Use the partial fraction decomposition: f = i/2 * (1/(z+i) - 1/(z-i))
    // 1/(z-a)^{n+1} contributes (-1)^n * n! / (z0-a)^{n+1} to n-th derivative
    const c = []; // coefficients c_n for Taylor at z0
    const amre = z0re, amim = z0im + 1; // z0 - (-i) = z0 + i
    const apre = z0re, apim = z0im - 1; // z0 - i
    const amd = amre*amre + amim*amim;
    const apd = apre*apre + apim*apim;
    if (amd < 1e-10 || apd < 1e-10) return null;
    // c_n = i/2 * [(-1)^n n! / (z0-(-i))^{n+1} - (-1)^n n! / (z0-i)^{n+1}]
    // = i/2 * (-1)^n n! * [1/(z0+i)^{n+1} - 1/(z0-i)^{n+1}]
    // Power of complex: (a+ib)^k
    function cpow(re, im, k) {
      if (k === 0) return [1, 0];
      let r = Math.sqrt(re*re+im*im), theta = Math.atan2(im, re);
      let rk = Math.pow(r, k), tk = theta * k;
      return [rk*Math.cos(tk), rk*Math.sin(tk)];
    }
    function cdiv(ar, ai, br, bi) {
      const d = br*br + bi*bi; if (d < 1e-12) return [NaN, NaN];
      return [(ar*br+ai*bi)/d, (ai*br-ar*bi)/d];
    }
    for (let n = 0; n < N; n++) {
      const [pm_re, pm_im] = cpow(amre, amim, n+1);
      const [pp_re, pp_im] = cpow(apre, apim, n+1);
      const [inv_am_re, inv_am_im] = cdiv(1, 0, pm_re, pm_im);
      const [inv_ap_re, inv_ap_im] = cdiv(1, 0, pp_re, pp_im);
      // diff = inv_am - inv_ap
      const diffRe = inv_am_re - inv_ap_re, diffIm = inv_am_im - inv_ap_im;
```

```
      // multiply by i/2 and (-1)^n (n! already absorbed into c_n = f^(n)(z0)/n!)
      // c_n = i/2 * (-1)^n * [diff]   (the n! cancels with 1/n!)
      const sign = (n%2===0) ? 1 : -1;
      // * i/2: (a+ib)*i = -b+ia
      const cr = sign * (-diffIm) / 2;
      const ci = sign * (diffRe) / 2;
      c.push([cr, ci]);
    }
    return c;
  }
  // Fallback: numerical coefficients via repeated evaluation
  // f^(n)(z0) ≈ (2h)^{-n} * sum_k C(n,k)*(-1)^{n-k} f(z0 + k*h) — forward differences
  // For simplicity, use a simple numerical approach
  const c = [];
  for (let n = 0; n < N; n++) {
    // n-th derivative via central finite differences (Richardson extrapolation would be better bu
    const hn = Math.pow(0.01, 1); // step
    let sumRe = 0, sumIm = 0;
    // Forward difference: f^(n)(z0) ≈ (1/h^n) sum_{k=0}^{n} (-1)^{n-k} C(n,k) f(z0 + k*h)
    let fact = 1;
    for (let k = 0; k <= n; k++) {
      const sign = ((n-k)%2===0) ? 1 : -1;
      const binom = fact / ((k <= n-k ? [1,1,2,6,24,120,720,5040][k]||1 : [1,1,2,6,24,120,720,5040
      const [fr, fi] = evalFunc(fname, z0re + k*hn, z0im);
      // C(n,k) = n! / (k! * (n-k)!)
      let cn = 1; for (let j=0;j<n;j++) cn*=(j<k?1:1); // just accumulate properly
      // Simple binomial
      let bk = 1; for (let j=0;j<k;j++) bk = bk*(n-j)/(j+1);
      sumRe += sign * bk * fr;
      sumIm += sign * bk * fi;
    }
    let nfact = 1; for (let j=1;j<=n;j++) nfact*=j;
    c.push([sumRe / Math.pow(hn, n) / nfact, sumIm / Math.pow(hn, n) / nfact]);
  }
  return c;
}

function taylorPartialSum(coeffs, z0re, z0im, re, im, N) {
  // Sum c_n (z - z0)^n
  const dre = re - z0re, dim = im - z0im;
  let sumRe = 0, sumIm = 0;
  let powRe = 1, powIm = 0;
  for (let n = 0; n < Math.min(coeffs.length, N); n++) {
    const [cr, ci] = coeffs[n];
    sumRe += cr*powRe - ci*powIm;
    sumIm += cr*powIm + ci*powRe;
    const newPowRe = powRe*dre - powIm*dim;
```

```
    const newPowIm = powRe*dim + powIm*dre;
    powRe = newPowRe; powIm = newPowIm;
  }
  return [sumRe, sumIm];
}

function hslToRgb(h, s, l) {
  h = ((h%1)+1)%1;
  const c = (1-Math.abs(2*l-1))*s, x = c*(1-Math.abs((h*6)%2-1)), m = l-c/2;
  let r=0,g=0,b=0;
  if(h<1/6){r=c;g=x;}else if(h<2/6){r=x;g=c;}else if(h<3/6){g=c;b=x;}
  else if(h<4/6){g=x;b=c;}else if(h<5/6){r=x;b=c;}else{r=c;b=x;}
  return [Math.round((r+m)*255),Math.round((g+m)*255),Math.round((b+m)*255)];
}

function toScreen(re, im) { return [CX + re*SCALE, CY - im*SCALE]; }

function render(fname, z0re, z0im, N) {
  const R = nearestDist(fname, z0re, z0im);
  const coeffs = taylorCoeffs(fname, z0re, z0im, N) || [];

  // Left panel: exact function with disk overlay
  const cL = document.createElement("canvas");
  cL.width = W; cL.height = H;
  const ctxL = cL.getContext("2d");
  const imgL = ctxL.createImageData(W, H); const dL = imgL.data;

  for (let px=0; px<W; px++) {
    for (let py=0; py<H; py++) {
      const re = (px-CX)/SCALE, im = -(py-CY)/SCALE;
      const [wr, wi] = evalFunc(fname, re, im);
      if (!isFinite(wr)||!isFinite(wi)||isNaN(wr)) {
        const idx=(py*W+px)*4;
        dL[idx]=255;dL[idx+1]=255;dL[idx+2]=255;dL[idx+3]=255; continue;
      }
      const arg = Math.atan2(wi, wr), mag = Math.sqrt(wr*wr+wi*wi);
      const hue = (arg/(2*Math.PI)+1)%1;
      const brightness = 0.5 + 0.35*Math.tanh(Math.log(mag+0.001)/2);
      const [r,g,b] = hslToRgb(hue, 0.8, brightness);
      const idx=(py*W+px)*4; dL[idx]=r;dL[idx+1]=g;dL[idx+2]=b;dL[idx+3]=255;
    }
  }
  ctxL.putImageData(imgL, 0, 0);

  // Convergence disk — gold dashed circle
  const [z0sx, z0sy] = toScreen(z0re, z0im);
  ctxL.setLineDash([6, 4]);
```

```
ctxL.strokeStyle = "#fbbf24"; ctxL.lineWidth = 2.5;
ctxL.beginPath();
if (R < 50) ctxL.arc(z0sx, z0sy, R*SCALE, 0, 2*Math.PI);
ctxL.stroke();
ctxL.setLineDash([]);

// Line to nearest singularity
const sings = singularities(fname);
if (sings.length > 0) {
  let nearS = sings[0], nearD = Infinity;
  for (const s of sings) {
    const d = Math.sqrt((z0re-s.re)**2+(z0im-s.im)**2);
    if (d < nearD) { nearD = d; nearS = s; }
  }
  const [nsx, nsy] = toScreen(nearS.re, nearS.im);
  ctxL.strokeStyle = "rgba(251,191,36,0.6)"; ctxL.lineWidth = 1.5; ctxL.setLineDash([3,3]);
  ctxL.beginPath(); ctxL.moveTo(z0sx, z0sy); ctxL.lineTo(nsx, nsy); ctxL.stroke();
  ctxL.setLineDash([]);
  // Nearest singularity ×
  ctxL.strokeStyle="#ef4444"; ctxL.lineWidth=2;
  ctxL.beginPath(); ctxL.moveTo(nsx-6,nsy-6); ctxL.lineTo(nsx+6,nsy+6); ctxL.stroke();
  ctxL.beginPath(); ctxL.moveTo(nsx+6,nsy-6); ctxL.lineTo(nsx-6,nsy+6); ctxL.stroke();
}

// Axes
const [ox, oy] = toScreen(0, 0);
ctxL.strokeStyle="rgba(255,255,255,0.5)"; ctxL.lineWidth=0.8;
ctxL.beginPath(); ctxL.moveTo(0,oy); ctxL.lineTo(W,oy); ctxL.stroke();
ctxL.beginPath(); ctxL.moveTo(ox,0); ctxL.lineTo(ox,H); ctxL.stroke();

// z₀ dot
ctxL.fillStyle="#fbbf24"; ctxL.beginPath(); ctxL.arc(z0sx, z0sy, 6, 0, 2*Math.PI); ctxL.fill();
ctxL.strokeStyle="#fff"; ctxL.lineWidth=1.5; ctxL.stroke();
ctxL.fillStyle="#fff"; ctxL.font="11px sans-serif";
ctxL.fillText(`z₀`, z0sx+8, z0sy-4);
ctxL.fillText(`R = ${R < 50 ? R.toFixed(3) : "∞"}`, z0sx+8, z0sy+10);

ctxL.fillStyle="rgba(0,0,0,0.7)"; ctxL.font="11px sans-serif";
ctxL.fillText("exact f(z)", 6, 16);

// Right panel: partial sum
const cR = document.createElement("canvas");
cR.width = W; cR.height = H;
const ctxR = cR.getContext("2d");
const imgR = ctxR.createImageData(W, H); const dR = imgR.data;

for (let px=0; px<W; px++) {
```

```
      for (let py=0; py<H; py++) {
        const re = (px-CX)/SCALE, im = -(py-CY)/SCALE;
        let wr, wi;
        if (coeffs.length > 0) {
          [wr, wi] = taylorPartialSum(coeffs, z0re, z0im, re, im, N);
        } else {
          [wr, wi] = evalFunc(fname, re, im);
        }
        if (!isFinite(wr)||!isFinite(wi)||isNaN(wr)||Math.abs(wr)>200||Math.abs(wi)>200) {
          const idx=(py*W+px)*4;
          dR[idx]=240;dR[idx+1]=240;dR[idx+2]=240;dR[idx+3]=255; continue;
        }
        const arg = Math.atan2(wi, wr), mag = Math.sqrt(wr*wr+wi*wi);
        const hue = (arg/(2*Math.PI)+1)%1;
        const brightness = 0.5 + 0.35*Math.tanh(Math.log(mag+0.001)/2);
        const [r,g,b] = hslToRgb(hue, 0.8, brightness);
        const idx=(py*W+px)*4; dR[idx]=r;dR[idx+1]=g;dR[idx+2]=b;dR[idx+3]=255;
      }
    }
    ctxR.putImageData(imgR, 0, 0);

    // Disk overlay on right panel
    ctxR.setLineDash([6,4]);
    ctxR.strokeStyle="#fbbf24"; ctxR.lineWidth=2.5;
    ctxR.beginPath();
    if (R < 50) ctxR.arc(z0sx, z0sy, R*SCALE, 0, 2*Math.PI);
    ctxR.stroke(); ctxR.setLineDash([]);
    ctxR.fillStyle="#fbbf24"; ctxR.beginPath(); ctxR.arc(z0sx, z0sy, 6, 0, 2*Math.PI); ctxR.fill(
    ctxR.strokeStyle="#fff"; ctxR.lineWidth=1.5; ctxR.stroke();
    const [ox2, oy2] = toScreen(0, 0);
    ctxR.strokeStyle="rgba(255,255,255,0.5)"; ctxR.lineWidth=0.8;
    ctxR.beginPath(); ctxR.moveTo(0,oy2); ctxR.lineTo(W,oy2); ctxR.stroke();
    ctxR.beginPath(); ctxR.moveTo(ox2,0); ctxR.lineTo(ox2,H); ctxR.stroke();
    ctxR.fillStyle="rgba(0,0,0,0.7)"; ctxR.font="11px sans-serif";
    ctxR.fillText(`Taylor partial sum, N=${N} terms`, 6, 16);
    ctxR.fillStyle="rgba(255,255,255,0.85)"; ctxR.font="10px sans-serif";
    ctxR.fillText("Inside disk: series matches f. Outside: diverges.", 6, H-8);

    const panelDiv = document.createElement("div");
    panelDiv.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap;";
    panelDiv.appendChild(cL); panelDiv.appendChild(cR);

    const infoDiv = document.createElement("div");
    infoDiv.style.cssText = "margin-top:0.5rem; font-family:sans-serif; font-size:0.88em; backgro
    infoDiv.innerHTML = `<strong>Convergence radius R = ${R < 50 ? R.toFixed(4) : "∞"}</strong> –

    const wrap = document.createElement("div");
```

```
    wrap.appendChild(panelDiv); wrap.appendChild(infoDiv);
    return wrap;
  }

  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem
  const title = document.createElement("div");
  title.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  title.textContent = "Taylor series convergence disk: R = distance to nearest singularity";
  container.appendChild(title);
  container.appendChild(funcSel);
  container.appendChild(z0reSl);
  container.appendChild(z0imSl);
  container.appendChild(nTermsSl);
  const vizDiv = document.createElement("div");
  container.appendChild(vizDiv);
  const note = document.createElement("div");
  note.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  note.textContent = "Left: exact f(z) with domain colouring (hue = arg f, brightness = log|f|). Rig
  container.appendChild(note);

  function update() {
    vizDiv.innerHTML = "";
    vizDiv.appendChild(render(funcSel.value, z0reSl.value, z0imSl.value, nTermsSl.value));
  }
  funcSel.addEventListener("input", update);
  z0reSl.addEventListener("input", update);
  z0imSl.addEventListener("input", update);
  nTermsSl.addEventListener("input", update);
  update();

  return container;
}
```

---

## 49.2. Laurent series

At a point $z_0$ where $f$ is not analytic, the Taylor series does not converge. The **Laurent series** extends the power series to include negative powers:

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z - z_0)^n$$

where the coefficients are:

$$c_n = \frac{1}{2\pi i} \oint_C \frac{f(z)}{(z - z_0)^{n+1}}\, dz$$

This converges in an **annulus** $r < |z - z_0| < R$ (with $r$ possibly 0 and $R$ possibly $\infty$).

The **principal part** consists of the negative-power terms $\sum_{n=1}^{\infty} c_{-n}(z - z_0)^{-n}$; the **analytic part** is the non-negative power series $\sum_{n=0}^{\infty} c_n(z - z_0)^n$. Whether the principal part is absent, has finitely many terms, or has infinitely many terms classifies the type of singularity at $z_0$ — which is the point of the next section.

```
//| echo: false
{
  const W = 400, H = 360, CX = W/2, CY = H/2, SCALE = 66;
  const svgNS = "http://www.w3.org/2000/svg";

  const funcSel9 = Inputs.select(["1/(z(z-1))", "e^{1/z} (essential)", "z/sin(z)"], {
    value: "1/(z(z-1))", label: "f(z)"
  });
  const rInnerSl = Inputs.range([0.01, 1.4], {value: 0.2, step: 0.01, label: "Inner barrier r"});
  const rOuterSl = Inputs.range([0.5, 3.5], {value: 1.5, step: 0.05, label: "Outer barrier R"});
  const nTermsSl = Inputs.range([1, 20], {value: 6, step: 1, label: "N terms (e^{1/z} panel)"});

  // Singularity metadata per function
  const funcMeta = {
    "1/(z(z-1))": {
      sings: [{re:0,im:0,label:"z=0 (simple pole)"},{re:1,im:0,label:"z=1 (simple pole)"}],
      ppTerms: 2, singType: "Two simple poles", annulusDesc: "0 < |z| < 1 (between poles)"
    },
    "e^{1/z} (essential)": {
      sings: [{re:0,im:0,label:"z=0 (essential)"}],
      ppTerms: Infinity, singType: "Essential singularity (infinitely many principal-part terms)"
    },
    "z/sin(z)": {
      sings: [{re:0,im:0,label:"z=0 (removable)"},{re:Math.PI,im:0,label:"z=π (simple pole)"},{re
      ppTerms: 1, singType: "Removable at 0; simple poles at ±nπ", annulusDesc: "0 < |z| < π"
    }
  };

  function evalExact9(fname, re, im) {
    switch(fname) {
      case "1/(z(z-1))": {
        const d2=re*re+im*im; if(d2<1e-8) return [NaN,NaN];
        const dr=re-1,di=im,d3=dr*dr+di*di; if(d3<1e-8) return [NaN,NaN];
        return [dr/d3-re/d2, -di/d3+im/d2];
      }
      case "e^{1/z} (essential)": {
        const d=re*re+im*im; if(d<1e-8) return [NaN,NaN];
        const invRe=re/d, invIm=-im/d;
```

```
      return [Math.exp(invRe)*Math.cos(invIm), Math.exp(invRe)*Math.sin(invIm)];
    }
    case "z/sin(z)": {
      const sr=Math.sin(re)*Math.cosh(im), si=Math.cos(re)*Math.sinh(im);
      const d=sr*sr+si*si; if(d<1e-8) return (re*re+im*im<1e-8)?[1,0]:[NaN,NaN];
      return [(re*sr+im*si)/d, (im*sr-re*si)/d];
    }
    default: return [NaN,NaN];
  }
}

function evalPartial9(fname, re, im, N) {
  // Genuine N-term partial sums of Laurent series at z0=0
  switch(fname) {
    case "1/(z(z-1))": {
      // For 0<|z|<1: -1/z - 1 - z - z^2 - ... (principal part: -1/z; analytic: -sum z^n)
      const d=re*re+im*im; if(d<1e-8) return [NaN,NaN];
      const invRe=re/d, invIm=-im/d;
      // -1/z term
      let sumRe = -invRe, sumIm = -invIm;
      // -z^n terms for n=0..N-2
      let powRe=1, powIm=0;
      for(let k=0;k<N-1;k++) {
        sumRe -= powRe; sumIm -= powIm;
        const nRe=powRe*re-powIm*im; const nIm=powRe*im+powIm*re;
        powRe=nRe; powIm=nIm;
      }
      return [sumRe, sumIm];
    }
    case "e^{1/z} (essential)": {
      const d=re*re+im*im; if(d<1e-8) return [NaN,NaN];
      const invRe=re/d, invIm=-im/d;
      let sumRe=1, sumIm=0, powRe=1, powIm=0, fact=1;
      for(let k=1;k<=Math.min(N,25);k++) {
        const nRe=powRe*invRe-powIm*invIm; const nIm=powRe*invIm+powIm*invRe;
        powRe=nRe; powIm=nIm; fact*=k;
        sumRe+=powRe/fact; sumIm+=powIm/fact;
      }
      return [sumRe, sumIm];
    }
    case "z/sin(z)": {
      // Laurent around 0: z/sin(z) = 1 + z²/6 + 7z⁴/360 + ...  (all non-negative powers; removabl
      // Treat as Taylor (no principal part)
      const coeffs=[1, 0, 1/6, 0, 7/360, 0, 31/15120, 0, 127/604800, 0];
      let sumRe=0, sumIm=0, powRe=1, powIm=0;
      for(let k=0;k<Math.min(N,coeffs.length);k++) {
        sumRe+=coeffs[k]*powRe; sumIm+=coeffs[k]*powIm;
```

```
        const nRe=powRe*re-powIm*im; const nIm=powRe*im+powIm*re;
        powRe=nRe; powIm=nIm;
      }
      return [sumRe, sumIm];
    }
    default: return [NaN,NaN];
  }
}

function computeMaxError9(fname, rInner, rOuter, N, SAMPLES) {
  // Estimate max |f_exact - f_partial| on a circle of radius midR
  const midR = Math.sqrt(rInner * rOuter); // geometric mean
  let maxErr = 0;
  for(let k=0;k<SAMPLES;k++) {
    const theta = 2*Math.PI*k/SAMPLES;
    const re = midR*Math.cos(theta), im = midR*Math.sin(theta);
    const [er,ei] = evalExact9(fname, re, im);
    const [pr,pi] = evalPartial9(fname, re, im, N);
    if(!isFinite(er)||!isFinite(pr)) continue;
    const err = Math.sqrt((er-pr)**2+(ei-pi)**2);
    if(err>maxErr) maxErr=err;
  }
  return maxErr;
}

function hslToRgb9(h,s,l) {
  h=((h%1)+1)%1;
  const c=(1-Math.abs(2*l-1))*s,x=c*(1-Math.abs((h*6)%2-1)),m=l-c/2;
  let r=0,g=0,b=0;
  if(h<1/6){r=c;g=x;}else if(h<2/6){r=x;g=c;}else if(h<3/6){g=c;b=x;}
  else if(h<4/6){g=x;b=c;}else if(h<5/6){r=x;b=c;}else{r=c;b=x;}
  return [Math.round((r+m)*255),Math.round((g+m)*255),Math.round((b+m)*255)];
}

function toScr9(re,im) { return [CX+re*SCALE, CY-im*SCALE]; }

function renderPanel9(fname, rInner, rOuter, N, usePartial) {
  const canvas=document.createElement("canvas");
  canvas.width=W; canvas.height=H;
  const ctx=canvas.getContext("2d");
  const img=ctx.createImageData(W,H); const data=img.data;

  for(let px=0;px<W;px++) {
    for(let py=0;py<H;py++) {
      const re=(px-CX)/SCALE, im=-(py-CY)/SCALE;
      const dist=Math.sqrt(re*re+im*im);
      const inAnnulus = dist > rInner && dist < rOuter;
```

```
      if(!inAnnulus) {
        const idx=(py*W+px)*4; data[idx]=210;data[idx+1]=210;data[idx+2]=210;data[idx+3]=255; cont
      }

      const [wr,wi] = usePartial ? evalPartial9(fname,re,im,N) : evalExact9(fname,re,im);
      if(!isFinite(wr)||!isFinite(wi)||isNaN(wr)||Math.abs(wr)>1e6||Math.abs(wi)>1e6) {
        const idx=(py*W+px)*4; data[idx]=255;data[idx+1]=255;data[idx+2]=255;data[idx+3]=255; cont
      }
      const arg=Math.atan2(wi,wr), mag=Math.sqrt(wr*wr+wi*wi);
      const hue=(arg/(2*Math.PI)+1)%1;
      const brightness=0.5+0.35*Math.tanh(Math.log(mag+0.001)/2);
      const [r,g,b]=hslToRgb9(hue,0.8,brightness);
      const idx=(py*W+px)*4; data[idx]=r;data[idx+1]=g;data[idx+2]=b;data[idx+3]=255;
    }
  }
  ctx.putImageData(img,0,0);

  const [ox,oy]=toScr9(0,0);
  // Axes
  ctx.strokeStyle="rgba(255,255,255,0.4)"; ctx.lineWidth=0.8;
  ctx.beginPath(); ctx.moveTo(0,oy); ctx.lineTo(W,oy); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(ox,0); ctx.lineTo(ox,H); ctx.stroke();

  // Inner barrier (blue)
  ctx.strokeStyle="#3b82f6"; ctx.lineWidth=2.5; ctx.setLineDash([]);
  ctx.beginPath(); ctx.arc(ox,oy,rInner*SCALE,0,2*Math.PI); ctx.stroke();
  // Inner label
  ctx.fillStyle="#3b82f6"; ctx.font="bold 9px sans-serif";
  ctx.fillText("inner singularity", ox+rInner*SCALE*0.6,oy-rInner*SCALE-4);
  ctx.fillText("barrier r="+rInner.toFixed(2), ox+rInner*SCALE*0.6, oy-rInner*SCALE+8);

  // Outer barrier (orange)
  ctx.strokeStyle="#f97316"; ctx.lineWidth=2.5;
  ctx.beginPath(); ctx.arc(ox,oy,Math.min(rOuter*SCALE,W*0.72),0,2*Math.PI); ctx.stroke();
  ctx.fillStyle="#f97316"; ctx.font="bold 9px sans-serif";
  const angOuter = -0.5;
  const ox2=ox+Math.min(rOuter*SCALE,W*0.72)*Math.cos(angOuter), oy2=oy+Math.min(rOuter*SCALE,W*0
  ctx.fillText("outer singularity", ox2+4, oy2-3);
  ctx.fillText("barrier R="+rOuter.toFixed(2), ox2+4, oy2+9);

  // Singularity markers
  const meta = funcMeta[fname];
  meta.sings.forEach(s => {
    const [sx,sy]=toScr9(s.re,s.im);
    ctx.strokeStyle="#dc2626"; ctx.lineWidth=2;
    ctx.beginPath(); ctx.moveTo(sx-6,sy-6); ctx.lineTo(sx+6,sy+6); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(sx+6,sy-6); ctx.lineTo(sx-6,sy+6); ctx.stroke();
```

```
  });

  // Title
  ctx.fillStyle="rgba(0,0,0,0.75)"; ctx.font="bold 10px sans-serif";
  ctx.fillText(usePartial?`N=${N}-term partial sum`:"exact f(z)", 5, H-7);

  return canvas;
}

function renderLaurent9(fname, rInner, rOuter, N) {
  if(rOuter <= rInner + 0.01) rOuter = rInner + 0.15;
  const meta = funcMeta[fname];
  const isEssential = fname === "e^{1/z} (essential)";

  const panelsDiv = document.createElement("div");
  panelsDiv.style.cssText = "display:flex; gap:0.5rem; flex-wrap:wrap;";

  const exactCanvas = renderPanel9(fname, rInner, rOuter, N, false);
  const partialCanvas = renderPanel9(fname, rInner, rOuter, N, true);

  // Label wrappers
  function labeled(canvas, labelText) {
    const d = document.createElement("div");
    const lbl = document.createElement("div");
    lbl.style.cssText = "font-size:0.8em; font-weight:600; text-align:center; margin-bottom:2px
    lbl.textContent = labelText;
    d.appendChild(lbl); d.appendChild(canvas);
    return d;
  }

  panelsDiv.appendChild(labeled(exactCanvas, "Exact f(z) in annulus"));
  if(isEssential) panelsDiv.appendChild(labeled(partialCanvas, `Partial sum (N=${N} terms of e^

  const infoDiv = document.createElement("div");
  infoDiv.style.cssText = "margin-top:0.5rem; font-family:sans-serif; font-size:0.85em;";

  let ppDesc = meta.ppTerms === Infinity
    ? `<strong style="color:#dc2626">∞ principal-part terms</strong> — essential singularity`
    : `<strong>${meta.ppTerms}</strong> principal-part term${meta.ppTerms===1?"":"s"}`;

  let errStr = "";
  if(isEssential) {
    const maxErr = computeMaxError9(fname, rInner, rOuter, N, 64);
    errStr = `<div style="margin-top:0.3rem;color:#374151;"><strong>Max error on midpoint circl
  }

  infoDiv.innerHTML = `
```

```
      <div><strong>Singularity type:</strong> <span style="color:#1e40af">${meta.singType}</span></d
      <div><strong>Principal part:</strong> ${ppDesc}</div>
      <div><strong>Convergence annulus:</strong> ${rInner.toFixed(3)} &lt; |z| &lt; ${rOuter.toFixed
      ${errStr}
    `;

    const wrap = document.createElement("div");
    wrap.appendChild(panelsDiv); wrap.appendChild(infoDiv);
    return wrap;
  }

  const container=document.createElement("div");
  container.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem 0
  const title9=document.createElement("div");
  title9.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  title9.textContent="Laurent annulus: convergence barriers and singularity type";
  container.appendChild(title9);
  container.appendChild(funcSel9);
  container.appendChild(rInnerSl);
  container.appendChild(rOuterSl);
  container.appendChild(nTermsSl);

  const vizDiv9=document.createElement("div");
  container.appendChild(vizDiv9);

  const note9=document.createElement("div");
  note9.style.cssText="margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  note9.textContent="Domain colouring (hue = arg f, brightness = log|f|) shown only inside the annul
  container.appendChild(note9);

  function update9() {
    let rI = parseFloat(rInnerSl.value), rO = parseFloat(rOuterSl.value);
    if(rO <= rI + 0.01) rO = rI + 0.15;
    vizDiv9.innerHTML="";
    vizDiv9.appendChild(renderLaurent9(funcSel9.value, rI, rO, parseInt(nTermsSl.value)));
  }
  funcSel9.addEventListener("input", update9);
  rInnerSl.addEventListener("input", update9);
  rOuterSl.addEventListener("input", update9);
  nTermsSl.addEventListener("input", update9);
  update9();

  return container;
}
```

**Computing Laurent series in practice.** For functions that are products, quotients, or compositions of known series, it is usually more efficient to manipulate the known Taylor series directly rather than

computing coefficients by integration.

**Example.** $\sin z/z^3$ near $z = 0$:

$$\frac{\sin z}{z^3} = \frac{1}{z^3}\left(z - \frac{z^3}{6} + \frac{z^5}{120} - \cdots\right) = \frac{1}{z^2} - \frac{1}{6} + \frac{z^2}{120} - \cdots$$

The principal part has only one negative-power term, $1/z^2$.

```
//| echo: false
{
  // Laurent coefficient spectrum — missed opportunity 4
  // Bar chart of c_n for n=−6..+4; red bars = principal part (n<0), blue = analytic (n≥0)
  // Right panel: domain colouring built by adding k principal-part terms one by one
  const W2 = 320, H2 = 280, CX2 = W2/2, CY2 = H2/2, SCALE2 = 55;

  const funcSelSpec = Inputs.select([
    "1/z(z-1)  [2 pp terms]",
    "e^{1/z}    [∞ pp terms]",
    "sin(z)/z   [0 pp terms — removable]",
    "1/z²       [1 pp term — order-2 pole]"
  ], { value: "1/z(z-1)  [2 pp terms]", label: "f(z)" });

  const kSlider = Inputs.range([0, 6], { value: 0, step: 1, label: "Principal-part terms added (k

  // Laurent coefficients c_n for n = -6..+4 (index 0 = n=-6)
  function getLaurentCoeffs(fname) {
    // Returns array of {n, c} for n=-6..4
    const result = [];
    for(let n=-6; n<=4; n++) {
      let c = 0;
      if(fname.startsWith("1/z(z-1)")) {
        // 1/(z(z-1)) = -1/z - 1 - z - z^2 - ... for |z|<1
        if(n===-1) c=-1;
        else if(n>=0) c=-1;
        else c=0;
      } else if(fname.startsWith("e^{1/z}")) {
        // e^{1/z} = sum_{k=0}^∞ (1/z)^k / k! = sum_{k=0}^∞ z^{-k} / k!
        if(n<=0) c = 1/factorial(-n);
        else c = 0;
      } else if(fname.startsWith("sin(z)/z")) {
        // sin(z)/z = 1 - z²/6 + z⁴/120 - ...
        if(n===0) c=1; else if(n===2) c=-1/6; else if(n===4) c=1/120; else c=0;
      } else if(fname.startsWith("1/z²")) {
        if(n===-2) c=1; else c=0;
      }
      result.push({n, c});
    }
```

```
  return result;
}

function factorial(k) {
  if(k<=0) return 1; let f=1; for(let i=2;i<=k;i++) f*=i; return f;
}

// Evaluate partial sum using k principal-part terms
function evalKTerms(fname, re, im, k) {
  const coeffs = getLaurentCoeffs(fname);
  // Principal-part coefficients: n < 0, sorted from most-negative to -1
  const ppCoeffs = coeffs.filter(c=>c.n<0&&c.c!==0).sort((a,b)=>a.n-b.n);
  const analyticCoeffs = coeffs.filter(c=>c.n>=0);
  // Take first k principal-part terms (smallest n first)
  const activeCoeffs = [
    ...ppCoeffs.slice(0, k),
    ...analyticCoeffs
  ];
  const d2 = re*re+im*im; if(d2<1e-8) return [NaN,NaN];
  let sumRe=0, sumIm=0;
  for(const {n, c} of activeCoeffs) {
    if(c===0) continue;
    // z^n = r^n * e^{i*n*theta}
    const r = Math.sqrt(d2);
    const theta = Math.atan2(im, re);
    const rn = Math.pow(r, n);
    if(!isFinite(rn)) continue;
    sumRe += c * rn * Math.cos(n*theta);
    sumIm += c * rn * Math.sin(n*theta);
  }
  return [sumRe, sumIm];
}

function hslToRgbSpec(h,s,l) {
  h=((h%1)+1)%1;
  const c=(1-Math.abs(2*l-1))*s,x=c*(1-Math.abs((h*6)%2-1)),m=l-c/2;
  let r=0,g=0,b=0;
  if(h<1/6){r=c;g=x;}else if(h<2/6){r=x;g=c;}else if(h<3/6){g=c;b=x;}
  else if(h<4/6){g=x;b=c;}else if(h<5/6){r=x;b=c;}else{r=c;b=x;}
  return [Math.round((r+m)*255),Math.round((g+m)*255),Math.round((b+m)*255)];
}

function renderSpectrum(fname, k) {
  const coeffs = getLaurentCoeffs(fname);
  const ppCoeffs = coeffs.filter(c=>c.n<0&&c.c!==0).sort((a,b)=>a.n-b.n);
  const kCapped = Math.min(k, ppCoeffs.length);
```

```
// --- Bar chart ---
const BW = 380, BH = 200;
const barCanvas = document.createElement("canvas");
barCanvas.width = BW; barCanvas.height = BH;
const bctx = barCanvas.getContext("2d");
bctx.fillStyle="#f9fafb";
bctx.fillRect(0,0,BW,BH);

const nVals = coeffs.map(c=>c.n);
const n0 = nVals[0], n1 = nVals[nVals.length-1];
const totalBars = n1-n0+1;
const barW = Math.floor((BW-40) / totalBars) - 2;
const xOff = 30;
const zeroY = BH - 35;
const maxAbs = Math.max(...coeffs.map(c=>Math.abs(c.c)), 0.01);
const scaleY = (BH - 55) / maxAbs;

// Zero line
bctx.strokeStyle="#9ca3af"; bctx.lineWidth=1;
bctx.beginPath(); bctx.moveTo(xOff, zeroY); bctx.lineTo(BW-10, zeroY); bctx.stroke();

coeffs.forEach(({n,c}) => {
  const barIdx = n - n0;
  const x = xOff + barIdx * (barW+2) + 1;
  const h = Math.abs(c) * scaleY;
  const y = c >= 0 ? zeroY - h : zeroY;

  // Highlight active terms
  const isPP = n < 0;
  const ppIdx = ppCoeffs.findIndex(p=>p.n===n);
  const isActive = isPP ? (ppIdx >= 0 && ppIdx < kCapped) : true;
  const isAnalytic = n >= 0;

  bctx.fillStyle = isPP
    ? (isActive ? "#dc2626" : "rgba(220,38,38,0.25)")
    : (c!==0 ? "#3b82f6" : "#e5e7eb");
  bctx.fillRect(x, y, barW, Math.max(h, 1));

  // n label
  bctx.fillStyle = "#374151"; bctx.font="9px sans-serif"; bctx.textAlign="center";
  bctx.fillText(n.toString(), x+barW/2, zeroY+12);

  // c value for non-zero
  if(Math.abs(c)>0.001) {
    bctx.fillStyle = isPP ? (isActive?"#dc2626":"#f87171") : "#3b82f6";
    bctx.font="8px sans-serif";
    const valStr = Number.isInteger(c)?c.toString():c.toFixed(3);
```

```
      bctx.fillText(valStr, x+barW/2, c>=0 ? zeroY-h-3 : zeroY+h+20);
    }
  });
  bctx.textAlign="left";

  // Axis label
  bctx.fillStyle="#374151"; bctx.font="10px sans-serif";
  bctx.fillText("n", BW-15, zeroY+3);
  bctx.fillStyle="#dc2626"; bctx.font="10px sans-serif";
  bctx.fillText("red = principal part (n<0)", 30, 14);
  bctx.fillStyle="#3b82f6";
  bctx.fillText("blue = analytic part (n≥0)", 220, 14);

  // --- Domain colouring panel ---
  const dcCanvas = document.createElement("canvas");
  dcCanvas.width=W2; dcCanvas.height=H2;
  const dctx=dcCanvas.getContext("2d");
  const img2=dctx.createImageData(W2,H2); const dd=img2.data;

  for(let px=0;px<W2;px++) {
    for(let py=0;py<H2;py++) {
      const re=(px-CX2)/SCALE2, im=-(py-CY2)/SCALE2;
      const [wr,wi]=evalKTerms(fname,re,im,kCapped);
      if(!isFinite(wr)||!isFinite(wi)||isNaN(wr)||Math.abs(wr)>1e5) {
        const idx=(py*W2+px)*4; dd[idx]=255;dd[idx+1]=255;dd[idx+2]=255;dd[idx+3]=255; continue;
      }
      const arg=Math.atan2(wi,wr), mag=Math.sqrt(wr*wr+wi*wi);
      const hue=(arg/(2*Math.PI)+1)%1;
      const brightness=0.5+0.35*Math.tanh(Math.log(mag+0.001)/2);
      const [r,g,b]=hslToRgbSpec(hue,0.8,brightness);
      const idx=(py*W2+px)*4; dd[idx]=r;dd[idx+1]=g;dd[idx+2]=b;dd[idx+3]=255;
    }
  }
  dctx.putImageData(img2,0,0);

  // Axes on dc panel
  const [dcox,dcoy]=[CX2,CY2];
  dctx.strokeStyle="rgba(255,255,255,0.4)"; dctx.lineWidth=0.8;
  dctx.beginPath(); dctx.moveTo(0,dcoy); dctx.lineTo(W2,dcoy); dctx.stroke();
  dctx.beginPath(); dctx.moveTo(dcox,0); dctx.lineTo(dcox,H2); dctx.stroke();
  dctx.fillStyle="rgba(0,0,0,0.7)"; dctx.font="10px sans-serif";
  dctx.fillText(`${kCapped} pp term${kCapped===1?"":"s"} added`, 5, H2-7);

  const stepLabel = document.createElement("div");
  stepLabel.style.cssText="text-align:center; font-size:0.85em; color:#374151; margin:0.25rem 0;"
  stepLabel.textContent = kCapped === 0
    ? "Step 0: analytic part only (no principal-part terms)"
```

```
      : `Step ${kCapped}: adding c_{${ppCoeffs[kCapped-1]?.n}} term to the reconstruction`;

    const panelDiv = document.createElement("div");
    panelDiv.style.cssText="display:flex; gap:0.5rem; flex-wrap:wrap; align-items:flex-start;";
    panelDiv.appendChild(barCanvas); panelDiv.appendChild(dcCanvas);

    const wrap = document.createElement("div");
    wrap.appendChild(panelDiv);
    wrap.appendChild(stepLabel);
    return wrap;
  }

  const container2 = document.createElement("div");
  container2.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1re
  const title2 = document.createElement("div");
  title2.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  title2.textContent="Laurent coefficient spectrum: principal part vs analytic part";
  container2.appendChild(title2);
  container2.appendChild(funcSelSpec);
  container2.appendChild(kSlider);
  const vizDiv2 = document.createElement("div");
  container2.appendChild(vizDiv2);
  const note2 = document.createElement("div");
  note2.style.cssText="margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  note2.textContent="Bar chart: red = principal-part coefficients (n<0), blue = analytic-part coe
  container2.appendChild(note2);

  function updateSpec() {
    vizDiv2.innerHTML="";
    vizDiv2.appendChild(renderSpectrum(funcSelSpec.value, parseInt(kSlider.value)));
  }
  funcSelSpec.addEventListener("input", updateSpec);
  kSlider.addEventListener("input", updateSpec);
  updateSpec();

  return container2;
}
```

---

## 49.3. Classification of isolated singularities

An **isolated singularity** of $f$ at $z_0$ is a point where $f$ is not analytic but is analytic in some punctured disk $0 < |z - z_0| < \delta$. The Laurent series reveals the type:

**Removable singularity.** The principal part is absent (no negative-power terms). $f$ can be assigned a value at $z_0$ to make it analytic. Example: $\sin z/z$ at $z = 0$ — the Laurent series is $1 - z^2/6 + \cdots$, which extends continuously to $z = 0$ with value 1.

**Pole of order $m$.** The principal part has exactly $m$ terms:

$$f(z) = \frac{c_{-m}}{(z - z_0)^m} + \cdots + \frac{c_{-1}}{z - z_0} + c_0 + c_1(z - z_0) + \cdots, \quad c_{-m} \neq 0$$

$|f(z)| \to \infty$ as $z \to z_0$. A **simple pole** is a pole of order 1.

**Essential singularity.** The principal part has infinitely many terms. The behaviour near $z_0$ has no limit — finite or infinite. Example: $e^{1/z}$ at $z = 0$. Approach the origin along the positive real axis and $e^{1/z} \to +\infty$; along the negative real axis, $e^{1/z} \to 0$; along the imaginary axis, $e^{1/z}$ spirals and oscillates. In any small neighbourhood of 0, the function takes every complex value (with at most one exception).

```
//| echo: false
{
  // Singularity classifier — log-scale zoom, domain colouring, 4 singularity types
  const W_SC = 420, H_SC = 380, CX_SC = W_SC/2, CY_SC = H_SC/2;

  const funcSelSC = Inputs.select([
    "sin(z)/z   — removable",
    "1/z        — simple pole (order 1)",
    "1/z²       — order-2 pole",
    "e^{1/z}    — essential singularity"
  ], { value: "sin(z)/z   — removable", label: "Singularity type" });

  // Zoom: log scale, viewport half-width from 2 down to 0.002
  const zoomSl = Inputs.range([Math.log10(0.002), Math.log10(2)], {
    value: Math.log10(0.5), step: 0.02, label: "log₁₀(viewport half-width)"
  });

  function evalSC(fname, re, im) {
    if(fname.startsWith("sin(z)/z")) {
      // sin(z)/z: at z=0 value is 1 (removable singularity)
      const r2=re*re+im*im; if(r2<1e-12) return [1,0];
      // sin(z) = sin(x)cosh(y) + i cos(x)sinh(y)
      const sr=Math.sin(re)*Math.cosh(im), si=Math.cos(re)*Math.sinh(im);
      const d=r2; // divide by z: (sr+isi)/(re+iim)
      return [(sr*re+si*im)/d, (si*re-sr*im)/d];
    }
    if(fname.startsWith("1/z ")) {
      const d=re*re+im*im; if(d<1e-12) return [NaN,NaN];
      return [re/d, -im/d];
    }
    if(fname.startsWith("1/z²")) {
      const d=re*re+im*im; if(d<1e-12) return [NaN,NaN];
      const d2=d*d;
      return [(re*re-im*im)/d2, -2*re*im/d2];
```

```
  }
  if(fname.startsWith("e^{1/z}")) {
    const d=re*re+im*im; if(d<1e-12) return [NaN,NaN];
    const invRe=re/d, invIm=-im/d;
    return [Math.exp(invRe)*Math.cos(invIm), Math.exp(invRe)*Math.sin(invIm)];
  }
  return [1,0];
}

function hslToRgbSC(h,s,l) {
  h=((h%1)+1)%1;
  const c=(1-Math.abs(2*l-1))*s,x=c*(1-Math.abs((h*6)%2-1)),m=l-c/2;
  let r=0,g=0,b=0;
  if(h<1/6){r=c;g=x;}else if(h<2/6){r=x;g=c;}else if(h<3/6){g=c;b=x;}
  else if(h<4/6){g=x;b=c;}else if(h<5/6){r=x;b=c;}else{r=c;b=x;}
  return [Math.round((r+m)*255),Math.round((g+m)*255),Math.round((b+m)*255)];
}

function renderSC(fname, logHalfWidth) {
  const halfW = Math.pow(10, logHalfWidth);
  const scale = W_SC / (2 * halfW);

  const canvas = document.createElement("canvas");
  canvas.width = W_SC; canvas.height = H_SC;
  const ctx = canvas.getContext("2d");
  const img = ctx.createImageData(W_SC, H_SC); const data = img.data;

  for(let px=0; px<W_SC; px++) {
    for(let py=0; py<H_SC; py++) {
      const re = (px - CX_SC) / scale;
      const im = -(py - CY_SC) / scale;
      const [wr,wi] = evalSC(fname, re, im);
      if(!isFinite(wr)||!isFinite(wi)||isNaN(wr)) {
        const idx=(py*W_SC+px)*4; data[idx]=255;data[idx+1]=255;data[idx+2]=255;data[idx+3]=255
      }
      const arg=Math.atan2(wi,wr), mag=Math.sqrt(wr*wr+wi*wi);
      const hue=(arg/(2*Math.PI)+1)%1;
      const brightness=0.5+0.35*Math.tanh(Math.log(mag+0.001)/2);
      const [r,g,b]=hslToRgbSC(hue,0.8,brightness);
      const idx=(py*W_SC+px)*4; data[idx]=r;data[idx+1]=g;data[idx+2]=b;data[idx+3]=255;
    }
  }
  ctx.putImageData(img,0,0);

  // Axes
  ctx.strokeStyle="rgba(255,255,255,0.5)"; ctx.lineWidth=0.8;
  ctx.beginPath(); ctx.moveTo(0,H_SC/2); ctx.lineTo(W_SC,H_SC/2); ctx.stroke();
```

```
ctx.beginPath(); ctx.moveTo(W_SC/2,0); ctx.lineTo(W_SC/2,H_SC); ctx.stroke();

// Scale indicator
const tickPx = scale; // 1 unit in pixels
ctx.strokeStyle="rgba(255,255,255,0.85)"; ctx.lineWidth=2;
const ty = H_SC-18;
ctx.beginPath(); ctx.moveTo(W_SC/2-tickPx/2,ty); ctx.lineTo(W_SC/2+tickPx/2,ty); ctx.stroke();
ctx.fillStyle="rgba(255,255,255,0.9)"; ctx.font="10px sans-serif"; ctx.textAlign="center";
ctx.fillText(`|z| = ${halfW < 0.01 ? halfW.toExponential(2) : halfW.toFixed(4)}`, W_SC/2, ty-4)
ctx.fillText("← 1 unit →", W_SC/2, ty+12);
ctx.textAlign="left";

// Type annotation
const isEss = fname.startsWith("e^{1/z}");
const isRem = fname.startsWith("sin(z)/z");
let typeColor = isEss ? "#f97316" : isRem ? "#059669" : "#3b82f6";
let typeLabel = isEss ? "Essential singularity" : isRem ? "Removable singularity" : fname.starts

ctx.fillStyle="rgba(0,0,0,0.7)";
ctx.fillRect(0,0,W_SC,22);
ctx.fillStyle=typeColor; ctx.font="bold 11px sans-serif";
ctx.fillText(typeLabel, 6, 15);

// Persistent annotation for essential singularity at fine zoom
const infoDiv = document.createElement("div");
infoDiv.style.cssText = "margin-top:0.4rem; font-family:sans-serif; font-size:0.86em; padding:0

if(isEss && halfW < 0.1) {
  infoDiv.style.background="#fff7ed";
  infoDiv.style.border="1px solid #fed7aa";
  infoDiv.innerHTML = `<strong style="color:#f97316">No pattern emerges — every colour appears i
} else if(isRem) {
  infoDiv.style.background="#f0fdf4";
  infoDiv.style.border="1px solid #bbf7d0";
  infoDiv.innerHTML = `<strong style="color:#059669">Removable singularity:</strong> as you zoom
} else if(fname.startsWith("1/z²")) {
  infoDiv.style.background="#eff6ff";
  infoDiv.style.border="1px solid #bfdbfe";
  infoDiv.innerHTML = `<strong style="color:#3b82f6">Pole of order 2:</strong> the hue wheel wir
} else {
  infoDiv.style.background="#eff6ff";
  infoDiv.style.border="1px solid #bfdbfe";
  infoDiv.innerHTML = `<strong style="color:#3b82f6">Simple pole (order 1):</strong> the hue whe
}

const wrap = document.createElement("div");
wrap.appendChild(canvas); wrap.appendChild(infoDiv);
```

```
    return wrap;
  }

  const containerSC = document.createElement("div");
  containerSC.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:
  const titleSC = document.createElement("div");
  titleSC.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  titleSC.textContent = "Singularity classifier: zoom in on isolated singularities";
  containerSC.appendChild(titleSC);
  containerSC.appendChild(funcSelSC);
  containerSC.appendChild(zoomSl);
  const vizDivSC = document.createElement("div");
  containerSC.appendChild(vizDivSC);
  const noteSC = document.createElement("div");
  noteSC.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;"
  noteSC.textContent = "Domain colouring: hue = arg f, brightness = log|f| — identical to Ch1/Ch2
  containerSC.appendChild(noteSC);

  function updateSC() {
    vizDivSC.innerHTML = "";
    vizDivSC.appendChild(renderSC(funcSelSC.value, parseFloat(zoomSl.value)));
  }
  funcSelSC.addEventListener("input", updateSC);
  zoomSl.addEventListener("input", updateSC);
  updateSC();

  return containerSC;
}
```

---

## 49.4. Residues

The **residue** of $f$ at an isolated singularity $z_0$ is the Laurent coefficient $c_{-1}$:

$$\mathrm{Res}[f, z_0] = c_{-1}$$

**Why it matters.** Chapter 2 proved by direct parametrisation that $\oint_{|z-z_0|=r}(z-z_0)^n\,dz = 2\pi i$ when $n = -1$ and $= 0$ for every other integer $n$. (The same substitution $z = z_0 + re^{i\theta}$ works for all integers, not just $n = 0$ as computed there.) So when the Laurent series is integrated term by term around a small circle enclosing $z_0$, every term vanishes except the $c_{-1}$ term:

$$\oint_C \sum_{n=-\infty}^{\infty} c_n(z-z_0)^n\,dz = c_{-1}\cdot 2\pi i$$

```
//| echo: false
{
  // Why only c₋₁ survives — missed opportunity 2
  // Circle with z = r e^{iθ} travelling; integer slider n; arrows show integrand contribution
  // Right panel: accumulated contour integral as curve in C-plane — closes to 0 for n≠-1, to 2πi fo

  const W_C1 = 320, H_C1 = 300, CX_C1 = W_C1/2, CY_C1 = H_C1/2, SC_C1 = 90;
  const W_C2 = 300, H_C2 = 300, CX_C2 = W_C2/2, CY_C2 = H_C2/2;

  const nSlider = Inputs.range([-4, 3], { value: -1, step: 1, label: "Power n in z^n" });
  const tSlider = Inputs.range([0, 1], { value: 0, step: 0.004, label: "Progress t (0→1 = full circl

  const rCircle = 1.0; // circle radius |z| = r = 1

  // Integral of z^n dz around circle of radius r, from 0 to 2πt
  // z = r e^{iθ}, dz = ir e^{iθ} dθ
  // integrand = r^n e^{inθ} * ir e^{iθ} = i r^{n+1} e^{i(n+1)θ}
  function computeIntegralC1(n, t, STEPS) {
    const dtheta = 2*Math.PI*t / Math.max(STEPS,1);
    let sumRe=0, sumIm=0;
    for(let k=0;k<STEPS;k++) {
      const theta = 2*Math.PI * k*t / STEPS;
      const dthE = 2*Math.PI * t / STEPS;
      // integrand z^n dz at midpoint
      const thM = theta + dthE/2;
      const r = rCircle;
      // z^n = r^n e^{inθ}
      const znRe = Math.pow(r,n)*Math.cos(n*thM);
      const znIm = Math.pow(r,n)*Math.sin(n*thM);
      // dz = ir e^{iθ} dθ = r(-sin θ + i cos θ) dθ
      const dzRe = -r*Math.sin(thM)*dthE;
      const dzIm =  r*Math.cos(thM)*dthE;
      sumRe += znRe*dzRe - znIm*dzIm;
      sumIm += znRe*dzIm + znIm*dzRe;
    }
    return [sumRe, sumIm];
  }

  function toScrC1(re,im) { return [CX_C1+re*SC_C1, CY_C1-im*SC_C1]; }
  function toScrC2(re,im) { // C2: scale to fit result; n=-1 gives 2πi ~ 6.28
    const s2 = H_C2*0.35 / (2*Math.PI);
    return [CX_C2+re*s2, CY_C2-im*s2];
  }

  function renderC1(n, tVal) {
    const STEPS = 200;
    const canvas = document.createElement("canvas");
```

```
canvas.width=W_C1; canvas.height=H_C1;
const ctx=canvas.getContext("2d");
ctx.fillStyle="#fafafa"; ctx.fillRect(0,0,W_C1,H_C1);

// Grid
ctx.strokeStyle="#e5e7eb"; ctx.lineWidth=0.5;
for(let i=-2;i<=2;i++) {
  const [lx]=toScrC1(i,0); ctx.beginPath(); ctx.moveTo(lx,0); ctx.lineTo(lx,H_C1); ctx.stroke
  const [,ly]=toScrC1(0,i); ctx.beginPath(); ctx.moveTo(0,ly); ctx.lineTo(W_C1,ly); ctx.strok
}

// Axes
const [ox,oy]=toScrC1(0,0);
ctx.strokeStyle="#9ca3af"; ctx.lineWidth=1;
ctx.beginPath(); ctx.moveTo(0,oy); ctx.lineTo(W_C1,oy); ctx.stroke();
ctx.beginPath(); ctx.moveTo(ox,0); ctx.lineTo(ox,H_C1); ctx.stroke();

// Full circle outline
ctx.strokeStyle="rgba(156,163,175,0.4)"; ctx.lineWidth=1.5; ctx.setLineDash([4,3]);
ctx.beginPath(); ctx.arc(ox,oy,rCircle*SC_C1,0,2*Math.PI); ctx.stroke(); ctx.setLineDash([]);

// Traced portion of circle (orange for n=-1, blue otherwise)
const traceColor = n===-1 ? "#f97316" : "#3b82f6";
ctx.strokeStyle=traceColor; ctx.lineWidth=2.5;
ctx.beginPath();
for(let k=0;k<=STEPS;k++) {
  const theta = 2*Math.PI*(k/STEPS)*tVal;
  const [px,py]=toScrC1(rCircle*Math.cos(theta), rCircle*Math.sin(theta));
  k===0?ctx.moveTo(px,py):ctx.lineTo(px,py);
}
ctx.stroke();

// Arrows showing integrand direction at a few points along the traced arc
const arrowCount = 8;
for(let a=0;a<arrowCount;a++) {
  const frac = (a+0.5)/arrowCount;
  if(frac >= tVal && tVal < 0.98) continue;
  const theta = 2*Math.PI*frac;
  const [zx,zy] = [rCircle*Math.cos(theta), rCircle*Math.sin(theta)];
  // Integrand contribution direction: z^n * dz (dz direction = tangent = i*z/r)
  // direction of z^n * dz at this point
  const argZn = n * theta;
  const argDz = theta + Math.PI/2; // tangent direction
  const argContrib = argZn + argDz;
  const arrowLen = 0.28;
  const [ax1,ay1]=toScrC1(zx,zy);
  const ax2=ax1+arrowLen*SC_C1*Math.cos(argContrib);
```

```
    const ay2=ay1-arrowLen*SC_C1*Math.sin(argContrib);
    ctx.strokeStyle=n===-1?"#f97316":"rgba(59,130,246,0.7)"; ctx.lineWidth=1.5;
    ctx.beginPath(); ctx.moveTo(ax1,ay1); ctx.lineTo(ax2,ay2); ctx.stroke();
    // Arrowhead
    const headLen=7, headAngle=0.4;
    const dir=Math.atan2(ay1-ay2,ax2-ax1);
    ctx.beginPath();
    ctx.moveTo(ax2,ay2);
    ctx.lineTo(ax2-headLen*Math.cos(dir-headAngle), ay2+headLen*Math.sin(dir-headAngle));
    ctx.moveTo(ax2,ay2);
    ctx.lineTo(ax2-headLen*Math.cos(dir+headAngle), ay2+headLen*Math.sin(dir+headAngle));
    ctx.stroke();
  }

  // Current point z = re^{iθ}
  const thetaNow = 2*Math.PI*tVal;
  const [cpx,cpy]=toScrC1(rCircle*Math.cos(thetaNow), rCircle*Math.sin(thetaNow));
  ctx.fillStyle=traceColor; ctx.beginPath(); ctx.arc(cpx,cpy,6,0,2*Math.PI); ctx.fill();
  ctx.strokeStyle="#fff"; ctx.lineWidth=1.5; ctx.stroke();
  ctx.fillStyle="#374151"; ctx.font="10px sans-serif";
  ctx.fillText(`z = e^{iθ}`, cpx+8, cpy-4);

  // Origin mark
  ctx.fillStyle="#374151"; ctx.font="9px sans-serif";
  ctx.fillText("0", ox+4, oy-4);

  // Title
  ctx.fillStyle="#374151"; ctx.font="bold 10px sans-serif";
  const nLabel = n>=0?`+${n}`:n.toString();
  ctx.fillText(`Integrand: z^{${nLabel}} dz on |z|=1`, 5, 14);

  return canvas;
}

function renderC2(n, tVal) {
  const STEPS = 200;
  const canvas = document.createElement("canvas");
  canvas.width=W_C2; canvas.height=H_C2;
  const ctx=canvas.getContext("2d");
  ctx.fillStyle="#fafafa"; ctx.fillRect(0,0,W_C2,H_C2);

  // Axes
  const [ox,oy]=toScrC2(0,0);
  ctx.strokeStyle="#9ca3af"; ctx.lineWidth=1;
  ctx.beginPath(); ctx.moveTo(0,oy); ctx.lineTo(W_C2,oy); ctx.stroke();
  ctx.beginPath(); ctx.moveTo(ox,0); ctx.lineTo(ox,H_C2); ctx.stroke();
  ctx.fillStyle="#374151"; ctx.font="9px sans-serif";
```

```
ctx.fillText("Re", W_C2-20,oy-3); ctx.fillText("Im", ox+3,10);

// Mark 2πi on Im axis for n=-1
if(n===-1) {
  const [,tpiY]=toScrC2(0,2*Math.PI);
  ctx.strokeStyle="#fbbf24"; ctx.lineWidth=1; ctx.setLineDash([3,2]);
  ctx.beginPath(); ctx.moveTo(ox-5,tpiY); ctx.lineTo(ox+5,tpiY); ctx.stroke(); ctx.setLineDas
  ctx.fillStyle="#fbbf24"; ctx.font="10px sans-serif";
  ctx.fillText("2πi", ox+6, tpiY+4);
}

// Accumulated integral curve up to t
const curveColor = n===-1?"#f97316":"#3b82f6";
ctx.strokeStyle=curveColor; ctx.lineWidth=2;
ctx.beginPath();
for(let k=0;k<=STEPS;k++) {
  const tFrac = k/STEPS;
  if(tFrac > tVal + 0.001) break;
  const [ir,ii] = computeIntegralC1(n, tFrac, 80);
  const [px,py] = toScrC2(ir,ii);
  k===0?ctx.moveTo(px,py):ctx.lineTo(px,py);
}
ctx.stroke();

// Current accumulated value
const [curRe, curIm] = computeIntegralC1(n, tVal, 80);
const [cpx,cpy]=toScrC2(curRe,curIm);
ctx.fillStyle=curveColor; ctx.beginPath(); ctx.arc(cpx,cpy,5,0,2*Math.PI); ctx.fill();

// Final value annotation
if(tVal > 0.97) {
  const [fRe, fIm] = computeIntegralC1(n, 1, 200);
  const closed = Math.sqrt(fRe*fRe+fIm*fIm) < 0.1;
  if(n===-1) {
    ctx.fillStyle="#f97316"; ctx.font="bold 10px sans-serif";
    ctx.fillText("→ 2πi ✓", cpx+8, cpy-4);
  } else {
    ctx.fillStyle="#3b82f6"; ctx.font="bold 10px sans-serif";
    ctx.fillText("→ 0", cpx+8, cpy-4);
  }
}

// n=-1 gold highlight box
if(n===-1) {
  ctx.fillStyle="rgba(251,191,36,0.15)"; ctx.fillRect(ox-60,0,120,H_C2);
}
```

```
  ctx.fillStyle="#374151"; ctx.font="bold 10px sans-serif";
  ctx.fillText("∮ z^n dz (accumulating)", 4, 14);
  ctx.fillStyle=n===-1?"#f97316":"#374151"; ctx.font="10px sans-serif";
  const resultLabel = n===-1 ? "n = −1: integral → 2πi" : `n = ${n}: integral → 0`;
  ctx.fillText(resultLabel, 4, H_C2-7);

  return canvas;
}

function renderC1C2(n, tVal) {
  const c1 = renderC1(n, tVal);
  const c2 = renderC2(n, tVal);
  const panelDiv = document.createElement("div");
  panelDiv.style.cssText = "display:flex; gap:0.5rem; flex-wrap:wrap;";

  function labeled2(canvas, lbl) {
    const d=document.createElement("div");
    const l=document.createElement("div");
    l.style.cssText="font-size:0.8em;font-weight:600;text-align:center;margin-bottom:2px;color:#37
    l.textContent=lbl; d.appendChild(l); d.appendChild(canvas); return d;
  }

  panelDiv.appendChild(labeled2(c1, "Circle: z = e^{iθ}, arrows = integrand direction"));
  panelDiv.appendChild(labeled2(c2, "Accumulated integral in C"));

  const nLabel = n>=0?`+${n}`:n.toString();
  const infoDiv = document.createElement("div");
  infoDiv.style.cssText = "margin-top:0.4rem; font-family:sans-serif; font-size:0.86em;";

  if(n===-1) {
    infoDiv.style.cssText += "background:#fff7ed; border:1px solid #fed7aa; border-radius:4px; pad
    infoDiv.innerHTML = `<strong style="color:#f97316">n = −1 (the residue term):</strong> the int
  } else {
    infoDiv.style.cssText += "background:#eff6ff; border:1px solid #bfdbfe; border-radius:4px; pad
    infoDiv.innerHTML = `<strong style="color:#3b82f6">n = ${nLabel}:</strong> the integrand contr
  }

  const wrap = document.createElement("div");
  wrap.appendChild(panelDiv); wrap.appendChild(infoDiv);
  return wrap;
}

const containerC1 = document.createElement("div");
containerC1.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1re
const titleC1 = document.createElement("div");
titleC1.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
titleC1.textContent = "Why only c₋₁ survives: ∮ zⁿ dz = 2πi if n = −1, else 0";
```

```
  containerC1.appendChild(titleC1);
  containerC1.appendChild(nSlider);
  containerC1.appendChild(tSlider);
  const vizDivC1 = document.createElement("div");
  containerC1.appendChild(vizDivC1);
  const noteC1 = document.createElement("div");
  noteC1.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;"
  noteC1.textContent = "Left: the contour |z|=1 with arrows showing the direction of z^n dz at ea
  containerC1.appendChild(noteC1);

  function updateC1() {
    vizDivC1.innerHTML="";
    vizDivC1.appendChild(renderC1C2(parseInt(nSlider.value), parseFloat(tSlider.value)));
  }
  nSlider.addEventListener("input", updateC1);
  tSlider.addEventListener("input", updateC1);
  updateC1();

  return containerC1;
}
```

### 49.4.1. Computing residues

**At a simple pole:** $\mathrm{Res}[f, z_0] = \lim_{z \to z_0} (z - z_0) f(z)$.

**At a simple pole of $p/q$ where $q(z_0) = 0$, $q'(z_0) \neq 0$:**

$$\mathrm{Res}\left[\frac{p}{q}, z_0\right] = \frac{p(z_0)}{q'(z_0)}$$

This is just L'Hôpital applied to the simple-pole limit. Quick check: $\mathrm{Res}[1/(z^2 - 1), 1]$ — here $p = 1$, $q = z^2 - 1$, $q'(z) = 2z$, so the residue is $1/(2 \cdot 1) = 1/2$.

**At a pole of order $m$:**

$$\mathrm{Res}[f, z_0] = \frac{1}{(m-1)!} \lim_{z \to z_0} \frac{d^{m-1}}{dz^{m-1}} \left[(z - z_0)^m f(z)\right]$$

Quick check: $\mathrm{Res}[e^z/(z-1)^2, 1]$ — multiply by $(z-1)^2$ to get $e^z$, differentiate once (since $m = 2$, so $m - 1 = 1$), divide by $1! = 1$: $d(e^z)/dz\big|_{z=1} = e$.

---

## 49.5. The residue theorem

**Residue theorem.** Let $f$ be analytic on and inside a simple closed contour $C$ (counterclockwise) except for isolated singularities $z_1, \dots, z_k$ inside $C$. Then:

$$\oint_C f(z)\,dz = 2\pi i \sum_{j=1}^{k} \operatorname{Res}[f, z_j]$$

This is the master formula of complex analysis. Cauchy's integral formula is the special case where $f$ has only simple poles.

---

## 49.6. Evaluating real integrals

### 49.6.1. Type 1: trigonometric integrals $\int_0^{2\pi} R(\cos\theta, \sin\theta)\,d\theta$

Substitute $z = e^{i\theta}$: $\cos\theta = (z + z^{-1})/2$, $\sin\theta = (z - z^{-1})/(2i)$, $d\theta = dz/(iz)$. The integral becomes a contour integral on $|z| = 1$.

**Example.** $\displaystyle\int_0^{2\pi} \frac{d\theta}{3 + 2\cos\theta}$. Substituting gives $\oint_{|z|=1} \frac{dz}{iz(3+z+z^{-1})} = \oint_{|z|=1} \frac{dz}{i(z^2+3z+1)}$. The roots of $z^2 + 3z + 1$ are $z = (-3 \pm \sqrt{5})/2$; only $z_1 = (-3 + \sqrt{5})/2 \approx -0.38$ lies inside $|z| = 1$. Residue: $1/(i(2z_1 + 3)) = 1/(i\sqrt{5})$. Integral: $2\pi i \cdot 1/(i\sqrt{5}) = 2\pi/\sqrt{5}$. Exercise 5 follows the same pattern.

### 49.6.2. Type 2: improper integrals $\int_{-\infty}^{\infty} R(x)\,dx$

Close the contour with a large semicircle in the upper (or lower) half-plane. The semicircular arc's contribution vanishes as the radius grows — but the condition depends on the form of the integrand. For a plain rational $R(x)$ with no oscillatory factor, the ML estimation lemma guarantees the arc vanishes when the degree of the denominator exceeds the degree of the numerator by at least **2**. When the integrand takes the form $e^{iax}R(x)$ with $a > 0$ (for example, $e^{ix}/(1 + x^2)$, whose real part gives the hook integral), **Jordan's lemma** applies and degree excess of at least **1** suffices. The residue theorem then gives the real integral as $2\pi i$ times the sum of residues in the upper half-plane.

**Example.** $\displaystyle\int_{-\infty}^{\infty} \frac{dx}{1 + x^2}$. Poles of $1/(1 + z^2)$: $z = \pm i$. Only $z = i$ is in the upper half-plane. Residue: $1/(2i)$. Integral: $2\pi i \cdot 1/(2i) = \pi$.

The hook integral $\int_{-\infty}^{\infty} \cos x/(1 + x^2)\,dx$ uses the same idea with one extra step: write $\cos x = \operatorname{Re}(e^{ix})$, so the integral equals $\operatorname{Re}\int_{-\infty}^{\infty} e^{ix}/(1 + x^2)\,dx$. The integrand is now of the form $e^{iax}R(x)$ with $a = 1$, so Jordan's lemma applies. The pole at $z = i$ gives residue $e^{i \cdot i}/(2i) = e^{-1}/(2i)$, and the integral equals $\operatorname{Re}(2\pi i \cdot e^{-1}/(2i)) = \pi/e$.

```
//| echo: false
{
  const W = 400, H = 360, CX = W/2, CY = H/2, SCALE = 70;
  const svgNS = "http://www.w3.org/2000/svg";

  const funcData = {
    "1/(1+z²)": {
      poles: [{re:0,im:1,res:{re:0,im:-0.5},label:"i"},{re:0,im:-1,res:{re:0,im:0.5},label:"−i"}]
      realIntegral: "π",
      realValue: Math.PI,
      eval: (re,im) => {
        const dr=re, di=im+1, d=dr*dr+di*di;
        if(d<1e-8) return [NaN,NaN];
        const dr2=re, di2=im-1, d2=dr2*dr2+di2*di2;
        // 1/(z²+1) = 1/((z+i)(z-i))
        // just colour: use |f|
        // f = 1/(1+z^2): real = (1-y^2+x^2-2x^2)/... let's compute directly
        const r2 = 1 + re*re - im*im; const i2 = 2*re*im;
        const den = r2*r2 + i2*i2;
        if(den<1e-10) return [NaN,NaN];
        return [r2/den, -i2/den];
      }
    },
    "1/((z²+1)(z²+4))": {
      poles: [
        {re:0,im:1,res:{re:0,im:-1/6},label:"i"},
        {re:0,im:-1,res:{re:0,im:1/6},label:"−i"},
        {re:0,im:2,res:{re:0,im:1/12},label:"2i"},
        {re:0,im:-2,res:{re:0,im:-1/12},label:"−2i"}
      ],
      realIntegral: "π/6",
      realValue: Math.PI/6,
      eval: (re,im) => {
        // f = 1/((z^2+1)(z^2+4))
        const a1=re*re-im*im+1, b1=2*re*im, a2=re*re-im*im+4, b2=2*re*im;
        const ra=a1*a2-b1*b2, ia=a1*b2+a2*b1;
        const d=ra*ra+ia*ia; if(d<1e-10) return [NaN,NaN];
        return [ra/d,-ia/d];
      }
    },
    "z²/((z²−1)(z+2))": {
      poles: [
        {re:1,im:0,res:{re:1/6,im:0},label:"1"},
        {re:-1,im:0,res:{re:-1/2,im:0},label:"−1"},
        {re:-2,im:0,res:{re:4/3,im:0},label:"−2"}
      ],
      realIntegral: "(sum of residues)",
```

```
    realValue: null,
    eval: (re,im) => {
      const z2r=re*re-im*im, z2i=2*re*im;
      const d1r=z2r-1,d1i=z2i;
      const d2r=re+2,d2i=im;
      const d12r=d1r*d2r-d1i*d2i, d12i=d1r*d2i+d1i*d2r;
      const nd=d12r*d12r+d12i*d12i; if(nd<1e-10) return [NaN,NaN];
      return [(z2r*d12r+z2i*d12i)/nd,(z2i*d12r-z2r*d12i)/nd];
    }
  }
};

const funcSelector = Inputs.select(Object.keys(funcData), {
  value: "1/(1+z²)", label: "f(z)"
});
const radiusSlider = Inputs.range([0.1, 4.5], {value: 1.5, step: 0.05, label: "Contour radius R"})

function hslToRgb(h,s,l) {
  h=((h%1)+1)%1;
  const c=(1-Math.abs(2*l-1))*s,x=c*(1-Math.abs((h*6)%2-1)),m=l-c/2;
  let r=0,g=0,b=0;
  if(h<1/6){r=c;g=x;}else if(h<2/6){r=x;g=c;}else if(h<3/6){g=c;b=x;}
  else if(h<4/6){g=x;b=c;}else if(h<5/6){r=x;b=c;}else{r=c;b=x;}
  return [Math.round((r+m)*255),Math.round((g+m)*255),Math.round((b+m)*255)];
}

function toScreen(re,im) { return [CX+re*SCALE, CY-im*SCALE]; }

function render3(fname, R) {
  const fd = funcData[fname];

  const canvas = document.createElement("canvas");
  canvas.width=W; canvas.height=H;
  const ctx=canvas.getContext("2d");
  const img=ctx.createImageData(W,H); const data=img.data;

  for (let px=0;px<W;px++) {
    for (let py=0;py<H;py++) {
      const re=(px-CX)/SCALE, im=-(py-CY)/SCALE;
      const [wr,wi]=fd.eval(re,im);
      if(!isFinite(wr)||!isFinite(wi)) {
        const idx=(py*W+px)*4;
        data[idx]=255;data[idx+1]=255;data[idx+2]=255;data[idx+3]=255;
        continue;
      }
      const arg=Math.atan2(wi,wr), mag=Math.sqrt(wr*wr+wi*wi);
      const hue=(arg/(2*Math.PI)+1)%1;
```

```
      const logmag=Math.log(mag+0.001);
      const brightness=0.5+0.35*Math.tanh(logmag/2);
      const [r,g,b]=hslToRgb(hue,0.75,brightness);
      const idx=(py*W+px)*4;
      data[idx]=r;data[idx+1]=g;data[idx+2]=b;data[idx+3]=255;
    }
  }
  ctx.putImageData(img,0,0);

  // Contour
  const [cx0,cy0]=toScreen(0,0);
  ctx.strokeStyle="rgba(255,255,255,0.9)"; ctx.lineWidth=2;
  ctx.beginPath(); ctx.arc(cx0,cy0,R*SCALE,0,2*Math.PI); ctx.stroke();
  ctx.fillStyle="rgba(255,255,255,0.8)"; ctx.font="11px sans-serif";
  ctx.fillText(`C: |z| = ${R.toFixed(2)}`,5,14);

  // Pole markers, labels, and winding number indicators
  const enclosed = [];
  fd.poles.forEach(p => {
    const [psx,psy]=toScreen(p.re,p.im);
    const inC = Math.sqrt(p.re*p.re+p.im*p.im) < R;
    if(inC) enclosed.push(p);
    ctx.strokeStyle=inC?"#fbbf24":"rgba(255,255,255,0.7)";
    ctx.lineWidth=2;
    ctx.beginPath(); ctx.moveTo(psx-7,psy-7); ctx.lineTo(psx+7,psy+7); ctx.stroke();
    ctx.beginPath(); ctx.moveTo(psx+7,psy-7); ctx.lineTo(psx-7,psy+7); ctx.stroke();
    ctx.fillStyle=inC?"#fbbf24":"rgba(255,255,255,0.7)";
    ctx.font="10px sans-serif";
    ctx.fillText(p.label,psx+9,psy+4);
    // Winding number badge — n=1 for each simple pole inside the contour
    if(inC) {
      ctx.fillStyle="#1e40af"; ctx.beginPath(); ctx.arc(psx-12,psy-12,10,0,2*Math.PI); ctx.fill
      ctx.fillStyle="white"; ctx.font="bold 10px sans-serif"; ctx.textAlign="center";
      ctx.fillText("n=1", psx-12, psy-8);
      ctx.textAlign="left";
      // Small label connecting to Ch1/Ch2
      ctx.fillStyle="#fbbf24"; ctx.font="9px sans-serif";
      ctx.fillText("winding=1", psx+9, psy+16);
    }
  });

  // Residue sum
  let sumRe=0, sumIm=0;
  enclosed.forEach(p => { sumRe+=p.res.re; sumIm+=p.res.im; });
  const integral2piRe = -2*Math.PI*sumIm;
  const integral2piIm = 2*Math.PI*sumRe;
```

```
  const infoDiv=document.createElement("div");
  infoDiv.style.cssText="margin-top:0.5rem; font-family:sans-serif; font-size:0.85em; background:#
  const encStr=enclosed.length===0?"none":enclosed.map(p=>p.label).join(", ");
  const resStr=enclosed.length===0?"0":(`${sumRe.toFixed(4)}${sumIm>=0?"+":""}${sumIm.toFixed(4)}i
  const intStr=enclosed.length===0?"0":`${integral2piRe.toFixed(4)} + ${integral2piIm.toFixed(4)}i
  const realStr=(fd.realValue!==null&&enclosed.some(p=>p.im>0))
    ?`<div style="color:#059669">Real integral = \u22122\u03c0 \xd7 Im(\u03a3Res) = ${integral2piR
    :"";
  const windingNote = enclosed.length>0
    ? `<div style="color:#1e40af; font-size:0.9em; margin-top:0.3rem;">Each enclosed pole has wind
    : "";
  infoDiv.innerHTML=`
    <div><strong>Enclosed poles:</strong> ${encStr}</div>
    <div><strong>Sum of residues:</strong> ${resStr}</div>
    <div><strong>\u222e<sub>C</sub> f\u202fdz = 2\u03c0i \xd7 \u03a3Res = </strong><span style="co
    ${realStr}
    ${windingNote}
  `;

  const wrap=document.createElement("div");
  wrap.appendChild(canvas);
  wrap.appendChild(infoDiv);
  return wrap;
}

const container=document.createElement("div");
container.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem 0
const title=document.createElement("div");
title.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
title.textContent="Residue theorem explorer: sweeping poles with the contour";
container.appendChild(title);
container.appendChild(funcSelector);
container.appendChild(radiusSlider);

const vizDiv3=document.createElement("div");
container.appendChild(vizDiv3);

const note=document.createElement("div");
note.style.cssText="margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
note.textContent="Domain colouring shows f(z): poles appear as white blooms. Gold \u00d7 marks = p
container.appendChild(note);

function update3() {
  vizDiv3.innerHTML="";
  vizDiv3.appendChild(render3(funcSelector.value, radiusSlider.value));
}
funcSelector.addEventListener("input", update3);
```

```
  radiusSlider.addEventListener("input", update3);
  update3();

  return container;
}
```

```
//| echo: false
{
  // Semicircle contour closure — missed opportunity 3
  // Real axis + growing R semicircle, domain colouring background, poles marked distinctly
  // ML bound vs Jordan's lemma toggle, default function 1/(1+z²)
  // Colour palette: orange real axis, blue semicircle, red × poles, green result box

  const W_SM = 440, H_SM = 320, CX_SM = W_SM/2, CY_SM = H_SM - 50, SCALE_SM = 65;

  const funcSelSM = Inputs.select([
    "1/(1+z²)   — poles at ±i",
    "1/((z²+1)(z²+4))   — poles at ±i, ±2i",
    "e^{iz}/(1+z²)   — Jordan's lemma case"
  ], { value: "1/(1+z²)   — poles at ±i", label: "f(z)" });

  const rSliderSM = Inputs.range([0.5, 4.5], { value: 1.5, step: 0.05, label: "Semicircle radius

  const lemmaSel = Inputs.select(["ML bound", "Jordan's lemma"], {
    value: "ML bound", label: "Arc vanishing argument"
  });

  const smFuncData = {
    "1/(1+z²)   — poles at ±i": {
      poles: [{re:0,im:1,res:{re:0,im:-0.5},label:"i",half:"upper"},
              {re:0,im:-1,res:{re:0,im:0.5},label:"−i",half:"lower"}],
      eval: (re,im)=>{const a=1+re*re-im*im,b=2*re*im;const d=a*a+b*b;if(d<1e-10)return[NaN,NaN];
      realResult:"□", realVal:Math.PI,
      mlDegExcess: 2, jordanApplies: false,
      mlText: "Denominator deg 2, numerator deg 0 — excess 2 □ 2. ML bound: |1/(1+z²)| □ 1/(R²−1)
      jordanText: "Jordan's lemma requires an e^{iaz} oscillatory factor — not present here. Use
      note: "Upper half-plane: only z=i enclosed."
    },
    "1/((z²+1)(z²+4))   — poles at ±i, ±2i": {
      poles: [{re:0,im:1,res:{re:0,im:-1/6},label:"i",half:"upper"},
              {re:0,im:-1,res:{re:0,im:1/6},label:"−i",half:"lower"},
              {re:0,im:2,res:{re:0,im:1/12},label:"2i",half:"upper"},
              {re:0,im:-2,res:{re:0,im:-1/12},label:"−2i",half:"lower"}],
      eval: (re,im)=>{
        const a1=re*re-im*im+1,b1=2*re*im,a2=re*re-im*im+4,b2=2*re*im;
        const ra=a1*a2-b1*b2,ia=a1*b2+a2*b1;const d=ra*ra+ia*ia;if(d<1e-10)return[NaN,NaN];
        return[ra/d,-ia/d];
```

```
      },
      realResult:"π/6", realVal:Math.PI/6,
      mlDegExcess: 4, jordanApplies: false,
      mlText: "Denominator deg 4, numerator deg 0 — excess 4 ≫ 2. Arc integral vanishes via ML bound
      jordanText: "No oscillatory factor — Jordan's lemma does not apply. ML bound sufficient.",
      note: "Upper half-plane: poles at i and 2i enclosed when R > 2."
    },
    "e^{iz}/(1+z²)   — Jordan's lemma case": {
      poles: [{re:0,im:1,res:{re:0,im:-0.5*Math.E/Math.E,re_override:Math.exp(-1)/(2)},label:"i",hal
              {re:0,im:-1,res:{re:0,im:0.5},label:"−i",half:"lower"}],
      eval: (re,im)=>{
        // e^{iz} * 1/(1+z^2)
        // e^{iz} = e^{i(re+iim)} = e^{-im}*(cos(re)+i*sin(re))
        const eR=Math.exp(-im)*Math.cos(re), eI=Math.exp(-im)*Math.sin(re);
        const a=1+re*re-im*im,b=2*re*im;const d=a*a+b*b;if(d<1e-10)return[NaN,NaN];
        return[(eR*a+eI*b)/d,(eI*a-eR*b)/d];
      },
      realResult:"π/e (real part)", realVal:Math.PI*Math.exp(-1),
      mlDegExcess: 1, jordanApplies: true,
      mlText: "Degree excess = 1. ML bound alone gives |arc| ≤ πR/(R²−1) → 0 only at O(1/R) — barely
      jordanText: "Jordan's lemma applies: integrand = e^{iz}·R(z) with R(z)→0 uniformly. Arc integr
      note: "Re(∫e^{ix}/(1+x²)dx) = π/e — the hook integral in the chapter."
    }
  };

  function hslToRgbSM(h,s,l) {
    h=((h%1)+1)%1;
    const c=(1-Math.abs(2*l-1))*s,x=c*(1-Math.abs((h*6)%2-1)),m=l-c/2;
    let r=0,g=0,b=0;
    if(h<1/6){r=c;g=x;}else if(h<2/6){r=x;g=c;}else if(h<3/6){g=c;b=x;}
    else if(h<4/6){g=x;b=c;}else if(h<5/6){r=x;b=c;}else{r=c;b=x;}
    return [Math.round((r+m)*255),Math.round((g+m)*255),Math.round((b+m)*255)];
  }

  function toScrSM(re,im) { return [CX_SM+re*SCALE_SM, CY_SM-im*SCALE_SM]; }

  function renderSM(fname, R, lemmaMode) {
    const fd = smFuncData[fname];

    const canvas = document.createElement("canvas");
    canvas.width=W_SM; canvas.height=H_SM;
    const ctx=canvas.getContext("2d");
    const img=ctx.createImageData(W_SM,H_SM); const data=img.data;

    // Domain colouring — only upper half-plane
    for(let px=0;px<W_SM;px++) {
      for(let py=0;py<H_SM;py++) {
```

```
        const re=(px-CX_SM)/SCALE_SM, im=-(py-CY_SM)/SCALE_SM;
        if(im < -0.1) {
          const idx=(py*W_SM+px)*4; data[idx]=245;data[idx+1]=245;data[idx+2]=245;data[idx+3]=255
        }
        const [wr,wi]=fd.eval(re,im);
        if(!isFinite(wr)||!isFinite(wi)||isNaN(wr)) {
          const idx=(py*W_SM+px)*4; data[idx]=255;data[idx+1]=255;data[idx+2]=255;data[idx+3]=255
        }
        const arg=Math.atan2(wi,wr),mag=Math.sqrt(wr*wr+wi*wi);
        const hue=(arg/(2*Math.PI)+1)%1;
        const brightness=0.5+0.35*Math.tanh(Math.log(mag+0.001)/2);
        // Dim below real axis
        const dimFactor = im < 0 ? 0.4 : 1.0;
        const [r,g,b]=hslToRgbSM(hue,0.8*dimFactor,0.5+(brightness-0.5)*dimFactor);
        const idx=(py*W_SM+px)*4; data[idx]=r;data[idx+1]=g;data[idx+2]=b;data[idx+3]=255;
      }
    }
    ctx.putImageData(img,0,0);

    const [ox,oy]=toScrSM(0,0);

    // Real axis (orange)
    ctx.strokeStyle="#f97316"; ctx.lineWidth=3;
    const [rx1]=toScrSM(-R-0.3,0); const [rx2]=toScrSM(R+0.3,0);
    ctx.beginPath(); ctx.moveTo(rx1,oy); ctx.lineTo(rx2,oy); ctx.stroke();
    // Arrow at right end
    ctx.fillStyle="#f97316";
    ctx.beginPath(); ctx.moveTo(rx2,oy); ctx.lineTo(rx2-10,oy-5); ctx.lineTo(rx2-10,oy+5); ctx.fi
    ctx.fillStyle="#f97316"; ctx.font="10px sans-serif"; ctx.fillText("real axis (−R to R)", rx1+

    // Semicircle (blue)
    ctx.strokeStyle="#3b82f6"; ctx.lineWidth=2.5;
    ctx.beginPath(); ctx.arc(ox,oy,R*SCALE_SM,Math.PI,0,false); ctx.stroke();
    // Arrow on semicircle
    const arcAngle = Math.PI/4;
    const arcX=ox+R*SCALE_SM*Math.cos(arcAngle), arcY=oy-R*SCALE_SM*Math.sin(arcAngle);
    ctx.strokeStyle="#3b82f6"; ctx.lineWidth=2;
    ctx.beginPath(); ctx.moveTo(arcX,arcY); ctx.lineTo(arcX-8,-8+arcY); ctx.lineTo(arcX+5,arcY-10
    ctx.fillStyle="#3b82f6"; ctx.font="10px sans-serif";
    ctx.fillText(`⍰_R (R=${R.toFixed(2)})`, arcX+8, arcY-5);

    // R label on real axis
    const [rPx]=toScrSM(R,0); const [nrPx]=toScrSM(-R,0);
    ctx.fillStyle="#f97316"; ctx.font="10px sans-serif";
    ctx.fillText("R", rPx+3, oy-6); ctx.fillText("−R", nrPx-20, oy-6);

    // Poles
```

```
    fd.poles.forEach(p => {
      const [psx,psy]=toScrSM(p.re,p.im);
      const inUpper = p.im > 0;
      const enclosed = inUpper && Math.sqrt(p.re*p.re+p.im*p.im) < R;

      ctx.strokeStyle=enclosed?"#fbbf24":"#dc2626"; ctx.lineWidth=2.5;
      ctx.beginPath(); ctx.moveTo(psx-8,psy-8); ctx.lineTo(psx+8,psy+8); ctx.stroke();
      ctx.beginPath(); ctx.moveTo(psx+8,psy-8); ctx.lineTo(psx-8,psy+8); ctx.stroke();
      ctx.fillStyle=enclosed?"#fbbf24":"#dc2626"; ctx.font="10px sans-serif";
      ctx.fillText(p.label, psx+10, psy+4);

      // Winding number badge for enclosed
      if(enclosed) {
        ctx.fillStyle="#1e40af"; ctx.beginPath(); ctx.arc(psx-14,psy-14,11,0,2*Math.PI); ctx.fill()
        ctx.fillStyle="white"; ctx.font="bold 9px sans-serif"; ctx.textAlign="center";
        ctx.fillText("n=1", psx-14, psy-10);
        ctx.textAlign="left";
      }
    });

    // Axis labels
    ctx.strokeStyle="rgba(255,255,255,0.5)"; ctx.lineWidth=0.8;
    ctx.beginPath(); ctx.moveTo(ox,0); ctx.lineTo(ox,H_SM); ctx.stroke();
    ctx.fillStyle="rgba(200,200,200,0.9)"; ctx.font="9px sans-serif";
    ctx.fillText("i", ox+3, oy-SCALE_SM+3); ctx.fillText("2i", ox+3, oy-2*SCALE_SM+3);

    // Result computation
    const encPoles = fd.poles.filter(p => p.im>0 && Math.sqrt(p.re*p.re+p.im*p.im)<R);
    let sumRe=0,sumIm=0;
    encPoles.forEach(p=>{sumRe+=p.res.re;sumIm+=p.res.im;});
    const intRe=-2*Math.PI*sumIm, intIm=2*Math.PI*sumRe;

    // Result box (green)
    if(encPoles.length>0) {
      ctx.fillStyle="rgba(5,150,105,0.92)";
      ctx.beginPath(); if(ctx.roundRect) ctx.roundRect(5,5,205,46,5); else ctx.rect(5,5,205,46); ctx
      ctx.fillStyle="white"; ctx.font="bold 10px sans-serif";
      ctx.fillText("∮_C f dz = 2πi × ΣRes", 10, 20);
      ctx.font="10px sans-serif";
      const resultStr=`= ${intRe.toFixed(4)} + ${intIm.toFixed(4)}i`;
      ctx.fillText(resultStr, 10, 35);
      if(fd.realVal&&encPoles.some(p=>p.im>0)) {
        ctx.fillStyle="#bbf7d0"; ctx.font="9px sans-serif";
        ctx.fillText(`Real integral ≈ ${Math.abs(intRe).toFixed(4)} [exact: ${fd.realResult}]`, 10,
      }
    }
```

```
    // Lemma annotation
    const lemmaText = lemmaMode==="Jordan's lemma" ? fd.jordanText : fd.mlText;
    const lemmaDiv = document.createElement("div");
    lemmaDiv.style.cssText="margin-top:0.4rem; font-family:sans-serif; font-size:0.83em; padding:
    lemmaDiv.innerHTML=`<strong>${lemmaMode}:</strong> ${lemmaText}`;

    const infoDiv = document.createElement("div");
    infoDiv.style.cssText="margin-top:0.4rem; font-family:sans-serif; font-size:0.84em;";
    const encStr=encPoles.length===0?"none (R too small)":encPoles.map(p=>p.label).join(", ");
    infoDiv.innerHTML=`<strong>Enclosed poles (upper half-plane):</strong> ${encStr} — <em>${fd.n

    const wrap=document.createElement("div");
    wrap.appendChild(canvas); wrap.appendChild(infoDiv); wrap.appendChild(lemmaDiv);
    return wrap;
  }

  const containerSM=document.createElement("div");
  containerSM.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r
  const titleSM=document.createElement("div");
  titleSM.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  titleSM.textContent="Semicircle contour closure: real integrals via the upper half-plane";
  containerSM.appendChild(titleSM);
  containerSM.appendChild(funcSelSM);
  containerSM.appendChild(rSliderSM);
  containerSM.appendChild(lemmaSel);
  const vizDivSM=document.createElement("div");
  containerSM.appendChild(vizDivSM);
  const noteSM=document.createElement("div");
  noteSM.style.cssText="margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  noteSM.textContent="Orange line = real axis −R to R; blue arc = semicircle ?_R. Domain colourin
  containerSM.appendChild(noteSM);

  function updateSM(){
    vizDivSM.innerHTML="";
    vizDivSM.appendChild(renderSM(funcSelSM.value, parseFloat(rSliderSM.value), lemmaSel.value));
  }
  funcSelSM.addEventListener("input",updateSM);
  rSliderSM.addEventListener("input",updateSM);
  lemmaSel.addEventListener("input",updateSM);
  updateSM();

  return containerSM;
}
```

The residue theorem and Laurent series complete the core tools of complex integration. The next chapter takes a different direction: conformal mapping uses analytic functions to deform geometry — turning complicated domains into simple ones where Laplace's equation can be solved directly.

## 49.7. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align-it
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell); row.appendChild(eqCell);
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "Solution complete.";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
```

```
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSt
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons()
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn); controls.appendChild(resetBtn); controls.appendChild(counter);
  container.appendChild(stepsDiv); container.appendChild(controls);
  renderSteps(); updateButtons();
  return container;
}
```

---

### 49.7.1. Exercise 1: Laurent series around a pole

Find the Laurent series of $f(z) = \dfrac{1}{z(z-1)}$ valid for $0 < |z| < 1$, and classify the singularity at $z = 0$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Partial fractions",
    eq: "\\frac{1}{z(z-1)} = \\frac{A}{z} + \\frac{B}{z-1} \\implies A = -1,\\; B = 1",
    note: "Multiply both sides by z: at z=0, A = 1/(0−1) = −1. Multiply by (z−1): at z=1, B = 1/1
  },
  {
    op: "Expand 1/(z−1) using geometric series for |z| < 1",
    eq: "\\frac{1}{z-1} = \\frac{-1}{1-z} = -\\sum_{n=0}^{\\infty}z^n"
  },
  {
    op: "Combine",
    eq: "\\frac{1}{z(z-1)} = -\\frac{1}{z} - 1 - z - z^2 - z^3 - \\cdots \\quad (0 < |z| < 1)"
  },
  {
    op: "Classify the singularity at z = 0",
    eq: "\\text{Principal part: } -1/z \\text{ (one term)} \\implies \\text{simple pole of order
    note: "The residue at z = 0 is the coefficient c_{−1} = −1."
```

```
  }
])
```

---

### 49.7.2. Exercise 2: Classifying $\sin z/z^2$ at $z = 0$

```
//| echo: false
makeStepperHTML([
  {
    op: "Expand sin z in Taylor series",
    eq: "\\sin z = z - \\frac{z^3}{6} + \\frac{z^5}{120} - \\cdots"
  },
  {
    op: "Divide by z²",
    eq: "\\frac{\\sin z}{z^2} = \\frac{1}{z} - \\frac{z}{6} + \\frac{z^3}{120} - \\cdots"
  },
  {
    op: "Principal part has one term: 1/z",
    eq: "\\text{Simple pole at } z = 0, \\quad \\text{Res}\\!\\left[\\frac{\\sin z}{z^2},\\, 0\\righ
  },
  {
    op: "Compare with sin(z)/z and sin(z)/z³",
    eq: "\\frac{\\sin z}{z} \\to 1 \\text{ (removable)}, \\quad \\frac{\\sin z}{z^3} \\text{ has pri
    note: "The order of the pole equals (order of zero of denominator) − (order of zero of numerator
  }
])
```

---

### 49.7.3. Exercise 3: Residue at a simple pole

Find $\operatorname{Res}\left[\dfrac{z^2+1}{(z-2)(z+3)},\ z = 2\right]$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Apply the simple-pole formula",
    eq: "\\text{Res}[f, 2] = \\lim_{z\\to 2}(z-2)f(z) = \\lim_{z\\to 2}\\frac{z^2+1}{z+3}"
  },
  {
    op: "Evaluate at z = 2",
    eq: "= \\frac{4+1}{2+3} = \\frac{5}{5} = 1"
```

```
  },
  {
    op: "Alternatively, use p/q□ formula with q = (z−2)(z+3)",
    eq: "q'(z) = 2z+1, \\quad q'(2) = 5, \\quad p(2) = 5 \\implies \\text{Res} = 5/5 = 1 \\checkm
  }
])
```

---

### 49.7.4. Exercise 4: Residue at a double pole

Find $\operatorname{Res}\left[\dfrac{e^z}{(z-1)^2},\ z=1\right]$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Apply the order-m formula with m = 2",
    eq: "\\text{Res}[f,1] = \\frac{1}{(2-1)!}\\lim_{z\\to 1}\\frac{d}{dz}\\left[(z-1)^2\\cdot\\fr
  },
  {
    op: "Evaluate",
    eq: "\\frac{d}{dz}e^z\\bigg|_{z=1} = e^1 = e"
  },
  {
    op: "Verify via Laurent expansion",
    eq: "\\frac{e^z}{(z-1)^2} = \\frac{e\\,e^{z-1}}{(z-1)^2} = \\frac{e}{(z-1)^2}\\left[1 + (z-1)
    note: "Coefficient of (z−1)^{−1} is e. □"
  }
])
```

---

### 49.7.5. Exercise 5: Trigonometric integral

Evaluate $\displaystyle\int_0^{2\pi} \frac{d\theta}{2+\cos\theta}$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Substitute z = e^{i□}: cos □ = (z+1/z)/2, d□ = dz/(iz)",
    eq: "\\int_0^{2\\pi}\\frac{d\\theta}{2+\\cos\\theta} = \\oint_{|z|=1}\\frac{1}{2+\\frac{z+z^{
  },
  {
```

```
    op: "Find roots of z²+4z+1 = 0",
    eq: "z = \\frac{-4\\pm\\sqrt{12}}{2} = -2\\pm\\sqrt{3}",
    note: "z₁ = −2+√3 ≈ −0.27 (inside |z|=1); z₂ = −2−√3 ≈ −3.73 (outside)."
  },
  {
    op: "Compute Res[2/(i(z²+4z+1)), z₁]",
    eq: "\\text{Res} = \\frac{2}{i(2z_1+4)} = \\frac{2}{i\\cdot 2\\sqrt{3}} = \\frac{1}{i\\sqrt{3}}
  },
  {
    op: "Apply the residue theorem",
    eq: "\\int_0^{2\\pi}\\frac{d\\theta}{2+\\cos\\theta} = 2\\pi i\\cdot\\frac{-i}{\\sqrt{3}} = \\fr
  }
])
```

---

### 49.7.6. Exercise 6: Improper real integral via residues

Evaluate $\displaystyle\int_{-\infty}^{\infty} \frac{dx}{(1+x^2)^2}$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Poles of 1/(1+z²)² at z = ±i, each of order 2",
    eq: "\\frac{1}{(1+z^2)^2} = \\frac{1}{(z-i)^2(z+i)^2}"
  },
  {
    op: "Close contour with upper semicircle; only z = i is in upper half-plane",
    eq: "\\text{Res}\\left[\\frac{1}{(z-i)^2(z+i)^2},\\,i\\right] = \\lim_{z\\to i}\\frac{d}{dz}\\fr
  },
  {
    op: "Differentiate",
    eq: "\\frac{d}{dz}(z+i)^{-2} = -2(z+i)^{-3} \\implies \\text{Res} = \\frac{-2}{(2i)^3} = \\frac
  },
  {
    op: "Apply residue theorem (semicircle integral → 0 as R → ∞)",
    eq: "\\int_{-\\infty}^{\\infty}\\frac{dx}{(1+x^2)^2} = 2\\pi i\\cdot\\frac{-i}{4} = \\frac{2\\pi
  }
])
```

# 50. Conformal mapping

Angle-preserving transformations and applications to potential theory

## 50.1. Conformal mappings

An analytic function $w = f(z)$ is **conformal at** $z_0$ if $f'(z_0) \neq 0$. At such a point, the mapping:

1. **Preserves angles** between curves (in both magnitude and orientation).
2. **Scales uniformly**: all infinitesimal lengths at $z_0$ are multiplied by $|f'(z_0)|$.
3. **Rotates uniformly**: all directions at $z_0$ are rotated by $\arg f'(z_0)$.

*Why angles are preserved.* If two curves $\gamma_1$ and $\gamma_2$ meet at $z_0$ at angle $\alpha$, their images under $f$ meet at $f(z_0)$ at the same angle. The chain rule gives the tangent direction of the image curve as $f'(z_0) \cdot \gamma'(t_0)$ — the original tangent vector multiplied by $f'(z_0)$. Write $f'(z_0) = |f'(z_0)|e^{i\varphi}$ where $\varphi = \arg f'(z_0)$. Multiplying a complex number by $e^{i\varphi}$ rotates it by $\varphi$ and multiplying by $|f'(z_0)|$ scales it. Both curves get the same rotation by $\varphi$, so the angle between them is unchanged.

```
//| echo: false
{
  // --- Conformal angle-preservation explorer ---
  // Dual panel: left z-plane (draggable z0, two cross-arms), right w-plane (images).
  // Domain colouring background in w-panel. Live angle readout + critical point detection.

  const svgNS = "http://www.w3.org/2000/svg";
  const W = 300, H = 280, SCALE = 65;
  const CX = W / 2, CY = H / 2;

  // --- Complex arithmetic helpers ---
  function cadd([ar, ai], [br, bi]) { return [ar + br, ai + bi]; }
  function csub([ar, ai], [br, bi]) { return [ar - br, ai - bi]; }
  function cmul([ar, ai], [br, bi]) { return [ar*br - ai*bi, ar*bi + ai*br]; }
  function cdiv([ar, ai], [br, bi]) {
    const d = br*br + bi*bi;
    return [(ar*br + ai*bi)/d, (ai*br - ar*bi)/d];
  }
  function cexp([r, i]) { const er = Math.exp(r); return [er*Math.cos(i), er*Math.sin(i)]; }
  function csin([r, i]) {
    return [Math.sin(r)*Math.cosh(i), Math.cos(r)*Math.sinh(i)];
```

```
}
function cconj([r, i]) { return [r, -i]; }

// Functions and their derivatives
const funcs = {
  "z²":    { f: ([r,i]) => cmul([r,i],[r,i]),
             fp: ([r,i]) => cmul([2,0],[r,i]) },
  "e⍰":    { f: ([r,i]) => cexp([r,i]),
             fp: ([r,i]) => cexp([r,i]) },
  "sin z": { f: ([r,i]) => csin([r,i]),
             fp: ([r,i]) => [Math.cos(r)*Math.cosh(i), -Math.sin(r)*Math.sinh(i)] },
  "1/z":   { f: ([r,i]) => { const d=r*r+i*i; return d<1e-12?[NaN,NaN]:[r/d,-i/d]; },
             fp: ([r,i]) => { const d=r*r+i*i; return d<1e-12?[NaN,NaN]:[(i*i-r*r)/(d*d),(-2*r*i
  "z̄ (reflection)": { f: ([r,i]) => cconj([r,i]),
             fp: ([r,i]) => [NaN, NaN] }   // not analytic
};

// HSL helper (same formula as Ch 1)
function hslToRgb(h, s, l) {
  let r, g, b;
  if (s === 0) { r = g = b = l; }
  else {
    const hue2rgb = (p, q, t) => {
      if (t < 0) t += 1; if (t > 1) t -= 1;
      if (t < 1/6) return p + (q-p)*6*t;
      if (t < 1/2) return q;
      if (t < 2/3) return p + (q-p)*(2/3-t)*6;
      return p;
    };
    const q = l < 0.5 ? l*(1+s) : l+s-l*s;
    const p = 2*l-q;
    r = hue2rgb(p,q,h+1/3); g = hue2rgb(p,q,h); b = hue2rgb(p,q,h-1/3);
  }
  return [Math.round(r*255), Math.round(g*255), Math.round(b*255)];
}

// Screen ↔ complex coordinate helpers
function toScreen(re, im) { return [CX + re*SCALE, CY - im*SCALE]; }
function fromScreen(px, py) { return [(px - CX)/SCALE, (CY - py)/SCALE]; }

// State
let z0 = [0.8, 0.5];
let funcName = "z²";

// Build UI
const funcSel = Inputs.select(Object.keys(funcs), { label: "f(z)", value: "z²" });
```

```
const z0ReSlider = Inputs.range([-2, 2], { value: 0.8, step: 0.05, label: "Re(z₀)" });
const z0ImSlider = Inputs.range([-2, 2], { value: 0.5, step: 0.05, label: "Im(z₀)" });

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1r

const heading = document.createElement("div");
heading.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif; font-si
heading.textContent = "Conformal angle-preservation explorer";
container.appendChild(heading);
container.appendChild(funcSel);
container.appendChild(z0ReSlider);
container.appendChild(z0ImSlider);

const panels = document.createElement("div");
panels.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-top:0.5rem;";

const leftSvgEl = document.createElementNS(svgNS, "svg");
leftSvgEl.setAttribute("width", W); leftSvgEl.setAttribute("height", H);
leftSvgEl.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; background:#f8fafc; cur

// Right panel: canvas for domain colouring + SVG overlay
const rightWrap = document.createElement("div");
rightWrap.style.position = "relative";
rightWrap.style.display = "inline-block";

const rightCanvas = document.createElement("canvas");
rightCanvas.width = W; rightCanvas.height = H;
rightCanvas.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; display:block;";

const rightSvgEl = document.createElementNS(svgNS, "svg");
rightSvgEl.setAttribute("width", W); rightSvgEl.setAttribute("height", H);
rightSvgEl.style.cssText = "position:absolute; top:0; left:0; pointer-events:none;";

rightWrap.appendChild(rightCanvas);
rightWrap.appendChild(rightSvgEl);

panels.appendChild(leftSvgEl);
panels.appendChild(rightWrap);
container.appendChild(panels);

const readout = document.createElement("div");
readout.style.cssText = "margin-top:0.5rem; font-family:sans-serif; font-size:0.85em; padding:0
container.appendChild(readout);

const note = document.createElement("div");
note.style.cssText = "margin-top:0.4rem; font-size:0.82em; color:#6b7280; font-style:italic;";
```

```
note.textContent = "Left: z-plane with two cross-arms at z₀ (red and blue). Right: their images un
container.appendChild(note);

// Draw domain colouring on canvas
function drawDomainColouring(ctx, fn) {
  const imageData = ctx.createImageData(W, H);
  const data = imageData.data;
  for (let py = 0; py < H; py++) {
    for (let px = 0; px < W; px++) {
      const re = (px - CX) / SCALE;
      const im = (CY - py) / SCALE;
      let hue, brightness;
      const [wr, wi] = fn([re, im]);
      if (!isFinite(wr) || !isFinite(wi)) {
        hue = 0; brightness = 1.0;
      } else {
        const arg = Math.atan2(wi, wr);
        const mag = Math.sqrt(wr*wr + wi*wi);
        hue = (arg / (2*Math.PI) + 1) % 1;
        const logmag = Math.log(mag + 0.001);
        brightness = 0.5 + 0.35 * Math.tanh(logmag / 2);
      }
      // Desaturate slightly so overlaid arms are readable
      const [r, g, b] = hslToRgb(hue, 0.55, brightness * 0.85 + 0.1);
      const idx = (py * W + px) * 4;
      data[idx] = r; data[idx+1] = g; data[idx+2] = b; data[idx+3] = 255;
    }
  }
  ctx.putImageData(imageData, 0, 0);
}

function clearSvg(svgEl) {
  while (svgEl.firstChild) svgEl.removeChild(svgEl.firstChild);
}

function makeLine(x1, y1, x2, y2, stroke, sw, dashed) {
  const l = document.createElementNS(svgNS, "line");
  l.setAttribute("x1", x1); l.setAttribute("y1", y1);
  l.setAttribute("x2", x2); l.setAttribute("y2", y2);
  l.setAttribute("stroke", stroke); l.setAttribute("stroke-width", sw);
  if (dashed) l.setAttribute("stroke-dasharray", "4,3");
  return l;
}

function makeCircle(cx, cy, r, fill, stroke, sw) {
  const c = document.createElementNS(svgNS, "circle");
  c.setAttribute("cx", cx); c.setAttribute("cy", cy); c.setAttribute("r", r);
```

```
  c.setAttribute("fill", fill); c.setAttribute("stroke", stroke); c.setAttribute("stroke-width"
  return c;
}

function makeText(x, y, txt, fill, size, weight) {
  const t = document.createElementNS(svgNS, "text");
  t.setAttribute("x", x); t.setAttribute("y", y);
  t.setAttribute("fill", fill); t.setAttribute("font-size", size);
  t.setAttribute("font-family", "sans-serif");
  if (weight) t.setAttribute("font-weight", weight);
  t.textContent = txt;
  return t;
}

const ARM_LEN = 0.55; // arm half-length in complex units

function render() {
  const fn = funcs[funcName];
  const ctx = rightCanvas.getContext("2d");
  drawDomainColouring(ctx, fn.f);

  // --- Left panel: z-plane ---
  clearSvg(leftSvgEl);

  // Axes
  const axG = document.createElementNS(svgNS, "g");
  axG.setAttribute("stroke", "#d1d5db"); axG.setAttribute("stroke-width", "1");
  const [ax1x] = toScreen(-3, 0); const [ax2x] = toScreen(3, 0);
  const [, ay1] = toScreen(0, -2.5); const [, ay2] = toScreen(0, 2.5);
  const [ox, oy] = toScreen(0, 0);
  axG.appendChild(makeLine(ax1x, oy, ax2x, oy, "#d1d5db", 1));
  axG.appendChild(makeLine(ox, ay1, ox, ay2, "#d1d5db", 1));
  leftSvgEl.appendChild(axG);

  // Tick marks
  for (let v = -2; v <= 2; v++) {
    if (v === 0) continue;
    const [tx] = toScreen(v, 0); const [, ty] = toScreen(0, v);
    leftSvgEl.appendChild(makeLine(tx, oy-3, tx, oy+3, "#9ca3af", 1));
    leftSvgEl.appendChild(makeLine(ox-3, ty, ox+3, ty, "#9ca3af", 1));
  }

  // Cross-arms at z0 (red horizontal, blue vertical)
  const [z0x, z0y] = toScreen(z0[0], z0[1]);
  const [rx1, ry1] = toScreen(z0[0] - ARM_LEN, z0[1]);
  const [rx2, ry2] = toScreen(z0[0] + ARM_LEN, z0[1]);
  const [bx1, by1] = toScreen(z0[0], z0[1] - ARM_LEN);
```

```
const [bx2, by2] = toScreen(z0[0], z0[1] + ARM_LEN);

leftSvgEl.appendChild(makeLine(rx1, ry1, rx2, ry2, "#dc2626", 2.5));
leftSvgEl.appendChild(makeLine(bx1, by1, bx2, by2, "#3b82f6", 2.5));
leftSvgEl.appendChild(makeCircle(z0x, z0y, 5, "#fff", "#374151", 1.5));

// Label z0
leftSvgEl.appendChild(makeText(z0x + 7, z0y - 7, `z₀ = ${z0[0].toFixed(2)}+${z0[1].toFixed(2)}i
leftSvgEl.appendChild(makeText(5, 15, "z-plane", "#6b7280", "11"));
leftSvgEl.appendChild(makeText(5, H - 6, "Red arm: horizontal  Blue arm: vertical", "#9ca3af", "

// --- Right panel: w-plane overlaid SVG ---
clearSvg(rightSvgEl);

// Compute images of arm endpoints
const armStep = 0.04; // parametric step for arm curve
const ARM_N = Math.round(ARM_LEN / armStep);

function armCurve(baseRe, baseIm, dRe, dIm) {
  // Trace the image of a short line segment from z0 in direction (dRe, dIm)
  const pts = [];
  for (let k = -ARM_N; k <= ARM_N; k++) {
    const t = k * armStep;
    const z = [baseRe + t * dRe, baseIm + t * dIm];
    const w = fn.f(z);
    if (isFinite(w[0]) && isFinite(w[1])) {
      const [sx, sy] = toScreen(w[0], w[1]);
      pts.push([sx, sy]);
    }
  }
  return pts;
}

function ptsToPath(pts) {
  if (pts.length === 0) return "";
  return pts.map((p, i) => (i === 0 ? `M ${p[0]} ${p[1]}` : `L ${p[0]} ${p[1]}`)).join(" ");
}

const redPts = armCurve(z0[0], z0[1], 1, 0);
const bluePts = armCurve(z0[0], z0[1], 0, 1);

function makePath(d, stroke, sw) {
  const p = document.createElementNS(svgNS, "path");
  p.setAttribute("d", d); p.setAttribute("stroke", stroke); p.setAttribute("stroke-width", sw);
  p.setAttribute("fill", "none");
  return p;
}
```

```
if (redPts.length > 1) rightSvgEl.appendChild(makePath(ptsToPath(redPts), "#dc2626", 2.5));
if (bluePts.length > 1) rightSvgEl.appendChild(makePath(ptsToPath(bluePts), "#3b82f6", 2.5));

// Centre dot at f(z0)
const w0 = fn.f(z0);
if (isFinite(w0[0]) && isFinite(w0[1])) {
  const [wx, wy] = toScreen(w0[0], w0[1]);
  rightSvgEl.appendChild(makeCircle(wx, wy, 5, "#fff", "#374151", 1.5));
}

rightSvgEl.appendChild(makeText(5, 15, "w-plane", "#e5e7eb", "11"));

// --- Compute angle between image arms (tangent vectors) ---
const fpz = fn.fp(z0);
let angleStr = "", angleColour = "#059669", critNote = "";

if (!isFinite(fpz[0]) || !isFinite(fpz[1]) || funcName === "z̄ (reflection)") {
  // Not analytic
  angleStr = "not analytic — not conformal";
  angleColour = "#dc2626";
  critNote = "Conjugation reverses orientation — angles are reflected, not preserved.";
} else {
  const fpMag = Math.sqrt(fpz[0]*fpz[0] + fpz[1]*fpz[1]);
  if (fpMag < 0.001) {
    // Critical point
    angleStr = "angle doubled (critical point)";
    angleColour = "#dc2626";
    critNote = "f′(z₀) ≈ 0 — conformality fails here.";
  } else {
    // Compute angle between tangent image vectors
    // Red arm tangent in z: (1,0); image tangent = fp * (1,0) = fp
    const tRed = [fpz[0], fpz[1]];
    // Blue arm tangent in z: (0,1) (= i); image tangent = fp * i = (-fpz[1], fpz[0])
    const tBlue = [-fpz[1], fpz[0]];
    const dot = tRed[0]*tBlue[0] + tRed[1]*tBlue[1];
    const mR = Math.sqrt(tRed[0]**2 + tRed[1]**2);
    const mB = Math.sqrt(tBlue[0]**2 + tBlue[1]**2);
    const cosA = Math.max(-1, Math.min(1, dot / (mR * mB)));
    const angleDeg = Math.acos(cosA) * 180 / Math.PI;
    const isCrit = Math.abs(angleDeg - 90) > 5;
    angleStr = `${angleDeg.toFixed(1)}°`;
    angleColour = isCrit ? "#dc2626" : "#059669";
    critNote = isCrit ? "critical point — angle doubled" : "conformal (angle preserved)";
  }

  const fpArg = Math.atan2(fpz[1], fpz[0]) * 180 / Math.PI;
```

```
      readout.innerHTML = `
        <span style="color:${angleColour}; font-weight:600;">Angle between image arms: ${angleStr}</
        ${critNote ? `<span style="color:${angleColour};"> — ${critNote}</span>` : ""}
        <br>
        |f′(z₀)| = <strong>${fpMag.toFixed(3)}</strong> (scale factor)
         | 
        arg f′(z₀) = <strong>${fpArg.toFixed(1)}°</strong> (rotation applied to both arms)
      `;
      return;
    }

    readout.innerHTML = `
      <span style="color:${angleColour}; font-weight:600;">${angleStr}</span>
      ${critNote ? `<br><span style="color:${angleColour}; font-size:0.92em;">${critNote}</span>` :
    `;
  }

  funcSel.addEventListener("input", () => { funcName = funcSel.value; render(); });
  z0ReSlider.addEventListener("input", () => { z0 = [z0ReSlider.value, z0[1]]; render(); });
  z0ImSlider.addEventListener("input", () => { z0 = [z0[0], z0ImSlider.value]; render(); });

  render();
  return container;
}
```

At a **critical point** where $f'(z_0) = 0$, conformality fails: angles are multiplied by the order of the zero.

**Example.** For $w = z^2$ at $z = 0$: $f'(0) = 0$, so the map is not conformal there. The angle between any two curves through the origin is doubled. Away from the origin, $f'(z) = 2z \neq 0$ and the map is conformal everywhere else.

```
//| echo: false
{
  // --- Critical-point angle-doubling diagram (static SVG) ---
  // Left: z-plane, two curves meeting at z=+1 at angle θ.
  // Right: w-plane, image curves meeting at w=2 at angle 2θ.
  // Faint aerofoil profile in background of right panel (Joukowski flat-plate limit).
  const svgNS = "http://www.w3.org/2000/svg";
  const W = 290, H = 220, SCALE = 60;

  function toSc(re, im, cx, cy, sc) { return [cx + re*sc, cy - im*sc]; }

  function makeSvg(w, h, bg) {
    const s = document.createElementNS(svgNS, "svg");
    s.setAttribute("width", w); s.setAttribute("height", h);
    s.style.cssText = `background:${bg}; border:1px solid #e5e7eb; border-radius:6px;`;
    return s;
  }
```

```
function line(x1,y1,x2,y2,stroke,sw,dash) {
  const l=document.createElementNS(svgNS,"line");
  l.setAttribute("x1",x1);l.setAttribute("y1",y1);l.setAttribute("x2",x2);l.setAttribute("y2",y
  l.setAttribute("stroke",stroke);l.setAttribute("stroke-width",sw);
  if(dash)l.setAttribute("stroke-dasharray",dash);
  return l;
}

function txt(x,y,s,fill,size,anchor) {
  const t=document.createElementNS(svgNS,"text");
  t.setAttribute("x",x);t.setAttribute("y",y);t.setAttribute("fill",fill);
  t.setAttribute("font-size",size);t.setAttribute("font-family","sans-serif");
  if(anchor)t.setAttribute("text-anchor",anchor);
  t.textContent=s;
  return t;
}

function arc(cx,cy,r,a1,a2,stroke,sw) {
  // SVG arc from angle a1 to a2 (radians, measured from positive x, anti-clockwise in maths =
  const x1=cx+r*Math.cos(a1), y1=cy-r*Math.sin(a1);
  const x2=cx+r*Math.cos(a2), y2=cy-r*Math.sin(a2);
  const sweep=0; // CCW in SVG coords = CW in maths
  const d=`M ${x1} ${y1} A ${r} ${r} 0 0 0 ${x2} ${y2}`;
  const p=document.createElementNS(svgNS,"path");
  p.setAttribute("d",d);p.setAttribute("stroke",stroke);p.setAttribute("stroke-width",sw);
  p.setAttribute("fill","none");p.setAttribute("stroke-dasharray","4,3");
  return p;
}

// Left panel: z-plane, critical point at z=+1
const leftSvg = makeSvg(W, H, "#f8fafc");
const CX = W*0.40, CY = H*0.55;

// Axes
leftSvg.appendChild(line(10,CY,W-10,CY,"#d1d5db",1));
leftSvg.appendChild(line(CX,10,CX,H-10,"#d1d5db",1));

// Critical point at z=+1
const [cpx,cpy] = toSc(1,0,CX,CY,SCALE);

// Two curves meeting at z=+1: one going up-left at ~60°, one going up at 90°
// Represent as straight lines (tangent direction is what matters)
const THETA = Math.PI / 3; // 60 degrees
const len = 0.9 * SCALE;
// Curve 1: direction angle THETA from z=+1
leftSvg.appendChild(line(cpx,cpy, cpx+len*Math.cos(THETA+Math.PI),cpy-len*Math.sin(THETA+Math.P
leftSvg.appendChild(line(cpx,cpy, cpx+len*Math.cos(THETA),cpy-len*Math.sin(THETA),"#dc2626",2.2
```

```
// Curve 2: direction 90° from z=+1 (i.e. straight up/down)
leftSvg.appendChild(line(cpx,cpy, cpx+len*Math.cos(Math.PI/2+Math.PI),cpy-len*Math.sin(Math.PI/2+M
leftSvg.appendChild(line(cpx,cpy, cpx+len*Math.cos(Math.PI/2),cpy-len*Math.sin(Math.PI/2),"#3b82f6

// Arc showing angle θ between the curves
leftSvg.appendChild(arc(cpx,cpy,22,Math.PI/2,THETA,"#374151",1.5));

// Labels
const el=document.createElementNS(svgNS,"circle");
el.setAttribute("cx",cpx);el.setAttribute("cy",cpy);el.setAttribute("r","5");
el.setAttribute("fill","#8b5cf6");el.setAttribute("stroke","#fff");el.setAttribute("stroke-width",
leftSvg.appendChild(el);

leftSvg.appendChild(txt(cpx+7,cpy-7,"z = +1","#8b5cf6","10"));
leftSvg.appendChild(txt(cpx+25,cpy-30,"θ","#374151","13"));
leftSvg.appendChild(txt(5,15,"z-plane","#6b7280","11"));
leftSvg.appendChild(txt(W/2,H-5,"Critical point of w = z² + 1/z","#9ca3af","9","middle"));

// Right panel: w-plane, image at w=2
const rightSvg = makeSvg(W, H, "#f8fafc");
const WCX = W*0.28, WCY = H*0.55;
const WSCALE = 48;

// Faint aerofoil silhouette (Joukowski flat-plate at w=2)
const aeroPath = document.createElementNS(svgNS,"path");
const apts = [];
for(let k=0;k<360;k++){
  const theta=k*Math.PI/180;
  const r=1.08, dx=0.08;
  const zRe=-dx+r*Math.cos(theta), zIm=r*Math.sin(theta);
  const d2=zRe*zRe+zIm*zIm;
  const wRe=zRe+zRe/d2, wIm=zIm-zIm/d2;
  const [px,py]=toSc(wRe,wIm,WCX,WCY,WSCALE);
  apts.push(`${k===0?"M":"L"} ${px} ${py}`);
}
aeroPath.setAttribute("d",apts.join(" ")+" Z");
aeroPath.setAttribute("fill","rgba(249,115,22,0.08)");
aeroPath.setAttribute("stroke","rgba(249,115,22,0.25)");
aeroPath.setAttribute("stroke-width","1");
rightSvg.appendChild(aeroPath);

// Axes
rightSvg.appendChild(line(10,WCY,W-10,WCY,"#d1d5db",1));
rightSvg.appendChild(line(WCX,10,WCX,H-10,"#d1d5db",1));

// Image point at w=2
const [wpx,wpy] = toSc(2,0,WCX,WCY,WSCALE);
```

```
const THETA2 = THETA * 2; // doubled angle

// Image curve 1: at angle THETA2
rightSvg.appendChild(line(wpx,wpy,wpx+len*Math.cos(THETA2+Math.PI),wpy-len*Math.sin(THETA2+Math
rightSvg.appendChild(line(wpx,wpy,wpx+len*Math.cos(THETA2),wpy-len*Math.sin(THETA2),"#dc2626",2
// Image curve 2: at 180° (twice 90°) — goes straight left and right = cusp
rightSvg.appendChild(line(wpx,wpy,wpx-len,wpy,"#3b82f6",2.2));
rightSvg.appendChild(line(wpx,wpy,wpx+len,wpy,"#3b82f6",2.2));

// Arc showing doubled angle 2θ
rightSvg.appendChild(arc(wpx,wpy,22,Math.PI,THETA2,"#374151",1.5));

const we=document.createElementNS(svgNS,"circle");
we.setAttribute("cx",wpx);we.setAttribute("cy",wpy);we.setAttribute("r","5");
we.setAttribute("fill","#8b5cf6");we.setAttribute("stroke","#fff");we.setAttribute("stroke-widt
rightSvg.appendChild(we);

rightSvg.appendChild(txt(wpx+7,wpy-7,"w = 2 (trailing edge)","#8b5cf6","10"));
rightSvg.appendChild(txt(wpx+25,wpy-30,"2θ","#374151","13"));
rightSvg.appendChild(txt(5,15,"w-plane","#6b7280","11"));
rightSvg.appendChild(txt(W/2,H-5,"Angle doubled — trailing-edge cusp","#9ca3af","9","middle"));

// Arrow between panels
const arrow = document.createElementNS(svgNS,"text");

const wrap = document.createElement("div");
wrap.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:0.75rem 1rem; margin
const heading = document.createElement("div");
heading.style.cssText = "font-weight:600; font-family:sans-serif; font-size:0.9em; margin-botto
heading.textContent = "Critical-point geometry: angle doubling at the trailing edge";
wrap.appendChild(heading);

const row = document.createElement("div");
row.style.cssText = "display:flex; gap:0.5rem; align-items:center; flex-wrap:wrap;";
row.appendChild(leftSvg);

const midLabel = document.createElement("div");
midLabel.style.cssText = "font-size:1.1em; font-weight:600; color:#6b7280; padding:0 0.25rem;";
midLabel.textContent = "w = z + 1/z";
row.appendChild(midLabel);
row.appendChild(rightSvg);
wrap.appendChild(row);

const cap = document.createElement("div");
cap.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
cap.textContent = "At z = +1 (a critical point of w = z + 1/z, since dw/dz = 1 − 1/z² = 0 there
wrap.appendChild(cap);
```

```
  return wrap;
}
```

---

## 50.2. Möbius transformations

The **Möbius transformation** (or **fractional linear transformation**) is:

$$w = \frac{az + b}{cz + d}, \qquad ad - bc \neq 0$$

The condition $ad - bc \neq 0$ ensures that $w$ is not constant. These transformations compose cleanly — applying one after another gives another Möbius transformation — which is why they are the most useful class of conformal maps in practice.

**Key property: circles and lines map to circles and lines.** In the extended complex plane — the ordinary plane with a single point at infinity added — a straight line can be viewed as a circle that passes through that point at infinity. Under a Möbius transformation, circles (including lines viewed this way) always map to circles (including lines).

**Three-point determination.** Specify three distinct points and their images and the transformation is fixed. Given pairs $(z_1 \mapsto w_1)$, $(z_2 \mapsto w_2)$, $(z_3 \mapsto w_3)$, the transformation comes from the **cross-ratio** equation:

$$\frac{(w - w_1)(w_2 - w_3)}{(w - w_3)(w_2 - w_1)} = \frac{(z - z_1)(z_2 - z_3)}{(z - z_3)(z_2 - z_1)}$$

The right side is a known number for each $z$. Cross-multiply to get a linear equation in $w$, then solve — the result is the unique Möbius transformation. Exercise 3 works through this procedure.

**Standard mappings.**

- $w = \dfrac{z - i}{z + i}$: maps the upper half-plane $\mathrm{Im}(z) > 0$ to the unit disk $|w| < 1$, and the real axis to the unit circle.
- $w = \dfrac{1 + z}{1 - z}$: maps $|z| < 1$ (unit disk) to $\mathrm{Re}(w) > 0$ (right half-plane).

### 50.2.1. Fixed points

A **fixed point** is a point that the transformation sends to itself. For $w = (az + b)/(cz + d)$, setting $z = w$ gives $cz^2 + (d - a)z - b = 0$. A generic Möbius transformation has two fixed points (possibly coincident or at $\infty$). Knowing the fixed points characterises the geometric type of the transformation — rotation, dilation, translation, or a combination.

```
//| echo: false
{
  // --- Möbius circles-to-circles explorer ---
  // LEFT: z-plane with 3 horizontal lines (blue) and 3 vertical lines (orange).
  // RIGHT: w-plane images in same colours.
  // Presets: inversion, half-plane-to-disk, disk-to-half-plane, translation, custom (a/b/c/d sli
  // Fixed-point toggle: gold stars on both panels.
  // Live annotation: "real axis maps to: [circle/line]".

  const svgNS = "http://www.w3.org/2000/svg";
  const W = 300, H = 280, SCALE = 55;
  const CX = W/2, CY = H/2;

  function ts(re,im,cx,cy,sc){return [cx+re*sc, cy-im*sc];}

  // Möbius: w = (az+b)/(cz+d)
  function mobius(zr,zi,a,b,c,d){
    // numerator = a*z+b, denominator = c*z+d (all complex)
    const [ar,ai]=a, [br,bi]=b, [cr,ci]=c, [dr,di]=d;
    const nr=ar*zr-ai*zi+br, ni=ar*zi+ai*zr+bi;
    const dr2=cr*zr-ci*zi+dr, di2=cr*zi+ci*zr+di;
    const den=dr2*dr2+di2*di2;
    if(den<1e-14)return[NaN,NaN];
    return[(nr*dr2+ni*di2)/den,(ni*dr2-nr*di2)/den];
  }

  // Fixed points: solve c*z^2 + (d-a)*z - b = 0 (all complex)
  // Using quadratic formula with complex arithmetic
  function fixedPoints(a,b,c,d){
    const [ar,ai]=a,[br,bi]=b,[cr,ci]=c,[dr,di]=d;
    // c*z^2 + (d-a)*z - b = 0
    const Ar=cr, Ai=ci;
    const Br=dr-ar, Bi=di-ai;
    const Cr=-br, Ci=-bi;
    // discriminant = B^2 - 4AC
    const B2r=Br*Br-Bi*Bi, B2i=2*Br*Bi;
    const ACr=Ar*Cr-Ai*Ci, ACi=Ar*Ci+Ai*Cr;
    const discr=B2r-4*ACr, disci=B2i-4*ACi;
    // sqrt of disc
    const discMag=Math.sqrt(discr*discr+disci*disci);
    const discArg=Math.atan2(disci,discr);
    const sqrtMag=Math.sqrt(discMag);
    const sqrtDisc1=[sqrtMag*Math.cos(discArg/2), sqrtMag*Math.sin(discArg/2)];
    const sqrtDisc2=[-sqrtDisc1[0],-sqrtDisc1[1]];
    const twoAr=2*Ar, twoAi=2*Ai;
    const twoDen=twoAr*twoAr+twoAi*twoAi;
    if(twoDen<1e-12)return[]; // c=0: translation/scaling, infinite fixed point
```

```
  function cdiv2(nr,ni,dr,di){const d2=dr*dr+di*di;return[(nr*dr+ni*di)/d2,(ni*dr-nr*di)/d2];}
  const z1=cdiv2(-Br+sqrtDisc1[0],-Bi+sqrtDisc1[1],twoAr,twoAi);
  const z2=cdiv2(-Br+sqrtDisc2[0],-Bi+sqrtDisc2[1],twoAr,twoAi);
  const pts=[];
  if(isFinite(z1[0])&&isFinite(z1[1]))pts.push(z1);
  if(isFinite(z2[0])&&isFinite(z2[1])&&(Math.abs(z2[0]-z1[0])>0.001||Math.abs(z2[1]-z1[1])>0.001)
  return pts;
}

// Preset transformations: [a,b,c,d] as complex pairs
const presets = {
  "Inversion  w = 1/z":         { a:[0,0],b:[1,0],c:[1,0],d:[0,0] },
  "Half-plane → disk  w = (z−i)/(z+i)":
                                  { a:[1,0],b:[0,-1],c:[1,0],d:[0,1] },
  "Disk → half-plane  w = i(1−z)/(1+z)":
                                  { a:[0,1],b:[0,1],c:[1,0],d:[1,0] },  // w=i(1-z)/(1+z): a=0+i,
  "Translation  w = z+1":        { a:[1,0],b:[1,0],c:[0,0],d:[1,0] },
  "Custom":                      { a:[1,0],b:[1,0],c:[0,0],d:[1,0] }
};

let preset = "Inversion  w = 1/z";
let showFP = false;
let customA=[1,0],customB=[1,0],customC=[0,0],customD=[1,0];

const presetSel = Inputs.select(Object.keys(presets), { label: "Preset", value: preset });
const fpToggle = Inputs.toggle({ label: "Show fixed points", value: false });

const aReS = Inputs.range([-2,2],{value:1,step:0.1,label:"Re(a)"});
const aImS = Inputs.range([-2,2],{value:0,step:0.1,label:"Im(a)"});
const bReS = Inputs.range([-2,2],{value:1,step:0.1,label:"Re(b)"});
const bImS = Inputs.range([-2,2],{value:0,step:0.1,label:"Im(b)"});
const cReS = Inputs.range([-2,2],{value:0,step:0.1,label:"Re(c)"});
const cImS = Inputs.range([-2,2],{value:0,step:0.1,label:"Im(c)"});
const dReS = Inputs.range([-2,2],{value:1,step:0.1,label:"Re(d)"});
const dImS = Inputs.range([-2,2],{value:0,step:0.1,label:"Im(d)"});

const customSliders = document.createElement("div");
customSliders.style.cssText = "display:none; flex-wrap:wrap; gap:0.1rem; margin:0.25rem 0;";
[aReS,aImS,bReS,bImS,cReS,cImS,dReS,dImS].forEach(s=>customSliders.appendChild(s));

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem

const heading = document.createElement("div");
heading.style.cssText = "font-weight:600; margin-bottom:0.5rem; font-family:sans-serif; font-size
heading.textContent = "Möbius transformation: circles to circles";
container.appendChild(heading);
```

```
container.appendChild(presetSel);
container.appendChild(fpToggle);
container.appendChild(customSliders);

const panels = document.createElement("div");
panels.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-top:0.5rem;";

function makeSvgEl(label){
  const s=document.createElementNS(svgNS,"svg");
  s.setAttribute("width",W);s.setAttribute("height",H);
  s.style.cssText="background:#f8fafc;border:1px solid #e5e7eb;border-radius:6px;";
  return s;
}

const leftSvg = makeSvgEl("z-plane");
const rightSvg = makeSvgEl("w-plane");
panels.appendChild(leftSvg);
panels.appendChild(rightSvg);
container.appendChild(panels);

const annotation = document.createElement("div");
annotation.style.cssText = "margin-top:0.5rem; font-family:sans-serif; font-size:0.85em; paddin
container.appendChild(annotation);

const noteEl = document.createElement("div");
noteEl.style.cssText = "margin-top:0.4rem; font-size:0.82em; color:#6b7280; font-style:italic;"
noteEl.textContent = "Blue lines: Re(z) = const (horizontal). Orange lines: Im(z) = const (vert
container.appendChild(noteEl);

function clearSvg(s){while(s.firstChild)s.removeChild(s.firstChild);}
function mkLine(s,x1,y1,x2,y2,stroke,sw,dash){
  const l=document.createElementNS(svgNS,"line");
  l.setAttribute("x1",x1);l.setAttribute("y1",y1);l.setAttribute("x2",x2);l.setAttribute("y2",y
  l.setAttribute("stroke",stroke);l.setAttribute("stroke-width",sw);
  if(dash)l.setAttribute("stroke-dasharray",dash);
  s.appendChild(l);
}
function mkTxt(s,x,y,str,fill,size){
  const t=document.createElementNS(svgNS,"text");
  t.setAttribute("x",x);t.setAttribute("y",y);t.setAttribute("fill",fill);
  t.setAttribute("font-size",size);t.setAttribute("font-family","sans-serif");
  t.textContent=str;s.appendChild(t);
}
function mkStar(s,x,y){
  // Simple star: 5 triangles
  const g=document.createElementNS(svgNS,"text");
  g.setAttribute("x",x-6);g.setAttribute("y",y+5);
```

```
  g.setAttribute("fill","#fbbf24");g.setAttribute("font-size","14");
  g.setAttribute("font-family","sans-serif");
  g.textContent="★";s.appendChild(g);
}

// Draw a curve by mapping N points along horizontal or vertical line
function drawMappedLine(svgEl, isHorizontal, cval, colour, sw, a, b, c, d) {
  let N=200;
  let pts=[];
  let prevOk=true;
  let pathD="";
  for(let k=0;k<=N;k++){
    let t=(k/N)*6-3; // range [-3,3] in the 'free' coordinate
    let zr=isHorizontal?t:cval;
    let zi=isHorizontal?cval:t;
    const [wr,wi]=mobius(zr,zi,a,b,c,d);
    if(!isFinite(wr)||!isFinite(wi)||Math.abs(wr)>6||Math.abs(wi)>6){
      prevOk=false; continue;
    }
    const [px,py]=ts(wr,wi,CX,CY,SCALE);
    pathD+=prevOk?` L ${px} ${py}`:`M ${px} ${py}`;
    prevOk=true;
  }
  if(!pathD)return;
  const p=document.createElementNS(svgNS,"path");
  p.setAttribute("d",pathD.replace(/^ L/,"M"));
  p.setAttribute("stroke",colour);p.setAttribute("stroke-width",sw);p.setAttribute("fill","none")
  svgEl.appendChild(p);
}

function render(){
  clearSvg(leftSvg); clearSvg(rightSvg);

  const p = presets[preset];
  const {a,b,c,d} = preset==="Custom"
    ? {a:customA,b:customB,c:customC,d:customD}
    : p;

  // Check ad-bc ≠ 0
  const [ar,ai]=a,[br,bi]=b,[cr,ci]=c,[dr,di]=d;
  const adR=ar*dr-ai*di, adI=ar*di+ai*dr;
  const bcR=br*cr-bi*ci, bcI=br*ci+bi*cr;
  const detR=adR-bcR, detI=adI-bcI;
  const det=Math.sqrt(detR*detR+detI*detI);
  if(det<0.01){
    annotation.style.color="#dc2626";
    annotation.textContent="ad − bc ≠ 0: degenerate (not a valid Möbius transformation). Adjust a
```

```
}

// Axes both panels
[leftSvg,rightSvg].forEach(s=>{
  const [ox,oy]=ts(0,0,CX,CY,SCALE);
  mkLine(s,10,oy,W-10,oy,"#e5e7eb",1);
  mkLine(s,ox,10,ox,H-10,"#e5e7eb",1);
});

// Input lines in z-plane (left panel, static)
const hLines = [-1.5, 0, 1.5]; // Im = const (horizontal extended-plane circles)
const vLines = [-1.5, 0, 1.5]; // Re = const (vertical extended-plane circles)

hLines.forEach(im=>{
  const [x1,y1]=ts(-3,im,CX,CY,SCALE); const [x2,y2]=ts(3,im,CX,CY,SCALE);
  mkLine(leftSvg,x1,y1,x2,y2,"#3b82f6",im===0?2:1.5);
});
vLines.forEach(re=>{
  const [x1,y1]=ts(re,-2.5,CX,CY,SCALE); const [x2,y2]=ts(re,2.5,CX,CY,SCALE);
  mkLine(leftSvg,x1,y1,x2,y2,"#f97316",re===0?2:1.5);
});

mkTxt(leftSvg,5,15,"z-plane","#6b7280","11");

// Fixed points in z-plane
const fps=fixedPoints(a,b,c,d);
if(showFP){
  fps.forEach(([fr,fi])=>{
    const [fx,fy]=ts(fr,fi,CX,CY,SCALE);
    if(fx>0&&fx<W&&fy>0&&fy<H)mkStar(leftSvg,fx,fy);
  });
}

// Right panel: mapped images
hLines.forEach(im=>drawMappedLine(rightSvg,true,im,"#3b82f6",im===0?2:1.5,a,b,c,d));
vLines.forEach(re=>drawMappedLine(rightSvg,false,re,"#f97316",re===0?2:1.5,a,b,c,d));

mkTxt(rightSvg,5,15,"w-plane","#6b7280","11");

// Fixed points in w-plane (same z coords since they map to themselves)
if(showFP){
  fps.forEach(([fr,fi])=>{
    const [fx,fy]=ts(fr,fi,CX,CY,SCALE);
    if(fx>0&&fx<W&&fy>0&&fy<H)mkStar(rightSvg,fx,fy);
  });
}
```

```
    // Annotation: where does the real axis (Im=0) map?
    const testPts=[-2,-1,0,1,2].map(x=>mobius(x,0,a,b,c,d));
    const imParts=testPts.filter(([r,i])=>isFinite(r)&&isFinite(i)).map(([r,i])=>Math.abs(i));
    const allReal=imParts.every(v=>v<0.01);
    const targetDesc=allReal?"the real axis (a line)":"a circle (or arc)";
    annotation.style.color="#374151";
    annotation.innerHTML=`Real axis (Im z = 0) maps to: <strong>${targetDesc}</strong>
      ${det>0.01?` |  ad − bc = ${detR.toFixed(2)}${detI>=0?"+":""}${detI.toFixed(2)}i`:"
  }

  presetSel.addEventListener("input",()=>{
    preset=presetSel.value;
    customSliders.style.display=preset=="Custom"?"flex":"none";
    render();
  });
  fpToggle.addEventListener("input",()=>{showFP=fpToggle.value;render();});
  [aReS,aImS,bReS,bImS,cReS,cImS,dReS,dImS].forEach(s=>s.addEventListener("input",()=>{
    customA=[+aReS.value,+aImS.value];
    customB=[+bReS.value,+bImS.value];
    customC=[+cReS.value,+cImS.value];
    customD=[+dReS.value,+dImS.value];
    render();
  }));

  render();
  return container;
}
```

```
//| echo: false
{
  // --- Möbius fixed-point geometry: pencils of circles ---
  // Four transformation types: Elliptic, Hyperbolic, Loxodromic, Parabolic.
  // Blue pencil through fixed points; orange orthogonal family.
  // Square tracer dot iterated under T.

  const svgNS = "http://www.w3.org/2000/svg";
  const W = 380, H = 300, SCALE = 60;
  const CX = W/2, CY = H/2;

  function ts(re,im){return[CX+re*SCALE,CY-im*SCALE];}
  function cmul([ar,ai],[br,bi]){return[ar*br-ai*bi,ar*bi+ai*br];}
  function cadd([ar,ai],[br,bi]){return[ar+br,ai+bi];}
  function csub([ar,ai],[br,bi]){return[ar-br,ai-bi];}
  function cdiv([ar,ai],[br,bi]){const d=br*br+bi*bi;return[(ar*br+ai*bi)/d,(ai*br-ar*bi)/d];}

  // Canonical form: conjugate to T(z) = k*z where k is the multiplier
  // We work directly with explicit Möbius maps defined per type.
```

```
const types = {
  "Elliptic": {
    // T(z) = e^(i?/3) * z  →  pure rotation, fixed points at 0 and ∞
    // In full Möbius form: a=e^(i?/3), b=0, c=0, d=1
    a:[Math.cos(Math.PI/3),Math.sin(Math.PI/3)], b:[0,0], c:[0,0], d:[1,0],
    fp1:[0,0], fp2:null, // 0 and ∞
    desc:"Elliptic: |multiplier| = 1. Orbits are circles around the fixed points. Blue family:
  },
  "Hyperbolic": {
    // T(z) = 2*z  →  real dilation, fixed points 0, ∞
    a:[2,0], b:[0,0], c:[0,0], d:[1,0],
    fp1:[0,0], fp2:null,
    desc:"Hyperbolic: multiplier is real > 0. Orbits spiral outward along radii. Blue: circles
  },
  "Loxodromic": {
    // T(z) = 1.3*e^(i?/5)*z  →  spiral, fixed points 0, ∞
    a:[1.3*Math.cos(Math.PI/5),1.3*Math.sin(Math.PI/5)], b:[0,0], c:[0,0], d:[1,0],
    fp1:[0,0], fp2:null,
    desc:"Loxodromic: multiplier has |k|?1 and arg(k)?0. Orbits are loxodromes — spirals that c
  },
  "Parabolic": {
    // T(z) = z + 1  →  translation, one fixed point at ∞
    a:[1,0], b:[1,0], c:[0,0], d:[1,0],
    fp1:null, fp2:null,
    desc:"Parabolic: single fixed point (at ∞). Orbits are parallel horizontal lines. Both penc
  }
};

let typeName = "Elliptic";
let orbitStep = 0;
let orbitPts = [[0.5, 0.8]]; // starting tracer position

const typeSel = Inputs.select(Object.keys(types),{label:"Type",value:"Elliptic"});
const iterBtn_label = Inputs.button("Iterate T →");
const resetOrbitBtn = Inputs.button("Reset orbit");

const container = document.createElement("div");
container.style.cssText="border:1px solid #e5e7eb;border-radius:8px;padding:1rem;margin:1rem 0;
const heading=document.createElement("div");
heading.style.cssText="font-weight:600;margin-bottom:0.5rem;font-family:sans-serif;font-size:0.
heading.textContent="Möbius fixed-point geometry";
container.appendChild(heading);
container.appendChild(typeSel);

const ctrlRow=document.createElement("div");
ctrlRow.style.cssText="display:flex;gap:0.5rem;align-items:center;margin:0.25rem 0;flex-wrap:wr
ctrlRow.appendChild(iterBtn_label);
```

```
ctrlRow.appendChild(resetOrbitBtn);
container.appendChild(ctrlRow);

const svgEl=document.createElementNS(svgNS,"svg");
svgEl.setAttribute("width",W);svgEl.setAttribute("height",H);
svgEl.style.cssText="background:#f8fafc;border:1px solid #e5e7eb;border-radius:6px;display:block;m
container.appendChild(svgEl);

const desc=document.createElement("div");
desc.style.cssText="margin-top:0.5rem;font-size:0.85em;color:#374151;padding:0.4rem 0.75rem;backgr
container.appendChild(desc);

const noteEl=document.createElement("div");
noteEl.style.cssText="margin-top:0.4rem;font-size:0.82em;color:#6b7280;font-style:italic;";
noteEl.textContent="Blue circles: pencil through fixed points. Orange dashed: orthogonal pencil (i
container.appendChild(noteEl);

function mobius(zr,zi,a,b,c,d){
  const [ar,ai]=a,[br,bi]=b,[cr,ci]=c,[dr,di]=d;
  const nr=ar*zr-ai*zi+br,ni=ar*zi+ai*zr+bi;
  const dr2=cr*zr-ci*zi+dr,di2=cr*zi+ci*zr+di;
  const den=dr2*dr2+di2*di2;
  if(den<1e-14)return[NaN,NaN];
  return[(nr*dr2+ni*di2)/den,(ni*dr2-nr*di2)/den];
}

function clearSvg(s){while(s.firstChild)s.removeChild(s.firstChild);}

function mkTxt(s,x,y,str,fill,size){
  const t=document.createElementNS(svgNS,"text");
  t.setAttribute("x",x);t.setAttribute("y",y);t.setAttribute("fill",fill);
  t.setAttribute("font-size",size);t.setAttribute("font-family","sans-serif");
  t.textContent=str;s.appendChild(t);
}

function mkStar(s,x,y){
  const g=document.createElementNS(svgNS,"text");
  g.setAttribute("x",x-6);g.setAttribute("y",y+5);
  g.setAttribute("fill","#fbbf24");g.setAttribute("font-size","14");
  g.textContent="★";s.appendChild(g);
}

// Draw a circle (or line) by parametric point set
function drawCircle(svg,cx,cy,r,stroke,sw,dash){
  const N=120;
  let d="";
  for(let k=0;k<=N;k++){
```

```
    const t=k/N*2*Math.PI;
    const wx=cx+r*Math.cos(t),wy=cy+r*Math.sin(t);
    d+=k===0?`M ${wx} ${wy}`:`L ${wx} ${wy}`;
  }
  d+=" Z";
  const p=document.createElementNS(svgNS,"path");
  p.setAttribute("d",d);p.setAttribute("stroke",stroke);p.setAttribute("stroke-width",sw);
  p.setAttribute("fill","none");if(dash)p.setAttribute("stroke-dasharray",dash);
  svg.appendChild(p);
}

function render(){
  clearSvg(svgEl);
  const T=types[typeName];
  const {a,b,c,d}=T;

  // Axes
  const [ox,oy]=ts(0,0);
  const axG=document.createElementNS(svgNS,"g");
  const hline=document.createElementNS(svgNS,"line");
  hline.setAttribute("x1","10");hline.setAttribute("y1",oy);hline.setAttribute("x2",W-10);hline
  hline.setAttribute("stroke","#e5e7eb");hline.setAttribute("stroke-width","1");
  const vline=document.createElementNS(svgNS,"line");
  vline.setAttribute("x1",ox);vline.setAttribute("y1","10");vline.setAttribute("x2",ox);vline.s
  vline.setAttribute("stroke","#e5e7eb");vline.setAttribute("stroke-width","1");
  svgEl.appendChild(hline);svgEl.appendChild(vline);

  // For Elliptic and Hyperbolic: fixed points at 0 and ∞
  // Blue pencil: circles (or lines) through 0 and ∞ = lines through origin
  // Orange pencil: circles centred at origin = orthogonal to those lines
  if(typeName==="Elliptic"||typeName==="Loxodromic"){
    // Orange pencil: concentric circles around origin (invariant circles for elliptic)
    [0.5,1.0,1.5,2.0,2.5].forEach(r=>{
      const [cx2,cy2]=ts(0,0);
      drawCircle(svgEl,cx2,cy2,r*SCALE,"#f97316",typeName==="Elliptic"?1.5:1,"4,3");
    });
    // Blue pencil: lines through origin (pencil through 0 and ∞) at various angles
    [0,30,60,90,120,150].forEach(angDeg=>{
      const ang=angDeg*Math.PI/180;
      const [x1,y1]=ts(3*Math.cos(ang),3*Math.sin(ang));
      const [x2,y2]=ts(-3*Math.cos(ang),-3*Math.sin(ang));
      const l=document.createElementNS(svgNS,"line");
      l.setAttribute("x1",x1);l.setAttribute("y1",y1);l.setAttribute("x2",x2);l.setAttribute("y
      l.setAttribute("stroke","#3b82f6");l.setAttribute("stroke-width","1.2");
      svgEl.appendChild(l);
    });
  } else if(typeName==="Hyperbolic"){
```

```
      // Blue: lines through origin
      [0,30,60,90,120,150].forEach(angDeg=>{
        const ang=angDeg*Math.PI/180;
        const [x1,y1]=ts(3*Math.cos(ang),3*Math.sin(ang));
        const [x2,y2]=ts(-3*Math.cos(ang),-3*Math.sin(ang));
        const l=document.createElementNS(svgNS,"line");
        l.setAttribute("x1",x1);l.setAttribute("y1",y1);l.setAttribute("x2",x2);l.setAttribute("y2",
        l.setAttribute("stroke","#3b82f6");l.setAttribute("stroke-width","1.2");
        svgEl.appendChild(l);
      });
      // Orange: concentric circles
      [0.5,1.0,1.5,2.0,2.5].forEach(r=>{
        const [cx2,cy2]=ts(0,0);
        drawCircle(svgEl,cx2,cy2,r*SCALE,"#f97316",1.5,"4,3");
      });
    } else {
      // Parabolic: parallel horizontal lines (blue) and vertical (orange)
      [-2,-1,0,1,2].forEach(v=>{
        const [x1,y1]=ts(-3,v);const [x2,y2]=ts(3,v);
        const l=document.createElementNS(svgNS,"line");
        l.setAttribute("x1",x1);l.setAttribute("y1",y1);l.setAttribute("x2",x2);l.setAttribute("y2",
        l.setAttribute("stroke","#3b82f6");l.setAttribute("stroke-width","1.2");
        svgEl.appendChild(l);
        const [vx1,vy1]=ts(v,-2.5);const [vx2,vy2]=ts(v,2.5);
        const l2=document.createElementNS(svgNS,"line");
        l2.setAttribute("x1",vx1);l2.setAttribute("y1",vy1);l2.setAttribute("x2",vx2);l2.setAttribut
        l2.setAttribute("stroke","#f97316");l2.setAttribute("stroke-width","1.2");
        l2.setAttribute("stroke-dasharray","4,3");
        svgEl.appendChild(l2);
      });
    }

    // Fixed point star(s)
    if(T.fp1){const[fx,fy]=ts(...T.fp1);mkStar(svgEl,fx,fy);}

    // Tracer orbit
    if(orbitPts.length>1){
      let pathD=orbitPts.map(([r,i],k)=>{const[px,py]=ts(r,i);return`${k===0?"M":"L"} ${px} ${py}`;
      const orbitPath=document.createElementNS(svgNS,"path");
      orbitPath.setAttribute("d",pathD);orbitPath.setAttribute("stroke","#8b5cf6");
      orbitPath.setAttribute("stroke-width","1.5");orbitPath.setAttribute("fill","none");
      orbitPath.setAttribute("stroke-dasharray","3,2");
      svgEl.appendChild(orbitPath);
    }

    // Draw tracer square at current position
    const [tr,ti]=orbitPts[orbitPts.length-1];
```

```
    const[tx,ty]=ts(tr,ti);
    const sq=document.createElementNS(svgNS,"rect");
    sq.setAttribute("x",tx-5);sq.setAttribute("y",ty-5);sq.setAttribute("width","10");sq.setAttri
    sq.setAttribute("fill","#8b5cf6");sq.setAttribute("stroke","#fff");sq.setAttribute("stroke-wi
    svgEl.appendChild(sq);

    // Panel label and type annotation
    mkTxt(svgEl,5,15,typeName,"#6b7280","11");
    desc.textContent=T.desc;
  }

  typeSel.addEventListener("input",()=>{
    typeName=typeSel.value;
    orbitPts=[[0.5,0.8]];
    render();
  });

  iterBtn_label.addEventListener("input",()=>{
    const T=types[typeName];
    const{a,b,c,d}=T;
    const[lr,li]=orbitPts[orbitPts.length-1];
    const nxt=mobius(lr,li,a,b,c,d);
    if(isFinite(nxt[0])&&isFinite(nxt[1])&&Math.abs(nxt[0])<5&&Math.abs(nxt[1])<5){
      orbitPts.push(nxt);
      if(orbitPts.length>20)orbitPts.shift();
    }
    render();
  });

  resetOrbitBtn.addEventListener("input",()=>{
    orbitPts=[[0.5,0.8]];render();
  });

  render();
  return container;
}
```

---

## 50.3. Standard conformal maps

### 50.3.1. $w = z^2$

Squaring doubles the argument and squares the modulus. The upper half-plane $\{z : \mathrm{Im}(z) > 0\}$ maps to the full complex plane minus the non-negative real axis: argument ranges from $(0, \pi)$ in the $z$-plane, doubling to $(0, 2\pi)$ in the $w$-plane.

A more useful sub-case: the first quadrant $\{0 < \arg z < \pi/2\}$ maps to the upper half-plane $\{0 < \arg w < \pi\}$. Doubling a right angle gives a straight angle.

### 50.3.2. $w = e^z$

Write $z = x + iy$. Then $w = e^x e^{iy}$, so $|w| = e^x$ (any positive real value) and $\arg w = y$. The horizontal strip $\{0 < \mathrm{Im}(z) < \pi\}$ maps to the upper half-plane $\{\mathrm{Im}(w) > 0\}$: the imaginary part of $z$ becomes the argument of $w$.

Check the boundaries directly. The lower edge $y = 0$ gives $e^{x+0i} = e^x > 0$, which lands on the positive real axis ($\arg w = 0$). The upper edge $y = \pi$ gives $e^{x+i\pi} = -e^x < 0$, which lands on the negative real axis ($\arg w = \pi$). The interior maps to the open upper half-plane.

More generally, the strip $\{a < \mathrm{Im}(z) < b\}$ maps to the sector $\{a < \arg w < b\}$.

### 50.3.3. $w = \log z$

The inverse of $w = e^z$: it maps the upper half-plane to the horizontal strip $\{0 < \mathrm{Im}(w) < \pi\}$.

```
//| echo: false
{
  // --- Standard conformal maps: grid deformation ---
  // LEFT: z-plane grid (Re=const blue, Im=const orange).
  // RIGHT: image curves in same colours after mapping.
  // Segmented selector: z², e□, log z, (z−i)/(z+i).
  // Animate button morphs from identity to final over ~2s.

  const svgNS = "http://www.w3.org/2000/svg";
  const W = 300, H = 280;
  const SCALE = 55, CX = W/2, CY = H/2;

  // Functions
  function fzsq([r,i])   { return [r*r-i*i, 2*r*i]; }
  function fexp([r,i])   { const e=Math.exp(r); return [e*Math.cos(i),e*Math.sin(i)]; }
  function flog([r,i])   {
    const mag=Math.sqrt(r*r+i*i);
    if(mag<1e-12)return[NaN,NaN];
    return [Math.log(mag), Math.atan2(i,r)];
  }
  function fhalf2disk([r,i]) {
    // (z-i)/(z+i)
    const nr=r, ni=i-1;
    const dr=r, di=i+1;
    const den=dr*dr+di*di; if(den<1e-14)return[NaN,NaN];
    return[(nr*dr+ni*di)/den,(ni*dr-nr*di)/den];
  }
```

```
const maps = {
  "w = z²":    { fn: fzsq,       reRange:[-2,2], imRange:[-2,2],
                 note:"First quadrant → upper half-plane (right-angle doubles to straight angle
  "w = e□":    { fn: fexp,       reRange:[-2,2], imRange:[0,Math.PI],
                 note:"Horizontal strip {0 < Im z < □} → upper half-plane. Im z becomes arg w."
  "w = log z": { fn: flog,       reRange:[0.1,3], imRange:[-2,2],
                 note:"Inverse of e□: upper half-plane → strip {0 < Im w < □}." },
  "w = (z−i)/(z+i)": { fn: fhalf2disk, reRange:[-3,3], imRange:[0,3],
                 note:"Upper half-plane → unit disk. Real axis → unit circle." }
};

let mapName = "w = z²";
let animating = false;
let animT = 1.0; // 0 = identity, 1 = full map
let animReq = null;

const mapSel = Inputs.select(Object.keys(maps),{label:"Map",value:"w = z²"});
const animBtn = Inputs.button("Animate mapping");

const container = document.createElement("div");
container.style.cssText="border:1px solid #e5e7eb;border-radius:8px;padding:1rem;margin:1rem 0;
const heading=document.createElement("div");
heading.style.cssText="font-weight:600;margin-bottom:0.5rem;font-family:sans-serif;font-size:0.
heading.textContent="Standard conformal maps: grid deformation";
container.appendChild(heading);
container.appendChild(mapSel);
container.appendChild(animBtn);

const panels=document.createElement("div");
panels.style.cssText="display:flex;gap:0.75rem;flex-wrap:wrap;margin-top:0.5rem;";

function mkSvg(){
  const s=document.createElementNS(svgNS,"svg");
  s.setAttribute("width",W);s.setAttribute("height",H);
  s.style.cssText="background:#f8fafc;border:1px solid #e5e7eb;border-radius:6px;";
  return s;
}
const leftSvg=mkSvg(), rightSvg=mkSvg();
panels.appendChild(leftSvg);panels.appendChild(rightSvg);
container.appendChild(panels);

const noteDiv=document.createElement("div");
noteDiv.style.cssText="margin-top:0.5rem;font-size:0.85em;padding:0.4rem 0.75rem;background:#f0
container.appendChild(noteDiv);

const captionDiv=document.createElement("div");
captionDiv.style.cssText="margin-top:0.3rem;font-size:0.82em;color:#6b7280;font-style:italic;";
```

```
captionDiv.textContent="Blue lines: Re(z) = const. Orange lines: Im(z) = const. Click Animate to w
container.appendChild(captionDiv);

function ts(re,im){return[CX+re*SCALE,CY-im*SCALE];}
function clearSvg(s){while(s.firstChild)s.removeChild(s.firstChild);}

function lerp([ar,ai],[br,bi],t){return[ar*(1-t)+br*t, ai*(1-t)+bi*t];}

function drawAxes(svg,label){
  const [ox,oy]=ts(0,0);
  const lnH=document.createElementNS(svgNS,"line");
  lnH.setAttribute("x1","5");lnH.setAttribute("y1",oy);lnH.setAttribute("x2",W-5);lnH.setAttribute
  lnH.setAttribute("stroke","#d1d5db");lnH.setAttribute("stroke-width","1");svg.appendChild(lnH);
  const lnV=document.createElementNS(svgNS,"line");
  lnV.setAttribute("x1",ox);lnV.setAttribute("y1","5");lnV.setAttribute("x2",ox);lnV.setAttribute
  lnV.setAttribute("stroke","#d1d5db");lnV.setAttribute("stroke-width","1");svg.appendChild(lnV);
  const lbl=document.createElementNS(svgNS,"text");
  lbl.setAttribute("x","5");lbl.setAttribute("y","14");lbl.setAttribute("fill","#6b7280");
  lbl.setAttribute("font-size","11");lbl.setAttribute("font-family","sans-serif");
  lbl.textContent=label;svg.appendChild(lbl);
}

function drawMappedCurve(svg,pts,stroke,sw,t){
  if(pts.length<2)return;
  // Interpolate between identity and mapped position
  let pathD="";let prev=true;
  pts.forEach(([zr,zi,wr,wi],k)=>{
    const [ir,ii]=lerp([zr,zi],[wr,wi],t);
    if(!isFinite(ir)||!isFinite(ii)||Math.abs(ir)>5||Math.abs(ii)>5){prev=false;return;}
    const [px,py]=ts(ir,ii);
    if(px<-10||px>W+10||py<-10||py>H+10){prev=false;return;}
    pathD+=prev?` L ${px} ${py}`:`M ${px} ${py}`;
    prev=true;
  });
  if(!pathD)return;
  const p=document.createElementNS(svgNS,"path");
  p.setAttribute("d",pathD.replace(/^ L/,"M"));
  p.setAttribute("stroke",stroke);p.setAttribute("stroke-width",sw);p.setAttribute("fill","none")
  svg.appendChild(p);
}

// Precompute sample points for each grid line
function buildGridLines(mapCfg){
  const {fn,reRange,imRange}=mapCfg;
  const [rMin,rMax]=reRange, [iMin,iMax]=imRange;
  const N=80;
  const hLines=[], vLines=[];
```

```
  // Horizontal lines: Im = const, Re varies
  const imVals=[];
  for(let k=0;k<=6;k++) imVals.push(iMin + k*(iMax-iMin)/6);
  imVals.forEach(im=>{
    const pts=[];
    for(let k=0;k<=N;k++){
      const re=rMin+k*(rMax-rMin)/N;
      const [wr,wi]=fn([re,im]);
      pts.push([re,im,wr,wi]);
    }
    hLines.push({pts, boundary:Math.abs(im-iMin)<0.01||Math.abs(im-iMax)<0.01});
  });

  // Vertical lines: Re = const, Im varies
  const reVals=[];
  for(let k=0;k<=6;k++) reVals.push(rMin + k*(rMax-rMin)/6);
  reVals.forEach(re=>{
    const pts=[];
    for(let k=0;k<=N;k++){
      const im=iMin+k*(iMax-iMin)/N;
      const [wr,wi]=fn([re,im]);
      pts.push([re,im,wr,wi]);
    }
    vLines.push({pts, boundary:Math.abs(re-rMin)<0.01||Math.abs(re-rMax)<0.01});
  });

  return{hLines,vLines};
}

let cachedGrid=null;
let cachedMapName=null;

function render(){
  const mapCfg=maps[mapName];
  if(cachedMapName!==mapName){
    cachedGrid=buildGridLines(mapCfg);
    cachedMapName=mapName;
  }
  const{hLines,vLines}=cachedGrid;

  clearSvg(leftSvg);clearSvg(rightSvg);
  drawAxes(leftSvg,"z-plane");
  drawAxes(rightSvg,"w-plane (t = "+animT.toFixed(2)+")");

  // Left: always identity (t=0)
  hLines.forEach(({pts,boundary})=>drawMappedCurve(leftSvg,pts,"#3b82f6",boundary?2:1,0));
  vLines.forEach(({pts,boundary})=>drawMappedCurve(leftSvg,pts,"#f97316",boundary?2:1,0));
```

```
    // Right: interpolated at animT
    hLines.forEach(({pts,boundary})=>drawMappedCurve(rightSvg,pts,"#3b82f6",boundary?2:1,animT));
    vLines.forEach(({pts,boundary})=>drawMappedCurve(rightSvg,pts,"#f97316",boundary?2:1,animT));

    noteDiv.textContent=mapCfg.note;
  }

  function runAnimation(){
    if(animating)return;
    animating=true;animT=0;
    const startTime=performance.now();
    const dur=2000;
    function step(now){
      animT=Math.min(1,(now-startTime)/dur);
      render();
      if(animT<1)animReq=requestAnimationFrame(step);
      else animating=false;
    }
    animReq=requestAnimationFrame(step);
  }

  mapSel.addEventListener("input",()=>{mapName=mapSel.value;animT=1;cachedGrid=null;render();});
  animBtn.addEventListener("input",()=>{if(!animating)runAnimation();});

  render();
  return container;
}
```

---

## 50.4. The Joukowski transformation

$$w = z + \frac{1}{z}$$

This map sits at the foundation of thin-aerofoil theory. The behaviour depends on which circle in the $z$-plane you start with.

**The unit circle maps to a degenerate shape.** For $z = e^{i\theta}$:

$$w = e^{i\theta} + e^{-i\theta} = 2\cos\theta \in [-2, 2]$$

The unit circle collapses to the real interval $[-2, 2]$ — a flat plate.

**Circles near the unit circle map to aerofoil-like shapes.** For $z = re^{i\theta}$ with $r > 1$:

$$w = \left(r + \frac{1}{r}\right)\cos\theta + i\left(r - \frac{1}{r}\right)\sin\theta$$

This is an **ellipse** with semi-axes $a = r + 1/r$ and $b = r - 1/r$. As $r \to 1^+$, the ellipse degenerates toward the flat plate.

**Off-centre circles.** Shifting the circle in the $z$-plane before applying the Joukowski map breaks the ellipse's symmetry and produces a curved aerofoil profile with a sharp trailing edge. This is the Joukowski aerofoil — the basis of early wing design.

**Why it works for aerodynamics.** The complex potential for incompressible irrotational flow around a cylinder of radius $r$ is known: $F(z) = U(z + r^2/z)$ for uniform flow at speed $U$. Composing with the Joukowski map gives the complex potential for flow around the aerofoil. The Kutta–Joukowski theorem then gives the lift per unit span as $L = \rho U \Gamma$, where $\Gamma = \oint \mathbf{v} \cdot d\mathbf{s}$ is the **circulation** — the line integral of the tangential velocity component around the aerofoil.

```
//| echo: false
{
  // --- Joukowski map explorer (upgraded) ---
  // LEFT: z-plane circle + Euler-traced streamlines (parametric, no Newton iteration).
  // RIGHT: w-plane aerofoil + mapped streamlines + lift arrow + critical point dashed link.
  // Shape label appears above the w-panel as a bold heading.

  const WP = 320, HP = 300, SCALE = 65;
  const svgNS = "http://www.w3.org/2000/svg";

  const rSlider4 = Inputs.range([1.0, 2.0], {value: 1.15, step: 0.01, label: "Circle radius r"});
  const offsetSlider = Inputs.range([0.0, 0.3], {value: 0.12, step: 0.01, label: "x-offset \u03b4
  const circSlider = Inputs.range([0, Math.PI*2], {value: 0, step: 0.1, label: "Circulation \u039

  function joukowski(zRe, zIm) {
    const d = zRe*zRe + zIm*zIm;
    if(d < 1e-10) return [NaN, NaN];
    return [zRe + zRe/d, zIm - zIm/d];
  }

  function toScreen(re, im, cx, cy, scale) {
    return [cx + re*scale, cy - im*scale];
  }

  // --- Complex velocity for flow around cylinder + circulation ---
  // dF/dz = U(1 - r²/z²) + i*Gamma/(2*pi*z)
  // vx = Re(dF/dz), vy = -Im(dF/dz)
  function complexVelocity(zr, zi, r, circ) {
    const d2 = zr*zr + zi*zi; if(d2<1e-12)return[NaN,NaN];
    const z2r=zr*zr-zi*zi,z2i=2*zr*zi; const z2d=z2r*z2r+z2i*z2i;
    const r2oz2r=r*r*z2r/z2d,r2oz2i=-r*r*z2i/z2d;
    const t1r=1-r2oz2r,t1i=-r2oz2i;
    const fac=circ/(2*Math.PI);
    const t2r=-fac*(-zi/d2),t2i=fac*(zr/d2);
    return[t1r+t2r,t1i+t2i];
  }
```

```
function traceEuler(x0,y0,r,delta,circ,dt,maxSteps){
  const pts=[];let x=x0,y=y0;
  for(let k=0;k<maxSteps;k++){
    const dx=x+delta,dy=y;
    if(dx*dx+dy*dy<r*r*0.92)break;
    if(Math.abs(x)>4.5||Math.abs(y)>3.5)break;
    pts.push([x,y]);
    const[dvr,dvi]=complexVelocity(x,y,r,circ);
    if(!isFinite(dvr)||!isFinite(dvi))break;
    const vx=dvr,vy=-dvi;const vmag=Math.sqrt(vx*vx+vy*vy);
    if(vmag<1e-8)break;
    const step=dt/vmag;x+=vx*step;y+=vy*step;
  }
  return pts;
}

function renderJoukowski(r, delta, circ) {
  const CX = WP/2 + 5, CY = HP/2;
  const WCX = WP/2, WCY = HP/2;
  const WSCALE = 42;

  // Build aerofoil curve
  const N = 360;
  const aerofoilPts = [];
  for(let k=0; k<N; k++) {
    const theta = (2*Math.PI*k)/(N-1);
    const zRe = -delta + r*Math.cos(theta);
    const zIm = r*Math.sin(theta);
    const [wRe, wIm] = joukowski(zRe, zIm);
    if(isFinite(wRe) && isFinite(wIm)) aerofoilPts.push([wRe, wIm]);
  }

  let shapeLabel="";
  if(r<=1.02&&delta<0.02)shapeLabel="Flat plate [-2, 2]";
  else if(delta<0.02)shapeLabel="Ellipse";
  else shapeLabel="Joukowski aerofoil";

  // Euler-traced streamlines
  const dt=0.02,maxSteps=600;
  const zStreamlines=[];
  for(let sy=-2.5;sy<=2.5;sy+=0.35){
    const fwd=traceEuler(-3.5,sy,r,delta,circ,dt,maxSteps);
    let xb=-3.5,yb=sy;const bwd=[];
    for(let k=0;k<120;k++){
      const dx=xb+delta,dy=yb;if(dx*dx+dy*dy<r*r*0.92)break;
      if(Math.abs(xb)>4.5||Math.abs(yb)>3.5)break;
      bwd.unshift([xb,yb]);
```

```
      const[dvr,dvi]=complexVelocity(xb,yb,r,circ);
      if(!isFinite(dvr)||!isFinite(dvi))break;
      const vx=dvr,vy=-dvi;const vmag=Math.sqrt(vx*vx+vy*vy);if(vmag<1e-8)break;
      xb-=vx*(dt/vmag);yb-=vy*(dt/vmag);
    }
    const full=[...bwd,...fwd];if(full.length>2)zStreamlines.push(full);
  }
  const wStreamlines=zStreamlines.map(pts=>pts.map(([zx,zy])=>{const[wr,wi]=joukowski(zx,zy);re

  // Left panel: z-plane
  const leftSvg = document.createElementNS(svgNS, "svg");
  leftSvg.setAttribute("width", WP); leftSvg.setAttribute("height", HP);
  leftSvg.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:6px;";

  // Faint grid
  for(let i=-3;i<=3;i++){
    const gl=document.createElementNS(svgNS,"line");
    const[lx]=toScreen(i,0,CX,CY,SCALE),[,ly1]=toScreen(0,-2.2,CX,CY,SCALE),[,ly2]=toScreen(0,2
    gl.setAttribute("x1",lx);gl.setAttribute("y1",ly1);gl.setAttribute("x2",lx);gl.setAttribute
    gl.setAttribute("stroke","#f3f4f6");gl.setAttribute("stroke-width","0.5");leftSvg.appendChi
    const gl2=document.createElementNS(svgNS,"line");
    const[lx1]=toScreen(-3,0,CX,CY,SCALE),[lx2]=toScreen(3,0,CX,CY,SCALE),[,ly]=toScreen(0,i,CX
    gl2.setAttribute("x1",lx1);gl2.setAttribute("y1",ly);gl2.setAttribute("x2",lx2);gl2.setAttr
    gl2.setAttribute("stroke","#f3f4f6");gl2.setAttribute("stroke-width","0.5");leftSvg.appendC
  }

  // Euler-traced streamlines in z-plane
  zStreamlines.forEach(pts=>{
    let d="";let prev=true;
    pts.forEach(([x,y])=>{
      const[px,py]=toScreen(x,y,CX,CY,SCALE);
      if(px<-5||px>WP+5||py<-5||py>HP+5){prev=false;return;}
      d+=prev?` L ${px} ${py}`:`M ${px} ${py}`;prev=true;
    });
    if(!d)return;
    const p=document.createElementNS(svgNS,"path");
    p.setAttribute("d",d.replace(/^ L/,"M"));
    p.setAttribute("stroke","rgba(59,130,246,0.45)");p.setAttribute("stroke-width","1.2");p.set
    leftSvg.appendChild(p);
  });

  // Circle
  const [circCx,circCy]=toScreen(-delta,0,CX,CY,SCALE);
  const circleEl=document.createElementNS(svgNS,"circle");
  circleEl.setAttribute("cx",circCx);circleEl.setAttribute("cy",circCy);circleEl.setAttribute("r"
  circleEl.setAttribute("fill","rgba(249,115,22,0.08)");circleEl.setAttribute("stroke","#f97316
  leftSvg.appendChild(circleEl);
```

```
// Axes
const [ox,oy]=toScreen(0,0,CX,CY,SCALE);
const axH=document.createElementNS(svgNS,"line");
axH.setAttribute("x1",toScreen(-3.2,0,CX,CY,SCALE)[0]);axH.setAttribute("y1",oy);
axH.setAttribute("x2",toScreen(3.2,0,CX,CY,SCALE)[0]);axH.setAttribute("y2",oy);
axH.setAttribute("stroke","#9ca3af");axH.setAttribute("stroke-width","1");leftSvg.appendChild(ax
const axV=document.createElementNS(svgNS,"line");
axV.setAttribute("x1",ox);axV.setAttribute("y1",toScreen(0,-2.2,CX,CY,SCALE)[1]);
axV.setAttribute("x2",ox);axV.setAttribute("y2",toScreen(0,2.2,CX,CY,SCALE)[1]);
axV.setAttribute("stroke","#9ca3af");axV.setAttribute("stroke-width","1");leftSvg.appendChild(ax

// Critical points ±1
[1,-1].forEach(xc=>{
  const[cpx,cpy]=toScreen(xc,0,CX,CY,SCALE);
  const cp=document.createElementNS(svgNS,"circle");
  cp.setAttribute("cx",cpx);cp.setAttribute("cy",cpy);cp.setAttribute("r","5");
  cp.setAttribute("fill","#8b5cf6");cp.setAttribute("stroke","#fff");cp.setAttribute("stroke-wi
  leftSvg.appendChild(cp);
  const lbl=document.createElementNS(svgNS,"text");
  lbl.setAttribute("x",cpx+6);lbl.setAttribute("y",cpy-6);lbl.setAttribute("fill","#8b5cf6");
  lbl.setAttribute("font-size","9");lbl.setAttribute("font-family","sans-serif");
  lbl.textContent=xc>0?"z=+1":"z=−1";leftSvg.appendChild(lbl);
});

const lbl1=document.createElementNS(svgNS,"text");
lbl1.setAttribute("fill","#6b7280");lbl1.setAttribute("font-size","10");lbl1.setAttribute("font-
lbl1.setAttribute("x",5);lbl1.setAttribute("y",14);lbl1.textContent="z-plane";leftSvg.appendChil

// Right panel: w-plane
// Shape label as bold heading above panel
const shapeLabelDiv=document.createElement("div");
shapeLabelDiv.style.cssText="font-size:14px;font-weight:700;font-family:sans-serif;color:#f9731
shapeLabelDiv.textContent=shapeLabel;

const rightSvg=document.createElementNS(svgNS,"svg");
rightSvg.setAttribute("width",WP);rightSvg.setAttribute("height",HP);
rightSvg.style.cssText="background:#f8fafc;border:1px solid #e5e7eb;border-radius:6px;";

// Faint grid
for(let i=-4;i<=4;i++){
  const gl=document.createElementNS(svgNS,"line");
  const[lx]=toScreen(i,0,WCX,WCY,WSCALE),[,ly1]=toScreen(0,-3.5,WCX,WCY,WSCALE),[,ly2]=toScreen
  gl.setAttribute("x1",lx);gl.setAttribute("y1",ly1);gl.setAttribute("x2",lx);gl.setAttribute("y
  gl.setAttribute("stroke","#f3f4f6");gl.setAttribute("stroke-width","0.5");rightSvg.appendChil
  const gl2=document.createElementNS(svgNS,"line");
  const[lx1]=toScreen(-4,0,WCX,WCY,WSCALE),[lx2]=toScreen(4,0,WCX,WCY,WSCALE),[,ly]=toScreen(0,
  gl2.setAttribute("x1",lx1);gl2.setAttribute("y1",ly);gl2.setAttribute("x2",lx2);gl2.setAttribu
```

```
    gl2.setAttribute("stroke","#f3f4f6");gl2.setAttribute("stroke-width","0.5");rightSvg.append
  }

  // Mapped streamlines in w-plane
  wStreamlines.forEach(pts=>{
    let d="";let prev=true;
    pts.forEach(([x,y])=>{
      const[px,py]=toScreen(x,y,WCX,WCY,WSCALE);
      if(px<-5||px>WP+5||py<-5||py>HP+5){prev=false;return;}
      d+=prev?` L ${px} ${py}`:`M ${px} ${py}`;prev=true;
    });
    if(!d)return;
    const p=document.createElementNS(svgNS,"path");
    p.setAttribute("d",d.replace(/^ L/,"M"));
    p.setAttribute("stroke","rgba(59,130,246,0.45)");p.setAttribute("stroke-width","1.2");p.set
    rightSvg.appendChild(p);
  });

  // Aerofoil curve
  if(aerofoilPts.length>2){
    const aeroD=aerofoilPts.map(([wr,wi],i)=>{
      const[px,py]=toScreen(wr,wi,WCX,WCY,WSCALE);
      return`${i===0?"M":"L"} ${px} ${py}`;
    }).join(" ")+" Z";
    const aeroPath=document.createElementNS(svgNS,"path");
    aeroPath.setAttribute("d",aeroD);
    aeroPath.setAttribute("fill","rgba(249,115,22,0.12)");aeroPath.setAttribute("stroke","#f973
    rightSvg.appendChild(aeroPath);
  }

  // Critical point image at w=2, dashed link
  const[cpImgX,cpImgY]=toScreen(2,0,WCX,WCY,WSCALE);
  const cpImgDot=document.createElementNS(svgNS,"circle");
  cpImgDot.setAttribute("cx",cpImgX);cpImgDot.setAttribute("cy",cpImgY);cpImgDot.setAttribute("
  cpImgDot.setAttribute("fill","#8b5cf6");cpImgDot.setAttribute("stroke","#fff");cpImgDot.setAt
  rightSvg.appendChild(cpImgDot);
  const dashLink=document.createElementNS(svgNS,"line");
  dashLink.setAttribute("x1",cpImgX+22);dashLink.setAttribute("y1",cpImgY-18);
  dashLink.setAttribute("x2",cpImgX);dashLink.setAttribute("y2",cpImgY);
  dashLink.setAttribute("stroke","#8b5cf6");dashLink.setAttribute("stroke-width","1.2");dashLin
  rightSvg.appendChild(dashLink);
  const cpLbl=document.createElementNS(svgNS,"text");
  cpLbl.setAttribute("x",cpImgX+24);cpLbl.setAttribute("y",cpImgY-20);cpLbl.setAttribute("fill"
  cpLbl.setAttribute("font-size","9");cpLbl.setAttribute("font-family","sans-serif");
  cpLbl.textContent="w=2 (trailing edge)";rightSvg.appendChild(cpLbl);

  // w-plane axes
```

```
const[wox,woy]=toScreen(0,0,WCX,WCY,WSCALE);
const waxH=document.createElementNS(svgNS,"line");
waxH.setAttribute("x1",toScreen(-4,0,WCX,WCY,WSCALE)[0]);waxH.setAttribute("y1",woy);
waxH.setAttribute("x2",toScreen(4,0,WCX,WCY,WSCALE)[0]);waxH.setAttribute("y2",woy);
waxH.setAttribute("stroke","#9ca3af");waxH.setAttribute("stroke-width","1");rightSvg.appendChilc
const waxV=document.createElementNS(svgNS,"line");
waxV.setAttribute("x1",wox);waxV.setAttribute("y1",toScreen(0,-3,WCX,WCY,WSCALE)[1]);
waxV.setAttribute("x2",wox);waxV.setAttribute("y2",toScreen(0,3,WCX,WCY,WSCALE)[1]);
waxV.setAttribute("stroke","#9ca3af");waxV.setAttribute("stroke-width","1");rightSvg.appendChilc

// Lift arrow: green, height proportional to Gamma
const liftMag=Math.min(circ*WSCALE*0.12,HP*0.35);
if(circ>0.05){
  const arrowX=toScreen(3.2,0,WCX,WCY,WSCALE)[0];
  const arrowBaseY=woy+10;const arrowTipY=arrowBaseY-liftMag;
  const shaft=document.createElementNS(svgNS,"line");
  shaft.setAttribute("x1",arrowX);shaft.setAttribute("y1",arrowBaseY);
  shaft.setAttribute("x2",arrowX);shaft.setAttribute("y2",arrowTipY);
  shaft.setAttribute("stroke","#059669");shaft.setAttribute("stroke-width","2.5");rightSvg.appen
  const ah=document.createElementNS(svgNS,"polygon");
  ah.setAttribute("points",`${arrowX},${arrowTipY-8} ${arrowX-5},${arrowTipY+4} ${arrowX+5},${ar
  ah.setAttribute("fill","#059669");rightSvg.appendChild(ah);
  const ll=document.createElementNS(svgNS,"text");
  ll.setAttribute("x",arrowX+7);ll.setAttribute("y",arrowTipY+4);ll.setAttribute("fill","#05966
  ll.setAttribute("font-size","10");ll.setAttribute("font-family","sans-serif");ll.setAttribute
  ll.textContent="L = \u03c1U\u0393";rightSvg.appendChild(ll);
}

const wlbl=document.createElementNS(svgNS,"text");
wlbl.setAttribute("fill","#6b7280");wlbl.setAttribute("font-size","10");wlbl.setAttribute("font-
wlbl.setAttribute("x",5);wlbl.setAttribute("y",14);wlbl.textContent="w-plane";rightSvg.appendChi

// Lift indicator bar
const liftDiv=document.createElement("div");
liftDiv.style.cssText="margin-top:0.4rem;font-family:sans-serif;font-size:0.85em;padding:0.5rem
const liftBarW=circ>0.05?Math.min(circ/6.3*80,80).toFixed(0):4;
const liftBarCol=circ>0.05?"#059669":"#d1d5db";
liftDiv.innerHTML=`<strong>Kutta\u2013Joukowski:</strong> <span style="display:inline-block;widt
  <div style="color:#6b7280;font-size:0.82em;margin-top:0.2rem;">Green arrow (w-plane) grows wit

const panelsDiv=document.createElement("div");
panelsDiv.style.cssText="display:flex;gap:0.75rem;flex-wrap:wrap;margin-top:0.5rem;align-items:f
panelsDiv.appendChild(leftSvg);
const rightCol=document.createElement("div");
rightCol.appendChild(shapeLabelDiv);rightCol.appendChild(rightSvg);
panelsDiv.appendChild(rightCol);
```

```
    const wrap=document.createElement("div");
    wrap.appendChild(panelsDiv);wrap.appendChild(liftDiv);
    return wrap;
  }

  const container=document.createElement("div");
  container.style.cssText="border:1px solid #e5e7eb; border-radius:8px; padding:1rem; margin:1rem
  const title=document.createElement("div");
  title.style.cssText="font-weight:600; margin-bottom:0.5rem; font-family:sans-serif;";
  title.textContent="Joukowski map explorer: circle to aerofoil (w = z + 1/z)";
  container.appendChild(title);
  container.appendChild(rSlider4);
  container.appendChild(offsetSlider);
  container.appendChild(circSlider);

  const vizDiv4=document.createElement("div");
  container.appendChild(vizDiv4);

  const note4=document.createElement("div");
  note4.style.cssText="margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic;";
  note4.textContent="Left: circle in z-plane, streamlines from Euler integration of potential flo
  container.appendChild(note4);

  function update4() {
    vizDiv4.innerHTML="";
    vizDiv4.appendChild(renderJoukowski(rSlider4.value, offsetSlider.value, circSlider.value));
  }
  rSlider4.addEventListener("input", update4);
  offsetSlider.addEventListener("input", update4);
  circSlider.addEventListener("input", update4);
  update4();

  return container;
}
```

---

## 50.5. Application to potential theory

Conformal mapping is the main analytic tool for two-dimensional field problems — fluid flow, electrostatics, heat conduction — because it transforms the Laplace equation in a complicated domain into the Laplace equation in a simple one, and solutions to Laplace's equation compose under analytic maps.

### 50.5.1. Complex potential

For a two-dimensional **incompressible irrotational** flow (or electrostatic field, or heat conduction problem), the **complex potential** $F(z) = \Phi(x, y) + i\Psi(x, y)$ has: - $\Phi$: velocity potential (or electrostatic potential, or temperature) - $\Psi$: stream function (or flux lines)

The **complex velocity** is $dF/dz = v_x - iv_y$, where $(v_x, v_y)$ is the physical velocity vector. The sign convention ($v_x - iv_y$ rather than $v_x + iv_y$) is standard in potential flow and follows from $F = \Phi + i\Psi$ with $v_x = \Phi_x$, $v_y = \Phi_y$.

Since $F$ is analytic, both $\Phi$ and $\Psi$ satisfy Laplace's equation, and they are harmonic conjugates.

### 50.5.2. Transforming the domain

If $w = f(z)$ is a conformal map, and $F(w)$ is the complex potential in the $w$-domain, then $F(f(z))$ is the complex potential in the $z$-domain.

In practice: solve Laplace's equation in a simple domain (disk, half-plane, strip), choose a conformal map from that domain to the complicated geometry, and the composition gives the solution in the complicated domain.

**Example.** Take $G(w) = w$ (uniform flow in the $w$-plane) and the conformal map $f(z) = z^2$. The composed potential in the $z$-plane is $G(f(z)) = z^2$. That is, substitute $w = z^2$ directly: the result is a new function of $z$, not $G$ evaluated at $z$. With $f(z) = z^2$, the $z$-plane flow has streamlines $\Psi = \mathrm{Im}(z^2) = 2xy = \text{const}$ — the rectangular hyperbola pattern of flow in a 90° corner.

```
//| echo: false
{
  // --- Potential theory domain transformation ---
  // THREE panels: z-plane (first quadrant, corner flow), centre label, w-plane (upper half-plane).
  // Streamlines ⍰ = 2xy = C (blue), equipotentials ⍰ = x²−y² = C (orange dashed).
  // Solid grey walls (axes of first quadrant).
  // Gold tracer dot moving simultaneously in both panels.
  // Animate button. Toggle: Fluid flow / Electrostatics (swaps label emphasis).
  // The map w = z² takes first quadrant → upper half-plane.

  const svgNS = "http://www.w3.org/2000/svg";
  const WP = 280, HP = 240, SCALE = 55;
  const CX = 20, CY = HP - 20; // origin at bottom-left corner for first quadrant

  function ts_fq(re, im) {
    // First-quadrant panel: origin at bottom-left
    return [CX + re*SCALE, CY - im*SCALE];
  }
  function ts_uhp(re, im, cx, cy, sc) {
    // Upper half-plane panel: centred origin
    return [cx + re*sc, cy - im*sc];
  }
```

```
const animBtn = Inputs.button("Animate tracer");
const modeToggle = Inputs.select(["Fluid flow","Electrostatics"],{label:"Interpretation",value:

let tracerT = 0;
let animating = false;
let animReq = null;

const container = document.createElement("div");
container.style.cssText = "border:1px solid #e5e7eb;border-radius:8px;padding:1rem;margin:1rem
const heading = document.createElement("div");
heading.style.cssText = "font-weight:600;margin-bottom:0.5rem;font-family:sans-serif;font-size:
heading.textContent = "Potential theory: corner flow maps to half-plane flow";
container.appendChild(heading);

const ctrlRow = document.createElement("div");
ctrlRow.style.cssText = "display:flex;gap:0.5rem;align-items:center;flex-wrap:wrap;margin-botto
ctrlRow.appendChild(animBtn);
ctrlRow.appendChild(modeToggle);
container.appendChild(ctrlRow);

const panelRow = document.createElement("div");
panelRow.style.cssText = "display:flex;gap:0.5rem;align-items:center;flex-wrap:wrap;";

function mkSvg(w, h) {
  const s = document.createElementNS(svgNS, "svg");
  s.setAttribute("width", w); s.setAttribute("height", h);
  s.style.cssText = "background:#f8fafc;border:1px solid #e5e7eb;border-radius:6px;";
  return s;
}

const leftSvg = mkSvg(WP, HP);
const rightSvg = mkSvg(WP, HP);

const midLabel = document.createElement("div");
midLabel.style.cssText = "font-size:1em;font-weight:600;color:#6b7280;padding:0 0.25rem;text-al
midLabel.innerHTML = "w = z²<br><span style='font-size:0.8em;color:#9ca3af;'>→</span>";

panelRow.appendChild(leftSvg);
panelRow.appendChild(midLabel);
panelRow.appendChild(rightSvg);
container.appendChild(panelRow);

const readoutDiv = document.createElement("div");
readoutDiv.style.cssText = "margin-top:0.5rem;font-size:0.85em;font-family:sans-serif;padding:0
container.appendChild(readoutDiv);

const noteDiv = document.createElement("div");
```

```
noteDiv.style.cssText = "margin-top:0.4rem;font-size:0.82em;color:#6b7280;font-style:italic;";
noteDiv.textContent = "Blue: streamlines ψ = 2xy = C (fluid flow) or equipotentials (electrostatic
container.appendChild(noteDiv);

function mkLine(s,x1,y1,x2,y2,stroke,sw,dash){
  const l=document.createElementNS(svgNS,"line");
  l.setAttribute("x1",x1);l.setAttribute("y1",y1);l.setAttribute("x2",x2);l.setAttribute("y2",y2)
  l.setAttribute("stroke",stroke);l.setAttribute("stroke-width",sw);
  if(dash)l.setAttribute("stroke-dasharray",dash);
  s.appendChild(l);
}
function mkPath(s,d,stroke,sw,dash){
  const p=document.createElementNS(svgNS,"path");
  p.setAttribute("d",d);p.setAttribute("stroke",stroke);p.setAttribute("stroke-width",sw);
  p.setAttribute("fill","none");if(dash)p.setAttribute("stroke-dasharray",dash);
  s.appendChild(p);
}
function mkTxt(s,x,y,str,fill,size,weight){
  const t=document.createElementNS(svgNS,"text");
  t.setAttribute("x",x);t.setAttribute("y",y);t.setAttribute("fill",fill);
  t.setAttribute("font-size",size);t.setAttribute("font-family","sans-serif");
  if(weight)t.setAttribute("font-weight",weight);
  t.textContent=str;s.appendChild(t);
}
function mkDot(s,x,y,fill,r){
  const c=document.createElementNS(svgNS,"circle");
  c.setAttribute("cx",x);c.setAttribute("cy",y);c.setAttribute("r",r||"6");
  c.setAttribute("fill",fill);c.setAttribute("stroke","#fff");c.setAttribute("stroke-width","1.5")
  s.appendChild(c);
}

// Streamlines ψ = 2xy = C; for x in [0,2.5], y = C/(2x)
// Equipotentials φ = x²-y² = C; parametrise as x = sqrt((C+t²)/2)... or just plot
function clearSvg(s){while(s.firstChild)s.removeChild(s.firstChild);}

function render(){
  clearSvg(leftSvg);clearSvg(rightSvg);
  const mode = modeToggle.value;

  // --- LEFT: z-plane, first quadrant ---
  // Solid grey walls (axes)
  const [ox, oy] = ts_fq(0,0);
  mkLine(leftSvg, ox, oy, ts_fq(2.5,0)[0], ts_fq(2.5,0)[1], "#6b7280", 2.5); // x-axis
  mkLine(leftSvg, ox, oy, ts_fq(0,2.2)[0], ts_fq(0,2.2)[1], "#6b7280", 2.5); // y-axis

  // Streamlines ψ = 2xy = C (blue): y = C/(2x), x in [0.01, 2.5]
  const psiVals = [0.3, 0.7, 1.2, 1.8, 2.5];
```

```
psiVals.forEach(C=>{
  let d="";
  for(let k=1;k<=80;k++){
    const x=0.08+k*2.42/80;
    const y=C/(2*x);
    if(y>2.3)continue;
    const[px,py]=ts_fq(x,y);
    d+=k===1?`M ${px} ${py}`:`L ${px} ${py}`;
  }
  if(d)mkPath(leftSvg,d,mode===="Fluid flow"?"#3b82f6":"#f97316",1.5,mode===="Fluid flow"?null:
});

// Equipotentials ⍰ = x²-y² = C (orange dashed): rectangular hyperbolae
// x² - y² = C; parametrise: x = sqrt(C + y²) for C>0
const phiVals = [-1.5, -0.8, 0.2, 0.8, 1.5, 2.2];
phiVals.forEach(C=>{
  let d="";let prev=false;
  for(let k=0;k<=80;k++){
    const y=k*2.3/80;
    const x2=C+y*y;
    if(x2<0.001)continue;
    const x=Math.sqrt(x2);
    if(x>2.5)continue;
    const[px,py]=ts_fq(x,y);
    if(!prev){d=`M ${px} ${py}`;prev=true;}else d+=`L ${px} ${py}`;
  }
  if(d)mkPath(leftSvg,d,mode===="Fluid flow"?"#f97316":"#3b82f6",1.2,"4,3");
});

// Tracer dot: moves along streamline ⍰ = 1.2, parameterised by angle
// On streamline 2xy = C: (t/sqrt(2), C/(sqrt(2)*t)) for t in [eps, ...]
const C_tracer = 1.2;
const tVal = 0.4 + tracerT * 1.6; // t goes from 0.4 to 2.0
const tx_z = tVal;
const ty_z = C_tracer / (2 * tVal);
if(ty_z > 0 && ty_z < 2.5 && tx_z < 2.5){
  const[dpx,dpy]=ts_fq(tx_z,ty_z);
  mkDot(leftSvg,dpx,dpy,"#fbbf24");
}

// Axis labels
mkTxt(leftSvg,ts_fq(2.4,0)[0]+2,ts_fq(0,0)[1],"Re z","#6b7280","10");
mkTxt(leftSvg,ts_fq(0,2.1)[0]+4,ts_fq(0,2.1)[1],"Im z","#6b7280","10");
mkTxt(leftSvg,5,15,"z-plane (first quadrant)","#6b7280","10");

const wallLabel = mode===="Fluid flow"?"solid walls":"conductor boundaries";
mkTxt(leftSvg,ox+6,oy-6,wallLabel,"#6b7280","9");
```

```
  // --- RIGHT: w-plane, upper half-plane ---
  const WSCALE=40, WCX=WP/2, WCY=HP-20;
  const [wox,woy]=ts_uhp(0,0,WCX,WCY,WSCALE);

  // Real axis (solid wall in w-plane)
  mkLine(rightSvg,10,woy,WP-10,woy,"#6b7280",2.5);

  // w = z² maps: ψ_w = Im(w) = Im(z²) = 2xy = ψ_z (same value!)
  // So streamlines Im(w) = C are horizontal lines Im(w) = C
  psiVals.forEach(C=>{
    const [,py]=ts_uhp(0,C,WCX,WCY,WSCALE);
    if(py<5||py>HP-5)return;
    mkLine(rightSvg,10,py,WP-10,py,mode==="Fluid flow"?"#3b82f6":"#f97316",1.5,mode==="Fluid flow"
    mkTxt(rightSvg,WP-28,py-3,`ψ=${C.toFixed(1)}`,"rgba(59,130,246,0.7)","8");
  });

  // Equipotentials φ_w = Re(w) = Re(z²) = x²-y² = C are vertical lines Re(w) = C
  phiVals.forEach(C=>{
    const[px]=ts_uhp(C,0,WCX,WCY,WSCALE);
    if(px<5||px>WP-5)return;
    mkLine(rightSvg,px,5,px,woy,mode==="Fluid flow"?"#f97316":"#3b82f6",1.2,"4,3");
  });

  // Tracer in w-plane: w = z² for the z-tracer
  const tw_r = tx_z*tx_z - ty_z*ty_z;
  const tw_i = 2*tx_z*ty_z;
  const[wpx,wpy]=ts_uhp(tw_r,tw_i,WCX,WCY,WSCALE);
  if(wpx>5&&wpx<WP-5&&wpy>5&&wpy<woy)mkDot(rightSvg,wpx,wpy,"#fbbf24");

  // Labels
  mkTxt(rightSvg,5,15,"w-plane (upper half-plane)","#6b7280","10");
  mkTxt(rightSvg,WP/2,HP-5,"real axis (boundary)","#6b7280","9","middle");

  // Readout
  const psiLabel = mode==="Fluid flow"?"stream function ψ":"flux function ψ";
  const phiLabel = mode==="Fluid flow"?"velocity potential φ":"electrostatic potential φ";
  const blueLabel = mode==="Fluid flow"?"streamlines (blue)":"equipotentials (blue)";
  const orangeLabel = mode==="Fluid flow"?"equipotentials (orange dashed)":"field lines (orange da
  readoutDiv.innerHTML=`<strong>${mode}:</strong> ${blueLabel}: ${psiLabel} = 2xy = C  | 
    <span style="color:#6b7280;font-size:0.9em;">w = z² maps the first-quadrant corner flow to hor
}

// Animate tracer along streamline
function runAnimation(){
  if(animating)return;
  animating=true;tracerT=0;
  const startTime=performance.now(),dur=3000;
```

```
    function step(now){
      tracerT=((now-startTime)/dur)%1;
      render();
      animReq=requestAnimationFrame(step);
    }
    animReq=requestAnimationFrame(step);
    // Stop after 3 cycles
    setTimeout(()=>{animating=false;if(animReq)cancelAnimationFrame(animReq);},9000);
  }

  animBtn.addEventListener("input",()=>{if(!animating)runAnimation();});
  modeToggle.addEventListener("input",()=>render());

  render();
  return container;
}
```

---

## 50.6. Exercises

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(steps) {
  let current = 0;
  const totalSteps = steps.length;
  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; m
  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";
  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
      const row = document.createElement("div");
      row.style.cssText = "display:grid; grid-template-columns:220px 1fr; gap:0.35rem 1rem; align
      const opCell = document.createElement("span");
      opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
      opCell.textContent = s.op;
      const eqCell = document.createElement("span");
      eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });
      row.appendChild(opCell); row.appendChild(eqCell);
```

```
      if (s.note) {
        const noteRow = document.createElement("div");
        noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0.2r
        noteRow.textContent = s.note;
        row.appendChild(noteRow);
      }
      stepsDiv.appendChild(row);
    }
    if (current === totalSteps) {
      const done = document.createElement("div");
      done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
      done.textContent = "Solution complete.";
      stepsDiv.appendChild(done);
    }
  }
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";
  const nextBtn = document.createElement("button");
  nextBtn.textContent = "Next step →";
  nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px; bac
  const resetBtn = document.createElement("button");
  resetBtn.textContent = "Reset";
  resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px; bac
  const counter = document.createElement("span");
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";
  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSteps
  }
  nextBtn.onclick = () => { if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
  resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };
  controls.appendChild(nextBtn); controls.appendChild(resetBtn); controls.appendChild(counter);
  container.appendChild(stepsDiv); container.appendChild(controls);
  renderSteps(); updateButtons();
  return container;
}
```

---

### 50.6.1. Exercise 1: Image of a circle under $w = z^2$

Find the image of the circle $|z| = 3$ under $w = z^2$. Then find the image of the half-circle $|z| = 3$, $\mathrm{Im}(z) \geq 0$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Parametrise |z| = 3",
    eq: "z = 3e^{i\\theta}, \\quad w = z^2 = 9e^{2i\\theta}"
  },
  {
    op: "Image of full circle",
    eq: "|w| = 9 \\text{ and } \\arg w = 2\\theta \\in [0,4\\pi) \\implies \\text{image is } |w|=
  },
  {
    op: "Image of upper half-circle: □ ∈ [0, □]",
    eq: "\\arg w = 2\\theta \\in [0, 2\\pi) \\implies \\text{full circle } |w| = 9 \\text{ traver
    note: "The upper semicircle maps onto the full circle |w| = 9 — the map w = z² is 2-to-1."
  },
  {
    op: "Critical points check",
    eq: "w'(z) = 2z = 0 \\text{ only at } z=0 \\notin |z|=3 \\implies \\text{map is conformal on
    note: "Angles are preserved at every point of the circle."
  }
])
```

---

### 50.6.2. Exercise 2: Image of a strip under $w = e^z$

Find the image of the horizontal strip $\{0 < \mathrm{Im}(z) < \pi/2\}$ under $w = e^z$.

```
//| echo: false
makeStepperHTML([
  {
    op: "Write w = e^{x+iy} = e^x e^{iy}",
    eq: "|w| = e^x \\in (0, \\infty), \\quad \\arg w = y \\in \\left(0, \\frac{\\pi}{2}\\right)"
  },
  {
    op: "Image in the w-plane",
    eq: "\\left\\{w : \\arg w \\in \\left(0, \\tfrac{\\pi}{2}\\right)\\right\\} = \\text{first qu
  },
  {
    op: "Check boundary images",
    eq: "y=0 \\mapsto \\arg w = 0 \\text{ (positive real axis)};\\quad y=\\pi/2 \\mapsto \\arg w
  },
  {
    op: "Conformality check: w□(z) = e^z □ 0 everywhere",
    eq: "\\text{The map is conformal throughout the strip.}",
    note: "The strip's right angles at its corner (at infinity) map to the right angle at w = 0 i
```

```
  }
])
```

---

### 50.6.3. Exercise 3: Möbius transformation from three point pairs

Find the Möbius transformation mapping $z_1 = 0 \mapsto w_1 = i$, $z_2 = 1 \mapsto w_2 = 0$, $z_3 = -1 \mapsto w_3 = \infty$.

```
//| echo: false
makeStepperHTML([
  {
    op: "z₃ maps to ∞ means the denominator vanishes at z = −1",
    eq: "w = \\frac{az+b}{c(z+1)} \\quad \\text{(denominator has root at } z=-1\\text{)}"
  },
  {
    op: "Apply w(0) = i",
    eq: "\\frac{b}{c} = i \\implies b = ic"
  },
  {
    op: "Apply w(1) = 0: numerator vanishes at z = 1",
    eq: "a(1) + b = 0 \\implies a = -b = -ic"
  },
  {
    op: "Write the transformation (set c = 1)",
    eq: "w = \\frac{-iz + i}{z + 1} = \\frac{i(1-z)}{z+1}"
  },
  {
    op: "Verify all three pairs",
    eq: "w(0) = i/1 = i\\checkmark, \\quad w(1) = 0/2 = 0\\checkmark, \\quad w(-1) = i(2)/0 = \\inft
  }
])
```

---

### 50.6.4. Exercise 4: Joukowski map — circle to ellipse

Apply $w = z + 1/z$ to the circle $|z| = 2$. Find the semi-axes of the resulting ellipse.

```
//| echo: false
makeStepperHTML([
  {
    op: "Parametrise |z| = 2",
    eq: "z = 2e^{i\\theta} \\implies w = 2e^{i\\theta} + \\tfrac{1}{2}e^{-i\\theta}"
  },
```

```
  {
    op: "Separate real and imaginary parts",
    eq: "w = \\left(2+\\tfrac{1}{2}\\right)\\cos\\theta + i\\left(2-\\tfrac{1}{2}\\right)\\sin\\t
  },
  {
    op: "Identify as an ellipse",
    eq: "\\frac{(\\text{Re}\\,w)^2}{(5/2)^2} + \\frac{(\\text{Im}\\,w)^2}{(3/2)^2} = \\cos^2\\the
  },
  {
    op: "Semi-axes",
    eq: "a = \\tfrac{5}{2} = 2.5 \\text{ (horizontal)}, \\quad b = \\tfrac{3}{2} = 1.5 \\text{ (v
    note: "General formula for |z| = r: a = r + 1/r, b = r − 1/r. As r → 1⁺, b → 0 and the ellips
  }
])
```

---

### 50.6.5. Exercise 5: Potential theory — flow in a corner

The complex potential $F(w) = w$ represents uniform horizontal flow in the $w$-plane. Apply the conformal map $w = z^2$ to find the complex potential for flow in the first quadrant ($x > 0$, $y > 0$) of the $z$-plane.

```
//| echo: false
makeStepperHTML([
  {
    op: "Compose: F(z) = F(w)|_{w=z²} = z²",
    eq: "F(z) = z^2 = (x^2-y^2) + 2ixy"
  },
  {
    op: "Identify potential ⍰ and stream function ⍰",
    eq: "\\Phi = x^2 - y^2 \\text{ (equipotential lines: rectangular hyperbolae)}"
  },
  {
    op: "Stream function",
    eq: "\\Psi = 2xy = \\text{const} \\text{ (streamlines: rectangular hyperbolae } xy = C\\text{
  },
  {
    op: "Verify ⍰ satisfies Laplace's equation",
    eq: "\\Phi_{xx} + \\Phi_{yy} = 2 + (-2) = 0 \\checkmark"
  },
  {
    op: "Velocity field",
    eq: "\\frac{dF}{dz} = 2z = 2(x+iy) \\implies v_x - iv_y = 2z \\implies v_x = 2x,\\; v_y = -2y
    note: "Flow moves outward along the x-axis and inward along the y-axis — the stagnation-point
  }
])
```

With these four chapters you have the core analytic methods: complex differentiation and the Cauchy-Riemann equations, contour integration and Cauchy's theorem, Laurent series and residues, and conformal mapping. They connect directly to the rest of engineering mathematics — the Laplace equation in any two-dimensional domain, potential flow around aerofoils, inverse Laplace transforms via residues, and stability analysis in the complex frequency plane.

# Part XI.

# Volume 7 — Numerical methods

# 51. Numerical methods

When the exact answer isn't the point

The heat equation through an irregular fin. The trajectory of a satellite perturbed by the moon's gravity. The implied volatility of an options contract given its market price. Three equations you can write down in seconds. None of them have closed-form solutions.

This is not unusual. Most equations that describe something real cannot be solved exactly. Numerical methods are what you use instead — systematic procedures that produce answers you can trust, with errors you can bound.

Three families of problems appear everywhere:

- **Root-finding**: find $x$ such that $f(x) = 0$.
- **Interpolation**: given values at a set of points, estimate the value elsewhere.
- **Numerical integration**: compute $\int_a^b f(x)\,dx$ when $f$ has no antiderivative in closed form.

Each family has the same structure: start with something computable, iterate or accumulate, and keep track of how wrong you might be.

---

## 51.1. Root-finding

### 51.1.1. The problem

You have a function $f(x)$ — read "$f$ of $x$", the function whose root you want — and you need to find $x^*$ such that $f(x^*) = 0$. This is every nonlinear equation in disguise. If you need to solve $g(x) = h(x)$, define $f(x) = g(x) - h(x)$ and find its root.

### 51.1.2. Bisection method

The simplest idea: if $f(a) < 0$ and $f(b) > 0$, and $f$ is continuous, then somewhere in $(a, b)$ the function must cross zero. The midpoint $m = (a + b)/2$ is either a root, or the root lies in $(a, m)$, or it lies in $(m, b)$. Evaluate $f(m)$ and keep the half that changes sign. Repeat.

**Algorithm:**

1. Choose $[a, b]$ with $f(a)$ and $f(b)$ having opposite signs.

2. Set $m = (a + b)/2$.
3. If $|f(m)| < \varepsilon$ (your tolerance), stop. $m$ is the root.
4. If $f(a) \cdot f(m) < 0$, the root is in $[a, m]$: set $b = m$. Otherwise set $a = m$.
5. Return to step 2.

After $n$ steps the bracket width is $(b-a)/2^n$, so the error is bounded by $(b-a)/2^{n+1}$. This is the **bisection error bound**. To guarantee an error below $\varepsilon$, you need at most $\lceil \log_2((b-a)/\varepsilon) \rceil$ iterations.

The method is slow — the error halves each step — but it always converges as long as the function is continuous and the initial bracket is valid. Nothing else is assumed.

**Worked example.** Find the positive real root of $f(x) = x^3 - x - 2$, starting in $[1, 2]$.

Note that $f(1) = 1 - 1 - 2 = -2 < 0$ and $f(2) = 8 - 2 - 2 = 4 > 0$. A root exists in $[1, 2]$.

| Step | $a$ | $b$ | $m$ | $f(m)$ | New bracket |
|---|---|---|---|---|---|
| 1 | 1 | 2 | 1.5 | $1.5^3 - 1.5 - 2 = -0.125$ | $[1.5, 2]$ |
| 2 | 1.5 | 2 | 1.75 | $1.75^3 - 1.75 - 2 \approx 1.609$ | $[1.5, 1.75]$ |
| 3 | 1.5 | 1.75 | 1.625 | $1.625^3 - 1.625 - 2 \approx 0.666$ | $[1.5, 1.625]$ |

After three steps the root is bracketed in $[1.5, 1.625]$, width $0.125 = 1/2^3$. The exact root is $x^* \approx 1.5214$.

Use the controls to watch the bracket halve and the guaranteed uncertainty shrink step by step.

```
//| echo: false
{
  const funcs = {
    cubic: { label:"x^3 - x - 2", f:x => x**3 - x - 2, a:1, b:2 },
    cosx: { label:"cos(x) - x", f:x => Math.cos(x) - x, a:0, b:1 }
  };
  const W = 720, H = 360;

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  function runBisection(name, n) {
    const { f, a: a0, b: b0 } = funcs[name];
    let a = a0, b = b0;
    const states = [{ a, b, m:(a+b)/2, fm:f((a+b)/2) }];
    for (let k = 0; k < n; k++) {
      const m = (a + b) / 2;
      const fm = f(m);
      if (f(a) * fm <= 0) b = m; else a = m;
      states.push({ a, b, m:(a+b)/2, fm:f((a+b)/2) });
    }
    return states.at(-1);
  }
```

```
const container = document.createElement("div");
container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem
const selLabel = document.createElement("label"); selLabel.textContent = "Function";
const select = document.createElement("select");
Object.entries(funcs).forEach(([k, v]) => {
  const opt = document.createElement("option");
  opt.value = k; opt.textContent = v.label;
  select.appendChild(opt);
});
const selOut = document.createElement("span"); selOut.textContent = "";
controls.append(selLabel, select, selOut);
const stepLabel = document.createElement("label"); stepLabel.textContent = "Iteration";
const step = document.createElement("input");
step.type = "range"; step.min = "0"; step.max = "8"; step.step = "1"; step.value = "3";
const stepOut = document.createElement("span");
stepOut.style.cssText = "font-variant-numeric:tabular-nums; min-width:3rem;";
controls.append(stepLabel, step, stepOut);

const summary = document.createElement("div");
summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3
const bracketBox = document.createElement("span");
const midBox = document.createElement("span");
const widthBox = document.createElement("span");
summary.append(bracketBox, midBox, widthBox);

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

function redraw() {
  const name = select.value;
  const info = funcs[name];
  const state = runBisection(name, Number(step.value));
  selOut.textContent = info.label;
  stepOut.textContent = step.value;
  svg.replaceChildren();

  const xMin = info.a - 0.2, xMax = info.b + 0.2;
  const yVals = Array.from({ length: 200 }, (_, i) => info.f(xMin + i / 199 * (xMax - xMin)));
```

```
  const yMin = Math.min(...yVals, -0.5), yMax = Math.max(...yVals, 0.5);
  const sx = x => 55 + (x - xMin) / (xMax - xMin) * 620;
  const sy = y => 290 - (y - yMin) / (yMax - yMin) * 210;

  const axis = svgEl("line");
  axis.setAttribute("x1", "55"); axis.setAttribute("y1", String(sy(0)));
  axis.setAttribute("x2", "675"); axis.setAttribute("y2", String(sy(0)));
  axis.setAttribute("stroke", "#64748b");
  axis.setAttribute("stroke-width", "1.2");
  svg.appendChild(axis);

  const path = [];
  for (let i = 0; i < 200; i++) {
    const x = xMin + i / 199 * (xMax - xMin);
    path.push(`${i === 0 ? "M" : "L"} ${sx(x)} ${sy(info.f(x))}`);
  }
  const curve = svgEl("path");
  curve.setAttribute("d", path.join(" "));
  curve.setAttribute("fill", "none");
  curve.setAttribute("stroke", "#2563eb");
  curve.setAttribute("stroke-width", "3");
  svg.appendChild(curve);

  const band = svgEl("rect");
  band.setAttribute("x", String(sx(state.a)));
  band.setAttribute("y", "40");
  band.setAttribute("width", String(sx(state.b) - sx(state.a)));
  band.setAttribute("height", "250");
  band.setAttribute("fill", "rgba(245,158,11,0.18)");
  svg.appendChild(band);

  [["a", state.a, "#dc2626"], ["m", state.m, "#0f172a"], ["b", state.b, "#16a34a"]].forEach(([labe
    const line = svgEl("line");
    line.setAttribute("x1", String(sx(x))); line.setAttribute("y1", "40");
    line.setAttribute("x2", String(sx(x))); line.setAttribute("y2", "290");
    line.setAttribute("stroke", color);
    line.setAttribute("stroke-width", "2.5");
    svg.appendChild(line);
    const t = svgEl("text");
    t.setAttribute("x", String(sx(x))); t.setAttribute("y", "32");
    t.setAttribute("text-anchor", "middle");
    t.setAttribute("font-size", "12");
    t.setAttribute("fill", color);
    t.textContent = `${label} = ${Number(x).toFixed(4)}`;
    svg.appendChild(t);
  });
```

```
    bracketBox.textContent = `current bracket = [${state.a.toFixed(4)}, ${state.b.toFixed(4)}]`;
    midBox.textContent = `midpoint = ${state.m.toFixed(4)}`;
    widthBox.textContent = `width = ${(state.b - state.a).toFixed(4)}`;
    caption.textContent = "Bisection never loses the root: it simply keeps the half-interval wher
  }

  [select, step].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, summary, svg, caption);
  redraw();
  return container;
}
```

### 51.1.3. Newton-Raphson method

Bisection is robust but slow. If $f$ is differentiable, you can do much better. The idea: at your current estimate $x_n$ — read "$x$ sub $n$", the $n$th iterate — draw the tangent line to $y = f(x)$. Follow the tangent to where it hits zero; call that $x_{n+1}$.

The tangent line at $x_n$ has equation:

$$y - f(x_n) = f'(x_n)(x - x_n)$$

where $f'(x_n)$ — read "$f$ prime at $x_n$" — is the derivative of $f$ at the current point. Setting $y = 0$ and solving for $x$:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

This is the **Newton-Raphson iteration**. Apply it repeatedly until $|f(x_n)|$ is below your tolerance.

**Why it converges so fast.** The error $e_n = x_n - x^*$ satisfies:

Substituting a Taylor expansion of $f(x_n)$ about the true root $x^*$ into the Newton iteration and simplifying:

$$e_{n+1} \approx -\frac{f''(x^*)}{2f'(x^*)} e_n^2$$

The next error is proportional to the *square* of the current error. If you are accurate to 3 decimal places, one more step gives you roughly 6. This is **quadratic convergence** — the number of correct digits doubles each step. Bisection adds about 0.3 correct digits per step; Newton-Raphson roughly doubles them.

The method fails if $f'(x_n) = 0$ (the tangent is flat) or if the starting point is far from the root (the iteration diverges or cycles). Always check convergence.

**Worked example.** Compute $\sqrt{2}$ using Newton-Raphson. The positive root of $f(x) = x^2 - 2$ is $\sqrt{2}$, so $f'(x) = 2x$.

The iteration is:

$$x_{n+1} = x_n - \frac{x_n^2 - 2}{2x_n} = \frac{x_n + 2/x_n}{2}$$

Starting from $x_0 = 1$:

| $n$ | $x_n$ | $f(x_n) = x_n^2 - 2$ | $\lvert f(x_n)\rvert$ |
|---|---|---|---|
| 0 | 1.000000 | −1.000000 | 1.000000 |
| 1 | 1.500000 | 0.250000 | 0.250000 |
| 2 | 1.416667 | 0.006944 | 0.006944 |
| 3 | 1.414216 | 0.000006 | $6 \times 10^{-6}$ |

Three steps to six-digit accuracy. Quadratic convergence in action.

Use the stepper to see each tangent line generate the next iterate.

```
//| echo: false
{
  const f = x => x * x - 2;
  const fp = x => 2 * x;
  const xs = [1.0];
  for (let i = 0; i < 4; i++) {
    const x = xs.at(-1);
    xs.push(x - f(x) / fp(x));
  }

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
  const actions = document.createElement("div");
  actions.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.75rem;";
  const prevBtn = document.createElement("button"); prevBtn.textContent = "Previous";
  const nextBtn = document.createElement("button"); nextBtn.textContent = "Next iteration";
  const stage = document.createElement("strong");
  actions.append(prevBtn, nextBtn, stage);

  const readout = document.createElement("div");
  readout.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3341
  const xBox = document.createElement("span");
  const fBox = document.createElement("span");
  const errBox = document.createElement("span");
  readout.append(xBox, fBox, errBox);

  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 720 360");
  svg.style.width = "100%";
```

```
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const log = document.createElement("table");
log.style.cssText = "border-collapse:collapse; width:100%; max-width:420px; margin-top:0.75rem;

const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

let idx = 0;
function redraw() {
  const x = xs[idx];
  const xNext = xs[Math.min(idx + 1, xs.length - 1)];
  stage.textContent = `iteration n = ${idx}`;
  xBox.textContent = `x_n = ${x.toFixed(6)}`;
  fBox.textContent = `f(x_n) = ${f(x).toFixed(6)}`;
  errBox.textContent = `|f(x_n)| = ${Math.abs(f(x)).toExponential(3)}`;

  svg.replaceChildren();
  const xMin = 0.7, xMax = 1.7, yMin = -1.2, yMax = 1.2;
  const sx = x => 55 + (x - xMin) / (xMax - xMin) * 620;
  const sy = y => 280 - (y - yMin) / (yMax - yMin) * 220;

  const axis = svgEl("line");
  axis.setAttribute("x1", "55"); axis.setAttribute("y1", String(sy(0)));
  axis.setAttribute("x2", "675"); axis.setAttribute("y2", String(sy(0)));
  axis.setAttribute("stroke", "#64748b"); axis.setAttribute("stroke-width", "1.2");
  svg.appendChild(axis);

  const path = [];
  for (let i = 0; i < 240; i++) {
    const xv = xMin + i / 239 * (xMax - xMin);
    path.push(`${i === 0 ? "M" : "L"} ${sx(xv)} ${sy(f(xv))}`);
  }
  const curve = svgEl("path");
  curve.setAttribute("d", path.join(" "));
  curve.setAttribute("fill", "none");
  curve.setAttribute("stroke", "#2563eb");
  curve.setAttribute("stroke-width", "3");
  svg.appendChild(curve);

  const m = fp(x), b = f(x) - m * x;
  const t1 = xMin, t2 = xMax;
  const tangent = svgEl("line");
  tangent.setAttribute("x1", String(sx(t1)));
  tangent.setAttribute("y1", String(sy(m * t1 + b)));
```

```
    tangent.setAttribute("x2", String(sx(t2)));
    tangent.setAttribute("y2", String(sy(m * t2 + b)));
    tangent.setAttribute("stroke", "#f59e0b");
    tangent.setAttribute("stroke-width", "2.5");
    svg.appendChild(tangent);

    const p = svgEl("circle");
    p.setAttribute("cx", String(sx(x)));
    p.setAttribute("cy", String(sy(f(x))));
    p.setAttribute("r", "5.5");
    p.setAttribute("fill", "#0f172a");
    svg.appendChild(p);

    const drop = svgEl("line");
    drop.setAttribute("x1", String(sx(xNext))); drop.setAttribute("y1", String(sy(0)));
    drop.setAttribute("x2", String(sx(xNext))); drop.setAttribute("y2", "305");
    drop.setAttribute("stroke", "#dc2626");
    drop.setAttribute("stroke-width", "2");
    svg.appendChild(drop);

    log.replaceChildren();
    const head = document.createElement("tr");
    ["n","x_n","|f(x_n)|"].forEach(h => {
      const th = document.createElement("th");
      th.textContent = h;
      th.style.cssText = "border:1px solid #cbd5e1; padding:0.35rem 0.5rem; text-align:center;";
      head.appendChild(th);
    });
    log.appendChild(head);
    xs.slice(0, idx + 2).forEach((val, n) => {
      const tr = document.createElement("tr");
      [String(n), val.toFixed(6), Math.abs(f(val)).toExponential(3)].forEach(text => {
        const td = document.createElement("td");
        td.textContent = text;
        td.style.cssText = "border:1px solid #cbd5e1; padding:0.35rem 0.5rem; text-align:center;";
        tr.appendChild(td);
      });
      log.appendChild(tr);
    });

    caption.textContent = "The orange tangent uses the local linear approximation at x_n. Its x-inte
  }

  prevBtn.addEventListener("click", () => { idx = Math.max(0, idx - 1); redraw(); });
  nextBtn.addEventListener("click", () => { idx = Math.min(xs.length - 2, idx + 1); redraw(); });

  container.append(actions, readout, svg, log, caption);
```

```
  redraw();
  return container;
}
```

### 51.1.4. Secant method

Newton-Raphson requires the derivative $f'(x)$. When that is expensive or unavailable, replace it with a finite-difference approximation using the two most recent iterates:

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

This is the **secant method**. It needs two starting values $x_0$ and $x_1$. The convergence order is approximately 1.618 (the golden ratio) — super-linear but slower than quadratic. No derivative evaluation required.

> **Which root-finder to use**
> Bisection is your fallback: slow, guaranteed, needs only function values and a sign change. Newton-Raphson is your workhorse when you have a derivative and a decent starting point: fast, elegant, can fail. The secant method sits between them: no derivative, faster than bisection, needs two starting points. In practice, hybrid methods (Brent's method, Illinois algorithm) combine bisection safety with Newton-like speed.

---

## 51.2. Interpolation

### 51.2.1. The problem

You have $n + 1$ data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$ and you want to estimate $f(x)$ at a new $x$ lying between them. The classical approach: find a polynomial that passes exactly through all the given points, then evaluate it at $x$.

### 51.2.2. Lagrange interpolation

For $n + 1$ points there is exactly one polynomial of degree $\leq n$ passing through all of them. Write it as:

$$P_n(x) = \sum_{k=0}^{n} y_k \, L_k(x)$$

where each **Lagrange basis polynomial** $L_k(x)$ is:

$$L_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j}$$

Read this as: $L_k(x)$ equals 1 at $x_k$ and equals 0 at every other node $x_j$. So $P_n(x_k) = y_k$ for each $k$ — the polynomial interpolates every point exactly.

**Worked example.** Interpolate $f$ at $x = 1.5$ using the points $(1, 1)$, $(2, 8)$, $(3, 27)$ — the cube function sampled at three integers.

With nodes $x_0 = 1$, $x_1 = 2$, $x_2 = 3$:

$$L_0(x) = \frac{(x-2)(x-3)}{(1-2)(1-3)} = \frac{(x-2)(x-3)}{2}$$

$$L_1(x) = \frac{(x-1)(x-3)}{(2-1)(2-3)} = \frac{(x-1)(x-3)}{-1} = -(x-1)(x-3)$$

$$L_2(x) = \frac{(x-1)(x-2)}{(3-1)(3-2)} = \frac{(x-1)(x-2)}{2}$$

At $x = 1.5$:

$$L_0(1.5) = \frac{(1.5-2)(1.5-3)}{2} = \frac{(-0.5)(-1.5)}{2} = \frac{0.75}{2} = 0.375$$

$$L_1(1.5) = -(1.5-1)(1.5-3) = -(0.5)(-1.5) = 0.75$$

$$L_2(1.5) = \frac{(1.5-1)(1.5-2)}{2} = \frac{(0.5)(-0.5)}{2} = -0.125$$

$$P_2(1.5) = 1 \cdot 0.375 + 8 \cdot 0.75 + 27 \cdot (-0.125) = 0.375 + 6 - 3.375 = 3$$

The exact value is $1.5^3 = 3.375$. The polynomial through three points of $x^3$ is not perfect — it is a degree-2 polynomial approximating a degree-3 function. Adding the fourth node $(4, 64)$ would give exact interpolation.

### 51.2.3. Newton's divided-difference form

The Lagrange form is conceptually clean but computationally inconvenient: to add a new node you must recompute everything. Newton's divided-difference form builds the polynomial incrementally.

Define the **divided differences** recursively:

$$f[x_k] = y_k$$

$$f[x_k, x_{k+1}] = \frac{f[x_{k+1}] - f[x_k]}{x_{k+1} - x_k}$$

$$f[x_k, \dots, x_{k+j}] = \frac{f[x_{k+1}, \dots, x_{k+j}] - f[x_k, \dots, x_{k+j-1}]}{x_{k+j} - x_k}$$

To see how this builds, use two of the cube-function points: $x_0 = 1$, $y_0 = 1$ and $x_1 = 2$, $y_1 = 8$. The first divided difference is $f[x_0, x_1] = (8 - 1)/(2 - 1) = 7$. Newton's form gives $P_1(x) = 1 + 7(x - 1)$ — the straight line through $(1, 1)$ and $(2, 8)$. Adding a third node brings in the next divided difference and bends the curve to pass through that point too.

The interpolating polynomial is then:

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots$$

Adding a new node $x_{n+1}$ requires only one new divided difference and one new term — the previous work is retained.

### 51.2.4. Runge's phenomenon and why splines exist

High-degree polynomial interpolation over equally-spaced nodes can oscillate wildly near the endpoints, even when the underlying function is smooth. This is Runge's phenomenon. The standard fix is **spline interpolation**: use low-degree polynomials (typically cubics) on each sub-interval, stitched together with continuity conditions. Splines are covered in the next chapter.

---

## 51.3. Numerical integration

### 51.3.1. The problem

Compute $\int_a^b f(x)\,dx$ when $f$ has no closed-form antiderivative, or when $f$ is known only at discrete points (sampled data from a sensor, say).

Partition $[a, b]$ into $n$ equal panels of width $h = (b - a)/n$. Let $x_i = a + ih$ for $i = 0, 1, \dots, n$, and write $f_i = f(x_i)$.

### 51.3.2. Trapezoid rule

Approximate $f$ on each panel by a straight line joining $(x_i, f_i)$ and $(x_{i+1}, f_{i+1})$. The area of each trapezoid is $\frac{h}{2}(f_i + f_{i+1})$. Summing over all panels:

$$\int_a^b f(x)\,dx \approx T_n = \frac{h}{2}\big(f_0 + 2f_1 + 2f_2 + \cdots + 2f_{n-1} + f_n\big)$$

The error is $O(h^2)$ — halving the step size $h$ reduces the error by a factor of four. More precisely:

$$\text{Error} = -\frac{(b-a)h^2}{12} f''(\xi)$$

for some $\xi \in (a, b)$ — a point whose exact location is unknown, but which lies somewhere in the integration interval. The formula says: the error depends on the curvature of $f$ somewhere in $(a, b)$. You don't need to know where $\xi$ is; you only need an upper bound on $|f''|$ over the interval. The error is proportional to the second derivative of $f$ — large curvature means larger error.

**Worked example.** Estimate $\int_0^1 e^x\,dx$ with $n = 4$ panels.

$h = 0.25$, nodes at $x = 0, 0.25, 0.5, 0.75, 1$.

| $x_i$ | $f_i = e^{x_i}$ |
|---|---|
| 0 | 1.000000 |
| 0.25 | 1.284025 |
| 0.5 | 1.648721 |
| 0.75 | 2.117000 |
| 1 | 2.718282 |

$$T_4 = \frac{0.25}{2}\big(1 + 2(1.284025) + 2(1.648721) + 2(2.117000) + 2.718282\big)$$

$$= 0.125 \times (1 + 2.568050 + 3.297442 + 4.234000 + 2.718282)$$

$$= 0.125 \times 13.817774 = 1.727222$$

The exact value is $e - 1 \approx 1.718282$. Error: $|1.727222 - 1.718282| \approx 0.008940$.

### 51.3.3. Simpson's 1/3 rule

Replace the straight-line approximation on each pair of panels with a quadratic through three consecutive nodes. Require $n$ even.

$$\int_a^b f(x)\,dx \approx S_n = \frac{h}{3}\big(f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 4f_{n-1} + f_n\big)$$

The coefficients alternate $4, 2, 4, 2, \ldots, 4$ for interior nodes. The error is $O(h^4)$:

$$\text{Error} = -\frac{(b-a)h^4}{180} f^{(4)}(\xi)$$

Halving $h$ reduces the error by a factor of sixteen. Simpson's rule is exact for polynomials of degree up to 3, even though it uses degree-2 approximations — a consequence of symmetry.

**Worked example.** Apply Simpson's 1/3 rule to $\int_0^1 e^x\,dx$ with $n = 4$.

Using the same nodes as above:

$$S_4 = \frac{0.25}{3}\big(f_0 + 4f_1 + 2f_2 + 4f_3 + f_4\big)$$

$$= \frac{0.25}{3}\big(1 + 4(1.284025) + 2(1.648721) + 4(2.117000) + 2.718282\big)$$

$$= \frac{0.25}{3}\big(1 + 5.136100 + 3.297442 + 8.468000 + 2.718282\big)$$

$$= \frac{0.25}{3} \times 20.619824 = 1.718319$$

Error: $|1.718319 - 1.718282| \approx 0.000037$. A factor of 241 smaller than the trapezoid rule with the same number of function evaluations.

### 51.3.4. Simpson's 3/8 rule

A variant using groups of three panels instead of two, again with a cubic approximant. Require $n$ divisible by 3.

$$\int_a^b f(x)\,dx \approx \frac{3h}{8}\big(f_0 + 3f_1 + 3f_2 + 2f_3 + 3f_4 + 3f_5 + 2f_6 + \cdots + f_n\big)$$

Error is also $O(h^4)$. The 1/3 rule is more commonly used; the 3/8 rule appears when $n$ is odd so the standard Simpson pairing fails.

Use the panel slider to compare how trapezoid and Simpson approximations improve as the partition is refined.

```
//| echo: false
{
  const f = x => Math.sin(Math.PI * x) + 0.4 * x * x;
  const exact = 2 / Math.PI + 0.4 / 3;
  function trap(n) {
    const h = 1 / n;
    let s = 0.5 * (f(0) + f(1));
    for (let i = 1; i < n; i++) s += f(i * h);
    return h * s;
  }
  function simpson(n) {
    if (n % 2 === 1) n += 1;
    const h = 1 / n;
    let s = f(0) + f(1);
    for (let i = 1; i < n; i++) s += (i % 2 === 0 ? 2 : 4) * f(i * h);
    return h * s / 3;
  }

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem; a
  const label = document.createElement("label"); label.textContent = "Panels n";
  const slider = document.createElement("input");
  slider.type = "range"; slider.min = "2"; slider.max = "16"; slider.step = "2"; slider.value = "4"
  const out = document.createElement("span"); out.style.cssText = "font-variant-numeric:tabular-nums
  controls.append(label, slider, out);

  const summary = document.createElement("div");
  summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3341
  const trapBox = document.createElement("span");
  const simpBox = document.createElement("span");
  const exactBox = document.createElement("span");
  summary.append(trapBox, simpBox, exactBox);

  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 720 360");
  svg.style.width = "100%";
  svg.style.height = "auto";
  svg.style.border = "1px solid #e2e8f0";
  svg.style.background = "#fff";

  const caption = document.createElement("p");
  caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

  function redraw() {
```

```
    const n = Number(slider.value);
    out.textContent = String(n);
    const tVal = trap(n);
    const sVal = simpson(n);
    svg.replaceChildren();
    const sx = x => 55 + x * 620;
    const sy = y => 300 - y / 1.45 * 220;

    const axisX = svgEl("line");
    axisX.setAttribute("x1", "55"); axisX.setAttribute("y1", "300");
    axisX.setAttribute("x2", "675"); axisX.setAttribute("y2", "300");
    axisX.setAttribute("stroke", "#64748b"); axisX.setAttribute("stroke-width", "1.2");
    svg.appendChild(axisX);

    for (let i = 0; i < n; i++) {
      const x0 = i / n, x1 = (i + 1) / n;
      const poly = svgEl("polygon");
      poly.setAttribute("points", `${sx(x0)},300 ${sx(x0)},${sy(f(x0))} ${sx(x1)},${sy(f(x1))} ${
      poly.setAttribute("fill", "rgba(59,130,246,0.12)");
      poly.setAttribute("stroke", "#93c5fd");
      svg.appendChild(poly);
    }

    const curve = svgEl("path");
    curve.setAttribute("d", Array.from({ length: 220 }, (_, i) => {
      const x = i / 219;
      return `${i === 0 ? "M" : "L"} ${sx(x)} ${sy(f(x))}`;
    }).join(" "));
    curve.setAttribute("fill", "none");
    curve.setAttribute("stroke", "#0f172a");
    curve.setAttribute("stroke-width", "3");
    svg.appendChild(curve);

    trapBox.textContent = `trapezoid = ${tVal.toFixed(5)} (error ${Math.abs(tVal - exact).toFixed
    simpBox.textContent = `Simpson = ${sVal.toFixed(5)} (error ${Math.abs(sVal - exact).toFixed(5
    exactBox.textContent = `exact = ${exact.toFixed(5)}`;
    caption.textContent = "The blue panels show the trapezoid construction. Simpson's rule uses t
  }

  slider.addEventListener("input", redraw);
  container.append(controls, summary, svg, caption);
  redraw();
  return container;
}
```

### 51.3.5. Gaussian quadrature

The rules above use evenly-spaced nodes. If you are free to choose both the node locations and the weights, you can do much better.

**Gauss-Legendre quadrature** on $[-1, 1]$ with $n$ nodes is exact for polynomials of degree up to $2n - 1$ — twice the order you might expect. The nodes are the roots of the Legendre polynomial $P_n(x)$, and the weights are determined by the condition of exactness.

The Legendre polynomials are a specific family of functions whose roots have an extremal property: placing quadrature nodes there squeezes the most accuracy out of a given number of evaluations. You do not derive these roots by hand — they are tabulated in every numerical methods reference and built into every serious quadrature library. For $n = 2$, the nodes and weights are:

For $n = 2$: nodes $x_{1,2} = \pm 1/\sqrt{3}$, weights $w_{1,2} = 1$:

$$\int_{-1}^{1} f(x)\,dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

This is exact for all polynomials of degree $\leq 3$.

To integrate over $[a, b]$, substitute $x = \frac{a+b}{2} + \frac{b-a}{2}t$ where $t \in [-1, 1]$:

$$\int_a^b f(x)\,dx = \frac{b-a}{2}\int_{-1}^{1} f\left(\frac{a+b}{2} + \frac{b-a}{2}t\right)dt$$

**Worked example.** Estimate $\int_0^1 e^x\,dx$ using 2-point Gauss-Legendre.

Change variables to $[-1, 1]$: $x = \frac{1}{2} + \frac{1}{2}t$, $dx = \frac{1}{2}\,dt$.

Nodes in $t$: $t_{1,2} = \pm 1/\sqrt{3} \approx \pm 0.577350$.

Corresponding $x$-values: $x_1 = 0.5 - 0.288675 = 0.211325$, $x_2 = 0.5 + 0.288675 = 0.788675$.

$$\int_0^1 e^x\,dx \approx \frac{1}{2}\left(e^{0.211325} + e^{0.788675}\right) = \frac{1}{2}(1.235369 + 2.200819) = 1.718094$$

Error: $|1.718094 - 1.718282| \approx 0.000188$. Two function evaluations; better accuracy than the 5-point trapezoid rule.

**Summary: error orders**

| Method | Error | Error reduction when $n$ doubles |
|---|---|---|
| Trapezoid | $O(h^2)$ | 4× smaller |
| Simpson's 1/3 | $O(h^4)$ | 16× smaller |
| Gauss–Legendre ($n$ pts) | exact for degree $\leq 2n-1$ | — |

When $f$ is smooth and you control the nodes, Gaussian quadrature wins. When $f$ comes from sampled data at fixed points, use Simpson or trapezoid.

---

## 51.4. Where this goes

The three families here — root-finding, interpolation, numerical integration — are the building blocks for almost everything else in numerical computation. **Numerical linear algebra** (the next chapter in this section) extends root-finding ideas to systems: you want $\mathbf{x}$ such that $A\mathbf{x} = \mathbf{b}$, and for large systems you solve it iteratively using methods (Gauss-Seidel, conjugate gradient) that are direct descendants of the Newton-Raphson idea.

Numerical integration connects back to **ordinary differential equations**: Euler's method for ODEs is the forward-rectangle (left-endpoint) rule applied to the integral form of the ODE — at each step you approximate the integrand by its value at the left endpoint, which is first-order accurate. Runge-Kutta methods are higher-order quadrature applied at each step. Knowing the error orders from this chapter tells you immediately why RK4 (fourth-order) is preferred over Euler (first-order): halving the step size reduces RK4's error by 16×, Euler's by only 2×.

**Where these methods appear**

- **Structural engineering (FEA):** each element stiffness matrix is assembled using Gaussian quadrature over the element domain.
- **Signal processing:** interpolation underlies sample-rate conversion (resampling audio or images); Newton-Raphson is used in carrier phase estimation.
- **Options pricing:** Newton-Raphson is the standard method for computing implied volatility from the Black-Scholes formula — the exact inverse doesn't exist.
- **Astrophysics:** orbital trajectories are integrated numerically step by step using adaptive-step Runge-Kutta, which uses error estimates derived from the quadrature theory here.
- **Machine learning:** most optimisers (Adam, L-BFGS) are Newton-like methods applied to high-dimensional loss functions. The convergence order matters enormously at scale.

---

## 51.5. Exercises

These are puzzles. Each has a clean numerical answer. The interesting work is carrying the method through correctly and checking the error bound.

**1.** Apply three iterations of bisection to find the positive root of $f(x) = x^3 - x - 2 = 0$, starting with the bracket $[1, 2]$. State the midpoint after each iteration and the final bracket width.

---

**2.** Apply three iterations of Newton-Raphson to compute $\sqrt{2}$, starting from $x_0 = 1$. Use $f(x) = x^2 - 2$. Record $x_n$ and $|f(x_n)|$ at each step and comment on the convergence rate.

---

**3.** Use the Lagrange interpolation formula to estimate $f(1.5)$, given the data points $(1, 1)$, $(2, 8)$, $(3, 27)$. Compare with the exact value $1.5^3 = 3.375$ and explain the discrepancy.

---

**4.** Apply the trapezoid rule to $\int_0^1 e^x \, dx$ with $n = 4$ panels. Compute the numerical result and the absolute error against the exact value $e - 1 \approx 1.718282$.

---

**5.** Apply Simpson's 1/3 rule to the same integral $\int_0^1 e^x \, dx$ with $n = 4$ panels. Compare your error with Exercise 4 and confirm the improvement is consistent with the $O(h^4)$ order.

---

**6.** A diode circuit satisfies $I \cdot R + V_T \ln(I/I_0) = V_s$, where $R = 100\,\Omega$, $V_T = 0.026\,\text{V}$, $I_0 = 10^{-12}\,\text{A}$, $V_s = 5\,\text{V}$. Starting from $I_0^* = 0.04\,\text{A}$, apply one step of Newton-Raphson to find a better estimate. *(Linearise the equation about the starting point.)*

# 52. Numerical linear algebra

Solving systems that can't be solved by hand

You have a system of equations. In principle, Gaussian elimination gives the answer. In practice, you type it into a computer, and the answer is wrong — not obviously wrong, but subtly, fatally wrong.

The issue isn't the algorithm. The issue is that computers don't do exact arithmetic. Every floating-point operation introduces a tiny rounding error. Tiny errors compound. In a badly conditioned system, a rounding error in the twelfth decimal place can shift the answer by a factor of a thousand.

This chapter is about how to do it right: pivoting to avoid catastrophic cancellation, LU decomposition to separate the expensive part from the cheap part, and the condition number to know in advance whether the answer you're computing is worth trusting.

## 52.1. What the notation is saying

A linear system $A\mathbf{x} = \mathbf{b}$ means: find the vector $\mathbf{x}$ — read as "the solution vector" — such that the matrix $A$, read as "the coefficient matrix", transforms $\mathbf{x}$ into the right-hand side $\mathbf{b}$.

In component form for a $3 \times 3$ system:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

The goal is to reduce $A$ to upper-triangular form so that back-substitution can extract $x_3$, then $x_2$, then $x_1$ in sequence.

**LU decomposition** writes $A = LU$, where $L$ is a lower-triangular matrix (ones on the diagonal, entries only below) and $U$ is the upper-triangular matrix produced by elimination. This notation should be read as: "$A$ factors into $L$ times $U$".

The **condition number** $\kappa(A)$, read "kappa of $A$", quantifies how much the solution can change relative to a small change in the data. It is defined as

$$\kappa(A) = \|A\|_\infty \cdot \|A^{-1}\|_\infty$$

where $\|A\|_\infty$, read "the infinity-norm of $A$", is the largest row sum of absolute values:

$$\|A\|_\infty = \max_i \sum_j |a_{ij}|$$

A rough rule: if $\kappa(A) \approx 10^k$, the solution loses approximately $k$ digits of accuracy to rounding errors. A condition number of $10^{12}$ on a machine with 15-digit floating-point precision leaves you with about 3 reliable digits.

For iterative methods, $x^{(k)}$ denotes the iterate at step $k$ — the approximate solution after $k$ steps, converging toward the true $\mathbf{x}$ as $k$ grows.

## 52.2. Gaussian elimination with partial pivoting

### 52.2.1. Why naive elimination fails

The exact solution of the system below is $x_1 = 1$, $x_2 = 1$ — you can check: $0.0001(1) + 1(1) = 1.0001 \approx 1$ and $1(1) + 1(1) = 2$. Now watch what happens when a computer solves it.

Consider this $2 \times 2$ system in floating-point arithmetic with 4 significant figures:

$$\begin{pmatrix} 0.0001 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Naive elimination: subtract $(1/0.0001) = 10000$ times row 1 from row 2.

Row 2 becomes:

$$[1 - 10000 \times 0.0001, \quad 1 - 10000 \times 1] = [0, \quad 1 - 10000]$$

The $1-10000$ gives $-9999$, fine. But consider the right-hand side: $2-10000\times1 = -9998$. Back-substituting: $x_2 = -9998/-9999 \approx 1.0001$. Then $x_1 = (1 - 1 \times 1.0001)/0.0001 \approx -1$.

The exact answer is $x_2 \approx 1$, $x_1 \approx 1$. Naive elimination gives $x_1 \approx -1$. That is not a small rounding error — that is the wrong sign with the wrong magnitude. The problem: dividing by the tiny pivot 0.0001 amplified the rounding error in the subtracted row by a factor of 10000. The error in the fourth decimal place became the dominant term in the result.

This is **catastrophic cancellation**: subtracting two nearly equal large numbers, leaving a result that is almost entirely rounding error. It is not a pathological edge case. Any system with a small pivot relative to the other entries is vulnerable — and such systems arise routinely from physical models with variables at very different scales.

### 52.2.2. Partial pivoting: the fix

Before eliminating column $j$, scan the entries in column $j$ from row $j$ downward. Swap the row with the **largest absolute value** in that column into the pivot position. This ensures the multiplier $m_{ij} = a_{ij}/a_{jj}$ satisfies $|m_{ij}| \leq 1$, bounding the growth of rounding errors.

**Algorithm** (partial pivoting GE on $n \times n$ augmented matrix $[A|\mathbf{b}]$):

For $j = 1, 2, \ldots, n-1$:

1. Find $p = \arg\max_{i \geq j} |a_{ij}|$ (index of largest entry in column $j$, rows $j$ to $n$)
2. If $p \neq j$: swap rows $p$ and $j$ in $[A|\mathbf{b}]$
3. For $i = j+1, \ldots, n$:
   - Compute multiplier $m_{ij} = a_{ij}/a_{jj}$
   - Row $i \leftarrow$ Row $i - m_{ij} \times$ Row $j$

After $n-1$ passes, $A$ is upper triangular. Recover $\mathbf{x}$ by back-substitution:

$$x_n = \frac{b_n}{a_{nn}}, \qquad x_i = \frac{1}{a_{ii}}\left(b_i - \sum_{k=i+1}^{n} a_{ik}x_k\right), \quad i = n-1, \ldots, 1$$

### 52.2.3. Worked example: partial pivoting on a 3×3 system

Solve:

$$\begin{pmatrix} 1 & 2 & 1 \\ 3 & 2 & 4 \\ 2 & 1 & 2 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 20 \\ 12 \end{pmatrix}$$

Write the augmented matrix:

$$[A|\mathbf{b}] = \left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 3 & 2 & 4 & 20 \\ 2 & 1 & 2 & 12 \end{array}\right]$$

**Pass 1 (column 1):** The largest entry in column 1 is $|3|$ in row 2. Swap rows 1 and 2:

$$\left[\begin{array}{ccc|c} 3 & 2 & 4 & 20 \\ 1 & 2 & 1 & 8 \\ 2 & 1 & 2 & 12 \end{array}\right]$$

Eliminate below pivot $a_{11} = 3$:

- Row 2 $\leftarrow$ Row 2 $-(1/3)$ Row 1: $[1-1,\ 2-2/3,\ 1-4/3 \mid 8-20/3] = [0,\ 4/3,\ -1/3 \mid 4/3]$
- Row 3 $\leftarrow$ Row 3 $-(2/3)$ Row 1: $[2-2,\ 1-4/3,\ 2-8/3 \mid 12-40/3] = [0,\ -1/3,\ -2/3 \mid -4/3]$

$$\left[\begin{array}{ccc|c} 3 & 2 & 4 & 20 \\ 0 & 4/3 & -1/3 & 4/3 \\ 0 & -1/3 & -2/3 & -4/3 \end{array}\right]$$

**Pass 2 (column 2):** Largest in column 2, rows 2–3: $|4/3| > |-1/3|$. No swap needed. Pivot is $4/3$.

- Row 3 $\leftarrow$ Row 3 $-(-1/3)/(4/3)\times$ Row 2 = Row 3 $+(1/4)$ Row 2: $[0,\ -1/3+1/3,\ -2/3-1/12 \mid -4/3+1/3] = [0,\ 0,\ -3/4 \mid -1]$

$$\left[\begin{array}{ccc|c} 3 & 2 & 4 & 20 \\ 0 & 4/3 & -1/3 & 4/3 \\ 0 & 0 & -3/4 & -1 \end{array}\right]$$

**Back-substitution:**

$$x_3 = \frac{-1}{-3/4} = \frac{4}{3}$$

$$x_2 = \frac{4/3 - (-1/3)(4/3)}{4/3} = \frac{4/3 + 4/9}{4/3} = \frac{16/9}{4/3} = \frac{16}{9} \cdot \frac{3}{4} = \frac{4}{3}$$

$$x_1 = \frac{20 - 2(4/3) - 4(4/3)}{3} = \frac{20 - 8/3 - 16/3}{3} = \frac{20-8}{3} = \frac{12}{3} = 4$$

**Solution:** $x_1 = 4$, $x_2 = 4/3$, $x_3 = 4/3$.

**Check:** Row 1: $1(4) + 2(4/3) + 1(4/3) = 4 + 8/3 + 4/3 = 4 + 4 = 8$ ☒ Row 2: $3(4) + 2(4/3) + 4(4/3) = 12 + 8/3 + 16/3 = 12 + 8 = 20$ ☒ Row 3: $2(4) + 1(4/3) + 2(4/3) = 8 + 4/3 + 8/3 = 8 + 4 = 12$ ☒

Use the stepper to see the row swap, the pivot choice, and the elimination multipliers that create the upper-triangular system.

```
//| echo: false
{
  const steps = [
    {
      title: "Initial augmented matrix",
      pivot: [0,0],
      swap: "swap R1 and R2 for a stronger pivot",
      rows: [
        ["1","2","1","8"],
        ["3","2","4","20"],
        ["2","1","2","12"]
      ]
    },
    {
      title: "After partial pivoting",
      pivot: [0,0],
```

```
    swap: "R2 moved to the top",
    rows: [
      ["3","2","4","20"],
      ["1","2","1","8"],
      ["2","1","2","12"]
    ]
  },
  {
    title: "Eliminate below pivot 1",
    pivot: [1,1],
    swap: "multipliers m21 = 1/3, m31 = 2/3",
    rows: [
      ["3","2","4","20"],
      ["0","4/3","-1/3","4/3"],
      ["0","-1/3","-2/3","-4/3"]
    ]
  },
  {
    title: "Pivot 2 and finish",
    pivot: [1,1],
    swap: "multiplier m32 = -1/4",
    rows: [
      ["3","2","4","20"],
      ["0","4/3","-1/3","4/3"],
      ["0","0","-3/4","-1"]
    ]
  }
];

const container = document.createElement("div");
container.style.cssText = "max-width:700px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const actions = document.createElement("div");
actions.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.75rem;";
const prevBtn = document.createElement("button"); prevBtn.textContent = "Previous";
const nextBtn = document.createElement("button"); nextBtn.textContent = "Next step";
const stage = document.createElement("strong");
actions.append(prevBtn, nextBtn, stage);

const note = document.createElement("div");
note.style.cssText = "margin-bottom:0.6rem; color:#334155;";

const table = document.createElement("table");
table.style.cssText = "border-collapse:collapse; width:100%; max-width:640px; font-size:0.95rem
const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

let idx = 0;
```

```
  function redraw() {
    const step = steps[idx];
    stage.textContent = step.title;
    note.textContent = step.swap;
    table.replaceChildren();
    const head = document.createElement("tr");
    ["x1","x2","x3","rhs"].forEach(h => {
      const th = document.createElement("th");
      th.textContent = h;
      th.style.cssText = "border:1px solid #cbd5e1; padding:0.45rem 0.6rem; text-align:center;";
      head.appendChild(th);
    });
    table.appendChild(head);
    step.rows.forEach((row, i) => {
      const tr = document.createElement("tr");
      row.forEach((val, j) => {
        const td = document.createElement("td");
        td.textContent = val;
        td.style.cssText = "border:1px solid #cbd5e1; padding:0.45rem 0.6rem; text-align:center;";
        if (i === step.pivot[0] && j === step.pivot[1]) td.style.background = "#fde68a";
        if (idx === 0 && i === 0 && j === 0) td.style.background = "#fecaca";
        if (idx >= 2 && i > 0 && j === 0) td.style.color = "#2563eb";
        tr.appendChild(td);
      });
      table.appendChild(tr);
    });
    caption.textContent = idx === 0
      ? "The red entry would be a weak pivot. Partial pivoting swaps in the largest available entry
      : idx < steps.length - 1
      ? "Amber marks the current pivot. Blue text highlights entries being eliminated below the pivo
      : "At the end, the matrix is upper triangular and ready for back substitution.";
  }

  prevBtn.addEventListener("click", () => { idx = Math.max(0, idx - 1); redraw(); });
  nextBtn.addEventListener("click", () => { idx = Math.min(steps.length - 1, idx + 1); redraw(); })

  container.append(actions, note, table, caption);
  redraw();
  return container;
}
```

## 52.3. LU decomposition

Gaussian elimination — without pivoting, for clarity — can be read as computing a factorisation $A = LU$. The matrix $L$ records the multipliers used during elimination; $U$ is the upper-triangular result.

### 52.3.1. Doolittle form

In the Doolittle convention, $L$ has **ones on its diagonal**:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}$$

where $m_{ij}$ is the multiplier used to zero $a_{ij}$ during elimination.

The $U$ matrix is exactly the upper-triangular matrix at the end of Gaussian elimination. The $L$ matrix costs nothing extra — the multipliers are computed anyway and simply stored in the lower portion of the working array.

### 52.3.2. Why LU earns its keep

Solving $A\mathbf{x} = \mathbf{b}$ once costs $O(n^3)$ operations — where "operations" means floating-point additions and multiplications, roughly $n^3/3$ in total, because each elimination step touches every remaining row-column pair. This is unavoidable. But if you need to solve the same $A$ for multiple right-hand sides $\mathbf{b}_1, \mathbf{b}_2, \ldots$, the LU factorisation pays back:

- **Build** $A = LU$: $O(n^3)$ — done once.
- **Solve** $L\mathbf{y} = \mathbf{b}$ (forward substitution): $O(n^2)$ per right-hand side.
- **Solve** $U\mathbf{x} = \mathbf{y}$ (back substitution): $O(n^2)$ per right-hand side.

For $k$ right-hand sides the total cost is $O(n^3) + k \cdot O(n^2)$, versus $k \cdot O(n^3)$ if you re-eliminate each time. For a $1000 \times 1000$ system solved for 50 right-hand sides, that's roughly 50× faster.

This matters in FEM analysis, where the stiffness matrix $K$ is fixed while the load vector $\mathbf{f}$ changes with each load case.

### 52.3.3. Worked example: Doolittle LU on a 3×3 system

Factorise and solve:

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad \mathbf{b}_2 = \begin{pmatrix} 2 \\ 0 \\ 4 \end{pmatrix}$$

**Step 1: Build $U$ and record multipliers.**

Column 1, pivot $u_{11} = 2$: - $m_{21} = 4/2 = 2$. Row 2 ← Row 2 − 2 Row 1: $[0, 1, 1 \mid \cdots]$ - $m_{31} = 8/2 = 4$. Row 3 ← Row 3 − 4 Row 1: $[0, 3, 5 \mid \cdots]$

Column 2, pivot $u_{22} = 1$: - $m_{32} = 3/1 = 3$. Row 3 ← Row 3 − 3 Row 2: $[0, 0, 2 \mid \cdots]$

$$U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}, \qquad L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{pmatrix}$$

**Verify:** $LU$ should equal $A$.

$$LU = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix} = A$$

**Step 2: Solve $L\mathbf{y} = \mathbf{b}_1$ (forward substitution).**

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

$y_1 = 1, \quad y_2 = 1 - 2(1) = -1, \quad y_3 = 1 - 4(1) - 3(-1) = 0$

**Step 3: Solve $U\mathbf{x} = \mathbf{y}$ (back substitution).**

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix}$$

$x_3 = 0/2 = 0, \quad x_2 = (-1 - 1 \cdot 0)/1 = -1, \quad x_1 = (1 - 1(-1) - 1(0))/2 = 1$

**Solution for $\mathbf{b}_1$: $\mathbf{x} = (1, -1, 0)^T$**

**Check:** $2(1) + 1(-1) + 1(0) = 1$ ☒, $\quad 4(1) + 3(-1) + 3(0) = 1$ ☒, $\quad 8(1) + 7(-1) + 9(0) = 1$ ☒

**Step 4: Reuse $L, U$ for $\mathbf{b}_2 = (2, 0, 4)^T$.**

Forward substitution $L\mathbf{y} = \mathbf{b}_2$:

$y_1 = 2, \quad y_2 = 0 - 2(2) = -4, \quad y_3 = 4 - 4(2) - 3(-4) = 4 - 8 + 12 = 8$

Back substitution $U\mathbf{x} = \mathbf{y}$:

$x_3 = 8/2 = 4, \quad x_2 = (-4 - 1 \cdot 4)/1 = -8, \quad x_1 = (2 - 1(-8) - 1(4))/2 = (2 + 8 - 4)/2 = 3$

**Solution for $\mathbf{b}_2$: $\mathbf{x} = (3, -8, 4)^T$**

**Check:** $2(3)+1(-8)+1(4) = 6-8+4 = 2$ ☒, $\quad 4(3)+3(-8)+3(4) = 12-24+12 = 0$ ☒, $\quad 8(3)+7(-8)+9(4) = 24 - 56 + 36 = 4$ ☒

> **Why LU works**
> Solving $A\mathbf{x} = \mathbf{b}$ is equivalent to solving $LU\mathbf{x} = \mathbf{b}$. Let $\mathbf{y} = U\mathbf{x}$. Then $L\mathbf{y} = \mathbf{b}$ — a lower-triangular system, solved in one forward pass. Then $U\mathbf{x} = \mathbf{y}$ — an upper-triangular system, solved in one backward pass. The original $O(n^3)$ work goes into building $L$ and $U$. Once those are in hand, each new right-hand side costs only $O(n^2)$.

## 52.4. Condition number

### 52.4.1. Definition

The condition number $\kappa(A)$ measures how much relative error in $\mathbf{b}$ is amplified into relative error in $\mathbf{x}$.

Using the $\infty$-norm:

$$\kappa(A) = \|A\|_\infty \cdot \|A^{-1}\|_\infty$$

where $\|A\|_\infty = \max_i \sum_j |a_{ij}|$ is the maximum absolute row sum.

A system is **well-conditioned** if $\kappa(A)$ is small (close to 1). It is **ill-conditioned** if $\kappa(A) \gg 1$.

The digits-of-accuracy rule: with $t$-digit floating-point arithmetic, you can expect roughly $t - \log_{10} \kappa(A)$ correct digits in the solution.

### 52.4.2. Ill-conditioning in practice: the 3×3 Hilbert matrix

The Hilbert matrix $H$ with entries $h_{ij} = 1/(i + j - 1)$ is the classic ill-conditioning example:

$$H_3 = \begin{pmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{pmatrix}$$

**Step 1: $\|H_3\|_\infty$.**

Row sums: Row 1: $1 + 0.5 + 0.333 = 1.833$; Row 2: $0.5 + 0.333 + 0.25 = 1.083$; Row 3: $0.333 + 0.25 + 0.2 = 0.783$.

$$\|H_3\|_\infty = 1.833$$

**Step 2: $\|H_3^{-1}\|_\infty$.**

The exact inverse of $H_3$ is:

$$H_3^{-1} = \begin{pmatrix} 9 & -36 & 30 \\ -36 & 192 & -180 \\ 30 & -180 & 180 \end{pmatrix}$$

Row sums of $|H_3^{-1}|$: Row 1: $9 + 36 + 30 = 75$; Row 2: $36 + 192 + 180 = 408$; Row 3: $30 + 180 + 180 = 390$.

$$\|H_3^{-1}\|_\infty = 408$$

**Step 3: $\kappa_\infty(H_3)$.**

$$\kappa_\infty(H_3) = 1.833 \times 408 \approx 748$$

So solving $H_3\mathbf{x} = \mathbf{b}$ loses about $\log_{10}(748) \approx 2.9$ digits of accuracy. On a 15-digit machine you get about 12 reliable digits — acceptable. For $H_{10}$, $\kappa \approx 10^{13}$, leaving roughly 2 reliable digits. That system cannot be usefully solved by direct methods in standard double precision.

> **When to worry**
> If $\kappa(A) > 10^6$ in double precision, inspect the problem formulation before trusting any solution. The issue usually lies in the model (ill-scaled variables, near-dependent equations) rather than the solver. Rescaling rows and columns (equilibration) often reduces $\kappa$ by several orders of magnitude.

Use the matrix sliders to move toward or away from singularity, then nudge the right-hand side and compare how much the solution moves.

```
//| echo: false
{
  function inv2(a, b, c, d) {
    const det = a * d - b * c;
    if (Math.abs(det) < 1e-9) return null;
    return { det, m: [d / det, -b / det, -c / det, a / det] };
  }
  function normInf(a, b, c, d) { return Math.max(Math.abs(a) + Math.abs(b), Math.abs(c) + Math.abs(
  function mul(inv, v1, v2) { return [inv.m[0] * v1 + inv.m[1] * v2, inv.m[2] * v1 + inv.m[3] * v2]

  const container = document.createElement("div");
  container.style.cssText = "max-width:780px; margin:1rem 0 1.25rem 0; font-family:inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.45rem 0.75rem;

  function row(labelText, min, max, step, value) {
    const label = document.createElement("label");
    label.textContent = labelText;
    const input = document.createElement("input");
    input.type = "range";
    input.min = String(min);
    input.max = String(max);
    input.step = String(step);
    input.value = String(value);
    const out = document.createElement("span");
    out.style.cssText = "font-variant-numeric:tabular-nums; min-width:4rem;";
    controls.append(label, input, out);
    return { input, out };
  }

  const aRow = row("a", -5, 5, 0.1, 2.0);
  const bRow = row("b", -5, 5, 0.1, 1.8);
  const cRow = row("c", -5, 5, 0.1, 2.1);
  const dRow = row("d", -5, 5, 0.1, 1.9);
  const rhs1 = row("b₁", -5, 5, 0.1, 1.0);
```

```
const rhs2 = row("b₂", -5, 5, 0.1, 1.0);
const delta = row("⍰ added to b₁", -0.5, 0.5, 0.01, 0.05);

const summary = document.createElement("div");
summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.7rem; color:#3
const detBox = document.createElement("span");
const kappaBox = document.createElement("span");
const sensBox = document.createElement("span");
summary.append(detBox, kappaBox, sensBox);

const matrixWrap = document.createElement("div");
matrixWrap.style.cssText = "display:grid; grid-template-columns:repeat(3, auto); gap:1rem 1.5re
const aTable = document.createElement("table");
const xTable = document.createElement("table");
const xpTable = document.createElement("table");
[aTable, xTable, xpTable].forEach(t => t.style.cssText = "border-collapse:collapse; font-size:0

const caption = document.createElement("p");
caption.style.cssText = "margin:0.7rem 0 0 0; color:#475569;";

function tableFrom(table, title, rows, accent) {
  table.replaceChildren();
  const cap = document.createElement("caption");
  cap.textContent = title;
  cap.style.cssText = `caption-side:top; text-align:left; font-weight:700; margin-bottom:0.25re
  table.appendChild(cap);
  rows.forEach(r => {
    const tr = document.createElement("tr");
    r.forEach(v => {
      const td = document.createElement("td");
      td.textContent = v;
      td.style.cssText = "border:1px solid #cbd5e1; padding:0.45rem 0.65rem; text-align:center;
      tr.appendChild(td);
    });
    table.appendChild(tr);
  });
}

function redraw() {
  const a = Number(aRow.input.value), b = Number(bRow.input.value), c = Number(cRow.input.value
  const v1 = Number(rhs1.input.value), v2 = Number(rhs2.input.value), dv = Number(delta.input.v
  [aRow, bRow, cRow, dRow, rhs1, rhs2, delta].forEach(r => r.out.textContent = Number(r.input.v
  const inv = inv2(a, b, c, d);
  tableFrom(aTable, "A", [[a.toFixed(1), b.toFixed(1)], [c.toFixed(1), d.toFixed(1)]], "#0f172a
  if (!inv) {
    tableFrom(xTable, "x = A⁻¹b", [["singular", "matrix"]], "#dc2626");
    tableFrom(xpTable, "x + ⍰x", [["not defined", ""]], "#dc2626");
```

```
      detBox.textContent = `det(A) = ${(a * d - b * c).toFixed(3)}`;
      kappaBox.textContent = "□∞(A) = ∞";
      sensBox.textContent = "small change in b can produce arbitrarily large change in x";
      caption.textContent = "At singularity the inverse does not exist, so the linear system cannot
      return;
    }
    const x = mul(inv, v1, v2);
    const xp = mul(inv, v1 + dv, v2);
    const ainvNorm = normInf(...inv.m);
    const kappa = normInf(a, b, c, d) * ainvNorm;
    const dxRel = Math.max(Math.abs(xp[0] - x[0]), Math.abs(xp[1] - x[1])) / Math.max(1e-9, Math.max
    tableFrom(xTable, "x = A⁻¹b", [[x[0].toFixed(3)], [x[1].toFixed(3)]], "#2563eb");
    tableFrom(xpTable, "x + □x", [[xp[0].toFixed(3)], [xp[1].toFixed(3)]], "#16a34a");
    detBox.textContent = `det(A) = ${inv.det.toFixed(3)}`;
    kappaBox.textContent = `□∞(A) □ ${kappa.toFixed(2)}`;
    sensBox.textContent = `relative solution change □ ${(100 * dxRel).toFixed(1)}%`;
    caption.textContent = kappa > 50
      ? "This matrix is ill-conditioned: a modest perturbation in the data is being amplified strong
      : "This matrix is reasonably conditioned: the solution changes, but not wildly, when the right
  }

  [aRow, bRow, cRow, dRow, rhs1, rhs2, delta].forEach(r => r.input.addEventListener("input", redraw)
  matrixWrap.append(aTable, xTable, xpTable);
  container.append(controls, summary, matrixWrap, caption);
  redraw();
  return container;
}
```

## 52.5. Iterative methods

Direct methods (GE, LU) compute the exact answer (up to rounding) in a fixed number of operations. Iterative methods start from an initial guess and update it repeatedly. They're preferred when $A$ is large and sparse — a $10^6 \times 10^6$ FEM stiffness matrix, for example, where storing $L$ and $U$ in full is impractical.

### 52.5.1. Jacobi iteration

Rewrite each equation for its diagonal variable, holding all other variables fixed at their current values.

For row $i$ of $A\mathbf{x} = \mathbf{b}$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

All updates use values from the **previous** iterate $x^{(k)}$. This makes Jacobi naturally parallelisable — each new $x_i^{(k+1)}$ is independent of the others being computed in the same step.

### 52.5.2. Gauss-Seidel iteration

The same formula, but use the **latest available** values — if $x_j^{(k+1)}$ has already been computed in this sweep, use it immediately:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right)$$

Gauss-Seidel typically converges in roughly half the iterations of Jacobi because it incorporates fresh information sooner. The trade-off: it cannot be parallelised as straightforwardly.

### 52.5.3. Convergence criterion: diagonal dominance

Both methods are guaranteed to converge if $A$ is **strictly diagonally dominant**:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for all } i$$

This means each diagonal entry exceeds the sum of all off-diagonal entries in its row, in absolute value. FEM stiffness matrices and many physical systems (heat diffusion, network flow) naturally satisfy this condition.

When diagonal dominance holds, the error is guaranteed to shrink at each step and the iteration converges from any starting point. (For those familiar with eigenvalues: the convergence criterion is that the spectral radius of the iteration matrix — the largest absolute eigenvalue of the matrix that multiplies the error — is less than 1.)

### 52.5.4. Worked example: Jacobi and Gauss-Seidel on a 3×3 system

Solve from $\mathbf{x}^{(0)} = (0, 0, 0)^T$:

$$\begin{pmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 6 \\ 25 \\ -11 \end{pmatrix}$$

Check diagonal dominance: Row 1: $10 > |-1| + |2| = 3$ ☒; Row 2: $11 > |-1| + |-1| = 2$ ☒; Row 3: $10 > |2| + |-1| = 3$ ☒. Convergence guaranteed.

The exact solution is $\mathbf{x} = (1, 2, -1)^T$.

#### 52.5.4.1. Jacobi, 3 iterations

Iteration formulas:

$$x_1^{(k+1)} = \frac{1}{10}(6 + x_2^{(k)} - 2x_3^{(k)})$$

$$x_2^{(k+1)} = \frac{1}{11}(25 + x_1^{(k)} + x_3^{(k)})$$

$$x_3^{(k+1)} = \frac{1}{10}(-11 - 2x_1^{(k)} + x_2^{(k)})$$

$k = 0 \to k = 1$**:** Using $\mathbf{x}^{(0)} = (0, 0, 0)^T$:

$x_1^{(1)} = (6 + 0 - 0)/10 = 0.6$

$x_2^{(1)} = (25 + 0 + 0)/11 \approx 2.2727$

$x_3^{(1)} = (-11 - 0 + 0)/10 = -1.1$

$k = 1 \to k = 2$**:**

$x_1^{(2)} = (6 + 2.2727 - 2(-1.1))/10 = (6 + 2.2727 + 2.2)/10 = 10.4727/10 = 1.0473$

$x_2^{(2)} = (25 + 0.6 + (-1.1))/11 = 24.5/11 = 2.2273$

$x_3^{(2)} = (-11 - 2(0.6) + 2.2727)/10 = (-11 - 1.2 + 2.2727)/10 = -9.9273/10 = -0.9927$

$k = 2 \to k = 3$**:**

$x_1^{(3)} = (6 + 2.2273 - 2(-0.9927))/10 = (6 + 2.2273 + 1.9854)/10 = 10.2127/10 = 1.0213$

$x_2^{(3)} = (25 + 1.0473 + (-0.9927))/11 = (25 + 0.0546)/11 = 25.0546/11 \approx 2.278$

$x_3^{(3)} = (-11 - 2(1.0473) + 2.2273)/10 = (-11 - 2.0946 + 2.2273)/10 = -10.8673/10 = -1.087$

After 3 iterations: $\mathbf{x}^{(3)} \approx (1.021, 2.278, -1.087)^T$. Errors are shrinking — heading toward the exact solution $(1, 2, -1)^T$, but Jacobi converges slowly on this system.

#### 52.5.4.2. Gauss-Seidel, 3 iterations

$k = 0 \to k = 1$**:** Using **updated** values as soon as available:

$x_1^{(1)} = (6 + 0 - 0)/10 = 0.6$ (uses $x_2^{(0)} = 0$, $x_3^{(0)} = 0$)

$x_2^{(1)} = (25 + \mathbf{0.6} + 0)/11 = 25.6/11 \approx 2.3273$ (uses new $x_1^{(1)} = 0.6$)

$x_3^{(1)} = (-11 - 2(\mathbf{0.6}) + \mathbf{2.3273})/10 = (-11 - 1.2 + 2.3273)/10 = -9.8727/10 = -0.9873$ (uses new $x_1^{(1)}$ and $x_2^{(1)}$)

$k = 1 \to k = 2$**:**

$x_1^{(2)} = (6 + 2.3273 - 2(-0.9873))/10 = (6 + 2.3273 + 1.9746)/10 = 10.3019/10 = 1.0302$

$x_2^{(2)} = (25 + 1.0302 + (-0.9873))/11 = 25.0429/11 \approx 2.2766$

$x_3^{(2)} = (-11 - 2(1.0302) + 2.2766)/10 = (-11 - 2.0604 + 2.2766)/10 = -10.7838/10 = -1.0784$

$k = 2 \to k = 3$**:**

$x_1^{(3)} = (6 + 2.2766 - 2(-1.0784))/10 = (6 + 2.2766 + 2.1568)/10 = 10.4334/10 = 1.0433$

$x_2^{(3)} = (25 + 1.0433 + (-1.0784))/11 = 24.9649/11 \approx 2.2695$

$x_3^{(3)} = (-11 - 2(1.0433) + 2.2695)/10 = (-11 - 2.0866 + 2.2695)/10 = -10.8171/10 = -1.0817$

After 3 iterations, Gauss-Seidel is at $\mathbf{x}^{(3)} \approx (1.043, 2.270, -1.082)^T$. Errors are still about 4%, similar to Jacobi after 3 steps on this particular system. Gauss-Seidel does converge in roughly half the iterations of Jacobi in general, but the advantage shows more clearly in systems where the off-diagonal coupling is stronger or after more iterations have been taken.

**When to use iterative vs direct**

| Criterion | Direct (LU) | Iterative (GS/Jacobi) |
|---|---|---|
| Matrix size | Any, up to memory | Large sparse matrices |
| Sparsity | Not exploited | Natural fit |
| Multiple RHS | Fast after factorisation | Re-iterate each time |
| Accuracy | Machine precision | Stops at tolerance |
| Convergence guarantee | Always (if non-singular) | Diagonal dominance |

For FEM systems with $n > 10^5$, iterative methods are standard. For $n < 10^3$ with multiple right-hand sides, LU with pivoting is preferable.

## 52.6. Where this goes

This chapter handles $A\mathbf{x} = \mathbf{b}$ for fixed $A$ and $\mathbf{b}$. The next chapter in this section extends the same ideas to **numerical ODEs and PDEs**: discretising a differential equation produces a linear system at every time step or on every mesh — the same pivoted LU or iterative machinery applies, just called millions of times. The structure of the system (banded, sparse, symmetric) determines which variant to use, and the condition number of the discretised operator determines the mesh resolution needed to stay in the accurate regime.

The conditioning ideas carry directly into **optimisation** (Vol 7, optimisation section): the Hessian matrix of an objective function plays the role of $A$, and ill-conditioning of the Hessian is the primary reason gradient-descent methods stall or diverge. Knowing how to estimate $\kappa$ and how to precondition a matrix (rescaling so that $\kappa$ drops) is practical currency in large-scale numerical optimisation.

**Where this shows up**

- **Structural engineering:** Every finite element analysis assembles a stiffness matrix $K$ and solves $K\mathbf{u} = \mathbf{f}$ for displacements $\mathbf{u}$. LU factorisation with multiple load vectors is standard.
- **Computational fluid dynamics:** The pressure-correction step in incompressible flow solvers is a large sparse Poisson system — iterative solvers (conjugate gradient, a Krylov subspace method that exploits symmetry and positive-definiteness) are universal.
- **Machine learning:** Linear regression solves the normal equations $(X^TX)\boldsymbol{\beta} = X^T\mathbf{y}$; the condition number of $X^TX$ is why multicollinearity ruins regression estimates.
- **Finance:** Estimating a covariance matrix from historical returns produces an ill-conditioned system when assets are correlated — the condition number flags when the inverse is unreliable.

## 52.7. Exercises

These are engineering-layer puzzles. Each has a specific numerical answer. Work the full procedure — don't skip steps — because the errors that matter in practice happen in the steps that get skipped in rough work.

**Exercise 1.** Factorise the following matrix using Doolittle LU decomposition, then use the factorisation to solve for two right-hand sides.

$$A = \begin{pmatrix} 2 & 4 & 2 \\ 1 & 5 & 2 \\ 4 & 6 & 8 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 2 \\ 3 \\ 6 \end{pmatrix}, \quad \mathbf{b}_2 = \begin{pmatrix} 0 \\ 1 \\ -2 \end{pmatrix}$$

**Exercise 2.** Apply partial pivoting to the following near-singular system. Identify which row swap is required, perform the swap, and carry out elimination. Show what would go wrong without the swap.

$$\begin{pmatrix} 0.002 & 1 & 1 \\ 1 & 2 & 5 \\ 1 & -1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 8 \\ 5 \end{pmatrix}$$

**Exercise 3.** Compute $\kappa_\infty(A)$ for the matrix below, then interpret the result: if you solve $A\mathbf{x} = \mathbf{b}$ on a machine with 15 significant digits, how many reliable digits do you expect in the solution?

$$A = \begin{pmatrix} 3 & 1 \\ 2 & 4 \end{pmatrix}$$

**Exercise 4.** Apply 3 iterations of Jacobi iteration to the following system, starting from $\mathbf{x}^{(0)} = (0, 0, 0)^T$. Show the iterate after each step.

$$\begin{pmatrix} 5 & 1 & 1 \\ 1 & 6 & 1 \\ 1 & 1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 8 \\ 6 \end{pmatrix}$$

The exact solution is $\mathbf{x} = (1, 1, 1)^T$.

**Exercise 5.** Apply 3 iterations of Gauss-Seidel to the same system as Exercise 4, from the same starting point $\mathbf{x}^{(0)} = (0, 0, 0)^T$. Compare the convergence rate after 3 iterations.

**Exercise 6 (Engineering context).** A four-node plane truss is modelled by finite element analysis. After assembly and boundary conditions, the reduced stiffness system is:

$$K = \begin{pmatrix} 4 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 \\ 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 3 \end{pmatrix}, \quad \mathbf{f} = \begin{pmatrix} 0 \\ 0 \\ 10 \\ 5 \end{pmatrix}$$

(a) Verify that $K$ is strictly diagonally dominant, which guarantees that Gauss-Seidel converges for any load vector $\mathbf{f}$.

(b) Identify the structural feature of $K$ (bandwidth, sparsity pattern) that explains why iterative methods are preferred over LU decomposition for large truss problems of this type.

(c) Apply 2 iterations of Gauss-Seidel from $\mathbf{u}^{(0)} = (0, 0, 0, 0)^T$.

- op: "Identify structural feature: K is tridiagonal (bandwidth 1)" eq: "\text{Only entries }k_{i,i},\;k_{i,i\pm 1}\text{ are non-zero — all others are zero}" note: "For large n, LU decomposition of a tridiagonal matrix remains tridiagonal (no fill-in), but for general sparse FEM matrices, LU can produce dense factors. Iterative methods work directly on the sparse structure without fill-in."
- op: "Write Gauss-Seidel formulas for each DOF" eq: "u_1^{(k+1)}=\frac{1}{4}(0+u_2^{(k)}),\quad u_2^{(k+1)}=\frac{1}{4}(0+u_1^{(k+1)}+u_3^{(k)}),\quad u_3^{(k+1)}=\frac{1}{4}(10+u_2^{(k+1)}+u_4^{(k)}),\quad u_4^{(k+1)}=\frac{1}{3}(5+u_3^{(k+1)})" note: ~
- op: "Iteration 1 with u^(0)=(0,0,0,0)^T" eq: "u_1^{(1)}=0/4=0,\quad u_2^{(1)}=(0+0)/4=0,\quad u_3^{(1)}=(10+0+0)/4=2.5,\quad u_4^{(1)}=(5+2.5)/3=2.5" note: ~
- op: "Iteration 2 with u^(1)=(0,0,2.5,2.5)^T" eq: "u_1^{(2)}=0/4=0,\quad u_2^{(2)}=(0+0+2.5)/4=0.625,\quad u_3^{(2)}=(10+0.625+2.5)/4=3.281,\quad u_4^{(2)}=(5+3.281)/3=2.760" note: ~

- op: “Physical interpretation of the iterates” eq: “\text{Node 4 displacement grows from 0 to 2.5 to 2.760 (in whatever units the stiffness and force are expressed) — iteration revealing the load path through the truss}” note: “The iterative process is physically meaningful: each sweep propagates the applied loads one structural connection further through the mesh. No units are assigned in this dimensionless example; in a real FEM model the stiffness entries carry units of kN/m and forces carry kN, giving displacements in metres.”
- op: “Check convergence direction” eq: “\text{Residual }\|K\mathbf{u}^{(2)}-\mathbf{f}\|\infty:\quad K\mathbf{u}^{(2)}\approx(-0.625,\;-0.781,\;9.739,\;4.999)^T,\quad\text{vs }\mathbf{f}=(0,0,10,5)^T” note: ”The residual is shrinking; continue until $\|Ku - f\|_\infty$ < tolerance (e.g. $10^{-6}$).” –>

# 53. Numerical ODEs and PDEs

Stepping through time and space

## 53.1. The setup: replacing dy/dx with Δy/Δx

The initial-value problem has the form

$$y' = f(x, y), \quad y(x_0) = y_0.$$

Read this as: "the slope of $y$ at any point $(x, y)$ is given by the function $f$, and we know where we start." The question is: what is $y$ at some later $x$?

If $f$ is simple enough, calculus gives you $y(x)$ directly. When it isn't, you fall back on the most basic approximation: the tangent-line step.

The **step size** $h$ — a small positive number — is your control parameter. Smaller $h$ means more accuracy and more work. The methods below differ in how many function evaluations they use per step, and therefore in how quickly the error shrinks as $h$ decreases.

---

## 53.2. Euler's method

### 53.2.1. Forward Euler

Approximate $y'(x_n) \approx \dfrac{y_{n+1} - y_n}{h}$, rearrange, and you get

$$y_{n+1} = y_n + h\, f(x_n, y_n).$$

$y_n$ means "the approximate value of $y$ at step $n$." The update rule says: take the current value, add the step size times the slope at the current point.

**Local truncation error.** Taylor-expand $y(x_{n+1})$ about $x_n$:

$$y(x_{n+1}) = y(x_n) + hy'(x_n) + \frac{h^2}{2} y''(x_n) + \cdots$$

Forward Euler keeps only the first-derivative term, so the error in one step is $O(h^2)$. Over $N = (b-a)/h$ steps, the errors accumulate: **global error is** $O(h)$. Halve the step size, halve the global error.

### 53.2.2. Backward (implicit) Euler

$$y_{n+1} = y_n + h\, f(x_{n+1}, y_{n+1}).$$

The slope is evaluated at the *new* point, which makes this an equation you must solve for $y_{n+1}$ at each step — more expensive per step. The payoff is unconditional stability for stiff equations (equations where the true solution decays quickly and explicit methods require an extremely small $h$ to stay bounded). For non-stiff problems, forward Euler is fine.

> **Note**
>
> **What "stiff" means.** A stiff ODE has solution components that decay at very different rates. The fast-decaying component forces $h$ to be tiny for forward Euler to remain stable, even though that component itself becomes negligible quickly. Implicit methods remain stable at larger $h$ because they incorporate information from the next step.

### 53.2.3. Worked example: forward Euler on $y' = -2y$

Exact solution: $y = e^{-2x}$. Let $h = 0.5$, $y_0 = 1$.

| $n$ | $x_n$ | $y_n$ | $f = -2y_n$ | $y_{n+1} = y_n + hf$ |
|---|---|---|---|---|
| 0 | 0.0 | 1.0000 | −2.0000 | 1 + 0.5(−2) = 0.0000 |

That's the stability boundary. For an equation of the form $y' = \lambda y$— where $\lambda$ is the coefficient multiplying $y$, here $\lambda = -2$ — forward Euler multiplies the solution by $(1 + h\lambda)$ at each step. Stability requires $|1 + h\lambda| \leq 1$, which for $\lambda = -2$ gives $h \leq 1$. With $h = 0.5$ we land exactly at the edge: the solution collapses to zero and stays there. For $h > 1$ the factor $(1 + h\lambda)$ exceeds 1 in magnitude, and the computed solution oscillates with growing amplitude — blowing up — even though the true solution decays smoothly to zero. Use $h = 0.2$ to see the method tracking the decay:

| $n$ | $x_n$ | $y_n$ | $f = -2y_n$ | $y_{n+1}$ | Exact $e^{-2x}$ |
|---|---|---|---|---|---|
| 0 | 0.0 | 1.0000 | −2.0000 | 0.6000 | — |
| 1 | 0.2 | 0.6000 | −1.2000 | 0.3600 | 0.6703 |
| 2 | 0.4 | 0.3600 | −0.7200 | 0.2160 | 0.4493 |

The error at $x = 0.4$ is about 0.089, or roughly 20%. Three steps with $h = 0.2$ on a decaying exponential is not a recipe for accuracy — but it illustrates the mechanics.

Use the step-size slider to compare how Euler and RK4 accumulate error on the same simple growth problem.

```
//| echo: false
{
  const exact = x => Math.exp(x);
  const f = (x, y) => y;

  function solve(h) {
    const xs = [0], euler = [1], rk4 = [1];
    const n = Math.round(2 / h);
    for (let i = 0; i < n; i++) {
      const x = xs[i], yE = euler[i], yR = rk4[i];
      xs.push(x + h);
      euler.push(yE + h * f(x, yE));
      const k1 = f(x, yR);
      const k2 = f(x + h / 2, yR + h * k1 / 2);
      const k3 = f(x + h / 2, yR + h * k2 / 2);
      const k4 = f(x + h, yR + h * k3);
      rk4.push(yR + h * (k1 + 2 * k2 + 2 * k3 + k4) / 6);
    }
    return { xs, euler, rk4 };
  }

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem
  const label = document.createElement("label"); label.textContent = "Step size h";
  const slider = document.createElement("input");
  slider.type = "range"; slider.min = "0.1"; slider.max = "1"; slider.step = "0.1"; slider.value
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric:tabular-nums; min-width:3rem;";
  controls.append(label, slider, out);

  const summary = document.createElement("div");
  summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3
  const eBox = document.createElement("span");
  const rBox = document.createElement("span");
  const nBox = document.createElement("span");
  summary.append(eBox, rBox, nBox);

  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 720 380");
  svg.style.width = "100%";
  svg.style.height = "auto";
  svg.style.border = "1px solid #e2e8f0";
  svg.style.background = "#fff";
```

```
  const caption = document.createElement("p");
  caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

  function redraw() {
    const h = Number(slider.value);
    out.textContent = h.toFixed(1);
    const { xs, euler, rk4 } = solve(h);
    svg.replaceChildren();
    const xMin = 0, xMax = 2, yMin = 0.8, yMax = 8;
    const sx = x => 55 + (x - xMin) / (xMax - xMin) * 620;
    const sy = y => 325 - (y - yMin) / (yMax - yMin) * 255;

    const axisX = svgEl("line");
    axisX.setAttribute("x1", "55"); axisX.setAttribute("y1", "325");
    axisX.setAttribute("x2", "675"); axisX.setAttribute("y2", "325");
    axisX.setAttribute("stroke", "#64748b"); axisX.setAttribute("stroke-width", "1.2");
    const axisY = svgEl("line");
    axisY.setAttribute("x1", "55"); axisY.setAttribute("y1", "325");
    axisY.setAttribute("x2", "55"); axisY.setAttribute("y2", "45");
    axisY.setAttribute("stroke", "#64748b"); axisY.setAttribute("stroke-width", "1.2");
    svg.append(axisX, axisY);

    const exactPath = [];
    for (let i = 0; i < 220; i++) {
      const x = xMin + i / 219 * (xMax - xMin);
      exactPath.push(`${i === 0 ? "M" : "L"} ${sx(x)} ${sy(exact(x))}`);
    }
    [["#0f172a", exactPath.join(" "), 3], ["#dc2626", xs.map((x, i) => `${i === 0 ? "M" : "L"} ${sx
      const p = svgEl("path");
      p.setAttribute("d", d); p.setAttribute("fill", "none"); p.setAttribute("stroke", color); p.set
      svg.appendChild(p);
    });

    const eErr = Math.abs(euler.at(-1) - exact(2));
    const rErr = Math.abs(rk4.at(-1) - exact(2));
    eBox.textContent = `Euler error at x = 2: ${eErr.toFixed(4)}`;
    rBox.textContent = `RK4 error at x = 2: ${rErr.toFixed(4)}`;
    nBox.textContent = `steps: ${xs.length - 1}`;
    caption.textContent = "Black is the exact solution e^x. With the same step size, Euler drifts v
  }

  slider.addEventListener("input", redraw);
  container.append(controls, summary, svg, caption);
  redraw();
  return container;
}
```

## 53.3. Runge-Kutta methods

The key idea: use several evaluations of $f$ within a single step — at different intermediate points — then combine them into a weighted average slope. More work per step, but the error drops much faster with decreasing $h$.

### 53.3.1. RK2: two-slope methods

**Midpoint method.** Take a half-step to estimate the midpoint, then use the slope there:

$$k_1 = f(x_n, y_n), \qquad k_2 = f\Big(x_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_1\Big)$$

$$y_{n+1} = y_n + hk_2.$$

**Heun's method (modified Euler).** Use the slope at the beginning and the predicted end, average them:

$$k_1 = f(x_n, y_n), \qquad k_2 = f(x_n + h,\ y_n + hk_1)$$

$$y_{n+1} = y_n + \tfrac{h}{2}(k_1 + k_2).$$

Both are $O(h^2)$ per step, giving global error $O(h^2)$. Halving $h$ divides the error by 4.

### 53.3.2. RK4: the standard workhorse

Evaluate four slopes — $k_1, k_2, k_3, k_4$ — and combine with weights $1, 2, 2, 1$:

$$k_1 = f(x_n,\ y_n)$$

$$k_2 = f\Big(x_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_1\Big)$$

$$k_3 = f\Big(x_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_2\Big)$$

$$k_4 = f(x_n + h,\ y_n + hk_3)$$

$$y_{n+1} = y_n + \tfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

$k_1$ is the slope at the left end. $k_2$ and $k_3$ are two estimates of the midpoint slope (the second corrects the first). $k_4$ is the slope at the right end. The weights $1, 2, 2, 1$ come from Simpson's rule: the interval has three evaluation points — left end, midpoint, right end — with Simpson weights $1, 4, 1$. RK4 has two independent midpoint estimates rather than one, so their weight is split, giving $1, 2, 2, 1$ after normalising to sum to 6.

**Error.** Local error is $O(h^5)$; global error is $O(h^4)$. Halving $h$ divides the global error by 16. This is why RK4 is the default choice for non-stiff problems: it is usually accurate enough with step sizes that are not punishingly small.

> **Note**
>
> **Adaptive step size.** RK4 with a fixed $h$ wastes work in regions where the solution is smooth and underestimates error in regions where it changes rapidly. Adaptive methods (e.g. RK45, the Dormand-Prince method) compute two approximations of different orders at each step, compare them to estimate the local error, and adjust $h$ to keep the error within a specified tolerance. Embedded pairs like RK23 and RK45 reuse slope evaluations between the two orders, keeping the extra cost small. The implementation is routine; the idea is simply: use $h$ proportional to what the solution needs.

### 53.3.3. Worked example: RK4 on $y' = y$, one step

$y(0) = 1$, $h = 1$. Exact: $y(1) = e \approx 2.71828$.

$$k_1 = f(0, 1) = 1$$

$$k_2 = f(0.5,\ 1 + 0.5) = f(0.5, 1.5) = 1.5$$

$$k_3 = f(0.5,\ 1 + 0.5 \times 1.5) = f(0.5, 1.75) = 1.75$$

$$k_4 = f(1,\ 1 + 1 \times 1.75) = f(1, 2.75) = 2.75$$

$$y_1 = 1 + \tfrac{1}{6}(1 + 3 + 3.5 + 2.75) = 1 + \tfrac{10.25}{6} \approx 2.7083.$$

Error: $|2.7083 - 2.71828| \approx 0.00998$. With a single step of $h = 1$, RK4 gets the answer to 3 significant figures. Forward Euler with $h = 1$ gives $y_1 = 2.0$ — an error of 26%.

---

## 53.4. Systems of ODEs

Many physical problems involve second- (or higher-) order ODEs. A second-order ODE can always be rewritten as a system of two first-order ODEs, and then Euler or RK4 applies to the system directly.

**Reduction procedure.** Given

$$y'' + p(x)y' + q(x)y = r(x),$$

introduce $u_1 = y$ and $u_2 = y'$. Then:

$$u_1' = u_2$$

$$u_2' = r(x) - p(x)u_2 - q(x)u_1.$$

Write this as $\mathbf{u}' = \mathbf{F}(x, \mathbf{u})$ where $\mathbf{u} = [u_1, u_2]^T$. Apply Euler or RK4 to the vector equation: each $k_i$ becomes a two-component vector.

### 53.4.1. Worked example: damped oscillator

$y'' + 0.5y' + 4y = 0$, $y(0) = 1$, $y'(0) = 0$, $h = 0.1$.

Let $u_1 = y$, $u_2 = y'$. Then:

$$u_1' = u_2, \qquad u_2' = -4u_1 - 0.5u_2.$$

One Euler step:

$$k_1^{(1)} = u_2^{(0)} = 0, \qquad k_1^{(2)} = -4(1) - 0.5(0) = -4.$$

$$u_1^{(1)} = 1 + 0.1 \times 0 = 1.0, \qquad u_2^{(1)} = 0 + 0.1 \times (-4) = -0.4.$$

So after one step: $y(0.1) \approx 1.0$, $y'(0.1) \approx -0.4$. The method is mechanical once the system is written down; the reduction step is the only place judgment is required.

---

## 53.5. Finite differences for PDEs

The same idea — replace a derivative with a finite ratio — extends to PDEs, where you discretise both time and space.

### 53.5.1. The 1D heat equation

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad 0 \le x \le L, \quad t \ge 0,$$

with given initial condition $u(x, 0)$ and boundary conditions. $\alpha$ is thermal diffusivity (a positive constant).

Lay a grid: $x_j = j\Delta x$ for $j = 0, 1, \ldots, M$ and $t^n = n\Delta t$ for $n = 0, 1, \ldots$. Write $u_j^n$ for the approximation to $u(x_j, t^n)$.

**FTCS scheme (forward-time, central-space).** Use a forward difference in time and a central difference in space:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \alpha \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{(\Delta x)^2}.$$

Define the **Fourier number** (also called the mesh ratio)

$$r = \frac{\alpha \Delta t}{(\Delta x)^2}.$$

Rearrange:

$$u_j^{n+1} = r u_{j-1}^n + (1 - 2r)\, u_j^n + r u_{j+1}^n.$$

This is explicit: each new value depends only on known values at the previous time level.

**Stability condition.** Look at the FTCS update coefficients: $r$, $(1 - 2r)$, $r$. When $r \le 1/2$, the coefficient $(1 - 2r)$ is non-negative, so the new value at each interior node is a weighted average of the neighbouring values — physically reasonable, bounded, and stable. When $r > 1/2$, the coefficient $(1 - 2r)$ turns negative. The centre node's own value now enters with a negative weight: the update is no longer an average. A small perturbation at one node gets amplified rather than smoothed; each time step multiplies the perturbation, and the solution oscillates with growing amplitude until it overflows. The physical heat equation diffuses and decays; the numerical scheme with an oversized time step does the opposite — it invents energy out of rounding errors. Von Neumann stability analysis makes this precise: $r \le \frac{1}{2}$ is the condition. In practice, for $\alpha = 1$, $\Delta x = 0.1$: the maximum stable $\Delta t$ is $\frac{1}{2}(0.1)^2 = 0.005$.

Use the controls to see the FTCS heat solver behave like a smoother when the stability condition holds, and like a source of fake oscillations when it does not.

```
//| echo: false
{
  const alpha = 1;
  function stepHeat(dx, dt, steps) {
    const n = Math.round(1 / dx);
    let u = Array.from({ length: n + 1 }, (_, j) => {
      const z = (j * dx - 0.5) / 0.08;
      return Math.exp(-(z * z));
```

```
  });
  u[0] = 0; u[n] = 0;
  const r = alpha * dt / (dx * dx);
  for (let s = 0; s < steps; s++) {
    const next = u.slice();
    for (let j = 1; j < n; j++) next[j] = r * u[j - 1] + (1 - 2 * r) * u[j] + r * u[j + 1];
    next[0] = 0; next[n] = 0;
    u = next;
  }
  return { u, r, n };
}

function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
const container = document.createElement("div");
container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem
function addSlider(labelText, min, max, step, value) {
  const label = document.createElement("label"); label.textContent = labelText;
  const input = document.createElement("input");
  input.type = "range"; input.min = String(min); input.max = String(max); input.step = String(s
  const out = document.createElement("span"); out.style.cssText = "font-variant-numeric:tabular
  controls.append(label, input, out);
  return { input, out };
}
const dxRow = addSlider("x", 0.05, 0.2, 0.01, 0.10);
const dtRow = addSlider("t", 0.001, 0.02, 0.001, 0.005);
const tRow = addSlider("Time steps", 1, 40, 1, 12);

const summary = document.createElement("div");
summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3
const rBox = document.createElement("span");
const stateBox = document.createElement("span");
summary.append(rBox, stateBox);

const svg = svgEl("svg");
svg.setAttribute("viewBox", "0 0 720 360");
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

function redraw() {
  const dx = Number(dxRow.input.value), dt = Number(dtRow.input.value), steps = Number(tRow.inp
```

```
  dxRow.out.textContent = dx.toFixed(2);
  dtRow.out.textContent = dt.toFixed(3);
  tRow.out.textContent = String(steps);
  const { u, r, n } = stepHeat(dx, dt, steps);
  svg.replaceChildren();
  const sx = x => 55 + x * 620;
  const sy = y => 300 - y * 220;

  const axisX = svgEl("line");
  axisX.setAttribute("x1", "55"); axisX.setAttribute("y1", "300");
  axisX.setAttribute("x2", "675"); axisX.setAttribute("y2", "300");
  axisX.setAttribute("stroke", "#64748b"); axisX.setAttribute("stroke-width", "1.2");
  svg.appendChild(axisX);

  const path = u.map((val, j) => `${j === 0 ? "M" : "L"} ${sx(j / n)} ${sy(val)}`).join(" ");
  const curve = svgEl("path");
  curve.setAttribute("d", path);
  curve.setAttribute("fill", "none");
  curve.setAttribute("stroke", r <= 0.5 ? "#2563eb" : "#dc2626");
  curve.setAttribute("stroke-width", "3");
  svg.appendChild(curve);

  if (r > 0.5) {
    const warn = svgEl("rect");
    warn.setAttribute("x", "110"); warn.setAttribute("y", "55");
    warn.setAttribute("width", "510"); warn.setAttribute("height", "42");
    warn.setAttribute("rx", "8");
    warn.setAttribute("fill", "rgba(220,38,38,0.14)");
    svg.appendChild(warn);
    const t = svgEl("text");
    t.setAttribute("x", "365"); t.setAttribute("y", "81");
    t.setAttribute("text-anchor", "middle");
    t.setAttribute("font-size", "15");
    t.setAttribute("font-weight", "700");
    t.setAttribute("fill", "#b91c1c");
    t.textContent = "r > 1/2: unstable update, oscillations can grow";
    svg.appendChild(t);
  }

  rBox.textContent = `r = t/x² = ${r.toFixed(3)}`;
  stateBox.textContent = r <= 0.5 ? "stable weighted-average regime" : "unstable regime";
  caption.textContent = "For r  1/2 the scheme behaves like diffusion and smooths the spike. Once
}

[dxRow.input, dtRow.input, tRow.input].forEach(el => el.addEventListener("input", redraw));
container.append(controls, summary, svg, caption);
redraw();
```

```
  return container;
}
```

### 53.5.2. Crank-Nicolson: unconditional stability

Average the spatial derivative between time levels $n$ and $n+1$:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \frac{\alpha}{2}\left[\frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{(\Delta x)^2} + \frac{u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}}{(\Delta x)^2}\right].$$

The unknowns at time level $n+1$ appear on both sides. Collecting them gives a tridiagonal linear system to solve at each time step — implicit, but only marginally more expensive than FTCS because tridiagonal systems have a special structure: each equation involves only its immediate neighbours, so a forward sweep followed by back-substitution solves the whole system in $O(N)$ operations. This procedure is called the **Thomas algorithm** — Gaussian elimination stripped down to exploit the tridiagonal sparsity pattern. The method is unconditionally stable: any $r$ is admissible, so you can take large time steps without blowup. The global error is $O((\Delta t)^2 + (\Delta x)^2)$ — second order in both.

### 53.5.3. Boundary-value problems: tridiagonal systems from central differences

For a 1D BVP:

$$y'' = f(x), \quad y(a) = \alpha, \quad y(b) = \beta,$$

discretise the interior points $x_1, \dots, x_{N-1}$ with spacing $h = (b-a)/N$. The central-difference approximation $y'' \approx (y_{i-1} - 2y_i + y_{i+1})/h^2$ at each interior point gives:

$$y_{i-1} - 2y_i + y_{i+1} = h^2 f(x_i), \quad i = 1, \dots, N-1.$$

Collect these equations (incorporating $y_0 = \alpha$ and $y_N = \beta$ into the first and last equations) and you get a tridiagonal linear system. The structure is

$$\begin{pmatrix} -2 & 1 & & \\ 1 & -2 & 1 & \\ & \ddots & \ddots & 1 \\ & & 1 & -2 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{N-1} \end{pmatrix} = h^2 \begin{pmatrix} f(x_1) - \alpha/h^2 \\ f(x_2) \\ \vdots \\ f(x_{N-1}) - \beta/h^2 \end{pmatrix}.$$

Solve with the Thomas algorithm (forward sweep + back substitution) in $O(N)$ operations.

> **Note**
>
> **Why the tridiagonal structure matters.** Gaussian elimination on a dense $N \times N$ system costs $O(N^3)$. Exploiting the tridiagonal structure reduces this to $O(N)$. For $N = 1000$ spatial nodes, that is a factor of $10^9$ in computational work. Recognising sparsity is not a nicety; it is what makes

large-scale simulation possible.

---

## 53.6. Worked example: FTCS one step

Heat equation on $[0, 1]$ with 5 spatial nodes ($\Delta x = 0.25$), $\alpha = 1$, $\Delta t = 0.02$.

First check stability: $r = 1 \times 0.02/(0.25)^2 = 0.02/0.0625 = 0.32 < 0.5$. Stable.

Initial condition: $u(x, 0) = \sin(\pi x)$. Interior nodes: $x_1 = 0.25$, $x_2 = 0.50$, $x_3 = 0.75$.

Values at $t = 0$: $u_0^0 = 0$, $u_1^0 = \sin(0.25\pi) \approx 0.7071$, $u_2^0 = \sin(0.5\pi) = 1.0$, $u_3^0 = \sin(0.75\pi) \approx 0.7071$, $u_4^0 = 0$.

Apply $u_j^1 = 0.32u_{j-1}^0 + 0.36u_j^0 + 0.32u_{j+1}^0$ (coefficients: $r = 0.32$, $1 - 2r = 0.36$):

$$u_1^1 = 0.32(0) + 0.36(0.7071) + 0.32(1.0) = 0.2546 + 0.3200 = 0.5746$$

$$u_2^1 = 0.32(0.7071) + 0.36(1.0) + 0.32(0.7071) = 0.2263 + 0.3600 + 0.2263 = 0.8126$$

$$u_3^1 = 0.5746 \quad \text{(by symmetry)}$$

Exact solution: $u(x, t) = e^{-\pi^2 t} \sin(\pi x)$. At $t = 0.02$: $e^{-0.197} \approx 0.821$. So $u(0.5, 0.02)^{\text{exact}} = 0.821$. Numerical: 0.8126. Error $\approx 0.008$.

---

## 53.7. Where this goes

The methods here are the direct input to **adaptive ODE solvers** (e.g. `scipy.integrate.solve_ivp`, MATLAB's `ode45`). Those solvers use embedded Runge-Kutta pairs — the same structure as above — with automatic step-size control. Understanding the underlying stepping scheme lets you diagnose solver failures: a stiff ODE switched to an explicit method, or a step-size tolerance too loose for the problem's dynamics.

Finite differences for PDEs extend naturally to **finite element methods** (FEM), where the domain is discretised into irregular elements rather than a regular grid. FEM is the foundation of structural analysis, fluid simulation, and electromagnetic modelling. The tridiagonal system from 1D BVPs generalises to sparse banded systems in higher dimensions — still tractable, still solved by exploiting the sparsity pattern.

## 53.8. Applications

Where numerical ODE/PDE methods appear

- **Structural engineering.** Finite element codes (ANSYS, Abaqus) step through nonlinear load-displacement ODEs and solve elliptic PDEs for stress fields.
- **Aerospace.** Flight simulators use RK4 (or RK45) to integrate the 6-DOF equations of motion. Autopilot *design* uses frequency-domain tools (Laplace, Bode analysis), but the plant model is validated by time-domain simulation using exactly these stepping methods.
- **Climate science.** Atmospheric models discretise the primitive equations on a spherical grid with time steps of minutes; finite-difference or spectral schemes advance the state.
- **Biomedical.** Hodgkin-Huxley neuron models are stiff ODEs solved with implicit methods; cardiac electrophysiology models are stiff PDE systems.
- **Computer graphics.** Game physics engines use symplectic Euler (a variant of forward Euler that conserves energy better) to integrate rigid-body dynamics at 60–120 Hz.
- **Finance.** The Black-Scholes PDE for option pricing is discretised with Crank-Nicolson on a log-price grid.

---

## 53.9. Exercises

**Exercise 1.** A bacterial culture's population decreases due to an antibiotic. The concentration $y$ (mg/L) satisfies $dy/dx = -2y$, $y(0) = 1$. Apply forward Euler with $h = 0.1$ for three steps. Compare with the exact values $e^{-0.2}$, $e^{-0.4}$, $e^{-0.6}$.

**Exercise 2.** Apply RK4 to $dy/dx = -2y$, $y(0) = 1$ with a single step of $h = 0.3$. Compute all four slopes $k_1, k_2, k_3, k_4$ and the resulting $y_1$. Compare with the exact $e^{-0.6}$.

**Exercise 3.** A second-order system satisfies $y'' - 3y' + 2y = 0$, $y(0) = 0$, $y'(0) = 1$, $h = 0.1$.

(a) Reduce to a first-order system.
(b) Apply one step of forward Euler to get $y(0.1)$ and $y'(0.1)$.

**Exercise 4.** A 1D diffusion problem is modelled with diffusivity $\alpha = 0.5$ (dimensionless or in consistent units scaled to the rod length). The rod lies on $[0, 1]$ with 5 spatial nodes ($\Delta x = 0.25$). Initial condition: $u(x, 0) = \sin(\pi x)$. Boundary conditions: $u(0, t) = u(1, t) = 0$. Take $\Delta t = 0.01$.

Apply one FTCS time step. Give the value of $u$ at each interior node.

**Exercise 5.** Using the parameters from Exercise 4, verify analytically that the scheme is stable. Now choose $\Delta t = 0.04$ (with the same $\Delta x = 0.25$, $\alpha = 0.5$). Compute $r$ and explain what will happen if you run the scheme forward in time.

**Exercise 6.** Solve the boundary-value problem $y'' = -\pi^2 \sin(\pi x)$, $y(0) = 0$, $y(1) = 0$ numerically using central differences with 4 interior nodes ($h = 0.2$). Set up the $4 \times 4$ linear system and solve it. The exact solution is $y(x) = \sin(\pi x)$.

# Part XII.

# Volume 7 — Optimisation and probability

# 54. Linear programming

The best answer in a constrained space

A factory can make two products — call them chairs and tables. Each chair uses 2 board-hours of cutting time and 1 hour of finishing. Each table uses 1 board-hour of cutting and 3 hours of finishing. The factory has 8 cutting hours and 9 finishing hours available per day. Chairs sell at a $30 margin; tables at $50. How many of each should it make?

This is not a puzzle you can answer by intuition. Make only chairs and you run out of cutting time. Make only tables and finishing is the bottleneck. The mix that squeezes out the most profit sits somewhere in the middle, at a specific corner of the region defined by those constraints — and that is exactly what linear programming finds.

## 54.1. LP formulation

A **linear programme** (LP) has three components: *decision variables*, an *objective function*, and *constraints*.

**Decision variables.** These are the quantities you control. Write them as $x_1, x_2, \dots, x_n$ — read as "x-sub-one, x-sub-two, up to x-sub-n" — or as the vector $\mathbf{x}$, read as "bold x." For the factory problem, $x_1$ is the number of chairs made per day and $x_2$ is the number of tables.

**Objective function.** This is what you are optimising. It is always a linear combination of the decision variables: $z = c_1x_1 + c_2x_2 + \cdots + c_nx_n$, written compactly as $z = \mathbf{c}^\top\mathbf{x}$, read "z equals c-transpose times x." The coefficients $\mathbf{c}$ are your profit margins (for maximisation) or costs (for minimisation).

**Constraints.** Limits on what the variables can be. Each constraint is a linear inequality: $a_{i1}x_1 + a_{i2}x_2 + \cdots \leq b_i$. Non-negativity $\mathbf{x} \geq \mathbf{0}$ — read "x is non-negative" — is almost always included separately; you cannot make a negative number of chairs.

**The factory problem in LP form:**

$$\text{maximise} \quad z = 30x_1 + 50x_2$$

subject to:

$$2x_1 + x_2 \leq 8 \qquad \text{(cutting hours)}$$

$$x_1 + 3x_2 \leq 9 \qquad \text{(finishing hours)}$$

$$x_1, x_2 \geq 0$$

**Standard form.** The simplex method requires equality constraints. Convert each $\leq$ constraint to an equation by adding a **slack variable** $s_i \geq 0$, read "slack variable i." Slack absorbs the difference between the left side and the right side:

$$2x_1 + x_2 + s_1 = 8, \quad s_1 \geq 0$$

$$x_1 + 3x_2 + s_2 = 9, \quad s_2 \geq 0$$

Now all constraints are equations, all variables are non-negative, and the objective is linear. This is standard form: maximise $\mathbf{c}^\top \mathbf{x}$ subject to $A\mathbf{x} = \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$.

**A second example: diet/resource allocation.** A hospital nutrition service needs to provide at least 60 g of protein, at least 800 mg of calcium, and at most 2000 kcal per patient per day from two food types. Food A costs \$1.20 per unit, food B costs \$0.80 per unit. Formulate the LP to minimise daily cost.

Let $x_1$ = units of food A, $x_2$ = units of food B.

| | Food A | Food B | Requirement |
|---|---|---|---|
| Protein (g/unit) | 15 | 10 | ≥ 60 |
| Calcium (mg/unit) | 200 | 150 | ≥ 800 |
| Energy (kcal/unit) | 400 | 300 | ≤ 2000 |
| Cost (\$/unit) | 1.20 | 0.80 | minimise |

$$\text{minimise} \quad z = 1.20x_1 + 0.80x_2$$

subject to:

$$15x_1 + 10x_2 \geq 60$$

$$200x_1 + 150x_2 \geq 800$$

$$400x_1 + 300x_2 \leq 2000$$

$$x_1, x_2 \geq 0$$

For $\geq$ constraints, introduce **surplus variables** $s_i \geq 0$: $15x_1 + 10x_2 - s_1 = 60$. The minus sign makes $s_i$ the excess above the minimum. This means a surplus variable records how far you are above a required minimum, rather than how much unused capacity remains below a maximum.

Note that the diet problem's standard form, with its surplus variables and $\geq$ constraints, is not directly ready for the simplex method as presented below. With surplus variables, the origin is no longer a feasible starting point (it violates the minimum requirements), and finding an initial basic feasible solution requires an additional technique — *artificial variables* — that is covered in more advanced treatments. The simplex worked example uses only $\leq$ constraints to keep the starting point clear.

## 54.2. Graphical method

For two decision variables you can draw the **feasible region** — the set of all $(x_1, x_2)$ satisfying every constraint — and read off the optimum. More importantly, you can *see* the geometry that the simplex method will exploit algebraically when there are too many variables to draw.

For the factory problem, plot each constraint boundary:

- $2x_1 + x_2 = 8$: line through $(4, 0)$ and $(0, 8)$. Feasible side: below/left.
- $x_1 + 3x_2 = 9$: line through $(9, 0)$ and $(0, 3)$. Feasible side: below/left.
- $x_1, x_2 \geq 0$: first quadrant only.

The feasible region is a convex polygon — a quadrilateral with four vertices: $(0, 0)$, $(4, 0)$, $(3, 2)$, and $(0, 3)$.

**Corner-point theorem.** If an LP has an optimal solution, there is an optimal solution at a vertex (corner point) of the feasible region. This is the foundational geometric fact: the objective function $z = 30x_1 + 50x_2$ is a family of parallel lines (iso-profit lines). As you push the line in the direction of increasing profit, the last point of contact with the feasible region is a vertex.

Evaluate the objective at each vertex:

| Vertex | $z = 30x_1 + 50x_2$ |
|---|---|
| $(0, 0)$ | 0 |
| $(4, 0)$ | 120 |
| $(3, 2)$ | 190 |
| $(0, 3)$ | 150 |

Maximum profit is **$190** per day, achieved by making 3 chairs and 2 tables.

The intersection point $(3, 2)$ comes from solving $2x_1 + x_2 = 8$ and $x_1 + 3x_2 = 9$ simultaneously. Subtract twice the second from the first: $-5x_2 = -10$, so $x_2 = 2$, $x_1 = 3$.

> **Why the optimum is always at a vertex**
> The feasible region of an LP is a convex polytope — any two feasible points can be connected by a straight line that stays entirely inside the region. The objective function is linear, so it increases at a constant rate in one direction. If the optimum were in the interior, you could always move a little further in the improving direction and still stay feasible, reaching a higher objective value. The only place where you genuinely cannot improve further is a boundary. And on a boundary, you can keep moving along the edge until you hit a vertex. This is why the simplex method only needs to visit vertices.

Use the sliders to watch the feasible region change and see the optimum stay locked to a corner.

```
//| echo: false
{
  const W = 700, H = 420;
  const xMax = 8;
```

```
const yMax = 5;
let b1 = 8;
let b2 = 9;

function svgEl(tag) {
  return document.createElementNS("http://www.w3.org/2000/svg", tag);
}

function yFromC1(x) {
  return b1 - 2 * x;
}

function yFromC2(x) {
  return (b2 - x) / 3;
}

const container = document.createElement("div");
container.style.cssText = "max-width: 760px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem; a
function sliderRow(labelText, min, max, value) {
  const label = document.createElement("label");
  label.textContent = labelText;
  const input = document.createElement("input");
  input.type = "range";
  input.min = String(min);
  input.max = String(max);
  input.step = "0.5";
  input.value = String(value);
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; min-width:3.4rem;";
  controls.append(label, input, out);
  return { input, out };
}
const c1 = sliderRow("Cutting-hours RHS", 5, 11, b1);
const c2 = sliderRow("Finishing-hours RHS", 6, 12, b2);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding:0
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
  k.textContent = label;
  const v = document.createElement("div");
```

```
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const vertexBox = card("Optimal vertex");
const valueBox = card("Objective z");
const bindBox = card("Active constraints");

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:white; border:1px solid #e5e7eb; border-radius:1

const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

const left = 60, right = 24, top = 26, bottom = 42;
const plotW = W - left - right, plotH = H - top - bottom;
const sx = x => left + plotW * x / xMax;
const sy = y => top + plotH * (1 - y / yMax);

function feasible(pt) {
  const [x, y] = pt;
  return x >= -1e-9 && y >= -1e-9 && 2 * x + y <= b1 + 1e-9 && x + 3 * y <= b2 + 1e-9;
}

function sortPoly(pts) {
  const cx = pts.reduce((s, p) => s + p[0], 0) / pts.length;
  const cy = pts.reduce((s, p) => s + p[1], 0) / pts.length;
  return pts.slice().sort((a, b) => Math.atan2(a[1] - cy, a[0] - cx) - Math.atan2(b[1] - cy, b[
}

function redraw() {
  b1 = parseFloat(c1.input.value);
  b2 = parseFloat(c2.input.value);
  c1.out.textContent = b1.toFixed(1);
  c2.out.textContent = b2.toFixed(1);
  while (svg.firstChild) svg.removeChild(svg.firstChild);

  for (let x = 0; x <= xMax; x += 1) {
    const line = svgEl("line");
    line.setAttribute("x1", sx(x));
    line.setAttribute("x2", sx(x));
    line.setAttribute("y1", sy(0));
    line.setAttribute("y2", sy(yMax));
    line.setAttribute("stroke", "#f3f4f6");
```

```
    svg.appendChild(line);
  }
  for (let y = 0; y <= yMax; y += 1) {
    const line = svgEl("line");
    line.setAttribute("x1", sx(0));
    line.setAttribute("x2", sx(xMax));
    line.setAttribute("y1", sy(y));
    line.setAttribute("y2", sy(y));
    line.setAttribute("stroke", "#f3f4f6");
    svg.appendChild(line);
  }

  const axX = svgEl("line");
  axX.setAttribute("x1", sx(0)); axX.setAttribute("x2", sx(xMax));
  axX.setAttribute("y1", sy(0)); axX.setAttribute("y2", sy(0));
  axX.setAttribute("stroke", "#6b7280"); axX.setAttribute("stroke-width", "1.5");
  svg.appendChild(axX);
  const axY = svgEl("line");
  axY.setAttribute("x1", sx(0)); axY.setAttribute("x2", sx(0));
  axY.setAttribute("y1", sy(0)); axY.setAttribute("y2", sy(yMax));
  axY.setAttribute("stroke", "#6b7280"); axY.setAttribute("stroke-width", "1.5");
  svg.appendChild(axY);

  const intersection = [(3 * b1 - b2) / 5, (2 * b2 - b1) / 5];
  const candidates = [[0, 0], [b1 / 2, 0], [0, b2 / 3], intersection];
  const feasiblePts = candidates.filter(feasible);
  const poly = sortPoly(feasiblePts);

  const polyNode = svgEl("polygon");
  polyNode.setAttribute("points", poly.map(([x, y]) => `${sx(x)},${sy(y)}`).join(" "));
  polyNode.setAttribute("fill", "rgba(20,184,166,0.22)");
  polyNode.setAttribute("stroke", "#0f766e");
  polyNode.setAttribute("stroke-width", "2");
  svg.appendChild(polyNode);

  [
    { fn: yFromC1, color: "#2563eb", x2: Math.min(xMax, b1 / 2), label: "2x₁ + x₂ = RHS" },
    { fn: yFromC2, color: "#f59e0b", x2: Math.min(xMax, b2), label: "x₁ + 3x₂ = RHS" }
  ].forEach(item => {
    const line = svgEl("line");
    line.setAttribute("x1", sx(0));
    line.setAttribute("y1", sy(item.fn(0)));
    line.setAttribute("x2", sx(item.x2));
    line.setAttribute("y2", sy(item.fn(item.x2)));
    line.setAttribute("stroke", item.color);
    line.setAttribute("stroke-width", "2.5");
    svg.appendChild(line);
```

```
  const text = svgEl("text");
  text.setAttribute("x", sx(item.x2) - 8);
  text.setAttribute("y", sy(item.fn(item.x2)) - 8);
  text.setAttribute("text-anchor", "end");
  text.setAttribute("font-size", "12");
  text.setAttribute("fill", item.color);
  text.textContent = item.label;
  svg.appendChild(text);
});

let best = feasiblePts[0];
let bestZ = -Infinity;
feasiblePts.forEach(pt => {
  const z = 30 * pt[0] + 50 * pt[1];
  if (z > bestZ) {
    best = pt;
    bestZ = z;
  }
});

const zLine = svgEl("line");
zLine.setAttribute("x1", sx(0));
zLine.setAttribute("y1", sy(bestZ / 50));
zLine.setAttribute("x2", sx(bestZ / 30));
zLine.setAttribute("y2", sy(0));
zLine.setAttribute("stroke", "#7c3aed");
zLine.setAttribute("stroke-width", "2.2");
zLine.setAttribute("stroke-dasharray", "7 5");
svg.appendChild(zLine);

feasiblePts.forEach(([x, y]) => {
  const dot = svgEl("circle");
  dot.setAttribute("cx", sx(x));
  dot.setAttribute("cy", sy(y));
  dot.setAttribute("r", x === best[0] && y === best[1] ? "6.5" : "4");
  dot.setAttribute("fill", x === best[0] && y === best[1] ? "#7c3aed" : "#0f766e");
  svg.appendChild(dot);
});

const label = svgEl("text");
label.setAttribute("x", sx(best[0]) + 10);
label.setAttribute("y", sy(best[1]) - 10);
label.setAttribute("font-size", "12");
label.setAttribute("fill", "#6d28d9");
label.textContent = `best = (${best[0].toFixed(2)}, ${best[1].toFixed(2)})`;
svg.appendChild(label);
```

```
    vertexBox.textContent = `(${best[0].toFixed(2)}, ${best[1].toFixed(2)})`;
    valueBox.textContent = `${bestZ.toFixed(1)}`;
    const active = [];
    if (Math.abs(2 * best[0] + best[1] - b1) < 1e-6) active.push("cutting");
    if (Math.abs(best[0] + 3 * best[1] - b2) < 1e-6) active.push("finishing");
    if (Math.abs(best[0]) < 1e-6) active.push("x₁ = 0");
    if (Math.abs(best[1]) < 1e-6) active.push("x₂ = 0");
    bindBox.textContent = active.join(", ");
    caption.textContent = "The shaded polygon is the feasible region. The dashed purple line is the
  }

  [c1.input, c2.input].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, readout, svg, caption);
  redraw();
  return container;
}
```

What the graphical method shows directly is what the simplex method does systematically: start at one vertex (here, the origin), check whether any neighbour is better, move to the best neighbour, and repeat. The graphical picture makes the logic self-evident. The simplex method turns that logic into algebra so it can work in a hundred dimensions instead of two.

## 54.3. Simplex method

The graphical method breaks down beyond two variables. The simplex method does the same thing — walk from vertex to vertex, improving the objective each step — algebraically, using the tableau.

**Setup: slack variables and initial tableau.** Return to the factory problem in standard form with $x_1, x_2$ (original variables) and $s_1, s_2$ (slacks):

$$2x_1 + x_2 + s_1 = 8$$

$$x_1 + 3x_2 + s_2 = 9$$

$$z = 30x_1 + 50x_2 \quad \Longrightarrow \quad -30x_1 - 50x_2 + z = 0$$

The **basic feasible solution** (BFS) at the origin: set non-basic variables $x_1 = x_2 = 0$, read off basic variables $s_1 = 8$, $s_2 = 9$, $z = 0$. This is the vertex $(0, 0)$.

**Initial tableau:**

| Basic | $x_1$ | $x_2$ | $s_1$ | $s_2$ | RHS |
|---|---|---|---|---|---|
| $s_1$ | 2 | 1 | 1 | 0 | 8 |
| $s_2$ | 1 | 3 | 0 | 1 | 9 |
| $z$ | −30 | −50 | 0 | 0 | 0 |

**Pivot selection.** Look at the objective row (bottom row). The most negative coefficient is $-50$ in the $x_2$ column: $x_2$ enters the basis. The column $x_2$ is the *entering column*.

To choose the *leaving variable*, compute the minimum ratio: RHS/entering-column coefficient for rows with positive pivot-column entries only. Rows with zero or negative entries in the pivot column are skipped: a zero entry means increasing $x_2$ has no effect on that constraint, and a negative entry means the constraint becomes *easier* to satisfy as $x_2$ increases — neither constrains the move.

$$\frac{8}{1} = 8 \qquad \frac{9}{3} = 3$$

Minimum ratio is 3, so $s_2$ leaves. The pivot element is the entry in row $s_2$, column $x_2$: that is 3.

**Pivot operation.** Divide the $s_2$ row by 3 to make the pivot element 1:

$$\text{new row 2}: \quad \tfrac{1}{3}x_1 + x_2 + 0 \cdot s_1 + \tfrac{1}{3}s_2 = 3$$

Eliminate $x_2$ from every other row by adding/subtracting multiples of the new row 2:

- Row 1 ← Row 1 − 1 × new row 2: $(2 - \frac{1}{3})x_1 + 0 \cdot x_2 + s_1 - \frac{1}{3}s_2 = 8 - 3 \Rightarrow \frac{5}{3}x_1 + s_1 - \frac{1}{3}s_2 = 5$
- Objective row ← Obj row + 50 × new row 2: $(-30 + \frac{50}{3})x_1 + 0 \cdot x_2 + 0 \cdot s_1 + \frac{50}{3}s_2 = 150 \Rightarrow -\frac{40}{3}x_1 + \frac{50}{3}s_2 = 150$

**Tableau after first pivot ($x_2$ enters, $s_2$ leaves):**

| Basic | $x_1$ | $x_2$ | $s_1$ | $s_2$ | RHS |
|---|---|---|---|---|---|
| $s_1$ | $5/3$ | 0 | 1 | $-1/3$ | 5 |
| $x_2$ | $1/3$ | 1 | 0 | $1/3$ | 3 |
| $z$ | $-40/3$ | 0 | 0 | $50/3$ | 150 |

Current BFS: $x_1 = 0$, $x_2 = 3$, $s_1 = 5$, $s_2 = 0$, $z = 150$. This is vertex $(0, 3)$.

**Second pivot.** Most negative objective coefficient: $-40/3$ in the $x_1$ column. $x_1$ enters. Minimum ratio:

$$\frac{5}{5/3} = 3 \qquad \frac{3}{1/3} = 9$$

$s_1$ leaves. Pivot element: $5/3$. Divide row 1 by $5/3$:

$$\text{new row 1}: \quad x_1 + 0 \cdot x_2 + \tfrac{3}{5}s_1 - \tfrac{1}{5}s_2 = 3$$

Eliminate $x_1$ from other rows:

- Row 2 ← Row 2 − $\frac{1}{3}$ × new row 1: $0 \cdot x_1 + x_2 - \frac{1}{5}s_1 + \frac{2}{5}s_2 = 2$
- Obj row ← Obj row + $\frac{40}{3}$ × new row 1: $0 \cdot x_1 + 8 \cdot s_1 + 14 \cdot s_2 = 190$

  Let's be precise: $-\frac{40}{3} + \frac{40}{3} \cdot 1 = 0$; $s_1$ coefficient: $0 + \frac{40}{3} \cdot \frac{3}{5} = 8$; $s_2$ coefficient: $\frac{50}{3} + \frac{40}{3} \cdot (-\frac{1}{5}) = \frac{50}{3} - \frac{8}{3} = 14$; RHS: $150 + \frac{40}{3} \cdot 3 = 150 + 40 = 190$.

**Final tableau:**

| Basic | $x_1$ | $x_2$ | $s_1$ | $s_2$ | RHS |
|---|---|---|---|---|---|
| $x_1$ | 1 | 0 | $3/5$ | $-1/5$ | 3 |
| $x_2$ | 0 | 1 | $-1/5$ | $2/5$ | 2 |
| $z$ | 0 | 0 | 8 | 14 | 190 |

All objective-row coefficients are non-negative. **Termination condition met.** Optimal solution: $x_1 = 3$, $x_2 = 2$, $z = 190$. This means there is no adjacent basic feasible solution that can improve the objective any further, so the current vertex is optimal.

This matches the graphical answer exactly.

> **Why simplex terminates**
> Each pivot strictly improves the objective (assuming no degeneracy — no zero RHS values). There are finitely many vertices. So the method must reach an optimum in a finite number of steps. In practice, simplex visits far fewer vertices than the total count even for large LPs.

Use the stepper to see how simplex moves from one basic feasible solution to the next by pivoting on an entering column and a leaving row.

```
//| echo: false
{
  const steps = [
    {
      title: "Initial tableau",
      entering: "x2",
      leaving: "s2",
      bfs: "x1 = 0, x2 = 0, z = 0",
      rows: [
        ["s1", "2", "1", "1", "0", "8"],
        ["s2", "1", "3", "0", "1", "9"],
        ["z", "-30", "-50", "0", "0", "0"]
      ]
    },
    {
      title: "After pivot 1",
      entering: "x1",
      leaving: "s1",
      bfs: "x2 = 3, x1 = 0, z = 150",
      rows: [
        ["s1", "5/3", "0", "1", "-1/3", "5"],
        ["x2", "1/3", "1", "0", "1/3", "3"],
        ["z", "-40/3", "0", "0", "50/3", "150"]
      ]
    },
    {
      title: "Final tableau",
```

```
    entering: "none",
    leaving: "none",
    bfs: "x1 = 3, x2 = 2, z = 190",
    rows: [
      ["x1", "1", "0", "3/5", "-1/5", "3"],
      ["x2", "0", "1", "-1/5", "2/5", "2"],
      ["z", "0", "0", "8", "14", "190"]
    ]
  }
];

const container = document.createElement("div");
container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const actions = document.createElement("div");
actions.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.7rem;";
const prevBtn = document.createElement("button");
prevBtn.textContent = "Previous";
const nextBtn = document.createElement("button");
nextBtn.textContent = "Next pivot";
const stage = document.createElement("strong");
actions.append(prevBtn, nextBtn, stage);

const summary = document.createElement("div");
summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.7rem; color:#3
const enterBox = document.createElement("span");
const leaveBox = document.createElement("span");
const bfsBox = document.createElement("span");
summary.append(enterBox, leaveBox, bfsBox);

const table = document.createElement("table");
table.style.cssText = "border-collapse:collapse; width:100%; max-width:700px; font-size:0.95rem
const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

let idx = 0;
function cell(text, header = false) {
  const el = document.createElement(header ? "th" : "td");
  el.textContent = text;
  el.style.cssText = "border:1px solid #cbd5e1; padding:0.45rem 0.55rem; text-align:center;";
  return el;
}

function redraw() {
  const step = steps[idx];
  stage.textContent = step.title;
  enterBox.textContent = `entering variable: ${step.entering}`;
  leaveBox.textContent = `leaving variable: ${step.leaving}`;
```

```
    bfsBox.textContent = `current BFS: ${step.bfs}`;
    table.replaceChildren();

    const thead = document.createElement("thead");
    const hr = document.createElement("tr");
    ["Basic", "x1", "x2", "s1", "s2", "RHS"].forEach(h => hr.appendChild(cell(h, true)));
    thead.appendChild(hr);
    table.appendChild(thead);

    const tbody = document.createElement("tbody");
    step.rows.forEach(row => {
      const tr = document.createElement("tr");
      row.forEach((val, j) => {
        const td = cell(val);
        if (j === 0 && val === step.leaving) td.style.background = "#fde68a";
        if (j > 0 && ["x1","x2","s1","s2"][j - 1] === step.entering) td.style.background = "#bfdbfe"
        if (j === 0 && val === "z") td.style.fontWeight = "700";
        tr.appendChild(td);
      });
      tbody.appendChild(tr);
    });
    table.appendChild(tbody);

    caption.textContent = idx < steps.length - 1
      ? "Blue cells mark the entering column and amber marks the leaving row. The next pivot moves t
      : "At the final tableau all objective-row coefficients are non-negative, so no adjacent basic
  }

  prevBtn.addEventListener("click", () => { idx = Math.max(0, idx - 1); redraw(); });
  nextBtn.addEventListener("click", () => { idx = Math.min(steps.length - 1, idx + 1); redraw(); })

  container.append(actions, summary, table, caption);
  redraw();
  return container;
}
```

## 54.4. Duality

Every LP — the **primal** — has a companion LP called the **dual**. The dual is not just a mathematical curiosity; it has direct economic meaning and provides a powerful optimality check.

**Constructing the dual.** Start from the primal in standard inequality form:

$$\text{maximise} \quad \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad A\mathbf{x} \le \mathbf{b},\ \mathbf{x} \ge \mathbf{0}$$

The dual introduces one dual variable $y_i$ — read "y-sub-i" — for each primal constraint. The dual is:

$$\text{minimise} \quad \mathbf{b}^\top \mathbf{y} \quad \text{subject to} \quad A^\top \mathbf{y} \geq \mathbf{c},\ \mathbf{y} \geq \mathbf{0}$$

The roles reverse: primal constraints become dual variables, primal variables become dual constraints, primal objective coefficients become dual RHS values, and primal RHS values become dual objective coefficients. The maximisation becomes minimisation.

**Dual of the factory problem.** Primal: maximise $30x_1 + 50x_2$ subject to $2x_1 + x_2 \leq 8$, $x_1 + 3x_2 \leq 9$, $x_1, x_2 \geq 0$.

Let $y_1, y_2$ be the dual variables (one per primal constraint):

$$\text{minimise} \quad 8y_1 + 9y_2$$

subject to:

$$2y_1 + y_2 \geq 30 \qquad \text{(column 1 of } A^\top\text{)}$$

$$y_1 + 3y_2 \geq 50 \qquad \text{(column 2 of } A^\top\text{)}$$

$$y_1, y_2 \geq 0$$

**Weak duality theorem.** For any primal feasible $\mathbf{x}$ and any dual feasible $\mathbf{y}$:

$$\mathbf{c}^\top \mathbf{x} \leq \mathbf{b}^\top \mathbf{y}$$

*Proof.* The dual constraint says $A^\top \mathbf{y} \geq \mathbf{c}$, meaning each component satisfies $(A^\top \mathbf{y})_j \geq c_j$. Since $x_j \geq 0$, multiplying through and summing over all $j$ gives $\mathbf{c}^\top \mathbf{x} \leq (A^\top \mathbf{y})^\top \mathbf{x} = \mathbf{y}^\top A\mathbf{x}$. The primal constraint $A\mathbf{x} \leq \mathbf{b}$ combined with $\mathbf{y} \geq \mathbf{0}$ then gives $\mathbf{y}^\top A\mathbf{x} \leq \mathbf{y}^\top \mathbf{b} = \mathbf{b}^\top \mathbf{y}$. Chaining both inequalities: $\mathbf{c}^\top \mathbf{x} \leq \mathbf{b}^\top \mathbf{y}$. □

The strong duality theorem (proved via simplex theory) says the inequality is tight at optimality: primal and dual optimal values are equal.

**Economic interpretation: shadow prices.** The optimal dual variables have a direct meaning. $y_1$ is the **shadow price** of the first constraint (cutting hours): it tells you how much the optimal objective increases if you add one unit of that resource. At optimality, $y_1 = 8$ and $y_2 = 14$ — each additional cutting hour is worth \$8, each additional finishing hour is worth \$14. You can verify: $8y_1 + 9y_2 = 8 \times 8 + 9 \times 14 = 64 + 126 = 190$. Dual optimal matches primal optimal — strong duality holds. This means the dual variables are not abstract extras: they price the scarce resources in exactly the units of the original objective.

**Complementary slackness.** At optimality, either a primal constraint is tight (active, $A_i\mathbf{x} = b_i$) or the corresponding dual variable is zero ($y_i = 0$); and either a dual constraint is tight or the corresponding primal variable is zero. Formally: $y_i(b_i - A_i\mathbf{x}) = 0$ and $(A^\top \mathbf{y} - \mathbf{c})_j x_j = 0$ for all $i, j$. This gives you a system of equations to verify optimality without running the full simplex.

## 54.5. Sensitivity analysis

The optimal basis found by simplex remains optimal as long as the problem data stays within certain ranges. If the profit margin on $x_1$ changes from 30 to $30 + \Delta$, the basis $\{x_1, x_2\}$ stays optimal while the objective-row coefficient of $s_1$ (currently 8) remains non-negative and the coefficient of $s_2$ (currently 14) remains non-negative. The $x_1$ basic row is $[1, 0, 3/5, -1/5, 3]$, so changing $c_1$ by $\Delta$ shifts the $s_1$ coefficient to $8 - \frac{3}{5}\Delta$ and the $s_2$ coefficient to $14 + \frac{1}{5}\Delta$. Requiring both to remain non-negative: $8 - \frac{3}{5}\Delta \geq 0 \Rightarrow \Delta \leq \frac{40}{3}$ and $14 + \frac{1}{5}\Delta \geq 0 \Rightarrow \Delta \geq -70$. The basis is stable for $-70 \leq \Delta \leq \frac{40}{3}$, i.e., $c_1 \in [-40, 43\frac{1}{3}]$. Similarly, if the RHS $b_1 = 8$ changes, the solution coordinates shift but the same basis remains feasible as long as all basic variables stay non-negative — the range can be read from the current tableau's column for $s_1$. This analysis is done without re-running simplex.

## 54.6. Where this goes

Chapter 2 of this section develops **network flow problems** — shortest path, maximum flow, minimum cost flow — all of which can be formulated as LPs with special structure (network matrices) that allows faster algorithms than general simplex. Network LPs arise everywhere: logistics, circuit analysis, pipeline routing.

The broader field of **integer programming** (mixed-integer LP) handles the case where some or all variables must be integers — scheduling problems, facility location, combinatorial optimisation. Branch-and-bound solves these by solving a sequence of LP relaxations. The LP theory in this chapter is the computational core of those methods.

**Where LP shows up**

- A refinery blends crude feedstocks to meet fuel-grade specifications at minimum cost — an LP with 50–500 variables, solved daily.
- An airline schedules crews to cover flights while meeting duty-time regulations — an integer LP with millions of variables in practice.
- A fund manager maximises expected portfolio return subject to risk limits and position constraints — LP or quadratic programming depending on the risk model.
- A nutritionist formulates minimum-cost meal plans meeting dietary requirements — the diet problem, which drove early LP development.
- A chip manufacturer allocates fab capacity across product lines to maximise margin — LP with supply chain constraints.

## 54.7. Exercises

These are puzzles. Each has a clean optimal solution. The interesting work is in the formulation — translating the situation into variables, objective, and constraints — and in executing the simplex steps carefully.

**Exercise 1.** A workshop makes two products, shelves and benches. Each shelf uses 3 kg of timber and 1 hour of labour. Each bench uses 2 kg of timber and 4 hours of labour. The workshop has 24 kg of timber and 16 hours of labour available per week. Profit is \$20 per shelf and \$45 per bench. Formulate the LP to maximise weekly profit.

**Exercise 2.** Solve the LP from Exercise 1 graphically. Plot both constraint boundaries, shade the feasible region, evaluate the objective at every vertex, and identify the optimal solution.

**Exercise 3.** Convert the LP from Exercise 1 to standard form by introducing slack variables. Write out the full system of equality constraints and identify the initial basic feasible solution (the origin vertex).

**Exercise 4.** Solve the following LP using the simplex tableau method. Show all pivots.

$$\text{maximise} \quad z = 5x_1 + 4x_2$$

subject to:

$$6x_1 + 4x_2 \leq 24$$
$$x_1 + 2x_2 \leq 6$$
$$x_1, x_2 \geq 0$$

**Exercise 5.** Write the dual of the LP in Exercise 4. Then use complementary slackness to find the dual optimal solution $(y_1, y_2)$. Verify strong duality by checking that primal and dual objective values agree at optimality.

**Exercise 6.** In Exercise 4, the optimal basis has $x_1$ basic (value 3) and $x_2$ basic (value $3/2$). By how much can the objective coefficient of $x_1$ (currently 5) change before the current basis is no longer optimal? Give the range.

# 55. Graphs and networks

Paths, flows, and spanning structures

You want to drive from Edmonton to Calgary in the least time, not just the fewest kilometres. You need to wire up five remote sensors to a data logger using as little cable as possible. You want to know the maximum throughput a pipeline network can deliver before a bottleneck binds.

Different questions. The same underlying structure: nodes, connections, weights. Once you have that structure, a handful of classical algorithms — each with a clean proof of correctness — answer all of them.

## 55.1. Graph fundamentals

A **graph** $G = (V, E)$ — read "a graph G consisting of vertex set V and edge set E" — is a set of **vertices** (also called nodes) $V$ together with a set of **edges** $E$, where each edge connects two vertices.

In an **undirected** graph, the edge $\{u, v\}$ can be traversed in either direction. In a **directed** graph (digraph), each edge $(u, v)$ — read "from u to v" — has a direction: you can go from $u$ to $v$ but not necessarily back.

A **weighted** graph assigns a real number $w(u, v)$ — "the weight of the edge from $u$ to $v$" — to each edge. Weights model distance, cost, travel time, capacity, or any other quantity associated with a connection.

### 55.1.1. Adjacency matrix

The **adjacency matrix** $A$ of a graph on $n$ vertices is the $n \times n$ matrix where entry $A_{ij} = 1$ if there is an edge from vertex $i$ to vertex $j$, and $A_{ij} = 0$ otherwise. For a weighted graph, replace the 1 with the edge weight. For an undirected graph, $A$ is symmetric: $A_{ij} = A_{ji}$.

Example: a simple undirected graph with four vertices $\{1, 2, 3, 4\}$ and edges $\{1, 2\}$, $\{1, 3\}$, $\{2, 4\}$, $\{3, 4\}$ has adjacency matrix

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}.$$

### 55.1.2. Degree, paths, and connectivity

The **degree** $d(v)$ — "the degree of vertex $v$" — is the number of edges incident to $v$. In the example above, $d(1) = 2$, $d(2) = 2$, $d(3) = 2$, $d(4) = 2$.

A **path** from $u$ to $v$ is a sequence of vertices starting at $u$ and ending at $v$ where consecutive vertices are connected by an edge, and no vertex repeats. A **cycle** is a closed path: it starts and ends at the same vertex. A graph is **connected** if there is a path between every pair of vertices.

A **tree** is a connected graph with no cycles. Equivalently, a tree on $n$ vertices has exactly $n - 1$ edges — removing any edge disconnects it, adding any edge creates a cycle. Trees are the minimal connected structures, which is exactly why they appear in spanning problems.

## 55.2. Shortest path: Dijkstra's algorithm

**Problem.** Given a weighted graph with non-negative edge weights and a source vertex $s$, find the shortest-distance path from $s$ to every other vertex.

**Algorithm.** Maintain a set $S$ of vertices whose shortest distances are finalised, and for every vertex outside $S$, a tentative distance $\text{dist}[v]$. Initially $\text{dist}[s] = 0$ and $\text{dist}[v] = \infty$ for all $v \neq s$.

Repeat until $S = V$:

1. Pick the vertex $u \notin S$ with the smallest $\text{dist}[u]$.
2. Add $u$ to $S$ (its distance is now final).
3. For each neighbour $v$ of $u$: if $\text{dist}[u] + w(u, v) < \text{dist}[v]$, update $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$.

### 55.2.1. Worked trace — 6-node graph

Consider the following undirected weighted graph with vertices $\{A, B, C, D, E, F\}$ and source $A$:

| Edge | Weight |
|---|---|
| A–B | 4 |
| A–C | 2 |
| B–C | 5 |
| B–D | 10 |
| C–E | 3 |
| E–D | 4 |
| D–F | 11 |
| E–F | 7 |

Initial distances: $A = 0$, $B = \infty$, $C = \infty$, $D = \infty$, $E = \infty$, $F = \infty$.

**Step 1.** Smallest tentative distance is $A$ (dist = 0). Finalise $A$. Update neighbours: $\text{dist}[B] \leftarrow 4$, $\text{dist}[C] \leftarrow 2$.

**Step 2.** Smallest non-finalised: $C$ (dist = 2). Finalise $C$. Update neighbours: $B$: $2 + 5 = 7 > 4$, no change. $E$: $2 + 3 = 5 \leftarrow 5$.

**Step 3.** Smallest non-finalised: $B$ (dist = 4). Finalise $B$. Update neighbours: $C$: already finalised. $D$: $4 + 10 = 14 \leftarrow 14$.

**Step 4.** Smallest non-finalised: $E$ (dist = 5). Finalise $E$. Update neighbours: $D$: $5 + 4 = 9 < 14$, update $\text{dist}[D] \leftarrow 9$. $F$: $5 + 7 = 12 \leftarrow 12$.

**Step 5.** Smallest non-finalised: $D$ (dist = 9). Finalise $D$. Update neighbours: $F$: $9 + 11 = 20 > 12$, no change.

**Step 6.** Smallest non-finalised: $F$ (dist = 12). Finalise $F$.

Final shortest distances from $A$: $B = 4$, $C = 2$, $D = 9$, $E = 5$, $F = 12$.

The shortest path to $F$ is $A \to C \to E \to F$ with total weight $2 + 3 + 7 = 12$. This means Dijkstra is not merely producing distances one by one. It is building a growing tree of already-certified shortest paths.

Use the stepper to watch the shortest-path tree grow one certified vertex at a time.

```
//| echo: false
{
  const pos = {
    A:[90,170], B:[220,80], C:[220,250],
    D:[380,85], E:[380,250], F:[560,170]
  };
  const edges = [
    ["A","B",4],["A","C",2],["B","C",1],["B","D",5],
    ["C","D",8],["C","E",3],["D","E",2],["D","F",11],["E","F",7]
  ];
  const steps = [
    { current:"A", finalised:["A"], dist:{A:0,B:4,C:2,D:"∞",E:"∞",F:"∞"}, tree:[] },
    { current:"C", finalised:["A","C"], dist:{A:0,B:3,C:2,D:10,E:5,F:"∞"}, tree:[["A","C"]] },
    { current:"B", finalised:["A","C","B"], dist:{A:0,B:3,C:2,D:8,E:5,F:"∞"}, tree:[["A","C"],["C
    { current:"E", finalised:["A","C","B","E"], dist:{A:0,B:3,C:2,D:8,E:5,F:12}, tree:[["A","C"],
    { current:"D", finalised:["A","C","B","E","D"], dist:{A:0,B:3,C:2,D:8,E:5,F:12}, tree:[["A","
    { current:"F", finalised:["A","C","B","E","D","F"], dist:{A:0,B:3,C:2,D:8,E:5,F:12}, tree:[["
  ];

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }

  const container = document.createElement("div");
  container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
  const actions = document.createElement("div");
  actions.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.7rem;";
  const prevBtn = document.createElement("button"); prevBtn.textContent = "Previous";
  const nextBtn = document.createElement("button"); nextBtn.textContent = "Next step";
  const stage = document.createElement("strong");
  actions.append(prevBtn, nextBtn, stage);

  const readout = document.createElement("div");
  readout.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3
```

```
const currentBox = document.createElement("span");
const pathBox = document.createElement("span");
readout.append(currentBox, pathBox);

const svg = svgEl("svg");
svg.setAttribute("viewBox", "0 0 640 340");
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

let idx = 0;
function redraw() {
  const step = steps[idx];
  stage.textContent = `Step ${idx + 1} of ${steps.length}`;
  currentBox.textContent = `newly finalised vertex: ${step.current}`;
  pathBox.textContent = `current shortest path to F: ${idx < 3 ? "not yet certified" : "A → C → E
  svg.replaceChildren();

  edges.forEach(([u,v,w]) => {
    const [x1,y1] = pos[u], [x2,y2] = pos[v];
    const inTree = step.tree.some(([a,b]) => (a === u && b === v) || (a === v && b === u));
    const line = svgEl("line");
    line.setAttribute("x1", x1); line.setAttribute("y1", y1);
    line.setAttribute("x2", x2); line.setAttribute("y2", y2);
    line.setAttribute("stroke", inTree ? "#2563eb" : "#94a3b8");
    line.setAttribute("stroke-width", inTree ? "5" : "2");
    svg.appendChild(line);
    const label = svgEl("text");
    label.setAttribute("x", (x1 + x2) / 2);
    label.setAttribute("y", (y1 + y2) / 2 - 6);
    label.setAttribute("text-anchor", "middle");
    label.setAttribute("font-size", "12");
    label.setAttribute("fill", "#334155");
    label.textContent = String(w);
    svg.appendChild(label);
  });

  Object.entries(pos).forEach(([node,[x,y]]) => {
    const finalised = step.finalised.includes(node);
    const current = step.current === node;
    const c = svgEl("circle");
    c.setAttribute("cx", x); c.setAttribute("cy", y); c.setAttribute("r", 22);
    c.setAttribute("fill", current ? "#f59e0b" : finalised ? "#1d4ed8" : "#e2e8f0");
```

```
      c.setAttribute("stroke", "#0f172a");
      c.setAttribute("stroke-width", "1.2");
      svg.appendChild(c);
      const t = svgEl("text");
      t.setAttribute("x", x); t.setAttribute("y", y + 4);
      t.setAttribute("text-anchor", "middle");
      t.setAttribute("font-size", "14");
      t.setAttribute("font-weight", "700");
      t.setAttribute("fill", finalised || current ? "#fff" : "#0f172a");
      t.textContent = node;
      svg.appendChild(t);
      const d = svgEl("text");
      d.setAttribute("x", x); d.setAttribute("y", y + 38);
      d.setAttribute("text-anchor", "middle");
      d.setAttribute("font-size", "12");
      d.setAttribute("fill", "#334155");
      d.textContent = `dist ${step.dist[node]}`;
      svg.appendChild(d);
    });

    caption.textContent = idx < steps.length - 1
      ? "Blue edges are already certified as part of the shortest-path tree. The amber vertex is
      : "By the end, every vertex is finalised and the blue edges form the shortest-path tree roo
  }

  prevBtn.addEventListener("click", () => { idx = Math.max(0, idx - 1); redraw(); });
  nextBtn.addEventListener("click", () => { idx = Math.min(steps.length - 1, idx + 1); redraw();

  container.append(actions, readout, svg, caption);
  redraw();
  return container;
}
```

### 55.2.2. Correctness sketch

Dijkstra is a greedy algorithm. Before the proof, it's worth asking: *why would the greedy choice ever go wrong?* The answer is: it could go wrong if a later, distant vertex $y$ had a shortcut that made a longer-looking path to $u$ end up shorter. Non-negative edge weights prevent exactly this — you can never shorten a path by adding edges.

The key claim: when a vertex $u$ is moved into $S$, its tentative distance $\text{dist}[u]$ is the true shortest distance.

Proof by contradiction: suppose some path to $u$ is shorter, and let $y$ be the first vertex on that path not yet in $S$ when $u$ is picked. The portion of that path from the source to $y$ has length $\text{dist}[y]$ (since Dijkstra has already found the shortest path to $y$, which lies on our assumed shorter path). Any further edges from $y$ to $u$ have non-negative weight, so the full path is at least $\text{dist}[y]$ in length. But $\text{dist}[y] \geq \text{dist}[u]$,

because Dijkstra would have picked $y$ before $u$ if $y$ were closer. So the assumed shorter path is at least as long as $\text{dist}[u]$ — a contradiction.

This argument fails if any edge weight is negative — a later edge could make up for a large earlier one. For graphs with negative weights (but no negative-weight cycles), **Bellman-Ford** is the correct algorithm: it relaxes all edges $|V| - 1$ times, running in $O(|V||E|)$ time rather than Dijkstra's $O((|V| + |E|) \log |V|)$. One use case is currency arbitrage detection.

## 55.3. Minimum spanning tree

**Problem.** Given a connected weighted undirected graph, find a spanning tree — a tree that includes all vertices — with minimum total edge weight.

Two classical algorithms solve this; both are correct, and both rely on the same underlying property.

### 55.3.1. Cut property

A **cut** is a partition of $V$ into two non-empty sets $S$ and $V \setminus S$. An edge **crosses** the cut if one endpoint is in $S$ and the other is in $V \setminus S$. The cut property states: for any cut $(S, V \setminus S)$, if $e$ is the unique minimum-weight edge crossing that cut, then $e$ belongs to every minimum spanning tree; more generally, any minimum-weight edge crossing a given cut belongs to some minimum spanning tree. This is the single fact both Kruskal and Prim exploit.

Here is how this applies: in Kruskal's algorithm, every time we consider an edge $e = \{u, v\}$, the two components containing $u$ and $v$ define a cut. The edge $e$ is the cheapest edge crossing that cut (because we process edges in weight order and skip any that close a cycle). The cut property says it is safe to include $e$ in the MST. Every addition Kruskal makes is justified by exactly this argument. This means the cut property is the reason the greedy step is safe, not just a fact that happens to be true in the background.

### 55.3.2. Kruskal's algorithm

1. Sort all edges by weight (ascending).
2. Initialise $T = \emptyset$ (no edges selected yet).
3. Consider edges in order. Add edge $e = \{u, v\}$ to $T$ if it does not create a cycle (equivalently, if $u$ and $v$ are in different connected components of $T$).
4. Stop when $T$ has $n - 1$ edges.

Use a **union-find** data structure to test and merge components efficiently. Total time: $O(|E| \log |E|)$ (dominated by sorting).

**Worked example.** Six-node graph with edges (sorted by weight):

| Edge | Weight |
|---|---|
| C–E | 3 |
| A–C | 2 |
| A–B | 4 |
| E–D | 4 |

| Edge | Weight |
|---|---|
| B–C | 5 |
| E–F | 7 |
| B–D | 10 |
| D–F | 11 |

Sorted: A–C (2), C–E (3), A–B (4), E–D (4), B–C (5), E–F (7), B–D (10), D–F (11).

- Add A–C (2): no cycle. $T = \{$A–C$\}$.
- Add C–E (3): no cycle. $T = \{$A–C, C–E$\}$.
- Add A–B (4): no cycle. $T = \{$A–C, C–E, A–B$\}$.
- Add E–D (4): no cycle. $T = \{$A–C, C–E, A–B, E–D$\}$.
- Add B–C (5): creates cycle A–B–C–A. Skip.
- Add E–F (7): no cycle. $T = \{$A–C, C–E, A–B, E–D, E–F$\}$. Now $|T| = 5 = n - 1$. Done.

MST total weight: $2 + 3 + 4 + 4 + 7 = 20$.

### 55.3.3. Prim's algorithm

1. Start from an arbitrary seed vertex $r$. Mark $r$ as in the tree.
2. At each step, add the minimum-weight edge that connects a vertex inside the current tree to one outside it.
3. Repeat until all vertices are in the tree.

**Worked example** (same graph, starting from $A$):

- Tree: $\{A\}$. Cheapest crossing edge: A–C (2). Add C.
- Tree: $\{A, C\}$. Cheapest crossing edge: C–E (3). Add E.
- Tree: $\{A, C, E\}$. Cheapest crossing edge: A–B (4) or E–D (4). Take A–B. Add B.
- Tree: $\{A, C, E, B\}$. Cheapest crossing edge: E–D (4). Add D.
- Tree: $\{A, C, E, B, D\}$. Cheapest crossing edge: E–F (7). Add F.

MST edges: A–C, C–E, A–B, E–D, E–F. Total weight: 20. Same result as Kruskal.

Both algorithms always produce an MST. When the minimum-weight crossing edge for any cut is unique, the MST is unique. With ties, multiple MSTs of the same total weight may exist.

## 55.4. Network flows

**Problem.** A directed weighted graph has a distinguished **source** $s$ (supply) and **sink** $t$ (demand). Each directed edge $(u, v)$ has a **capacity** $c(u, v) \geq 0$ — the maximum flow it can carry. The **flow** $f(u, v)$ on each edge must satisfy:

- **Capacity constraint:** $0 \leq f(u, v) \leq c(u, v)$ for all edges.
- **Flow conservation:** for every vertex $v \neq s, t$, the total flow in equals the total flow out: $\sum_u f(u, v) = \sum_w f(v, w)$.

The **value of the flow** is the total flow leaving $s$ (equivalently, arriving at $t$). The problem is to maximise this value.

### 55.4.1. Ford-Fulkerson method

The key idea behind Ford-Fulkerson is that earlier routing decisions can be wrong, and the algorithm needs a way to undo them. If we sent flow along a path that turned out to block a better option, we want to be able to redirect it. The **residual graph** makes this possible by including a reverse edge for every forward edge: sending flow "backwards" on a reverse edge is equivalent to cancelling some previously routed flow.

1. Start with zero flow on all edges.
2. Find an **augmenting path** — any directed path from $s$ to $t$ in the **residual graph** $G_f$, where edge $(u, v)$ has residual capacity $c(u, v) - f(u, v)$ (how much more we can send forward) and the reverse edge $(v, u)$ has residual capacity $f(u, v)$ (how much previously routed flow we can cancel). This means the residual graph records both remaining room and the option to revise earlier routing choices.
3. Send as much flow along the augmenting path as the bottleneck edge allows.
4. Repeat until no augmenting path exists.

### 55.4.2. Max-flow min-cut theorem

A **cut** $(S, T)$ with $s \in S$ and $t \in T$ separates source from sink. The **capacity of the cut** is the sum of capacities of edges going from $S$ to $T$. The max-flow min-cut theorem states:

> **The maximum flow value equals the minimum cut capacity.**

The forward direction is clear: no flow can exceed the capacity of any cut separating $s$ from $t$, because every unit of flow must cross the cut. The deep direction is that Ford-Fulkerson achieves this bound — when no augmenting path remains, the set of vertices reachable from $s$ in the residual graph defines a cut whose capacity equals the current flow value.

In practice: to find the bottleneck in a network, find the minimum cut. The edges in the minimum cut are the ones whose capacity, if increased, would allow more total flow.

### 55.4.3. Worked example — 5-node network

Directed graph: source $S$, sink $T$, intermediate vertices $A$, $B$, $C$.

| Edge | Capacity |
|---|---|
| S⟶A | 10 |
| S⟶B | 8 |
| A⟶C | 9 |
| A⟶B | 3 |
| B⟶T | 10 |
| C⟶T | 7 |
| C⟶B | 2 |

| Edge | Capacity |
|---|---|

**Round 1.** Augmenting path $S \to A \to C \to T$. Bottleneck: $\min(10, 9, 7) = 7$. Send 7 units. Updated flows: S⟶A = 7, A⟶C = 7, C⟶T = 7.

**Round 2.** Augmenting path $S \to A \to B \to T$. Bottleneck: $\min(3, 3, 10) = 3$. Send 3 units. Updated flows: S⟶A = 10, A⟶B = 3, B⟶T = 3.

**Round 3.** Augmenting path $S \to B \to T$. Bottleneck: $\min(8, 7) = 7$. Send 7 units. Updated flows: S⟶B = 7, B⟶T = 10.

**Round 4.** Path $S \to A$: S⟶A is saturated (10/10). $S \to B$: B⟶T is saturated (10/10). No augmenting path exists. Done.

Maximum flow = 7 + 3 + 7 = 17 units.

**Minimum cut.** After Round 3, the residual capacities are: S⟶A: 0, S⟶B: 1, A⟶C: 2, A⟶B: 0, B⟶T: 0, C⟶T: 0, C⟶B: 2, plus reverse edges A⟵S: 10, B⟵S: 7, C⟵A: 7, B⟵A: 3, T⟵B: 10, T⟵C: 7.

Starting from $S$ in the residual graph: S⟶B has residual 1, so $B$ is reachable. From $B$: B⟶T has residual 0 (blocked); the reverse edge B⟵A carries residual 3, so $A$ is reachable. From $A$: A⟶C has residual 2, so $C$ is reachable. From $C$: C⟶T has residual 0 (blocked); C⟶B forward has residual 2 but $B$ is already in the set.

Reachable set: $\{S, B, A, C\}$. Non-reachable: $\{T\}$.

The minimum cut consists of the edges crossing from $\{S, B, A, C\}$ to $\{T\}$: that is B⟶T (capacity 10) and C⟶T (capacity 7). Min cut capacity = 10 + 7 = 17, which equals the max flow.

The minimum cut {B⟶T, C⟶T} identifies the bottleneck: these are the edges whose capacity limits the total throughput of the network.

Use the capacity sliders to see how the bottleneck edges control the maximum throughput of the whole network.

```
//| echo: false
{
  const pos = { S:[90,170], A:[240,90], B:[240,250], C:[400,170], T:[560,170] };
  const base = { SA:10, SB:8, AC:5, AB:3, BC:4, AT:4, BT:10, CT:7 };

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }

  function maxFlow(c) {
    const nodes = ["S","A","B","C","T"];
    const cap = {};
    const add = (u,v,w) => {
      cap[u] ||= {}; cap[v] ||= {};
      cap[u][v] = w;
      cap[v][u] ||= 0;
    };
    add("S","A",c.SA); add("S","B",c.SB); add("A","C",c.AC); add("A","B",c.AB);
```

```
  add("B","C",c.BC); add("A","T",c.AT); add("B","T",c.BT); add("C","T",c.CT);
  const resid = JSON.parse(JSON.stringify(cap));
  const flow = {};
  let total = 0;
  while (true) {
    const parent = { S:null };
    const queue = ["S"];
    while (queue.length && !("T" in parent)) {
      const u = queue.shift();
      for (const v of nodes) {
        if (!(v in parent) && resid[u]?.[v] > 1e-9) {
          parent[v] = u;
          queue.push(v);
        }
      }
    }
    if (!("T" in parent)) break;
    let aug = Infinity, v = "T";
    while (parent[v] !== null) {
      const u = parent[v];
      aug = Math.min(aug, resid[u][v]);
      v = u;
    }
    v = "T";
    while (parent[v] !== null) {
      const u = parent[v];
      resid[u][v] -= aug;
      resid[v][u] = (resid[v][u] || 0) + aug;
      flow[u] ||= {};
      flow[u][v] = (flow[u][v] || 0) + aug;
      v = u;
    }
    total += aug;
  }
  const reachable = new Set(["S"]);
  const queue = ["S"];
  while (queue.length) {
    const u = queue.shift();
    for (const v of nodes) {
      if (!reachable.has(v) && resid[u]?.[v] > 1e-9) {
        reachable.add(v);
        queue.push(v);
      }
    }
  }
  return { total, flow, resid, reachable };
}
```

```
const container = document.createElement("div");
container.style.cssText = "max-width:780px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.45rem 0.75re

const rows = {};
Object.entries(base).forEach(([key,val]) => {
  const label = document.createElement("label");
  label.textContent = `${key[0]} → ${key[1]} capacity`;
  const input = document.createElement("input");
  input.type = "range";
  input.min = "0";
  input.max = "14";
  input.step = "1";
  input.value = String(val);
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric:tabular-nums; min-width:2rem;";
  controls.append(label, input, out);
  rows[key] = { input, out };
});

const summary = document.createElement("div");
summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3
const flowBox = document.createElement("span");
const cutBox = document.createElement("span");
summary.append(flowBox, cutBox);

const svg = svgEl("svg");
svg.setAttribute("viewBox", "0 0 650 340");
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

const edgeMap = [["S","A","SA"],["S","B","SB"],["A","C","AC"],["A","B","AB"],["B","C","BC"],["A

function redraw() {
  const caps = {};
  Object.entries(rows).forEach(([k, row]) => {
    caps[k] = Number(row.input.value);
    row.out.textContent = row.input.value;
  });
  const { total, flow, reachable } = maxFlow(caps);
  svg.replaceChildren();
```

```
  const cutEdges = [];

  edgeMap.forEach(([u,v,key]) => {
    const [x1,y1] = pos[u], [x2,y2] = pos[v];
    const isCut = reachable.has(u) && !reachable.has(v);
    if (isCut) cutEdges.push(`${u}→${v}`);
    const line = svgEl("line");
    line.setAttribute("x1", x1); line.setAttribute("y1", y1);
    line.setAttribute("x2", x2); line.setAttribute("y2", y2);
    line.setAttribute("stroke", isCut ? "#dc2626" : "#94a3b8");
    line.setAttribute("stroke-width", isCut ? "5" : "3");
    svg.appendChild(line);

    const label = svgEl("text");
    label.setAttribute("x", (x1 + x2) / 2);
    label.setAttribute("y", (y1 + y2) / 2 - 8);
    label.setAttribute("text-anchor", "middle");
    label.setAttribute("font-size", "12");
    label.setAttribute("fill", "#0f172a");
    label.textContent = `${flow[u]?.[v] || 0}/${caps[key]}`;
    svg.appendChild(label);
  });

  Object.entries(pos).forEach(([node,[x,y]]) => {
    const c = svgEl("circle");
    c.setAttribute("cx", x); c.setAttribute("cy", y); c.setAttribute("r", 22);
    c.setAttribute("fill", reachable.has(node) ? "#1d4ed8" : "#e2e8f0");
    c.setAttribute("stroke", "#0f172a");
    c.setAttribute("stroke-width", "1.2");
    svg.appendChild(c);
    const t = svgEl("text");
    t.setAttribute("x", x); t.setAttribute("y", y + 4);
    t.setAttribute("text-anchor", "middle");
    t.setAttribute("font-size", "14");
    t.setAttribute("font-weight", "700");
    t.setAttribute("fill", reachable.has(node) ? "#fff" : "#0f172a");
    t.textContent = node;
    svg.appendChild(t);
  });

  flowBox.textContent = `maximum flow = ${total}`;
  cutBox.textContent = `highlighted min-cut edges = ${cutEdges.join(", ") || "none"}`;
  caption.textContent = "Edge labels show flow/capacity. Red edges are the current minimum cut: in
}

Object.values(rows).forEach(({input}) => input.addEventListener("input", redraw));
container.append(controls, summary, svg, caption);
```

```
  redraw();
  return container;
}
```

**Why Ford-Fulkerson terminates**
With integer capacities, each augmentation increases the flow by at least 1. Since the flow is bounded above by the sum of capacities out of $s$, the algorithm terminates. The reverse edges in the residual graph are what make it work correctly: they allow the algorithm to "undo" a routing decision and reroute flow along a more productive path.

## 55.5. Where this goes

The three problems in this chapter — shortest path, MST, network flow — are the foundational problems of **combinatorial optimisation**. They appear directly inside more complex algorithms: Dijkstra is the inner loop of many routing protocols; MST computation underlies Christofides' approximation algorithm for the travelling salesman problem; max-flow is equivalent to maximum bipartite matching via a reduction. If you proceed to the study of algorithms or operations research, you will see all three again.

The other extension worth knowing is **integer programming**, where variables are constrained to take integer values. Many practical network problems — crew scheduling, facility location, vehicle routing — are integer programs. Chapter 1 of this section covered the continuous relaxation; the integer case is in general NP-hard, but the network structure of the problems above makes them tractable.

**Where this shows up**

- A civil engineer routing roads or pipelines through terrain minimises a weighted path or tree.
- A telecoms network engineer provisioning bandwidth uses max-flow to find the network's throughput limit.
- A supply-chain analyst models goods movement as a flow problem and finds bottleneck warehouse links.
- A machine learning engineer working on graph neural networks needs the adjacency matrix representation as the input to the network.
- A biologist mapping evolutionary relationships uses minimum spanning trees on genetic distance matrices.

## 55.6. Exercises

These are convergent problems. Each has a definite answer. The work is in applying the algorithm correctly and reading the result.

**Exercise 1.** Run Dijkstra's algorithm on the following 5-node undirected weighted graph, starting from vertex $P$.

| Edge | Weight |
|---|---|
| P–Q | 6 |
| P–R | 2 |
| R–Q | 3 |
| R–S | 7 |
| Q–S | 1 |
| Q–T | 5 |
| S–T | 4 |

Trace all distance updates. State the final shortest distance to each vertex and give the shortest path to $T$.

**Exercise 2.** Find the MST of the following 6-node graph using Kruskal's algorithm. List the edges in the order they are added to the tree and give the total MST weight.

| Edge | Weight |
|---|---|
| 1–2 | 7 |
| 1–3 | 5 |
| 2–4 | 8 |
| 2–3 | 9 |
| 3–4 | 15 |
| 3–5 | 6 |
| 4–5 | 4 |
| 4–6 | 5 |
| 5–6 | 2 |

**Exercise 3.** Find the MST of the same 6-node graph from Exercise 2 using Prim's algorithm, starting from vertex 1. Verify that the total MST weight equals the result from Exercise 2.

**Exercise 4.** Find the maximum flow from $S$ to $T$ in the following 4-node network. Identify the minimum cut.

| Edge | Capacity |
|---|---|
| S→A | 8 |
| S→B | 6 |
| A→T | 5 |
| A→B | 4 |
| B→T | 9 |

**Exercise 5.** Let $G$ be a tree on $n$ vertices. Prove that the adjacency matrix $A$ of $G$ contains exactly $n - 1$ ones in each of its upper and lower triangular parts (i.e., $G$ has exactly $n - 1$ edges), using only the definition of a tree as a connected acyclic graph.

**Exercise 6 (Engineering application).** A water treatment facility distributes purified water via a network of pipes from a treatment plant (source $S$) to a distribution reservoir (sink $T$). The intermediate pump stations are $P$, $Q$, $R$.

| Pipe | Capacity (ML/day) |
|---|---|
| S→P | 12 |
| S→Q | 10 |
| P→Q | 3 |
| P→R | 8 |
| Q→T | 7 |
| R→Q | 4 |
| R→T | 9 |

Model this as a max-flow problem. Find the maximum daily throughput from $S$ to $T$ and identify the bottleneck cut — the set of pipes that, if their capacity were increased, would allow more water to reach $T$.

# 56. Probability theory

Quantifying uncertainty

A component comes off a production line. You don't know whether it's defective. You test 1000 and 23 fail. What can you say about the next one? About the next batch of 500?

These questions have precise answers. Getting them requires a framework for describing uncertainty — assigning likelihoods to outcomes, combining them consistently, and extracting the quantities that are actually useful.

## 56.1. Probability axioms

### 56.1.1. Sample space and events

The **sample space** $\Omega$ (read: "omega") is the complete set of possible outcomes of an experiment. For a coin flip, $\Omega = \{\text{heads}, \text{tails}\}$. For rolling a six-sided die, $\Omega = \{1, 2, 3, 4, 5, 6\}$. For the lifetime of a component in hours, $\Omega = [0, \infty)$.

An **event** $A$ is any subset of $\Omega$ — a collection of outcomes you care about. The event "roll an even number" is the subset $\{2, 4, 6\} \subset \Omega$.

### 56.1.2. Kolmogorov axioms

A **probability measure** $P$ assigns a number to each event. Three axioms define what that assignment must satisfy:

1. **Non-negativity:** $P(A) \geq 0$ for every event $A$.
2. **Normalisation:** $P(\Omega) = 1$ — something must happen.
3. **Additivity:** For mutually exclusive events $A$ and $B$ (events that cannot both occur), $P(A \cup B) = P(A) + P(B)$.

These axioms are the whole foundation. Everything else is a consequence.

From them you can show:

- $P(\emptyset) = 0$ (the impossible event has probability zero)
- $P(A^c) = 1 - P(A)$, where $A^c$ is the complement of $A$ (everything not in $A$)
- If $A \subset B$ then $P(A) \leq P(B)$
- The inclusion-exclusion rule: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

The third axiom extends to any countable collection of mutually exclusive events (finitely many, or an infinite but listable sequence): $P(A_1 \cup A_2 \cup \cdots) = P(A_1) + P(A_2) + \cdots$

### 56.1.3. Conditional probability

The **conditional probability** of $A$ given $B$ — written $P(A \mid B)$, read "probability of $A$ given $B$" — is defined as:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

This is the fraction of $B$'s probability that overlaps with $A$. If you know $B$ has occurred, you're restricting attention to the outcomes in $B$ and asking what fraction of those are also in $A$. This means conditioning changes the universe you are measuring inside: once $B$ is known to have happened, all probabilities are rescaled relative to $B$.

```
//| echo: false
{
  // ── Conditional probability area model (OPPORTUNITY 1) ──────────────────
  const W=520, H=320, PAD={top:20,right:180,bottom:20,left:20};
  const BLUE="rgba(59,130,246,0.55)", ORANGE="rgba(249,115,22,0.55)";
  const BLUE_STROKE="#3b82f6", ORANGE_STROKE="#f97316";
  const BLEND="rgba(139,92,246,0.55)";

  // A: ellipse centred at (ax,ay) with radii (arx,ary)
  // B: ellipse centred at (bx,by) with radii (brx,bry)
  let state={ax:0.3,ay:0.5,arx:0.22,ary:0.30,bx:0.62,by:0.5,brx:0.24,bry:0.30};
  let condOnB=false;
  let dragging=null, dragStart=null;
  let svgRect=null;

  function ellipseArea(rx,ry) { return Math.PI*rx*ry; } // in unit-square coords
  function intersectionArea(state) {
    // Monte Carlo estimate
    const {ax,ay,arx,ary,bx,by,brx,bry}=state;
    let inA=0,inB=0,inBoth=0;
    const M=3000;
    let s=42;
    function lcg(){s=(s*1664525+1013904223)&0xffffffff;return(s>>>0)/0xffffffff;}
    for(let i=0;i<M;i++){
      const x=lcg(),y=lcg();
      const a=((x-ax)/arx)**2+((y-ay)/ary)**2<=1;
      const b=((x-bx)/brx)**2+((y-by)/bry)**2<=1;
      if(a)inA++;if(b)inB++;if(a&&b)inBoth++;
    }
    return {pA:inA/M, pB:inB/M, pAB:inBoth/M};
  }
```

```
const container=document.createElement("div");
container.style.cssText="font-family:inherit; max-width:560px; margin:1rem 0;";

const svgEl=document.createElementNS("http://www.w3.org/2000/svg","svg");
svgEl.setAttribute("width",W); svgEl.setAttribute("height",H);
svgEl.style.cssText="border:1px solid #e5e7eb; border-radius:6px; background:#fff; display:bloc
container.appendChild(svgEl);

// controls
const ctrlRow=document.createElement("div");
ctrlRow.style.cssText="display:flex; gap:0.5rem; align-items:center; margin-top:0.5rem; flex-wr
const condBtn=document.createElement("button");
condBtn.textContent="Condition on B";
condBtn.style.cssText="padding:0.3rem 0.8rem; border:1px solid #d1d5db; border-radius:4px; curs
condBtn.onclick=()=>{condOnB=!condOnB; condBtn.style.background=condOnB?ORANGE_STROKE:"#f9fafb"
const resetBtn=document.createElement("button");
resetBtn.textContent="Reset";
resetBtn.style.cssText="padding:0.3rem 0.7rem; border:1px solid #e5e7eb; border-radius:4px; cur
resetBtn.onclick=()=>{state={ax:0.3,ay:0.5,arx:0.22,ary:0.30,bx:0.62,by:0.5,brx:0.24,bry:0.30};
ctrlRow.appendChild(condBtn); ctrlRow.appendChild(resetBtn);
const hint=document.createElement("span");
hint.style.cssText="font-size:0.78em; color:#6b7280;"; hint.textContent="Drag A or B to reposit
ctrlRow.appendChild(hint);
container.appendChild(ctrlRow);

const labelDiv=document.createElement("div");
labelDiv.style.cssText="margin-top:0.35rem; font-size:0.82em; color:#6b7280; font-style:italic;
labelDiv.textContent='Conditioning on B means zooming in to B. P(A|B) is the fraction of B that
container.appendChild(labelDiv);

function toSVG(ux,uy) {
  const sqW=W-PAD.left-PAD.right, sqH=H-PAD.top-PAD.bottom;
  return [PAD.left+ux*sqW, PAD.top+uy*sqH];
}
function fromSVG(sx,sy) {
  const sqW=W-PAD.left-PAD.right, sqH=H-PAD.top-PAD.bottom;
  return [(sx-PAD.left)/sqW, (sy-PAD.top)/sqH];
}

function draw() {
  while(svgEl.firstChild) svgEl.removeChild(svgEl.firstChild);
  const sqW=W-PAD.left-PAD.right, sqH=H-PAD.top-PAD.bottom;
  const {ax,ay,arx,ary,bx,by,brx,bry}=state;

  // background rect (Ω)
  const omegaRect=document.createElementNS("http://www.w3.org/2000/svg","rect");
  omegaRect.setAttribute("x",PAD.left); omegaRect.setAttribute("y",PAD.top);
```

```
omegaRect.setAttribute("width",sqW); omegaRect.setAttribute("height",sqH);
omegaRect.setAttribute("fill", condOnB?"#f3f4f6":"#f9fafb");
omegaRect.setAttribute("stroke","#d1d5db"); omegaRect.setAttribute("rx","4");
svgEl.appendChild(omegaRect);
const omLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
omLbl.setAttribute("x",PAD.left+6); omLbl.setAttribute("y",PAD.top+16);
omLbl.setAttribute("font-size","13"); omLbl.setAttribute("fill","#9ca3af");
omLbl.textContent="Ω"; svgEl.appendChild(omLbl);

const [axSVG,aySVG]=toSVG(ax,ay);
const [bxSVG,bySVG]=toSVG(bx,by);
const arxSVG=arx*sqW, arySVG=ary*sqH;
const brxSVG=brx*sqW, brySVG=bry*sqH;

function makeEllipse(cx,cy,rx,ry,fill,stroke,opacity=1) {
  const el=document.createElementNS("http://www.w3.org/2000/svg","ellipse");
  el.setAttribute("cx",cx); el.setAttribute("cy",cy);
  el.setAttribute("rx",rx); el.setAttribute("ry",ry);
  el.setAttribute("fill",fill); el.setAttribute("stroke",stroke);
  el.setAttribute("stroke-width","2"); el.setAttribute("opacity",opacity);
  return el;
}

if(!condOnB) {
  svgEl.appendChild(makeEllipse(axSVG,aySVG,arxSVG,arySVG,BLUE,BLUE_STROKE));
  svgEl.appendChild(makeEllipse(bxSVG,bySVG,brxSVG,brySVG,ORANGE,ORANGE_STROKE));
} else {
  // grey out everything, show B clearly, show A∩B
  svgEl.appendChild(makeEllipse(axSVG,aySVG,arxSVG,arySVG,BLUE,BLUE_STROKE,0.18));
  svgEl.appendChild(makeEllipse(bxSVG,bySVG,brxSVG,brySVG,ORANGE,ORANGE_STROKE,0.55));
  // A∩B region approximated visually as darkened intersection
  const overEl=document.createElementNS("http://www.w3.org/2000/svg","ellipse");
  // Draw a smaller "intersection" ellipse centred at the midpoint, scaled
  const ixc=(axSVG+bxSVG)/2, iyc=(aySVG+bySVG)/2;
  const irx=Math.max(2,Math.min(arxSVG,brxSVG)*0.65);
  const iry=Math.max(2,Math.min(arySVG,brySVG)*0.65);
  const dist=Math.sqrt((axSVG-bxSVG)**2+(aySVG-bySVG)**2);
  if(dist<(arxSVG+brxSVG)*0.85) {
    const intEl=makeEllipse(ixc,iyc,irx,iry,BLEND,BLUE_STROKE,0.7);
    svgEl.appendChild(intEl);
  }
}

// Labels A and B
const aLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
aLbl.setAttribute("x",axSVG); aLbl.setAttribute("y",aySVG+4);
aLbl.setAttribute("text-anchor","middle"); aLbl.setAttribute("font-size","16");
```

```
  aLbl.setAttribute("font-weight","700"); aLbl.setAttribute("fill","#1d4ed8"); aLbl.textContent
  svgEl.appendChild(aLbl);
  const bLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  bLbl.setAttribute("x",bxSVG); bLbl.setAttribute("y",bySVG+4);
  bLbl.setAttribute("text-anchor","middle"); bLbl.setAttribute("font-size","16");
  bLbl.setAttribute("font-weight","700"); bLbl.setAttribute("fill","#c2410c"); bLbl.textContent
  svgEl.appendChild(bLbl);

  // Readout panel (right side)
  const panelX=W-PAD.right+14, panelY=PAD.top;
  const {pA,pB,pAB}=intersectionArea(state);
  const pAgivenB=pB>0.001?pAB/pB:0;
  const indep=Math.abs(pAB-pA*pB)<0.02;
  const lines=[
    {label:"P(A):",val:pA.toFixed(2),color:"#3b82f6"},
    {label:"P(B):",val:pB.toFixed(2),color:"#f97316"},
    {label:"P(A∩B):",val:pAB.toFixed(2),color:"#7c3aed"},
    {label:"P(A|B):",val:pAgivenB.toFixed(2),color:"#374151",big:true},
  ];
  lines.forEach(({label,val,color,big},i)=>{
    const lbl=document.createElementNS("http://www.w3.org/2000/svg","text");
    lbl.setAttribute("x",panelX); lbl.setAttribute("y",panelY+25+i*26);
    lbl.setAttribute("font-size",big?"13":"12"); lbl.setAttribute("fill",color);
    lbl.setAttribute("font-weight",big?"700":"400");
    lbl.textContent=`${label} ${val}`;
    svgEl.appendChild(lbl);
  });
  if(indep) {
    const indLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
    indLbl.setAttribute("x",panelX); indLbl.setAttribute("y",panelY+135);
    indLbl.setAttribute("font-size","10"); indLbl.setAttribute("fill","#059669");
    indLbl.textContent="∩ ∩ Independent";
    svgEl.appendChild(indLbl);
  }
  if(condOnB) {
    const condNote=document.createElementNS("http://www.w3.org/2000/svg","text");
    condNote.setAttribute("x",panelX); condNote.setAttribute("y",panelY+160);
    condNote.setAttribute("font-size","9"); condNote.setAttribute("fill","#6b7280");
    condNote.textContent="Inside B only.";
    svgEl.appendChild(condNote);
  }
}

// drag handling
function getPos(e) {
  svgRect=svgEl.getBoundingClientRect();
  const cx=e.clientX-svgRect.left, cy=e.clientY-svgRect.top;
```

```
    return fromSVG(cx,cy);
  }
  svgEl.addEventListener("mousedown",e=>{
    const [ux,uy]=getPos(e);
    const {ax,ay,arx,ary,bx,by,brx,bry}=state;
    const inA=((ux-ax)/arx)**2+((uy-ay)/ary)**2<=1;
    const inB=((ux-bx)/brx)**2+((uy-by)/bry)**2<=1;
    if(inB) dragging="B"; else if(inA) dragging="A";
    dragStart=[ux,uy];
  });
  window.addEventListener("mousemove",e=>{
    if(!dragging) return;
    const [ux,uy]=getPos(e);
    const [dx,dy]=[ux-dragStart[0],uy-dragStart[1]];
    dragStart=[ux,uy];
    if(dragging=="A") { state.ax=Math.max(0.1,Math.min(0.9,state.ax+dx)); state.ay=Math.max(0.1,Mat
    if(dragging=="B") { state.bx=Math.max(0.1,Math.min(0.9,state.bx+dx)); state.by=Math.max(0.1,Mat
    draw();
  });
  window.addEventListener("mouseup",()=>{ dragging=null; });

  draw();
  return container;
}
```

Two events $A$ and $B$ are **independent** if knowing $B$ occurred tells you nothing about $A$:

$$P(A \mid B) = P(A) \iff P(A \cap B) = P(A) \cdot P(B)$$

The equivalence is just one substitution. Starting from $P(A \mid B) = P(A)$ and using the definition of conditional probability:

$$\frac{P(A \cap B)}{P(B)} = P(A) \implies P(A \cap B) = P(A)\,P(B)$$

Conversely, dividing $P(A \cap B) = P(A)\,P(B)$ by $P(B)$ (when $P(B) > 0$) returns $P(A \mid B) = P(A)$.

### 56.1.4. Bayes' theorem

Rearranging the definition of conditional probability:

$$P(A \cap B) = P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$$

Solving for $P(A \mid B)$:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

This is **Bayes' theorem**. It lets you reverse the conditioning: if you know $P(B \mid A)$, you can compute $P(A \mid B)$.

The denominator $P(B)$ is expanded using the **total probability rule**. If $A_1, A_2, \ldots, A_n$ partition $\Omega$ (exhaustive, mutually exclusive), then:

$$P(B) = \sum_{i=1}^{n} P(B \mid A_i) \cdot P(A_i)$$

> **Why this works**
> Bayes' theorem doesn't involve any new mathematics — it's just two ways of writing the same joint probability $P(A \cap B)$, set equal. What makes it powerful is the direction: you have the likelihood $P(B \mid A)$ (how probable is the evidence given the hypothesis?) and you want $P(A \mid B)$ (how probable is the hypothesis given the evidence?). Bayes is the bridge between them.

**Worked example: Medical test.** A disease affects 1% of the population. A test for it has sensitivity 99% (correctly identifies 99% of sick patients) and specificity 95% (correctly clears 95% of healthy patients). A randomly selected person tests positive. What is the probability they actually have the disease?

Define: - $D$: the person has the disease - $T^+$: the test is positive

Given: $P(D) = 0.01$, $P(T^+ \mid D) = 0.99$ (sensitivity), $P(T^+ \mid D^c) = 0.05$

Note also that $P(D^c) = 1 - P(D) = 1 - 0.01 = 0.99$ — the complement of the prevalence, which happens to equal the sensitivity by coincidence.

First compute the total probability of a positive test, using the two ways a positive result can occur:

$$P(T^+) = P(T^+ \mid D) \cdot P(D) + P(T^+ \mid D^c) \cdot P(D^c)$$

$$= \underbrace{0.99}_{\text{sensitivity}} \times 0.01 + 0.05 \times \underbrace{0.99}_{P(D^c)=1-0.01} = 0.0099 + 0.0495 = 0.0594$$

Now apply Bayes:

$$P(D \mid T^+) = \frac{P(T^+ \mid D) \cdot P(D)}{P(T^+)} = \frac{0.99 \times 0.01}{0.0594} \approx 0.167$$

Even with a positive result from a 99%-sensitive test, the probability of actually having the disease is only about 17%. This is not a failure of the test — it's a consequence of the disease being rare. Most of the positive tests come from the large healthy population, not the small sick one. The prior probability $P(D)$ matters enormously.

## 56.2. Random variables

A **random variable** $X$ is a function from the sample space $\Omega$ to the real numbers — it assigns a numerical value to each outcome. The randomness is in the outcome; the function is deterministic.

**Discrete random variables** take values in a countable set $\{x_1, x_2, \ldots\}$. The distribution is completely described by the **probability mass function** (PMF):

$$p(x_i) = P(X = x_i), \quad \sum_i p(x_i) = 1$$

**Continuous random variables** take values in an interval (or union of intervals). The distribution is described by the **probability density function** (PDF) $f(x)$, where:

$$P(a \leq X \leq b) = \int_a^b f(x)\,dx, \quad \int_{-\infty}^{\infty} f(x)\,dx = 1$$

Note that $f(x)$ is *not* a probability — it can exceed 1. It's a density: the probability of $X$ falling in a small interval $[x, x + dx]$ is approximately $f(x)\,dx$.

A concrete example: if $X \sim \text{Uniform}(0,\ 0.5)$, then $f(x) = 2$ everywhere on $[0, 0.5]$. The density is 2, yet $P(0 \leq X \leq 0.5) = \int_0^{0.5} 2\,dx = 1$. The density can exceed 1; the integral over any interval is still between 0 and 1. This means the density is a probability-per-unit-length, not a probability by itself. Probabilities come from area under the curve, not from the height alone.

The **cumulative distribution function** (CDF) is defined for both types:

$$F(x) = P(X \leq x)$$

For continuous $X$: $F(x) = \int_{-\infty}^{x} f(t)\,dt$, and $f(x) = F'(x)$.

### 56.2.1. Expectation

The **expected value** $E[X]$ — also written $\mu$ (mu) — is the long-run average of $X$ over many repetitions:

**Discrete:** $E[X] = \sum_i x_i\, p(x_i)$

**Continuous:** $E[X] = \int_{-\infty}^{\infty} x\, f(x)\,dx$

Expectation is linear: $E[aX + b] = a\,E[X] + b$ for constants $a$, $b$.

For a function $g(X)$:

$$E[g(X)] = \sum_i g(x_i)\, p(x_i) \quad \text{(discrete)}$$

$$E[g(X)] = \int_{-\infty}^{\infty} g(x)\, f(x)\,dx \quad \text{(continuous)}$$

### 56.2.2. Variance

The **variance** $\mathrm{Var}(X)$ — also written $\sigma^2$ (sigma squared) — measures the average squared deviation from the mean:

$$\mathrm{Var}(X) = E\big[(X-\mu)^2\big] = E[X^2] - (E[X])^2$$

The second form is usually easier to compute. The derivation:

$$E\big[(X-\mu)^2\big] = E[X^2 - 2\mu X + \mu^2] = E[X^2] - 2\mu E[X] + \mu^2$$

Since $E[X] = \mu$, the last two terms combine: $-2\mu \cdot \mu + \mu^2 = -2\mu^2 + \mu^2 = -\mu^2$. So:

$$= E[X^2] - \mu^2$$

This means variance is the average squared distance from the mean. The alternative formula $E[X^2] - \mu^2$ is usually easier to compute, but it is measuring the same spread.

The **standard deviation** $\sigma = \sqrt{\mathrm{Var}(X)}$ is in the same units as $X$, which makes it interpretable.

Variance scales with the square: $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$.

For independent random variables $X$ and $Y$: $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$.

## 56.3. Key discrete distributions

### 56.3.1. Bernoulli

A single trial with probability $p$ of success, $1 - p$ of failure.

$$P(X = 1) = p, \quad P(X = 0) = 1 - p$$

$$E[X] = p, \quad \mathrm{Var}(X) = p(1-p)$$

The Bernoulli is the building block for everything that follows.

### 56.3.2. Binomial

$n$ independent Bernoulli trials, each with success probability $p$. $X$ counts the total number of successes.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, 1, \dots, n$$

The binomial coefficient $\binom{n}{k}$ — read "n choose k" — counts the number of ways to arrange $k$ successes among $n$ trials:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

**Mean and variance.** Since $X$ is the sum of $n$ independent Bernoulli trials, linearity of expectation and variance additivity give:

$$E[X] = np, \quad \mathrm{Var}(X) = np(1-p)$$

**Normal approximation.** When $np \geq 5$ and $n(1-p) \geq 5$, the binomial is well approximated by a normal distribution with the same mean and variance. This is one preview of the CLT, which appears later in this chapter and explains why sums and averages often become approximately normal.

### 56.3.3. Poisson

Models the number of events occurring in a fixed interval of time or space, when events occur independently at a constant average rate.

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

The parameter $\lambda > 0$ — read "lambda" — is both the mean and the variance:

$$E[X] = \lambda, \quad \mathrm{Var}(X) = \lambda$$

**As a limit of Binomial.** Set $\lambda = np$ and let $n \to \infty$, $p \to 0$. The binomial PMF converges to the Poisson. This is why the Poisson appears when events are rare but trials are many: the number of typing errors per page, radioactive decays per second, server requests per minute.

**Derivation of the mean.** Using the Poisson PMF:

$$E[X] = \sum_{k=0}^{\infty} k \cdot \frac{\lambda^k e^{-\lambda}}{k!} = e^{-\lambda} \sum_{k=1}^{\infty} \frac{\lambda^k}{(k-1)!} = e^{-\lambda} \cdot \lambda \sum_{j=0}^{\infty} \frac{\lambda^j}{j!} = e^{-\lambda} \cdot \lambda \cdot e^{\lambda} = \lambda$$

```
//| echo: false
{
  // ── Distribution landscape explorer ──────────────────────────────────────
  const W = 500, H = 280, PAD = { top: 24, right: 20, bottom: 40, left: 48 };
  const ACCENT = "#3b82f6";
  const ACCENT_MUTED = "rgba(59,130,246,0.18)";

  // ── state ────────────────────────────────────────────────────────────────
  const distNames = ["Bernoulli", "Binomial", "Poisson", "Exponential", "Normal"];
  let distIdx = 4;     // default: Normal
  let params = { p: 0.5, n: 10, lam: 2, mu: 0, sigma: 1 };

  // ── helpers ──────────────────────────────────────────────────────────────
  function factorial(k) { let r=1; for(let i=2;i<=k;i++) r*=i; return r; }
  function normalPDF(x,mu,sig) { return Math.exp(-0.5*((x-mu)/sig)**2)/(sig*Math.sqrt(2*Math.PI))
  function normalCDF(x,mu,sig) {
    const z=(x-mu)/(sig*Math.SQRT2);
    return 0.5*(1+erf(z));
  }
  function erf(x) {
    const sign = x<0?-1:1; x=Math.abs(x);
    const a1=0.254829592,a2=-0.284496736,a3=1.421413741,a4=-1.453152027,a5=1.061405429,p=0.327591
    const t=1/(1+p*x);
    const y=1-(((((a5*t+a4)*t)+a3)*t+a2)*t+a1)*t*Math.exp(-x*x);
    return sign*y;
  }
  function poissonPMF(k,lam) { return Math.pow(lam,k)*Math.exp(-lam)/factorial(k); }
  function binomPMF(k,n,p) {
    if(k<0||k>n) return 0;
    const logC = logBinom(n,k);
    return Math.exp(logC + k*Math.log(p) + (n-k)*Math.log(1-p));
  }
  function logBinom(n,k) {
    let r=0; for(let i=0;i<k;i++) r+=Math.log(n-i)-Math.log(i+1); return r;
  }

  function getDistInfo() {
    const d = distNames[distIdx];
    if(d=="Bernoulli") {
      const p=params.p; const mu=p; const variance=p*(1-p); const sigma=Math.sqrt(variance);
      const xs=[0,1]; const ys=[1-p,p];
      return { discrete:true, xs, ys, mu, variance, sigma, label:`Bernoulli(p=${p.toFixed(2)})` }
    }
    if(d=="Binomial") {
      const {n,p}=params; const mu=n*p; const variance=n*p*(1-p); const sigma=Math.sqrt(variance)
      const xs=[...Array(n+1).keys()]; const ys=xs.map(k=>binomPMF(k,n,p));
      return { discrete:true, xs, ys, mu, variance, sigma, label:`Binomial(n=${n}, p=${p.toFixed(
```

```
  }
  if(d=="Poisson") {
    const lam=params.lam; const mu=lam; const variance=lam; const sigma=Math.sqrt(lam);
    const maxK=Math.ceil(lam+4*sigma+5);
    const xs=[...Array(maxK+1).keys()]; const ys=xs.map(k=>poissonPMF(k,lam));
    return { discrete:true, xs, ys, mu, variance, sigma, label:`Poisson(□=${lam.toFixed(1)})` };
  }
  if(d=="Exponential") {
    const lam=params.lam; const mu=1/lam; const variance=1/(lam*lam); const sigma=1/lam;
    const xmax=mu+5*sigma; const nPts=200;
    const xs=Array.from({length:nPts},(_,i)=>i*xmax/(nPts-1));
    const ys=xs.map(x=>x>=0?lam*Math.exp(-lam*x):0);
    return { discrete:false, xs, ys, mu, variance, sigma, label:`Exponential(□=${lam.toFixed(2)})
  }
  // Normal
  const {mu,sigma}=params; const variance=sigma*sigma;
  const xmin=mu-4*sigma, xmax=mu+4*sigma; const nPts=200;
  const xs=Array.from({length:nPts},(_,i)=>xmin+i*(xmax-xmin)/(nPts-1));
  const ys=xs.map(x=>normalPDF(x,mu,sigma));
  return { discrete:false, xs, ys, mu, variance, sigma, label:`Normal(□=${mu.toFixed(1)}, □=${sigm
}

// —— build UI ————————————————————————————————————————————————————
const container = document.createElement("div");
container.style.cssText = "font-family:inherit; max-width:680px; margin:1rem 0;";

// dist selector
const selectorRow = document.createElement("div");
selectorRow.style.cssText = "display:flex; gap:0.4rem; flex-wrap:wrap; margin-bottom:0.75rem;";
const btns = distNames.map((name,i)=>{
  const b=document.createElement("button");
  b.textContent=name; b.dataset.idx=i;
  b.style.cssText = "padding:0.3rem 0.7rem; border-radius:4px; border:1px solid #d1d5db; cursor:po
  b.onclick=()=>{ distIdx=i; update(); };
  selectorRow.appendChild(b);
  return b;
});
container.appendChild(selectorRow);

// main layout: left (canvas) + right (readouts)
const mainRow = document.createElement("div");
mainRow.style.cssText = "display:flex; gap:1rem; align-items:flex-start;";

// svg
const svg = document.createElementNS("http://www.w3.org/2000/svg","svg");
svg.setAttribute("width", W); svg.setAttribute("height", H);
svg.style.cssText = "border:1px solid #e5e7eb; border-radius:6px; background:#fff; flex-shrink:0;"
```

```
mainRow.appendChild(svg);

// readout panel
const readouts = document.createElement("div");
readouts.style.cssText = "min-width:170px; padding:0.75rem; background:#f8fafc; border:1px soli
mainRow.appendChild(readouts);
container.appendChild(mainRow);

// sliders area
const slidersDiv = document.createElement("div");
slidersDiv.style.cssText = "margin-top:0.75rem; padding:0.75rem; background:#f8fafc; border:1px
container.appendChild(slidersDiv);

// label
const labelDiv = document.createElement("div");
labelDiv.style.cssText = "margin-top:0.5rem; font-size:0.82em; color:#6b7280; font-style:italic
labelDiv.textContent = "Every distribution is a landscape: a shape, a centre, and a spread.";
container.appendChild(labelDiv);

function makeSlider(labelText, key, min, max, step, getValue, setValue) {
  const row = document.createElement("div");
  row.style.cssText = "display:flex; align-items:center; gap:0.5rem; margin-bottom:0.4rem;";
  const lbl = document.createElement("label");
  lbl.style.cssText = "font-size:0.85em; min-width:80px;";
  lbl.textContent = labelText;
  const inp = document.createElement("input");
  inp.type="range"; inp.min=min; inp.max=max; inp.step=step; inp.value=getValue();
  inp.style.cssText = "flex:1;";
  const val = document.createElement("span");
  val.style.cssText = "font-size:0.85em; min-width:35px; text-align:right;";
  val.textContent = Number(getValue()).toFixed(step<0.1?2:step<1?1:0);
  inp.oninput = ()=>{ setValue(parseFloat(inp.value)); val.textContent=Number(inp.value).toFixe
  row.appendChild(lbl); row.appendChild(inp); row.appendChild(val);
  return {row, inp, val};
}

// ── draw ──────────────────────────────────────────────────────────────
function draw(info) {
  while(svg.firstChild) svg.removeChild(svg.firstChild);
  const cw = W - PAD.left - PAD.right;
  const ch = H - PAD.top - PAD.bottom;

  const xMin = info.discrete ? Math.min(...info.xs)-0.5 : (info.xmin !== undefined ? info.xmin
  const xMax = info.discrete ? Math.max(...info.xs)+0.5 : (info.xmax !== undefined ? info.xmax
  const yMax = Math.max(...info.ys)*1.15;

  const sx = x => PAD.left + (x-xMin)/(xMax-xMin)*cw;
```

```
const sy = y => PAD.top + ch - y/yMax*ch;

// grid lines (light)
for(let i=0;i<=4;i++) {
  const y = i*yMax/4;
  const line = document.createElementNS("http://www.w3.org/2000/svg","line");
  line.setAttribute("x1",PAD.left); line.setAttribute("x2",PAD.left+cw);
  line.setAttribute("y1",sy(y)); line.setAttribute("y2",sy(y));
  line.setAttribute("stroke","#f0f0f0"); line.setAttribute("stroke-width","1");
  svg.appendChild(line);
}

// ±1σ band shading (continuous only)
if(!info.discrete) {
  const {mu,sigma,xs,ys}=info;
  const x1c=Math.max(xMin,mu-sigma), x2c=Math.min(xMax,mu+sigma);
  // build polygon
  const bandXs=xs.filter(x=>x>=x1c&&x<=x2c);
  if(bandXs.length>1) {
    const pts=[`${sx(x1c)},${sy(0)}`,...bandXs.map((x,j)=>{
      const y=ys[xs.indexOf(x)]; return `${sx(x)},${sy(y)}`;
    }),`${sx(x2c)},${sy(0)}`].join(" ");
    const poly=document.createElementNS("http://www.w3.org/2000/svg","polygon");
    poly.setAttribute("points",pts); poly.setAttribute("fill",ACCENT_MUTED);
    svg.appendChild(poly);
  }
}

// distribution shape
if(info.discrete) {
  info.xs.forEach((x,i)=>{
    const y=info.ys[i];
    const line=document.createElementNS("http://www.w3.org/2000/svg","line");
    line.setAttribute("x1",sx(x)); line.setAttribute("x2",sx(x));
    line.setAttribute("y1",sy(0)); line.setAttribute("y2",sy(y));
    line.setAttribute("stroke",ACCENT); line.setAttribute("stroke-width","2");
    svg.appendChild(line);
    const dot=document.createElementNS("http://www.w3.org/2000/svg","circle");
    dot.setAttribute("cx",sx(x)); dot.setAttribute("cy",sy(y)); dot.setAttribute("r","4");
    dot.setAttribute("fill",ACCENT);
    svg.appendChild(dot);
  });
} else {
  const pts=info.xs.map((x,i)=>`${sx(x)},${sy(info.ys[i])}`).join(" ");
  const area=document.createElementNS("http://www.w3.org/2000/svg","polygon");
  area.setAttribute("points",`${sx(info.xs[0])},${sy(0)} ${pts} ${sx(info.xs[info.xs.length-1])}
  area.setAttribute("fill",ACCENT); area.setAttribute("fill-opacity","0.35");
```

```
    svg.appendChild(area);
    const path=document.createElementNS("http://www.w3.org/2000/svg","polyline");
    path.setAttribute("points",pts); path.setAttribute("fill","none");
    path.setAttribute("stroke",ACCENT); path.setAttribute("stroke-width","2");
    svg.appendChild(path);
  }

  // mean line
  const muX=sx(info.mu);
  const muLine=document.createElementNS("http://www.w3.org/2000/svg","line");
  muLine.setAttribute("x1",muX); muLine.setAttribute("x2",muX);
  muLine.setAttribute("y1",PAD.top); muLine.setAttribute("y2",PAD.top+ch);
  muLine.setAttribute("stroke","#374151"); muLine.setAttribute("stroke-width","1.5");
  muLine.setAttribute("stroke-dasharray","5,3");
  svg.appendChild(muLine);
  const muLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  muLbl.setAttribute("x",muX+4); muLbl.setAttribute("y",PAD.top+10);
  muLbl.setAttribute("font-size","11"); muLbl.setAttribute("fill","#374151");
  muLbl.textContent=`□=${info.mu.toFixed(2)}`;
  svg.appendChild(muLbl);

  // ±1□ bracket
  const s1x1=sx(info.mu-info.sigma), s1x2=sx(info.mu+info.sigma);
  const bracketY=PAD.top+ch+14;
  const bline=document.createElementNS("http://www.w3.org/2000/svg","line");
  bline.setAttribute("x1",Math.max(PAD.left,s1x1)); bline.setAttribute("x2",Math.min(PAD.left+c
  bline.setAttribute("y1",bracketY); bline.setAttribute("y2",bracketY);
  bline.setAttribute("stroke",ACCENT); bline.setAttribute("stroke-width","2");
  svg.appendChild(bline);
  const blbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  blbl.setAttribute("x",(Math.max(PAD.left,s1x1)+Math.min(PAD.left+cw,s1x2))/2);
  blbl.setAttribute("y",bracketY+12); blbl.setAttribute("text-anchor","middle");
  blbl.setAttribute("font-size","10"); blbl.setAttribute("fill",ACCENT);
  blbl.textContent="±1□";
  svg.appendChild(blbl);

  // axes
  const xAxis=document.createElementNS("http://www.w3.org/2000/svg","line");
  xAxis.setAttribute("x1",PAD.left); xAxis.setAttribute("x2",PAD.left+cw);
  xAxis.setAttribute("y1",sy(0)); xAxis.setAttribute("y2",sy(0));
  xAxis.setAttribute("stroke","#374151"); xAxis.setAttribute("stroke-width","1");
  svg.appendChild(xAxis);
  const yAxis=document.createElementNS("http://www.w3.org/2000/svg","line");
  yAxis.setAttribute("x1",PAD.left); yAxis.setAttribute("x2",PAD.left);
  yAxis.setAttribute("y1",PAD.top); yAxis.setAttribute("y2",sy(0));
  yAxis.setAttribute("stroke","#374151"); yAxis.setAttribute("stroke-width","1");
  svg.appendChild(yAxis);
```

```
  // y-axis label
  const yLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  yLbl.setAttribute("x",10); yLbl.setAttribute("y",PAD.top+ch/2);
  yLbl.setAttribute("font-size","11"); yLbl.setAttribute("fill","#6b7280");
  yLbl.setAttribute("text-anchor","middle");
  yLbl.setAttribute("transform",`rotate(-90,10,${PAD.top+ch/2})`);
  yLbl.textContent = info.discrete ? "Probability" : "Density";
  svg.appendChild(yLbl);

  // x-axis ticks
  const nTicks=5;
  for(let i=0;i<=nTicks;i++) {
    const xv=xMin+i*(xMax-xMin)/nTicks;
    const tick=document.createElementNS("http://www.w3.org/2000/svg","line");
    tick.setAttribute("x1",sx(xv)); tick.setAttribute("x2",sx(xv));
    tick.setAttribute("y1",sy(0)); tick.setAttribute("y2",sy(0)+5);
    tick.setAttribute("stroke","#374151"); tick.setAttribute("stroke-width","1");
    svg.appendChild(tick);
    const lbl=document.createElementNS("http://www.w3.org/2000/svg","text");
    lbl.setAttribute("x",sx(xv)); lbl.setAttribute("y",sy(0)+18);
    lbl.setAttribute("text-anchor","middle"); lbl.setAttribute("font-size","10");
    lbl.setAttribute("fill","#6b7280");
    lbl.textContent = info.discrete ? Math.round(xv) : xv.toFixed(1);
    svg.appendChild(lbl);
  }

  // distribution label top-left
  const titleLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  titleLbl.setAttribute("x",PAD.left+4); titleLbl.setAttribute("y",PAD.top-6);
  titleLbl.setAttribute("font-size","11"); titleLbl.setAttribute("fill","#374151");
  titleLbl.setAttribute("font-weight","500");
  titleLbl.textContent=info.label;
  svg.appendChild(titleLbl);
}

function buildSliders() {
  slidersDiv.innerHTML="";
  const d=distNames[distIdx];
  const title=document.createElement("div");
  title.style.cssText="font-size:0.82em; font-weight:600; color:#374151; margin-bottom:0.4rem;";
  title.textContent="Parameters";
  slidersDiv.appendChild(title);
  if(d=="Bernoulli") {
    const {row}=makeSlider("p",null,0.01,0.99,0.01,()=>params.p,v=>params.p=v);
    slidersDiv.appendChild(row);
  }
  if(d=="Binomial") {
```

```
    const {row:r1}=makeSlider("n",null,1,50,1,()=>params.n,v=>params.n=v);
    const {row:r2}=makeSlider("p",null,0.01,0.99,0.01,()=>params.p,v=>params.p=v);
    slidersDiv.appendChild(r1); slidersDiv.appendChild(r2);
  }
  if(d=="Poisson") {
    const {row}=makeSlider("□",null,0.1,20,0.1,()=>params.lam,v=>params.lam=v);
    slidersDiv.appendChild(row);
  }
  if(d=="Exponential") {
    const {row}=makeSlider("□ (rate)",null,0.1,5,0.05,()=>params.lam,v=>params.lam=v);
    slidersDiv.appendChild(row);
    const note=document.createElement("div");
    note.style.cssText="font-size:0.78em; color:#6b7280; margin-top:0.2rem;";
    note.textContent="Mean = 1/□ (rate □ mean)";
    slidersDiv.appendChild(note);
  }
  if(d=="Normal") {
    const {row:r1}=makeSlider("□",null,-5,5,0.1,()=>params.mu,v=>params.mu=v);
    const {row:r2}=makeSlider("□",null,0.1,5,0.1,()=>params.sigma,v=>params.sigma=v);
    slidersDiv.appendChild(r1); slidersDiv.appendChild(r2);
  }
}

function updateReadouts(info) {
  const isSymmetric = ["Bernoulli","Normal"].includes(distNames[distIdx]) ||
    (distNames[distIdx]=="Binomial" && Math.abs(params.p-0.5)<0.01);
  const medianNote = isSymmetric ? "Symmetric: median = mean" : "Skewed: median □ mean";
  readouts.innerHTML = `
    <div style="font-weight:600; color:#374151; margin-bottom:0.5rem; font-size:0.9em;">Statist
    <div style="margin-bottom:0.35rem;"><span style="color:#6b7280;">Mean:</span> <strong>${inf
    <div style="margin-bottom:0.35rem;"><span style="color:#6b7280;">Variance:</span> <strong>$
    <div style="margin-bottom:0.35rem;"><span style="color:#6b7280;">Std dev (□):</span> <stron
    <div style="margin-top:0.6rem; font-size:0.82em; color:#6b7280; font-style:italic;">${media
  `;
}

function update() {
  // highlight active button
  btns.forEach((b,i)=>{
    b.style.background = i===distIdx ? ACCENT : "#f9fafb";
    b.style.color = i===distIdx ? "#fff" : "#374151";
    b.style.borderColor = i===distIdx ? ACCENT : "#d1d5db";
  });
  const info=getDistInfo();
  draw(info);
  buildSliders();
  updateReadouts(info);
```

```
  }

  update();
  return container;
}
```

## 56.4. Key continuous distributions

### 56.4.1. Uniform

$X \sim \text{Uniform}(a, b)$ — every value in $[a, b]$ is equally likely.

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

$$E[X] = \frac{a+b}{2}, \quad \text{Var}(X) = \frac{(b-a)^2}{12}$$

The CDF is: $F(x) = (x-a)/(b-a)$ for $a \leq x \leq b$.

### 56.4.2. Exponential

Models the time until the first event in a Poisson process — waiting times, component lifetimes, time between arrivals.

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

$$F(x) = 1 - e^{-\lambda x}$$

**Derivation of mean.** Integrate by parts:

$$E[X] = \int_0^\infty x \cdot \lambda e^{-\lambda x} \, dx$$

Let $u = x$, $dv = \lambda e^{-\lambda x} \, dx$. Then $du = dx$, $v = -e^{-\lambda x}$.

$$E[X] = \left[-xe^{-\lambda x}\right]_0^\infty + \int_0^\infty e^{-\lambda x} \, dx = 0 + \left[-\frac{1}{\lambda} e^{-\lambda x}\right]_0^\infty = \frac{1}{\lambda}$$

**Derivation of variance.** First compute $E[X^2]$:

$$E[X^2] = \int_0^\infty x^2 \cdot \lambda e^{-\lambda x} \, dx = \frac{2}{\lambda^2}$$

One quick route is integration by parts twice. Let $u = x^2$ and $dv = \lambda e^{-\lambda x}\,dx$, so $du = 2x\,dx$ and $v = -e^{-\lambda x}$:

$$E[X^2] = \left[-x^2 e^{-\lambda x}\right]_0^\infty + 2\int_0^\infty x e^{-\lambda x}\,dx$$

The boundary term is 0, and the remaining integral is the same type used in the mean calculation. Evaluating it gives $2/\lambda^2$.

$$\mathrm{Var}(X) = E[X^2] - (E[X])^2 = \frac{2}{\lambda^2} - \frac{1}{\lambda^2} = \frac{1}{\lambda^2}$$

**Memoryless property.** The exponential is the only continuous distribution with no memory:

$$P(X > s + t \mid X > s) = P(X > t) \quad \text{for all } s, t \geq 0$$

If the component has already survived $s$ hours, the probability it survives another $t$ hours is the same as if it were brand new. The past waiting time gives no information about the future.

**Proof.** Using the survival function $P(X > x) = e^{-\lambda x}$:

$$P(X > s + t \mid X > s) = \frac{P(X > s + t)}{P(X > s)} = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} = e^{-\lambda t} = P(X > t)$$

### 56.4.3. Normal

$X \sim N(\mu, \sigma^2)$ — the most important distribution in applied probability, for reasons the CLT will make clear.

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Parameters: $\mu$ is the mean, $\sigma^2$ is the variance.

$$E[X] = \mu, \quad \mathrm{Var}(X) = \sigma^2$$

There is no closed form for the CDF — it's evaluated numerically and tabulated as the **standard normal CDF** $\Phi(z)$, where $\Phi$ (capital phi) is the CDF of $Z \sim N(0, 1)$.

**Standardisation.** Any normal $X \sim N(\mu, \sigma^2)$ can be converted to the standard normal by:

$$Z = \frac{X - \mu}{\sigma}$$

Then $P(X \leq x) = P\left(Z \leq \frac{x-\mu}{\sigma}\right) = \Phi\left(\frac{x-\mu}{\sigma}\right)$.

The transformation $Z = (X - \mu)/\sigma$ — "subtract the mean, divide by the standard deviation" — centres the distribution at zero and scales it to unit variance. Every probability question about $X \sim N(\mu, \sigma^2)$ reduces to looking up a value in the standard normal table.

```
//| echo: false
{
  // ── Normal distribution — sigma bands (Priority 1) ──────────────────────
  const W = 560, H = 300, PAD = { top: 30, right: 30, bottom: 50, left: 50 };
  const ACCENT = "#3b82f6";
  const BANDS = [
    { mult: 3, fill: "rgba(59,130,246,0.10)", pct: "99.7%", yFrac: 0.18 },
    { mult: 2, fill: "rgba(59,130,246,0.20)", pct: "95.4%", yFrac: 0.40 },
    { mult: 1, fill: "rgba(59,130,246,0.38)", pct: "68.3%", yFrac: 0.68 },
  ];
  const GREY = "#9ca3af";

  function erf(x) {
    const sign=x<0?-1:1; x=Math.abs(x);
    const a1=0.254829592,a2=-0.284496736,a3=1.421413741,a4=-1.453152027,a5=1.061405429,p=0.3275911;
    const t=1/(1+p*x);
    return sign*(1-(((((a5*t+a4)*t)+a3)*t+a2)*t+a1)*t*Math.exp(-x*x));
  }
  function normalPDF(x,mu,sig) { return Math.exp(-0.5*((x-mu)/sig)**2)/(sig*Math.sqrt(2*Math.PI)); }
  function normalCDF(x,mu,sig) { return 0.5*(1+erf((x-mu)/(sig*Math.SQRT2))); }

  let mu=0, sigma=1, showCDF=false, cursorX=0;

  const container=document.createElement("div");
  container.style.cssText="font-family:inherit; max-width:600px; margin:1rem 0;";

  // sliders row
  const sliderRow=document.createElement("div");
  sliderRow.style.cssText="display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.75rem; align-iter

  function makeSlider2(label,min,max,step,def,cb) {
    const wrap=document.createElement("div");
    wrap.style.cssText="display:flex; align-items:center; gap:0.4rem;";
    const lbl=document.createElement("label");
    lbl.style.cssText="font-size:0.85em; min-width:20px;";
    lbl.textContent=label;
    const inp=document.createElement("input");
    inp.type="range"; inp.min=min; inp.max=max; inp.step=step; inp.value=def;
    inp.style.cssText="width:120px;";
    const val=document.createElement("span");
    val.style.cssText="font-size:0.85em; min-width:30px;";
    val.textContent=Number(def).toFixed(step<0.1?1:step<1?1:0);
    inp.oninput=()=>{ val.textContent=Number(inp.value).toFixed(step<0.1?1:step<1?1:0); cb(parseFloa
    wrap.appendChild(lbl); wrap.appendChild(inp); wrap.appendChild(val);
```

```
  return wrap;
}
sliderRow.appendChild(makeSlider2("□",-5,5,0.1,0,v=>{mu=v;draw();}));
sliderRow.appendChild(makeSlider2("□",0.2,4,0.1,1,v=>{sigma=v;draw();}));

// CDF toggle
const cdfBtn=document.createElement("button");
cdfBtn.textContent="Show CDF";
cdfBtn.style.cssText="padding:0.3rem 0.7rem; border-radius:4px; border:1px solid #d1d5db; curso
cdfBtn.onclick=()=>{showCDF=!showCDF; cdfBtn.textContent=showCDF?"Hide CDF":"Show CDF"; cdfBtn.
sliderRow.appendChild(cdfBtn);
container.appendChild(sliderRow);

const svgEl=document.createElementNS("http://www.w3.org/2000/svg","svg");
svgEl.setAttribute("width",W); svgEl.setAttribute("height",H);
svgEl.style.cssText="border:1px solid #e5e7eb; border-radius:6px; background:#fff; display:bloc
container.appendChild(svgEl);

// cursor live readout
const cursorReadout=document.createElement("div");
cursorReadout.style.cssText="font-size:0.82em; color:#6b7280; margin-top:0.35rem; min-height:1.
container.appendChild(cursorReadout);

const labelDiv=document.createElement("div");
labelDiv.style.cssText="margin-top:0.3rem; font-size:0.82em; color:#6b7280; font-style:italic;"
labelDiv.textContent="□ sets the centre. □ sets the width. The shaded areas — 68%, 95%, 99.7% —
container.appendChild(labelDiv);

// draggable cursor state
let dragging=false;
let svgRect=null;

function xToData(px) {
  const cw=W-PAD.left-PAD.right;
  const xmin=mu-4.5*sigma, xmax=mu+4.5*sigma;
  return xmin+(px-PAD.left)/cw*(xmax-xmin);
}

svgEl.addEventListener("mousedown",e=>{
  if(!showCDF) return;
  dragging=true; svgRect=svgEl.getBoundingClientRect();
  cursorX=xToData(e.clientX-svgRect.left); draw();
});
window.addEventListener("mousemove",e=>{
  if(!dragging||!showCDF) return;
  cursorX=xToData(e.clientX-svgRect.left); draw();
});
```

```
window.addEventListener("mouseup",()=>{ dragging=false; });

function draw() {
  while(svgEl.firstChild) svgEl.removeChild(svgEl.firstChild);
  const cw=W-PAD.left-PAD.right;
  const ch=H-PAD.top-PAD.bottom;
  const xmin=mu-4.5*sigma, xmax=mu+4.5*sigma;
  const N=250;
  const xs=Array.from({length:N},(_,i)=>xmin+i*(xmax-xmin)/(N-1));
  const ys=xs.map(x=>normalPDF(x,mu,sigma));
  const yMax=normalPDF(mu,mu,sigma)*1.18;
  const sx=x=>PAD.left+(x-xmin)/(xmax-xmin)*cw;
  const sy=y=>PAD.top+ch-y/yMax*ch;

  // second axis for CDF
  const syCDF = y => PAD.top+ch - y*ch;

  // draw shaded sigma bands
  BANDS.forEach(({mult,fill,pct,yFrac})=>{
    const xl=mu-mult*sigma, xr=mu+mult*sigma;
    const bandXs=xs.filter(x=>x>=xl&&x<=xr);
    const pts=[
      `${sx(xl)},${sy(0)}`,
      ...bandXs.map(x=>`${sx(x)},${sy(normalPDF(x,mu,sigma))}`),
      `${sx(xr)},${sy(0)}`
    ].join(" ");
    const poly=document.createElementNS("http://www.w3.org/2000/svg","polygon");
    poly.setAttribute("points",pts); poly.setAttribute("fill",fill);
    svgEl.appendChild(poly);

    // band percentage label
    const lbl=document.createElementNS("http://www.w3.org/2000/svg","text");
    lbl.setAttribute("x",sx(mu)); lbl.setAttribute("y",PAD.top+ch*(1-yFrac));
    lbl.setAttribute("text-anchor","middle"); lbl.setAttribute("font-size","11");
    lbl.setAttribute("fill","#1e40af"); lbl.setAttribute("font-weight","500");
    lbl.textContent=pct;
    svgEl.appendChild(lbl);
  });

  // PDF curve
  const pdfPts=xs.map((x,i)=>`${sx(x)},${sy(ys[i])}`).join(" ");
  const pdfLine=document.createElementNS("http://www.w3.org/2000/svg","polyline");
  pdfLine.setAttribute("points",pdfPts); pdfLine.setAttribute("fill","none");
  pdfLine.setAttribute("stroke",ACCENT); pdfLine.setAttribute("stroke-width","2.5");
  svgEl.appendChild(pdfLine);

  // ▯ bracket below x-axis
```

```
const bY=PAD.top+ch+20;
const bLine=document.createElementNS("http://www.w3.org/2000/svg","line");
bLine.setAttribute("x1",sx(mu-sigma)); bLine.setAttribute("x2",sx(mu+sigma));
bLine.setAttribute("y1",bY); bLine.setAttribute("y2",bY);
bLine.setAttribute("stroke",ACCENT); bLine.setAttribute("stroke-width","2");
svgEl.appendChild(bLine);
// ticks at bracket ends
[-1,1].forEach(s=>{
  const t=document.createElementNS("http://www.w3.org/2000/svg","line");
  t.setAttribute("x1",sx(mu+s*sigma)); t.setAttribute("x2",sx(mu+s*sigma));
  t.setAttribute("y1",bY-4); t.setAttribute("y2",bY+4);
  t.setAttribute("stroke",ACCENT); t.setAttribute("stroke-width","2");
  svgEl.appendChild(t);
});
const bLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
bLbl.setAttribute("x",sx(mu)); bLbl.setAttribute("y",bY+13);
bLbl.setAttribute("text-anchor","middle"); bLbl.setAttribute("font-size","10");
bLbl.setAttribute("fill",ACCENT);
bLbl.textContent=`σ = ${sigma.toFixed(1)}`;
svgEl.appendChild(bLbl);

// CDF curve
if(showCDF) {
  const cdfPts=xs.map(x=>`${sx(x)},${syCDF(normalCDF(x,mu,sigma))}`).join(" ");
  const cdfLine=document.createElementNS("http://www.w3.org/2000/svg","polyline");
  cdfLine.setAttribute("points",cdfPts); cdfLine.setAttribute("fill","none");
  cdfLine.setAttribute("stroke","#f59e0b"); cdfLine.setAttribute("stroke-width","2");
  cdfLine.setAttribute("stroke-dasharray","4,2");
  svgEl.appendChild(cdfLine);
  // right y-axis
  const ryLine=document.createElementNS("http://www.w3.org/2000/svg","line");
  ryLine.setAttribute("x1",PAD.left+cw); ryLine.setAttribute("x2",PAD.left+cw);
  ryLine.setAttribute("y1",PAD.top); ryLine.setAttribute("y2",PAD.top+ch);
  ryLine.setAttribute("stroke","#f59e0b"); ryLine.setAttribute("stroke-width","1");
  svgEl.appendChild(ryLine);
  const ryLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  ryLbl.setAttribute("x",PAD.left+cw+10); ryLbl.setAttribute("y",PAD.top+ch/2);
  ryLbl.setAttribute("font-size","10"); ryLbl.setAttribute("fill","#f59e0b");
  ryLbl.setAttribute("text-anchor","middle");
  ryLbl.setAttribute("transform",`rotate(90,${PAD.left+cw+10},${PAD.top+ch/2})`);
  ryLbl.textContent="Φ(x)";
  svgEl.appendChild(ryLbl);
  // cursor
  const cxp=Math.max(xmin,Math.min(xmax,cursorX));
  const cxv=sx(cxp);
  const cursorLine=document.createElementNS("http://www.w3.org/2000/svg","line");
  cursorLine.setAttribute("x1",cxv); cursorLine.setAttribute("x2",cxv);
```

```
    cursorLine.setAttribute("y1",PAD.top); cursorLine.setAttribute("y2",PAD.top+ch);
    cursorLine.setAttribute("stroke","#6b7280"); cursorLine.setAttribute("stroke-width","1.5");
    cursorLine.setAttribute("stroke-dasharray","3,2");
    svgEl.appendChild(cursorLine);
    const pval=normalCDF(cxp,mu,sigma);
    const dot=document.createElementNS("http://www.w3.org/2000/svg","circle");
    dot.setAttribute("cx",cxv); dot.setAttribute("cy",syCDF(pval)); dot.setAttribute("r","5");
    dot.setAttribute("fill","#f59e0b"); dot.setAttribute("stroke","#fff"); dot.setAttribute("strok
    svgEl.appendChild(dot);
    cursorReadout.textContent=`x = ${cxp.toFixed(2)}  →  P(X ≤ x) = Φ(${((cxp-mu)/sigma).toFixed(2
    svgEl.style.cursor="col-resize";
  } else {
    cursorReadout.textContent="Toggle 'Show CDF' and drag to read off P(X ≤ x).";
    svgEl.style.cursor="default";
  }

  // mean dashed line
  const muLine=document.createElementNS("http://www.w3.org/2000/svg","line");
  muLine.setAttribute("x1",sx(mu)); muLine.setAttribute("x2",sx(mu));
  muLine.setAttribute("y1",PAD.top); muLine.setAttribute("y2",PAD.top+ch);
  muLine.setAttribute("stroke","#374151"); muLine.setAttribute("stroke-width","1.5");
  muLine.setAttribute("stroke-dasharray","5,3");
  svgEl.appendChild(muLine);
  const muLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  muLbl.setAttribute("x",sx(mu)+4); muLbl.setAttribute("y",PAD.top+13);
  muLbl.setAttribute("font-size","11"); muLbl.setAttribute("fill","#374151");
  muLbl.textContent=`μ=${mu.toFixed(1)}`;
  svgEl.appendChild(muLbl);

  // x-axis
  const xAxisEl=document.createElementNS("http://www.w3.org/2000/svg","line");
  xAxisEl.setAttribute("x1",PAD.left); xAxisEl.setAttribute("x2",PAD.left+cw);
  xAxisEl.setAttribute("y1",sy(0)); xAxisEl.setAttribute("y2",sy(0));
  xAxisEl.setAttribute("stroke","#374151"); xAxisEl.setAttribute("stroke-width","1");
  svgEl.appendChild(xAxisEl);

  // x-axis ticks
  const ticks=[-4,-3,-2,-1,0,1,2,3,4].map(z=>mu+z*sigma).filter(v=>v>=xmin&&v<=xmax);
  ticks.forEach(v=>{
    const tick=document.createElementNS("http://www.w3.org/2000/svg","line");
    tick.setAttribute("x1",sx(v)); tick.setAttribute("x2",sx(v));
    tick.setAttribute("y1",sy(0)); tick.setAttribute("y2",sy(0)+5);
    tick.setAttribute("stroke","#374151"); tick.setAttribute("stroke-width","1");
    svgEl.appendChild(tick);
    const lbl=document.createElementNS("http://www.w3.org/2000/svg","text");
    lbl.setAttribute("x",sx(v)); lbl.setAttribute("y",sy(0)+16);
    lbl.setAttribute("text-anchor","middle"); lbl.setAttribute("font-size","10");
```

```
      lbl.setAttribute("fill","#6b7280");
      lbl.textContent=v.toFixed(1);
      svgEl.appendChild(lbl);
    });

    // y-axis
    const yAxisEl=document.createElementNS("http://www.w3.org/2000/svg","line");
    yAxisEl.setAttribute("x1",PAD.left); yAxisEl.setAttribute("x2",PAD.left);
    yAxisEl.setAttribute("y1",PAD.top); yAxisEl.setAttribute("y2",sy(0));
    yAxisEl.setAttribute("stroke","#374151"); yAxisEl.setAttribute("stroke-width","1");
    svgEl.appendChild(yAxisEl);
    const yLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
    yLbl.setAttribute("x",12); yLbl.setAttribute("y",PAD.top+H/3);
    yLbl.setAttribute("text-anchor","middle"); yLbl.setAttribute("font-size","11");
    yLbl.setAttribute("fill","#6b7280");
    yLbl.setAttribute("transform",`rotate(-90,12,${PAD.top+H/3})`);
    yLbl.textContent="Density";
    svgEl.appendChild(yLbl);
  }

  draw();
  return container;
}
```

## 56.5. Joint distributions

For two random variables $X$ and $Y$, the **joint distribution** describes their behaviour together. The key concept for most applications is independence.

$X$ and $Y$ are **independent** if knowing the value of one gives no information about the other:

$$f_{X,Y}(x, y) = f_X(x) \cdot f_Y(y)$$

That is, the joint density factors into the product of the marginals.

**Covariance.** A measure of how $X$ and $Y$ move together:

$$\mathrm{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] = E[XY] - E[X]\,E[Y]$$

If $X$ and $Y$ are independent, $E[XY] = E[X]\,E[Y]$, so $\mathrm{Cov}(X, Y) = 0$. (The converse is not generally true: zero covariance does not imply independence.)

**Correlation coefficient.** Covariance is scale-dependent — multiplying $X$ by 2 doubles the covariance. The correlation normalises this:

$$\rho(X,Y) = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}, \quad -1 \le \rho \le 1$$

$\rho = 1$ means perfect positive linear relationship; $\rho = -1$ means perfect negative; $\rho = 0$ means no linear relationship.

```
//| echo: false
{
  // ── Correlation scatter cloud (OPPORTUNITY 4) ──────────────────────────
  const W=420, H=380, PAD={top:30,right:30,bottom:50,left:50};
  const ACCENT="#3b82f6";
  const N_PTS=200;

  let rho=0;
  let seed=99;
  function lcg() { seed=(seed*1664525+1013904223)&0xffffffff; return (seed>>>0)/0xffffffff; }
  function boxMuller() {
    const u1=lcg()||1e-10, u2=lcg();
    return [Math.sqrt(-2*Math.log(u1))*Math.cos(2*Math.PI*u2),
            Math.sqrt(-2*Math.log(u1))*Math.sin(2*Math.PI*u2)];
  }
  function genPoints(r) {
    seed=99;
    const pts=[];
    for(let i=0;i<N_PTS;i++) {
      const [z1,z2]=boxMuller();
      // Cholesky: x=z1, y=r*z1+sqrt(1-r^2)*z2
      pts.push([z1, r*z1+Math.sqrt(Math.max(0,1-r*r))*z2]);
    }
    return pts;
  }

  const container=document.createElement("div");
  container.style.cssText="font-family:inherit; max-width:460px; margin:1rem 0;";

  const sliderRow=document.createElement("div");
  sliderRow.style.cssText="display:flex; align-items:center; gap:0.6rem; margin-bottom:0.75rem;";
  const slLbl=document.createElement("label");
  slLbl.style.cssText="font-size:0.85em;"; slLbl.textContent="ρ:";
  const slInp=document.createElement("input");
  slInp.type="range"; slInp.min=-1; slInp.max=1; slInp.step=0.01; slInp.value=0;
  slInp.style.cssText="width:200px;";
  const slVal=document.createElement("span");
  slVal.style.cssText="font-size:0.9em; font-weight:600; min-width:40px;";
  slVal.textContent="0.00";
  slInp.oninput=()=>{ rho=parseFloat(slInp.value); slVal.textContent=rho.toFixed(2); draw(); };
  sliderRow.appendChild(slLbl); sliderRow.appendChild(slInp); sliderRow.appendChild(slVal);
```

```
container.appendChild(sliderRow);

const svgEl=document.createElementNS("http://www.w3.org/2000/svg","svg");
svgEl.setAttribute("width",W); svgEl.setAttribute("height",H);
svgEl.style.cssText="border:1px solid #e5e7eb; border-radius:6px; background:#fff; display:bloc
container.appendChild(svgEl);

const readoutDiv=document.createElement("div");
readoutDiv.style.cssText="font-size:0.82em; color:#374151; margin-top:0.4rem;";
container.appendChild(readoutDiv);

const labelDiv=document.createElement("div");
labelDiv.style.cssText="margin-top:0.3rem; font-size:0.82em; color:#6b7280; font-style:italic;"
labelDiv.textContent="? is the shape of the data cloud: circular, elliptical, or linear.";
container.appendChild(labelDiv);

function draw() {
  while(svgEl.firstChild) svgEl.removeChild(svgEl.firstChild);
  const cw=W-PAD.left-PAD.right;
  const ch=H-PAD.top-PAD.bottom;
  const r=Math.max(-0.999,Math.min(0.999,rho));
  const pts=genPoints(r);
  const range=3.5;
  const sx=x=>PAD.left+(x+range)/(2*range)*cw;
  const sy=y=>PAD.top+(range-y)/(2*range)*ch;

  // grid
  for(let v=-3;v<=3;v++) {
    const gx=document.createElementNS("http://www.w3.org/2000/svg","line");
    gx.setAttribute("x1",sx(v)); gx.setAttribute("x2",sx(v));
    gx.setAttribute("y1",PAD.top); gx.setAttribute("y2",PAD.top+ch);
    gx.setAttribute("stroke","#f3f4f6"); gx.setAttribute("stroke-width","1");
    svgEl.appendChild(gx);
    const gy=document.createElementNS("http://www.w3.org/2000/svg","line");
    gy.setAttribute("x1",PAD.left); gy.setAttribute("x2",PAD.left+cw);
    gy.setAttribute("y1",sy(v)); gy.setAttribute("y2",sy(v));
    gy.setAttribute("stroke","#f3f4f6"); gy.setAttribute("stroke-width","1");
    svgEl.appendChild(gy);
  }

  // axes
  ["x","y"].forEach(axis=>{
    const l=document.createElementNS("http://www.w3.org/2000/svg","line");
    if(axis==="x") { l.setAttribute("x1",PAD.left); l.setAttribute("x2",PAD.left+cw); l.setAttr
    else { l.setAttribute("x1",sx(0)); l.setAttribute("x2",sx(0)); l.setAttribute("y1",PAD.top)
    l.setAttribute("stroke","#9ca3af"); l.setAttribute("stroke-width","1");
    svgEl.appendChild(l);
```

```
  });

  // scatter points
  pts.forEach(([x,y])=>{
    const dot=document.createElementNS("http://www.w3.org/2000/svg","circle");
    dot.setAttribute("cx",sx(x)); dot.setAttribute("cy",sy(y)); dot.setAttribute("r","3");
    dot.setAttribute("fill",ACCENT); dot.setAttribute("fill-opacity","0.5");
    svgEl.appendChild(dot);
  });

  // 1-sigma ellipse of bivariate normal
  const ellN=80;
  const ellPts=Array.from({length:ellN+1},(_,i)=>{
    const t=2*Math.PI*i/ellN;
    const ex=Math.cos(t);
    const ey=r*Math.cos(t)+Math.sqrt(Math.max(0,1-r*r))*Math.sin(t);
    return `${sx(ex)},${sy(ey)}`;
  }).join(" ");
  const ellipse=document.createElementNS("http://www.w3.org/2000/svg","polyline");
  ellipse.setAttribute("points",ellPts); ellipse.setAttribute("fill","none");
  ellipse.setAttribute("stroke","#374151"); ellipse.setAttribute("stroke-width","2");
  ellipse.setAttribute("stroke-dasharray","4,2");
  svgEl.appendChild(ellipse);

  // axis labels
  const xLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  xLbl.setAttribute("x",PAD.left+cw/2); xLbl.setAttribute("y",PAD.top+ch+38);
  xLbl.setAttribute("text-anchor","middle"); xLbl.setAttribute("font-size","12");
  xLbl.setAttribute("fill","#374151"); xLbl.textContent="X";
  svgEl.appendChild(xLbl);
  const yLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  yLbl.setAttribute("x",14); yLbl.setAttribute("y",PAD.top+ch/2);
  yLbl.setAttribute("text-anchor","middle"); yLbl.setAttribute("font-size","12");
  yLbl.setAttribute("fill","#374151");
  yLbl.setAttribute("transform",`rotate(-90,14,${PAD.top+ch/2})`);
  yLbl.textContent="Y";
  svgEl.appendChild(yLbl);

  // rho annotation
  const rAnn=document.createElementNS("http://www.w3.org/2000/svg","text");
  rAnn.setAttribute("x",PAD.left+8); rAnn.setAttribute("y",PAD.top+16);
  rAnn.setAttribute("font-size","12"); rAnn.setAttribute("fill","#374151");
  rAnn.setAttribute("font-weight","500");
  rAnn.textContent=`ρ = ${r.toFixed(2)}`;
  svgEl.appendChild(rAnn);

  // interpretive note
```

```
    const absR=Math.abs(r);
    let strength = absR < 0.3 ? "Weak" : absR < 0.7 ? "Moderate" : "Strong";
    let direction = r>0.02 ? "Positive: Y tends to increase with X" : r<-0.02 ? "Negative: Y tend
    if(absR<0.05) direction="Knowing X tells you nothing about Y";
    if(absR>0.97) direction=r>0?"Perfect positive linear relationship":"Perfect negative linear r
    const angleRad=Math.atan(r); const angleDeg=(angleRad*180/Math.PI).toFixed(0);
    readoutDiv.textContent=`ρ = ${r.toFixed(2)} | ${strength} relationship | ${direction}`;
  }

  draw();
  return container;
}
```

**Variance of a sum.** For any $X$ and $Y$ (not necessarily independent):

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X,Y)$$

If they are independent, the covariance term vanishes.

## 56.6. Central Limit Theorem

Let $X_1, X_2, \ldots, X_n$ be independent, identically distributed random variables with mean $\mu$ and variance $\sigma^2 < \infty$. Define the sample mean:

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$$

**Central Limit Theorem (CLT):** As $n \to \infty$,

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0,1)$$

Read $\xrightarrow{d}$ as "converges in distribution to" — as $n$ grows, the distribution of the standardised mean gets closer and closer to the standard normal.

In practice: for large $n$, the distribution of $\bar{X}_n$ is approximately $N(\mu, \sigma^2/n)$.

**Three things the CLT is saying:**

1. The sample mean converges to the true mean $\mu$ — not just as a hope, but with a rate: the spread is $\sigma/\sqrt{n}$, shrinking like $1/\sqrt{n}$.
2. The limiting distribution is always normal, regardless of the distribution of the individual $X_i$. You do not need $X_i$ to be normal — exponential, uniform, Bernoulli, anything — the average becomes normal.

3. The only requirements are: independent, identically distributed (abbreviated i.i.d.), finite variance. No further conditions.

**Conditions for practical use.** The approximation is usually adequate when $n \geq 30$, and excellent for $n \geq 50$. For distributions that are already close to normal, smaller $n$ suffices. For very skewed distributions (e.g. heavy-tailed), larger $n$ may be needed.

**Worked example: Sample mean from an exponential population.**

Components have lifetimes $X_i \sim \text{Exp}(\lambda = 0.5)$, so $\mu = 1/\lambda = 2$ hours and $\sigma^2 = 1/\lambda^2 = 4$.

Take a sample of $n = 50$ components. By the CLT, the sample mean is approximately:

$$\bar{X}_{50} \approx N\left(2,\ \frac{4}{50}\right) = N(2,\ 0.08)$$

Standard deviation of the mean: $\sigma/\sqrt{n} = 2/\sqrt{50} \approx 0.283$.

Find $P(\bar{X}_{50} > 2.4)$:

$$P(\bar{X}_{50} > 2.4) = P\left(Z > \frac{2.4 - 2}{0.283}\right) = P(Z > 1.414) = 1 - \Phi(1.414) \approx 0.079$$

There is about an 8% chance the sample mean exceeds 2.4 hours. The individual lifetimes are exponential and highly right-skewed — but the average over 50 of them behaves almost exactly like a normal random variable.

```
//| echo: false
{
  // ── CLT demonstration (VIZ 2, extended) ──────────────────────────────
  const W=540, H=300, PAD={top:30,right:20,bottom:50,left:48};
  const ACCENT="#3b82f6", GREY="#d1d5db";
  const LAMBDA=0.5, MU=1/LAMBDA, SIGMA2=1/(LAMBDA*LAMBDA), SIGMA=1/LAMBDA;
  const N_SAMPLES=2000;

  let nSlider=5;

  // LCG-based deterministic "random" for reproducibility
  let seed=42;
  function lcgNext() { seed=(seed*1664525+1013904223)&0xffffffff; return (seed>>>0)/0xffffffff; }
  function resetSeed(s=42) { seed=s; }
  function expSample() { return -Math.log(1-lcgNext())/LAMBDA; }
  function normalPDF(x,mu,sig) { return Math.exp(-0.5*((x-mu)/sig)**2)/(sig*Math.sqrt(2*Math.PI)); }
  function erf(x) {
    const sign=x<0?-1:1; x=Math.abs(x);
    const a1=0.254829592,a2=-0.284496736,a3=1.421413741,a4=-1.453152027,a5=1.061405429,p=0.3275911;
    const t=1/(1+p*x);
    return sign*(1-(((((a5*t+a4)*t)+a3)*t+a2)*t+a1)*t*Math.exp(-x*x));
  }
```

```
const container=document.createElement("div");
container.style.cssText="font-family:inherit; max-width:580px; margin:1rem 0;";

// slider row
const sliderRow=document.createElement("div");
sliderRow.style.cssText="display:flex; align-items:center; gap:0.6rem; margin-bottom:0.75rem; f
const slLbl=document.createElement("label");
slLbl.style.cssText="font-size:0.85em;";
slLbl.textContent="Sample size n:";
const slInp=document.createElement("input");
slInp.type="range"; slInp.min=1; slInp.max=200; slInp.step=1; slInp.value=nSlider;
slInp.style.cssText="width:200px;";
const slVal=document.createElement("span");
slVal.style.cssText="font-size:0.88em; font-weight:600; min-width:30px;";
slVal.textContent=nSlider;
slInp.oninput=()=>{ nSlider=parseInt(slInp.value); slVal.textContent=nSlider; draw(); };
// snap buttons
const snaps=[1,5,30,100];
const snapBtns=snaps.map(v=>{
  const b=document.createElement("button");
  b.textContent=`n=${v}`;
  b.style.cssText="padding:0.2rem 0.5rem; border:1px solid #d1d5db; border-radius:3px; cursor:p
  b.onclick=()=>{ nSlider=v; slInp.value=v; slVal.textContent=v; draw(); };
  return b;
});
sliderRow.appendChild(slLbl); sliderRow.appendChild(slInp); sliderRow.appendChild(slVal);
snaps.forEach((v,i)=>sliderRow.appendChild(snapBtns[i]));
container.appendChild(sliderRow);

const mainSvg=document.createElementNS("http://www.w3.org/2000/svg","svg");
mainSvg.setAttribute("width",W); mainSvg.setAttribute("height",H);
mainSvg.style.cssText="border:1px solid #e5e7eb; border-radius:6px; background:#fff; display:bl
container.appendChild(mainSvg);

const infoDiv=document.createElement("div");
infoDiv.style.cssText="font-size:0.82em; color:#374151; margin-top:0.4rem;";
container.appendChild(infoDiv);

const captionDiv=document.createElement("div");
captionDiv.style.cssText="margin-top:0.3rem; font-size:0.82em; color:#6b7280; font-style:italic
captionDiv.textContent="The source distribution (upper right) is strongly right-skewed. Watch w
container.appendChild(captionDiv);

function getSampleMeans(n) {
  resetSeed(123);
  const means=[];
  for(let s=0;s<N_SAMPLES;s++) {
```

```
    let sum=0;
    for(let i=0;i<n;i++) sum+=expSample();
    means.push(sum/n);
  }
  return means;
}

function draw() {
  while(mainSvg.firstChild) mainSvg.removeChild(mainSvg.firstChild);
  const n=nSlider;
  const cw=W-PAD.left-PAD.right;
  const ch=H-PAD.top-PAD.bottom;

  const means=getSampleMeans(n);
  const theoSE=SIGMA/Math.sqrt(n);

  // histogram
  const xMin=Math.max(0, MU-5*theoSE);
  const xMax=MU+5*theoSE;
  const nBins=Math.max(15,Math.min(50,Math.round(Math.sqrt(N_SAMPLES))));
  const binWidth=(xMax-xMin)/nBins;
  const bins=Array(nBins).fill(0);
  means.forEach(v=>{
    const b=Math.floor((v-xMin)/binWidth);
    if(b>=0&&b<nBins) bins[b]++;
  });
  const maxCount=Math.max(...bins);
  // convert to density
  const densities=bins.map(c=>c/(N_SAMPLES*binWidth));
  const yMax=Math.max(...densities)*1.25;

  const sx=x=>PAD.left+(x-xMin)/(xMax-xMin)*cw;
  const sy=y=>PAD.top+ch-y/yMax*ch;

  // draw bars
  bins.forEach((c,i)=>{
    const x0=xMin+i*binWidth;
    const d=c/(N_SAMPLES*binWidth);
    const rect=document.createElementNS("http://www.w3.org/2000/svg","rect");
    rect.setAttribute("x",sx(x0)+0.5); rect.setAttribute("y",sy(d));
    rect.setAttribute("width",Math.max(1,sx(x0+binWidth)-sx(x0)-1));
    rect.setAttribute("height",Math.max(0,sy(0)-sy(d)));
    rect.setAttribute("fill",GREY); rect.setAttribute("stroke","#fff"); rect.setAttribute("stroke-
    mainSvg.appendChild(rect);
  });

  // theoretical normal overlay
```

```
const nPts=200;
const overlayXs=Array.from({length:nPts},(_,i)=>xMin+i*(xMax-xMin)/(nPts-1));
const overlayYs=overlayXs.map(x=>normalPDF(x,MU,theoSE));
const overlayPts=overlayXs.map((x,i)=>`${sx(x)},${sy(overlayYs[i])}`).join(" ");
const overlayLine=document.createElementNS("http://www.w3.org/2000/svg","polyline");
overlayLine.setAttribute("points",overlayPts); overlayLine.setAttribute("fill","none");
overlayLine.setAttribute("stroke",ACCENT); overlayLine.setAttribute("stroke-width","2.5");
mainSvg.appendChild(overlayLine);

// mean marker
const muLine=document.createElementNS("http://www.w3.org/2000/svg","line");
muLine.setAttribute("x1",sx(MU)); muLine.setAttribute("x2",sx(MU));
muLine.setAttribute("y1",PAD.top); muLine.setAttribute("y2",PAD.top+ch);
muLine.setAttribute("stroke","#374151"); muLine.setAttribute("stroke-width","1.5");
muLine.setAttribute("stroke-dasharray","5,3");
mainSvg.appendChild(muLine);

// axes
const xAx=document.createElementNS("http://www.w3.org/2000/svg","line");
xAx.setAttribute("x1",PAD.left); xAx.setAttribute("x2",PAD.left+cw);
xAx.setAttribute("y1",sy(0)); xAx.setAttribute("y2",sy(0));
xAx.setAttribute("stroke","#374151"); xAx.setAttribute("stroke-width","1");
mainSvg.appendChild(xAx);
const yAx=document.createElementNS("http://www.w3.org/2000/svg","line");
yAx.setAttribute("x1",PAD.left); yAx.setAttribute("x2",PAD.left);
yAx.setAttribute("y1",PAD.top); yAx.setAttribute("y2",sy(0));
yAx.setAttribute("stroke","#374151"); yAx.setAttribute("stroke-width","1");
mainSvg.appendChild(yAx);

// x ticks
const nTicks=5;
for(let i=0;i<=nTicks;i++) {
  const xv=xMin+i*(xMax-xMin)/nTicks;
  const tk=document.createElementNS("http://www.w3.org/2000/svg","line");
  tk.setAttribute("x1",sx(xv)); tk.setAttribute("x2",sx(xv));
  tk.setAttribute("y1",sy(0)); tk.setAttribute("y2",sy(0)+5);
  tk.setAttribute("stroke","#374151"); tk.setAttribute("stroke-width","1");
  mainSvg.appendChild(tk);
  const lbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  lbl.setAttribute("x",sx(xv)); lbl.setAttribute("y",sy(0)+18);
  lbl.setAttribute("text-anchor","middle"); lbl.setAttribute("font-size","10");
  lbl.setAttribute("fill","#6b7280");
  lbl.textContent=xv.toFixed(2);
  mainSvg.appendChild(lbl);
}

// x-axis label
```

```
  const xLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  xLbl.setAttribute("x",PAD.left+cw/2); xLbl.setAttribute("y",PAD.top+ch+38);
  xLbl.setAttribute("text-anchor","middle"); xLbl.setAttribute("font-size","11");
  xLbl.setAttribute("fill","#374151"); xLbl.textContent="Sample mean x̄ ";
  mainSvg.appendChild(xLbl);

  // y-axis label
  const yLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  yLbl.setAttribute("x",12); yLbl.setAttribute("y",PAD.top+ch/2);
  yLbl.setAttribute("text-anchor","middle"); yLbl.setAttribute("font-size","11");
  yLbl.setAttribute("fill","#6b7280");
  yLbl.setAttribute("transform",`rotate(-90,12,${PAD.top+ch/2})`);
  yLbl.textContent="Density";
  mainSvg.appendChild(yLbl);

  // inset: source Exp(0.5) distribution
  const iW=90, iH=60, iX=W-PAD.right-iW-5, iY=PAD.top+2;
  const iBg=document.createElementNS("http://www.w3.org/2000/svg","rect");
  iBg.setAttribute("x",iX-5); iBg.setAttribute("y",iY-5);
  iBg.setAttribute("width",iW+10); iBg.setAttribute("height",iH+10);
  iBg.setAttribute("fill","#f8fafc"); iBg.setAttribute("stroke","#e5e7eb");
  iBg.setAttribute("rx","3");
  mainSvg.appendChild(iBg);
  // inset curve
  const iXs=Array.from({length:80},(_,i)=>i*8/79);
  const iYs=iXs.map(x=>LAMBDA*Math.exp(-LAMBDA*x));
  const iYMax=LAMBDA*1.1;
  const isX=x=>iX+x/8*iW;
  const isY=y=>iY+iH-y/iYMax*iH;
  const iPts=iXs.map((x,i)=>`${isX(x)},${isY(iYs[i])}`).join(" ");
  const iArea=document.createElementNS("http://www.w3.org/2000/svg","polygon");
  iArea.setAttribute("points",`${isX(0)},${isY(0)} ${iPts} ${isX(8)},${isY(0)}`);
  iArea.setAttribute("fill","rgba(156,163,175,0.3)"); mainSvg.appendChild(iArea);
  const iLine=document.createElementNS("http://www.w3.org/2000/svg","polyline");
  iLine.setAttribute("points",iPts); iLine.setAttribute("fill","none");
  iLine.setAttribute("stroke","#9ca3af"); iLine.setAttribute("stroke-width","1.5");
  mainSvg.appendChild(iLine);
  const iLbl=document.createElementNS("http://www.w3.org/2000/svg","text");
  iLbl.setAttribute("x",iX+iW/2); iLbl.setAttribute("y",iY+iH+10);
  iLbl.setAttribute("text-anchor","middle"); iLbl.setAttribute("font-size","9");
  iLbl.setAttribute("fill","#6b7280"); iLbl.textContent="Source: Exp(0.5)";
  mainSvg.appendChild(iLbl);

  // info readout
  infoDiv.textContent=`n = ${n} | Theoretical mean: ${MU.toFixed(1)} | Theoretical SE: ${theoSE.to
}
```

```
  draw();
  return container;
}
```

## 56.7. Where this goes

This chapter provides the foundation for **Chapter 2: Mathematical statistics** (this volume). Everything in that chapter — maximum likelihood estimation, confidence intervals, hypothesis tests — takes the distributions developed here and uses them to make inferences about unknown parameters from data. The CLT is the engine: it justifies using normal-distribution machinery on sample means regardless of the underlying population distribution, which is why $z$-tests and $t$-tests work in practice.

The connections extend further into other sections of Vol 7. The Poisson distribution arises in **queuing theory** (a special case of stochastic processes). The normal distribution underlies error analysis in **numerical methods**: when you propagate measurement uncertainties through a computation, the CLT explains why the output errors are approximately normal. In **engineering statistics**, the same distributions appear in reliability theory, quality control (Six Sigma thresholds are expressed in $\sigma$ units), and signal detection.

**Where this shows up**

- A reliability engineer models component lifetimes as exponential and uses the memoryless property to compute replacement schedules.
- An actuary prices insurance using the Poisson distribution to model the number of claims per period.
- A machine learning engineer interprets model output probabilities using the Bayes framework — the model's output is $P(Y \mid X)$, not $P(X \mid Y)$.
- A signal processing engineer applies the CLT to justify that thermal noise in electronic circuits is modelled as Gaussian.
- A quality control engineer uses the binomial distribution to decide whether a batch rejection threshold is appropriate for a given defect rate.

## 56.8. Exercises

These are puzzles. Each has a clean numerical answer. The interesting part is identifying which distribution applies and setting up the probability correctly.

---

**Exercise 1.** A factory has two machines. Machine A produces 60% of output and has a 4% defect rate. Machine B produces the remaining 40% and has a 1% defect rate. An inspector picks a component at random and finds it defective. What is the probability it came from Machine A?

---

**Exercise 2.** A quality control test checks batches of 8 components. Each component, independently, has a failure probability of 0.1. Find the probability that at least 3 components in a batch fail.

---

**Exercise 3.** A call centre receives calls at an average rate of 4 per minute. In a 30-second window, what is the probability of receiving 3 or more calls?

---

**Exercise 4.** The time to failure of a device follows an exponential distribution with rate $\lambda = 0.02$ per hour. Find: (a) $P(T > 100)$, (b) $E[T]$, (c) $\mathrm{Var}(T)$.

---

**Exercise 5.** A manufacturing process produces items whose length $X \sim N(75, 100)$ mm (mean 75 mm, variance 100 mm²). The specification requires length greater than 85 mm. Find $P(X > 85)$.

---

**Exercise 6.** Let $X_1, X_2, \ldots, X_{50}$ be i.i.d. Exp(1) random variables. Their mean is $\mu = 1$ and variance is $\sigma^2 = 1$. Use the CLT to approximate $P(\bar{X}_{50} > 1.2)$.

# 57. Mathematical statistics

Inference from data

A production line fills bottles to a stated volume. A sample of 25 bottles shows a mean fill of 497 mL against a target of 500 mL. Is that a real problem, or just expected random variation? You need a principled way to decide.

A new catalyst cuts reaction time in a lab test. But the test used only eight runs. How confident are you that the effect is real and not noise?

You have data. The question is what the data can actually tell you — and, just as importantly, what it cannot.

## 57.1. Point estimation

You observe $n$ independent realisations $x_1, x_2, \dots, x_n$ from a distribution that has an unknown parameter $\theta$ — a proportion, a rate, a mean. A **point estimate** $\hat{\theta}$ — read "theta-hat" — is a single number computed from the sample to serve as your best guess at $\theta$.

The most systematic method for finding point estimates is **maximum likelihood estimation (MLE)**: choose the value of $\theta$ that makes the observed data most probable.

### 57.1.1. Setting up the likelihood

If the $x_i$ are i.i.d. (independent, identically distributed) from a distribution with density (or mass function) $f(x;\theta)$, the **likelihood** is the joint probability of the observed data, viewed as a function of $\theta$:

$$L(\theta) = \prod_{i=1}^{n} f(x_i;\theta)$$

Read this as: the product, over all $n$ observations, of the density evaluated at each $x_i$. The MLE is the $\theta$ that maximises this.

Products are hard to differentiate. Take the natural log — it's monotone, so the maximum is preserved. The **log-likelihood** is:

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} \log f(x_i;\theta)$$

Read "$\ell(\theta)$" as "ell of theta". Now differentiate and set equal to zero.

### 57.1.2. MLE for the Bernoulli distribution

Let $X_i \sim \text{Bernoulli}(p)$: each observation is 1 (success) or 0 (failure), with unknown success probability $p$. Then $f(x; p) = p^x(1-p)^{1-x}$, so:

For $x = 1$, this gives $f(1; p) = p$. For $x = 0$, it gives $f(0; p) = 1 - p$. So this compact formula is just the Bernoulli mass function written in one line.

$$\ell(p) = \sum_{i=1}^{n} [x_i \log p + (1 - x_i) \log(1 - p)] = \left(\sum x_i\right) \log p + \left(n - \sum x_i\right) \log(1 - p)$$

Let $S = \sum x_i$ (the count of successes). Differentiate with respect to $p$ and set to zero:

$$\frac{d\ell}{dp} = \frac{S}{p} - \frac{n - S}{1 - p} = 0$$

Multiply through by $p(1 - p)$:

$$S(1 - p) - (n - S)p = 0 \implies S - Sp - np + Sp = 0 \implies S = np$$

$$\boxed{\hat{p} = \frac{S}{n} = \bar{X}}$$

The MLE is the sample proportion. This confirms the intuition. This means maximum likelihood recovers the estimate you would probably guess from common sense: the best estimate of a success probability is the observed fraction of successes.

### 57.1.3. MLE for the Exponential distribution

Let $X_i \sim \text{Exp}(\lambda)$: density $f(x; \lambda) = \lambda e^{-\lambda x}$ for $x \geq 0$. The log-likelihood is:

$$\ell(\lambda) = \sum_{i=1}^{n} [\log \lambda - \lambda x_i] = n \log \lambda - \lambda \sum_{i=1}^{n} x_i$$

Differentiate and set to zero:

$$\frac{d\ell}{d\lambda} = \frac{n}{\lambda} - \sum x_i = 0 \implies \lambda = \frac{n}{\sum x_i}$$

$$\boxed{\hat{\lambda} = \frac{1}{\bar{X}}}$$

The MLE of the rate is the reciprocal of the sample mean. If the mean lifetime in the sample is 5 hours, you estimate the failure rate as 0.2 failures per hour.

### 57.1.4. Unbiasedness and consistency

A point estimate is **unbiased** if $E[\hat{\theta}] = \theta$ — the estimate is correct on average. The sample mean $\bar{X}$ is unbiased for $\mu$; the sample variance $s^2 = \frac{1}{n-1}\sum(X_i - \bar{X})^2$ is unbiased for $\sigma^2$ (the $n-1$ denominator matters here — the $n$ version is biased). The reason is that $\bar{X}$ is estimated from the same data: once the mean is fixed, only $n-1$ of the deviations $(X_i - \bar{X})$ are free to vary, so the sum of squared deviations has $n-1$ degrees of freedom, not $n$.

An estimate is **consistent** if $\hat{\theta} \to \theta$ in probability as $n \to \infty$ — more data drives the estimate to the truth. The MLE is consistent under mild regularity conditions.

> **Why MLE works**
> The likelihood is the probability of your data given $\theta$. Maximising it over $\theta$ answers: "which parameter value would make this data least surprising?" That question has a principled answer even when no other method is obvious — just write down the distribution, form the product, and differentiate.

## 57.2. Confidence intervals

A point estimate is a single number. It conceals the uncertainty. A **confidence interval** (CI) pairs the estimate with a margin that quantifies how much the estimate might be off.

### 57.2.1. The problem with small samples

If $\sigma$ (the population standard deviation) were known, the sampling distribution of $\bar{X}$ is exactly $N(\mu, \sigma^2/n)$, and you could build an interval using the standard normal $z$-distribution. But $\sigma$ is almost never known — you estimate it from the sample as $s$. When $n$ is small, this extra source of uncertainty matters.

Student's $t$-distribution handles this. The statistic

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}}$$

follows a $t$-distribution with $\nu = n - 1$ degrees of freedom.

The name "Student" comes from a pseudonym used by William Gosset, a statistician working for Guinness who published under that pen name.

Read "$s/\sqrt{n}$" as the **standard error of the mean** — the typical spread of sample means around the true mean. The $t$-distribution looks like the standard normal but with heavier tails; as $\nu \to \infty$ it converges to the normal.

### 57.2.2. Constructing a 95% CI for $\mu$

Let $t_{\alpha/2,\,n-1}$ be the upper $\alpha/2$ critical value of the $t$-distribution with $n-1$ degrees of freedom — the value such that $P(T > t_{\alpha/2,\,n-1}) = \alpha/2$. For a 95% CI, $\alpha = 0.05$.

Inverting $|T| \le t_{\alpha/2,\,n-1}$ gives the interval. Substituting $T = (\bar{X} - \mu)/(s/\sqrt{n})$ and expanding the absolute value:

$$-t_{\alpha/2,\,n-1} \le \frac{\bar{X} - \mu}{s/\sqrt{n}} \le t_{\alpha/2,\,n-1}$$

Multiply through by $s/\sqrt{n}$ and rearrange to isolate $\mu$:

$$\bar{X} - t_{\alpha/2,\,n-1} \cdot \frac{s}{\sqrt{n}} \;\le\; \mu \;\le\; \bar{X} + t_{\alpha/2,\,n-1} \cdot \frac{s}{\sqrt{n}}$$

Written compactly:

$$\bar{X} \pm t_{\alpha/2,\,n-1} \cdot \frac{s}{\sqrt{n}}$$

This is the 95% confidence interval for $\mu$. This means a confidence interval is always doing the same structural job: estimate at the centre, uncertainty as the margin.

**Worked example.** A materials lab measures Young's modulus for 10 specimens of a polymer. The sample mean is $\bar{x} = 3.42$ GPa and the sample standard deviation is $s = 0.18$ GPa. The $t$-table gives $t_{0.025,\,9} = 2.262$.

$$\text{CI} = 3.42 \pm 2.262 \times \frac{0.18}{\sqrt{10}} = 3.42 \pm 2.262 \times 0.0569 = 3.42 \pm 0.129$$

The 95% CI is $(3.29,\ 3.55)$ GPa.

### 57.2.3. What a confidence interval means — and what it does not

The 95% CI does **not** mean "there is a 95% probability that $\mu$ lies in this interval." After the data are observed, $\mu$ is either in the interval or it isn't — there is no probability involved.

The correct interpretation: if you repeated this experiment many times and built a 95% CI each time, about 95% of those intervals would contain the true $\mu$. The *procedure* has 95% coverage. Any particular interval is just one instance of that procedure.

Use the board below to see that long-run coverage idea directly.

```
//| echo: false
{
  const TRUE_MU = 0;
  const SIGMA = 1;
  const N_INTERVALS = 100;
  const W = 720;
  const ROW_H = 5;
  const xMin = -1.2;
  const xMax = 1.2;
  const zCrit = 1.96;
  let runId = 0;

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  function mulberry32(seed) {
    return function() {
      let t = seed += 0x6D2B79F5;
      t = Math.imul(t ^ (t >>> 15), t | 1);
      t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
      return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
    };
  }

  function normalSample(rng) {
    const u1 = Math.max(rng(), 1e-12);
    const u2 = rng();
    return Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
  }

  function mean(xs) {
    return xs.reduce((a, b) => a + b, 0) / xs.length;
  }

  function sx(x) {
    return 56 + (x - xMin) * (W - 112) / (xMax - xMin);
  }

  const container = document.createElement("div");
  container.style.cssText = "max-width: 780px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; flex-wrap:wrap; margin-bo

  const nWrap = document.createElement("label");
  nWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
```

```
const nText = document.createElement("span");
nText.textContent = "Sample size n";
const nSlider = document.createElement("input");
nSlider.type = "range";
nSlider.min = "5";
nSlider.max = "80";
nSlider.step = "1";
nSlider.value = "20";
const nValue = document.createElement("span");
nValue.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
nWrap.append(nText, nSlider, nValue);

const rerun = document.createElement("button");
rerun.textContent = "Generate 100 samples";
rerun.style.cssText = "padding:0.5rem 0.9rem; border:1px solid #d1d5db; border-radius:8px; backgro
controls.append(nWrap, rerun);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding:0
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
  k.textContent = label;
  const v = document.createElement("div");
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const coverBox = card("Coverage");
const hitBox = card("Intervals containing ");
const missBox = card("Misses");

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${N_INTERVALS * ROW_H + 50}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:white; border:1px solid #e5e7eb; border-radius:10px

const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function renderBoard() {
  runId += 1;
  const n = parseInt(nSlider.value, 10);
  const rng = mulberry32(12345 + 97 * runId + 13 * n);
```

```
nValue.textContent = `${n}`;

while (svg.firstChild) svg.removeChild(svg.firstChild);

const axis = svgEl("line");
axis.setAttribute("x1", sx(xMin));
axis.setAttribute("x2", sx(xMax));
axis.setAttribute("y1", "24");
axis.setAttribute("y2", "24");
axis.setAttribute("stroke", "#9ca3af");
axis.setAttribute("stroke-width", "1.5");
svg.appendChild(axis);

[-1, -0.5, 0, 0.5, 1].forEach(v => {
  const tick = svgEl("line");
  tick.setAttribute("x1", sx(v));
  tick.setAttribute("x2", sx(v));
  tick.setAttribute("y1", "18");
  tick.setAttribute("y2", "30");
  tick.setAttribute("stroke", v === 0 ? "#2563eb" : "#9ca3af");
  tick.setAttribute("stroke-width", v === 0 ? "2.5" : "1.2");
  svg.appendChild(tick);

  const label = svgEl("text");
  label.setAttribute("x", sx(v));
  label.setAttribute("y", "15");
  label.setAttribute("text-anchor", "middle");
  label.setAttribute("font-size", "11");
  label.setAttribute("fill", "#6b7280");
  label.textContent = v.toFixed(v % 1 === 0 ? 0 : 1);
  svg.appendChild(label);
});

const muLine = svgEl("line");
muLine.setAttribute("x1", sx(TRUE_MU));
muLine.setAttribute("x2", sx(TRUE_MU));
muLine.setAttribute("y1", "8");
muLine.setAttribute("y2", `${N_INTERVALS * ROW_H + 38}`);
muLine.setAttribute("stroke", "#2563eb");
muLine.setAttribute("stroke-width", "2.5");
muLine.setAttribute("stroke-dasharray", "4 4");
svg.appendChild(muLine);

let hits = 0;
for (let i = 0; i < N_INTERVALS; i++) {
  const sample = Array.from({ length: n }, () => TRUE_MU + SIGMA * normalSample(rng));
  const xbar = mean(sample);
```

```
      const half = zCrit * SIGMA / Math.sqrt(n);
      const lo = xbar - half;
      const hi = xbar + half;
      const contains = lo <= TRUE_MU && TRUE_MU <= hi;
      if (contains) hits += 1;

      const y = 40 + i * ROW_H;
      const interval = svgEl("line");
      interval.setAttribute("x1", sx(lo));
      interval.setAttribute("x2", sx(hi));
      interval.setAttribute("y1", y);
      interval.setAttribute("y2", y);
      interval.setAttribute("stroke", contains ? "#10b981" : "#ef4444");
      interval.setAttribute("stroke-width", contains ? "2" : "2.4");
      svg.appendChild(interval);

      const dot = svgEl("circle");
      dot.setAttribute("cx", sx(xbar));
      dot.setAttribute("cy", y);
      dot.setAttribute("r", "1.9");
      dot.setAttribute("fill", contains ? "#065f46" : "#b91c1c");
      svg.appendChild(dot);
    }

    const misses = N_INTERVALS - hits;
    coverBox.textContent = `${(100 * hits / N_INTERVALS).toFixed(1)}%`;
    hitBox.textContent = `${hits} of ${N_INTERVALS}`;
    missBox.textContent = `${misses}`;
    missBox.style.color = misses > 0 ? "#b91c1c" : "#065f46";
    caption.textContent = "Green intervals contain the true mean □ = 0. Red intervals miss. The exac
  }

  nSlider.addEventListener("input", renderBoard);
  rerun.addEventListener("click", renderBoard);

  container.append(controls, readout, svg, caption);
  renderBoard();
  return container;
}
```

## 57.3. Hypothesis testing

Point estimates and CIs describe what the data show. Hypothesis testing asks a sharper question: is there enough evidence to reject a specific claim?

### 57.3.1. The framework

The **null hypothesis** $H_0$ is the claim under scrutiny — the default, conservative position. The **alternative hypothesis** $H_1$ is what you'd conclude if $H_0$ is implausible given the data.

A **test statistic** is a function of the data whose distribution under $H_0$ is known. You compute the observed value, then ask: if $H_0$ were true, how often would you see a value this extreme or more extreme? That probability is the **p-value**.

If the p-value is below the **significance level** $\alpha$ (typically 0.05), you reject $H_0$.

Two error types:

| | $H_0$ true | $H_0$ false |
|---|---|---|
| Reject $H_0$ | Type I error (false positive), probability $\alpha$ | Correct |
| Fail to reject $H_0$ | Correct | Type II error (false negative), probability $\beta$ |

Choosing $\alpha = 0.05$ means you're willing to falsely reject a true $H_0$ in 5% of experiments. Reducing $\alpha$ reduces Type I errors but increases Type II errors. The power of a test is $1 - \beta$ — the probability of correctly detecting a real effect.

### 57.3.2. One-sample $t$-test

**Hypotheses.** You want to test whether a population mean $\mu$ equals a specified value $\mu_0$:

$$H_0 : \mu = \mu_0 \qquad H_1 : \mu \neq \mu_0 \quad \text{(two-sided)}$$

**Test statistic.**

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

Under $H_0$ this follows $t_{n-1}$.

**Decision rule.** Reject $H_0$ if $|t| > t_{\alpha/2,\, n-1}$.

**Worked example.** A valve is designed to open at 200 kPa. Quality control samples $n = 20$ valves. The sample mean opening pressure is $\bar{x} = 203.4$ kPa with $s = 5.2$ kPa. Test $H_0 : \mu = 200$ at $\alpha = 0.05$.

$$t = \frac{203.4 - 200}{5.2/\sqrt{20}} = \frac{3.4}{1.163} = 2.923$$

The critical value is $t_{0.025,\,19} = 2.093$. Since $2.923 > 2.093$, reject $H_0$. There is statistically significant evidence that the mean opening pressure differs from 200 kPa.

### 57.3.3. Two-sample $t$-test

To compare means $\mu_1$ and $\mu_2$ from two independent populations (assuming equal variances), form the pooled variance:

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$$

The test statistic is:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{s_p\sqrt{1/n_1 + 1/n_2}}$$

Under $H_0 : \mu_1 = \mu_2$, this follows $t_{n_1+n_2-2}$. Reject when $|t| > t_{\alpha/2, n_1+n_2-2}$.

Welch's $t$-test removes the equal-variance assumption by adjusting the degrees of freedom; it's the default in most software. The pooled test is a little more powerful when equal variances really are plausible, while Welch is safer when they may not be.

### 57.3.4. $\chi^2$ goodness-of-fit

When the data are counts in categories, the $\chi^2$ statistic tests whether the observed frequencies are consistent with a specified distribution. For $k$ categories with observed counts $O_i$ and expected counts $E_i = np_i$:

$$\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$$

Under $H_0$, this follows approximately $\chi^2_{k-1}$. Reject if $\chi^2 > \chi^2_{\alpha, k-1}$. The rule of thumb $E_i \geq 5$ in every cell ensures the approximation is reliable.

## 57.4. Linear regression

Data rarely have a single variable of interest in isolation. Regression asks how one variable changes with another — and how reliably.

### 57.4.1. The model

The **simple linear regression model** is:

$$Y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad i = 1, \dots, n$$

$Y_i$ is the response (outcome), $x_i$ is the predictor (input), $\beta_0$ is the **intercept** (read "beta-zero" — the expected response when $x = 0$), $\beta_1$ is the **slope** (read "beta-one" — the change in expected response per unit increase in $x$), and $\varepsilon_i$ is the error term, assumed i.i.d. $N(0, \sigma^2)$.

### 57.4.2. OLS estimators

**Ordinary least squares** minimises the **sum of squared residuals**:

$$\mathrm{SS}_{res} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{n}(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

Read "$\mathrm{SS}_{res}$" as "residual sum of squares" — the total vertical distance squared between the data points and the fitted line.

Differentiate $\mathrm{SS}_{res}$ with respect to $\hat{\beta}_0$ and $\hat{\beta}_1$ and set both partial derivatives to zero. This gives the **normal equations**. Here "normal" means perpendicular or orthogonal, not "normal distribution":

$$\frac{\partial \mathrm{SS}_{res}}{\partial \hat{\beta}_0} = -2\sum(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0$$

$$\frac{\partial \mathrm{SS}_{res}}{\partial \hat{\beta}_1} = -2\sum x_i(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0$$

From the first equation: distribute the sum and use $\sum_{i=1}^{n} 1 = n$:

$$\sum(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0 \implies n\bar{y} - n\hat{\beta}_0 - \hat{\beta}_1 \cdot n\bar{x} = 0$$

Divide by $n$: $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$.

Substituting into the second and solving:

$$\boxed{\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}}$$

$$\boxed{\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}}$$

where $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y})$ and $S_{xx} = \sum(x_i - \bar{x})^2$. This means the fitted line is pinned to the centre of the data cloud at $(\bar{x}, \bar{y})$ and tilts according to how strongly $x$ and $y$ move together.

### 57.4.3. $R^2$ — coefficient of determination

Define the **total sum of squares** $\mathrm{SS}_{tot} = \sum(y_i - \bar{y})^2$ and the **residual sum of squares** $\mathrm{SS}_{res} = \sum(y_i - \hat{y}_i)^2$. The coefficient of determination is:

$$R^2 = 1 - \frac{\mathrm{SS}_{res}}{\mathrm{SS}_{tot}}$$

$R^2 = 1$ means the line passes through every data point. $R^2 = 0$ means the line explains none of the variation — it is no better than simply predicting $\bar{y}$ for every observation. In simple linear regression, $R^2 = r^2$ where $r$ is the Pearson correlation coefficient, the sample analogue of the population correlation $\rho$ from the previous chapter.

### 57.4.4. $t$-test for $\beta_1$

Under the model assumptions, the standardised slope estimate

$$t = \frac{\hat{\beta}_1 - 0}{s_e / \sqrt{S_{xx}}}$$

follows $t_{n-2}$ under $H_0 : \beta_1 = 0$, where $s_e^2 = \mathrm{SS}_{res}/(n-2)$ is the estimated error variance. The denominator is $n-2$ rather than $n-1$ because two parameters ($\beta_0$ and $\beta_1$) are estimated from the data, consuming two degrees of freedom. Reject $H_0$ if $|t| > t_{\alpha/2, n-2}$. Failing to reject means there is no statistically significant linear relationship between $x$ and $Y$.

### 57.4.5. Residual analysis

The regression assumptions — linearity, constant variance, normality of errors, independence — are checked through the residuals $\hat{\varepsilon}_i = y_i - \hat{y}_i$.

Plot residuals against fitted values $\hat{y}_i$:

- **Random scatter around zero** — assumptions satisfied.
- **Curved pattern** — linearity is violated; consider a transformation or a polynomial term.
- **Fan shape (variance increasing with $\hat{y}_i$)** — heteroscedasticity; consider a log or square-root transformation of $Y$.
- **Outliers** — single points far from zero; investigate individually.

A normal Q-Q plot of residuals checks the normality assumption. Points falling close to the diagonal line support the assumption.

Use the sliders to test lines against the data, then snap to the least-squares fit and compare the change in SSR and $R^2$.

```
//| echo: false
{
  const pts = [
    [0.5,1.3],[0.9,1.8],[1.4,2.5],[1.7,2.4],[2.1,3.1],
    [2.5,3.6],[2.9,3.4],[3.2,4.2],[3.6,4.5],[4.0,4.8],
    [4.3,5.3],[4.8,5.5],[5.1,5.8],[5.5,6.3],[5.9,6.1],
    [6.2,6.8],[6.7,7.0],[7.1,7.6],[7.5,7.7],[7.9,8.2]
  ];
  const n = pts.length;
  const meanX = pts.reduce((s, [x]) => s + x, 0) / n;
  const meanY = pts.reduce((s, [, y]) => s + y, 0) / n;
  const sxx = pts.reduce((s, [x]) => s + (x - meanX) ** 2, 0);
  const sxy = pts.reduce((s, [x, y]) => s + (x - meanX) * (y - meanY), 0);
  const beta1Hat = sxy / sxx;
  const beta0Hat = meanY - beta1Hat * meanX;
  const sst = pts.reduce((s, [, y]) => s + (y - meanY) ** 2, 0);
  const W = 720, H = 420, xMin = 0, xMax = 8.5, yMin = 0.5, yMax = 9;

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }
  function sx(v) { return 55 + (v - xMin) / (xMax - xMin) * 620; }
  function sy(v) { return 360 - (v - yMin) / (yMax - yMin) * 290; }
  function metrics(b0, b1) {
    const ssr = pts.reduce((s, [x, y]) => s + (y - (b0 + b1 * x)) ** 2, 0);
    return { ssr, r2: 1 - ssr / sst };
  }

  const container = document.createElement("div");
  container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem

  function sliderRow(labelText, min, max, value, step) {
    const label = document.createElement("label");
    label.textContent = labelText;
    const input = document.createElement("input");
    input.type = "range";
    input.min = String(min);
    input.max = String(max);
    input.step = String(step);
    input.value = String(value);
    const out = document.createElement("span");
    out.style.cssText = "font-variant-numeric:tabular-nums; min-width:4rem;";
    controls.append(label, input, out);
    return { input, out };
  }
```

```
const b0Row = sliderRow("Intercept β0", 0.5, 4, 1.2, 0.05);
const b1Row = sliderRow("Slope β1", 0.3, 1.3, 0.75, 0.01);

const actions = document.createElement("div");
actions.style.cssText = "display:flex; gap:0.6rem; margin:0.1rem 0 0.75rem 0;";
const fitBtn = document.createElement("button");
fitBtn.textContent = "Fit OLS";
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset guess";
actions.append(fitBtn, resetBtn);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3341
const eqBox = document.createElement("span");
const ssrBox = document.createElement("span");
const r2Box = document.createElement("span");
readout.append(eqBox, ssrBox, r2Box);

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

container.append(controls, actions, readout, svg, caption);

function redraw() {
  const b0 = Number(b0Row.input.value);
  const b1 = Number(b1Row.input.value);
  b0Row.out.textContent = b0.toFixed(2);
  b1Row.out.textContent = b1.toFixed(2);
  const { ssr, r2 } = metrics(b0, b1);

  svg.replaceChildren();
  const axisX = svgEl("line");
  axisX.setAttribute("x1", "55"); axisX.setAttribute("y1", "360");
  axisX.setAttribute("x2", "675"); axisX.setAttribute("y2", "360");
  axisX.setAttribute("stroke", "#64748b");
  axisX.setAttribute("stroke-width", "1.2");
  const axisY = svgEl("line");
  axisY.setAttribute("x1", "55"); axisY.setAttribute("y1", "360");
  axisY.setAttribute("x2", "55"); axisY.setAttribute("y2", "50");
  axisY.setAttribute("stroke", "#64748b");
```

```
  axisY.setAttribute("stroke-width", "1.2");
  svg.append(axisX, axisY);

  for (let x = 1; x <= 8; x++) {
    const t = svgEl("text");
    t.setAttribute("x", String(sx(x)));
    t.setAttribute("y", "378");
    t.setAttribute("text-anchor", "middle");
    t.setAttribute("font-size", "12");
    t.setAttribute("fill", "#64748b");
    t.textContent = String(x);
    svg.appendChild(t);
  }

  pts.forEach(([x, y]) => {
    const resid = y - (b0 + b1 * x);
    const c = svgEl("line");
    c.setAttribute("x1", String(sx(x)));
    c.setAttribute("y1", String(sy(y)));
    c.setAttribute("x2", String(sx(x)));
    c.setAttribute("y2", String(sy(b0 + b1 * x)));
    c.setAttribute("stroke", resid >= 0 ? "#93c5fd" : "#fca5a5");
    c.setAttribute("stroke-width", "2");
    svg.appendChild(c);

    const p = svgEl("circle");
    p.setAttribute("cx", String(sx(x)));
    p.setAttribute("cy", String(sy(y)));
    p.setAttribute("r", "4.5");
    p.setAttribute("fill", "#0f172a");
    svg.appendChild(p);
  });

  const line = svgEl("line");
  line.setAttribute("x1", String(sx(xMin)));
  line.setAttribute("y1", String(sy(b0 + b1 * xMin)));
  line.setAttribute("x2", String(sx(xMax)));
  line.setAttribute("y2", String(sy(b0 + b1 * xMax)));
  line.setAttribute("stroke", "#ea580c");
  line.setAttribute("stroke-width", "3");
  svg.appendChild(line);

  eqBox.textContent = `ŷ = ${b0.toFixed(2)} + ${b1.toFixed(2)}x`;
  ssrBox.textContent = `SSR = ${ssr.toFixed(3)}`;
  r2Box.textContent = `R² = ${r2.toFixed(3)}`;
  caption.textContent = "Vertical segments are residuals. Least squares chooses the line that m
}
```

```
  [b0Row.input, b1Row.input].forEach(el => el.addEventListener("input", redraw));
  fitBtn.addEventListener("click", () => {
    b0Row.input.value = String(beta0Hat);
    b1Row.input.value = String(beta1Hat);
    redraw();
  });
  resetBtn.addEventListener("click", () => {
    b0Row.input.value = "1.2";
    b1Row.input.value = "0.75";
    redraw();
  });

  redraw();
  return container;
}
```

## 57.5. One-way ANOVA

When you want to compare means across three or more groups, running multiple $t$-tests inflates the Type I error rate. One-way ANOVA (analysis of variance) tests all groups simultaneously.

### 57.5.1. The F-test

Suppose you have $k$ groups, with $n_j$ observations in group $j$. Let $\bar{y}_{j\cdot}$ denote the mean of group $j$ and $\bar{y}$ the overall mean. Define:

$$\mathrm{SS}_{between} = \sum_{j=1}^{k} n_j(\bar{y}_{j\cdot} - \bar{y})^2 \qquad \text{(variation explained by group membership)}$$

$$\mathrm{SS}_{within} = \sum_{j=1}^{k} \sum_{i=1}^{n_j} (y_{ij} - \bar{y}_{j\cdot})^2 \qquad \text{(variation within groups)}$$

The **mean squares** are $\mathrm{MS}_{between} = \mathrm{SS}_{between}/(k-1)$ and $\mathrm{MS}_{within} = \mathrm{SS}_{within}/(N-k)$, where $N = \sum n_j$.

The **F-statistic** is:

$$F = \frac{\mathrm{MS}_{between}}{\mathrm{MS}_{within}}$$

Under $H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$, $F$ follows $F_{k-1,\,N-k}$. Reject $H_0$ if $F > F_{\alpha,\,k-1,\,N-k}$. A large $F$ means between-group variation is large relative to within-group variation — the groups genuinely differ.

> **Why F instead of multiple $t$-tests?**
> With $k = 3$ groups, running all pairwise $t$-tests at $\alpha = 0.05$ gives a familywise error rate of approximately $1 - (0.95)^3 \approx 0.14$. You'd falsely reject a true null about 14% of the time rather than 5%. ANOVA controls this by testing all groups in a single F-test. If ANOVA rejects $H_0$, post-hoc pairwise tests (Tukey HSD, Bonferroni) then identify which groups differ.

**Engineering application.** Three machine settings — A, B, C — produce polymer film. Quality engineers measure tensile strength (MPa) from five runs at each setting:

| Setting A | Setting B | Setting C |
|---|---|---|
| 41.2 | 38.5 | 44.1 |
| 40.8 | 39.2 | 43.8 |
| 42.1 | 37.9 | 45.0 |
| 41.5 | 38.8 | 44.3 |
| 40.4 | 39.6 | 43.4 |

Group means: $\bar{y}_A = 41.2$, $\bar{y}_B = 38.8$, $\bar{y}_C = 44.12$. Overall mean $\bar{y} = 41.37$.

$\text{SS}_{between} = 5(41.2 - 41.37)^2 + 5(38.8 - 41.37)^2 + 5(44.12 - 41.37)^2 = 5(0.0289) + 5(6.6049) + 5(7.5625) = 0.1445 + 33.02 + 37.81 = 70.98$

$\text{MS}_{between} = 70.98/2 = 35.49$

Computing $\text{SS}_{within}$ from deviations within each group gives approximately 4.61, so $\text{MS}_{within} = 4.61/12 = 0.384$.

$$F = \frac{35.49}{0.384} = 92.4$$

The critical value $F_{0.05,\,2,\,12} = 3.89$. Since $92.4 \gg 3.89$, reject $H_0$. The machine settings produce significantly different tensile strengths — the effect is not noise.

## 57.6. Where this goes

The methods here assume a parametric form for the distributions — Bernoulli, exponential, normal. **Non-parametric tests** (Wilcoxon, Mann-Whitney, Kruskal-Wallis) make weaker assumptions at the cost of some power; they become relevant when distributions are heavily skewed or sample sizes are very small. **Bayesian inference** reframes the problem entirely: instead of asking "what does the data tell us about $\theta$?" it asks "given a prior belief about $\theta$ and the data, what should our updated belief be?" Both extensions build on exactly the likelihood functions you derived here.

Multiple linear regression extends the single-predictor model to $Y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon$. The OLS estimator in matrix form is $\hat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \mathbf{y}$, which requires the linear algebra from Volume 7's linear algebra section. Generalised linear models (GLMs) then extend regression to non-normal responses — logistic regression for binary outcomes, Poisson regression for counts — connecting directly to the MLE framework you already have.

**Where this shows up**

- **Reliability engineering:** MLE estimates failure rates from field data; confidence intervals bound the estimate; hypothesis tests determine whether a design change made a difference.
- **Clinical trials:** two-sample $t$-tests compare treatment and control groups; sample size is set to achieve target power.
- **Process control:** $t$-tests on subgroup means detect shifts in a manufacturing process mean; ANOVA compares multiple line configurations simultaneously.
- **Geophysics:** linear regression of seismic travel time against distance estimates subsurface velocity; the $t$-test for $\beta_1$ determines whether the relationship is real.
- **Machine learning:** the OLS solution is closed-form linear regression; $R^2$ and residual analysis evaluate model fit before any regularisation is added.

## 57.7. Exercises

These are puzzles. Each has a clean numerical answer. The work is in setting up the likelihood, test statistic, or normal equations correctly before computing.

---

**Exercise 1.** An engineer records the time to failure (hours) for five identical electronic components under accelerated testing:

$$x_1 = 820, \quad x_2 = 1\,040, \quad x_3 = 650, \quad x_4 = 930, \quad x_5 = 760$$

Assuming lifetimes are Exponential($\lambda$), find $\hat{\lambda}$ (the MLE) and its standard error. Use the fact that for the Exponential MLE, $\mathrm{SE}(\hat{\lambda}) = \hat{\lambda}/\sqrt{n}$.

---

**Exercise 2.** A tensile test on 16 specimens of an aluminium alloy yields a sample mean yield strength of $\bar{x} = 48.2$ MPa and sample standard deviation $s = 6.4$ MPa. Construct a 95% confidence interval for the population mean yield strength. Use $t_{0.025,\,15} = 2.131$.

---

**Exercise 3.** A CNC milling process is specified to produce shafts with a mean diameter of 50.00 mm. A quality technician measures a random sample of 25 shafts and finds $\bar{x} = 47.8$ mm and $s = 5.1$ mm. Test the null hypothesis $H_0 : \mu = 50$ against $H_1 : \mu \neq 50$ at significance level $\alpha = 0.05$. Use $t_{0.025,\,24} = 2.064$. State your conclusion.

---

**Exercise 4.** Two aluminium alloys are compared for tensile strength. Alloy 1 ($n_1 = 10$, $\bar{x}_1 = 312$ MPa, $s_1 = 14$ MPa) and Alloy 2 ($n_2 = 12$, $\bar{x}_2 = 298$ MPa, $s_2 = 18$ MPa). Assuming equal population variances, test $H_0 : \mu_1 = \mu_2$ against $H_1 : \mu_1 \neq \mu_2$ at $\alpha = 0.05$. Use $t_{0.025,\,20} = 2.086$.

---

**Exercise 5.** A process engineer suspects that curing temperature $x$ (°C) affects bond strength $Y$ (N/mm²). Five runs are recorded:

| $x$ | 120 | 130 | 140 | 150 | 160 |
|---|---|---|---|---|---|
| $Y$ | 8.1 | 9.4 | 10.8 | 11.5 | 12.9 |

(a) Compute the OLS estimates $\hat{\beta}_0$ and $\hat{\beta}_1$.
(b) Compute $R^2$.
(c) Test $H_0 : \beta_1 = 0$ at $\alpha = 0.05$, given $t_{0.025,\,3} = 3.182$.

---

**Exercise 6.** Three CNC machine settings produce the following daily output yields (units per shift):

| Setting 1 | Setting 2 | Setting 3 |
|---|---|---|
| 102 | 98 | 115 |
| 105 | 101 | 112 |
| 99 | 97 | 118 |
| 103 | 100 | 114 |

Perform a one-way ANOVA F-test for equality of means at $\alpha = 0.05$. Use $F_{0.05,\,2,\,9} = 4.26$. State whether setting affects yield.

# Part XIII.

# Volume 8 — Upper-year engineering mathematics

# 58. Volume 8 — Upper-year engineering mathematics

Volume 8 is where the mathematics stops appearing as a course title and starts appearing as the working language of upper-year engineering.

By this point the reader should already have the Vol 7 toolkit: ODEs, linear algebra, vector calculus, transforms, PDEs, numerical methods, optimisation, and probability. The job now is different. Not "learn the next theorem" in isolation, but learn how mathematical structures reappear inside controls, signals, transport, simulation, estimation, reliability, and design.

This volume is anchored to the kinds of mathematics that show up across years 3 and 4 of University of Alberta engineering programmes: feedback loops, sampled systems, continuum laws, discretised models, noisy data, uncertain failure, and constrained nonlinear tradeoffs.

Optional viewpoints from other domains are built in on purpose. The same tools matter in atmospheric science, quantitative finance, machine learning, geophysics, remote sensing, and operations research. Those side doors are not distractions; they help the reader see the mathematics as a transferable way of thinking instead of a discipline-specific dialect.

The pipeline for this volume is different too. The authoring job is not simply to continue adding topics. It is to test each chapter from several angles at once:

- mathematical dependence: does this genuinely grow out of Volume 7?
- engineering honesty: does this match how upper-year students meet the tool?
- student readability: does the embedded context obscure the mathematics?
- visual design: can the system be seen moving, not just described?
- cross-domain transfer: does the optional viewpoint reveal the same structure?

## 58.1. Chapters

| Chapter | Title | Core move |
|---|---|---|
| 1 | Control, feedback, and stability | Turn dynamics into decisions about response and robustness |
| 2 | Discrete-time systems and signal processing | Move from continuous models to sampled systems and filters |
| 3 | Continuum mechanics and transport | Express materials, fluids, heat, and diffusion as field laws |
| 4 | Computational methods for engineering models | Turn governing equations into mesh-based computations |

| Chapter | Title | Core move |
|---|---|---|
| 5 | Estimation, inverse problems, and filtering | Recover hidden states and parameters from noisy observations |
| 6 | Reliability, stochastic systems, and quality | Model failure, queues, variability, and risk over time |
| 7 | Nonlinear optimisation for design and operations | Choose the best feasible design when tradeoffs are real |

## 58.2. U of A anchors

This volume is not a copy of any one programme calendar, but it is anchored to the kinds of mathematical work that appear in current upper-year UAlberta engineering courses: control systems, signals and systems, process dynamics, engineering statistics, process data analytics, and simulation-heavy design.

## 58.3. Pipeline notes

The first pipeline pass on Volume 8 produced three main decisions:

1. Keep the centre of gravity on engineering practice, not on a second pass through generic advanced mathematics.
2. Make every chapter carry at least one optional viewpoint from another domain, but only where the structure is truly parallel.
3. Treat visualisation as mandatory, because upper-year mathematics is now about systems, flows, feedback loops, noisy measurements, and tradeoff surfaces.

## 58.4. Optional viewpoints

| Viewpoint | What it adds |
|---|---|
| Hard sciences | Stability, waves, transport, and inverse problems in physical systems |
| Computing and data | Signal processing, state estimation, control software, and learned models |
| Finance and quant | Filtering, stochastic evolution, optimisation under uncertainty |
| Geography and environment | Flow, dispersion, remote sensing inversion, and spatial risk |

# 59. Control, feedback, and stability

Making a dynamic system behave

Solving a differential equation tells you what a system will do. Control asks a different question: whether that behaviour is acceptable, how fast it settles, how much it overshoots, and how sensitive it is to disturbances and modelling error.

The central move is this. You measure the output $y(t)$, compare it to the target $r(t)$, form the error $e(t) = r(t) - y(t)$, and choose a control input $u(t)$ that reshapes the dynamics in your favour. Stability and transient response are not side remarks — they are the point.

---

## 59.1. The control problem

Start with four signals.

- $r(t)$ is the **reference**: what you want.
- $y(t)$ is the **output**: what the system is actually doing.
- $e(t) = r(t) - y(t)$ is the **error**.
- $u(t)$ is the **control input** you apply to the plant.

The **plant** is the system being controlled: a motor, furnace, vehicle, converter, or biochemical process. In Laplace-transform language the plant is often represented by a transfer function $G(s)$, read "G of s." The controller is represented by $C(s)$, read "C of s."

In **open loop**, you choose $u(t)$ without using the measured output. In **closed loop**, you feed back the measured output and let the error signal drive the control action. That is the essential difference.

**Unity feedback** means the measured output $Y(s)$ is fed back directly, with no additional gain in the feedback path. The block-diagram algebra is:

$$E(s) = R(s) - Y(s)$$

$$U(s) = C(s)E(s)$$

$$Y(s) = G(s)U(s)$$

Substitute in sequence:

$$Y(s) = G(s)C(s)\big(R(s) - Y(s)\big)$$

Rearrange:

$$\big(1 + C(s)G(s)\big)Y(s) = C(s)G(s)R(s)$$

So the closed-loop transfer function is

$$T(s) = \frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{1 + C(s)G(s)}$$

This means feedback does not merely add another block to the diagram. It changes the input-output law of the whole system.

The equation $1 + C(s)G(s) = 0$ is the **characteristic equation** of the closed-loop system, and its roots are the **closed-loop poles**. They determine whether the response decays, oscillates, grows, or diverges.

*Adjust the gain K below. Watch the step response, pole location, and error readouts change simultaneously.*

```
//| echo: false
closedLoopViz = {
  const d3 = await require("d3@7");

  // ---- Layout constants ----
  const W = 640, H = 480;
  const BLUE = "#2563eb", GREEN = "#059669", RED = "#ef4444", AMBER = "#f59e0b";
  const GREY_DASH = "#9ca3af";

  const container = d3.create("div")
    .style("font-family", "sans-serif")
    .style("max-width", W + "px");

  // K slider
  const sliderRow = container.append("div")
    .style("display", "flex")
    .style("align-items", "center")
    .style("gap", "0.75rem")
    .style("margin-bottom", "0.5rem");

  sliderRow.append("label")
    .style("font-size", "0.9rem")
    .style("white-space", "nowrap")
    .text("Gain K:");

  const slider = sliderRow.append("input")
    .attr("type", "range")
    .attr("min", 0.1)
```

```
  .attr("max", 20)
  .attr("step", 0.1)
  .attr("value", 2)
  .style("flex", "1");

const kLabel = sliderRow.append("span")
  .style("font-weight", "bold")
  .style("min-width", "2.5rem");

// Main SVG
const svg = container.append("svg")
  .attr("width", W)
  .attr("height", H)
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("aria-label", "Closed-loop feedback explorer");

// ---- Block diagram (top half, y 0-210) ----
const BD = svg.append("g").attr("transform", "translate(0,0)");

// Draw block diagram elements
// Signal arrows: r -> summing junction -> controller -> plant -> y (output)
// feedback path from y back to summing junction

// Positions
const SJ = {x: 130, y: 80};   // summing junction centre
const CB = {x: 260, y: 60, w: 80, h: 40}; // controller box
const PB = {x: 420, y: 60, w: 80, h: 40}; // plant box
const outX = 550;               // output signal x

// r arrow
BD.append("line")
  .attr("x1", 50).attr("y1", SJ.y)
  .attr("x2", SJ.x - 18).attr("y2", SJ.y)
  .attr("stroke", "#333").attr("stroke-width", 1.5)
  .attr("marker-end", "url(#arrowBlack)");
BD.append("text").attr("x", 30).attr("y", SJ.y - 8)
  .attr("font-size", "13px").attr("fill", "#333").text("r(t)");

// summing junction circle
BD.append("circle")
  .attr("cx", SJ.x).attr("cy", SJ.y).attr("r", 18)
  .attr("fill", "none").attr("stroke", "#333").attr("stroke-width", 1.5);
BD.append("text").attr("x", SJ.x - 6).attr("y", SJ.y + 5)
  .attr("font-size", "14px").attr("fill", "#333").text("+");
BD.append("text").attr("x", SJ.x - 5).attr("y", SJ.y + 22)
  .attr("font-size", "11px").attr("fill", "#ef4444").text("−");
```

```
// e(t) label between SJ and controller
BD.append("line")
  .attr("x1", SJ.x + 18).attr("y1", SJ.y)
  .attr("x2", CB.x).attr("y2", SJ.y)
  .attr("stroke", "#333").attr("stroke-width", 1.5)
  .attr("marker-end", "url(#arrowBlack)");
BD.append("text").attr("x", SJ.x + 22).attr("y", SJ.y - 8)
  .attr("font-size", "12px").attr("fill", "#6b7280").text("e(t)");

// Controller box
BD.append("rect")
  .attr("x", CB.x).attr("y", CB.y).attr("width", CB.w).attr("height", CB.h)
  .attr("rx", 6).attr("fill", "#f3f4f6").attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
const kBoxLabel = BD.append("text")
  .attr("x", CB.x + CB.w / 2).attr("y", CB.y + CB.h / 2 + 5)
  .attr("text-anchor", "middle").attr("font-size", "14px").attr("fill", BLUE);

// Arrow controller to plant
BD.append("line")
  .attr("x1", CB.x + CB.w).attr("y1", SJ.y)
  .attr("x2", PB.x).attr("y2", SJ.y)
  .attr("stroke", "#333").attr("stroke-width", 1.5)
  .attr("marker-end", "url(#arrowBlack)");
BD.append("text").attr("x", CB.x + CB.w + 5).attr("y", SJ.y - 8)
  .attr("font-size", "12px").attr("fill", "#6b7280").text("u(t)");

// Plant box
BD.append("rect")
  .attr("x", PB.x).attr("y", PB.y).attr("width", PB.w).attr("height", PB.h)
  .attr("rx", 6).attr("fill", "#f3f4f6").attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
BD.append("text")
  .attr("x", PB.x + PB.w / 2).attr("y", PB.y + PB.h / 2 + 5)
  .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#374151")
  .text("G(s)=1/(5s+1)");

// Arrow plant to output
BD.append("line")
  .attr("x1", PB.x + PB.w).attr("y1", SJ.y)
  .attr("x2", outX).attr("y2", SJ.y)
  .attr("stroke", "#333").attr("stroke-width", 1.5)
  .attr("marker-end", "url(#arrowBlack)");
BD.append("text").attr("x", outX + 5).attr("y", SJ.y + 5)
  .attr("font-size", "13px").attr("fill", "#333").text("y(t)");

// Feedback path (y -> down -> left -> up -> summing junction bottom)
const fbY = SJ.y + 70;
BD.append("polyline")
```

```
  .attr("points", `${outX - 5},${SJ.y} ${outX - 5},${fbY} ${SJ.x},${fbY} ${SJ.x},${SJ.y + 18}`)
  .attr("fill", "none").attr("stroke", "#333").attr("stroke-width", 1.5)
  .attr("marker-end", "url(#arrowBlack)");
BD.append("text").attr("x", (SJ.x + outX) / 2).attr("y", fbY + 16)
  .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#6b7280")
  .text("unity feedback");

// Arrowhead marker def
const defs = svg.append("defs");
defs.append("marker")
  .attr("id", "arrowBlack").attr("markerWidth", 8).attr("markerHeight", 8)
  .attr("refX", 6).attr("refY", 3).attr("orient", "auto")
  .append("path").attr("d", "M0,0 L0,6 L8,3 z").attr("fill", "#333");

// ---- Step response panel (bottom left, y 220-480, x 0-320) ----
const SP_X = 30, SP_Y = 200, SP_W = 280, SP_H = 200;
const spG = svg.append("g").attr("transform", `translate(${SP_X},${SP_Y})`);
spG.append("text").attr("x", SP_W / 2).attr("y", 0)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#374151")
  .text("Step response y(t)");
// Axes
spG.append("line").attr("x1", 30).attr("y1", 10).attr("x2", 30).attr("y2", SP_H - 20)
  .attr("stroke", "#6b7280").attr("stroke-width", 1.5);
spG.append("line").attr("x1", 30).attr("y1", SP_H - 20).attr("x2", SP_W - 10).attr("y2", SP_H -
  .attr("stroke", "#6b7280").attr("stroke-width", 1.5);
spG.append("text").attr("x", SP_W - 8).attr("y", SP_H - 17)
  .attr("font-size", "10px").attr("fill", "#6b7280").text("t");
spG.append("text").attr("x", 0).attr("y", 10)
  .attr("font-size", "10px").attr("fill", "#6b7280").text("y");

// Reference line (dashed grey at y=1)
const spXmin = 30, spXmax = SP_W - 10, spYmin = 10, spYmax = SP_H - 20;
const yScale = d3.scaleLinear([0, 1.5], [spYmax, spYmin]);

spG.append("line")
  .attr("x1", spXmin).attr("y1", yScale(1))
  .attr("x2", spXmax).attr("y2", yScale(1))
  .attr("stroke", GREY_DASH).attr("stroke-width", 1).attr("stroke-dasharray", "5,3");
spG.append("text").attr("x", spXmax + 2).attr("y", yScale(1) + 4)
  .attr("font-size", "10px").attr("fill", GREY_DASH).text("r=1");

// Error shading group and response path
const errorShade = spG.append("path")
  .attr("fill", RED).attr("fill-opacity", 0.12).attr("stroke", "none");
const responsePath = spG.append("path")
  .attr("fill", "none").attr("stroke", BLUE).attr("stroke-width", 2);
```

```
// ---- Pole location panel (bottom right, y 220-480, x 330-640) ----
const PP_X = 340, PP_Y = 200, PP_W = 260, PP_H = 200;
const ppG = svg.append("g").attr("transform", `translate(${PP_X},${PP_Y})`);
ppG.append("text").attr("x", PP_W / 2).attr("y", 0)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#374151")
  .text("Pole location (real axis)");

// s-plane: x-axis only, range -6 to 0.5
const poleXscale = d3.scaleLinear([-6, 0.5], [10, PP_W - 10]);
const poleY = 80;
// Stable region shading
ppG.append("rect")
  .attr("x", 10).attr("y", 20)
  .attr("width", poleXscale(0) - 10).attr("height", 100)
  .attr("fill", GREEN).attr("fill-opacity", 0.08);
// Axis
ppG.append("line")
  .attr("x1", 10).attr("y1", poleY).attr("x2", PP_W - 10).attr("y2", poleY)
  .attr("stroke", "#6b7280").attr("stroke-width", 1.5);
// Imaginary axis (vertical dashed)
ppG.append("line")
  .attr("x1", poleXscale(0)).attr("y1", 10)
  .attr("x2", poleXscale(0)).attr("y2", 150)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1).attr("stroke-dasharray", "4,3");
ppG.append("text").attr("x", poleXscale(0) + 2).attr("y", 16)
  .attr("font-size", "9px").attr("fill", "#9ca3af").text("stability boundary");

// Axis ticks
[-5,-4,-3,-2,-1,0].forEach(v => {
  const x = poleXscale(v);
  ppG.append("line").attr("x1", x).attr("y1", poleY - 5)
    .attr("x2", x).attr("y2", poleY + 5).attr("stroke", "#9ca3af");
  ppG.append("text").attr("x", x).attr("y", poleY + 18)
    .attr("text-anchor", "middle").attr("font-size", "10px").attr("fill", "#6b7280")
    .text(v);
});
ppG.append("text").attr("x", PP_W - 8).attr("y", poleY + 5)
  .attr("font-size", "10px").attr("fill", "#6b7280").text("Re(s)");
ppG.append("text").attr("x", poleXscale(-3)).attr("y", 16)
  .attr("text-anchor", "middle").attr("font-size", "10px").attr("fill", GREEN)
  .text("stable region");

const poleCircle = ppG.append("circle")
  .attr("cy", poleY).attr("r", 8)
  .attr("fill", BLUE).attr("stroke", "#1e40af").attr("stroke-width", 1.5);
const poleLabelEl = ppG.append("text")
  .attr("text-anchor", "middle").attr("y", poleY - 15)
```

```
  .attr("font-size", "11px").attr("fill", BLUE).attr("font-weight", "bold");

// Readouts
const readoutG = svg.append("g").attr("transform", `translate(${PP_X}, ${PP_Y + 140})`);
const settlingTxt = readoutG.append("text")
  .attr("font-size", "12px").attr("fill", AMBER);
const errorTxt = readoutG.append("text")
  .attr("y", 20).attr("font-size", "12px").attr("fill", AMBER);

// ---- Update function ----
function update(K) {
  kLabel.text(K.toFixed(1));
  kBoxLabel.text(`C(s) = ${K.toFixed(1)}`);

  // Plant G(s) = 1/(5s+1), C(s) = K
  // Closed-loop pole: s = -(1+K)/5
  const pole = -(1 + K) / 5;
  const tau_cl = 5 / (1 + K);          // closed-loop time constant
  const ss_gain = K / (1 + K);         // steady-state output for unit step
  const ss_error = 1 / (1 + K);        // steady-state error

  // Settling time approx 4*tau_cl
  const t_settle = 4 * tau_cl;

  // Step response: y(t) = ss_gain * (1 - e^{-t/tau_cl})
  const tMax = Math.max(30, t_settle * 1.6);
  const nPts = 200;
  const pts = d3.range(nPts).map(i => {
    const t = (i / (nPts - 1)) * tMax;
    return [t, ss_gain * (1 - Math.exp(-t / tau_cl))];
  });

  const tScale = d3.scaleLinear([0, tMax], [spXmin, spXmax]);

  // Error shading area
  const areaGen = d3.area()
    .x(d => tScale(d[0]))
    .y0(yScale(1))
    .y1(d => yScale(Math.min(d[1], 1.4)));
  errorShade.attr("d", areaGen(pts));

  // Response path
  const lineGen = d3.line().x(d => tScale(d[0])).y(d => yScale(Math.min(d[1], 1.4)));
  responsePath.attr("d", lineGen(pts));

  // Pole position
  poleCircle.attr("cx", poleXscale(pole));
```

```
    poleLabelEl.attr("x", poleXscale(pole)).text(`s = ${pole.toFixed(2)}`);

    settlingTxt.text(`Settling time ⍰ ${t_settle.toFixed(1)} s`);
    errorTxt.text(`Steady-state error = ${(ss_error * 100).toFixed(1)}%`);
  }

  slider.on("input", function() { update(+this.value); });
  update(2);

  return container.node();
}
```

## 59.2. What stability means

A control loop is **stable** if small disturbances do not produce outputs that grow without bound. In continuous-time linear systems, the quick rule is:

- if every closed-loop pole has negative real part, the response decays
- if any closed-loop pole has positive real part, the response grows
- if poles sit on the imaginary axis, the system is marginal and small model errors may push it into trouble

This is why pole location is not bookkeeping. It is a statement about physical behaviour.

The same stability condition applies in state-space form — vectors and matrices instead of transfer functions, but the same underlying dynamics. A transfer function $G(s)$ and a state-space model $(A, B)$ are two descriptions of the same system.

Here $K$ is a row vector of gains — distinct from the scalar proportional gain used above. For a state-space model

$$\dot{x} = Ax + Bu, \qquad u = -Kx$$

substituting $u = -Kx$ into the equation gives

$$\dot{x} = Ax + B(-Kx) = (A - BK)x$$

Now the eigenvalues $\lambda(A - BK)$ play the same role that closed-loop poles play in transfer-function form. They tell you what the state does in time.

> **Why this works**
>
> Feedback changes the equation you are solving. Without control, the plant's dynamics are built into $G(s)$ or $A$. With control, the measured output feeds back into the input channel, so the effective denominator changes from the plant's own poles to a new set determined by the loop. You are not accepting the poles, modes, or eigenvalues the plant came with. You are choosing them.

## 59.3. The core method

For a first pass through a control problem, the workflow is usually:

1. Write the plant model in transfer-function or state-space form.
2. Decide what "good behaviour" means: stable, fast enough, low overshoot, small steady-state error, acceptable control effort.
3. Choose a controller structure: proportional, PI, PID, state feedback, or another architecture appropriate to the system.
4. Form the closed-loop model.
5. Inspect the poles or eigenvalues and connect them back to the time response.
6. Tune, then check what the tuning costs you elsewhere.

The important habit is to keep the interpretation attached to the algebra. A gain value is not just a number. It changes speed, error, sensitivity, and sometimes noise amplification.

## 59.4. Worked example 1: cruise control with proportional feedback

Suppose a simplified vehicle-speed model has plant transfer function

$$G(s) = \frac{1}{5s + 1}$$

where input is throttle command and output is speed deviation from the desired operating point. Use a proportional controller

$$C(s) = K$$

with unity feedback.

The closed-loop transfer function is

$$T(s) = \frac{KG(s)}{1 + KG(s)} = \frac{K/(5s + 1)}{1 + K/(5s + 1)} = \frac{K}{5s + 1 + K}$$

So the closed-loop pole is at

$$s = -\frac{1 + K}{5}$$

For any $K > -1$, the pole is in the left half-plane, so the linear model is stable. If $K > 0$, increasing $K$ moves the pole further left, which makes the response faster.

The steady-state gain is

$$T(0) = \frac{K}{1 + K}$$

so proportional feedback alone does not remove step-tracking error completely. For a unit step reference, the steady-state output is $K/(1+K)$ and the steady-state error is

$$1 - \frac{K}{1+K} = \frac{1}{1+K}$$

This is the standard tradeoff. Larger $K$ reduces error and speeds the loop, but in a more realistic model it also increases sensitivity to unmodelled dynamics, actuator limits, and measurement noise.

*The two-panel chart below shows both sides of this tradeoff at once. Drag the cursor to explore.*

```
//| echo: false
gainTradeoffViz = {
  const d3 = await require("d3@7");

  const W = 560, H = 340;
  const BLUE = "#2563eb", RED = "#ef4444", AMBER = "#f59e0b";
  const ML = 55, MR = 20, panelH = 130, gap = 30;

  const container = d3.create("div")
    .style("font-family", "sans-serif")
    .style("max-width", W + "px");

  const svg = container.append("svg")
    .attr("width", W).attr("height", H)
    .attr("viewBox", `0 0 ${W} ${H}`)
    .attr("aria-label", "Proportional gain tradeoff chart");

  const innerW = W - ML - MR;
  const Kmax = 20;
  const kScale = d3.scaleLinear([0, Kmax], [0, innerW]);

  // ---- Top panel: pole location ----
  const topG = svg.append("g").attr("transform", `translate(${ML}, 20)`);
  const poleScale = d3.scaleLinear([0, -5], [panelH, 0]);

  // axes
  topG.append("line").attr("x1", 0).attr("y1", panelH)
    .attr("x2", innerW).attr("y2", panelH).attr("stroke", "#6b7280").attr("stroke-width", 1.5);
  topG.append("line").attr("x1", 0).attr("y1", 0)
    .attr("x2", 0).attr("y2", panelH).attr("stroke", "#6b7280").attr("stroke-width", 1.5);

  topG.append("text").attr("x", innerW / 2).attr("y", panelH + 22)
    .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#374151").text("K");
  topG.append("text").attr("x", -40).attr("y", panelH / 2 + 4)
    .attr("font-size", "11px").attr("fill", "#374151").text("pole s");

  // annotation
  topG.append("text").attr("x", innerW * 0.55).attr("y", 18)
```

```
  .attr("font-size", "10px").attr("fill", "#6b7280").text("farther left = faster response");

// pole curve: s = -(1+K)/5
const polePts = d3.range(201).map(i => {
  const K = i / 200 * Kmax;
  return [kScale(K), poleScale(-(1 + K) / 5)];
});
topG.append("path")
  .attr("d", d3.line()(polePts))
  .attr("fill", "none").attr("stroke", BLUE).attr("stroke-width", 2);

// y-axis ticks
[-1,-2,-3,-4,-5].forEach(v => {
  topG.append("line").attr("x1", -4).attr("y1", poleScale(v))
    .attr("x2", 0).attr("y2", poleScale(v)).attr("stroke", "#9ca3af");
  topG.append("text").attr("x", -8).attr("y", poleScale(v) + 4)
    .attr("text-anchor", "end").attr("font-size", "9px").attr("fill", "#6b7280").text(v);
});
// x-axis ticks
[0, 5, 10, 15, 20].forEach(v => {
  topG.append("text").attr("x", kScale(v)).attr("y", panelH + 14)
    .attr("text-anchor", "middle").attr("font-size", "9px").attr("fill", "#6b7280").text(v);
});

// cursor line and dot (top)
const topCursor = topG.append("line")
  .attr("y1", 0).attr("y2", panelH)
  .attr("stroke", AMBER).attr("stroke-width", 1.5).attr("stroke-dasharray", "4,3");
const topDot = topG.append("circle").attr("r", 5).attr("fill", AMBER);

// ---- Bottom panel: steady-state error ----
const botG = svg.append("g").attr("transform", `translate(${ML}, ${20 + panelH + gap})`);
const errScale = d3.scaleLinear([0, 1], [panelH, 0]);

botG.append("line").attr("x1", 0).attr("y1", panelH)
  .attr("x2", innerW).attr("y2", panelH).attr("stroke", "#6b7280").attr("stroke-width", 1.5);
botG.append("line").attr("x1", 0).attr("y1", 0)
  .attr("x2", 0).attr("y2", panelH).attr("stroke", "#6b7280").attr("stroke-width", 1.5);

botG.append("text").attr("x", innerW / 2).attr("y", panelH + 22)
  .attr("text-anchor", "middle").attr("font-size", "11px").attr("fill", "#374151").text("K");
botG.append("text").attr("x", -40).attr("y", panelH / 2 + 4)
  .attr("font-size", "11px").attr("fill", "#374151").text("error");

botG.append("text").attr("x", innerW * 0.35).attr("y", panelH - 8)
  .attr("font-size", "10px").attr("fill", "#6b7280")
  .text("never reaches zero with proportional control only");
```

```
const errPts = d3.range(201).map(i => {
  const K = i / 200 * Kmax;
  return [kScale(K), errScale(1 / (1 + K))];
});
botG.append("path")
  .attr("d", d3.line()(errPts))
  .attr("fill", "none").attr("stroke", RED).attr("stroke-width", 2);

[0, 5, 10, 15, 20].forEach(v => {
  botG.append("text").attr("x", kScale(v)).attr("y", panelH + 14)
    .attr("text-anchor", "middle").attr("font-size", "9px").attr("fill", "#6b7280").text(v);
});
[0, 0.5, 1].forEach(v => {
  botG.append("line").attr("x1", -4).attr("y1", errScale(v))
    .attr("x2", 0).attr("y2", errScale(v)).attr("stroke", "#9ca3af");
  botG.append("text").attr("x", -8).attr("y", errScale(v) + 4)
    .attr("text-anchor", "end").attr("font-size", "9px").attr("fill", "#6b7280")
    .text(v.toFixed(1));
});

const botCursor = botG.append("line")
  .attr("y1", 0).attr("y2", panelH)
  .attr("stroke", AMBER).attr("stroke-width", 1.5).attr("stroke-dasharray", "4,3");
const botDot = botG.append("circle").attr("r", 5).attr("fill", AMBER);

// Readout text
const readoutEl = svg.append("text")
  .attr("x", ML + 10).attr("y", 20 + panelH * 2 + gap * 2 + 10)
  .attr("font-size", "12px").attr("fill", AMBER);

// Drag interaction on full SVG
let currentK = 5;
function updateCursor(K) {
  K = Math.max(0, Math.min(Kmax, K));
  currentK = K;
  const cx = kScale(K);
  const poleVal = -(1 + K) / 5;
  const errVal = 1 / (1 + K);
  topCursor.attr("x1", cx).attr("x2", cx);
  topDot.attr("cx", cx).attr("cy", poleScale(poleVal));
  botCursor.attr("x1", cx).attr("x2", cx);
  botDot.attr("cx", cx).attr("cy", errScale(errVal));
  readoutEl.text(`K = ${K.toFixed(1)}  |  pole = ${poleVal.toFixed(3)}  |  error = ${(errVal * 10
}

svg.style("cursor", "crosshair")
  .on("mousemove", function(event) {
```

```
      const [mx] = d3.pointer(event);
      const K = (mx - ML) / innerW * Kmax;
      updateCursor(K);
    });

  updateCursor(5);
  return container.node();
}
```

## 59.5. Worked example 2: state feedback for a two-state system

Consider the linear system

$$\dot{x} = Ax + Bu$$

with

$$A = \begin{pmatrix} 0 & 1 \\ -2 & -1 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Choose state feedback

$$u = -Kx, \qquad K = \begin{pmatrix} 3 & 2 \end{pmatrix}$$

Then

$$A - BK = \begin{pmatrix} 0 & 1 \\ -2 & -1 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 & 2 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -5 & -3 \end{pmatrix}$$

Its characteristic polynomial is $\det(\lambda I - (A - BK))$, evaluated as:

$$\det(\lambda I - (A - BK)) = \det \begin{pmatrix} \lambda & -1 \\ 5 & \lambda + 3 \end{pmatrix} = \lambda(\lambda + 3) + 5 = \lambda^2 + 3\lambda + 5$$

The eigenvalues are

$$\lambda = \frac{-3 \pm \sqrt{9 - 20}}{2} = -\frac{3}{2} \pm \frac{\sqrt{11}}{2} i$$

Both eigenvalues have negative real part, so the closed-loop system is stable. Compared with the uncontrolled system, the controller has shifted the system's modes. That is the state-space version of pole placement.

*The interactive below lets you explore how the gain vector $K = (k_1, k_2)$ shapes the eigenvalues and phase portrait.*

```
//| echo: false
stateFeedbackViz = {
  const d3 = await require("d3@7");

  const W = 640, PH = 280;
  const BLUE = "#2563eb", GREEN = "#059669", RED = "#ef4444", AMBER = "#f59e0b";

  const container = d3.create("div")
    .style("font-family", "sans-serif")
    .style("max-width", W + "px");

  // Sliders
  const sliderDiv = container.append("div")
    .style("display", "flex").style("gap", "1.5rem").style("margin-bottom", "0.5rem");

  function makeSlider(parent, label, min, max, step, val) {
    const wrap = parent.append("div").style("flex", "1");
    wrap.append("label").style("font-size", "0.88rem").style("display", "block")
      .text(label);
    const inp = wrap.append("input").attr("type", "range")
      .attr("min", min).attr("max", max).attr("step", step).attr("value", val)
      .style("width", "100%");
    const lbl = wrap.append("span").style("font-size", "0.88rem").style("font-weight", "bold");
    return {inp, lbl};
  }

  const s1 = makeSlider(sliderDiv, "k₁:", 0, 6, 0.1, 3);
  const s2 = makeSlider(sliderDiv, "k₂:", 0, 6, 0.1, 2);

  const svg = container.append("svg")
    .attr("width", W).attr("height", PH)
    .attr("viewBox", `0 0 ${W} ${PH}`)
    .attr("aria-label", "State feedback phase portrait");

  const panW = W / 2 - 20;

  // ---- Left: s-plane ----
  const spG = svg.append("g").attr("transform", "translate(10, 10)");
  const spXsc = d3.scaleLinear([-5, 1], [30, panW - 10]);
  const spYsc = d3.scaleLinear([-4, 4], [PH - 40, 30]);

  // Stable region shading
  spG.append("rect").attr("x", 30).attr("y", 30)
    .attr("width", spXsc(0) - 30).attr("height", PH - 70)
    .attr("fill", GREEN).attr("fill-opacity", 0.1);

  // Axes
```

```
spG.append("line").attr("x1", 30).attr("y1", spYsc(0))
  .attr("x2", panW - 10).attr("y2", spYsc(0)).attr("stroke", "#9ca3af").attr("stroke-width", 1)
spG.append("line").attr("x1", spXsc(0)).attr("y1", 30)
  .attr("x2", spXsc(0)).attr("y2", PH - 40)
  .attr("stroke", "#9ca3af").attr("stroke-width", 1).attr("stroke-dasharray", "4,3");

spG.append("text").attr("x", panW / 2).attr("y", 18)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#374151")
  .text("s-plane eigenvalues");
spG.append("text").attr("x", spXsc(0) + 2).attr("y", 28)
  .attr("font-size", "9px").attr("fill", "#9ca3af").text("stability boundary");

[-4,-3,-2,-1,0].forEach(v => {
  spG.append("text").attr("x", spXsc(v)).attr("y", spYsc(0) + 14)
    .attr("text-anchor", "middle").attr("font-size", "9px").attr("fill", "#9ca3af").text(v);
});
[-3,-2,-1,1,2,3].forEach(v => {
  spG.append("text").attr("x", spXsc(0) + 5).attr("y", spYsc(v) + 4)
    .attr("font-size", "9px").attr("fill", "#9ca3af").text(`${v}i`);
});
spG.append("text").attr("x", panW - 8).attr("y", spYsc(0) + 4)
  .attr("font-size", "10px").attr("fill", "#9ca3af").text("Re");
spG.append("text").attr("x", spXsc(0) - 14).attr("y", 34)
  .attr("font-size", "10px").attr("fill", "#9ca3af").text("Im");

const ev1 = spG.append("circle").attr("r", 7).attr("stroke-width", 2);
const ev2 = spG.append("circle").attr("r", 7).attr("stroke-width", 2);
const evLabel = spG.append("text")
  .attr("text-anchor", "middle").attr("font-size", "11px").attr("font-weight", "bold");
const stabilityAnnot = spG.append("text")
  .attr("x", spXsc(-2.5)).attr("y", PH - 45)
  .attr("text-anchor", "middle").attr("font-size", "11px").attr("font-weight", "bold");

// ---- Right: phase portrait ----
const ppG = svg.append("g").attr("transform", `translate(${W / 2 + 10}, 10)`);
const ppXsc = d3.scaleLinear([-3, 3], [30, panW - 10]);
const ppYsc = d3.scaleLinear([-3, 3], [PH - 40, 30]);

ppG.append("text").attr("x", panW / 2).attr("y", 18)
  .attr("text-anchor", "middle").attr("font-size", "12px").attr("fill", "#374151")
  .text("(x₁, x₂) phase portrait");

// Axes
ppG.append("line").attr("x1", 30).attr("y1", ppYsc(0))
  .attr("x2", panW - 10).attr("y2", ppYsc(0)).attr("stroke", "#d1d5db").attr("stroke-width", 1)
ppG.append("line").attr("x1", ppXsc(0)).attr("y1", 30)
  .attr("x2", ppXsc(0)).attr("y2", PH - 40).attr("stroke", "#d1d5db").attr("stroke-width", 1);
```

```
ppG.append("text").attr("x", panW - 5).attr("y", ppYsc(0) + 4)
  .attr("font-size", "10px").attr("fill", "#9ca3af").text("x₁");
ppG.append("text").attr("x", ppXsc(0) + 3).attr("y", 28)
  .attr("font-size", "10px").attr("fill", "#9ca3af").text("x₂");

const trajectoryPath = ppG.append("path")
  .attr("fill", "none").attr("stroke-width", 2);
const ppAnnot = ppG.append("text")
  .attr("x", panW / 2).attr("y", PH - 44)
  .attr("text-anchor", "middle").attr("font-size", "11px").attr("font-weight", "bold");

function update(k1, k2) {
  s1.lbl.text(k1.toFixed(1));
  s2.lbl.text(k2.toFixed(1));

  // A - BK where A=[[0,1],[-2,-1]], B=[[0],[1]], K=[k1,k2]
  // A-BK = [[0,1],[-2-k1,-1-k2]]
  const a = 0, b = 1, c = -2 - k1, d = -1 - k2;
  // char poly: λ² - (a+d)λ + (ad-bc)
  const tr = a + d;
  const det = a * d - b * c;
  const disc = tr * tr - 4 * det;

  let re1, im1, re2, im2, stable;
  if (disc >= 0) {
    re1 = (tr + Math.sqrt(disc)) / 2;
    re2 = (tr - Math.sqrt(disc)) / 2;
    im1 = 0; im2 = 0;
    stable = re1 < 0 && re2 < 0;
  } else {
    re1 = re2 = tr / 2;
    im1 = Math.sqrt(-disc) / 2;
    im2 = -im1;
    stable = re1 < 0;
  }

  const col = stable ? BLUE : RED;
  ev1.attr("cx", spXsc(re1)).attr("cy", spYsc(im1))
    .attr("fill", col).attr("stroke", col === BLUE ? "#1e40af" : "#991b1b");
  ev2.attr("cx", spXsc(re2)).attr("cy", spYsc(im2))
    .attr("fill", col).attr("stroke", col === BLUE ? "#1e40af" : "#991b1b");

  const evStr = disc < 0
    ? `λ = ${re1.toFixed(2)} ± ${Math.abs(im1).toFixed(2)}i`
    : `λ = ${re1.toFixed(2)}, ${re2.toFixed(2)}`;
  evLabel.attr("x", spXsc(re1)).attr("y", spYsc(im1) - 12)
    .attr("fill", col).text(evStr);
```

```
    stabilityAnnot.attr("fill", stable ? GREEN : RED)
      .text(stable ? "Re(λ) < 0: decays to origin" : "Re(λ) ≥ 0: grows away from origin");

    // Euler trajectory starting near (1, 1)
    const dt = 0.02, steps = 600;
    let x1 = 1.0, x2 = 1.0;
    const path = [[x1, x2]];
    const clip = 4;
    for (let i = 0; i < steps; i++) {
      const u = -(k1 * x1 + k2 * x2);
      const dx1 = a * x1 + b * x2;
      const dx2 = c * x1 + d * x2;
      x1 += dt * dx1;
      x2 += dt * dx2;
      if (Math.abs(x1) > clip || Math.abs(x2) > clip) break;
      path.push([x1, x2]);
    }

    const lineGen = d3.line().x(d => ppXsc(d[0])).y(d => ppYsc(d[1]));
    trajectoryPath.attr("d", lineGen(path)).attr("stroke", stable ? GREEN : RED);
    ppAnnot.attr("fill", stable ? GREEN : RED)
      .text(stable ? "decays to origin" : "grows away from origin");
  }

  s1.inp.on("input", function() { update(+this.value, +s2.inp.property("value")); });
  s2.inp.on("input", function() { update(+s1.inp.property("value"), +this.value); });
  update(3, 2);

  return container.node();
}
```

This is the language used constantly in robotics, flight control, and embedded control software. The code may be digital and the sensors noisy, but the core mathematical question is still: what did your feedback law do to the modes?

## 59.6. Worked example 3: stabilising a scientific instrument

An optics experiment requires a laser intensity to hold near a target despite thermal drift. A simplified model of the plant is slow and first-order, so the experimentalist begins with a proportional controller for the same reason an engineer would: it is low-complexity and straightforward to analyse.

If the gain is too low, the output drifts and tracking is poor. If the gain is too high, noise and delay in the sensor chain start to matter. The mathematics is identical to the motor-speed example. What changes is the vocabulary.

Control theory is not owned by electrical engineering. It is a general mathematical structure for shaping dynamics under measurement.

## 59.7. Where this goes

The most direct continuation is Estimation, inverse problems, and filtering (Volume 8, Chapter 5). Real control systems often cannot measure every state they need directly. Once you ask how to control a system, the next question is usually how to estimate what you cannot observe cleanly.

This chapter also informs how you read later Volume 8 material on sampled systems and computational models. A simulation is not a controller. A model can be numerically stable and still be a poor control design. That distinction runs through the rest of the volume.

**Where this appears in practice**

- motor-speed control in electric drives
- altitude hold and attitude stabilisation in drones
- furnace and reactor temperature control
- active vibration suppression in flexible structures
- instrument stabilisation in optics and experimental physics
- state-feedback and observer design in robotics and autonomous systems

## 59.8. Exercises

Each exercise asks you to connect the algebra to physical behaviour.

```
//| echo: false
katex = require("katex@0.16")
```

```
//| echo: false
function makeStepperHTML(exerciseNum, steps) {
  let current = 0;
  const totalSteps = steps.length;

  const container = document.createElement("div");
  container.style.cssText = "border:1px solid #e5e7eb; border-radius:8px; padding:1rem 1.25rem; marg

  const stepsDiv = document.createElement("div");
  stepsDiv.style.marginBottom = "0.75rem";

  function renderSteps() {
    stepsDiv.innerHTML = "";
    for (let i = 0; i < current; i++) {
      const s = steps[i];
```

```
    const row = document.createElement("div");
    row.style.cssText = "display:grid; grid-template-columns:200px 1fr; gap:0.5rem 1rem; align-

    const opCell = document.createElement("span");
    opCell.style.cssText = "font-size:0.85em; color:#6b7280; font-style:italic;";
    opCell.textContent = s.op;

    const eqCell = document.createElement("span");
    eqCell.innerHTML = katex.renderToString(s.eq, { throwOnError: false, displayMode: false });

    row.appendChild(opCell);
    row.appendChild(eqCell);

    if (s.note) {
      const noteRow = document.createElement("div");
      noteRow.style.cssText = "grid-column:2; font-size:0.82em; color:#6b7280; padding-bottom:0
      noteRow.textContent = s.note;
      row.appendChild(noteRow);
    }

    stepsDiv.appendChild(row);
  }

  if (current === totalSteps) {
    const done = document.createElement("div");
    done.style.cssText = "margin-top:0.5rem; font-size:0.9em; color:#059669; font-weight:500;";
    done.textContent = "□ Solution complete";
    stepsDiv.appendChild(done);
  }
}

const controls = document.createElement("div");
controls.style.cssText = "display:flex; gap:0.5rem; align-items:center;";

const nextBtn = document.createElement("button");
nextBtn.textContent = "Next step →";
nextBtn.style.cssText = "padding:0.42rem 0.95rem; border:1px solid #2563eb; border-radius:5px;
nextBtn.onclick = () => {
  if (current < totalSteps) { current++; renderSteps(); updateButtons(); }
};

const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset";
resetBtn.style.cssText = "padding:0.42rem 0.8rem; border:1px solid #cbd5e1; border-radius:5px;
resetBtn.onclick = () => { current = 0; renderSteps(); updateButtons(); };

const counter = document.createElement("span");
```

```
  counter.style.cssText = "font-size:0.82em; color:#9ca3af; margin-left:0.25rem;";

  function updateButtons() {
    nextBtn.disabled = current === totalSteps;
    nextBtn.style.opacity = current === totalSteps ? "0.4" : "1";
    counter.textContent = current === 0 ? "Click to reveal steps" : `Step ${current} of ${totalSteps
  }

  controls.appendChild(nextBtn);
  controls.appendChild(resetBtn);
  controls.appendChild(counter);

  container.appendChild(stepsDiv);
  container.appendChild(controls);

  renderSteps();
  updateButtons();
  return container;
}
```

### 59.8.1. Exercise 1

A thermal plant is modelled by

$$G(s) = \frac{2}{10s + 1}$$

with proportional controller $C(s) = K$ and unity feedback.

1. Derive the closed-loop transfer function.
2. Find the closed-loop pole.
3. For a unit-step reference, compute the steady-state error.
4. Compare what changes when $K = 1$ and when $K = 4$.

```
//| echo: false
makeStepperHTML(1, [
  { op: "Write the closed-loop formula",
    eq: "T(s) = \\dfrac{C(s)G(s)}{1 + C(s)G(s)} = \\dfrac{2K/(10s+1)}{1 + 2K/(10s+1)}",
    note: "Substitute the plant and controller directly into the unity-feedback formula." },
  { op: "Simplify the fraction",
    eq: "T(s) = \\dfrac{2K}{10s + 1 + 2K}",
    note: "Multiply numerator and denominator by 10s+1." },
  { op: "Read off the pole",
    eq: "10s + 1 + 2K = 0 \\quad \\Rightarrow \\quad s = -\\dfrac{1+2K}{10}",
    note: "The pole comes from the denominator." },
  { op: "Compute the steady-state gain",
    eq: "T(0) = \\dfrac{2K}{1+2K}",
```

```
    note: "Set s=0 to get the zero-frequency gain." },
  { op: "Compute the steady-state error for a unit step",
    eq: "e_{ss} = 1 - \\dfrac{2K}{1+2K} = \\dfrac{1}{1+2K}",
    note: "Output plus tracking error must sum to the unit reference." },
  { op: "Check the two gains",
    eq: "K=1: \\; s=-0.3, \\; e_{ss}=\\tfrac{1}{3} \\qquad K=4: \\; s=-0.9, \\; e_{ss}=\\tfrac{1}
    note: "Larger gain speeds the loop and reduces the residual error." }
])
```

### 59.8.2. Exercise 2

A two-state model has

$$A = \begin{pmatrix} 0 & 1 \\ -1 & -0.5 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \qquad K = \begin{pmatrix} 4 & 1.5 \end{pmatrix}$$

Compute $A - BK$ and decide whether the closed-loop system is stable.

```
//| echo: false
makeStepperHTML(2, [
  { op: "Write the closed-loop matrix",
    eq: "A-BK = \\begin{pmatrix} 0 & 1 \\\\ -1 & -0.5 \\end{pmatrix} - \\begin{pmatrix} 0 \\\\ 1
    note: "State feedback replaces A with A-BK." },
  { op: "Multiply BK",
    eq: "BK = \\begin{pmatrix} 0 & 0 \\\\ 4 & 1.5 \\end{pmatrix}",
    note: "The outer product fills only the second row because B has zero in its first entry." },
  { op: "Subtract",
    eq: "A-BK = \\begin{pmatrix} 0 & 1 \\\\ -5 & -2 \\end{pmatrix}" },
  { op: "Form the characteristic polynomial",
    eq: "\\det(\\lambda I - (A-BK)) = \\lambda^2 + 2\\lambda + 5",
    note: "Take the determinant of the 2x2 matrix \\lambda I-(A-BK)." },
  { op: "Find the eigenvalues",
    eq: "\\lambda = -1 \\pm 2i",
    note: "The real part is negative in both cases." },
  { op: "Check",
    eq: "\\Re(\\lambda) = -1 < 0 \\; \\Rightarrow \\; \\text{closed-loop system is stable}",
    note: "Negative real part means decaying oscillation rather than growing oscillation." }
])
```

### 59.8.3. Exercise 3

A lab instrument is modelled by a first-order plant $G(s) = a/(Ts + 1)$. The current proportional gain gives a closed-loop pole at $s = -0.2$. Following the same pole formula as Worked example 1, the operator proposes increasing the gain so the pole moves to $s = -0.8$.

Write a short design note answering:

1. What qualitative change will the lab see in the time response?
2. Why might this still be a bad idea if the sensor is noisy or delayed?

```
//| echo: false
makeStepperHTML(3, [
  { op: "Interpret the pole location",
    eq: "s=-0.8 \\text{ is farther left than } s=-0.2",
    note: "Farther left in the complex plane means faster exponential decay." },
  { op: "State the time-response consequence",
    eq: "\\text{settling becomes faster, so the instrument returns to target more quickly}",
    note: "A more negative real pole gives a shorter time constant." },
  { op: "State the risk",
    eq: "\\text{higher gain can amplify measurement noise and expose unmodelled delay}",
    note: "A controller that looks better in the simple model may behave worse in the real loop." },
  { op: "Check",
    eq: "\\text{better nominal speed does not guarantee better robustness}",
    note: "This is the main control tradeoff the design note should communicate." }
])
```

### 59.8.4. Exercise 4

Choose one system from your own field: a drive, drone, room heater, queueing server with autoscaling, or experimental instrument. Identify:

1. the reference
2. the measured output
3. the error signal
4. the control input
5. one reason high gain might help
6. one reason high gain might hurt

Write the answer as a one-page systems sketch, not as prose only.

*The diagram below shows the same unity-feedback loop relabelled for five different domains. Use it as a template for your sketch.*

```
//| echo: false
feedbackVocabViz = {
  const d3 = await require("d3@7");

  const W = 620, H = 260;
  const AMBER = "#f59e0b";

  const domains = {
    motor: {
      label: "Motor speed",
      ref: "target RPM",
      error: "speed error",
```

```
    ctrl: "drive current",
    dist: "load torque",
    out: "measured RPM"
  },
  drone: {
    label: "Drone altitude",
    ref: "target altitude",
    error: "altitude error",
    ctrl: "rotor thrust",
    dist: "wind gust",
    out: "measured altitude"
  },
  heater: {
    label: "Room heater",
    ref: "setpoint temperature",
    error: "temp error",
    ctrl: "heating power",
    dist: "heat loss",
    out: "measured temp"
  },
  instrument: {
    label: "Lab instrument",
    ref: "target intensity",
    error: "intensity error",
    ctrl: "actuator signal",
    dist: "thermal drift",
    out: "measured output"
  },
  server: {
    label: "Server autoscaling",
    ref: "target QPS",
    error: "load error",
    ctrl: "instance count",
    dist: "traffic spike",
    out: "measured QPS"
  }
};

const container = d3.create("div")
  .style("font-family", "sans-serif")
  .style("max-width", W + "px");

// Selector
const selRow = container.append("div")
  .style("display", "flex").style("align-items", "center")
  .style("gap", "0.75rem").style("margin-bottom", "0.5rem");
selRow.append("label").style("font-size", "0.9rem").text("Domain:");
```

```
const sel = selRow.append("select").style("font-size", "0.9rem").style("padding", "2px 6px");
Object.entries(domains).forEach(([k, v]) => {
  sel.append("option").attr("value", k).text(v.label);
});

const svg = container.append("svg")
  .attr("width", W).attr("height", H)
  .attr("viewBox", `0 0 ${W} ${H}`)
  .attr("aria-label", "Feedback vocabulary relabeller");

// Defs for arrowhead
const defs = svg.append("defs");
defs.append("marker")
  .attr("id", "arrowFV").attr("markerWidth", 8).attr("markerHeight", 8)
  .attr("refX", 6).attr("refY", 3).attr("orient", "auto")
  .append("path").attr("d", "M0,0 L0,6 L8,3 z").attr("fill", "#333");

// Fixed layout
const SJ = {x: 120, y: 120};
const CB = {x: 230, y: 100, w: 90, h: 40};
const PB = {x: 400, y: 100, w: 90, h: 40};
const outX = 540;
const fbY = 185;

// Draw static structure
function arrow(x1, y1, x2, y2) {
  svg.append("line")
    .attr("x1", x1).attr("y1", y1).attr("x2", x2).attr("y2", y2)
    .attr("stroke", "#374151").attr("stroke-width", 1.5)
    .attr("marker-end", "url(#arrowFV)");
}

// r -> SJ
arrow(40, SJ.y, SJ.x - 18, SJ.y);
// SJ -> CB
arrow(SJ.x + 18, SJ.y, CB.x, SJ.y);
// CB -> PB
arrow(CB.x + CB.w, SJ.y, PB.x, SJ.y);
// PB -> out
arrow(PB.x + PB.w, SJ.y, outX, SJ.y);
// feedback
svg.append("polyline")
  .attr("points", `${outX - 5},${SJ.y} ${outX - 5},${fbY} ${SJ.x},${fbY} ${SJ.x},${SJ.y + 18}`)
  .attr("fill", "none").attr("stroke", "#374151").attr("stroke-width", 1.5)
  .attr("marker-end", "url(#arrowFV)");

// Summing junction
```

```
svg.append("circle").attr("cx", SJ.x).attr("cy", SJ.y).attr("r", 18)
  .attr("fill", "none").attr("stroke", "#374151").attr("stroke-width", 1.5);
svg.append("text").attr("x", SJ.x - 6).attr("y", SJ.y + 5)
  .attr("font-size", "14px").attr("fill", "#374151").text("+");
svg.append("text").attr("x", SJ.x - 5).attr("y", SJ.y + 22)
  .attr("font-size", "11px").attr("fill", "#ef4444").text("−");

// Controller box
svg.append("rect").attr("x", CB.x).attr("y", CB.y).attr("width", CB.w).attr("height", CB.h)
  .attr("rx", 6).attr("fill", "#f3f4f6").attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
svg.append("text").attr("x", CB.x + CB.w / 2).attr("y", CB.y + CB.h / 2 + 5)
  .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#2563eb").text("Contro

// Plant box
svg.append("rect").attr("x", PB.x).attr("y", PB.y).attr("width", PB.w).attr("height", PB.h)
  .attr("rx", 6).attr("fill", "#f3f4f6").attr("stroke", "#9ca3af").attr("stroke-width", 1.5);
svg.append("text").attr("x", PB.x + PB.w / 2).attr("y", PB.y + PB.h / 2 + 5)
  .attr("text-anchor", "middle").attr("font-size", "13px").attr("fill", "#374151").text("Plant"

// Dynamic labels
const refLbl = svg.append("text").attr("x", 25).attr("y", SJ.y - 10)
  .attr("font-size", "11px").attr("text-anchor", "middle");
const errLbl = svg.append("text").attr("x", SJ.x + 55).attr("y", SJ.y - 10)
  .attr("font-size", "11px").attr("text-anchor", "middle");
const ctrlLbl = svg.append("text").attr("x", CB.x + CB.w + 45).attr("y", SJ.y - 10)
  .attr("font-size", "11px").attr("text-anchor", "middle");
const distLbl = svg.append("text").attr("x", (PB.x + PB.x + PB.w) / 2).attr("y", PB.y - 8)
  .attr("font-size", "11px").attr("text-anchor", "middle").attr("fill", "#6b7280");
const outLbl = svg.append("text").attr("x", outX + 5).attr("y", SJ.y + 5)
  .attr("font-size", "11px").attr("text-anchor", "start");

// Disturbance arrow
svg.append("line")
  .attr("x1", (PB.x + PB.x + PB.w) / 2).attr("y1", PB.y - 30)
  .attr("x2", (PB.x + PB.x + PB.w) / 2).attr("y2", PB.y)
  .attr("stroke", "#6b7280").attr("stroke-width", 1.2).attr("stroke-dasharray", "3,2")
  .attr("marker-end", "url(#arrowFV)");

let prevLabels = {};

function flashLabel(el, newText) {
  if (prevLabels[el.attr("class") || el.property("_id")] === newText) return;
  el.attr("fill", AMBER);
  el.text(newText);
  setTimeout(() => el.attr("fill", "#374151"), 800);
}
```

```
  function updateDomain(key) {
    const d = domains[key];
    flashLabel(refLbl, d.ref);
    flashLabel(errLbl, d.error);
    flashLabel(ctrlLbl, d.ctrl);
    flashLabel(distLbl, d.dist);
    flashLabel(outLbl, d.out);
    prevLabels = d;
  }

  sel.on("change", function() { updateDomain(this.value); });
  updateDomain("motor");

  return container.node();
}
```

# 60. Discrete-time systems and signal processing

Sampling, filtering, and recurrence

A sensor does not hand you a continuous function. It hands you numbers at intervals. A microphone gives a stream of samples. A controller updates once per cycle. Once time is sampled, the mathematics changes shape: derivatives become recurrences, spectra pick up aliasing, and filters become algorithms.

Upper-year engineering is full of systems that are modelled in continuous time but built in discrete time. A motor may obey a differential equation. The controller that stabilises it still runs on a clock. A seismic wave may be continuous in the ground. The data still arrives as a sampled trace.

Sampling turns a continuous object into a sequence, and once you have a sequence, the natural questions become delay, recurrence, convolution, spectrum, and stability. That structural shift — not signal processing vocabulary — is what this chapter is about.

---

## 60.1. The discrete-time view

A continuous-time signal is written $x(t)$. A discrete-time signal is written $x[n]$, read "x of n." The variable $n$ is not time itself. It is an index. The physical time attached to sample $n$ is

$$t_n = nT_s$$

where $T_s$ is the **sampling period**. The sampling frequency is

$$f_s = \frac{1}{T_s}$$

The same physical system can therefore be described in two ways:

- continuous-time: what the signal is doing between samples
- discrete-time: what the sampled sequence records at the clock instants

This sounds like a harmless change of notation. It is not. Once the signal is a sequence, operations that looked infinitesimal in continuous time become difference equations and delay operators.

If a discrete-time system takes an input sequence $x[n]$ and produces an output sequence $y[n]$, the simplest and most important class is the **linear time-invariant** (LTI) discrete system. Linear means the system obeys superposition. Time-invariant means the rule does not change between steps — the coefficients are fixed constants, not functions of $n$. For those systems, present output depends on present and past samples in a fixed way.

One common form is a **finite impulse response** (FIR) filter:

$$y[n] = \sum_{k=0}^{M} b_k x[n-k]$$

"Finite" here means the output depends on only a bounded window of $M + 1$ past inputs, so the filter's memory is bounded. The output at time step $n$ is a weighted combination of current and previous input samples.

Filters that also feed back past outputs — so that $y[n]$ depends on earlier $y$ values as well as earlier $x$ values — are called **infinite impulse response** (IIR) filters. Because the output feeds back into itself, a single impulse at the input can in principle affect every future output, giving the filter an unbounded memory. In stable IIR filters the impulse response decays geometrically, so the influence of distant past inputs becomes negligible in practice. IIR filters appear in the z-transform and state-space frameworks; the stability rules derived later in this chapter apply to them.

```
//| echo: false
{
  const width = 700, height = 230;
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 740px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; margin-bottom:0.75rem;";
  const stepWrap = document.createElement("label");
  stepWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  stepWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Window positio
  const step = document.createElement("input");
  step.type = "range"; step.min = "2"; step.max = "10"; step.step = "1"; step.value = "5";
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
  stepWrap.append(step, out);
  controls.appendChild(stepWrap);
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", `0 0 ${width} ${height}`);
  svg.setAttribute("width", "100%");
  svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10px
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";
```

```
  const xVals = [0,1,2,4,3,5,2,1,0,2,1,0];
  const hVals = [0.2,0.3,0.5];

  function rect(x,y,w,h,fill,stroke="#cbd5e1"){const r=svgEl("rect"); r.setAttribute("x",x); r.se
  function text(x,y,t,fill="#374151",size=12,anchor="middle",weight="400"){const n=svgEl("text");

  function redraw(){
    while(svg.firstChild) svg.removeChild(svg.firstChild);
    const n = parseInt(step.value,10);
    out.textContent = `${n}`;
    text(100, 28, "input sequence x[n]", "#374151", 12, "middle", "700");
    text(100, 122, "filter weights h[k]", "#374151", 12, "middle", "700");
    text(100, 202, "output sample y[n]", "#374151", 12, "middle", "700");
    for(let i=0;i<xVals.length;i++){
      const x = 150 + i*42;
      const active = i >= n-2 && i <= n;
      rect(x, 40, 34, 34, active ? "#dbeafe" : "#f8fafc", active ? "#2563eb" : "#cbd5e1");
      text(x+17, 62, `${xVals[i]}`, active ? "#1d4ed8" : "#374151", 12);
      text(x+17, 87, `${i}`, "#6b7280", 10);
    }
    for(let k=0;k<hVals.length;k++){
      const x = 150 + (n-2+k)*42;
      rect(x, 105, 34, 28, "#ecfdf5", "#059669");
      text(x+17, 123, `${hVals[k].toFixed(1)}`, "#047857", 11);
      const l=svgEl("line"); l.setAttribute("x1",x+17); l.setAttribute("x2",x+17); l.setAttribute
    }
    const y = hVals[0]*xVals[n-2] + hVals[1]*xVals[n-1] + hVals[2]*xVals[n];
    rect(150 + n*42 - 10, 182, 54, 28, "#fef3c7", "#f59e0b");
    text(150 + n*42 + 17, 200, `${y.toFixed(1)}`, "#b45309", 12, "middle", "700");
    text(430, 164, `y[${n}] = 0.2x[${n-2}] + 0.3x[${n-1}] + 0.5x[${n}]`, "#374151", 12, "middle")
    caption.textContent = "An FIR filter computes each output sample from a finite sliding window
  }

  step.addEventListener("input", redraw);
  container.append(controls, svg, caption);
  redraw();
  return container;
}
```

Use the controls to compare the real continuous oscillation with what the sampler can actually preserve.

```
//| echo: false
{
  const width = 640;
  const height = 340;
  const pad = { top: 24, right: 18, bottom: 48, left: 48 };
  const plotW = width - pad.left - pad.right;
```

```
const plotH = height - pad.top - pad.bottom;

const BLUE = "#2563eb";
const AMBER = "#f59e0b";
const RED = "#ef4444";
const GREEN = "#059669";
const GREY = "#6b7280";
const GRID = "#d1d5db";
const BG = "#f8fafc";

const duration = 1;
const nCurve = 900;

const wrapFreq = (f, fs) => {
  const wrapped = ((f % fs) + fs) % fs;
  return wrapped <= fs / 2 ? wrapped : fs - wrapped;
};

const makeSvg = (tag) => document.createElementNS("http://www.w3.org/2000/svg", tag);

const xScale = t => pad.left + (t / duration) * plotW;
const yScale = y => pad.top + plotH * (0.5 - 0.42 * y);

function linePath(points) {
  return points.map((p, i) => `${i === 0 ? "M" : "L"} ${xScale(p.t).toFixed(2)} ${yScale(p.y).toF
}

function samplesFor(f, fs) {
  const count = Math.floor(duration * fs) + 1;
  const pts = [];
  for (let n = 0; n < count; n += 1) {
    const t = n / fs;
    pts.push({ t, y: Math.sin(2 * Math.PI * f * t) });
  }
  return pts;
}

function curveFor(f) {
  const pts = [];
  for (let i = 0; i <= nCurve; i += 1) {
    const t = (i / nCurve) * duration;
    pts.push({ t, y: Math.sin(2 * Math.PI * f * t) });
  }
  return pts;
}

const container = document.createElement("div");
```

```
container.style.cssText = "max-width: 680px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

const controls = document.createElement("div");
controls.style.cssText = "display: grid; grid-template-columns: auto 1fr auto; gap: 0.5rem 0.75

const fLabel = document.createElement("label");
fLabel.textContent = "True frequency f (Hz)";
const fSlider = document.createElement("input");
fSlider.type = "range";
fSlider.min = "1";
fSlider.max = "20";
fSlider.step = "0.1";
fSlider.value = "3";
const fValue = document.createElement("span");
fValue.style.cssText = "font-variant-numeric: tabular-nums; min-width: 4.5rem;";

const fsLabel = document.createElement("label");
fsLabel.textContent = "Sampling frequency f_s (Hz)";
const fsSlider = document.createElement("input");
fsSlider.type = "range";
fsSlider.min = "4";
fsSlider.max = "40";
fsSlider.step = "1";
fsSlider.value = "12";
const fsValue = document.createElement("span");
fsValue.style.cssText = "font-variant-numeric: tabular-nums; min-width: 4.5rem;";

controls.append(fLabel, fSlider, fValue, fsLabel, fsSlider, fsValue);

const readout = document.createElement("div");
readout.style.cssText = "display: grid; grid-template-columns: repeat(5, minmax(0, 1fr)); gap:

const cards = Array.from({ length: 5 }, () => {
  const card = document.createElement("div");
  card.style.cssText = "background: #f8fafc; border: 1px solid #e5e7eb; border-radius: 8px; pad
  const k = document.createElement("div");
  k.style.cssText = "font-size: 0.76rem; color: #6b7280; margin-bottom: 0.15rem;";
  const v = document.createElement("div");
  v.style.cssText = "font-size: 0.96rem; font-variant-numeric: tabular-nums;";
  card.append(k, v);
  readout.append(card);
  return { card, key: k, value: v };
});

const [trueCard, sampleCard, nyquistCard, apparentCard, statusCard] = cards;
trueCard.key.textContent = "True f";
sampleCard.key.textContent = "Sampling f_s";
```

```
nyquistCard.key.textContent = "Nyquist";
apparentCard.key.textContent = "Apparent f";
statusCard.key.textContent = "Status";

const svg = makeSvg("svg");
svg.setAttribute("viewBox", `0 0 ${width} ${height}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display: block; background: white; border: 1px solid #e5e7eb; border-radius:

const bg = makeSvg("rect");
bg.setAttribute("x", "0");
bg.setAttribute("y", "0");
bg.setAttribute("width", width);
bg.setAttribute("height", height);
bg.setAttribute("fill", BG);
svg.appendChild(bg);

const gridGroup = makeSvg("g");
svg.appendChild(gridGroup);
for (let i = 0; i <= 10; i += 1) {
  const x = pad.left + (i / 10) * plotW;
  const line = makeSvg("line");
  line.setAttribute("x1", x);
  line.setAttribute("x2", x);
  line.setAttribute("y1", pad.top);
  line.setAttribute("y2", pad.top + plotH);
  line.setAttribute("stroke", GRID);
  line.setAttribute("stroke-width", "1");
  line.setAttribute("opacity", "0.7");
  gridGroup.appendChild(line);
}
[-1, -0.5, 0, 0.5, 1].forEach(v => {
  const y = yScale(v);
  const line = makeSvg("line");
  line.setAttribute("x1", pad.left);
  line.setAttribute("x2", pad.left + plotW);
  line.setAttribute("y1", y);
  line.setAttribute("y2", y);
  line.setAttribute("stroke", GRID);
  line.setAttribute("stroke-width", v === 0 ? "1.5" : "1");
  line.setAttribute("opacity", v === 0 ? "1" : "0.7");
  gridGroup.appendChild(line);
});

const sampleComb = makeSvg("g");
const apparentPath = makeSvg("path");
const truePath = makeSvg("path");
```

```
const pointGroup = makeSvg("g");
svg.append(sampleComb, apparentPath, truePath, pointGroup);

truePath.setAttribute("fill", "none");
truePath.setAttribute("stroke", BLUE);
truePath.setAttribute("stroke-width", "3");

apparentPath.setAttribute("fill", "none");
apparentPath.setAttribute("stroke-width", "2.5");
apparentPath.setAttribute("stroke-dasharray", "7 5");

const xAxis = makeSvg("line");
xAxis.setAttribute("x1", pad.left);
xAxis.setAttribute("x2", pad.left + plotW);
xAxis.setAttribute("y1", pad.top + plotH);
xAxis.setAttribute("y2", pad.top + plotH);
xAxis.setAttribute("stroke", GREY);
xAxis.setAttribute("stroke-width", "1.5");
svg.appendChild(xAxis);

const yAxis = makeSvg("line");
yAxis.setAttribute("x1", pad.left);
yAxis.setAttribute("x2", pad.left);
yAxis.setAttribute("y1", pad.top);
yAxis.setAttribute("y2", pad.top + plotH);
yAxis.setAttribute("stroke", GREY);
yAxis.setAttribute("stroke-width", "1.5");
svg.appendChild(yAxis);

const axisLabels = [
  { x: width / 2, y: height - 10, text: "time (s)", anchor: "middle" },
  { x: 16, y: pad.top + plotH / 2, text: "signal", anchor: "middle", rotate: true }
];
axisLabels.forEach(({ x, y, text, anchor, rotate }) => {
  const label = makeSvg("text");
  label.setAttribute("x", x);
  label.setAttribute("y", y);
  label.setAttribute("fill", GREY);
  label.setAttribute("font-size", "13");
  label.setAttribute("text-anchor", anchor);
  if (rotate) label.setAttribute("transform", `rotate(-90 ${x} ${y})`);
  label.textContent = text;
  svg.appendChild(label);
});

for (let i = 0; i <= 10; i += 1) {
  const x = pad.left + (i / 10) * plotW;
```

```
  const tick = makeSvg("text");
  tick.setAttribute("x", x);
  tick.setAttribute("y", pad.top + plotH + 18);
  tick.setAttribute("fill", GREY);
  tick.setAttribute("font-size", "12");
  tick.setAttribute("text-anchor", "middle");
  tick.textContent = (i / 10).toFixed(1);
  svg.appendChild(tick);
}

[-1, 0, 1].forEach(v => {
  const tick = makeSvg("text");
  tick.setAttribute("x", pad.left - 10);
  tick.setAttribute("y", yScale(v) + 4);
  tick.setAttribute("fill", GREY);
  tick.setAttribute("font-size", "12");
  tick.setAttribute("text-anchor", "end");
  tick.textContent = String(v);
  svg.appendChild(tick);
});

const legend = document.createElement("div");
legend.style.cssText = "display: flex; gap: 1rem; flex-wrap: wrap; margin-top: 0.55rem; font-size
const legendItem = (color, dash, text) => {
  const item = document.createElement("div");
  item.style.cssText = "display: inline-flex; align-items: center; gap: 0.45rem;";
  const swatch = document.createElement("span");
  swatch.style.cssText = `display:inline-block; width:1.6rem; height:0; border-top:3px ${dash ? "d
  const label = document.createElement("span");
  label.textContent = text;
  item.append(swatch, label);
  return item;
};
legend.append(
  legendItem(BLUE, false, "true continuous wave"),
  legendItem(RED, true, "apparent aliased wave"),
  legendItem(AMBER, false, "sampled values")
);

const note = document.createElement("p");
note.style.cssText = "font-size: 0.9rem; color: #374151; margin: 0.55rem 0 0 0;";

function render() {
  const f = parseFloat(fSlider.value);
  const fs = parseFloat(fsSlider.value);
  const nyquist = fs / 2;
  const apparent = wrapFreq(f, fs);
```

```
  const aliased = f > nyquist + 1e-9;

  fValue.textContent = `${f.toFixed(1)} Hz`;
  fsValue.textContent = `${fs.toFixed(0)} Hz`;

  trueCard.value.textContent = `${f.toFixed(1)} Hz`;
  sampleCard.value.textContent = `${fs.toFixed(0)} Hz`;
  nyquistCard.value.textContent = `${nyquist.toFixed(1)} Hz`;
  apparentCard.value.textContent = `${apparent.toFixed(1)} Hz`;
  statusCard.value.textContent = aliased ? "Aliasing" : "Safe";
  statusCard.value.style.color = aliased ? RED : GREEN;

  const truePts = curveFor(f);
  const apparentPts = curveFor(apparent);
  const samplePts = samplesFor(f, fs);

  truePath.setAttribute("d", linePath(truePts));
  apparentPath.setAttribute("d", linePath(apparentPts));
  apparentPath.setAttribute("stroke", aliased ? RED : BLUE);
  apparentPath.setAttribute("opacity", aliased ? "0.95" : "0.45");

  sampleComb.replaceChildren();
  pointGroup.replaceChildren();
  samplePts.forEach(pt => {
    const x = xScale(pt.t);
    const stem = makeSvg("line");
    stem.setAttribute("x1", x);
    stem.setAttribute("x2", x);
    stem.setAttribute("y1", yScale(0));
    stem.setAttribute("y2", yScale(pt.y));
    stem.setAttribute("stroke", AMBER);
    stem.setAttribute("stroke-width", "1.5");
    stem.setAttribute("opacity", "0.65");
    sampleComb.appendChild(stem);

    const circle = makeSvg("circle");
    circle.setAttribute("cx", x);
    circle.setAttribute("cy", yScale(pt.y));
    circle.setAttribute("r", "4");
    circle.setAttribute("fill", AMBER);
    circle.setAttribute("stroke", "#92400e");
    circle.setAttribute("stroke-width", "1");
    pointGroup.appendChild(circle);
  });

  note.textContent = aliased
    ? `Because f = ${f.toFixed(1)} Hz is above the Nyquist limit ${nyquist.toFixed(1)} Hz, the
```

```
      : `Because f = ${f.toFixed(1)} Hz stays below the Nyquist limit ${nyquist.toFixed(1)} Hz, the
  }

  fSlider.addEventListener("input", render);
  fsSlider.addEventListener("input", render);

  container.append(controls, readout, svg, legend, note);
  render();
  return container;
}
```

## 60.2. Why sampling can lie to you

Suppose the continuous signal is

$$x(t) = \sin(2\pi f t)$$

and you sample it every $T_s$ seconds. Then the discrete sequence is

$$x[n] = \sin(2\pi f n T_s) = \sin\left(2\pi \frac{f}{f_s} n\right)$$

The discrete sequence only sees the ratio $f/f_s$. Two different continuous frequencies can therefore produce sampled sequences that are indistinguishable, if the sampling frequency is too low. The samples no longer carry enough information to tell you which continuous oscillation was actually present. This means aliasing is not a small numerical error. It is a structural loss of information created at the moment of sampling.

The **Nyquist–Shannon sampling theorem** states the safe condition:

- to represent frequency content up to $f_{\max}$, sample faster than $2f_{\max}$

That does not solve every practical problem, but it tells you where obvious failure begins. Sampling below that rate invites aliasing: high-frequency content reappears as a false lower-frequency pattern in the data.

This is one of the reasons discrete-time systems need their own mathematics. The sampled sequence is not a transparent window into the continuous signal. It is a transformed object with its own constraints.

## 60.3. Difference equations and recurrence

Continuous-time linear systems are often described by differential equations. Discrete-time linear systems are naturally described by **difference equations**.

A first-order example is

$$y[n+1] = ay[n] + bu[n]$$

This says: the next output is built from the current output and the current input. The constant $a$ carries memory. The constant $b$ tells you how strongly the input influences the update.

The stability rule for the homogeneous system

$$y[n+1] = ay[n]$$

is very simple:

- if $|a| < 1$, the sequence decays to zero
- if $|a| > 1$, the sequence grows in magnitude
- if $|a| = 1$, the sequence neither decays nor grows and small modelling changes can matter a great deal

This is the discrete-time analogue of looking at pole location in continuous time. In one case you care about left-half-plane location. In the other you care about whether the relevant quantity sits inside the unit circle.

> **Why this works**
>
> In continuous time, a mode behaves like $e^{st}$, so the sign of $\Re(s)$ decides whether it decays or grows. In discrete time, a mode behaves like $a^n$, so the magnitude of $a$ decides whether repeated updates shrink or blow up. Exponentials become powers, and left-half-plane stability becomes unit-circle stability.

```
//| echo: false
{
  const width = 700, height = 280;
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 740px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; margin-bottom:0.75rem;";
  const wrap = document.createElement("label");
  wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  wrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Mode location s"
  const slider = document.createElement("input");
  slider.type = "range"; slider.min = "-1.4"; slider.max = "1.4"; slider.step = "0.02"; slider.va
  const out = document.createElement("span");
```

```
out.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
wrap.append(slider, out);
controls.appendChild(wrap);
const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${width} ${height}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10px
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function redraw(){
  while(svg.firstChild) svg.removeChild(svg.firstChild);
  const a = parseFloat(slider.value);
  out.textContent = a.toFixed(2);
  const left = { x: 40, y: 30, w: 260, h: 220 };
  const right = { x: 380, y: 30, w: 260, h: 220 };
  const mapSx = x => left.x + left.w * (x + 2) / 4;
  const mapSy = y => left.y + left.h * (1 - (y + 2) / 4);
  const mapZx = x => right.x + right.w * (x + 1.6) / 3.2;
  const mapZy = y => right.y + right.h * (1 - (y + 1.6) / 3.2);
  const text=(x,y,t,fill="#374151",size=12,anchor="middle",weight="400")=>{const n=svgEl("text");
  const line=(x1,y1,x2,y2,stroke,w=1.5,dash="")=>{const l=svgEl("line"); l.setAttribute("x1",x1);
  text(left.x + left.w/2, 20, "continuous-time s-plane", "#374151", 12, "middle", "700");
  line(mapSx(0), mapSy(-2), mapSx(0), mapSy(2), "#9ca3af", 1.8, "5 4");
  line(mapSx(-2), mapSy(0), mapSx(1), mapSy(0), "#6b7280", 1.5);
  const stableShade = svgEl("rect");
  stableShade.setAttribute("x", mapSx(-2)); stableShade.setAttribute("y", mapSy(2)); stableShade.s
  stableShade.setAttribute("fill","rgba(5,150,105,0.12)");
  svg.appendChild(stableShade);
  const pole = svgEl("circle"); pole.setAttribute("cx", mapSx(a)); pole.setAttribute("cy", mapSy(
  text(mapSx(-1), left.y + left.h + 18, "stable: Re(s) < 0", "#059669", 12);
  text(mapSx(a), mapSy(0) - 10, `s = ${a.toFixed(2)}`, "#2563eb", 11);

  text(right.x + right.w/2, 20, "discrete-time z-plane", "#374151", 12, "middle", "700");
  line(mapZx(-1.6), mapZy(0), mapZx(1.6), mapZy(0), "#6b7280", 1.5);
  line(mapZx(0), mapZy(-1.6), mapZx(0), mapZy(1.6), "#6b7280", 1.5);
  const unit = svgEl("circle"); unit.setAttribute("cx", mapZx(0)); unit.setAttribute("cy", mapZy(
  const z = Math.exp(a);
  const mode = svgEl("circle"); mode.setAttribute("cx", mapZx(z)); mode.setAttribute("cy", mapZy(
  text(right.x + right.w/2, right.y + right.h + 18, "stable: |z| < 1", "#059669", 12);
  text(mapZx(z), mapZy(0) - 10, `z ≈ ${z.toFixed(2)}`, "#b45309", 11);
  caption.textContent = "In continuous time, decay means the mode sits left of the imaginary axis
}

slider.addEventListener("input", redraw);
container.append(controls, svg, caption);
redraw();
```

```
  return container;
}
```

## 60.4. The core method

A first pass through a discrete-time systems problem usually goes like this:

1. Identify the sampling period $T_s$ and therefore the sampling frequency $f_s$.
2. Decide whether the data is a sampled version of a continuous signal, a genuinely discrete process, or a digital controller state.
3. Write the update rule or filter equation.
4. Interpret every term as memory, delay, or input contribution.
5. Check the relevant stability test: for the first-order case, the coefficient magnitude must be less than 1. For higher-order systems, stability requires all poles — the roots of the system's characteristic equation — to lie inside the unit circle in the complex plane. This is the discrete-time counterpart of the left-half-plane test from continuous-time control.
6. Connect the algebra back to behaviour: smoothing, lag, oscillation, alias, or amplification.

The habit is the same one from the control chapter: do not leave the recurrence as pure symbol manipulation. Ask what it means for the sequence the machine is actually computing.

## 60.5. Worked example 1: a moving-average sensor filter

A noisy sensor signal $x[n]$ is filtered by

$$y[n] = \frac{1}{3}\big(x[n] + x[n-1] + x[n-2]\big)$$

This is a three-point moving average.

It smooths rapid sample-to-sample variation by replacing the present sample with the average of the present and previous two samples. If one sample spikes upward because of noise, the output spike is diluted by the neighbouring samples.

The price is delay. A filter that averages over time cannot react instantly to a real rapid change. This is the first recurring engineering tradeoff in discrete-time signal processing:

- more smoothing usually means more lag
- less lag usually means less smoothing

In embedded control, that tradeoff matters because a laggy filter can make an otherwise good controller feel slow or even unstable.

## 60.6. Worked example 2: discrete-time stability of a sampled update

Consider the update rule

$$y[n+1] = 0.6\, y[n] + u[n]$$

If the input is zero, then

$$y[n+1] = 0.6\, y[n]$$

Starting from $y[0]$, repeated substitution gives

$$y[n] = 0.6^n y[0]$$

Because $|0.6| < 1$, the homogeneous response decays to zero. So the system's memory fades over time.

Now compare with

$$y[n+1] = 1.1\, y[n] + u[n]$$

With zero input, the response is

$$y[n] = 1.1^n y[0]$$

and because $|1.1| > 1$, the sequence grows. A digital implementation with this update law is unstable.

In first-order discrete-time systems, the entire stability story is often carried by one number.

## 60.7. Worked example 3: aliasing in a seismic record

A geophysicist samples ground motion at $f_s = 100$ Hz. A strong sinusoidal component is present at $f = 70$ Hz.

This is above the Nyquist limit of

$$\frac{f_s}{2} = 50 \text{ Hz}$$

so the signal will alias. The discrete record cannot faithfully represent a 70 Hz oscillation at that sampling rate. Because the sampler runs at 100 Hz, a frequency $f$ that exceeds the Nyquist limit folds back by the amount it overshoots the full sampling rate: the alias frequency is $|f - f_s| = |70 - 100| = 30$ Hz. In the sampled record, the 70 Hz component appears as a spurious 30 Hz oscillation.

The engineering lesson is not "compute a formula and move on." It is that data collection itself is part of the mathematics. Sampling choices determine what questions the later analysis can answer honestly.

This is why signal processing belongs naturally beside estimation and inverse problems in Volume 8. If the measurement process already lost information, no later inference method can magically restore it.

## 60.8. Where this goes

The most direct continuation is Estimation, inverse problems, and filtering. Once a system is sampled, state estimation, filtering, and inference are usually carried out in discrete time. Kalman filtering, recursive estimation, and streaming data assimilation all inherit the language of recurrences, sampling, and noise that this chapter sets up.

This chapter sits beside control deliberately. Upper-year engineers constantly move between the continuous-time model of the plant and the discrete-time implementation of the controller. The control chapter is about shaping behaviour in continuous time. This one is about what happens when that shaping must be done on a clock.

**Applications**

- digital filters for noisy sensors
- sampled-data control in motors and robotics
- audio and vibration signal analysis
- seismic and geophysical trace processing
- preprocessing streaming time-series data
- telemetry, embedded systems, and real-time monitoring

## 60.9. Exercises

These are project-style exercises. Explain the behaviour as well as the calculation.

### 60.9.1. Exercise 1

A sensor is sampled every 0.01 s.

1. Find the sampling frequency.
2. Find the Nyquist frequency.
3. Decide whether a 30 Hz sinusoid can be represented without aliasing.
4. Decide whether a 70 Hz sinusoid can be represented without aliasing.

### 60.9.2. Exercise 2

Consider the discrete-time system

$$y[n+1] = 0.8\, y[n] + 2u[n]$$

1. Determine whether the homogeneous system is stable.
2. Starting from $y[0] = 5$ with $u[n] = 0$, compute $y[1]$, $y[2]$, and $y[3]$.
3. Explain what the sequence is doing physically.

### 60.9.3. Exercise 3

A noisy accelerometer is filtered using

$$y[n] = \frac{1}{4}\big(x[n] + x[n-1] + x[n-2] + x[n-3]\big)$$

Write a short engineering memo answering:

1. What kind of behaviour this filter is trying to suppress
2. Why it introduces lag
3. Why that lag could matter in a feedback controller

### 60.9.4. Exercise 4

Choose one sampled-data setting from your field: a microphone, IMU, stock-price stream, satellite time series, or power-system sensor.

Prepare a one-page systems sketch naming:

1. the physical quantity being sampled
2. a plausible sampling frequency
3. one source of aliasing or information loss
4. one filter or recurrence you might apply
5. one reason filtering helps
6. one reason filtering can distort what you care about

Use the sandbox below to compare smoothing against delay directly.

```
//| echo: false
{
  const N = 40;
  const width = 700, height = 280;
  const pad = { top: 22, right: 18, bottom: 40, left: 44 };
  const plotW = width - pad.left - pad.right;
  const plotH = height - pad.top - pad.bottom;

  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width: 740px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-botto
  const kindWrap = document.createElement("label");
  kindWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  kindWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Filter" }));
  const kind = document.createElement("select");
  kind.style.cssText = "padding:0.35rem 0.55rem; border:1px solid #d1d5db; border-radius:8px; backgr
  ["3-point moving average", "5-point moving average"].forEach(v => {
```

```
  const opt = document.createElement("option");
  opt.value = v; opt.textContent = v;
  kind.appendChild(opt);
});
kindWrap.appendChild(kind);
const noiseWrap = document.createElement("label");
noiseWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
noiseWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Noise level
const noise = document.createElement("input");
noise.type = "range"; noise.min = "0"; noise.max = "1.5"; noise.step = "0.05"; noise.value = "0
const noiseVal = document.createElement("span");
noiseVal.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
noiseWrap.append(noise, noiseVal);
controls.append(kindWrap, noiseWrap);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
  k.textContent = label;
  const v = document.createElement("div");
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const lagBox = card("Estimated lag");
const noiseBox = card("Residual roughness");
const gainBox = card("Main effect");

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${width} ${height}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function redraw() {
  noiseVal.textContent = parseFloat(noise.value).toFixed(2);
  while (svg.firstChild) svg.removeChild(svg.firstChild);
  const raw = [];
  const trueSignal = [];
  for (let i = 0; i < N; i += 1) {
    const base = 1.3 * Math.sin(0.28 * i) + 0.6 * Math.sin(0.9 + 0.08 * i);
```

```
    const spike = i > 20 ? 0.9 : 0;
    const trueVal = base + spike;
    const noisy = trueVal + parseFloat(noise.value) * (Math.sin(1.97 * i + 0.4) + 0.5 * Math.cos(2
    trueSignal.push(trueVal);
    raw.push(noisy);
  }
  const window = kind.value.startsWith("3") ? 3 : 5;
  const filtered = raw.map((_, i) => {
    let s = 0, c = 0;
    for (let j = Math.max(0, i - window + 1); j <= i; j += 1) { s += raw[j]; c += 1; }
    return s / c;
  });
  const ymin = Math.min(...raw, ...filtered, ...trueSignal) - 0.4;
  const ymax = Math.max(...raw, ...filtered, ...trueSignal) + 0.4;
  const sx = i => pad.left + plotW * i / (N - 1);
  const sy = y => pad.top + plotH * (1 - (y - ymin) / (ymax - ymin));
  for (let i = 0; i < N; i += 5) {
    const line = svgEl("line");
    line.setAttribute("x1", sx(i)); line.setAttribute("x2", sx(i));
    line.setAttribute("y1", sy(ymin)); line.setAttribute("y2", sy(ymax));
    line.setAttribute("stroke", "#f3f4f6"); svg.appendChild(line);
  }
  const axisX = svgEl("line");
  axisX.setAttribute("x1", sx(0)); axisX.setAttribute("x2", sx(N - 1)); axisX.setAttribute("y1", s
  axisX.setAttribute("stroke", "#6b7280"); svg.appendChild(axisX);
  const truthPath = svgEl("polyline");
  truthPath.setAttribute("points", trueSignal.map((v, i) => `${sx(i)},${sy(v)}`).join(" "));
  truthPath.setAttribute("fill", "none"); truthPath.setAttribute("stroke", "#9ca3af"); truthPath.s
  svg.appendChild(truthPath);
  raw.forEach((v, i) => {
    const dot = svgEl("circle");
    dot.setAttribute("cx", sx(i)); dot.setAttribute("cy", sy(v)); dot.setAttribute("r", "2.7"); do
    svg.appendChild(dot);
  });
  const filtPath = svgEl("polyline");
  filtPath.setAttribute("points", filtered.map((v, i) => `${sx(i)},${sy(v)}`).join(" "));
  filtPath.setAttribute("fill", "none"); filtPath.setAttribute("stroke", "#f59e0b"); filtPath.setA
  svg.appendChild(filtPath);
  const label = (x, y, t, fill) => { const n = svgEl("text"); n.setAttribute("x", x); n.setAttribu
  label(74, 28, "true signal", "#6b7280");
  label(170, 28, "raw samples", "#2563eb");
  label(285, 28, "filtered output", "#b45309");

  const roughness = filtered.slice(1).reduce((s, v, i) => s + Math.abs(v - filtered[i]), 0) / (N -
  lagBox.textContent = `${((window - 1) / 2).toFixed(1)} samples`;
  noiseBox.textContent = roughness.toFixed(2);
  gainBox.textContent = window === 3 ? "less lag, less smoothing" : "more smoothing, more delay";
```

```
    caption.textContent = "The raw points are noisy, the amber line is the moving average, and th
  }

  [kind, noise].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, readout, svg, caption);
  redraw();
  return container;
}
```

# 61. Continuum mechanics and transport

Field laws for stress, flow, heat, and diffusion

Heat leaves a wall, stress moves through a beam, solute spreads through a river, and pressure drives flow through a pipe. Different materials, different scales, different equations on the page. Underneath, the same move repeats: write what is conserved, write how the material responds, and let the field describe the rest.

Structural analysis, heat transfer, porous flow, and contaminant transport do not share the same units or applications. They share the same mathematical architecture.

The architecture has two parts. A **conservation law** says something cannot appear or disappear except through flow across a boundary or production inside the region. A **constitutive law** says how the material responds: how heat flows down a temperature gradient, how solute diffuses down a concentration gradient, how stress relates to strain in a simple elastic solid.

---

## 61.1. The local balance law

Take a small control volume inside a body, pipe, soil column, or fluid region. Let $u$ be the amount of something stored per unit volume. Depending on the problem, $u$ might be thermal energy, mass concentration, momentum density, or another conserved quantity.

Let $\mathbf{q}$ be the **flux** of that quantity. Flux is a vector quantity: its magnitude measures how much passes through a unit area per unit time, and its direction is the direction of net transport.

The generic local balance law is

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{q} = s$$

where:

- $\partial u/\partial t$ is the local rate of storage
- $\nabla \cdot \mathbf{q}$ is the net outward flux
- $s$ is a source term

This one line is the template behind most continuum modelling. Change the meaning of $u$, $\mathbf{q}$, and $s$, and you get a different physical system. The bookkeeping structure stays the same. This means the conservation architecture is reusable: once you understand the balance law, many different physical theories become variations on one pattern.

The divergence term matters because a continuum model is local. It does not ask "how much crossed the outer boundary of the whole pipe?" first. It asks what is happening in an arbitrarily small piece. Once the local law is right, the global law follows by integration.

Use the controls to see the same balance law operate across several physical interpretations.

```
//| echo: false
{
  const W = 620, H = 300;
  let inflow = 6;
  let outflow = 4;
  let source = 1;
  let mode = "heat";

  const MODES = {
    heat: { store: "thermal energy", fluxIn: "heat in", fluxOut: "heat out", source: "heater", units
    solute: { store: "solute mass", fluxIn: "solute in", fluxOut: "solute out", source: "reaction so
    fluid: { store: "fluid mass", fluxIn: "mass in", fluxOut: "mass out", source: "injection", units
  };

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  const clamp = (v, a, b) => Math.max(a, Math.min(b, v));
  const arrowWidth = v => 2 + 2.2 * Math.abs(v);

  const container = document.createElement("div");
  container.style.cssText = "max-width: 680px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem; a
  function sliderRow(labelText, min, max, value) {
    const label = document.createElement("label");
    label.textContent = labelText;
    const input = document.createElement("input");
    input.type = "range";
    input.min = String(min);
    input.max = String(max);
    input.step = "0.5";
    input.value = String(value);
    const out = document.createElement("span");
    out.style.cssText = "font-variant-numeric: tabular-nums; min-width:3.6rem;";
    controls.append(label, input, out);
```

```
  return { input, out };
}
const inRow = sliderRow("Inflow", 0, 10, inflow);
const outRow = sliderRow("Outflow", 0, 10, outflow);
const srcRow = sliderRow("Source", -4, 4, source);

const modeRow = document.createElement("div");
modeRow.style.cssText = "display:flex; gap:0.5rem; align-items:center; margin-bottom:0.65rem;";
const modeLabel = document.createElement("span");
modeLabel.textContent = "Mode";
const select = document.createElement("select");
select.style.cssText = "padding:0.35rem 0.5rem; border:1px solid #d1d5db; border-radius:8px; ba
[["heat","Heat"],["solute","Solute"],["fluid","Fluid mass"]].forEach(([v,l]) => {
  const opt = document.createElement("option");
  opt.value = v; opt.textContent = l;
  select.appendChild(opt);
});
select.value = mode;
modeRow.append(modeLabel, select);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
  k.textContent = label;
  const v = document.createElement("div");
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const eqBox = card("Balance");
const rateBox = card("∂u/∂t");
const stateBox = card("Interpretation");

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:white; border:1px solid #e5e7eb; border-radius:1

const square = svgEl("rect");
square.setAttribute("x", "200");
square.setAttribute("y", "70");
square.setAttribute("width", "160");
```

```
  square.setAttribute("height", "120");
  square.setAttribute("fill", "#f8fafc");
  square.setAttribute("stroke", "#6b7280");
  square.setAttribute("stroke-width", "2");
  svg.appendChild(square);

  const barBg = svgEl("rect");
  barBg.setAttribute("x", "470");
  barBg.setAttribute("y", "70");
  barBg.setAttribute("width", "42");
  barBg.setAttribute("height", "120");
  barBg.setAttribute("fill", "#f3f4f6");
  barBg.setAttribute("stroke", "#9ca3af");
  barBg.setAttribute("stroke-width", "1.5");
  svg.appendChild(barBg);

  const barFill = svgEl("rect");
  svg.appendChild(barFill);

  const labels = {};
  ["center","store","bar","in","out","src"].forEach(k => {
    labels[k] = svgEl("text");
    labels[k].setAttribute("font-size", "13");
    labels[k].setAttribute("fill", "#374151");
    svg.appendChild(labels[k]);
  });
  labels.center.setAttribute("x", "280");
  labels.center.setAttribute("y", "135");
  labels.center.setAttribute("text-anchor", "middle");
  labels.center.setAttribute("font-weight", "700");

  labels.store.setAttribute("x", "280");
  labels.store.setAttribute("y", "155");
  labels.store.setAttribute("text-anchor", "middle");
  labels.store.setAttribute("font-size", "12");
  labels.store.setAttribute("fill", "#6b7280");

  labels.bar.setAttribute("x", "491");
  labels.bar.setAttribute("y", "206");
  labels.bar.setAttribute("text-anchor", "middle");
  labels.bar.setAttribute("font-size", "12");
  labels.bar.setAttribute("fill", "#6b7280");

  function makeArrow(color) {
    const g = svgEl("g");
    const line = svgEl("line");
    const tip1 = svgEl("line");
```

```
  const tip2 = svgEl("line");
  [line, tip1, tip2].forEach(n => {
    n.setAttribute("stroke", color);
    n.setAttribute("stroke-linecap", "round");
    g.appendChild(n);
  });
  svg.appendChild(g);
  return { g, line, tip1, tip2 };
}
const inArrow = makeArrow("#2563eb");
const outArrow = makeArrow("#f59e0b");
const srcArrow = makeArrow("#7c3aed");

const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function setArrow(obj, x1, y1, x2, y2, width) {
  obj.line.setAttribute("x1", x1);
  obj.line.setAttribute("y1", y1);
  obj.line.setAttribute("x2", x2);
  obj.line.setAttribute("y2", y2);
  obj.line.setAttribute("stroke-width", width);
  const dx = x2 - x1, dy = y2 - y1;
  const len = Math.hypot(dx, dy) || 1;
  const ux = dx / len, uy = dy / len;
  const bx = x2 - 12 * ux, by = y2 - 12 * uy;
  obj.tip1.setAttribute("x1", x2);
  obj.tip1.setAttribute("y1", y2);
  obj.tip1.setAttribute("x2", bx - 6 * uy);
  obj.tip1.setAttribute("y2", by + 6 * ux);
  obj.tip1.setAttribute("stroke-width", width);
  obj.tip2.setAttribute("x1", x2);
  obj.tip2.setAttribute("y1", y2);
  obj.tip2.setAttribute("x2", bx + 6 * uy);
  obj.tip2.setAttribute("y2", by - 6 * ux);
  obj.tip2.setAttribute("stroke-width", width);
}

function redraw() {
  inflow = parseFloat(inRow.input.value);
  outflow = parseFloat(outRow.input.value);
  source = parseFloat(srcRow.input.value);
  mode = select.value;

  inRow.out.textContent = inflow.toFixed(1);
  outRow.out.textContent = outflow.toFixed(1);
  srcRow.out.textContent = source.toFixed(1);
```

```
  const acc = inflow - outflow + source;
  const m = MODES[mode];

  labels.center.textContent = "control volume";
  labels.store.textContent = m.store;
  labels.bar.textContent = "storage";
  labels.in.setAttribute("x", "115");
  labels.in.setAttribute("y", "120");
  labels.in.setAttribute("text-anchor", "middle");
  labels.in.textContent = m.fluxIn;
  labels.out.setAttribute("x", "445");
  labels.out.setAttribute("y", "120");
  labels.out.setAttribute("text-anchor", "middle");
  labels.out.textContent = m.fluxOut;
  labels.src.setAttribute("x", "280");
  labels.src.setAttribute("y", "48");
  labels.src.setAttribute("text-anchor", "middle");
  labels.src.textContent = m.source;

  setArrow(inArrow, 80, 130, 198, 130, arrowWidth(inflow));
  setArrow(outArrow, 362, 130, 440, 130, arrowWidth(outflow));
  if (source >= 0) {
    setArrow(srcArrow, 280, 30, 280, 68, arrowWidth(source));
  } else {
    setArrow(srcArrow, 280, 68, 280, 30, arrowWidth(source));
  }

  const fillH = 25 + 95 * clamp((acc + 10) / 20, 0, 1);
  barFill.setAttribute("x", "470");
  barFill.setAttribute("y", 190 - fillH);
  barFill.setAttribute("width", "42");
  barFill.setAttribute("height", fillH);
  barFill.setAttribute("fill", acc >= 0 ? "rgba(245,158,11,0.65)" : "rgba(239,68,68,0.65)");

  eqBox.textContent = `${inflow.toFixed(1)} - ${outflow.toFixed(1)} + ${source.toFixed(1)}`;
  rateBox.textContent = `${acc >= 0 ? "+" : ""}${acc.toFixed(1)} ${m.units}`;
  rateBox.style.color = acc >= 0 ? "#b45309" : "#b91c1c";
  stateBox.textContent = acc > 0 ? "storage rises" : acc < 0 ? "storage falls" : "balanced";
  caption.textContent = `In one-dimensional bookkeeping form, the local balance reads accumulatio
}

[inRow.input, outRow.input, srcRow.input].forEach(el => el.addEventListener("input", redraw));
select.addEventListener("change", redraw);

container.append(controls, modeRow, readout, svg, caption);
redraw();
return container;
```

```
}
```

## 61.2. Constitutive laws

A balance law tells you what must be accounted for. It does not say how the material responds. That extra relation is the constitutive law.

Two standard examples are:

**Fourier's law of heat conduction**

$$\mathbf{q} = -k\nabla T$$

Heat flux points down the temperature gradient. The constant $k$ is thermal conductivity. The minus sign means heat flows from hot to cold.

**Fick's law of diffusion**

$$\mathbf{J} = -D\nabla c$$

Mass flux points down the concentration gradient. The constant $D$ is the diffusivity.

The structure is the same in both. A gradient creates a driving force. The material constant tells you how strongly the medium responds.

In elementary elasticity, the constitutive law is often written as a linear stress-strain relation. In one dimension,

$$\sigma = E\varepsilon$$

where $\sigma$ is **stress** (force per unit area, Pa), $\varepsilon$ is **strain** (fractional change in length, dimensionless), and $E$ is Young's modulus (the material stiffness).

The structure is the same: one field quantity is related to another by a material response parameter.

This 1D relation extends to three dimensions as a tensor equation relating the full strain tensor to the full stress tensor through a stiffness matrix. The scalar form here captures the logic; discipline courses in mechanics of materials and structural analysis build out the three-dimensional version.

## 61.3. Why signs and directions matter

Continuum laws are full of minus signs, outward normals, and directional derivatives. These are not decorative details. They are the physics.

If temperature increases to the right, then $\nabla T$ points right. Fourier's law says the heat flux points left. Miss the minus sign and you literally model heat flowing from cold to hot.

The same applies to every constitutive law in this chapter. Continuum mathematics is geometry with units attached. Direction is part of the answer.

```
//| echo: false
{
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 720px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; margin-bottom:0.75rem;";
  const wrap = document.createElement("label");
  wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  wrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Field slope" }));
  const slider = document.createElement("input");
  slider.type = "range"; slider.min = "-2"; slider.max = "2"; slider.step = "0.1"; slider.value = "1
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
  wrap.append(slider, out);
  controls.appendChild(wrap);
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 680 240");
  svg.setAttribute("width", "100%");
  svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10px
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

  function redraw(){
    while(svg.firstChild) svg.removeChild(svg.firstChild);
    const m = parseFloat(slider.value);
    out.textContent = m.toFixed(1);
    const text=(x,y,t,fill="#374151",size=12,anchor="middle",weight="400")=>{const n=svgEl("text");
    const line=(x1,y1,x2,y2,stroke,w=1.5,dash="")=>{const l=svgEl("line"); l.setAttribute("x1",x1);
    const rect=svgEl("rect"); rect.setAttribute("x","70"); rect.setAttribute("y","72"); rect.setAtti
    const leftVal = 1.5 + m, rightVal = 1.5 - m;
    text(70, 58, leftVal > rightVal ? "high field" : "low field", "#374151", 11, "start");
    text(490, 58, leftVal > rightVal ? "low field" : "high field", "#374151", 11, "end");
    line(90, 90, 470, 90 - 18*m, "#2563eb", 3);
    text(280, 52, "field profile", "#2563eb", 12);
    const dir = m > 0 ? 1 : m < 0 ? -1 : 0;
    if (dir !== 0) {
```

```
      line(300, 150, 300 + 90*dir, 150, "#2563eb", 3);
      line(300 + 90*dir, 150, 300 + 90*dir - 10*dir, 145, "#2563eb", 3);
      line(300 + 90*dir, 150, 300 + 90*dir - 10*dir, 155, "#2563eb", 3);
      line(300, 190, 300 - 90*dir, 190, "#ef4444", 3);
      line(300 - 90*dir, 190, 300 - 90*dir + 10*dir, 185, "#ef4444", 3);
      line(300 - 90*dir, 190, 300 - 90*dir + 10*dir, 195, "#ef4444", 3);
    }
    text(300, 172, m > 0 ? "gradient points right" : m < 0 ? "gradient points left" : "zero gradi
    text(300, 212, dir !== 0 ? "flux points opposite" : "no net flux", "#b91c1c", 12);
    text(560, 96, "q = -k∇T", "#374151", 13);
    caption.textContent = "When the field rises to the right, the gradient points right. Fourier'
  }

  slider.addEventListener("input", redraw);
  container.append(controls, svg, caption);
  redraw();
  return container;
}
```

**Where the PDE comes from**

Conservation laws come from bookkeeping on a region. Constitutive laws come from empirical or theoretical descriptions of how a material responds. The PDE appears when you combine them.
For example, if thermal energy density is proportional to temperature and heat flux obeys Fourier's law, then storage plus divergence of heat flux produces the heat equation. The PDE is not the starting point. It is the compressed result of conservation plus material behaviour.

## 61.4. The core method

A first pass through a continuum or transport problem usually goes like this:

1. Identify the conserved quantity: mass, energy, momentum, or solute.
2. Write the local balance law for storage, flux, and source.
3. Choose the constitutive law that links flux or stress to the driving field.
4. Substitute the constitutive law into the balance law.
5. Inspect the resulting PDE or field equation and connect each term back to a physical mechanism.
6. Add boundary and initial conditions only after the governing structure is clear.

That last point matters. Students often see the PDE first and the modelling logic second. In upper-year work, reversing that order helps. If you know where every term came from, the equation stops feeling arbitrary.

## 61.5. Worked example 1: heat conduction in one dimension

Let $T(x,t)$ be temperature in a rod. Suppose there is no internal heat source. The local energy balance says:

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{q} = 0$$

In one dimension, thermal energy per unit volume is $u = \rho c_p T$, where $\rho$ is mass density (kg m$^{-3}$) and $c_p$ is specific heat capacity (J kg$^{-1}$ K$^{-1}$). Substituting into the generic balance law gives

$$\rho c_p \frac{\partial T}{\partial t} + \frac{\partial q}{\partial x} = 0$$

Use Fourier's law in one dimension:

$$q = -k\frac{\partial T}{\partial x}$$

Substitute:

$$\rho c_p \frac{\partial T}{\partial t} - \frac{\partial}{\partial x}\left(k\frac{\partial T}{\partial x}\right) = 0$$

If $k$ is constant, this simplifies to

$$\rho c_p \frac{\partial T}{\partial t} = k\frac{\partial^2 T}{\partial x^2}$$

or

$$\frac{\partial T}{\partial t} = \alpha\frac{\partial^2 T}{\partial x^2}, \qquad \alpha = \frac{k}{\rho c_p}$$

This is the heat equation. The second spatial derivative tells you how much the temperature field differs from its local linear trend. Where the field is strongly curved, temperature changes quickly in time.

## 61.6. Worked example 2: contaminant diffusion in groundwater

Let $c(x,t)$ be solute concentration in a one-dimensional porous medium. Ignore advection for the moment and assume no chemical reaction source.

The balance law is

$$\frac{\partial c}{\partial t} + \frac{\partial J}{\partial x} = 0$$

Use Fick's law:

$$J = -D\frac{\partial c}{\partial x}$$

Substitute:

$$\frac{\partial c}{\partial t} - \frac{\partial}{\partial x}\left(D\frac{\partial c}{\partial x}\right) = 0$$

For constant diffusivity:

$$\frac{\partial c}{\partial t} = D\frac{\partial^2 c}{\partial x^2}$$

This has the same structure as the heat equation. Physically the fields mean something different — but the mathematical skeleton is shared. Recognising that is most of what this chapter is for.

In real groundwater and river-transport problems, advection — the bulk movement of solute carried by flowing water — usually dominates or competes with diffusion. In the generic framework, advection enters through the flux term: the total flux is $J = vc - D\,\partial c/\partial x$, where $vc$ is the solute carried by the bulk flow and $-D\,\partial c/\partial x$ is the diffusive part from Fick's law. Substituting into the balance law and rearranging gives

$$\frac{\partial c}{\partial t} + v\frac{\partial c}{\partial x} = D\frac{\partial^2 c}{\partial x^2}$$

The pure-diffusion case here applies when the pore velocity is low (dense clay, for example) or when setting up an initial model to isolate the diffusive contribution before adding flow. The full advection-diffusion equation is taken up in environmental and geotechnical engineering courses.

## 61.7. Worked example 3: linear elasticity in one dimension

Take a slender bar under axial load. In the simplest small-strain setting, strain is

$$\varepsilon = \frac{dw}{dx}$$

where $w(x)$ is axial displacement (using $w$ to avoid confusion with the generic stored quantity $u$ used earlier). Hooke's law gives

$$\sigma = E\varepsilon = E\frac{dw}{dx}$$

In the generic framework, stress plays the role of flux: it is the quantity that transmits the internal load across a cross-section. The "storage" term vanishes in static equilibrium, so the balance law reduces to a divergence-only statement with a source. The general 1D force balance in a body with distributed body force $b$ (force per unit volume, e.g. self-weight, corresponding to the source term $s$ in the generic law) is

$$\frac{d\sigma}{dx} + b = 0$$

If body forces are negligible ($b = 0$), this reduces to

$$\frac{d\sigma}{dx} = 0$$

Substitute the constitutive relation:

$$\frac{d}{dx}\left(E\frac{dw}{dx}\right) = 0$$

For constant $E$:

$$E\frac{d^2w}{dx^2} = 0 \qquad \Rightarrow \qquad \frac{d^2w}{dx^2} = 0$$

This is much simpler than the heat and diffusion examples, but the logic is the same. Balance law plus constitutive law gives the governing equation.

## 61.8. Where this goes

The most direct continuation is Computational methods for engineering models. Once the governing equations are written, most upper-year work shifts from derivation to solution. Real geometries, mixed materials, and realistic boundary conditions usually push these problems toward numerical methods, meshes, and solver design.

Environmental transport, remote sensing inversion, and geophysics all use the same structure. The quantity and units change. The control-volume logic does not.

**Applications**

- heat flow through walls, fins, and machine components
- diffusion and mixing in process equipment
- contaminant transport in groundwater and rivers
- stress and deformation in structural members
- porous flow and subsurface energy transport
- continuum models in earth and environmental systems

## 61.9. Exercises

Each exercise asks you to derive a governing equation. State the physical meaning of each term as well as the final equation.

### 61.9.1. Exercise 1

Starting from the one-dimensional heat balance

$$\rho c_p \frac{\partial T}{\partial t} + \frac{\partial q}{\partial x} = 0$$

and Fourier's law

$$q = -k \frac{\partial T}{\partial x}$$

derive the heat equation for constant $k$.

### 61.9.2. Exercise 2

Write the diffusion equation for concentration $c(x, t)$ from

$$\frac{\partial c}{\partial t} + \frac{\partial J}{\partial x} = 0, \qquad J = -D \frac{\partial c}{\partial x}$$

assuming $D$ is constant. Then explain why the equation has the same form as the heat equation.

### 61.9.3. Exercise 3

For a one-dimensional elastic bar with axial displacement $w(x)$,

$$\varepsilon = \frac{dw}{dx}, \qquad \sigma = E\varepsilon$$

and static equilibrium

$$\frac{d\sigma}{dx} = 0,$$

derive the governing equation for displacement when $E$ is constant.

### 61.9.4. Exercise 4

Choose one field setting from your own area: a wall, pipe, aquifer, beam, soil column, battery, or river reach.

Prepare a one-page systems sketch naming:

1. the conserved quantity
2. the flux
3. the constitutive law
4. one plausible source term
5. one important boundary condition
6. one reason the resulting equation will probably need numerical solution

```
//| echo: false
{
  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width: 900px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const board = document.createElement("div");
  board.style.cssText = "display:grid; grid-template-columns:repeat(3,minmax(0,1fr)); gap:0.75rem;"
  const models = [
    { title: "Heat conduction", field: "temperature T", grad: "∇T", response: "q = -k∇T", eq: "∂T/∂t
    { title: "Diffusion", field: "concentration c", grad: "∇c", response: "J = -D∇c", eq: "∂c/∂t = D
    { title: "Elasticity", field: "displacement u", grad: "□ = ux", response: "□ = E□", eq: "□x + f
  ];
  models.forEach(model => {
    const panel = document.createElement("div");
    panel.style.cssText = "background:#fff; border:1px solid #e5e7eb; border-radius:10px; padding:0
    const h = document.createElement("div");
    h.style.cssText = "font-size:0.9rem; font-weight:700; color:#111827; margin-bottom:0.35rem;";
    h.textContent = model.title;
    const svg = svgEl("svg");
    svg.setAttribute("viewBox", "0 0 240 180");
```

```
    svg.setAttribute("width", "100%");
    svg.style.display = "block";
    const rect = svgEl("rect");
    rect.setAttribute("x", "28"); rect.setAttribute("y", "46"); rect.setAttribute("width", "160")
    rect.setAttribute("fill", "#f8fafc"); rect.setAttribute("stroke", "#cbd5e1");
    svg.appendChild(rect);
    const grad = svgEl("line");
    grad.setAttribute("x1", "46"); grad.setAttribute("y1", "68"); grad.setAttribute("x2", "170");
    grad.setAttribute("stroke", model.color); grad.setAttribute("stroke-width", "3");
    svg.appendChild(grad);
    const arrow = svgEl("line");
    arrow.setAttribute("x1", "172"); arrow.setAttribute("y1", "68"); arrow.setAttribute("x2", "20
    arrow.setAttribute("stroke", model.color); arrow.setAttribute("stroke-width", "3");
    svg.appendChild(arrow);
    const tip1 = svgEl("line");
    tip1.setAttribute("x1", "206"); tip1.setAttribute("y1", "68"); tip1.setAttribute("x2", "196")
    tip1.setAttribute("stroke", model.color); tip1.setAttribute("stroke-width", "3");
    svg.appendChild(tip1);
    const tip2 = svgEl("line");
    tip2.setAttribute("x1", "206"); tip2.setAttribute("y1", "68"); tip2.setAttribute("x2", "196")
    tip2.setAttribute("stroke", model.color); tip2.setAttribute("stroke-width", "3");
    svg.appendChild(tip2);
    [["42","40",model.left],["174","40",model.right],["108","118",model.field],["205","61",model.
      const text = svgEl("text");
      text.setAttribute("x", x); text.setAttribute("y", y);
      text.setAttribute("font-size", idx === 2 ? "12" : "11");
      text.setAttribute("fill", idx === 3 ? model.color : "#374151");
      text.setAttribute("text-anchor", idx === 2 ? "middle" : "start");
      text.textContent = t;
      svg.appendChild(text);
    });
    const eq = document.createElement("div");
    eq.style.cssText = "font-size:0.96rem; color:#111827; margin-top:0.45rem; font-family:serif;"
    eq.textContent = model.eq;
    const note = document.createElement("div");
    note.style.cssText = "font-size:0.85rem; color:#4b5563; margin-top:0.25rem;";
    note.textContent = `${model.grad} creates the response term. The governing law keeps the same
    panel.append(h, svg, eq, note);
    board.appendChild(panel);
  });
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";
  caption.textContent = "These are not three unrelated subjects. In each case, a field varies in
  container.append(board, caption);
  return container;
}
```

# 62. Computational methods for engineering models

Meshes, weak forms, and conservation in computation

The governing equations may be right and still produce a useless simulation. A coarse mesh smears a boundary layer. A bad time step destabilises a model. A solver converges to something, but not to the physics you meant. Upper-year engineering lives in that gap between the equation on paper and the model you actually trust.

This chapter is about the mathematical move that closes that gap. A continuum equation is not yet computable. A computer needs finitely many unknowns, finitely many equations, and an update rule or linear system it can actually solve. Discretisation supplies that finite structure.

Every discretisation keeps some truths and sacrifices others. One method preserves conservation neatly. Another handles curved geometry more naturally. Another is easy to implement but unstable for large time steps. Knowing which sacrifices the physics can tolerate is what makes a simulation trustworthy.

---

## 62.1. From equation to algebraic system

Take the one-dimensional diffusion equation

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2}$$

defined on a grid with spatial spacing $\Delta x$ and time step $\Delta t$. The continuous field $u(x,t)$ becomes a table of approximate values $u_i^n$, read "u at grid point i and time level n."

Replace $\partial u/\partial t$ with a forward difference $(u_i^{n+1} - u_i^n)/\Delta t$ and $\partial^2 u/\partial x^2$ with a central difference $(u_{i+1}^n - 2u_i^n + u_{i-1}^n)/(\Delta x)^2$:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} \approx D\frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{(\Delta x)^2}$$

Rearrange:

$$u_i^{n+1} = u_i^n + \frac{D\Delta t}{(\Delta x)^2}\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right)$$

The PDE has become a recurrence. Once the spatial grid is fixed, the whole problem is now algebra.

This is the central translation of computational engineering:

- fields become arrays
- derivatives become finite differences, flux balances, or basis-function integrals
- boundary conditions become algebraic constraints
- the PDE becomes either a time-marching update or a linear/nonlinear system to solve

Use the selector to see how the same diffusion law changes shape as it is translated into three different computational languages.

```
//| echo: false
{
  const METHODS = {
    fd: {
      label: "Finite difference",
      gridTitle: "grid picture",
      algebraHTML: "u<sub>i</sub><sup>n+1</sup> = u<sub>i</sub><sup>n</sup> + r(u<sub>i+1</sub><sup>
      note: "Local derivatives become stencil differences at neighbouring nodes."
    },
    fv: {
      label: "Finite volume",
      gridTitle: "control-volume picture",
      algebraHTML: "du<sub>i</sub>/dt = (q<sub>i-1/2</sub> - q<sub>i+1/2</sub>)/Δx",
      note: "The balance is written on a cell, and fluxes enter through its faces."
    },
    fem: {
      label: "Finite element",
      gridTitle: "basis-function picture",
      algebraHTML: "M u̇ + K u = f",
      note: "The field is expanded in basis functions and the PDE becomes a matrix system."
    }
  };

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  const container = document.createElement("div");
  container.style.cssText = "max-width: 860px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.6rem; align-items:center; margin-bottom:0.75rem;";
  const label = document.createElement("span");
```

```
label.textContent = "Viewpoint";
const select = document.createElement("select");
select.style.cssText = "padding:0.35rem 0.5rem; border:1px solid #d1d5db; border-radius:8px; ba
[["fd","Finite difference"],["fv","Finite volume"],["fem","Finite element"]].forEach(([v,l]) =>
  const opt = document.createElement("option");
  opt.value = v; opt.textContent = l; select.appendChild(opt);
});
controls.append(label, select);

const panels = document.createElement("div");
panels.style.cssText = "display:grid; grid-template-columns:repeat(3,minmax(0,1fr)); gap:0.75re

function makePanel(title) {
  const panel = document.createElement("div");
  panel.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:10px; padd
  const h = document.createElement("div");
  h.style.cssText = "font-size:0.82rem; color:#6b7280; margin-bottom:0.5rem; text-transform:upp
  h.textContent = title;
  panel.appendChild(h);
  panels.appendChild(panel);
  return panel;
}

const pdePanel = makePanel("PDE");
const gridPanel = makePanel("Geometry");
const algPanel = makePanel("Algebra");

pdePanel.innerHTML += "<div style='font-size:1rem; color:#111827; line-height:1.5;'>Diffusion l

const gridSvg = svgEl("svg");
gridSvg.setAttribute("viewBox", "0 0 240 140");
gridSvg.setAttribute("width", "100%");
gridSvg.style.display = "block";
gridPanel.appendChild(gridSvg);

const gridTitle = document.createElement("div");
gridTitle.style.cssText = "font-size:0.88rem; color:#4b5563; margin-top:0.45rem;";
gridPanel.appendChild(gridTitle);

const algEq = document.createElement("div");
algEq.style.cssText = "font-size:1rem; color:#111827; line-height:1.6;";
algPanel.appendChild(algEq);
const algNote = document.createElement("div");
algNote.style.cssText = "font-size:0.88rem; color:#4b5563; margin-top:0.45rem;";
algPanel.appendChild(algNote);

function clearSvg() {
```

```
  while (gridSvg.firstChild) gridSvg.removeChild(gridSvg.firstChild);
}

function text(x, y, value, fill = "#374151", size = 12, anchor = "middle") {
  const t = svgEl("text");
  t.setAttribute("x", x);
  t.setAttribute("y", y);
  t.setAttribute("fill", fill);
  t.setAttribute("font-size", size);
  t.setAttribute("text-anchor", anchor);
  t.textContent = value;
  gridSvg.appendChild(t);
}

function redraw() {
  const method = METHODS[select.value];
  clearSvg();

  const base = svgEl("line");
  base.setAttribute("x1", "20");
  base.setAttribute("x2", "220");
  base.setAttribute("y1", "95");
  base.setAttribute("y2", "95");
  base.setAttribute("stroke", "#9ca3af");
  base.setAttribute("stroke-width", "2");
  gridSvg.appendChild(base);

  if (select.value === "fd") {
    [50, 95, 140, 185].forEach((x, idx) => {
      const c = svgEl("circle");
      c.setAttribute("cx", x);
      c.setAttribute("cy", "95");
      c.setAttribute("r", idx === 1 ? "7" : "5");
      c.setAttribute("fill", idx === 1 ? "#2563eb" : "#f59e0b");
      gridSvg.appendChild(c);
    });
    const brace = svgEl("line");
    brace.setAttribute("x1", "50"); brace.setAttribute("x2", "140");
    brace.setAttribute("y1", "62"); brace.setAttribute("y2", "62");
    brace.setAttribute("stroke", "#2563eb"); brace.setAttribute("stroke-width", "2");
    gridSvg.appendChild(brace);
    text(95, 55, "stencil at i", "#2563eb", 12);
    text(50, 115, "i-1", "#374151", 11);
    text(95, 115, "i", "#374151", 11);
    text(140, 115, "i+1", "#374151", 11);
  } else if (select.value === "fv") {
    [45, 95, 145, 195].forEach(x => {
```

```
      const line = svgEl("line");
      line.setAttribute("x1", x); line.setAttribute("x2", x);
      line.setAttribute("y1", "62"); line.setAttribute("y2", "128");
      line.setAttribute("stroke", "#9ca3af"); line.setAttribute("stroke-width", "2");
      gridSvg.appendChild(line);
    });
    const cell = svgEl("rect");
    cell.setAttribute("x", "95");
    cell.setAttribute("y", "62");
    cell.setAttribute("width", "50");
    cell.setAttribute("height", "66");
    cell.setAttribute("fill", "rgba(245,158,11,0.2)");
    cell.setAttribute("stroke", "#f59e0b");
    cell.setAttribute("stroke-width", "2");
    gridSvg.appendChild(cell);
    text(120, 58, "cell i", "#b45309", 12);
    text(95, 145, "q_{i-1/2}", "#2563eb", 11);
    text(145, 145, "q_{i+1/2}", "#2563eb", 11);
  } else {
    const pts = [[40,110],[90,40],[140,110],[190,70]];
    for (let i = 0; i < pts.length - 1; i++) {
      const l = svgEl("line");
      l.setAttribute("x1", pts[i][0]); l.setAttribute("y1", pts[i][1]);
      l.setAttribute("x2", pts[i+1][0]); l.setAttribute("y2", pts[i+1][1]);
      l.setAttribute("stroke", "#7c3aed"); l.setAttribute("stroke-width", "3");
      gridSvg.appendChild(l);
    }
    [90,140].forEach(x => {
      const l = svgEl("line");
      l.setAttribute("x1", x); l.setAttribute("y1", "25");
      l.setAttribute("x2", x); l.setAttribute("y2", "120");
      l.setAttribute("stroke", "#e5e7eb"); l.setAttribute("stroke-width", "1.5");
      gridSvg.appendChild(l);
    });
    text(115, 24, "hat functions / elements", "#6d28d9", 12);
    text(90, 134, "node i", "#374151", 11);
    text(140, 134, "node i+1", "#374151", 11);
  }

  gridTitle.textContent = method.gridTitle;
  algEq.innerHTML = `<span style="font-family:serif;">${method.algebraHTML}</span>`;
  algNote.textContent = method.note;
}

select.addEventListener("change", redraw);
container.append(controls, panels);
redraw();
```

```
  return container;
}
```

## 62.2. Three dominant viewpoints

Upper-year engineering simulation is often organised around three families of discretisation.

**Finite difference**

Replace derivatives directly with local difference quotients on a grid. Conceptually simple. Especially natural on structured meshes and textbook geometries.

**Finite volume**

Integrate the balance law over control volumes, then approximate fluxes at faces. Especially attractive when conservation matters strongly, as in fluid flow, transport, and environmental modelling.

**Finite element**

Represent the solution using basis functions $\phi_i$ over elements and solve a weak or variational form. The weak form replaces the requirement that the PDE holds exactly at every point with the requirement that a weighted integral of the residual is zero. This relaxed condition is easier to satisfy on irregular meshes, which makes finite element methods especially useful on complicated geometries and in structural mechanics.

These are not three unrelated subjects. They are three ways of answering the same question: how should a continuum law be turned into finite equations?

## 62.3. Stability, consistency, and convergence

A discretisation is not good merely because it can be coded.

Three recurring tests matter:

- **consistency**: does the discrete equation approach the continuous one as the mesh is refined?
- **stability**: do errors remain controlled instead of blowing up?
- **convergence**: does the computed solution approach the true solution as the grid is refined?

Students often meet these separately. In practice they are linked by the Lax equivalence theorem. For a linear, well-posed PDE — one where small changes in the input produce small changes in the solution — and a consistent discretisation, stability is both necessary and sufficient for convergence. A consistent method that is unstable may be useless. A stable method that is too diffusive may hide the physics you care about.

For the explicit diffusion update above, the ratio

$$r = \frac{D\Delta t}{(\Delta x)^2}$$

controls stability. For the 1D explicit diffusion scheme the stability bound is

$$r = \frac{D\Delta t}{(\Delta x)^2} \leq \frac{1}{2}$$

This means mesh spacing and timestep cannot be chosen independently. Refining space without tightening time can destroy stability.

If $r$ exceeds $\frac{1}{2}$, the numerical solution can oscillate or blow up even when the physical problem is perfectly well behaved. This bound means halving the grid spacing forces the time step to shrink by a factor of four — a practical cost of explicit time-stepping.

Implicit methods remove or relax this restriction at the cost of solving a linear system at each time step. Backward Euler is unconditionally stable (any $\Delta t$ is permitted) but introduces more numerical diffusion than a comparable explicit step. Crank-Nicolson averages the current and future spatial operators, recovering second-order accuracy in time while remaining stable for all step sizes. For fine spatial grids or stiff problems, solving a system at each step is almost always the correct exchange.

```
//| echo: false
{
  const width = 700, height = 290;
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 740px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-bo
  const dxWrap = document.createElement("label");
  dxWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  dxWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Δx" }));
  const dx = document.createElement("input");
  dx.type = "range"; dx.min = "0.05"; dx.max = "0.30"; dx.step = "0.01"; dx.value = "0.10";
  const dxVal = document.createElement("span"); dxVal.style.cssText = "font-variant-numeric: tabu
  dxWrap.append(dx, dxVal);
  const dtWrap = document.createElement("label");
  dtWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  dtWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Δt" }));
  const dt = document.createElement("input");
  dt.type = "range"; dt.min = "0.001"; dt.max = "0.04"; dt.step = "0.001"; dt.value = "0.005";
  const dtVal = document.createElement("span"); dtVal.style.cssText = "font-variant-numeric: tabu
  dtWrap.append(dt, dtVal);
  controls.append(dxWrap, dtWrap);
  const readout = document.createElement("div");
  readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
  function card(label){const box=document.createElement("div"); box.style.cssText="background:#f8
  const rBox = card("r = DΔt/Δx²");
  const explicitBox = card("Explicit");
  const implicitBox = card("Backward Euler");
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", `0 0 ${width} ${height}`);
  svg.setAttribute("width", "100%");
```

```
svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10px
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function redraw(){
  while(svg.firstChild) svg.removeChild(svg.firstChild);
  const dxv = parseFloat(dx.value), dtv = parseFloat(dt.value), D = 1;
  dxVal.textContent = dxv.toFixed(2);
  dtVal.textContent = dtv.toFixed(3);
  const r = D * dtv / (dxv * dxv);
  rBox.textContent = r.toFixed(2);
  explicitBox.textContent = r <= 0.5 ? "stable region" : "unstable region";
  explicitBox.style.color = r <= 0.5 ? "#166534" : "#b91c1c";
  implicitBox.textContent = "stable for all shown points";
  const left = 70, top = 24, plotW = 260, plotH = 220;
  const sx = x => left + plotW * (x - 0.05) / 0.25;
  const sy = y => top + plotH * (1 - (y - 0.001) / 0.039);
  const line=(x1,y1,x2,y2,stroke,w=1.5,dash="")=>{const l=svgEl("line"); l.setAttribute("x1",x1);
  const text=(x,y,t,fill="#374151",size=12,anchor="middle",weight="400")=>{const n=svgEl("text");
  line(sx(0.05), sy(0.001), sx(0.30), sy(0.001), "#6b7280");
  line(sx(0.05), sy(0.001), sx(0.05), sy(0.04), "#6b7280");
  text(left + plotW/2, 16, "explicit scheme", "#374151", 12, "middle", "700");
  const boundaryPts = [];
  for(let x=0.05;x<=0.30+1e-9;x+=0.005){ boundaryPts.push(`${sx(x)},${sy(0.5*x*x)}`); }
  const stablePoly = svgEl("polygon");
  stablePoly.setAttribute("points", `${sx(0.05)},${sy(0.001)} ${sx(0.30)},${sy(0.001)} ${boundaryI
  stablePoly.setAttribute("fill","rgba(5,150,105,0.12)");
  stablePoly.setAttribute("stroke","none");
  svg.appendChild(stablePoly);
  const boundary = svgEl("polyline"); boundary.setAttribute("points", boundaryPts.join(" ")); bour
  text(sx(0.20), sy(0.010), "stable", "#059669", 13, "middle", "700");
  text(sx(0.14), sy(0.030), "unstable", "#b91c1c", 13, "middle", "700");
  const pt = svgEl("circle"); pt.setAttribute("cx", sx(dxv)); pt.setAttribute("cy", sy(dtv)); pt.s
  text(sx(dxv)+16, sy(dtv)-8, `(${dxv.toFixed(2)}, ${dtv.toFixed(3)})`, "#374151", 11, "start");
  const box = svgEl("rect"); box.setAttribute("x","420"); box.setAttribute("y","44"); box.setAttri
  text(530, 66, "implicit schemes", "#374151", 12, "middle", "700");
  text(530, 104, "Backward Euler", "#2563eb", 12, "middle");
  text(530, 126, "no CFL-style stability wall", "#374151", 12, "middle");
  text(530, 162, "tradeoff:", "#6b7280", 12, "middle", "700");
  text(530, 182, "stable for large ⍰t", "#059669", 12, "middle");
  text(530, 202, "but more diffusive and costlier per step", "#b45309", 12, "middle");
  caption.textContent = "For the explicit diffusion update, ⍰x and ⍰t cannot be chosen independent
}

[dx, dt].forEach(el => el.addEventListener("input", redraw));
container.append(controls, readout, svg, caption);
redraw();
```

```
  return container;
}
```

**Discretisation always introduces error**

Replacing a continuous field with finitely many unknowns introduces error automatically. The right question is never "is there error?" but "what kind of error, how large, and does it destroy the physical meaning of the model?"
Computational mathematics is therefore not separate from modelling. The numerical method is part of the model.

## 62.4. The core method

A first pass through a computational engineering model usually goes like this:

1. Write the governing equation and the boundary/initial conditions.
2. Decide which discrete viewpoint matches the physics and geometry best.
3. Define the grid, mesh, or basis functions.
4. Derive the algebraic update or system.
5. Check the main stability or conditioning constraint.
6. Interpret the numerical output in physical terms and test mesh sensitivity.

The last step matters as much as the derivation. If halving $\Delta x$ changes the answer substantially, the previous mesh was telling you more about the discretisation than about the system.

## 62.5. Worked example 1: explicit diffusion update

Take

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2}$$

with $D = 1$, $\Delta x = 0.1$, and $\Delta t = 0.002$.

Then

$$r = \frac{D\Delta t}{(\Delta x)^2} = \frac{1(0.002)}{0.01} = 0.2$$

For the standard explicit 1D diffusion scheme, the stability requirement is $r \leq \frac{1}{2}$. Since $0.2 < 0.5$, this time step is within the stable range and is plausibly acceptable for this grid.

Now change only the time step to $\Delta t = 0.008$. Then

$$r = \frac{0.008}{0.01} = 0.8$$

and the scheme is no longer stable — $0.8 > 0.5$. Same PDE. Same material parameter. Different numerical behaviour because the discretisation changed.

This is why simulation judgement is mathematical judgement. The equation on the page did not fail. The computational choice did.

## 62.6. Worked example 2: a finite-volume mass balance

Suppose a pollutant concentration is tracked over a control volume of width $\Delta x$. The half-integer subscripts label the cell faces: $F_{i-1/2}$ is the flux at the left face of cell $i$ (between cells $i-1$ and $i$), and $F_{i+1/2}$ is the flux at the right face (between cells $i$ and $i+1$). Then a finite-volume update has the form

$$\frac{c_i^{n+1} - c_i^n}{\Delta t} = -\frac{F_{i+1/2}^n - F_{i-1/2}^n}{\Delta x} + s_i^n$$

This is the discrete balance law in its most honest form:

- future storage minus current storage
- equals net flux imbalance
- plus any local source

The attraction of the finite-volume approach is that conservation is built into the discretisation. If what leaves one cell enters the next, the algebra itself remembers that.

That is why finite volume is so common in transport and environmental systems. The method echoes the underlying physics instead of merely approximating the derivatives.

## 62.7. Worked example 3: why finite elements dominate structural analysis

In a structural problem, geometry is often awkward: holes, supports, joints, changing thickness, mixed materials. A regular grid is not always natural.

Finite elements instead approximate the displacement field using basis or shape functions:

$$u_h(x) = \sum_i U_i \phi_i(x)$$

The unknowns are the nodal coefficients $U_i$. The governing equations become a matrix system

$$K\mathbf{U} = \mathbf{f}$$

where $K$ is the **stiffness matrix** — a different $K$ from the control gain in Chapter 1 (Feedback control and state-space models), but the same letter is standard in both fields.

The important mathematical point is not the assembly details. It is that the continuum problem has been projected onto a finite-dimensional space spanned by the $\phi_i$. Geometry and material behaviour

are therefore represented through the choice of mesh and basis functions, not just through a derivative stencil.

A second important point is that $K$ is almost always **sparse**: each basis function $\phi_i$ overlaps only with its neighbours, so most entries of $K$ are zero. A mesh with millions of nodes still produces a matrix with only a few non-zeros per row. This sparsity is why large FEM problems are computationally tractable — but solver choice matters.

**Direct sparse solvers** (such as UMFPACK or CHOLMOD) factorize $K$ in one shot. They work well at moderate problem sizes but produce fill-in: new non-zero entries created during factorization that were zero in the original $K$. Fill-in grows faster than the problem does, which limits how far direct methods can scale.

**Iterative solvers** — Krylov methods such as conjugate gradient (for symmetric positive-definite $K$, meaning $K = K^T$ and $\mathbf{x}^T K \mathbf{x} > 0$ for all nonzero $\mathbf{x}$, which holds for many standard structural problems) or GMRES (for non-symmetric cases such as convection-dominated flow) — access $K$ only through matrix-vector products. They can reach millions of unknowns where direct methods cannot.

Both solver families operate on $K$ stored in compressed sparse row (CSR) or an equivalent format. At scale, understanding these choices is inseparable from using finite element analysis.

## 62.8. Where this goes

This chapter draws on two prior threads: the numerical methods sequence (finite differences, ODE solvers, linear systems) and continuum transport (the conservation laws that finite volume discretises directly). If the stability analysis or the flux-balance notation felt unfamiliar, those are the chapters to revisit.

The next natural continuation is Nonlinear optimisation for design and operations. Once a simulation model can be trusted to represent the system, design questions become computational questions: which shape, parameter, schedule, or control setting produces the best feasible outcome?

This chapter also deepens what you should mean by "model." In upper-year work, the numerical method is not downstream implementation detail. It is part of the model's claim to credibility.

**Applications**

- heat-transfer simulation in machine components
- finite-element structural analysis
- finite-volume flow and transport models
- weather, hydrology, and terrain-based environmental models
- sparse linear systems in scientific computing
- mesh-sensitivity and timestep-sensitivity studies

## 62.9. Exercises

These are project-style exercises. Do not stop at the formula. Explain what the numerical choice means for the model.

### 62.9.1. Exercise 1

For the explicit diffusion scheme with $D = 2$, $\Delta x = 0.05$, and $\Delta t = 0.0004$, compute

$$r = \frac{D\Delta t}{(\Delta x)^2}$$

and decide whether the time step is plausibly stable for the standard explicit scheme.

### 62.9.2. Exercise 2

A finite-volume cell has inflow flux $F_{i-1/2} = 3$, outflow flux $F_{i+1/2} = 5$, source term $s_i = 1$, and width $\Delta x = 2$.

Write the sign of the net flux contribution and explain whether storage tends to increase or decrease before the source term is included.

### 62.9.3. Exercise 3

Write a short comparison note explaining when you would prefer:

1. finite difference
2. finite volume
3. finite element

Use one example application for each method.

### 62.9.4. Exercise 4

Choose one simulation problem from your own field and prepare a one-page model brief naming:

1. the governing equation
2. the boundary and initial conditions
3. the likely discretisation family
4. one likely stability or conditioning concern
5. one mesh- or timestep-sensitivity check you would run before trusting the result

```
//| echo: false
{
  const width = 720, height = 300;
  const pad = { top: 22, right: 18, bottom: 40, left: 44 };
  const plotW = width - pad.left - pad.right;
  const plotH = height - pad.top - pad.bottom;
  function svgEl(tag) { return document.createElementNS("http://www.w3.org/2000/svg", tag); }
  const container = document.createElement("div");
  container.style.cssText = "max-width: 760px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-bo
  const resWrap = document.createElement("label");
  resWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  resWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Grid resoluti
  const resSel = document.createElement("select");
  resSel.style.cssText = "padding:0.35rem 0.55rem; border:1px solid #d1d5db; border-radius:8px; b
  ["Coarse (□x = 0.20)", "Fine (□x = 0.10)"].forEach(v => { const o=document.createElement("optio
  resWrap.appendChild(resSel);
  const dtWrap = document.createElement("label");
  dtWrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  dtWrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Timestep" }));
  const dtSel = document.createElement("select");
  dtSel.style.cssText = "padding:0.35rem 0.55rem; border:1px solid #d1d5db; border-radius:8px; ba
  ["Small □t", "Large □t"].forEach(v => { const o=document.createElement("option"); o.value=v; o.
  dtWrap.appendChild(dtSel);
  controls.append(resWrap, dtWrap);
  const readout = document.createElement("div");
  readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
  function card(label){const box=document.createElement("div"); box.style.cssText="background:#f8
  const errBox = card("Error estimate");
  const costBox = card("Runtime cost");
  const trustBox = card("Trust signal");
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", `0 0 ${width} ${height}`);
  svg.setAttribute("width", "100%");
  svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

  function redraw() {
    while (svg.firstChild) svg.removeChild(svg.firstChild);
    const fine = resSel.value.startsWith("Fine");
    const dx = fine ? 0.1 : 0.2;
    const dt = dtSel.value.startsWith("Small") ? 0.005 : 0.02;
    const xNodes = [];
    for (let x = 0; x <= 1 + 1e-9; x += dx) xNodes.push(parseFloat(x.toFixed(4)));
    const profile = xNodes.map(x => Math.sin(Math.PI * x) * Math.exp(-2.2 * dt / dx));
```

```
    const ref = xNodes.map(x => Math.sin(Math.PI * x) * Math.exp(-1.6));
    const sx = x => pad.left + plotW * x;
    const sy = y => pad.top + plotH * (1 - (y + 0.05) / 1.1);
    for (let x = 0; x <= 1; x += 0.1) { const l=svgEl("line"); l.setAttribute("x1",sx(x)); l.setAtt
    const axis = svgEl("line");
    axis.setAttribute("x1", sx(0)); axis.setAttribute("x2", sx(1)); axis.setAttribute("y1", sy(0));
    svg.appendChild(axis);
    const refPath = svgEl("polyline");
    refPath.setAttribute("points", ref.map((v,i)=>`${sx(xNodes[i])},${sy(v)}`).join(" "));
    refPath.setAttribute("fill","none"); refPath.setAttribute("stroke","#9ca3af"); refPath.setAttrib
    svg.appendChild(refPath);
    const profPath = svgEl("polyline");
    profPath.setAttribute("points", profile.map((v,i)=>`${sx(xNodes[i])},${sy(v)}`).join(" "));
    profPath.setAttribute("fill","none"); profPath.setAttribute("stroke", fine ? "#2563eb" : "#f59e0
    svg.appendChild(profPath);
    xNodes.forEach((x,i)=>{ const c=svgEl("circle"); c.setAttribute("cx",sx(x)); c.setAttribute("cy"
    const t1 = svgEl("text"); t1.setAttribute("x","78"); t1.setAttribute("y","30"); t1.setAttribute
    const t2 = svgEl("text"); t2.setAttribute("x","206"); t2.setAttribute("y","30"); t2.setAttribute
    const err = profile.reduce((s,v,i)=>s+Math.abs(v-ref[i]),0)/profile.length + (dt > 0.01 ? 0.08
    const cost = xNodes.length * (1 / dt) * 0.02;
    errBox.textContent = err.toFixed(3);
    costBox.textContent = `${cost.toFixed(1)} units`;
    trustBox.textContent = fine && dt < 0.01 ? "highest trust, highest cost" : !fine && dt > 0.01 ?
    caption.textContent = "The grey dashed curve is the reference solution. Finer grids and smaller
  }

  [resSel, dtSel].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, readout, svg, caption);
  redraw();
  return container;
}
```

# 63. Estimation, inverse problems, and filtering

Recovering hidden states from noisy measurements

You can measure vibration but not crack length. You can observe GPS, camera, and IMU signals but not the true state of the vehicle. You can see reflected radiation from a landscape but not the quantity you really care about on the ground. Inverse mathematics begins when the thing you want is not the thing you can measure.

The forward problem asks: if the state were known, what measurements would it produce? The inverse problem asks: given the measurements, what can you recover about the state or parameters?

The inverse direction is unstable. Small measurement errors can produce large errors in the recovered quantity. A good estimation method therefore does two jobs at once: it produces an estimate and it keeps track of how much trust the data deserves.

---

## 63.1. Forward and inverse viewpoints

Let $\mathbf{x}$ be the hidden state you care about and $\mathbf{y}$ the measured data you actually have. A simple measurement model is

$$\mathbf{y} = H\mathbf{x} + \varepsilon$$

where $H$ maps the state into observation space and $\varepsilon$ represents measurement noise.

The **forward problem** is:

- given $\mathbf{x}$, compute $\mathbf{y}$

The **inverse problem** is:

- given $\mathbf{y}$, estimate $\mathbf{x}$

The inverse direction is harder because the map may be noisy, incomplete, or many-to-one. Some parts of the state may be visible in the data. Others may be weakly observable or not observable at all. Estimation is not algebraic rearrangement. It is controlled inference under limited information.

Use the controls to see how the estimate shifts as you trust the sensor more or less strongly.

```
//| echo: false
{
  const width = 660;
  const height = 360;
  const pad = { top: 24, right: 18, bottom: 48, left: 48 };
  const plotW = width - pad.left - pad.right;
  const plotH = height - pad.top - pad.bottom;

  const BLUE = "#2563eb";
  const AMBER = "#f59e0b";
  const GREEN = "#059669";
  const RED = "#ef4444";
  const GREY = "#6b7280";
  const GRID = "#d1d5db";
  const BG = "#f8fafc";

  const N = 36;

  let noise = 1.2;
  let gain = 0.45;

  function makeSvg(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  function rng(i) {
    const x = Math.sin(91.17 * (i + 1) + 0.37) * 43758.5453123;
    return x - Math.floor(x);
  }

  function xScale(i) {
    return pad.left + (i / (N - 1)) * plotW;
  }

  function buildSeries(noiseLevel, K) {
    const truth = [];
    const meas = [];
    const prior = [];
    const post = [];

    let xhat = 17.5;
    for (let i = 0; i < N; i += 1) {
      const t = i;
      const trueVal =
        18
        + 0.18 * i
        + 1.3 * Math.sin(0.25 * i)
```

```
      + 0.55 * Math.cos(0.11 * i + 0.5);
    const measurement = trueVal + (rng(i) - 0.5) * 2 * noiseLevel;

    const prediction = i === 0 ? xhat : xhat + 0.18;
    const updated = prediction + K * (measurement - prediction);

    truth.push({ i, y: trueVal });
    meas.push({ i, y: measurement });
    prior.push({ i, y: prediction });
    post.push({ i, y: updated, innovation: measurement - prediction });

    xhat = updated;
    t;
  }

  const ys = [
    ...truth.map(p => p.y),
    ...meas.map(p => p.y),
    ...prior.map(p => p.y),
    ...post.map(p => p.y)
  ];
  const ymin = Math.min(...ys) - 1;
  const ymax = Math.max(...ys) + 1;
  const yScale = y => pad.top + plotH * (1 - (y - ymin) / (ymax - ymin));

  return { truth, meas, prior, post, yScale, ymin, ymax };
}

function pathFor(points, yScale) {
  return points.map((p, idx) => `${idx === 0 ? "M" : "L"} ${xScale(p.i).toFixed(2)} ${yScale(p.
}

const container = document.createElement("div");
container.style.cssText = "max-width: 700px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem

const noiseLabel = document.createElement("label");
noiseLabel.textContent = "Measurement noise";
const noiseSlider = document.createElement("input");
noiseSlider.type = "range";
noiseSlider.min = "0.2";
noiseSlider.max = "3.0";
noiseSlider.step = "0.1";
noiseSlider.value = String(noise);
const noiseValue = document.createElement("span");
```

```
noiseValue.style.cssText = "font-variant-numeric: tabular-nums; min-width:4rem;";

const gainLabel = document.createElement("label");
gainLabel.textContent = "Update weight K";
const gainSlider = document.createElement("input");
gainSlider.type = "range";
gainSlider.min = "0.05";
gainSlider.max = "0.95";
gainSlider.step = "0.05";
gainSlider.value = String(gain);
const gainValue = document.createElement("span");
gainValue.style.cssText = "font-variant-numeric: tabular-nums; min-width:4rem;";

controls.append(noiseLabel, noiseSlider, noiseValue, gainLabel, gainSlider, gainValue);

const readout = document.createElement("div");
readout.style.cssText = "display:grid; grid-template-columns:repeat(5,minmax(0,1fr)); gap:0.5rem;
function makeCard(keyText) {
  const card = document.createElement("div");
  card.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding:0
  const key = document.createElement("div");
  key.style.cssText = "font-size:0.76rem; color:#6b7280; margin-bottom:0.15rem;";
  key.textContent = keyText;
  const value = document.createElement("div");
  value.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  card.append(key, value);
  readout.append(card);
  return value;
}
const truthCard = makeCard("Current truth");
const measCard = makeCard("Measurement");
const priorCard = makeCard("Prior");
const postCard = makeCard("Updated estimate");
const innovationCard = makeCard("Innovation");

const svg = makeSvg("svg");
svg.setAttribute("viewBox", `0 0 ${width} ${height}`);
svg.setAttribute("width", "100%");
svg.style.cssText = "display:block; background:white; border:1px solid #e5e7eb; border-radius:10px

const bg = makeSvg("rect");
bg.setAttribute("x", "0");
bg.setAttribute("y", "0");
bg.setAttribute("width", width);
bg.setAttribute("height", height);
bg.setAttribute("fill", BG);
svg.appendChild(bg);
```

```
const grid = makeSvg("g");
const paths = makeSvg("g");
const dots = makeSvg("g");
svg.append(grid, paths, dots);

const truthPath = makeSvg("path");
truthPath.setAttribute("fill", "none");
truthPath.setAttribute("stroke", BLUE);
truthPath.setAttribute("stroke-width", "3");

const priorPath = makeSvg("path");
priorPath.setAttribute("fill", "none");
priorPath.setAttribute("stroke", GREY);
priorPath.setAttribute("stroke-width", "2");
priorPath.setAttribute("stroke-dasharray", "6 5");

const postPath = makeSvg("path");
postPath.setAttribute("fill", "none");
postPath.setAttribute("stroke", GREEN);
postPath.setAttribute("stroke-width", "3");

paths.append(truthPath, priorPath, postPath);

const xAxis = makeSvg("line");
xAxis.setAttribute("x1", pad.left);
xAxis.setAttribute("x2", pad.left + plotW);
xAxis.setAttribute("y1", pad.top + plotH);
xAxis.setAttribute("y2", pad.top + plotH);
xAxis.setAttribute("stroke", GREY);
xAxis.setAttribute("stroke-width", "1.5");
svg.appendChild(xAxis);

const yAxis = makeSvg("line");
yAxis.setAttribute("x1", pad.left);
yAxis.setAttribute("x2", pad.left);
yAxis.setAttribute("y1", pad.top);
yAxis.setAttribute("y2", pad.top + plotH);
yAxis.setAttribute("stroke", GREY);
yAxis.setAttribute("stroke-width", "1.5");
svg.appendChild(yAxis);

const note = document.createElement("p");
note.style.cssText = "font-size:0.9rem; color:#374151; margin:0.55rem 0 0 0;";

const legend = document.createElement("div");
legend.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-top:0.55rem; font-size:0
function legendItem(color, dash, text, dot = false) {
```

```
    const item = document.createElement("div");
    item.style.cssText = "display:inline-flex; align-items:center; gap:0.45rem;";
    const swatch = document.createElement("span");
    if (dot) {
      swatch.style.cssText = `display:inline-block; width:0.7rem; height:0.7rem; border-radius:999px
    } else {
      swatch.style.cssText = `display:inline-block; width:1.6rem; height:0; border-top:3px ${dash ?
    }
    const label = document.createElement("span");
    label.textContent = text;
    item.append(swatch, label);
    return item;
  }
  legend.append(
    legendItem(BLUE, false, "true state"),
    legendItem(AMBER, false, "measurements", true),
    legendItem(GREY, true, "prior estimate"),
    legendItem(GREEN, false, "updated estimate")
  );

  function render() {
    noise = parseFloat(noiseSlider.value);
    gain = parseFloat(gainSlider.value);
    noiseValue.textContent = noise.toFixed(1);
    gainValue.textContent = gain.toFixed(2);

    const { truth, meas, prior, post, yScale, ymin, ymax } = buildSeries(noise, gain);

    grid.replaceChildren();
    [-1, -0.5, 0, 0.5, 1].forEach(() => {});
    for (let i = 0; i <= 6; i += 1) {
      const x = pad.left + (i / 6) * plotW;
      const line = makeSvg("line");
      line.setAttribute("x1", x);
      line.setAttribute("x2", x);
      line.setAttribute("y1", pad.top);
      line.setAttribute("y2", pad.top + plotH);
      line.setAttribute("stroke", GRID);
      line.setAttribute("stroke-width", "1");
      line.setAttribute("opacity", "0.75");
      grid.appendChild(line);
    }
    for (let i = 0; i <= 4; i += 1) {
      const yVal = ymin + (i / 4) * (ymax - ymin);
      const y = yScale(yVal);
      const line = makeSvg("line");
      line.setAttribute("x1", pad.left);
```

```
      line.setAttribute("x2", pad.left + plotW);
      line.setAttribute("y1", y);
      line.setAttribute("y2", y);
      line.setAttribute("stroke", GRID);
      line.setAttribute("stroke-width", "1");
      line.setAttribute("opacity", "0.75");
      grid.appendChild(line);

      const label = makeSvg("text");
      label.setAttribute("x", pad.left - 10);
      label.setAttribute("y", y + 4);
      label.setAttribute("fill", GREY);
      label.setAttribute("font-size", "12");
      label.setAttribute("text-anchor", "end");
      label.textContent = yVal.toFixed(1);
      grid.appendChild(label);
    }

    truthPath.setAttribute("d", pathFor(truth, yScale));
    priorPath.setAttribute("d", pathFor(prior, yScale));
    postPath.setAttribute("d", pathFor(post, yScale));

    dots.replaceChildren();
    meas.forEach((p, idx) => {
      const circle = makeSvg("circle");
      circle.setAttribute("cx", xScale(p.i));
      circle.setAttribute("cy", yScale(p.y));
      circle.setAttribute("r", idx === N - 1 ? "4.5" : "3.2");
      circle.setAttribute("fill", AMBER);
      circle.setAttribute("stroke", "#92400e");
      circle.setAttribute("stroke-width", "1");
      dots.appendChild(circle);
    });

    const last = N - 1;
    truthCard.textContent = truth[last].y.toFixed(2);
    measCard.textContent = meas[last].y.toFixed(2);
    priorCard.textContent = prior[last].y.toFixed(2);
    postCard.textContent = post[last].y.toFixed(2);
    innovationCard.textContent = post[last].innovation >= 0
      ? `+${post[last].innovation.toFixed(2)}`
      : post[last].innovation.toFixed(2);
    innovationCard.style.color = Math.abs(post[last].innovation) > noise ? RED : GREY;

    note.textContent = gain > 0.7
      ? "High K makes the estimate chase the measurements closely, which improves responsiveness
      : gain < 0.3
```

```
        ? "Low K makes the estimate trust the prior strongly, which smooths noise but can lag behind
        : "Intermediate K balances responsiveness against noise suppression: the estimate moves towa
  }

  noiseSlider.addEventListener("input", render);
  gainSlider.addEventListener("input", render);

  container.append(controls, readout, svg, legend, note);
  render();
  return container;
}
```

## 63.2. The simplest update logic

Suppose you have a prior estimate $\hat{x}^-$ and then receive a new measurement $y$. In the scalar case where the measurement directly observes the state ($H = 1$), a simple update rule is

$$\hat{x}^+ = \hat{x}^- + K(y - \hat{x}^-)$$

where:

- $\hat{x}^-$ is the estimate before the measurement update
- $y - \hat{x}^-$ is the innovation or residual
- $K$ is the **update weight** — the same letter used for control gain in Vol 8 Chapter 1 and for the stiffness matrix in Vol 8 Chapter 4; here it plays a different role as a scalar between 0 and 1

In the general case with a non-trivial observation operator $H$, the innovation becomes $y - H\hat{x}^-$, but the structure of the weighted update is the same.

If $K$ is close to 1, you trust the new measurement strongly. If $K$ is close to 0, you trust the existing estimate strongly. Filtering methods differ in how they compute or adapt this weight, but the logic is already visible here. This means the update is a controlled compromise, not a hard switch from model to measurement.

The rule makes the compromise explicit. Estimation is never "use the measurement" or "use the model." It is always a weighted combination of both.

## 63.3. Parameter estimation and state estimation

Not every inverse problem asks for the same thing.

**State estimation**

You want the current hidden state of a dynamic system: position, velocity, temperature field, internal concentration, or machine condition.

**Parameter estimation**

You want the unknown constant or slowly varying quantity in the model: diffusivity, drag coefficient, reaction rate, calibration factor.

**Inverse field recovery**

You want an entire spatially distributed quantity from indirect measurements: subsurface moisture, atmospheric state, or interior temperature.

The mathematics overlaps substantially. Keeping the common structure in view is what stops them looking like three unrelated problems.

## 63.4. The core method

A first pass through an estimation or inverse problem usually goes like this:

1. Decide whether the unknown is a state, a parameter, or a field.
2. Write the forward relation from hidden quantity to measurement.
3. Name the main source of uncertainty: measurement noise, model error, missing data, or poor observability.
4. Choose an update or fitting rule.
5. Interpret the **residuals** — the differences between what the model predicted and what was actually measured — not just the estimate itself.
6. Ask what information the data still cannot recover.

The last question is the one most often skipped. A polished estimate can still be misleading if the measurements never contained the needed information.

> **Why this works**
>
> The update works because it combines two sources that are wrong in different ways. The model prediction is coherent but incomplete. The measurement is immediate but noisy. The weights are chosen so each source corrects the other's weakness.
> That is why estimation sits at the intersection of control, signal processing, and statistics. It draws on all three.

## 63.5. Worked example 1: one-step scalar filter

Suppose the prior estimate of a temperature is $\hat{x}^- = 18$ degrees, a new sensor reading is $y = 20$ degrees, and the update weight is $K = 0.3$. Then

$$\hat{x}^+ = \hat{x}^- + K(y - \hat{x}^-) = 18 + 0.3(20 - 18) = 18 + 0.6 = 18.6$$

The updated estimate moves toward the measurement, but does not jump all the way there. That is exactly what you want when the sensor is informative but not perfect.

If the sensor were known to be extremely reliable, a larger $K$ would make sense. If the measurement were very noisy, a smaller $K$ would be more honest.

```
//| echo: false
{
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 720px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-botto
  function sliderRow(labelText,min,max,step,value){const wrap=document.createElement("label"); wrap
  const prior = sliderRow("Prior estimate x̂⁻", 0, 30, 0.5, 14);
  const meas = sliderRow("Measurement y", 0, 30, 0.5, 20);
  const gain = sliderRow("Update weight K", 0, 1, 0.05, 0.3);
  const readout = document.createElement("div");
  readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
  function card(label){const box=document.createElement("div"); box.style.cssText="background:#f8fa
  const innovationBox = card("Innovation");
  const updateBox = card("Updated estimate x̂⁺");
  const trustBox = card("Interpretation");
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 680 160");
  svg.setAttribute("width", "100%");
  svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10px
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

  function redraw(){
    while(svg.firstChild) svg.removeChild(svg.firstChild);
    const xminus = parseFloat(prior.input.value), y = parseFloat(meas.input.value), K = parseFloat(g
    prior.out.textContent = xminus.toFixed(1); meas.out.textContent = y.toFixed(1); gain.out.textCon
    const innovation = y - xminus;
    const xplus = xminus + K * innovation;
    const map = v => 70 + 540 * v / 30;
    const line = (x1,y1,x2,y2,stroke,w=2,dash="")=>{const l=svgEl("line"); l.setAttribute("x1",x1);
    const text=(x,y0,t,fill="#374151",size=12,anchor="middle",weight="400")=>{const n=svgEl("text")
    line(map(0), 88, map(30), 88, "#6b7280", 1.8);
    for(let t=0;t<=30;t+=5){ line(map(t),82,map(t),94,"#9ca3af",1.2); text(map(t),108,`${t}`,"#6b728
    const p = svgEl("circle"); p.setAttribute("cx",map(xminus)); p.setAttribute("cy","88"); p.setAtt
    const m = svgEl("circle"); m.setAttribute("cx",map(y)); m.setAttribute("cy","88"); m.setAttribut
    const u = svgEl("circle"); u.setAttribute("cx",map(xplus)); u.setAttribute("cy","88"); u.setAtt
    line(map(xminus), 60, map(y), 60, "#d1d5db", 4);
    line(map(xminus), 60, map(xplus), 60, "#059669", 5);
    text(map(xminus), 48, "prior x̂⁻", "#2563eb", 11);
    text(map(y), 48, "measurement y", "#b45309", 11);
    text(map(xplus), 132, "updated x̂⁺", "#059669", 11);
    innovationBox.textContent = `${innovation >= 0 ? "+" : ""}${innovation.toFixed(1)}`;
    updateBox.textContent = xplus.toFixed(2);
    trustBox.textContent = K < 0.3 ? "stays near prior" : K > 0.7 ? "jumps toward measurement" : "ba
    caption.textContent = "The update does not jump blindly to the measurement. It moves partway fro
```

```
  }

  [prior.input, meas.input, gain.input].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, readout, svg, caption);
  redraw();
  return container;
}
```

## 63.6. Worked example 2: linear least-squares parameter fit

Suppose a calibration model predicts measurements by

$$y_i \approx \theta x_i$$

for known inputs $x_i$ and unknown scale parameter $\theta$. With observations

$$(x_i, y_i) = (1, 2.1), (2, 3.9), (3, 6.2)$$

the least-squares estimate minimises

$$\sum_i (y_i - \theta x_i)^2$$

Setting $\frac{d}{d\theta} \sum_i (y_i - \theta x_i)^2 = 0$ gives $-2 \sum_i x_i (y_i - \theta x_i) = 0$. Expanding gives $\sum_i x_i y_i = \theta \sum_i x_i^2$, which rearranges to

$$\hat{\theta} = \frac{\sum_i x_i y_i}{\sum_i x_i^2} = \frac{1(2.1) + 2(3.9) + 3(6.2)}{1^2 + 2^2 + 3^2} = \frac{28.5}{14} \approx 2.036$$

This is parameter estimation rather than state estimation, but the structure is the same: use noisy measurements to infer a hidden quantity in the model.

## 63.7. Worked example 3: remote sensing as an inverse problem

A satellite does not measure soil moisture directly. It measures reflected or emitted radiation. A forward model links surface state to what the instrument would see. The inverse problem works backward from the measured signal toward the surface property of interest.

The mathematics is the same as in engineering system identification:

- there is a hidden state or parameter
- there is an observation operator
- there is noise and model mismatch
- the estimate must balance data fit against plausibility

Remote sensing and process monitoring look unrelated on the surface. The mathematical structure is nearly identical.

## 63.8. The Kalman filter as the canonical example

The update rule in Worked Example 1 is not just a pedagogical toy. It is the core of the **Kalman filter**, which structures estimation around a predict-then-update cycle:

1. **Predict**: use the system model to propagate the state estimate forward one step, together with an estimate of how uncertain that prediction is.
2. **Update**: when a measurement arrives, compute how much to trust it relative to the prediction. The Kalman gain $K$ is chosen to minimise the mean-squared estimation error — the average squared distance between the estimate and the true state. When the noise is Gaussian this gives the globally optimal estimator. For any noise distribution with finite variance, it is the best linear unbiased estimator — the Gauss-Markov result, which says no linear estimator of the same form can have smaller variance.

The two-step pattern generalises. Extended Kalman filters handle nonlinear models by linearising around the current estimate. Particle filters handle nonlinear dynamics and non-Gaussian noise by representing uncertainty as a weighted cloud of samples. Variational data assimilation in weather prediction is a related method at continental scale. Hidden Markov models use the same predict-update logic for discrete state spaces. Bayesian filtering is the general probabilistic framing that contains all of these as special cases.

The scalar example in this chapter already shows the logic. The rest is bookkeeping to handle vectors, covariances, and system models.

## 63.9. Where this goes

The next natural continuation is Chapter 6 of this volume, Reliability, stochastic systems, and quality. Once you are estimating hidden states and parameters from uncertain data, the next question is often what that uncertainty means for failure, quality, risk, or operational decision-making over time.

There is also a direct connection back to control. Modern control systems often operate on estimated states rather than directly measured ones. Estimation is not a sidebar to control theory. It is often the missing half.

**Where this mathematics appears**

- condition monitoring in machines and structures
- navigation and sensor fusion
- process monitoring and soft sensing
- parameter estimation in simulation models
- remote sensing and environmental inversion
- streaming latent-state estimation in data systems

## 63.10. Exercises

These are project-style exercises. State what the estimate means and what the data still does not tell you.

### 63.10.1. Exercise 1

Use the scalar update rule

$$\hat{x}^+ = \hat{x}^- + K(y - \hat{x}^-)$$

with $\hat{x}^- = 12$, $y = 15$, and $K = 0.4$.

Compute the updated estimate and explain whether the update trusts the measurement strongly or cautiously.

### 63.10.2. Exercise 2

Given data

$$(x_i, y_i) = (1, 1.8), (2, 4.1), (4, 8.3),$$

estimate $\theta$ in the model $y_i \approx \theta x_i$ using

$$\hat{\theta} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}$$

and explain what the parameter means physically.

### 63.10.3. Exercise 3

A monitoring system can measure motor current and casing temperature, but not bearing wear directly.

Write a short systems note answering:

1. what the hidden state is
2. what the measurements are
3. one plausible forward model relation
4. one reason the inverse problem may be ambiguous or poorly observable — for instance, two different hidden states that would produce very similar measurements (the formal term for this instability is “ill-conditioned”)

### 63.10.4. Exercise 4

Choose one inverse problem from your own field and prepare a one-page model brief naming:

1. the hidden quantity
2. the measured quantity
3. the forward map
4. the main noise source
5. one update or fitting rule you would try first
6. one thing the estimate still cannot guarantee

```
//| echo: false
{
  const contexts = {
    process: { label: "Process monitoring", state: "reactor temperature field", obs: "thermocouple r
    navigation: { label: "Navigation", state: "vehicle position + velocity", obs: "GPS/IMU observati
    remote: { label: "Remote sensing", state: "soil moisture profile", obs: "radiative transfer H",
  };
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 740px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; margin-bottom:0.75rem;";
  const wrap = document.createElement("label");
  wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  wrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Context" }));
  const select = document.createElement("select");
  select.style.cssText = "padding:0.35rem 0.55rem; border:1px solid #d1d5db; border-radius:8px; back
  Object.entries(contexts).forEach(([k,v])=>{const o=document.createElement("option"); o.value=k; o
  wrap.appendChild(select); controls.appendChild(wrap);
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 680 220");
  svg.setAttribute("width", "100%");
  svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10px
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

  function drawArrow(x1,y1,x2,y2,color="#374151"){const l=svgEl("line"); l.setAttribute("x1",x1); l
  function text(x,y,t,fill="#374151",size=12,anchor="middle",weight="400"){const n=svgEl("text"); n

  function redraw(){
    while(svg.firstChild) svg.removeChild(svg.firstChild);
    const c = contexts[select.value];
    const boxes = [
      { x: 30, y: 78, w: 130, h: 54, fill: "#eff6ff", stroke: "#2563eb", title: "hidden state", body
      { x: 215, y: 78, w: 130, h: 54, fill: "#f8fafc", stroke: "#9ca3af", title: "observation operat
      { x: 400, y: 78, w: 110, h: 54, fill: "#fef3c7", stroke: "#f59e0b", title: "measurement", body
      { x: 555, y: 78, w: 100, h: 54, fill: "#ecfdf5", stroke: "#059669", title: "updated estimate",
```

```
    ];
    boxes.forEach(b => { const r=svgEl("rect"); r.setAttribute("x",b.x); r.setAttribute("y",b.y);
    drawArrow(160,105,215,105);
    drawArrow(345,105,400,105);
    drawArrow(510,105,555,105);
    drawArrow(605,132,605,182,"#7c3aed");
    drawArrow(605,182,95,182,"#7c3aed");
    drawArrow(95,182,95,132,"#7c3aed");
    text(452,172,"predict-update loop","#7c3aed",12,"middle","700");
    text(455,58,"same abstract pipeline in every field","#6b7280",12);
    caption.textContent = "The box structure stays the same across process monitoring, navigation
  }

  select.addEventListener("input", redraw);
  container.append(controls, svg, caption);
  redraw();
  return container;
}
```

# 64. Reliability, stochastic systems, and quality

Failure, variability, and risk over time

One bearing fails early, another lasts for years. A queue stays calm all morning, then suddenly grows faster than it clears. A production line meets tolerance most days but not all days. Randomness is not a side issue here. It changes maintenance, staffing, safety, and design.

Engineering decisions are made before the exact future is known. You do not know which unit will fail next, exactly when a line will back up, or how many defects will appear in the next batch. What you can do is model the variability honestly and let that model shape action.

Three linked ideas run through this chapter: lifetime, queueing, and quality. They look like separate professional topics. They are all ways of asking how randomness accumulates into operational consequence.

---

## 64.1. Reliability as survival over time

Let $T$ be a random lifetime. The **reliability function** is

$$R(t) = P(T > t)$$

This is the probability that the component or system survives beyond time $t$.

The **failure distribution** is

$$F(t) = P(T \leq t) = 1 - R(t)$$

If the lifetime is exponentially distributed with constant hazard rate $\lambda$, then

$$R(t) = e^{-\lambda t}$$

and

$$F(t) = 1 - e^{-\lambda t}$$

This is the most common first model: simple, and with a clear interpretation — the system has a constant failure tendency per unit time.

The **hazard rate** $\lambda(t)$ describes the instantaneous failure tendency conditional on survival up to time $t$. Let $f(t) = F'(t)$ be the probability density of the lifetime. Since $R(t) = 1 - F(t)$, differentiating gives $R'(t) = -f(t)$. The hazard rate is then the density of failure conditional on having survived so far:

$$\lambda(t) = \frac{f(t)}{R(t)} = \frac{-R'(t)}{R(t)}$$

For the exponential model $\lambda(t) = \lambda$ (constant), which is why the scalar $\lambda$ appears in both the survival function and the hazard. This means hazard is about conditional risk right now, given survival so far, not about total failure probability over the whole lifetime.

### 64.1.1. Weibull model for non-constant hazard

The exponential model's constant hazard is a special case that real components often do not satisfy. The **Weibull distribution** is the standard extension:

$$R(t) = e^{-(t/\eta)^\beta}$$

where $\eta > 0$ is the scale (characteristic life) and $\beta > 0$ is the shape parameter. The hazard rate is

$$\lambda(t) = \frac{\beta}{\eta}\left(\frac{t}{\eta}\right)^{\beta-1}$$

When $\beta = 1$ the Weibull reduces to the exponential. When $\beta > 1$ the hazard increases with age — wear-out behaviour typical of fatigue, corrosion, and mechanical degradation. When $\beta < 1$ the hazard decreases — early-life failures (infant mortality). Reliability engineers fit $\beta$ from field data and use the value to decide whether early screening, scheduled replacement, or condition monitoring is the appropriate response.

In some systems hazard is approximately constant over the operational window of interest and the exponential model is adequate. In others the shape parameter matters for every maintenance and safety decision.

The explorer below links survival, cumulative failure, and hazard at the same time marker.

```
//| echo: false
{
  const W = 760, H = 250;
  let beta = 1.4;
  let eta = 120;
  let tm = 90;

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }
```

```
const container = document.createElement("div");
container.style.cssText = "max-width: 820px; margin: 1rem 0 1.25rem 0; font-family: inherit;";

const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem
function sliderRow(labelText, min, max, step, value) {
  const label = document.createElement("label");
  label.textContent = labelText;
  const input = document.createElement("input");
  input.type = "range";
  input.min = String(min);
  input.max = String(max);
  input.step = String(step);
  input.value = String(value);
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; min-width:4.2rem;";
  controls.append(label, input, out);
  return { input, out };
}
const betaRow = sliderRow("Weibull shape □", 0.4, 3, 0.1, beta);
const etaRow = sliderRow("Scale □", 40, 220, 5, eta);
const tmRow = sliderRow("Maintenance marker t□", 10, 220, 5, tm);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
function card(label) {
  const box = document.createElement("div");
  box.style.cssText = "background:#f8fafc; border:1px solid #e5e7eb; border-radius:8px; padding
  const k = document.createElement("div");
  k.style.cssText = "font-size:0.76rem; color:#6b7280;";
  k.textContent = label;
  const v = document.createElement("div");
  v.style.cssText = "font-size:0.96rem; font-variant-numeric: tabular-nums;";
  box.append(k, v);
  readout.appendChild(box);
  return v;
}
const regimeBox = card("Behaviour");
const rBox = card("R(t□)");
const fBox = card("F(t□)");
const hBox = card("□(t□)");

const board = document.createElement("div");
board.style.cssText = "display:grid; grid-template-columns:repeat(3,minmax(0,1fr)); gap:0.75rem

function makePanel(title) {
  const panel = document.createElement("div");
```

```
  panel.style.cssText = "background:#fff; border:1px solid #e5e7eb; border-radius:10px; padding:0
  const h = document.createElement("div");
  h.style.cssText = "font-size:0.8rem; color:#6b7280; text-transform:uppercase; letter-spacing:0.0
  h.textContent = title;
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", `0 0 ${W / 3 - 20} ${H}`);
  svg.setAttribute("width", "100%");
  svg.style.display = "block";
  panel.append(h, svg);
  board.appendChild(panel);
  return svg;
}

const svgR = makePanel("Reliability R(t)");
const svgF = makePanel("Failure F(t)");
const svgH = makePanel("Hazard ?(t)");
const caption = document.createElement("p");
caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";

function drawCurve(svg, fn, yMax, color, markerValue) {
  while (svg.firstChild) svg.removeChild(svg.firstChild);
  const w = W / 3 - 20, h = H;
  const left = 34, right = 14, top = 10, bottom = 26;
  const plotW = w - left - right, plotH = h - top - bottom;
  const sx = t => left + plotW * t / 220;
  const sy = y => top + plotH * (1 - y / yMax);

  for (let i = 0; i <= 4; i++) {
    const gy = yMax * i / 4;
    const line = svgEl("line");
    line.setAttribute("x1", sx(0));
    line.setAttribute("x2", sx(220));
    line.setAttribute("y1", sy(gy));
    line.setAttribute("y2", sy(gy));
    line.setAttribute("stroke", "#f3f4f6");
    svg.appendChild(line);
  }
  const xAxis = svgEl("line");
  xAxis.setAttribute("x1", sx(0)); xAxis.setAttribute("x2", sx(220));
  xAxis.setAttribute("y1", sy(0)); xAxis.setAttribute("y2", sy(0));
  xAxis.setAttribute("stroke", "#6b7280");
  svg.appendChild(xAxis);
  const yAxis = svgEl("line");
  yAxis.setAttribute("x1", sx(0)); yAxis.setAttribute("x2", sx(0));
  yAxis.setAttribute("y1", sy(0)); yAxis.setAttribute("y2", sy(yMax));
  yAxis.setAttribute("stroke", "#6b7280");
  svg.appendChild(yAxis);
```

```
  const pts = [];
  for (let t = 0; t <= 220; t += 2) pts.push(`${sx(t)},${sy(fn(t))}`);
  const curve = svgEl("polyline");
  curve.setAttribute("points", pts.join(" "));
  curve.setAttribute("fill", "none");
  curve.setAttribute("stroke", color);
  curve.setAttribute("stroke-width", "2.4");
  svg.appendChild(curve);

  const mark = svgEl("line");
  mark.setAttribute("x1", sx(tm)); mark.setAttribute("x2", sx(tm));
  mark.setAttribute("y1", sy(0)); mark.setAttribute("y2", sy(yMax));
  mark.setAttribute("stroke", "#7c3aed");
  mark.setAttribute("stroke-dasharray", "5 4");
  svg.appendChild(mark);

  const dot = svgEl("circle");
  dot.setAttribute("cx", sx(tm));
  dot.setAttribute("cy", sy(markerValue));
  dot.setAttribute("r", "4.3");
  dot.setAttribute("fill", "#7c3aed");
  svg.appendChild(dot);
}

function redraw() {
  beta = parseFloat(betaRow.input.value);
  eta = parseFloat(etaRow.input.value);
  tm = parseFloat(tmRow.input.value);
  betaRow.out.textContent = beta.toFixed(1);
  etaRow.out.textContent = eta.toFixed(0);
  tmRow.out.textContent = tm.toFixed(0);

  const R = t => Math.exp(-Math.pow(t / eta, beta));
  const F = t => 1 - R(t);
  const Hfn = t => beta === 1 && t === 0 ? 1 / eta : (beta / eta) * Math.pow(Math.max(t, 1e-6)
  const hMax = Math.max(0.03, ...Array.from({ length: 111 }, (_, i) => Hfn(2 * i)));

  drawCurve(svgR, R, 1, "#2563eb", R(tm));
  drawCurve(svgF, F, 1, "#f59e0b", F(tm));
  drawCurve(svgH, Hfn, hMax, "#ef4444", Hfn(tm));

  regimeBox.textContent = beta < 1 ? "infant mortality" : beta > 1 ? "wear-out" : "constant haz
  regimeBox.style.color = beta < 1 ? "#b45309" : beta > 1 ? "#b91c1c" : "#2563eb";
  rBox.textContent = R(tm).toFixed(3);
  fBox.textContent = F(tm).toFixed(3);
  hBox.textContent = Hfn(tm).toFixed(4);
  caption.textContent = beta < 1
```

```
      ? "β < 1 means the hazard falls with age: weak units fail early, and survivors become safer. T
      : beta > 1
        ? "β > 1 means the hazard rises with age: wear-out dominates, so delaying replacement become
        : "β = 1 gives the exponential model: the hazard stays flat, so age alone does not make imme
  }

  [betaRow.input, etaRow.input, tmRow.input].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, readout, board, caption);
  redraw();
  return container;
}
```

## 64.2. Queueing as accumulation under uncertainty

Instead of asking when a unit fails, ask what happens when random arrivals meet random service.

In a queueing system:

- arrivals are uncertain in time
- service completions are uncertain in time
- if arrivals temporarily outpace service, a queue forms

Two core quantities are:

$$L_q = \text{expected number waiting in queue}$$

$$W_q = \text{expected waiting time in queue}$$

These are connected by **Little's Law**:

$$L_q = \lambda_a W_q$$

The expected queue length equals the arrival rate times the expected waiting time. Little's Law holds under mild conditions regardless of the arrival or service distribution — it is a conservation statement about flow rather than an assumption about randomness.

For the system to reach a steady state, arrivals must not permanently outpace service: the condition is $\rho < 1$. If $\rho \geq 1$ the queue grows without bound — the system is unstable and no finite long-run average exists. When $\rho < 1$ but is close to 1, the queue length grows sharply, even when mean demand still looks safely below capacity.

Variability matters even when average values look harmless. A system with average arrival rate only slightly below service rate can still produce long queues if the variability is large enough.

For the simplest tractable case — Poisson arrivals, exponential service, one server (M/M/1) — the expected queue length has a closed form:

$$L_q = \frac{\rho^2}{1-\rho}$$

This grows steeply as $\rho \to 1$. The worked example below shows the numbers.

This is why service reliability in computing and waiting-time models in operations research belong in the same chapter as bearing lifetime and defect rates. They are all stochastic consequences of uncertain events over time.

```
//| echo: false
{
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 720px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; margin-bottom:0.75rem;";
  const wrap = document.createElement("label");
  wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  wrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Traffic intensit
  const rho = document.createElement("input");
  rho.type = "range"; rho.min = "0.1"; rho.max = "0.98"; rho.step = "0.01"; rho.value = "0.75";
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
  wrap.append(rho, out);
  controls.appendChild(wrap);
  const readout = document.createElement("div");
  readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
  function card(label){const box=document.createElement("div"); box.style.cssText="background:#f8
  const lqBox = card("Expected queue Lq");
  const regimeBox = card("Regime");
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 680 240");
  svg.setAttribute("width", "100%");
  svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";
  function redraw(){
    while(svg.firstChild) svg.removeChild(svg.firstChild);
    const r = parseFloat(rho.value);
    out.textContent = r.toFixed(2);
    const lq = (r*r)/(1-r);
    const sx = x => 60 + 540 * (x - 0.1) / 0.88;
    const sy = y => 210 - 170 * y / 20;
    const line=(x1,y1,x2,y2,stroke,w=1.5,dash="")=>{const l=svgEl("line"); l.setAttribute("x1",x1
    const text=(x,y,t,fill="#374151",size=12,anchor="middle",weight="400")=>{const n=svgEl("text"
    line(sx(0.1),sy(0),sx(0.98),sy(0),"#6b7280"); line(sx(0.1),sy(0),sx(0.1),sy(20),"#6b7280");
    const pts=[]; for(let x=0.1;x<=0.98;x+=0.01){ pts.push(`${sx(x)},${sy(Math.min(20,(x*x)/(1-x)
    const path=svgEl("polyline"); path.setAttribute("points",pts.join(" ")); path.setAttribute("f
```

```
    line(sx(0.9), sy(0), sx(0.9), sy(20), "#ef4444", 1.5, "5 4");
    const dot=svgEl("circle"); dot.setAttribute("cx",sx(r)); dot.setAttribute("cy",sy(Math.min(20,lq
    text(sx(0.9), 30, "near-saturation zone", "#b91c1c", 11);
    lqBox.textContent = lq.toFixed(2);
    regimeBox.textContent = r < 0.7 ? "comfortable" : r < 0.9 ? "sensitive" : "queue blows up fast"
    regimeBox.style.color = r < 0.7 ? "#166534" : r < 0.9 ? "#b45309" : "#b91c1c";
    caption.textContent = "As traffic intensity approaches 1, expected queue length rises sharply. T
  }
  rho.addEventListener("input", redraw);
  container.append(controls, readout, svg, caption);
  redraw();
  return container;
}
```

## 64.3. Quality as probability of staying in tolerance

A quality problem can often be phrased as:

- what is the probability that a product characteristic falls outside the acceptable region?

If a machined diameter is modelled as approximately normal with mean $\mu$ and standard deviation $\sigma$, then defect probability is the area in the tails beyond the specification limits.

In manufacturing engineering this is operationalised through the **process capability index**. The most common form is

$$C_p = \frac{\text{USL} - \text{LSL}}{6\sigma}$$

where USL (upper specification limit) and LSL (lower specification limit) are the boundaries of acceptable product and $6\sigma$ is the spread of the process distribution. A value of $C_p \geq 1.33$ is a common minimum standard in production; it means the process spread fits inside the tolerance window with margin. The one-sided version $C_{pk}$ accounts for centring:

$$C_{pk} = \min\left(\frac{\text{USL} - \mu}{3\sigma}, \frac{\mu - \text{LSL}}{3\sigma}\right)$$

A process can have adequate spread ($C_p$ high) but still produce defects if the mean is off-centre ($C_{pk}$ low). Capability indices are the bridge between the tail-area probability from the normal model and the tolerance decisions made in a production environment.

This is not only a manufacturing question. The same logic appears in service level agreements, anomaly thresholds, and risk tolerances in data systems.

```
//| echo: false
{
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 720px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; align-items:end; margin-bo
  function sliderRow(labelText,min,max,step,value){const wrap=document.createElement("label"); wr
  const mu = sliderRow("Process mean □", 8.5, 11.5, 0.1, 10.0);
  const sigma = sliderRow("Spread □", 0.2, 1.2, 0.05, 0.5);
  const readout = document.createElement("div");
  readout.style.cssText = "display:flex; gap:0.75rem; flex-wrap:wrap; margin-bottom:0.65rem;";
  function card(label){const box=document.createElement("div"); box.style.cssText="background:#f8
  const cpBox = card("Cp");
  const cpkBox = card("Cpk");
  const statusBox = card("Interpretation");
  const svg = svgEl("svg");
  svg.setAttribute("viewBox", "0 0 680 240");
  svg.setAttribute("width", "100%");
  svg.style.cssText = "display:block; background:#fff; border:1px solid #e5e7eb; border-radius:10
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";
  const normal = z => Math.exp(-0.5*z*z)/Math.sqrt(2*Math.PI);
  function redraw(){
    while(svg.firstChild) svg.removeChild(svg.firstChild);
    const m = parseFloat(mu.input.value), s = parseFloat(sigma.input.value);
    mu.out.textContent = m.toFixed(1); sigma.out.textContent = s.toFixed(2);
    const USL = 11, LSL = 9;
    const mapX = x => 60 + 540 * (x - 7) / 6;
    const mapY = y => 200 - 140 * y / 0.9;
    const line=(x1,y1,x2,y2,stroke,w=1.5,dash="")=>{const l=svgEl("line"); l.setAttribute("x1",x1
    const text=(x,y,t,fill="#374151",size=12,anchor="middle",weight="400")=>{const n=svgEl("text"
    line(mapX(7), mapY(0), mapX(13), mapY(0), "#6b7280");
    const pts=[]; for(let x=7;x<=13;x+=0.03){ pts.push(`${mapX(x)},${mapY(normal((x-m)/s)/s)}`);
    const curve=svgEl("polyline"); curve.setAttribute("points",pts.join(" ")); curve.setAttribute
    line(mapX(LSL), mapY(0), mapX(LSL), mapY(0.85), "#ef4444", 2, "5 4");
    line(mapX(USL), mapY(0), mapX(USL), mapY(0.85), "#ef4444", 2, "5 4");
    text(mapX(LSL), 44, "LSL", "#b91c1c", 11);
    text(mapX(USL), 44, "USL", "#b91c1c", 11);
    text(mapX(m), 58, "□", "#059669", 12, "middle", "700");
    const cp = (USL-LSL)/(6*s);
    const cpk = Math.min((USL-m)/(3*s),(m-LSL)/(3*s));
    cpBox.textContent = cp.toFixed(2);
    cpkBox.textContent = cpk.toFixed(2);
    statusBox.textContent = cpk >= 1.33 ? "spread and centring both look acceptable" : cp >= 1.33
    caption.textContent = "Cp measures whether the process spread fits inside the tolerance windo
  }
```

```
  [mu.input, sigma.input].forEach(el => el.addEventListener("input", redraw));
  container.append(controls, readout, svg, caption);
  redraw();
  return container;
}
```

**Why this structure works**

Reliability, queueing, and quality are all asking the same underlying question: given a random process, what is the probability that a threshold is crossed? For reliability the threshold is failure; for queueing it is demand exceeding capacity; for quality it is a measurement leaving the specification band.
That shared structure means the same probabilistic tools — tail probabilities, expected values, densities — apply across all three, and a result in one domain often has a direct analogue in another.

## 64.4. The core method

A first pass through a stochastic reliability or quality problem usually goes like this:

1. Identify the random quantity: lifetime, arrivals, service time, or process variation.
2. Choose the summary function that matches the decision: reliability, hazard, queue length, waiting time, or defect probability.
3. Compute the key probability or expected value.
4. Interpret it in operational terms.
5. Ask whether the assumed stochastic model is realistic enough for the decision you are about to make.

That final question is essential. A neat model with the wrong variability structure can be more dangerous than admitting uncertainty openly.

## 64.5. Worked example 1: exponential reliability

Suppose a component has constant failure rate

$$\lambda = 0.002\ \mathrm{h}^{-1}$$

Then the reliability at 100 hours is

$$R(100) = e^{-0.002(100)} = e^{-0.2} \approx 0.819$$

So there is about an 81.9% chance the component survives beyond 100 hours. Equivalently, the probability of failure by 100 hours is

$$F(100) = 1 - 0.819 = 0.181$$

This is already enough to frame a maintenance question. If a system contains many such components, or if failure consequence is high, 18.1% may be too risky to tolerate over that interval.

## 64.6. Worked example 2: a queue near saturation

A service system receives jobs at average rate

$$\lambda_a = 8 \text{ jobs/h}$$

and can serve at average rate

$$\mu = 10 \text{ jobs/h}$$

The traffic intensity is

$$\rho = \frac{\lambda_a}{\mu} = 0.8$$

For an M/M/1 queue (Poisson arrivals, exponential service, single server), the expected number waiting in queue is

$$L_q = \frac{\rho^2}{1-\rho} = \frac{0.64}{0.2} = 3.2 \text{ jobs}$$

By Little's Law the expected waiting time in queue is

$$W_q = \frac{L_q}{\lambda_a} = \frac{3.2}{8} = 0.4 \text{ h}$$

So at 80% utilisation, a job waits on average 24 minutes before being served. If utilisation rises to $\rho = 0.9$, $L_q = 8.1$ jobs and $W_q = 1.01$ h — more than twice as long for a 12.5% increase in load. That nonlinear blowup is the engineering message.

Systems that look adequate on average can still feel overloaded because variability is experienced in real time, not as a long-run mean.

## 64.7. Worked example 3: defect probability from a normal model

Suppose a manufactured diameter is approximately normal with mean

$$\mu = 10.0 \text{ mm}$$

and standard deviation

$$\sigma = 0.1 \text{ mm}$$

The upper specification limit is 10.2 mm. Standardise:

$$z = \frac{10.2 - 10.0}{0.1} = 2$$

So the probability of exceeding the upper limit is about

$$P(Z > 2) \approx 0.0228$$

or 2.28%.

That tail probability is the mathematical link between observed variation and quality cost. It is also the same logic used in anomaly detection and service thresholds outside manufacturing.

## 64.8. Where this goes

The next continuation is Nonlinear optimisation for design and operations. Once risk, waiting, and failure are quantified, design decisions become tradeoffs between cost, performance, safety, and uncertainty.

**Applications**

- maintenance interval planning
- reliability engineering and survival analysis (exponential and Weibull models)
- queueing in production, logistics, and computing (M/M/1 and related models)
- defect rates and process capability ($C_p$, $C_{pk}$)
- service availability and incident risk
- uncertainty-aware operations decisions

## 64.9. Exercises

These are project-style exercises. State the operational meaning of the number you compute.

### 64.9.1. Exercise 1

A component has exponential lifetime with failure rate

$$\lambda = 0.005\ \mathrm{h}^{-1}$$

Compute the probability it survives beyond 50 hours.

### 64.9.2. Exercise 2

A queueing system has average arrival rate 12 jobs/h and service rate 15 jobs/h.

1. Compute the traffic intensity $\rho$.
2. Explain whether the system has generous slack or operates near saturation.
3. Suppose demand grows to 14 jobs/h without any change in service rate. What happens to $\rho$, and at what arrival rate does the system become dangerously loaded (say, $\rho > 0.95$)? What operational decision does this suggest?

### 64.9.3. Exercise 3

A process characteristic is approximately normal with mean 25 and standard deviation 2. The upper specification limit is 29.

Compute the corresponding $z$ value and explain what tail probability it refers to.

### 64.9.4. Exercise 4

Choose one stochastic setting from your field and prepare a one-page model brief naming:

1. the random quantity
2. the decision you need to support
3. the probability or expectation that matters
4. one source of model mismatch
5. one action you would change if the risk turned out to be higher than expected

```
//| echo: false
{
  function svgEl(tag){return document.createElementNS("http://www.w3.org/2000/svg", tag);}
  const container = document.createElement("div");
  container.style.cssText = "max-width: 920px; margin: 1rem 0 1.25rem 0; font-family: inherit;";
  const controls = document.createElement("div");
  controls.style.cssText = "display:flex; gap:0.75rem; align-items:end; margin-bottom:0.75rem;";
  const wrap = document.createElement("label");
  wrap.style.cssText = "display:grid; gap:0.25rem; font-size:0.9rem; color:#374151;";
  wrap.appendChild(Object.assign(document.createElement("span"), { textContent: "Variability" }))
  const slider = document.createElement("input");
```

```
  slider.type = "range"; slider.min = "0.2"; slider.max = "1.4"; slider.step = "0.05"; slider.value
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric: tabular-nums; color:#6b7280;";
  wrap.append(slider, out); controls.appendChild(wrap);
  const board = document.createElement("div");
  board.style.cssText = "display:grid; grid-template-columns:repeat(3,minmax(0,1fr)); gap:0.75rem;"
  const caption = document.createElement("p");
  caption.style.cssText = "font-size:0.9rem; color:#4b5563; margin:0.55rem 0 0 0;";
  function panel(title, build){const p=document.createElement("div"); p.style.cssText="background:#
  const panels = [
    panel("Reliability", (svg,v)=>{ while(svg.firstChild) svg.removeChild(svg.firstChild); const pts
    panel("Queueing", (svg,v)=>{ while(svg.firstChild) svg.removeChild(svg.firstChild); const rho=Ma
    panel("Quality", (svg,v)=>{ while(svg.firstChild) svg.removeChild(svg.firstChild); const normal=
  ];
  function redraw(){const v=parseFloat(slider.value); out.textContent=v.toFixed(2); panels.forEach(p
  slider.addEventListener("input", redraw);
  container.append(controls, board, caption);
  redraw();
  return container;
}
```

# 65. Nonlinear optimisation for design and operations

Tradeoffs, constraints, and the shape of a best solution

Make the structure lighter without making it fail. Increase throughput without violating a pressure limit. Improve prediction quality without blowing the compute budget. Once the world stops being linear, "the best answer" is no longer a corner point on a clean polygon. It is a shape in a curved space, and getting there becomes part geometry, part computation, part judgment.

This chapter gathers the themes that the earlier chapters developed separately. Simulation creates a model of system behaviour. Estimation tells you what is known and uncertain. Reliability tells you what risk costs you. Optimisation turns all of that into a decision rule.

When the objective and constraints are nonlinear in the design variables, the geometry changes, the algorithms change, and the meaning of "best" becomes more local and more conditional.

---

## 65.1. From linear to nonlinear thinking

In linear programming, the feasible set is a polytope and the optimum sits at a vertex. In nonlinear optimisation, neither statement is generally true.

You now have:

- a nonlinear objective $f(\mathbf{x})$
- nonlinear constraints $g_i(\mathbf{x}) \leq 0$
- possibly many local optima instead of one clean global answer

The design variable vector $\mathbf{x}$ may represent dimensions, control settings, schedule variables, material choices, or hyperparameters. The mathematics does not care what professional name the variables carry. It cares how objective and constraints bend the space.

The simplest unconstrained update rule is gradient descent:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$$

The negative gradient points in the direction of steepest local decrease. Choose a step size $\alpha$, move, and repeat.

In practice the difficulties are exactly the ones you would expect in a curved space: the descent direction is only locally good, the step size can be too large or too small, and constraints can make the locally best direction infeasible.

**Convergence and local optima.** Gradient descent converges to a *local* minimum, not necessarily a global one. For *convex* problems — where the feasible set is convex and the objective has no local minima that are not also global — any stationary point found is a global optimum and convergence guarantees are strong. For *non-convex* problems, gradient descent may stop at a local minimum or saddle point. The algorithm has no way to know whether a better basin exists elsewhere in the space. Global optimisation (basin-hopping, simulated annealing, branch-and-bound for mixed-integer problems) is a separate, harder task. Most real engineering design problems and essentially all deep-learning objectives are non-convex.

**Step size.** A fixed $\alpha$ that is too large causes divergence; too small makes convergence impractically slow. In practice, the step size is set by a *line search* — a procedure that finds an $\alpha$ satisfying a formal sufficient-decrease condition. The Armijo condition requires that the objective drop by at least a fixed fraction of what a full gradient step would give; the stronger Wolfe conditions additionally bound how much the gradient is allowed to change. Either condition ensures progress without committing to a fixed schedule. Alternatively, an adaptive schedule decreases $\alpha$ over iterations. Automatic differentiation and adaptive-rate optimisers have made fixed-rate gradient descent rare in production systems.

> **Computing note: batch, stochastic, and mini-batch gradient descent**
>
> The rule above evaluates the full gradient over all data at each step — this is **batch gradient descent**. In machine learning, where $f$ is a loss summed over millions of examples, evaluating the full gradient per step is computationally prohibitive.
> **Stochastic gradient descent** (SGD) in its strict form uses a *single* randomly chosen example per step. The gradient estimate is very noisy but each step is cheap. In practice, training uses **mini-batch SGD**: the gradient is estimated from a randomly sampled subset (a mini-batch, typically 32–512 examples), balancing noise against computational cost.
> Modern optimisers — **Adam**, **AdaGrad**, **RMSProp** — extend mini-batch SGD by maintaining per-parameter step sizes. AdaGrad accumulates the squared gradients and divides by their square root, shrinking the effective step for parameters that receive large or frequent updates. RMSProp uses an exponential moving average of squared gradients rather than accumulating them, preventing the step size from decaying to zero. Adam combines an RMSProp-style second moment estimate with a first moment estimate (momentum), making it an adaptive gradient method with memory of both direction and curvature. All three are still gradient-based and inherit the local-minimum limitation of gradient descent.
> The convergence theory for stochastic methods differs from the deterministic case (requires diminishing step-size schedules or noise conditions), but the geometric intuition is the same: move opposite to an estimate of the gradient, iterate, and check for convergence.

Use the point controls and the descent button to compare a free downhill move with a move that has to stay inside the feasible set.

```
//| echo: false
{
  const W = 720, H = 430;
  const xMin = -3, xMax = 3, yMin = -3, yMax = 3;
```

```
const alpha = 0.18;
let x = 1.9, y = 1.6;
let constrained = true;

function svgEl(tag) {
  return document.createElementNS("http://www.w3.org/2000/svg", tag);
}

function f(x, y) {
  return 0.55 * (x - 1.4) ** 2 + 1.2 * (y + 0.7) ** 2 + 0.45 * x * y + 0.08 * x ** 4;
}

function grad(x, y) {
  return {
    gx: 1.1 * (x - 1.4) + 0.45 * y + 0.32 * x ** 3,
    gy: 2.4 * (y + 0.7) + 0.45 * x
  };
}

function feasible(x, y) {
  return x * x + (y - 0.2) * (y - 0.2) <= 5.4 && y >= -1.8;
}

function projectToFeasible(px, py) {
  let yProj = Math.max(py, -1.8);
  const dx = px;
  const dy = yProj - 0.2;
  const r = Math.sqrt(dx * dx + dy * dy);
  if (r > Math.sqrt(5.4)) {
    const scale = Math.sqrt(5.4) / r;
    return { x: dx * scale, y: 0.2 + dy * scale };
  }
  return { x: px, y: yProj };
}

function sx(v) { return 50 + (v - xMin) / (xMax - xMin) * 620; }
function sy(v) { return 370 - (v - yMin) / (yMax - yMin) * 320; }

const container = document.createElement("div");
container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem

function sliderRow(labelText, min, max, value, step) {
  const label = document.createElement("label");
  label.textContent = labelText;
  const input = document.createElement("input");
```

```
  input.type = "range";
  input.min = String(min);
  input.max = String(max);
  input.step = String(step);
  input.value = String(value);
  const out = document.createElement("span");
  out.style.cssText = "font-variant-numeric:tabular-nums; min-width:4rem;";
  controls.append(label, input, out);
  return { input, out };
}

const xRow = sliderRow("Point x", -2.5, 2.5, x, 0.05);
const yRow = sliderRow("Point y", -2.2, 2.5, y, 0.05);
const modeWrap = document.createElement("label");
modeWrap.style.cssText = "display:flex; gap:0.5rem; align-items:center; margin-top:0.25rem;";
const modeBox = document.createElement("input");
modeBox.type = "checkbox";
modeBox.checked = constrained;
const modeText = document.createElement("span");
modeText.textContent = "Constrained step";
modeWrap.append(modeBox, modeText);
controls.append(modeWrap);

const actions = document.createElement("div");
actions.style.cssText = "display:flex; gap:0.6rem; margin:0.35rem 0 0.8rem 0;";
const stepBtn = document.createElement("button");
stepBtn.textContent = "Take descent step";
const resetBtn = document.createElement("button");
resetBtn.textContent = "Reset point";
actions.append(stepBtn, resetBtn);

const readout = document.createElement("div");
readout.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3341
const pointBox = document.createElement("span");
const fBox = document.createElement("span");
const gradBox = document.createElement("span");
const feasBox = document.createElement("span");
readout.append(pointBox, fBox, gradBox, feasBox);

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";
svg.setAttribute("role", "img");
svg.setAttribute("aria-label", "Nonlinear optimisation contour map with feasible region and gradie
```

```
const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

container.append(controls, actions, readout, svg, caption);

function redraw() {
  x = Number(xRow.input.value);
  y = Number(yRow.input.value);
  constrained = modeBox.checked;
  xRow.out.textContent = x.toFixed(2);
  yRow.out.textContent = y.toFixed(2);
  svg.replaceChildren();

  for (let i = 0; i < 40; i++) {
    for (let j = 0; j < 32; j++) {
      const gx = xMin + (i + 0.5) / 40 * (xMax - xMin);
      const gy = yMin + (j + 0.5) / 32 * (yMax - yMin);
      const val = f(gx, gy);
      const t = Math.max(0, Math.min(1, (val - 0.2) / 8));
      const rect = svgEl("rect");
      rect.setAttribute("x", String(50 + i * (620 / 40)));
      rect.setAttribute("y", String(50 + (31 - j) * (320 / 32)));
      rect.setAttribute("width", String(620 / 40 + 0.6));
      rect.setAttribute("height", String(320 / 32 + 0.6));
      const light = 94 - 22 * t;
      rect.setAttribute("fill", `hsl(205 70% ${light}%)`);
      svg.appendChild(rect);
    }
  }

  const axes = svgEl("g");
  [["x", 350, 390, 670, 390], ["y", 50, 210, 50, 50]].forEach(([label, x1, y1, x2, y2]) => {
    const line = svgEl("line");
    line.setAttribute("x1", x1); line.setAttribute("y1", y1);
    line.setAttribute("x2", x2); line.setAttribute("y2", y2);
    line.setAttribute("stroke", "#64748b");
    line.setAttribute("stroke-width", "1.2");
    axes.appendChild(line);
    const text = svgEl("text");
    text.setAttribute("x", x2);
    text.setAttribute("y", y2 === 390 ? 408 : 42);
    text.setAttribute("fill", "#475569");
    text.setAttribute("font-size", "12");
    text.textContent = label;
    axes.appendChild(text);
  });
  svg.appendChild(axes);
```

```
const feasiblePath = [];
for (let k = 0; k <= 120; k++) {
  const th = Math.PI * (1 - k / 120);
  feasiblePath.push(`${k === 0 ? "M" : "L"} ${sx(Math.sqrt(5.4) * Math.cos(th))} ${sy(0.2 + Mat
}
feasiblePath.push(`L ${sx(-Math.sqrt(5.4))} ${sy(-1.8)}`);
feasiblePath.push(`L ${sx(Math.sqrt(5.4))} ${sy(-1.8)} Z`);
const patch = svgEl("path");
patch.setAttribute("d", feasiblePath.join(" "));
patch.setAttribute("fill", "rgba(16,185,129,0.16)");
patch.setAttribute("stroke", "#059669");
patch.setAttribute("stroke-width", "2");
svg.appendChild(patch);

const g = grad(x, y);
const gLen = Math.hypot(g.gx, g.gy) || 1;
const scale = 0.55 / gLen;
const x2 = x + g.gx * scale;
const y2 = y + g.gy * scale;
const arrow = svgEl("line");
arrow.setAttribute("x1", String(sx(x)));
arrow.setAttribute("y1", String(sy(y)));
arrow.setAttribute("x2", String(sx(x2)));
arrow.setAttribute("y2", String(sy(y2)));
arrow.setAttribute("stroke", "#dc2626");
arrow.setAttribute("stroke-width", "3");
svg.appendChild(arrow);

const head = svgEl("circle");
head.setAttribute("cx", String(sx(x2)));
head.setAttribute("cy", String(sy(y2)));
head.setAttribute("r", "4");
head.setAttribute("fill", "#dc2626");
svg.appendChild(head);

const pt = svgEl("circle");
pt.setAttribute("cx", String(sx(x)));
pt.setAttribute("cy", String(sy(y)));
pt.setAttribute("r", "6");
pt.setAttribute("fill", feasible(x, y) ? "#1d4ed8" : "#b91c1c");
svg.appendChild(pt);

pointBox.textContent = `point = (${x.toFixed(2)}, ${y.toFixed(2)})`;
fBox.textContent = `f(x,y) = ${f(x, y).toFixed(3)}`;
gradBox.textContent = `|∇f| = ${gLen.toFixed(3)}`;
feasBox.textContent = feasible(x, y) ? "feasible" : "outside feasible set";
caption.textContent = constrained
```

```
      ? "The green region is feasible. In constrained mode, a descent step is clipped back to tha
      : "In unconstrained mode, the step follows the negative gradient directly, even if that wou
  }

  [xRow.input, yRow.input, modeBox].forEach(el => el.addEventListener("input", redraw));
  stepBtn.addEventListener("click", () => {
    const g = grad(x, y);
    let next = { x: x - alpha * g.gx, y: y - alpha * g.gy };
    if (modeBox.checked) next = projectToFeasible(next.x, next.y);
    xRow.input.value = String(next.x);
    yRow.input.value = String(next.y);
    redraw();
  });
  resetBtn.addEventListener("click", () => {
    xRow.input.value = "1.9";
    yRow.input.value = "1.6";
    modeBox.checked = true;
    redraw();
  });

  redraw();
  return container;
}
```

## 65.2. Constrained optimisation and the Lagrangian idea

Suppose you want to minimise $f(\mathbf{x})$ subject to an equality constraint

$$h(\mathbf{x}) = 0$$

At an optimum, you usually cannot move in an arbitrary direction because many directions leave the feasible surface. The Lagrangian is

$$\mathscr{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda h(\mathbf{x})$$

The stationary conditions are

$$\nabla_{\mathbf{x}} \mathscr{L} = 0, \qquad h(\mathbf{x}) = 0$$

This means constrained optimality is a balance condition: at the solution, the objective cannot improve without pushing against the geometry of the constraint.

Geometrically, this means the gradient of the objective lines up with the gradient of the constraint. You cannot improve the objective without leaving feasibility.

For inequality constraints the full Karush-Kuhn-Tucker (KKT) conditions are needed. A constraint $g_i(\mathbf{x}) \leq 0$ is *active* at a point if $g_i(\mathbf{x}) = 0$ (the point sits on the constraint boundary) and *inactive* if the strict inequality holds (the constraint has slack). The KKT conditions extend the Lagrangian stationarity conditions by requiring that each active constraint contributes a non-negative multiplier and that inactive constraints contribute nothing ($\lambda_i = 0$). The intuition is the same as the equality case: at an optimum you cannot improve the objective without violating at least one binding constraint, so the gradients of objective and active constraints must be in balance.

## 65.3. Tradeoffs are the real subject

Nonlinear optimisation is a tradeoff chapter, not just an algorithm chapter.

If you minimise mass, stress may rise. If you maximise throughput, queueing risk may rise. If you improve prediction accuracy, compute cost may rise. The objective function is therefore not neutral. It encodes what you are willing to care about and what you are willing to pay.

That is why sensitivity analysis matters. A computed optimum is only useful if you can explain how it shifts when assumptions, weights, or constraints change.

> **A self-check**
>
> Before running any solver, you should be able to answer three questions about your formulation: (1) What does the objective reward or penalise? (2) Are the constraints physically motivated, or are some just computational convenience? (3) If the optimum shifts when you tighten a constraint by 10%, does that shift make physical sense?
> If you cannot answer all three, the formulation is not ready.

## 65.4. The core method

A first pass through a nonlinear optimisation problem usually goes like this:

1. Choose the design variables.
2. Write the objective in a way that matches the actual decision.
3. Write the constraints in physically meaningful form.
4. Inspect the geometry qualitatively before trusting an algorithm.
5. Use a numerical method appropriate to smoothness and constraint structure.
6. Check whether the result is feasible, local or global, and sensitive to modelling assumptions.

This is where all the earlier Volume 8 chapters return. If the model, estimate, or risk terms are poor, the optimisation result will be polished nonsense.

## 65.5. Worked example 1: unconstrained quadratic design objective

Minimise

$$f(x) = x^2 - 4x + 7$$

The derivative is

$$f'(x) = 2x - 4$$

Set it to zero:

$$2x - 4 = 0 \qquad \Rightarrow \qquad x = 2$$

The second derivative is

$$f''(x) = 2 > 0$$

so $x = 2$ is a local minimum. The minimum value is

$$f(2) = 4 - 8 + 7 = 3$$

The core logic is visible here in its simplest form: stationary point, then curvature check.

## 65.6. Worked example 2: equality-constrained optimisation

Minimise

$$f(x, y) = x^2 + y^2$$

subject to

$$x + y - 1 = 0$$

Form the Lagrangian:

$$\mathscr{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Take derivatives:

$$\frac{\partial \mathscr{L}}{\partial x} = 2x + \lambda = 0$$

$$\frac{\partial \mathscr{L}}{\partial y} = 2y + \lambda = 0$$

$$x + y - 1 = 0$$

The first two equations each give $\lambda = -2x$ and $\lambda = -2y$, so

$$x = y$$

Then the constraint gives

$$2x = 1 \qquad \Rightarrow \qquad x = y = \frac{1}{2}$$

So the feasible point closest to the origin is

$$\left(\frac{1}{2}, \frac{1}{2}\right)$$

This is the simplest geometric picture of constrained optimisation: the objective's level sets touch the feasible set at the best allowable point.

## 65.7. Worked example 3: design under a penalty tradeoff

Suppose an engineer uses the objective

$$J(x) = (x - 3)^2 + 0.5x^4$$

One term rewards staying near a target value $x = 3$; the quartic term penalises large magnitude. Real design objectives often mix performance and regularisation in exactly this way.

Different penalty weights produce different optima. That means the "best" design is partly a statement about priorities, not just about calculus.

In machine learning the same structure recurs. **Ridge regression** (L2 regularisation) adds a penalty $\lambda\|\mathbf{w}\|^2$ to the loss function, trading fit quality for smaller parameter magnitudes. The penalty weight $\lambda$ plays the same role as the 0.5 here: it encodes how strongly the solver is pushed away from large solutions, and changing it shifts the optimum. **Lasso** (L1 regularisation) uses $\lambda\|\mathbf{w}\|_1$ and produces sparse solutions — many weights driven to exactly zero — for a geometric reason: the L1 ball has corners on the coordinate axes, and when the objective's level sets first touch the constraint boundary they typically land at a corner. Note that the L1 norm is not differentiable at zero, so the standard Lagrangian gradient condition does not directly apply there; the analysis uses *subgradients*, which generalise gradients to non-smooth functions, but the physical intuition (level set touching constraint boundary at a corner) holds without that machinery. Portfolio risk penalties and operations scheduling with nonlinear costs follow the same two-term structure.

## 65.8. Where this goes

There is no single next topic from here. The continuation is the practice of using the whole Volume 8 stack together: models, estimates, risks, and objectives inside one decision loop.

The mathematics is no longer a supporting subject. It is the working medium of design judgment.

**Applications**

- shape and parameter optimisation in engineering design
- calibration and tuning of controllers
- operations and scheduling under nonlinear costs
- portfolio and risk optimisation
- hyperparameter and training-objective tuning in ML systems
- multi-objective design under uncertainty

## 65.9. Exercises

These are project-style exercises. Always explain what "better" means in the problem, not only where the derivative vanishes.

### 65.9.1. Exercise 1

Minimise

$$f(x) = x^2 - 6x + 10$$

Find the minimiser and the minimum value.

### 65.9.2. Exercise 2

Use Lagrange multipliers to minimise

$$f(x, y) = x^2 + 4y^2$$

subject to

$$x + y = 3$$

### 65.9.3. Exercise 3

An engineering team is choosing a design variable $x$ to minimise

$$J(x) = (x - 2)^2 + 0.2x^4$$

Write a short interpretation note answering:

1. what each term in the objective is doing
2. why this is not a linear optimisation problem
3. why changing the penalty weight 0.2 would change the preferred design

### 65.9.4. Exercise 4

Choose one optimisation problem from your own field and prepare a one-page design brief naming:

1. the design variables
2. the objective
3. the constraints
4. one source of nonlinearity
5. one uncertainty or reliability issue that should influence the formulation
6. one reason a computed optimum might still be a bad decision

Use the weight slider to change what the objective values most, then watch the best feasible design move along the curved constraints.

```
//| echo: false
{
  const W = 720, H = 420;
  const xMin = -3, xMax = 3, yMin = -2.5, yMax = 3;

  function svgEl(tag) {
    return document.createElementNS("http://www.w3.org/2000/svg", tag);
  }

  function feasible(x, y) {
    const g1 = 0.35 * (x - 0.4) ** 2 - 1.2;
    const g2 = x * x + (y - 0.1) ** 2 - 5.4;
    return y >= g1 && g2 <= 0;
  }

  function objective(x, y, w) {
    const f1 = (x - 1.7) ** 2 + 0.4 * y ** 2;
    const f2 = 0.6 * (x + 0.9) ** 2 + (y - 1.5) ** 2;
    return w * f1 + (1 - w) * f2;
  }
```

```
function grad(x, y, w) {
  return {
    gx: 2 * w * (x - 1.7) + 1.2 * (1 - w) * (x + 0.9),
    gy: 0.8 * w * y + 2 * (1 - w) * (y - 1.5)
  };
}

function sx(v) { return 55 + (v - xMin) / (xMax - xMin) * 610; }
function sy(v) { return 365 - (v - yMin) / (yMax - yMin) * 300; }

const container = document.createElement("div");
container.style.cssText = "max-width:760px; margin:1rem 0 1.25rem 0; font-family:inherit;";
const controls = document.createElement("div");
controls.style.cssText = "display:grid; grid-template-columns:auto 1fr auto; gap:0.5rem 0.75rem

const label = document.createElement("label");
label.textContent = "Weight on target-matching term";
const slider = document.createElement("input");
slider.type = "range";
slider.min = "0";
slider.max = "1";
slider.step = "0.01";
slider.value = "0.55";
const out = document.createElement("span");
out.style.cssText = "font-variant-numeric:tabular-nums; min-width:4rem;";
controls.append(label, slider, out);

const summary = document.createElement("div");
summary.style.cssText = "display:flex; gap:1rem; flex-wrap:wrap; margin-bottom:0.6rem; color:#3
const optBox = document.createElement("span");
const objBox = document.createElement("span");
const activeBox = document.createElement("span");
const gradBox = document.createElement("span");
summary.append(optBox, objBox, activeBox, gradBox);

const svg = svgEl("svg");
svg.setAttribute("viewBox", `0 0 ${W} ${H}`);
svg.style.width = "100%";
svg.style.height = "auto";
svg.style.border = "1px solid #e2e8f0";
svg.style.background = "#fff";

const caption = document.createElement("p");
caption.style.cssText = "margin:0.65rem 0 0 0; color:#475569;";

container.append(controls, summary, svg, caption);
```

```
function redraw() {
  const w = Number(slider.value);
  out.textContent = w.toFixed(2);
  svg.replaceChildren();

  let best = null;
  for (let i = 0; i <= 140; i++) {
    for (let j = 0; j <= 120; j++) {
      const x = xMin + i / 140 * (xMax - xMin);
      const y = yMin + j / 120 * (yMax - yMin);
      const rect = svgEl("rect");
      rect.setAttribute("x", String(55 + i * (610 / 140)));
      rect.setAttribute("y", String(65 + (119 - j) * (300 / 120)));
      rect.setAttribute("width", String(610 / 140 + 0.5));
      rect.setAttribute("height", String(300 / 120 + 0.5));
      if (feasible(x, y)) {
        const val = objective(x, y, w);
        const t = Math.max(0, Math.min(1, val / 9));
        rect.setAttribute("fill", `hsl(32 90% ${94 - 22 * t}%)`);
        if (!best || val < best.val) best = { x, y, val };
      } else {
        rect.setAttribute("fill", "#f8fafc");
      }
      svg.appendChild(rect);
    }
  }

  const circle = svgEl("ellipse");
  circle.setAttribute("cx", String(sx(0)));
  circle.setAttribute("cy", String(sy(0.1)));
  circle.setAttribute("rx", String(Math.abs(sx(Math.sqrt(5.4)) - sx(0))));
  circle.setAttribute("ry", String(Math.abs(sy(0.1 + Math.sqrt(5.4)) - sy(0.1))));
  circle.setAttribute("fill", "none");
  circle.setAttribute("stroke", "#475569");
  circle.setAttribute("stroke-width", "2");
  svg.appendChild(circle);

  const parabolaPts = [];
  for (let i = 0; i <= 140; i++) {
    const x = xMin + i / 140 * (xMax - xMin);
    const y = 0.35 * (x - 0.4) ** 2 - 1.2;
    parabolaPts.push(`${i === 0 ? "M" : "L"} ${sx(x)} ${sy(y)}`);
  }
  const parabola = svgEl("path");
  parabola.setAttribute("d", parabolaPts.join(" "));
  parabola.setAttribute("fill", "none");
  parabola.setAttribute("stroke", "#7c3aed");
```

```
    parabola.setAttribute("stroke-width", "2.5");
    svg.appendChild(parabola);

    const point = svgEl("circle");
    point.setAttribute("cx", String(sx(best.x)));
    point.setAttribute("cy", String(sy(best.y)));
    point.setAttribute("r", "6");
    point.setAttribute("fill", "#dc2626");
    svg.appendChild(point);

    const g = grad(best.x, best.y, w);
    const gLen = Math.hypot(g.gx, g.gy) || 1;
    const arrow = svgEl("line");
    arrow.setAttribute("x1", String(sx(best.x)));
    arrow.setAttribute("y1", String(sy(best.y)));
    arrow.setAttribute("x2", String(sx(best.x + 0.55 * g.gx / gLen)));
    arrow.setAttribute("y2", String(sy(best.y + 0.55 * g.gy / gLen)));
    arrow.setAttribute("stroke", "#1d4ed8");
    arrow.setAttribute("stroke-width", "3");
    svg.appendChild(arrow);

    const g1Val = best.y - (0.35 * (best.x - 0.4) ** 2 - 1.2);
    const g2Val = 5.4 - (best.x * best.x + (best.y - 0.1) ** 2);
    const active = [];
    if (g1Val < 0.12) active.push("parabolic floor");
    if (g2Val < 0.12) active.push("elliptic boundary");
    if (!active.length) active.push("none");

    optBox.textContent = `best feasible point = (${best.x.toFixed(2)}, ${best.y.toFixed(2)})`;
    objBox.textContent = `objective = ${best.val.toFixed(3)}`;
    activeBox.textContent = `active constraints = ${active.join(", ")}`;
    gradBox.textContent = `|∇f| = ${gLen.toFixed(3)}`;
    caption.textContent = "As the weight shifts, the red optimum slides to the part of the feasib
  }

  slider.addEventListener("input", redraw);
  redraw();
  return container;
}
```